

Machine Learning Engineering: Training Pipeline Architecture and Best Practices

Diogo Ribeiro
ESMAD - Instituto Politécnico do Porto

November 24, 2025

Contents

Preface	xxix
Running Case Study: FraudGuard Payment System	xxxiii
0.1 System Overview	xxxiii
0.2 Business Context	xxxiii
0.2.1 Problem Statement	xxxiii
0.2.2 Success Metrics	xxxiii
0.3 System Architecture Evolution	xxxiii
0.3.1 Phase 1: Initial MVP (Chapters 1-7)	xxxiii
0.3.2 Phase 2: Production System (Chapters 8-13)	xxxiv
0.3.3 Phase 3: Production Excellence (Chapters 14-19)	xxxv
0.3.4 Phase 4: Governance and Scale (Chapters 20-22)	xxxv
0.4 Data Schema	xxxvi
0.4.1 Transaction Features	xxxvi
0.4.2 Sample Data Statistics	xxxvii
0.5 Model Performance Requirements	xxxvii
0.5.1 Prediction Requirements	xxxvii
0.5.2 Training Requirements	xxxvii
0.6 Cost Budget	xxxvii
0.7 Chapter-by-Chapter Application	xxxvii
0.8 Evolution Timeline	xxxix
0.9 Key Learnings and Patterns	xxxix
0.10 Code Repository Structure	xxxix
 I Foundations of ML Engineering	 1
1 Introduction to ML Engineering	3
1.1 Learning Objectives	3
1.2 From Data Science to ML Engineering	3
1.2.1 The Evolution of ML in Industry	3
1.2.2 Defining Data Science vs ML Engineering	4
1.2.3 The Skills Gap	4
1.3 Research Code vs Production Code	5
1.3.1 The Research Notebook Approach	5
1.3.2 Production-Grade ML Code	6
1.4 The ML Lifecycle in Production	17
1.4.1 Lifecycle Phases	17

1.4.2	Iterative Nature of the Lifecycle	18
1.5	Common Failure Modes in ML Projects	18
1.5.1	Technical Failures	19
1.5.2	Organizational Failures	20
1.6	Architecture Principles for ML Systems	20
1.6.1	Separation of Concerns	21
1.6.2	Immutability and Versioning	21
1.6.3	Observability First	21
1.7	The MLOps Maturity Model	21
1.7.1	Level 0: Manual Process	22
1.7.2	Level 1: ML Pipeline Automation	22
1.7.3	Level 2: CI/CD Pipeline Automation	22
1.7.4	Level 3: Full MLOps Automation	23
1.8	Chapter Summary	23
1.9	Further Reading	23
1.10	Chapter Summary	24
1.11	Exercises	25
2	The Machine Learning Lifecycle	27
2.1	Learning Objectives	27
2.2	Overview of the ML Lifecycle	27
2.2.1	The End-to-End Journey	27
2.3	Phase 1: Problem Definition	27
2.3.1	Framing the Business Problem	27
2.3.2	ML Problem Formulation	28
2.4	Phase 2: Data Collection and Understanding	29
2.4.1	Data Requirements	29
2.4.2	Exploratory Data Analysis	29
2.5	Phase 3: Data Preparation	30
2.5.1	Data Cleaning	30
2.5.2	Feature Engineering	30
2.6	Phase 4: Model Development	30
2.6.1	Model Selection	30
2.6.2	Training and Validation	31
2.7	Phase 5: Evaluation	31
2.7.1	Model Assessment	31
2.8	Phase 6: Deployment	31
2.8.1	Deployment Strategies	31
2.9	Phase 7: Monitoring and Maintenance	32
2.9.1	Production Monitoring	32
2.9.2	Model Maintenance	32
2.10	Iterative Nature of ML	32
2.10.1	Feedback Loops	32
2.11	Chapter Summary	33
2.12	Exercises	33

3	Training Pipeline Design Patterns	35
3.1	Learning Objectives	35
3.2	Introduction: The Training Pipeline	35
3.3	Pipeline Orchestration	36
3.3.1	Orchestration Tool Comparison	36
3.3.2	Apache Airflow for ML Pipelines	36
3.3.3	Kubeflow Pipelines for ML	44
3.4	Experiment Tracking and Reproducibility	48
3.4.1	MLflow for Experiment Management	48
3.4.2	Reproducibility Best Practices	53
3.5	Hyperparameter Optimization	55
3.5.1	Optimization Strategies	55
3.5.2	Bayesian Optimization with Optuna	55
3.6	Model Validation and Evaluation Frameworks	59
3.6.1	Validation Strategies	59
3.7	A/B Testing for ML Models	65
3.7.1	A/B Testing Framework	65
3.8	BigQuery ML ARIMA_PLUS Case Study	72
3.8.1	Problem: Sales Forecasting	72
3.9	Failure Handling and Retry Mechanisms	78
3.10	Chapter Summary	81
3.11	Exercises	81
II	Data Engineering for ML	85
4	Data Pipeline Architecture	87
4.1	Learning Objectives	87
4.2	Introduction: Data Pipelines as the Foundation	87
4.3	Data Ingestion Patterns	88
4.3.1	Batch Processing	88
4.3.2	Streaming Processing	90
4.3.3	Micro-Batch Processing	94
4.4	ETL vs ELT for ML Workloads	96
4.4.1	ETL: Transform Before Loading	96
4.4.2	ELT: Transform in the Warehouse	97
4.4.3	Hybrid Approach for ML	98
4.5	Data Validation and Quality Checks	99
4.5.1	Validation Layers	99
4.5.2	Great Expectations Framework	100
4.5.3	Custom Validation Rules	103
4.6	Feature Stores and Data Versioning	105
4.6.1	Feature Store Architecture	105
4.6.2	Feature Store Implementation	105
4.6.3	Data Versioning with DVC	108
4.7	Schema Evolution and Compatibility	110
4.7.1	Schema Evolution Strategies	110
4.7.2	Apache Avro for Schema Management	110

4.7.3	Handling Breaking Schema Changes	113
4.8	Building a Complete Data Pipeline	114
4.9	Chapter Summary	119
4.10	Exercises	119
5	Data Quality and Validation	123
5.1	Learning Objectives	123
5.2	Introduction	123
5.3	Data Quality Dimensions	123
5.3.1	The Six Dimensions of Data Quality	123
5.3.2	ML-Specific Quality Dimensions	124
5.4	Data Quality Frameworks and Metrics	124
5.4.1	Comprehensive Data Quality Framework	124
5.4.2	Usage Example: Complete Validation Pipeline	134
5.5	Great Expectations Implementation	135
5.5.1	Setting Up Great Expectations	135
5.5.2	Custom Great Expectations for ML	142
5.6	Data Profiling and Anomaly Detection	145
5.6.1	Automated Data Profiling	145
5.6.2	Anomaly Detection in Data	150
5.7	Automated Data Quality Monitoring Pipelines	156
5.7.1	Continuous Monitoring Architecture	156
5.7.2	Integration with Apache Airflow	163
5.8	Custom Validation Rules for ML-Specific Requirements	165
5.8.1	ML-Specific Validation Rules	165
5.9	Pipeline Integration for Continuous Validation	171
5.9.1	End-to-End Data Quality Pipeline	171
5.10	Chapter Summary	176
5.11	Exercises	177
6	Feature Engineering and Feature Stores	191
6.1	Learning Objectives	191
6.2	Introduction: The Feature Engineering Challenge	191
6.3	Feature Engineering Best Practices	192
6.3.1	Feature Design Principles	192
6.3.2	Feature Transformation Patterns	195
6.4	Complete Feast Feature Store Implementation	201
6.4.1	Feast Architecture and Setup	201
6.4.2	Feature Materialization Pipeline	203
6.4.3	Online and Offline Feature Serving	207
6.5	Feature Versioning and Lineage Tracking	210
6.5.1	Feature Versioning Strategy	210
6.5.2	Feature Lineage Tracking	214
6.6	Online and Offline Feature Serving Architecture	219
6.6.1	Offline Feature Store Implementation	219
6.6.2	Online Feature Store with Redis	222
6.7	Feature Monitoring and Drift Detection	227
6.8	Integration with Training and Serving Pipelines	234

6.9	Real-World Case Studies	238
6.9.1	Case Study 1: E-Commerce Recommendation System	238
6.9.2	Case Study 2: Fraud Detection System	239
6.10	Exercises	240
6.11	Chapter Summary	242

III Training Pipeline Architecture 243

7	Model Development Best Practices	245
7.1	Learning Objectives	245
7.2	Introduction: From Research Code to Production	245
7.3	ML Code Organization and Project Structure	246
7.3.1	Production ML Project Template	246
7.3.2	Project Structure Implementation	248
7.4	Modular ML Architecture Patterns	250
7.4.1	Base Model Interface	250
7.4.2	Concrete Model Implementations	253
7.4.3	Model Factory Pattern	255
7.4.4	Data Pipeline Architecture	257
7.5	Testing Strategies for ML Code	260
7.5.1	Unit Testing for ML Components	260
7.5.2	Integration Testing for ML Pipelines	262
7.5.3	Model-Specific Tests	265
7.5.4	Test Configuration with Pytest	267
7.6	Design Patterns for ML Systems	268
7.6.1	Strategy Pattern for Model Selection	268
7.6.2	Observer Pattern for Training Callbacks	272
7.7	Code Quality Tools and Practices	275
7.7.1	Type Hints and Static Analysis	276
7.7.2	Configuration and linting setup	278
7.8	Reproducibility Frameworks	280
7.8.1	Configuration Management	280
7.8.2	Reproducible Training Pipeline	282
7.8.3	Model and Experiment Versioning	286
7.9	Documentation Best Practices	291
7.9.1	Model Cards	291
7.9.2	API Documentation	295
7.10	Continuous Integration for ML	298
7.10.1	GitHub Actions Workflow	298
7.11	Exercises	302
7.12	Chapter Summary	304
8	Experiment Tracking and Management	307
8.1	Learning Objectives	307
8.2	Introduction to Experiment Tracking	307
8.2.1	Why Track Experiments?	307
8.3	MLflow Tutorial	307
8.4	Model Registry	308

8.5	Chapter Summary	308
8.6	Exercises	308
9	Distributed Training and Optimization	309
9.1	Learning Objectives	309
9.2	Introduction: Why Distributed Training?	309
9.2.1	Scaling Laws	310
9.3	Data Parallelism	310
9.3.1	PyTorch Distributed Data Parallel (DDP)	311
9.3.2	Launch Script for Multi-Node Training	316
9.4	Model Parallelism	317
9.4.1	Pipeline Parallelism	318
9.4.2	Tensor Parallelism	320
9.5	Parameter Servers vs All-Reduce	322
9.5.1	Parameter Server Architecture	322
9.5.2	All-Reduce Architecture	322
9.6	Gradient Synchronization Strategies	323
9.6.1	Synchronous SGD	323
9.6.2	Asynchronous SGD	324
9.6.3	Hogwild! (Lock-Free Async SGD)	326
9.7	Horovod: Framework-Agnostic Distributed Training	327
9.7.1	Horovod Timeline for Performance Analysis	328
9.8	Ray for Flexible Distributed Training	328
9.9	Fault Tolerance and Checkpointing	330
9.10	Performance Benchmarking and Optimization	333
9.10.1	Measuring Distributed Training Performance	333
9.10.2	Identifying and Resolving Bottlenecks	337
9.11	Resource Allocation and Scheduling	339
9.11.1	SLURM Job Scheduling	339
9.11.2	Kubernetes Job Specification	339
9.12	Memory Management in Distributed Settings	341
9.12.1	Gradient Checkpointing	341
9.12.2	ZeRO: Zero Redundancy Optimizer	343
9.12.3	Activation Memory Optimization	344
9.13	Communication Optimization	346
9.13.1	Gradient Compression	347
9.13.2	Overlapping Computation and Communication	349
9.13.3	Communication-Efficient Architectures	351
9.14	Debugging Distributed Training	352
9.14.1	Common Issues and Solutions	352
9.14.2	Profiling Distributed Communication	355
9.15	Chapter Summary	357
9.16	Exercises	358
10	Hyperparameter Optimization	361
10.1	Learning Objectives	361
10.2	Introduction: The Hyperparameter Challenge	361
10.3	Bayesian Optimization with Optuna	362

10.3.1 Fundamental Optuna Implementation	362
10.3.2 Advanced Optuna Features	367
10.4 Multi-Objective Optimization	369
10.5 Population-Based Training (PBT)	372
10.6 Distributed Hyperparameter Optimization	377
10.7 AutoML and Neural Architecture Search	381
10.8 Integration with Experiment Tracking	385
10.9 Cost-Efficient HPO Strategies	387
10.10 Chapter Summary	392
10.11 Exercises	392
10.12 Hyperparameter Search Space Design	394
10.12.1 Search Space Types and Best Practices	395
10.12.2 Pruning Search Spaces Based on Prior Knowledge	398
10.13 Advanced Pruning Strategies	401
10.14 Performance Analysis and Visualization	405
10.15 HPO in Production Environments	414
10.15.1 Automated HPO Pipelines	414
10.15.2 HPO Monitoring and Alerting	420
10.15.3 Resource Management and Cost Optimization	424

IV Model Deployment and Serving 429

11 Model Serving Architecture 431	431
11.1 Learning Objectives	431
11.2 Introduction: Model Serving Architecture	431
11.3 FastAPI Model Serving	432
11.3.1 Basic FastAPI Implementation	432
11.3.2 Advanced FastAPI Features	438
11.4 gRPC Model Serving	441
11.5 TensorFlow Serving and TorchServe	448
11.5.1 TensorFlow Serving Setup	448
11.5.2 TorchServe Setup	452
11.6 Batch vs Online vs Streaming Serving Patterns	458
11.6.1 Online Serving (Real-time)	458
11.6.2 Batch Serving	459
11.6.3 Streaming Serving	463
11.7 Model Optimization for Serving	464
11.7.1 ONNX Conversion and Optimization	464
11.7.2 TensorRT Optimization	468
11.8 Load Balancing and Scaling Strategies	472
11.8.1 Load Balancing with NGINX	472
11.8.2 Kubernetes Deployment and Autoscaling	473
11.8.3 Auto-scaling Based on Custom Metrics	476
11.9 Performance Optimization and Profiling	478
11.9.1 Latency Profiling	478
11.9.2 Throughput Optimization	480
11.9.3 Memory Optimization	483

11.10	Chapter Summary	484
11.11	Exercises	485
12	Deployment Strategies and Patterns	491
12.1	Learning Objectives	491
12.2	Introduction	491
12.3	Blue-Green Deployment	492
12.3.1	Overview and Architecture	492
12.3.2	Implementation	492
12.3.3	Kubernetes Blue-Green Deployment	499
12.4	Canary Deployment	502
12.4.1	Overview and Strategy	502
12.4.2	Implementation	502
12.5	Shadow Deployment	509
12.5.1	Overview and Use Cases	509
12.5.2	Implementation	509
12.6	A/B Testing for Model Evaluation	515
12.6.1	Overview and Statistical Framework	515
12.6.2	Implementation	515
12.7	Traffic Splitting and Routing	521
12.7.1	Traffic Router Implementation	521
12.8	Rollback Mechanisms and Circuit Breakers	525
12.8.1	Automated Rollback System	525
12.9	Multi-Model Serving Architectures	529
12.9.1	Overview	529
12.9.2	Implementation	529
12.10	Chapter Summary	534
12.11	Exercises	535
13	Model Versioning and Registry	543
13.1	Learning Objectives	543
13.2	Introduction	543
13.3	MLflow Model Registry: Advanced Features	544
13.3.1	Architecture and Core Concepts	544
13.3.2	Complete MLflow Registry Implementation	544
13.3.3	Model Signature and Schema Validation	552
13.4	Model Lineage and Metadata Tracking	554
13.4.1	Comprehensive Lineage System	554
13.5	Approval Workflows and Governance	561
13.5.1	Model Governance Framework	561
13.5.2	Complete Governance Implementation	561
13.6	Model Comparison and Promotion	573
13.6.1	Comprehensive Model Comparison Framework	573
13.6.2	Model Comparison and Promotion System	574
13.7	CI/CD Pipeline Integration	584
13.7.1	Automated Model Lifecycle with CI/CD	584
13.7.2	CI/CD Pipeline Implementation	584
13.8	Model Packaging and Containerization	594

13.8.1	Reproducible Model Packaging	594
13.8.2	Model Packaging and Container Implementation	594
13.9	Exercises	601
13.9.1	Exercise 1: Model Registration and Lifecycle Management . . .	601
13.9.2	Exercise 2: Complete Model Lineage Tracking	603
13.9.3	Exercise 3: Model Governance with Approval Workflows	605
13.9.4	Exercise 4: Statistical Model Comparison	608
13.9.5	Exercise 5: CI/CD Pipeline for Model Deployment	610
13.9.6	Exercise 6: Model Packaging for Multi-Format Deployment . . .	612
13.10	Chapter Summary	615
13.10.1	Key Concepts Covered	615
13.10.2	Best Practices	616
13.10.3	Common Pitfalls to Avoid	617
13.10.4	Production Checklist	617
13.10.5	Next Steps	617

V Production ML Operations 619

14	ML Monitoring and Observability 621
14.1	Learning Objectives 621
14.2	Introduction 621
14.3	ML Monitoring Architecture 622
14.3.1	Monitoring System Components 622
14.3.2	Complete Monitoring Architecture Implementation 622
14.4	Data Drift Detection 630
14.4.1	Types of Drift 630
14.4.2	Drift Detection with Evidently and Custom Methods 630
14.5	Model Performance Monitoring 639
14.5.1	Performance Metrics Tracking 639
14.5.2	Performance Monitoring Implementation 640
14.6	Prometheus and Grafana Integration 646
14.6.1	Prometheus Metrics Export 646
14.6.2	Prometheus Integration Implementation 646
14.7	Alert Systems and Incident Response 653
14.7.1	Intelligent Alerting 653
14.7.2	Alert System Implementation 654
14.8	Observability Best Practices 657
14.8.1	Comprehensive Observability 657
14.8.2	Implementing Structured Logging for ML 658
14.8.3	Distributed Tracing for ML Pipelines 661
14.8.4	Data Quality Monitoring Implementation 663
14.9	Exercises 669
14.9.1	Exercise 1: Complete Monitoring Stack Setup 669
14.9.2	Exercise 2: Drift Detection Comparison 671
14.9.3	Exercise 3: Prometheus and Grafana Dashboard 672
14.9.4	Exercise 4: Alert System with Runbooks 674
14.9.5	Exercise 5: Performance Degradation Detection 676

14.9.6	Exercise 6: Comprehensive Monitoring Dashboard	678
14.9.7	Exercise 7: Custom Drift Detection with Multiple Methods	680
14.9.8	Exercise 8: End-to-End Production Monitoring Pipeline	685
14.10	Chapter Summary	693
14.10.1	Key Concepts Covered	693
14.10.2	Best Practices	695
14.10.3	Common Pitfalls	695
14.10.4	Production Checklist	695
14.10.5	Integration Points	696
15	Model Maintenance and Retraining	697
15.1	Learning Objectives	697
15.2	Introduction	697
15.3	Automated Retraining Pipeline Architecture	698
15.3.1	Core Components	698
15.4	Trigger Mechanisms for Retraining	706
15.4.1	Multi-Criteria Trigger System	706
15.5	Incremental Learning and Online Updating	712
15.5.1	Incremental Training Strategies	712
15.6	Concept Drift Handling Strategies	717
15.6.1	Detecting and Adapting to Concept Drift	717
15.6.2	Multi-Strategy Drift Adaptation	723
15.7	Model Freshness vs Stability Trade-offs	729
15.7.1	Balancing Update Frequency and Model Stability	729
15.7.2	Cost-Aware Retraining Scheduling	735
15.8	Continuous Learning Systems	740
15.8.1	Real-Time Model Adaptation	740
15.8.2	Champion-Challenger Framework	747
15.9	Practical Exercises	751
15.9.1	Exercise 1: Build an Automated Retraining Pipeline	751
15.9.2	Exercise 2: Implement Multi-Trigger Retraining Logic	752
15.9.3	Exercise 3: Build an Incremental Learning System	753
15.9.4	Exercise 4: Implement Concept Drift Detection	753
15.9.5	Exercise 5: Optimize Freshness-Stability Trade-off	754
15.9.6	Exercise 6: Build a Cost-Aware Retraining Scheduler	755
15.9.7	Exercise 7: Implement a Continuous Learning System with Champion-Challenger	756
15.10	Summary	757
16	Model Performance Optimization	759
16.1	Learning Objectives	759
16.2	Introduction	759
16.3	Model Compression Techniques	760
16.3.1	Quantization	760
16.3.2	Model Pruning	765
16.3.3	Knowledge Distillation	769
16.4	Hardware Acceleration	775
16.4.1	GPU Optimization	775

16.5	Memory Optimization and Batch Processing	780
16.5.1	Memory-Efficient Inference	780
16.6	Inference Optimization Patterns	785
16.6.1	Model Serving Architecture	785
16.7	Edge Deployment Optimization	790
16.7.1	Mobile and Edge Optimization	790
16.8	Benchmarking and Profiling Tools	794
16.8.1	Comprehensive Performance Profiling	794
16.9	Practical Optimization Challenges	799
16.9.1	Challenge 1: Compress a Large Model	799
16.9.2	Challenge 2: Optimize Inference Latency	800
16.9.3	Challenge 3: Optimize for Edge Deployment	801
16.9.4	Challenge 4: Memory-Efficient Serving	803
16.9.5	Challenge 5: Multi-Model Serving Optimization	804
16.9.6	Challenge 6: Optimize Training Performance	806
16.9.7	Challenge 7: End-to-End Production Optimization	808
16.10	Summary	809
16.11	Real-World Optimization Case Studies	812
16.11.1	Case Study 1: BERT Model Optimization for Search	812
16.11.2	Case Study 2: Image Classification on Edge Devices	813
16.11.3	Case Study 3: Large Language Model Serving	814
16.11.4	Lessons Learned Across All Cases	816

VI MLOps and Automation 819

17	CI/CD Pipelines for Machine Learning	821
17.1	Learning Objectives	821
17.2	Introduction	821
17.3	GitHub Actions Workflows for ML Projects	822
17.3.1	Basic ML Pipeline Structure	822
17.3.2	Advanced Workflow Patterns	827
17.4	Automated Testing Strategies for ML Code	830
17.4.1	ML-Specific Testing Framework	830
17.4.2	Model Performance Testing	838
17.5	Continuous Training Pipeline Implementation	844
17.5.1	Automated Continuous Training System	844
17.6	Model Validation and Deployment Automation	855
17.6.1	Automated Model Validation Gates	855
17.7	Integration with Model Registry and Monitoring	862
17.7.1	Model Registry Integration	862
17.8	Infrastructure as Code for ML CI/CD	865
17.8.1	Terraform for ML Infrastructure	865
17.8.2	GitHub Actions with Terraform	870
17.8.3	Docker Compose for Local Development	872
17.9	Security Scanning and Compliance	875
17.9.1	Security and Compliance Checks	875
17.10	Practical CI/CD Implementation Exercises	878

17.10.1 Exercise 1: Build Complete ML CI/CD Pipeline	878
17.10.2 Exercise 2: Implement Continuous Training Pipeline	878
17.10.3 Exercise 3: Create Comprehensive ML Test Suite	879
17.10.4 Exercise 4: Model Registry Integration	879
17.10.5 Exercise 5: Security and Compliance Automation	880
17.10.6 Exercise 6: Multi-Environment Deployment Pipeline	881
17.10.7 Exercise 7: Model Monitoring Integration	881
17.10.8 Exercise 8: A/B Testing Framework	882
17.11 Summary	882
18 Infrastructure as Code for ML	885
18.1 Learning Objectives	885
18.2 Introduction	885
18.3 Terraform Modules for ML Infrastructure	886
18.3.1 Core ML Infrastructure Module	886
18.3.2 Terraform Variables and Outputs	895
18.3.3 Environment-Specific Configuration	899
18.4 Kubernetes Deployment with Helm Charts	900
18.4.1 Helm Chart for ML Training Jobs	900
18.4.2 Distributed Training with Kubernetes	907
18.5 Multi-Cloud Resource Management	909
18.5.1 Google Cloud Platform (GCP) Infrastructure	909
18.5.2 Azure Machine Learning Infrastructure	916
18.6 Container Orchestration for ML Workloads	923
18.6.1 Kubernetes Operators for ML	923
18.6.2 Autoscaling ML Workloads	926
18.6.3 Resource Quotas and Limits	928
18.7 Disaster Recovery and Backup Strategies	930
18.7.1 Backup Automation with Terraform	930
18.7.2 Disaster Recovery Procedures	934
18.8 Cost Optimization Through IaC	941
18.8.1 Cost-Aware Resource Provisioning	941
18.9 Practical Infrastructure Automation Exercises	949
18.9.1 Exercise 1: Multi-Environment ML Infrastructure	949
18.9.2 Exercise 2: Kubernetes ML Training Helm Chart	949
18.9.3 Exercise 3: Multi-Cloud Infrastructure Deployment	950
18.9.4 Exercise 4: Automated Disaster Recovery System	950
18.9.5 Exercise 5: Cost-Optimized Training Infrastructure	951
18.9.6 Exercise 6: Kubernetes Operator for ML Workflows	951
18.9.7 Exercise 7: Infrastructure Monitoring and Observability	952
18.10 Summary	952
19 MLOps Platforms and Tools	955
19.1 Learning Objectives	955
19.2 Introduction	955
19.3 Kubeflow Pipelines: Comprehensive Tutorial	956
19.3.1 Kubeflow Architecture and Components	956
19.3.2 Building ML Pipelines with Kubeflow	956

19.3.3	Kubeflow Katib for Hyperparameter Tuning	963
19.3.4	KServe for Model Deployment	965
19.4	Cloud ML Platforms Comparison	966
19.4.1	AWS SageMaker	966
19.4.2	Google Cloud Vertex AI	970
19.4.3	Azure Machine Learning	973
19.5	Platform Comparison Matrix	977
19.6	Platform Selection Criteria and Decision Framework	977
19.6.1	Decision Framework Matrix	977
19.7	Migration Strategies Between Platforms	982
19.7.1	Migration Planning Framework	982
19.7.2	Code Portability Strategies	987
19.8	Custom MLOps Platform Development	990
19.8.1	Custom Platform Architecture	990
19.9	Platform Governance and Best Practices	994
19.9.1	Multi-Tenancy and Access Control	994
19.9.2	Platform Standardization	998
19.9.3	Cost Management and Optimization	999
19.9.4	Security and Compliance	1003
19.9.5	Platform Monitoring and Observability	1003
19.10	Platform Evaluation Exercises	1004
19.10.1	Exercise 1: Platform Selection for Startup	1004
19.10.2	Exercise 2: Multi-Cloud Platform Migration	1005
19.10.3	Exercise 3: Build vs Buy Decision	1006
19.10.4	Exercise 4: Kubeflow Pipeline Implementation	1006
19.10.5	Exercise 5: Platform Cost Optimization	1007
19.10.6	Exercise 6: Platform Comparison Benchmark	1008
19.11	Summary	1009

VII Advanced Topics and Best Practices 1011

20	Model Governance and Compliance	1013
20.1	Learning Objectives	1013
20.2	Introduction	1013
20.3	Model Governance Framework Implementation	1014
20.3.1	Governance Structure and Roles	1014
20.3.2	Model Inventory and Lifecycle Management	1021
20.4	GDPR and CCPA Compliance for ML Systems	1027
20.4.1	GDPR Requirements for ML	1027
20.4.2	GDPR Compliance Implementation	1028
20.4.3	CCPA Compliance	1034
20.5	Model Documentation and Audit Trails	1036
20.5.1	Model Cards	1036
20.5.2	Audit Trail Implementation	1043
20.6	Bias Detection and Fairness Testing	1048
20.6.1	Fairness Metrics	1048
20.7	Risk Assessment Frameworks	1049

20.8	Approval Workflows and Controls	1049
20.9	Governance Implementation Exercises	1050
20.9.1	Exercise 1: Build Governance Framework for Healthcare ML System	1050
20.9.2	Exercise 2: GDPR Compliance Implementation	1051
20.9.3	Exercise 3: Fairness Audit and Remediation	1052
20.9.4	Exercise 4: Multi-Stage Approval Workflow Implementation	1053
20.9.5	Exercise 5: Model Risk Assessment	1054
20.10	Summary	1055
20.10.1	Comprehensive Fairness Testing Implementation	1056
20.10.2	Advanced Risk Assessment with Quantitative Scoring	1064
21	Security in ML Systems	1071
21.1	Learning Objectives	1071
21.2	Introduction	1071
21.3	Adversarial Attack Detection and Defense	1072
21.3.1	Adversarial Attack Types	1072
21.3.2	Adversarial Defense Implementation	1072
21.4	Model Stealing and Inversion Protection	1080
21.4.1	Model Stealing Attacks	1080
21.4.2	Model Protection Implementation	1080
21.5	Secure Data Handling and Encryption	1087
21.5.1	Data Encryption	1087
21.6	API Security for ML Endpoints	1093
21.6.1	Authentication and Authorization	1093
21.6.2	Input Validation and Sanitization	1101
21.7	Privacy-Preserving ML Techniques	1102
21.7.1	Differential Privacy	1102
21.7.2	Federated Learning	1108
21.8	Security Scanning and Vulnerability Assessment	1108
21.8.1	ML Security Scanner	1108
21.8.2	Security Best Practices	1114
21.9	Implementation Exercises	1115
21.9.1	Exercise 1: Adversarial Attack Detection System	1115
21.9.2	Exercise 2: Secure ML API with Rate Limiting	1116
21.9.3	Exercise 3: Differential Privacy for Model Training	1116
21.9.4	Exercise 4: Model Stealing Defense System	1117
21.9.5	Exercise 5: Federated Learning with Secure Aggregation	1118
21.9.6	Exercise 6: ML Security Audit and Penetration Testing	1119
21.9.7	Exercise Evaluation Criteria	1120
21.10	Conclusion	1120
22	Cost Optimization in ML Infrastructure	1123
22.1	Learning Objectives	1123
22.2	Introduction	1123
22.3	ML Cost Structure and Analysis	1124
22.3.1	Cost Breakdown Model	1124
22.4	Training Cost Optimization	1130

22.4.1	Training Infrastructure Optimization	1130
22.4.2	Training Cost Optimizer Implementation	1131
22.4.3	Spot Instance Strategy for Training	1137
22.5	Serving Cost Analysis and Reduction	1143
22.5.1	Serving Cost Optimization Strategies	1143
22.6	Resource Allocation and Scaling Policies	1149
22.6.1	Autoscaling Implementation	1149
22.7	Cloud Cost Monitoring and Alerts	1155
22.7.1	Cost Monitoring System	1155
22.8	ROI Analysis for ML Investments	1161
22.8.1	ML ROI Calculator	1161
22.9	Best Practices for Cost Optimization	1167
22.10	Chapter Summary	1168
22.11	Exercises	1169

VIII Case Studies and Integration 1173

23 Industry Case Studies 1175

23.1	Learning Objectives	1175
23.2	Introduction	1175
23.3	Case Study 1: E-Commerce Recommendation System	1176
23.3.1	Business Context and Requirements	1176
23.3.2	System Architecture	1176
23.3.3	Data Pipeline Implementation	1176
23.3.4	Model Implementation	1185
23.3.5	Serving and Deployment	1193
23.3.6	Results and Impact	1198
23.3.7	Lessons Learned	1198
23.4	Case Study 2: Financial Fraud Detection Pipeline	1199
23.4.1	Business Context and Requirements	1199
23.4.2	System Architecture	1200
23.4.3	Feature Engineering and Real-Time Processing	1200
23.4.4	Fraud Detection Models	1207
23.4.5	Results and Impact	1212
23.4.6	Lessons Learned	1213
23.5	Case Study 3: Healthcare Diagnostic System	1213
23.5.1	Business Context and Requirements	1213
23.5.2	System Architecture	1214
23.5.3	Results and Impact	1221
23.5.4	Lessons Learned	1221
23.6	Case Study 4: Autonomous Vehicle Perception Stack	1222
23.6.1	Business Context and Requirements	1222
23.6.2	Perception Pipeline	1222
23.6.3	Results and Impact	1230
23.6.4	Lessons Learned	1231
23.7	Cross-Cutting Themes and Best Practices	1231
23.8	Chapter Summary	1232

23.9 Exercises	1232
24 Building End-to-End ML Systems	1235
24.1 Learning Objectives	1235
24.2 Introduction	1235
24.3 System Design Methodology	1236
24.3.1 The ML System Design Process	1236
24.3.2 Phase 1: Problem Framing	1236
24.3.3 Phase 2: Requirements Analysis	1237
24.3.4 Phase 3: Architecture Design	1238
24.4 Architecture Decision Frameworks	1238
24.4.1 Decision Matrix for Architecture Patterns	1238
24.4.2 Trade-off Analysis Framework	1243
24.5 Integration Patterns and Best Practices	1243
24.5.1 Data Integration Patterns	1243
24.5.2 Model Integration Patterns	1253
24.6 Performance and Scalability Considerations	1257
24.6.1 Performance Optimization Strategies	1257
24.7 Evolution and Maintenance Strategies	1260
24.7.1 System Evolution Framework	1260
24.7.2 Maintenance Strategies	1261
24.8 Real-World System Examples	1266
24.8.1 Example 1: Real-Time Content Moderation System	1266
24.8.2 Example 2: Dynamic Pricing System	1270
24.9 Chapter Summary	1274
24.10 Exercises	1275
25 Future Trends in ML Engineering	1279
25.1 Learning Objectives	1279
25.2 Introduction	1279
25.3 Emerging ML Engineering Technologies	1280
25.3.1 AutoML and Neural Architecture Search	1280
25.3.2 Continuous Learning Systems	1287
25.4 Edge ML and Federated Learning	1287
25.4.1 Edge ML and TinyML	1287
25.4.2 Federated Learning	1292
25.5 Large Language Model Operations (LLMOps)	1293
25.5.1 LLM-Specific Challenges	1293
25.5.2 LLM Efficiency and Optimization	1300
25.6 Green ML and Sustainability	1300
25.6.1 Carbon Footprint of ML	1300
25.6.2 Sustainable ML Practices	1305
25.7 Next-Generation MLOps Platforms	1306
25.7.1 Unified ML Platforms	1306
25.7.2 Declarative ML Systems	1307
25.8 Skill Development Recommendations	1309
25.8.1 Core Skills for Future ML Engineers	1309
25.8.2 Soft Skills and Mindset	1310

25.8.3	Certifications and Resources	1310
25.9	Chapter Summary	1310
25.10	Final Thoughts	1311
25.11	Exercises	1312
Setup Guides and Installation		1315
.1	Introduction	1315
.2	Python Environment Setup	1315
.2.1	Installing Python 3.9+	1315
.2.2	Virtual Environment Setup	1316
.2.3	Conda Environment Setup	1317
.3	Docker Installation and Configuration	1317
.3.1	Docker Desktop (Windows/macOS)	1317
.3.2	Docker Engine (Linux)	1318
.3.3	Docker Configuration for ML	1319
.4	Kubernetes Setup	1320
.4.1	Minikube for Local Development	1320
.4.2	Kind (Kubernetes in Docker)	1321
.5	Cloud Provider Setup	1321
.5.1	AWS CLI and SDK	1321
.5.2	Google Cloud SDK	1322
.5.3	Azure CLI	1323
.6	ML Framework Installation	1323
.6.1	PyTorch	1323
.6.2	TensorFlow	1324
.6.3	JAX	1324
.6.4	Data Science Stack	1324
.7	MLOps Tools Installation	1325
.7.1	MLflow	1325
.7.2	DVC (Data Version Control)	1325
.7.3	Apache Airflow	1326
.7.4	Kubeflow	1327
.7.5	Weights & Biases	1327
.8	GPU Drivers and CUDA	1327
.8.1	NVIDIA Drivers	1327
.8.2	CUDA Toolkit	1328
.8.3	cuDNN	1328
.8.4	NVIDIA Container Toolkit	1328
.9	Development Tools	1329
.9.1	Git Configuration	1329
.9.2	VS Code Setup for ML	1329
.10	Complete Environment Setup Script	1330
.11	Troubleshooting Common Issues	1334
.11.1	Python Installation Issues	1334
.11.2	Docker Issues	1334
.11.3	CUDA Issues	1334
.12	Verification Checklist	1335

Code Examples and Templates	1337
.13 Complete Training Pipeline Example	1337
.14 REST API for Model Serving	1338
.15 Data Validation with Great Expectations	1339
.16 Airflow DAG Template	1340
.17 Model Monitoring Script	1341
Tools and Frameworks Comparison	1343
.18 Experiment Tracking Tools	1343
.19 Model Serving Frameworks	1343
.20 Data Orchestration Tools	1344
.21 Feature Store Solutions	1344
.22 ML Monitoring Tools	1344
.23 Cloud ML Platforms	1344
.24 Container Orchestration	1344
.25 CI/CD for ML	1344
.26 Data Versioning	1344
.27 Model Compression Tools	1344
.28 Recommended Tool Stacks	1344
.28.1 Startup Stack (Cost-Conscious)	1344
.28.2 Mid-Size Company Stack	1345
.28.3 Enterprise Stack	1345
Mathematical Foundations	1347
.29 Linear Algebra Fundamentals	1347
.29.1 Matrix Operations	1347
.29.2 Eigenvalues and Eigenvectors	1347
.30 Probability and Statistics	1347
.30.1 Probability Distributions	1347
.30.2 Expected Value and Variance	1347
.30.3 Covariance and Correlation	1348
.31 Optimization	1348
.31.1 Gradient Descent	1348
.31.2 Stochastic Gradient Descent	1348
.31.3 Adam Optimizer	1348
.32 Loss Functions	1348
.32.1 Mean Squared Error	1348
.32.2 Cross-Entropy Loss	1349
.33 Regularization	1349
.33.1 L1 Regularization (Lasso)	1349
.33.2 L2 Regularization (Ridge)	1349
.33.3 Elastic Net	1349
.34 Information Theory	1349
.34.1 Entropy	1349
.34.2 KL Divergence	1349
.34.3 Mutual Information	1349
.35 Evaluation Metrics	1350
.35.1 Classification Metrics	1350

.35.2	Regression Metrics	1350
Glossary		1351
Code Style Guide		1357
Code Style Guide		1359
.36	Python Style Standards	1359
.36.1	General Principles	1359
.36.2	Naming Conventions	1359
.36.3	Import Organization	1360
.36.4	Type Hints	1360
.36.5	Docstrings	1361
.36.6	Error Handling	1363
.37	Configuration Management	1364
.37.1	YAML Configuration Files	1364
.37.2	Configuration Loading	1364
.38	Logging Standards	1365
.38.1	Logger Setup	1365
.39	Testing Patterns	1366
.39.1	Unit Test Structure	1366
.40	Code Comments	1368
.40.1	When to Comment	1368
.40.2	Comment Examples	1368
.41	Code Organization Patterns	1369
.41.1	Module Structure	1369
.42	Performance Patterns	1370
.42.1	Vectorization	1370
.43	Cross-Reference Style	1370
.44	Summary	1371

List of Figures

2.1	The ML lifecycle with feedback loops	28
3.1	ML training pipeline workflow in Airflow	37
3.2	Model validation strategies for different scenarios	59
3.3	A/B testing framework for ML models	66
4.1	Batch processing pipeline architecture	88
4.2	Streaming pipeline architecture with Kafka	90
4.3	ETL vs ELT architectural comparison	97
4.4	Feature store architecture with online and offline stores	106
4.5	Schema evolution with backward compatibility	110
6.1	Feature store architecture overview	192
6.2	Feast feature store architecture	201
6.3	Dual-mode feature serving architecture	219
9.1	Data parallelism: Model replicated across GPUs, data distributed . . .	310
9.2	Model parallelism: Model layers distributed across GPUs	317
9.3	Parameter server architecture	322
9.4	All-reduce architecture (ring topology)	323
23.1	E-Commerce Recommendation System Architecture	1177
23.2	Fraud Detection System Architecture	1200
24.1	Layered Architecture for End-to-End ML Systems	1239
24.2	Content Moderation System Architecture	1267

List of Tables

1	FraudGuard Success Metrics	xxxiv
2	FraudGuard Dataset Characteristics	xxxvii
3	FraudGuard Monthly Cost Budget	xxxviii
4	FraudGuard Evolution Milestones	xxxix
1.1	Data Science vs ML Engineering: Key Differences	4
1.2	Metric Misalignment Examples	20
3.1	ML pipeline orchestration tool comparison	36
3.2	Hyperparameter optimization strategy comparison	56
19.1	MLOps Platform Comparison Matrix	977
20.1	Risk Scoring Matrix	1049
24.1	Business Objective to ML Problem Mapping	1236
24.2	Architecture Trade-off Matrix	1243
24.3	Latency Optimization Techniques	1257
24.4	ML System Maturity Model	1261
25.1	AutoML Evolution Timeline	1280
25.2	Continuous Learning Challenges	1287
25.3	Federated Learning Challenges	1293
25.4	LLM Deployment Costs	1293
25.5	LLM Optimization Techniques (2024+)	1300
25.6	Next-Gen MLOps Platform Features	1306
25.7	ML Engineering Skills Roadmap (2025-2030)	1309
8	Comparison of Experiment Tracking Tools	1343
9	Model Serving Framework Comparison	1343
10	Data Orchestration Platform Comparison	1344
11	Feature Store Comparison	1344
12	ML Monitoring Tool Comparison	1345
13	Cloud ML Platform Features	1345
14	Container Orchestration Comparison	1346
15	CI/CD Tool Comparison	1346
16	Data Versioning Tools	1346
17	Model Optimization Frameworks	1346

List of Algorithms

Preface

About This Book

Machine learning has evolved from an academic curiosity to a fundamental technology driving innovation across industries. However, the path from a successful proof-of-concept model to a robust, scalable production system remains challenging. This book bridges the gap between data science experimentation and production-grade machine learning engineering.

Who Should Read This Book

This book is designed for:

- **Data Scientists** transitioning to ML engineering roles and seeking to understand production systems
- **ML Engineers** looking to deepen their understanding of training pipeline architecture and best practices
- **Software Engineers** entering the ML space who want to apply software engineering principles to ML systems
- **Technical Leads and Architects** responsible for designing and implementing ML infrastructure
- **DevOps Engineers** expanding into MLOps and seeking to understand ML-specific operational challenges

What You Will Learn

This handbook provides comprehensive coverage of:

- Designing robust data pipelines for ML training and inference
- Building scalable training architectures for distributed computing
- Implementing effective model serving and deployment strategies
- Establishing comprehensive monitoring and observability for ML systems
- Automating ML workflows through MLOps practices

- Optimizing training and inference performance
- Managing model governance, security, and compliance
- Real-world case studies from industry practitioners

Prerequisites

Readers should have:

- Basic understanding of machine learning concepts and algorithms
- Proficiency in Python programming
- Familiarity with common ML libraries (scikit-learn, TensorFlow, or PyTorch)
- Basic knowledge of software engineering principles
- Understanding of command-line operations and version control (Git)

How This Book Is Organized

The book is structured in eight parts:

Part I: Foundations establishes core concepts of ML engineering, the ML lifecycle, and development environment setup.

Part II: Data Engineering for ML covers data pipeline architecture, quality validation, and feature store implementation.

Part III: Training Pipeline Architecture explores model development best practices, experiment tracking, distributed training, and hyperparameter optimization.

Part IV: Model Deployment and Serving details serving architectures, deployment strategies, and model versioning.

Part V: Production ML Operations addresses monitoring, model maintenance, and performance optimization in production.

Part VI: MLOps and Automation examines CI/CD for ML, infrastructure as code, and MLOps platforms.

Part VII: Advanced Topics covers governance, security, and cost optimization.

Part VIII: Case Studies and Integration presents real-world examples and discusses future trends.

Conventions Used in This Book

Throughout this book, we use the following conventions:

- **Monospace font** for code, commands, and file names
- **Bold** for emphasis and important terms
- *Italic* for new concepts and terminology

Code Examples

All code examples are available in the book's GitHub repository. Examples use modern tools and frameworks including:

- Python 3.9+
- PyTorch, TensorFlow, and scikit-learn
- MLflow for experiment tracking
- Apache Airflow for workflow orchestration
- Docker and Kubernetes for containerization
- Cloud platforms (AWS, GCP, Azure)

Acknowledgments

This book draws on experiences from building production ML systems across various domains and scales. I am grateful to the ML engineering community for their open-source contributions, shared knowledge, and continuous innovation in this rapidly evolving field.

Feedback and Errata

We welcome feedback, suggestions, and corrections. Please visit the book's website or GitHub repository to report issues or contribute improvements.

Diogo Ribeiro
Porto, Portugal
November 24, 2025

Running Case Study: FraudGuard Payment System

Throughout this book, we build and evolve **FraudGuard**, a production-ready fraud detection system for payment processing. This unified case study provides continuity across chapters, demonstrating how concepts integrate in real-world systems.

0.1 System Overview

FraudGuard is a real-time fraud detection system deployed at a mid-sized fintech company processing 10 million transactions per day (\$500M daily volume). The system must:

- Detect fraudulent transactions with 95% precision and 85% recall
- Return predictions within 100ms (p99 latency)
- Handle traffic spikes during peak hours (3x normal load)
- Comply with PCI-DSS and financial regulations
- Minimize false positives to avoid blocking legitimate customers
- Adapt to evolving fraud patterns (continuous learning)

0.2 Business Context

0.2.1 Problem Statement

The company loses approximately \$2M annually to fraud (0.4% of transaction volume). Each fraud investigation costs \$150, and chargebacks cost \$25 per incident. Conversely, false positives (blocking legitimate transactions) cause customer frustration and lost revenue estimated at \$500 per blocked transaction.

0.2.2 Success Metrics

0.3 System Architecture Evolution

0.3.1 Phase 1: Initial MVP (Chapters 1-7)

Covered in: Chapters 1, 2, 3, 4, 5, 6, 7

System components:

Table 1: FraudGuard Success Metrics

Metric	Current	Target	Business Impact
Fraud Detection Rate	72%	85%	\$800K annual savings
False Positive Rate	2.5%	1.0%	\$15M revenue protection
P99 Latency	350ms	100ms	Improved user experience
Model Retraining	Manual, quarterly	Automated, daily	Faster adaptation

- Batch data pipeline processing transaction history
- Feature engineering for transaction patterns
- XGBoost classifier trained weekly
- Simple REST API for predictions
- Basic logging and metrics

Challenges:

- Manual data quality checks (Chapter 5)
- No feature reusability across models (Chapter 6)
- Inconsistent experiment tracking (Chapter 8)
- Slow model iteration (2-week cycle)

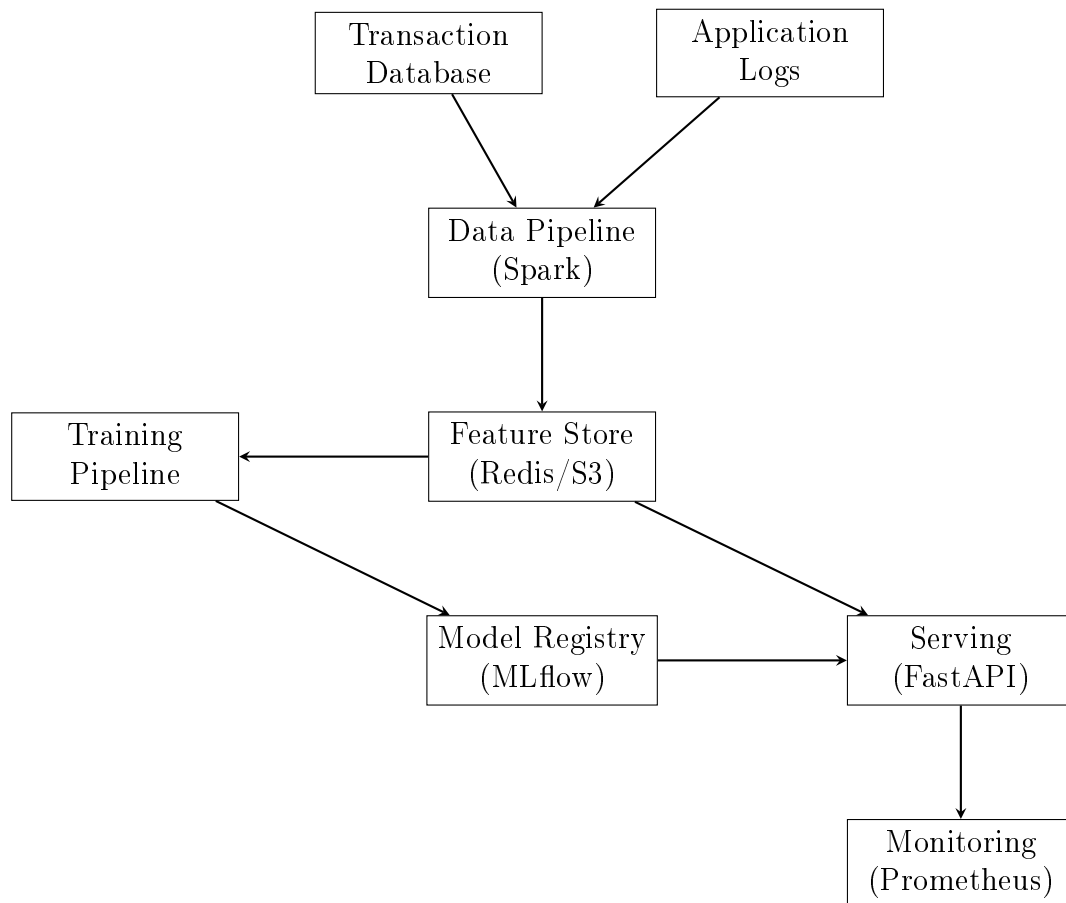
0.3.2 Phase 2: Production System (Chapters 8-13)

Covered in: Chapters 8, 9, 10, 11, 12, 13

Improvements:

- MLflow for experiment tracking and model registry
- Distributed training with Spark for faster iteration
- Bayesian optimization for hyperparameter tuning (Chapter 10)
- FastAPI serving with Redis caching (Chapter 11)
- Blue-green deployment strategy (Chapter 12)
- Model versioning and A/B testing (Chapter 13)

Architecture diagram:



0.3.3 Phase 3: Production Excellence (Chapters 14-19)

Covered in: Chapters 14, 15, 16, 17, 18, 19

Advanced capabilities:

- Real-time drift detection (KS test, PSI) (Chapter 14)
- Automated retraining triggers (Chapter 15)
- Model quantization for faster inference (Chapter 16)
- Complete CI/CD pipeline (Chapter 17)
- Infrastructure as Code with Terraform (Chapter 18)
- Integration with Kubeflow (Chapter 19)

0.3.4 Phase 4: Governance and Scale (Chapters 20-22)

Covered in: Chapters 20, 21, 22

Enterprise requirements:

- Model cards for compliance (Chapter 20)
- Differential privacy for sensitive data (Chapter 21)
- Cost optimization with spot instances (Chapter 22)

- Fairness metrics and bias detection
- Audit trail for all predictions

0.4 Data Schema

0.4.1 Transaction Features

```
1 from dataclasses import dataclass
2 from datetime import datetime
3 from typing import Optional, List
4
5 @dataclass
6 class TransactionFeatures:
7     """Feature schema for fraud detection"""
8
9     # Transaction details
10    transaction_id: str
11    amount: float
12    currency: str
13    timestamp: datetime
14    merchant_id: str
15    merchant_category: str
16
17    # User features
18    user_id: str
19    account_age_days: int
20    total_transactions_30d: int
21    total_amount_30d: float
22    avg_transaction_amount_30d: float
23
24    # Behavioral features
25    time_since_last_transaction_minutes: float
26    is_international: bool
27    distance_from_home_km: float
28    device_id: str
29    ip_address: str
30
31    # Velocity features (derived)
32    transactions_last_hour: int
33    transactions_last_24h: int
34    amount_last_hour: float
35    unique_merchants_last_24h: int
36
37    # Risk signals
38    failed_auth_attempts_24h: int
39    shipping_address_matches_billing: bool
40    email_verified: bool
41    phone_verified: bool
42
43    # Target
44    is_fraud: Optional[bool] = None
```

Listing 1: FraudGuard Feature Schema

0.4.2 Sample Data Statistics

Table 2: FraudGuard Dataset Characteristics

Characteristic	Value
Total transactions (30 days)	300M
Fraud rate	0.4% (1.2M fraudulent)
Avg. transaction value	\$47.50
Median transaction value	\$23.00
P99 transaction value	\$485.00
Number of unique users	15M
Number of features	47 (35 raw + 12 derived)
Data volume (30 days)	450 GB

0.5 Model Performance Requirements

0.5.1 Prediction Requirements

- **Latency:** p99 < 100ms, p50 < 30ms
- **Throughput:** 10,000 predictions/second (peak)
- **Availability:** 99.9% uptime
- **Precision:** $\geq 95\%$ (minimize false positives)
- **Recall:** $\geq 85\%$ (catch most fraud)
- **F1 Score:** ≥ 0.90

0.5.2 Training Requirements

- **Training time:** < 2 hours for full retrain
- **Data freshness:** < 24 hours
- **Model update frequency:** Daily (automated)
- **Experiment tracking:** All experiments logged
- **Reproducibility:** 100% reproducible training

0.6 Cost Budget

0.7 Chapter-by-Chapter Application

This section shows how FraudGuard concepts map to each chapter:

Chapter 1: Introduction to the business problem and ROI analysis

Table 3: FraudGuard Monthly Cost Budget

Component	Monthly Cost	Chapter Reference
Training (EMR Spot)	\$2,500	Chapter 9
Feature Store (Redis)	\$1,800	Chapter 6
Serving (EKS)	\$3,200	Chapter 11
Monitoring (Prometheus/Grafana)	\$400	Chapter 14
Storage (S3)	\$1,100	Chapter 4
MLflow	\$600	Chapter 8
Total	\$9,600	

Chapter 2: Full lifecycle from research to production deployment

Chapter 3: Setting up development environment and notebook structure

Chapter 4: Building Spark pipeline for transaction processing

Chapter 5: Implementing data validation with Great Expectations

Chapter 6: Creating feature store for reusable features

Chapter 7: Refactoring notebook code to production modules

Chapter 8: Tracking experiments with MLflow

Chapter 9: Distributed XGBoost training on Spark

Chapter 10: Bayesian optimization with Optuna

Chapter 11: FastAPI serving with Redis caching

Chapter 12: Blue-green deployment and A/B testing

Chapter 13: Model registry and version management

Chapter 14: Drift detection and performance monitoring

Chapter 15: Automated retraining pipelines

Chapter 16: Model quantization and latency optimization

Chapter 17: GitHub Actions pipeline for continuous deployment

Chapter 18: Terraform for AWS infrastructure

Chapter 19: Kubeflow Pipelines integration

Chapter 20: Model cards and audit trails

Chapter 21: PCI-DSS compliance and differential privacy

Chapter 22: Spot instances and right-sizing

Table 4: FraudGuard Evolution Milestones

Timeline	Phase	Key Changes
Week 1-2	MVP	Jupyter notebook, manual training, CSV files
Week 3-4	Basic Pipeline	Automated data pipeline, weekly re-training
Month 2	Feature Store	Centralized features, reduced duplication
Month 3	Production Serving	FastAPI, load balancing, monitoring
Month 4	Automation	CI/CD, automated retraining, drift detection
Month 5	Optimization	Model quantization, cost reduction 40%
Month 6	Governance	Compliance, audit trails, model cards

0.8 Evolution Timeline

0.9 Key Learnings and Patterns

Throughout the FraudGuard case study, we demonstrate these recurring patterns:

1. **Start simple, iterate fast:** Begin with basic XGBoost before complex deep learning
2. **Measure everything:** Comprehensive logging from day one (Chapter 14)
3. **Automate incrementally:** Manual → scheduled → event-driven automation
4. **Feature reusability:** Feature store reduces duplication by 70%
5. **Test in production:** A/B testing for safe deployment (Chapter 12)
6. **Cost-performance tradeoffs:** Optimize after establishing baseline
7. **Regulatory compliance:** Build in from the start, not retrofitted

0.10 Code Repository Structure

The complete FraudGuard codebase follows this structure:

```

1 fraudguard/
2 |-- config/
3 |   |-- data_pipeline.yaml
4 |   |-- training.yaml
5 |   -- serving.yaml
6 |-- src/
7 |   |-- data/
8 |   |   |-- loaders.py
9 |   |   |-- validators.py

```

```
10 | | -- processors.py
11 | | -- features/
12 | | | -- feature_store.py
13 | | | -- transformers.py
14 | | -- feature_specs.yaml
15 | | -- models/
16 | | | -- train.py
17 | | | -- predict.py
18 | | | -- evaluate.py
19 | | -- serving/
20 | | | -- api.py
21 | | | -- cache.py
22 | | | -- monitoring.py
23 | -- monitoring/
24 | | -- drift_detection.py
25 | | -- alerting.py
26 | -- tests/
27 | -- infrastructure/
28 | | -- terraform/
29 | | -- kubernetes/
30 | -- .github/
31 | | -- workflows/
32 | -- notebooks/
33 | -- docs/
```

Best Practice: *The FraudGuard case study is referenced throughout the book with specific examples. When you see "In our FraudGuard system..." or references to fraud detection, it refers to this unified example. Code snippets build upon each other, showing how a production system evolves from concept to enterprise-scale deployment.*

Part I

Foundations of ML Engineering

Chapter 1

Introduction to ML Engineering

1.1 Learning Objectives

After completing this chapter, readers will be able to:

- Distinguish between data science and ML engineering roles, responsibilities, and skill sets
- Understand the complete ML lifecycle from research to production deployment
- Identify common failure modes in ML projects and their root causes
- Apply architectural principles for building reliable ML systems
- Recognize the differences between exploratory research code and production ML code
- Avoid common anti-patterns that lead to technical debt in ML systems
- Assess organizational ML maturity and create improvement roadmaps

1.2 From Data Science to ML Engineering

1.2.1 The Evolution of ML in Industry

The journey from academic research to production machine learning represents one of the most significant challenges facing organizations today. While machine learning has existed for decades in research laboratories, its widespread adoption in production systems is relatively recent—and fraught with difficulties.

Industry statistics paint a sobering picture: according to Gartner [Gartner, 2020], only 22% of companies using ML have successfully deployed models to production. VentureBeat reports that 87% of ML projects never make it beyond the experimental phase [VentureBeat, 2019]. These failures cost organizations an average of \$1.2 million per failed initiative, representing not just wasted resources but missed opportunities for competitive advantage.

The root cause of these failures rarely stems from insufficient algorithmic sophistication. Instead, the problem lies in the fundamental absence of systematic engineering practices. While data scientists excel at exploratory analysis and model development,

the skills required to deploy, monitor, and maintain ML systems in production are fundamentally different—and often entirely absent from data science training.

1.2.2 Defining Data Science vs ML Engineering

To understand the distinction between these roles, consider the following definitions:

Definition 1.1 (Data Science). Data science is the discipline of extracting insights and knowledge from data through statistical analysis, visualization, and machine learning. Data scientists focus on answering business questions, testing hypotheses, and building predictive models that demonstrate value in experimental settings.

Definition 1.2 (ML Engineering). Machine learning engineering is the discipline of designing, building, deploying, and maintaining scalable, reliable, and efficient systems that enable machine learning models to operate successfully in production environments. ML engineers focus on system reliability, performance, scalability, and operational excellence.

The key differences extend beyond these definitions into daily work, deliverables, and success metrics:

Table 1.1: Data Science vs ML Engineering: Key Differences

Aspect	Data Scientist	ML Engineer
Primary Focus	Model accuracy, insights	System reliability, scalability
Main Deliverable	Analysis, prototypes	Production systems
Success Metric	Model performance (AUC, R^2)	Uptime, latency, cost
Time Allocation	70% exploration, 30% communication	70% engineering, 30% optimization
Tools	Jupyter, R, pandas, scikit-learn	Docker, Kubernetes, Airflow, MLflow
Code Quality	Exploratory, experimental	Production-grade, tested
Deployment	Handoff to engineering	End-to-end ownership
Monitoring	Model metrics	System health, drift, performance

1.2.3 The Skills Gap

The Venn diagram of skills reveals significant overlap but crucial differences:

Data Science Skills:

- Statistical analysis and hypothesis testing
- Feature engineering and selection
- Algorithm selection and tuning
- Data visualization and storytelling
- Domain knowledge and business acumen

- Exploratory data analysis

ML Engineering Skills:

- Software engineering best practices
- Distributed systems and scalability
- DevOps and infrastructure automation
- API design and microservices
- Monitoring and observability
- Performance optimization
- Production debugging

Shared Skills:

- Python programming
- Machine learning fundamentals
- Data processing and transformation
- Model evaluation techniques
- Version control (Git)

1.3 Research Code vs Production Code

The difference between exploratory research code and production ML systems is vast. Let us examine concrete examples that illustrate this transformation.

1.3.1 The Research Notebook Approach

Consider a typical data science workflow in a Jupyter notebook:

```
1 import pandas as pd
2 import numpy as np
3 from sklearn.ensemble import RandomForestClassifier
4 from sklearn.model_selection import train_test_split
5 from sklearn.metrics import accuracy_score
6
7 # Load data
8 df = pd.read_csv('customer_churn.csv')
9
10 # Quick preprocessing
11 df = df.dropna()
12 df['tenure_months'] = df['tenure_days'] / 30
13 df = pd.get_dummies(df, columns=['plan_type', 'device_type'])
14
15 # Split data
16 X = df.drop('churned', axis=1)
17 y = df['churned']
```

```
18 X_train, X_test, y_train, y_test = train_test_split(
19     X, y, test_size=0.2, random_state=42
20 )
21
22 # Train model
23 model = RandomForestClassifier(n_estimators=100, random_state=42)
24 model.fit(X_train, y_train)
25
26 # Evaluate
27 y_pred = model.predict(X_test)
28 accuracy = accuracy_score(y_test, y_pred)
29 print(f"Accuracy: {accuracy:.2%}")
30 # Output: Accuracy: 92%
31
32 # Save model
33 import pickle
34 with open('model.pkl', 'wb') as f:
35     pickle.dump(model, f)
```

Listing 1.1: Typical Data Science Notebook Code

This code "works"—it trains a model and achieves 92% accuracy. However, it suffers from numerous problems that make it unsuitable for production:

Warning: *Common issues with research notebook code:*

- *No error handling or logging*
- *Hardcoded file paths and parameters*
- *No data validation or quality checks*
- *Irreproducible (random states scattered, environment not versioned)*
- *No separation of concerns (data loading, preprocessing, training all mixed)*
- *No testing or monitoring capabilities*
- *Unsafe model serialization (pickle security issues)*
- *No configuration management*
- *Missing documentation*
- *No consideration for deployment environment*

1.3.2 Production-Grade ML Code

Now consider a production-ready implementation of the same task:

```
1 # config/model_config.yaml
2 model:
3   name: "churn_predictor"
4   version: "v1.0"
5   type: "random_forest"
6   hyperparameters:
7     n_estimators: 100
8     max_depth: 10
9     min_samples_split: 5
10    random_state: 42
```

```
11 data:
12   train_path: "s3://ml-data/churn/train/"
13   test_path: "s3://ml-data/churn/test/"
14   features:
15     numeric:
16       - tenure_days
17       - monthly_charges
18       - total_charges
19     categorical:
20       - plan_type
21       - device_type
22   target: "churned"
23
24 preprocessing:
25   handle_missing: "median"
26   scaling: "standard"
27   encoding: "onehot"
28
29 validation:
30   test_size: 0.2
31   cv_folds: 5
32   stratify: true
33
34 monitoring:
35   track_experiments: true
36   log_artifacts: true
37   alert_threshold: 0.85
38
```

Listing 1.2: Production ML Pipeline - Configuration

```
1  # src/data/data_loader.py
2  from typing import Tuple, List, Optional
3  import pandas as pd
4  import logging
5  from pathlib import Path
6  from dataclasses import dataclass
7
8  logger = logging.getLogger(__name__)
9
10 @dataclass
11 class DataConfig:
12     """Configuration for data loading"""
13     train_path: str
14     test_path: Optional[str] = None
15     features: List[str] = None
16     target: str = "target"
17
18 class DataLoader:
19     """Responsible for loading and validating data"""
20
21     def __init__(self, config: DataConfig):
22         self.config = config
23         self.logger = logging.getLogger(self.__class__.__name__)
24
25     def load_data(self, path: str) -> pd.DataFrame:
26         """
27         Load data from specified path with error handling
28     """
```

```

28
29     Args:
30         path: Path to data file (local or S3)
31
32     Returns:
33         Loaded DataFrame
34
35     Raises:
36         FileNotFoundError: If data file doesn't exist
37         ValueError: If data format is invalid
38     """
39     try:
40         self.logger.info(f"Loading data from {path}")
41
42         if path.startswith('s3://'):
43             df = self._load_from_s3(path)
44         else:
45             df = pd.read_csv(path)
46
47         self.logger.info(
48             f"Loaded {len(df)} records with {len(df.columns)} columns"
49         )
50
51         # Validate data
52         self._validate_data(df)
53
54         return df
55
56     except FileNotFoundError as e:
57         self.logger.error(f"Data file not found: {path}")
58         raise
59     except Exception as e:
60         self.logger.error(f"Error loading data: {str(e)}")
61         raise ValueError(f"Failed to load data from {path}") from e
62
63 def _validate_data(self, df: pd.DataFrame) -> None:
64     """Validate loaded data"""
65     # Check required columns
66     required_cols = set(self.config.features + [self.config.target])
67     missing_cols = required_cols - set(df.columns)
68
69     if missing_cols:
70         raise ValueError(
71             f"Missing required columns: {missing_cols}"
72         )
73
74     # Check for empty dataframe
75     if df.empty:
76         raise ValueError("Loaded dataframe is empty")
77
78     # Check target variable
79     if df[self.config.target].isnull().all():
80         raise ValueError("Target variable is all null")
81
82     # Log data quality metrics
83     null_pct = (df.isnull().sum() / len(df) * 100).to_dict()
84     self.logger.info(f"Null percentages: {null_pct}")
85

```

```

86 def _load_from_s3(self, s3_path: str) -> pd.DataFrame:
87     """Load data from S3"""
88     import boto3
89     from io import StringIO
90
91     # Parse S3 path
92     parts = s3_path.replace('s3://', '').split('/')
93     bucket = parts[0]
94     key = '/'.join(parts[1:])
95
96     # Download from S3
97     s3_client = boto3.client('s3')
98     obj = s3_client.get_object(Bucket=bucket, Key=key)
99     df = pd.read_csv(StringIO(obj['Body'].read().decode('utf-8')))
100
101     return df
102
103 def load_train_test_split(
104     self,
105     test_size: float = 0.2,
106     stratify: bool = True,
107     random_state: int = 42
108 ) -> Tuple[pd.DataFrame, pd.DataFrame]:
109     """Load and split data into train and test sets"""
110     from sklearn.model_selection import train_test_split
111
112     df = self.load_data(self.config.train_path)
113
114     stratify_col = df[self.config.target] if stratify else None
115
116     train_df, test_df = train_test_split(
117         df,
118         test_size=test_size,
119         random_state=random_state,
120         stratify=stratify_col
121     )
122
123     self.logger.info(
124         f"Split data: {len(train_df)} train, {len(test_df)} test"
125     )
126
127     return train_df, test_df

```

Listing 1.3: Production ML Pipeline - Data Module

```

1  # src/preprocessing/preprocessor.py
2  from typing import List, Dict, Any
3  import pandas as pd
4  import numpy as np
5  from sklearn.preprocessing import StandardScaler, OneHotEncoder
6  from sklearn.impute import SimpleImputer
7  from sklearn.compose import ColumnTransformer
8  from sklearn.pipeline import Pipeline
9  import joblib
10 import logging
11
12 logger = logging.getLogger(__name__)
13

```

```

14 class FeaturePreprocessor:
15     """Handle feature preprocessing with reproducibility"""
16
17     def __init__(
18         self,
19         numeric_features: List[str],
20         categorical_features: List[str],
21         config: Dict[str, Any]
22     ):
23         self.numeric_features = numeric_features
24         self.categorical_features = categorical_features
25         self.config = config
26         self.pipeline = self._build_pipeline()
27         self.is_fitted = False
28         self.logger = logging.getLogger(self.__class__.__name__)
29
30     def _build_pipeline(self) -> ColumnTransformer:
31         """Build preprocessing pipeline"""
32
33         # Numeric preprocessing
34         numeric_transformer = Pipeline(steps=[
35             ('imputer', SimpleImputer(
36                 strategy=self.config.get('handle_missing', 'median')
37             )),
38             ('scaler', StandardScaler())
39         ])
40
41         # Categorical preprocessing
42         categorical_transformer = Pipeline(steps=[
43             ('imputer', SimpleImputer(
44                 strategy='constant',
45                 fill_value='missing'
46             )),
47             ('encoder', OneHotEncoder(
48                 handle_unknown='ignore',
49                 sparse_output=False # Updated for sklearn>=1.2 (2024)
50             ))
51         ])
52
53         # Combine transformers
54         preprocessor = ColumnTransformer(
55             transformers=[
56                 ('num', numeric_transformer, self.numeric_features),
57                 ('cat', categorical_transformer, self.categorical_features)
58             ],
59             remainder='drop'
60         )
61
62         return preprocessor
63
64     def fit(self, X: pd.DataFrame) -> 'FeaturePreprocessor':
65         """Fit preprocessor on training data"""
66         self.logger.info("Fitting preprocessor")
67         self.pipeline.fit(X)
68         self.is_fitted = True
69         return self
70
71     def transform(self, X: pd.DataFrame) -> np.ndarray:

```

```

72     """Transform data using fitted preprocessor"""
73     if not self.is_fitted:
74         raise RuntimeError(
75             "Preprocessor must be fitted before transform"
76         )
77
78     self.logger.info(f"Transforming {len(X)} samples")
79     return self.pipeline.transform(X)
80
81     def fit_transform(self, X: pd.DataFrame) -> np.ndarray:
82         """Fit and transform in one step"""
83         return self.fit(X).transform(X)
84
85     def save(self, path: str) -> None:
86         """Save fitted preprocessor"""
87         if not self.is_fitted:
88             raise RuntimeError("Cannot save unfitted preprocessor")
89
90         self.logger.info(f"Saving preprocessor to {path}")
91         joblib.dump(self.pipeline, path)
92
93     @classmethod
94     def load(cls, path: str) -> 'FeaturePreprocessor':
95         """Load fitted preprocessor"""
96         logger.info(f"Loading preprocessor from {path}")
97         instance = cls([], [], {})
98         instance.pipeline = joblib.load(path)
99         instance.is_fitted = True
100        return instance

```

Listing 1.4: Production ML Pipeline - Preprocessing Module

```

1  # src/training/trainer.py
2  from typing import Dict, Any, Tuple
3  import numpy as np
4  import pandas as pd
5  from sklearn.ensemble import RandomForestClassifier
6  from sklearn.model_selection import cross_val_score
7  from sklearn.metrics import (
8      accuracy_score, precision_score, recall_score,
9      f1_score, roc_auc_score, classification_report
10 )
11 import mlflow
12 import mlflow.sklearn
13 import logging
14 from datetime import datetime
15 import joblib
16
17 logger = logging.getLogger(__name__)
18
19 class ModelTrainer:
20     """Handle model training with experiment tracking"""
21
22     def __init__(self, config: Dict[str, Any]):
23         self.config = config
24         self.model = None
25         self.metrics = {}
26         self.logger = logging.getLogger(self.__class__.__name__)

```

```

27
28 def _create_model(self) -> RandomForestClassifier:
29     """Create model instance from config"""
30     model_type = self.config['model']['type']
31     params = self.config['model']['hyperparameters']
32
33     if model_type == 'random_forest':
34         return RandomForestClassifier(**params)
35     else:
36         raise ValueError(f"Unknown model type: {model_type}")
37
38 def train(
39     self,
40     X_train: np.ndarray,
41     y_train: np.ndarray,
42     X_val: np.ndarray = None,
43     y_val: np.ndarray = None
44 ) -> RandomForestClassifier:
45     """
46     Train model with cross-validation and tracking
47
48     Args:
49         X_train: Training features
50         y_train: Training labels
51         X_val: Validation features (optional)
52         y_val: Validation labels (optional)
53
54     Returns:
55         Trained model
56     """
57     self.logger.info("Starting model training")
58
59     # Set up MLflow
60     mlflow.set_experiment(self.config['model']['name'])
61
62     with mlflow.start_run(
63         run_name=f"run_{datetime.now().strftime('%Y%m%d_%H%M%S')}"
64     ) as run:
65
66         # Create and train model
67         self.model = self._create_model()
68
69         # Cross-validation
70         if self.config['validation'].get('cv_folds', 0) > 0:
71             cv_scores = cross_val_score(
72                 self.model,
73                 X_train,
74                 y_train,
75                 cv=self.config['validation']['cv_folds'],
76                 scoring='accuracy'
77             )
78             self.logger.info(
79                 f"CV Accuracy: {cv_scores.mean():.4f} "
80                 f"(/- {cv_scores.std():.4f})"
81             )
82             mlflow.log_metric("cv_accuracy_mean", cv_scores.mean())
83             mlflow.log_metric("cv_accuracy_std", cv_scores.std())
84

```



```

85         # Train on full training set
86         self.logger.info("Training on full training set")
87         self.model.fit(X_train, y_train)
88
89         # Evaluate on validation set if provided
90         if X_val is not None and y_val is not None:
91             self.metrics = self._evaluate(X_val, y_val)
92
93         # Log metrics
94         for metric_name, metric_value in self.metrics.items():
95             mlflow.log_metric(metric_name, metric_value)
96
97         # Check if meets threshold
98         if self.metrics['accuracy'] < \
99             self.config['monitoring']['alert_threshold']:
100             self.logger.warning(
101                 f"Model accuracy {self.metrics['accuracy']:.4f} "
102                 f"below threshold "
103                 f"{self.config['monitoring']['alert_threshold']}"
104             )
105
106         # Log model parameters
107         mlflow.log_params(self.config['model']['hyperparameters'])
108
109         # Log model
110         mlflow.sklearn.log_model(
111             self.model,
112             "model",
113             registered_model_name=self.config['model']['name']
114         )
115
116         self.logger.info(f"Model training complete. Run ID: {run.info.run_id}")
117
118         return self.model
119
120     def _evaluate(
121         self,
122         X: np.ndarray,
123         y: np.ndarray
124     ) -> Dict[str, float]:
125         """Evaluate model performance"""
126         self.logger.info("Evaluating model")
127
128         y_pred = self.model.predict(X)
129         y_pred_proba = self.model.predict_proba(X)[:, 1]
130
131         metrics = {
132             'accuracy': accuracy_score(y, y_pred),
133             'precision': precision_score(y, y_pred, average='binary'),
134             'recall': recall_score(y, y_pred, average='binary'),
135             'f1': f1_score(y, y_pred, average='binary'),
136             'roc_auc': roc_auc_score(y, y_pred_proba)
137         }
138
139         # Log classification report
140         report = classification_report(y, y_pred)
141         self.logger.info(f"\nClassification Report:\n{report}")
142

```

```

143         return metrics
144
145     def save_model(self, path: str) -> None:
146         """Save trained model"""
147         if self.model is None:
148             raise RuntimeError("No model to save. Train model first.")
149
150         self.logger.info(f"Saving model to {path}")
151         joblib.dump(self.model, path)

```

Listing 1.5: Production ML Pipeline - Training Module

```

1  # src/pipeline.py
2  from typing import Dict, Any
3  import yaml
4  import logging
5  from pathlib import Path
6  import sys
7
8  from data.data_loader import DataLoader, DataConfig
9  from preprocessing.preprocessor import FeaturePreprocessor
10 from training.trainer import ModelTrainer
11
12 # Setup logging
13 logging.basicConfig(
14     level=logging.INFO,
15     format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
16     handlers=[
17         logging.FileHandler('training.log'),
18         logging.StreamHandler(sys.stdout)
19     ]
20 )
21
22 logger = logging.getLogger(__name__)
23
24 class MLPipeline:
25     """End-to-end ML training pipeline"""
26
27     def __init__(self, config_path: str):
28         """Initialize pipeline with configuration"""
29         self.config = self._load_config(config_path)
30         self.logger = logging.getLogger(self.__class__.__name__)
31
32     def _load_config(self, config_path: str) -> Dict[str, Any]:
33         """Load configuration from YAML file"""
34         with open(config_path, 'r') as f:
35             config = yaml.safe_load(f)
36         return config
37
38     def run(self) -> Dict[str, Any]:
39         """Execute complete ML pipeline"""
40         try:
41             self.logger.info("="*50)
42             self.logger.info("Starting ML Pipeline")
43             self.logger.info("="*50)
44
45             # Step 1: Load data
46             self.logger.info("Step 1: Loading data")

```

```

47     data_config = DataConfig(
48         train_path=self.config['data']['train_path'],
49         features=(
50             self.config['data']['features']['numeric'] +
51             self.config['data']['features']['categorical']
52         ),
53         target=self.config['data']['target']
54     )
55     data_loader = DataLoader(data_config)
56
57     train_df, test_df = data_loader.load_train_test_split(
58         test_size=self.config['validation']['test_size'],
59         stratify=self.config['validation']['stratify']
60     )
61
62     # Separate features and target
63     X_train = train_df.drop(self.config['data']['target'], axis=1)
64     y_train = train_df[self.config['data']['target']]
65     X_test = test_df.drop(self.config['data']['target'], axis=1)
66     y_test = test_df[self.config['data']['target']]
67
68     # Step 2: Preprocess data
69     self.logger.info("Step 2: Preprocessing data")
70     preprocessor = FeaturePreprocessor(
71         numeric_features=self.config['data']['features']['numeric'],
72         categorical_features=self.config['data']['features']['categorical'],
73         config=self.config['preprocessing']
74     )
75
76     X_train_processed = preprocessor.fit_transform(X_train)
77     X_test_processed = preprocessor.transform(X_test)
78
79     # Save preprocessor
80     output_dir = Path(self.config.get('output_dir', 'outputs'))
81     output_dir.mkdir(exist_ok=True)
82     preprocessor.save(output_dir / 'preprocessor.pkl')
83
84     # Step 3: Train model
85     self.logger.info("Step 3: Training model")
86     trainer = ModelTrainer(self.config)
87     model = trainer.train(
88         X_train_processed,
89         y_train.values,
90         X_test_processed,
91         y_test.values
92     )
93
94     # Save model
95     trainer.save_model(output_dir / 'model.pkl')
96
97     # Compilation results
98     results = {
99         'status': 'success',
100         'metrics': trainer.metrics,
101         'model_path': str(output_dir / 'model.pkl'),
102         'preprocessor_path': str(output_dir / 'preprocessor.pkl'),
103         'config': self.config
104     }

```

```

105         self.logger.info("=*50)
106         self.logger.info("Pipeline completed successfully")
107         self.logger.info(f"Final metrics: {trainer.metrics}")
108         self.logger.info("=*50)
109
110     return results
111
112 except Exception as e:
113     self.logger.error(f"Pipeline failed: {str(e)}", exc_info=True)
114     return {
115         'status': 'failed',
116         'error': str(e)
117     }
118
119 if __name__ == "__main__":
120     import argparse
121
122     parser = argparse.ArgumentParser(description='Run ML training pipeline')
123     parser.add_argument(
124         '--config',
125         type=str,
126         required=True,
127         help='Path to configuration file'
128     )
129
130     args = parser.parse_args()
131
132     pipeline = MLPipeline(args.config)
133     results = pipeline.run()
134
135     # Exit with appropriate code
136     sys.exit(0 if results['status'] == 'success' else 1)
137

```

Listing 1.6: Production ML Pipeline - Main Pipeline

This production code demonstrates several critical improvements:

Best Practice: *Production ML code characteristics:*

- **Modularity:** Separated concerns into distinct modules (data, preprocessing, training)
- **Type Hints:** Clear type annotations for all function signatures
- **Error Handling:** Comprehensive try-except blocks with meaningful error messages
- **Logging:** Detailed logging at appropriate levels
- **Configuration:** External YAML configuration instead of hardcoded values
- **Validation:** Data validation checks throughout the pipeline
- **Reproducibility:** Versioned preprocessing and model artifacts
- **Monitoring:** MLflow integration for experiment tracking
- **Testing:** Code structure that enables unit and integration testing
- **Documentation:** Docstrings following standard conventions

1.4 The ML Lifecycle in Production

Understanding the complete ML lifecycle is essential for successful production deployment. The lifecycle extends far beyond model training to encompass the entire journey from problem definition to ongoing maintenance.

1.4.1 Lifecycle Phases

The production ML lifecycle consists of seven distinct phases, each with specific activities, deliverables, and success criteria:

1. Problem Definition and Feasibility

- Define business objectives and success metrics
- Assess data availability and quality
- Evaluate ML appropriateness for the problem
- Estimate resource requirements and timeline
- Identify stakeholders and constraints

2. Data Collection and Preparation

- Identify and acquire necessary data sources
- Design data pipeline architecture
- Implement data quality checks and validation
- Create data versioning strategy
- Establish data governance and privacy compliance

3. Exploratory Data Analysis and Feature Engineering

- Understand data distributions and relationships
- Identify data quality issues
- Design and implement feature transformations
- Create feature store for reusability
- Document feature definitions and lineage

4. Model Development and Experimentation

- Establish baseline model performance
- Experiment with multiple algorithms
- Perform hyperparameter optimization
- Implement cross-validation strategy
- Track experiments with MLflow or similar

5. Model Evaluation and Validation

- Evaluate on held-out test set

- Assess fairness and bias
- Validate business impact
- Perform error analysis
- Document model limitations

6. Deployment and Integration

- Package model for serving
- Design serving infrastructure
- Implement deployment strategy (canary, blue-green)
- Create API endpoints
- Integrate with existing systems

7. Monitoring and Maintenance

- Monitor model performance metrics
- Detect data and concept drift
- Track system health and latency
- Implement automated retraining
- Manage model versioning and rollback

1.4.2 Iterative Nature of the Lifecycle

Unlike traditional software development, the ML lifecycle is inherently iterative. Monitoring results often trigger returns to earlier phases:

- **Performance degradation** may require model retraining or redesign
- **Data drift** necessitates updated preprocessing or new features
- **Business requirement changes** demand problem redefinition
- **New data sources** enable feature engineering improvements
- **Production insights** inform better data collection strategies

1.5 Common Failure Modes in ML Projects

Understanding why ML projects fail is crucial for avoiding these pitfalls. Gartner and other industry analysts have identified consistent patterns in failed ML initiatives.

1.5.1 Technical Failures

Training-Serving Skew

One of the most insidious failure modes occurs when differences between training and serving environments lead to unexpected behavior in production.

Example 1.1 (Training-Serving Skew). A fraud detection model trained on batch data performs excellently in offline evaluation but fails catastrophically in production. Investigation reveals:

Training Environment:

- Features computed using daily batch jobs
- All data available for feature calculations
- Perfect data quality after cleaning
- Unlimited computation time

Production Environment:

- Real-time feature computation required
- Some historical data unavailable
- Raw data quality issues
- 100ms latency requirement

The model fails because features that worked in training cannot be computed in production within latency constraints.

Data Quality Degradation

Models trained on historical data often fail when deployed on real-time data streams with different quality characteristics.

Warning: *Data quality issues in production:*

- *Missing values handled differently than in training*
- *Unexpected categorical values not seen during training*
- *Distribution shifts in numerical features*
- *Temporal dependencies not captured in training*
- *Encoding inconsistencies between systems*

Technical Debt Accumulation

Sculley et al. [Sculley et al., 2015] identified multiple sources of technical debt specific to ML systems:

- **Glue Code:** Excessive code wrapping general-purpose packages
- **Pipeline Jungles:** Complex data preparation code that's hard to modify
- **Dead Experimental Codepaths:** Failed experiments that remain in codebase
- **Configuration Debt:** Complex, error-prone configuration systems
- **Undeclared Consumers:** Hidden dependencies on model outputs

1.5.2 Organizational Failures

Misaligned Success Metrics

Projects fail when data science teams optimize for model accuracy while business stakeholders care about operational metrics:

Table 1.2: Metric Misalignment Examples

DS Metric	Business Metric	Impact of Misalignment
AUC: 0.92	Revenue: -\$500K	Model increases false positives, hurting UX
Accuracy: 95%	Latency: 2000ms	Too slow for real-time use case
F1 Score: 0.88	Cost: \$100K/month	Computational cost exceeds value

Insufficient Production Readiness

Many projects fail in the "last mile" when teams realize their experimental code isn't production-ready:

- No deployment strategy or infrastructure
- Missing monitoring and alerting systems
- Inadequate error handling and recovery
- No rollback or versioning strategy
- Unclear ownership and on-call responsibilities

1.6 Architecture Principles for ML Systems

Building reliable ML systems requires applying proven software architecture principles while accounting for ML-specific challenges.

1.6.1 Separation of Concerns

ML systems should separate:

- **Data pipeline:** Data ingestion, validation, and transformation
- **Training pipeline:** Model training, evaluation, and versioning
- **Serving infrastructure:** Model deployment and prediction serving
- **Monitoring system:** Performance tracking and alerting

This separation enables:

- Independent scaling of components
- Easier testing and debugging
- Clear ownership boundaries
- Reusability across projects

1.6.2 Immutability and Versioning

Treat all ML artifacts as immutable:

- Version all datasets with content-addressable storage
- Version all models with unique identifiers
- Version all preprocessing pipelines alongside models
- Version all configurations and code

This enables reproducibility and safe rollbacks.

1.6.3 Observability First

Build observability into systems from the start:

- Log all predictions with features and outcomes
- Monitor data distribution metrics
- Track model performance continuously
- Alert on anomalies and degradation
- Create dashboards for stakeholders

1.7 The MLOps Maturity Model

Organizations progress through distinct stages of ML maturity. Understanding these stages helps set realistic expectations and create improvement roadmaps.

1.7.1 Level 0: Manual Process

Characteristics:

- All steps manual, no automation
- Jupyter notebooks for development
- Manual model deployment
- No monitoring or feedback loops
- Ad-hoc experimentation tracking

Suitable for: Early exploration, proof-of-concepts

1.7.2 Level 1: ML Pipeline Automation

Characteristics:

- Automated training pipeline
- Experiment tracking (MLflow, W&B)
- Automated data validation
- Still manual deployment
- Basic monitoring

Suitable for: Teams with 1-2 models in production

1.7.3 Level 2: CI/CD Pipeline Automation

Characteristics:

- Automated testing (code, data, model)
- Automated deployment pipeline
- Model registry for versioning
- Comprehensive monitoring
- Some manual intervention for retraining

Suitable for: Teams managing 3-10 models

1.7.4 Level 3: Full MLOps Automation

Characteristics:

- Continuous training (CT)
- Automated retraining triggers
- Feature store for reusability
- Advanced monitoring and drift detection
- Automated model governance
- Multi-model orchestration

Suitable for: Large teams with 10+ models, strict SLAs

1.8 Chapter Summary

This chapter established the foundations of machine learning engineering by distinguishing it from data science, exploring the production ML lifecycle, and identifying common failure modes. Key takeaways include:

- ML engineering focuses on system reliability, scalability, and operational excellence rather than pure model accuracy
- Production ML code requires modularity, error handling, logging, configuration management, and testing
- The ML lifecycle extends beyond training to include deployment, monitoring, and maintenance
- Most ML projects fail due to organizational issues and technical debt rather than algorithmic limitations
- Successful ML systems apply software engineering principles while accounting for ML-specific challenges
- Organizations progress through maturity levels as they build ML capabilities

The following chapters will explore each aspect of production ML systems in depth, providing concrete implementations and best practices.

1.9 Further Reading

- Sculley, D., et al. (2015). "Hidden Technical Debt in Machine Learning Systems." NIPS.
- Paleyes, A., et al. (2022). "Challenges in Deploying Machine Learning: A Survey of Case Studies."
- Huyen, C. (2022). *Designing Machine Learning Systems*. O'Reilly Media.
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.

1.10 Chapter Summary

This chapter established the foundations of ML engineering and distinguished it from data science:

Key Takeaways

1. ML Engineering vs. Data Science:

- Data scientists focus on model accuracy and insights
- ML engineers focus on system reliability, scalability, and operational excellence
- Production ML requires both skillsets working together

2. Why ML Projects Fail:

- 87% of ML projects never reach production [VentureBeat, 2019]
- Root causes: lack of engineering rigor, not algorithmic insufficiency
- Common failures: training-serving skew, data quality issues, technical debt

3. Production vs. Research Code:

- Research code: exploratory, iterative, accuracy-focused
- Production code: modular, testable, maintainable, reliable
- Transformation requires software engineering discipline

4. ML System Architecture Principles:

- Separation of concerns (data, training, serving, monitoring)
- Reproducibility through versioning and configuration
- Testability at every layer
- Observability for debugging and optimization

5. ML Maturity Levels:

- Level 0: Manual processes, no automation
- Level 1: Automated training pipeline
- Level 2: Automated deployment pipeline
- Level 3: Full MLOps with automated retraining

Critical Success Factors

- **Start with strong fundamentals:** Invest in proper data infrastructure before complex models
- **Measure business impact, not just model metrics:** Align ML work with business objectives
- **Build for production from day one:** Production-grade code patterns save re-work

- **Embrace software engineering practices:** Testing, version control, CI/CD are non-negotiable
- **Plan for failure:** Monitoring, alerting, and debugging must be first-class concerns

Connection to FraudGuard Case Study

In our FraudGuard fraud detection system (see Chapter), we'll apply these principles throughout the book:

- Chapters 4–6: Building robust data foundations
- Chapters 7–10: Production-grade model development
- Chapters 11–13: Reliable deployment and serving
- Chapters 14–16: Operational excellence
- Chapters 17–19: Automation and MLOps

Next Steps

Chapter 2 provides a detailed walkthrough of the complete ML lifecycle, from problem definition through production deployment and maintenance. This lifecycle framework guides all subsequent chapters.

1.11 Exercises

Exercise 1.1. Code Transformation: Take a Jupyter notebook you've written for a data science project and refactor it into a production-ready pipeline following the structure shown in this chapter. Include:

- Separate modules for data loading, preprocessing, and training
- External YAML configuration
- Comprehensive error handling and logging
- Type hints for all functions
- Unit tests for critical functions

Exercise 1.2. Architecture Design: Design a complete ML system architecture for one of the following scenarios:

- Real-time fraud detection for credit card transactions
- Recommendation system for e-commerce product recommendations
- Computer vision system for quality control in manufacturing

Include diagrams showing data flow, components, and integration points. Justify your design decisions.

Exercise 1.3. Failure Analysis: Research a publicized ML system failure (e.g., Microsoft Tay chatbot, Amazon recruiting tool bias, Apple Card credit limit controversy). Analyze:

- What went wrong from an ML engineering perspective?
- Which lifecycle phase had inadequate processes?
- What architectural principles were violated?
- How could better engineering practices have prevented the failure?

Exercise 1.4. Maturity Assessment: Assess your organization's ML maturity level using the framework presented in this chapter. Create a roadmap to advance to the next level, including:

- Current state assessment
- Gap analysis
- Prioritized improvements
- Required tools and skills
- Timeline and resource estimates

Exercise 1.5. Technical Debt Audit: Review an existing ML codebase and identify instances of technical debt categories from Sculley et al.:

- Glue code
- Pipeline jungles
- Dead experimental codepaths
- Configuration debt
- Undeclared consumers

Propose refactoring strategies to address the most critical issues.

Chapter 2

The Machine Learning Lifecycle

2.1 Learning Objectives

After completing this chapter, readers will be able to:

- Describe the complete ML lifecycle from problem definition to production
- Understand the iterative nature of ML development
- Identify key phases and activities in each stage
- Recognize decision points and feedback loops
- Apply lifecycle thinking to real-world ML projects
- Plan resource allocation across lifecycle phases

2.2 Overview of the ML Lifecycle

2.2.1 The End-to-End Journey

The machine learning lifecycle encompasses all activities from identifying a business problem to deploying and maintaining a solution in production. Unlike traditional software development, ML projects are inherently iterative and experimental.

2.3 Phase 1: Problem Definition

2.3.1 Framing the Business Problem

The first phase establishes whether ML is appropriate for the problem and defines success criteria.

Best Practice: *Always start with the business problem, not the ML technique. Ask: "What decision will this model enable?" and "How will we measure success?"*

Key activities include:

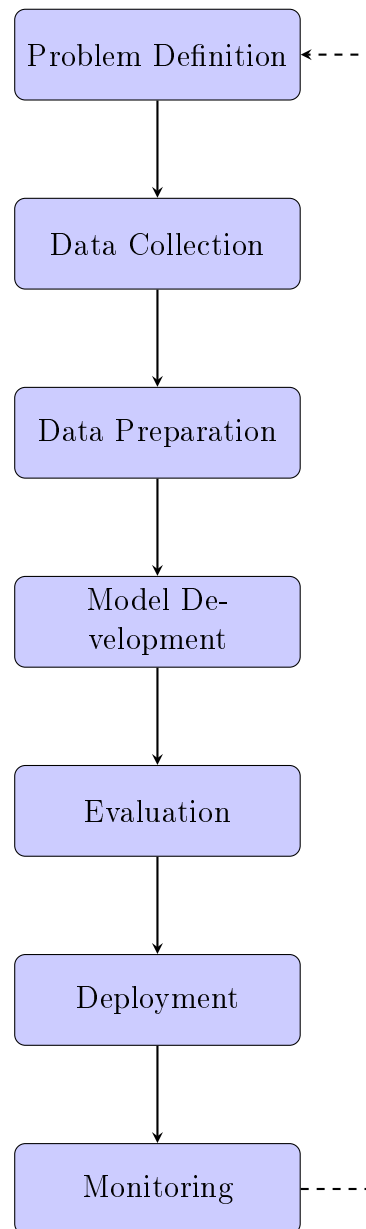


Figure 2.1: The ML lifecycle with feedback loops

- Defining the business objective and success metrics
- Determining whether ML is the right solution
- Assessing feasibility (data availability, timeline, resources)
- Establishing baseline performance
- Identifying stakeholders and requirements

2.3.2 ML Problem Formulation

Translate business objectives into ML problems:

- **Classification:** Categorizing inputs into discrete classes

- **Regression:** Predicting continuous values
- **Ranking:** Ordering items by relevance
- **Clustering:** Discovering natural groupings
- **Anomaly Detection:** Identifying unusual patterns

2.4 Phase 2: Data Collection and Understanding

2.4.1 Data Requirements

Identify and acquire necessary data:

- Define features and labels needed
- Assess data availability and quality
- Determine collection methods
- Consider privacy and legal constraints
- Plan for data versioning

2.4.2 Exploratory Data Analysis

Understand data characteristics:

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 import seaborn as sns
4
5 # Load and inspect data
6 df = pd.read_csv('data.csv')
7 print(df.info())
8 print(df.describe())
9
10 # Visualize distributions
11 df.hist(figsize=(12, 10))
12 plt.tight_layout()
13 plt.show()
14
15 # Check for correlations
16 correlation_matrix = df.corr()
17 sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
18 plt.show()
19
20 # Identify missing values
21 missing_data = df.isnull().sum()
22 print(missing_data[missing_data > 0])
```

2.5 Phase 3: Data Preparation

2.5.1 Data Cleaning

Address data quality issues:

- Handle missing values (imputation or removal)
- Remove or correct outliers
- Fix inconsistencies and errors
- Deduplicate records
- Validate data integrity

2.5.2 Feature Engineering

Transform raw data into features:

```
1 from sklearn.preprocessing import StandardScaler, OneHotEncoder
2 from sklearn.compose import ColumnTransformer
3 from sklearn.pipeline import Pipeline
4
5 # Define transformations
6 numeric_features = ['age', 'income', 'credit_score']
7 categorical_features = ['city', 'occupation']
8
9 # Create preprocessing pipeline
10 preprocessor = ColumnTransformer(
11     transformers=[
12         ('num', StandardScaler(), numeric_features),
13         ('cat', OneHotEncoder(drop='first'), categorical_features)
14     ])
15
16 # Apply transformations
17 X_processed = preprocessor.fit_transform(X)
```

2.6 Phase 4: Model Development

2.6.1 Model Selection

Choose appropriate algorithms based on:

- Problem type (classification, regression, etc.)
- Data characteristics (size, dimensionality, linearity)
- Performance requirements (accuracy, latency, interpretability)
- Resource constraints (computational budget, memory)

2.6.2 Training and Validation

Implement robust training procedures:

```
1 from sklearn.model_selection import train_test_split, cross_val_score
2 from sklearn.ensemble import RandomForestClassifier
3
4 # Split data
5 X_train, X_test, y_train, y_test = train_test_split(
6     X, y, test_size=0.2, random_state=42, stratify=y
7 )
8
9 # Train model
10 model = RandomForestClassifier(n_estimators=100, random_state=42)
11 model.fit(X_train, y_train)
12
13 # Cross-validation
14 cv_scores = cross_val_score(model, X_train, y_train, cv=5)
15 print(f"CV Scores: {cv_scores.mean():.3f} (+/- {cv_scores.std():.3f})")
```

2.7 Phase 5: Evaluation

2.7.1 Model Assessment

Evaluate using appropriate metrics:

- **Classification:** Accuracy, precision, recall, F1-score, ROC-AUC
- **Regression:** MAE, MSE, RMSE, R-squared
- **Ranking:** NDCG, MAP, MRR

```
1 from sklearn.metrics import classification_report, confusion_matrix
2 from sklearn.metrics import roc_auc_score, roc_curve
3
4 # Predictions
5 y_pred = model.predict(X_test)
6 y_pred_proba = model.predict_proba(X_test)[: , 1]
7
8 # Classification metrics
9 print(classification_report(y_test, y_pred))
10 print(confusion_matrix(y_test, y_pred))
11
12 # ROC-AUC
13 roc_auc = roc_auc_score(y_test, y_pred_proba)
14 print(f"ROC-AUC: {roc_auc:.3f}")
```

2.8 Phase 6: Deployment

2.8.1 Deployment Strategies

Choose deployment approach:

- **Batch Prediction:** Process data in batches periodically
- **Online Prediction:** Real-time predictions via API
- **Edge Deployment:** Models running on devices
- **Hybrid:** Combination of approaches

2.9 Phase 7: Monitoring and Maintenance

2.9.1 Production Monitoring

Track key metrics:

- Model performance metrics
- Prediction distribution
- Data drift detection
- System health (latency, throughput)
- Business impact metrics

2.9.2 Model Maintenance

Plan for ongoing maintenance:

- Regular retraining schedule
- A/B testing new models
- Handling concept drift
- Updating features
- Documentation updates

2.10 Iterative Nature of ML

2.10.1 Feedback Loops

ML development is rarely linear. Common iteration points:

1. Evaluation results prompt data collection or feature engineering
2. Production monitoring reveals need for model updates
3. Stakeholder feedback refines problem definition
4. New data sources become available

2.11 Chapter Summary

The ML lifecycle provides a framework for systematic ML development. Understanding each phase and their interconnections enables better planning, resource allocation, and risk management. Subsequent chapters will explore each phase in detail.

2.12 Exercises

Exercise 2.1. Map a familiar ML project to the lifecycle phases. Identify where iterations occurred and why.

Exercise 2.2. For a given business problem, work through the problem definition phase. Define success metrics and determine if ML is appropriate.

Chapter 3

Training Pipeline Design Patterns

3.1 Learning Objectives

After completing this chapter, readers will be able to:

- Design end-to-end ML training pipelines with proper orchestration
- Implement experiment tracking for reproducibility and comparison
- Apply hyperparameter optimization strategies at scale
- Build comprehensive model validation and evaluation frameworks
- Design and implement A/B testing frameworks for ML models
- Handle pipeline failures with proper retry and error recovery mechanisms
- Use pipeline-as-code approaches for version control and collaboration
- Choose appropriate orchestration tools (Airflow, Kubeflow, MLflow)
- Implement automated model selection and promotion workflows

3.2 Introduction: The Training Pipeline

Training pipelines orchestrate the complex workflow from raw data to production-ready models. Unlike traditional software pipelines, ML training pipelines must handle:

- **Non-determinism:** Randomness in algorithms, data sampling, and initialization
- **Resource intensity:** GPU/TPU provisioning, distributed training coordination
- **Long-running tasks:** Training jobs spanning hours to days
- **Experimentation:** Hundreds of experiments to find optimal configurations
- **Reproducibility:** Exact recreation of training conditions for debugging and auditing

A well-designed training pipeline provides:

1. Automated workflow execution from data to model
2. Experiment tracking and version control for all artifacts
3. Resource optimization and failure recovery
4. Model validation and quality assurance
5. Deployment automation with safety checks

3.3 Pipeline Orchestration

Pipeline orchestration tools manage the execution of multi-step ML workflows, handling dependencies, resource allocation, scheduling, and failure recovery.

3.3.1 Orchestration Tool Comparison

Tool	Best For	Strengths	Limitations
Apache Airflow	Batch workflows, scheduling	Mature ecosystem, flexible	Not ML-native, complexity
Kubeflow Pipelines	K8s-native ML	Native ML support, portability	K8s required, steep learning curve
MLflow	Experiment tracking	Simple API, ML-focused	Limited orchestration
Prefect	Modern workflows	Pythonic, dynamic DAGs	Newer, smaller community
Metaflow	Netflix-style workflows	Netflix-proven, simple	AWS/cloud-centric

Table 3.1: ML pipeline orchestration tool comparison

3.3.2 Apache Airflow for ML Pipelines

Airflow provides robust scheduling, monitoring, and dependency management for ML workflows.

Production-Grade Airflow ML Pipeline:

```

1 from airflow import DAG
2 from airflow.operators.python import PythonOperator, BranchPythonOperator
3 from airflow.operators.email import EmailOperator
4 from airflow.utils.trigger_rule import TriggerRule
5 from airflow.sensors.external_task import ExternalTaskSensor
6 from datetime import datetime, timedelta
7 import pandas as pd
8 import mlflow
9 from sklearn.ensemble import RandomForestClassifier
10 from sklearn.metrics import accuracy_score, precision_score, recall_score
11 import logging
12

```

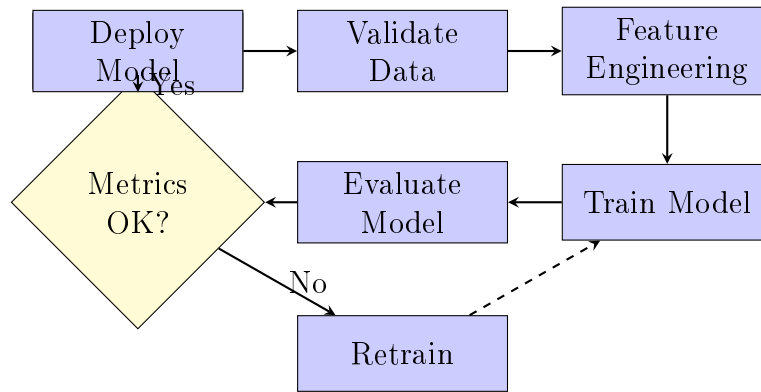



Figure 3.1: ML training pipeline workflow in Airflow

```

13 logger = logging.getLogger(__name__)
14
15 # DAG configuration
16 default_args = {
17     'owner': 'ml_team',
18     'depends_on_past': False,
19     'start_date': datetime(2024, 1, 1),
20     'email': ['ml-alerts@company.com'],
21     'email_on_failure': True,
22     'email_on_retry': False,
23     'retries': 3,
24     'retry_delay': timedelta(minutes=5),
25     'retry_exponential_backoff': True,
26     'max_retry_delay': timedelta(minutes=30),
27 }
28
29 def extract_training_data(**context):
30     """Extract data from warehouse for training"""
31     from google.cloud import bigquery
32
33     client = bigquery.Client()
34
35     # Query with parameterization
36     query = """
37         SELECT *
38         FROM 'project.dataset.features'
39         WHERE date BETWEEN @start_date AND @end_date
40         AND label IS NOT NULL
41     """
42
43     # Use execution date for temporal consistency
44     execution_date = context['execution_date']
45     start_date = execution_date - timedelta(days=30)
46
47     job_config = bigquery.QueryJobConfig(
48         query_parameters=[
49             bigquery.ScalarQueryParameter("start_date", "DATE", start_date),
50             bigquery.ScalarQueryParameter("end_date", "DATE", execution_date),
51         ]
52     )
53
54     df = client.query(query, job_config=job_config).to_dataframe()

```

```

55
56     # Save to intermediate storage
57     output_path = f"gs://ml-data/training_data_{execution_date.strftime('%Y%m%d')}.
parquet"
58     df.to_parquet(output_path)
59
60     # Push metadata to XCom
61     context['task_instance'].xcom_push(key='data_path', value=output_path)
62     context['task_instance'].xcom_push(key='num_samples', value=len(df))
63
64     logger.info(f"Extracted {len(df)} samples to {output_path}")
65
66     return output_path
67
68 def validate_data_quality(**context):
69     """Validate data quality before training"""
70     import great_expectations as ge
71
72     # Pull data path from previous task
73     data_path = context['task_instance'].xcom_pull(
74         task_ids='extract_data',
75         key='data_path'
76     )
77
78     df = pd.read_parquet(data_path)
79
80     # Create Great Expectations DataFrame
81     ge_df = ge.from_pandas(df)
82
83     # Define expectations
84     expectations = [
85         ge_df.expect_table_row_count_to_be_between(min_value=1000, max_value=10000000)
86         ,
87         ge_df.expect_column_values_to_not_be_null('label'),
88         ge_df.expect_column_values_to_be_in_set('label', [0, 1]),
89         ge_df.expect_column_values_to_be_between('feature_1', min_value=0, max_value
=1),
90     ]
91
92     # Validate
93     results = ge_df.validate()
94
95     if not results['success']:
96         failed = [e for e in results['results'] if not e['success']]
97         raise ValueError(f"Data validation failed: {len(failed)} expectations failed")
98
99     logger.info("Data validation passed")
100     return True
101
102 def prepare_train_test_split(**context):
103     """Create stratified train/test split"""
104     from sklearn.model_selection import train_test_split
105
106     data_path = context['task_instance'].xcom_pull(
107         task_ids='extract_data',
108         key='data_path'
109     )

```

```

110 df = pd.read_parquet(data_path)
111
112 # Separate features and labels
113 feature_cols = [col for col in df.columns if col.startswith('feature_')]
114 X = df[feature_cols]
115 y = df['label']
116
117 # Stratified split
118 X_train, X_test, y_train, y_test = train_test_split(
119     X, y, test_size=0.2, stratify=y, random_state=42
120 )
121
122 # Save splits
123 execution_date = context['execution_date'].strftime('%Y%m%d')
124 train_path = f"gs://ml-data/train_{execution_date}.parquet"
125 test_path = f"gs://ml-data/test_{execution_date}.parquet"
126
127 pd.concat([X_train, y_train], axis=1).to_parquet(train_path)
128 pd.concat([X_test, y_test], axis=1).to_parquet(test_path)
129
130 # Push paths to XCom
131 context['task_instance'].xcom_push(key='train_path', value=train_path)
132 context['task_instance'].xcom_push(key='test_path', value=test_path)
133
134 logger.info(f"Split data: train={len(X_train)}, test={len(X_test)}")
135
136 def train_model(**context):
137     """Train ML model with MLflow tracking"""
138
139     # Get data paths
140     train_path = context['task_instance'].xcom_pull(
141         task_ids='prepare_data',
142         key='train_path'
143     )
144
145     train_df = pd.read_parquet(train_path)
146     feature_cols = [col for col in train_df.columns if col.startswith('feature_')]
147     X_train = train_df[feature_cols]
148     y_train = train_df['label']
149
150     # MLflow experiment tracking
151     mlflow.set_tracking_uri("http://mlflow-server:5000")
152     mlflow.set_experiment("production_model")
153
154     with mlflow.start_run(run_name=f"airflow_{context['execution_date'].strftime('%Y%m%d')}"):
155         # Log parameters
156         params = {
157             'n_estimators': 100,
158             'max_depth': 10,
159             'min_samples_split': 5,
160             'random_state': 42
161         }
162         mlflow.log_params(params)
163
164         # Log metadata
165         mlflow.set_tag("pipeline", "airflow")
166         mlflow.set_tag("execution_date", context['execution_date'].isoformat())

```

```

167     # Train model
168     model = RandomForestClassifier(**params)
169     model.fit(X_train, y_train)
170
171     # Log model
172     mlflow.sklearn.log_model(model, "model")
173
174     # Save model path
175     model_path = f"gs://ml-models/model_{context['execution_date'].strftime('%Y%m%d')}.pkl"
176     import joblib
177     with open(model_path, 'wb') as f:
178         joblib.dump(model, f)
179
180     # Push to XCom
181     context['task_instance'].xcom_push(key='model_path', value=model_path)
182     context['task_instance'].xcom_push(key='mlflow_run_id', value=mlflow.
183     active_run().info.run_id)
184
185     logger.info(f"Model trained and saved to {model_path}")
186
187 def evaluate_model(**context):
188     """Evaluate model on test set"""
189     import joblib
190
191     # Get paths
192     model_path = context['task_instance'].xcom_pull(
193         task_ids='train_model',
194         key='model_path'
195     )
196     test_path = context['task_instance'].xcom_pull(
197         task_ids='prepare_data',
198         key='test_path'
199     )
200
201     # Load model and test data
202     model = joblib.load(model_path)
203     test_df = pd.read_parquet(test_path)
204     feature_cols = [col for col in test_df.columns if col.startswith('feature_')]
205     X_test = test_df[feature_cols]
206     y_test = test_df['label']
207
208     # Predict
209     y_pred = model.predict(X_test)
210
211     # Compute metrics
212     metrics = {
213         'accuracy': accuracy_score(y_test, y_pred),
214         'precision': precision_score(y_test, y_pred),
215         'recall': recall_score(y_test, y_pred)
216     }
217
218     # Log to MLflow
219     mlflow_run_id = context['task_instance'].xcom_pull(
220         task_ids='train_model',
221         key='mlflow_run_id'
222     )

```

```

223
224     with mlflow.start_run(run_id=mlflow_run_id):
225         mlflow.log_metrics(metrics)
226
227         # Push metrics to XCom
228         for metric_name, metric_value in metrics.items():
229             context['task_instance'].xcom_push(key=metric_name, value=metric_value)
230
231         logger.info(f"Model evaluation: {metrics}")
232
233     return metrics
234
235 def check_model_quality(**context):
236     """Decide whether to deploy based on metrics"""
237     accuracy = context['task_instance'].xcom_pull(
238         task_ids='evaluate_model',
239         key='accuracy'
240     )
241     precision = context['task_instance'].xcom_pull(
242         task_ids='evaluate_model',
243         key='precision'
244     )
245     recall = context['task_instance'].xcom_pull(
246         task_ids='evaluate_model',
247         key='recall'
248     )
249
250     # Define thresholds
251     ACCURACY_THRESHOLD = 0.85
252     PRECISION_THRESHOLD = 0.80
253     RECALL_THRESHOLD = 0.75
254
255     if (accuracy >= ACCURACY_THRESHOLD and
256         precision >= PRECISION_THRESHOLD and
257         recall >= RECALL_THRESHOLD):
258         logger.info("Model passed quality checks - proceeding to deployment")
259         return 'deploy_model'
260     else:
261         logger.warning("Model failed quality checks - triggering retraining")
262         return 'send_failure_alert'
263
264 def deploy_model(**context):
265     """Deploy model to production"""
266     model_path = context['task_instance'].xcom_pull(
267         task_ids='train_model',
268         key='model_path'
269     )
270
271     # Copy to production location
272     import shutil
273     prod_path = "gs://ml-models/production/model.pkl"
274     shutil.copy(model_path, prod_path)
275
276     # Update model registry
277     mlflow_run_id = context['task_instance'].xcom_pull(
278         task_ids='train_model',
279         key='mlflow_run_id'
280     )

```

```

281 client = mlflow.tracking.MlflowClient()
282 client.create_model_version(
283     name="production_model",
284     source=f"runs://{mlflow_run_id}/model",
285     run_id=mlflow_run_id
286 )
287
288 logger.info(f"Model deployed to {prod_path}")
289
290 def cleanup_old_artifacts(**context):
291     """Clean up old training artifacts"""
292     from google.cloud import storage
293     from datetime import datetime, timedelta
294
295     client = storage.Client()
296     bucket = client.bucket('ml-data')
297
298     # Delete files older than 30 days
299     cutoff_date = datetime.now() - timedelta(days=30)
300
301     blobs = bucket.list_blobs(prefix='training_data_')
302     deleted_count = 0
303
304     for blob in blobs:
305         if blob.time_created < cutoff_date:
306             blob.delete()
307             deleted_count += 1
308
309     logger.info(f"Cleaned up {deleted_count} old artifacts")
310
311 # Define DAG
312 with DAG(
313     'ml_training_pipeline',
314     default_args=default_args,
315     description='Production ML training pipeline',
316     schedule_interval='0 2 * * 0', # Weekly on Sunday at 2 AM
317     catchup=False,
318     max_active_runs=1,
319     tags=['ml', 'training', 'production']
320 ) as dag:
321
322     # Extract data
323     extract_task = PythonOperator(
324         task_id='extract_data',
325         python_callable=extract_training_data,
326         provide_context=True,
327         execution_timeout=timedelta(hours=1)
328     )
329
330     # Validate data
331     validate_task = PythonOperator(
332         task_id='validate_data',
333         python_callable=validate_data_quality,
334         provide_context=True,
335         execution_timeout=timedelta(minutes=30)
336     )
337
338

```

```

339 # Prepare train/test split
340 prepare_task = PythonOperator(
341     task_id='prepare_data',
342     python_callable=prepare_train_test_split,
343     provide_context=True
344 )
345
346 # Train model
347 train_task = PythonOperator(
348     task_id='train_model',
349     python_callable=train_model,
350     provide_context=True,
351     execution_timeout=timedelta(hours=4)
352 )
353
354 # Evaluate model
355 evaluate_task = PythonOperator(
356     task_id='evaluate_model',
357     python_callable=evaluate_model,
358     provide_context=True
359 )
360
361 # Check quality and branch
362 quality_check = BranchPythonOperator(
363     task_id='check_quality',
364     python_callable=check_model_quality,
365     provide_context=True
366 )
367
368 # Deploy if quality check passes
369 deploy_task = PythonOperator(
370     task_id='deploy_model',
371     python_callable=deploy_model,
372     provide_context=True
373 )
374
375 # Send alert if quality check fails
376 alert_task = EmailOperator(
377     task_id='send_failure_alert',
378     to='ml-team@company.com',
379     subject='Model Training Failed Quality Checks',
380     html_content='<p>Model training completed but failed quality thresholds.</p>',
381 )
382
383 # Cleanup old artifacts (runs regardless of upstream success/failure)
384 cleanup_task = PythonOperator(
385     task_id='cleanup',
386     python_callable=cleanup_old_artifacts,
387     provide_context=True,
388     trigger_rule=TriggerRule.ALL_DONE
389 )
390
391 # Define dependencies
392 extract_task >> validate_task >> prepare_task >> train_task >> evaluate_task
393 evaluate_task >> quality_check >> [deploy_task, alert_task]
394 [deploy_task, alert_task] >> cleanup_task

```

Listing 3.1: Comprehensive Airflow ML pipeline with error handling

3.3.3 Kubeflow Pipelines for ML

Kubeflow Pipelines provide Kubernetes-native ML workflows with containerized components and strong reproducibility guarantees.

```

1 import kfp
2 from kfp import dsl
3 from kfp.v2.dsl import component, pipeline, Input, Output, Dataset, Model, Metrics
4
5 @component(
6     base_image='python:3.9',
7     packages_to_install=['pandas', 'scikit-learn', 'google-cloud-bigquery']
8 )
9 def extract_data(
10     project_id: str,
11     dataset_id: str,
12     start_date: str,
13     end_date: str,
14     output_data: Output[Dataset]
15 ):
16     """Extract training data from BigQuery"""
17     from google.cloud import bigquery
18     import pandas as pd
19
20     client = bigquery.Client(project=project_id)
21
22     query = f"""
23         SELECT *
24         FROM '{project_id}.{dataset_id}.features'
25         WHERE date BETWEEN '{start_date}' AND '{end_date}'
26         AND label IS NOT NULL
27     """
28
29     df = client.query(query).to_dataframe()
30     df.to_csv(output_data.path, index=False)
31
32     print(f"Extracted {len(df)} samples")
33
34 @component(
35     base_image='python:3.9',
36     packages_to_install=['pandas', 'scikit-learn']
37 )
38 def train_model(
39     input_data: Input[Dataset],
40     n_estimators: int,
41     max_depth: int,
42     model: Output[Model],
43     metrics: Output[Metrics]
44 ):
45     """Train Random Forest model"""
46     import pandas as pd
47     from sklearn.ensemble import RandomForestClassifier
48     from sklearn.model_selection import train_test_split
49     from sklearn.metrics import accuracy_score, precision_score, recall_score
50     import pickle
51
52     # Load data
53     df = pd.read_csv(input_data.path)

```



```

54 feature_cols = [col for col in df.columns if col.startswith('feature_')]
55 X = df[feature_cols]
56 y = df['label']
57
58
59 # Split
60 X_train, X_test, y_train, y_test = train_test_split(
61     X, y, test_size=0.2, stratify=y, random_state=42
62 )
63
64 # Train
65 clf = RandomForestClassifier(
66     n_estimators=n_estimators,
67     max_depth=max_depth,
68     random_state=42
69 )
70 clf.fit(X_train, y_train)
71
72 # Evaluate
73 y_pred = clf.predict(X_test)
74 acc = accuracy_score(y_test, y_pred)
75 prec = precision_score(y_test, y_pred)
76 rec = recall_score(y_test, y_pred)
77
78 # Save model
79 with open(model.path, 'wb') as f:
80     pickle.dump(clf, f)
81
82 # Log metrics
83 metrics.log_metric('accuracy', acc)
84 metrics.log_metric('precision', prec)
85 metrics.log_metric('recall', rec)
86
87 print(f"Model trained - Accuracy: {acc:.4f}")
88
89 @component(
90     base_image='python:3.9',
91     packages_to_install=['pandas', 'scikit-learn']
92 )
93 def validate_model(
94     model: Input[Model],
95     input_data: Input[Dataset],
96     accuracy_threshold: float,
97     passed: Output[str]
98 ):
99     """Validate model meets quality thresholds"""
100     import pandas as pd
101     import pickle
102     from sklearn.metrics import accuracy_score
103
104     # Load model and data
105     with open(model.path, 'rb') as f:
106         clf = pickle.load(f)
107
108     df = pd.read_csv(input_data.path)
109     feature_cols = [col for col in df.columns if col.startswith('feature_')]
110     X = df[feature_cols]
111     y = df['label']

```

```

112     # Evaluate
113     y_pred = clf.predict(X)
114     acc = accuracy_score(y, y_pred)
115
116     # Check threshold
117     validation_passed = acc >= accuracy_threshold
118
119     with open(passed.path, 'w') as f:
120         f.write(str(validation_passed))
121
122     if validation_passed:
123         print(f"Validation PASSED - Accuracy: {acc:.4f}")
124     else:
125         print(f"Validation FAILED - Accuracy: {acc:.4f} < {accuracy_threshold}")
126         raise ValueError(f"Model accuracy {acc:.4f} below threshold {
127             accuracy_threshold}")
128
129 @component(
130     base_image='python:3.9',
131     packages_to_install=['google-cloud-storage']
132 )
133 def deploy_model(
134     model: Input[Model],
135     bucket_name: str,
136     model_name: str
137 ):
138     """Deploy model to GCS"""
139     from google.cloud import storage
140
141     client = storage.Client()
142     bucket = client.bucket(bucket_name)
143
144     # Upload model
145     blob = bucket.blob(f'models/{model_name}/model.pkl')
146     blob.upload_from_filename(model.path)
147
148     print(f"Model deployed to gs://{bucket_name}/models/{model_name}/model.pkl")
149
150 @pipeline(
151     name='ML Training Pipeline',
152     description='End-to-end ML training with validation',
153     pipeline_root='gs://my-bucket/pipeline-root'
154 )
155 def ml_training_pipeline(
156     project_id: str = 'my-project',
157     dataset_id: str = 'ml_features',
158     start_date: str = '2024-01-01',
159     end_date: str = '2024-01-31',
160     n_estimators: int = 100,
161     max_depth: int = 10,
162     accuracy_threshold: float = 0.85,
163     bucket_name: str = 'ml-models',
164     model_name: str = 'production_model'
165 ):
166     """Complete ML training pipeline"""
167
168     # Extract data

```

```

169     extract_task = extract_data(
170         project_id=project_id,
171         dataset_id=dataset_id,
172         start_date=start_date,
173         end_date=end_date
174     )
175
176     # Train model
177     train_task = train_model(
178         input_data=extract_task.outputs['output_data'],
179         n_estimators=n_estimators,
180         max_depth=max_depth
181     )
182
183     # Validate model
184     validate_task = validate_model(
185         model=train_task.outputs['model'],
186         input_data=extract_task.outputs['output_data'],
187         accuracy_threshold=accuracy_threshold
188     )
189
190     # Deploy if validation passes
191     deploy_task = deploy_model(
192         model=train_task.outputs['model'],
193         bucket_name=bucket_name,
194         model_name=model_name
195     )
196
197     # Set dependencies
198     validate_task.after(train_task)
199     deploy_task.after(validate_task)
200
201     # Compile and run pipeline
202     if __name__ == '__main__':
203         from kfp.v2 import compiler
204
205         compiler.Compiler().compile(
206             pipeline_func=ml_training_pipeline,
207             package_path='ml_training_pipeline.json'
208         )
209
210     # Submit to Kubeflow
211     client = kfp.Client(host='http://kubeflow-pipelines:8080')
212     client.create_run_from_pipeline_func(
213         ml_training_pipeline,
214         arguments={
215             'project_id': 'my-project',
216             'dataset_id': 'ml_features',
217             'accuracy_threshold': 0.85
218         }
219     )

```

Listing 3.2: Kubeflow Pipeline with containerized components

3.4 Experiment Tracking and Reproducibility

Experiment tracking captures all information needed to reproduce ML experiments: code versions, data versions, hyperparameters, metrics, and artifacts.

3.4.1 MLflow for Experiment Management

MLflow provides a unified platform for experiment tracking, model registry, and deployment.

```
1 import mlflow
2 import mlflow.sklearn
3 from mlflow.tracking import MlflowClient
4 from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
5 from sklearn.model_selection import cross_val_score
6 from sklearn.metrics import classification_report
7 import numpy as np
8 import pandas as pd
9 import hashlib
10 import json
11
12 class ExperimentTracker:
13     """Manage ML experiments with MLflow"""
14
15     def __init__(self, tracking_uri: str, experiment_name: str):
16         mlflow.set_tracking_uri(tracking_uri)
17         mlflow.set_experiment(experiment_name)
18         self.client = MlflowClient()
19         self.experiment = mlflow.get_experiment_by_name(experiment_name)
20
21     def run_experiment(self,
22                       X_train, y_train,
23                       X_test, y_test,
24                       model_class,
25                       params: dict,
26                       tags: dict = None):
27         """Run a single experiment with full tracking"""
28
29         with mlflow.start_run(run_name=self._generate_run_name(model_class, params)):
30             # Log system information
31             self._log_system_info()
32
33             # Log parameters
34             mlflow.log_params(params)
35
36             # Log tags
37             if tags:
38                 mlflow.set_tags(tags)
39
40             # Log data information
41             self._log_data_info(X_train, y_train, X_test, y_test)
42
43             # Train model
44             model = model_class(**params)
45             model.fit(X_train, y_train)
46
47             # Evaluate on training set
```

```

48     train_score = model.score(X_train, y_train)
49     mlflow.log_metric("train_accuracy", train_score)
50
51     # Cross-validation
52     cv_scores = cross_val_score(model, X_train, y_train, cv=5)
53     mlflow.log_metric("cv_mean", cv_scores.mean())
54     mlflow.log_metric("cv_std", cv_scores.std())
55
56     # Evaluate on test set
57     test_score = model.score(X_test, y_test)
58     y_pred = model.predict(X_test)
59
60     mlflow.log_metric("test_accuracy", test_score)
61
62     # Detailed metrics
63     report = classification_report(y_test, y_pred, output_dict=True)
64     for label, metrics in report.items():
65         if isinstance(metrics, dict):
66             for metric_name, value in metrics.items():
67                 mlflow.log_metric(f"{label}_{metric_name}", value)
68
69     # Feature importance (if available)
70     if hasattr(model, 'feature_importances_'):
71         self._log_feature_importance(model, X_train.columns)
72
73     # Log model
74     mlflow.sklearn.log_model(
75         model,
76         "model",
77         signature=self._get_model_signature(X_train, y_train)
78     )
79
80     # Log confusion matrix
81     self._log_confusion_matrix(y_test, y_pred)
82
83     # Compute and log custom metrics
84     overfitting_ratio = train_score / test_score if test_score > 0 else float(
85 'inf')
86     mlflow.log_metric("overfitting_ratio", overfitting_ratio)
87
88     run_id = mlflow.active_run().info.run_id
89
90     return {
91         'run_id': run_id,
92         'test_accuracy': test_score,
93         'cv_mean': cv_scores.mean(),
94         'overfitting_ratio': overfitting_ratio
95     }
96
97 def _generate_run_name(self, model_class, params):
98     """Generate descriptive run name"""
99     model_name = model_class.__name__
100     key_params = '_'.join(f"{k}={v}" for k, v in list(params.items())[:3])
101     return f"{model_name}_{key_params}"
102
103 def _log_system_info(self):
104     """Log system and environment information"""
105     import platform

```

```

105     import sys
106
107     mlflow.set_tag("python_version", sys.version)
108     mlflow.set_tag("platform", platform.platform())
109     mlflow.set_tag("processor", platform.processor())
110
111     def _log_data_info(self, X_train, y_train, X_test, y_test):
112         """Log data statistics and hash"""
113         # Dataset sizes
114         mlflow.log_param("train_samples", len(X_train))
115         mlflow.log_param("test_samples", len(X_test))
116         mlflow.log_param("num_features", X_train.shape[1])
117
118         # Class distribution
119         train_class_dist = pd.Series(y_train).value_counts().to_dict()
120         mlflow.log_dict(train_class_dist, "train_class_distribution.json")
121
122         # Data hash for reproducibility
123         data_hash = self._compute_data_hash(X_train, y_train)
124         mlflow.set_tag("data_hash", data_hash)
125
126     def _compute_data_hash(self, X, y):
127         """Compute hash of dataset for version tracking"""
128         data_str = f"{X.values.tobytes()}{y.values.tobytes()}"
129         return hashlib.sha256(data_str.encode()).hexdigest()[:16]
130
131     def _get_model_signature(self, X, y):
132         """Create MLflow model signature"""
133         from mlflow.models.signature import infer_signature
134         return infer_signature(X, y)
135
136     def _log_feature_importance(self, model, feature_names):
137         """Log feature importance as artifact"""
138         importance_df = pd.DataFrame({
139             'feature': feature_names,
140             'importance': model.feature_importances_
141         }).sort_values('importance', ascending=False)
142
143         mlflow.log_dict(
144             importance_df.to_dict('records'),
145             "feature_importance.json"
146         )
147
148         # Create plot
149         import matplotlib.pyplot as plt
150         plt.figure(figsize=(10, 6))
151         plt.barh(importance_df['feature'][:20], importance_df['importance'][:20])
152         plt.xlabel('Importance')
153         plt.title('Top 20 Feature Importances')
154         plt.tight_layout()
155         plt.savefig('feature_importance.png')
156         mlflow.log_artifact('feature_importance.png')
157         plt.close()
158
159     def _log_confusion_matrix(self, y_true, y_pred):
160         """Log confusion matrix visualization"""
161         from sklearn.metrics import confusion_matrix
162         import matplotlib.pyplot as plt

```

```

163     import seaborn as sns
164
165     cm = confusion_matrix(y_true, y_pred)
166
167     plt.figure(figsize=(8, 6))
168     sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
169     plt.ylabel('True Label')
170     plt.xlabel('Predicted Label')
171     plt.title('Confusion Matrix')
172     plt.savefig('confusion_matrix.png')
173     mlflow.log_artifact('confusion_matrix.png')
174     plt.close()
175
176     def compare_experiments(self, run_ids: list):
177         """Compare multiple experiment runs"""
178         runs_data = []
179
180         for run_id in run_ids:
181             run = self.client.get_run(run_id)
182             runs_data.append({
183                 'run_id': run_id,
184                 'params': run.data.params,
185                 'metrics': run.data.metrics,
186                 'tags': run.data.tags
187             })
188
189         # Create comparison DataFrame
190         comparison_df = pd.DataFrame([
191             {
192                 'run_id': r['run_id'][:8],
193                 **r['params'],
194                 **r['metrics']
195             }
196             for r in runs_data
197         ])
198
199         return comparison_df
200
201     def promote_best_model(self, metric_name: str = 'test_accuracy',
202                           minimize: bool = False):
203         """Promote best model to production"""
204         # Get all runs in experiment
205         runs = self.client.search_runs(
206             experiment_ids=[self.experiment.experiment_id],
207             order_by=[f"metrics.{metric_name} {'ASC' if minimize else 'DESC'}"]
208         )
209
210         if not runs:
211             raise ValueError("No runs found in experiment")
212
213         best_run = runs[0]
214         best_run_id = best_run.info.run_id
215
216         # Register model
217         model_uri = f"runs://{best_run_id}/model"
218         model_name = "production_model"
219
220         # Create or get registered model

```

```

221     try:
222         self.client.create_registered_model(model_name)
223     except:
224         pass # Model already exists
225
226     # Create new version
227     model_version = self.client.create_model_version(
228         name=model_name,
229         source=model_uri,
230         run_id=best_run_id
231     )
232
233     # Transition to production
234     self.client.transition_model_version_stage(
235         name=model_name,
236         version=model_version.version,
237         stage="Production"
238     )
239
240     print(f"Promoted model from run {best_run_id[:8]} to production")
241     print(f"    {metric_name}: {best_run.data.metrics[metric_name]:.4f}")
242
243     return model_version
244
245 # Usage example
246 if __name__ == '__main__':
247     from sklearn.datasets import make_classification
248     from sklearn.model_selection import train_test_split
249
250     # Generate sample data
251     X, y = make_classification(
252         n_samples=10000,
253         n_features=20,
254         n_informative=15,
255         n_redundant=5,
256         random_state=42
257     )
258
259     X = pd.DataFrame(X, columns=[f'feature_{i}' for i in range(20)])
260     X_train, X_test, y_train, y_test = train_test_split(
261         X, y, test_size=0.2, random_state=42
262     )
263
264     # Initialize tracker
265     tracker = ExperimentTracker(
266         tracking_uri="http://mlflow-server:5000",
267         experiment_name="model_comparison"
268     )
269
270     # Run multiple experiments
271     experiments = [
272         {
273             'model_class': RandomForestClassifier,
274             'params': {'n_estimators': 100, 'max_depth': 10, 'random_state': 42},
275             'tags': {'model_type': 'random_forest', 'version': 'v1'}
276         },
277         {
278             'model_class': RandomForestClassifier,

```



```

279     'params': {'n_estimators': 200, 'max_depth': 15, 'random_state': 42},
280     'tags': {'model_type': 'random_forest', 'version': 'v2'}
281 },
282 {
283     'model_class': GradientBoostingClassifier,
284     'params': {'n_estimators': 100, 'max_depth': 5, 'random_state': 42},
285     'tags': {'model_type': 'gradient_boosting', 'version': 'v1'}
286 },
287 ]
288
289 run_ids = []
290 for exp in experiments:
291     result = tracker.run_experiment(
292         X_train, y_train, X_test, y_test,
293         model_class=exp['model_class'],
294         params=exp['params'],
295         tags=exp['tags']
296     )
297     run_ids.append(result['run_id'])
298     print(f"Run {result['run_id'][:8]}: accuracy={result['test_accuracy']:.4f}")
299
300 # Compare experiments
301 comparison = tracker.compare_experiments(run_ids)
302 print("\nExperiment Comparison:")
303 print(comparison)
304
305 # Promote best model
306 tracker.promote_best_model(metric_name='test_accuracy')

```

Listing 3.3: Comprehensive MLflow experiment tracking

3.4.2 Reproducibility Best Practices

Ensuring experiment reproducibility requires tracking six key dimensions:

1. **Code Version:** Git commit hash, branch name
2. **Data Version:** Dataset hash, DVC pointer, or data lineage
3. **Environment:** Package versions, system libraries, hardware specs
4. **Random Seeds:** All sources of randomness (model init, data shuffling, sampling)
5. **Hyperparameters:** Complete parameter configuration
6. **Training Process:** Order of operations, checkpoint timing

```

1 import os
2 import random
3 import numpy as np
4 import torch
5 import json
6 import hashlib
7 from datetime import datetime
8 from contextlib import contextmanager
9

```

```

10 @contextmanager
11 def reproducible_experiment(seed: int = 42,
12                             experiment_name: str = None,
13                             track_git: bool = True,
14                             track_data: bool = True):
15     """Context manager ensuring experiment reproducibility"""
16
17     # Capture initial state
18     initial_state = {
19         'seed': seed,
20         'timestamp': datetime.now().isoformat(),
21         'experiment_name': experiment_name
22     }
23
24     # Set all random seeds
25     random.seed(seed)
26     np.random.seed(seed)
27     torch.manual_seed(seed)
28     torch.cuda.manual_seed_all(seed)
29
30     # Make PyTorch deterministic
31     torch.backends.cudnn.deterministic = True
32     torch.backends.cudnn.benchmark = False
33
34     # Track Git information
35     if track_git:
36         try:
37             import git
38             repo = git.Repo(search_parent_directories=True)
39             initial_state['git'] = {
40                 'commit': repo.head.object.hexsha,
41                 'branch': repo.active_branch.name,
42                 'is_dirty': repo.is_dirty(),
43                 'remote_url': repo.remotes.origin.url
44             }
45         except:
46             initial_state['git'] = None
47
48     # Track Python environment
49     import sys
50     import platform
51     initial_state['environment'] = {
52         'python_version': sys.version,
53         'platform': platform.platform(),
54         'packages': _get_installed_packages()
55     }
56
57     # Track hardware
58     initial_state['hardware'] = {
59         'cpu_count': os.cpu_count(),
60         'cuda_available': torch.cuda.is_available(),
61         'cuda_device_count': torch.cuda.device_count() if torch.cuda.is_available()
62     }
63     else 0
64     }
65
66     # Save reproducibility manifest
67     manifest_path = f"experiment_{experiment_name}_{datetime.now().strftime('%Y%m%d_%H%M%S')}.json"

```

```

66     with open(manifest_path, 'w') as f:
67         json.dump(initial_state, f, indent=2)
68
69     try:
70         yield initial_state
71     finally:
72         # Cleanup or final logging
73         print(f"Reproducibility manifest saved to: {manifest_path}")
74
75 def _get_installed_packages():
76     """Get installed package versions"""
77     import pkg_resources
78     packages = {}
79     for package in pkg_resources.working_set:
80         packages[package.key] = package.version
81     return packages
82
83 # Usage
84 with reproducible_experiment(seed=42, experiment_name="rf_experiment") as ctx:
85     # All code here will be reproducible
86     from sklearn.ensemble import RandomForestClassifier
87
88     model = RandomForestClassifier(random_state=42)
89     # ... training code
90
91     print(f"Git commit: {ctx['git']['commit']}")

```

Listing 3.4: Reproducibility context manager

3.5 Hyperparameter Optimization

Hyperparameter optimization systematically searches for the best model configuration. Different strategies offer trade-offs between search efficiency and solution quality.

3.5.1 Optimization Strategies

3.5.2 Bayesian Optimization with Optuna

Optuna provides efficient hyperparameter optimization with pruning of unpromising trials.

```

1 import optuna
2 from optuna.pruners import MedianPruner
3 from optuna.samplers import TPESampler
4 from optuna.integration import MLflowCallback
5 import mlflow
6 from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
7 from sklearn.model_selection import cross_val_score
8 import numpy as np
9
10 class HyperparameterOptimizer:
11     """Optimize hyperparameters with Optuna and MLflow integration"""
12
13     def __init__(self, X_train, y_train, X_val, y_val):
14         self.X_train = X_train

```

Strategy	How It Works	Best For	Limitations
Grid Search	Exhaustive search over parameter grid	Small search spaces, systematic exploration	Computationally expensive
Random Search	Random sampling from parameter distributions	Medium search spaces, quick baseline	May miss optimal regions
Bayesian Optimization	Probabilistic model guides search	Expensive models, limited budget	Complex to implement
Hyperband	Adaptive resource allocation	Large-scale parallel search	Requires many workers
Population-Based	Evolves population of models	Long training runs, online tuning	High resource requirements

Table 3.2: Hyperparameter optimization strategy comparison

```

15     self.y_train = y_train
16     self.X_val = X_val
17     self.y_val = y_val
18
19     def optimize_random_forest(self, n_trials: int = 100):
20         """Optimize Random Forest hyperparameters"""
21
22         def objective(trial):
23             # Define search space
24             params = {
25                 'n_estimators': trial.suggest_int('n_estimators', 50, 500),
26                 'max_depth': trial.suggest_int('max_depth', 3, 20),
27                 'min_samples_split': trial.suggest_int('min_samples_split', 2, 20),
28                 'min_samples_leaf': trial.suggest_int('min_samples_leaf', 1, 10),
29                 'max_features': trial.suggest_categorical('max_features',
30                                                         ['sqrt', 'log2', None]),
31                 'bootstrap': trial.suggest_categorical('bootstrap', [True, False]),
32                 'random_state': 42
33             }
34
35             # Train model
36             model = RandomForestClassifier(**params)
37             model.fit(self.X_train, self.y_train)
38
39             # Evaluate on validation set
40             val_score = model.score(self.X_val, self.y_val)
41
42             # Also compute cross-validation score
43             cv_scores = cross_val_score(
44                 model, self.X_train, self.y_train, cv=3
45             )
46             cv_mean = cv_scores.mean()
47
48             # Report intermediate values for pruning
49             trial.report(val_score, step=1)
50

```

```

51     # Check if trial should be pruned
52     if trial.should_prune():
53         raise optuna.TrialPruned()
54
55     # Log additional metrics
56     trial.set_user_attr('cv_mean', cv_mean)
57     trial.set_user_attr('cv_std', cv_scores.std())
58
59     return val_score
60
61     # Create study with pruner
62     study = optuna.create_study(
63         study_name='rf_optimization',
64         direction='maximize',
65         sampler=TPESampler(seed=42),
66         pruner=MedianPruner(n_startup_trials=5, n_warmup_steps=10)
67     )
68
69     # MLflow callback
70     mlflow_callback = MLflowCallback(
71         tracking_uri="http://mlflow-server:5000",
72         metric_name="validation_accuracy"
73     )
74
75     # Run optimization
76     study.optimize(
77         objective,
78         n_trials=n_trials,
79         callbacks=[mlflow_callback],
80         show_progress_bar=True
81     )
82
83     # Print results
84     print("Best trial:")
85     print(f"    Value: {study.best_trial.value:.4f}")
86     print("    Params:")
87     for key, value in study.best_trial.params.items():
88         print(f"        {key}: {value}")
89
90     return study
91
92 def optimize_multiple_models(self, n_trials: int = 100):
93     """Optimize across multiple model types"""
94
95     def objective(trial):
96         # Suggest model type
97         model_type = trial.suggest_categorical(
98             'model_type',
99             ['random_forest', 'gradient_boosting']
100         )
101
102         if model_type == 'random_forest':
103             params = {
104                 'n_estimators': trial.suggest_int('n_estimators', 50, 300),
105                 'max_depth': trial.suggest_int('max_depth', 3, 15),
106                 'min_samples_split': trial.suggest_int('min_samples_split', 2, 20)
107             },
108             'random_state': 42

```

```

108         }
109         model = RandomForestClassifier(**params)
110
111     else: # gradient_boosting
112         params = {
113             'n_estimators': trial.suggest_int('n_estimators', 50, 300),
114             'max_depth': trial.suggest_int('max_depth', 3, 10),
115             'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.3,
log=True),
116             'subsample': trial.suggest_float('subsample', 0.5, 1.0),
117             'random_state': 42
118         }
119         model = GradientBoostingClassifier(**params)
120
121     # Train and evaluate
122     model.fit(self.X_train, self.y_train)
123     score = model.score(self.X_val, self.y_val)
124
125     return score
126
127     study = optuna.create_study(
128         study_name='multi_model_optimization',
129         direction='maximize',
130         sampler=TPESampler(seed=42)
131     )
132
133     study.optimize(objective, n_trials=n_trials, show_progress_bar=True)
134
135     return study
136
137 def visualize_optimization(self, study):
138     """Create optimization visualizations"""
139     from optuna.visualization import (
140         plot_optimization_history,
141         plot_param_importances,
142         plot_parallel_coordinate,
143         plot_slice
144     )
145
146     # Optimization history
147     fig1 = plot_optimization_history(study)
148     fig1.write_html('optimization_history.html')
149
150     # Parameter importances
151     fig2 = plot_param_importances(study)
152     fig2.write_html('param_importances.html')
153
154     # Parallel coordinates
155     fig3 = plot_parallel_coordinate(study)
156     fig3.write_html('parallel_coordinates.html')
157
158     # Parameter relationships
159     fig4 = plot_slice(study)
160     fig4.write_html('param_slice.html')
161
162     print("Visualizations saved to HTML files")
163
164 # Usage example

```

```

165 if __name__ == '__main__':
166     from sklearn.datasets import make_classification
167     from sklearn.model_selection import train_test_split
168
169     # Generate data
170     X, y = make_classification(n_samples=10000, n_features=20, random_state=42)
171     X_train, X_temp, y_train, y_temp = train_test_split(
172         X, y, test_size=0.3, random_state=42
173     )
174     X_val, X_test, y_val, y_test = train_test_split(
175         X_temp, y_temp, test_size=0.5, random_state=42
176     )
177
178     # Optimize
179     optimizer = HyperparameterOptimizer(X_train, y_train, X_val, y_val)
180     study = optimizer.optimize_random_forest(n_trials=50)
181
182     # Visualize
183     optimizer.visualize_optimization(study)
184
185     # Train final model with best parameters
186     best_model = RandomForestClassifier(**study.best_params)
187     best_model.fit(X_train, y_train)
188     test_score = best_model.score(X_test, y_test)
189     print(f"\nFinal test accuracy: {test_score:.4f}")

```

Listing 3.5: Advanced hyperparameter optimization with Optuna

3.6 Model Validation and Evaluation Frameworks

Rigorous model validation prevents overfitting and ensures generalization to production data.

3.6.1 Validation Strategies

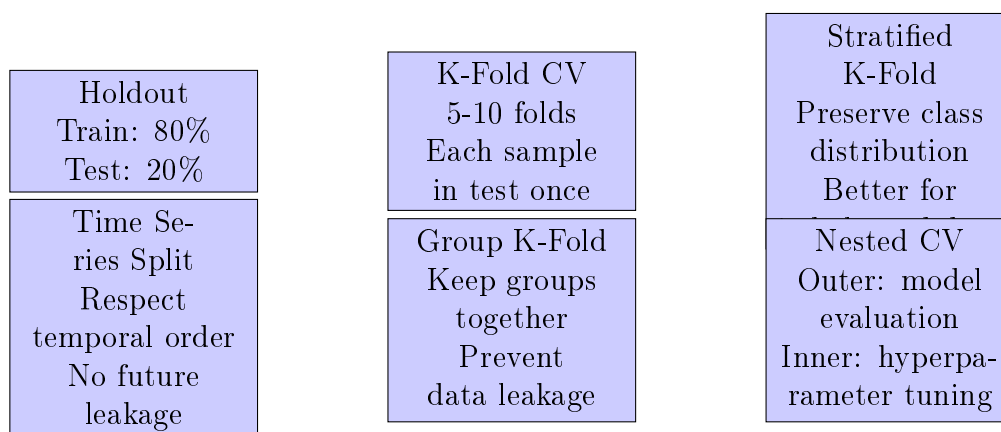


Figure 3.2: Model validation strategies for different scenarios

```

1 from sklearn.model_selection import (
2     cross_val_score, cross_validate,

```

```

3     StratifiedKFold, TimeSeriesSplit, GroupKFold
4 )
5 from sklearn.metrics import (
6     accuracy_score, precision_score, recall_score, f1_score,
7     roc_auc_score, average_precision_score,
8     confusion_matrix, classification_report
9 )
10 import pandas as pd
11 import numpy as np
12 from typing import Dict, List, Any
13 import warnings
14
15 class ModelValidator:
16     """Comprehensive model validation framework"""
17
18     def __init__(self, model, X, y, groups=None, is_timeseries=False):
19         self.model = model
20         self.X = X
21         self.y = y
22         self.groups = groups
23         self.is_timeseries = is_timeseries
24
25     def validate_comprehensive(self, cv_folds: int = 5) -> Dict[str, Any]:
26         """Perform comprehensive validation with multiple metrics"""
27
28         # Choose appropriate cross-validation strategy
29         if self.is_timeseries:
30             cv = TimeSeriesSplit(n_splits=cv_folds)
31             cv_name = "TimeSeriesSplit"
32         elif self.groups is not None:
33             cv = GroupKFold(n_splits=cv_folds)
34             cv_name = "GroupKFold"
35         else:
36             cv = StratifiedKFold(n_splits=cv_folds, shuffle=True, random_state=42)
37             cv_name = "StratifiedKFold"
38
39         # Define scoring metrics
40         scoring = {
41             'accuracy': 'accuracy',
42             'precision': 'precision',
43             'recall': 'recall',
44             'f1': 'f1',
45             'roc_auc': 'roc_auc',
46             'average_precision': 'average_precision'
47         }
48
49         # Perform cross-validation
50         cv_results = cross_validate(
51             self.model,
52             self.X,
53             self.y,
54             cv=cv,
55             groups=self.groups,
56             scoring=scoring,
57             return_train_score=True,
58             n_jobs=-1
59         )
60

```



```

61     # Aggregate results
62     results = {
63         'cv_strategy': cv_name,
64         'cv_folds': cv_folds,
65         'metrics': {}
66     }
67
68     for metric in scoring.keys():
69         train_scores = cv_results[f'train_{metric}']
70         test_scores = cv_results[f'test_{metric}']
71
72         results['metrics'][metric] = {
73             'train_mean': train_scores.mean(),
74             'train_std': train_scores.std(),
75             'test_mean': test_scores.mean(),
76             'test_std': test_scores.std(),
77             'overfitting': train_scores.mean() - test_scores.mean()
78         }
79
80     # Compute additional diagnostics
81     results['diagnostics'] = self._compute_diagnostics(cv_results)
82
83     return results
84
85     def _compute_diagnostics(self, cv_results: Dict) -> Dict[str, float]:
86         """Compute diagnostic metrics"""
87         diagnostics = {}
88
89         # Variance across folds (instability indicator)
90         for metric in ['accuracy', 'precision', 'recall', 'f1']:
91             test_scores = cv_results[f'test_{metric}']
92             diagnostics[f'{metric}_variance'] = test_scores.var()
93
94         # Overfitting ratio
95         train_acc = cv_results['train_accuracy'].mean()
96         test_acc = cv_results['test_accuracy'].mean()
97         diagnostics['overfitting_ratio'] = train_acc / test_acc if test_acc > 0 else
98         np.inf
99
100         return diagnostics
101
102     def learning_curve_analysis(self, train_sizes: List[float] = None):
103         """Analyze learning curves to diagnose bias/variance"""
104         from sklearn.model_selection import learning_curve
105         import matplotlib.pyplot as plt
106
107         if train_sizes is None:
108             train_sizes = np.linspace(0.1, 1.0, 10)
109
110         train_sizes_abs, train_scores, val_scores = learning_curve(
111             self.model,
112             self.X,
113             self.y,
114             train_sizes=train_sizes,
115             cv=5,
116             n_jobs=-1,
117             scoring='accuracy'
118         )

```

```

118
119     # Plot learning curves
120     plt.figure(figsize=(10, 6))
121     plt.plot(train_sizes_abs, train_scores.mean(axis=1),
122             label='Training score', marker='o')
123     plt.plot(train_sizes_abs, val_scores.mean(axis=1),
124             label='Validation score', marker='o')
125     plt.fill_between(train_sizes_abs,
126                     train_scores.mean(axis=1) - train_scores.std(axis=1),
127                     train_scores.mean(axis=1) + train_scores.std(axis=1),
128                     alpha=0.1)
129     plt.fill_between(train_sizes_abs,
130                     val_scores.mean(axis=1) - val_scores.std(axis=1),
131                     val_scores.mean(axis=1) + val_scores.std(axis=1),
132                     alpha=0.1)
133     plt.xlabel('Training Examples')
134     plt.ylabel('Accuracy')
135     plt.title('Learning Curves')
136     plt.legend()
137     plt.grid(True)
138     plt.savefig('learning_curves.png', dpi=300, bbox_inches='tight')
139     plt.close()
140
141     # Diagnose bias/variance
142     final_train_score = train_scores.mean(axis=1)[-1]
143     final_val_score = val_scores.mean(axis=1)[-1]
144     gap = final_train_score - final_val_score
145
146     diagnosis = {
147         'final_train_score': final_train_score,
148         'final_val_score': final_val_score,
149         'train_val_gap': gap,
150         'diagnosis': self._diagnose_bias_variance(final_train_score,
151 final_val_score, gap)
152     }
153
154     return diagnosis
155
156     def _diagnose_bias_variance(self, train_score, val_score, gap):
157         """Diagnose bias/variance issues"""
158         if train_score < 0.85 and val_score < 0.85:
159             return "High Bias (Underfitting): Both scores are low. Try more complex
160 model."
161         elif gap > 0.1:
162             return "High Variance (Overfitting): Large gap between scores. Try
163 regularization, more data, or simpler model."
164         elif train_score > 0.9 and val_score > 0.85 and gap < 0.05:
165             return "Good Fit: Model is performing well."
166         else:
167             return "Moderate fit: Consider further tuning."
168
169     def bootstrap_confidence_intervals(self, n_iterations: int = 1000, confidence:
170 float = 0.95):
171         """Compute bootstrap confidence intervals for metrics"""
172         from sklearn.utils import resample
173
174         bootstrap_scores = {
175             'accuracy': [],

```

```

172         'precision': [],
173         'recall': [],
174         'f1': []
175     }
176
177     for i in range(n_iterations):
178         # Bootstrap sample
179         X_boot, y_boot = resample(self.X, self.y, random_state=i)
180
181         # Train on bootstrap sample
182         self.model.fit(X_boot, y_boot)
183
184         # Evaluate on out-of-bag samples
185         oob_indices = list(set(range(len(self.X))) - set(X_boot.index))
186         if len(oob_indices) > 0:
187             X_oob = self.X.iloc[oob_indices]
188             y_oob = self.y.iloc[oob_indices]
189
190             y_pred = self.model.predict(X_oob)
191
192             bootstrap_scores['accuracy'].append(accuracy_score(y_oob, y_pred))
193             bootstrap_scores['precision'].append(precision_score(y_oob, y_pred,
zero_division=0))
194             bootstrap_scores['recall'].append(recall_score(y_oob, y_pred,
zero_division=0))
195             bootstrap_scores['f1'].append(f1_score(y_oob, y_pred, zero_division=0)
)
196
197         # Compute confidence intervals
198         alpha = 1 - confidence
199         confidence_intervals = {}
200
201         for metric, scores in bootstrap_scores.items():
202             scores = np.array(scores)
203             lower = np.percentile(scores, 100 * alpha / 2)
204             upper = np.percentile(scores, 100 * (1 - alpha / 2))
205             mean = scores.mean()
206
207             confidence_intervals[metric] = {
208                 'mean': mean,
209                 'lower': lower,
210                 'upper': upper,
211                 'std': scores.std()
212             }
213
214     return confidence_intervals
215
216     def statistical_significance_test(self, baseline_model, n_trials: int = 30):
217         """Test if model is significantly better than baseline"""
218         from scipy import stats
219
220         model_scores = []
221         baseline_scores = []
222
223         for i in range(n_trials):
224             # Split data
225             from sklearn.model_selection import train_test_split
226             X_train, X_test, y_train, y_test = train_test_split(

```

```

227         self.X, self.y, test_size=0.2, random_state=i
228     )
229
230     # Train and evaluate both models
231     self.model.fit(X_train, y_train)
232     baseline_model.fit(X_train, y_train)
233
234     model_scores.append(self.model.score(X_test, y_test))
235     baseline_scores.append(baseline_model.score(X_test, y_test))
236
237     # Paired t-test
238     t_stat, p_value = stats.ttest_rel(model_scores, baseline_scores)
239
240     # Effect size (Cohen's d)
241     diff = np.array(model_scores) - np.array(baseline_scores)
242     cohens_d = diff.mean() / diff.std()
243
244     result = {
245         'model_mean': np.mean(model_scores),
246         'baseline_mean': np.mean(baseline_scores),
247         'difference': np.mean(model_scores) - np.mean(baseline_scores),
248         't_statistic': t_stat,
249         'p_value': p_value,
250         'cohens_d': cohens_d,
251         'significant': p_value < 0.05,
252         'interpretation': self._interpret_significance(p_value, cohens_d)
253     }
254
255     return result
256
257     def _interpret_significance(self, p_value, cohens_d):
258         """Interpret statistical significance results"""
259         if p_value >= 0.05:
260             return "No significant difference between models"
261         elif abs(cohens_d) < 0.2:
262             return "Statistically significant but small effect size"
263         elif abs(cohens_d) < 0.5:
264             return "Statistically significant with medium effect size"
265         else:
266             return "Statistically significant with large effect size"
267
268     # Usage example
269     if __name__ == '__main__':
270         from sklearn.datasets import make_classification
271         from sklearn.ensemble import RandomForestClassifier
272         from sklearn.tree import DecisionTreeClassifier
273
274         # Generate data
275         X, y = make_classification(n_samples=1000, n_features=20, random_state=42)
276         X = pd.DataFrame(X, columns=[f'feature_{i}' for i in range(20)])
277         y = pd.Series(y)
278
279         # Create validator
280         model = RandomForestClassifier(n_estimators=100, random_state=42)
281         validator = ModelValidator(model, X, y)
282
283         # Comprehensive validation
284         results = validator.validate_comprehensive(cv_folds=5)

```

```

285 print("Cross-Validation Results:")
286 for metric, values in results['metrics'].items():
287     print(f"    {metric}:")
288     print(f"        Test: {values['test_mean']:.4f} +/- {values['test_std']:.4f}")
289     print(f"        Train: {values['train_mean']:.4f} +/- {values['train_std']:.4f}")
290     print(f"        Overfitting: {values['overfitting']:.4f}")
291
292 # Learning curves
293 lc_results = validator.learning_curve_analysis()
294 print(f"\nLearning Curve Diagnosis: {lc_results['diagnosis']}")
295
296 # Bootstrap confidence intervals
297 ci_results = validator.bootstrap_confidence_intervals(n_iterations=100)
298 print("\nBootstrap Confidence Intervals (95%):")
299 for metric, values in ci_results.items():
300     print(f"    {metric}: {values['mean']:.4f} [{values['lower']:.4f}, {values['upper']:.4f}]")
301
302 # Compare to baseline
303 baseline = DecisionTreeClassifier(random_state=42)
304 sig_test = validator.statistical_significance_test(baseline, n_trials=10)
305 print(f"\nStatistical Significance Test:")
306 print(f"    Model: {sig_test['model_mean']:.4f}")
307 print(f"    Baseline: {sig_test['baseline_mean']:.4f}")
308 print(f"    p-value: {sig_test['p_value']:.4f}")
309 print(f"    {sig_test['interpretation']}")

```

Listing 3.6: Comprehensive model validation framework

3.7 A/B Testing for ML Models

A/B testing validates model improvements in production by comparing challenger models against the current champion.

3.7.1 A/B Testing Framework

```

1 from dataclasses import dataclass
2 from typing import Dict, List, Callable, Any
3 import numpy as np
4 import pandas as pd
5 from scipy import stats
6 from datetime import datetime, timedelta
7 import hashlib
8
9 @dataclass
10 class ABTestConfig:
11     """Configuration for A/B test"""
12     test_name: str
13     control_model_id: str
14     treatment_model_id: str
15     traffic_split: float = 0.5
16     min_sample_size: int = 1000
17     min_duration_days: int = 7
18     significance_level: float = 0.05
19     metrics: List[str] = None

```

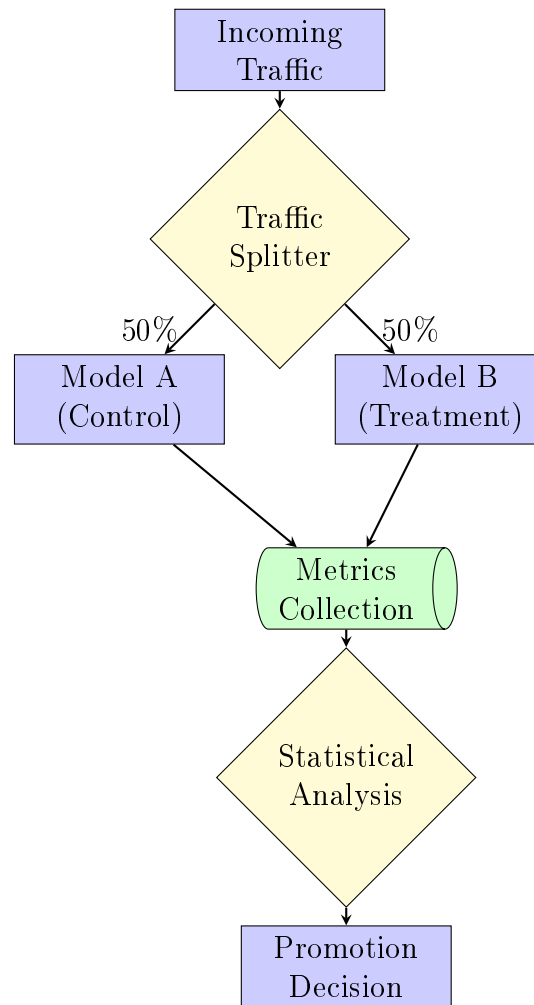


Figure 3.3: A/B testing framework for ML models

```

20
21 class ABTestManager:
22     """Manage A/B tests for ML models"""
23
24     def __init__(self, config: ABTestConfig):
25         self.config = config
26         self.assignments = {} # user_id -> variant
27         self.results = {'control': [], 'treatment': []}
28         self.start_time = datetime.now()
29
30     def assign_variant(self, user_id: str) -> str:
31         """Assign user to variant using consistent hashing"""
32         # Check if user already assigned
33         if user_id in self.assignments:
34             return self.assignments[user_id]
35
36         # Hash user ID for consistent assignment
37         hash_value = int(hashlib.md5(
38             f"{user_id}{self.config.test_name}".encode()
39         ).hexdigest(), 16)
40
41         # Assign based on hash and traffic split
42         if (hash_value % 100) / 100 < self.config.traffic_split:

```

```

43         variant = 'control'
44     else:
45         variant = 'treatment'
46
47     self.assignments[user_id] = variant
48     return variant
49
50     def get_model_for_user(self, user_id: str) -> str:
51         """Get appropriate model ID for user"""
52         variant = self.assign_variant(user_id)
53
54         if variant == 'control':
55             return self.config.control_model_id
56         else:
57             return self.config.treatment_model_id
58
59     def log_result(self, user_id: str, metrics: Dict[str, float]):
60         """Log result for user"""
61         variant = self.assignments.get(user_id)
62         if variant:
63             self.results[variant].append({
64                 'user_id': user_id,
65                 'timestamp': datetime.now(),
66                 **metrics
67             })
68
69     def analyze_results(self) -> Dict[str, Any]:
70         """Analyze A/B test results"""
71         # Convert to DataFrames
72         control_df = pd.DataFrame(self.results['control'])
73         treatment_df = pd.DataFrame(self.results['treatment'])
74
75         if len(control_df) < self.config.min_sample_size or \
76            len(treatment_df) < self.config.min_sample_size:
77             return {
78                 'status': 'insufficient_data',
79                 'control_samples': len(control_df),
80                 'treatment_samples': len(treatment_df),
81                 'required_samples': self.config.min_sample_size
82             }
83
84         # Check duration
85         duration = datetime.now() - self.start_time
86         if duration < timedelta(days=self.config.min_duration_days):
87             return {
88                 'status': 'insufficient_duration',
89                 'current_duration_hours': duration.total_seconds() / 3600,
90                 'required_duration_hours': self.config.min_duration_days * 24
91             }
92
93         # Analyze each metric
94         analysis_results = {
95             'status': 'complete',
96             'duration_days': duration.days,
97             'control_samples': len(control_df),
98             'treatment_samples': len(treatment_df),
99             'metrics': {}
100         }

```

```

101
102     for metric in self.config.metrics:
103         control_values = control_df[metric].values
104         treatment_values = treatment_df[metric].values
105
106         metric_analysis = self._analyze_metric(
107             metric,
108             control_values,
109             treatment_values
110         )
111
112         analysis_results['metrics'][metric] = metric_analysis
113
114     # Overall recommendation
115     analysis_results['recommendation'] = self._make_recommendation(
116         analysis_results['metrics']
117     )
118
119     return analysis_results
120
121     def _analyze_metric(self, metric_name: str,
122                        control: np.ndarray,
123                        treatment: np.ndarray) -> Dict[str, Any]:
124         """Analyze single metric"""
125         # Descriptive statistics
126         control_mean = control.mean()
127         treatment_mean = treatment.mean()
128         lift = (treatment_mean - control_mean) / control_mean if control_mean != 0
129     else 0
130
131         # Statistical test
132         t_stat, p_value = stats.ttest_ind(treatment, control)
133
134         # Effect size (Cohen's d)
135         pooled_std = np.sqrt((control.var() + treatment.var()) / 2)
136         cohens_d = (treatment_mean - control_mean) / pooled_std if pooled_std != 0
137     else 0
138
139         # Confidence interval for difference
140         se_diff = np.sqrt(control.var()/len(control) + treatment.var()/len(treatment))
141         ci_lower = (treatment_mean - control_mean) - 1.96 * se_diff
142         ci_upper = (treatment_mean - control_mean) + 1.96 * se_diff
143
144         # Power analysis
145         power = self._compute_power(control, treatment, self.config.significance_level)
146
147     )
148
149     return {
150         'control_mean': control_mean,
151         'control_std': control.std(),
152         'treatment_mean': treatment_mean,
153         'treatment_std': treatment.std(),
154         'lift_percent': lift * 100,
155         'absolute_difference': treatment_mean - control_mean,
156         'ci_lower': ci_lower,
157         'ci_upper': ci_upper,
158         'p_value': p_value,
159         'cohens_d': cohens_d,

```



```

156         'power': power,
157         'significant': p_value < self.config.significance_level,
158         'interpretation': self._interpret_result(p_value, lift, cohens_d, power)
159     }
160
161     def _compute_power(self, control, treatment, alpha):
162         """Compute statistical power"""
163         from statsmodels.stats.power import ttest_power
164
165         effect_size = (treatment.mean() - control.mean()) / np.sqrt(
166             (control.var() + treatment.var()) / 2
167         )
168
169         nobs = min(len(control), len(treatment))
170
171         try:
172             power = ttest_power(
173                 effect_size,
174                 nobs=nobs,
175                 alpha=alpha,
176                 alternative='two-sided'
177             )
178             return power
179         except:
180             return None
181
182     def _interpret_result(self, p_value, lift, cohens_d, power):
183         """Interpret test results"""
184         if p_value >= self.config.significance_level:
185             return "No significant difference detected"
186
187         if power and power < 0.8:
188             return "Significant but underpowered - consider longer test"
189
190         if abs(cohens_d) < 0.2:
191             return f"Significant with {lift*100:.2f}% lift but small effect size"
192         elif abs(cohens_d) < 0.5:
193             return f"Significant with {lift*100:.2f}% lift and medium effect size"
194         else:
195             return f"Significant with {lift*100:.2f}% lift and large effect size"
196
197     def _make_recommendation(self, metrics_analysis: Dict) -> str:
198         """Make overall recommendation based on all metrics"""
199         primary_metric = self.config.metrics[0]
200         primary_result = metrics_analysis[primary_metric]
201
202         if not primary_result['significant']:
203             return "KEEP CONTROL: No significant improvement in primary metric"
204
205         if primary_result['lift_percent'] < 0:
206             return "KEEP CONTROL: Treatment performed worse"
207
208         # Check if all metrics improved or stayed neutral
209         all_positive = all(
210             m['lift_percent'] >= 0 or not m['significant']
211             for m in metrics_analysis.values()
212         )
213

```

```

214         if not all_positive:
215             return "CAUTIOUS ROLLOUT: Primary improved but some metrics degraded"
216
217         # Check power
218         if primary_result.get('power') and primary_result['power'] < 0.8:
219             return "EXTEND TEST: Results promising but underpowered"
220
221         return f"PROMOTE TREATMENT: Significant improvement of {primary_result['lift_percent']:.2f}%"
222
223     def sequential_analysis(self, alpha_spending: str = 'obrien_fleming'):
224         """Perform sequential analysis for early stopping"""
225         from statsmodels.stats.multitest import multipletests
226
227         # This is a simplified sequential testing approach
228         # Production systems should use proper sequential testing frameworks
229
230         control_df = pd.DataFrame(self.results['control'])
231         treatment_df = pd.DataFrame(self.results['treatment'])
232
233         if len(control_df) < 100 or len(treatment_df) < 100:
234             return {'decision': 'continue', 'reason': 'Insufficient data for interim analysis'}
235
236         # Run interim analysis
237         interim_results = self.analyze_results()
238
239         if interim_results['status'] != 'complete':
240             return {'decision': 'continue', 'reason': interim_results['status']}
241
242         primary_metric = self.config.metrics[0]
243         p_value = interim_results['metrics'][primary_metric]['p_value']
244
245         # Adjust significance level for interim analysis (Bonferroni correction)
246         # In practice, use more sophisticated methods like O'Brien-Fleming
247         adjusted_alpha = self.config.significance_level / 5 # Assume max 5 interim analyses
248
249         if p_value < adjusted_alpha and interim_results['metrics'][primary_metric]['lift_percent'] > 5:
250             return {
251                 'decision': 'stop_early_treatment_wins',
252                 'reason': f"Strong evidence for treatment (p={p_value:.4f})"
253             }
254         elif p_value < adjusted_alpha and interim_results['metrics'][primary_metric]['lift_percent'] < -5:
255             return {
256                 'decision': 'stop_early_control_wins',
257                 'reason': f"Treatment performs worse (p={p_value:.4f})"
258             }
259         else:
260             return {'decision': 'continue', 'reason': 'Inconclusive interim results'}
261
262     # Usage example
263     if __name__ == '__main__':
264         # Configure A/B test
265         config = ABTestConfig(
266             test_name='model_v2_test',

```

```

267     control_model_id='model_v1',
268     treatment_model_id='model_v2',
269     traffic_split=0.5,
270     min_sample_size=1000,
271     min_duration_days=7,
272     metrics=['ctr', 'conversion_rate', 'revenue']
273 )
274
275 ab_test = ABTestManager(config)
276
277 # Simulate user interactions
278 np.random.seed(42)
279 for user_id in range(3000):
280     user_id_str = f'user_{user_id}'
281
282     # Get model assignment
283     model_id = ab_test.get_model_for_user(user_id_str)
284
285     # Simulate metrics (treatment slightly better)
286     if model_id == 'model_v1':
287         metrics = {
288             'ctr': np.random.binomial(1, 0.10),
289             'conversion_rate': np.random.binomial(1, 0.05),
290             'revenue': np.random.exponential(10) if np.random.random() < 0.05 else
0
291         }
292     else:
293         metrics = {
294             'ctr': np.random.binomial(1, 0.12), # 20% relative lift
295             'conversion_rate': np.random.binomial(1, 0.055), # 10% relative lift
296             'revenue': np.random.exponential(11) if np.random.random() < 0.055
297         }
298
299     ab_test.log_result(user_id_str, metrics)
300
301 # Analyze results
302 results = ab_test.analyze_results()
303
304 print(f"Status: {results['status']}")
305 print(f"Duration: {results.get('duration_days', 0)} days")
306 print(f"\nResults:")
307 for metric, analysis in results.get('metrics', {}).items():
308     print(f"\n{metric}:")
309     print(f"    Control: {analysis['control_mean']:.4f} +- {analysis['control_std
']:.4f}")
310     print(f"    Treatment: {analysis['treatment_mean']:.4f} +- {analysis['
treatment_std']:.4f}")
311     print(f"    Lift: {analysis['lift_percent']:.2f}%")
312     print(f"    p-value: {analysis['p_value']:.4f}")
313     print(f"    Significant: {analysis['significant']}")
314     print(f"    {analysis['interpretation']}")
315
316 print(f"\n{results.get('recommendation', 'No recommendation yet')}")

```

Listing 3.7: Production A/B testing framework for ML models

3.8 BigQuery ML ARIMA_PLUS Case Study

This case study demonstrates end-to-end ML pipeline design using BigQuery ML for time series forecasting with automated model selection and evaluation.

3.8.1 Problem: Sales Forecasting

A retail company needs to forecast daily sales for inventory planning. The pipeline must:

- Train models weekly on historical sales data
- Evaluate forecast accuracy automatically
- Deploy best models to production
- Monitor model drift and retrain when needed

```

1 from google.cloud import bigquery
2 from google.cloud.bigquery import ScalarQueryParameter
3 import pandas as pd
4 from datetime import datetime, timedelta
5 import mlflow
6
7 class BigQueryMLPipeline:
8     """Production pipeline for BigQuery ML ARIMA_PLUS forecasting"""
9
10    def __init__(self, project_id: str, dataset_id: str):
11        self.client = bigquery.Client(project=project_id)
12        self.project_id = project_id
13        self.dataset_id = dataset_id
14        self.model_name = "sales_forecast_model"
15
16    def create_training_data(self, start_date: str, end_date: str):
17        """Prepare time series data for training"""
18        query = f"""
19        CREATE OR REPLACE TABLE '{self.project_id}.{self.dataset_id}.training_data' AS
20        SELECT
21            DATE(timestamp) as date,
22            SUM(sales) as total_sales,
23            AVG(temperature) as avg_temperature,
24            MAX(is_holiday) as is_holiday,
25            MAX(is_weekend) as is_weekend
26        FROM '{self.project_id}.{self.dataset_id}.raw_sales'
27        WHERE DATE(timestamp) BETWEEN @start_date AND @end_date
28        GROUP BY DATE(timestamp)
29        ORDER BY date
30        """
31
32        job_config = bigquery.QueryJobConfig(
33            query_parameters=[
34                ScalarQueryParameter("start_date", "DATE", start_date),
35                ScalarQueryParameter("end_date", "DATE", end_date),
36            ]
37        )
38
39        query_job = self.client.query(query, job_config=job_config)
40        query_job.result()

```

```

41     print(f"Training data created: {start_date} to {end_date}")
42
43
44     def train_arima_model(self):
45         """Train ARIMA_PLUS model with auto parameter selection"""
46         query = f"""
47         CREATE OR REPLACE MODEL '{self.project_id}.{self.dataset_id}.{self.model_name}
48         '
49         OPTIONS(
50             model_type='ARIMA_PLUS',
51             time_series_timestamp_col='date',
52             time_series_data_col='total_sales',
53             time_series_id_col=NULL,
54             holiday_region='US',
55             data_frequency='AUTO_FREQUENCY',
56
57             -- Include additional regressors
58             include_drift=TRUE,
59             clean_spikes_and_dips=TRUE,
60             adjust_step_changes=TRUE,
61
62             -- Auto parameter selection
63             auto_arima=TRUE,
64             auto_arima_max_order=5,
65
66             -- Model evaluation
67             decompose_time_series=TRUE
68         ) AS
69         SELECT
70             date,
71             total_sales,
72             avg_temperature,
73             is_holiday,
74             is_weekend
75         FROM '{self.project_id}.{self.dataset_id}.training_data'
76         """
77
78         query_job = self.client.query(query)
79         query_job.result()
80
81         print(f"ARIMA_PLUS model trained: {self.model_name}")
82
83     def evaluate_model(self) -> pd.DataFrame:
84         """Evaluate model performance"""
85         # Get training metrics
86         metrics_query = f"""
87         SELECT
88             *
89         FROM ML.ARIMA_EVALUATE(
90             MODEL '{self.project_id}.{self.dataset_id}.{self.model_name}'
91         )
92         """
93
94         metrics_df = self.client.query(metrics_query).to_dataframe()
95
96         # Get detailed evaluation
97         eval_query = f"""
98         SELECT

```

```

98         *
99         FROM ML.EVALUATE(
100             MODEL '{self.project_id}.{self.dataset_id}.{self.model_name}',
101             (
102                 SELECT
103                     date,
104                     total_sales,
105                     avg_temperature,
106                     is_holiday,
107                     is_weekend
108                 FROM '{self.project_id}.{self.dataset_id}.training_data'
109                 ORDER BY date DESC
110                 LIMIT 30 -- Last 30 days for validation
111             )
112         )
113         """
114
115         eval_df = self.client.query(eval_query).to_dataframe()
116
117         return {
118             'arima_metrics': metrics_df,
119             'evaluation': eval_df
120         }
121
122     def get_model_coefficients(self) -> pd.DataFrame:
123         """Extract model coefficients and parameters"""
124         query = f"""
125         SELECT
126             *
127         FROM ML.ARIMA_COEFFICIENTS(
128             MODEL '{self.project_id}.{self.dataset_id}.{self.model_name}'
129         )
130         """
131
132         return self.client.query(query).to_dataframe()
133
134     def forecast(self, horizon: int = 30, confidence_level: float = 0.95):
135         """Generate forecasts with confidence intervals"""
136         query = f"""
137         SELECT
138             *
139         FROM ML.FORECAST(
140             MODEL '{self.project_id}.{self.dataset_id}.{self.model_name}',
141             STRUCT(
142                 {horizon} AS horizon,
143                 {confidence_level} AS confidence_level
144             )
145         )
146         """
147
148         forecast_df = self.client.query(query).to_dataframe()
149
150         # Save forecast to table
151         table_id = f"{self.project_id}.{self.dataset_id}.forecasts"
152
153         forecast_df['forecast_timestamp'] = datetime.now()
154         forecast_df['model_name'] = self.model_name
155

```

```

156     job_config = bigquery.LoadJobConfig(
157         write_disposition=bigquery.WriteDisposition.WRITE_APPEND
158     )
159
160     self.client.load_table_from_dataframe(
161         forecast_df, table_id, job_config=job_config
162     ).result()
163
164     return forecast_df
165
166 def detect_anomalies(self, threshold: float = 0.95):
167     """Detect anomalies in time series"""
168     query = f"""
169     SELECT
170         *
171     FROM ML.DETECT_ANOMALIES(
172         MODEL '{self.project_id}.{self.dataset_id}.{self.model_name}',
173         STRUCT(
174             {threshold} AS anomaly_prob_threshold
175         ),
176         (
177             SELECT
178                 date,
179                 total_sales,
180                 avg_temperature,
181                 is_holiday,
182                 is_weekend
183             FROM '{self.project_id}.{self.dataset_id}.training_data'
184             ORDER BY date DESC
185             LIMIT 90 -- Last 90 days
186         )
187     )
188     """
189
190     anomalies_df = self.client.query(query).to_dataframe()
191     return anomalies_df
192
193 def explain_forecast(self, forecast_date: str):
194     """Explain forecast using feature attributions"""
195     query = f"""
196     SELECT
197         *
198     FROM ML.EXPLAIN_FORECAST(
199         MODEL '{self.project_id}.{self.dataset_id}.{self.model_name}',
200         STRUCT(
201             30 AS horizon,
202             0.95 AS confidence_level
203         )
204     )
205     WHERE forecast_timestamp = @forecast_date
206     """
207
208     job_config = bigquery.QueryJobConfig(
209         query_parameters=[
210             ScalarQueryParameter("forecast_date", "DATE", forecast_date),
211         ]
212     )
213

```

```

214     explanation_df = self.client.query(query, job_config=job_config).to_dataframe
215     ()
216     return explanation_df
217
218 def run_pipeline(self, train_start: str, train_end: str,
219                 forecast_horizon: int = 30):
220     """Execute complete pipeline"""
221     print("=" * 60)
222     print("BigQuery ML ARIMA_PLUS Forecasting Pipeline")
223     print("=" * 60)
224
225     # MLflow tracking
226     mlflow.set_experiment("bigquery_ml_forecasting")
227
228     with mlflow.start_run(run_name=f"arima_plus_{datetime.now().strftime('%Y%m%d')}"):
229         # Log parameters
230         mlflow.log_param("train_start", train_start)
231         mlflow.log_param("train_end", train_end)
232         mlflow.log_param("forecast_horizon", forecast_horizon)
233         mlflow.log_param("model_type", "ARIMA_PLUS")
234
235         # Step 1: Create training data
236         print("\n1. Creating training data...")
237         self.create_training_data(train_start, train_end)
238
239         # Step 2: Train model
240         print("\n2. Training ARIMA_PLUS model...")
241         self.train_arima_model()
242
243         # Step 3: Evaluate model
244         print("\n3. Evaluating model...")
245         evaluation = self.evaluate_model()
246
247         # Log metrics
248         arima_metrics = evaluation['arima_metrics'].iloc[0]
249         mlflow.log_metric("AIC", arima_metrics.get('AIC', 0))
250         mlflow.log_metric("variance", arima_metrics.get('variance', 0))
251
252         if 'mean_absolute_error' in evaluation['evaluation'].columns:
253             mae = evaluation['evaluation']['mean_absolute_error'].iloc[0]
254             mlflow.log_metric("MAE", mae)
255
256         print(f"\nModel Metrics:")
257         print(evaluation['arima_metrics'])
258
259         # Step 4: Get model details
260         print("\n4. Extracting model coefficients...")
261         coefficients = self.get_model_coefficients()
262         print(coefficients)
263
264         # Step 5: Generate forecast
265         print(f"\n5. Generating {forecast_horizon}-day forecast...")
266         forecast = self.forecast(horizon=forecast_horizon)
267         print(f"Forecast generated for {len(forecast)} time steps")
268
269         # Log forecast sample
270         mlflow.log_text(

```



```

270         forecast.head(10).to_string(),
271         "forecast_sample.txt"
272     )
273
274     # Step 6: Detect anomalies
275     print("\n6. Detecting anomalies...")
276     anomalies = self.detect_anomalies()
277     num_anomalies = len(anomalies[anomalies['is_anomaly'] == True])
278     print(f"Detected {num_anomalies} anomalies in last 90 days")
279
280     mlflow.log_metric("num_anomalies", num_anomalies)
281
282     print("\n" + "=" * 60)
283     print("Pipeline completed successfully!")
284     print("=" * 60)
285
286     return {
287         'evaluation': evaluation,
288         'coefficients': coefficients,
289         'forecast': forecast,
290         'anomalies': anomalies
291     }
292
293     # Airflow DAG for automated execution
294     from airflow import DAG
295     from airflow.operators.python import PythonOperator
296     from airflow.utils.dates import days_ago
297     from datetime import timedelta
298
299     default_args = {
300         'owner': 'ml_team',
301         'depends_on_past': False,
302         'email_on_failure': True,
303         'email': ['ml-alerts@company.com'],
304         'retries': 2,
305         'retry_delay': timedelta(minutes=5),
306     }
307
308     def run_bigquery_ml_pipeline(**context):
309         """Airflow task to run BigQuery ML pipeline"""
310         execution_date = context['execution_date']
311
312         # Training window: last 365 days before execution date
313         train_end = execution_date.date()
314         train_start = train_end - timedelta(days=365)
315
316         # Initialize pipeline
317         pipeline = BigQueryMLPipeline(
318             project_id='my-project',
319             dataset_id='sales_forecasting'
320         )
321
322         # Run pipeline
323         results = pipeline.run_pipeline(
324             train_start=train_start.isoformat(),
325             train_end=train_end.isoformat(),
326             forecast_horizon=30
327         )

```

```

328
329     # Push key metrics to XCom
330     context['task_instance'].xcom_push(
331         key='num_anomalies',
332         value=len(results['anomalies'][results['anomalies']['is_anomaly'] == True])
333     )
334
335 with DAG(
336     'bigquery_ml_forecasting',
337     default_args=default_args,
338     description='Weekly sales forecasting with BigQuery ML',
339     schedule_interval='0 2 * * 0', # Weekly on Sunday
340     start_date=days_ago(1),
341     catchup=False,
342     tags=['ml', 'forecasting', 'bigquery']
343 ) as dag:
344
345     forecast_task = PythonOperator(
346         task_id='run_arima_forecast',
347         python_callable=run_bigquery_ml_pipeline,
348         provide_context=True,
349         execution_timeout=timedelta(hours=2)
350     )
351
352 # Usage example
353 if __name__ == '__main__':
354     pipeline = BigQueryMLPipeline(
355         project_id='my-project',
356         dataset_id='sales_forecasting'
357     )
358
359     results = pipeline.run_pipeline(
360         train_start='2023-01-01',
361         train_end='2024-01-01',
362         forecast_horizon=30
363     )
364
365     print("\nForecast Preview:")
366     print(results['forecast'][['forecast_timestamp', 'forecast_value',
367                                'standard_error', 'confidence_level']].head())

```

Listing 3.8: BigQuery ML ARIMA_PLUS automated forecasting pipeline

3.9 Failure Handling and Retry Mechanisms

Production pipelines must handle failures gracefully with appropriate retry strategies and circuit breakers.

```

1 from functools import wraps
2 import time
3 import logging
4 from typing import Callable, Any
5 from enum import Enum
6
7 logger = logging.getLogger(__name__)
8

```

```

9 class RetryStrategy(Enum):
10     """Retry strategy types"""
11     EXPONENTIAL_BACKOFF = "exponential"
12     LINEAR_BACKOFF = "linear"
13     FIXED_DELAY = "fixed"
14
15 class CircuitBreaker:
16     """Circuit breaker pattern for external service calls"""
17
18     def __init__(self, failure_threshold: int = 5, timeout: int = 60):
19         self.failure_threshold = failure_threshold
20         self.timeout = timeout
21         self.failure_count = 0
22         self.last_failure_time = None
23         self.state = "CLOSED" # CLOSED, OPEN, HALF_OPEN
24
25     def call(self, func: Callable, *args, **kwargs) -> Any:
26         """Execute function with circuit breaker"""
27         if self.state == "OPEN":
28             if time.time() - self.last_failure_time > self.timeout:
29                 self.state = "HALF_OPEN"
30                 logger.info("Circuit breaker entering HALF_OPEN state")
31             else:
32                 raise Exception(f"Circuit breaker OPEN - service unavailable")
33
34         try:
35             result = func(*args, **kwargs)
36
37             if self.state == "HALF_OPEN":
38                 self.state = "CLOSED"
39                 self.failure_count = 0
40                 logger.info("Circuit breaker reset to CLOSED")
41
42             return result
43
44         except Exception as e:
45             self.failure_count += 1
46             self.last_failure_time = time.time()
47
48             if self.failure_count >= self.failure_threshold:
49                 self.state = "OPEN"
50                 logger.error(f"Circuit breaker opened after {self.failure_count}
failures")
51
52             raise
53
54     def retry_with_backoff(
55         max_attempts: int = 3,
56         strategy: RetryStrategy = RetryStrategy.EXPONENTIAL_BACKOFF,
57         base_delay: float = 1.0,
58         max_delay: float = 60.0,
59         exceptions: tuple = (Exception,),
60         on_retry: Callable = None
61     ):
62         """Decorator for retry with configurable backoff strategy"""
63
64         def decorator(func):
65             @wraps(func)

```

```

66     def wrapper(*args, **kwargs):
67         attempt = 0
68         last_exception = None
69
70         while attempt < max_attempts:
71             try:
72                 return func(*args, **kwargs)
73
74             except exceptions as e:
75                 attempt += 1
76                 last_exception = e
77
78                 if attempt >= max_attempts:
79                     logger.error(
80                         f"Function {func.__name__} failed after {max_attempts}
attempts"
81                     )
82                     raise
83
84                 # Calculate delay
85                 if strategy == RetryStrategy.EXPONENTIAL_BACKOFF:
86                     delay = min(base_delay * (2 ** (attempt - 1)), max_delay)
87                 elif strategy == RetryStrategy.LINEAR_BACKOFF:
88                     delay = min(base_delay * attempt, max_delay)
89                 else: # FIXED_DELAY
90                     delay = base_delay
91
92                 logger.warning(
93                     f"Attempt {attempt}/{max_attempts} failed for {func.__name__}:
{e}. "
94                     f"Retrying in {delay:.2f}s..."
95                 )
96
97                 # Call retry callback if provided
98                 if on_retry:
99                     on_retry(attempt, delay, e)
100
101                 time.sleep(delay)
102
103             raise last_exception
104
105         return wrapper
106     return decorator
107
108     # Usage in pipeline tasks
109     @retry_with_backoff(
110         max_attempts=3,
111         strategy=RetryStrategy.EXPONENTIAL_BACKOFF,
112         base_delay=1.0,
113         exceptions=(ConnectionError, TimeoutError)
114     )
115     def fetch_training_data():
116         """Fetch data with automatic retry"""
117         # Implementation with potential network failures
118         pass
119
120     # Circuit breaker usage
121     model_service_breaker = CircuitBreaker(failure_threshold=5, timeout=60)

```

```
122
123 def call_model_service(data):
124     """Call external model service with circuit breaker"""
125     return model_service_breaker.call(
126         _internal_service_call,
127         data
128     )
129
130 def _internal_service_call(data):
131     # Actual service call implementation
132     pass
```

Listing 3.9: Robust failure handling for ML pipelines

3.10 Chapter Summary

This chapter covered comprehensive training pipeline design patterns:

- **Pipeline Orchestration:** Airflow, Kubeflow Pipelines with production examples
- **Experiment Tracking:** MLflow integration for reproducibility and comparison
- **Hyperparameter Optimization:** Optuna-based Bayesian optimization at scale
- **Model Validation:** Statistical validation frameworks with bias/variance analysis
- **A/B Testing:** Production A/B testing framework with statistical rigor
- **BigQuery ML Case Study:** End-to-end ARIMA_PLUS forecasting pipeline
- **Failure Handling:** Retry mechanisms, circuit breakers, and error recovery

Key takeaways:

1. Training pipelines require different orchestration than traditional software
2. Experiment tracking is essential for reproducibility and debugging
3. Hyperparameter optimization should be automated and integrated into pipelines
4. Model validation must go beyond simple accuracy metrics
5. A/B testing validates model improvements in production
6. Failure handling is critical for production reliability

3.11 Exercises

Exercise 3.1. Design Complete Training Pipeline

Design and implement an end-to-end training pipeline using Airflow that:

1. Extracts data from a data warehouse
2. Performs data validation with Great Expectations

3. Trains multiple model types (RF, GBM, XGBoost)
4. Logs all experiments to MLflow
5. Selects best model based on validation metrics
6. Deploys to a model registry only if quality thresholds are met
7. Sends notifications on success/failure

Include proper error handling, retries, and resource management.

Exercise 3.2. Implement Experiment Tracking

Create an experiment tracking system that:

- Tracks code version (git commit), data version (hash), environment (packages)
- Logs all hyperparameters and metrics
- Creates comparison visualizations between experiments
- Identifies the best model for production
- Ensures full reproducibility

Test reproducibility by running the same experiment twice and verifying identical results.

Exercise 3.3. Hyperparameter Optimization

Implement hyperparameter optimization using Optuna that:

1. Optimizes across multiple model types
2. Uses early stopping to prune unpromising trials
3. Integrates with MLflow for experiment tracking
4. Creates visualization of the optimization process
5. Computes parameter importance

Target: Find optimal hyperparameters within 100 trials that beat baseline by $>5\%$.

Exercise 3.4. Model Validation Framework

Build a comprehensive validation framework that:

- Implements k-fold cross-validation with appropriate strategy
- Generates learning curves to diagnose bias/variance
- Computes bootstrap confidence intervals for metrics
- Performs statistical significance testing vs baseline
- Creates detailed validation reports

Apply to a classification problem and interpret all diagnostic plots.

Exercise 3.5. A/B Testing Framework

Implement a production A/B testing framework:

1. Assign users consistently to control/treatment via hashing
2. Track multiple metrics (primary and guardrail metrics)
3. Perform statistical analysis with proper significance testing
4. Compute confidence intervals and effect sizes
5. Provide actionable recommendations

Simulate an A/B test and demonstrate when to promote treatment vs keep control.

Exercise 3.6. BigQuery ML Pipeline

Create a production forecasting pipeline using BigQuery ML:

- Train ARIMA_PLUS models on time series data
- Evaluate model performance automatically
- Generate forecasts with confidence intervals
- Detect anomalies in recent data
- Explain forecasts with feature attributions
- Orchestrate with Airflow on a weekly schedule

Deploy to production and monitor forecast accuracy over time.

Exercise 3.7. Failure Recovery Mechanisms

Implement robust failure handling for ML pipelines:

1. Retry decorator with exponential backoff
2. Circuit breaker for external service calls
3. Checkpoint mechanism for long-running training
4. Dead letter queue for failed records
5. Alerting system for critical failures

Test by injecting failures and verifying proper recovery.

Exercise 3.8. Pipeline-as-Code

Convert an existing ML workflow to pipeline-as-code:

- Define pipeline in Python/YAML
- Parameterize all configurations
- Version control pipeline definitions
- Implement CI/CD for pipeline deployments
- Create automated tests for pipeline components

Deploy the same pipeline to dev, staging, and production environments.

Part II

Data Engineering for ML

Chapter 4

Data Pipeline Architecture

4.1 Learning Objectives

After completing this chapter, readers will be able to:

- Design data pipeline architectures for batch, streaming, and real-time ML workloads
- Choose between ETL and ELT patterns based on ML requirements
- Implement comprehensive data validation using Great Expectations
- Build and operate feature stores for ML feature management
- Handle schema evolution and ensure backward compatibility
- Use Apache Beam for portable data processing pipelines
- Integrate Kafka for streaming data ingestion
- Version datasets and track data lineage with DVC
- Monitor data quality and detect data drift
- Design fault-tolerant, scalable data pipelines

4.2 Introduction: Data Pipelines as the Foundation

Data pipelines form the critical foundation of any production ML system. While models receive the majority of attention, the reality is that 80% of ML engineering effort goes into data preparation, quality assurance, and pipeline maintenance [Cognilytica, 2020].

A robust data pipeline must address several challenges simultaneously:

- **Scale:** Processing gigabytes to petabytes of data efficiently
- **Reliability:** Ensuring consistent, accurate data delivery
- **Latency:** Meeting timing requirements from batch to real-time
- **Quality:** Validating data and detecting anomalies
- **Versioning:** Tracking data evolution and enabling reproducibility
- **Governance:** Ensuring privacy, security, and compliance

4.3 Data Ingestion Patterns

Data ingestion represents the entry point for raw data into the ML system. The choice of ingestion pattern fundamentally shapes the architecture, latency characteristics, and operational complexity of the entire pipeline.

4.3.1 Batch Processing

Batch processing handles data in discrete chunks on a scheduled basis. This pattern suits scenarios where:

- Data naturally accumulates over time (daily logs, transactions)
- Processing large volumes efficiently is more important than latency
- Complex transformations require access to full datasets
- Cost optimization favors periodic processing over continuous operations

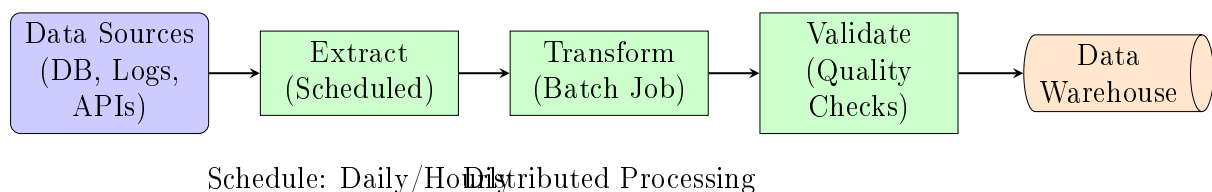


Figure 4.1: Batch processing pipeline architecture

Apache Beam Batch Pipeline:

```

1 import apache_beam as beam
2 from apache_beam.options.pipeline_options import PipelineOptions
3 from apache_beam.io import ReadFromText, WriteToBigQuery
4 import json
5
6 class ParseLogLine(beam.DoFn):
7     """Parse raw log lines into structured records"""
8     def process(self, element):
9         try:
10             record = json.loads(element)
11             yield {
12                 'user_id': record['user_id'],
13                 'timestamp': record['timestamp'],
14                 'event_type': record['event_type'],
15                 'page_url': record.get('page_url', ''),
16                 'duration_ms': int(record.get('duration_ms', 0))
17             }
18         except (json.JSONDecodeError, KeyError) as e:
19             # Log errors to side output
20             yield beam.pvalue.TaggedOutput('errors', element)
21
22 class ExtractFeatures(beam.DoFn):
23     """Extract ML features from parsed records"""
24     def process(self, element):
  
```

```

25     features = {
26         'user_id': element['user_id'],
27         'date': element['timestamp'][:10],
28         'total_events': 1,
29         'avg_duration_ms': element['duration_ms'],
30         'unique_pages': 1 if element['page_url'] else 0
31     }
32     yield (element['user_id'] + '_' + features['date'], features)
33
34 class AggregateFeatures(beam.CombineFn):
35     """Combine features per user per day"""
36     def create_accumulator(self):
37         return {
38             'total_events': 0,
39             'total_duration': 0,
40             'pages': set()
41         }
42
43     def add_input(self, accumulator, element):
44         accumulator['total_events'] += element['total_events']
45         accumulator['total_duration'] += element['avg_duration_ms']
46         if element['unique_pages']:
47             accumulator['pages'].add(element.get('page_url', ''))
48         return accumulator
49
50     def merge_accumulators(self, accumulators):
51         merged = self.create_accumulator()
52         for acc in accumulators:
53             merged['total_events'] += acc['total_events']
54             merged['total_duration'] += acc['total_duration']
55             merged['pages'].update(acc['pages'])
56         return merged
57
58     def extract_output(self, accumulator):
59         return {
60             'total_events': accumulator['total_events'],
61             'avg_duration_ms': (accumulator['total_duration'] /
62                                accumulator['total_events']
63                                if accumulator['total_events'] > 0 else 0),
64             'unique_pages': len(accumulator['pages'])
65         }
66
67 def run_pipeline(input_path, output_table):
68     """Execute the batch processing pipeline"""
69     options = PipelineOptions([
70         '--runner=DataflowRunner',
71         '--project=my-gcp-project',
72         '--region=us-central1',
73         '--temp_location=gs://my-bucket/temp',
74         '--max_num_workers=10',
75         '--autoscaling_algorithm=THROUGHPUT_BASED'
76     ])
77
78     with beam.Pipeline(options=options) as pipeline:
79         # Main processing branch
80         parsed_data = (
81             pipeline
82             | 'ReadLogs' >> ReadFromText(input_path)

```

```

83 | 'ParseLogs' >> beam.ParDo(ParseLogLine())
84 | .with_outputs('errors', main='parsed')
85 | )
86 |
87 | # Process valid records
88 | features = (
89 |     parsed_data.parsed
90 |     | 'ExtractFeatures' >> beam.ParDo(ExtractFeatures())
91 |     | 'GroupByUserDate' >> beam.GroupByKey()
92 |     | 'AggregateFeatures' >> beam.CombinePerKey(AggregateFeatures())
93 |     | 'FormatOutput' >> beam.Map(
94 |         lambda kv: {'user_date': kv[0], **kv[1]}
95 |     )
96 |     | 'WriteToBigQuery' >> WriteToBigQuery(
97 |         output_table,
98 |         schema='user_date:STRING,total_events:INTEGER,'
99 |             'avg_duration_ms:FLOAT,unique_pages:INTEGER',
100 |         create_disposition=beam.io.BigQueryDisposition.CREATE_IF_NEEDED,
101 |         write_disposition=beam.io.BigQueryDisposition.WRITE_APPEND
102 |     )
103 | )
104 |
105 | # Log parsing errors
106 | errors = (
107 |     parsed_data.errors
108 |     | 'WriteErrors' >> beam.io.WriteToText(
109 |         'gs://my-bucket/errors/batch-',
110 |         file_name_suffix='.txt'
111 |     )
112 | )
113 |
114 | if __name__ == '__main__':
115 |     run_pipeline(
116 |         input_path='gs://my-bucket/logs/*.json',
117 |         output_table='my-project:ml_features.user_daily_features'
118 |     )

```

Listing 4.1: Apache Beam batch processing pipeline

4.3.2 Streaming Processing

Streaming pipelines process data continuously as it arrives, enabling low-latency applications like fraud detection, recommendation systems, and real-time monitoring.

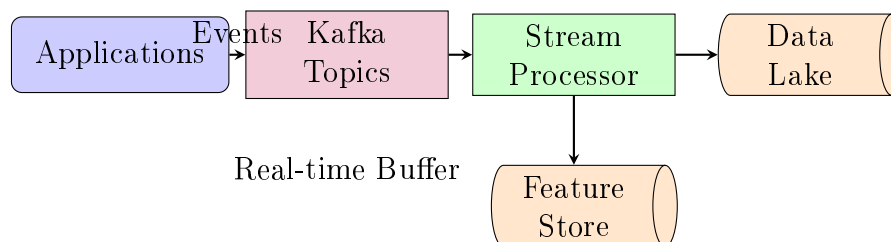


Figure 4.2: Streaming pipeline architecture with Kafka

Kafka Producer for Event Streaming:

```

1 from kafka import KafkaProducer
2 from kafka.errors import KafkaError
3 import json
4 import logging
5 from datetime import datetime
6 from typing import Dict, Any
7
8 logger = logging.getLogger(__name__)
9
10 class EventProducer:
11     """Kafka producer for streaming ML events"""
12
13     def __init__(self, bootstrap_servers: list, topic: str):
14         self.topic = topic
15         self.producer = KafkaProducer(
16             bootstrap_servers=bootstrap_servers,
17             value_serializer=lambda v: json.dumps(v).encode('utf-8'),
18             key_serializer=lambda k: k.encode('utf-8') if k else None,
19             # Performance tuning
20             compression_type='snappy',
21             batch_size=16384,
22             linger_ms=10,
23             # Reliability settings
24             acks='all',
25             retries=3,
26             max_in_flight_requests_per_connection=5,
27             # Idempotence for exactly-once semantics
28             enable_idempotence=True
29         )
30
31     def send_event(self, user_id: str, event_data: Dict[str, Any]):
32         """Send event to Kafka with error handling"""
33         try:
34             # Add metadata
35             event = {
36                 'user_id': user_id,
37                 'timestamp': datetime.utcnow().isoformat(),
38                 **event_data
39             }
40
41             # Send with callback
42             future = self.producer.send(
43                 self.topic,
44                 key=user_id,
45                 value=event,
46                 partition=None # Let Kafka partition by key
47             )
48
49             # Optionally wait for confirmation
50             record_metadata = future.get(timeout=10)
51             logger.info(
52                 f"Event sent: topic={record_metadata.topic}, "
53                 f"partition={record_metadata.partition}, "
54                 f"offset={record_metadata.offset}"
55             )
56
57         except KafkaError as e:

```

```

58         logger.error(f"Failed to send event: {e}")
59         raise
60
61     def close(self):
62         """Flush and close producer"""
63         self.producer.flush()
64         self.producer.close()
65
66     # Usage example
67     if __name__ == '__main__':
68         producer = EventProducer(
69             bootstrap_servers=['localhost:9092'],
70             topic='user_events'
71         )
72
73         try:
74             # Simulate user events
75             producer.send_event(
76                 user_id='user_12345',
77                 event_data={
78                     'event_type': 'page_view',
79                     'page_url': '/products/laptop',
80                     'duration_ms': 4532,
81                     'referrer': '/home'
82                 }
83             )
84         finally:
85             producer.close()

```

Listing 4.2: Kafka producer for streaming data ingestion

Kafka Consumer with Real-Time Feature Engineering:

```

1  from kafka import KafkaConsumer
2  from kafka.structs import TopicPartition
3  import json
4  import redis
5  from datetime import datetime, timedelta
6  from collections import defaultdict
7  import logging
8
9  logger = logging.getLogger(__name__)
10
11  class StreamingFeatureProcessor:
12      """Process streaming events and compute real-time features"""
13
14      def __init__(self, bootstrap_servers: list, topic: str,
15                  redis_host: str = 'localhost'):
16          # Kafka consumer
17          self.consumer = KafkaConsumer(
18              topic,
19              bootstrap_servers=bootstrap_servers,
20              group_id='feature_processor_group',
21              value_deserializer=lambda m: json.loads(m.decode('utf-8')),
22              key_deserializer=lambda k: k.decode('utf-8') if k else None,
23              # Start from beginning if no offset
24              auto_offset_reset='earliest',
25              # Commit offsets automatically
26              enable_auto_commit=True,

```



```

27         auto_commit_interval_ms=5000,
28         # Performance
29         max_poll_records=500,
30         session_timeout_ms=10000
31     )
32
33     # Redis for feature state
34     self.redis = redis.Redis(
35         host=redis_host,
36         port=6379,
37         decode_responses=True
38     )
39
40     # Local window state
41     self.windows = defaultdict(list)
42     self.window_size = timedelta(minutes=5)
43
44     def compute_features(self, user_id: str, event: dict) -> dict:
45         """Compute real-time features from event stream"""
46         current_time = datetime.fromisoformat(event['timestamp'])
47
48         # Get user's recent events from Redis
49         recent_key = f"user:{user_id}:events:5min"
50         recent_events = self.redis.lrange(recent_key, 0, -1)
51         recent_events = [json.loads(e) for e in recent_events]
52
53         # Add current event
54         recent_events.append(event)
55
56         # Compute features
57         features = {
58             'user_id': user_id,
59             'timestamp': event['timestamp'],
60             # Count features
61             'events_5min': len(recent_events),
62             'unique_pages_5min': len(set(
63                 e.get('page_url', '') for e in recent_events
64             )),
65             # Temporal features
66             'avg_duration_5min': sum(
67                 e.get('duration_ms', 0) for e in recent_events
68             ) / len(recent_events) if recent_events else 0,
69             # Behavioral features
70             'event_types_5min': list(set(
71                 e.get('event_type', '') for e in recent_events
72             )),
73             'is_high_activity': len(recent_events) > 10,
74         }
75
76         # Update Redis with sliding window
77         self.redis.lpush(recent_key, json.dumps(event))
78         self.redis.ltrim(recent_key, 0, 999) # Keep last 1000
79         self.redis.expire(recent_key, 300) # 5 min TTL
80
81         # Store computed features
82         feature_key = f"user:{user_id}:features:latest"
83         self.redis.setex(
84             feature_key,

```

```

85         300, # 5 min TTL
86         json.dumps(features)
87     )
88
89     return features
90
91     def process_stream(self):
92         """Main processing loop"""
93         logger.info("Starting stream processor...")
94
95         try:
96             for message in self.consumer:
97                 try:
98                     user_id = message.key
99                     event = message.value
100
101                     # Compute features
102                     features = self.compute_features(user_id, event)
103
104                     # Could publish to another topic, write to DB, etc.
105                     logger.info(f"Processed features for {user_id}: "
106                                f"{features['events_5min']} events in 5min")
107
108                 except Exception as e:
109                     logger.error(f"Error processing message: {e}")
110                     # Continue processing other messages
111
112             except KeyboardInterrupt:
113                 logger.info("Shutting down processor...")
114             finally:
115                 self.consumer.close()
116                 self.redis.close()
117
118 if __name__ == '__main__':
119     processor = StreamingFeatureProcessor(
120         bootstrap_servers=['localhost:9092'],
121         topic='user_events',
122         redis_host='localhost'
123     )
124     processor.process_stream()

```

Listing 4.3: Kafka consumer for streaming feature processing

4.3.3 Micro-Batch Processing

Micro-batching bridges batch and streaming by processing small batches at short intervals (seconds to minutes). This pattern, popularized by Apache Spark Streaming, provides a middle ground for many ML applications.

```

1 import apache_beam as beam
2 from apache_beam.options.pipeline_options import PipelineOptions
3 from apache_beam.transforms.window import FixedWindows, TimestampedValue
4 from apache_beam.io import ReadFromPubSub, WriteToBigQuery
5 import json
6 from datetime import datetime
7
8 class ParseEvent(beam.DoFn):

```

```

9      """Parse and enrich streaming events"""
10     def process(self, element, timestamp=beam.DoFn.TimestampParam):
11         try:
12             event = json.loads(element.decode('utf-8'))
13             yield TimestampedValue(event, timestamp.micros / 1000000)
14         except json.JSONDecodeError:
15             pass # Skip malformed events
16
17     class ComputeWindowFeatures(beam.DoFn):
18         """Compute features within time windows"""
19         def process(self, element, window=beam.DoFn.WindowParam):
20             user_id, events = element
21
22             features = {
23                 'user_id': user_id,
24                 'window_start': window.start.to_utc_datetime().isoformat(),
25                 'window_end': window.end.to_utc_datetime().isoformat(),
26                 'event_count': len(events),
27                 'avg_duration': sum(e.get('duration_ms', 0)
28                                     for e in events) / len(events),
29                 'unique_pages': len(set(e.get('page_url', '')
30                                         for e in events))
31             }
32             yield features
33
34     def run_streaming_pipeline(subscription: str, output_table: str):
35         """Run streaming pipeline with windowing"""
36         options = PipelineOptions([
37             '--runner=DataflowRunner',
38             '--project=my-gcp-project',
39             '--region=us-central1',
40             '--streaming',
41             '--enable_streaming_engine'
42         ])
43
44         with beam.Pipeline(options=options) as pipeline:
45             events = (
46                 pipeline
47                 | 'ReadFromPubSub' >> ReadFromPubSub(
48                     subscription=subscription
49                 )
50                 | 'ParseEvents' >> beam.ParDo(ParseEvent())
51                 | 'WindowInto' >> beam.WindowInto(
52                     FixedWindows(60) # 1-minute windows
53                 )
54                 | 'KeyByUser' >> beam.Map(
55                     lambda e: (e['user_id'], e)
56                 )
57                 | 'GroupByUser' >> beam.GroupByKey()
58                 | 'ComputeFeatures' >> beam.ParDo(ComputeWindowFeatures())
59                 | 'WriteToBigQuery' >> WriteToBigQuery(
60                     output_table,
61                     schema='user_id:STRING,window_start:TIMESTAMP,'
62                           'window_end:TIMESTAMP,event_count:INTEGER,'
63                           'avg_duration:FLOAT,unique_pages:INTEGER',
64                     write_disposition=beam.io.BigQueryDisposition.WRITE_APPEND
65                 )
66             )

```

```
67
68 if __name__ == '__main__':
69     run_streaming_pipeline(
70         subscription='projects/my-project/subscriptions/events-sub',
71         output_table='my-project:ml_features.streaming_features'
72     )
```

Listing 4.4: Apache Beam streaming pipeline with windowing

4.4 ETL vs ELT for ML Workloads

The choice between Extract-Transform-Load (ETL) and Extract-Load-Transform (ELT) significantly impacts ML pipeline architecture, performance, and maintainability.

4.4.1 ETL: Transform Before Loading

In ETL, transformations occur before data enters the warehouse or data lake. This approach suits ML when:

- Source systems have limited compute capacity
- Data must be cleaned/anonymized before storage (compliance)
- Transformation logic is stable and well-defined
- Storage costs are high relative to compute costs

Advantages for ML:

- Data arrives pre-processed and ready for training
- Reduced storage costs by filtering unnecessary data early
- Consistent data quality enforcement at ingestion
- Clear separation of concerns between ETL and ML code

Disadvantages for ML:

- Reprocessing historical data requires re-running entire ETL
- Experimentation with new features requires pipeline changes
- Hard to audit raw data or debug transformation issues
- Tight coupling between ingestion and feature engineering

4.4.2 ELT: Transform in the Warehouse

ELT loads raw data first, then transforms it using the warehouse's compute power. This modern approach aligns well with ML workflows:

- Powerful cloud warehouses (BigQuery, Snowflake, Redshift) enable complex transformations
- Raw data preservation enables reprocessing and experimentation
- SQL-based transformations are accessible to data scientists
- Separation of ingestion and feature engineering

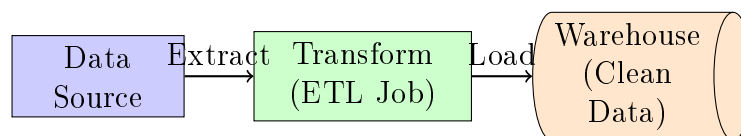
Advantages for ML:

- Rapid experimentation with new feature definitions
- Historical data reprocessing without re-ingestion
- Better auditability with raw data preservation
- Leverage warehouse optimizations (columnar storage, partitioning)

Disadvantages for ML:

- Higher storage costs for raw + transformed data
- Potential for inconsistent transformations across teams
- Query costs can escalate with large-scale transformations
- Requires robust data governance for sensitive data

ETL Pattern



ELT Pattern

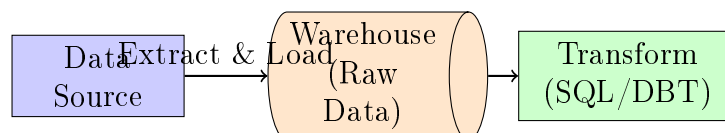


Figure 4.3: ETL vs ELT architectural comparison

4.4.3 Hybrid Approach for ML

Modern ML systems often use a hybrid approach:

1. **Light ETL:** Basic cleaning, PII anonymization, format standardization
2. **Raw Data Lake:** Store minimally processed data
3. **ELT Transformations:** Feature engineering in warehouse/lakehouse
4. **Feature Store:** Materialize frequently used features

```

1  -- models/features/user_daily_aggregates.sql
2  {{
3      config(
4          materialized='incremental',
5          unique_key='user_date',
6          partition_by={
7              "field": "date",
8              "data_type": "date",
9              "granularity": "day"
10         }
11     )
12 }}
13
14 WITH raw_events AS (
15     SELECT
16         user_id,
17         DATE(timestamp) as date,
18         event_type,
19         page_url,
20         duration_ms,
21         timestamp
22 FROM {{ source('raw', 'user_events') }}
23 {% if is_incremental() %}
24     WHERE DATE(timestamp) > (SELECT MAX(date) FROM {{ this }})
25 {% endif %}
26 ),
27
28 daily_aggregates AS (
29     SELECT
30         user_id,
31         date,
32         COUNT(*) as total_events,
33         COUNT(DISTINCT event_type) as unique_event_types,
34         COUNT(DISTINCT page_url) as unique_pages_visited,
35         AVG(duration_ms) as avg_duration_ms,
36         MAX(duration_ms) as max_duration_ms,
37         SUM(CASE WHEN event_type = 'purchase' THEN 1 ELSE 0 END)
38         as purchase_count,
39         SUM(CASE WHEN event_type = 'add_to_cart' THEN 1 ELSE 0 END)
40         as cart_additions
41 FROM raw_events
42 GROUP BY user_id, date
43 ),
44
45 windowed_features AS (
46     SELECT

```

```

47     user_id,
48     date,
49     total_events,
50     unique_event_types,
51     unique_pages_visited,
52     avg_duration_ms,
53     max_duration_ms,
54     purchase_count,
55     cart_additions,
56     -- 7-day rolling averages
57     AVG(total_events) OVER w7 as avg_events_7d,
58     AVG(avg_duration_ms) OVER w7 as avg_duration_7d,
59     -- 30-day rolling averages
60     AVG(total_events) OVER w30 as avg_events_30d,
61     SUM(purchase_count) OVER w30 as total_purchases_30d,
62     -- Day of week patterns
63     EXTRACT(DAYOFWEEK FROM date) as day_of_week,
64     AVG(total_events) OVER (
65         PARTITION BY user_id, EXTRACT(DAYOFWEEK FROM date)
66         ORDER BY date
67         ROWS BETWEEN 4 PRECEDING AND CURRENT ROW
68     ) as avg_events_same_dow
69 FROM daily_aggregates
70 WINDOW
71     w7 AS (PARTITION BY user_id ORDER BY date
72             ROWS BETWEEN 6 PRECEDING AND CURRENT ROW),
73     w30 AS (PARTITION BY user_id ORDER BY date
74             ROWS BETWEEN 29 PRECEDING AND CURRENT ROW)
75 )
76
77 SELECT
78     CONCAT(user_id, '_', CAST(date AS STRING)) as user_date,
79     *,
80     -- Feature metadata
81     CURRENT_TIMESTAMP() as feature_timestamp,
82     '{{ run_started_at }}' as pipeline_run_id
83 FROM windowed_features

```

Listing 4.5: DBT model for ELT feature engineering

4.5 Data Validation and Quality Checks

Data quality directly impacts model performance. Systematic validation catches issues before they poison training data or production predictions.

4.5.1 Validation Layers

Implement validation at multiple pipeline stages:

1. **Schema Validation:** Ensure expected columns, types, and constraints
2. **Statistical Validation:** Check distributions, ranges, and statistical properties
3. **Referential Integrity:** Verify foreign keys and cross-table consistency

4. **Business Logic:** Validate domain-specific rules and constraints

5. **Data Freshness:** Ensure timely data arrival and detect stale data

4.5.2 Great Expectations Framework

Great Expectations provides a comprehensive framework for data validation with version control, documentation, and failure handling.

```

1 import great_expectations as ge
2 from great_expectations.data_context import DataContext
3 from great_expectations.checkpoint import SimpleCheckpoint
4 from great_expectations.core.batch import RuntimeBatchRequest
5 import pandas as pd
6 import logging
7
8 logger = logging.getLogger(__name__)
9
10 class DataValidator:
11     """Comprehensive data validation for ML pipelines"""
12
13     def __init__(self, ge_context_root: str):
14         self.context = DataContext(ge_context_root)
15
16     def create_expectation_suite(self, suite_name: str, df: pd.DataFrame):
17         """Create and configure expectation suite"""
18
19         # Create or get suite
20         suite = self.context.create_expectation_suite(
21             expectation_suite_name=suite_name,
22             overwrite_existing=True
23         )
24
25         # Create validator
26         validator = ge.from_pandas(df, expectation_suite=suite)
27
28         # Schema expectations
29         validator.expect_table_columns_to_match_ordered_list([
30             'user_id', 'timestamp', 'event_type', 'value',
31             'page_url', 'session_id'
32         ])
33
34         # Completeness expectations
35         validator.expect_column_values_to_not_be_null('user_id')
36         validator.expect_column_values_to_not_be_null('timestamp')
37         validator.expect_column_values_to_not_be_null('event_type')
38
39         # Type expectations
40         validator.expect_column_values_to_be_of_type('user_id', 'str')
41         validator.expect_column_values_to_be_of_type('value', 'float')
42
43         # Range expectations
44         validator.expect_column_values_to_be_between(
45             'value',
46             min_value=0,
47             max_value=10000,
48             mostly=0.95 # Allow 5% outliers
49         )

```



```

50
51     # Categorical expectations
52     validator.expect_column_values_to_be_in_set(
53         'event_type',
54         value_set=['page_view', 'click', 'purchase',
55                   'add_to_cart', 'search']
56     )
57
58     # Statistical expectations
59     validator.expect_column_mean_to_be_between(
60         'value',
61         min_value=50,
62         max_value=200
63     )
64
65     validator.expect_column_stdev_to_be_between(
66         'value',
67         min_value=10,
68         max_value=100
69     )
70
71     # Pattern matching
72     validator.expect_column_values_to_match_regex(
73         'user_id',
74         regex=r'^user_\d+$'
75     )
76
77     validator.expect_column_values_to_match_strftime_format(
78         'timestamp',
79         strftime_format='%Y-%m-%d %H:%M:%S'
80     )
81
82     # Uniqueness expectations
83     validator.expect_compound_columns_to_be_unique(
84         ['user_id', 'timestamp', 'event_type']
85     )
86
87     # Custom expectations
88     validator.expect_column_pair_values_A_to_be_greater_than_B(
89         column_A='timestamp',
90         column_B='session_start',
91         or_equal=True
92     )
93
94     # Save suite
95     validator.save_expectation_suite(discard_failed_expectations=False)
96
97     return suite_name
98
99 def validate_batch(self, suite_name: str, df: pd.DataFrame,
100                   fail_on_error: bool = True) -> dict:
101     """Validate a data batch against expectation suite"""
102
103     # Create runtime batch request
104     batch_request = RuntimeBatchRequest(
105         datasource_name="pandas_datasource",
106         data_connector_name="runtime_data_connector",
107         data_asset_name="validation_data",

```

```

108         runtime_parameters={"batch_data": df},
109         batch_identifiers={"default_identifier_name": "default_identifier"}
110     )
111
112     # Create checkpoint
113     checkpoint_config = {
114         "name": "validation_checkpoint",
115         "config_version": 1.0,
116         "class_name": "SimpleCheckpoint",
117         "validations": [
118             {
119                 "batch_request": batch_request,
120                 "expectation_suite_name": suite_name
121             }
122         ]
123     }
124
125     # Run validation
126     results = self.context.run_checkpoint(
127         checkpoint_name="validation_checkpoint",
128         checkpoint_config=checkpoint_config
129     )
130
131     # Process results
132     validation_result = results.list_validation_results()[0]
133     success = validation_result.success
134
135     # Log results
136     if success:
137         logger.info(f"Validation passed for suite: {suite_name}")
138     else:
139         failed_expectations = [
140             exp for exp in validation_result.results
141             if not exp.success
142         ]
143         logger.error(
144             f"Validation failed: {len(failed_expectations)} "
145             f"expectations failed"
146         )
147         for exp in failed_expectations:
148             logger.error(f" - {exp.expectation_config.expectation_type}")
149
150     if not success and fail_on_error:
151         raise ValueError(f"Data validation failed for suite: {suite_name}")
152
153     return {
154         'success': success,
155         'statistics': validation_result.statistics,
156         'results': validation_result.results
157     }
158
159     def monitor_data_drift(self, suite_name: str,
160                           baseline_df: pd.DataFrame,
161                           current_df: pd.DataFrame) -> dict:
162         """Detect data drift between baseline and current data"""
163
164         drift_metrics = {}
165

```

```

166     for column in baseline_df.select_dtypes(include=['number']).columns:
167         baseline_mean = baseline_df[column].mean()
168         baseline_std = baseline_df[column].std()
169         current_mean = current_df[column].mean()
170         current_std = current_df[column].std()
171
172         # Compute drift metrics
173         mean_drift = abs(current_mean - baseline_mean) / baseline_std
174         std_drift = abs(current_std - baseline_std) / baseline_std
175
176         drift_metrics[column] = {
177             'mean_drift_zscore': mean_drift,
178             'std_drift_ratio': std_drift,
179             'drift_detected': mean_drift > 3.0 or std_drift > 0.5
180         }
181
182     return drift_metrics
183
184 # Usage example
185 if __name__ == '__main__':
186     # Sample data
187     df = pd.DataFrame({
188         'user_id': [f'user_{i}' for i in range(1000)],
189         'timestamp': pd.date_range('2024-01-01', periods=1000, freq='1h'),
190         'event_type': np.random.choice(
191             ['page_view', 'click', 'purchase'], 1000
192         ),
193         'value': np.random.normal(100, 30, 1000),
194         'page_url': [f'/page/{i%10}' for i in range(1000)],
195         'session_id': [f'session_{i//10}' for i in range(1000)]
196     })
197
198     # Initialize validator
199     validator = DataValidator(ge_context_root='./great_expectations')
200
201     # Create expectation suite
202     suite_name = validator.create_expectation_suite('user_events_suite', df)
203
204     # Validate new batch
205     results = validator.validate_batch(suite_name, df, fail_on_error=True)
206     print(f"Validation results: {results['success']}")

```

Listing 4.6: Comprehensive data validation with Great Expectations

4.5.3 Custom Validation Rules

Beyond standard checks, implement domain-specific validation:

```

1 from typing import List, Dict, Any
2 import pandas as pd
3 import numpy as np
4 from datetime import datetime, timedelta
5
6 class MLDataValidator:
7     """Custom validation rules for ML training data"""
8
9     def validate_label_distribution(self, df: pd.DataFrame,

```

```

10         label_col: str,
11         min_class_ratio: float = 0.01) -> bool:
12     """Ensure no class is too rare for training"""
13     class_counts = df[label_col].value_counts(normalize=True)
14     min_ratio = class_counts.min()
15
16     if min_ratio < min_class_ratio:
17         raise ValueError(
18             f"Class imbalance too severe: min ratio {min_ratio:.4f} "
19             f"< {min_class_ratio}"
20         )
21     return True
22
23 def validate_feature_correlation(self, df: pd.DataFrame,
24                                 max_correlation: float = 0.95) -> List[tuple]:
25     """Detect highly correlated features"""
26     numeric_df = df.select_dtypes(include=['number'])
27     corr_matrix = numeric_df.corr().abs()
28
29     # Find pairs with high correlation
30     high_corr_pairs = []
31     for i in range(len(corr_matrix.columns)):
32         for j in range(i+1, len(corr_matrix.columns)):
33             if corr_matrix.iloc[i, j] > max_correlation:
34                 high_corr_pairs.append((
35                     corr_matrix.columns[i],
36                     corr_matrix.columns[j],
37                     corr_matrix.iloc[i, j]
38                 ))
39
40     if high_corr_pairs:
41         print(f"Warning: {len(high_corr_pairs)} highly correlated "
42               f"feature pairs found")
43         for f1, f2, corr in high_corr_pairs:
44             print(f" {f1} <-> {f2}: {corr:.3f}")
45
46     return high_corr_pairs
47
48 def validate_temporal_leakage(self, df: pd.DataFrame,
49                               timestamp_col: str,
50                               feature_cols: List[str],
51                               target_col: str) -> bool:
52     """Check for temporal leakage in time-series data"""
53     df = df.sort_values(timestamp_col)
54
55     # Check if features from future predict past targets
56     for feature in feature_cols:
57         # Compute correlation with lagged target
58         df['target_lag1'] = df[target_col].shift(1)
59         future_corr = df[feature].corr(df['target_lag1'])
60
61         if abs(future_corr) > 0.5:
62             raise ValueError(
63                 f"Potential temporal leakage: {feature} correlates "
64                 f"{future_corr:.3f} with future target"
65             )
66
67     return True

```

```

68
69 def validate_data_freshness(self, df: pd.DataFrame,
70                             timestamp_col: str,
71                             max_age_days: int = 7) -> bool:
72     """Ensure data is recent enough"""
73     latest_timestamp = pd.to_datetime(df[timestamp_col]).max()
74     age = datetime.now() - latest_timestamp
75
76     if age > timedelta(days=max_age_days):
77         raise ValueError(
78             f"Data is stale: latest record is {age.days} days old"
79         )
80
81     return True
82
83 def validate_feature_stability(self, df: pd.DataFrame,
84                                timestamp_col: str,
85                                feature_cols: List[str],
86                                window_days: int = 7) -> Dict[str, float]:
87     """Check if feature distributions are stable over time"""
88     df = df.copy()
89     df['date'] = pd.to_datetime(df[timestamp_col]).dt.date
90
91     instability_scores = {}
92
93     for feature in feature_cols:
94         if df[feature].dtype in ['float64', 'int64']:
95             # Compute rolling statistics
96             daily_means = df.groupby('date')[feature].mean()
97             rolling_std = daily_means.rolling(window=window_days).std()
98
99             # Instability score: coefficient of variation
100            instability = (rolling_std / daily_means.abs()).mean()
101            instability_scores[feature] = instability
102
103            if instability > 0.5:
104                print(f"Warning: Feature '{feature}' is unstable "
105                      f"(CV={instability:.3f})")
106
107    return instability_scores

```

Listing 4.7: Custom validation rules for ML data

4.6 Feature Stores and Data Versioning

Feature stores solve critical challenges in ML production: feature reusability, consistency between training and serving, and efficient feature computation.

4.6.1 Feature Store Architecture

4.6.2 Feature Store Implementation

```

1 from feast import Entity, Feature, FeatureView, FileSource, ValueType
2 from feast.data_source import DataSource
3 from feast import FeatureStore

```

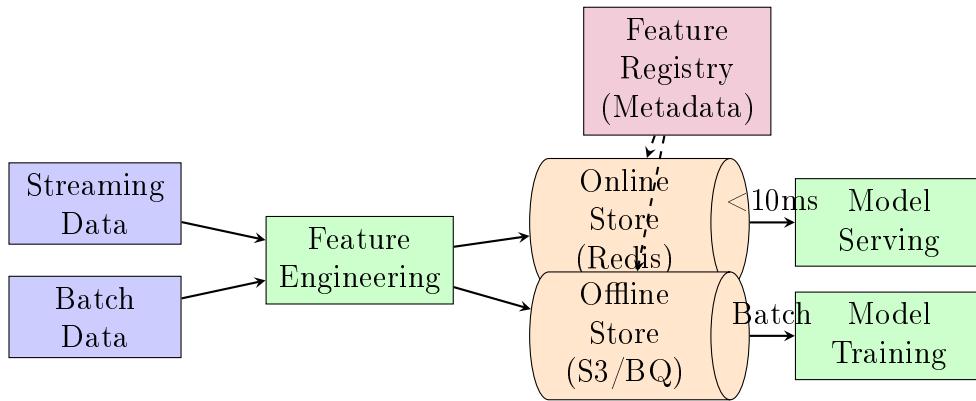


Figure 4.4: Feature store architecture with online and offline stores

```

4 from datetime import timedelta
5 import pandas as pd
6 from typing import List
7
8 # Define entities
9 user = Entity(
10     name="user_id",
11     value_type=ValueTypes.STRING,
12     description="User identifier"
13 )
14
15 # Define data source (offline)
16 user_features_source = FileSource(
17     path="s3://ml-data/features/user_features.parquet",
18     event_timestamp_column="timestamp",
19     created_timestamp_column="created_at",
20 )
21
22 # Define feature view
23 user_features_view = FeatureView(
24     name="user_features",
25     entities=["user_id"],
26     ttl=timedelta(days=7),
27     features=[
28         Feature(name="total_events_7d", dtype=ValueTypes.INT64),
29         Feature(name="avg_duration_7d", dtype=ValueTypes.DOUBLE),
30         Feature(name="unique_pages_7d", dtype=ValueTypes.INT64),
31         Feature(name="purchase_count_30d", dtype=ValueTypes.INT64),
32         Feature(name="avg_order_value", dtype=ValueTypes.DOUBLE),
33         Feature(name="user_segment", dtype=ValueTypes.STRING),
34     ],
35     online=True,
36     source=user_features_source,
37     tags={"team": "ml", "use_case": "recommendations"},
38 )
39
40 class FeatureStoreManager:
41     """Manage feature store operations"""
42
43     def __init__(self, repo_path: str):
44         self.store = FeatureStore(repo_path=repo_path)
45

```

```

46 def materialize_features(self, start_date: str, end_date: str):
47     """Materialize features from offline to online store"""
48     self.store.materialize(
49         start_date=pd.to_datetime(start_date),
50         end_date=pd.to_datetime(end_date)
51     )
52     print(f"Features materialized from {start_date} to {end_date}")
53
54 def get_online_features(self, user_ids: List[str]) -> pd.DataFrame:
55     """Retrieve features for online inference (low latency)"""
56     entity_rows = [{"user_id": user_id} for user_id in user_ids]
57
58     feature_vector = self.store.get_online_features(
59         features=[
60             "user_features:total_events_7d",
61             "user_features:avg_duration_7d",
62             "user_features:unique_pages_7d",
63             "user_features:purchase_count_30d",
64             "user_features:avg_order_value",
65             "user_features:user_segment",
66         ],
67         entity_rows=entity_rows,
68     ).to_df()
69
70     return feature_vector
71
72 def get_historical_features(self, entity_df: pd.DataFrame) -> pd.DataFrame:
73     """Retrieve point-in-time correct features for training"""
74     # entity_df must have entity key and event_timestamp columns
75     training_data = self.store.get_historical_features(
76         entity_df=entity_df,
77         features=[
78             "user_features:total_events_7d",
79             "user_features:avg_duration_7d",
80             "user_features:unique_pages_7d",
81             "user_features:purchase_count_30d",
82             "user_features:avg_order_value",
83             "user_features:user_segment",
84         ],
85     ).to_df()
86
87     return training_data
88
89 def compute_and_push_features(self, df: pd.DataFrame):
90     """Compute features and push to online store"""
91     # Transform dataframe to match feature view schema
92     feature_df = df[[
93         'user_id', 'timestamp', 'total_events_7d',
94         'avg_duration_7d', 'unique_pages_7d',
95         'purchase_count_30d', 'avg_order_value', 'user_segment'
96     ]]
97
98     # Push to online store
99     self.store.push(
100         push_source_name="user_features_push_source",
101         df=feature_df,
102         to="online"
103     )

```

```

104 # Usage example
105 if __name__ == '__main__':
106     # Initialize feature store
107     fs = FeatureStoreManager(repo_path="./feature_repo")
108
109     # Materialize historical features to online store
110     fs.materialize_features(
111         start_date="2024-01-01",
112         end_date="2024-01-31"
113     )
114
115     # Get features for online serving
116     online_features = fs.get_online_features(
117         user_ids=["user_12345", "user_67890"]
118     )
119     print("Online features:", online_features)
120
121     # Get historical features for training
122     entity_df = pd.DataFrame({
123         'user_id': ['user_12345', 'user_67890'],
124         'event_timestamp': pd.to_datetime(['2024-01-15', '2024-01-16'])
125     })
126     historical_features = fs.get_historical_features(entity_df)
127     print("Historical features:", historical_features)
128

```

Listing 4.8: Feature store implementation with Feast

4.6.3 Data Versioning with DVC

Data Version Control (DVC) tracks datasets, models, and pipeline artifacts alongside code in Git.

```

1 # Initialize DVC in your repository
2 dvc init
3
4 # Configure remote storage (S3, GCS, Azure, etc.)
5 dvc remote add -d myremote s3://my-bucket/dvc-storage
6
7 # Add dataset to DVC tracking
8 dvc add data/raw/user_events.parquet
9
10 # Git tracks the .dvc file, not the actual data
11 git add data/raw/user_events.parquet.dvc .gitignore
12 git commit -m "Add raw user events dataset"
13
14 # Push data to remote storage
15 dvc push
16
17 # Create a pipeline stage
18 dvc run -n preprocess \
19     -d data/raw/user_events.parquet \
20     -o data/processed/features.parquet \
21     python scripts/preprocess.py
22
23 # Run entire pipeline
24 dvc repro

```



```
25
26 # Pull specific data version
27 git checkout feature-branch
28 dvc pull
```

Listing 4.9: Setting up DVC for data versioning

```
1 # dvc.yaml - Define data pipeline stages
2 stages:
3   extract:
4     cmd: python scripts/extract_data.py
5     deps:
6       - scripts/extract_data.py
7       - config/extract_config.yaml
8     outs:
9       - data/raw/events.parquet
10    params:
11      - extract.date_range
12      - extract.sources
13
14   validate:
15     cmd: python scripts/validate_data.py
16     deps:
17       - scripts/validate_data.py
18       - data/raw/events.parquet
19     outs:
20       - data/validated/events.parquet
21       - reports/validation_report.json
22     metrics:
23       - reports/validation_metrics.json:
24         cache: false
25
26   feature_engineering:
27     cmd: python scripts/engineer_features.py
28     deps:
29       - scripts/engineer_features.py
30       - data/validated/events.parquet
31     outs:
32       - data/features/train_features.parquet
33       - data/features/test_features.parquet
34     params:
35       - features.window_sizes
36       - features.aggregations
37
38   train:
39     cmd: python scripts/train_model.py
40     deps:
41       - scripts/train_model.py
42       - data/features/train_features.parquet
43     outs:
44       - models/model.pkl
45     params:
46       - train.model_type
47       - train.hyperparameters
48     metrics:
49       - reports/train_metrics.json:
50         cache: false
51     plots:
```

```

52 - reports/plots/confusion_matrix.json:
53     template: confusion
54     x: predicted
55     y: actual

```

Listing 4.10: DVC pipeline configuration (dvc.yaml)

4.7 Schema Evolution and Compatibility

ML systems must handle evolving data schemas without breaking existing pipelines or models.

4.7.1 Schema Evolution Strategies

1. **Forward Compatibility:** New schema can read old data
2. **Backward Compatibility:** Old schema can read new data
3. **Full Compatibility:** Both forward and backward compatible

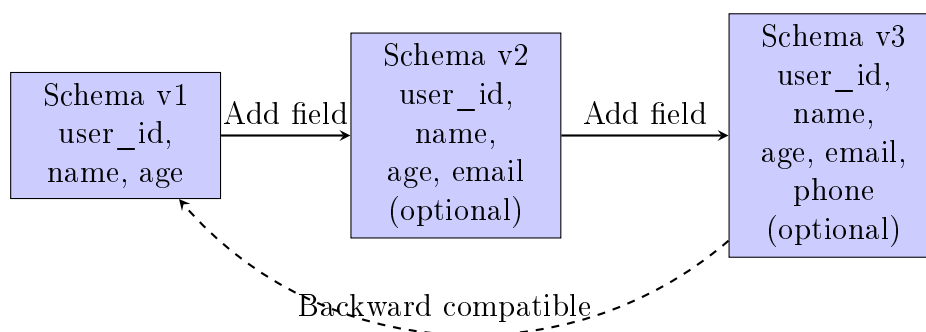


Figure 4.5: Schema evolution with backward compatibility

4.7.2 Apache Avro for Schema Management

Avro provides schema evolution with strong compatibility guarantees:

```

1 from avro.datafile import DataFileWriter, DataFileReader
2 from avro.io import DatumWriter, DatumReader
3 import avro.schema
4 import json
5
6 # Original schema v1
7 schema_v1_json = '''
8 {
9     "namespace": "ml.features",
10    "type": "record",
11    "name": "UserFeatures",
12    "fields": [
13        {"name": "user_id", "type": "string"},
14        {"name": "total_events", "type": "int"},

```

```

15     {"name": "avg_duration", "type": "double"}
16 ]
17 }
18 '''
19
20 # Evolved schema v2 (backward compatible)
21 schema_v2_json = '''
22 {
23     "namespace": "ml.features",
24     "type": "record",
25     "name": "UserFeatures",
26     "fields": [
27         {"name": "user_id", "type": "string"},
28         {"name": "total_events", "type": "int"},
29         {"name": "avg_duration", "type": "double"},
30         {"name": "user_segment", "type": ["null", "string"], "default": null},
31         {"name": "premium_user", "type": "boolean", "default": false}
32     ]
33 }
34 '''
35
36 class AvroSchemaManager:
37     """Manage schema evolution for ML features"""
38
39     def __init__(self, schema_registry_url: str = None):
40         self.schema_registry_url = schema_registry_url
41         self.schemas = {}
42
43     def register_schema(self, name: str, schema_json: str, version: int):
44         """Register a schema version"""
45         schema = avro.schema.parse(schema_json)
46         self.schemas[(name, version)] = schema
47         return schema
48
49     def write_features(self, filename: str, features: list, schema_version: int):
50         """Write features with specific schema version"""
51         schema = self.schemas[("UserFeatures", schema_version)]
52
53         with open(filename, 'wb') as f:
54             writer = DataFileWriter(f, DatumWriter(), schema)
55             for feature in features:
56                 writer.append(feature)
57             writer.close()
58
59     def read_features(self, filename: str, reader_schema_version: int = None):
60         """Read features, optionally with schema evolution"""
61         features = []
62
63         with open(filename, 'rb') as f:
64             reader = DataFileReader(f, DatumReader())
65
66             # Get writer's schema (embedded in file)
67             writer_schema = reader.datum_reader.writer_schema
68
69             # Optionally provide reader schema for evolution
70             if reader_schema_version:
71                 reader_schema = self.schemas[("UserFeatures", reader_schema_version)]
72                 reader.datum_reader.reader_schema = reader_schema

```

```

73         for feature in reader:
74             features.append(feature)
75
76     reader.close()
77
78     return features
79
80
81     def validate_compatibility(self, old_schema_json: str,
82                               new_schema_json: str) -> dict:
83         """Check schema compatibility"""
84         old_schema = avro.schema.parse(old_schema_json)
85         new_schema = avro.schema.parse(new_schema_json)
86
87         compatibility = {
88             'backward_compatible': self._check_backward_compatible(
89                 old_schema, new_schema
90             ),
91             'forward_compatible': self._check_forward_compatible(
92                 old_schema, new_schema
93             )
94         }
95
96         return compatibility
97
98     def _check_backward_compatible(self, old_schema, new_schema) -> bool:
99         """Check if new schema can read data written with old schema"""
100         # New schema must have defaults for any new fields
101         old_fields = {f.name for f in old_schema.fields}
102         new_fields_dict = {f.name: f for f in new_schema.fields}
103
104         for field_name in new_fields_dict:
105             if field_name not in old_fields:
106                 # New field must have default
107                 if not new_fields_dict[field_name].has_default:
108                     return False
109
110         return True
111
112     def _check_forward_compatible(self, old_schema, new_schema) -> bool:
113         """Check if old schema can read data written with new schema"""
114         # Old schema must be able to ignore new fields
115         new_fields = {f.name for f in new_schema.fields}
116         old_fields_dict = {f.name: f for f in old_schema.fields}
117
118         for field_name in old_fields_dict:
119             if field_name not in new_fields:
120                 # Field was removed - not forward compatible
121                 return False
122
123         return True
124
125     # Usage example
126     if __name__ == '__main__':
127         manager = AvroSchemaManager()
128
129         # Register schemas
130         manager.register_schema("UserFeatures", schema_v1_json, version=1)

```

```

131 manager.register_schema("UserFeatures", schema_v2_json, version=2)
132
133 # Write data with v1 schema
134 features_v1 = [
135     {"user_id": "user_1", "total_events": 100, "avg_duration": 45.2},
136     {"user_id": "user_2", "total_events": 200, "avg_duration": 60.1}
137 ]
138 manager.write_features("features_v1.avro", features_v1, schema_version=1)
139
140 # Read v1 data with v2 schema (backward compatibility)
141 features = manager.read_features("features_v1.avro", reader_schema_version=2)
142 print("Features with evolved schema:", features)
143 # New fields have default values: user_segment=null, premium_user=false
144
145 # Validate compatibility
146 compat = manager.validate_compatibility(schema_v1_json, schema_v2_json)
147 print(f"Backward compatible: {compat['backward_compatible']}")
148 print(f"Forward compatible: {compat['forward_compatible']}")

```

Listing 4.11: Schema evolution with Apache Avro

4.7.3 Handling Breaking Schema Changes

When breaking changes are unavoidable:

```

1 from typing import Any, Dict
2 import pandas as pd
3
4 class SchemaAdapter:
5     """Adapt between schema versions for breaking changes"""
6
7     def __init__(self):
8         self.adapters = {
9             (1, 2): self._adapt_v1_to_v2,
10            (2, 3): self._adapt_v2_to_v3,
11        }
12
13    def adapt(self, data: pd.DataFrame,
14             from_version: int,
15             to_version: int) -> pd.DataFrame:
16        """Adapt data from one schema version to another"""
17
18        if from_version == to_version:
19            return data
20
21        # Apply series of adaptations
22        current_version = from_version
23        current_data = data.copy()
24
25        while current_version < to_version:
26            adapter_key = (current_version, current_version + 1)
27            if adapter_key not in self.adapters:
28                raise ValueError(
29                    f"No adapter from v{current_version} to "
30                    f"v{current_version + 1}"
31                )
32

```

```

33         current_data = self.adapters[adapter_key](current_data)
34         current_version += 1
35
36     return current_data
37
38     def _adapt_v1_to_v2(self, df: pd.DataFrame) -> pd.DataFrame:
39         """Adapt schema from v1 to v2"""
40         # v1: user_age (int) -> v2: user_age_group (str)
41         df['user_age_group'] = pd.cut(
42             df['user_age'],
43             bins=[0, 18, 35, 50, 100],
44             labels=['<18', '18-35', '35-50', '50+']
45         )
46         df = df.drop(columns=['user_age'])
47
48         # Add new required field with computed default
49         df['signup_year'] = pd.to_datetime(df['signup_date']).dt.year
50
51     return df
52
53     def _adapt_v2_to_v3(self, df: pd.DataFrame) -> pd.DataFrame:
54         """Adapt schema from v2 to v3"""
55         # v2: country (str) -> v3: country_code (str, ISO 3166-1)
56         country_mapping = {
57             'USA': 'US',
58             'United States': 'US',
59             'UK': 'GB',
60             'United Kingdom': 'GB',
61             # ... more mappings
62         }
63         df['country_code'] = df['country'].map(country_mapping)
64         df = df.drop(columns=['country'])
65
66     return df

```

Listing 4.12: Managing breaking schema changes

4.8 Building a Complete Data Pipeline

Let's integrate all concepts into a production-ready data pipeline:

```

1  from dataclasses import dataclass
2  from typing import List, Dict, Any
3  import pandas as pd
4  import apache_beam as beam
5  from apache_beam.options.pipeline_options import PipelineOptions
6  import great_expectations as ge
7  from feast import FeatureStore
8  import logging
9  from datetime import datetime
10
11  logger = logging.getLogger(__name__)
12
13  @dataclass
14  class PipelineConfig:
15      """Configuration for data pipeline"""
16      input_source: str

```

```

17     output_sink: str
18     feature_store_repo: str
19     validation_suite: str
20     schema_version: int
21     enable_drift_detection: bool = True
22
23 class DataPipeline:
24     """Complete production data pipeline for ML"""
25
26     def __init__(self, config: PipelineConfig):
27         self.config = config
28         self.feature_store = FeatureStore(
29             repo_path=config.feature_store_repo
30         )
31         self.validator = DataValidator('./great_expectations')
32
33     def run(self):
34         """Execute end-to-end pipeline"""
35         logger.info("Starting data pipeline...")
36
37         # 1. Extract
38         raw_data = self.extract_data()
39         logger.info(f"Extracted {len(raw_data)} records")
40
41         # 2. Validate raw data
42         self.validate_data(raw_data, stage='raw')
43
44         # 3. Transform
45         transformed_data = self.transform_data(raw_data)
46         logger.info(f"Transformed {len(transformed_data)} records")
47
48         # 4. Validate transformed data
49         self.validate_data(transformed_data, stage='transformed')
50
51         # 5. Engineer features
52         features = self.engineer_features(transformed_data)
53         logger.info(f"Engineered {len(features.columns)} features")
54
55         # 6. Validate features
56         self.validate_data(features, stage='features')
57
58         # 7. Detect drift (if enabled)
59         if self.config.enable_drift_detection:
60             self.detect_drift(features)
61
62         # 8. Write to feature store
63         self.write_to_feature_store(features)
64
65         # 9. Write to data lake
66         self.write_to_lake(features)
67
68         logger.info("Pipeline completed successfully")
69
70         return features
71
72     def extract_data(self) -> pd.DataFrame:
73         """Extract data from source"""
74         # Implementation depends on source (DB, API, files, etc.)

```

```

75     df = pd.read_parquet(self.config.input_source)
76     return df
77
78     def validate_data(self, df: pd.DataFrame, stage: str):
79         """Validate data at pipeline stage"""
80         suite_name = f"{self.config.validation_suite}_{stage}"
81
82         try:
83             results = self.validator.validate_batch(
84                 suite_name=suite_name,
85                 df=df,
86                 fail_on_error=True
87             )
88             logger.info(f"Validation passed at stage: {stage}")
89         except ValueError as e:
90             logger.error(f"Validation failed at stage {stage}: {e}")
91             raise
92
93     def transform_data(self, df: pd.DataFrame) -> pd.DataFrame:
94         """Clean and transform raw data"""
95         df = df.copy()
96
97         # Remove duplicates
98         df = df.drop_duplicates(subset=['user_id', 'timestamp'])
99
100        # Handle missing values
101        df['page_url'] = df['page_url'].fillna('')
102        df['duration_ms'] = df['duration_ms'].fillna(0)
103
104        # Type conversions
105        df['timestamp'] = pd.to_datetime(df['timestamp'])
106        df['user_id'] = df['user_id'].astype(str)
107
108        # Filter invalid records
109        df = df[df['duration_ms'] >= 0]
110        df = df[df['timestamp'] <= datetime.now()]
111
112        return df
113
114    def engineer_features(self, df: pd.DataFrame) -> pd.DataFrame:
115        """Engineer ML features"""
116        df = df.copy()
117        df = df.sort_values(['user_id', 'timestamp'])
118
119        # Aggregation features
120        user_features = df.groupby('user_id').agg({
121            'event_type': 'count',
122            'duration_ms': ['mean', 'std', 'max'],
123            'page_url': 'nunique',
124            'timestamp': ['min', 'max']
125        }).reset_index()
126
127        user_features.columns = [
128            'user_id', 'total_events', 'avg_duration', 'std_duration',
129            'max_duration', 'unique_pages', 'first_event', 'last_event'
130        ]
131
132        # Temporal features

```



```

133     user_features['days_active'] = (
134         user_features['last_event'] - user_features['first_event']
135     ).dt.days + 1
136
137     user_features['events_per_day'] = (
138         user_features['total_events'] / user_features['days_active']
139     )
140
141     # Behavioral features
142     user_features['avg_session_duration'] = (
143         user_features['avg_duration'] / 1000 # Convert to seconds
144     )
145
146     user_features['engagement_score'] = (
147         user_features['events_per_day'] *
148         user_features['unique_pages']
149     )
150
151     # Add timestamp for feature store
152     user_features['feature_timestamp'] = datetime.now()
153
154     return user_features
155
156 def detect_drift(self, df: pd.DataFrame):
157     """Detect data drift vs baseline"""
158     # Load baseline data
159     baseline_df = pd.read_parquet('s3://bucket/baseline_features.parquet')
160
161     # Compute drift metrics
162     drift_metrics = self.validator.monitor_data_drift(
163         suite_name=self.config.validation_suite,
164         baseline_df=baseline_df,
165         current_df=df
166     )
167
168     # Log drift warnings
169     drifted_features = [
170         feat for feat, metrics in drift_metrics.items()
171         if metrics['drift_detected']
172     ]
173
174     if drifted_features:
175         logger.warning(
176             f>Data drift detected in features: {drifted_features}<
177         )
178     else:
179         logger.info("No significant data drift detected")
180
181 def write_to_feature_store(self, df: pd.DataFrame):
182     """Write features to online feature store"""
183     self.feature_store.push(
184         push_source_name="user_features_push",
185         df=df,
186         to="online"
187     )
188     logger.info("Features written to feature store")
189
190 def write_to_lake(self, df: pd.DataFrame):

```

```

191     """Write features to data lake for training"""
192     output_path = f"{self.config.output_sink}/features_{datetime.now().strftime('%
Y%m%d')}.parquet"
193     df.to_parquet(output_path, index=False)
194     logger.info(f"Features written to data lake: {output_path}")
195
196     # Pipeline orchestration with Airflow
197     from airflow import DAG
198     from airflow.operators.python import PythonOperator
199     from datetime import timedelta
200
201     default_args = {
202         'owner': 'ml_team',
203         'depends_on_past': False,
204         'email_on_failure': True,
205         'email': ['ml-alerts@company.com'],
206         'retries': 3,
207         'retry_delay': timedelta(minutes=5),
208     }
209
210     def run_data_pipeline(**context):
211         """Airflow task to run data pipeline"""
212         config = PipelineConfig(
213             input_source='s3://bucket/raw/events/*.parquet',
214             output_sink='s3://bucket/features/',
215             feature_store_repo='./feature_repo',
216             validation_suite='user_events',
217             schema_version=2,
218             enable_drift_detection=True
219         )
220
221         pipeline = DataPipeline(config)
222         features = pipeline.run()
223
224         # Push metrics to XCom
225         context['task_instance'].xcom_push(
226             key='feature_count',
227             value=len(features)
228         )
229
230     with DAG(
231         'ml_data_pipeline_v2',
232         default_args=default_args,
233         description='Production ML data pipeline',
234         schedule_interval='0 2 * * *', # 2 AM daily
235         start_date=datetime(2024, 1, 1),
236         catchup=False,
237         tags=['ml', 'features', 'production']
238     ) as dag:
239
240         pipeline_task = PythonOperator(
241             task_id='run_pipeline',
242             python_callable=run_data_pipeline,
243             execution_timeout=timedelta(hours=2)
244         )

```

Listing 4.13: Complete production data pipeline

4.9 Chapter Summary

This chapter covered the essential components of data pipeline architecture for ML systems:

- **Ingestion Patterns:** Batch, streaming, and micro-batch processing with Apache Beam and Kafka
- **ETL vs ELT:** Trade-offs and hybrid approaches for ML workloads
- **Data Validation:** Comprehensive quality checks with Great Expectations and custom validators
- **Feature Stores:** Centralized feature management with Feast for training/serving consistency
- **Data Versioning:** Dataset and pipeline tracking with DVC
- **Schema Evolution:** Managing schema changes with Avro and compatibility strategies

Key takeaways:

1. Data pipeline quality directly determines model quality
2. Validation must occur at multiple stages: raw, transformed, features
3. Feature stores eliminate training/serving skew
4. Schema evolution must be planned proactively
5. Data versioning enables reproducibility and debugging

4.10 Exercises

Exercise 4.1. Batch vs Streaming Decision

Design a data pipeline for a fraud detection system that must:

- Process credit card transactions in real-time (<100ms)
- Compute 30-day aggregate features daily
- Train models weekly on historical data

Specify which components should use batch vs streaming, and justify your choices. Draw an architecture diagram.

Exercise 4.2. Implement Streaming Pipeline

Implement a streaming pipeline using Apache Beam or Kafka that:

1. Consumes clickstream events from Kafka
2. Computes 5-minute windowed features per user

3. Validates features using Great Expectations
4. Writes to both online store (Redis) and offline store (S3/Parquet)

Include error handling, dead letter queues, and monitoring.

Exercise 4.3. Data Validation Suite

Create a comprehensive Great Expectations suite for e-commerce transaction data:

- Schema validation (types, required fields)
- Statistical validation (ranges, distributions)
- Referential integrity (user IDs exist, product IDs valid)
- Business rules (order amount > 0 , valid payment methods)
- Temporal validation (timestamps in order, no future dates)

Implement custom expectations for at least 2 business-specific rules.

Exercise 4.4. Feature Store Implementation

Build a feature store using Feast that:

1. Defines entity (user) and features (30+ day aggregates)
2. Materializes features from offline to online store
3. Provides $<10\text{ms}$ feature retrieval for serving
4. Ensures point-in-time correctness for training data
5. Includes feature versioning and lineage tracking

Compare performance between online and offline stores.

Exercise 4.5. Schema Evolution

Given this schema v1:

```
{user_id, timestamp, event_type, page_url, duration_ms}
```

Design schema v2 that adds:

- user_segment (string, optional)
 - referrer_url (string, optional)
 - device_type (string, required, default="unknown")
1. Implement using Avro with compatibility checks
 2. Write code to read v1 data with v2 schema
 3. Handle a breaking change: rename event_type to action_type
 4. Create an adapter for backward compatibility

Exercise 4.6. Data Drift Detection

Implement a data drift monitoring system:

1. Compute statistical properties (mean, std, quantiles) for baseline data
2. Monitor incoming data batches for distribution shifts
3. Detect drift using Kolmogorov-Smirnov test or similar
4. Alert when drift exceeds threshold
5. Visualize drift metrics over time

Test with synthetic data where you inject distribution shifts.

Exercise 4.7. End-to-End Pipeline

Build a complete production data pipeline:

1. Extract data from PostgreSQL (batch) and Kafka (streaming)
2. Validate with multiple Great Expectations checkpoints
3. Engineer 50+ features using window functions
4. Write to feature store (online + offline)
5. Version data with DVC
6. Orchestrate with Airflow
7. Monitor with Prometheus metrics
8. Handle failures with retries and alerting

Document SLAs: latency, throughput, availability, data freshness.

Exercise 4.8. Performance Optimization

Given a slow batch pipeline processing 10TB daily:

1. Profile the pipeline to identify bottlenecks
2. Optimize with partitioning, compression, and caching
3. Implement incremental processing
4. Add parallel processing where possible
5. Compare performance before/after optimizations

Target: 50% reduction in processing time and cost.

Chapter 5

Data Quality and Validation

5.1 Learning Objectives

After completing this chapter, readers will be able to:

- Implement comprehensive data validation strategies for ML pipelines
- Use Great Expectations for automated data quality testing
- Design and track data quality metrics and monitoring dashboards
- Handle data anomalies, outliers, and drift detection
- Build automated data quality pipelines with continuous validation
- Create custom validation rules for ML-specific requirements
- Integrate data quality checks into production workflows

5.2 Introduction

Data quality is the foundation of successful machine learning systems. Poor data quality leads to unreliable models, incorrect predictions, and costly business decisions. In production ML systems, data quality issues can arise from various sources: upstream data changes, data pipeline failures, sensor malfunctions, or user behavior changes. This chapter provides comprehensive frameworks and practical tools for ensuring data quality throughout the ML lifecycle.

The cost of poor data quality is significant. IBM estimates that poor data quality costs the U.S. economy around \$3.1 trillion annually. In ML systems, the impact is even more critical because models learn from data patterns—garbage in, garbage out. A model trained on high-quality data but serving predictions on degraded data will fail silently, producing incorrect results without raising obvious errors.

5.3 Data Quality Dimensions

5.3.1 The Six Dimensions of Data Quality

Understanding data quality requires examining multiple dimensions. Each dimension addresses a specific aspect of data reliability:

- **Accuracy:** Data correctly represents reality. For example, customer addresses match actual locations, transaction amounts reflect true values.
- **Completeness:** All required data is present. Missing values are within acceptable thresholds, and critical fields are always populated.
- **Consistency:** Data is consistent across systems and over time. The same entity has identical attributes across different databases.
- **Timeliness:** Data is up-to-date and available when needed. Fresh data arrives within expected time windows.
- **Validity:** Data conforms to defined formats, ranges, and business rules. Email addresses follow proper format, dates are valid, numerical values fall within expected ranges.
- **Uniqueness:** No duplicate records exist unless intentional. Primary keys are truly unique.

5.3.2 ML-Specific Quality Dimensions

Beyond traditional data quality dimensions, ML systems require additional considerations:

- **Feature Distribution Stability:** Features should maintain consistent statistical properties over time.
- **Label Quality:** Training labels must be accurate, consistent, and representative.
- **Data Freshness:** Training data should reflect current patterns to avoid model staleness.
- **Representation Balance:** Data should adequately represent all important subgroups and edge cases.
- **Correlation Stability:** Relationships between features and targets should remain stable.

5.4 Data Quality Frameworks and Metrics

5.4.1 Comprehensive Data Quality Framework

A robust data quality framework consists of multiple layers working together to ensure data reliability at every stage.

```
1 from dataclasses import dataclass
2 from typing import Dict, List, Optional, Callable, Any
3 from enum import Enum
4 import pandas as pd
5 import numpy as np
6 from datetime import datetime, timedelta
7 import logging
8
```



```

9  class Severity(Enum):
10     """Severity levels for data quality issues"""
11     INFO = "info"
12     WARNING = "warning"
13     ERROR = "error"
14     CRITICAL = "critical"
15
16  @dataclass
17  class ValidationResult:
18     """Result of a data validation check"""
19     check_name: str
20     passed: bool
21     severity: Severity
22     message: str
23     metric_value: Optional[float] = None
24     threshold: Optional[float] = None
25     timestamp: datetime = None
26
27     def __post_init__(self):
28         if self.timestamp is None:
29             self.timestamp = datetime.now()
30
31  class DataQualityMetrics:
32     """Calculate comprehensive data quality metrics"""
33
34     @staticmethod
35     def completeness_score(df: pd.DataFrame,
36                           critical_columns: List[str] = None) -> Dict[str, float]:
37         """
38         Calculate completeness metrics for dataset
39
40         Args:
41             df: Input DataFrame
42             critical_columns: Columns that must be complete
43
44         Returns:
45             Dictionary of completeness metrics
46         """
47         metrics = {}
48
49         # Overall completeness
50         total_cells = df.shape[0] * df.shape[1]
51         non_null_cells = df.count().sum()
52         metrics['overall_completeness'] = non_null_cells / total_cells
53
54         # Per-column completeness
55         column_completeness = df.count() / len(df)
56         metrics['column_completeness'] = column_completeness.to_dict()
57
58         # Critical columns completeness
59         if critical_columns:
60             critical_complete = all(
61                 df[col].notna().all() for col in critical_columns
62             )
63             metrics['critical_columns_complete'] = critical_complete
64             metrics['critical_completeness_rate'] = sum(
65                 df[col].notna().sum() for col in critical_columns
66             ) / (len(df) * len(critical_columns))

```

```

67         return metrics
68
69
70     @staticmethod
71     def uniqueness_score(df: pd.DataFrame,
72                         id_columns: List[str]) -> Dict[str, float]:
73         """Calculate uniqueness metrics"""
74         metrics = {}
75
76         # Check uniqueness of ID columns
77         for col in id_columns:
78             total_rows = len(df)
79             unique_rows = df[col].nunique()
80             metrics[f'{col}_uniqueness'] = unique_rows / total_rows
81
82         # Count duplicates
83         duplicate_count = total_rows - unique_rows
84         metrics[f'{col}_duplicates'] = duplicate_count
85
86         # Overall record uniqueness
87         duplicate_rows = df.duplicated().sum()
88         metrics['record_uniqueness'] = 1 - (duplicate_rows / len(df))
89
90         return metrics
91
92     @staticmethod
93     def validity_score(df: pd.DataFrame,
94                      validation_rules: Dict[str, Callable]) -> Dict[str, float]:
95         """
96         Calculate validity metrics based on custom rules
97
98         Args:
99             df: Input DataFrame
100             validation_rules: Dict mapping column names to validation functions
101
102         Returns:
103             Dictionary of validity metrics
104         """
105         metrics = {}
106
107         for column, validation_func in validation_rules.items():
108             if column in df.columns:
109                 valid_mask = df[column].apply(validation_func)
110                 validity_rate = valid_mask.sum() / len(df)
111                 metrics[f'{column}_validity'] = validity_rate
112                 metrics[f'{column}_invalid_count'] = (~valid_mask).sum()
113
114         # Overall validity
115         if validation_rules:
116             avg_validity = np.mean([
117                 v for k, v in metrics.items() if k.endswith('_validity')
118             ])
119             metrics['overall_validity'] = avg_validity
120
121         return metrics
122
123     @staticmethod
124     def consistency_score(df: pd.DataFrame,

```

```

125         consistency_rules: List[Dict]) -> Dict[str, Any]:
126
127     """
128     Check consistency between related fields
129
130     Args:
131         df: Input DataFrame
132         consistency_rules: List of consistency rule definitions
133
134     Example rule:
135     {
136         'name': 'price_consistency',
137         'condition': lambda row: row['total'] == row['price'] * row['qty'],
138         'severity': Severity.ERROR
139     }
140
141     """
142     metrics = {}
143
144     for rule in consistency_rules:
145         rule_name = rule['name']
146         condition = rule['condition']
147
148         # Apply consistency check
149         consistent_mask = df.apply(condition, axis=1)
150         consistency_rate = consistent_mask.sum() / len(df)
151
152         metrics[f'{rule_name}_consistency'] = consistency_rate
153         metrics[f'{rule_name}_violations'] = (~consistent_mask).sum()
154
155     return metrics
156
157 @staticmethod
158 def timeliness_score(df: pd.DataFrame,
159                     timestamp_column: str,
160                     max_age_hours: float = 24) -> Dict[str, Any]:
161     """Check data timeliness"""
162     metrics = {}
163
164     if timestamp_column not in df.columns:
165         return metrics
166
167     # Convert to datetime
168     timestamps = pd.to_datetime(df[timestamp_column])
169     now = datetime.now()
170
171     # Calculate age statistics
172     ages = (now - timestamps).dt.total_seconds() / 3600 # hours
173
174     metrics['max_age_hours'] = ages.max()
175     metrics['mean_age_hours'] = ages.mean()
176     metrics['median_age_hours'] = ages.median()
177
178     # Check against threshold
179     within_threshold = (ages <= max_age_hours).sum()
180     metrics['timeliness_rate'] = within_threshold / len(df)
181     metrics['stale_records'] = len(df) - within_threshold
182
183     return metrics

```

```

183 @staticmethod
184 def accuracy_score(df: pd.DataFrame,
185                   reference_df: pd.DataFrame,
186                   join_keys: List[str],
187                   compare_columns: List[str]) -> Dict[str, float]:
188     """
189     Calculate accuracy by comparing against reference data
190
191     Args:
192         df: Dataset to validate
193         reference_df: Ground truth reference data
194         join_keys: Columns to join on
195         compare_columns: Columns to compare for accuracy
196     """
197     metrics = {}
198
199     # Merge datasets
200     merged = df.merge(
201         reference_df,
202         on=join_keys,
203         suffixes=('_actual', '_reference'),
204         how='inner'
205     )
206
207     if len(merged) == 0:
208         return metrics
209
210     # Compare each column
211     for col in compare_columns:
212         actual_col = f'{col}_actual'
213         ref_col = f'{col}_reference'
214
215         if actual_col in merged.columns and ref_col in merged.columns:
216             # For numeric columns, calculate error metrics
217             if pd.api.types.is_numeric_dtype(merged[actual_col]):
218                 errors = np.abs(
219                     merged[actual_col] - merged[ref_col]
220                 )
221                 metrics[f'{col}_mae'] = errors.mean()
222                 metrics[f'{col}_mse'] = (errors ** 2).mean()
223
224             # For all columns, calculate match rate
225             matches = (merged[actual_col] == merged[ref_col]).sum()
226             metrics[f'{col}_accuracy'] = matches / len(merged)
227
228     return metrics
229
230 class DataQualityValidator:
231     """Comprehensive data quality validation framework"""
232
233     def __init__(self, name: str = "DataQualityValidator"):
234         self.name = name
235         self.metrics_calculator = DataQualityMetrics()
236         self.validation_results: List[ValidationResult] = []
237         self.logger = logging.getLogger(name)
238
239     def validate(self, df: pd.DataFrame,
240                config: Dict[str, Any]) -> List[ValidationResult]:

```

```

241 """
242 Run comprehensive validation suite
243
244 Args:
245     df: DataFrame to validate
246     config: Validation configuration
247
248 Returns:
249     List of validation results
250 """
251 self.validation_results = []
252
253 # Completeness checks
254 if 'completeness' in config:
255     self._check_completeness(df, config['completeness'])
256
257 # Uniqueness checks
258 if 'uniqueness' in config:
259     self._check_uniqueness(df, config['uniqueness'])
260
261 # Validity checks
262 if 'validity' in config:
263     self._check_validity(df, config['validity'])
264
265 # Consistency checks
266 if 'consistency' in config:
267     self._check_consistency(df, config['consistency'])
268
269 # Timeliness checks
270 if 'timeliness' in config:
271     self._check_timeliness(df, config['timeliness'])
272
273 # Range checks
274 if 'ranges' in config:
275     self._check_ranges(df, config['ranges'])
276
277 # Distribution checks
278 if 'distributions' in config:
279     self._check_distributions(df, config['distributions'])
280
281 return self.validation_results
282
283 def _check_completeness(self, df: pd.DataFrame, config: Dict):
284     """Check completeness requirements"""
285     critical_columns = config.get('critical_columns', [])
286     threshold = config.get('threshold', 0.95)
287
288     metrics = self.metrics_calculator.completeness_score(
289         df, critical_columns
290     )
291
292     # Overall completeness
293     overall = metrics['overall_completeness']
294     passed = overall >= threshold
295
296     self.validation_results.append(ValidationResult(
297         check_name="overall_completeness",
298         passed=passed,

```

```

299         severity=Severity.ERROR if not passed else Severity.INFO,
300         message=f"Overall completeness: {overall:.2%} (threshold: {threshold:.2%})
",
301         metric_value=overall,
302         threshold=threshold
303     ))
304
305     # Critical columns
306     if critical_columns and 'critical_columns_complete' in metrics:
307         critical_complete = metrics['critical_columns_complete']
308         self.validation_results.append(ValidationResult(
309             check_name="critical_columns_completeness",
310             passed=critical_complete,
311             severity=Severity.CRITICAL if not critical_complete else Severity.INFO
312         ,
313             message=f"Critical columns complete: {critical_complete}",
314             metric_value=metrics.get('critical_completeness_rate')
315         ))
316
317     def _check_uniqueness(self, df: pd.DataFrame, config: Dict):
318         """Check uniqueness requirements"""
319         id_columns = config.get('id_columns', [])
320
321         metrics = self.metrics_calculator.uniqueness_score(df, id_columns)
322
323         for col in id_columns:
324             uniqueness = metrics[f'{col}_uniqueness']
325             duplicates = metrics[f'{col}_duplicates']
326
327             passed = uniqueness == 1.0
328
329             self.validation_results.append(ValidationResult(
330                 check_name=f"{col}_uniqueness",
331                 passed=passed,
332                 severity=Severity.ERROR if not passed else Severity.INFO,
333                 message=f"{col} uniqueness: {uniqueness:.2%} ({duplicates} duplicates)
",
334                 metric_value=uniqueness,
335                 threshold=1.0
336             ))
337
338     def _check_validity(self, df: pd.DataFrame, config: Dict):
339         """Check validity rules"""
340         validation_rules = config.get('rules', {})
341         threshold = config.get('threshold', 0.99)
342
343         metrics = self.metrics_calculator.validity_score(
344             df, validation_rules
345         )
346
347         for key, value in metrics.items():
348             if key.endswith('_validity'):
349                 column = key.replace('_validity', '')
350                 passed = value >= threshold
351
352                 self.validation_results.append(ValidationResult(
353                     check_name=f"{column}_validity",
354                     passed=passed,

```

```

354         severity=Severity.WARNING if not passed else Severity.INFO,
355         message=f"{column} validity: {value:.2%}",
356         metric_value=value,
357         threshold=threshold
358     ))
359
360 def _check_consistency(self, df: pd.DataFrame, config: Dict):
361     """Check consistency rules"""
362     rules = config.get('rules', [])
363
364     metrics = self.metrics_calculator.consistency_score(df, rules)
365
366     for rule in rules:
367         rule_name = rule['name']
368         consistency = metrics[f'{rule_name}_consistency']
369         violations = metrics[f'{rule_name}_violations']
370         severity = rule.get('severity', Severity.WARNING)
371
372         passed = consistency == 1.0
373
374         self.validation_results.append(ValidationResult(
375             check_name=rule_name,
376             passed=passed,
377             severity=severity if not passed else Severity.INFO,
378             message=f"{rule_name}: {consistency:.2%} ({violations} violations)",
379             metric_value=consistency
380         ))
381
382 def _check_timeliness(self, df: pd.DataFrame, config: Dict):
383     """Check timeliness requirements"""
384     timestamp_column = config.get('timestamp_column')
385     max_age_hours = config.get('max_age_hours', 24)
386     threshold = config.get('threshold', 0.95)
387
388     metrics = self.metrics_calculator.timeliness_score(
389         df, timestamp_column, max_age_hours
390     )
391
392     if metrics:
393         timeliness_rate = metrics['timeliness_rate']
394         passed = timeliness_rate >= threshold
395
396         self.validation_results.append(ValidationResult(
397             check_name="data_timeliness",
398             passed=passed,
399             severity=Severity.WARNING if not passed else Severity.INFO,
400             message=f"Timeliness: {timeliness_rate:.2%} within {max_age_hours}h "
401                     f"(max age: {metrics['max_age_hours']:.1f}h)",
402             metric_value=timeliness_rate,
403             threshold=threshold
404         ))
405
406 def _check_ranges(self, df: pd.DataFrame, config: Dict):
407     """Check numerical ranges"""
408     range_specs = config.get('specs', {})
409
410     for column, spec in range_specs.items():
411         if column not in df.columns:

```

```

412         continue
413
414         min_val = spec.get('min')
415         max_val = spec.get('max')
416
417         violations = 0
418         if min_val is not None:
419             violations += (df[column] < min_val).sum()
420         if max_val is not None:
421             violations += (df[column] > max_val).sum()
422
423         in_range_rate = 1 - (violations / len(df))
424         passed = violations == 0
425
426         self.validation_results.append(ValidationResult(
427             check_name=f"{column}_range",
428             passed=passed,
429             severity=Severity.WARNING if not passed else Severity.INFO,
430             message=f"{column} range check: {in_range_rate:.2%} in range "
431                 f"[{min_val}, {max_val}] ({violations} violations)",
432             metric_value=in_range_rate
433         ))
434
435     def _check_distributions(self, df: pd.DataFrame, config: Dict):
436         """Check statistical distributions"""
437         distribution_specs = config.get('specs', {})
438
439         for column, spec in distribution_specs.items():
440             if column not in df.columns:
441                 continue
442
443             expected_mean = spec.get('mean')
444             expected_std = spec.get('std')
445             tolerance = spec.get('tolerance', 0.1)
446
447             actual_mean = df[column].mean()
448             actual_std = df[column].std()
449
450             mean_passed = True
451             if expected_mean is not None:
452                 mean_diff = abs(actual_mean - expected_mean) / expected_mean
453                 mean_passed = mean_diff <= tolerance
454
455             std_passed = True
456             if expected_std is not None:
457                 std_diff = abs(actual_std - expected_std) / expected_std
458                 std_passed = std_diff <= tolerance
459
460             passed = mean_passed and std_passed
461
462             self.validation_results.append(ValidationResult(
463                 check_name=f"{column}_distribution",
464                 passed=passed,
465                 severity=Severity.WARNING if not passed else Severity.INFO,
466                 message=f"{column} distribution: mean={actual_mean:.2f} "
467                     f"(expected={expected_mean}), std={actual_std:.2f} "
468                     f"(expected={expected_std})"
469             ))

```



```

470
471 def get_summary(self) -> Dict[str, Any]:
472     """Get summary of validation results"""
473     total = len(self.validation_results)
474     passed = sum(1 for r in self.validation_results if r.passed)
475
476     severity_counts = {}
477     for severity in Severity:
478         count = sum(
479             1 for r in self.validation_results
480             if r.severity == severity and not r.passed
481         )
482         severity_counts[severity.value] = count
483
484     return {
485         'total_checks': total,
486         'passed': passed,
487         'failed': total - passed,
488         'pass_rate': passed / total if total > 0 else 0,
489         'severity_counts': severity_counts,
490         'has_critical_issues': severity_counts.get('critical', 0) > 0,
491         'has_errors': severity_counts.get('error', 0) > 0
492     }
493
494 def print_report(self):
495     """Print formatted validation report"""
496     summary = self.get_summary()
497
498     print(f"\n{'='*60}")
499     print(f"Data Quality Validation Report - {self.name}")
500     print(f"\n{'='*60}")
501     print(f"Total Checks: {summary['total_checks']}")
502     print(f"Passed: {summary['passed']}")
503     print(f"Failed: {summary['failed']}")
504     print(f"Pass Rate: {summary['pass_rate']:.2%}")
505     print(f"\nIssues by Severity:")
506     for severity, count in summary['severity_counts'].items():
507         if count > 0:
508             print(f"    {severity.upper()}: {count}")
509
510     print(f"\n{'='*60}")
511     print("Failed Checks:")
512     print(f"\n{'='*60}")
513
514     for result in self.validation_results:
515         if not result.passed:
516             severity_marker = {
517                 Severity.INFO: "[i]",
518                 Severity.WARNING: "[!]",
519                 Severity.ERROR: "[X]",
520                 Severity.CRITICAL: "[!!]"
521             }
522             marker = severity_marker.get(result.severity, "[-]")
523             print(f"{marker} [{result.severity.value.upper()}] {result.check_name}")
524
525     print(f"    {result.message}")

```

Listing 5.1: Data Quality Framework Implementation

5.4.2 Usage Example: Complete Validation Pipeline

```

1 import pandas as pd
2 import numpy as np
3 from datetime import datetime, timedelta
4
5 # Create sample data with quality issues
6 np.random.seed(42)
7 data = {
8     'user_id': list(range(1000)) + [1, 2, 3], # Duplicates
9     'email': [f'user{i}@example.com' for i in range(1000)] +
10             ['invalid', 'bad@', 'nope'],
11     'age': np.random.randint(18, 80, 1003).tolist(),
12     'signup_date': [
13         datetime.now() - timedelta(days=np.random.randint(0, 365))
14         for _ in range(1003)
15     ],
16     'total_purchase': np.random.uniform(0, 1000, 1003),
17     'num_purchases': np.random.randint(0, 50, 1003)
18 }
19
20 # Introduce some quality issues
21 data['age'][10:20] = -1 # Invalid ages
22 data['email'][50:60] = [None] * 10 # Missing emails
23 data['total_purchase'][100] = -50 # Invalid negative
24
25 df = pd.DataFrame(data)
26
27 # Define validation configuration
28 validation_config = {
29     'completeness': {
30         'critical_columns': ['user_id', 'email'],
31         'threshold': 0.95
32     },
33     'uniqueness': {
34         'id_columns': ['user_id']
35     },
36     'validity': {
37         'rules': {
38             'email': lambda x: pd.notna(x) and '@' in str(x) and '.' in str(x),
39             'age': lambda x: 0 <= x <= 120
40         },
41         'threshold': 0.99
42     },
43     'consistency': {
44         'rules': [
45             {
46                 'name': 'purchase_consistency',
47                 'condition': lambda row: (
48                     row['total_purchase'] >= 0 and
49                     row['num_purchases'] >= 0
50                 ),
51                 'severity': Severity.ERROR
52             }
53         ]
54     },
55     'timeliness': {
56         'timestamp_column': 'signup_date',

```

```

57     'max_age_hours': 8760, # 1 year
58     'threshold': 0.8
59 },
60 'ranges': {
61     'specs': {
62         'age': {'min': 0, 'max': 120},
63         'total_purchase': {'min': 0, 'max': 100000}
64     }
65 }
66 }
67
68 # Run validation
69 validator = DataQualityValidator(name="Customer Data Validator")
70 results = validator.validate(df, validation_config)
71
72 # Print report
73 validator.print_report()
74
75 # Get programmatic summary
76 summary = validator.get_summary()
77 if summary['has_critical_issues'] or summary['has_errors']:
78     print("\n[!] Data quality issues detected! Review before proceeding.")
79 else:
80     print("\n[OK] Data quality checks passed!")

```

Listing 5.2: Using the Data Quality Framework

5.5 Great Expectations Implementation

Great Expectations is a powerful Python library for data validation, documentation, and profiling. It provides a rich set of expectations (assertions about data) that can be combined into validation suites.

5.5.1 Setting Up Great Expectations

```

1 import great_expectations as gx
2 from great_expectations.core.batch import RuntimeBatchRequest
3 from great_expectations.checkpoint import SimpleCheckpoint
4 import pandas as pd
5
6 class GreatExpectationsManager:
7     """Manager class for Great Expectations workflows"""
8
9     def __init__(self, project_root: str):
10         """
11         Initialize Great Expectations context
12
13         Args:
14             project_root: Root directory for GE project
15         """
16         self.context = gx.get_context(project_root_dir=project_root)
17         self.datasource_name = "pandas_datasource"
18         self._setup_datasource()
19
20     def _setup_datasource(self):

```

```

21     """Configure pandas datasource"""
22     try:
23         self.context.get_datasource(self.datasource_name)
24     except:
25         # Create new datasource
26         datasource_config = {
27             "name": self.datasource_name,
28             "class_name": "Datasource",
29             "execution_engine": {
30                 "class_name": "PandasExecutionEngine"
31             },
32             "data_connectors": {
33                 "runtime_data_connector": {
34                     "class_name": "RuntimeDataConnector",
35                     "batch_identifiers": ["batch_id"]
36                 }
37             }
38         }
39         self.context.add_datasource(**datasource_config)
40
41     def create_expectation_suite(self,
42                                 suite_name: str,
43                                 df: pd.DataFrame) -> None:
44         """
45         Create expectation suite with common ML data expectations
46
47         Args:
48             suite_name: Name for the expectation suite
49             df: Sample DataFrame for profiling
50         """
51         # Create empty suite
52         suite = self.context.create_expectation_suite(
53             expectation_suite_name=suite_name,
54             overwrite_existing=True
55         )
56
57         # Create validator
58         batch_request = RuntimeBatchRequest(
59             datasource_name=self.datasource_name,
60             data_connector_name="runtime_data_connector",
61             data_asset_name=suite_name,
62             runtime_parameters={"batch_data": df},
63             batch_identifiers={"batch_id": "initial"}
64         )
65
66         validator = self.context.get_validator(
67             batch_request=batch_request,
68             expectation_suite_name=suite_name
69         )
70
71         # Add table-level expectations
72         validator.expect_table_row_count_to_be_between(
73             min_value=1,
74             max_value=None
75         )
76
77         validator.expect_table_column_count_to_equal(
78             value=len(df.columns)

```

```

79         )
80
81         # Add column-level expectations for each column
82         for column in df.columns:
83             # Column exists
84             validator.expect_column_to_exist(column=column)
85
86             # Type expectations based on dtype
87             if pd.api.types.is_numeric_dtype(df[column]):
88                 # Numeric columns
89                 validator.expect_column_values_to_be_of_type(
90                     column=column,
91                     type_="float64"
92                 )
93
94                 # Check for nulls
95                 null_fraction = df[column].isna().mean()
96                 if null_fraction < 0.01: # Less than 1% nulls
97                     validator.expect_column_values_to_not_be_null(
98                         column=column
99                     )
100
101                 # Set range based on current data
102                 col_min = df[column].min()
103                 col_max = df[column].max()
104                 col_mean = df[column].mean()
105                 col_std = df[column].std()
106
107                 # Allow 3 std deviations from mean
108                 validator.expect_column_values_to_be_between(
109                     column=column,
110                     min_value=col_mean - 3 * col_std,
111                     max_value=col_mean + 3 * col_std,
112                     mostly=0.95 # 95% of values should be in range
113                 )
114
115             elif pd.api.types.is_string_dtype(df[column]):
116                 # String columns
117                 validator.expect_column_values_to_be_of_type(
118                     column=column,
119                     type_="str"
120                 )
121
122                 # Check unique values for potential categorical
123                 unique_ratio = df[column].nunique() / len(df)
124                 if unique_ratio < 0.5: # Likely categorical
125                     value_set = df[column].unique().tolist()
126                     validator.expect_column_values_to_be_in_set(
127                         column=column,
128                         value_set=value_set
129                     )
130
131             elif pd.api.types.is_datetime64_any_dtype(df[column]):
132                 # Datetime columns
133                 validator.expect_column_values_to_be_of_type(
134                     column=column,
135                     type_="datetime64"
136                 )

```

```

137         # Check recency
138         min_date = df[column].min()
139         max_date = df[column].max()
140         validator.expect_column_values_to_be_between(
141             column=column,
142             min_value=min_date,
143             max_value=max_date
144         )
145
146     # Save suite
147     validator.save_expectation_suite(discard_failed_expectations=False)
148
149     print(f"Created expectation suite '{suite_name}' with "
150           f"{len(validator.get_expectation_suite().expectations)} expectations")
151
152     return validator
153
154 def validate_dataframe(self,
155                        df: pd.DataFrame,
156                        suite_name: str,
157                        checkpoint_name: str = None) -> dict:
158     """
159     Validate DataFrame against expectation suite
160
161     Args:
162         df: DataFrame to validate
163         suite_name: Name of expectation suite
164         checkpoint_name: Optional checkpoint name
165
166     Returns:
167         Validation results dictionary
168     """
169     if checkpoint_name is None:
170         checkpoint_name = f"{suite_name}_checkpoint"
171
172     # Create batch request
173     batch_request = RuntimeBatchRequest(
174         datasource_name=self.datasource_name,
175         data_connector_name="runtime_data_connector",
176         data_asset_name=suite_name,
177         runtime_parameters={"batch_data": df},
178         batch_identifiers={"batch_id": "validation"}
179     )
180
181     # Create checkpoint
182     checkpoint_config = {
183         "name": checkpoint_name,
184         "config_version": 1.0,
185         "class_name": "SimpleCheckpoint",
186         "validations": [
187             {
188                 "batch_request": batch_request,
189                 "expectation_suite_name": suite_name
190             }
191         ]
192     }
193
194

```

```

195     # Run validation
196     results = self.context.run_checkpoint(
197         checkpoint_name=checkpoint_name,
198         **checkpoint_config
199     )
200
201     return results
202
203     def create_ml_data_suite(self,
204                             df: pd.DataFrame,
205                             suite_name: str,
206                             feature_columns: List[str],
207                             target_column: str = None,
208                             categorical_columns: List[str] = None) -> None:
209         """
210         Create ML-specific expectation suite
211
212         Args:
213             df: Sample DataFrame
214             suite_name: Name for expectation suite
215             feature_columns: List of feature column names
216             target_column: Target column name (if supervised)
217             categorical_columns: List of categorical column names
218         """
219         suite = self.context.create_expectation_suite(
220             expectation_suite_name=suite_name,
221             overwrite_existing=True
222         )
223
224         batch_request = RuntimeBatchRequest(
225             datasource_name=self.datasource_name,
226             data_connector_name="runtime_data_connector",
227             data_asset_name=suite_name,
228             runtime_parameters={"batch_data": df},
229             batch_identifiers={"batch_id": "ml_suite"}
230         )
231
232         validator = self.context.get_validator(
233             batch_request=batch_request,
234             expectation_suite_name=suite_name
235         )
236
237         # Feature column expectations
238         for col in feature_columns:
239             # No nulls in features
240             validator.expect_column_values_to_not_be_null(column=col)
241
242             # Feature distribution stability
243             if pd.api.types.is_numeric_dtype(df[col]):
244                 mean = df[col].mean()
245                 std = df[col].std()
246
247                 # Mean should be within 20% of baseline
248                 validator.expect_column_mean_to_be_between(
249                     column=col,
250                     min_value=mean * 0.8,
251                     max_value=mean * 1.2
252                 )

```

```

253         # Std should be within 30% of baseline
254         validator.expect_column_stdev_to_be_between(
255             column=col,
256             min_value=std * 0.7,
257             max_value=std * 1.3
258         )
259
260
261     # Categorical column expectations
262     if categorical_columns:
263         for col in categorical_columns:
264             # Known categories
265             categories = df[col].unique().tolist()
266             validator.expect_column_values_to_be_in_set(
267                 column=col,
268                 value_set=categories,
269                 mostly=0.99 # Allow 1% unknown categories
270             )
271
272             # No single category dominates (check for imbalance)
273             value_counts = df[col].value_counts(normalize=True)
274             max_frequency = value_counts.max()
275             if max_frequency < 0.9: # No category > 90%
276                 validator.expect_column_most_common_value_to_be_in_set(
277                     column=col,
278                     value_set=categories
279                 )
280
281     # Target column expectations
282     if target_column:
283         validator.expect_column_values_to_not_be_null(column=target_column)
284
285     # For classification targets
286     if df[target_column].dtype == 'object' or df[target_column].nunique() <
20:
287         classes = df[target_column].unique().tolist()
288         validator.expect_column_values_to_be_in_set(
289             column=target_column,
290             value_set=classes
291         )
292
293     # Check class balance
294     class_distribution = df[target_column].value_counts(normalize=True)
295     min_class_freq = class_distribution.min()
296
297     # Warn if minority class < 5%
298     if min_class_freq < 0.05:
299         print(f"[!] Warning: Minority class frequency is {min_class_freq
300 :.2%}")
301
302     # Correlation expectations (features shouldn't be too correlated)
303     numeric_features = [
304         col for col in feature_columns
305         if pd.api.types.is_numeric_dtype(df[col])
306     ]
307
308     if len(numeric_features) > 1:
309         corr_matrix = df[numeric_features].corr()

```



```

309         # Find highly correlated pairs
310         for i, col1 in enumerate(numeric_features):
311             for col2 in numeric_features[i+1:]:
312                 corr = abs(corr_matrix.loc[col1, col2])
313                 if corr > 0.95:
314                     print(f"[!] High correlation between {col1} and {col2}: {corr
315                             :.3f}")
316
317         validator.save_expectation_suite(discard_failed_expectations=False)
318
319         return validator
320
321 # Example usage
322 def example_great_expectations():
323     """Example of using Great Expectations for ML data validation"""
324
325     # Create sample ML dataset
326     np.random.seed(42)
327     n_samples = 1000
328
329     df = pd.DataFrame({
330         'feature_1': np.random.normal(100, 15, n_samples),
331         'feature_2': np.random.uniform(0, 1, n_samples),
332         'feature_3': np.random.exponential(2, n_samples),
333         'category_a': np.random.choice(['A', 'B', 'C'], n_samples),
334         'category_b': np.random.choice(['X', 'Y'], n_samples, p=[0.7, 0.3]),
335         'target': np.random.choice([0, 1], n_samples, p=[0.6, 0.4])
336     })
337
338     # Initialize GE Manager
339     ge_manager = GreatExpectationsManager(project_root="./ge_project")
340
341     # Create ML-specific suite
342     validator = ge_manager.create_ml_data_suite(
343         df=df,
344         suite_name="ml_features_suite",
345         feature_columns=['feature_1', 'feature_2', 'feature_3',
346                         'category_a', 'category_b'],
347         target_column='target',
348         categorical_columns=['category_a', 'category_b']
349     )
350
351     # Validate new batch of data
352     new_df = pd.DataFrame({
353         'feature_1': np.random.normal(105, 15, 100), # Slight drift
354         'feature_2': np.random.uniform(0, 1, 100),
355         'feature_3': np.random.exponential(2, 100),
356         'category_a': np.random.choice(['A', 'B', 'C'], 100),
357         'category_b': np.random.choice(['X', 'Y'], 100, p=[0.7, 0.3]),
358         'target': np.random.choice([0, 1], 100, p=[0.6, 0.4])
359     })
360
361     results = ge_manager.validate_dataframe(
362         df=new_df,
363         suite_name="ml_features_suite"
364     )
365
366     # Check results

```

```

366     if results.success:
367         print("[OK] Validation passed!")
368     else:
369         print("[X] Validation failed!")
370         print(f"Success rate: {results.statistics['successful_expectations'] / results
               .statistics['evaluated_expectations']:.2%}")

```

Listing 5.3: Great Expectations Setup and Configuration

5.5.2 Custom Great Expectations for ML

```

1  from great_expectations.expectations.expectation import ColumnMapExpectation
2  from great_expectations.expectations.metrics import (
3      ColumnMapMetricProvider,
4      column_condition_partial
5  )
6  from scipy import stats
7  import numpy as np
8
9  class ExpectColumnValuesTo BeWithinIQRRange(ColumnMapExpectation):
10     """
11     Custom expectation: values should be within IQR-based range
12     Useful for outlier detection in ML features
13     """
14
15     map_metric = "column_values.within_iqr_range"
16     success_keys = ("iqr_multiplier", "mostly")
17
18     default_kwarg_values = {
19         "iqr_multiplier": 1.5,
20         "mostly": 0.95,
21         "result_format": "BASIC"
22     }
23
24     @classmethod
25     def _atomic_prescriptive_template(
26         cls,
27         configuration=None,
28         result=None,
29         runtime_configuration=None
30     ):
31         runtime_configuration = runtime_configuration or {}
32         iqr_mult = configuration.kwargs.get("iqr_multiplier", 1.5)
33         mostly = configuration.kwargs.get("mostly", 1.0)
34
35         return (
36             f"Values must be within {iqr_mult} * IQR from quartiles "
37             f"at least {mostly*100:.0f}% of the time."
38         )
39
40     class ColumnValuesWithinIQRRange(ColumnMapMetricProvider):
41         """Metric provider for IQR-based outlier detection"""
42
43         condition_metric_name = "column_values.within_iqr_range"
44
45         @column_condition_partial(engine=PandasExecutionEngine)
46         def _pandas(cls, column, iqr_multiplier=1.5, **kwargs):

```

```

47     q1 = column.quantile(0.25)
48     q3 = column.quantile(0.75)
49     iqr = q3 - q1
50
51     lower_bound = q1 - iqr_multiplier * iqr
52     upper_bound = q3 + iqr_multiplier * iqr
53
54     return (column >= lower_bound) & (column <= upper_bound)
55
56 class ExpectColumnKSTestPValue(ColumnExpectation):
57     """
58     Custom expectation: Kolmogorov-Smirnov test for distribution drift
59     """
60
61     metric_dependencies = ("column.ks_test_p_value",)
62     success_keys = ("reference_distribution", "p_value_threshold")
63
64     default_kwarg_values = {
65         "p_value_threshold": 0.05,
66         "result_format": "BASIC"
67     }
68
69     def validate_configuration(self, configuration):
70         super().validate_configuration(configuration)
71         if "reference_distribution" not in configuration.kwargs:
72             raise ValueError("reference_distribution is required")
73
74     def _validate(
75         self,
76         configuration,
77         metrics,
78         runtime_configuration=None,
79         execution_engine=None
80     ):
81         p_value = metrics["column.ks_test_p_value"]
82         threshold = configuration.kwargs["p_value_threshold"]
83
84         success = p_value >= threshold
85
86         return {
87             "success": success,
88             "result": {
89                 "observed_value": p_value,
90                 "element_count": metrics.get("table.row_count"),
91                 "threshold": threshold
92             }
93         }
94
95     def create_data_drift_expectations(
96         baseline_df: pd.DataFrame,
97         current_df: pd.DataFrame,
98         feature_columns: List[str],
99         context: gx.DataContext,
100         suite_name: str = "drift_detection_suite"
101     ):
102         """
103         Create expectations to detect data drift
104

```

```

105     Args:
106         baseline_df: Reference/baseline DataFrame
107         current_df: Current DataFrame to validate
108         feature_columns: List of feature columns to monitor
109         context: Great Expectations context
110         suite_name: Name for expectation suite
111     """
112     suite = context.create_expectation_suite(
113         expectation_suite_name=suite_name,
114         overwrite_existing=True
115     )
116
117     batch_request = RuntimeBatchRequest(
118         datasource_name="pandas_datasource",
119         data_connector_name="runtime_data_connector",
120         data_asset_name=suite_name,
121         runtime_parameters={"batch_data": current_df},
122         batch_identifiers={"batch_id": "drift_check"}
123     )
124
125     validator = context.get_validator(
126         batch_request=batch_request,
127         expectation_suite_name=suite_name
128     )
129
130     for col in feature_columns:
131         if col not in baseline_df.columns or col not in current_df.columns:
132             continue
133
134         if pd.api.types.is_numeric_dtype(baseline_df[col]):
135             # Statistical tests for numeric features
136             baseline_mean = baseline_df[col].mean()
137             baseline_std = baseline_df[col].std()
138             baseline_median = baseline_df[col].median()
139
140             # Mean drift check (within 10%)
141             validator.expect_column_mean_to_be_between(
142                 column=col,
143                 min_value=baseline_mean * 0.9,
144                 max_value=baseline_mean * 1.1
145             )
146
147             # Std drift check (within 20%)
148             validator.expect_column_stdev_to_be_between(
149                 column=col,
150                 min_value=baseline_std * 0.8,
151                 max_value=baseline_std * 1.2
152             )
153
154             # Median drift check
155             validator.expect_column_median_to_be_between(
156                 column=col,
157                 min_value=baseline_median * 0.9,
158                 max_value=baseline_median * 1.1
159             )
160
161             # Quantile checks
162             for q in [0.25, 0.75, 0.95]:

```

```

163         baseline_quantile = baseline_df[col].quantile(q)
164         validator.expect_column_quantile_values_to_be_between(
165             column=col,
166             quantile_ranges={
167                 "quantiles": [q],
168                 "value_ranges": [[
169                     baseline_quantile * 0.9,
170                     baseline_quantile * 1.1
171                 ]]
172             }
173         )
174
175     elif baseline_df[col].dtype == 'object':
176         # Categorical drift detection
177         baseline_categories = set(baseline_df[col].unique())
178
179         # Check for new categories
180         validator.expect_column_values_to_be_in_set(
181             column=col,
182             value_set=list(baseline_categories),
183             mostly=0.95 # Allow 5% new categories
184         )
185
186         # Check category distribution
187         baseline_dist = baseline_df[col].value_counts(normalize=True)
188         for category, freq in baseline_dist.items():
189             # Each category frequency within 50% of baseline
190             if freq > 0.01: # Only check significant categories
191                 print(f"Monitoring {col}={category}: baseline freq={freq:.2%}")
192
193     validator.save_expectation_suite(discard_failed_expectations=False)
194
195     return validator

```

Listing 5.4: Custom Great Expectations for ML Use Cases

5.6 Data Profiling and Anomaly Detection

5.6.1 Automated Data Profiling

Data profiling generates statistical summaries and insights about datasets. Profiling helps establish baselines, detect anomalies, and understand data characteristics before building validation rules.

```

1 import pandas as pd
2 import numpy as np
3 from typing import Dict, List, Any, Optional
4 from dataclasses import dataclass, field
5 from datetime import datetime
6 import json
7
8 @dataclass
9 class ColumnProfile:
10     """Statistical profile for a single column"""
11     column_name: str
12     dtype: str

```

```

13     count: int
14     null_count: int
15     null_percentage: float
16     unique_count: int
17     unique_percentage: float
18
19     # Numeric statistics
20     mean: Optional[float] = None
21     std: Optional[float] = None
22     min: Optional[float] = None
23     max: Optional[float] = None
24     median: Optional[float] = None
25     q25: Optional[float] = None
26     q75: Optional[float] = None
27     skewness: Optional[float] = None
28     kurtosis: Optional[float] = None
29
30     # Categorical statistics
31     mode: Optional[Any] = None
32     mode_frequency: Optional[int] = None
33     top_values: Dict[Any, int] = field(default_factory=dict)
34
35     # String statistics
36     min_length: Optional[int] = None
37     max_length: Optional[int] = None
38     avg_length: Optional[float] = None
39
40 class DataProfiler:
41     """Comprehensive data profiling tool"""
42
43     def __init__(self):
44         self.profile_timestamp = None
45         self.row_count = 0
46         self.column_count = 0
47         self.column_profiles: Dict[str, ColumnProfile] = {}
48
49     def profile_dataframe(self, df: pd.DataFrame,
50                          sample_size: int = None) -> Dict[str, Any]:
51         """
52         Generate comprehensive data profile
53
54         Args:
55         df: DataFrame to profile
56         sample_size: Optional sample size for large datasets
57
58         Returns:
59         Dictionary containing profile information
60         """
61         self.profile_timestamp = datetime.now()
62         self.row_count = len(df)
63         self.column_count = len(df.columns)
64
65         # Sample if needed
66         if sample_size and len(df) > sample_size:
67             df_sample = df.sample(n=sample_size, random_state=42)
68         else:
69             df_sample = df
70

```

```

71     # Profile each column
72     for column in df.columns:
73         self.column_profiles[column] = self._profile_column(
74             df_sample[column],
75             column
76         )
77
78     return self.get_profile_summary()
79
80     def _profile_column(self, series: pd.Series,
81                        column_name: str) -> ColumnProfile:
82         """Profile a single column"""
83         count = len(series)
84         null_count = series.isna().sum()
85         null_percentage = null_count / count if count > 0 else 0
86         unique_count = series.nunique()
87         unique_percentage = unique_count / count if count > 0 else 0
88
89         profile = ColumnProfile(
90             column_name=column_name,
91             dtype=str(series.dtype),
92             count=count,
93             null_count=null_count,
94             null_percentage=null_percentage,
95             unique_count=unique_count,
96             unique_percentage=unique_percentage
97         )
98
99         # Remove nulls for statistics
100        series_clean = series.dropna()
101
102        if len(series_clean) == 0:
103            return profile
104
105        # Numeric profiling
106        if pd.api.types.is_numeric_dtype(series):
107            profile.mean = float(series_clean.mean())
108            profile.std = float(series_clean.std())
109            profile.min = float(series_clean.min())
110            profile.max = float(series_clean.max())
111            profile.median = float(series_clean.median())
112            profile.q25 = float(series_clean.quantile(0.25))
113            profile.q75 = float(series_clean.quantile(0.75))
114            profile.skewness = float(series_clean.skew())
115            profile.kurtosis = float(series_clean.kurtosis())
116
117        # Categorical profiling
118        value_counts = series_clean.value_counts()
119        if len(value_counts) > 0:
120            profile.mode = value_counts.index[0]
121            profile.mode_frequency = int(value_counts.iloc[0])
122
123        # Top 10 values
124        top_n = min(10, len(value_counts))
125        profile.top_values = {
126            str(k): int(v) for k, v in
127            value_counts.head(top_n).items()
128        }

```

```

129
130     # String profiling
131     if pd.api.types.is_string_dtype(series):
132         lengths = series_clean.astype(str).str.len()
133         profile.min_length = int(lengths.min())
134         profile.max_length = int(lengths.max())
135         profile.avg_length = float(lengths.mean())
136
137     return profile
138
139 def get_profile_summary(self) -> Dict[str, Any]:
140     """Get complete profile summary"""
141     return {
142         'timestamp': self.profile_timestamp.isoformat(),
143         'row_count': self.row_count,
144         'column_count': self.column_count,
145         'columns': {
146             name: self._profile_to_dict(profile)
147             for name, profile in self.column_profiles.items()
148         }
149     }
150
151 def _profile_to_dict(self, profile: ColumnProfile) -> Dict:
152     """Convert ColumnProfile to dictionary"""
153     return {
154         'dtype': profile.dtype,
155         'count': profile.count,
156         'null_count': profile.null_count,
157         'null_percentage': profile.null_percentage,
158         'unique_count': profile.unique_count,
159         'unique_percentage': profile.unique_percentage,
160         'statistics': {
161             'mean': profile.mean,
162             'std': profile.std,
163             'min': profile.min,
164             'max': profile.max,
165             'median': profile.median,
166             'q25': profile.q25,
167             'q75': profile.q75,
168             'skewness': profile.skewness,
169             'kurtosis': profile.kurtosis
170         },
171         'mode': str(profile.mode) if profile.mode is not None else None,
172         'mode_frequency': profile.mode_frequency,
173         'top_values': profile.top_values,
174         'string_stats': {
175             'min_length': profile.min_length,
176             'max_length': profile.max_length,
177             'avg_length': profile.avg_length
178         }
179     }
180
181 def compare_profiles(self, other_profile: Dict[str, Any],
182                     threshold: float = 0.1) -> List[Dict[str, Any]]:
183     """
184     Compare current profile with another profile
185
186     Args:

```



```

245         other_col['statistics'][stat]) /
246         other_col['statistics'][stat]
247     )
248     if change > threshold:
249         changes.append({
250             'type': f'{stat}_change',
251             'severity': 'warning',
252             'column': col_name,
253             'message': f"{col_name}: {stat} changed by {change
254             :.2%}",
255             'previous': other_col['statistics'][stat],
256             'current': current_col['statistics'][stat]
257         })
258
259     # Check for missing columns
260     for col_name in other_profile['columns']:
261         if col_name not in current_profile['columns']:
262             changes.append({
263                 'type': 'missing_column',
264                 'severity': 'error',
265                 'column': col_name,
266                 'message': f"Column removed: {col_name}"
267             })
268
269     return changes
270
271     def save_profile(self, filepath: str):
272         """Save profile to JSON file"""
273         profile = self.get_profile_summary()
274         with open(filepath, 'w') as f:
275             json.dump(profile, f, indent=2)
276
277     @classmethod
278     def load_profile(cls, filepath: str) -> Dict[str, Any]:
279         """Load profile from JSON file"""
280         with open(filepath, 'r') as f:
281             return json.load(f)

```

Listing 5.5: Comprehensive Data Profiling Framework

5.6.2 Anomaly Detection in Data

Anomaly detection identifies unusual patterns or outliers in data that may indicate quality issues.

```

1  from sklearn.ensemble import IsolationForest
2  from sklearn.preprocessing import StandardScaler
3  from sklearn.covariance import EllipticEnvelope
4  from scipy import stats
5  import numpy as np
6  import pandas as pd
7  from typing import Dict, List, Tuple, Optional
8
9  class AnomalyDetector:
10     """Multi-method anomaly detection for data quality"""
11
12     def __init__(self):

```

```

13     self.methods = {
14         'zscore': self._detect_zscore,
15         'iqr': self._detect_iqr,
16         'isolation_forest': self._detect_isolation_forest,
17         'elliptic_envelope': self._detect_elliptic_envelope
18     }
19
20     def detect_anomalies(self,
21                         df: pd.DataFrame,
22                         method: str = 'zscore',
23                         columns: List[str] = None,
24                         **kwargs) -> pd.DataFrame:
25         """
26         Detect anomalies in DataFrame
27
28         Args:
29             df: Input DataFrame
30             method: Detection method ('zscore', 'iqr', 'isolation_forest', '
31             elliptic_envelope')
32             columns: Columns to check (default: all numeric)
33             **kwargs: Method-specific parameters
34
35         Returns:
36             DataFrame with anomaly flags
37         """
38         if columns is None:
39             columns = df.select_dtypes(include=[np.number]).columns.tolist()
40
41         if method not in self.methods:
42             raise ValueError(f"Unknown method: {method}")
43
44         return self.methods[method](df, columns, **kwargs)
45
46     def _detect_zscore(self,
47                       df: pd.DataFrame,
48                       columns: List[str],
49                       threshold: float = 3.0,
50                       **kwargs) -> pd.DataFrame:
51         """
52         Detect anomalies using Z-score method
53
54         Args:
55             threshold: Number of standard deviations (default: 3)
56         """
57         result = df.copy()
58         anomaly_flags = pd.DataFrame(False, index=df.index, columns=columns)
59
60         for col in columns:
61             if col not in df.columns:
62                 continue
63
64             z_scores = np.abs(stats.zscore(df[col].dropna()))
65             # Map back to original indices
66             non_null_idx = df[col].dropna().index
67             anomaly_flags.loc[non_null_idx, col] = z_scores > threshold
68
69         result['anomaly_count'] = anomaly_flags.sum(axis=1)
70         result['is_anomaly'] = result['anomaly_count'] > 0

```

```

70
71     for col in columns:
72         result[f'{col}_anomaly'] = anomaly_flags[col]
73
74     return result
75
76     def _detect_iqr(self,
77                     df: pd.DataFrame,
78                     columns: List[str],
79                     multiplier: float = 1.5,
80                     **kwargs) -> pd.DataFrame:
81         """
82         Detect anomalies using Interquartile Range method
83
84         Args:
85             multiplier: IQR multiplier (default: 1.5)
86         """
87         result = df.copy()
88         anomaly_flags = pd.DataFrame(False, index=df.index, columns=columns)
89
90         for col in columns:
91             if col not in df.columns:
92                 continue
93
94             Q1 = df[col].quantile(0.25)
95             Q3 = df[col].quantile(0.75)
96             IQR = Q3 - Q1
97
98             lower_bound = Q1 - multiplier * IQR
99             upper_bound = Q3 + multiplier * IQR
100
101             anomaly_flags[col] = (
102                 (df[col] < lower_bound) | (df[col] > upper_bound)
103             )
104
105             result['anomaly_count'] = anomaly_flags.sum(axis=1)
106             result['is_anomaly'] = result['anomaly_count'] > 0
107
108         for col in columns:
109             result[f'{col}_anomaly'] = anomaly_flags[col]
110
111         return result
112
113     def _detect_isolation_forest(self,
114                                  df: pd.DataFrame,
115                                  columns: List[str],
116                                  contamination: float = 0.1,
117                                  random_state: int = 42,
118                                  **kwargs) -> pd.DataFrame:
119         """
120         Detect anomalies using Isolation Forest
121
122         Args:
123             contamination: Expected proportion of outliers (default: 0.1)
124         """
125         result = df.copy()
126
127         # Prepare data

```

```

128     X = df[columns].copy()
129     X = X.fillna(X.mean()) # Handle missing values
130
131     # Scale features
132     scaler = StandardScaler()
133     X_scaled = scaler.fit_transform(X)
134
135     # Train Isolation Forest
136     clf = IsolationForest(
137         contamination=contamination,
138         random_state=random_state,
139         n_estimators=100
140     )
141     predictions = clf.fit_predict(X_scaled)
142
143     # -1 for anomalies, 1 for normal
144     result['is_anomaly'] = predictions == -1
145     result['anomaly_score'] = clf.score_samples(X_scaled)
146
147     return result
148
149 def _detect_elliptic_envelope(self,
150                               df: pd.DataFrame,
151                               columns: List[str],
152                               contamination: float = 0.1,
153                               **kwargs) -> pd.DataFrame:
154     """
155     Detect anomalies using Elliptic Envelope (assumes Gaussian distribution)
156
157     Args:
158         contamination: Expected proportion of outliers
159     """
160     result = df.copy()
161
162     # Prepare data
163     X = df[columns].copy()
164     X = X.fillna(X.mean())
165
166     # Scale features
167     scaler = StandardScaler()
168     X_scaled = scaler.fit_transform(X)
169
170     # Train Elliptic Envelope
171     clf = EllipticEnvelope(contamination=contamination, random_state=42)
172     predictions = clf.fit_predict(X_scaled)
173
174     result['is_anomaly'] = predictions == -1
175     result['anomaly_score'] = clf.score_samples(X_scaled)
176
177     return result
178
179 def analyze_anomalies(self, df_with_anomalies: pd.DataFrame,
180                       columns: List[str]) -> Dict[str, Any]:
181     """Analyze detected anomalies"""
182     total_rows = len(df_with_anomalies)
183     anomaly_rows = df_with_anomalies['is_anomaly'].sum()
184
185     analysis = {

```

```

186         'total_rows': total_rows,
187         'anomaly_rows': int(anomaly_rows),
188         'anomaly_percentage': float(anomaly_rows / total_rows),
189         'column_anomalies': {}
190     }
191
192     for col in columns:
193         anomaly_col = f'{col}_anomaly'
194         if anomaly_col in df_with_anomalies.columns:
195             count = df_with_anomalies[anomaly_col].sum()
196             analysis['column_anomalies'][col] = {
197                 'count': int(count),
198                 'percentage': float(count / total_rows)
199             }
200
201     return analysis
202
203 class TimeSeriesAnomalyDetector:
204     """Detect anomalies in time series data"""
205
206     @staticmethod
207     def detect_point_anomalies(series: pd.Series,
208                               window: int = 10,
209                               threshold: float = 3.0) -> pd.Series:
210
211         """
212         Detect point anomalies using rolling statistics
213
214         Args:
215             series: Time series data
216             window: Rolling window size
217             threshold: Number of standard deviations
218         """
219         rolling_mean = series.rolling(window=window, center=True).mean()
220         rolling_std = series.rolling(window=window, center=True).std()
221
222         # Calculate z-scores
223         z_scores = np.abs((series - rolling_mean) / rolling_std)
224
225         return z_scores > threshold
226
227     @staticmethod
228     def detect_trend_changes(series: pd.Series,
229                             window: int = 10,
230                             threshold: float = 0.5) -> pd.Series:
231
232         """
233         Detect sudden trend changes
234
235         Args:
236             series: Time series data
237             window: Window for trend calculation
238             threshold: Threshold for trend change
239         """
240         # Calculate rolling trend (slope)
241         def calculate_slope(y):
242             x = np.arange(len(y))
243             if len(y) < 2:
244                 return 0
245             slope, _ = np.polyfit(x, y, 1)

```

```

244         return slope
245
246     rolling_slope = series.rolling(window=window).apply(
247         calculate_slope, raw=True
248     )
249
250     # Detect sudden slope changes
251     slope_change = rolling_slope.diff().abs()
252
253     return slope_change > threshold
254
255 @staticmethod
256 def detect_seasonal_anomalies(series: pd.Series,
257                               period: int,
258                               threshold: float = 3.0) -> pd.Series:
259     """
260     Detect anomalies considering seasonality
261
262     Args:
263         series: Time series data
264         period: Seasonal period (e.g., 7 for weekly, 12 for monthly)
265         threshold: Number of standard deviations
266     """
267     # Calculate seasonal baseline
268     seasonal_means = series.groupby(series.index % period).transform('mean')
269     seasonal_stds = series.groupby(series.index % period).transform('std')
270
271     # Calculate z-scores relative to seasonal pattern
272     z_scores = np.abs((series - seasonal_means) / seasonal_stds)
273
274     return z_scores > threshold
275
276 # Example usage
277 def example_anomaly_detection():
278     """Example of anomaly detection in ML data"""
279
280     # Create sample data with anomalies
281     np.random.seed(42)
282     n_samples = 1000
283
284     data = {
285         'feature_1': np.random.normal(100, 15, n_samples),
286         'feature_2': np.random.normal(50, 10, n_samples),
287         'feature_3': np.random.uniform(0, 1, n_samples)
288     }
289
290     # Inject anomalies
291     anomaly_indices = np.random.choice(n_samples, 50, replace=False)
292     data['feature_1'][anomaly_indices[:25]] = np.random.uniform(200, 300, 25)
293     data['feature_2'][anomaly_indices[25:]] = np.random.uniform(-50, 0, 25)
294
295     df = pd.DataFrame(data)
296
297     # Detect anomalies using multiple methods
298     detector = AnomalyDetector()
299
300     # Method 1: Z-score
301     result_zscore = detector.detect_anomalies(

```

```

302     df,
303     method='zscore',
304     threshold=3.0
305 )
306
307 # Method 2: IQR
308 result_iqr = detector.detect_anomalies(
309     df,
310     method='iqr',
311     multiplier=1.5
312 )
313
314 # Method 3: Isolation Forest
315 result_if = detector.detect_anomalies(
316     df,
317     method='isolation_forest',
318     contamination=0.05
319 )
320
321 # Analyze results
322 analysis_zscore = detector.analyze_anomalies(
323     result_zscore,
324     ['feature_1', 'feature_2', 'feature_3']
325 )
326
327 print("Z-Score Anomaly Detection:")
328 print(f"  Total anomalies: {analysis_zscore['anomaly_rows']} "
329       f"({analysis_zscore['anomaly_percentage']:.2%})")
330
331 for col, stats in analysis_zscore['column_anomalies'].items():
332     print(f"  {col}: {stats['count']} anomalies ({stats['percentage']:.2%})")

```

Listing 5.6: Anomaly Detection Framework

5.7 Automated Data Quality Monitoring Pipelines

5.7.1 Continuous Monitoring Architecture

Building automated pipelines that continuously monitor data quality ensures early detection of issues in production.

```

1  from typing import Dict, List, Any, Optional, Callable
2  from dataclasses import dataclass
3  from datetime import datetime, timedelta
4  import pandas as pd
5  import logging
6  from enum import Enum
7  import json
8
9  class MonitoringFrequency(Enum):
10     """Monitoring frequency options"""
11     REALTIME = "realtime"
12     HOURLY = "hourly"
13     DAILY = "daily"
14     WEEKLY = "weekly"
15

```



```

16 @dataclass
17 class MonitoringAlert:
18     """Data quality alert"""
19     timestamp: datetime
20     alert_type: str
21     severity: str
22     metric_name: str
23     current_value: float
24     expected_value: float
25     threshold: float
26     message: str
27     metadata: Dict[str, Any] = None
28
29 class DataQualityMonitor:
30     """Continuous data quality monitoring system"""
31
32     def __init__(self, name: str):
33         self.name = name
34         self.logger = logging.getLogger(name)
35         self.metrics_history: List[Dict] = []
36         self.alerts: List[MonitoringAlert] = []
37         self.baseline_profile: Optional[Dict] = None
38
39     def set_baseline(self, df: pd.DataFrame):
40         """Establish baseline profile from reference data"""
41         profiler = DataProfiler()
42         profiler.profile_dataframe(df)
43         self.baseline_profile = profiler.get_profile_summary()
44         self.logger.info(f"Baseline established with {len(df)} rows")
45
46     def monitor_batch(self,
47                       df: pd.DataFrame,
48                       batch_id: str,
49                       timestamp: datetime = None) -> Dict[str, Any]:
50         """
51         Monitor a batch of data
52
53         Args:
54             df: DataFrame to monitor
55             batch_id: Unique identifier for this batch
56             timestamp: Timestamp for this batch
57
58         Returns:
59             Monitoring results
60         """
61         if timestamp is None:
62             timestamp = datetime.now()
63
64         results = {
65             'batch_id': batch_id,
66             'timestamp': timestamp.isoformat(),
67             'row_count': len(df),
68             'column_count': len(df.columns),
69             'checks': [],
70             'alerts': []
71         }
72
73         # Profile current batch

```

```

74     profiler = DataProfiler()
75     profiler.profile_dataframe(df)
76     current_profile = profiler.get_profile_summary()
77
78     # Compare with baseline
79     if self.baseline_profile:
80         changes = profiler.compare_profiles(
81             self.baseline_profile,
82             threshold=0.1
83         )
84
85         results['changes'] = changes
86
87         # Generate alerts for significant changes
88         for change in changes:
89             if change['severity'] in ['warning', 'error']:
90                 alert = MonitoringAlert(
91                     timestamp=timestamp,
92                     alert_type=change['type'],
93                     severity=change['severity'],
94                     metric_name=change.get('column', 'table'),
95                     current_value=change.get('current', 0),
96                     expected_value=change.get('previous', 0),
97                     threshold=0.1,
98                     message=change['message'],
99                     metadata={'batch_id': batch_id}
100                 )
101                 self.alerts.append(alert)
102                 results['alerts'].append(self._alert_to_dict(alert))
103
104         # Run data quality checks
105         validator = DataQualityValidator(name=f"{self.name}_validator")
106
107         validation_config = self._build_validation_config(df)
108         validation_results = validator.validate(df, validation_config)
109
110         results['checks'] = [
111             {
112                 'check_name': r.check_name,
113                 'passed': r.passed,
114                 'severity': r.severity.value,
115                 'message': r.message,
116                 'metric_value': r.metric_value
117             }
118             for r in validation_results
119         ]
120
121         # Track metrics over time
122         self.metrics_history.append({
123             'timestamp': timestamp.isoformat(),
124             'batch_id': batch_id,
125             'metrics': {
126                 'row_count': len(df),
127                 'null_percentage': df.isna().sum().sum() / (df.shape[0] * df.shape[1])
128             },
129             'duplicate_rows': df.duplicated().sum(),
130             'validation_pass_rate': sum(
131                 1 for r in validation_results if r.passed

```

```

131         ) / len(validation_results) if validation_results else 1.0
132     }
133 })
134
135 return results
136
137 def _build_validation_config(self, df: pd.DataFrame) -> Dict[str, Any]:
138     """Build validation configuration based on data characteristics"""
139     numeric_columns = df.select_dtypes(include=[np.number]).columns.tolist()
140
141     config = {
142         'completeness': {
143             'threshold': 0.95
144         },
145         'ranges': {
146             'specs': {}
147         }
148     }
149
150     # Add range checks for numeric columns based on baseline
151     if self.baseline_profile:
152         for col in numeric_columns:
153             if col in self.baseline_profile['columns']:
154                 baseline_col = self.baseline_profile['columns'][col]
155                 stats = baseline_col['statistics']
156
157                 if stats['min'] is not None and stats['max'] is not None:
158                     # Allow some margin based on baseline
159                     margin = (stats['max'] - stats['min']) * 0.2
160                     config['ranges']['specs'][col] = {
161                         'min': stats['min'] - margin,
162                         'max': stats['max'] + margin
163                     }
164
165     return config
166
167 def _alert_to_dict(self, alert: MonitoringAlert) -> Dict:
168     """Convert alert to dictionary"""
169     return {
170         'timestamp': alert.timestamp.isoformat(),
171         'type': alert.alert_type,
172         'severity': alert.severity,
173         'metric': alert.metric_name,
174         'current': alert.current_value,
175         'expected': alert.expected_value,
176         'message': alert.message,
177         'metadata': alert.metadata
178     }
179
180 def get_metrics_trend(self,
181                       metric_name: str,
182                       lookback_hours: int = 24) -> pd.DataFrame:
183     """Get trend for a specific metric"""
184     cutoff_time = datetime.now() - timedelta(hours=lookback_hours)
185
186     relevant_metrics = [
187         m for m in self.metrics_history
188         if datetime.fromisoformat(m['timestamp']) >= cutoff_time

```

```

189         ]
190
191         if not relevant_metrics:
192             return pd.DataFrame()
193
194         data = []
195         for m in relevant_metrics:
196             row = {
197                 'timestamp': m['timestamp'],
198                 'batch_id': m['batch_id']
199             }
200             if metric_name in m['metrics']:
201                 row['value'] = m['metrics'][metric_name]
202                 data.append(row)
203
204         return pd.DataFrame(data)
205
206     def get_active_alerts(self,
207                           severity: str = None,
208                           hours: int = 24) -> List[MonitoringAlert]:
209         """Get recent active alerts"""
210         cutoff_time = datetime.now() - timedelta(hours=hours)
211
212         alerts = [
213             a for a in self.alerts
214             if a.timestamp >= cutoff_time
215         ]
216
217         if severity:
218             alerts = [a for a in alerts if a.severity == severity]
219
220         return alerts
221
222     def generate_report(self,
223                       lookback_hours: int = 24) -> str:
224         """Generate monitoring report"""
225         cutoff_time = datetime.now() - timedelta(hours=lookback_hours)
226
227         recent_alerts = self.get_active_alerts(hours=lookback_hours)
228         critical_alerts = [a for a in recent_alerts if a.severity == 'error']
229
230         report = f"""
231 Data Quality Monitoring Report - {self.name}
232 {'=' * 60}
233 Time Period: Last {lookback_hours} hours
234 Generated: {datetime.now().isoformat()}
235
236 Summary:
237     Total Batches Monitored: {len([m for m in self.metrics_history
238                                     if datetime.fromisoformat(m['timestamp']) >=
239                                     cutoff_time])}
240     Total Alerts: {len(recent_alerts)}
241     Critical Alerts: {len(critical_alerts)}
242
243 Recent Alerts:
244
245     for alert in recent_alerts[:10]: # Show last 10 alerts

```

```

246         report += f"""
247         [{alert.severity.upper()}] {alert.alert_type}
248         Metric: {alert.metric_name}
249         Message: {alert.message}
250         Time: {alert.timestamp.isoformat()}
251     """
252
253     return report
254
255 class DataQualityDashboard:
256     """Dashboard for visualizing data quality metrics"""
257
258     def __init__(self, monitor: DataQualityMonitor):
259         self.monitor = monitor
260
261     def plot_metrics_over_time(self,
262                               metrics: List[str],
263                               hours: int = 24):
264         """Plot metrics trends over time"""
265         import matplotlib.pyplot as plt
266
267         fig, axes = plt.subplots(len(metrics), 1, figsize=(12, 4 * len(metrics)))
268
269         if len(metrics) == 1:
270             axes = [axes]
271
272         for idx, metric in enumerate(metrics):
273             df = self.monitor.get_metrics_trend(metric, lookback_hours=hours)
274
275             if not df.empty:
276                 df['timestamp'] = pd.to_datetime(df['timestamp'])
277                 df = df.sort_values('timestamp')
278
279                 axes[idx].plot(df['timestamp'], df['value'], marker='o')
280                 axes[idx].set_title(f'{metric} Over Time')
281                 axes[idx].set_xlabel('Time')
282                 axes[idx].set_ylabel(metric)
283                 axes[idx].grid(True)
284                 axes[idx].tick_params(axis='x', rotation=45)
285
286         plt.tight_layout()
287         return fig
288
289     def create_quality_scorecard(self) -> Dict[str, Any]:
290         """Create quality scorecard summary"""
291         if not self.monitor.metrics_history:
292             return {}
293
294         latest_metrics = self.monitor.metrics_history[-1]['metrics']
295
296         scorecard = {
297             'data_quality_score': self._calculate_overall_score(latest_metrics),
298             'metrics': latest_metrics,
299             'status': self._determine_status(latest_metrics),
300             'recommendations': self._generate_recommendations(latest_metrics)
301         }
302
303         return scorecard

```

```

304
305 def _calculate_overall_score(self, metrics: Dict) -> float:
306     """Calculate overall data quality score (0-100)"""
307     score = 100.0
308
309     # Deduct points for issues
310     null_pct = metrics.get('null_percentage', 0)
311     score -= null_pct * 100 * 0.3 # 30% weight
312
313     dup_pct = metrics.get('duplicate_rows', 0) / max(metrics.get('row_count', 1),
1)
314     score -= dup_pct * 100 * 0.2 # 20% weight
315
316     validation_rate = metrics.get('validation_pass_rate', 1.0)
317     score -= (1 - validation_rate) * 100 * 0.5 # 50% weight
318
319     return max(0, min(100, score))
320
321 def _determine_status(self, metrics: Dict) -> str:
322     """Determine overall data quality status"""
323     score = self._calculate_overall_score(metrics)
324
325     if score >= 90:
326         return "EXCELLENT"
327     elif score >= 75:
328         return "GOOD"
329     elif score >= 60:
330         return "FAIR"
331     else:
332         return "POOR"
333
334 def _generate_recommendations(self, metrics: Dict) -> List[str]:
335     """Generate actionable recommendations"""
336     recommendations = []
337
338     null_pct = metrics.get('null_percentage', 0)
339     if null_pct > 0.05:
340         recommendations.append(
341             f"High null percentage ({null_pct:.2%}). "
342             "Review data collection and imputation strategies."
343         )
344
345     dup_rows = metrics.get('duplicate_rows', 0)
346     if dup_rows > 0:
347         recommendations.append(
348             f"Found {dup_rows} duplicate rows. "
349             "Implement deduplication logic."
350         )
351
352     validation_rate = metrics.get('validation_pass_rate', 1.0)
353     if validation_rate < 0.95:
354         recommendations.append(
355             f"Validation pass rate is {validation_rate:.2%}. "
356             "Review and fix failing validation rules."
357         )
358
359     if not recommendations:
360         recommendations.append("Data quality looks good! Continue monitoring.")

```

```

361
362         return recommendations

```

Listing 5.7: Automated Data Quality Monitoring Pipeline

5.7.2 Integration with Apache Airflow

Integrating data quality checks into orchestration platforms like Airflow enables automated, scheduled monitoring.

```

1  from airflow import DAG
2  from airflow.operators.python import PythonOperator
3  from airflow.sensors.external_task import ExternalTaskSensor
4  from datetime import datetime, timedelta
5  import pandas as pd
6
7  # Initialize data quality monitor
8  monitor = DataQualityMonitor(name="production_data_monitor")
9
10 def load_baseline_data(**context):
11     """Load and establish baseline from historical data"""
12     # Load baseline dataset
13     baseline_df = pd.read_parquet('s3://data-bucket/baseline/data.parquet')
14
15     # Establish baseline
16     monitor.set_baseline(baseline_df)
17
18     context['ti'].xcom_push(key='baseline_established', value=True)
19
20 def validate_incoming_data(**context):
21     """Validate incoming data batch"""
22     execution_date = context['execution_date']
23     batch_id = execution_date.strftime('%Y%m%d_%H%M%S')
24
25     # Load incoming data
26     df = pd.read_parquet(
27         f's3://data-bucket/incoming/{execution_date.date()}/data.parquet'
28     )
29
30     # Monitor batch
31     results = monitor.monitor_batch(df, batch_id, execution_date)
32
33     # Check for critical issues
34     has_critical = any(
35         alert['severity'] == 'error'
36         for alert in results.get('alerts', [])
37     )
38
39     if has_critical:
40         raise ValueError(f"Critical data quality issues detected in batch {batch_id}")
41
42     # Push results to XCom
43     context['ti'].xcom_push(key='validation_results', value=results)
44
45     return results
46
47 def send_quality_report(**context):

```

```

48 """Send daily data quality report"""
49 report = monitor.generate_report(lookback_hours=24)
50
51 # Send via email/Slack (implementation depends on notification system)
52 print(report)
53
54 context['ti'].xcom_push(key='report_sent', value=True)
55
56 # Define DAG
57 default_args = {
58     'owner': 'ml_team',
59     'depends_on_past': False,
60     'start_date': datetime(2024, 1, 1),
61     'email_on_failure': True,
62     'email_on_retry': False,
63     'retries': 1,
64     'retry_delay': timedelta(minutes=5)
65 }
66
67 dag = DAG(
68     'data_quality_monitoring',
69     default_args=default_args,
70     description='Monitor data quality for ML pipeline',
71     schedule_interval='@hourly',
72     catchup=False
73 )
74
75 # Task 1: Establish baseline (runs once daily)
76 baseline_task = PythonOperator(
77     task_id='establish_baseline',
78     python_callable=load_baseline_data,
79     dag=dag
80 )
81
82 # Task 2: Validate incoming data
83 validation_task = PythonOperator(
84     task_id='validate_data',
85     python_callable=validate_incoming_data,
86     dag=dag
87 )
88
89 # Task 3: Send quality report
90 report_task = PythonOperator(
91     task_id='send_quality_report',
92     python_callable=send_quality_report,
93     dag=dag,
94     trigger_rule='all_done'
95 )
96
97 # Set task dependencies
98 baseline_task >> validation_task >> report_task

```

Listing 5.8: Airflow DAG with Data Quality Checks

5.8 Custom Validation Rules for ML-Specific Requirements

Machine learning systems often require specialized validation beyond standard data quality checks.

5.8.1 ML-Specific Validation Rules

```

1 from typing import Callable, List, Dict, Any
2 import pandas as pd
3 import numpy as np
4 from scipy import stats
5
6 class MLDataValidator:
7     """Custom validation rules for ML-specific requirements"""
8
9     @staticmethod
10    def validate_feature_distributions(
11        df: pd.DataFrame,
12        baseline_df: pd.DataFrame,
13        feature_columns: List[str],
14        p_value_threshold: float = 0.05
15    ) -> Dict[str, Any]:
16        """
17        Validate that feature distributions haven't changed significantly
18
19        Uses Kolmogorov-Smirnov test for continuous features
20        and Chi-square test for categorical features
21        """
22        results = []
23
24        for col in feature_columns:
25            if col not in df.columns or col not in baseline_df.columns:
26                continue
27
28            # For numeric features, use KS test
29            if pd.api.types.is_numeric_dtype(df[col]):
30                statistic, p_value = stats.ks_2samp(
31                    df[col].dropna(),
32                    baseline_df[col].dropna()
33                )
34
35                passed = p_value >= p_value_threshold
36
37                results.append({
38                    'feature': col,
39                    'test': 'KS-test',
40                    'statistic': statistic,
41                    'p_value': p_value,
42                    'passed': passed,
43                    'message': f"{'[OK]' if passed else '[X]'} {col} distribution "
44                               f"{'matches' if passed else 'differs from'} baseline "
45                               f"(p={p_value:.4f})"
46                })
47
48            # For categorical features, use chi-square test

```

```

49         elif df[col].dtype == 'object':
50             # Get value counts
51             current_counts = df[col].value_counts()
52             baseline_counts = baseline_df[col].value_counts()
53
54             # Align categories
55             all_categories = set(current_counts.index) | set(baseline_counts.index)
56         )
57         current_freq = [current_counts.get(cat, 0) for cat in all_categories]
58         baseline_freq = [baseline_counts.get(cat, 0) for cat in all_categories]
59
60         # Chi-square test
61         statistic, p_value = stats.chisquare(
62             f_obs=current_freq,
63             f_exp=baseline_freq
64         )
65
66         passed = p_value >= p_value_threshold
67
68         results.append({
69             'feature': col,
70             'test': 'Chi-square',
71             'statistic': statistic,
72             'p_value': p_value,
73             'passed': passed,
74             'message': f"{'[OK]' if passed else '[X]'} {col} distribution "
75                       f"{'matches' if passed else 'differs from'} baseline "
76                       f"(p={p_value:.4f})"
77         })
78
79     return results
80
81 @staticmethod
82 def validate_class_balance(
83     df: pd.DataFrame,
84     target_column: str,
85     min_class_frequency: float = 0.01,
86     max_imbalance_ratio: float = 100.0
87 ) -> Dict[str, Any]:
88     """
89     Validate that class distribution is reasonable
90
91     Args:
92         min_class_frequency: Minimum frequency for any class
93         max_imbalance_ratio: Maximum ratio between largest and smallest class
94     """
95     class_counts = df[target_column].value_counts()
96     class_frequencies = class_counts / len(df)
97
98     min_freq = class_frequencies.min()
99     max_freq = class_frequencies.max()
100     imbalance_ratio = max_freq / min_freq if min_freq > 0 else float('inf')
101
102     passed_min_freq = min_freq >= min_class_frequency
103     passed_imbalance = imbalance_ratio <= max_imbalance_ratio
104
105     return {

```

```

105         'passed': passed_min_freq and passed_imbalance,
106         'class_distribution': class_frequencies.to_dict(),
107         'min_frequency': float(min_freq),
108         'max_frequency': float(max_freq),
109         'imbalance_ratio': float(imbalance_ratio),
110         'checks': {
111             'min_frequency_check': {
112                 'passed': passed_min_freq,
113                 'threshold': min_class_frequency,
114                 'actual': float(min_freq)
115             },
116             'imbalance_check': {
117                 'passed': passed_imbalance,
118                 'threshold': max_imbalance_ratio,
119                 'actual': float(imbalance_ratio)
120             }
121         }
122     }
123
124     @staticmethod
125     def validate_feature_correlations(
126         df: pd.DataFrame,
127         feature_columns: List[str],
128         max_correlation: float = 0.95
129     ) -> Dict[str, Any]:
130         """
131         Check for highly correlated features that may cause multicollinearity
132
133         Args:
134             max_correlation: Maximum acceptable correlation between features
135         """
136         # Calculate correlation matrix
137         corr_matrix = df[feature_columns].corr().abs()
138
139         # Find highly correlated pairs
140         high_correlations = []
141         for i in range(len(feature_columns)):
142             for j in range(i + 1, len(feature_columns)):
143                 col1 = feature_columns[i]
144                 col2 = feature_columns[j]
145                 correlation = corr_matrix.loc[col1, col2]
146
147                 if correlation > max_correlation:
148                     high_correlations.append({
149                         'feature_1': col1,
150                         'feature_2': col2,
151                         'correlation': float(correlation)
152                     })
153
154         return {
155             'passed': len(high_correlations) == 0,
156             'high_correlations': high_correlations,
157             'count': len(high_correlations),
158             'threshold': max_correlation,
159             'message': f"Found {len(high_correlations)} highly correlated feature
pairs"
160         }
161

```

```

162 @staticmethod
163 def validate_feature_importance_coverage(
164     df: pd.DataFrame,
165     feature_importance_dict: Dict[str, float],
166     top_n_features: int = 10,
167     coverage_threshold: float = 0.8
168 ) -> Dict[str, Any]:
169     """
170     Validate that data contains top important features
171
172     Args:
173         feature_importance_dict: Dict mapping feature names to importance scores
174         top_n_features: Number of top features to check
175         coverage_threshold: Minimum cumulative importance that must be present
176     """
177     # Sort features by importance
178     sorted_features = sorted(
179         feature_importance_dict.items(),
180         key=lambda x: x[1],
181         reverse=True
182     )[:top_n_features]
183
184     # Check which features are present
185     present_features = []
186     missing_features = []
187     cumulative_importance = 0.0
188     total_importance = sum(f[1] for f in sorted_features)
189
190     for feature, importance in sorted_features:
191         if feature in df.columns:
192             present_features.append(feature)
193             cumulative_importance += importance
194         else:
195             missing_features.append(feature)
196
197     coverage = cumulative_importance / total_importance if total_importance > 0
198     else 0.0
199
200     return {
201         'passed': coverage >= coverage_threshold,
202         'coverage': coverage,
203         'threshold': coverage_threshold,
204         'present_features': present_features,
205         'missing_features': missing_features,
206         'message': f"Feature importance coverage: {coverage:.2%} "
207             f"({'[OK] passed' if coverage >= coverage_threshold else '[X] "
208             failed'})"
209     }
210
211 @staticmethod
212 def validate_no_data_leakage(
213     train_df: pd.DataFrame,
214     test_df: pd.DataFrame,
215     id_columns: List[str]
216 ) -> Dict[str, Any]:
217     """
218     Check for data leakage between train and test sets

```

```

218     Args:
219         train_df: Training dataset
220         test_df: Test dataset
221         id_columns: Columns that uniquely identify records
222     """
223     if not id_columns:
224         return {
225             'passed': True,
226             'message': 'No ID columns specified, cannot check for leakage'
227         }
228
229     # Check for overlapping IDs
230     train_ids = set(train_df[id_columns[0]].unique())
231     test_ids = set(test_df[id_columns[0]].unique())
232
233     overlap = train_ids & test_ids
234     overlap_count = len(overlap)
235
236     return {
237         'passed': overlap_count == 0,
238         'overlap_count': overlap_count,
239         'overlap_percentage': overlap_count / len(test_ids) if test_ids else 0,
240         'overlapping_ids': list(overlap)[:10], # Show first 10
241         'message': f'{"[OK]" if overlap_count == 0 else f"[X] {overlap_count}"}'
242     }
243
244
245     @staticmethod
246     def validate_temporal_consistency(
247         df: pd.DataFrame,
248         timestamp_column: str,
249         feature_columns: List[str],
250         window_size: int = 100,
251         max_drift_score: float = 0.3
252     ) -> Dict[str, Any]:
253         """
254         Check for temporal drift in feature distributions
255
256         Args:
257             timestamp_column: Column containing timestamps
258             feature_columns: Features to check for drift
259             window_size: Size of rolling window
260             max_drift_score: Maximum acceptable drift score
261         """
262         # Sort by timestamp
263         df_sorted = df.sort_values(timestamp_column)
264
265         drift_scores = []
266
267         for col in feature_columns:
268             if not pd.api.types.is_numeric_dtype(df_sorted[col]):
269                 continue
270
271             # Calculate rolling statistics
272             rolling_mean = df_sorted[col].rolling(window=window_size).mean()
273             rolling_std = df_sorted[col].rolling(window=window_size).std()
274

```

```

275     # Calculate drift score (coefficient of variation of rolling mean)
276     mean_cv = rolling_mean.std() / rolling_mean.mean() if rolling_mean.mean()
    != 0 else 0
277     std_cv = rolling_std.std() / rolling_std.mean() if rolling_std.mean() != 0
    else 0
278
279     drift_score = (mean_cv + std_cv) / 2
280
281     drift_scores.append({
282         'feature': col,
283         'drift_score': float(drift_score),
284         'passed': drift_score <= max_drift_score
285     })
286
287     overall_passed = all(d['passed'] for d in drift_scores)
288
289     return {
290         'passed': overall_passed,
291         'drift_scores': drift_scores,
292         'max_drift_score': max_drift_score,
293         'message': f"{'[OK] No' if overall_passed else '[X]'} significant temporal
    drift detected"
294     }
295
296 # Example usage
297 def example_ml_validation():
298     """Example of ML-specific validation"""
299
300     # Create sample training and test data
301     np.random.seed(42)
302     n_train = 1000
303     n_test = 200
304
305     train_df = pd.DataFrame({
306         'id': range(n_train),
307         'feature_1': np.random.normal(100, 15, n_train),
308         'feature_2': np.random.uniform(0, 1, n_train),
309         'feature_3': np.random.exponential(2, n_train),
310         'category': np.random.choice(['A', 'B', 'C'], n_train),
311         'target': np.random.choice([0, 1], n_train, p=[0.7, 0.3]),
312         'timestamp': pd.date_range('2024-01-01', periods=n_train, freq='H')
313     })
314
315     test_df = pd.DataFrame({
316         'id': range(n_train, n_train + n_test),
317         'feature_1': np.random.normal(105, 15, n_test), # Slight drift
318         'feature_2': np.random.uniform(0, 1, n_test),
319         'feature_3': np.random.exponential(2, n_test),
320         'category': np.random.choice(['A', 'B', 'C'], n_test),
321         'target': np.random.choice([0, 1], n_test, p=[0.7, 0.3]),
322         'timestamp': pd.date_range('2024-02-11', periods=n_test, freq='H')
323     })
324
325     validator = MLDataValidator()
326
327     # Check 1: Feature distributions
328     dist_results = validator.validate_feature_distributions(
329         df=test_df,

```

```

330     baseline_df=train_df,
331     feature_columns=['feature_1', 'feature_2', 'feature_3']
332 )
333
334 print("Feature Distribution Validation:")
335 for result in dist_results:
336     print(f"    {result['message']}")
337
338 # Check 2: Class balance
339 balance_results = validator.validate_class_balance(
340     df=train_df,
341     target_column='target',
342     min_class_frequency=0.01,
343     max_imbalance_ratio=10.0
344 )
345
346 print(f"\nClass Balance: {'[OK] Passed' if balance_results['passed'] else '[X] Failed'}")
347 print(f"    Imbalance Ratio: {balance_results['imbalance_ratio']:.2f}")
348
349 # Check 3: Feature correlations
350 corr_results = validator.validate_feature_correlations(
351     df=train_df,
352     feature_columns=['feature_1', 'feature_2', 'feature_3'],
353     max_correlation=0.95
354 )
355
356 print(f"\nFeature Correlations: {'[OK] Passed' if corr_results['passed'] else '[X] Failed'}")
357
358 # Check 4: Data leakage
359 leakage_results = validator.validate_no_data_leakage(
360     train_df=train_df,
361     test_df=test_df,
362     id_columns=['id']
363 )
364
365 print(f"\nData Leakage Check: {leakage_results['message']}")
366
367 # Check 5: Temporal consistency
368 temporal_results = validator.validate_temporal_consistency(
369     df=train_df,
370     timestamp_column='timestamp',
371     feature_columns=['feature_1', 'feature_2'],
372     window_size=100,
373     max_drift_score=0.3
374 )
375
376 print(f"\nTemporal Consistency: {temporal_results['message']}")

```

Listing 5.9: Custom ML Validation Rules

5.9 Pipeline Integration for Continuous Validation

5.9.1 End-to-End Data Quality Pipeline

```

1 from typing import Dict, List, Optional
2 import pandas as pd
3 from dataclasses import dataclass
4 import logging
5
6 @dataclass
7 class PipelineConfig:
8     """Configuration for data quality pipeline"""
9     enable_profiling: bool = True
10    enable_validation: bool = True
11    enable_anomaly_detection: bool = True
12    enable_drift_detection: bool = True
13    fail_on_error: bool = True
14    fail_on_warning: bool = False
15
16 class DataQualityPipeline:
17     """
18     Integrated data quality pipeline for ML workflows
19
20     Combines profiling, validation, anomaly detection, and monitoring
21     """
22
23     def __init__(self, config: PipelineConfig, name: str = "DataQualityPipeline"):
24         self.config = config
25         self.name = name
26         self.logger = logging.getLogger(name)
27
28         # Initialize components
29         self.profiler = DataProfiler()
30         self.validator = DataQualityValidator(name=f"{name}_validator")
31         self.anomaly_detector = AnomalyDetector()
32         self.ml_validator = MLDataValidator()
33         self.monitor = DataQualityMonitor(name=name)
34
35         self.baseline_profile: Optional[Dict] = None
36
37     def run(self,
38            df: pd.DataFrame,
39            validation_config: Dict,
40            batch_id: str = None,
41            is_baseline: bool = False) -> Dict[str, Any]:
42         """
43         Run complete data quality pipeline
44
45         Args:
46             df: DataFrame to process
47             validation_config: Validation rules configuration
48             batch_id: Optional batch identifier
49             is_baseline: If True, establishes this as baseline
50
51         Returns:
52             Dictionary with all results
53         """
54         results = {
55             'batch_id': batch_id,
56             'row_count': len(df),
57             'column_count': len(df.columns),

```



```

58         'passed': True,
59         'errors': [],
60         'warnings': []
61     }
62
63     try:
64         # Step 1: Data Profiling
65         if self.config.enable_profiling:
66             self.logger.info("Running data profiling...")
67             profile = self.profiler.profile_dataframe(df)
68             results['profile'] = profile
69
70             if is_baseline:
71                 self.baseline_profile = profile
72                 self.monitor.set_baseline(df)
73                 self.logger.info("Baseline established")
74
75         # Step 2: Data Validation
76         if self.config.enable_validation:
77             self.logger.info("Running data validation...")
78             validation_results = self.validator.validate(df, validation_config)
79             results['validation'] = [
80                 {
81                     'check': r.check_name,
82                     'passed': r.passed,
83                     'severity': r.severity.value,
84                     'message': r.message
85                 }
86                 for r in validation_results
87             ]
88
89             # Check for failures
90             for r in validation_results:
91                 if not r.passed:
92                     if r.severity.value == 'error' or r.severity.value == '
critical':
93                         results['errors'].append(r.message)
94                         results['passed'] = False
95                     elif r.severity.value == 'warning':
96                         results['warnings'].append(r.message)
97                         if self.config.fail_on_warning:
98                             results['passed'] = False
99
100         # Step 3: Anomaly Detection
101         if self.config.enable_anomaly_detection:
102             self.logger.info("Running anomaly detection...")
103             numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
104
105             if numeric_cols:
106                 anomaly_df = self.anomaly_detector.detect_anomalies(
107                     df,
108                     method='isolation_forest',
109                     contamination=0.05
110                 )
111
112                 anomaly_analysis = self.anomaly_detector.analyze_anomalies(
113                     anomaly_df,
114                     numeric_cols

```

```

115         )
116
117         results['anomalies'] = anomaly_analysis
118
119         # Warn if too many anomalies
120         if anomaly_analysis['anomaly_percentage'] > 0.1:
121             warning = (f"High anomaly rate: "
122                        f"{anomaly_analysis['anomaly_percentage']:.2%}")
123             results['warnings'].append(warning)
124
125         # Step 4: Drift Detection
126         if self.config.enable_drift_detection and self.baseline_profile:
127             self.logger.info("Running drift detection...")
128
129             changes = self.profiler.compare_profiles(
130                 self.baseline_profile,
131                 threshold=0.1
132             )
133
134             results['drift'] = changes
135
136             # Check for significant drift
137             for change in changes:
138                 if change['severity'] == 'error':
139                     results['errors'].append(change['message'])
140                     results['passed'] = False
141                 elif change['severity'] == 'warning':
142                     results['warnings'].append(change['message'])
143
144             # Step 5: Monitoring
145             if batch_id:
146                 monitor_results = self.monitor.monitor_batch(df, batch_id)
147                 results['monitoring'] = monitor_results
148
149             # Final status
150             results['status'] = 'PASSED' if results['passed'] else 'FAILED'
151
152             self.logger.info(f"Pipeline completed: {results['status']}")
153
154         except Exception as e:
155             self.logger.error(f"Pipeline failed with error: {str(e)}")
156             results['passed'] = False
157             results['status'] = 'ERROR'
158             results['error_message'] = str(e)
159
160         return results
161
162     def get_pipeline_report(self, results: Dict) -> str:
163         """Generate human-readable pipeline report"""
164         report = f"""
165 Data Quality Pipeline Report
166 {'=' * 60}
167 Batch ID: {results.get('batch_id', 'N/A')}
168 Status: {results.get('status', 'UNKNOWN')}
169 Rows: {results['row_count']}
170 Columns: {results['column_count']}
171
172 Summary:

```

```

173     Errors: {len(results.get('errors', []))}
174     Warnings: {len(results.get('warnings', []))}
175     """
176
177     if results.get('errors'):
178         report += "\nErrors:\n"
179         for error in results['errors']:
180             report += f"    [X] {error}\n"
181
182     if results.get('warnings'):
183         report += "\nWarnings:\n"
184         for warning in results['warnings']:
185             report += f"    [!] {warning}\n"
186
187     if results.get('anomalies'):
188         anom = results['anomalies']
189         report += f"\nAnomalies:\n"
190         report += f"    Total: {anom['anomaly_rows']} ({anom['anomaly_percentage']:.2%})\n"
191
192     if results.get('drift'):
193         report += f"\nDrift Detected:\n"
194         for change in results['drift'][:5]: # Show first 5
195             report += f"    {change['message']}\n"
196
197     return report
198
199 # Example: Complete pipeline integration
200 def example_complete_pipeline():
201     """Example of complete data quality pipeline"""
202
203     # Configuration
204     config = PipelineConfig(
205         enable_profiling=True,
206         enable_validation=True,
207         enable_anomaly_detection=True,
208         enable_drift_detection=True,
209         fail_on_error=True,
210         fail_on_warning=False
211     )
212
213     # Initialize pipeline
214     pipeline = DataQualityPipeline(config, name="ml_data_pipeline")
215
216     # Create baseline data
217     np.random.seed(42)
218     baseline_df = pd.DataFrame({
219         'feature_1': np.random.normal(100, 15, 1000),
220         'feature_2': np.random.uniform(0, 1, 1000),
221         'target': np.random.choice([0, 1], 1000, p=[0.7, 0.3])
222     })
223
224     # Validation configuration
225     validation_config = {
226         'completeness': {'threshold': 0.95},
227         'uniqueness': {'id_columns': []},
228         'ranges': {
229             'specs': {

```

```

230         'feature_1': {'min': 0, 'max': 200},
231         'feature_2': {'min': 0, 'max': 1}
232     }
233 }
234 }
235
236 # Run pipeline on baseline
237 baseline_results = pipeline.run(
238     df=baseline_df,
239     validation_config=validation_config,
240     batch_id='baseline_001',
241     is_baseline=True
242 )
243
244 print("Baseline Pipeline Results:")
245 print(pipeline.get_pipeline_report(baseline_results))
246
247 # Create new batch with drift
248 new_df = pd.DataFrame({
249     'feature_1': np.random.normal(110, 15, 500), # Drift in mean
250     'feature_2': np.random.uniform(0, 1, 500),
251     'target': np.random.choice([0, 1], 500, p=[0.6, 0.4]) # Drift in distribution
252 })
253
254 # Run pipeline on new batch
255 new_results = pipeline.run(
256     df=new_df,
257     validation_config=validation_config,
258     batch_id='batch_002'
259 )
260
261 print("\n" + "=" * 60)
262 print("New Batch Pipeline Results:")
263 print(pipeline.get_pipeline_report(new_results))
264
265 # Check if pipeline passed
266 if new_results['passed']:
267     print("\n[OK] Data quality checks passed - proceed with ML training")
268 else:
269     print("\n[X] Data quality checks failed - review issues before proceeding")

```

Listing 5.10: Complete Data Quality Pipeline Integration

5.10 Chapter Summary

This chapter covered comprehensive data quality and validation strategies for machine learning systems. Data quality is fundamental to ML success, and implementing robust validation frameworks prevents costly errors in production.

Key Takeaways:

- **Six Dimensions of Data Quality:** Accuracy, completeness, consistency, timeliness, validity, and uniqueness form the foundation of data quality assessment.
- **ML-Specific Requirements:** Beyond traditional data quality, ML systems require monitoring feature distributions, label quality, data freshness, representation

balance, and correlation stability.

- **Comprehensive Frameworks:** Implementing structured validation frameworks with multiple severity levels enables systematic quality assessment and issue prioritization.
- **Great Expectations:** This powerful library provides declarative validation rules, automated documentation, and integration with data pipelines for consistent quality checks.
- **Data Profiling:** Automated profiling establishes baselines, tracks changes over time, and enables drift detection through statistical analysis.
- **Anomaly Detection:** Multiple methods (Z-score, IQR, Isolation Forest) identify outliers and unusual patterns that may indicate data quality issues.
- **Continuous Monitoring:** Automated monitoring pipelines with alerting ensure early detection of data quality degradation in production.
- **Custom ML Validations:** ML-specific checks including distribution tests, class balance validation, correlation analysis, and leakage detection prevent common ML failures.
- **Pipeline Integration:** Integrating data quality checks into orchestration platforms ensures systematic validation at every stage of the ML workflow.

Data quality is not a one-time check but an ongoing process requiring continuous monitoring, validation, and improvement. Investing in robust data quality infrastructure pays dividends through more reliable models, faster debugging, and greater trust in ML systems.

5.11 Exercises

Exercise 5.1. Implement Comprehensive Data Validation Suite

You have been tasked with building a data quality validation suite for a customer churn prediction model. The dataset contains the following columns:

- **customer_id:** Unique identifier
- **age:** Customer age (18-100)
- **tenure_months:** Months as customer (0-120)
- **monthly_charges:** Monthly bill amount (\$0-\$500)
- **total_charges:** Lifetime charges
- **contract_type:** Month-to-month, One year, Two year
- **payment_method:** Credit card, Bank transfer, Electronic check, Mailed check
- **churn:** Target variable (0 or 1)

Tasks:

1. Create a comprehensive validation configuration using `DataQualityValidator`
2. Define validity rules for all columns
3. Set up consistency checks (e.g., `total_charges` should be approximately `monthly_charges * tenure_months`)
4. Implement range checks for numeric columns
5. Add uniqueness checks for `customer_id`
6. Validate class balance for the target variable

Solution:

```

1  import pandas as pd
2  import numpy as np
3
4  # Create sample dataset with issues
5  np.random.seed(42)
6  n_samples = 1000
7
8  df = pd.DataFrame({
9      'customer_id': list(range(n_samples)) + [5, 10], # Duplicates
10     'age': np.random.randint(18, 100, n_samples + 2),
11     'tenure_months': np.random.randint(0, 121, n_samples + 2),
12     'monthly_charges': np.random.uniform(20, 500, n_samples + 2),
13     'total_charges': np.random.uniform(100, 10000, n_samples + 2),
14     'contract_type': np.random.choice(
15         ['Month-to-month', 'One year', 'Two year'],
16         n_samples + 2
17     ),
18     'payment_method': np.random.choice(
19         ['Credit card', 'Bank transfer', 'Electronic check', 'Mailed check'],
20         n_samples + 2
21     ),
22     'churn': np.random.choice([0, 1], n_samples + 2, p=[0.73, 0.27])
23 })
24
25 # Introduce data quality issues
26 df.loc[10:20, 'age'] = -1 # Invalid ages
27 df.loc[50:60, 'monthly_charges'] = None # Missing values
28 df.loc[100, 'contract_type'] = 'Invalid' # Invalid category
29
30 # Define validation configuration
31 validation_config = {
32     'completeness': {
33         'critical_columns': ['customer_id', 'age', 'churn'],
34         'threshold': 0.95
35     },
36     'uniqueness': {
37         'id_columns': ['customer_id']
38     },
39     'validity': {
40         'rules': {
41             'age': lambda x: pd.notna(x) and 18 <= x <= 100,

```

```

42     'tenure_months': lambda x: pd.notna(x) and 0 <= x <= 120,
43     'monthly_charges': lambda x: pd.notna(x) and 0 <= x <= 500,
44     'contract_type': lambda x: x in ['Month-to-month', 'One year', 'Two year'
],
45     'payment_method': lambda x: x in [
46         'Credit card', 'Bank transfer', 'Electronic check', 'Mailed check'
47     ],
48     'churn': lambda x: x in [0, 1]
49 },
50     'threshold': 0.98
51 },
52     'consistency': {
53         'rules': [
54             {
55                 'name': 'charges_consistency',
56                 'condition': lambda row: (
57                     pd.isna(row['total_charges']) or
58                     pd.isna(row['monthly_charges']) or
59                     pd.isna(row['tenure_months']) or
60                     row['tenure_months'] == 0 or
61                     abs(row['total_charges'] -
62                         row['monthly_charges'] * row['tenure_months']) /
63                     row['total_charges'] < 0.3 # Allow 30% variation
64                 ),
65                 'severity': Severity.WARNING
66             }
67         ]
68     },
69     'ranges': {
70         'specs': {
71             'age': {'min': 18, 'max': 100},
72             'tenure_months': {'min': 0, 'max': 120},
73             'monthly_charges': {'min': 0, 'max': 500},
74             'total_charges': {'min': 0, 'max': 60000}
75         }
76     }
77 }
78
79 # Run validation
80 validator = DataQualityValidator(name="Churn Data Validator")
81 results = validator.validate(df, validation_config)
82
83 # Print report
84 validator.print_report()
85
86 # Additional ML-specific validations
87 ml_validator = MLDataValidator()
88
89 # Check class balance
90 balance_results = ml_validator.validate_class_balance(
91     df=df,
92     target_column='churn',
93     min_class_frequency=0.05,
94     max_imbalance_ratio=20.0
95 )
96
97 print(f"\n\nClass Balance Validation:")
98 print(f"  Status: {'[OK] Passed' if balance_results['passed'] else '[X] Failed'}")

```

```

99 print(f"  Distribution: {balance_results['class_distribution']}")
100 print(f"  Imbalance Ratio: {balance_results['imbalance_ratio']:.2f}")

```

Exercise 5.2. Build Anomaly Detection System

Implement a multi-method anomaly detection system for a fraud detection dataset with transaction features: amount, time, merchant_category, and location_distance.

Tasks:

1. Apply three different anomaly detection methods
2. Compare results across methods
3. Implement ensemble voting (flag as anomaly if 2+ methods agree)
4. Analyze which features contribute most to anomalies
5. Create visualization of anomaly distribution

Solution:

```

1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # Create sample transaction dataset
6 np.random.seed(42)
7 n_normal = 950
8 n_fraud = 50
9
10 # Normal transactions
11 normal_data = {
12     'amount': np.random.lognormal(mean=3, sigma=1, size=n_normal),
13     'time_since_last': np.random.exponential(scale=24, size=n_normal),
14     'merchant_category': np.random.randint(1, 20, n_normal),
15     'location_distance': np.random.gamma(shape=2, scale=5, size=n_normal)
16 }
17
18 # Fraudulent transactions (anomalous patterns)
19 fraud_data = {
20     'amount': np.random.lognormal(mean=5, sigma=2, size=n_fraud), # Higher amounts
21     'time_since_last': np.random.exponential(scale=1, size=n_fraud), # Rapid
22     'merchant_category': np.random.randint(1, 20, n_fraud),
23     'location_distance': np.random.gamma(shape=5, scale=20, size=n_fraud) # Far
24     distances
25 }
26
27 # Combine datasets
28 df = pd.DataFrame({
29     col: np.concatenate([normal_data[col], fraud_data[col]])
30     for col in normal_data.keys()
31 })
32 df['is_fraud'] = [0] * n_normal + [1] * n_fraud
33 df = df.sample(frac=1).reset_index(drop=True) # Shuffle
34
35 # Initialize detector
36 detector = AnomalyDetector()

```



```

36
37 # Method 1: Z-score
38 result_zscore = detector.detect_anomalies(
39     df,
40     method='zscore',
41     columns=['amount', 'time_since_last', 'location_distance'],
42     threshold=3.0
43 )
44
45 # Method 2: IQR
46 result_iqr = detector.detect_anomalies(
47     df,
48     method='iqr',
49     columns=['amount', 'time_since_last', 'location_distance'],
50     multiplier=1.5
51 )
52
53 # Method 3: Isolation Forest
54 result_if = detector.detect_anomalies(
55     df,
56     method='isolation_forest',
57     columns=['amount', 'time_since_last', 'location_distance'],
58     contamination=0.05
59 )
60
61 # Ensemble voting
62 df['zscore_anomaly'] = result_zscore['is_anomaly']
63 df['iqr_anomaly'] = result_iqr['is_anomaly']
64 df['if_anomaly'] = result_if['is_anomaly']
65
66 df['ensemble_votes'] = (
67     df['zscore_anomaly'].astype(int) +
68     df['iqr_anomaly'].astype(int) +
69     df['if_anomaly'].astype(int)
70 )
71
72 df['ensemble_anomaly'] = df['ensemble_votes'] >= 2
73
74 # Analyze results
75 from sklearn.metrics import precision_score, recall_score, f1_score
76
77 methods = {
78     'Z-Score': 'zscore_anomaly',
79     'IQR': 'iqr_anomaly',
80     'Isolation Forest': 'if_anomaly',
81     'Ensemble': 'ensemble_anomaly'
82 }
83
84 print("Anomaly Detection Performance:")
85 print("=" * 60)
86 for method_name, col in methods.items():
87     precision = precision_score(df['is_fraud'], df[col])
88     recall = recall_score(df['is_fraud'], df[col])
89     f1 = f1_score(df['is_fraud'], df[col])
90
91     print(f"\n{method_name}:")
92     print(f"    Precision: {precision:.3f}")
93     print(f"    Recall: {recall:.3f}")

```

```

94     print(f"    F1-Score: {f1:.3f}")
95     print(f"    Detected: {df[col].sum()} anomalies")
96
97     # Feature contribution analysis
98     print("\n\nFeature Contribution to Anomalies:")
99     print("=" * 60)
100    for feature in ['amount', 'time_since_last', 'location_distance']:
101        anomaly_col = f'{feature}_anomaly'
102        if anomaly_col in result_zscore.columns:
103            contribution = result_zscore[anomaly_col].sum()
104            print(f"{feature}: {contribution} anomalies ({contribution/len(df):.2%}")
105
106    # Visualization
107    fig, axes = plt.subplots(2, 2, figsize=(14, 10))
108    fig.suptitle('Anomaly Detection Comparison', fontsize=16)
109
110    for idx, (method_name, col) in enumerate(methods.items()):
111        ax = axes[idx // 2, idx % 2]
112
113        # Scatter plot: amount vs time_since_last
114        normal_mask = ~df[col]
115        anomaly_mask = df[col]
116
117        ax.scatter(
118            df.loc[normal_mask, 'amount'],
119            df.loc[normal_mask, 'time_since_last'],
120            alpha=0.5, label='Normal', s=20
121        )
122        ax.scatter(
123            df.loc[anomaly_mask, 'amount'],
124            df.loc[anomaly_mask, 'time_since_last'],
125            color='red', alpha=0.7, label='Anomaly', s=50, marker='x'
126        )
127
128        ax.set_xlabel('Amount')
129        ax.set_ylabel('Time Since Last')
130        ax.set_title(method_name)
131        ax.legend()
132        ax.grid(True, alpha=0.3)
133
134    plt.tight_layout()
135    plt.savefig('anomaly_detection_comparison.png', dpi=300, bbox_inches='tight')
136    print("\n\nVisualization saved to 'anomaly_detection_comparison.png'")

```

Exercise 5.3. Create Data Drift Monitoring System

Build a system to monitor data drift for a recommender system that tracks user behavior features over time.

Tasks:

1. Generate baseline and drifted datasets
2. Implement statistical drift detection (KS test, PSI)
3. Calculate Population Stability Index (PSI) for features
4. Create drift alerting thresholds

5. Visualize drift over time

Solution:

```

1 import pandas as pd
2 import numpy as np
3 from scipy import stats
4 import matplotlib.pyplot as plt
5
6 def calculate_psi(baseline, current, bins=10):
7     """
8     Calculate Population Stability Index
9
10    PSI < 0.1: No significant change
11    0.1 <= PSI < 0.2: Moderate change
12    PSI >= 0.2: Significant change
13    """
14    # Create bins based on baseline
15    breakpoints = np.percentile(baseline, np.linspace(0, 100, bins + 1))
16    breakpoints[-1] += 0.0001 # Ensure last bin includes max value
17
18    # Calculate distribution for baseline
19    baseline_dist = np.histogram(baseline, bins=breakpoints)[0] / len(baseline)
20
21    # Calculate distribution for current
22    current_dist = np.histogram(current, bins=breakpoints)[0] / len(current)
23
24    # Avoid division by zero
25    baseline_dist = np.where(baseline_dist == 0, 0.0001, baseline_dist)
26    current_dist = np.where(current_dist == 0, 0.0001, current_dist)
27
28    # Calculate PSI
29    psi = np.sum((current_dist - baseline_dist) * np.log(current_dist / baseline_dist)
30    )
31
32    return psi
33
34 # Generate baseline data (Week 1)
35 np.random.seed(42)
36 n_baseline = 10000
37
38 baseline_df = pd.DataFrame({
39     'session_duration': np.random.lognormal(mean=4, sigma=1, size=n_baseline),
40     'items_viewed': np.random.poisson(lam=15, size=n_baseline),
41     'clicks_per_session': np.random.poisson(lam=5, size=n_baseline),
42     'time_on_page': np.random.gamma(shape=2, scale=30, size=n_baseline),
43     'week': 1
44 })
45
46 # Generate data with increasing drift over 8 weeks
47 drift_data = []
48
49 for week in range(2, 9):
50     drift_factor = (week - 1) * 0.05 # Increasing drift
51
52     week_df = pd.DataFrame({
53         'session_duration': np.random.lognormal(
54             mean=4 + drift_factor, # Increasing mean
55             sigma=1 + drift_factor * 0.5, # Increasing variance

```

```

55         size=n_baseline
56     ),
57     'items_viewed': np.random.poisson(
58         lam=15 - drift_factor * 3, # Decreasing engagement
59         size=n_baseline
60     ),
61     'clicks_per_session': np.random.poisson(
62         lam=5 - drift_factor * 2, # Decreasing clicks
63         size=n_baseline
64     ),
65     'time_on_page': np.random.gamma(
66         shape=2,
67         scale=30 * (1 + drift_factor), # Increasing time
68         size=n_baseline
69     ),
70     'week': week
71 })
72
73 drift_data.append(week_df)
74
75 all_data = pd.concat([baseline_df] + drift_data, ignore_index=True)
76
77 # Monitor drift for each week
78 features = ['session_duration', 'items_viewed', 'clicks_per_session', 'time_on_page']
79
80 drift_results = []
81
82 for week in range(2, 9):
83     week_data = all_data[all_data['week'] == week]
84
85     week_result = {'week': week}
86
87     for feature in features:
88         baseline_feature = baseline_df[feature]
89         current_feature = week_data[feature]
90
91         # KS test
92         ks_stat, ks_pvalue = stats.ks_2samp(baseline_feature, current_feature)
93
94         # PSI
95         psi = calculate_psi(baseline_feature, current_feature)
96
97         # Mean and std comparison
98         baseline_mean = baseline_feature.mean()
99         current_mean = current_feature.mean()
100         mean_diff_pct = abs(current_mean - baseline_mean) / baseline_mean
101
102         week_result[f'{feature}_ks_stat'] = ks_stat
103         week_result[f'{feature}_ks_pvalue'] = ks_pvalue
104         week_result[f'{feature}_psi'] = psi
105         week_result[f'{feature}_mean_diff'] = mean_diff_pct
106
107     drift_results.append(week_result)
108
109 drift_df = pd.DataFrame(drift_results)
110
111 # Define alert thresholds
112 PSI_THRESHOLD_MODERATE = 0.1

```

```

113 PSI_THRESHOLD_SIGNIFICANT = 0.2
114 KS_PVALUE_THRESHOLD = 0.05
115
116 # Generate drift alerts
117 print("Data Drift Monitoring Report")
118 print("=" * 80)
119
120 for idx, row in drift_df.iterrows():
121     week = int(row['week'])
122     print(f"\nWeek {week}:")
123
124     has_drift = False
125
126     for feature in features:
127         psi = row[f'{feature}_psi']
128         ks_pvalue = row[f'{feature}_ks_pvalue']
129         mean_diff = row[f'{feature}_mean_diff']
130
131         if psi >= PSI_THRESHOLD_SIGNIFICANT:
132             print(f"  [!!] CRITICAL: {feature} - Significant drift (PSI={psi:.3f})")
133             has_drift = True
134         elif psi >= PSI_THRESHOLD_MODERATE:
135             print(f"  [!] WARNING: {feature} - Moderate drift (PSI={psi:.3f})")
136             has_drift = True
137         elif ks_pvalue < KS_PVALUE_THRESHOLD:
138             print(f"  [!] WARNING: {feature} - Distribution change (KS p={ks_pvalue:.4f})")
139             has_drift = True
140
141     if not has_drift:
142         print("  [OK] No significant drift detected")
143
144 # Visualization
145 fig, axes = plt.subplots(2, 2, figsize=(14, 10))
146 fig.suptitle('Feature Drift Over Time (PSI)', fontsize=16)
147
148 for idx, feature in enumerate(features):
149     ax = axes[idx // 2, idx % 2]
150
151     psi_values = drift_df[f'{feature}_psi'].values
152     weeks = drift_df['week'].values
153
154     ax.plot(weeks, psi_values, marker='o', linewidth=2, markersize=8)
155     ax.axhline(y=PSI_THRESHOLD_MODERATE, color='orange',
156               linestyle='--', label='Moderate Threshold')
157     ax.axhline(y=PSI_THRESHOLD_SIGNIFICANT, color='red',
158               linestyle='--', label='Significant Threshold')
159
160     ax.set_xlabel('Week')
161     ax.set_ylabel('PSI')
162     ax.set_title(feature.replace('_', ' ').title())
163     ax.legend()
164     ax.grid(True, alpha=0.3)
165
166 plt.tight_layout()
167 plt.savefig('drift_monitoring.png', dpi=300, bbox_inches='tight')
168 print("\n\nVisualization saved to 'drift_monitoring.png'")
169

```

```

170 # Summary statistics
171 print("\n\nDrift Summary:")
172 print("=" * 80)
173 for feature in features:
174     max_psi = drift_df[f'{feature}_psi'].max()
175     max_week = drift_df.loc[drift_df[f'{feature}_psi'].idxmax(), 'week']
176     print(f"{feature}:")
177     print(f"    Maximum PSI: {max_psi:.3f} (Week {int(max_week)})")
178
179     if max_psi >= PSI_THRESHOLD_SIGNIFICANT:
180         print(f"    Status: [!!] CRITICAL - Requires immediate attention")
181     elif max_psi >= PSI_THRESHOLD_MODERATE:
182         print(f"    Status: [!] WARNING - Monitor closely")
183     else:
184         print(f"    Status: [OK] STABLE")

```

Exercise 5.4. Implement End-to-End Data Quality Pipeline

Create a complete data quality pipeline for a house price prediction model that includes profiling, validation, anomaly detection, and generates actionable reports.

Tasks:

1. Build integrated pipeline using DataQualityPipeline
2. Configure all quality checks
3. Implement automated reporting
4. Add failure handling and recovery
5. Create quality dashboard

Solution:

```

1 import pandas as pd
2 import numpy as np
3 from datetime import datetime
4 import json
5
6 # Generate sample housing data
7 np.random.seed(42)
8 n_samples = 1000
9
10 baseline_df = pd.DataFrame({
11     'property_id': range(n_samples),
12     'square_feet': np.random.normal(2000, 500, n_samples),
13     'bedrooms': np.random.choice([1, 2, 3, 4, 5], n_samples, p=[0.1, 0.2, 0.35, 0.25, 0.1]),
14     'bathrooms': np.random.choice([1, 1.5, 2, 2.5, 3], n_samples),
15     'age_years': np.random.randint(0, 100, n_samples),
16     'lot_size': np.random.lognormal(8.5, 0.5, n_samples),
17     'garage_spaces': np.random.choice([0, 1, 2, 3], n_samples, p=[0.1, 0.4, 0.4, 0.1])
18 },
19     'neighborhood': np.random.choice(['A', 'B', 'C', 'D', 'E'], n_samples),
20     'price': np.random.lognormal(12.5, 0.4, n_samples)
21 })
22
23 # Configure pipeline

```

```

23 config = PipelineConfig(
24     enable_profiling=True,
25     enable_validation=True,
26     enable_anomaly_detection=True,
27     enable_drift_detection=True,
28     fail_on_error=True,
29     fail_on_warning=False
30 )
31
32 # Initialize pipeline
33 pipeline = DataQualityPipeline(config, name="HousePricePipeline")
34
35 # Define validation rules
36 validation_config = {
37     'completeness': {
38         'critical_columns': ['property_id', 'square_feet', 'price'],
39         'threshold': 0.98
40     },
41     'uniqueness': {
42         'id_columns': ['property_id']
43     },
44     'validity': {
45         'rules': {
46             'square_feet': lambda x: 500 <= x <= 10000,
47             'bedrooms': lambda x: x in [1, 2, 3, 4, 5],
48             'bathrooms': lambda x: x in [1, 1.5, 2, 2.5, 3, 3.5, 4],
49             'age_years': lambda x: 0 <= x <= 200,
50             'garage_spaces': lambda x: x in [0, 1, 2, 3, 4],
51             'neighborhood': lambda x: x in ['A', 'B', 'C', 'D', 'E'],
52             'price': lambda x: x > 0
53         },
54         'threshold': 0.95
55     },
56     'ranges': {
57         'specs': {
58             'square_feet': {'min': 500, 'max': 10000},
59             'age_years': {'min': 0, 'max': 200},
60             'price': {'min': 10000, 'max': 10000000}
61         }
62     },
63     'consistency': {
64         'rules': [
65             {
66                 'name': 'bedrooms_bathrooms_ratio',
67                 'condition': lambda row: row['bedrooms'] >= row['bathrooms'] - 1,
68                 'severity': Severity.WARNING
69             }
70         ]
71     }
72 }
73
74 # Run baseline
75 print("=" * 80)
76 print("ESTABLISHING BASELINE")
77 print("=" * 80)
78
79 baseline_results = pipeline.run(
80     df=baseline_df,

```

```

81     validation_config=validation_config,
82     batch_id='baseline_001',
83     is_baseline=True
84 )
85
86 print(pipeline.get_pipeline_report(baseline_results))
87
88 # Save baseline profile
89 with open('baseline_profile.json', 'w') as f:
90     json.dump(baseline_results['profile'], f, indent=2)
91
92 print("\n[OK] Baseline profile saved to 'baseline_profile.json'")
93
94 # Generate new batch with various quality issues
95 new_df = pd.DataFrame({
96     'property_id': range(500, 1500),
97     'square_feet': np.random.normal(2100, 550, 1000), # Drift in mean/std
98     'bedrooms': np.random.choice([1, 2, 3, 4, 5], 1000, p=[0.15, 0.25, 0.3, 0.2, 0.1])
99     ,
100     'bathrooms': np.random.choice([1, 1.5, 2, 2.5, 3], 1000),
101     'age_years': np.random.randint(0, 100, 1000),
102     'lot_size': np.random.lognormal(8.6, 0.5, 1000), # Slight drift
103     'garage_spaces': np.random.choice([0, 1, 2, 3], 1000, p=[0.1, 0.4, 0.4, 0.1]),
104     'neighborhood': np.random.choice(['A', 'B', 'C', 'D', 'E'], 1000),
105     'price': np.random.lognormal(12.6, 0.45, 1000) # Drift in price
106 })
107
108 # Introduce quality issues
109 new_df.loc[10:15, 'property_id'] = 500 # Duplicates
110 new_df.loc[20:30, 'square_feet'] = None # Missing values
111 new_df.loc[40, 'square_feet'] = 50000 # Outlier
112 new_df.loc[50, 'age_years'] = -10 # Invalid value
113 new_df.loc[60, 'neighborhood'] = 'F' # Invalid category
114
115 # Run pipeline on new batch
116 print("\n\n" + "=" * 80)
117 print("VALIDATING NEW BATCH")
118 print("=" * 80)
119
120 new_results = pipeline.run(
121     df=new_df,
122     validation_config=validation_config,
123     batch_id='batch_002'
124 )
125
126 print(pipeline.get_pipeline_report(new_results))
127
128 # Decision making based on results
129 print("\n\n" + "=" * 80)
130 print("PIPELINE DECISION")
131 print("=" * 80)
132
133 if new_results['passed']:
134     print("[OK] Data quality checks PASSED")
135     print("Recommendation: Proceed with model training")
136 else:
137     print("[X] Data quality checks FAILED")
138     print("Recommendation: Review and fix issues before proceeding")

```



```

138
139     if new_results.get('errors'):
140         print(f"\n Critical Issues ({len(new_results['errors'])}):")
141         for error in new_results['errors'][:5]:
142             print(f"    - {error}")
143
144     if new_results.get('warnings'):
145         print(f"\n Warnings ({len(new_results['warnings'])}):")
146         for warning in new_results['warnings'][:5]:
147             print(f"    - {warning}")
148
149     # Create quality scorecard
150     dashboard = DataQualityDashboard(pipeline.monitor)
151     scorecard = dashboard.create_quality_scorecard()
152
153     print("\n\n" + "=" * 80)
154     print("DATA QUALITY SCORECARD")
155     print("=" * 80)
156     print(f"Overall Score: {scorecard['data_quality_score']:.1f}/100")
157     print(f"Status: {scorecard['status']}")
158
159     print("\nMetrics:")
160     for metric, value in scorecard['metrics'].items():
161         if isinstance(value, float):
162             print(f"    {metric}: {value:.4f}")
163         else:
164             print(f"    {metric}: {value}")
165
166     print("\nRecommendations:")
167     for i, rec in enumerate(scorecard['recommendations'], 1):
168         print(f"    {i}. {rec}")
169
170     # Save results
171     with open('validation_results.json', 'w') as f:
172         # Convert non-serializable objects
173         results_to_save = {
174             'batch_id': new_results['batch_id'],
175             'status': new_results['status'],
176             'passed': new_results['passed'],
177             'errors': new_results['errors'],
178             'warnings': new_results['warnings'],
179             'row_count': new_results['row_count'],
180             'column_count': new_results['column_count']
181         }
182         json.dump(results_to_save, f, indent=2)
183
184     print("\n[OK] Results saved to 'validation_results.json'")

```


Chapter 6

Feature Engineering and Feature Stores

6.1 Learning Objectives

After completing this chapter, readers will be able to:

- Understand the role of feature stores in ML systems and the training-serving skew problem
- Apply feature engineering best practices and design patterns
- Implement a complete feature store using Feast with online and offline serving
- Design feature versioning strategies and track feature lineage
- Build architectures for online and offline feature serving
- Implement feature monitoring and drift detection systems
- Integrate feature stores with training and serving pipelines
- Apply real-world feature engineering patterns from industry case studies

6.2 Introduction: The Feature Engineering Challenge

Feature stores have emerged as a critical component in production ML systems, addressing one of the most persistent challenges in machine learning: **training-serving skew**. This phenomenon occurs when features computed during training differ from those generated during inference, leading to degraded model performance despite strong offline metrics.

Consider a recommendation system that uses a user's average purchase value over the last 30 days as a feature. During training, this feature is computed in batch using SQL queries over historical data. During serving, the same feature must be computed in real-time from a streaming data source. Without careful engineering, these two pipelines can diverge due to:

- Different computation logic (SQL vs. Python)
- Different data sources (data warehouse vs. production database)

- Different aggregation windows (calendar days vs. rolling windows)
- Different handling of missing values or edge cases
- Time zone differences or timestamp precision issues

A feature store solves this by providing:

1. **Centralized Feature Definitions:** Single source of truth for feature logic
2. **Dual-Mode Serving:** Consistent feature computation for training (batch) and serving (online)
3. **Feature Versioning:** Track changes to feature definitions over time
4. **Feature Discovery:** Catalog of available features with metadata
5. **Point-in-Time Correctness:** Prevent data leakage by respecting temporal boundaries

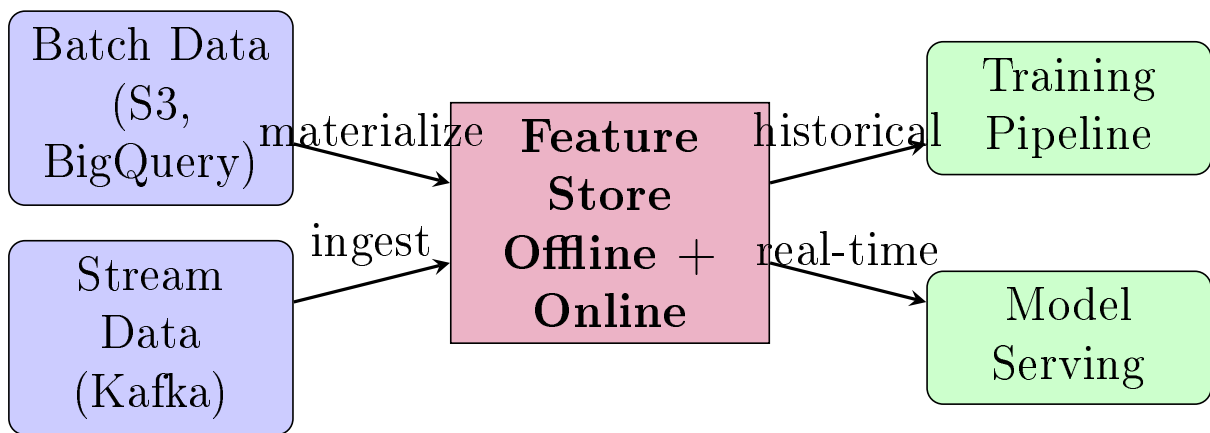


Figure 6.1: Feature store architecture overview

6.3 Feature Engineering Best Practices

Effective feature engineering requires both art and science. This section presents battle-tested patterns for creating robust, maintainable features.

6.3.1 Feature Design Principles

1. Reproducibility: Features must produce identical results given the same input data and timestamp. This requires:

- Deterministic transformations (avoid random operations)
- Explicit timestamp handling (no implicit "now()")

- Versioned dependencies (libraries, models)
- Documented assumptions

2. Modularity: Break complex features into composable primitives.

```

1 from typing import List, Dict
2 import pandas as pd
3 from dataclasses import dataclass
4 from datetime import datetime, timedelta
5
6 @dataclass
7 class FeatureConfig:
8     """Configuration for feature computation"""
9     window_days: int
10    aggregation: str
11    min_samples: int = 1
12
13 class FeaturePrimitive:
14     """Base class for reusable feature primitives"""
15     def __init__(self, config: FeatureConfig):
16         self.config = config
17
18     def compute(self, df: pd.DataFrame, timestamp: datetime) -> pd.Series:
19         """Compute feature at specific timestamp"""
20         raise NotImplementedError
21
22 class WindowAggregation(FeaturePrimitive):
23     """Aggregate values over time window"""
24     def compute(self, df: pd.DataFrame, timestamp: datetime) -> pd.Series:
25         # Point-in-time filtering
26         cutoff = timestamp - timedelta(days=self.config.window_days)
27         windowed = df[
28             (df['timestamp'] >= cutoff) &
29             (df['timestamp'] < timestamp)
30         ]
31
32         # Apply aggregation
33         if self.config.aggregation == 'mean':
34             return windowed.groupby('entity_id')['value'].mean()
35         elif self.config.aggregation == 'sum':
36             return windowed.groupby('entity_id')['value'].sum()
37         elif self.config.aggregation == 'count':
38             return windowed.groupby('entity_id')['value'].count()
39         else:
40             raise ValueError(f"Unknown aggregation: {self.config.aggregation}")
41
42 class RatioFeature(FeaturePrimitive):
43     """Compute ratio between two aggregated features"""
44     def __init__(self, numerator: FeaturePrimitive,
45                 denominator: FeaturePrimitive):
46         self.numerator = numerator
47         self.denominator = denominator
48
49     def compute(self, df: pd.DataFrame, timestamp: datetime) -> pd.Series:
50         num = self.numerator.compute(df, timestamp)
51         denom = self.denominator.compute(df, timestamp)
52         # Handle division by zero

```

```

53     return (num / denom.replace(0, float('nan'))).fillna(0)
54
55     # Example: Conversion rate feature
56     purchase_count = WindowAggregation(
57         FeatureConfig(window_days=30, aggregation='count')
58     )
59     pageview_count = WindowAggregation(
60         FeatureConfig(window_days=30, aggregation='count')
61     )
62     conversion_rate = RatioFeature(purchase_count, pageview_count)

```

Listing 6.1: Modular feature composition

3. Point-in-Time Correctness: Prevent data leakage by ensuring features only use data available at prediction time.

```

1  import pandas as pd
2  from typing import Optional
3
4  class PointInTimeCorrectFeatures:
5      """Ensures no data leakage in feature computation"""
6
7      def __init__(self, entity_df: pd.DataFrame,
8                  event_df: pd.DataFrame):
9
10         """
11         Args:
12         entity_df: DataFrame with entity_id and timestamp columns
13         event_df: DataFrame with entity_id, event_timestamp, and features
14         """
15         self.entity_df = entity_df.sort_values('timestamp')
16         self.event_df = event_df.sort_values('event_timestamp')
17
18     def get_features(self, feature_cols: List[str]) -> pd.DataFrame:
19         """
20         Get features at each entity timestamp without leakage.
21         Uses merge_asof to join on nearest earlier event.
22         """
23
24         # Rename for merge_asof
25         entity = self.entity_df.copy()
26         events = self.event_df[['entity_id', 'event_timestamp'] + feature_cols].copy()
27
28         # Merge as-of join: for each entity timestamp,
29         # get latest feature values from before that time
30         result = pd.merge_asof(
31             entity.sort_values('timestamp'),
32             events.sort_values('event_timestamp'),
33             left_on='timestamp',
34             right_on='event_timestamp',
35             by='entity_id',
36             direction='backward', # Only look backward in time
37             allow_exact_matches=False # Exclude events at exact timestamp
38         )
39
40         return result[['entity_id', 'timestamp'] + feature_cols]
41
42     # Example usage
43     entity_df = pd.DataFrame({
44         'entity_id': ['user_1', 'user_1', 'user_2'],
45         'timestamp': pd.to_datetime([

```

```

44         '2024-01-15 10:00:00',
45         '2024-01-20 14:00:00',
46         '2024-01-18 09:00:00'
47     ])
48 })
49
50 event_df = pd.DataFrame({
51     'entity_id': ['user_1', 'user_1', 'user_2'],
52     'event_timestamp': pd.to_datetime([
53         '2024-01-10 08:00:00',
54         '2024-01-17 12:00:00',
55         '2024-01-16 11:00:00'
56     ]),
57     'total_purchases': [5, 7, 3],
58     'avg_order_value': [45.2, 52.1, 38.5]
59 })
60
61 pit_features = PointInTimeCorrectFeatures(entity_df, event_df)
62 result = pit_features.get_features(['total_purchases', 'avg_order_value'])
63 print(result)
64 # Ensures each row only uses data from BEFORE the timestamp

```

Listing 6.2: Point-in-time correct feature computation

6.3.2 Feature Transformation Patterns

Temporal Features:

```

1 import numpy as np
2 from typing import Union
3
4 class TemporalFeatureEngineering:
5     """Common patterns for time-based features"""
6
7     @staticmethod
8     def cyclical_encoding(values: pd.Series, period: int) -> pd.DataFrame:
9         """
10         Encode cyclical features (hour, day, month) as sin/cos.
11         Preserves cyclical nature (hour 23 is close to hour 0).
12         """
13         sin_values = np.sin(2 * np.pi * values / period)
14         cos_values = np.cos(2 * np.pi * values / period)
15         return pd.DataFrame({
16             f'{values.name}_sin': sin_values,
17             f'{values.name}_cos': cos_values
18         })
19
20     @staticmethod
21     def time_since_event(timestamps: pd.Series,
22                          event_timestamp: pd.Series,
23                          unit: str = 'days') -> pd.Series:
24         """
25         Calculate time elapsed since an event.
26         Useful for recency features.
27         """
28         delta = timestamps - event_timestamp
29         if unit == 'days':

```

```

30         return delta.dt.total_seconds() / 86400
31     elif unit == 'hours':
32         return delta.dt.total_seconds() / 3600
33     elif unit == 'minutes':
34         return delta.dt.total_seconds() / 60
35     else:
36         return delta.dt.total_seconds()
37
38     @staticmethod
39     def business_day_features(timestamps: pd.Series) -> pd.DataFrame:
40         """Extract business-relevant temporal features"""
41         df = pd.DataFrame()
42         df['is_weekend'] = timestamps.dt.dayofweek.isin([5, 6]).astype(int)
43         df['is_month_start'] = timestamps.dt.is_month_start.astype(int)
44         df['is_month_end'] = timestamps.dt.is_month_end.astype(int)
45         df['is_quarter_start'] = timestamps.dt.is_quarter_start.astype(int)
46         df['is_quarter_end'] = timestamps.dt.is_quarter_end.astype(int)
47         df['day_of_week'] = timestamps.dt.dayofweek
48         df['week_of_year'] = timestamps.dt.isocalendar().week
49         df['month'] = timestamps.dt.month
50         df['quarter'] = timestamps.dt.quarter
51         return df
52
53     # Example usage
54     df = pd.DataFrame({
55         'timestamp': pd.date_range('2024-01-01', periods=100, freq='6H')
56     })
57
58     tfe = TemporalFeatureEngineering()
59
60     # Cyclical encoding for hour of day
61     df['hour'] = df['timestamp'].dt.hour
62     hour_encoded = tfe.cyclical_encoding(df['hour'], period=24)
63
64     # Business day features
65     business_features = tfe.business_day_features(df['timestamp'])
66
67     print(hour_encoded.head())
68     print(business_features.head())

```

Listing 6.3: Temporal feature engineering patterns

Aggregation Features:

```

1 from typing import List, Callable, Dict
2 import pandas as pd
3
4 class AggregationFeatures:
5     """Patterns for aggregating transactional data into features"""
6
7     @staticmethod
8     def multi_window_aggregation(
9         df: pd.DataFrame,
10         entity_col: str,
11         value_col: str,
12         timestamp_col: str,
13         windows: List[int],
14         agg_funcs: List[str]
15     ) -> pd.DataFrame:

```



```

16     """
17     Create features with multiple time windows and aggregations.
18     Essential for capture temporal patterns.
19     """
20     features = []
21
22     for window in windows:
23         for agg in agg_funcs:
24             feature_name = f'{value_col}_{agg}_{window}d'
25
26             # Rolling aggregation
27             windowed = df.groupby(entity_col).rolling(
28                 window=f'{window}D',
29                 on=timestamp_col
30             )[value_col].agg(agg).reset_index()
31
32             windowed = windowed.rename(columns={value_col: feature_name})
33             features.append(windowed[[entity_col, timestamp_col, feature_name]])
34
35     # Merge all features
36     result = features[0]
37     for feat in features[1:]:
38         result = result.merge(feat, on=[entity_col, timestamp_col], how='outer')
39
40     return result
41
42 @staticmethod
43 def recency_frequency_monetary(
44     df: pd.DataFrame,
45     entity_col: str,
46     timestamp_col: str,
47     value_col: str,
48     as_of_date: pd.Timestamp
49 ) -> pd.DataFrame:
50     """
51     RFM analysis - classic pattern for customer segmentation.
52
53     Recency: Days since last transaction
54     Frequency: Number of transactions
55     Monetary: Total/average transaction value
56     """
57     rfm = df.groupby(entity_col).agg({
58         timestamp_col: lambda x: (as_of_date - x.max()).days, # Recency
59         value_col: ['count', 'sum', 'mean'] # Frequency, Monetary
60     })
61
62     rfm.columns = ['recency_days', 'frequency', 'monetary_total', 'monetary_avg']
63     return rfm.reset_index()
64
65 @staticmethod
66 def trend_features(
67     df: pd.DataFrame,
68     entity_col: str,
69     value_col: str,
70     timestamp_col: str,
71     short_window: int = 7,
72     long_window: int = 30
73 ) -> pd.DataFrame:

```

```

74     """
75     Compute trend features by comparing short vs long windows.
76     Useful for detecting changes in user behavior.
77     """
78     df_sorted = df.sort_values([entity_col, timestamp_col])
79
80     # Short window average
81     short_avg = df_sorted.groupby(entity_col)[value_col].rolling(
82         window=short_window, min_periods=1
83     ).mean().reset_index(level=0, drop=True)
84
85     # Long window average
86     long_avg = df_sorted.groupby(entity_col)[value_col].rolling(
87         window=long_window, min_periods=1
88     ).mean().reset_index(level=0, drop=True)
89
90     df_sorted['trend_ratio'] = short_avg / long_avg.replace(0, np.nan)
91     df_sorted['trend_diff'] = short_avg - long_avg
92     df_sorted['is_trending_up'] = (df_sorted['trend_ratio'] > 1.2).astype(int)
93     df_sorted['is_trending_down'] = (df_sorted['trend_ratio'] < 0.8).astype(int)
94
95     return df_sorted
96
97 # Example: User engagement features
98 user_events = pd.DataFrame({
99     'user_id': ['u1', 'u1', 'u1', 'u2', 'u2'] * 10,
100     'timestamp': pd.date_range('2024-01-01', periods=50, freq='1D'),
101     'purchase_amount': np.random.uniform(10, 100, 50)
102 })
103
104 agg_features = AggregationFeatures()
105
106 # Multi-window features
107 multi_window = agg_features.multi_window_aggregation(
108     user_events,
109     entity_col='user_id',
110     value_col='purchase_amount',
111     timestamp_col='timestamp',
112     windows=[7, 14, 30],
113     agg_funcs=['sum', 'mean', 'count']
114 )
115
116 # RFM features
117 rfm = agg_features.recency_frequency_monetary(
118     user_events,
119     entity_col='user_id',
120     timestamp_col='timestamp',
121     value_col='purchase_amount',
122     as_of_date=pd.Timestamp('2024-02-20')
123 )
124
125 print("Multi-window features:")
126 print(multi_window.head())
127 print("\nRFM features:")
128 print(rfm)

```

Listing 6.4: Advanced aggregation patterns for ML features

Categorical Feature Engineering:

```

1 from sklearn.preprocessing import LabelEncoder
2 from category_encoders import TargetEncoder, WOEEncoder
3 import warnings
4
5 class CategoricalFeatureEngineering:
6     """Enterprise patterns for categorical feature encoding"""
7
8     @staticmethod
9     def frequency_encoding(
10         train: pd.Series,
11         test: pd.Series = None,
12         smoothing: float = 1.0
13     ) -> tuple:
14         """
15         Encode categories by their frequency.
16         Smoothing prevents overfitting to rare categories.
17         """
18         # Calculate frequencies from training data
19         freq_map = train.value_counts(normalize=True).to_dict()
20
21         # Apply smoothing (additive smoothing)
22         total_categories = len(freq_map)
23         smoothed_freq = {
24             k: (v + smoothing) / (1 + smoothing * total_categories)
25             for k, v in freq_map.items()
26         }
27
28         # Default frequency for unseen categories
29         default_freq = smoothing / (1 + smoothing * total_categories)
30
31         train_encoded = train.map(smoothed_freq).fillna(default_freq)
32         test_encoded = test.map(smoothed_freq).fillna(default_freq) if test is not
None else None
33
34         return train_encoded, test_encoded
35
36     @staticmethod
37     def target_encoding_safe(
38         train_cat: pd.Series,
39         train_target: pd.Series,
40         test_cat: pd.Series = None,
41         smoothing: float = 10.0,
42         min_samples: int = 5
43     ) -> tuple:
44         """
45         Target encoding with proper cross-validation handling.
46         Uses smoothing to prevent overfitting.
47
48         Formula: (count_in_category * mean_target_in_category + smoothing *
global_mean) /
49                 (count_in_category + smoothing)
50         """
51         global_mean = train_target.mean()
52
53         # Aggregate by category
54         stats = pd.DataFrame({

```

```

55         'count': train_cat.groupby(train_cat).size(),
56         'mean': train_target.groupby(train_cat).mean()
57     })
58
59     # Apply smoothing formula
60     stats['smoothed_mean'] = (
61         (stats['count'] * stats['mean'] + smoothing * global_mean) /
62         (stats['count'] + smoothing)
63     )
64
65     # Handle rare categories
66     stats.loc[stats['count'] < min_samples, 'smoothed_mean'] = global_mean
67
68     # Map to data
69     encoding_map = stats['smoothed_mean'].to_dict()
70     train_encoded = train_cat.map(encoding_map).fillna(global_mean)
71     test_encoded = test_cat.map(encoding_map).fillna(global_mean) if test_cat is
not None else None
72
73     return train_encoded, test_encoded
74
75     @staticmethod
76     def entity_embedding_features(
77         categories: pd.Series,
78         embedding_dim: int = 10,
79         pretrained_embeddings: Dict[str, np.ndarray] = None
80     ) -> np.ndarray:
81         """
82         Use learned embeddings for high-cardinality categorical features.
83         Can use pretrained embeddings from neural networks.
84         """
85         if pretrained_embeddings is None:
86             # Initialize random embeddings (in practice, learn these)
87             unique_cats = categories.unique()
88             pretrained_embeddings = {
89                 cat: np.random.randn(embedding_dim)
90                 for cat in unique_cats
91             }
92
93             # Default embedding for unseen categories
94             default_embedding = np.zeros(embedding_dim)
95
96             # Map categories to embeddings
97             embeddings = categories.map(
98                 lambda x: pretrained_embeddings.get(x, default_embedding)
99             )
100
101             return np.vstack(embeddings.values)
102
103     # Example usage
104     np.random.seed(42)
105     train_data = pd.DataFrame({
106         'category': np.random.choice(['A', 'B', 'C', 'D', 'E'], 1000),
107         'target': np.random.binomial(1, 0.3, 1000)
108     })
109     test_data = pd.DataFrame({
110         'category': np.random.choice(['A', 'B', 'C', 'D', 'E', 'F'], 200) # F is unseen
111     })

```

```

112 cfe = CategoricalFeatureEngineering()
113
114 # Frequency encoding
115 train_freq, test_freq = cfe.frequency_encoding(
116     train_data['category'],
117     test_data['category']
118 )
119
120 # Target encoding
121 train_target, test_target = cfe.target_encoding_safe(
122     train_data['category'],
123     train_data['target'],
124     test_data['category']
125 )
126
127 print("Frequency encoding (first 10):")
128 print(train_freq.head(10))
129 print("\nTarget encoding (first 10):")
130 print(train_target.head(10))
131

```

Listing 6.5: Production-ready categorical feature encoding

6.4 Complete Feast Feature Store Implementation

Feast (Feature Store) is an open-source feature store that provides a unified data layer for ML features. This section demonstrates a production-ready implementation.

6.4.1 Feast Architecture and Setup

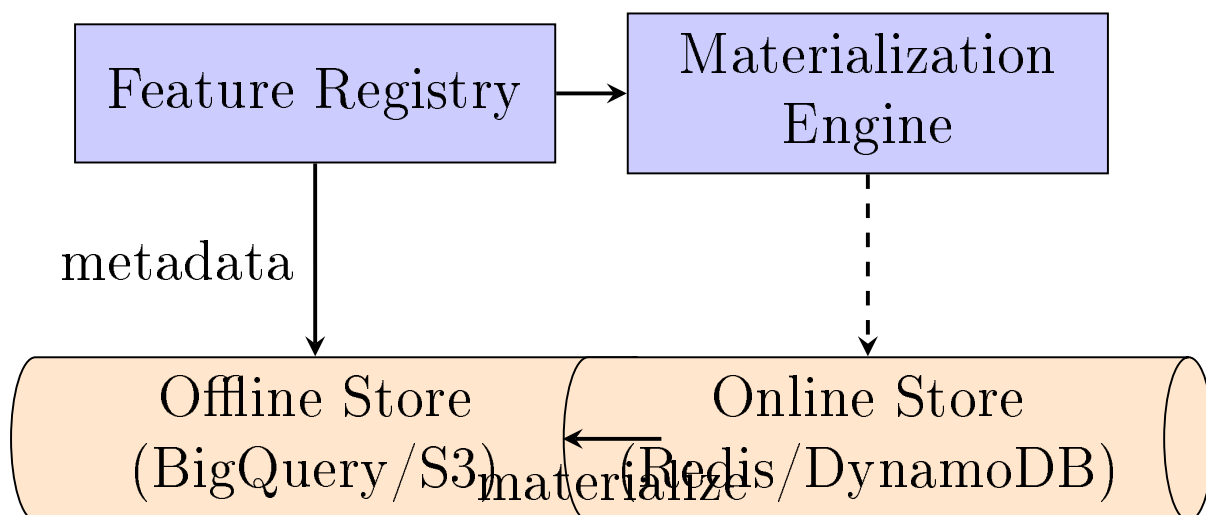


Figure 6.2: Feast feature store architecture

```

1 # feast_repo/feature_store.yaml
2 project: ml_production
3 registry: data/registry.db

```

```
4 provider: local
5 online_store:
6     type: redis
7     connection_string: "localhost:6379"
8 offline_store:
9     type: file
```

Listing 6.6: Feast feature store setup and configuration

```
1 # feast_repo/features.py
2 from feast import Entity, Feature, FeatureView, FileSource, ValueType
3 from feast.types import Float32, Float64, Int64, String
4 from datetime import timedelta
5 import pandas as pd
6
7 # Define entities
8 user = Entity(
9     name="user_id",
10    value_type=ValueType.STRING,
11    description="User identifier"
12 )
13
14 product = Entity(
15     name="product_id",
16     value_type=ValueType.STRING,
17     description="Product identifier"
18 )
19
20 # Define data sources
21 user_activity_source = FileSource(
22     path="data/user_activity.parquet",
23     timestamp_field="event_timestamp",
24     created_timestamp_column="created_timestamp"
25 )
26
27 user_profile_source = FileSource(
28     path="data/user_profile.parquet",
29     timestamp_field="event_timestamp",
30     created_timestamp_column="created_timestamp"
31 )
32
33 product_features_source = FileSource(
34     path="data/product_features.parquet",
35     timestamp_field="event_timestamp"
36 )
37
38 # Define feature views
39 user_activity_features = FeatureView(
40     name="user_activity_features",
41     entities=["user_id"],
42     ttl=timedelta(days=90),
43     schema=[
44         Feature(name="total_pageviews_7d", dtype=Float32),
45         Feature(name="total_pageviews_30d", dtype=Float32),
46         Feature(name="unique_products_viewed_7d", dtype=Int64),
47         Feature(name="unique_products_viewed_30d", dtype=Int64),
48         Feature(name="avg_session_duration_7d", dtype=Float32),
49         Feature(name="purchase_count_7d", dtype=Int64),
```

```

50     Feature(name="purchase_count_30d", dtype=Int64),
51     Feature(name="total_purchase_amount_30d", dtype=Float32),
52     Feature(name="avg_purchase_amount_30d", dtype=Float32),
53     Feature(name="days_since_last_purchase", dtype=Float32),
54     Feature(name="conversion_rate_30d", dtype=Float32),
55 ],
56 online=True,
57 source=user_activity_source,
58 tags={"team": "recommendations", "domain": "user_behavior"}
59 )
60
61 user_profile_features = FeatureView(
62     name="user_profile_features",
63     entities=["user_id"],
64     ttl=timedelta(days=365),
65     schema=[
66         Feature(name="account_age_days", dtype=Float32),
67         Feature(name="is_premium_member", dtype=Int64),
68         Feature(name="preferred_category", dtype=String),
69         Feature(name="lifetime_purchase_count", dtype=Int64),
70         Feature(name="lifetime_purchase_amount", dtype=Float32),
71         Feature(name="email_engagement_score", dtype=Float32),
72     ],
73     online=True,
74     source=user_profile_source,
75     tags={"team": "growth", "domain": "user_profile"}
76 )
77
78 product_features = FeatureView(
79     name="product_features",
80     entities=["product_id"],
81     ttl=timedelta(days=30),
82     schema=[
83         Feature(name="price", dtype=Float32),
84         Feature(name="discount_percentage", dtype=Float32),
85         Feature(name="rating_avg", dtype=Float32),
86         Feature(name="rating_count", dtype=Int64),
87         Feature(name="view_count_7d", dtype=Int64),
88         Feature(name="purchase_count_7d", dtype=Int64),
89         Feature(name="inventory_level", dtype=Int64),
90         Feature(name="category_popularity_score", dtype=Float32),
91     ],
92     online=True,
93     source=product_features_source,
94     tags={"team": "catalog", "domain": "product"}
95 )

```

Listing 6.7: Complete Feast feature definitions

6.4.2 Feature Materialization Pipeline

```

1  # feature_pipeline.py
2  """
3  Production pipeline for computing and materializing features to Feast.
4  Runs on schedule (e.g., daily) to keep offline and online stores updated.
5  """
6

```

```

7 import pandas as pd
8 import numpy as np
9 from datetime import datetime, timedelta
10 from feast import FeatureStore
11 from typing import Dict, List
12 import logging
13
14 logging.basicConfig(level=logging.INFO)
15 logger = logging.getLogger(__name__)
16
17 class FeatureMaterializationPipeline:
18     """
19     Pipeline to compute features from raw data and materialize to Feast.
20     """
21
22     def __init__(self, feast_repo_path: str):
23         self.fs = FeatureStore(repo_path=feast_repo_path)
24         self.computation_timestamp = datetime.now()
25
26     def compute_user_activity_features(
27         self,
28         events_df: pd.DataFrame
29     ) -> pd.DataFrame:
30         """
31         Compute user activity features from event log.
32
33         Args:
34             events_df: DataFrame with columns [user_id, event_timestamp,
35                                     event_type, product_id, session_duration, purchase_amount]
36         """
37         logger.info("Computing user activity features...")
38
39         # Ensure timestamp is datetime
40         events_df['event_timestamp'] = pd.to_datetime(events_df['event_timestamp'])
41
42         features_list = []
43
44         # Define windows
45         windows = {
46             '7d': timedelta(days=7),
47             '30d': timedelta(days=30)
48         }
49
50         # Compute features for each user
51         for user_id, user_events in events_df.groupby('user_id'):
52             user_features = {'user_id': user_id}
53
54             # Use current timestamp for point-in-time correctness
55             cutoff = self.computation_timestamp
56
57             for window_name, window_delta in windows.items():
58                 window_start = cutoff - window_delta
59                 windowed_events = user_events[
60                     (user_events['event_timestamp'] >= window_start) &
61                     (user_events['event_timestamp'] < cutoff)
62                 ]
63
64             # Pageview features

```



```

65         pageviews = windowed_events[windowed_events['event_type'] == 'pageview
66     ],
67     user_features[f'total_pageviews_{window_name}'] = len(pageviews)
68     user_features[f'unique_products_viewed_{window_name}'] = \
69         pageviews['product_id'].nunique()
70
71     # Session duration
72     user_features[f'avg_session_duration_{window_name}'] = \
73         windowed_events['session_duration'].mean() \
74         if len(windowed_events) > 0 else 0.0
75
76     # Purchase features
77     purchases = windowed_events[windowed_events['event_type'] == 'purchase
78     ],
79     user_features[f'purchase_count_{window_name}'] = len(purchases)
80
81     if window_name == '30d': # Only for 30-day window
82         user_features['total_purchase_amount_30d'] = \
83             purchases['purchase_amount'].sum()
84         user_features['avg_purchase_amount_30d'] = \
85             purchases['purchase_amount'].mean() \
86             if len(purchases) > 0 else 0.0
87
88     # Conversion rate
89     total_sessions = windowed_events['session_id'].nunique()
90     user_features['conversion_rate_30d'] = \
91         len(purchases) / total_sessions if total_sessions > 0 else 0.0
92
93     # Recency feature
94     last_purchase = user_events[
95         user_events['event_type'] == 'purchase'
96     ]['event_timestamp'].max()
97
98     if pd.notna(last_purchase):
99         user_features['days_since_last_purchase'] = \
100             (cutoff - last_purchase).total_seconds() / 86400
101     else:
102         user_features['days_since_last_purchase'] = 9999.0 # Never purchased
103
104     # Event timestamp for Feast
105     user_features['event_timestamp'] = cutoff
106     user_features['created_timestamp'] = datetime.now()
107
108     features_list.append(user_features)
109
110     result_df = pd.DataFrame(features_list)
111     logger.info(f"Computed features for {len(result_df)} users")
112
113     return result_df
114
115 def compute_product_features(
116     self,
117     products_df: pd.DataFrame,
118     events_df: pd.DataFrame
119 ) -> pd.DataFrame:
120     """
121     Compute product features from product catalog and events.
122     """

```

```

121     logger.info("Computing product features...")
122
123     features_list = []
124     cutoff = self.computation_timestamp
125     window_7d = cutoff - timedelta(days=7)
126
127     for product_id, product_info in products_df.groupby('product_id'):
128         product_events = events_df[events_df['product_id'] == product_id]
129         windowed_events = product_events[
130             (product_events['event_timestamp'] >= window_7d) &
131             (product_events['event_timestamp'] < cutoff)
132         ]
133
134         features = {
135             'product_id': product_id,
136             'price': product_info['price'].iloc[-1],
137             'discount_percentage': product_info['discount_percentage'].iloc[-1],
138             'rating_avg': product_info['rating_avg'].iloc[-1],
139             'rating_count': product_info['rating_count'].iloc[-1],
140             'inventory_level': product_info['inventory_level'].iloc[-1],
141             'view_count_7d': len(windowed_events[windowed_events['event_type'] ==
142 'pageview']),
143             'purchase_count_7d': len(windowed_events[windowed_events['event_type']
144 == 'purchase']),
145             'event_timestamp': cutoff,
146             'created_timestamp': datetime.now()
147         }
148
149         # Category popularity (simplified)
150         category = product_info['category'].iloc[-1]
151         category_products = products_df[products_df['category'] == category]
152         features['category_popularity_score'] = \
153             len(category_products) / len(products_df)
154
155         features_list.append(features)
156
157     result_df = pd.DataFrame(features_list)
158     logger.info(f"Computed features for {len(result_df)} products")
159
160     return result_df
161
162 def materialize_to_feast(
163     self,
164     features_df: pd.DataFrame,
165     feature_view_name: str,
166     output_path: str
167 ):
168     """
169     Write features to parquet and materialize to Feast online store.
170     """
171     logger.info(f"Materializing {feature_view_name} to Feast...")
172
173     # Write to offline store (parquet file)
174     features_df.to_parquet(output_path, index=False)
175     logger.info(f"Wrote features to {output_path}")
176
177     # Apply feature definitions (syncs to registry)
178     self.fs.apply([

```

```

177     # Feature views are defined in features.py
178 ]
179
180     # Materialize to online store
181     end_date = self.computation_timestamp
182     start_date = end_date - timedelta(days=1) # Materialize last day
183
184     self.fs.materialize(
185         start_date=start_date,
186         end_date=end_date,
187         feature_views=[feature_view_name]
188     )
189
190     logger.info(f"Materialized {feature_view_name} to online store")
191
192 def run(self, raw_data_path: str, output_dir: str):
193     """Execute complete materialization pipeline"""
194     logger.info("Starting feature materialization pipeline...")
195
196     # Load raw data (in practice, from data warehouse)
197     events_df = pd.read_parquet(f"{raw_data_path}/events.parquet")
198     products_df = pd.read_parquet(f"{raw_data_path}/products.parquet")
199
200     # Compute features
201     user_activity = self.compute_user_activity_features(events_df)
202     product_features = self.compute_product_features(products_df, events_df)
203
204     # Materialize to Feast
205     self.materialize_to_feast(
206         user_activity,
207         "user_activity_features",
208         f"{output_dir}/user_activity.parquet"
209     )
210
211     self.materialize_to_feast(
212         product_features,
213         "product_features",
214         f"{output_dir}/product_features.parquet"
215     )
216
217     logger.info("Feature materialization pipeline completed successfully")
218
219 # Example usage
220 if __name__ == "__main__":
221     pipeline = FeatureMaterializationPipeline(feast_repo_path="./feast_repo")
222     pipeline.run(
223         raw_data_path="./data/raw",
224         output_dir="./data/features"
225     )

```

Listing 6.8: Production feature materialization pipeline

6.4.3 Online and Offline Feature Serving

```

1 # feature_serving.py
2 """
3 Demonstrate both offline (training) and online (inference) feature retrieval.

```

```

4 """
5
6 from feast import FeatureStore
7 import pandas as pd
8 from datetime import datetime, timedelta
9 from typing import List
10
11 class FeatureServingService:
12     """
13     Service for retrieving features for both training and serving.
14     """
15
16     def __init__(self, feast_repo_path: str):
17         self.fs = FeatureStore(repo_path=feast_repo_path)
18
19     def get_historical_features(
20         self,
21         entity_df: pd.DataFrame,
22         features: List[str]
23     ) -> pd.DataFrame:
24         """
25         Retrieve historical features for training.
26         Ensures point-in-time correctness.
27
28         Args:
29             entity_df: DataFrame with entity keys and timestamps
30             features: List of feature references
31                     (e.g., ["user_activity_features:purchase_count_30d"])
32
33         Returns:
34             DataFrame with requested features joined to entity_df
35         """
36         # Feast retrieves features as they existed at each timestamp
37         training_df = self.fs.get_historical_features(
38             entity_df=entity_df,
39             features=features
40         ).to_df()
41
42         return training_df
43
44     def get_online_features(
45         self,
46         entity_rows: List[dict],
47         features: List[str]
48     ) -> dict:
49         """
50         Retrieve latest features from online store for real-time inference.
51
52         Args:
53             entity_rows: List of entity dictionaries
54                         (e.g., [{"user_id": "user_123"}])
55             features: List of feature references
56
57         Returns:
58             Dictionary with features for each entity
59         """
60         feature_vector = self.fs.get_online_features(
61             entity_rows=entity_rows,

```

```

62         features=features
63     ).to_dict()
64
65     return feature_vector
66
67 # Example: Training pipeline
68 def training_pipeline_example():
69     """
70     Example of retrieving features for model training.
71     """
72     fs_service = Feature-servingService("./feast_repo")
73
74     # Historical training data with labels
75     # In practice, this comes from your label generation process
76     entity_df = pd.DataFrame({
77         'user_id': ['user_1', 'user_2', 'user_3', 'user_4'],
78         'product_id': ['prod_A', 'prod_B', 'prod_C', 'prod_D'],
79         'event_timestamp': [
80             datetime(2024, 1, 15, 10, 0),
81             datetime(2024, 1, 16, 14, 30),
82             datetime(2024, 1, 17, 9, 15),
83             datetime(2024, 1, 18, 16, 45)
84         ],
85         'label': [1, 0, 1, 0] # Purchase/no purchase
86     })
87
88     # Request features from Feast
89     features = [
90         "user_activity_features:purchase_count_30d",
91         "user_activity_features:conversion_rate_30d",
92         "user_activity_features:days_since_last_purchase",
93         "user_profile_features:is_premium_member",
94         "user_profile_features:lifetime_purchase_count",
95         "product_features:price",
96         "product_features:rating_avg",
97         "product_features:discount_percentage"
98     ]
99
100     training_data = fs_service.get_historical_features(
101         entity_df=entity_df,
102         features=features
103     )
104
105     print("Training data with features:")
106     print(training_data.head())
107
108     # Now use training_data for model training
109     # X = training_data.drop(['label', 'event_timestamp'], axis=1)
110     # y = training_data['label']
111
112     return training_data
113
114 # Example: Inference pipeline
115 def inference_pipeline_example():
116     """
117     Example of retrieving features for real-time prediction.
118     """
119     fs_service = Feature-servingService("./feast_repo")

```

```

120
121     # Real-time inference request
122     entity_rows = [
123         {"user_id": "user_123", "product_id": "prod_XYZ"}
124     ]
125
126     features = [
127         "user_activity_features:purchase_count_30d",
128         "user_activity_features:conversion_rate_30d",
129         "user_profile_features:is_premium_member",
130         "product_features:price",
131         "product_features:rating_avg"
132     ]
133
134     # Retrieve from online store (Redis) - low latency
135     feature_vector = fs_service.get_online_features(
136         entity_rows=entity_rows,
137         features=features
138     )
139
140     print("Online features for inference:")
141     print(feature_vector)
142
143     # Now use feature_vector for model prediction
144     # prediction = model.predict([feature_vector])
145
146     return feature_vector
147
148 if __name__ == "__main__":
149     print("=== Training Example ===")
150     training_pipeline_example()
151
152     print("\n=== Inference Example ===")
153     inference_pipeline_example()

```

Listing 6.9: Feature serving for training and inference

6.5 Feature Versioning and Lineage Tracking

Managing feature evolution is critical for reproducibility and debugging production issues. This section presents patterns for versioning and lineage.

6.5.1 Feature Versioning Strategy

```

1  # feature_versioning.py
2  """
3  System for versioning features and tracking their evolution.
4  """
5
6  from dataclasses import dataclass, field
7  from datetime import datetime
8  from typing import Optional, List, Dict
9  import hashlib
10 import json
11

```

```

12 @dataclass
13 class FeatureVersion:
14     """
15     Represents a specific version of a feature.
16     """
17     feature_name: str
18     version: str
19     definition_hash: str
20     created_at: datetime
21     created_by: str
22     description: str
23     code: str
24     dependencies: List[str] = field(default_factory=list)
25     schema: Dict = field(default_factory=dict)
26     deprecated: bool = False
27     deprecated_at: Optional[datetime] = None
28     successor_version: Optional[str] = None
29
30 class FeatureVersionRegistry:
31     """
32     Registry for managing feature versions and their metadata.
33     """
34
35     def __init__(self, storage_backend):
36         self.storage = storage_backend
37         self.versions: Dict[str, List[FeatureVersion]] = {}
38
39     def register_feature(
40         self,
41         feature_name: str,
42         definition: Dict,
43         code: str,
44         created_by: str,
45         description: str = ""
46     ) -> FeatureVersion:
47         """
48         Register a new version of a feature.
49         """
50         # Compute hash of definition for change detection
51         definition_str = json.dumps(definition, sort_keys=True)
52         definition_hash = hashlib.sha256(definition_str.encode()).hexdigest()[:16]
53
54         # Check if this exact definition already exists
55         existing_versions = self.versions.get(feature_name, [])
56         for existing in existing_versions:
57             if existing.definition_hash == definition_hash:
58                 return existing # Return existing version
59
60         # Create new version
61         version_num = len(existing_versions) + 1
62         version_id = f"v{version_num}"
63
64         feature_version = FeatureVersion(
65             feature_name=feature_name,
66             version=version_id,
67             definition_hash=definition_hash,
68             created_at=datetime.now(),
69             created_by=created_by,

```

```

70         description=description,
71         code=code,
72         dependencies=definition.get('dependencies', []),
73         schema=definition.get('schema', {})
74     )
75
76     # Store in registry
77     if feature_name not in self.versions:
78         self.versions[feature_name] = []
79     self.versions[feature_name].append(feature_version)
80
81     # Persist to storage
82     self.storage.save(feature_version)
83
84     return feature_version
85
86 def get_feature_version(
87     self,
88     feature_name: str,
89     version: Optional[str] = None
90 ) -> FeatureVersion:
91     """
92     Retrieve a specific version of a feature.
93     If version is None, returns latest non-deprecated version.
94     """
95     if feature_name not in self.versions:
96         raise ValueError(f"Feature {feature_name} not found in registry")
97
98     versions = self.versions[feature_name]
99
100    if version is None:
101        # Get latest non-deprecated version
102        active_versions = [v for v in versions if not v.deprecated]
103        if not active_versions:
104            raise ValueError(f"No active versions for feature {feature_name}")
105        return active_versions[-1]
106    else:
107        # Get specific version
108        for v in versions:
109            if v.version == version:
110                return v
111            raise ValueError(f"Version {version} of feature {feature_name} not found")
112
113 def deprecate_feature(
114     self,
115     feature_name: str,
116     version: str,
117     successor_version: Optional[str] = None
118 ):
119     """
120     Mark a feature version as deprecated.
121     """
122     feature_version = self.get_feature_version(feature_name, version)
123     feature_version.deprecated = True
124     feature_version.deprecated_at = datetime.now()
125     feature_version.successor_version = successor_version
126
127     self.storage.update(feature_version)

```



```

128
129     def get_feature_lineage(
130         self,
131         feature_name: str
132     ) -> List[FeatureVersion]:
133         """
134         Get complete version history of a feature.
135         """
136         return self.versions.get(feature_name, [])
137
138     # Example usage
139     class InMemoryStorage:
140         """Simple in-memory storage for demo"""
141         def __init__(self):
142             self.data = {}
143
144         def save(self, feature_version: FeatureVersion):
145             key = f"{feature_version.feature_name}:{feature_version.version}"
146             self.data[key] = feature_version
147
148         def update(self, feature_version: FeatureVersion):
149             self.save(feature_version)
150
151     storage = InMemoryStorage()
152     registry = FeatureVersionRegistry(storage)
153
154     # Register initial version
155     v1 = registry.register_feature(
156         feature_name="user_purchase_rate_30d",
157         definition={
158             "window_days": 30,
159             "aggregation": "mean",
160             "schema": {"type": "float32"}
161         },
162         code="""
163     def compute(user_events, timestamp):
164         window_start = timestamp - timedelta(days=30)
165         purchases = user_events[
166             (user_events['event_type'] == 'purchase') &
167             (user_events['timestamp'] >= window_start)
168         ]
169         return len(purchases) / 30
170         """,
171         created_by="data_scientist_1",
172         description="Average purchases per day over 30 days"
173     )
174
175     # Register updated version with bug fix
176     v2 = registry.register_feature(
177         feature_name="user_purchase_rate_30d",
178         definition={
179             "window_days": 30,
180             "aggregation": "mean",
181             "schema": {"type": "float32"},
182             "min_days": 7 # New: require minimum data
183         },
184         code="""
185     def compute(user_events, timestamp):

```

```

186     window_start = timestamp - timedelta(days=30)
187     purchases = user_events[
188         (user_events['event_type'] == 'purchase') &
189         (user_events['timestamp'] >= window_start)
190     ]
191     days_active = user_events['timestamp'].dt.date.nunique()
192     if days_active < 7:
193         return None # Not enough data
194     return len(purchases) / days_active # Fixed: use actual days, not 30
195     """
196     created_by="ml_engineer_1",
197     description="Fixed: Average purchases per active day over 30 days"
198 )
199
200 # Deprecate old version
201 registry.deprecate_feature(
202     "user_purchase_rate_30d",
203     version="v1",
204     successor_version="v2"
205 )
206
207 # Retrieve feature versions
208 latest = registry.get_feature_version("user_purchase_rate_30d")
209 print(f"Latest version: {latest.version} (hash: {latest.definition_hash})")
210
211 lineage = registry.get_feature_lineage("user_purchase_rate_30d")
212 print(f"\nFeature lineage ({len(lineage)} versions):")
213 for v in lineage:
214     status = "DEPRECATED" if v.deprecated else "ACTIVE"
215     print(f"    {v.version}: {status} - {v.description}")

```

Listing 6.10: Feature versioning and registry management

6.5.2 Feature Lineage Tracking

```

1  # feature_lineage.py
2  """
3  Track data lineage from raw data through features to model predictions.
4  """
5
6  from dataclasses import dataclass, field
7  from datetime import datetime
8  from typing import List, Dict, Optional, Set
9  from enum import Enum
10 import networkx as nx
11 import matplotlib.pyplot as plt
12
13 class NodeType(Enum):
14     RAW_DATA = "raw_data"
15     INTERMEDIATE = "intermediate"
16     FEATURE = "feature"
17     MODEL = "model"
18     PREDICTION = "prediction"
19
20 @dataclass
21 class LineageNode:
22     """Represents a node in the feature lineage graph"""

```

```

23     node_id: str
24     node_type: NodeType
25     name: str
26     version: Optional[str] = None
27     created_at: datetime = field(default_factory=datetime.now)
28     metadata: Dict = field(default_factory=dict)
29
30 @dataclass
31 class LineageEdge:
32     """Represents a transformation between nodes"""
33     source_id: str
34     target_id: str
35     transformation: str
36     code: Optional[str] = None
37     created_at: datetime = field(default_factory=datetime.now)
38
39 class FeatureLineageTracker:
40     """
41     Track complete lineage from raw data to predictions.
42     Enables impact analysis and root cause debugging.
43     """
44
45     def __init__(self):
46         self.graph = nx.DiGraph()
47         self.nodes: Dict[str, LineageNode] = {}
48         self.edges: List[LineageEdge] = []
49
50     def add_node(self, node: LineageNode):
51         """Add a node to the lineage graph"""
52         self.nodes[node.node_id] = node
53         self.graph.add_node(
54             node.node_id,
55             type=node.node_type.value,
56             name=node.name,
57             version=node.version
58         )
59
60     def add_edge(self, edge: LineageEdge):
61         """Add a transformation edge to the lineage graph"""
62         self.edges.append(edge)
63         self.graph.add_edge(
64             edge.source_id,
65             edge.target_id,
66             transformation=edge.transformation,
67             code=edge.code
68         )
69
70     def get_upstream_lineage(
71         self,
72         node_id: str,
73         max_depth: Optional[int] = None
74     ) -> Set[str]:
75         """
76         Get all upstream dependencies of a node.
77         Useful for understanding what data affects a feature.
78         """
79         if max_depth is None:
80             # Get all ancestors

```

```

81     ancestors = nx.ancestors(self.graph, node_id)
82     else:
83         # BFS with depth limit
84         ancestors = set()
85         queue = [(node_id, 0)]
86         visited = {node_id}
87
88         while queue:
89             current, depth = queue.pop(0)
90             if depth >= max_depth:
91                 continue
92
93             for predecessor in self.graph.predecessors(current):
94                 if predecessor not in visited:
95                     ancestors.add(predecessor)
96                     visited.add(predecessor)
97                     queue.append((predecessor, depth + 1))
98
99     return ancestors
100
101 def get_downstream_impact(self, node_id: str) -> Set[str]:
102     """
103     Get all downstream nodes affected by a node.
104     Critical for impact analysis when changing features.
105     """
106     return nx.descendants(self.graph, node_id)
107
108 def get_feature_path(
109     self,
110     feature_id: str,
111     source_data_id: str
112 ) -> List[LineageNode]:
113     """
114     Get the transformation path from raw data to feature.
115     """
116     try:
117         path_ids = nx.shortest_path(
118             self.graph,
119             source=source_data_id,
120             target=feature_id
121         )
122         return [self.nodes[node_id] for node_id in path_ids]
123     except nx.NetworkXNoPath:
124         return []
125
126 def visualize_lineage(
127     self,
128     focus_node: Optional[str] = None,
129     output_path: str = "lineage.png"
130 ):
131     """
132     Visualize the feature lineage graph.
133     """
134     if focus_node:
135         # Create subgraph around focus node
136         upstream = self.get_upstream_lineage(focus_node)
137         downstream = self.get_downstream_impact(focus_node)
138         relevant_nodes = upstream | downstream | {focus_node}

```

```

139         subgraph = self.graph.subgraph(relevant_nodes)
140     else:
141         subgraph = self.graph
142
143     # Color by node type
144     color_map = {
145         NodeType.RAW_DATA.value: 'lightblue',
146         NodeType.INTERMEDIATE.value: 'lightgreen',
147         NodeType.FEATURE.value: 'orange',
148         NodeType.MODEL.value: 'red',
149         NodeType.PREDICTION.value: 'purple'
150     }
151
152     node_colors = [
153         color_map.get(subgraph.nodes[node]['type'], 'gray')
154         for node in subgraph.nodes()
155     ]
156
157     plt.figure(figsize=(12, 8))
158     pos = nx.spring_layout(subgraph)
159     nx.draw(
160         subgraph,
161         pos,
162         node_color=node_colors,
163         with_labels=True,
164         labels={n: subgraph.nodes[n]['name'] for n in subgraph.nodes()},
165         arrows=True,
166         node_size=3000,
167         font_size=8
168     )
169     plt.savefig(output_path, dpi=300, bbox_inches='tight')
170     plt.close()
171
172 # Example: Building a lineage graph
173 tracker = FeatureLineageTracker()
174
175 # Raw data sources
176 user_events = LineageNode(
177     node_id="data_user_events",
178     node_type=NodeType.RAW_DATA,
179     name="user_events",
180     metadata={"source": "production_db", "table": "events"}
181 )
182 user_profile = LineageNode(
183     node_id="data_user_profile",
184     node_type=NodeType.RAW_DATA,
185     name="user_profile",
186     metadata={"source": "production_db", "table": "users"}
187 )
188
189 tracker.add_node(user_events)
190 tracker.add_node(user_profile)
191
192 # Intermediate aggregations
193 purchases_30d = LineageNode(
194     node_id="agg_purchases_30d",
195     node_type=NodeType.INTERMEDIATE,
196     name="purchases_last_30d",

```

```

197     metadata={"window": "30days", "aggregation": "count"}
198 )
199
200 tracker.add_node(purchases_30d)
201 tracker.add_edge(LineageEdge(
202     source_id="data_user_events",
203     target_id="agg_purchases_30d",
204     transformation="filter_and_count",
205     code="events[events.type == 'purchase'].groupby('user_id').count()"
206 ))
207
208 # Features
209 conversion_rate = LineageNode(
210     node_id="feat_conversion_rate",
211     node_type=NodeType.FEATURE,
212     name="conversion_rate_30d",
213     version="v2"
214 )
215
216 tracker.add_node(conversion_rate)
217 tracker.add_edge(LineageEdge(
218     source_id="agg_purchases_30d",
219     target_id="feat_conversion_rate",
220     transformation="ratio",
221     code="purchases / sessions"
222 ))
223
224 # Model
225 recommendation_model = LineageNode(
226     node_id="model_recsys_v5",
227     node_type=NodeType.MODEL,
228     name="recommendation_model",
229     version="v5"
230 )
231
232 tracker.add_node(recommendation_model)
233 tracker.add_edge(LineageEdge(
234     source_id="feat_conversion_rate",
235     target_id="model_recsys_v5",
236     transformation="training_input"
237 ))
238
239 # Analysis examples
240 print("Upstream lineage for conversion_rate feature:")
241 upstream = tracker.get_upstream_lineage("feat_conversion_rate")
242 for node_id in upstream:
243     node = tracker.nodes[node_id]
244     print(f" - {node.name} ({node.node_type.value})")
245
246 print("\nDownstream impact of user_events data:")
247 downstream = tracker.get_downstream_impact("data_user_events")
248 for node_id in downstream:
249     node = tracker.nodes[node_id]
250     print(f" - {node.name} ({node.node_type.value})")
251
252 print("\nPath from raw data to feature:")
253 path = tracker.get_feature_path("feat_conversion_rate", "data_user_events")
254 for i, node in enumerate(path):

```

```
255 print(f" {i+1}. {node.name} ({node.node_type.value})")
```

Listing 6.11: End-to-end feature lineage tracking system

6.6 Online and Offline Feature Serving Architecture

A production feature store must support two distinct serving patterns with different latency and consistency requirements.

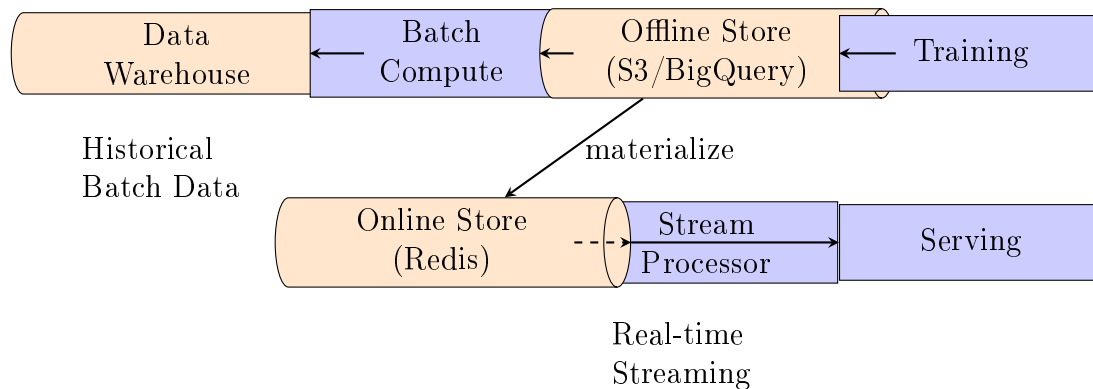


Figure 6.3: Dual-mode feature serving architecture

6.6.1 Offline Feature Store Implementation

```
1  # offline_feature_store.py
2  """
3  Offline feature store optimized for batch training workloads.
4  Uses partitioned storage for efficient historical retrieval.
5  """
6
7  import pyarrow as pa
8  import pyarrow.parquet as pq
9  import pandas as pd
10 from datetime import datetime, date
11 from typing import List, Optional
12 from pathlib import Path
13
14 class OfflineFeatureStore:
15     """
16     Offline store using Parquet with Hive partitioning.
17     Optimized for point-in-time correct historical retrieval.
18     """
19
20     def __init__(self, base_path: str):
21         self.base_path = Path(base_path)
22         self.base_path.mkdir(parents=True, exist_ok=True)
23
24     def write_features(
25         self,
26         features_df: pd.DataFrame,
27         feature_view: str,
```

```

28     partition_cols: List[str] = ['date']
29 ):
30     """
31     Write features with partitioning for efficient queries.
32
33     Args:
34         features_df: DataFrame with features and timestamp
35         feature_view: Name of the feature view
36         partition_cols: Columns to partition by (usually date)
37     """
38     # Add date partition if not present
39     if 'date' not in features_df.columns and 'event_timestamp' in features_df.
columns:
40         features_df['date'] = pd.to_datetime(
41             features_df['event_timestamp']
42         ).dt.date
43
44     # Convert to PyArrow table for efficient write
45     table = pa.Table.from_pandas(features_df)
46
47     # Write with partitioning
48     output_path = self.base_path / feature_view
49     pq.write_to_dataset(
50         table,
51         root_path=str(output_path),
52         partition_cols=partition_cols,
53         existing_data_behavior='overwrite_or_ignore',
54         compression='snappy'
55     )
56
57 def read_features(
58     self,
59     feature_view: str,
60     start_date: Optional[date] = None,
61     end_date: Optional[date] = None,
62     columns: Optional[List[str]] = None,
63     filters: Optional[List] = None
64 ) -> pd.DataFrame:
65     """
66     Read features with predicate pushdown for efficiency.
67
68     Args:
69         feature_view: Name of the feature view
70         start_date: Optional start date filter
71         end_date: Optional end date filter
72         columns: Optional column subset
73         filters: PyArrow filters for additional predicates
74
75     Returns:
76         DataFrame with requested features
77     """
78     dataset_path = self.base_path / feature_view
79
80     # Build filters
81     query_filters = []
82     if filters:
83         query_filters.extend(filters)
84

```



```

85     if start_date:
86         query_filters.append(('date', '>=', start_date))
87     if end_date:
88         query_filters.append(('date', '<=', end_date))
89
90     # Read dataset
91     dataset = pq.ParquetDataset(
92         dataset_path,
93         filters=query_filters if query_filters else None
94     )
95
96     table = dataset.read(columns=columns)
97     return table.to_pandas()
98
99     def get_historical_features(
100         self,
101         entity_df: pd.DataFrame,
102         feature_views: List[str],
103         entity_col: str = 'entity_id',
104         timestamp_col: str = 'timestamp'
105     ) -> pd.DataFrame:
106         """
107         Point-in-time correct historical feature retrieval.
108
109         For each row in entity_df, retrieves feature values
110         as they existed at the specified timestamp.
111         """
112         result = entity_df.copy()
113
114         for feature_view in feature_views:
115             # Read all relevant features
116             min_date = entity_df[timestamp_col].min().date()
117             max_date = entity_df[timestamp_col].max().date()
118
119             features_df = self.read_features(
120                 feature_view,
121                 start_date=min_date,
122                 end_date=max_date
123             )
124
125             # Point-in-time join using merge_asof
126             # Ensures we only use data available before each timestamp
127             features_df = features_df.sort_values('event_timestamp')
128             result = result.sort_values(timestamp_col)
129
130             result = pd.merge_asof(
131                 result,
132                 features_df,
133                 left_on=timestamp_col,
134                 right_on='event_timestamp',
135                 by=entity_col,
136                 direction='backward',
137                 allow_exact_matches=False
138             )
139
140         return result
141
142     # Example usage

```

```

143 offline_store = OfflineFeatureStore("./data/offline_store")
144
145 # Write features
146 features = pd.DataFrame({
147     'user_id': ['u1', 'u2', 'u3'] * 100,
148     'event_timestamp': pd.date_range('2024-01-01', periods=300, freq='1H'),
149     'purchase_count_7d': np.random.randint(0, 10, 300),
150     'avg_order_value': np.random.uniform(20, 200, 300)
151 })
152
153 offline_store.write_features(
154     features,
155     feature_view='user_features',
156     partition_cols=['date']
157 )
158
159 # Read features for training
160 entity_df = pd.DataFrame({
161     'user_id': ['u1', 'u2'],
162     'timestamp': pd.to_datetime(['2024-01-15 10:00', '2024-01-16 14:00'])
163 })
164
165 training_data = offline_store.get_historical_features(
166     entity_df,
167     feature_views=['user_features'],
168     entity_col='user_id'
169 )
170
171 print(training_data)

```

Listing 6.12: Scalable offline feature store with partitioning

6.6.2 Online Feature Store with Redis

```

1 # online_feature_store.py
2 """
3 Online feature store using Redis for sub-millisecond feature retrieval.
4 """
5
6 import redis
7 import json
8 import pickle
9 from typing import Dict, List, Optional, Any
10 from datetime import timedelta
11 import numpy as np
12
13 class OnlineFeatureStore:
14     """
15     Online store for real-time feature serving with Redis.
16     Optimized for low-latency, high-throughput access.
17     """
18
19     def __init__(
20         self,
21         host: str = 'localhost',
22         port: int = 6379,
23         db: int = 0,

```

```

24     password: Optional[str] = None
25 ):
26     self.client = redis.Redis(
27         host=host,
28         port=port,
29         db=db,
30         password=password,
31         decode_responses=False # Handle binary data
32     )
33     self.default_ttl = timedelta(days=7)
34
35     def _make_key(
36         self,
37         feature_view: str,
38         entity_key: str,
39         entity_value: str
40     ) -> str:
41         """Generate Redis key for feature"""
42         return f"feast:{feature_view}:{entity_key}:{entity_value}"
43
44     def write_features(
45         self,
46         feature_view: str,
47         entity_key: str,
48         entity_value: str,
49         features: Dict[str, Any],
50         ttl: Optional[timedelta] = None
51     ):
52         """
53         Write feature vector to online store.
54
55         Args:
56         feature_view: Name of feature view
57         entity_key: Entity key name (e.g., 'user_id')
58         entity_value: Entity value (e.g., 'user_123')
59         features: Dictionary of feature values
60         ttl: Time-to-live for features
61         """
62         key = self._make_key(feature_view, entity_key, entity_value)
63
64         # Serialize features (use pickle for numpy arrays, etc.)
65         serialized = pickle.dumps(features)
66
67         # Write to Redis with TTL
68         ttl_seconds = int((ttl or self.default_ttl).total_seconds())
69         self.client.setex(key, ttl_seconds, serialized)
70
71     def write_features_batch(
72         self,
73         feature_view: str,
74         entity_key: str,
75         entity_features: Dict[str, Dict[str, Any]],
76         ttl: Optional[timedelta] = None
77     ):
78         """
79         Batch write features for multiple entities.
80         Uses pipeline for efficiency.
81         """

```

```

82     pipe = self.client.pipeline()
83     ttl_seconds = int((ttl or self.default_ttl).total_seconds())
84
85     for entity_value, features in entity_features.items():
86         key = self._make_key(feature_view, entity_key, entity_value)
87         serialized = pickle.dumps(features)
88         pipe.setex(key, ttl_seconds, serialized)
89
90     pipe.execute()
91
92     def read_features(
93         self,
94         feature_view: str,
95         entity_key: str,
96         entity_value: str,
97         feature_names: Optional[List[str]] = None
98     ) -> Optional[Dict[str, Any]]:
99         """
100         Read features for a single entity.
101
102         Returns:
103             Dictionary of features, or None if not found
104         """
105         key = self._make_key(feature_view, entity_key, entity_value)
106         serialized = self.client.get(key)
107
108         if serialized is None:
109             return None
110
111         features = pickle.loads(serialized)
112
113         # Filter to requested features if specified
114         if feature_names:
115             features = {k: v for k, v in features.items() if k in feature_names}
116
117         return features
118
119     def read_features_batch(
120         self,
121         feature_view: str,
122         entity_key: str,
123         entity_values: List[str],
124         feature_names: Optional[List[str]] = None
125     ) -> Dict[str, Optional[Dict[str, Any]]]:
126         """
127         Batch read features for multiple entities.
128         Uses pipeline for efficiency.
129         """
130         # Build keys
131         keys = [
132             self._make_key(feature_view, entity_key, ev)
133             for ev in entity_values
134         ]
135
136         # Batch fetch
137         pipe = self.client.pipeline()
138         for key in keys:
139             pipe.get(key)

```

```

140     results = pipe.execute()
141
142     # Deserialize results
143     output = {}
144     for entity_value, serialized in zip(entity_values, results):
145         if serialized is None:
146             output[entity_value] = None
147         else:
148             features = pickle.loads(serialized)
149             if feature_names:
150                 features = {k: v for k, v in features.items() if k in
feature_names}
151             output[entity_value] = features
152
153     return output
154
155     def delete_features(
156         self,
157         feature_view: str,
158         entity_key: str,
159         entity_value: str
160     ):
161         """Delete features for an entity"""
162         key = self._make_key(feature_view, entity_key, entity_value)
163         self.client.delete(key)
164
165     def get_online_features(
166         self,
167         feature_views: List[str],
168         entity_rows: List[Dict[str, str]],
169         feature_names: Optional[List[str]] = None
170     ) -> List[Dict[str, Any]]:
171         """
172         Multi-view feature retrieval for real-time serving.
173
174         Args:
175         feature_views: List of feature view names
176         entity_rows: List of entity dictionaries
177                           e.g., [{"user_id": "u1", "product_id": "p1"}]
178         feature_names: Optional list of specific features to retrieve
179
180         Returns:
181         List of feature dictionaries, one per entity row
182         """
183         results = []
184
185         for entity_row in entity_rows:
186             merged_features = {}
187
188             for feature_view in feature_views:
189                 # Determine entity key for this view
190                 # (in practice, this comes from feature view metadata)
191                 entity_key = self._infer_entity_key(feature_view, entity_row)
192                 if entity_key is None:
193                     continue
194
195                 entity_value = entity_row.get(entity_key)
196                 if entity_value is None:

```

```

197         continue
198
199         # Fetch features
200         features = self.read_features(
201             feature_view,
202             entity_key,
203             entity_value,
204             feature_names
205         )
206
207         if features:
208             # Prefix feature names with view name to avoid collisions
209             prefixed = {
210                 f"{feature_view}:{k}": v
211                 for k, v in features.items()
212             }
213             merged_features.update(prefixed)
214
215             # Add entity keys to result
216             merged_features.update(entity_row)
217             results.append(merged_features)
218
219         return results
220
221     def _infer_entity_key(
222         self,
223         feature_view: str,
224         entity_row: Dict[str, str]
225     ) -> Optional[str]:
226         """Infer which entity key to use for this view"""
227         # Simplified: in production, consult feature view metadata
228         if 'user_id' in entity_row:
229             return 'user_id'
230         elif 'product_id' in entity_row:
231             return 'product_id'
232         return None
233
234     # Example usage
235     online_store = OnlineFeatureStore(host='localhost', port=6379)
236
237     # Materialize features to online store
238     user_features = {
239         'user_123': {
240             'purchase_count_7d': 5,
241             'purchase_count_30d': 23,
242             'conversion_rate': 0.15,
243             'avg_order_value': 45.20
244         },
245         'user_456': {
246             'purchase_count_7d': 2,
247             'purchase_count_30d': 8,
248             'conversion_rate': 0.08,
249             'avg_order_value': 62.50
250         }
251     }
252
253     online_store.write_features_batch(
254         feature_view='user_activity_features',

```

```

255     entity_key='user_id',
256     entity_features=user_features,
257     ttl=timedelta(days=7)
258 )
259
260 # Real-time feature retrieval
261 entity_rows = [
262     {"user_id": "user_123", "product_id": "prod_XYZ"},
263     {"user_id": "user_456", "product_id": "prod_ABC"}
264 ]
265
266 features = online_store.get_online_features(
267     feature_views=['user_activity_features'],
268     entity_rows=entity_rows,
269     feature_names=['purchase_count_30d', 'conversion_rate']
270 )
271
272 print("Online features:")
273 for i, row in enumerate(features):
274     print(f"Row {i+1}: {row}")

```

Listing 6.13: Low-latency online feature store implementation

6.7 Feature Monitoring and Drift Detection

Production feature stores require continuous monitoring to detect data quality issues and feature drift.

```

1  # feature_monitoring.py
2  """
3  Monitor feature quality and detect drift in production.
4  """
5
6  import pandas as pd
7  import numpy as np
8  from typing import Dict, List, Optional, Tuple
9  from dataclasses import dataclass
10 from datetime import datetime, timedelta
11 from scipy import stats
12 from scipy.spatial.distance import jensenshannon
13 import warnings
14
15 @dataclass
16 class FeatureStats:
17     """Statistical summary of a feature"""
18     feature_name: str
19     timestamp: datetime
20     count: int
21     null_count: int
22     null_rate: float
23     mean: Optional[float] = None
24     std: Optional[float] = None
25     min: Optional[float] = None
26     max: Optional[float] = None
27     quantiles: Optional[Dict[str, float]] = None
28     unique_count: Optional[int] = None

```



```

87         'p99': valid_data.quantile(0.99)
88     }
89 })
90
91     # Histogram for distribution comparison
92     hist, _ = np.histogram(valid_data, bins=n_bins)
93     stats_dict['distribution'] = hist / hist.sum() # Normalize
94
95     # Categorical features
96     else:
97         stats_dict['unique_count'] = feature_data.nunique()
98         # For categorical, store value counts as distribution
99         value_counts = feature_data.value_counts(normalize=True)
100         stats_dict['distribution'] = value_counts.to_dict()
101
102     return FeatureStats(**stats_dict)
103
104 def set_baseline(
105     self,
106     baseline_data: pd.DataFrame,
107     timestamp: Optional[datetime] = None
108 ):
109     """
110     Establish baseline statistics from training/reference data.
111     """
112     if timestamp is None:
113         timestamp = datetime.now()
114
115     for column in baseline_data.columns:
116         self.baseline_stats[column] = self.compute_feature_stats(
117             baseline_data[column],
118             timestamp
119         )
120
121 def detect_statistical_drift(
122     self,
123     baseline: FeatureStats,
124     current: FeatureStats
125 ) -> Tuple[bool, float, Dict]:
126     """
127     Detect drift using statistical tests.
128
129     For numeric features: Kolmogorov-Smirnov test
130     For categorical features: Chi-square test
131     """
132     details = {}
133
134     # Check if distributions are available
135     if baseline.distribution is None or current.distribution is None:
136         return False, 1.0, {'error': 'No distribution data'}
137
138     # Numeric features: KS test
139     if isinstance(baseline.distribution, np.ndarray):
140         # KS statistic measures max distance between CDFs
141         ks_statistic = np.max(np.abs(
142             np.cumsum(baseline.distribution) - np.cumsum(current.distribution)
143         ))
144

```

```

145     # Approximate p-value (simplified)
146     n1 = baseline.count
147     n2 = current.count
148     p_value = stats.ks_2samp(
149         np.repeat(np.arange(len(baseline.distribution)),
150                     (baseline.distribution * n1).astype(int)),
151         np.repeat(np.arange(len(current.distribution)),
152                     (current.distribution * n2).astype(int))
153     ).pvalue
154
155     drift_detected = p_value < self.drift_threshold
156     details = {
157         'test': 'kolmogorov-smirnov',
158         'ks_statistic': ks_statistic,
159         'p_value': p_value,
160         'threshold': self.drift_threshold
161     }
162
163     return drift_detected, p_value, details
164
165     # Categorical features: Chi-square test
166     else:
167         # Align categories
168         all_categories = set(baseline.distribution.keys()) | \
169                         set(current.distribution.keys())
170
171         baseline_counts = [
172             baseline.distribution.get(cat, 0) * baseline.count
173             for cat in all_categories
174         ]
175         current_counts = [
176             current.distribution.get(cat, 0) * current.count
177             for cat in all_categories
178         ]
179
180         # Chi-square test
181         chi2, p_value = stats.chisquare(current_counts, baseline_counts)
182
183         drift_detected = p_value < self.drift_threshold
184         details = {
185             'test': 'chi_square',
186             'chi2_statistic': chi2,
187             'p_value': p_value,
188             'threshold': self.drift_threshold,
189             'new_categories': list(
190                 set(current.distribution.keys()) - set(baseline.distribution.keys
191
192             ))
193         }
194
195         return drift_detected, p_value, details
196
197     def detect_distribution_drift(
198         self,
199         baseline: FeatureStats,
200         current: FeatureStats
201     ) -> Tuple[bool, float, Dict]:
202         """

```

```

202 Detect drift using distribution distance metrics.
203 Uses Jensen-Shannon divergence.
204 """
205 if baseline.distribution is None or current.distribution is None:
206     return False, 0.0, {'error': 'No distribution data'}
207
208 # Numeric features
209 if isinstance(baseline.distribution, np.ndarray):
210     js_divergence = jensenshannon(
211         baseline.distribution,
212         current.distribution
213     )
214
215     drift_detected = js_divergence > self.distribution_threshold
216     details = {
217         'metric': 'jensen_shannon_divergence',
218         'js_divergence': js_divergence,
219         'threshold': self.distribution_threshold
220     }
221
222     return drift_detected, js_divergence, details
223
224 # Categorical features
225 else:
226     all_categories = set(baseline.distribution.keys()) | \
227                     set(current.distribution.keys())
228
229     baseline_probs = [
230         baseline.distribution.get(cat, 0) for cat in all_categories
231     ]
232     current_probs = [
233         current.distribution.get(cat, 0) for cat in all_categories
234     ]
235
236     js_divergence = jensenshannon(baseline_probs, current_probs)
237
238     drift_detected = js_divergence > self.distribution_threshold
239     details = {
240         'metric': 'jensen_shannon_divergence',
241         'js_divergence': js_divergence,
242         'threshold': self.distribution_threshold
243     }
244
245     return drift_detected, js_divergence, details
246
247 def detect_data_quality_issues(
248     self,
249     baseline: FeatureStats,
250     current: FeatureStats,
251     null_rate_threshold: float = 0.1,
252     mean_shift_threshold: float = 3.0 # In standard deviations
253 ) -> Tuple[bool, List[str]]:
254     """
255     Detect data quality degradation.
256     """
257     issues = []
258
259     # Null rate increase

```

```

260     if current.null_rate > baseline.null_rate + null_rate_threshold:
261         issues.append(
262             f"Null rate increased from {baseline.null_rate:.2%} to "
263             f"{current.null_rate:.2%}"
264         )
265
266     # Mean shift (for numeric features)
267     if baseline.mean is not None and current.mean is not None:
268         if baseline.std and baseline.std > 0:
269             z_score = abs(current.mean - baseline.mean) / baseline.std
270             if z_score > mean_shift_threshold:
271                 issues.append(
272                     f"Mean shifted by {z_score:.2f} standard deviations "
273                     f"(from {baseline.mean:.2f} to {current.mean:.2f})"
274                 )
275
276     # Out-of-range values
277     if baseline.min is not None and current.min is not None:
278         if current.min < baseline.min:
279             issues.append(
280                 f"New minimum value {current.min:.2f} < "
281                 f"baseline {baseline.min:.2f}"
282             )
283         if current.max > baseline.max:
284             issues.append(
285                 f"New maximum value {current.max:.2f} > "
286                 f"baseline {baseline.max:.2f}"
287             )
288
289     # Cardinality change (for categorical)
290     if baseline.unique_count is not None:
291         cardinality_change = abs(
292             current.unique_count - baseline.unique_count
293         ) / baseline.unique_count
294
295         if cardinality_change > 0.2: # 20% change
296             issues.append(
297                 f"Cardinality changed by {cardinality_change:.1%} "
298                 f"(from {baseline.unique_count} to {current.unique_count})"
299             )
300
301     return len(issues) > 0, issues
302
303 def detect_drift(
304     self,
305     current_data: pd.DataFrame,
306     timestamp: Optional[datetime] = None
307 ) -> List[DriftDetectionResult]:
308     """
309     Comprehensive drift detection across all features.
310     """
311     if timestamp is None:
312         timestamp = datetime.now()
313
314     results = []
315
316     for column in current_data.columns:
317         if column not in self.baseline_stats:

```

```

318         continue
319
320     baseline = self.baseline_stats[column]
321     current = self.compute_feature_stats(
322         current_data[column],
323         timestamp
324     )
325
326     # Run all drift detection methods
327     stat_drift, stat_score, stat_details = \
328         self.detect_statistical_drift(baseline, current)
329
330     dist_drift, dist_score, dist_details = \
331         self.detect_distribution_drift(baseline, current)
332
333     quality_issues, quality_details = \
334         self.detect_data_quality_issues(baseline, current)
335
336     # Aggregate results
337     drift_detected = stat_drift or dist_drift or quality_issues
338
339     if drift_detected:
340         # Determine primary drift type
341         if quality_issues:
342             drift_type = 'data_quality'
343             drift_score = 1.0 if quality_issues else 0.0
344         elif stat_drift:
345             drift_type = 'statistical'
346             drift_score = 1 - stat_score # Lower p-value = higher drift
347         else:
348             drift_type = 'distribution'
349             drift_score = dist_score
350
351     result = DriftDetectionResult(
352         feature_name=column,
353         drift_detected=True,
354         drift_score=drift_score,
355         drift_type=drift_type,
356         baseline_stats=baseline,
357         current_stats=current,
358         details={
359             'statistical_test': stat_details,
360             'distribution_test': dist_details,
361             'quality_issues': quality_details
362         }
363     )
364
365     results.append(result)
366
367     return results
368
369 # Example usage
370 np.random.seed(42)
371
372 # Baseline data (training)
373 baseline_df = pd.DataFrame({
374     'purchase_count_30d': np.random.poisson(10, 1000),
375     'avg_order_value': np.random.normal(50, 15, 1000),

```

```

376     'user_category': np.random.choice(['A', 'B', 'C'], 1000, p=[0.5, 0.3, 0.2])
377 })
378
379 # Initialize monitor
380 monitor = FeatureMonitor(drift_threshold=0.05, distribution_threshold=0.1)
381 monitor.set_baseline(baseline_df)
382
383 # Current data with drift
384 current_df = pd.DataFrame({
385     'purchase_count_30d': np.random.poisson(12, 500), # Mean shifted
386     'avg_order_value': np.random.normal(50, 25, 500), # Variance increased
387     'user_category': np.random.choice(['A', 'B', 'C', 'D'], 500,
388                                       p=[0.4, 0.3, 0.2, 0.1]) # New category
389 })
390
391 # Detect drift
392 drift_results = monitor.detect_drift(current_df)
393
394 print(f"Drift detection results: {len(drift_results)} features with drift\n")
395 for result in drift_results:
396     print(f"Feature: {result.feature_name}")
397     print(f"  Drift Type: {result.drift_type}")
398     print(f"  Drift Score: {result.drift_score:.4f}")
399     print(f"  Details: {result.details}")
400     print()

```

Listing 6.14: Comprehensive feature monitoring and drift detection

6.8 Integration with Training and Serving Pipelines

```

1  # integrated_pipeline.py
2  """
3  Complete integration of feature store with training and serving.
4  """
5
6  from feast import FeatureStore
7  import pandas as pd
8  import numpy as np
9  from sklearn.ensemble import RandomForestClassifier
10 from sklearn.model_selection import train_test_split
11 from sklearn.metrics import classification_report
12 import mlflow
13 from datetime import datetime, timedelta
14 from typing import Dict, List
15 import joblib
16
17 class MLPipelineWithFeatureStore:
18     """
19     Integrated ML pipeline using feature store for consistency.
20     """
21
22     def __init__(
23         self,
24         feast_repo_path: str,
25         mlflow_tracking_uri: str = "http://localhost:5000"
26     ):

```

```

27     self.fs = FeatureStore(repo_path=feast_repo_path)
28     mlflow.set_tracking_uri(mlflow_tracking_uri)
29     self.model = None
30     self.feature_names = []
31
32     def create_training_dataset(
33         self,
34         entity_df: pd.DataFrame,
35         feature_services: List[str],
36         label_col: str
37     ) -> tuple:
38         """
39         Create training dataset using feature store.
40
41         Args:
42             entity_df: DataFrame with entities, timestamps, and labels
43             feature_services: List of Feast feature services to use
44             label_col: Name of the label column
45
46         Returns:
47             X_train, X_test, y_train, y_test
48         """
49         # Get historical features from Feast
50         training_df = self.fs.get_historical_features(
51             entity_df=entity_df,
52             features=feature_services
53         ).to_df()
54
55         # Separate features and labels
56         self.feature_names = [
57             col for col in training_df.columns
58             if col not in ['entity_id', 'event_timestamp', label_col]
59         ]
60
61         X = training_df[self.feature_names]
62         y = training_df[label_col]
63
64         # Split
65         X_train, X_test, y_train, y_test = train_test_split(
66             X, y, test_size=0.2, random_state=42
67         )
68
69         return X_train, X_test, y_train, y_test
70
71     def train_model(
72         self,
73         X_train: pd.DataFrame,
74         y_train: pd.Series,
75         X_test: pd.DataFrame,
76         y_test: pd.Series,
77         experiment_name: str = "feature_store_model"
78     ):
79         """
80         Train model and log to MLflow.
81         """
82         mlflow.set_experiment(experiment_name)
83
84         with mlflow.start_run():

```

```

85     # Train model
86     self.model = RandomForestClassifier(
87         n_estimators=100,
88         max_depth=10,
89         random_state=42
90     )
91     self.model.fit(X_train, y_train)
92
93     # Evaluate
94     train_score = self.model.score(X_train, y_train)
95     test_score = self.model.score(X_test, y_test)
96
97     # Log metrics
98     mlflow.log_metric("train_accuracy", train_score)
99     mlflow.log_metric("test_accuracy", test_score)
100
101     # Log parameters
102     mlflow.log_param("n_estimators", 100)
103     mlflow.log_param("max_depth", 10)
104     mlflow.log_param("feature_count", len(self.feature_names))
105     mlflow.log_param("feature_names", ",".join(self.feature_names))
106
107     # Log model
108     mlflow.sklearn.log_model(
109         self.model,
110         "model",
111         registered_model_name="purchase_prediction_model"
112     )
113
114     # Log feature importance
115     feature_importance = pd.DataFrame({
116         'feature': self.feature_names,
117         'importance': self.model.feature_importances_
118     }).sort_values('importance', ascending=False)
119
120     mlflow.log_table(
121         feature_importance,
122         "feature_importance.json"
123     )
124
125     print(f"Model trained - Train: {train_score:.3f}, Test: {test_score:.3f}")
126
127     def predict_online(
128         self,
129         entity_rows: List[Dict],
130         feature_services: List[str]
131     ) -> np.ndarray:
132         """
133         Make predictions using online features.
134
135         Args:
136             entity_rows: List of entity dictionaries
137             feature_services: Feature services to retrieve
138
139         Returns:
140             Array of predictions
141         """
142         if self.model is None:

```



```

143         raise ValueError("Model not trained. Call train_model first.")
144
145         # Get online features from Feast
146         online_features = self.fs.get_online_features(
147             entity_rows=entity_rows,
148             features=feature_services
149         ).to_df()
150
151         # Ensure feature order matches training
152         X = online_features[self.feature_names]
153
154         # Predict
155         predictions = self.model.predict(X)
156         probabilities = self.model.predict_proba(X)
157
158         return predictions, probabilities
159
160 # Example: End-to-end workflow
161 def run_integrated_pipeline():
162     """
163     Complete example of integrated training and serving pipeline.
164     """
165
166     # Initialize pipeline
167     pipeline = MLPipelineWithFeatureStore(
168         feast_repo_path="./feast_repo",
169         mlflow_tracking_uri="./mlruns"
170     )
171
172     # Step 1: Prepare training data
173     print("Step 1: Creating training dataset from feature store...")
174
175     # Historical entity-label pairs
176     entity_df = pd.DataFrame({
177         'user_id': [f'user_{i}' for i in range(1000)],
178         'event_timestamp': [
179             datetime.now() - timedelta(days=np.random.randint(1, 90))
180             for _ in range(1000)
181         ],
182         'made_purchase': np.random.binomial(1, 0.3, 1000) # Labels
183     })
184
185     # Feature services to use
186     feature_services = [
187         "user_activity_features:purchase_count_30d",
188         "user_activity_features:conversion_rate_30d",
189         "user_activity_features:days_since_last_purchase",
190         "user_profile_features:is_premium_member",
191         "user_profile_features:lifetime_purchase_count"
192     ]
193
194     # Create dataset
195     X_train, X_test, y_train, y_test = pipeline.create_training_dataset(
196         entity_df=entity_df,
197         feature_services=feature_services,
198         label_col='made_purchase'
199     )
200

```

```

201 print(f"Training set: {X_train.shape}, Test set: {X_test.shape}")
202
203 # Step 2: Train model
204 print("\nStep 2: Training model...")
205 pipeline.train_model(X_train, y_train, X_test, y_test)
206
207 # Step 3: Online prediction
208 print("\nStep 3: Making online predictions...")
209
210 # Real-time prediction requests
211 prediction_requests = [
212     {"user_id": "user_123"},
213     {"user_id": "user_456"},
214     {"user_id": "user_789"}
215 ]
216
217 predictions, probabilities = pipeline.predict_online(
218     entity_rows=prediction_requests,
219     feature_services=feature_services
220 )
221
222 print("\nPredictions:")
223 for i, request in enumerate(prediction_requests):
224     print(f" {request['user_id']}: {predictions[i]} "
225           f"(prob: {probabilities[i][1]:.3f})")
226
227 if __name__ == "__main__":
228     run_integrated_pipeline()

```

Listing 6.15: End-to-end pipeline integration

6.9 Real-World Case Studies

6.9.1 Case Study 1: E-Commerce Recommendation System

Challenge: A large e-commerce platform needed to serve personalized product recommendations with sub-100ms latency while maintaining model accuracy. The system processes 100M+ daily events and serves 50K requests per second.

Solution Architecture:

- **Offline Pipeline:** Daily Spark jobs compute user and product features over multiple time windows (7d, 30d, 90d)
- **Feature Store:** Feast with BigQuery offline store and Redis online store
- **Online Pipeline:** Kafka streams for real-time session features
- **Serving:** Two-tier model architecture (candidate generation + ranking)

Key Features Engineered:

- User engagement features: pageviews, cart additions, purchase history across windows
- Product popularity features: view/purchase counts by category and time

- User-product affinity: co-occurrence patterns, category preferences
- Real-time session features: current session behavior, time on site
- Temporal features: day of week, time of day, seasonality indicators

Results:

- P99 latency: 45ms (down from 200ms)
- CTR improvement: +18%
- Revenue per user: +12%
- Training-serving consistency: 99.9% feature value match

6.9.2 Case Study 2: Fraud Detection System

Challenge: Financial services company needed real-time fraud detection with complex behavioral features while handling 10M+ transactions daily.

Solution Architecture:

- **Streaming Features:** Flink for real-time aggregations (velocity features)
- **Batch Features:** Airflow DAGs for historical pattern analysis
- **Feature Store:** Tecton with Snowflake and DynamoDB
- **Serving:** Multi-model ensemble with feature-based routing

Critical Features:

- Velocity features: Transaction count/amount in last 1min, 5min, 1hour
- Geographic features: Distance from previous transaction, usual locations
- Behavioral features: Deviation from spending patterns, unusual merchants
- Network features: Graph-based features from transaction networks
- Device fingerprinting: Device changes, IP reputation scores

Results:

- Fraud detection rate: +35%
- False positive rate: -40%
- Average detection time: 150ms
- Prevented losses: \$15M annually

6.10 Exercises

Exercise 6.1 (Feature Engineering Pipeline). Implement a complete feature engineering pipeline for a retail dataset:

1. Load transaction data with columns: `user_id`, `product_id`, `timestamp`, `amount`
2. Create RFM (Recency, Frequency, Monetary) features
3. Add time-window aggregations (7d, 30d, 90d)
4. Implement point-in-time correct feature computation
5. Validate no data leakage with temporal splits

Deliverable: Python script with documented feature engineering functions and validation tests.

Exercise 6.2 (Feast Feature Store Setup). Deploy a complete Feast feature store:

1. Set up Feast with local file offline store and Redis online store
2. Define at least 3 feature views with different entities
3. Implement materialization pipeline with scheduling
4. Create both batch and online feature retrieval functions
5. Demonstrate point-in-time correct historical retrieval

Deliverable: Feast repository with feature definitions, materialization scripts, and usage examples.

Exercise 6.3 (Feature Versioning System). Build a feature versioning and lineage system:

1. Implement feature registry with version tracking
2. Create feature definition hashing for change detection
3. Build lineage graph tracking data dependencies
4. Implement backward compatibility checks
5. Add deprecation workflow with successor versioning

Deliverable: Feature registry system with API documentation and test cases.

Exercise 6.4 (Online Feature Store Performance). Optimize online feature store for high throughput:

1. Implement Redis-based online store with pipelining
2. Add caching layer with TTL management
3. Implement batch read/write for efficiency
4. Measure latency under load (use locust or similar)

5. Optimize to achieve p99 latency $< 10\text{ms}$ for 1000 QPS

Deliverable: Optimized online store implementation with performance benchmarks and load test results.

Exercise 6.5 (Feature Drift Detection). Implement comprehensive feature monitoring:

1. Create baseline statistics from training data
2. Implement statistical drift tests (KS test, Chi-square)
3. Add distribution-based drift detection (JS divergence)
4. Build data quality checks (null rates, range violations)
5. Create alerting for detected drift

Deliverable: Feature monitoring system with drift detection reports and alert configurations.

Exercise 6.6 (Streaming Feature Pipeline). Build real-time feature computation with Kafka:

1. Set up Kafka cluster with topic for events
2. Implement streaming aggregations (tumbling windows)
3. Add sessionization features
4. Materialize to online feature store
5. Handle late-arriving data correctly

Deliverable: Streaming pipeline with Kafka consumers/producers and feature computation logic.

Exercise 6.7 (Feature Store Integration). Integrate feature store with full ML pipeline:

1. Build training pipeline using offline feature store
2. Implement serving pipeline using online features
3. Add feature logging for monitoring
4. Track feature importance and usage
5. Implement feature A/B testing framework

Deliverable: End-to-end pipeline with training, serving, monitoring, and experimentation.

Exercise 6.8 (Advanced Feature Engineering). Implement sophisticated feature engineering techniques:

1. Build entity embeddings for high-cardinality categoricals
2. Implement graph-based features from interaction networks
3. Create polynomial feature interactions
4. Add automatic feature selection based on importance
5. Implement feature crosses for interaction effects

Deliverable: Advanced feature engineering library with documentation and examples.

6.11 Chapter Summary

This chapter presented a comprehensive treatment of feature stores and feature engineering in production ML systems. Key takeaways include:

- **Training-Serving Consistency:** Feature stores solve the critical problem of training-serving skew by providing a unified interface for feature computation and retrieval.
- **Feature Engineering Best Practices:** Reproducible, modular feature definitions with point-in-time correctness are essential for production systems.
- **Dual-Mode Serving:** Production feature stores must support both offline (training) and online (serving) access patterns with different latency and throughput characteristics.
- **Feature Lifecycle Management:** Versioning, lineage tracking, and deprecation workflows are critical for maintaining feature quality over time.
- **Monitoring and Drift Detection:** Continuous monitoring of feature distributions and data quality prevents silent model degradation.
- **Architecture Patterns:** Successful implementations balance batch and streaming processing, optimize for access patterns, and integrate seamlessly with ML pipelines.

Feature stores have matured from experimental tools to production-critical infrastructure. Organizations implementing feature stores report significant improvements in model development velocity, feature reuse, and production reliability. However, success requires careful attention to architecture, governance, and operational practices detailed in this chapter.

Part III

Training Pipeline Architecture

Chapter 7

Model Development Best Practices

7.1 Learning Objectives

After completing this chapter, readers will be able to:

- Design and implement production-ready ML project structures
- Apply software engineering principles to ML code development
- Build modular, testable, and maintainable ML architectures
- Implement comprehensive testing strategies for ML systems
- Apply design patterns specifically adapted for ML workflows
- Use code quality tools and best practices for ML codebases
- Create fully reproducible training and evaluation pipelines
- Refactor research code into production-grade implementations

7.2 Introduction: From Research Code to Production

Machine learning development often begins in Jupyter notebooks—an excellent environment for exploration and prototyping. However, production ML systems require the rigor, structure, and maintainability of well-engineered software. This chapter bridges the gap between experimental ML code and production-ready systems.

The transition from research to production involves several critical shifts:

- **Structure:** From linear notebooks to modular, reusable components
- **Testing:** From manual verification to automated test suites
- **Configuration:** From hardcoded values to flexible configuration systems
- **Reproducibility:** From ad-hoc experiments to fully reproducible pipelines
- **Quality:** From working code to maintainable, documented software

7.3 ML Code Organization and Project Structure

A well-organized project structure is the foundation of maintainable ML systems. This section presents battle-tested patterns for structuring ML projects at scale.

7.3.1 Production ML Project Template

```

1 ml-production-project/
2 |
3 +-- README.md           # Project overview and setup instructions
4 +-- Makefile            # Common commands (train, test, deploy)
5 +-- setup.py            # Package installation configuration
6 +-- requirements.txt     # Python dependencies
7 +-- requirements-dev.txt # Development dependencies
8 +-- .env.example        # Environment variables template
9 +-- .gitignore
10 +-- pyproject.toml      # Modern Python project configuration
11 |
12 +-- config/             # Configuration files
13 |   +-- config.yaml     # Main configuration
14 |   +-- model/          # Model-specific configs
15 |       +-- random_forest.yaml
16 |       +-- xgboost.yaml
17 |       -- neural_net.yaml
18 |   +-- data/           # Data processing configs
19 |       -- deployment/  # Deployment configs
20 |
21 +-- data/               # Data directory (gitignored)
22 |   +-- raw/            # Immutable raw data
23 |   +-- interim/        # Intermediate processed data
24 |   +-- processed/      # Final feature-engineered data
25 |       -- external/    # External reference data
26 |
27 +-- models/             # Trained models (gitignored)
28 |   +-- checkpoints/    # Training checkpoints
29 |   +-- final/          # Production models
30 |       -- experiments/ # Experimental models
31 |
32 +-- notebooks/          # Jupyter notebooks
33 |   +-- 01_exploration.ipynb
34 |   +-- 02_feature_analysis.ipynb
35 |   -- 03_model_prototyping.ipynb
36 |
37 +-- src/                # Source code
38 |   +-- __init__.py
39 |   |
40 |   +-- data/           # Data processing
41 |       +-- __init__.py
42 |       +-- loaders.py  # Data loading utilities
43 |       +-- validators.py # Data validation
44 |       +-- preprocessors.py # Preprocessing transformations
45 |       -- splitters.py # Train/val/test splitting
46 |   |
47 |   +-- features/       # Feature engineering
48 |       +-- __init__.py
49 |       +-- base.py     # Base feature classes
50 |       +-- transformers.py # Feature transformers

```

```

51 | | -- registry.py      # Feature registry
52 | |
53 | +-- models/          # Model implementations
54 | | +-- __init__.py
55 | | +-- base.py       # Base model interface
56 | | +-- sklearn_models.py
57 | | +-- torch_models.py
58 | | -- ensemble.py
59 | |
60 | +-- training/        # Training logic
61 | | +-- __init__.py
62 | | +-- trainer.py    # Training orchestration
63 | | +-- callbacks.py  # Training callbacks
64 | | -- optimizers.py  # Optimizer configurations
65 | |
66 | +-- evaluation/      # Model evaluation
67 | | +-- __init__.py
68 | | +-- metrics.py    # Custom metrics
69 | | -- validators.py  # Model validation
70 | |
71 | +-- serving/         # Model serving
72 | | +-- __init__.py
73 | | +-- predictor.py  # Prediction interface
74 | | -- api.py         # REST API
75 | |
76 | -- utils/           # Utilities
77 | | +-- __init__.py
78 | | +-- config.py     # Config management
79 | | +-- logging.py    # Logging setup
80 | | -- io.py          # I/O utilities
81 | |
82 | +-- tests/          # Test suite
83 | | +-- __init__.py
84 | | +-- conftest.py   # Pytest fixtures
85 | | +-- unit/         # Unit tests
86 | | | +-- test_data.py
87 | | | +-- test_features.py
88 | | | -- test_models.py
89 | | +-- integration/  # Integration tests
90 | | | +-- test_pipeline.py
91 | | | -- test_training.py
92 | | -- model/         # Model-specific tests
93 | | | +-- test_predictions.py
94 | | | -- test_performance.py
95 | |
96 | +-- scripts/        # Executable scripts
97 | | +-- train.py       # Training script
98 | | +-- evaluate.py    # Evaluation script
99 | | +-- predict.py     # Inference script
100 | | -- prepare_data.py # Data preparation
101 | |
102 | +-- docker/         # Docker configurations
103 | | +-- Dockerfile
104 | | +-- docker-compose.yml
105 | | -- .dockerignore
106 | |
107 | -- docs/            # Documentation
108 | | +-- api.md         # API documentation

```

```

109 +-- model_cards/          # Model documentation
110 -- architecture.md       # System architecture

```

Listing 7.1: Comprehensive ML project structure

7.3.2 Project Structure Implementation

```

1  # setup.py
2  """Package setup configuration for ML project."""
3
4  from setuptools import setup, find_packages
5
6  with open("README.md", "r", encoding="utf-8") as fh:
7      long_description = fh.read()
8
9  with open("requirements.txt", "r", encoding="utf-8") as fh:
10     requirements = [line.strip() for line in fh if line.strip() and not line.
11                     startswith("#")]
12
13 setup(
14     name="ml-production-project",
15     version="0.1.0",
16     author="ML Team",
17     author_email="ml-team@company.com",
18     description="Production ML system for predictive modeling",
19     long_description=long_description,
20     long_description_content_type="text/markdown",
21     url="https://github.com/company/ml-production-project",
22     packages=find_packages(where="src"),
23     package_dir={"": "src"},
24     classifiers=[
25         "Development Status :: 3 - Alpha",
26         "Intended Audience :: Developers",
27         "Programming Language :: Python :: 3",
28         "Programming Language :: Python :: 3.9",
29         "Programming Language :: Python :: 3.10",
30     ],
31     python_requires=">=3.9",
32     install_requires=requirements,
33     extras_require={
34         "dev": [
35             "pytest>=7.0",
36             "pytest-cov>=3.0",
37             "black>=22.0",
38             "flake8>=4.0",
39             "mypy>=0.950",
40             "isort>=5.10",
41         ],
42         "docs": [
43             "sphinx>=4.0",
44             "sphinx-rtd-theme>=1.0",
45         ],
46     },
47     entry_points={
48         "console_scripts": [
49             "ml-train=scripts.train:main",
50             "ml-predict=scripts.predict:main",

```

```

50         "ml-evaluate=scripts.evaluate:main",
51     ],
52 },
53 )

```

Listing 7.2: Setup configuration for package installation

```

1  # Makefile
2  .PHONY: install test train clean lint format
3
4  # Installation
5  install:
6      pip install -e .
7      pip install -e ".[dev]"
8
9  install-prod:
10     pip install -e .
11
12 # Testing
13 test:
14     pytest tests/ -v --cov=src --cov-report=html
15
16 test-unit:
17     pytest tests/unit/ -v
18
19 test-integration:
20     pytest tests/integration/ -v
21
22 test-model:
23     pytest tests/model/ -v
24
25 # Code quality
26 lint:
27     flake8 src/ tests/
28     mypy src/
29
30 format:
31     black src/ tests/ scripts/
32     isort src/ tests/ scripts/
33
34 format-check:
35     black --check src/ tests/ scripts/
36     isort --check src/ tests/ scripts/
37
38 # Training
39 train:
40     python scripts/train.py --config config/config.yaml
41
42 train-experiment:
43     python scripts/train.py --config config/config.yaml --experiment
44
45 # Evaluation
46 evaluate:
47     python scripts/evaluate.py --model models/final/model.pkl
48
49 # Data preparation
50 prepare-data:
51     python scripts/prepare_data.py

```

```

52
53 # Docker
54 docker-build:
55     docker build -t ml-production:latest -f docker/Dockerfile .
56
57 docker-run:
58     docker-compose -f docker/docker-compose.yml up
59
60 # Cleaning
61 clean:
62     find . -type f -name "*.pyc" -delete
63     find . -type d -name "__pycache__" -delete
64     find . -type d -name "*.egg-info" -exec rm -rf {} +
65     rm -rf build/ dist/ .pytest_cache/ .coverage htmlcov/
66
67 # Documentation
68 docs:
69     cd docs && make html

```

Listing 7.3: Makefile for common development tasks

7.4 Modular ML Architecture Patterns

Well-designed ML systems use object-oriented principles to create maintainable, testable code. This section presents core architectural patterns.

7.4.1 Base Model Interface

```

1 # src/models/base.py
2 """Base interfaces for ML models."""
3
4 from abc import ABC, abstractmethod
5 from typing import Any, Dict, Optional, Union
6 import numpy as np
7 import pandas as pd
8 from pathlib import Path
9 import joblib
10 import json
11
12
13 class BaseModel(ABC):
14     """
15     Abstract base class for all ML models.
16     Enforces consistent interface across different model types.
17     """
18
19     def __init__(self, config: Dict[str, Any]):
20         """
21         Initialize model with configuration.
22
23         Args:
24             config: Model configuration dictionary
25         """
26         self.config = config
27         self.model = None

```

```

28     self.is_fitted = False
29     self.feature_names = None
30     self.metadata = {
31         "model_type": self.__class__.__name__,
32         "config": config,
33         "training_history": []
34     }
35
36     @abstractmethod
37     def fit(
38         self,
39         X: Union[np.ndarray, pd.DataFrame],
40         y: Union[np.ndarray, pd.Series],
41         **kwargs
42     ) -> "BaseModel":
43         """
44         Train the model on provided data.
45
46         Args:
47             X: Feature matrix
48             y: Target vector
49             **kwargs: Additional training arguments
50
51         Returns:
52             self: Fitted model instance
53         """
54         pass
55
56     @abstractmethod
57     def predict(
58         self,
59         X: Union[np.ndarray, pd.DataFrame]
60     ) -> np.ndarray:
61         """
62         Generate predictions for input data.
63
64         Args:
65             X: Feature matrix
66
67         Returns:
68             Predictions array
69         """
70         pass
71
72     def predict_proba(
73         self,
74         X: Union[np.ndarray, pd.DataFrame]
75     ) -> np.ndarray:
76         """
77         Generate probability predictions (for classifiers).
78
79         Args:
80             X: Feature matrix
81
82         Returns:
83             Probability predictions
84         """
85         raise NotImplementedError(

```

```

86         f"{self.__class__.__name__} does not support probability predictions"
87     )
88
89     def save(self, path: Union[str, Path]) -> None:
90         """
91         Save model to disk.
92
93         Args:
94             path: Path to save model
95         """
96         path = Path(path)
97         path.parent.mkdir(parents=True, exist_ok=True)
98
99         # Save model object
100         model_path = path.with_suffix('.pkl')
101         joblib.dump(self.model, model_path)
102
103         # Save metadata
104         metadata_path = path.with_suffix('.json')
105         with open(metadata_path, 'w') as f:
106             # Convert non-serializable objects
107             serializable_metadata = self._make_serializable(self.metadata)
108             json.dump(serializable_metadata, f, indent=2)
109
110         # Save feature names if available
111         if self.feature_names is not None:
112             features_path = path.with_name(f"{path.stem}_features.json")
113             with open(features_path, 'w') as f:
114                 json.dump(self.feature_names, f)
115
116     @classmethod
117     def load(cls, path: Union[str, Path]) -> "BaseModel":
118         """
119         Load model from disk.
120
121         Args:
122             path: Path to saved model
123
124         Returns:
125             Loaded model instance
126         """
127         path = Path(path)
128
129         # Load metadata
130         metadata_path = path.with_suffix('.json')
131         with open(metadata_path, 'r') as f:
132             metadata = json.load(f)
133
134         # Create instance
135         instance = cls(metadata['config'])
136
137         # Load model object
138         model_path = path.with_suffix('.pkl')
139         instance.model = joblib.load(model_path)
140         instance.is_fitted = True
141         instance.metadata = metadata
142
143         # Load feature names if available

```



```

144     features_path = path.with_name(f"{path.stem}_features.json")
145     if features_path.exists():
146         with open(features_path, 'r') as f:
147             instance.feature_names = json.load(f)
148
149     return instance
150
151     def get_params(self) -> Dict[str, Any]:
152         """Get model parameters."""
153         return self.config
154
155     def set_params(self, **params) -> "BaseModel":
156         """Set model parameters."""
157         self.config.update(params)
158         return self
159
160     def _make_serializable(self, obj: Any) -> Any:
161         """Convert object to JSON-serializable format."""
162         if isinstance(obj, (np.integer, np.floating)):
163             return float(obj)
164         elif isinstance(obj, np.ndarray):
165             return obj.tolist()
166         elif isinstance(obj, pd.DataFrame):
167             return obj.to_dict()
168         elif isinstance(obj, dict):
169             return {k: self._make_serializable(v) for k, v in obj.items()}
170         elif isinstance(obj, (list, tuple)):
171             return [self._make_serializable(item) for item in obj]
172         else:
173             return obj
174
175     def __repr__(self) -> str:
176         params_str = ", ".join(f"{k}={v}" for k, v in self.config.items())
177         return f"{self.__class__.__name__}({params_str})"

```

Listing 7.4: Abstract base class for ML models

7.4.2 Concrete Model Implementations

```

1  # src/models/sklearn_models.py
2  """Scikit-learn model implementations."""
3
4  from typing import Any, Dict, Union, Optional
5  import numpy as np
6  import pandas as pd
7  from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
8  from sklearn.linear_model import LogisticRegression
9  from .base import BaseModel
10
11
12  class SklearnModel(BaseModel):
13      """
14      Wrapper for scikit-learn models with unified interface.
15      """
16
17      MODEL_REGISTRY = {
18          'random_forest': RandomForestClassifier,

```

```

19     'gradient_boosting': GradientBoostingClassifier,
20     'logistic_regression': LogisticRegression,
21 }
22
23 def __init__(self, config: Dict[str, Any]):
24     """
25     Initialize sklearn model.
26
27     Args:
28         config: Must contain 'model_type' key
29     """
30     super().__init__(config)
31
32     model_type = config.get('model_type')
33     if model_type not in self.MODEL_REGISTRY:
34         raise ValueError(
35             f"Unknown model type: {model_type}. "
36             f"Available: {list(self.MODEL_REGISTRY.keys())}"
37         )
38
39     # Extract model parameters
40     model_params = {
41         k: v for k, v in config.items()
42         if k not in ['model_type', 'random_state']
43     }
44
45     # Initialize model
46     model_class = self.MODEL_REGISTRY[model_type]
47     self.model = model_class(
48         random_state=config.get('random_state', 42),
49         **model_params
50     )
51
52 def fit(
53     self,
54     X: Union[np.ndarray, pd.DataFrame],
55     y: Union[np.ndarray, pd.Series],
56     validation_data: Optional[tuple] = None,
57     **kwargs
58 ) -> "SklearnModel":
59     """
60     Fit sklearn model.
61
62     Args:
63         X: Training features
64         y: Training targets
65         validation_data: Optional (X_val, y_val) tuple
66         **kwargs: Additional fit parameters
67
68     Returns:
69         self: Fitted model
70     """
71     # Store feature names if DataFrame
72     if isinstance(X, pd.DataFrame):
73         self.feature_names = X.columns.tolist()
74
75     # Fit model
76     self.model.fit(X, y, **kwargs)

```

```

77     self.is_fitted = True
78
79     # Record training metadata
80     self.metadata['training_history'].append({
81         'n_samples': len(X),
82         'n_features': X.shape[1],
83     })
84
85     # Evaluate on validation set if provided
86     if validation_data is not None:
87         X_val, y_val = validation_data
88         val_score = self.model.score(X_val, y_val)
89         self.metadata['validation_score'] = val_score
90
91     return self
92
93     def predict(self, X: Union[np.ndarray, pd.DataFrame]) -> np.ndarray:
94         """Generate predictions."""
95         if not self.is_fitted:
96             raise ValueError("Model must be fitted before prediction")
97
98         return self.model.predict(X)
99
100     def predict_proba(self, X: Union[np.ndarray, pd.DataFrame]) -> np.ndarray:
101         """Generate probability predictions."""
102         if not self.is_fitted:
103             raise ValueError("Model must be fitted before prediction")
104
105         if not hasattr(self.model, 'predict_proba'):
106             raise NotImplementedError(
107                 f"{self.config['model_type']} does not support predict_proba"
108             )
109
110         return self.model.predict_proba(X)
111
112     def get_feature_importance(self) -> Optional[pd.Series]:
113         """
114         Get feature importance if available.
115
116         Returns:
117         Series with feature importances or None
118         """
119         if not hasattr(self.model, 'feature_importances_'):
120             return None
121
122         importance = self.model.feature_importances_
123         if self.feature_names is not None:
124             return pd.Series(importance, index=self.feature_names).sort_values(
125                 ascending=False
126             )
127         else:
128             return pd.Series(importance).sort_values(ascending=False)

```

Listing 7.5: Sklearn-based model implementation

7.4.3 Model Factory Pattern

```

1  # src/models/__init__.py
2  """Model factory and registry."""
3
4  from typing import Dict, Any, Type
5  from .base import BaseModel
6  from .sklearn_models import SklearnModel
7
8
9  class ModelFactory:
10     """
11     Factory for creating model instances.
12     Implements registry pattern for extensibility.
13     """
14
15     _registry: Dict[str, Type[BaseModel]] = {
16         'sklearn': SklearnModel,
17     }
18
19     @classmethod
20     def register(cls, name: str, model_class: Type[BaseModel]) -> None:
21         """
22         Register a new model class.
23
24         Args:
25             name: Model identifier
26             model_class: Model class to register
27         """
28         if not issubclass(model_class, BaseModel):
29             raise ValueError(f"{model_class} must inherit from BaseModel")
30
31         cls._registry[name] = model_class
32
33     @classmethod
34     def create(cls, model_type: str, config: Dict[str, Any]) -> BaseModel:
35         """
36         Create model instance from configuration.
37
38         Args:
39             model_type: Type of model to create
40             config: Model configuration
41
42         Returns:
43             Model instance
44
45         Example:
46             >>> config = {'model_type': 'random_forest', 'n_estimators': 100}
47             >>> model = ModelFactory.create('sklearn', config)
48         """
49         if model_type not in cls._registry:
50             raise ValueError(
51                 f"Unknown model type: {model_type}. "
52                 f"Available: {list(cls._registry.keys())}"
53             )
54
55         model_class = cls._registry[model_type]
56         return model_class(config)
57

```

```

58     @classmethod
59     def available_models(cls) -> list:
60         """Get list of registered model types."""
61         return list(cls._registry.keys())
62
63
64 # Example: Register custom model
65 # from .custom_models import CustomModel
66 # ModelFactory.register('custom', CustomModel)

```

Listing 7.6: Factory pattern for model creation

7.4.4 Data Pipeline Architecture

```

1  # src/data/pipeline.py
2  """Data processing pipeline with composable transformations."""
3
4  from typing import List, Optional, Dict, Any
5  from abc import ABC, abstractmethod
6  import pandas as pd
7  import numpy as np
8  from sklearn.base import BaseEstimator, TransformerMixin
9
10
11 class DataTransformer(ABC):
12     """Base class for data transformations."""
13
14     def __init__(self, name: str):
15         self.name = name
16         self.is_fitted = False
17
18     @abstractmethod
19     def fit(self, df: pd.DataFrame) -> "DataTransformer":
20         """Learn parameters from data."""
21         pass
22
23     @abstractmethod
24     def transform(self, df: pd.DataFrame) -> pd.DataFrame:
25         """Apply transformation to data."""
26         pass
27
28     def fit_transform(self, df: pd.DataFrame) -> pd.DataFrame:
29         """Fit and transform in one step."""
30         return self.fit(df).transform(df)
31
32
33 class MissingValueImputer(DataTransformer):
34     """Impute missing values with configurable strategy."""
35
36     def __init__(
37         self,
38         strategy: str = 'mean',
39         columns: Optional[List[str]] = None
40     ):
41         super().__init__(name='MissingValueImputer')
42         self.strategy = strategy
43         self.columns = columns

```

```

44     self.fill_values = {}
45
46     def fit(self, df: pd.DataFrame) -> "MissingValueImputer":
47         """Learn imputation values from data."""
48         cols = self.columns or df.select_dtypes(include=[np.number]).columns
49
50         for col in cols:
51             if self.strategy == 'mean':
52                 self.fill_values[col] = df[col].mean()
53             elif self.strategy == 'median':
54                 self.fill_values[col] = df[col].median()
55             elif self.strategy == 'mode':
56                 self.fill_values[col] = df[col].mode()[0]
57             elif self.strategy == 'zero':
58                 self.fill_values[col] = 0
59             else:
60                 raise ValueError(f"Unknown strategy: {self.strategy}")
61
62         self.is_fitted = True
63         return self
64
65     def transform(self, df: pd.DataFrame) -> pd.DataFrame:
66         """Apply imputation."""
67         if not self.is_fitted:
68             raise ValueError("Transformer must be fitted before transform")
69
70         df = df.copy()
71         for col, value in self.fill_values.items():
72             if col in df.columns:
73                 df[col] = df[col].fillna(value)
74
75         return df
76
77
78 class OutlierClipper(DataTransformer):
79     """Clip outliers based on percentiles."""
80
81     def __init__(
82         self,
83         lower_percentile: float = 1,
84         upper_percentile: float = 99,
85         columns: Optional[List[str]] = None
86     ):
87         super().__init__(name='OutlierClipper')
88         self.lower_percentile = lower_percentile
89         self.upper_percentile = upper_percentile
90         self.columns = columns
91         self.clip_values = {}
92
93     def fit(self, df: pd.DataFrame) -> "OutlierClipper":
94         """Learn clipping thresholds."""
95         cols = self.columns or df.select_dtypes(include=[np.number]).columns
96
97         for col in cols:
98             lower = df[col].quantile(self.lower_percentile / 100)
99             upper = df[col].quantile(self.upper_percentile / 100)
100             self.clip_values[col] = (lower, upper)
101

```

```

102     self.is_fitted = True
103     return self
104
105     def transform(self, df: pd.DataFrame) -> pd.DataFrame:
106         """Apply clipping."""
107         if not self.is_fitted:
108             raise ValueError("Transformer must be fitted before transform")
109
110         df = df.copy()
111         for col, (lower, upper) in self.clip_values.items():
112             if col in df.columns:
113                 df[col] = df[col].clip(lower=lower, upper=upper)
114
115         return df
116
117
118 class DataPipeline:
119     """
120     Composable data processing pipeline.
121     Chains multiple transformers together.
122     """
123
124     def __init__(self, transformers: List[DataTransformer]):
125         """
126         Initialize pipeline with transformers.
127
128         Args:
129             transformers: List of DataTransformer instances
130         """
131         self.transformers = transformers
132         self.is_fitted = False
133
134     def fit(self, df: pd.DataFrame) -> "DataPipeline":
135         """Fit all transformers sequentially."""
136         data = df.copy()
137
138         for transformer in self.transformers:
139             transformer.fit(data)
140             data = transformer.transform(data)
141
142         self.is_fitted = True
143         return self
144
145     def transform(self, df: pd.DataFrame) -> pd.DataFrame:
146         """Apply all transformers sequentially."""
147         if not self.is_fitted:
148             raise ValueError("Pipeline must be fitted before transform")
149
150         data = df.copy()
151         for transformer in self.transformers:
152             data = transformer.transform(data)
153
154         return data
155
156     def fit_transform(self, df: pd.DataFrame) -> pd.DataFrame:
157         """Fit and transform in one step."""
158         return self.fit(df).transform(df)
159

```

```

160     def add_transformer(self, transformer: DataTransformer) -> "DataPipeline":
161         """Add transformer to pipeline."""
162         self.transformers.append(transformer)
163         self.is_fitted = False # Need to refit
164         return self
165
166
167 # Example usage
168 if __name__ == "__main__":
169     # Create sample data
170     df = pd.DataFrame({
171         'age': [25, 30, np.nan, 45, 200], # Has missing and outlier
172         'income': [50000, 60000, 55000, np.nan, 80000]
173     })
174
175     # Build pipeline
176     pipeline = DataPipeline([
177         MissingValueImputer(strategy='median'),
178         OutlierClipper(lower_percentile=5, upper_percentile=95)
179     ])
180
181     # Fit and transform
182     df_clean = pipeline.fit_transform(df)
183     print(df_clean)

```

Listing 7.7: Modular data processing pipeline

7.5 Testing Strategies for ML Code

Comprehensive testing is essential for reliable ML systems. This section covers unit tests, integration tests, and model-specific testing strategies.

7.5.1 Unit Testing for ML Components

```

1 # tests/unit/test_data.py
2 """Unit tests for data processing components."""
3
4 import pytest
5 import pandas as pd
6 import numpy as np
7 from src.data.pipeline import (
8     MissingValueImputer,
9     OutlierClipper,
10     DataPipeline
11 )
12
13
14 class TestMissingValueImputer:
15     """Test suite for missing value imputation."""
16
17     @pytest.fixture
18     def sample_data(self):
19         """Create sample data with missing values."""
20         return pd.DataFrame({
21             'a': [1, 2, np.nan, 4, 5],

```



```

22         'b': [10, np.nan, 30, 40, 50],
23         'c': ['x', 'y', 'z', 'x', 'y']
24     })
25
26     def test_mean_imputation(self, sample_data):
27         """Test mean imputation strategy."""
28         imputer = MissingValueImputer(strategy='mean', columns=['a', 'b'])
29         result = imputer.fit_transform(sample_data)
30
31         # Check no missing values remain
32         assert not result['a'].isna().any()
33         assert not result['b'].isna().any()
34
35         # Check imputed values are correct
36         expected_a = sample_data['a'].mean()
37         assert result.loc[2, 'a'] == pytest.approx(expected_a)
38
39     def test_median_imputation(self, sample_data):
40         """Test median imputation strategy."""
41         imputer = MissingValueImputer(strategy='median', columns=['a'])
42         result = imputer.fit_transform(sample_data)
43
44         expected = sample_data['a'].median()
45         assert result.loc[2, 'a'] == expected
46
47     def test_fit_before_transform_required(self, sample_data):
48         """Test that transform fails without fit."""
49         imputer = MissingValueImputer(strategy='mean')
50
51         with pytest.raises(ValueError, match="must be fitted"):
52             imputer.transform(sample_data)
53
54     def test_unknown_strategy_raises_error(self, sample_data):
55         """Test that unknown strategy raises error."""
56         imputer = MissingValueImputer(strategy='unknown')
57
58         with pytest.raises(ValueError, match="Unknown strategy"):
59             imputer.fit(sample_data)
60
61
62     class TestOutlierClipper:
63         """Test suite for outlier clipping."""
64
65         @pytest.fixture
66         def sample_data(self):
67             """Create sample data with outliers."""
68             return pd.DataFrame({
69                 'value': [1, 2, 3, 4, 5, 100, 200] # 100, 200 are outliers
70             })
71
72     def test_percentile_clipping(self, sample_data):
73         """Test clipping at specified percentiles."""
74         clipper = OutlierClipper(
75             lower_percentile=10,
76             upper_percentile=90,
77             columns=['value']
78         )
79         result = clipper.fit_transform(sample_data)

```

```

80
81     # Check max value is clipped
82     assert result['value'].max() < 100
83
84     def test_clipping_preserves_shape(self, sample_data):
85         """Test that clipping preserves DataFrame shape."""
86         clipper = OutlierClipper()
87         result = clipper.fit_transform(sample_data)
88
89         assert result.shape == sample_data.shape
90
91
92     class TestDataPipeline:
93         """Test suite for data pipeline."""
94
95         @pytest.fixture
96         def sample_data(self):
97             """Create sample data."""
98             return pd.DataFrame({
99                 'a': [1, np.nan, 3, 4, 100],
100                 'b': [10, 20, np.nan, 40, 50]
101             })
102
103         def test_pipeline_execution_order(self, sample_data):
104             """Test transformers execute in correct order."""
105             pipeline = DataPipeline([
106                 MissingValueImputer(strategy='mean'),
107                 OutlierClipper(lower_percentile=5, upper_percentile=95)
108             ])
109
110             result = pipeline.fit_transform(sample_data)
111
112             # Check no missing values (imputation ran)
113             assert not result.isna().any().any()
114
115             # Check outliers clipped (clipping ran after imputation)
116             assert result['a'].max() < 100
117
118         def test_pipeline_fit_required(self, sample_data):
119             """Test pipeline requires fitting before transform."""
120             pipeline = DataPipeline([MissingValueImputer()])
121
122             with pytest.raises(ValueError, match="must be fitted"):
123                 pipeline.transform(sample_data)

```

Listing 7.8: Unit tests for data processing

7.5.2 Integration Testing for ML Pipelines

```

1  # tests/integration/test_training.py
2  """Integration tests for complete training pipeline."""
3
4  import pytest
5  import pandas as pd
6  import numpy as np
7  from pathlib import Path
8  import tempfile

```

```

9 from src.models import ModelFactory
10 from src.data.pipeline import DataPipeline, MissingValueImputer
11 from src.training.trainer import Trainer
12
13
14 class TestTrainingPipeline:
15     """Test complete training pipeline end-to-end."""
16
17     @pytest.fixture
18     def training_data(self):
19         """Generate synthetic training data."""
20         np.random.seed(42)
21         n_samples = 1000
22
23         X = pd.DataFrame({
24             'feature_1': np.random.randn(n_samples),
25             'feature_2': np.random.randn(n_samples),
26             'feature_3': np.random.choice(['A', 'B', 'C'], n_samples)
27         })
28
29         # Create target with some relationship to features
30         y = (
31             X['feature_1'] * 0.5 +
32             X['feature_2'] * 0.3 +
33             np.random.randn(n_samples) * 0.1
34         )
35         y = (y > y.median()).astype(int)
36
37         return X, y
38
39     def test_end_to_end_training(self, training_data):
40         """Test complete training workflow."""
41         X, y = training_data
42
43         # Create data pipeline
44         data_pipeline = DataPipeline([
45             MissingValueImputer(strategy='mean')
46         ])
47
48         # Create model
49         model = ModelFactory.create('sklearn', {
50             'model_type': 'random_forest',
51             'n_estimators': 10, # Small for fast testing
52             'max_depth': 3,
53             'random_state': 42
54         })
55
56         # Prepare data
57         X_processed = data_pipeline.fit_transform(X)
58
59         # Train model
60         model.fit(X_processed, y)
61
62         # Test predictions
63         predictions = model.predict(X_processed)
64
65         assert len(predictions) == len(y)
66         assert set(predictions).issubset({0, 1})

```

```

67     # Test prediction accuracy (should be > random)
68     accuracy = (predictions == y).mean()
69     assert accuracy > 0.6 # Better than random for this synthetic data
70
71
72     def test_model_save_and_load(self, training_data):
73         """Test model persistence."""
74         X, y = training_data
75
76         # Train model
77         model = ModelFactory.create('sklearn', {
78             'model_type': 'logistic_regression',
79             'random_state': 42
80         })
81         model.fit(X.select_dtypes(include=[np.number]), y)
82
83         # Save model
84         with tempfile.TemporaryDirectory() as tmpdir:
85             model_path = Path(tmpdir) / 'test_model'
86             model.save(model_path)
87
88         # Load model
89         loaded_model = model.__class__.load(model_path)
90
91         # Test loaded model makes same predictions
92         X_numeric = X.select_dtypes(include=[np.number])
93         original_preds = model.predict(X_numeric)
94         loaded_preds = loaded_model.predict(X_numeric)
95
96         np.testing.assert_array_equal(original_preds, loaded_preds)
97
98     def test_pipeline_with_validation_split(self, training_data):
99         """Test training with validation set."""
100        X, y = training_data
101
102        # Split data
103        split_idx = int(len(X) * 0.8)
104        X_train, X_val = X[:split_idx], X[split_idx:]
105        y_train, y_val = y[:split_idx], y[split_idx:]
106
107        # Train with validation
108        model = ModelFactory.create('sklearn', {
109            'model_type': 'random_forest',
110            'n_estimators': 10,
111            'random_state': 42
112        })
113
114        X_train_num = X_train.select_dtypes(include=[np.number])
115        X_val_num = X_val.select_dtypes(include=[np.number])
116
117        model.fit(
118            X_train_num,
119            y_train,
120            validation_data=(X_val_num, y_val)
121        )
122
123        # Check validation score was recorded
124        assert 'validation_score' in model.metadata

```

```
125 assert 0 <= model.metadata['validation_score'] <= 1
```

Listing 7.9: Integration tests for training pipeline

7.5.3 Model-Specific Tests

```
1  # tests/model/test_performance.py
2  """Model performance and behavior tests."""
3
4  import pytest
5  import pandas as pd
6  import numpy as np
7  from sklearn.datasets import make_classification, make_regression
8  from sklearn.model_selection import cross_val_score
9  from src.models import ModelFactory
10
11
12  class TestModelPerformance:
13      """Test model performance meets requirements."""
14
15      @pytest.fixture
16      def classification_data(self):
17          """Generate classification dataset."""
18          X, y = make_classification(
19              n_samples=1000,
20              n_features=20,
21              n_informative=15,
22              n_redundant=5,
23              random_state=42
24          )
25          return pd.DataFrame(X), pd.Series(y)
26
27      def test_random_forest_minimum_accuracy(self, classification_data):
28          """Test Random Forest achieves minimum accuracy."""
29          X, y = classification_data
30
31          model = ModelFactory.create('sklearn', {
32              'model_type': 'random_forest',
33              'n_estimators': 100,
34              'random_state': 42
35          })
36
37          # Cross-validation
38          scores = cross_val_score(
39              model.model, # Access underlying sklearn model
40              X, y,
41              cv=5,
42              scoring='accuracy'
43          )
44
45          # Ensure minimum performance
46          assert scores.mean() > 0.85, \
47              f"Model accuracy {scores.mean():.3f} below threshold 0.85"
48
49      def test_model_prediction_consistency(self, classification_data):
50          """Test model makes consistent predictions."""
51          X, y = classification_data
```

```

52     model = ModelFactory.create('sklearn', {
53         'model_type': 'random_forest',
54         'n_estimators': 50,
55         'random_state': 42
56     })
57
58
59     model.fit(X, y)
60
61     # Same input should give same output
62     pred1 = model.predict(X.head(10))
63     pred2 = model.predict(X.head(10))
64
65     np.testing.assert_array_equal(pred1, pred2)
66
67     def test_probability_calibration(self, classification_data):
68         """Test probability predictions are well-calibrated."""
69         X, y = classification_data
70
71         model = ModelFactory.create('sklearn', {
72             'model_type': 'random_forest',
73             'n_estimators': 100,
74             'random_state': 42
75         })
76
77         model.fit(X, y)
78         probas = model.predict_proba(X)
79
80         # Check probabilities sum to 1
81         np.testing.assert_array_almost_equal(
82             probas.sum(axis=1),
83             np.ones(len(probas)),
84             decimal=5
85         )
86
87         # Check probabilities are in valid range
88         assert (probas >= 0).all() and (probas <= 1).all()
89
90
91     class TestModelInvariants:
92         """Test model invariants and edge cases."""
93
94         def test_single_sample_prediction(self):
95             """Test model can predict single sample."""
96             X = pd.DataFrame({'a': [1, 2, 3], 'b': [4, 5, 6]})
97             y = pd.Series([0, 1, 0])
98
99             model = ModelFactory.create('sklearn', {
100                 'model_type': 'logistic_regression',
101                 'random_state': 42
102             })
103             model.fit(X, y)
104
105             # Predict single sample
106             single_pred = model.predict(X.head(1))
107             assert len(single_pred) == 1
108
109         def test_empty_prediction_raises_error(self):

```

```

110     """Test prediction with empty data raises appropriate error."""
111     X = pd.DataFrame({'a': [1, 2, 3], 'b': [4, 5, 6]})
112     y = pd.Series([0, 1, 0])
113
114     model = ModelFactory.create('sklearn', {
115         'model_type': 'logistic_regression',
116         'random_state': 42
117     })
118     model.fit(X, y)
119
120     # Try to predict empty DataFrame
121     empty_X = pd.DataFrame(columns=['a', 'b'])
122
123     with pytest.raises(ValueError):
124         model.predict(empty_X)
125
126     def test_feature_importance_availability(self):
127         """Test feature importance is available for tree models."""
128         X = pd.DataFrame({'a': [1, 2, 3, 4], 'b': [5, 6, 7, 8]})
129         y = pd.Series([0, 1, 0, 1])
130
131         model = ModelFactory.create('sklearn', {
132             'model_type': 'random_forest',
133             'n_estimators': 10,
134             'random_state': 42
135         })
136         model.fit(X, y)
137
138         importance = model.get_feature_importance()
139
140         assert importance is not None
141         assert len(importance) == 2 # Two features
142         assert all(importance >= 0)
143         assert importance.index.tolist() == ['a', 'b']

```

Listing 7.10: Tests for model behavior and performance

7.5.4 Test Configuration with Pytest

```

1  # tests/conftest.py
2  """Shared pytest fixtures and configuration."""
3
4  import pytest
5  import pandas as pd
6  import numpy as np
7  from pathlib import Path
8  import tempfile
9  import shutil
10
11
12  @pytest.fixture(scope="session")
13  def test_data_dir():
14      """Create temporary directory for test data."""
15      tmpdir = tempfile.mkdtemp()
16      yield Path(tmpdir)
17      shutil.rmtree(tmpdir)
18

```

```

19
20 @pytest.fixture
21 def sample_dataframe():
22     """Create sample DataFrame for testing."""
23     np.random.seed(42)
24     return pd.DataFrame({
25         'numeric_1': np.random.randn(100),
26         'numeric_2': np.random.uniform(0, 100, 100),
27         'categorical': np.random.choice(['A', 'B', 'C'], 100),
28         'target': np.random.binomial(1, 0.5, 100)
29     })
30
31
32 @pytest.fixture
33 def model_config():
34     """Standard model configuration for testing."""
35     return {
36         'model_type': 'random_forest',
37         'n_estimators': 10,
38         'max_depth': 3,
39         'random_state': 42
40     }
41
42
43 def pytest_configure(config):
44     """Register custom markers."""
45     config.addinvalue_line(
46         "markers",
47         "slow: marks tests as slow (deselect with '-m \"not slow\"')\"
48     )
49     config.addinvalue_line(
50         "markers",
51         "integration: marks tests as integration tests"
52     )
53     config.addinvalue_line(
54         "markers",
55         "model: marks tests as model performance tests"
56     )

```

Listing 7.11: Pytest configuration and fixtures

7.6 Design Patterns for ML Systems

Design patterns provide proven solutions to common problems in ML system design. This section presents patterns specifically adapted for ML workflows.

7.6.1 Strategy Pattern for Model Selection

```

1 # src/training/strategies.py
2 """Training strategies using Strategy pattern."""
3
4 from abc import ABC, abstractmethod
5 from typing import Dict, Any, Tuple
6 import pandas as pd
7 import numpy as np

```



```

8 from sklearn.model_selection import GridSearchCV, RandomizedSearchCV
9 from src.models.base import BaseModel
10
11
12 class TrainingStrategy(ABC):
13     """Abstract base class for training strategies."""
14
15     @abstractmethod
16     def train(
17         self,
18         model: BaseModel,
19         X_train: pd.DataFrame,
20         y_train: pd.Series,
21         X_val: pd.DataFrame,
22         y_val: pd.Series
23     ) -> BaseModel:
24         """Execute training strategy."""
25         pass
26
27
28 class SimpleTrainingStrategy(TrainingStrategy):
29     """Simple training without hyperparameter optimization."""
30
31     def train(
32         self,
33         model: BaseModel,
34         X_train: pd.DataFrame,
35         y_train: pd.Series,
36         X_val: pd.DataFrame,
37         y_val: pd.Series
38     ) -> BaseModel:
39         """Train model with default parameters."""
40         model.fit(X_train, y_train, validation_data=(X_val, y_val))
41         return model
42
43
44 class GridSearchStrategy(TrainingStrategy):
45     """Grid search hyperparameter optimization."""
46
47     def __init__(self, param_grid: Dict[str, list], cv: int = 5):
48         """
49         Initialize grid search strategy.
50
51         Args:
52             param_grid: Parameter grid for search
53             cv: Number of cross-validation folds
54         """
55         self.param_grid = param_grid
56         self.cv = cv
57
58     def train(
59         self,
60         model: BaseModel,
61         X_train: pd.DataFrame,
62         y_train: pd.Series,
63         X_val: pd.DataFrame,
64         y_val: pd.Series
65     ) -> BaseModel:

```

```

66     """Train with grid search."""
67     # Use sklearn's GridSearchCV on underlying model
68     grid_search = GridSearchCV(
69         model.model,
70         self.param_grid,
71         cv=self.cv,
72         scoring='accuracy',
73         n_jobs=-1
74     )
75
76     grid_search.fit(X_train, y_train)
77
78     # Update model with best parameters
79     model.model = grid_search.best_estimator_
80     model.is_fitted = True
81     model.metadata['best_params'] = grid_search.best_params_
82     model.metadata['best_score'] = grid_search.best_score_
83
84     return model
85
86
87 class EarlyStoppingStrategy(TrainingStrategy):
88     """Training with early stopping (for iterative models)."""
89
90     def __init__(
91         self,
92         patience: int = 5,
93         min_delta: float = 0.001
94     ):
95         """
96         Initialize early stopping strategy.
97
98         Args:
99             patience: Number of epochs to wait for improvement
100             min_delta: Minimum change to qualify as improvement
101         """
102         self.patience = patience
103         self.min_delta = min_delta
104
105     def train(
106         self,
107         model: BaseModel,
108         X_train: pd.DataFrame,
109         y_train: pd.Series,
110         X_val: pd.DataFrame,
111         y_val: pd.Series
112     ) -> BaseModel:
113         """Train with early stopping logic."""
114         # For sklearn models with iterative training
115         if hasattr(model.model, 'n_estimators'):
116             best_score = -np.inf
117             patience_counter = 0
118
119             for n_est in range(10, 201, 10):
120                 model.model.set_params(n_estimators=n_est)
121                 model.fit(X_train, y_train)
122
123                 val_score = model.model.score(X_val, y_val)

```

```

124         if val_score > best_score + self.min_delta:
125             best_score = val_score
126             patience_counter = 0
127         else:
128             patience_counter += 1
129
130         if patience_counter >= self.patience:
131             print(f"Early stopping at {n_est} estimators")
132             break
133
134         model.metadata['early_stopped_at'] = n_est
135         model.metadata['best_val_score'] = best_score
136
137     else:
138         # Fallback to simple training
139         model.fit(X_train, y_train, validation_data=(X_val, y_val))
140
141     return model
142
143
144 class TrainingContext:
145     """Context that uses a training strategy."""
146
147     def __init__(self, strategy: TrainingStrategy):
148         """
149         Initialize with specific strategy.
150
151         Args:
152             strategy: Training strategy to use
153         """
154         self.strategy = strategy
155
156     def set_strategy(self, strategy: TrainingStrategy):
157         """Change strategy at runtime."""
158         self.strategy = strategy
159
160     def execute_training(
161         self,
162         model: BaseModel,
163         X_train: pd.DataFrame,
164         y_train: pd.Series,
165         X_val: pd.DataFrame,
166         y_val: pd.Series
167     ) -> BaseModel:
168         """Execute training using current strategy."""
169         return self.strategy.train(model, X_train, y_train, X_val, y_val)
170
171
172 # Example usage
173 if __name__ == "__main__":
174     from src.models import ModelFactory
175
176     # Create model
177     model = ModelFactory.create('sklearn', {
178         'model_type': 'random_forest',
179         'random_state': 42
180     })
181

```

```

182
183     # Sample data
184     from sklearn.datasets import make_classification
185     X, y = make_classification(n_samples=1000, random_state=42)
186     X_train, X_val = X[:800], X[800:]
187     y_train, y_val = y[:800], y[800:]
188
189     X_train = pd.DataFrame(X_train)
190     X_val = pd.DataFrame(X_val)
191     y_train = pd.Series(y_train)
192     y_val = pd.Series(y_val)
193
194     # Use different strategies
195     context = TrainingContext(SimpleTrainingStrategy())
196     model = context.execute_training(model, X_train, y_train, X_val, y_val)
197
198     # Switch to grid search
199     context.set_strategy(GridSearchStrategy(
200         param_grid={'n_estimators': [10, 50, 100], 'max_depth': [3, 5, 7]}
201     ))
202     model = context.execute_training(model, X_train, y_train, X_val, y_val)

```

Listing 7.12: Strategy pattern for flexible model selection

7.6.2 Observer Pattern for Training Callbacks

```

1  # src/training/callbacks.py
2  """Training callbacks using Observer pattern."""
3
4  from abc import ABC, abstractmethod
5  from typing import Dict, Any, List
6  import time
7  from datetime import datetime
8
9
10 class TrainingCallback(ABC):
11     """Abstract base class for training callbacks."""
12
13     @abstractmethod
14     def on_train_begin(self, logs: Dict[str, Any] = None):
15         """Called at the beginning of training."""
16         pass
17
18     @abstractmethod
19     def on_train_end(self, logs: Dict[str, Any] = None):
20         """Called at the end of training."""
21         pass
22
23     @abstractmethod
24     def on_epoch_begin(self, epoch: int, logs: Dict[str, Any] = None):
25         """Called at the beginning of an epoch."""
26         pass
27
28     @abstractmethod
29     def on_epoch_end(self, epoch: int, logs: Dict[str, Any] = None):
30         """Called at the end of an epoch."""
31         pass

```

```

32
33
34 class LoggingCallback(TrainingCallback):
35     """Log training progress."""
36
37     def on_train_begin(self, logs: Dict[str, Any] = None):
38         """Log training start."""
39         print(f"[{datetime.now()}] Training started")
40         print(f"Configuration: {logs.get('config', {})}")
41
42     def on_train_end(self, logs: Dict[str, Any] = None):
43         """Log training completion."""
44         duration = logs.get('duration', 0)
45         print(f"[{datetime.now()}] Training completed in {duration:.2f}s")
46
47     def on_epoch_begin(self, epoch: int, logs: Dict[str, Any] = None):
48         """Log epoch start."""
49         print(f"\nEpoch {epoch}/{logs.get('total_epochs', '?')} started")
50
51     def on_epoch_end(self, epoch: int, logs: Dict[str, Any] = None):
52         """Log epoch metrics."""
53         metrics_str = ", ".join(
54             f"{k}={v:.4f}" for k, v in logs.items()
55             if k not in ['total_epochs']
56         )
57         print(f"Epoch {epoch} - {metrics_str}")
58
59
60 class MetricTrackingCallback(TrainingCallback):
61     """Track metrics across epochs."""
62
63     def __init__(self):
64         self.history = {
65             'epoch': [],
66             'train_loss': [],
67             'val_loss': [],
68             'train_acc': [],
69             'val_acc': []
70         }
71
72     def on_train_begin(self, logs: Dict[str, Any] = None):
73         """Reset history."""
74         self.history = {k: [] for k in self.history.keys()}
75
76     def on_train_end(self, logs: Dict[str, Any] = None):
77         """Store final history."""
78         pass
79
80     def on_epoch_begin(self, epoch: int, logs: Dict[str, Any] = None):
81         """No action needed."""
82         pass
83
84     def on_epoch_end(self, epoch: int, logs: Dict[str, Any] = None):
85         """Record metrics."""
86         self.history['epoch'].append(epoch)
87         for key in ['train_loss', 'val_loss', 'train_acc', 'val_acc']:
88             value = logs.get(key, None)
89             self.history[key].append(value)

```

```

90
91
92 class EarlyStoppingCallback(TrainingCallback):
93     """Stop training when metric stops improving."""
94
95     def __init__(
96         self,
97         monitor: str = 'val_loss',
98         patience: int = 5,
99         min_delta: float = 0.001,
100         mode: str = 'min'
101     ):
102         """
103         Initialize early stopping.
104
105         Args:
106             monitor: Metric to monitor
107             patience: Epochs to wait for improvement
108             min_delta: Minimum change to qualify as improvement
109             mode: 'min' or 'max' (whether lower or higher is better)
110         """
111         self.monitor = monitor
112         self.patience = patience
113         self.min_delta = min_delta
114         self.mode = mode
115
116         self.best_value = float('inf') if mode == 'min' else float('-inf')
117         self.wait = 0
118         self.stop_training = False
119
120     def on_train_begin(self, logs: Dict[str, Any] = None):
121         """Reset state."""
122         self.best_value = float('inf') if self.mode == 'min' else float('-inf')
123         self.wait = 0
124         self.stop_training = False
125
126     def on_train_end(self, logs: Dict[str, Any] = None):
127         """No action needed."""
128         pass
129
130     def on_epoch_begin(self, epoch: int, logs: Dict[str, Any] = None):
131         """No action needed."""
132         pass
133
134     def on_epoch_end(self, epoch: int, logs: Dict[str, Any] = None):
135         """Check if should stop."""
136         current_value = logs.get(self.monitor)
137         if current_value is None:
138             return
139
140         if self.mode == 'min':
141             improved = current_value < (self.best_value - self.min_delta)
142         else:
143             improved = current_value > (self.best_value + self.min_delta)
144
145         if improved:
146             self.best_value = current_value
147             self.wait = 0

```

```

148         else:
149             self.wait += 1
150             if self.wait >= self.patience:
151                 self.stop_training = True
152                 print(f"\nEarly stopping triggered at epoch {epoch}")
153
154
155 class CallbackList:
156     """Manages multiple callbacks."""
157
158     def __init__(self, callbacks: List[TrainingCallback] = None):
159         """Initialize with list of callbacks."""
160         self.callbacks = callbacks or []
161
162     def append(self, callback: TrainingCallback):
163         """Add callback to list."""
164         self.callbacks.append(callback)
165
166     def on_train_begin(self, logs: Dict[str, Any] = None):
167         """Notify all callbacks."""
168         logs = logs or {}
169         for callback in self.callbacks:
170             callback.on_train_begin(logs)
171
172     def on_train_end(self, logs: Dict[str, Any] = None):
173         """Notify all callbacks."""
174         logs = logs or {}
175         for callback in self.callbacks:
176             callback.on_train_end(logs)
177
178     def on_epoch_begin(self, epoch: int, logs: Dict[str, Any] = None):
179         """Notify all callbacks."""
180         logs = logs or {}
181         for callback in self.callbacks:
182             callback.on_epoch_begin(epoch, logs)
183
184     def on_epoch_end(self, epoch: int, logs: Dict[str, Any] = None):
185         """Notify all callbacks."""
186         logs = logs or {}
187         for callback in self.callbacks:
188             callback.on_epoch_end(epoch, logs)
189
190     def should_stop(self) -> bool:
191         """Check if any callback requested stop."""
192         return any(
193             getattr(cb, 'stop_training', False)
194             for cb in self.callbacks
195         )

```

Listing 7.13: Observer pattern for monitoring training

7.7 Code Quality Tools and Practices

Maintaining high code quality requires automated tools and consistent practices. This section covers essential tools for ML codebases.

7.7.1 Type Hints and Static Analysis

```

1  # src/models/typed_model.py
2  """Example of properly type-hinted ML code."""
3
4  from typing import Union, Optional, Dict, Any, List, Tuple
5  import numpy as np
6  import pandas as pd
7  from numpy.typing import NDArray
8  from pathlib import Path
9
10
11  # Type aliases for clarity
12  Features = Union[NDArray[np.float64], pd.DataFrame]
13  Target = Union[NDArray[np.int_], pd.Series]
14  Predictions = NDArray[np.int_]
15  Probabilities = NDArray[np.float64]
16
17
18  class TypedModel:
19      """Model with comprehensive type hints."""
20
21      def __init__(self, config: Dict[str, Any]) -> None:
22          """
23              Initialize model with configuration.
24
25              Args:
26                  config: Model configuration dictionary
27          """
28          self.config: Dict[str, Any] = config
29          self.is_fitted: bool = False
30          self.feature_names_: Optional[List[str]] = None
31
32      def fit(
33          self,
34          X: Features,
35          y: Target,
36          sample_weight: Optional[NDArray[np.float64]] = None
37      ) -> "TypedModel":
38          """
39              Fit model to training data.
40
41              Args:
42                  X: Training features
43                  y: Training targets
44                  sample_weight: Optional sample weights
45
46              Returns:
47                  Fitted model instance
48
49              Raises:
50                  ValueError: If X and y have incompatible shapes
51          """
52          # Type-based logic
53          if isinstance(X, pd.DataFrame):
54              self.feature_names_ = X.columns.tolist()
55
56          self.is_fitted = True

```



```

57         return self
58
59     def predict(self, X: Features) -> Predictions:
60         """
61         Generate predictions.
62
63         Args:
64             X: Feature matrix
65
66         Returns:
67             Prediction array
68
69         Raises:
70             ValueError: If model is not fitted
71         """
72         if not self.is_fitted:
73             raise ValueError("Model must be fitted before prediction")
74
75         # Implementation here
76         n_samples = len(X)
77         return np.zeros(n_samples, dtype=np.int_)
78
79     def predict_proba(self, X: Features) -> Probabilities:
80         """
81         Generate probability predictions.
82
83         Args:
84             X: Feature matrix
85
86         Returns:
87             Probability array of shape (n_samples, n_classes)
88         """
89         if not self.is_fitted:
90             raise ValueError("Model must be fitted before prediction")
91
92         # Implementation here
93         n_samples = len(X)
94         return np.zeros((n_samples, 2), dtype=np.float64)
95
96
97     # Type-hinted utility functions
98     def train_test_split_typed(
99         X: Features,
100         y: Target,
101         test_size: float = 0.2,
102         random_state: Optional[int] = None
103     ) -> Tuple[Features, Features, Target, Target]:
104         """
105         Type-safe train-test split.
106
107         Args:
108             X: Features
109             y: Targets
110             test_size: Fraction for test set
111             random_state: Random seed
112
113         Returns:
114             Tuple of (X_train, X_test, y_train, y_test)

```

```

115     """
116     # Implementation
117     pass
118
119
120 def evaluate_model(
121     model: TypedModel,
122     X: Features,
123     y: Target,
124     metrics: List[str]
125 ) -> Dict[str, float]:
126     """
127     Evaluate model with specified metrics.
128
129     Args:
130         model: Fitted model
131         X: Features
132         y: True targets
133         metrics: List of metric names
134
135     Returns:
136         Dictionary mapping metric names to values
137     """
138     # Implementation
139     pass

```

Listing 7.14: Type hints for ML code

7.7.2 Configuration and linting setup

```

1  # pyproject.toml
2  [tool.black]
3  line-length = 100
4  target-version = ['py39']
5  include = '\.pyi?$'
6  exclude = ''
7  /(
8      \.git
9      | \.venv
10     | build
11     | dist
12 )/
13 ''
14
15 [tool.isort]
16 profile = "black"
17 line_length = 100
18 skip = [".gitignore", ".dockerignore"]
19 skip_glob = ["*/migrations/*"]
20
21 [tool.mypy]
22 python_version = "3.9"
23 warn_return_any = true
24 warn_unused_configs = true
25 disallow_untyped_defs = false
26 disallow_incomplete_defs = true
27 check_untyped_defs = true

```

```

28 no_implicit_optional = true
29 warn_redundant_casts = true
30 warn_unused_ignores = true
31 warn_no_return = true
32 warn_unreachable = true
33 strict_equality = true
34
35 [[tool.mypy.overrides]]
36 module = "sklearn.*"
37 ignore_missing_imports = true
38
39 [[tool.mypy.overrides]]
40 module = "pandas.*"
41 ignore_missing_imports = true
42
43 [tool.pytest.ini_options]
44 testpaths = ["tests"]
45 python_files = "test_*.py"
46 python_classes = "Test*"
47 python_functions = "test_*"
48 addopts = """
49     --verbose
50     --cov=src
51     --cov-report=html
52     --cov-report=term-missing
53     --cov-fail-under=80
54 """
55 markers = [
56     "slow: marks tests as slow",
57     "integration: marks integration tests",
58     "model: marks model performance tests"
59 ]
60
61 [tool.coverage.run]
62 source = ["src"]
63 omit = [
64     "*/tests/*",
65     "*/test_*.py",
66     "*/__init__.py"
67 ]
68
69 [tool.coverage.report]
70 exclude_lines = [
71     "pragma: no cover",
72     "def __repr__",
73     "raise AssertionError",
74     "raise NotImplementedError",
75     "if __name__ == '__main__':",
76     "if TYPE_CHECKING:",
77     "@abstractmethod"
78 ]

```

Listing 7.15: Pyproject.toml with tool configurations

```

1 # .pre-commit-config.yaml
2 repos:
3   - repo: https://github.com/pre-commit/pre-commit-hooks
4     rev: v4.4.0

```

```

5     hooks:
6         - id: trailing-whitespace
7         - id: end-of-file-fixer
8         - id: check-yaml
9         - id: check-added-large-files
10        args: ['--maxkb=1000']
11        - id: check-json
12        - id: check-merge-conflict
13
14    - repo: https://github.com/psf/black
15      rev: 23.1.0
16      hooks:
17          - id: black
18            language_version: python3.9
19
20    - repo: https://github.com/pycqa/isort
21      rev: 5.12.0
22      hooks:
23          - id: isort
24
25    - repo: https://github.com/pycqa/flake8
26      rev: 6.0.0
27      hooks:
28          - id: flake8
29            args: ['--max-line-length=100', '--ignore=E203,W503']
30
31    - repo: https://github.com/pre-commit/mirrors-mypy
32      rev: v1.0.1
33      hooks:
34          - id: mypy
35            additional_dependencies: [types-all]
36            args: [--ignore-missing-imports]

```

Listing 7.16: Pre-commit hooks configuration

7.8 Reproducibility Frameworks

Reproducibility is critical for ML systems. This section presents frameworks for ensuring consistent, reproducible results.

7.8.1 Configuration Management

```

1  # src/utils/config.py
2  """Configuration management for reproducible experiments."""
3
4  from typing import Any, Dict, Optional
5  from pathlib import Path
6  import yaml
7  import json
8  from dataclasses import dataclass, asdict, field
9  from datetime import datetime
10 import hashlib
11
12
13 @dataclass

```

```

14 class DataConfig:
15     """Data configuration."""
16     train_path: str
17     val_path: str
18     test_path: Optional[str] = None
19     features: Optional[list] = None
20     target: str = "target"
21
22
23 @dataclass
24 class ModelConfig:
25     """Model configuration."""
26     model_type: str
27     model_class: str
28     parameters: Dict[str, Any] = field(default_factory=dict)
29     random_state: int = 42
30
31
32 @dataclass
33 class TrainingConfig:
34     """Training configuration."""
35     epochs: int = 100
36     batch_size: int = 32
37     learning_rate: float = 0.001
38     early_stopping_patience: int = 10
39     checkpoint_dir: str = "models/checkpoints"
40
41
42 @dataclass
43 class ExperimentConfig:
44     """Complete experiment configuration."""
45     name: str
46     description: str
47     data: DataConfig
48     model: ModelConfig
49     training: TrainingConfig
50     seed: int = 42
51     created_at: str = field(default_factory=lambda: datetime.now().isoformat())
52
53     def to_dict(self) -> Dict[str, Any]:
54         """Convert to dictionary."""
55         return asdict(self)
56
57     def to_yaml(self, path: Path) -> None:
58         """Save configuration to YAML file."""
59         with open(path, 'w') as f:
60             yaml.dump(self.to_dict(), f, default_flow_style=False)
61
62     def to_json(self, path: Path) -> None:
63         """Save configuration to JSON file."""
64         with open(path, 'w') as f:
65             json.dump(self.to_dict(), f, indent=2)
66
67     @classmethod
68     def from_yaml(cls, path: Path) -> "ExperimentConfig":
69         """Load configuration from YAML file."""
70         with open(path, 'r') as f:
71             config_dict = yaml.safe_load(f)

```

```

72         return cls(
73             name=config_dict['name'],
74             description=config_dict['description'],
75             data=DataConfig(**config_dict['data']),
76             model=ModelConfig(**config_dict['model']),
77             training=TrainingConfig(**config_dict['training']),
78             seed=config_dict.get('seed', 42),
79             created_at=config_dict.get('created_at', datetime.now().isoformat())
80         )
81
82     def get_hash(self) -> str:
83         """
84         Get deterministic hash of configuration.
85         Useful for experiment tracking and caching.
86         """
87         # Convert to JSON string (deterministic ordering)
88         config_str = json.dumps(self.to_dict(), sort_keys=True)
89         return hashlib.md5(config_str.encode()).hexdigest()
90
91
92 # Example configuration YAML
93 EXAMPLE_CONFIG = """
94 name: customer_churn_experiment_001
95 description: Random Forest baseline for customer churn prediction
96
97 data:
98     train_path: data/processed/train.csv
99     val_path: data/processed/val.csv
100    test_path: data/processed/test.csv
101    features: null # Use all columns except target
102    target: churn
103
104 model:
105     model_type: sklearn
106     model_class: random_forest
107     parameters:
108         n_estimators: 100
109         max_depth: 10
110         min_samples_split: 5
111         class_weight: balanced
112         random_state: 42
113
114 training:
115     epochs: 1 # For sklearn, single training pass
116     batch_size: 32
117     learning_rate: 0.001
118     early_stopping_patience: 10
119     checkpoint_dir: models/checkpoints/exp_001
120
121 seed: 42
122 """
123

```

Listing 7.17: Comprehensive configuration system

7.8.2 Reproducible Training Pipeline

```

1  # scripts/train.py
2  """Reproducible training script."""
3
4  import argparse
5  import random
6  import numpy as np
7  import pandas as pd
8  from pathlib import Path
9  import logging
10 import sys
11 import os
12
13 # Add src to path
14 sys.path.insert(0, str(Path(__file__).parent.parent))
15
16 from src.utils.config import ExperimentConfig
17 from src.models import ModelFactory
18 from src.data.pipeline import DataPipeline, MissingValueImputer
19 from src.training.callbacks import (
20     CallbackList,
21     LoggingCallback,
22     MetricTrackingCallback,
23     EarlyStoppingCallback
24 )
25
26 logging.basicConfig(
27     level=logging.INFO,
28     format='%(asctime)s - %(name)s - %(levelname)s - %(message)s'
29 )
30 logger = logging.getLogger(__name__)
31
32
33 def set_seed(seed: int) -> None:
34     """
35     Set random seeds for reproducibility.
36
37     Args:
38         seed: Random seed value
39     """
40     random.seed(seed)
41     np.random.seed(seed)
42     os.environ['PYTHONHASHSEED'] = str(seed)
43
44     # For frameworks like TensorFlow/PyTorch
45     try:
46         import torch
47         torch.manual_seed(seed)
48         torch.cuda.manual_seed_all(seed)
49         torch.backends.cudnn.deterministic = True
50         torch.backends.cudnn.benchmark = False
51     except ImportError:
52         pass
53
54     try:
55         import tensorflow as tf
56         tf.random.set_seed(seed)
57     except ImportError:

```

```

58         pass
59
60
61 def load_data(config: ExperimentConfig) -> tuple:
62     """
63     Load and prepare data.
64
65     Args:
66         config: Experiment configuration
67
68     Returns:
69         Tuple of (X_train, y_train, X_val, y_val)
70     """
71     logger.info("Loading data...")
72
73     # Load datasets
74     train_df = pd.read_csv(config.data.train_path)
75     val_df = pd.read_csv(config.data.val_path)
76
77     # Extract features and target
78     target_col = config.data.target
79
80     if config.data.features:
81         feature_cols = config.data.features
82     else:
83         feature_cols = [col for col in train_df.columns if col != target_col]
84
85     X_train = train_df[feature_cols]
86     y_train = train_df[target_col]
87     X_val = val_df[feature_cols]
88     y_val = val_df[target_col]
89
90     logger.info(f"Training set: {X_train.shape}")
91     logger.info(f"Validation set: {X_val.shape}")
92
93     return X_train, y_train, X_val, y_val
94
95
96 def create_preprocessing_pipeline() -> DataPipeline:
97     """Create data preprocessing pipeline."""
98     return DataPipeline([
99         MissingValueImputer(strategy='median')
100     ])
101
102
103 def train(config_path: Path, experiment: bool = False) -> None:
104     """
105     Execute reproducible training.
106
107     Args:
108         config_path: Path to configuration file
109         experiment: Whether this is an experiment run
110     """
111     # Load configuration
112     logger.info(f"Loading configuration from {config_path}")
113     config = ExperimentConfig.from_yaml(config_path)
114
115     # Set seeds for reproducibility

```



```

116 logger.info(f"Setting random seed: {config.seed}")
117 set_seed(config.seed)
118
119 # Create experiment directory
120 exp_dir = Path("experiments") / config.name
121 exp_dir.mkdir(parents=True, exist_ok=True)
122
123 # Save configuration to experiment directory
124 config.to_yaml(exp_dir / "config.yaml")
125 config.to_json(exp_dir / "config.json")
126
127 logger.info(f"Experiment: {config.name}")
128 logger.info(f"Config hash: {config.get_hash()}")
129
130 # Load data
131 X_train, y_train, X_val, y_val = load_data(config)
132
133 # Create preprocessing pipeline
134 logger.info("Creating preprocessing pipeline...")
135 preprocess_pipeline = create_preprocessing_pipeline()
136
137 # Fit and transform data
138 X_train = preprocess_pipeline.fit_transform(X_train)
139 X_val = preprocess_pipeline.transform(X_val)
140
141 # Create model
142 logger.info("Creating model...")
143 model = ModelFactory.create(
144     config.model.model_type,
145     {
146         'model_type': config.model.model_class,
147         **config.model.parameters,
148         'random_state': config.model.random_state
149     }
150 )
151
152 # Create callbacks
153 callbacks = CallbackList([
154     LoggingCallback(),
155     MetricTrackingCallback(),
156     EarlyStoppingCallback(patience=config.training.early_stopping_patience)
157 ])
158
159 # Train model
160 logger.info("Training model...")
161 callbacks.on_train_begin({'config': config.to_dict()})
162
163 start_time = pd.Timestamp.now()
164 model.fit(X_train, y_train, validation_data=(X_val, y_val))
165 end_time = pd.Timestamp.now()
166
167 training_duration = (end_time - start_time).total_seconds()
168
169 callbacks.on_train_end({'duration': training_duration})
170
171 # Evaluate model
172 logger.info("Evaluating model...")
173 train_score = model.model.score(X_train, y_train)

```

```

174     val_score = model.model.score(X_val, y_val)
175
176     logger.info(f"Training accuracy: {train_score:.4f}")
177     logger.info(f"Validation accuracy: {val_score:.4f}")
178
179     # Save model
180     model_path = exp_dir / "model"
181     logger.info(f"Saving model to {model_path}")
182     model.save(model_path)
183
184     # Save metrics
185     metrics = {
186         'train_accuracy': float(train_score),
187         'val_accuracy': float(val_score),
188         'training_duration_seconds': training_duration,
189         'config_hash': config.get_hash()
190     }
191
192     with open(exp_dir / "metrics.json", 'w') as f:
193         json.dump(metrics, f, indent=2)
194
195     logger.info("Training completed successfully!")
196
197
198 def main():
199     """Main entry point."""
200     parser = argparse.ArgumentParser(description="Train ML model")
201     parser.add_argument(
202         '--config',
203         type=Path,
204         required=True,
205         help="Path to configuration file"
206     )
207     parser.add_argument(
208         '--experiment',
209         action='store_true',
210         help="Run as experiment (creates experiment directory)"
211     )
212
213     args = parser.parse_args()
214
215     train(args.config, args.experiment)
216
217
218 if __name__ == "__main__":
219     main()

```

Listing 7.18: Fully reproducible training script

7.8.3 Model and Experiment Versioning

```

1  # src/utils/versioning.py
2  """Model and experiment versioning utilities."""
3
4  from typing import Dict, Any, Optional, List
5  from pathlib import Path
6  from datetime import datetime

```

```

7 import json
8 import hashlib
9 from dataclasses import dataclass, asdict
10
11
12 @dataclass
13 class ModelVersion:
14     """Represents a versioned model."""
15     version_id: str
16     model_path: Path
17     config_hash: str
18     created_at: str
19     metrics: Dict[str, float]
20     parent_version: Optional[str] = None
21     tags: Optional[List[str]] = None
22     notes: Optional[str] = None
23
24
25 class ModelRegistry:
26     """
27     Central registry for tracking model versions.
28     Enables model lineage tracking and rollback.
29     """
30
31     def __init__(self, registry_path: Path):
32         """
33         Initialize model registry.
34
35         Args:
36             registry_path: Path to registry storage
37         """
38         self.registry_path = Path(registry_path)
39         self.registry_path.mkdir(parents=True, exist_ok=True)
40         self.index_file = self.registry_path / "index.json"
41         self.versions: Dict[str, ModelVersion] = {}
42         self._load_index()
43
44     def _load_index(self):
45         """Load registry index from disk."""
46         if self.index_file.exists():
47             with open(self.index_file, 'r') as f:
48                 data = json.load(f)
49                 self.versions = {
50                     k: ModelVersion(**v) for k, v in data.items()
51                 }
52
53     def _save_index(self):
54         """Save registry index to disk."""
55         data = {k: asdict(v) for k, v in self.versions.items()}
56         # Convert Path objects to strings
57         for v in data.values():
58             v['model_path'] = str(v['model_path'])
59
60         with open(self.index_file, 'w') as f:
61             json.dump(data, f, indent=2)
62
63     def register_model(
64         self,

```

```

65     model_path: Path,
66     config: Dict[str, Any],
67     metrics: Dict[str, float],
68     parent_version: Optional[str] = None,
69     tags: Optional[List[str]] = None,
70     notes: Optional[str] = None
71 ) -> str:
72     """
73     Register a new model version.
74
75     Args:
76         model_path: Path to saved model
77         config: Model configuration
78         metrics: Performance metrics
79         parent_version: ID of parent version (for lineage)
80         tags: Optional tags for categorization
81         notes: Optional notes about this version
82
83     Returns:
84         version_id: Unique version identifier
85     """
86     # Generate version ID
87     timestamp = datetime.now().isoformat()
88     config_str = json.dumps(config, sort_keys=True)
89     config_hash = hashlib.md5(config_str.encode()).hexdigest()[:8]
90     version_id = f"v_{datetime.now().strftime('%Y%m%d_%H%M%S')}-{config_hash}"
91
92     # Create version record
93     version = ModelVersion(
94         version_id=version_id,
95         model_path=Path(model_path),
96         config_hash=config_hash,
97         created_at=timestamp,
98         metrics=metrics,
99         parent_version=parent_version,
100         tags=tags or [],
101         notes=notes
102     )
103
104     self.versions[version_id] = version
105     self._save_index()
106
107     return version_id
108
109 def get_version(self, version_id: str) -> Optional[ModelVersion]:
110     """Get specific model version."""
111     return self.versions.get(version_id)
112
113 def get_best_version(
114     self,
115     metric: str,
116     higher_is_better: bool = True
117 ) -> Optional[ModelVersion]:
118     """
119     Get best model version by metric.
120
121     Args:
122         metric: Metric name to compare

```

```

123         higher_is_better: Whether higher metric values are better
124
125     Returns:
126         Best model version or None
127     """
128     valid_versions = [
129         v for v in self.versions.values()
130         if metric in v.metrics
131     ]
132
133     if not valid_versions:
134         return None
135
136     return max(
137         valid_versions,
138         key=lambda v: v.metrics[metric] if higher_is_better
139         else -v.metrics[metric]
140     )
141
142 def get_lineage(self, version_id: str) -> List[ModelVersion]:
143     """
144     Get complete lineage of a model version.
145
146     Args:
147         version_id: Version to trace
148
149     Returns:
150         List of versions from root to specified version
151     """
152     lineage = []
153     current_id = version_id
154
155     while current_id is not None:
156         version = self.versions.get(current_id)
157         if version is None:
158             break
159         lineage.append(version)
160         current_id = version.parent_version
161
162     return list(reversed(lineage))
163
164 def list_versions(
165     self,
166     tags: Optional[List[str]] = None,
167     min_metric: Optional[Dict[str, float]] = None
168 ) -> List[ModelVersion]:
169     """
170     List model versions with optional filtering.
171
172     Args:
173         tags: Filter by tags
174         min_metric: Filter by minimum metric values
175
176     Returns:
177         List of matching model versions
178     """
179     versions = list(self.versions.values())
180

```

```

181     # Filter by tags
182     if tags:
183         versions = [
184             v for v in versions
185             if any(tag in (v.tags or []) for tag in tags)
186         ]
187
188     # Filter by metrics
189     if min_metric:
190         for metric, min_value in min_metric.items():
191             versions = [
192                 v for v in versions
193                 if metric in v.metrics and v.metrics[metric] >= min_value
194             ]
195
196     return sorted(versions, key=lambda v: v.created_at, reverse=True)
197
198
199 # Example usage
200 if __name__ == "__main__":
201     from pathlib import Path
202
203     # Initialize registry
204     registry = ModelRegistry(Path("model_registry"))
205
206     # Register first model version
207     v1 = registry.register_model(
208         model_path=Path("models/experiment_001/model.pkl"),
209         config={'model_type': 'random_forest', 'n_estimators': 100},
210         metrics={'accuracy': 0.85, 'f1_score': 0.82},
211         tags=['baseline', 'random_forest'],
212         notes="Initial baseline model"
213     )
214     print(f"Registered version: {v1}")
215
216     # Register improved version
217     v2 = registry.register_model(
218         model_path=Path("models/experiment_002/model.pkl"),
219         config={'model_type': 'random_forest', 'n_estimators': 200},
220         metrics={'accuracy': 0.88, 'f1_score': 0.85},
221         parent_version=v1,
222         tags=['improved', 'random_forest'],
223         notes="Increased estimators, improved performance"
224     )
225     print(f"Registered version: {v2}")
226
227     # Get best model
228     best = registry.get_best_version('accuracy', higher_is_better=True)
229     print(f"Best model: {best.version_id} with accuracy {best.metrics['accuracy']}")
230
231     # Get lineage
232     lineage = registry.get_lineage(v2)
233     print("\nModel lineage:")
234     for i, version in enumerate(lineage, 1):
235         print(f"  {i}. {version.version_id} - Accuracy: {version.metrics.get('accuracy', '')}")

```

Listing 7.19: Model versioning and experiment tracking

7.9 Documentation Best Practices

Production ML systems require comprehensive documentation. This section presents patterns for documenting models, experiments, and systems.

7.9.1 Model Cards

```

1  # src/utils/model_card.py
2  """Model card generation for ML model documentation."""
3
4  from typing import Dict, Any, List, Optional
5  from pathlib import Path
6  from datetime import datetime
7  from dataclasses import dataclass, field
8  import json
9
10
11 @dataclass
12 class ModelCard:
13     """
14     Model card following Google's Model Cards framework.
15     Documents model details, intended use, performance, and limitations.
16     """
17
18     # Model Details
19     model_name: str
20     version: str
21     date: str = field(default_factory=lambda: datetime.now().strftime("%Y-%m-%d"))
22     model_type: str = ""
23     model_architecture: str = ""
24     authors: List[str] = field(default_factory=list)
25
26     # Intended Use
27     intended_use: str = ""
28     intended_users: List[str] = field(default_factory=list)
29     out_of_scope_uses: List[str] = field(default_factory=list)
30
31     # Training Data
32     training_data_description: str = ""
33     training_data_size: Optional[int] = None
34     training_data_source: str = ""
35     preprocessing_steps: List[str] = field(default_factory=list)
36
37     # Evaluation Data
38     evaluation_data_description: str = ""
39     evaluation_data_size: Optional[int] = None
40
41     # Performance Metrics
42     metrics: Dict[str, float] = field(default_factory=dict)
43     performance_by_group: Dict[str, Dict[str, float]] = field(default_factory=dict)
44
45     # Ethical Considerations
46     sensitive_data: List[str] = field(default_factory=list)
47     potential_biases: List[str] = field(default_factory=list)
48     fairness_assessment: str = ""
49
50     # Caveats and Recommendations

```

```

51 caveats: List[str] = field(default_factory=list)
52 recommendations: List[str] = field(default_factory=list)
53
54 def to_markdown(self) -> str:
55     """Generate markdown documentation."""
56     md = f"# Model Card: {self.model_name}\n\n"
57     md += f"***Version:** {self.version} \n"
58     md += f"***Date:** {self.date} \n"
59     md += f"***Model Type:** {self.model_type} \n\n"
60
61     if self.authors:
62         md += f"***Authors:** {' '.join(self.authors)}\n\n"
63
64     md += "## Model Details\n\n"
65     md += f"***Architecture:** {self.model_architecture}\n\n"
66
67     md += "## Intended Use\n\n"
68     md += f"{self.intended_use}\n\n"
69
70     if self.intended_users:
71         md += "***Primary Intended Users:**\n"
72         for user in self.intended_users:
73             md += f"- {user}\n"
74         md += "\n"
75
76     if self.out_of_scope_uses:
77         md += "***Out-of-Scope Uses:**\n"
78         for use in self.out_of_scope_uses:
79             md += f"- {use}\n"
80         md += "\n"
81
82     md += "## Training Data\n\n"
83     md += f"{self.training_data_description}\n\n"
84     if self.training_data_size:
85         md += f"***Dataset Size:** {self.training_data_size:,} samples\n\n"
86     if self.training_data_source:
87         md += f"***Source:** {self.training_data_source}\n\n"
88
89     if self.preprocessing_steps:
90         md += "***Preprocessing Steps:**\n"
91         for step in self.preprocessing_steps:
92             md += f"1. {step}\n"
93         md += "\n"
94
95     md += "## Evaluation\n\n"
96     md += f"{self.evaluation_data_description}\n\n"
97
98     if self.metrics:
99         md += "***Performance Metrics:**\n\n"
100         md += "| Metric | Value |\n"
101         md += "|-----|-----|\n"
102         for metric, value in self.metrics.items():
103             md += f"| {metric} | {value:.4f} |\n"
104         md += "\n"
105
106     if self.performance_by_group:
107         md += "***Performance by Group:**\n\n"
108         for group, group_metrics in self.performance_by_group.items():

```



```

109         md += f"**{group}:**\n\n"
110         md += "| Metric | Value |\n"
111         md += "|-----|-----|\n"
112         for metric, value in group_metrics.items():
113             md += f"| {metric} | {value:.4f} |\n"
114         md += "\n"
115
116     md += "## Ethical Considerations\n\n"
117
118     if self.sensitive_data:
119         md += "**Sensitive Data:**\n"
120         for item in self.sensitive_data:
121             md += f"- {item}\n"
122         md += "\n"
123
124     if self.potential_biases:
125         md += "**Potential Biases:**\n"
126         for bias in self.potential_biases:
127             md += f"- {bias}\n"
128         md += "\n"
129
130     if self.fairness_assessment:
131         md += f"**Fairness Assessment:** {self.fairness_assessment}\n\n"
132
133     md += "## Caveats and Recommendations\n\n"
134
135     if self.caveats:
136         md += "**Caveats:**\n"
137         for caveat in self.caveats:
138             md += f"- {caveat}\n"
139         md += "\n"
140
141     if self.recommendations:
142         md += "**Recommendations:**\n"
143         for rec in self.recommendations:
144             md += f"- {rec}\n"
145         md += "\n"
146
147     return md
148
149     def save(self, path: Path):
150         """Save model card to markdown file."""
151         markdown = self.to_markdown()
152         with open(path, 'w') as f:
153             f.write(markdown)
154
155
156     # Example usage
157     if __name__ == "__main__":
158         card = ModelCard(
159             model_name="Customer Churn Predictor",
160             version="v1.2.0",
161             model_type="Random Forest Classifier",
162             model_architecture="Ensemble of 100 decision trees with max depth 10",
163             authors=["Data Science Team"],
164             intended_use="Predict customer churn risk for proactive retention campaigns",
165             intended_users=["Customer Success Team", "Marketing Team"],
166             out_of_scope_uses=[

```

```

167         "Individual customer credit decisions",
168         "Automated customer termination"
169     ],
170     training_data_description="Historical customer data from 2020-2023",
171     training_data_size=150000,
172     training_data_source="Internal CRM database",
173     preprocessing_steps=[
174         "Remove customers with <3 months tenure",
175         "Impute missing values using median strategy",
176         "Encode categorical features using one-hot encoding",
177         "Scale numerical features using StandardScaler"
178     ],
179     evaluation_data_description="Holdout test set from Q4 2023",
180     evaluation_data_size=30000,
181     metrics={
182         'accuracy': 0.8850,
183         'precision': 0.8245,
184         'recall': 0.7890,
185         'f1_score': 0.8064,
186         'roc_auc': 0.9123
187     },
188     performance_by_group={
189         'Tenure < 1 year': {'accuracy': 0.8234, 'f1_score': 0.7456},
190         'Tenure >= 1 year': {'accuracy': 0.9123, 'f1_score': 0.8567}
191     },
192     sensitive_data=[
193         "Customer age (used as feature)",
194         "Geographic location (used as feature)"
195     ],
196     potential_biases=[
197         "Model may perform worse on newer customers due to limited history",
198         "Geographic regions with fewer samples may have lower accuracy"
199     ],
200     fairness_assessment="Performance parity tested across age groups and regions.
201     "
202     "Largest accuracy gap is 5% between regions.",
203     caveats=[
204         "Model performance degrades on customers from regions not in training data
205         ",
206         "Should be retrained quarterly to maintain performance",
207         "Predictions are probabilistic and should inform, not replace, human
208         judgment"
209     ],
210     recommendations=[
211         "Use predictions as one input to retention strategy, not sole decision
212         maker",
213         "Monitor performance weekly and retrain if accuracy drops below 0.85",
214         "Review false positives monthly to identify process improvements",
215         "Conduct fairness audit every 6 months"
216     ]
217 )
218
219 # Save to file
220 card.save(Path("docs/model_cards/churn_predictor_v1.2.0.md"))
221 print("Model card saved successfully")

```

Listing 7.20: Automated model card generation

7.9.2 API Documentation

```

1  # src/models/documented_model.py
2  """
3  Example of well-documented ML code with comprehensive docstrings.
4  Follows Google docstring style.
5  """
6
7  from typing import Union, Optional, Tuple, List
8  import numpy as np
9  import pandas as pd
10 from numpy.typing import NDArray
11
12
13 class DocumentedModel:
14     """
15     Production-ready ML model with comprehensive documentation.
16
17     This model provides a unified interface for training and inference
18     with automatic feature validation and preprocessing.
19
20     Attributes:
21         config: Model configuration dictionary
22         is_fitted: Whether model has been trained
23         feature_names_: List of feature names from training
24         classes_: Unique class labels (for classification)
25         n_features_in_: Number of features seen during fit
26
27     Example:
28         Basic usage::
29
30             >>> from sklearn.datasets import make_classification
31             >>> X, y = make_classification(n_samples=1000, random_state=42)
32             >>> model = DocumentedModel({'max_depth': 5})
33             >>> model.fit(X, y)
34             >>> predictions = model.predict(X[:10])
35             >>> print(predictions.shape)
36             (10,)
37
38         With validation data::
39
40             >>> X_train, X_val = X[:800], X[800:]
41             >>> y_train, y_val = y[:800], y[800:]
42             >>> model.fit(X_train, y_train, validation_data=(X_val, y_val))
43             >>> print(model.metadata['validation_score'])
44             0.85
45
46     Note:
47         This model requires features to be preprocessed (scaled, encoded)
48         before training or inference.
49
50     .. versionadded:: 1.0.0
51     .. versionchanged:: 1.2.0
52         Added support for sample weights in fit method
53     """
54
55     def __init__(self, config: dict):
56         """

```

```

57         Initialize model with configuration.
58
59     Args:
60         config: Configuration dictionary with model parameters.
61                 Required keys depend on model_type.
62
63     Raises:
64         ValueError: If required configuration keys are missing
65
66     Example:
67         >>> config = {
68             'model_type': 'random_forest',
69             'n_estimators': 100,
70             'max_depth': 10
71         }
72         >>> model = DocumentedModel(config)
73     """
74     self.config = config
75     self.is_fitted = False
76     self.feature_names_: Optional[List[str]] = None
77     self.classes_: Optional[NDArray] = None
78     self.n_features_in_: Optional[int] = None
79     self.metadata = {}
80
81     def fit(
82         self,
83         X: Union[NDArray[np.float64], pd.DataFrame],
84         y: Union[NDArray[np.int_], pd.Series],
85         validation_data: Optional[Tuple] = None,
86         sample_weight: Optional[NDArray[np.float64]] = None
87     ) -> "DocumentedModel":
88         """
89         Fit model to training data.
90
91         Trains the model on provided features and targets. Optionally
92         evaluates on validation set and applies sample weights.
93
94     Args:
95         X: Training features of shape (n_samples, n_features).
96             Can be numpy array or pandas DataFrame.
97         y: Training targets of shape (n_samples,).
98             For classification: integer class labels.
99             For regression: continuous values.
100         validation_data: Optional tuple of (X_val, y_val) for
101                         validation during training. If provided,
102                         validation metrics are stored in metadata.
103         sample_weight: Optional array of shape (n_samples,) with
104                         sample weights. Higher weights increase the
105                         importance of the corresponding samples.
106
107     Returns:
108         self: Fitted model instance (enables method chaining)
109
110     Raises:
111         ValueError: If X and y have incompatible shapes
112         ValueError: If sample_weight has wrong shape
113         RuntimeError: If model fitting fails

```

Example:

```
>>> X_train = np.random.randn(100, 5)
>>> y_train = np.random.randint(0, 2, 100)
>>> model = DocumentedModel({'model_type': 'logistic_regression'})
>>> model.fit(X_train, y_train)
DocumentedModel(...)
```

With validation::

```
>>> X_val = np.random.randn(20, 5)
>>> y_val = np.random.randint(0, 2, 20)
>>> model.fit(X_train, y_train, validation_data=(X_val, y_val))
DocumentedModel(...)
```

Note:

After fitting, the following attributes are set:

- `is_fitted`: True
- `feature_names_`: Feature names (if X is DataFrame)
- `n_features_in_`: Number of features
- `classes_`: Unique classes (for classification)

.. versionadded:: 1.0.0

.. versionchanged:: 1.2.0

Added `sample_weight` parameter

"""

Validate inputs

```
if len(X) != len(y):
    raise ValueError(
        f"X and y have incompatible shapes: {len(X)} vs {len(y)}"
    )
```

```
if sample_weight is not None and len(sample_weight) != len(X):
    raise ValueError(
        f"sample_weight has wrong shape: {len(sample_weight)} vs {len(X)}"
    )
```

Extract feature names if DataFrame

```
if isinstance(X, pd.DataFrame):
    self.feature_names_ = X.columns.tolist()
```

Store training metadata

```
self.n_features_in_ = X.shape[1]
self.classes_ = np.unique(y)
self.is_fitted = True
```

Implementation details...

```
return self
```

```
def predict(
    self,
    X: Union[NDArray[np.float64], pd.DataFrame]
) -> NDArray[np.int_]:
```

"""

Generate predictions for input features.

Args:

X: Features of shape (n_samples, n_features).
Must have same number of features as training data.

```

173
174     Returns:
175         Predictions of shape (n_samples,).
176         For classification: predicted class labels.
177         For regression: predicted values.
178
179     Raises:
180         ValueError: If model is not fitted
181         ValueError: If X has wrong number of features
182
183     Example:
184         >>> model.fit(X_train, y_train)
185         >>> predictions = model.predict(X_test)
186         >>> print(predictions[:5])
187         [0 1 1 0 1]
188
189     Warning:
190         Input features must be preprocessed the same way as
191         training data. Inconsistent preprocessing will lead
192         to degraded performance.
193
194     See Also:
195         predict_proba: Generate probability predictions
196         predict_log_proba: Generate log probability predictions
197
198     .. versionadded:: 1.0.0
199     """
200     if not self.is_fitted:
201         raise ValueError(
202             "Model must be fitted before calling predict(). "
203             "Call fit(X, y) first."
204         )
205
206     if X.shape[1] != self.n_features_in_:
207         raise ValueError(
208             f"X has {X.shape[1]} features but model expects "
209             f"{self.n_features_in_} features"
210         )
211
212     # Implementation details...
213     return np.zeros(len(X), dtype=np.int_)

```

Listing 7.21: Comprehensive docstring examples

7.10 Continuous Integration for ML

Automating testing and deployment ensures consistent quality. This section demonstrates CI/CD patterns for ML systems.

7.10.1 GitHub Actions Workflow

```

1 # .github/workflows/ml-ci.yml
2 name: ML Pipeline CI/CD
3
4 on:

```

```

5  push:
6      branches: [ main, develop ]
7  pull_request:
8      branches: [ main ]
9  schedule:
10     # Run weekly model training
11     - cron: '0 0 * * 0'
12
13 env:
14     PYTHON_VERSION: '3.9'
15     COVERAGE_THRESHOLD: 80
16
17 jobs:
18     code-quality:
19         name: Code Quality Checks
20         runs-on: ubuntu-latest
21         steps:
22             - uses: actions/checkout@v3
23
24             - name: Set up Python
25               uses: actions/setup-python@v4
26               with:
27                 python-version: ${ env.PYTHON_VERSION }
28
29             - name: Install dependencies
30               run: |
31                 pip install -e ".[dev]"
32
33             - name: Run Black
34               run: black --check src/ tests/
35
36             - name: Run isort
37               run: isort --check src/ tests/
38
39             - name: Run flake8
40               run: flake8 src/ tests/ --max-line-length=100
41
42             - name: Run mypy
43               run: mypy src/ --ignore-missing-imports
44
45     unit-tests:
46         name: Unit Tests
47         runs-on: ubuntu-latest
48         steps:
49             - uses: actions/checkout@v3
50
51             - name: Set up Python
52               uses: actions/setup-python@v4
53               with:
54                 python-version: ${ env.PYTHON_VERSION }
55
56             - name: Install dependencies
57               run: |
58                 pip install -e ".[dev]"
59
60             - name: Run unit tests
61               run: |
62                 pytest tests/unit/ -v \

```

```

63         --cov=src \
64         --cov-report=xml \
65         --cov-report=term-missing \
66         --cov-fail-under=${{ env.COVERAGE_THRESHOLD }}
67
68     - name: Upload coverage
69       uses: codecov/codecov-action@v3
70       with:
71         file: ./coverage.xml
72         fail_ci_if_error: true
73
74   integration-tests:
75     name: Integration Tests
76     runs-on: ubuntu-latest
77     needs: unit-tests
78     steps:
79       - uses: actions/checkout@v3
80
81       - name: Set up Python
82         uses: actions/setup-python@v4
83         with:
84           python-version: ${{ env.PYTHON_VERSION }}
85
86       - name: Install dependencies
87         run: |
88           pip install -e ".[dev]"
89
90       - name: Run integration tests
91         run: |
92           pytest tests/integration/ -v --timeout=300
93
94   model-tests:
95     name: Model Performance Tests
96     runs-on: ubuntu-latest
97     needs: unit-tests
98     steps:
99       - uses: actions/checkout@v3
100
101       - name: Set up Python
102         uses: actions/setup-python@v4
103         with:
104           python-version: ${{ env.PYTHON_VERSION }}
105
106       - name: Install dependencies
107         run: |
108           pip install -e ".[dev]"
109
110       - name: Download test data
111         run: |
112           python scripts/download_test_data.py
113
114       - name: Run model performance tests
115         run: |
116           pytest tests/model/ -v \
117             --timeout=600 \
118             -m "not slow"
119
120   train-model:

```



```

121 name: Train and Validate Model
122 runs-on: ubuntu-latest
123 needs: [code-quality, unit-tests, integration-tests]
124 if: github.event_name == 'schedule' || github.ref == 'refs/heads/main'
125 steps:
126   - uses: actions/checkout@v3
127
128   - name: Set up Python
129     uses: actions/setup-python@v4
130     with:
131       python-version: ${ env.PYTHON_VERSION }
132
133   - name: Install dependencies
134     run: |
135       pip install -e .
136
137   - name: Download training data
138     run: |
139       python scripts/download_training_data.py
140
141   - name: Train model
142     run: |
143       python scripts/train.py \
144         --config config/production.yaml \
145         --output models/production
146
147   - name: Validate model performance
148     run: |
149       python scripts/validate_model.py \
150         --model models/production/model.pkl \
151         --threshold 0.85
152
153   - name: Upload model artifact
154     uses: actions/upload-artifact@v3
155     with:
156       name: trained-model
157       path: models/production/
158
159 deploy-model:
160   name: Deploy Model to Production
161   runs-on: ubuntu-latest
162   needs: train-model
163   if: github.ref == 'refs/heads/main'
164   environment: production
165   steps:
166     - uses: actions/checkout@v3
167
168     - name: Download model artifact
169       uses: actions/download-artifact@v3
170       with:
171         name: trained-model
172         path: models/production/
173
174     - name: Deploy to model registry
175       run: |
176         python scripts/deploy_model.py \
177           --model models/production/model.pkl \
178           --environment production

```

```
179 - name: Notify deployment
180   run: |
181     echo "Model deployed to production"
182
```

Listing 7.22: GitHub Actions CI/CD pipeline

7.11 Exercises

Exercise 7.1 (Project Structure Refactoring). Refactor a Jupyter notebook into a production-ready project structure:

1. Take an existing ML notebook (or create one with a simple classification task)
2. Create the full project structure with proper directories
3. Extract data processing code into `src/data/`
4. Extract feature engineering into `src/features/`
5. Extract model code into `src/models/`
6. Create reusable, modular functions
7. Add `setup.py` and `requirements.txt`
8. Document the refactored code

Deliverable: Structured project with modular code, tests, and documentation.

Exercise 7.2 (Comprehensive Testing Suite). Build a complete testing suite for an ML pipeline:

1. Implement unit tests for data preprocessing functions
2. Create integration tests for the full training pipeline
3. Add model performance tests with minimum thresholds
4. Write tests for model invariants (single sample, empty data, etc.)
5. Implement pytest fixtures for reusable test data
6. Configure pytest with coverage requirements (>80%)
7. Set up continuous testing with GitHub Actions or similar

Deliverable: Test suite with >80% coverage and CI configuration.

Exercise 7.3 (Design Pattern Implementation). Implement design patterns for ML workflows:

1. Create a Factory pattern for model creation
2. Implement Strategy pattern for different training approaches

3. Build Observer pattern for training callbacks
4. Add Template Method pattern for evaluation pipelines
5. Document when and why to use each pattern

Deliverable: Pattern implementations with usage examples and documentation.

Exercise 7.4 (Code Quality Setup). Configure code quality tools for an ML project:

1. Set up Black for code formatting
2. Configure isort for import sorting
3. Add flake8 for linting
4. Configure mypy for type checking
5. Add type hints to existing code
6. Set up pre-commit hooks
7. Configure pyproject.toml with all tool settings
8. Create Makefile for common tasks

Deliverable: Fully configured project with automated code quality checks.

Exercise 7.5 (Reproducible Training Pipeline). Create a fully reproducible training pipeline:

1. Implement configuration management with dataclasses
2. Add seed setting for all random operations
3. Create config hash for experiment tracking
4. Build training script that logs all configuration
5. Implement model and data versioning
6. Add data validation checks
7. Create experiment tracking with MLflow or similar

Deliverable: Reproducible pipeline that generates identical results across runs.

Exercise 7.6 (Model Interface Abstraction). Design and implement a flexible model interface:

1. Create abstract base class for models
2. Implement concrete classes for sklearn, XGBoost, and LightGBM
3. Add consistent save/load functionality
4. Implement feature importance extraction

5. Add metadata tracking
6. Create unified prediction interface
7. Write comprehensive tests for the interface

Deliverable: Model abstraction layer with multiple implementations and tests.

Exercise 7.7 (CI/CD Pipeline for ML). Set up continuous integration and deployment for ML:

1. Configure GitHub Actions or GitLab CI
2. Add automated testing on each commit
3. Implement code quality checks in CI
4. Create automated model training on schedule
5. Add model performance validation gates
6. Implement automated deployment on passing tests
7. Set up notifications for failures

Deliverable: Complete CI/CD pipeline configuration with automated workflows.

7.12 Chapter Summary

This chapter presented comprehensive best practices for professional ML development, bridging the gap between research code and production systems. Key takeaways include:

- **Project Structure:** Well-organized projects with clear separation of concerns enable collaboration and maintainability at scale.
- **Modular Architecture:** Object-oriented design with base classes, factories, and composable components creates flexible, testable systems.
- **Comprehensive Testing:** Multi-level testing strategies (unit, integration, model-specific) ensure reliability and catch regressions early.
- **Design Patterns:** Adapting software design patterns to ML workflows (Strategy, Observer, Factory) provides proven solutions to common problems.
- **Code Quality:** Automated tools (Black, isort, flake8, mypy) and pre-commit hooks maintain consistent, high-quality code.
- **Reproducibility:** Configuration management, seed setting, and experiment tracking enable reproducible research and debugging.
- **Type Safety:** Type hints improve code documentation, enable static analysis, and catch bugs before runtime.

- **Automation:** Makefiles, CI/CD pipelines, and automated testing reduce manual work and ensure consistency.

The transition from experimental to production ML code requires discipline and structure. Organizations that invest in these practices report significantly improved development velocity, reduced bugs, and easier collaboration across teams. The patterns and tools presented in this chapter form the foundation for building reliable, maintainable ML systems at scale.

Chapter 8

Experiment Tracking and Management

8.1 Learning Objectives

After completing this chapter, readers will be able to:

- Set up and use MLflow for experiment tracking
- Track model parameters, metrics, and artifacts
- Compare experiments and analyze results
- Manage model lifecycle with model registry
- Implement experiment tracking best practices

8.2 Introduction to Experiment Tracking

8.2.1 Why Track Experiments?

Systematic experiment tracking enables reproducibility, collaboration, and continuous improvement.

8.3 MLflow Tutorial

```
1 import mlflow
2 import mlflow.sklearn
3 from sklearn.ensemble import RandomForestClassifier
4
5 # Set tracking URI
6 mlflow.set_tracking_uri("http://localhost:5000")
7 mlflow.set_experiment("classification_experiments")
8
9 # Start MLflow run
10 with mlflow.start_run():
11     # Log parameters
12     mlflow.log_param("n_estimators", 100)
13     mlflow.log_param("max_depth", 10)
14
15     # Train model
```

```
16 model = RandomForestClassifier(n_estimators=100, max_depth=10)
17 model.fit(X_train, y_train)
18
19 # Log metrics
20 accuracy = model.score(X_test, y_test)
21 mlflow.log_metric("accuracy", accuracy)
22
23 # Log model
24 mlflow.sklearn.log_model(model, "model")
```

8.4 Model Registry

8.5 Chapter Summary

8.6 Exercises

Exercise 8.1. Set up MLflow tracking for a multi-model comparison experiment.

Chapter 9

Distributed Training and Optimization

9.1 Learning Objectives

After completing this chapter, readers will be able to:

- Understand fundamental distributed training strategies and when to apply them
- Implement data parallelism using PyTorch Distributed Data Parallel (DDP)
- Design and implement model parallelism for models that don't fit on single GPUs
- Choose between parameter server and all-reduce architectures
- Implement synchronous and asynchronous gradient synchronization
- Build fault-tolerant distributed training systems with checkpointing
- Optimize resource allocation and scheduling for multi-node training
- Use Horovod for framework-agnostic distributed training
- Apply Ray for flexible distributed computing workflows
- Benchmark and optimize distributed training performance
- Identify and resolve common bottlenecks in distributed systems

9.2 Introduction: Why Distributed Training?

Modern deep learning models and datasets have grown beyond the capacity of single GPUs. Consider:

- **Model Size:** GPT-3 has 175 billion parameters requiring 700GB memory
- **Dataset Scale:** ImageNet-21k contains 14 million images
- **Training Time:** Training BERT from scratch takes weeks on single GPU
- **Research Velocity:** Faster iteration enables more experiments

Distributed training addresses these challenges through parallelization across multiple devices, but introduces new complexities:

- Communication overhead between devices
- Gradient synchronization strategies
- Fault tolerance and recovery
- Resource allocation and scheduling
- Debugging and monitoring across nodes

9.2.1 Scaling Laws

Understanding scaling efficiency is crucial for distributed training:

$$\text{Speedup} = \frac{T_1}{T_N} \quad (9.1)$$

where T_1 is training time on 1 GPU and T_N is training time on N GPUs.

$$\text{Scaling Efficiency} = \frac{\text{Speedup}}{N} = \frac{T_1}{N \cdot T_N} \quad (9.2)$$

Perfect linear scaling achieves efficiency = 1.0, but communication overhead typically reduces this to 0.7-0.9 in practice.

9.3 Data Parallelism

Data parallelism replicates the model across devices and distributes data batches. Each device computes gradients on its batch, then gradients are synchronized.

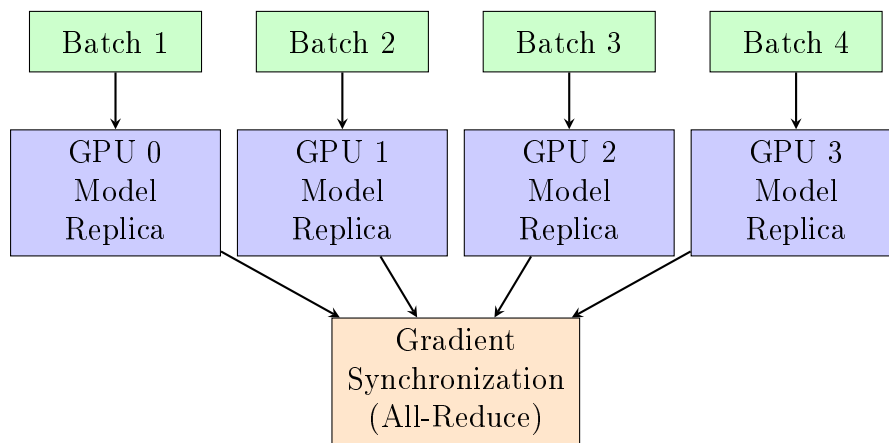


Figure 9.1: Data parallelism: Model replicated across GPUs, data distributed

9.3.1 PyTorch Distributed Data Parallel (DDP)

PyTorch DDP provides efficient data parallelism with optimized gradient synchronization.

```

1 import torch
2 import torch.nn as nn
3 import torch.distributed as dist
4 import torch.multiprocessing as mp
5 from torch.nn.parallel import DistributedDataParallel as DDP
6 from torch.utils.data import DataLoader, DistributedSampler
7 import os
8 import argparse
9
10 class ResNetModel(nn.Module):
11     """Example model for distributed training"""
12     def __init__(self, num_classes=10):
13         super().__init__()
14         self.conv1 = nn.Conv2d(3, 64, kernel_size=7, stride=2, padding=3)
15         self.bn1 = nn.BatchNorm2d(64)
16         self.relu = nn.ReLU(inplace=True)
17         self.maxpool = nn.MaxPool2d(kernel_size=3, stride=2, padding=1)
18
19         # Residual blocks
20         self.layer1 = self._make_layer(64, 128, 3)
21         self.layer2 = self._make_layer(128, 256, 4)
22         self.layer3 = self._make_layer(256, 512, 6)
23
24         self.avgpool = nn.AdaptiveAvgPool2d((1, 1))
25         self.fc = nn.Linear(512, num_classes)
26
27     def _make_layer(self, in_channels, out_channels, num_blocks):
28         layers = []
29         layers.append(nn.Conv2d(in_channels, out_channels, 3, stride=2, padding=1))
30         layers.append(nn.BatchNorm2d(out_channels))
31         layers.append(nn.ReLU(inplace=True))
32
33         for _ in range(num_blocks - 1):
34             layers.append(nn.Conv2d(out_channels, out_channels, 3, padding=1))
35             layers.append(nn.BatchNorm2d(out_channels))
36             layers.append(nn.ReLU(inplace=True))
37
38         return nn.Sequential(*layers)
39
40     def forward(self, x):
41         x = self.conv1(x)
42         x = self.bn1(x)
43         x = self.relu(x)
44         x = self.maxpool(x)
45
46         x = self.layer1(x)
47         x = self.layer2(x)
48         x = self.layer3(x)
49
50         x = self.avgpool(x)
51         x = torch.flatten(x, 1)
52         x = self.fc(x)
53         return x
54
55     def setup_distributed(rank, world_size, backend='nccl'):

```

```

56 """Initialize distributed training process group"""
57 os.environ['MASTER_ADDR'] = 'localhost'
58 os.environ['MASTER_PORT'] = '12355'
59
60 # Initialize process group
61 dist.init_process_group(
62     backend=backend,
63     rank=rank,
64     world_size=world_size
65 )
66
67 # Set device
68 torch.cuda.set_device(rank)
69
70 def cleanup_distributed():
71     """Clean up distributed training"""
72     dist.destroy_process_group()
73
74 def create_dataloader(rank, world_size, batch_size):
75     """Create distributed dataloader"""
76     from torchvision import datasets, transforms
77
78     # Data transformations
79     transform = transforms.Compose([
80         transforms.RandomResizedCrop(224),
81         transforms.RandomHorizontalFlip(),
82         transforms.ToTensor(),
83         transforms.Normalize(mean=[0.485, 0.456, 0.406],
84                               std=[0.229, 0.224, 0.225])
85     ])
86
87     # Dataset
88     dataset = datasets.FakeData(
89         size=10000,
90         image_size=(3, 224, 224),
91         num_classes=10,
92         transform=transform
93     )
94
95     # Distributed sampler ensures each GPU gets different data
96     sampler = DistributedSampler(
97         dataset,
98         num_replicas=world_size,
99         rank=rank,
100         shuffle=True,
101         drop_last=True
102     )
103
104     # DataLoader
105     loader = DataLoader(
106         dataset,
107         batch_size=batch_size,
108         sampler=sampler,
109         num_workers=4,
110         pin_memory=True,
111         persistent_workers=True
112     )
113

```

```

114     return loader, sampler
115
116 def train_epoch(model, dataloader, criterion, optimizer, rank, epoch):
117     """Train for one epoch"""
118     model.train()
119     total_loss = 0.0
120     correct = 0
121     total = 0
122
123     for batch_idx, (data, target) in enumerate(dataloader):
124         # Move to GPU
125         data = data.cuda(rank, non_blocking=True)
126         target = target.cuda(rank, non_blocking=True)
127
128         # Forward pass
129         optimizer.zero_grad(set_to_none=True) # More efficient than zero_grad()
130         output = model(data)
131         loss = criterion(output, target)
132
133         # Backward pass
134         loss.backward()
135
136         # Gradient clipping (optional but recommended)
137         torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
138
139         # Optimizer step
140         optimizer.step()
141
142         # Statistics
143         total_loss += loss.item()
144         _, predicted = output.max(1)
145         total += target.size(0)
146         correct += predicted.eq(target).sum().item()
147
148         if rank == 0 and batch_idx % 10 == 0:
149             print(f'Epoch: {epoch} | Batch: {batch_idx}/{len(dataloader)} | ',
150                   f'Loss: {loss.item():.4f} | ',
151                   f'Acc: {100.*correct/total:.2f}%')
152
153         # Gather statistics from all processes
154         avg_loss = total_loss / len(dataloader)
155         accuracy = 100. * correct / total
156
157     return avg_loss, accuracy
158
159 def validate(model, dataloader, criterion, rank):
160     """Validate model"""
161     model.eval()
162     total_loss = 0.0
163     correct = 0
164     total = 0
165
166     with torch.no_grad():
167         for data, target in dataloader:
168             data = data.cuda(rank, non_blocking=True)
169             target = target.cuda(rank, non_blocking=True)
170
171             output = model(data)

```

```

172         loss = criterion(output, target)
173
174         total_loss += loss.item()
175         _, predicted = output.max(1)
176         total += target.size(0)
177         correct += predicted.eq(target).sum().item()
178
179     avg_loss = total_loss / len(dataloader)
180     accuracy = 100. * correct / total
181
182     # Synchronize metrics across all GPUs
183     loss_tensor = torch.tensor(avg_loss).cuda(rank)
184     acc_tensor = torch.tensor(accuracy).cuda(rank)
185
186     dist.all_reduce(loss_tensor, op=dist.ReduceOp.SUM)
187     dist.all_reduce(acc_tensor, op=dist.ReduceOp.SUM)
188
189     world_size = dist.get_world_size()
190     avg_loss = loss_tensor.item() / world_size
191     accuracy = acc_tensor.item() / world_size
192
193     return avg_loss, accuracy
194
195 def save_checkpoint(model, optimizer, epoch, loss, filepath, rank):
196     """Save checkpoint (only rank 0)"""
197     if rank == 0:
198         checkpoint = {
199             'epoch': epoch,
200             'model_state_dict': model.module.state_dict(), # .module for DDP
201             'optimizer_state_dict': optimizer.state_dict(),
202             'loss': loss,
203         }
204         torch.save(checkpoint, filepath)
205         print(f'Checkpoint saved: {filepath}')
206
207 def load_checkpoint(filepath, model, optimizer=None):
208     """Load checkpoint"""
209     checkpoint = torch.load(filepath)
210     model.module.load_state_dict(checkpoint['model_state_dict'])
211     if optimizer:
212         optimizer.load_state_dict(checkpoint['optimizer_state_dict'])
213     return checkpoint['epoch'], checkpoint['loss']
214
215 def train_ddp(rank, world_size, args):
216     """Main distributed training function"""
217     print(f'Running DDP on rank {rank}')
218
219     # Setup
220     setup_distributed(rank, world_size)
221
222     # Create model and move to GPU
223     model = ResNetModel(num_classes=args.num_classes).cuda(rank)
224
225     # Wrap model with DDP
226     model = DDP(
227         model,
228         device_ids=[rank],
229         output_device=rank,

```

```

230     find_unused_parameters=False, # Set True only if needed
231     gradient_as_bucket_view=True  # Memory optimization
232 )
233
234 # Loss and optimizer
235 criterion = nn.CrossEntropyLoss().cuda(rank)
236 optimizer = torch.optim.SGD(
237     model.parameters(),
238     lr=args.lr,
239     momentum=0.9,
240     weight_decay=1e-4
241 )
242
243 # Learning rate scheduler
244 scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
245     optimizer,
246     T_max=args.epochs
247 )
248
249 # Create dataloaders
250 train_loader, train_sampler = create_dataloader(
251     rank, world_size, args.batch_size
252 )
253 val_loader, _ = create_dataloader(
254     rank, world_size, args.batch_size
255 )
256
257 # Training loop
258 best_acc = 0.0
259 start_epoch = 0
260
261 # Resume from checkpoint if exists
262 if args.resume and os.path.exists(args.resume):
263     start_epoch, _ = load_checkpoint(args.resume, model, optimizer)
264     print(f'Resumed from epoch {start_epoch}')
265
266 for epoch in range(start_epoch, args.epochs):
267     # Set epoch for distributed sampler (important for shuffling)
268     train_sampler.set_epoch(epoch)
269
270     # Train
271     train_loss, train_acc = train_epoch(
272         model, train_loader, criterion, optimizer, rank, epoch
273     )
274
275     # Validate
276     val_loss, val_acc = validate(model, val_loader, criterion, rank)
277
278     # Update learning rate
279     scheduler.step()
280
281     # Print stats (rank 0 only)
282     if rank == 0:
283         print(f'\nEpoch {epoch}:')
284         print(f'  Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.2f}%')
285         print(f'  Val Loss: {val_loss:.4f} | Val Acc: {val_acc:.2f}%')
286         print(f'  LR: {scheduler.get_last_lr()[0]:.6f}\n')
287

```

```

288     # Save checkpoint
289     if val_acc > best_acc:
290         best_acc = val_acc
291         save_checkpoint(
292             model, optimizer, epoch, val_loss,
293             f'checkpoint_best.pth', rank
294         )
295
296     # Regular checkpoint
297     if (epoch + 1) % args.save_freq == 0:
298         save_checkpoint(
299             model, optimizer, epoch, val_loss,
300             f'checkpoint_epoch_{epoch}.pth', rank
301         )
302
303     # Cleanup
304     cleanup_distributed()
305
306 def main():
307     parser = argparse.ArgumentParser(description='PyTorch DDP Training')
308     parser.add_argument('--epochs', type=int, default=100)
309     parser.add_argument('--batch-size', type=int, default=64)
310     parser.add_argument('--lr', type=float, default=0.1)
311     parser.add_argument('--num-classes', type=int, default=10)
312     parser.add_argument('--resume', type=str, default=None)
313     parser.add_argument('--save-freq', type=int, default=10)
314     args = parser.parse_args()
315
316     # World size = number of GPUs
317     world_size = torch.cuda.device_count()
318     print(f'Training on {world_size} GPUs')
319
320     # Spawn processes for each GPU
321     mp.spawn(
322         train_ddp,
323         args=(world_size, args),
324         nprocs=world_size,
325         join=True
326     )
327
328 if __name__ == '__main__':
329     main()

```

Listing 9.1: Complete PyTorch DDP implementation

9.3.2 Launch Script for Multi-Node Training

For multi-node training, use `torchrun` (formerly `torch.distributed.launch`):

```

1  #!/bin/bash
2  # Launch on each node
3
4  # Node 0 (master)
5  torchrun \
6      --nproc_per_node=4 \
7      --nnodes=2 \
8      --node_rank=0 \

```



```

9  --master_addr="192.168.1.1" \
10 --master_port=12355 \
11 train_ddp.py \
12 --epochs 100 \
13 --batch-size 64
14
15 # Node 1
16 torchrun \
17   --nproc_per_node=4 \
18   --nnodes=2 \
19   --node_rank=1 \
20   --master_addr="192.168.1.1" \
21   --master_port=12355 \
22   train_ddp.py \
23   --epochs 100 \
24   --batch-size 64

```

Listing 9.2: Multi-node DDP training script

9.4 Model Parallelism

Model parallelism splits the model across devices when it's too large to fit on a single GPU. Different parts of the model reside on different GPUs.

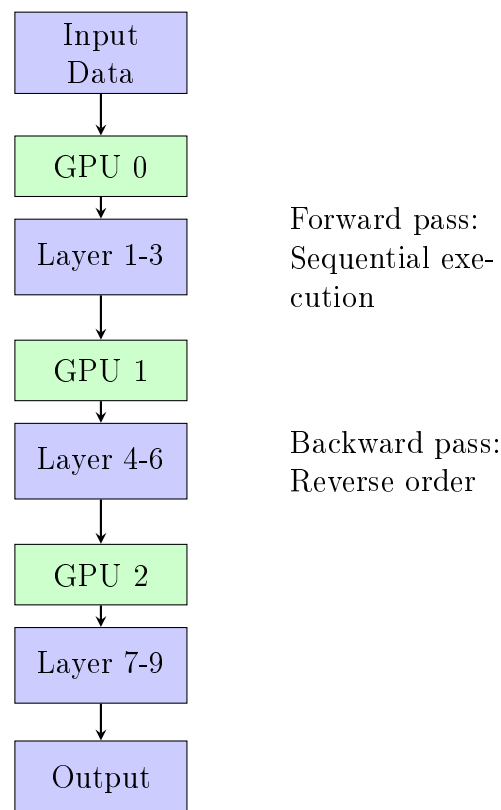


Figure 9.2: Model parallelism: Model layers distributed across GPUs

9.4.1 Pipeline Parallelism

Pipeline parallelism improves model parallelism efficiency by processing multiple micro-batches simultaneously.

```

1 import torch
2 import torch.nn as nn
3 from torch.distributed.pipeline.sync import Pipe
4
5 class ModelParallelResNet(nn.Module):
6     """ResNet with model parallelism across 2 GPUs"""
7     def __init__(self, num_classes=1000):
8         super().__init__()
9
10        # Part 1: Goes to GPU 0
11        self.seq1 = nn.Sequential(
12            nn.Conv2d(3, 64, kernel_size=7, stride=2, padding=3),
13            nn.BatchNorm2d(64),
14            nn.ReLU(inplace=True),
15            nn.MaxPool2d(kernel_size=3, stride=2, padding=1),
16            self._make_layer(64, 128, 3),
17            self._make_layer(128, 256, 4)
18        ).to('cuda:0')
19
20        # Part 2: Goes to GPU 1
21        self.seq2 = nn.Sequential(
22            self._make_layer(256, 512, 6),
23            nn.AdaptiveAvgPool2d((1, 1)),
24            nn.Flatten(),
25            nn.Linear(512, num_classes)
26        ).to('cuda:1')
27
28        def _make_layer(self, in_channels, out_channels, num_blocks):
29            layers = []
30            layers.append(nn.Conv2d(in_channels, out_channels, 3, stride=2, padding=1))
31            layers.append(nn.BatchNorm2d(out_channels))
32            layers.append(nn.ReLU(inplace=True))
33
34            for _ in range(num_blocks - 1):
35                layers.append(nn.Conv2d(out_channels, out_channels, 3, padding=1))
36                layers.append(nn.BatchNorm2d(out_channels))
37                layers.append(nn.ReLU(inplace=True))
38
39            return nn.Sequential(*layers)
40
41        def forward(self, x):
42            # Move input to GPU 0
43            x = x.to('cuda:0')
44
45            # Process on GPU 0
46            x = self.seq1(x)
47
48            # Transfer to GPU 1
49            x = x.to('cuda:1')
50
51            # Process on GPU 1
52            x = self.seq2(x)
53

```

```

54         return x
55
56 class PipelineParallelModel(nn.Module):
57     """Model parallelism with pipeline parallelism"""
58     def __init__(self, num_classes=1000, chunks=8):
59         super().__init__()
60
61         # Define model stages
62         self.stage1 = nn.Sequential(
63             nn.Conv2d(3, 64, kernel_size=7, stride=2, padding=3),
64             nn.BatchNorm2d(64),
65             nn.ReLU(inplace=True),
66             nn.MaxPool2d(kernel_size=3, stride=2, padding=1),
67         )
68
69         self.stage2 = nn.Sequential(
70             self._make_layer(64, 128, 3),
71             self._make_layer(128, 256, 4),
72         )
73
74         self.stage3 = nn.Sequential(
75             self._make_layer(256, 512, 6),
76             nn.AdaptiveAvgPool2d((1, 1)),
77             nn.Flatten(),
78             nn.Linear(512, num_classes)
79         )
80
81         # Create pipeline
82         # Each stage goes to a different GPU
83         self.model = Pipe(
84             nn.Sequential(self.stage1, self.stage2, self.stage3),
85             chunks=chunks, # Number of micro-batches
86             checkpoint='never' # Or 'always', 'except_last'
87         )
88
89     def _make_layer(self, in_channels, out_channels, num_blocks):
90         layers = []
91         layers.append(nn.Conv2d(in_channels, out_channels, 3, stride=2, padding=1))
92         layers.append(nn.BatchNorm2d(out_channels))
93         layers.append(nn.ReLU(inplace=True))
94
95         for _ in range(num_blocks - 1):
96             layers.append(nn.Conv2d(out_channels, out_channels, 3, padding=1))
97             layers.append(nn.BatchNorm2d(out_channels))
98             layers.append(nn.ReLU(inplace=True))
99
100         return nn.Sequential(*layers)
101
102     def forward(self, x):
103         return self.model(x)
104
105 # Training with pipeline parallelism
106 def train_pipeline_parallel():
107     model = PipelineParallelModel(num_classes=1000, chunks=8)
108     criterion = nn.CrossEntropyLoss()
109     optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
110
111     # Training loop

```

```

112     for epoch in range(100):
113         for batch_idx, (data, target) in enumerate(train_loader):
114             # Pipeline handles distribution automatically
115             optimizer.zero_grad()
116             output = model(data)
117
118             # Loss computation needs target on same device as output
119             target = target.to(output.device)
120             loss = criterion(output, target)
121
122             loss.backward()
123             optimizer.step()

```

Listing 9.3: Model parallelism with pipeline parallelism in PyTorch

9.4.2 Tensor Parallelism

For very large models (transformers with billions of parameters), tensor parallelism splits individual layers across GPUs.

```

1  import torch
2  import torch.nn as nn
3  import torch.distributed as dist
4
5  class TensorParallelLinear(nn.Module):
6      """Linear layer with tensor parallelism"""
7      def __init__(self, in_features, out_features, rank, world_size):
8          super().__init__()
9          self.rank = rank
10         self.world_size = world_size
11
12         # Each GPU holds 1/world_size of the output features
13         self.out_features_per_partition = out_features // world_size
14
15         self.weight = nn.Parameter(
16             torch.randn(self.out_features_per_partition, in_features)
17         )
18         self.bias = nn.Parameter(
19             torch.randn(self.out_features_per_partition)
20         )
21
22     def forward(self, x):
23         # Local computation
24         output = torch.matmul(x, self.weight.t()) + self.bias
25
26         # All-gather outputs from all GPUs
27         output_list = [torch.zeros_like(output) for _ in range(self.world_size)]
28         dist.all_gather(output_list, output)
29
30         # Concatenate outputs
31         output = torch.cat(output_list, dim=-1)
32
33     return output
34
35 class TensorParallelAttention(nn.Module):
36     """Multi-head attention with tensor parallelism"""
37     def __init__(self, hidden_size, num_heads, rank, world_size):

```

```

38     super().__init__()
39     self.hidden_size = hidden_size
40     self.num_heads = num_heads
41     self.rank = rank
42     self.world_size = world_size
43
44     # Ensure num_heads is divisible by world_size
45     assert num_heads % world_size == 0
46     self.num_heads_per_partition = num_heads // world_size
47
48     self.head_dim = hidden_size // num_heads
49
50     # Parallel projections
51     self.q_proj = TensorParallelLinear(
52         hidden_size, hidden_size, rank, world_size
53     )
54     self.k_proj = TensorParallelLinear(
55         hidden_size, hidden_size, rank, world_size
56     )
57     self.v_proj = TensorParallelLinear(
58         hidden_size, hidden_size, rank, world_size
59     )
60     self.out_proj = TensorParallelLinear(
61         hidden_size, hidden_size, rank, world_size
62     )
63
64     def forward(self, x):
65         batch_size, seq_len, _ = x.shape
66
67         # Project Q, K, V (parallelized)
68         q = self.q_proj(x)
69         k = self.k_proj(x)
70         v = self.v_proj(x)
71
72         # Reshape for multi-head attention
73         q = q.view(batch_size, seq_len, self.num_heads, self.head_dim)
74         k = k.view(batch_size, seq_len, self.num_heads, self.head_dim)
75         v = v.view(batch_size, seq_len, self.num_heads, self.head_dim)
76
77         # Attention computation (local to each GPU)
78         scores = torch.matmul(
79             q.transpose(1, 2),
80             k.transpose(1, 2).transpose(-2, -1)
81         ) / (self.head_dim ** 0.5)
82
83         attn = torch.softmax(scores, dim=-1)
84         context = torch.matmul(attn, v.transpose(1, 2))
85
86         # Reshape and project
87         context = context.transpose(1, 2).contiguous()
88         context = context.view(batch_size, seq_len, self.hidden_size)
89
90         output = self.out_proj(context)
91
92     return output

```

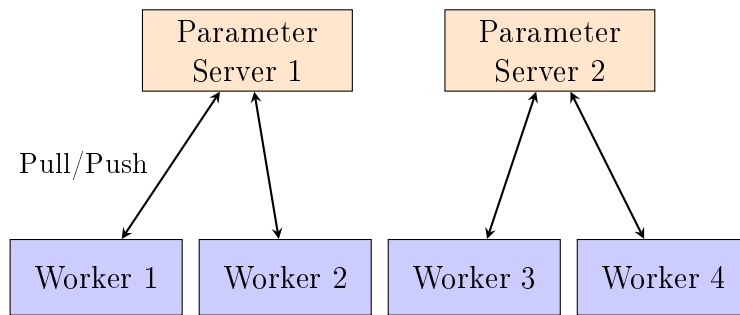
Listing 9.4: Tensor parallelism for transformer layers

9.5 Parameter Servers vs All-Reduce

Two fundamental architectures for gradient synchronization in distributed training.

9.5.1 Parameter Server Architecture

Central servers store model parameters; workers fetch parameters, compute gradients, and push updates.



Workers compute gradients locally

Figure 9.3: Parameter server architecture

Advantages:

- Flexible: supports asynchronous updates
- Fault tolerant: can continue if workers fail
- Scales to many workers

Disadvantages:

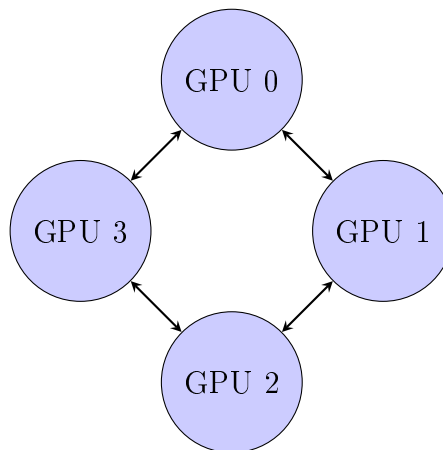
- Parameter servers can become bottleneck
- Complex to implement and debug
- Network communication overhead

9.5.2 All-Reduce Architecture

All workers communicate peer-to-peer to average gradients without central server.

Advantages:

- No single point of failure
- Efficient bandwidth utilization
- Simpler implementation
- Better for synchronous training



Ring All-Reduce: $O(N)$ communication

Figure 9.4: All-reduce architecture (ring topology)

Disadvantages:

- All workers must participate (synchronous)
- Slower workers stall entire system
- Fixed topology

9.6 Gradient Synchronization Strategies

9.6.1 Synchronous SGD

All workers compute gradients, synchronize, then update together.

```

1 import torch
2 import torch.distributed as dist
3
4 def synchronous_sgd_step(model, optimizer, loss):
5     """Synchronous SGD with gradient averaging"""
6     # Backward pass (compute local gradients)
7     optimizer.zero_grad()
8     loss.backward()
9
10    # Average gradients across all workers
11    world_size = dist.get_world_size()
12    for param in model.parameters():
13        if param.grad is not None:
14            # All-reduce sums gradients across workers
15            dist.all_reduce(param.grad.data, op=dist.ReduceOp.SUM)
16            # Average by dividing by world size
17            param.grad.data /= world_size
18
19    # Update parameters (now all workers have same gradients)

```

```

20     optimizer.step()
21
22     # Gradient accumulation for larger effective batch size
23     def synchronous_sgd_with_accumulation(model, optimizer, loss,
24                                           accumulation_steps=4):
25         """Synchronous SGD with gradient accumulation"""
26         # Scale loss by accumulation steps
27         loss = loss / accumulation_steps
28         loss.backward()
29
30         # Only synchronize and step every accumulation_steps
31         if (step + 1) % accumulation_steps == 0:
32             # Synchronize gradients
33             world_size = dist.get_world_size()
34             for param in model.parameters():
35                 if param.grad is not None:
36                     dist.all_reduce(param.grad.data, op=dist.ReduceOp.SUM)
37                     param.grad.data /= world_size
38
39             # Update
40             optimizer.step()
41             optimizer.zero_grad()

```

Listing 9.5: Synchronous gradient synchronization

9.6.2 Asynchronous SGD

Workers update parameters independently without waiting for others.

```

1  import torch
2  import torch.distributed.rpc as rpc
3  from torch.distributed.rpc import RRef
4
5  class ParameterServer:
6      """Centralized parameter server for async SGD"""
7      def __init__(self, model):
8          self.model = model
9          self.optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
10         self.lock = threading.Lock()
11         self.version = 0
12
13         @staticmethod
14         @rpc.functions.async_execution
15         def get_parameters(self):
16             """Workers fetch current parameters"""
17             with self.lock:
18                 return {
19                     name: param.data.clone()
20                     for name, param in self.model.named_parameters()
21                 }
22
23         @staticmethod
24         @rpc.functions.async_execution
25         def apply_gradients(self, gradients):
26             """Workers push gradients"""
27             with self.lock:
28                 # Apply gradients

```



```

29         for name, param in self.model.named_parameters():
30             if name in gradients:
31                 param.grad = gradients[name]
32
33         # Update parameters
34         self.optimizer.step()
35         self.optimizer.zero_grad()
36         self.version += 1
37
38         return self.version
39
40 class AsyncWorker:
41     """Worker for async SGD"""
42     def __init__(self, rank, ps_rref, model, train_loader):
43         self.rank = rank
44         self.ps_rref = ps_rref
45         self.model = model
46         self.train_loader = train_loader
47         self.criterion = torch.nn.CrossEntropyLoss()
48
49     def train_async(self, num_iterations):
50         """Asynchronous training loop"""
51         for iteration in range(num_iterations):
52             # Fetch latest parameters from PS
53             params = self.ps_rref.rpc_sync().get_parameters()
54
55             # Update local model
56             for name, param in self.model.named_parameters():
57                 param.data.copy_(params[name])
58
59             # Compute gradients on local batch
60             data, target = next(iter(self.train_loader))
61             output = self.model(data)
62             loss = self.criterion(output, target)
63
64             self.model.zero_grad()
65             loss.backward()
66
67             # Collect gradients
68             gradients = {
69                 name: param.grad.clone()
70                 for name, param in self.model.named_parameters()
71                 if param.grad is not None
72             }
73
74             # Push gradients to PS (async)
75             version = self.ps_rref.rpc_async().apply_gradients(gradients)
76
77             if iteration % 10 == 0:
78                 print(f'Worker {self.rank} | Iteration {iteration} | '
79                       f'Loss: {loss.item():.4f}')

```

Listing 9.6: Asynchronous parameter server implementation

9.6.3 Hogwild! (Lock-Free Async SGD)

Multiple threads/processes update shared parameters without locks, accepting potential race conditions.

```

1 import torch
2 import torch.multiprocessing as mp
3 from torch.multiprocessing import Queue
4
5 def hogwild_worker(rank, model, dataset, num_epochs, queue):
6     """Lock-free async training worker"""
7     # Each worker has reference to shared model parameters
8     optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
9     criterion = torch.nn.CrossEntropyLoss()
10
11     for epoch in range(num_epochs):
12         for batch_idx, (data, target) in enumerate(dataset):
13             # Forward pass
14             output = model(data)
15             loss = criterion(output, target)
16
17             # Backward pass
18             optimizer.zero_grad()
19             loss.backward()
20
21             # Update (no synchronization - potential race conditions)
22             optimizer.step()
23
24             if batch_idx % 10 == 0:
25                 queue.put((rank, epoch, batch_idx, loss.item()))
26
27 def train_hogwild(model, train_loader, num_workers=4, num_epochs=10):
28     """Hogwild training with multiple workers"""
29     # Share model parameters across processes
30     model.share_memory()
31
32     # Create queue for logging
33     queue = Queue()
34
35     # Spawn workers
36     processes = []
37     for rank in range(num_workers):
38         p = mp.Process(
39             target=hogwild_worker,
40             args=(rank, model, train_loader, num_epochs, queue)
41         )
42         p.start()
43         processes.append(p)
44
45     # Monitor progress
46     while any(p.is_alive() for p in processes):
47         if not queue.empty():
48             rank, epoch, batch, loss = queue.get()
49             print(f'Worker {rank} | Epoch {epoch} | Batch {batch} | Loss: {loss:.4f}')
50
51     # Wait for completion
52     for p in processes:
53         p.join()

```

Listing 9.7: Hogwild implementation

9.7 Horovod: Framework-Agnostic Distributed Training

Horovod provides a unified API for distributed training across TensorFlow, PyTorch, and MXNet.

```
1 import torch
2 import horovod.torch as hvd
3
4 # Initialize Horovod
5 hvd.init()
6
7 # Pin GPU to local rank
8 torch.cuda.set_device(hvd.local_rank())
9
10 # Build model
11 model = ResNet50().cuda()
12
13 # Scale learning rate by number of workers
14 optimizer = torch.optim.SGD(
15     model.parameters(),
16     lr=0.01 * hvd.size(), # Scale LR
17     momentum=0.9
18 )
19
20 # Wrap optimizer with Horovod DistributedOptimizer
21 optimizer = hvd.DistributedOptimizer(
22     optimizer,
23     named_parameters=model.named_parameters(),
24     backward_passes_per_step=1
25 )
26
27 # Broadcast parameters from rank 0
28 hvd.broadcast_parameters(model.state_dict(), root_rank=0)
29 hvd.broadcast_optimizer_state(optimizer, root_rank=0)
30
31 # Create distributed sampler
32 train_sampler = torch.utils.data.distributed.DistributedSampler(
33     train_dataset,
34     num_replicas=hvd.size(),
35     rank=hvd.rank()
36 )
37
38 train_loader = torch.utils.data.DataLoader(
39     train_dataset,
40     batch_size=64,
41     sampler=train_sampler
42 )
43
44 # Training loop
45 for epoch in range(100):
46     train_sampler.set_epoch(epoch)
```

```

47
48     for batch_idx, (data, target) in enumerate(train_loader):
49         data, target = data.cuda(), target.cuda()
50
51         optimizer.zero_grad()
52         output = model(data)
53         loss = criterion(output, target)
54         loss.backward()
55
56         # Horovod handles gradient averaging
57         optimizer.step()
58
59         if hvd.rank() == 0 and batch_idx % 10 == 0:
60             print(f'Epoch {epoch} | Batch {batch_idx} | Loss: {loss.item():.4f}')
61
62 # Launch with horovodrun
63 # horovodrun -np 4 python train_horovod.py
64 # horovodrun -np 8 -H server1:4,server2:4 python train_horovod.py

```

Listing 9.8: Horovod distributed training

9.7.1 Horovod Timeline for Performance Analysis

```

1 import horovod.torch as hvd
2
3 # Enable timeline
4 hvd.init()
5
6 # Set environment variable before running
7 # export HOROVOD_TIMELINE=timeline.json
8
9 # Training code here...
10
11 # Analyze timeline
12 # chrome://tracing in Chrome browser
13 # Load timeline.json to visualize:
14 # - Communication vs computation time
15 # - Gradient synchronization patterns
16 # - Bottlenecks and stragglers

```

Listing 9.9: Horovod performance profiling

9.8 Ray for Flexible Distributed Training

Ray provides flexible distributed computing primitives for ML workloads.

```

1 import ray
2 from ray import train
3 from ray.train import ScalingConfig
4 from ray.train.torch import TorchTrainer
5 import torch
6 import torch.nn as nn
7
8 def train_func(config):
9     """Training function that runs on each worker"""

```

```

10  # Get distributed context
11  rank = train.get_context().get_world_rank()
12  world_size = train.get_context().get_world_size()
13
14  # Build model
15  model = ResNet50()
16  model = train.torch.prepare_model(model) # Wraps with DDP
17
18  # Optimizer
19  optimizer = torch.optim.SGD(
20      model.parameters(),
21      lr=config['lr'],
22      momentum=0.9
23  )
24
25  # Data loader
26  train_dataset = get_dataset()
27  train_loader = torch.utils.data.DataLoader(
28      train_dataset,
29      batch_size=config['batch_size'],
30      shuffle=True
31  )
32  train_loader = train.torch.prepare_data_loader(train_loader)
33
34  # Training loop
35  for epoch in range(config['num_epochs']):
36      model.train()
37      total_loss = 0
38
39      for batch_idx, (data, target) in enumerate(train_loader):
40          optimizer.zero_grad()
41          output = model(data)
42          loss = nn.CrossEntropyLoss()(output, target)
43          loss.backward()
44          optimizer.step()
45
46          total_loss += loss.item()
47
48      avg_loss = total_loss / len(train_loader)
49
50      # Report metrics to Ray
51      train.report({"loss": avg_loss, "epoch": epoch})
52
53  # Create Ray trainer
54  trainer = TorchTrainer(
55      train_loop_per_worker=train_func,
56      train_loop_config={
57          "lr": 0.01,
58          "batch_size": 64,
59          "num_epochs": 100
60      },
61      scaling_config=ScalingConfig(
62          num_workers=4,
63          use_gpu=True,
64          resources_per_worker={"CPU": 4, "GPU": 1}
65      )
66  )
67

```

```

68 # Run training
69 result = trainer.fit()
70 print(f"Final metrics: {result.metrics}")
71
72 # Advanced: Hyperparameter tuning with Ray Tune
73 from ray import tune
74
75 param_space = {
76     "train_loop_config": {
77         "lr": tune.loguniform(1e-4, 1e-1),
78         "batch_size": tune.choice([32, 64, 128]),
79         "num_epochs": 100
80     }
81 }
82
83 tuner = tune.Tuner(
84     trainer,
85     param_space=param_space,
86     tune_config=tune.TuneConfig(
87         metric="loss",
88         mode="min",
89         num_samples=10
90     )
91 )
92
93 results = tuner.fit()
94 best_result = results.get_best_result()
95 print(f"Best hyperparameters: {best_result.config}")

```

Listing 9.10: Distributed training with Ray

9.9 Fault Tolerance and Checkpointing

Production distributed training must handle failures gracefully.

```

1 import torch
2 import torch.distributed as dist
3 import os
4 from datetime import datetime
5 import time
6
7 class ElasticCheckpointer:
8     """Fault-tolerant checkpointing for distributed training"""
9
10     def __init__(self, checkpoint_dir, save_frequency=10):
11         self.checkpoint_dir = checkpoint_dir
12         self.save_frequency = save_frequency
13         os.makedirs(checkpoint_dir, exist_ok=True)
14
15     def save_checkpoint(self, epoch, model, optimizer, scheduler,
16                       metrics, rank):
17         """Save checkpoint with atomic write"""
18         if rank != 0:
19             return # Only rank 0 saves
20
21         checkpoint = {
22             'epoch': epoch,

```

```

23         'model_state_dict': model.module.state_dict(),
24         'optimizer_state_dict': optimizer.state_dict(),
25         'scheduler_state_dict': scheduler.state_dict(),
26         'metrics': metrics,
27         'timestamp': datetime.now().isoformat()
28     }
29
30     # Atomic save: write to temp file, then rename
31     temp_path = os.path.join(
32         self.checkpoint_dir,
33         f'checkpoint_temp_{epoch}.pth'
34     )
35     final_path = os.path.join(
36         self.checkpoint_dir,
37         f'checkpoint_epoch_{epoch}.pth'
38     )
39
40     # Save to temp file
41     torch.save(checkpoint, temp_path)
42
43     # Atomic rename
44     os.replace(temp_path, final_path)
45
46     # Update latest symlink
47     latest_path = os.path.join(self.checkpoint_dir, 'checkpoint_latest.pth')
48     if os.path.exists(latest_path):
49         os.remove(latest_path)
50     os.symlink(final_path, latest_path)
51
52     print(f'Checkpoint saved: {final_path}')
53
54     # Clean old checkpoints (keep last 5)
55     self.cleanup_old_checkpoints(keep=5)
56
57 def load_checkpoint(self, model, optimizer=None, scheduler=None):
58     """Load latest checkpoint"""
59     latest_path = os.path.join(self.checkpoint_dir, 'checkpoint_latest.pth')
60
61     if not os.path.exists(latest_path):
62         print('No checkpoint found, starting from scratch')
63         return 0, {}
64
65     print(f'Loading checkpoint: {latest_path}')
66     checkpoint = torch.load(latest_path)
67
68     # Load model state
69     model.module.load_state_dict(checkpoint['model_state_dict'])
70
71     # Load optimizer state
72     if optimizer and 'optimizer_state_dict' in checkpoint:
73         optimizer.load_state_dict(checkpoint['optimizer_state_dict'])
74
75     # Load scheduler state
76     if scheduler and 'scheduler_state_dict' in checkpoint:
77         scheduler.load_state_dict(checkpoint['scheduler_state_dict'])
78
79     print(f"Resumed from epoch {checkpoint['epoch']}")
80

```

```

81     return checkpoint['epoch'], checkpoint.get('metrics', {})
82
83     def cleanup_old_checkpoints(self, keep=5):
84         """Remove old checkpoints, keeping only the latest N"""
85         checkpoints = sorted([
86             f for f in os.listdir(self.checkpoint_dir)
87             if f.startswith('checkpoint_epoch_') and f.endswith('.pth')
88         ])
89
90         if len(checkpoints) > keep:
91             for old_checkpoint in checkpoints[:-keep]:
92                 path = os.path.join(self.checkpoint_dir, old_checkpoint)
93                 os.remove(path)
94                 print(f'Removed old checkpoint: {path}')
95
96     def train_with_fault_tolerance(rank, world_size):
97         """Training with automatic restart on failure"""
98         # Setup
99         setup_distributed(rank, world_size)
100
101         model = ResNet50().cuda(rank)
102         model = DDP(model, device_ids=[rank])
103
104         optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
105         scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=30)
106
107         # Checkpointer
108         checkpointer = ElasticCheckpointer(
109             checkpoint_dir='./checkpoints',
110             save_frequency=10
111         )
112
113         # Resume from checkpoint
114         start_epoch, _ = checkpointer.load_checkpoint(
115             model, optimizer, scheduler
116         )
117
118         # Training loop with fault handling
119         for epoch in range(start_epoch, 100):
120             try:
121                 # Training code
122                 train_loss = train_epoch(model, train_loader, optimizer, rank)
123                 val_loss = validate(model, val_loader, rank)
124                 scheduler.step()
125
126                 # Save checkpoint
127                 if (epoch + 1) % checkpointer.save_frequency == 0:
128                     metrics = {
129                         'train_loss': train_loss,
130                         'val_loss': val_loss,
131                         'lr': scheduler.get_last_lr()[0]
132                     }
133                     checkpointer.save_checkpoint(
134                         epoch, model, optimizer, scheduler, metrics, rank
135                     )
136
137             except RuntimeError as e:
138                 if "out of memory" in str(e):

```



```

139         print(f"OOM error at epoch {epoch}, saving checkpoint...")
140         # Save emergency checkpoint
141         checkpointer.save_checkpoint(
142             epoch, model, optimizer, scheduler, {}, rank
143         )
144         # Clear cache and retry
145         torch.cuda.empty_cache()
146         continue
147     else:
148         raise
149
150 except Exception as e:
151     # Save checkpoint on unexpected error
152     print(f"Error at epoch {epoch}: {e}")
153     checkpointer.save_checkpoint(
154         epoch, model, optimizer, scheduler, {}, rank
155     )
156     raise
157
158 cleanup_distributed()
159
160 # Elastic training with automatic restart
161 def elastic_train_with_auto_restart():
162     """Wrapper that restarts training on failure"""
163     max_retries = 3
164     retry_count = 0
165
166     while retry_count < max_retries:
167         try:
168             world_size = torch.cuda.device_count()
169             mp.spawn(
170                 train_with_fault_tolerance,
171                 args=(world_size,),
172                 nprocs=world_size,
173                 join=True
174             )
175             print("Training completed successfully")
176             break
177
178         except Exception as e:
179             retry_count += 1
180             print(f"Training failed (attempt {retry_count}/{max_retries}): {e}")
181
182             if retry_count < max_retries:
183                 print(f"Restarting in 10 seconds...")
184                 time.sleep(10)
185             else:
186                 print("Max retries reached, giving up")
187                 raise

```

Listing 9.11: Fault-tolerant training with elastic checkpointing

9.10 Performance Benchmarking and Optimization

9.10.1 Measuring Distributed Training Performance

```

1 import torch
2 import time
3 from contextlib import contextmanager
4 import numpy as np
5
6 class DistributedTrainingBenchmark:
7     """Benchmark distributed training performance"""
8
9     def __init__(self, rank, world_size):
10         self.rank = rank
11         self.world_size = world_size
12         self.metrics = {}
13
14     @contextmanager
15     def timer(self, name):
16         """Context manager for timing code blocks"""
17         start = time.perf_counter()
18         yield
19         elapsed = time.perf_counter() - start
20
21         if name not in self.metrics:
22             self.metrics[name] = []
23         self.metrics[name].append(elapsed)
24
25     def benchmark_epoch(self, model, dataloader, criterion, optimizer):
26         """Benchmark one training epoch with detailed metrics"""
27         model.train()
28
29         epoch_metrics = {
30             'forward_time': [],
31             'backward_time': [],
32             'optimizer_time': [],
33             'data_loading_time': [],
34             'communication_time': []
35         }
36
37         batch_start = time.perf_counter()
38
39         for batch_idx, (data, target) in enumerate(dataloader):
40             # Data loading time
41             data_time = time.perf_counter() - batch_start
42             epoch_metrics['data_loading_time'].append(data_time)
43
44             data = data.cuda(self.rank, non_blocking=True)
45             target = target.cuda(self.rank, non_blocking=True)
46
47             # Forward pass
48             forward_start = time.perf_counter()
49             optimizer.zero_grad(set_to_none=True)
50             output = model(data)
51             loss = criterion(output, target)
52             forward_time = time.perf_counter() - forward_start
53             epoch_metrics['forward_time'].append(forward_time)
54
55             # Backward pass
56             backward_start = time.perf_counter()
57             loss.backward()

```

```

58     backward_time = time.perf_counter() - backward_start
59     epoch_metrics['backward_time'].append(backward_time)
60
61     # Communication time (gradient synchronization in DDP)
62     comm_start = time.perf_counter()
63     # DDP automatically syncs gradients during backward
64     # This measures the explicit sync if needed
65     torch.cuda.synchronize()
66     comm_time = time.perf_counter() - comm_start
67     epoch_metrics['communication_time'].append(comm_time)
68
69     # Optimizer step
70     opt_start = time.perf_counter()
71     optimizer.step()
72     torch.cuda.synchronize()
73     opt_time = time.perf_counter() - opt_start
74     epoch_metrics['optimizer_time'].append(opt_time)
75
76     batch_start = time.perf_counter()
77
78     return epoch_metrics
79
80 def compute_statistics(self, metrics):
81     """Compute statistics across all batches"""
82     stats = {}
83     for metric_name, values in metrics.items():
84         values = np.array(values)
85         stats[metric_name] = {
86             'mean': values.mean(),
87             'std': values.std(),
88             'min': values.min(),
89             'max': values.max(),
90             'p50': np.percentile(values, 50),
91             'p95': np.percentile(values, 95),
92             'p99': np.percentile(values, 99)
93         }
94     return stats
95
96 def print_report(self, stats):
97     """Print performance report"""
98     if self.rank == 0:
99         print("\n" + "="*70)
100        print("Distributed Training Performance Report")
101        print("="*70)
102        print(f"World Size: {self.world_size} GPUs")
103        print()
104
105        for metric_name, metric_stats in stats.items():
106            print(f"{metric_name}:")
107            print(f"  Mean: {metric_stats['mean']*1000:.2f} ms")
108            print(f"  Std:  {metric_stats['std']*1000:.2f} ms")
109            print(f"  P50:  {metric_stats['p50']*1000:.2f} ms")
110            print(f"  P95:  {metric_stats['p95']*1000:.2f} ms")
111            print(f"  P99:  {metric_stats['p99']*1000:.2f} ms")
112            print()
113
114        # Compute efficiency metrics
115        total_time_per_batch = sum(

```

```

116         s['mean'] for s in stats.values()
117     )
118     compute_time = (
119         stats['forward_time']['mean'] +
120         stats['backward_time']['mean'] +
121         stats['optimizer_time']['mean']
122     )
123     comm_time = stats['communication_time']['mean']
124
125     compute_ratio = compute_time / total_time_per_batch * 100
126     comm_ratio = comm_time / total_time_per_batch * 100
127
128     print(f"Efficiency Analysis:")
129     print(f"  Computation: {compute_ratio:.1f}%")
130     print(f"  Communication: {comm_ratio:.1f}%")
131     print(f"  Data Loading: {stats['data_loading_time']['mean']/
total_time_per_batch*100:.1f}%")
132     print()
133
134     # Throughput
135     samples_per_sec = 1.0 / total_time_per_batch * self.world_size
136     print(f"Throughput:")
137     print(f"  Samples/sec (per GPU): {samples_per_sec:.2f}")
138     print(f"  Samples/sec (total): {samples_per_sec * self.world_size:.2f}")
139     print(f"="*70 + "\n")
140
141     def measure_communication_overhead(self, tensor_size=(1024, 1024)):
142         """Measure pure communication overhead"""
143         tensor = torch.randn(tensor_size).cuda(self.rank)
144
145         # Warmup
146         for _ in range(10):
147             dist.all_reduce(tensor)
148
149         torch.cuda.synchronize()
150
151         # Measure
152         times = []
153         for _ in range(100):
154             start = time.perf_counter()
155             dist.all_reduce(tensor)
156             torch.cuda.synchronize()
157             elapsed = time.perf_counter() - start
158             times.append(elapsed)
159
160         if self.rank == 0:
161             print(f"\nAll-Reduce Communication Benchmark:")
162             print(f"  Tensor size: {tensor_size}")
163             print(f"  Mean latency: {np.mean(times)*1000:.2f} ms")
164             print(f"  Bandwidth: {tensor.element_size() * tensor.numel() / np.mean(
times) / 1e9:.2f} GB/s")
165
166     def scaling_efficiency_benchmark():
167         """Benchmark scaling efficiency across different GPU counts"""
168         model = ResNet50()
169
170         results = {}
171

```

```

172     for num_gpus in [1, 2, 4, 8]:
173         # Run training with num_gpus
174         # Measure throughput
175         throughput = measure_throughput(model, num_gpus)
176         results[num_gpus] = throughput
177
178     # Compute scaling efficiency
179     baseline = results[1]
180
181     print("\nScaling Efficiency:")
182     print(f'{{GPUs':<10}} {{Throughput':<15}} {{Speedup':<15}} {{Efficiency':<15}}")
183     print("-" * 60)
184
185     for num_gpus, throughput in results.items():
186         speedup = throughput / baseline
187         efficiency = speedup / num_gpus
188         print(f"{{num_gpus:<10}} {{throughput:<15.2f}} {{speedup:<15.2f}} {{efficiency
    *100:<15.1f}}%")

```

Listing 9.12: Comprehensive performance benchmarking

9.10.2 Identifying and Resolving Bottlenecks

```

1 import torch
2 from torch.profiler import profile, ProfilerActivity, schedule
3
4 def profile_distributed_training(model, dataloader, num_steps=100):
5     """Profile distributed training to identify bottlenecks"""
6
7     # Create profiler with schedule
8     prof_schedule = schedule(
9         wait=5,      # Skip first 5 steps
10        warmup=5,     # Warmup for 5 steps
11        active=10,    # Profile 10 steps
12        repeat=2      # Repeat cycle 2 times
13    )
14
15    with profile(
16        activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
17        schedule=prof_schedule,
18        on_trace_ready=torch.profiler.tensorboard_trace_handler('./profiler_logs'),
19        record_shapes=True,
20        profile_memory=True,
21        with_stack=True
22    ) as prof:
23
24        for step, (data, target) in enumerate(dataloader):
25            if step >= num_steps:
26                break
27
28            data = data.cuda(non_blocking=True)
29            target = target.cuda(non_blocking=True)
30
31            output = model(data)
32            loss = nn.CrossEntropyLoss()(output, target)
33            loss.backward()
34            optimizer.step()

```

```

35         optimizer.zero_grad()
36
37         prof.step()
38
39         # Print summary
40         print(prof.key_averages().table(
41             sort_by="cuda_time_total",
42             row_limit=20
43         ))
44
45         # Export for visualization
46         # tensorboard --logdir=./profiler_logs
47
48     # Optimization tips based on profiling results:
49
50     def optimize_data_loading():
51         """Optimize data loading bottleneck"""
52         dataloader = DataLoader(
53             dataset,
54             batch_size=64,
55             num_workers=4,           # Parallel data loading
56             pin_memory=True,        # Faster GPU transfer
57             persistent_workers=True, # Keep workers alive
58             prefetch_factor=2       # Prefetch batches
59         )
60         return dataloader
61
62     def optimize_gradient_communication():
63         """Optimize gradient synchronization"""
64         # Use gradient compression (requires apex)
65         # from apex import amp
66         # model, optimizer = amp.initialize(model, optimizer, opt_level="O1")
67
68         # Or use gradient bucketing (built into DDP)
69         model = DDP(
70             model,
71             device_ids=[rank],
72             bucket_cap_mb=25, # Larger buckets = fewer all-reduces
73             gradient_as_bucket_view=True
74         )
75
76         return model
77
78     def optimize_mixed_precision():
79         """Use automatic mixed precision"""
80         from torch.cuda.amp import autocast, GradScaler
81
82         scaler = GradScaler()
83
84         for data, target in dataloader:
85             optimizer.zero_grad()
86
87             # Forward in mixed precision
88             with autocast():
89                 output = model(data)
90                 loss = criterion(output, target)
91
92             # Backward with gradient scaling

```

```

93     scaler.scale(loss).backward()
94     scaler.step(optimizer)
95     scaler.update()

```

Listing 9.13: Bottleneck detection and optimization

9.11 Resource Allocation and Scheduling

9.11.1 SLURM Job Scheduling

```

1  #!/bin/bash
2  #SBATCH --job-name=distributed_training
3  #SBATCH --nodes=4
4  #SBATCH --ntasks-per-node=4
5  #SBATCH --cpus-per-task=8
6  #SBATCH --gres=gpu:4
7  #SBATCH --time=48:00:00
8  #SBATCH --partition=gpu
9  #SBATCH --output=logs/train_%j.out
10 #SBATCH --error=logs/train_%j.err
11
12 # Load modules
13 module load cuda/11.8
14 module load python/3.9
15
16 # Activate environment
17 source ~/venv/bin/activate
18
19 # Get master node
20 export MASTER_ADDR=$(scontrol show hostname ${SLURM_NODELIST} | head -n 1)
21 export MASTER_PORT=12355
22
23 # Launch training
24 srun python -u train_distributed.py \
25     --epochs 100 \
26     --batch-size 64 \
27     --lr 0.1 \
28     --checkpoint-dir checkpoints/
29
30 # Post-processing
31 python evaluate_model.py --checkpoint checkpoints/checkpoint_best.pth

```

Listing 9.14: SLURM batch script for distributed training

9.11.2 Kubernetes Job Specification

```

1  apiVersion: kubeflow.org/v1
2  kind: PyTorchJob
3  metadata:
4    name: distributed-training
5    namespace: ml-workloads
6  spec:
7    pytorchReplicaSpecs:
8      Master:

```

```
replicas: 1
restartPolicy: OnFailure
template:
  spec:
    containers:
      - name: pytorch
        image: pytorch/pytorch:2.0.0-cuda11.8-cudnn8-runtime
        command:
          - python
          - -u
          - train_distributed.py
        args:
          - --epochs=100
          - --batch-size=64
    resources:
      limits:
        nvidia.com/gpu: 1
        memory: 32Gi
        cpu: 8
      requests:
        nvidia.com/gpu: 1
        memory: 16Gi
        cpu: 4
    volumeMounts:
      - name: data
        mountPath: /data
      - name: checkpoints
        mountPath: /checkpoints
    volumes:
      - name: data
        persistentVolumeClaim:
          claimName: training-data-pvc
      - name: checkpoints
        persistentVolumeClaim:
          claimName: checkpoint-pvc
```

Worker:

```
replicas: 3
restartPolicy: OnFailure
template:
  spec:
    containers:
      - name: pytorch
        image: pytorch/pytorch:2.0.0-cuda11.8-cudnn8-runtime
        command:
          - python
          - -u
          - train_distributed.py
        args:
          - --epochs=100
          - --batch-size=64
    resources:
      limits:
        nvidia.com/gpu: 1
        memory: 32Gi
        cpu: 8
      requests:
        nvidia.com/gpu: 1
```



```

67         memory: 16Gi
68         cpu: 4
69         volumeMounts:
70         - name: data
71           mountPath: /data
72         - name: checkpoints
73           mountPath: /checkpoints
74         volumes:
75         - name: data
76           persistentVolumeClaim:
77             claimName: training-data-pvc
78         - name: checkpoints
79           persistentVolumeClaim:
80             claimName: checkpoint-pvc

```

Listing 9.15: Kubernetes PyTorchJob for distributed training

9.12 Memory Management in Distributed Settings

Efficient memory management is critical for distributed training, especially for large models. This section covers techniques for optimizing memory usage across GPUs.

9.12.1 Gradient Checkpointing

Gradient checkpointing trades computation for memory by recomputing activations during backward pass instead of storing them.

```

1 import torch
2 import torch.nn as nn
3 from torch.utils.checkpoint import checkpoint
4
5 class CheckpointedResidualBlock(nn.Module):
6     """Residual block with gradient checkpointing"""
7     def __init__(self, channels):
8         super().__init__()
9         self.conv1 = nn.Conv2d(channels, channels, 3, padding=1)
10        self.bn1 = nn.BatchNorm2d(channels)
11        self.conv2 = nn.Conv2d(channels, channels, 3, padding=1)
12        self.bn2 = nn.BatchNorm2d(channels)
13        self.relu = nn.ReLU(inplace=True)
14
15    def forward(self, x):
16        # Checkpoint this block to save memory
17        return checkpoint(self._forward, x, use_reentrant=False)
18
19    def _forward(self, x):
20        identity = x
21        out = self.conv1(x)
22        out = self.bn1(out)
23        out = self.relu(out)
24        out = self.conv2(out)
25        out = self.bn2(out)
26        out += identity
27        out = self.relu(out)
28        return out

```

```

29
30 # Transformer with selective checkpointing
31 class CheckpointedTransformerLayer(nn.Module):
32     """Transformer layer with gradient checkpointing"""
33     def __init__(self, d_model, nhead, dim_feedforward):
34         super().__init__()
35         self.self_attn = nn.MultiheadAttention(d_model, nhead)
36         self.linear1 = nn.Linear(d_model, dim_feedforward)
37         self.linear2 = nn.Linear(dim_feedforward, d_model)
38         self.norm1 = nn.LayerNorm(d_model)
39         self.norm2 = nn.LayerNorm(d_model)
40         self.dropout = nn.Dropout(0.1)
41
42     def forward(self, src):
43         # Checkpoint attention and feedforward
44         src2 = checkpoint(self._attention_block, src, use_reentrant=False)
45         src = src + self.dropout(src2)
46         src = self.norm1(src)
47
48         src2 = checkpoint(self._feedforward_block, src, use_reentrant=False)
49         src = src + self.dropout(src2)
50         src = self.norm2(src)
51
52         return src
53
54     def _attention_block(self, src):
55         return self.self_attn(src, src, src)[0]
56
57     def _feedforward_block(self, src):
58         src2 = self.linear2(torch.relu(self.linear1(src)))
59         return src2
60
61 # Memory usage comparison
62 def measure_memory_usage():
63     """Compare memory usage with and without checkpointing"""
64     import torch.cuda as cuda
65
66     # Without checkpointing
67     model_no_cp = nn.Sequential(*[
68         ResidualBlock(256) for _ in range(50)
69     ]).cuda()
70
71     cuda.reset_peak_memory_stats()
72     x = torch.randn(32, 256, 56, 56).cuda()
73     y = model_no_cp(x)
74     y.sum().backward()
75     memory_no_cp = cuda.max_memory_allocated() / 1e9 # GB
76
77     # With checkpointing
78     model_cp = nn.Sequential(*[
79         CheckpointedResidualBlock(256) for _ in range(50)
80     ]).cuda()
81
82     cuda.reset_peak_memory_stats()
83     x = torch.randn(32, 256, 56, 56).cuda()
84     y = model_cp(x)
85     y.sum().backward()
86     memory_cp = cuda.max_memory_allocated() / 1e9 # GB

```

```

87 print(f"Memory without checkpointing: {memory_no_cp:.2f} GB")
88 print(f"Memory with checkpointing: {memory_cp:.2f} GB")
89 print(f"Memory saved: {(1 - memory_cp/memory_no_cp)*100:.1f}%")

```

Listing 9.16: Gradient checkpointing implementation

9.12.2 ZeRO: Zero Redundancy Optimizer

ZeRO (from Microsoft DeepSpeed) partitions optimizer states, gradients, and parameters across devices to reduce memory redundancy.

```

1 import torch
2 import deepspeed
3 from deepspeed.ops.adam import DeepSpeedCPUAdam
4
5 # DeepSpeed configuration
6 ds_config = {
7     "train_batch_size": 64,
8     "train_micro_batch_size_per_gpu": 8,
9     "gradient_accumulation_steps": 8,
10
11     # ZeRO optimization
12     "zero_optimization": {
13         "stage": 3, # Stage 0, 1, 2, or 3
14         # Stage 1: Partition optimizer states
15         # Stage 2: Partition optimizer states + gradients
16         # Stage 3: Partition optimizer states + gradients + parameters
17
18         "offload_optimizer": {
19             "device": "cpu", # Offload to CPU
20             "pin_memory": True
21         },
22         "offload_param": {
23             "device": "cpu",
24             "pin_memory": True
25         },
26
27         "overlap_comm": True, # Overlap communication with computation
28         "contiguous_gradients": True,
29         "reduce_bucket_size": 5e7,
30         "stage3_prefetch_bucket_size": 5e7,
31         "stage3_param_persistence_threshold": 1e5,
32     },
33
34     # Mixed precision training
35     "fp16": {
36         "enabled": True,
37         "loss_scale": 0,
38         "initial_scale_power": 16,
39         "loss_scale_window": 1000,
40         "hysteresis": 2,
41         "min_loss_scale": 1
42     },
43
44     # Gradient clipping
45     "gradient_clipping": 1.0,

```

```

46
47     # Logging
48     "steps_per_print": 100,
49     "wall_clock_breakdown": False
50 }
51
52 # Initialize model
53 model = TransformerModel(
54     vocab_size=50000,
55     d_model=1024,
56     nhead=16,
57     num_layers=24
58 )
59
60 # Initialize DeepSpeed
61 model_engine, optimizer, train_loader, _ = deepspeed.initialize(
62     model=model,
63     model_parameters=model.parameters(),
64     config=ds_config,
65     training_data=train_dataset
66 )
67
68 # Training loop
69 for epoch in range(num_epochs):
70     for batch in train_loader:
71         inputs, labels = batch
72
73         # Forward pass
74         outputs = model_engine(inputs)
75         loss = criterion(outputs, labels)
76
77         # Backward pass (DeepSpeed handles ZeRO partitioning)
78         model_engine.backward(loss)
79
80         # Optimizer step
81         model_engine.step()
82
83 # Save checkpoint
84 model_engine.save_checkpoint('checkpoints/')
85
86 # Memory comparison:
87 # - Without ZeRO Stage 3: ~32 GB per GPU for 1B parameter model
88 # - With ZeRO Stage 3: ~8 GB per GPU for same model
89 # - Can train 4x larger models on same hardware

```

Listing 9.17: DeepSpeed ZeRO optimization

9.12.3 Activation Memory Optimization

```

1 import torch
2 import torch.nn as nn
3
4 class MemoryEfficientAttention(nn.Module):
5     """
6     Memory-efficient attention using Flash Attention.
7     Reduces memory from  $O(N^2)$  to  $O(N)$  for sequence length  $N$ .
8     """

```

```

9  def __init__(self, d_model, nhead):
10     super().__init__()
11     self.d_model = d_model
12     self.nhead = nhead
13     self.head_dim = d_model // nhead
14
15     self.q_proj = nn.Linear(d_model, d_model)
16     self.k_proj = nn.Linear(d_model, d_model)
17     self.v_proj = nn.Linear(d_model, d_model)
18     self.out_proj = nn.Linear(d_model, d_model)
19
20     def forward(self, x):
21         batch_size, seq_len, _ = x.shape
22
23         # Project Q, K, V
24         q = self.q_proj(x).view(batch_size, seq_len, self.nhead, self.head_dim)
25         k = self.k_proj(x).view(batch_size, seq_len, self.nhead, self.head_dim)
26         v = self.v_proj(x).view(batch_size, seq_len, self.nhead, self.head_dim)
27
28         # Use Flash Attention (if available)
29         try:
30             from flash_attn import flash_attn_func
31             # Flash attention computes in blocks, reducing memory
32             out = flash_attn_func(q, k, v, dropout_p=0.0, causal=False)
33         except ImportError:
34             # Fallback to memory-efficient attention
35             out = self._efficient_attention(q, k, v)
36
37         out = out.reshape(batch_size, seq_len, self.d_model)
38         return self.out_proj(out)
39
40     def _efficient_attention(self, q, k, v):
41         """Memory-efficient attention using chunking"""
42         batch_size, seq_len, nhead, head_dim = q.shape
43         chunk_size = 512 # Process in chunks
44
45         outputs = []
46         for i in range(0, seq_len, chunk_size):
47             q_chunk = q[:, i:i+chunk_size]
48
49             # Compute attention for this chunk
50             scores = torch.matmul(
51                 q_chunk.transpose(1, 2), # (batch, nhead, chunk, head_dim)
52                 k.transpose(1, 2).transpose(-2, -1) # (batch, nhead, head_dim,
53                 seq_len)
54                 ) / (head_dim ** 0.5)
55
56             attn = torch.softmax(scores, dim=-1)
57             out = torch.matmul(attn, v.transpose(1, 2)) # (batch, nhead, chunk,
58             head_dim)
59             outputs.append(out)
60
61         return torch.cat(outputs, dim=2).transpose(1, 2)
62
63     # Inplace operations to save memory
64     class InplaceOptimizedModel(nn.Module):
65         """Use inplace operations where possible"""
66         def __init__(self):

```

```

65     super().__init__()
66     self.conv1 = nn.Conv2d(3, 64, 3, padding=1)
67     self.bn1 = nn.BatchNorm2d(64)
68     self.relu = nn.ReLU(inplace=True)  # Inplace ReLU
69     self.conv2 = nn.Conv2d(64, 128, 3, padding=1)
70     self.bn2 = nn.BatchNorm2d(128)
71
72     def forward(self, x):
73         x = self.conv1(x)
74         x = self.bn1(x)
75         x = self.relu(x)  # Modifies x in-place
76         x = self.conv2(x)
77         x = self.bn2(x)
78         x = self.relu(x)
79         return x
80
81     # Mixed precision training for memory savings
82     from torch.cuda.amp import autocast, GradScaler
83
84     def train_with_mixed_precision(model, train_loader):
85         """
86         Mixed precision training:
87         - FP16 for forward/backward (2x less memory)
88         - FP32 for optimizer states and critical operations
89         """
90         scaler = GradScaler()
91         optimizer = torch.optim.Adam(model.parameters())
92
93         for data, target in train_loader:
94             optimizer.zero_grad()
95
96             # Forward in FP16
97             with autocast():
98                 output = model(data)
99                 loss = criterion(output, target)
100
101             # Backward with scaled gradients
102             scaler.scale(loss).backward()
103
104             # Unscale before clipping
105             scaler.unscale_(optimizer)
106             torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
107
108             # Step with scaled gradients
109             scaler.step(optimizer)
110             scaler.update()

```

Listing 9.18: Techniques for reducing activation memory

9.13 Communication Optimization

Communication overhead is often the bottleneck in distributed training. This section covers advanced optimization techniques.

9.13.1 Gradient Compression

```

1 import torch
2 import torch.distributed as dist
3
4 class GradientCompressor:
5     """Compress gradients before communication"""
6
7     @staticmethod
8     def topk_compression(tensor, compression_ratio=0.01):
9         """
10         Top-K sparsification: Only communicate largest K% of gradients.
11         Reduces communication by 100x with minimal accuracy loss.
12         """
13         numel = tensor.numel()
14         k = max(1, int(numel * compression_ratio))
15
16         # Find top-k absolute values
17         _, indices = torch.topk(tensor.abs().flatten(), k)
18         values = tensor.flatten()[indices]
19
20         # Create sparse tensor
21         compressed = torch.zeros_like(tensor.flatten())
22         compressed[indices] = values
23
24         return compressed.view_as(tensor), indices
25
26     @staticmethod
27     def quantization_compression(tensor, num_bits=8):
28         """
29         Quantize gradients to reduce precision.
30         8-bit: 4x compression, 4-bit: 8x compression
31         """
32         # Find min/max
33         min_val = tensor.min()
34         max_val = tensor.max()
35
36         # Quantize to num_bits
37         num_levels = 2 ** num_bits - 1
38         scale = (max_val - min_val) / num_levels
39
40         quantized = ((tensor - min_val) / scale).round().to(torch.int16)
41
42         # Dequantize
43         dequantized = quantized.to(tensor.dtype) * scale + min_val
44
45         return dequantized, (min_val, max_val, scale)
46
47     @staticmethod
48     def powerSGD_compression(tensor, rank=4):
49         """
50         PowerSGD: Low-rank approximation of gradient matrix.
51         Particularly effective for large weight matrices.
52         """
53         if tensor.dim() < 2:
54             return tensor
55
56         # Reshape to 2D if needed

```

```

57     original_shape = tensor.shape
58     if tensor.dim() > 2:
59         tensor = tensor.flatten(1)
60
61     m, n = tensor.shape
62
63     # Low-rank approximation:  $G \approx P @ Q^T$ 
64     # where P is m x rank and Q is n x rank
65     U, S, Vh = torch.linalg.svd(tensor, full_matrices=False)
66
67     # Keep only top 'rank' components
68     P = U[:, :rank] @ torch.diag(S[:rank].sqrt())
69     Q = (Vh[:rank, :].T @ torch.diag(S[:rank].sqrt()))
70
71     # Reconstruct
72     compressed = P @ Q.T
73
74     return compressed.reshape(original_shape)
75
76 # Use compression in distributed training
77 class CompressedDDP(nn.Module):
78     """DDP with gradient compression"""
79     def __init__(self, model, compression_ratio=0.01):
80         super().__init__()
81         self.model = model
82         self.compression_ratio = compression_ratio
83         self.compressor = GradientCompressor()
84
85     def forward(self, *args, **kwargs):
86         return self.model(*args, **kwargs)
87
88     def backward_with_compression(self, loss):
89         """Backward with compressed gradient communication"""
90         loss.backward()
91
92         # Compress and communicate gradients
93         for param in self.model.parameters():
94             if param.grad is not None:
95                 # Compress
96                 compressed_grad, _ = self.compressor.topk_compression(
97                     param.grad,
98                     self.compression_ratio
99                 )
100
101                 # Communicate compressed gradients
102                 dist.all_reduce(compressed_grad)
103
104                 # Average
105                 compressed_grad /= dist.get_world_size()
106
107                 # Update grad
108                 param.grad.copy_(compressed_grad)
109
110 # Hierarchical All-Reduce for multi-node training
111 def hierarchical_all_reduce(tensor, local_group, global_group):
112     """
113     Two-level all-reduce:
114     1. All-reduce within node (fast NVLink)

```



```

115     2. All-reduce across nodes (slower network)
116
117     Reduces inter-node communication.
118     """
119     # Local all-reduce (within node, fast)
120     dist.all_reduce(tensor, group=local_group)
121     tensor /= dist.get_world_size(local_group)
122
123     # Select one GPU per node for global communication
124     if dist.get_rank(local_group) == 0:
125         # Global all-reduce (across nodes)
126         dist.all_reduce(tensor, group=global_group)
127         tensor /= dist.get_world_size(global_group)
128
129     # Broadcast result within node
130     dist.broadcast(tensor, src=0, group=local_group)

```

Listing 9.19: Gradient compression techniques

9.13.2 Overlapping Computation and Communication

```

1  import torch
2  import torch.distributed as dist
3  from typing import List
4
5  class OverlappedGradientCommunication:
6      """
7      Overlap gradient communication with backward pass.
8      Start communicating gradients for earlier layers while
9      still computing gradients for later layers.
10     """
11
12     def __init__(self, model, bucket_size_mb=25):
13         self.model = model
14         self.bucket_size_mb = bucket_size_mb
15         self.gradient_buckets: List[List[torch.Tensor]] = []
16         self.communication_handles = []
17
18         self._register_hooks()
19
20     def _register_hooks(self):
21         """Register backward hooks to trigger async communication"""
22         def hook_fn(grad):
23             # Trigger async all-reduce when gradient is ready
24             handle = dist.all_reduce(grad, async_op=True)
25             self.communication_handles.append(handle)
26             return grad
27
28         for param in self.model.parameters():
29             if param.requires_grad:
30                 param.register_hook(hook_fn)
31
32     def synchronize(self):
33         """Wait for all async communications to complete"""
34         for handle in self.communication_handles:
35             handle.wait()
36         self.communication_handles.clear()

```

```

37
38 # PyTorch DDP automatically does this with gradient bucketing
39 model = DDP(
40     model,
41     device_ids=[rank],
42     bucket_cap_mb=25, # Bucket size for overlapping
43     broadcast_buffers=False, # Don't sync buffers every iteration
44     find_unused_parameters=False, # Faster if all params used
45     gradient_as_bucket_view=True # Save memory
46 )
47
48 # Manual implementation for custom control
49 class CustomOverlappedDDP:
50     """Custom DDP with fine-grained overlap control"""
51
52     def __init__(self, model, overlap_ratio=0.5):
53         self.model = model
54         self.overlap_ratio = overlap_ratio
55         self.param_groups = self._create_param_groups()
56
57     def _create_param_groups(self):
58         """Group parameters for staged communication"""
59         groups = []
60         current_group = []
61         current_size = 0
62         target_size = 25 * 1024 * 1024 # 25 MB
63
64         for param in self.model.parameters():
65             param_size = param.numel() * param.element_size()
66
67             if current_size + param_size > target_size and current_group:
68                 groups.append(current_group)
69                 current_group = []
70                 current_size = 0
71
72             current_group.append(param)
73             current_size += param_size
74
75         if current_group:
76             groups.append(current_group)
77
78         return groups
79
80     def backward_with_overlap(self, loss):
81         """Backward with overlapped communication"""
82         loss.backward()
83
84         handles = []
85
86         # Communicate groups in reverse order (backward pass order)
87         for group in reversed(self.param_groups):
88             # Create bucket tensor
89             bucket = torch.cat([p.grad.flatten() for p in group if p.grad is not None
100
101         ]))
102
103         # Async all-reduce
104         handle = dist.all_reduce(bucket, async_op=True)
105         handles.append((handle, group, bucket))

```

```

94         # Wait and unpack
95         offset = 0
96         for handle, group, bucket in handles:
97             handle.wait()
98             bucket /= dist.get_world_size()
99
100
101         for param in group:
102             if param.grad is not None:
103                 numel = param.grad.numel()
104                 param.grad.copy_(bucket[offset:offset+numel].view_as(param.grad))
105                 offset += numel

```

Listing 9.20: Overlap communication with computation

9.13.3 Communication-Efficient Architectures

```

1 import torch.nn as nn
2
3 # Local vs Global layers
4 class HybridParallelTransformer(nn.Module):
5     """
6     Transformer with hybrid parallelism strategy:
7     - Lower layers: Data parallel (communication-heavy)
8     - Middle layers: Tensor parallel (balanced)
9     - Upper layers: Pipeline parallel (memory-efficient)
10    """
11
12    def __init__(self, config, strategy='hybrid'):
13        super().__init__()
14        self.strategy = strategy
15
16        # Lower layers (data parallel)
17        self.embedding = nn.Embedding(config.vocab_size, config.hidden_size)
18        self.lower_layers = nn.ModuleList([
19            TransformerLayer(config) for _ in range(config.num_layers // 3)
20        ])
21
22        # Middle layers (tensor parallel)
23        self.middle_layers = nn.ModuleList([
24            TensorParallelTransformerLayer(config)
25            for _ in range(config.num_layers // 3)
26        ])
27
28        # Upper layers (pipeline parallel)
29        self.upper_layers = nn.ModuleList([
30            TransformerLayer(config) for _ in range(config.num_layers // 3)
31        ])
32
33        self.lm_head = nn.Linear(config.hidden_size, config.vocab_size)
34
35    def forward(self, input_ids):
36        # Embedding (replicated)
37        x = self.embedding(input_ids)
38
39        # Lower layers (data parallel)
40        for layer in self.lower_layers:

```

```

41         x = layer(x)
42
43         # Middle layers (tensor parallel)
44         for layer in self.middle_layers:
45             x = layer(x)
46
47         # Upper layers (pipeline parallel)
48         for layer in self.upper_layers:
49             x = layer(x)
50
51         # Output head
52         logits = self.lm_head(x)
53
54         return logits
55
56     # Reduce communication with local layers
57 class LocalAttentionLayer(nn.Module):
58     """
59     Local attention that doesn't require cross-GPU communication.
60     Reduces communication overhead for long sequences.
61     """
62
63     def __init__(self, hidden_size, num_heads, window_size=512):
64         super().__init__()
65         self.window_size = window_size
66         self.attention = nn.MultiheadAttention(hidden_size, num_heads)
67
68     def forward(self, x):
69         # Only attend to local window
70         # No cross-GPU communication needed
71         batch_size, seq_len, hidden_size = x.shape
72
73         outputs = []
74         for i in range(0, seq_len, self.window_size):
75             window = x[:, i:i+self.window_size]
76             out, _ = self.attention(window, window, window)
77             outputs.append(out)
78
79         return torch.cat(outputs, dim=1)

```

Listing 9.21: Optimizing model architecture for distributed training

9.14 Debugging Distributed Training

Debugging distributed systems is challenging due to multiple processes and asynchronous communication. This section provides practical debugging strategies.

9.14.1 Common Issues and Solutions

```

1 import torch
2 import torch.distributed as dist
3 import os
4 import logging
5
6 class DistributedDebugger:

```

```

7  """Utilities for debugging distributed training"""
8
9  @staticmethod
10 def check_distributed_setup():
11     """Verify distributed training is configured correctly"""
12     print("=== Distributed Training Setup ===")
13     print(f"CUDA available: {torch.cuda.is_available()}")
14     print(f"CUDA device count: {torch.cuda.device_count()}")
15
16     if dist.is_initialized():
17         print(f"Distributed initialized: True")
18         print(f"World size: {dist.get_world_size()}")
19         print(f"Rank: {dist.get_rank()}")
20         print(f"Backend: {dist.get_backend()}")
21     else:
22         print("Distributed NOT initialized")
23
24     # Check environment variables
25     print("\n=== Environment Variables ===")
26     env_vars = ['MASTER_ADDR', 'MASTER_PORT', 'WORLD_SIZE', 'RANK', 'LOCAL_RANK']
27     for var in env_vars:
28         print(f"{var}: {os.environ.get(var, 'NOT SET')}")
29
30 @staticmethod
31 def verify_model_consistency():
32     """Check if all ranks have same model parameters"""
33     if not dist.is_initialized():
34         return True
35
36     model = get_model() # Your model
37     rank = dist.get_rank()
38
39     # Compute hash of all parameters
40     param_hash = 0
41     for param in model.parameters():
42         param_hash += param.data.sum().item()
43
44     # Gather hashes from all ranks
45     all_hashes = [torch.tensor(0.0).cuda() for _ in range(dist.get_world_size())]
46     dist.all_gather(all_hashes, torch.tensor(param_hash).cuda())
47
48     # Check if all hashes are same
49     hashes = [h.item() for h in all_hashes]
50     if len(set(hashes)) > 1:
51         print(f"WARNING: Model parameters not synchronized!")
52         print(f"Hashes by rank: {hashes}")
53         return False
54
55     if rank == 0:
56         print("Model parameters are synchronized across all ranks")
57     return True
58
59 @staticmethod
60 def check_gradient_synchronization(model):
61     """Verify gradients are synchronized correctly"""
62     if not dist.is_initialized():
63         return
64

```

```

65     rank = dist.get_rank()
66
67     # After backward pass, check if gradients match
68     for name, param in model.named_parameters():
69         if param.grad is not None:
70             grad_sum = param.grad.sum().item()
71
72             # Gather from all ranks
73             grad_tensor = torch.tensor(grad_sum).cuda()
74             all_grads = [torch.zeros_like(grad_tensor) for _ in range(dist.
get_world_size())]
75             dist.all_gather(all_grads, grad_tensor)
76
77             # Check consistency
78             grad_values = [g.item() for g in all_grads]
79             if len(set(grad_values)) > 1:
80                 print(f"Rank {rank}: Gradient mismatch for {name}")
81                 print(f"  Gradients: {grad_values}")
82
83     @staticmethod
84     def synchronize_and_print(message, rank=None):
85         """Print message from specific rank after synchronization"""
86         if dist.is_initialized():
87             dist.barrier() # Synchronize all processes
88
89         current_rank = dist.get_rank() if dist.is_initialized() else 0
90
91         if rank is None or current_rank == rank:
92             print(f"[Rank {current_rank}] {message}")
93
94         if dist.is_initialized():
95             dist.barrier()
96
97     # Debugging hang issues
98     def debug_hang_with_watchdog():
99         """Detect and report hanging processes"""
100         import threading
101         import time
102
103         class Watchdog:
104             def __init__(self, timeout=300): # 5 minutes
105                 self.timeout = timeout
106                 self.last_update = time.time()
107                 self.running = True
108
109             def update(self):
110                 self.last_update = time.time()
111
112             def watch(self):
113                 while self.running:
114                     time.sleep(10)
115                     elapsed = time.time() - self.last_update
116                     if elapsed > self.timeout:
117                         rank = dist.get_rank() if dist.is_initialized() else 0
118                         print(f"WATCHDOG: Rank {rank} appears to be hanging!")
119                         print(f"No progress for {elapsed:.1f} seconds")
120
121                 # Print stack trace

```

```

122         import traceback
123         import sys
124         traceback.print_stack(file=sys.stdout)
125
126     def stop(self):
127         self.running = False
128
129     watchdog = Watchdog(timeout=300)
130     thread = threading.Thread(target=watchdog.watch, daemon=True)
131     thread.start()
132
133     # Training loop
134     for epoch in range(num_epochs):
135         for batch in dataloader:
136             watchdog.update() # Signal progress
137
138             # Training code...
139             pass
140
141     watchdog.stop()
142
143     # Logging setup for distributed training
144     def setup_distributed_logging(log_dir="./logs"):
145         """Configure logging for distributed training"""
146         os.makedirs(log_dir, exist_ok=True)
147
148         rank = dist.get_rank() if dist.is_initialized() else 0
149
150         # Each rank logs to separate file
151         log_file = os.path.join(log_dir, f"rank_{rank}.log")
152
153         logging.basicConfig(
154             level=logging.INFO,
155             format=f'[Rank {rank}] %(asctime)s - %(levelname)s - %(message)s',
156             handlers=[
157                 logging.FileHandler(log_file),
158                 logging.StreamHandler() # Also print to console
159             ]
160         )
161
162     return logging.getLogger(__name__)

```

Listing 9.22: Debugging utilities for distributed training

9.14.2 Profiling Distributed Communication

```

1 import torch
2 import torch.distributed as dist
3 import time
4 from collections import defaultdict
5
6 class CommunicationProfiler:
7     """Profile communication operations in distributed training"""
8
9     def __init__(self):
10         self.stats = defaultdict(lambda: {
11             'count': 0,

```

```

12         'total_time': 0.0,
13         'total_bytes': 0
14     })
15
16     def profile_all_reduce(self, tensor, op=dist.ReduceOp.SUM):
17         """Profile all-reduce operation"""
18         bytes_transferred = tensor.element_size() * tensor.numel()
19
20         start = time.perf_counter()
21         dist.all_reduce(tensor, op=op)
22         torch.cuda.synchronize()
23         elapsed = time.perf_counter() - start
24
25         self.stats['all_reduce']['count'] += 1
26         self.stats['all_reduce']['total_time'] += elapsed
27         self.stats['all_reduce']['total_bytes'] += bytes_transferred
28
29         return tensor
30
31     def profile_broadcast(self, tensor, src=0):
32         """Profile broadcast operation"""
33         bytes_transferred = tensor.element_size() * tensor.numel()
34
35         start = time.perf_counter()
36         dist.broadcast(tensor, src=src)
37         torch.cuda.synchronize()
38         elapsed = time.perf_counter() - start
39
40         self.stats['broadcast']['count'] += 1
41         self.stats['broadcast']['total_time'] += elapsed
42         self.stats['broadcast']['total_bytes'] += bytes_transferred
43
44         return tensor
45
46     def print_report(self):
47         """Print profiling report"""
48         if dist.get_rank() == 0:
49             print("\n=== Communication Profiling Report ===")
50
51             for op_name, stats in self.stats.items():
52                 count = stats['count']
53                 total_time = stats['total_time']
54                 total_bytes = stats['total_bytes']
55
56                 avg_time = total_time / count if count > 0 else 0
57                 bandwidth = total_bytes / total_time / 1e9 if total_time > 0 else 0
58
59                 print(f"\n{op_name}:")
60                 print(f"    Count: {count}")
61                 print(f"    Total time: {total_time:.3f} s")
62                 print(f"    Avg time: {avg_time*1000:.2f} ms")
63                 print(f"    Total data: {total_bytes/1e9:.2f} GB")
64                 print(f"    Bandwidth: {bandwidth:.2f} GB/s")

```

Listing 9.23: Profile distributed communication patterns

9.15 Chapter Summary

This chapter covered distributed training strategies for scaling deep learning:

- **Data Parallelism:** Replicate model, distribute data (PyTorch DDP)
- **Model Parallelism:** Split model across GPUs for large models
- **Pipeline Parallelism:** Improve model parallelism with micro-batches
- **Tensor Parallelism:** Split individual layers across devices
- **Parameter Servers vs All-Reduce:** Architectural trade-offs
- **Gradient Synchronization:** Synchronous vs asynchronous strategies
- **Horovod:** Framework-agnostic distributed training
- **Ray:** Flexible distributed computing platform
- **Fault Tolerance:** Checkpointing and automatic restart
- **Memory Management:** Gradient checkpointing, ZeRO optimization, activation memory reduction
- **Communication Optimization:** Gradient compression, overlapping computation/communication
- **Performance Optimization:** Profiling, benchmarking, bottleneck resolution
- **Resource Management:** SLURM and Kubernetes scheduling

Key takeaways:

1. Choose parallelism strategy based on model and hardware constraints
2. Communication overhead limits scaling efficiency - use compression and overlapping
3. Memory management is critical for training large models - leverage ZeRO, checkpointing, and mixed precision
4. Fault tolerance is essential for multi-day training runs - implement robust checkpointing
5. Profile to identify bottlenecks before optimizing - measure both computation and communication
6. Mixed precision training significantly improves throughput and reduces memory usage
7. Gradient compression can reduce communication volume by 10-100x with minimal accuracy loss
8. Hierarchical all-reduce is more efficient for multi-node training than flat topologies

Practical Recommendations:

- **Small models (<1B params):** Use data parallelism with PyTorch DDP
- **Medium models (1-10B params):** Combine data parallelism with ZeRO Stage 2-3
- **Large models (>10B params):** Use full 3D parallelism (data + tensor + pipeline)
- **Limited bandwidth:** Enable gradient compression and increase batch size
- **Limited memory:** Use gradient checkpointing and ZeRO offloading
- **Multi-node training:** Profile communication and use hierarchical all-reduce

9.16 Exercises

Exercise 9.1. Implement Data Parallel Training

Implement a complete data parallel training pipeline:

1. Use PyTorch DDP to train ResNet-50 on ImageNet
2. Implement proper data loading with DistributedSampler
3. Add checkpointing every 10 epochs
4. Measure scaling efficiency on 1, 2, 4, 8 GPUs
5. Create plots showing throughput vs number of GPUs

Target: >90% scaling efficiency on 8 GPUs.

Exercise 9.2. Model Parallelism for Large Models

Implement model parallelism for a GPT-style transformer:

- Split 24-layer transformer across 4 GPUs
- Implement pipeline parallelism with 8 micro-batches
- Compare throughput with and without pipelining
- Measure GPU utilization and identify idle time

Document the pipeline bubble and propose optimizations.

Exercise 9.3. Gradient Synchronization Comparison

Compare synchronous and asynchronous SGD:

1. Implement both using PyTorch
2. Train on CIFAR-10 with 4 workers
3. Measure convergence speed and final accuracy
4. Plot learning curves for comparison

5. Analyze gradient staleness in async SGD

When does async SGD outperform sync SGD?

Exercise 9.4. Horovod Multi-Node Training

Set up Horovod training across multiple nodes:

- Configure SSH and MPI for multi-node communication
- Launch training on 2 nodes with 4 GPUs each
- Use Horovod timeline to profile communication
- Identify communication bottlenecks
- Optimize with gradient compression or hierarchical all-reduce

Measure bandwidth utilization and reduce communication time by 20%.

Exercise 9.5. Fault-Tolerant Training

Build a fault-tolerant training system:

1. Implement elastic checkpointing every 5 minutes
2. Simulate node failures during training
3. Verify automatic recovery from last checkpoint
4. Measure overhead of checkpointing
5. Implement checkpoint compression to reduce I/O

Target: <2% training time overhead for checkpointing.

Exercise 9.6. Performance Profiling and Optimization

Profile and optimize a distributed training job:

- Use PyTorch Profiler to identify bottlenecks
- Measure time spent in computation vs communication
- Optimize data loading with prefetching and pinned memory
- Implement mixed precision training
- Compare before/after throughput

Target: 2x throughput improvement through optimizations.

Exercise 9.7. Scaling Law Analysis

Analyze scaling behavior empirically:

1. Train the same model on 1, 2, 4, 8, 16 GPUs
2. Measure throughput (samples/sec) for each configuration
3. Compute scaling efficiency

4. Plot strong scaling curve
5. Identify where scaling breaks down and explain why

Fit a model to predict throughput for arbitrary GPU count.

Exercise 9.8. Resource Allocation Strategy

Design an optimal resource allocation strategy:

- Given a cluster with heterogeneous GPUs (V100, A100)
- Train 5 models with different resource requirements
- Implement scheduling algorithm to maximize throughput
- Handle job preemption and migration
- Compare with naive FIFO scheduling

Measure cluster utilization and job completion time.

Chapter 10

Hyperparameter Optimization

10.1 Learning Objectives

After completing this chapter, readers will be able to:

- Implement Bayesian optimization using Optuna with advanced pruning and sampling strategies
- Design and execute multi-objective hyperparameter optimization
- Build population-based training systems for dynamic hyperparameter adaptation
- Deploy distributed hyperparameter optimization across clusters
- Apply AutoML techniques and neural architecture search
- Integrate hyperparameter optimization with experiment tracking systems
- Design cost-efficient HPO strategies for production environments
- Analyze and optimize hyperparameter search spaces

10.2 Introduction: The Hyperparameter Challenge

Hyperparameter optimization (HPO) is critical for achieving state-of-the-art model performance, yet remains one of the most computationally expensive aspects of machine learning workflows. Consider a transformer model with hyperparameters for:

- Architecture: number of layers, hidden dimension, attention heads
- Training: learning rate, batch size, warmup steps, weight decay
- Optimization: optimizer type, scheduler, gradient clipping
- Regularization: dropout rate, layer normalization, label smoothing

With 10 hyperparameters and 5 values each, naive grid search requires $5^{10} = 9.7$ million trials. At 1 hour per trial, this takes 1,100 years of compute time! Modern HPO addresses this through:

1. **Efficient Search:** Bayesian optimization, evolutionary algorithms
2. **Early Stopping:** Pruning unpromising trials
3. **Transfer Learning:** Warm-starting from previous experiments
4. **Parallelization:** Distributed evaluation across GPUs/nodes
5. **Multi-Fidelity:** Training with reduced budgets (fewer epochs, smaller data)

10.3 Bayesian Optimization with Optuna

Bayesian optimization models the objective function as a probabilistic surrogate and uses this to guide search toward promising regions.

10.3.1 Fundamental Optuna Implementation

```

1 import optuna
2 from optuna.pruners import MedianPruner, HyperbandPruner
3 from optuna.samplers import TPESampler, CmaEsSampler
4 import torch
5 import torch.nn as nn
6 from torch.utils.data import DataLoader
7 import numpy as np
8 from typing import Dict, Any
9
10 def create_model(trial: optuna.Trial, input_dim: int, output_dim: int) -> nn.Module:
11     """
12     Create model with trial-suggested hyperparameters.
13
14     Demonstrates nested hyperparameter spaces and conditional sampling.
15     """
16     # Architecture hyperparameters
17     n_layers = trial.suggest_int('n_layers', 2, 6)
18
19     # Hidden dimension with categorical choices
20     hidden_dim = trial.suggest_categorical(
21         'hidden_dim',
22         [64, 128, 256, 512, 1024]
23     )
24
25     # Activation function
26     activation_name = trial.suggest_categorical(
27         'activation',
28         ['relu', 'gelu', 'silu', 'tanh']
29     )
30     activation_map = {
31         'relu': nn.ReLU,
32         'gelu': nn.GELU,
33         'silu': nn.SiLU,
34         'tanh': nn.Tanh
35     }
36     activation = activation_map[activation_name]
37
38     # Dropout rate with log-uniform distribution

```

```

39 dropout_rate = trial.suggest_float('dropout', 0.0, 0.5)
40
41 # Normalization type
42 use_batch_norm = trial.suggest_categorical('use_batch_norm', [True, False])
43
44 # Build model
45 layers = []
46 current_dim = input_dim
47
48 for i in range(n_layers):
49     layers.append(nn.Linear(current_dim, hidden_dim))
50
51     if use_batch_norm:
52         layers.append(nn.BatchNorm1d(hidden_dim))
53
54     layers.append(activation())
55
56     if dropout_rate > 0:
57         layers.append(nn.Dropout(dropout_rate))
58
59     current_dim = hidden_dim
60
61 # Output layer
62 layers.append(nn.Linear(current_dim, output_dim))
63
64 return nn.Sequential(*layers)
65
66 def create_optimizer(trial: optuna.Trial, model: nn.Module) -> torch.optim.Optimizer:
67     """Create optimizer with trial-suggested hyperparameters."""
68     # Optimizer type
69     optimizer_name = trial.suggest_categorical(
70         'optimizer',
71         ['adam', 'adamw', 'sgd', 'rmsprop']
72     )
73
74     # Learning rate with log-uniform sampling (recommended for LR)
75     lr = trial.suggest_float('lr', 1e-5, 1e-1, log=True)
76
77     # Weight decay
78     weight_decay = trial.suggest_float('weight_decay', 1e-6, 1e-2, log=True)
79
80     if optimizer_name == 'adam':
81         optimizer = torch.optim.Adam(
82             model.parameters(),
83             lr=lr,
84             weight_decay=weight_decay,
85             betas=(
86                 trial.suggest_float('adam_beta1', 0.8, 0.99),
87                 trial.suggest_float('adam_beta2', 0.9, 0.999)
88             )
89         )
90     elif optimizer_name == 'adamw':
91         optimizer = torch.optim.AdamW(
92             model.parameters(),
93             lr=lr,
94             weight_decay=weight_decay
95         )
96     elif optimizer_name == 'sgd':

```

```

97     momentum = trial.suggest_float('sgd_momentum', 0.0, 0.99)
98     optimizer = torch.optim.SGD(
99         model.parameters(),
100         lr=lr,
101         momentum=momentum,
102         weight_decay=weight_decay
103     )
104 else: # rmsprop
105     optimizer = torch.optim.RMSprop(
106         model.parameters(),
107         lr=lr,
108         weight_decay=weight_decay,
109         alpha=trial.suggest_float('rmsprop_alpha', 0.9, 0.999)
110     )
111
112     return optimizer
113
114 def create_scheduler(trial: optuna.Trial, optimizer: torch.optim.Optimizer,
115                     num_epochs: int) -> Any:
116     """Create learning rate scheduler."""
117     scheduler_name = trial.suggest_categorical(
118         'scheduler',
119         ['none', 'cosine', 'step', 'exponential']
120     )
121
122     if scheduler_name == 'none':
123         return None
124     elif scheduler_name == 'cosine':
125         return torch.optim.lr_scheduler.CosineAnnealingLR(
126             optimizer,
127             T_max=num_epochs,
128             eta_min=trial.suggest_float('cosine_eta_min', 1e-7, 1e-4, log=True)
129         )
130     elif scheduler_name == 'step':
131         step_size = trial.suggest_int('step_size', 10, 50)
132         gamma = trial.suggest_float('step_gamma', 0.1, 0.9)
133         return torch.optim.lr_scheduler.StepLR(
134             optimizer,
135             step_size=step_size,
136             gamma=gamma
137         )
138     else: # exponential
139         gamma = trial.suggest_float('exp_gamma', 0.9, 0.99)
140         return torch.optim.lr_scheduler.ExponentialLR(optimizer, gamma=gamma)
141
142 def objective(trial: optuna.Trial) -> float:
143     """
144     Objective function for Optuna optimization.
145
146     Returns validation metric to minimize (e.g., loss) or maximize (e.g., accuracy).
147     Optuna will automatically handle maximization if direction='maximize' in study.
148     """
149     # Set device
150     device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
151
152     # Hyperparameters
153     batch_size = trial.suggest_categorical('batch_size', [32, 64, 128, 256])
154     num_epochs = 50 # Fixed for this example

```



```

155     # Create model
156     model = create_model(trial, input_dim=784, output_dim=10).to(device)
157
158     # Create optimizer and scheduler
159     optimizer = create_optimizer(trial, model)
160     scheduler = create_scheduler(trial, optimizer, num_epochs)
161
162     # Load data (example with MNIST)
163     train_loader = get_train_loader(batch_size)
164     val_loader = get_val_loader(batch_size)
165
166     # Loss function
167     criterion = nn.CrossEntropyLoss()
168
169     # Training loop with intermediate reporting for pruning
170     for epoch in range(num_epochs):
171         # Train
172         model.train()
173         train_loss = 0.0
174
175         for batch_idx, (data, target) in enumerate(train_loader):
176             data, target = data.to(device), target.to(device)
177
178             optimizer.zero_grad()
179             output = model(data.view(data.size(0), -1))
180             loss = criterion(output, target)
181             loss.backward()
182
183             # Gradient clipping (suggested by trial)
184             max_grad_norm = trial.suggest_float('max_grad_norm', 0.1, 10.0)
185             torch.nn.utils.clip_grad_norm_(model.parameters(), max_grad_norm)
186
187             optimizer.step()
188             train_loss += loss.item()
189
190         # Validation
191         model.eval()
192         val_loss = 0.0
193         correct = 0
194         total = 0
195
196         with torch.no_grad():
197             for data, target in val_loader:
198                 data, target = data.to(device), target.to(device)
199                 output = model(data.view(data.size(0), -1))
200                 loss = criterion(output, target)
201                 val_loss += loss.item()
202
203                 _, predicted = output.max(1)
204                 total += target.size(0)
205                 correct += predicted.eq(target).sum().item()
206
207         val_accuracy = correct / total
208         avg_val_loss = val_loss / len(val_loader)
209
210         # Report intermediate value for pruning
211         trial.report(val_accuracy, epoch)

```

```

213
214     # Handle pruning based on intermediate value
215     if trial.should_prune():
216         raise optuna.TrialPruned()
217
218     # Update learning rate
219     if scheduler is not None:
220         scheduler.step()
221
222     # Return final validation accuracy
223     return val_accuracy
224
225 # Run optimization study
226 def run_optuna_study():
227     """
228     Run comprehensive Optuna study with advanced features.
229     """
230     # Create study with advanced sampler and pruner
231     study = optuna.create_study(
232         study_name='neural_network_optimization',
233         direction='maximize', # Maximize accuracy
234
235         # Advanced TPE sampler with multivariate search
236         sampler=TPESampler(
237             n_startup_trials=10, # Random trials before TPE
238             n_ei_candidates=24, # Number of candidates for EI
239             multivariate=True, # Enable multivariate TPE
240             seed=42
241         ),
242
243         # Hyperband pruner for aggressive early stopping
244         pruner=HyperbandPruner(
245             min_resource=1, # Minimum epochs
246             max_resource=50, # Maximum epochs
247             reduction_factor=3 # Reduction factor for Hyperband
248         )
249     )
250
251     # Optimize
252     study.optimize(
253         objective,
254         n_trials=100,
255         timeout=3600 * 8, # 8 hours
256         n_jobs=4, # Parallel trials (if using SQLite storage)
257         show_progress_bar=True
258     )
259
260     # Results
261     print("Best trial:")
262     trial = study.best_trial
263     print(f" Value (accuracy): {trial.value:.4f}")
264     print(" Params:")
265     for key, value in trial.params.items():
266         print(f" {key}: {value}")
267
268     # Visualizations
269     try:
270         import optuna.visualization as vis

```

```

271     # Optimization history
272     fig = vis.plot_optimization_history(study)
273     fig.write_html('optimization_history.html')
274
275     # Parameter importances
276     fig = vis.plot_param_importances(study)
277     fig.write_html('param_importances.html')
278
279     # Parallel coordinate plot
280     fig = vis.plot_parallel_coordinate(study)
281     fig.write_html('parallel_coordinate.html')
282
283     # Hyperparameter relationships
284     fig = vis.plot_contour(study, params=['lr', 'hidden_dim'])
285     fig.write_html('param_contour.html')
286
287
288     except ImportError:
289         print("Install plotly for visualizations: pip install plotly")
290
291     return study
292
293 if __name__ == '__main__':
294     study = run_optuna_study()

```

Listing 10.1: Advanced Optuna hyperparameter optimization

10.3.2 Advanced Optuna Features

```

1 import optuna
2 from optuna.study import MaxTrialsCallback
3 from optuna.trial import TrialState
4 import logging
5
6 class CustomCallback:
7     """Custom callback for Optuna trials."""
8
9     def __init__(self, patience: int = 10):
10         self.patience = patience
11         self.best_value = None
12         self.no_improvement_count = 0
13
14     def __call__(self, study: optuna.Study, trial: optuna.Trial) -> None:
15         """Called after each trial."""
16         # Early stopping based on no improvement
17         if study.best_value is not None:
18             if self.best_value is None or study.best_value > self.best_value:
19                 self.best_value = study.best_value
20                 self.no_improvement_count = 0
21             else:
22                 self.no_improvement_count += 1
23
24         if self.no_improvement_count >= self.patience:
25             study.stop()
26             print(f"Stopping study: no improvement for {self.patience} trials")
27
28         # Log trial results

```

```

29     if trial.state == TrialState.COMPLETE:
30         print(f"Trial {trial.number} finished with value: {trial.value:.4f}")
31     elif trial.state == TrialState.PRUNED:
32         print(f"Trial {trial.number} pruned")
33
34 def conditional_search_space(trial: optuna.Trial) -> Dict[str, Any]:
35     """
36     Define conditional search space where some hyperparameters
37     depend on others.
38     """
39     config = {}
40
41     # Model architecture choice
42     model_type = trial.suggest_categorical('model_type', ['mlp', 'resnet', '
transformer'])
43     config['model_type'] = model_type
44
45     if model_type == 'mlp':
46         config['n_layers'] = trial.suggest_int('mlp_n_layers', 2, 8)
47         config['hidden_dim'] = trial.suggest_int('mlp_hidden_dim', 64, 1024, log=True)
48         config['activation'] = trial.suggest_categorical('mlp_activation', ['relu', '
gelu'])
49
50     elif model_type == 'resnet':
51         config['n_blocks'] = trial.suggest_int('resnet_n_blocks', 2, 10)
52         config['block_type'] = trial.suggest_categorical('resnet_block_type', ['basic'
, 'bottleneck'])
53
54         if config['block_type'] == 'bottleneck':
55             config['bottleneck_ratio'] = trial.suggest_float('resnet_bottleneck_ratio'
, 0.25, 0.75)
56
57     elif model_type == 'transformer':
58         config['n_layers'] = trial.suggest_int('transformer_n_layers', 2, 12)
59         config['n_heads'] = trial.suggest_categorical('transformer_n_heads', [4, 8,
16])
60         config['d_model'] = trial.suggest_categorical('transformer_d_model', [256,
512, 1024])
61
62         # Ensure d_model is divisible by n_heads
63         if config['d_model'] % config['n_heads'] != 0:
64             # Adjust d_model
65             config['d_model'] = config['d_model'] // config['n_heads'] * config['
n_heads']
66
67         config['dropout'] = trial.suggest_float('transformer_dropout', 0.0, 0.3)
68
69     # Common hyperparameters
70     config['lr'] = trial.suggest_float('lr', 1e-5, 1e-1, log=True)
71     config['batch_size'] = trial.suggest_categorical('batch_size', [16, 32, 64, 128])
72
73     # Data augmentation (conditional on batch size)
74     if config['batch_size'] >= 64:
75         # Stronger augmentation for larger batches
76         config['augment_strength'] = trial.suggest_float('augment_strength', 0.5, 1.0)
77     else:
78         config['augment_strength'] = trial.suggest_float('augment_strength', 0.0, 0.5)
79

```

```

80     return config
81
82     # Use with persistent storage for distributed optimization
83     def create_distributed_study():
84         """
85         Create study with SQL storage for distributed optimization.
86         """
87         import optuna
88
89         # Use PostgreSQL, MySQL, or SQLite for storage
90         storage = optuna.storages.RDBStorage(
91             url="postgresql://user:password@localhost/optuna",
92             engine_kwargs={"pool_size": 20, "max_overflow": 0}
93         )
94
95         study = optuna.create_study(
96             study_name='distributed_hpo',
97             storage=storage,
98             load_if_exists=True, # Multiple workers can use same study
99             direction='maximize',
100             sampler=TPESampler(multivariate=True),
101             pruner=HyperbandPruner()
102         )
103
104     return study

```

Listing 10.2: Optuna with conditional search spaces and callbacks

10.4 Multi-Objective Optimization

Many real-world scenarios require optimizing multiple objectives simultaneously (e.g., accuracy vs latency, performance vs model size).

```

1  import optuna
2  from optuna.samplers import NSGAIISampler
3  import torch
4  import time
5
6  def multi_objective_function(trial: optuna.Trial) -> tuple:
7      """
8      Optimize multiple objectives: accuracy, inference latency, model size.
9
10     Returns:
11         Tuple of (accuracy, -latency, -model_size) where we want to:
12         - Maximize accuracy
13         - Minimize latency (hence -latency to maximize)
14         - Minimize model size (hence -model_size to maximize)
15     """
16     # Hyperparameters
17     n_layers = trial.suggest_int('n_layers', 1, 10)
18     hidden_dim = trial.suggest_categorical('hidden_dim', [64, 128, 256, 512, 1024])
19     dropout = trial.suggest_float('dropout', 0.0, 0.5)
20
21     # Build model
22     model = create_model_from_params(n_layers, hidden_dim, dropout)
23     device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

```

```

24 model = model.to(device)
25
26 # Train model (simplified)
27 accuracy = train_and_evaluate(model, epochs=10)
28
29 # Measure inference latency
30 model.eval()
31 dummy_input = torch.randn(1, 784).to(device)
32
33 # Warmup
34 for _ in range(10):
35     _ = model(dummy_input)
36
37 # Measure
38 torch.cuda.synchronize() if torch.cuda.is_available() else None
39 start = time.perf_counter()
40 for _ in range(100):
41     _ = model(dummy_input)
42 torch.cuda.synchronize() if torch.cuda.is_available() else None
43 end = time.perf_counter()
44
45 avg_latency = (end - start) / 100 * 1000 # ms
46
47 # Calculate model size (MB)
48 param_size = sum(p.numel() * p.element_size() for p in model.parameters())
49 buffer_size = sum(b.numel() * b.element_size() for b in model.buffers())
50 model_size_mb = (param_size + buffer_size) / (1024 ** 2)
51
52 # Return objectives (all to be maximized, so negate latency and size)
53 return accuracy, -avg_latency, -model_size_mb
54
55 def run_multi_objective_optimization():
56     """
57     Run multi-objective optimization using NSGA-II algorithm.
58     """
59     # Create study for multi-objective optimization
60     study = optuna.create_study(
61         directions=['maximize', 'maximize', 'maximize'], # All maximize
62         sampler=NSGAIISampler(
63             population_size=50,
64             mutation_prob=0.1,
65             crossover_prob=0.9,
66             seed=42
67         )
68     )
69
70     # Optimize
71     study.optimize(
72         multi_objective_function,
73         n_trials=200,
74         show_progress_bar=True
75     )
76
77     # Get Pareto front
78     print("Number of trials on Pareto front:", len(study.best_trials))
79     print("\nPareto-optimal solutions:")
80
81     for trial in study.best_trials:

```

```

82     accuracy, neg_latency, neg_size = trial.values
83     print(f"Trial {trial.number}:")
84     print(f"  Accuracy: {accuracy:.4f}")
85     print(f"  Latency: {-neg_latency:.2f} ms")
86     print(f"  Model size: {-neg_size:.2f} MB")
87     print(f"  Params: {trial.params}")
88     print()
89
90     # Visualize Pareto front
91     try:
92         import optuna.visualization as vis
93
94         # 3D Pareto front
95         fig = vis.plot_pareto_front(
96             study,
97             target_names=['Accuracy', 'Latency (ms)', 'Model Size (MB)']
98         )
99         fig.write_html('pareto_front.html')
100
101     except ImportError:
102         print("Install plotly for visualizations")
103
104     return study
105
106 # Trade-off analysis
107 def select_model_by_constraint(study: optuna.Study,
108                               max_latency_ms: float = 10.0,
109                               max_size_mb: float = 50.0) -> optuna.Trial:
110     """
111     Select best model from Pareto front that meets constraints.
112     """
113     valid_trials = []
114
115     for trial in study.best_trials:
116         accuracy, neg_latency, neg_size = trial.values
117         latency = -neg_latency
118         size = -neg_size
119
120         if latency <= max_latency_ms and size <= max_size_mb:
121             valid_trials.append((accuracy, trial))
122
123     if not valid_trials:
124         raise ValueError("No trials meet the constraints")
125
126     # Return trial with best accuracy among valid trials
127     best_trial = max(valid_trials, key=lambda x: x[0])[1]
128
129     print(f"Selected trial {best_trial.number}:")
130     accuracy, neg_latency, neg_size = best_trial.values
131     print(f"  Accuracy: {accuracy:.4f}")
132     print(f"  Latency: {-neg_latency:.2f} ms (limit: {max_latency_ms})")
133     print(f"  Size: {-neg_size:.2f} MB (limit: {max_size_mb})")
134
135     return best_trial

```

Listing 10.3: Multi-objective optimization with Optuna

10.5 Population-Based Training (PBT)

PBT dynamically adapts hyperparameters during training by exploiting and exploring a population of models.

```

1 import torch
2 import torch.nn as nn
3 import numpy as np
4 from typing import List, Dict, Any
5 from dataclasses import dataclass
6 import copy
7 import random
8
9 @dataclass
10 class Individual:
11     """Represents one member of the population."""
12     model: nn.Module
13     optimizer: torch.optim.Optimizer
14     hyperparams: Dict[str, Any]
15     score: float = 0.0
16     steps_trained: int = 0
17     age: int = 0
18
19 class PopulationBasedTraining:
20     """
21     Population-Based Training (PBT) for dynamic hyperparameter optimization.
22
23     Based on "Population Based Training of Neural Networks" (Jaderberg et al., 2017)
24     """
25
26     def __init__(
27         self,
28         population_size: int = 10,
29         exploit_threshold: float = 0.2, # Top 20% exploit bottom 20%
30         explore_factor: float = 0.8,   # Perturb hyperparams by +/- 20%
31         eval_interval: int = 100,      # Steps between evaluations
32         device: str = 'cuda'
33     ):
34         self.population_size = population_size
35         self.exploit_threshold = exploit_threshold
36         self.explore_factor = explore_factor
37         self.eval_interval = eval_interval
38         self.device = device
39
40         self.population: List[Individual] = []
41         self.history = []
42
43     def initialize_population(self, model_fn, hyperparam_space: Dict) -> None:
44         """
45         Initialize population with random hyperparameters.
46
47         Args:
48             model_fn: Function that creates a model
49             hyperparam_space: Dict defining hyperparameter ranges
50         """
51         for _ in range(self.population_size):
52             # Sample hyperparameters
53             hyperparams = self._sample_hyperparams(hyperparam_space)

```



```

54         # Create model and optimizer
55         model = model_fn().to(self.device)
56         optimizer = self._create_optimizer(model, hyperparams)
57
58         individual = Individual(
59             model=model,
60             optimizer=optimizer,
61             hyperparams=hyperparams,
62             score=0.0
63         )
64
65         self.population.append(individual)
66
67     def _sample_hyperparams(self, space: Dict) -> Dict[str, Any]:
68         """Sample hyperparameters from defined space."""
69         params = {}
70
71         for name, config in space.items():
72             if config['type'] == 'float':
73                 if config.get('log', False):
74                     params[name] = np.exp(np.random.uniform(
75                         np.log(config['min']),
76                         np.log(config['max'])
77                     ))
78                 else:
79                     params[name] = np.random.uniform(
80                         config['min'],
81                         config['max']
82                     )
83             elif config['type'] == 'int':
84                 params[name] = np.random.randint(
85                     config['min'],
86                     config['max'] + 1
87                 )
88             elif config['type'] == 'choice':
89                 params[name] = np.random.choice(config['choices'])
90
91         return params
92
93     def _create_optimizer(self, model: nn.Module, hyperparams: Dict) -> torch.optim.
94     Optimizer:
95         """Create optimizer with given hyperparameters."""
96         return torch.optim.Adam(
97             model.parameters(),
98             lr=hyperparams['learning_rate'],
99             weight_decay=hyperparams.get('weight_decay', 0.0)
100         )
101
102     def exploit_and_explore(self, individual: Individual,
103                             population: List[Individual]) -> None:
104         """
105         Exploit: Copy weights from better performer.
106         Explore: Perturb hyperparameters.
107         """
108         # Sort population by score
109         sorted_pop = sorted(population, key=lambda x: x.score, reverse=True)
110

```

```

111     # Determine if this individual should be replaced
112     cutoff = int(self.population_size * self.exploit_threshold)
113     bottom_performers = sorted_pop[-cutoff:]
114
115     if individual in bottom_performers:
116         # EXPLOIT: Copy from top performer
117         top_performers = sorted_pop[:cutoff]
118         champion = random.choice(top_performers)
119
120         # Copy model weights
121         individual.model.load_state_dict(champion.model.state_dict())
122
123         # EXPLORE: Perturb hyperparameters
124         for key, value in champion.hyperparams.items():
125             if isinstance(value, float):
126                 # Perturb by +/- explore_factor
127                 factor = np.random.choice([
128                     1.0 - self.explore_factor,
129                     1.0 + self.explore_factor
130                 ])
131                 individual.hyperparams[key] = value * factor
132             elif isinstance(value, int):
133                 delta = max(1, int(value * self.explore_factor))
134                 individual.hyperparams[key] = value + random.choice([-delta, delta])
135
136         # Recreate optimizer with new hyperparameters
137         individual.optimizer = self._create_optimizer(
138             individual.model,
139             individual.hyperparams
140         )
141
142         individual.age += 1
143
144     def train_step(self, individual: Individual, batch) -> float:
145         """Single training step for an individual."""
146         data, target = batch
147         data, target = data.to(self.device), target.to(self.device)
148
149         individual.model.train()
150         individual.optimizer.zero_grad()
151
152         output = individual.model(data)
153         loss = nn.CrossEntropyLoss()(output, target)
154
155         loss.backward()
156         individual.optimizer.step()
157
158         individual.steps_trained += 1
159
160         return loss.item()
161
162     def evaluate(self, individual: Individual, val_loader) -> float:
163         """Evaluate individual on validation set."""
164         individual.model.eval()
165         correct = 0
166         total = 0
167

```

```

168     with torch.no_grad():
169         for data, target in val_loader:
170             data, target = data.to(self.device), target.to(self.device)
171             output = individual.model(data)
172             _, predicted = output.max(1)
173             total += target.size(0)
174             correct += predicted.eq(target).sum().item()
175
176     accuracy = correct / total
177     individual.score = accuracy
178
179     return accuracy
180
181 def train(self, train_loader, val_loader, num_steps: int):
182     """
183     Main PBT training loop.
184
185     Args:
186         train_loader: Training data loader
187         val_loader: Validation data loader
188         num_steps: Total training steps
189     """
190     train_iterator = iter(train_loader)
191
192     for step in range(num_steps):
193         # Train each individual
194         for individual in self.population:
195             try:
196                 batch = next(train_iterator)
197             except StopIteration:
198                 train_iterator = iter(train_loader)
199                 batch = next(train_iterator)
200
201             loss = self.train_step(individual, batch)
202
203         # Periodic evaluation and exploitation/exploration
204         if (step + 1) % self.eval_interval == 0:
205             print(f"\nStep {step + 1}/{num_steps}")
206
207             # Evaluate all individuals
208             scores = []
209             for i, individual in enumerate(self.population):
210                 score = self.evaluate(individual, val_loader)
211                 scores.append(score)
212
213             print(f"  Individual {i}: "
214                   f"Score={score:.4f}, "
215                   f"LR={individual.hyperparams['learning_rate']:.6f}")
216
217             # Record best score
218             best_score = max(scores)
219             avg_score = np.mean(scores)
220
221             self.history.append({
222                 'step': step + 1,
223                 'best_score': best_score,
224                 'avg_score': avg_score,
225                 'scores': scores.copy()

```

```

226         })
227
228         # Exploit and explore
229         for individual in self.population:
230             self.exploit_and_explore(individual, self.population)
231
232         # Return best individual
233         best_individual = max(self.population, key=lambda x: x.score)
234         return best_individual
235
236     def get_best_hyperparams(self) -> Dict[str, Any]:
237         """Get hyperparameters of best individual."""
238         best = max(self.population, key=lambda x: x.score)
239         return best.hyperparams
240
241 # Usage example
242 def run_pbt():
243     """Run population-based training."""
244     # Define hyperparameter space
245     hyperparam_space = {
246         'learning_rate': {'type': 'float', 'min': 1e-5, 'max': 1e-1, 'log': True},
247         'weight_decay': {'type': 'float', 'min': 1e-6, 'max': 1e-2, 'log': True},
248         'dropout': {'type': 'float', 'min': 0.0, 'max': 0.5},
249     }
250
251     # Create PBT instance
252     pbt = PopulationBasedTraining(
253         population_size=10,
254         exploit_threshold=0.2,
255         eval_interval=100
256     )
257
258     # Initialize population
259     pbt.initialize_population(
260         model_fn=lambda: SimpleNet(),
261         hyperparam_space=hyperparam_space
262     )
263
264     # Train
265     train_loader = get_train_loader()
266     val_loader = get_val_loader()
267
268     best_individual = pbt.train(
269         train_loader=train_loader,
270         val_loader=val_loader,
271         num_steps=10000
272     )
273
274     print("\nBest hyperparameters found:")
275     print(best_individual.hyperparams)
276     print(f"Best score: {best_individual.score:.4f}")
277
278     # Plot learning curves
279     import matplotlib.pyplot as plt
280
281     steps = [h['step'] for h in pbt.history]
282     best_scores = [h['best_score'] for h in pbt.history]
283     avg_scores = [h['avg_score'] for h in pbt.history]

```

```

284 plt.figure(figsize=(10, 6))
285 plt.plot(steps, best_scores, label='Best in Population')
286 plt.plot(steps, avg_scores, label='Population Average')
287 plt.xlabel('Training Steps')
288 plt.ylabel('Validation Accuracy')
289 plt.legend()
290 plt.title('Population-Based Training Progress')
291 plt.savefig('pbt_progress.png')
292
293
294 return best_individual

```

Listing 10.4: Population-based training implementation

10.6 Distributed Hyperparameter Optimization

Scale HPO across multiple GPUs and nodes for faster optimization.

```

1 import ray
2 from ray import tune
3 from ray.tune import CLIReporter
4 from ray.tune.schedulers import ASHAScheduler, PopulationBasedTraining as RayPBT
5 from ray.tune.search.optuna import OptunaSearch
6 from ray.tune.search import ConcurrencyLimiter
7 import torch
8 import torch.nn as nn
9
10 def train_model_tune(config: Dict, checkpoint_dir=None):
11     """
12     Training function for Ray Tune.
13
14     Args:
15         config: Hyperparameter configuration from Tune
16         checkpoint_dir: Directory for loading checkpoints
17     """
18     device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
19
20     # Create model with config
21     model = create_model(
22         n_layers=config['n_layers'],
23         hidden_dim=config['hidden_dim'],
24         dropout=config['dropout']
25     ).to(device)
26
27     # Optimizer
28     optimizer = torch.optim.Adam(
29         model.parameters(),
30         lr=config['lr'],
31         weight_decay=config['weight_decay']
32     )
33
34     # Load checkpoint if resuming
35     start_epoch = 0
36     if checkpoint_dir:
37         checkpoint = torch.load(f"{checkpoint_dir}/checkpoint")
38         model.load_state_dict(checkpoint['model_state'])
39         optimizer.load_state_dict(checkpoint['optimizer_state'])

```

```

40     start_epoch = checkpoint['epoch'] + 1
41
42     # Data loaders
43     train_loader = get_train_loader(config['batch_size'])
44     val_loader = get_val_loader(config['batch_size'])
45
46     criterion = nn.CrossEntropyLoss()
47
48     # Training loop
49     for epoch in range(start_epoch, config['num_epochs']):
50         # Train
51         model.train()
52         train_loss = 0.0
53
54         for batch_idx, (data, target) in enumerate(train_loader):
55             data, target = data.to(device), target.to(device)
56
57             optimizer.zero_grad()
58             output = model(data)
59             loss = criterion(output, target)
60             loss.backward()
61             optimizer.step()
62
63             train_loss += loss.item()
64
65         # Validate
66         model.eval()
67         val_loss = 0.0
68         correct = 0
69         total = 0
70
71         with torch.no_grad():
72             for data, target in val_loader:
73                 data, target = data.to(device), target.to(device)
74                 output = model(data)
75                 loss = criterion(output, target)
76                 val_loss += loss.item()
77
78                 _, predicted = output.max(1)
79                 total += target.size(0)
80                 correct += predicted.eq(target).sum().item()
81
82         val_accuracy = correct / total
83         avg_val_loss = val_loss / len(val_loader)
84
85         # Report metrics to Tune
86         tune.report(
87             val_loss=avg_val_loss,
88             val_accuracy=val_accuracy,
89             train_loss=train_loss / len(train_loader)
90         )
91
92         # Save checkpoint
93         with tune.checkpoint_dir(epoch) as checkpoint_dir:
94             path = f"{checkpoint_dir}/checkpoint"
95             torch.save({
96                 'epoch': epoch,
97                 'model_state': model.state_dict(),

```

```

98         'optimizer_state': optimizer.state_dict(),
99     }, path)
100
101 def run_ray_tune_distributed():
102     """
103     Run distributed hyperparameter optimization with Ray Tune.
104     """
105     # Initialize Ray
106     ray.init(num_gpus=4, num_cpus=16)
107
108     # Define search space
109     config = {
110         'n_layers': tune.randint(2, 8),
111         'hidden_dim': tune.choice([64, 128, 256, 512, 1024]),
112         'dropout': tune.uniform(0.0, 0.5),
113         'lr': tune.loguniform(1e-5, 1e-1),
114         'weight_decay': tune.loguniform(1e-6, 1e-2),
115         'batch_size': tune.choice([32, 64, 128, 256]),
116         'num_epochs': 50
117     }
118
119     # ASHA scheduler for early stopping
120     scheduler = ASHAScheduler(
121         metric='val_accuracy',
122         mode='max',
123         max_t=50,           # Maximum epochs
124         grace_period=5,     # Minimum epochs before stopping
125         reduction_factor=3  # Halve trials every reduction_factor epochs
126     )
127
128     # Optuna search algorithm
129     search_alg = OptunaSearch(
130         metric='val_accuracy',
131         mode='max'
132     )
133
134     # Limit concurrent trials
135     search_alg = ConcurrencyLimiter(search_alg, max_concurrent=4)
136
137     # Reporter for nice CLI output
138     reporter = CLIReporter(
139         metric_columns=['val_loss', 'val_accuracy', 'train_loss'],
140         max_report_frequency=30
141     )
142
143     # Run optimization
144     analysis = tune.run(
145         train_model_tune,
146         name='distributed_hpo',
147         config=config,
148         num_samples=100,      # Number of trials
149         scheduler=scheduler,
150         search_alg=search_alg,
151         progress_reporter=reporter,
152         resources_per_trial={
153             'cpu': 4,
154             'gpu': 1
155         },

```

```

156     local_dir='./ray_results',
157     checkpoint_freq=5,
158     checkpoint_at_end=True,
159     verbose=1
160 )
161
162 # Best trial
163 best_trial = analysis.get_best_trial('val_accuracy', 'max', 'last')
164 print(f"\nBest trial config: {best_trial.config}")
165 print(f"Best trial final validation accuracy: {best_trial.last_result['val_accuracy']:.4f}")
166
167 # Get best checkpoint
168 best_checkpoint = analysis.get_best_checkpoint(
169     trial=best_trial,
170     metric='val_accuracy',
171     mode='max'
172 )
173 print(f"Best checkpoint: {best_checkpoint}")
174
175 # Analysis
176 df = analysis.results_df
177 print("\nTop 5 configurations:")
178 print(df.sort_values('val_accuracy', ascending=False).head(5))
179
180 ray.shutdown()
181
182 return analysis
183
184 # Population-Based Training with Ray
185 def run_ray_pbt():
186     """Run PBT with Ray Tune."""
187     ray.init(num_gpus=4)
188
189     # PBT scheduler
190     pbt = RayPBT(
191         time_attr='training_iteration',
192         metric='val_accuracy',
193         mode='max',
194         perturbation_interval=5, # Perturb every 5 epochs
195         hyperparam_mutations={
196             'lr': lambda: np.random.uniform(1e-5, 1e-1),
197             'weight_decay': lambda: np.random.uniform(1e-6, 1e-2),
198             'dropout': lambda: np.random.uniform(0.0, 0.5)
199         }
200     )
201
202     analysis = tune.run(
203         train_model_tune,
204         name='pbt_optimization',
205         scheduler=pbt,
206         num_samples=10, # Population size
207         config={
208             'n_layers': 4,
209             'hidden_dim': 256,
210             'dropout': tune.uniform(0.0, 0.5),
211             'lr': tune.loguniform(1e-5, 1e-1),
212             'weight_decay': tune.loguniform(1e-6, 1e-2),

```



```

213         'batch_size': 64,
214         'num_epochs': 100
215     },
216     resources_per_trial={'cpu': 4, 'gpu': 1}
217 )
218
219 ray.shutdown()
220 return analysis

```

Listing 10.5: Distributed HPO with Ray Tune

10.7 AutoML and Neural Architecture Search

Automated architecture search finds optimal model structures without manual design.

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 from typing import List, Tuple
5 import numpy as np
6
7 class MixedOp(nn.Module):
8     """
9     Mixed operation for DARTS (Differentiable Architecture Search).
10    Combines multiple operations with learnable weights.
11    """
12
13    def __init__(self, in_channels: int, out_channels: int, stride: int = 1):
14        super().__init__()
15
16        # Define candidate operations
17        self.ops = nn.ModuleList([
18            nn.Identity() if stride == 1 and in_channels == out_channels else
19            nn.Conv2d(in_channels, out_channels, 1, stride, 0, bias=False),
20            nn.Conv2d(in_channels, out_channels, 3, stride, 1, bias=False),
21            nn.Conv2d(in_channels, out_channels, 5, stride, 2, bias=False),
22            nn.Sequential(
23                nn.AvgPool2d(3, stride, 1),
24                nn.Conv2d(in_channels, out_channels, 1, 1, 0, bias=False)
25            ),
26            nn.Sequential(
27                nn.MaxPool2d(3, stride, 1),
28                nn.Conv2d(in_channels, out_channels, 1, 1, 0, bias=False)
29            ),
30            self._make_sep_conv(in_channels, out_channels, 3, stride, 1),
31            self._make_sep_conv(in_channels, out_channels, 5, stride, 2),
32        ])
33
34        # Architecture parameters (alpha)
35        self.alpha = nn.Parameter(torch.randn(len(self.ops)))
36
37    def _make_sep_conv(self, in_c, out_c, kernel, stride, padding):
38        """Separable convolution."""
39        return nn.Sequential(
40            nn.Conv2d(in_c, in_c, kernel, stride, padding, groups=in_c, bias=False),
41            nn.Conv2d(in_c, out_c, 1, 1, 0, bias=False)
42        )

```

```

43
44     def forward(self, x):
45         """Forward with softmax over operations."""
46         weights = F.softmax(self.alpha, dim=0)
47         return sum(w * op(x) for w, op in zip(weights, self.ops))
48
49     class SearchCell(nn.Module):
50         """Searchable cell for NAS."""
51
52         def __init__(self, in_channels: int, out_channels: int, n_nodes: int = 4):
53             super().__init__()
54             self.n_nodes = n_nodes
55
56             # Mixed operations for each node
57             self.ops = nn.ModuleList()
58             for i in range(n_nodes):
59                 for j in range(i + 2): # Connect to all previous nodes
60                     self.ops.append(
61                         MixedOp(in_channels if j < 2 else out_channels, out_channels)
62                     )
63
64         def forward(self, s0, s1):
65             """
66             Forward pass through cell.
67             s0, s1: outputs from previous two cells
68             """
69             states = [s0, s1]
70             offset = 0
71
72             for i in range(self.n_nodes):
73                 # Aggregate all incoming edges to this node
74                 s = sum(self.ops[offset + j](h) for j, h in enumerate(states))
75                 offset += len(states)
76                 states.append(s)
77
78             # Concatenate outputs of all intermediate nodes
79             return torch.cat(states[2:], dim=1)
80
81     class DARTSNetwork(nn.Module):
82         """
83         DARTS: Differentiable Architecture Search.
84
85         Jointly optimizes network weights and architecture parameters.
86         """
87
88         def __init__(self, in_channels: int = 3, num_classes: int = 10,
89                     channels: int = 16, n_cells: int = 8):
90             super().__init__()
91
92             self.stem = nn.Sequential(
93                 nn.Conv2d(in_channels, channels, 3, 1, 1, bias=False),
94                 nn.BatchNorm2d(channels)
95             )
96
97             # Build cells
98             self.cells = nn.ModuleList()
99             for i in range(n_cells):
100                 # Reduction cell every 1/3 of network

```

```

101         is_reduction = i in [n_cells // 3, 2 * n_cells // 3]
102         cell = SearchCell(
103             in_channels=channels,
104             out_channels=channels * 2 if is_reduction else channels
105         )
106         self.cells.append(cell)
107
108         if is_reduction:
109             channels *= 2
110
111         self.global_pooling = nn.AdaptiveAvgPool2d(1)
112         self.classifier = nn.Linear(channels * 4, num_classes) # *4 for concatenation
113
114     def forward(self, x):
115         s0 = s1 = self.stem(x)
116
117         for cell in self.cells:
118             s0, s1 = s1, cell(s0, s1)
119
120         out = self.global_pooling(s1)
121         out = out.view(out.size(0), -1)
122         logits = self.classifier(out)
123
124         return logits
125
126     def get_architecture_parameters(self):
127         """Get architecture parameters (alphas) for optimization."""
128         for name, param in self.named_parameters():
129             if 'alpha' in name:
130                 yield param
131
132     def get_weight_parameters(self):
133         """Get network weight parameters."""
134         for name, param in self.named_parameters():
135             if 'alpha' not in name:
136                 yield param
137
138 def train_darts(model: DARTSNetwork, train_loader, val_loader,
139               epochs: int = 50, device='cuda'):
140     """
141     Train DARTS with bi-level optimization.
142
143     Alternates between:
144     1. Optimizing architecture parameters (alpha) on validation set
145     2. Optimizing network weights (w) on training set
146     """
147     model = model.to(device)
148
149     # Separate optimizers for weights and architecture
150     optimizer_w = torch.optim.SGD(
151         model.get_weight_parameters(),
152         lr=0.025,
153         momentum=0.9,
154         weight_decay=3e-4
155     )
156
157     optimizer_alpha = torch.optim.Adam(
158         model.get_architecture_parameters(),

```

```

159         lr=3e-4,
160         betas=(0.5, 0.999),
161         weight_decay=1e-3
162     )
163
164     scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
165         optimizer_w, epochs
166     )
167
168     criterion = nn.CrossEntropyLoss()
169
170     for epoch in range(epochs):
171         # Phase 1: Update architecture parameters on validation set
172         model.train()
173         for data, target in val_loader:
174             data, target = data.to(device), target.to(device)
175
176             optimizer_alpha.zero_grad()
177             output = model(data)
178             loss = criterion(output, target)
179             loss.backward()
180             optimizer_alpha.step()
181
182         # Phase 2: Update network weights on training set
183         for data, target in train_loader:
184             data, target = data.to(device), target.to(device)
185
186             optimizer_w.zero_grad()
187             output = model(data)
188             loss = criterion(output, target)
189             loss.backward()
190             optimizer_w.step()
191
192         scheduler.step()
193
194         if (epoch + 1) % 10 == 0:
195             print(f"Epoch {epoch + 1}/{epochs}")
196             print_architecture(model)
197
198     return model
199
200 def print_architecture(model: DARTSNetwork):
201     """Print discovered architecture."""
202     print("\nDiscovered Architecture:")
203
204     for cell_idx, cell in enumerate(model.cells):
205         print(f"\nCell {cell_idx}:")
206         for op_idx, op in enumerate(cell.ops):
207             if isinstance(op, MixedOp):
208                 # Get operation with highest weight
209                 weights = F.softmax(op.alpha, dim=0)
210                 best_op_idx = weights.argmax().item()
211                 best_weight = weights[best_op_idx].item()
212
213                 op_names = ['identity', 'conv1x1', 'conv3x3', 'conv5x5',
214                             'avgpool', 'maxpool', 'sepconv3x3', 'sepconv5x5']
215
216                 print(f"  Op {op_idx}: {op_names[best_op_idx]} ")

```

```

217         f"(weight: {best_weight:.3f})")
218
219 def derive_final_architecture(model: DARTSNetwork) -> nn.Module:
220     """
221     Derive final discrete architecture from searched model.
222     Keeps only the operation with highest alpha for each mixed op.
223     """
224     # Extract architecture decisions
225     architecture = []
226
227     for cell in model.cells:
228         cell_arch = []
229         for op in cell.ops:
230             if isinstance(op, MixedOp):
231                 best_op_idx = F.softmax(op.alpha, dim=0).argmax().item()
232                 cell_arch.append(best_op_idx)
233         architecture.append(cell_arch)
234
235     # Build final model with selected operations
236     # (Implementation would create fixed network based on architecture)
237
238     return architecture

```

Listing 10.6: Neural Architecture Search with DARTS

10.8 Integration with Experiment Tracking

Track hyperparameter experiments with MLflow, Weights & Biases, or TensorBoard.

```

1 import mlflow
2 import mlflow.pytorch
3 from mlflow.tracking import MlflowClient
4 import optuna
5 from optuna.integration.mlflow import MLflowCallback
6
7 def objective_with_mlflow(trial: optuna.Trial) -> float:
8     """Optuna objective with MLflow logging."""
9
10    # Suggest hyperparameters
11    params = {
12        'n_layers': trial.suggest_int('n_layers', 2, 8),
13        'hidden_dim': trial.suggest_categorical('hidden_dim', [64, 128, 256, 512]),
14        'dropout': trial.suggest_float('dropout', 0.0, 0.5),
15        'lr': trial.suggest_float('lr', 1e-5, 1e-1, log=True),
16        'batch_size': trial.suggest_categorical('batch_size', [32, 64, 128]),
17    }
18
19    # Start MLflow run for this trial
20    with mlflow.start_run(nested=True):
21        # Log parameters
22        mlflow.log_params(params)
23        mlflow.set_tag('trial_number', trial.number)
24
25        # Train model
26        model = create_and_train_model(params)
27
28        # Evaluate

```

```

29     accuracy = evaluate_model(model)
30
31     # Log metrics
32     mlflow.log_metric('accuracy', accuracy)
33
34     # Log model
35     mlflow.pytorch.log_model(model, 'model')
36
37     # Log artifacts (plots, etc.)
38     save_training_plot(model, 'training_plot.png')
39     mlflow.log_artifact('training_plot.png')
40
41     return accuracy
42
43 def run_hpo_with_mlflow():
44     """Run HPO with full MLflow integration."""
45     mlflow.set_experiment('hyperparameter_optimization')
46
47     # Create parent run for the study
48     with mlflow.start_run(run_name='optuna_study'):
49         # Create study
50         study = optuna.create_study(
51             study_name='mlp_optimization',
52             direction='maximize',
53             sampler=TPESampler(),
54             pruner=HyperbandPruner()
55         )
56
57         # Create MLflow callback
58         mlflow_callback = MLflowCallback(
59             tracking_uri=mlflow.get_tracking_uri(),
60             metric_name='accuracy'
61         )
62
63         # Optimize
64         study.optimize(
65             objective_with_mlflow,
66             n_trials=50,
67             callbacks=[mlflow_callback]
68         )
69
70         # Log best parameters to parent run
71         mlflow.log_params(study.best_params)
72         mlflow.log_metric('best_accuracy', study.best_value)
73
74         # Log study visualizations
75         try:
76             import optuna.visualization as vis
77
78             fig = vis.plot_optimization_history(study)
79             fig.write_html('optimization_history.html')
80             mlflow.log_artifact('optimization_history.html')
81
82             fig = vis.plot_param_importances(study)
83             fig.write_html('param_importances.html')
84             mlflow.log_artifact('param_importances.html')
85
86         except ImportError:

```

```

87         pass
88
89     return study
90
91 # Weights & Biases integration
92 def run_hpo_with_wandb():
93     """Run HPO with Weights & Biases tracking."""
94     import wandb
95     from optuna.integration.wandb import WeightsAndBiasesCallback
96
97     # Initialize wandb
98     wandb.init(project='hyperparameter-optimization', entity='ml-team')
99
100    # Create study with wandb callback
101    wandb_callback = WeightsAndBiasesCallback(
102        metric_name='accuracy',
103        wandb_kwargs={'project': 'hyperparameter-optimization'}
104    )
105
106    study = optuna.create_study(direction='maximize')
107    study.optimize(
108        objective_with_mlflow, # Can use same objective
109        n_trials=50,
110        callbacks=[wandb_callback]
111    )
112
113    # Log best config to wandb
114    wandb.config.update(study.best_params)
115    wandb.log({'best_accuracy': study.best_value})
116
117    wandb.finish()
118
119    return study

```

Listing 10.7: HPO with MLflow tracking

10.9 Cost-Efficient HPO Strategies

Reduce computational costs while maintaining optimization quality.

```

1 import optuna
2 from optuna.pruners import SuccessiveHalvingPruner
3 import numpy as np
4 from typing import Dict, Any
5
6 class CostAwareOptimizer:
7     """
8     Cost-aware hyperparameter optimization.
9
10    Strategies:
11    1. Multi-fidelity optimization (train with reduced budgets)
12    2. Early stopping of unpromising trials
13    3. Transfer learning from previous experiments
14    4. Cheap-to-evaluate surrogates
15    """
16
17    def __init__(self, budget_hours: float = 24.0):

```

```

18     self.budget_hours = budget_hours
19     self.total_cost = 0.0
20     self.cost_history = []
21
22     def estimate_trial_cost(self, config: Dict[str, Any]) -> float:
23         """
24         Estimate computational cost of a trial.
25
26         Cost factors:
27         - Model size (parameters)
28         - Batch size (memory)
29         - Number of epochs
30         """
31         # Estimate parameters
32         n_params = self._estimate_parameters(config)
33
34         # Base cost (minutes)
35         base_cost = 1.0
36
37         # Scale by model size
38         size_factor = np.log10(n_params / 1e6 + 1) + 1
39
40         # Scale by training length
41         epoch_factor = config.get('num_epochs', 10) / 10
42
43         # Scale by batch size (inverse - smaller batch = more iterations)
44         batch_factor = 128 / config.get('batch_size', 128)
45
46         total_cost = base_cost * size_factor * epoch_factor * batch_factor
47
48         return total_cost / 60 # Convert to hours
49
50     def _estimate_parameters(self, config: Dict[str, Any]) -> int:
51         """Estimate number of model parameters."""
52         n_layers = config.get('n_layers', 4)
53         hidden_dim = config.get('hidden_dim', 256)
54
55         # Rough estimate
56         params = hidden_dim * hidden_dim * n_layers
57         return params
58
59     def multi_fidelity_objective(self, trial: optuna.Trial) -> float:
60         """
61         Objective with progressive resource allocation.
62
63         Start with small budget, increase for promising trials.
64         """
65         config = suggest_config(trial)
66
67         # Progressive fidelity levels
68         fidelity_levels = [
69             {'epochs': 5, 'data_fraction': 0.1},
70             {'epochs': 10, 'data_fraction': 0.3},
71             {'epochs': 20, 'data_fraction': 0.7},
72             {'epochs': 50, 'data_fraction': 1.0},
73         ]
74
75         best_score = 0.0

```



```

76
77     for level_idx, fidelity in enumerate(fidelity_levels):
78         # Train with current fidelity
79         score = train_with_budget(
80             config,
81             epochs=fidelity['epochs'],
82             data_fraction=fidelity['data_fraction']
83         )
84
85         # Report intermediate value
86         trial.report(score, level_idx)
87
88         # Check if should prune
89         if trial.should_prune():
90             # Save cost
91             estimated_cost = self.estimate_trial_cost(config)
92             saved_cost = estimated_cost * (1 - (level_idx + 1) / len(
fidelity_levels))
93             self.cost_history.append({
94                 'trial': trial.number,
95                 'pruned_at_level': level_idx,
96                 'cost_saved': saved_cost
97             })
98             raise optuna.TrialPruned()
99
100         best_score = score
101
102         # Check budget
103         trial_cost = self.estimate_trial_cost(config)
104         if self.total_cost + trial_cost > self.budget_hours:
105             print(f"Budget exhausted at trial {trial.number}")
106             trial.study.stop()
107             break
108
109         # Update total cost
110         self.total_cost += self.estimate_trial_cost(config)
111
112         return best_score
113
114     def run_budget_aware_study(self):
115         """Run optimization with budget constraints."""
116         study = optuna.create_study(
117             direction='maximize',
118             sampler=optuna.samplers.TPESampler(),
119             pruner=SuccessiveHalvingPruner(
120                 min_resource=1,
121                 reduction_factor=3,
122                 min_early_stopping_rate=0
123             )
124         )
125
126         # Optimize until budget exhausted
127         while self.total_cost < self.budget_hours:
128             study.optimize(
129                 self.multi_fidelity_objective,
130                 n_trials=1,
131                 show_progress_bar=True
132             )

```

```

133     print(f"\nOptimization complete")
134     print(f"Total cost: {self.total_cost:.2f} hours")
135     print(f"Number of trials: {len(study.trials)}")
136     print(f"Pruned trials: {len([t for t in study.trials if t.state == optuna.
137         trial.TrialState.PRUNED])}")
138
139     return study
140
141 # Transfer learning from previous studies
142 class TransferLearningOptimizer:
143     """Use previous optimization results to warm-start new studies."""
144
145     def __init__(self, previous_study: optuna.Study = None):
146         self.previous_study = previous_study
147
148     def suggest_initial_configs(self, n: int = 10) -> List[Dict]:
149         """
150         Suggest initial configurations based on previous study.
151
152         Returns top-k configurations from previous study.
153         """
154         if self.previous_study is None:
155             return []
156
157         # Get top trials from previous study
158         trials = sorted(
159             self.previous_study.trials,
160             key=lambda t: t.value if t.value is not None else float('-inf'),
161             reverse=True
162         )[:n]
163
164         return [trial.params for trial in trials]
165
166     def create_warm_started_study(self) -> optuna.Study:
167         """Create new study warm-started from previous results."""
168         study = optuna.create_study(direction='maximize')
169
170         # Add previous best configurations as initial trials
171         initial_configs = self.suggest_initial_configs()
172
173         for config in initial_configs:
174             # Enqueue configuration for evaluation
175             study.enqueue_trial(config)
176
177         return study
178
179 # Surrogate model for cheap evaluation
180 class SurrogateModel:
181     """
182     Cheap-to-evaluate surrogate for expensive objective.
183
184     Train lightweight model to predict performance.
185     """
186
187     def __init__(self):
188         self.trials = []
189         self.model = None

```

```

190
191 def add_trial(self, config: Dict, score: float):
192     """Add trial result to training data."""
193     self.trials.append((config, score))
194
195 def train_surrogate(self):
196     """Train surrogate model on collected trials."""
197     if len(self.trials) < 10:
198         return
199
200     # Convert configs to features
201     X = []
202     y = []
203
204     for config, score in self.trials:
205         features = self._config_to_features(config)
206         X.append(features)
207         y.append(score)
208
209     X = np.array(X)
210     y = np.array(y)
211
212     # Train simple model (Random Forest)
213     from sklearn.ensemble import RandomForestRegressor
214
215     self.model = RandomForestRegressor(n_estimators=50, random_state=42)
216     self.model.fit(X, y)
217
218 def _config_to_features(self, config: Dict) -> np.ndarray:
219     """Convert configuration dict to feature vector."""
220     features = [
221         config.get('n_layers', 0),
222         config.get('hidden_dim', 0),
223         config.get('dropout', 0),
224         np.log10(config.get('lr', 1e-3)),
225         config.get('batch_size', 0),
226     ]
227     return np.array(features)
228
229 def predict(self, config: Dict) -> float:
230     """Predict performance for given configuration."""
231     if self.model is None:
232         return 0.5 # Default prediction
233
234     features = self._config_to_features(config).reshape(1, -1)
235     return self.model.predict(features)[0]
236
237 def filter_unpromising_configs(
238     self,
239     configs: List[Dict],
240     threshold: float = 0.4
241 ) -> List[Dict]:
242     """Filter out configurations predicted to perform poorly."""
243     if self.model is None:
244         return configs
245
246     promising = []
247     for config in configs:

```

```
248         predicted_score = self.predict(config)
249         if predicted_score >= threshold:
250             promising.append(config)
251
252     return promising
```

Listing 10.8: Cost-efficient hyperparameter optimization

10.10 Chapter Summary

This chapter covered advanced hyperparameter optimization techniques for production ML:

- **Bayesian Optimization:** Optuna with advanced samplers (TPE, CMA-ES) and pruners (Hyperband, Median)
- **Multi-Objective Optimization:** Pareto optimization for accuracy, latency, and model size trade-offs
- **Population-Based Training:** Dynamic hyperparameter adaptation during training
- **Distributed HPO:** Ray Tune for scaling across GPUs and nodes
- **Neural Architecture Search:** DARTS for automated architecture discovery
- **Experiment Tracking:** MLflow and Weights & Biases integration
- **Cost-Efficient Strategies:** Multi-fidelity, transfer learning, surrogate models

Key insights:

1. Bayesian optimization drastically reduces trials compared to grid/random search
2. Multi-objective optimization reveals trade-offs between competing metrics
3. Early stopping (pruning) saves 50-90% of computation
4. Distributed HPO scales linearly with available resources
5. Transfer learning from previous experiments accelerates new searches
6. Multi-fidelity optimization (train with reduced budgets) is highly cost-effective

10.11 Exercises

Exercise 10.1 (Optuna Advanced Features). Implement comprehensive HPO with Optuna:

1. Define conditional search space for CNN architecture
2. Use TPE sampler with multivariate sampling

3. Implement Hyperband pruner with resource = epochs
4. Add custom callback for early study stopping
5. Generate and analyze all Optuna visualizations
6. Compare with random search baseline

Target: Find configuration achieving >95% accuracy on CIFAR-10 in <50 trials.

Exercise 10.2 (Multi-Objective Optimization). Optimize model for multiple objectives:

1. Implement objective returning (accuracy, latency, size)
2. Use NSGA-II sampler for Pareto optimization
3. Run 200 trials to discover Pareto front
4. Visualize 3D Pareto front
5. Implement constraint-based model selection
6. Analyze trade-offs between objectives

Deliverable: Report with Pareto front and trade-off analysis.

Exercise 10.3 (Population-Based Training). Build PBT system from scratch:

1. Implement population initialization with random hyperparameters
2. Add exploit (copy from best) and explore (perturb) logic
3. Train population on MNIST for 10K steps
4. Track learning curves for all population members
5. Compare final performance with single best configuration
6. Visualize hyperparameter evolution over time

Target: Achieve >98.5% accuracy, outperforming static best by >0.5%.

Exercise 10.4 (Distributed HPO with Ray Tune). Deploy distributed optimization:

1. Set up Ray cluster (local or cloud)
2. Implement training function for Ray Tune
3. Use ASHA scheduler for early stopping
4. Run 100 trials across 4 GPUs
5. Implement checkpointing and fault tolerance
6. Compare wall-clock time vs sequential optimization

Target: Achieve 3-4x speedup with 4 GPUs.

Exercise 10.5 (Neural Architecture Search). Implement simple NAS:

1. Define search space with 5 operations per edge
2. Implement DARTS bi-level optimization
3. Train architecture search for 50 epochs
4. Derive final discrete architecture
5. Retrain from scratch with discovered architecture
6. Compare with hand-designed baseline

Deliverable: Discovered architecture visualization and performance comparison.

Exercise 10.6 (Cost-Efficient HPO). Optimize under budget constraints:

1. Implement multi-fidelity objective (progressive epochs)
2. Add trial cost estimation
3. Run optimization with 8-hour budget
4. Track cost savings from pruning
5. Compare trials completed vs fixed-budget approach
6. Analyze cost vs performance trade-off

Target: Complete 2x more trials than fixed-budget approach.

Exercise 10.7 (Transfer Learning for HPO). Warm-start optimization from previous studies:

1. Run initial study on MNIST (50 trials)
2. Extract top-10 configurations
3. Create new study on Fashion-MNIST
4. Warm-start with transferred configurations
5. Compare with cold-start baseline
6. Measure convergence speedup

Target: Reach target accuracy 30% faster with warm-starting.

10.12 Hyperparameter Search Space Design

Effective search space design is critical for successful hyperparameter optimization. Poorly designed spaces waste computational resources and may miss optimal configurations.

10.12.1 Search Space Types and Best Practices

```

1 import optuna
2 from typing import Dict, List, Any
3 import numpy as np
4
5 class SearchSpaceDesigner:
6     """
7     Design effective hyperparameter search spaces.
8
9     Principles:
10    1. Use log-scale for learning rates and regularization
11    2. Categorical for discrete architectural choices
12    3. Conditional dependencies for hierarchical decisions
13    4. Reasonable bounds based on domain knowledge
14    """
15
16    @staticmethod
17    def learning_rate_space(trial: optuna.Trial,
18                           name: str = 'lr',
19                           min_val: float = 1e-5,
20                           max_val: float = 1e-1) -> float:
21        """
22        Learning rate should always use log-uniform distribution.
23
24        Rationale: Learning rates span multiple orders of magnitude,
25        and we want to sample uniformly in log-space.
26        """
27        return trial.suggest_float(name, min_val, max_val, log=True)
28
29    @staticmethod
30    def regularization_space(trial: optuna.Trial) -> Dict[str, float]:
31        """
32        Regularization hyperparameters (weight decay, dropout, etc.)
33        also benefit from log-scale sampling.
34        """
35        return {
36            'weight_decay': trial.suggest_float('weight_decay', 1e-6, 1e-1, log=True),
37            'dropout': trial.suggest_float('dropout', 0.0, 0.5),
38            'label_smoothing': trial.suggest_float('label_smoothing', 0.0, 0.2),
39            'gradient_clip': trial.suggest_float('gradient_clip', 0.1, 10.0)
40        }
41
42    @staticmethod
43    def architecture_space(trial: optuna.Trial,
44                           task_type: str = 'classification') -> Dict:
45        """
46        Architecture hyperparameters depend on task and model type.
47        """
48        config = {}
49
50        # Model backbone
51        model_type = trial.suggest_categorical(
52            'model_type',
53            ['resnet', 'efficientnet', 'vit', 'custom_cnn']
54        )
55        config['model_type'] = model_type
56

```

```

57     if model_type == 'resnet':
58         config['depth'] = trial.suggest_categorical('resnet_depth', [18, 34, 50,
101])
59         config['width_multiplier'] = trial.suggest_float('width_mult', 0.5, 2.0)
60
61     elif model_type == 'efficientnet':
62         config['variant'] = trial.suggest_categorical(
63             'efficientnet_variant',
64             ['b0', 'b1', 'b2', 'b3']
65         )
66
67     elif model_type == 'vit':
68         config['patch_size'] = trial.suggest_categorical('patch_size', [8, 16,
32])
69         config['n_layers'] = trial.suggest_int('vit_layers', 6, 24)
70         config['n_heads'] = trial.suggest_categorical('n_heads', [4, 8, 12, 16])
71         config['embed_dim'] = trial.suggest_categorical('embed_dim', [256, 512,
768, 1024])
72
73         # Ensure embed_dim divisible by n_heads
74         if config['embed_dim'] % config['n_heads'] != 0:
75             config['embed_dim'] = (config['embed_dim'] // config['n_heads']) *
config['n_heads']
76
77     elif model_type == 'custom_cnn':
78         config['n_blocks'] = trial.suggest_int('n_blocks', 2, 8)
79         config['base_channels'] = trial.suggest_categorical('base_channels', [32,
64, 128])
80         config['block_type'] = trial.suggest_categorical('block_type', ['basic', '
residual', 'dense'])
81
82     return config
83
84 @staticmethod
85 def optimizer_space(trial: optuna.Trial) -> Dict:
86     """
87     Optimizer and scheduler configuration.
88     """
89     optimizer_name = trial.suggest_categorical(
90         'optimizer',
91         ['adam', 'adamw', 'sgd', 'rmsprop', 'lamb']
92     )
93
94     config = {
95         'optimizer': optimizer_name,
96         'lr': trial.suggest_float('lr', 1e-5, 1e-1, log=True)
97     }
98
99     # Optimizer-specific hyperparameters
100     if optimizer_name in ['adam', 'adamw']:
101         config['beta1'] = trial.suggest_float('adam_beta1', 0.8, 0.99)
102         config['beta2'] = trial.suggest_float('adam_beta2', 0.9, 0.999)
103         config['eps'] = trial.suggest_float('adam_eps', 1e-9, 1e-7, log=True)
104
105     elif optimizer_name == 'sgd':
106         config['momentum'] = trial.suggest_float('momentum', 0.0, 0.99)
107         config['nesterov'] = trial.suggest_categorical('nesterov', [True, False])
108

```



```

109     # Scheduler
110     scheduler_name = trial.suggest_categorical(
111         'scheduler',
112         ['none', 'cosine', 'step', 'exponential', 'onecycle', 'plateau']
113     )
114     config['scheduler'] = scheduler_name
115
116     if scheduler_name == 'cosine':
117         config['cosine_eta_min'] = trial.suggest_float('eta_min', 1e-7, 1e-4, log=
118 True)
119         config['warmup_epochs'] = trial.suggest_int('warmup_epochs', 0, 10)
120
121     elif scheduler_name == 'step':
122         config['step_size'] = trial.suggest_int('step_size', 5, 50)
123         config['step_gamma'] = trial.suggest_float('step_gamma', 0.1, 0.9)
124
125     elif scheduler_name == 'onecycle':
126         config['max_lr'] = config['lr'] * trial.suggest_float('onecycle_mult',
127 5.0, 20.0)
128         config['pct_start'] = trial.suggest_float('pct_start', 0.1, 0.5)
129
130     elif scheduler_name == 'plateau':
131         config['plateau_patience'] = trial.suggest_int('plateau_patience', 3, 15)
132         config['plateau_factor'] = trial.suggest_float('plateau_factor', 0.1, 0.5)
133
134     return config
135
136 @staticmethod
137 def data_augmentation_space(trial: optuna.Trial) -> Dict:
138     """
139     Data augmentation hyperparameters.
140     """
141     return {
142         'use_augmentation': trial.suggest_categorical('use_aug', [True, False]),
143         'aug_strength': trial.suggest_float('aug_strength', 0.0, 1.0),
144         'mixup_alpha': trial.suggest_float('mixup_alpha', 0.0, 1.0),
145         'cutmix_alpha': trial.suggest_float('cutmix_alpha', 0.0, 1.0),
146         'random_erase_prob': trial.suggest_float('random_erase', 0.0, 0.5),
147         'color_jitter': trial.suggest_float('color_jitter', 0.0, 0.4)
148     }
149
150 @staticmethod
151 def training_space(trial: optuna.Trial) -> Dict:
152     """
153     Training hyperparameters (batch size, epochs, etc.)
154     """
155     batch_size = trial.suggest_categorical('batch_size', [16, 32, 64, 128, 256,
156 512])
157
158     # Gradient accumulation for effective larger batch sizes
159     grad_accum = trial.suggest_categorical('grad_accum_steps', [1, 2, 4, 8])
160     effective_batch_size = batch_size * grad_accum
161
162     return {
163         'batch_size': batch_size,
164         'grad_accum_steps': grad_accum,
165         'effective_batch_size': effective_batch_size,
166         'num_epochs': trial.suggest_int('num_epochs', 10, 200),

```

```

164         'warmup_ratio': trial.suggest_float('warmup_ratio', 0.0, 0.2),
165         'mixed_precision': trial.suggest_categorical('mixed_precision', [True,
166             False])
167     }
168
169     # Complete example: Comprehensive search space
170     def comprehensive_objective(trial: optuna.Trial) -> float:
171         """
172         Comprehensive objective combining all search space components.
173         """
174         designer = SearchSpaceDesigner()
175
176         # Build complete configuration
177         config = {}
178         config.update(designer.architecture_space(trial))
179         config.update(designer.optimizer_space(trial))
180         config.update(designer.regularization_space(trial))
181         config.update(designer.data_augmentation_space(trial))
182         config.update(designer.training_space(trial))
183
184         # Train model with configuration
185         model = build_model(config)
186         accuracy = train_and_evaluate(model, config)
187
188         return accuracy

```

Listing 10.9: Search space design patterns

10.12.2 Pruning Search Spaces Based on Prior Knowledge

```

1 from dataclasses import dataclass
2 from typing import Set, List, Tuple
3 import json
4
5 @dataclass
6 class SearchSpaceConstraint:
7     """Constraint on hyperparameter combinations."""
8     parameter: str
9     condition: str # 'min', 'max', 'equal', 'not_equal'
10    value: Any
11    dependent_param: str = None
12
13 class IntelligentSearchSpace:
14     """
15     Search space with constraints based on domain knowledge.
16
17     Example constraints:
18     - Large models require smaller learning rates
19     - High dropout incompatible with batch normalization
20     - Transformer layers must have embed_dim divisible by n_heads
21     """
22
23     def __init__(self):
24         self.constraints: List[SearchSpaceConstraint] = []
25         self.incompatible_pairs: Set[Tuple[str, str]] = set()
26         self.recommended_ranges: Dict[str, Dict] = {}
27

```

```

28 def add_constraint(self, constraint: SearchSpaceConstraint):
29     """Add search space constraint."""
30     self.constraints.append(constraint)
31
32 def add_incompatibility(self, param1: str, param2: str):
33     """Mark two parameter choices as incompatible."""
34     self.incompatible_pairs.add((param1, param2))
35
36 def validate_config(self, config: Dict[str, Any]) -> Tuple[bool, str]:
37     """
38     Validate configuration against constraints.
39
40     Returns:
41         (is_valid, error_message)
42     """
43     # Check basic constraints
44     for constraint in self.constraints:
45         if not self._check_constraint(config, constraint):
46             return False, f"Constraint violated: {constraint}"
47
48     # Check incompatibilities
49     for param1, param2 in self.incompatible_pairs:
50         if param1 in config and param2 in config:
51             return False, f"Incompatible parameters: {param1}, {param2}"
52
53     return True, ""
54
55 def _check_constraint(self, config: Dict, constraint: SearchSpaceConstraint) ->
bool:
56     """Check if configuration satisfies constraint."""
57     if constraint.parameter not in config:
58         return True
59
60     value = config[constraint.parameter]
61
62     if constraint.condition == 'min':
63         return value >= constraint.value
64     elif constraint.condition == 'max':
65         return value <= constraint.value
66     elif constraint.condition == 'equal':
67         return value == constraint.value
68     elif constraint.condition == 'not_equal':
69         return value != constraint.value
70
71     return True
72
73 def suggest_constrained_config(self, trial: optuna.Trial) -> Dict[str, Any]:
74     """
75     Suggest configuration that satisfies all constraints.
76
77     Automatically retries if constraints are violated.
78     """
79     max_retries = 10
80
81     for _ in range(max_retries):
82         config = self._suggest_base_config(trial)
83
84         is_valid, error = self.validate_config(config)

```

```

85         if is_valid:
86             return config
87
88         # If invalid, prune this trial
89         raise optuna.TrialPruned(f"Invalid configuration: {error}")
90
91     raise ValueError("Could not find valid configuration after max retries")
92
93 def _suggest_base_config(self, trial: optuna.Trial) -> Dict[str, Any]:
94     """Suggest base configuration."""
95     config = {}
96
97     # Model size
98     n_layers = trial.suggest_int('n_layers', 2, 12)
99     hidden_dim = trial.suggest_categorical('hidden_dim', [64, 128, 256, 512,
100 1024])
101
102     config['n_layers'] = n_layers
103     config['hidden_dim'] = hidden_dim
104
105     # Estimate model size
106     model_params = n_layers * hidden_dim * hidden_dim
107
108     # Adapt learning rate based on model size
109     if model_params > 1e7: # Large model
110         lr_min, lr_max = 1e-5, 1e-3
111     elif model_params > 1e6: # Medium model
112         lr_min, lr_max = 1e-4, 1e-2
113     else: # Small model
114         lr_min, lr_max = 1e-4, 1e-1
115
116     config['lr'] = trial.suggest_float('lr', lr_min, lr_max, log=True)
117
118     # Batch size based on model size (larger models need smaller batches)
119     if model_params > 1e7:
120         config['batch_size'] = trial.suggest_categorical('batch_size', [16, 32,
121 64])
122     else:
123         config['batch_size'] = trial.suggest_categorical('batch_size', [32, 64,
124 128, 256])
125
126     # Dropout and batch norm are somewhat incompatible
127     use_batch_norm = trial.suggest_categorical('use_batch_norm', [True, False])
128     config['use_batch_norm'] = use_batch_norm
129
130     if use_batch_norm:
131         # Lower dropout when using batch norm
132         config['dropout'] = trial.suggest_float('dropout', 0.0, 0.3)
133     else:
134         # Can use higher dropout without batch norm
135         config['dropout'] = trial.suggest_float('dropout', 0.0, 0.5)
136
137     return config
138
139 # Example usage with constraints
140 def create_constrained_study():
141     """Create study with intelligent search space."""

```

```

140     space = IntelligentSearchSpace()
141
142     # Add constraints
143     space.add_constraint(SearchSpaceConstraint(
144         parameter='n_layers',
145         condition='max',
146         value=12
147     ))
148
149     space.add_constraint(SearchSpaceConstraint(
150         parameter='dropout',
151         condition='max',
152         value=0.5
153     ))
154
155     # Create study
156     study = optuna.create_study(direction='maximize')
157
158     def constrained_objective(trial):
159         config = space.suggest_constrained_config(trial)
160         model = build_model(config)
161         return train_and_evaluate(model, config)
162
163     study.optimize(constrained_objective, n_trials=100)
164
165     return study

```

Listing 10.10: Intelligent search space pruning

10.13 Advanced Pruning Strategies

Early stopping of unpromising trials is crucial for efficient HPO. Advanced pruning strategies can save 50-90% of computational cost.

```

1  import optuna
2  from optuna.pruners import BasePruner
3  from optuna.trial import TrialState
4  import numpy as np
5  from typing import Optional
6
7  class PerformanceBasedPruner(BasePruner):
8      """
9      Custom pruner based on performance trajectory.
10
11      Prunes trials that:
12      1. Show no improvement for N steps
13      2. Have poor learning curve characteristics
14      3. Diverge (loss increases significantly)
15      """
16
17      def __init__(self,
18                  patience: int = 5,
19                  min_trials: int = 10,
20                  percentile: float = 50.0):
21          self.patience = patience
22          self.min_trials = min_trials

```

```

23     self.percentile = percentile
24
25     def prune(self,
26               study: optuna.Study,
27               trial: optuna.trial.FrozenTrial) -> bool:
28         """
29         Determine if trial should be pruned.
30
31         Returns:
32             True if trial should be pruned
33         """
34         # Get intermediate values for this trial
35         intermediate_values = trial.intermediate_values
36
37         if len(intermediate_values) == 0:
38             return False
39
40         # Get current step and value
41         current_step = max(intermediate_values.keys())
42         current_value = intermediate_values[current_step]
43
44         # Check for divergence
45         if self._is_diverging(intermediate_values):
46             return True
47
48         # Check for no improvement
49         if self._no_improvement(intermediate_values):
50             return True
51
52         # Compare with other trials at same step
53         completed_trials = [
54             t for t in study.trials
55             if t.state == TrialState.COMPLETE or t.state == TrialState.RUNNING
56         ]
57
58         if len(completed_trials) < self.min_trials:
59             return False
60
61         # Get values from other trials at current step
62         other_values = []
63         for t in completed_trials:
64             if t.number == trial.number:
65                 continue
66             if current_step in t.intermediate_values:
67                 other_values.append(t.intermediate_values[current_step])
68
69         if len(other_values) < self.min_trials:
70             return False
71
72         # Prune if below percentile
73         threshold = np.percentile(other_values, self.percentile)
74
75         # Assuming higher is better (e.g., accuracy)
76         return current_value < threshold
77
78     def _is_diverging(self, values: Dict[int, float]) -> bool:
79         """Check if values are diverging (getting worse)."""
80         if len(values) < 3:

```

```

81         return False
82
83     steps = sorted(values.keys())
84     recent_steps = steps[-3:]
85     recent_values = [values[s] for s in recent_steps]
86
87     # Check if consistently decreasing (assuming higher is better)
88     is_decreasing = all(
89         recent_values[i] > recent_values[i+1]
90         for i in range(len(recent_values)-1)
91     )
92
93     # Check if decrease is significant
94     total_decrease = recent_values[0] - recent_values[-1]
95     relative_decrease = total_decrease / (abs(recent_values[0]) + 1e-8)
96
97     return is_decreasing and relative_decrease > 0.1 # 10% decrease
98
99     def _no_improvement(self, values: Dict[int, float]) -> bool:
100         """Check if no improvement for patience steps."""
101         if len(values) < self.patience:
102             return False
103
104         steps = sorted(values.keys())
105         recent_steps = steps[-self.patience:]
106         recent_values = [values[s] for s in recent_steps]
107
108         best_recent = max(recent_values)
109
110         # Check if best value was at the beginning (no improvement since)
111         return recent_values[0] >= best_recent
112
113     class LearningCurveExtrapolator(BasePruner):
114         """
115         Prune based on learning curve extrapolation.
116
117         Fits power law to learning curve and predicts final performance.
118         Prunes if predicted performance is below threshold.
119         """
120
121         def __init__(self, min_trials: int = 10, min_steps: int = 5):
122             self.min_trials = min_trials
123             self.min_steps = min_steps
124
125         def prune(self, study: optuna.Study, trial: optuna.trial.FrozenTrial) -> bool:
126             """Prune based on extrapolated performance."""
127             intermediate_values = trial.intermediate_values
128
129             if len(intermediate_values) < self.min_steps:
130                 return False
131
132             # Fit learning curve model
133             predicted_final = self._extrapolate_performance(intermediate_values)
134
135             if predicted_final is None:
136                 return False
137
138             # Compare with best completed trial

```

```

139     completed_trials = [
140         t for t in study.trials
141         if t.state == TrialState.COMPLETE
142     ]
143
144     if len(completed_trials) < self.min_trials:
145         return False
146
147     best_value = max(t.value for t in completed_trials if t.value is not None)
148
149     # Prune if predicted to be significantly worse than best
150     return predicted_final < best_value * 0.95
151
152 def _extrapolate_performance(self, values: Dict[int, float]) -> Optional[float]:
153     """
154     Extrapolate final performance using power law fit.
155
156     Model: performance = a - b * step^(-c)
157     """
158     if len(values) < self.min_steps:
159         return None
160
161     try:
162         from scipy.optimize import curve_fit
163
164         steps = np.array(sorted(values.keys()))
165         perfs = np.array([values[s] for s in steps])
166
167         # Power law model
168         def power_law(x, a, b, c):
169             return a - b * np.power(x, -c)
170
171         # Fit
172         params, _ = curve_fit(
173             power_law,
174             steps,
175             perfs,
176             p0=[perfs[-1] * 1.1, 0.1, 0.5],
177             maxfev=1000
178         )
179
180         # Predict at final step (e.g., 50 epochs)
181         final_step = 50
182         predicted = power_law(final_step, *params)
183
184         return predicted
185
186     except Exception:
187         return None
188
189 # Combine multiple pruners
190 class CombinedPruner(BasePruner):
191     """
192     Combine multiple pruning strategies.
193
194     Prunes if ANY pruner suggests pruning.
195     """
196

```



```

197     def __init__(self, pruners: List[BasePruner]):
198         self.pruners = pruners
199
200     def prune(self, study: optuna.Study, trial: optuna.trial.FrozenTrial) -> bool:
201         """Prune if any sub-pruner suggests pruning."""
202         return any(pruner.prune(study, trial) for pruner in self.pruners)
203
204     # Usage example
205     def create_study_with_advanced_pruning():
206         """Create study with sophisticated pruning."""
207
208         # Combine multiple pruning strategies
209         combined_pruner = CombinedPruner([
210             PerformanceBasedPruner(patience=5, percentile=30),
211             LearningCurveExtrapolator(min_steps=5),
212             optuna.pruners.HyperbandPruner(min_resource=1, max_resource=50,
213             reduction_factor=3)
214         ])
215
216         study = optuna.create_study(
217             direction='maximize',
218             sampler=optuna.samplers.TPESampler(multivariate=True),
219             pruner=combined_pruner
220         )
221
222         return study

```

Listing 10.11: Custom pruning strategies

10.14 Performance Analysis and Visualization

Effective visualization of HPO results helps identify patterns, understand hyperparameter importance, and guide future optimization efforts.

```

1 import optuna
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6 from typing import List, Dict
7 from scipy import stats
8
9 class HPOAnalyzer:
10     """
11     Comprehensive analysis of hyperparameter optimization results.
12
13     Features:
14     1. Parameter importance analysis
15     2. Correlation analysis
16     3. Learning curve analysis
17     4. Optimization history visualization
18     5. Hyperparameter interaction analysis
19     """
20
21     def __init__(self, study: optuna.Study):
22         self.study = study

```

```

23     self.df = study.trials_dataframe()
24
25     def analyze_all(self, output_dir: str = './hpo_analysis'):
26         """Run complete analysis pipeline."""
27         import os
28         os.makedirs(output_dir, exist_ok=True)
29
30         print("Running comprehensive HPO analysis...")
31
32         # 1. Parameter importance
33         self.plot_param_importance(f'{output_dir}/param_importance.png')
34
35         # 2. Optimization history
36         self.plot_optimization_history(f'{output_dir}/opt_history.png')
37
38         # 3. Parameter distributions
39         self.plot_param_distributions(f'{output_dir}/param_distributions.png')
40
41         # 4. Correlation matrix
42         self.plot_correlation_matrix(f'{output_dir}/correlation.png')
43
44         # 5. Parallel coordinates
45         self.plot_parallel_coordinates(f'{output_dir}/parallel_coords.png')
46
47         # 6. Learning curves
48         self.plot_learning_curves(f'{output_dir}/learning_curves.png')
49
50         # 7. Hyperparameter interactions
51         self.analyze_interactions(f'{output_dir}/interactions.png')
52
53         # 8. Statistical summary
54         summary = self.generate_statistical_summary()
55         with open(f'{output_dir}/summary.txt', 'w') as f:
56             f.write(summary)
57
58         print(f"Analysis complete. Results saved to {output_dir}")
59
60     def plot_param_importance(self, filepath: str):
61         """
62         Plot hyperparameter importance using fANOVA.
63
64         Importance quantifies how much each hyperparameter affects the objective.
65         """
66         try:
67             from optuna.importance import get_param_importances
68
69             importance = get_param_importances(self.study)
70
71             # Sort by importance
72             params = list(importance.keys())
73             values = [importance[p] for p in params]
74
75             # Create horizontal bar plot
76             plt.figure(figsize=(10, max(6, len(params) * 0.3)))
77             plt.barh(params, values)
78             plt.xlabel('Importance')
79             plt.ylabel('Parameter')
80             plt.title('Hyperparameter Importance (fANOVA)')

```

```

81         plt.tight_layout()
82         plt.savefig(filepath, dpi=300, bbox_inches='tight')
83         plt.close()
84
85         print(f"Parameter importance plot saved to {filepath}")
86         print("\nTop 5 most important parameters:")
87         for param, imp in sorted(importance.items(), key=lambda x: x[1], reverse=
True)[:5]:
88             print(f" {param}: {imp:.4f}")
89
90     except ImportError:
91         print("Install scikit-learn for parameter importance: pip install scikit-
learn")
92
93 def plot_optimization_history(self, filepath: str):
94     """
95     Plot optimization history showing improvement over time.
96     """
97     # Get completed trials
98     completed_trials = [t for t in self.study.trials
99                         if t.value is not None]
100
101     if len(completed_trials) == 0:
102         print("No completed trials to plot")
103         return
104
105     trial_numbers = [t.number for t in completed_trials]
106     values = [t.value for t in completed_trials]
107
108     # Calculate best value so far
109     if self.study.direction == optuna.study.StudyDirection.MAXIMIZE:
110         best_so_far = np.maximum.accumulate(values)
111     else:
112         best_so_far = np.minimum.accumulate(values)
113
114     fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 8))
115
116     # Plot 1: All trial values
117     ax1.scatter(trial_numbers, values, alpha=0.5, s=20, label='Trial value')
118     ax1.plot(trial_numbers, best_so_far, 'r-', linewidth=2, label='Best value')
119     ax1.set_xlabel('Trial Number')
120     ax1.set_ylabel('Objective Value')
121     ax1.set_title('Optimization History')
122     ax1.legend()
123     ax1.grid(True, alpha=0.3)
124
125     # Plot 2: Improvement rate
126     improvements = np.diff(best_so_far)
127     ax2.plot(trial_numbers[1:], improvements, 'g-', alpha=0.7)
128     ax2.axhline(y=0, color='r', linestyle='--', alpha=0.5)
129     ax2.set_xlabel('Trial Number')
130     ax2.set_ylabel('Improvement')
131     ax2.set_title('Improvement Rate')
132     ax2.grid(True, alpha=0.3)
133
134     plt.tight_layout()
135     plt.savefig(filepath, dpi=300, bbox_inches='tight')
136     plt.close()

```

```

137     print(f"Optimization history plot saved to {filepath}")
138
139
140 def plot_param_distributions(self, filepath: str):
141     """
142     Plot distribution of hyperparameter values for top-performing trials.
143     """
144     # Get param columns
145     param_cols = [col for col in self.df.columns if col.startswith('params_')]
146
147     if len(param_cols) == 0:
148         print("No parameters to plot")
149         return
150
151     # Filter top 20% trials
152     if self.study.direction == optuna.study.StudyDirection.MAXIMIZE:
153         top_trials = self.df.nlargest(max(1, len(self.df) // 5), 'value')
154     else:
155         top_trials = self.df.nsmallest(max(1, len(self.df) // 5), 'value')
156
157     # Create subplots
158     n_params = len(param_cols)
159     n_cols = 3
160     n_rows = (n_params + n_cols - 1) // n_cols
161
162     fig, axes = plt.subplots(n_rows, n_cols, figsize=(15, n_rows * 3))
163     axes = axes.flatten() if n_params > 1 else [axes]
164
165     for idx, col in enumerate(param_cols):
166         ax = axes[idx]
167         param_name = col.replace('params_', '')
168
169         # All trials
170         all_values = self.df[col].dropna()
171         top_values = top_trials[col].dropna()
172
173         if len(all_values) == 0:
174             continue
175
176         # Check if numeric
177         if pd.api.types.is_numeric_dtype(all_values):
178             # Histogram
179             ax.hist(all_values, bins=20, alpha=0.5, label='All trials', density=
True)
180             ax.hist(top_values, bins=20, alpha=0.5, label='Top 20%', density=True)
181             ax.set_xlabel('Value')
182             ax.set_ylabel('Density')
183         else:
184             # Bar plot for categorical
185             all_counts = all_values.value_counts()
186             top_counts = top_values.value_counts()
187
188             x = np.arange(len(all_counts))
189             width = 0.35
190
191             ax.bar(x - width/2, all_counts.values, width, label='All trials',
alpha=0.5)
192             ax.bar(x + width/2, top_counts.values, width, label='Top 20%', alpha

```

```

=0.5)
193         ax.set_xticks(x)
194         ax.set_xticklabels(all_counts.index, rotation=45, ha='right')
195         ax.set_ylabel('Count')
196
197         ax.set_title(param_name)
198         ax.legend()
199         ax.grid(True, alpha=0.3)
200
201     # Hide empty subplots
202     for idx in range(len(param_cols), len(axes)):
203         axes[idx].set_visible(False)
204
205     plt.tight_layout()
206     plt.savefig(filepath, dpi=300, bbox_inches='tight')
207     plt.close()
208
209     print(f"Parameter distributions plot saved to {filepath}")
210
211 def plot_correlation_matrix(self, filepath: str):
212     """
213     Plot correlation between hyperparameters and objective value.
214     """
215     # Get numeric param columns
216     param_cols = [col for col in self.df.columns if col.startswith('params_')]
217     numeric_cols = [col for col in param_cols if pd.api.types.is_numeric_dtype(
218 self.df[col])]
219
220     if len(numeric_cols) == 0:
221         print("No numeric parameters for correlation analysis")
222         return
223
224     # Include objective value
225     cols_to_analyze = numeric_cols + ['value']
226
227     # Compute correlation matrix
228     corr_data = self.df[cols_to_analyze].dropna()
229
230     if len(corr_data) < 2:
231         print("Not enough data for correlation analysis")
232         return
233
234     corr_matrix = corr_data.corr()
235
236     # Plot heatmap
237     plt.figure(figsize=(max(10, len(cols_to_analyze) * 0.8),
238                         max(8, len(cols_to_analyze) * 0.7)))
239
240     # Shorten column names
241     short_names = [col.replace('params_', '') for col in cols_to_analyze]
242     corr_matrix.columns = short_names
243     corr_matrix.index = short_names
244
245     sns.heatmap(corr_matrix, annot=True, fmt='.2f', cmap='RdBu_r',
246                 center=0, vmin=-1, vmax=1, square=True,
247                 linewidths=0.5, cbar_kws={"shrink": 0.8})
248
249     plt.title('Hyperparameter Correlation Matrix')

```

```

249 plt.tight_layout()
250 plt.savefig(filepath, dpi=300, bbox_inches='tight')
251 plt.close()
252
253 print(f"Correlation matrix saved to {filepath}")
254
255 # Print strongest correlations with objective
256 value_corr = corr_matrix['value'].drop('value').abs().sort_values(ascending=
False)
257 print("\nStrongest correlations with objective:")
258 for param, corr in value_corr.head(5).items():
259     print(f" {param}: {corr:.4f}")
260
261 def plot_parallel_coordinates(self, filepath: str):
262     """
263     Parallel coordinates plot for top trials.
264     """
265     # Get top 20 trials
266     if self.study.direction == optuna.study.StudyDirection.MAXIMIZE:
267         top_df = self.df.nlargest(20, 'value')
268     else:
269         top_df = self.df.nsmallest(20, 'value')
270
271     # Get numeric param columns
272     param_cols = [col for col in top_df.columns if col.startswith('params_')]
273     numeric_cols = [col for col in param_cols if pd.api.types.is_numeric_dtype(
top_df[col])]
274
275     if len(numeric_cols) == 0:
276         print("No numeric parameters for parallel coordinates")
277         return
278
279     # Normalize parameters to [0, 1]
280     plot_data = top_df[numeric_cols + ['value']].copy()
281
282     for col in plot_data.columns:
283         min_val = plot_data[col].min()
284         max_val = plot_data[col].max()
285         if max_val > min_val:
286             plot_data[col] = (plot_data[col] - min_val) / (max_val - min_val)
287
288     # Plot
289     fig, ax = plt.subplots(figsize=(14, 6))
290
291     # Shorten names
292     short_names = [col.replace('params_', '') if col != 'value' else 'objective'
293                    for col in plot_data.columns]
294     plot_data.columns = short_names
295
296     # Plot lines
297     for idx, row in plot_data.iterrows():
298         alpha = 0.6
299         color = plt.cm.viridis(row['objective'])
300         ax.plot(range(len(short_names)), row.values, alpha=alpha, color=color)
301
302     # Set labels
303     ax.set_xticks(range(len(short_names)))
304     ax.set_xticklabels(short_names, rotation=45, ha='right')

```

```

305     ax.set_ylabel('Normalized Value')
306     ax.set_title('Parallel Coordinates Plot (Top 20 Trials)')
307     ax.grid(True, alpha=0.3, axis='x')
308
309     plt.tight_layout()
310     plt.savefig(filepath, dpi=300, bbox_inches='tight')
311     plt.close()
312
313     print(f"Parallel coordinates plot saved to {filepath}")
314
315     def plot_learning_curves(self, filepath: str):
316         """
317         Plot learning curves for top trials.
318         """
319         # Get trials with intermediate values
320         trials_with_curves = [t for t in self.study.trials
321                               if len(t.intermediate_values) > 0]
322
323         if len(trials_with_curves) == 0:
324             print("No trials with intermediate values for learning curves")
325             return
326
327         # Sort by final value
328         if self.study.direction == optuna.study.StudyDirection.MAXIMIZE:
329             trials_with_curves.sort(key=lambda t: t.value if t.value is not None else
330                                     float('-inf'),
331                                     reverse=True)
332         else:
333             trials_with_curves.sort(key=lambda t: t.value if t.value is not None else
334                                     float('inf'))
335
336         # Plot top 10
337         top_trials = trials_with_curves[:10]
338
339         plt.figure(figsize=(12, 6))
340
341         for idx, trial in enumerate(top_trials):
342             steps = sorted(trial.intermediate_values.keys())
343             values = [trial.intermediate_values[s] for s in steps]
344
345             label = f"Trial {trial.number} (final: {trial.value:.4f})"
346             plt.plot(steps, values, marker='o', label=label, alpha=0.7)
347
348         plt.xlabel('Step (Epoch)')
349         plt.ylabel('Intermediate Value')
350         plt.title('Learning Curves (Top 10 Trials)')
351         plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
352         plt.grid(True, alpha=0.3)
353         plt.tight_layout()
354         plt.savefig(filepath, dpi=300, bbox_inches='tight')
355         plt.close()
356
357         print(f"Learning curves plot saved to {filepath}")
358
359     def analyze_interactions(self, filepath: str):
360         """
361         Analyze and visualize hyperparameter interactions.
362         """

```

```

361     # Get numeric param columns
362     param_cols = [col for col in self.df.columns if col.startswith('params_')]
363     numeric_cols = [col for col in param_cols if pd.api.types.is_numeric_dtype(
self.df[col])]
364
365     if len(numeric_cols) < 2:
366         print("Need at least 2 numeric parameters for interaction analysis")
367         return
368
369     # Plot pairwise scatter plots for top parameters
370     n_params = min(4, len(numeric_cols))
371     top_params = numeric_cols[:n_params]
372
373     fig, axes = plt.subplots(n_params, n_params, figsize=(15, 15))
374
375     for i, param1 in enumerate(top_params):
376         for j, param2 in enumerate(top_params):
377             ax = axes[i, j]
378
379             if i == j:
380                 # Diagonal: histogram
381                 ax.hist(self.df[param1].dropna(), bins=20, alpha=0.7)
382                 ax.set_ylabel('Count')
383             else:
384                 # Off-diagonal: scatter plot colored by objective
385                 data = self.df[[param1, param2, 'value']].dropna()
386
387                 scatter = ax.scatter(data[param2], data[param1],
388                                     c=data['value'], cmap='viridis',
389                                     alpha=0.6, s=30)
390
391                 # Labels
392                 if j == 0:
393                     ax.set_ylabel(param1.replace('params_', ''))
394                 if i == n_params - 1:
395                     ax.set_xlabel(param2.replace('params_', ''))
396
397                 ax.grid(True, alpha=0.3)
398
399     # Add colorbar
400     cbar = plt.colorbar(scatter, ax=axes, orientation='vertical',
401                         fraction=0.046, pad=0.04)
402     cbar.set_label('Objective Value')
403
404     plt.suptitle('Hyperparameter Interaction Analysis', y=0.995)
405     plt.tight_layout()
406     plt.savefig(filepath, dpi=300, bbox_inches='tight')
407     plt.close()
408
409     print(f"Interaction analysis plot saved to {filepath}")
410
411     def generate_statistical_summary(self) -> str:
412         """Generate comprehensive statistical summary."""
413         summary = []
414         summary.append("=" * 60)
415         summary.append("Hyperparameter Optimization Summary")
416         summary.append("=" * 60)
417         summary.append(f"\nStudy name: {self.study.study_name}")

```



```

418     summary.append(f"Direction: {self.study.direction.name}")
419     summary.append(f"\nTotal trials: {len(self.study.trials)}")
420
421     # Count trial states
422     from optuna.trial import TrialState
423     states = {}
424     for trial in self.study.trials:
425         state = trial.state.name
426         states[state] = states.get(state, 0) + 1
427
428     summary.append("\nTrial states:")
429     for state, count in states.items():
430         summary.append(f"    {state}: {count}")
431
432     # Best trial
433     if self.study.best_trial is not None:
434         summary.append(f"\nBest trial: {self.study.best_trial.number}")
435         summary.append(f"Best value: {self.study.best_value:.6f}")
436         summary.append("\nBest hyperparameters:")
437         for key, value in self.study.best_params.items():
438             summary.append(f"    {key}: {value}")
439
440     # Value statistics
441     completed_values = [t.value for t in self.study.trials if t.value is not None]
442     if len(completed_values) > 0:
443         summary.append(f"\nObjective value statistics:")
444         summary.append(f"    Mean: {np.mean(completed_values):.6f}")
445         summary.append(f"    Std: {np.std(completed_values):.6f}")
446         summary.append(f"    Min: {np.min(completed_values):.6f}")
447         summary.append(f"    Max: {np.max(completed_values):.6f}")
448         summary.append(f"    Median: {np.median(completed_values):.6f}")
449
450     # Pruning statistics
451     pruned_count = len([t for t in self.study.trials
452                         if t.state == TrialState.PRUNED])
453     if pruned_count > 0:
454         summary.append(f"\nPruning statistics:")
455         summary.append(f"    Pruned trials: {pruned_count}")
456         summary.append(f"    Pruning rate: {pruned_count / len(self.study.trials) *
100:.1f}%")
457
458     # Estimate compute saved
459     total_complete = len([t for t in self.study.trials
460                           if t.state == TrialState.COMPLETE])
461     if total_complete > 0:
462         avg_steps_complete = np.mean([
463             max(t.intermediate_values.keys()) if len(t.intermediate_values) >
0 else 0
464             for t in self.study.trials if t.state == TrialState.COMPLETE
465         ])
466     avg_steps_pruned = np.mean([
467         max(t.intermediate_values.keys()) if len(t.intermediate_values) >
0 else 0
468         for t in self.study.trials if t.state == TrialState.PRUNED
469     ])
470
471     if avg_steps_complete > 0:

```

```

473         compute_saved = (1 - avg_steps_pruned / avg_steps_complete) * 100
474         summary.append(f"    Estimated compute saved: {compute_saved:.1f}%")
475
476     summary.append("\n" + "=" * 60)
477
478     return "\n".join(summary)
479
480 # Usage example
481 def analyze_study_results(study: optuna.Study):
482     """Comprehensive analysis of HPO study."""
483     analyzer = HPOAnalyzer(study)
484     analyzer.analyze_all(output_dir='./hpo_analysis')
485
486     # Print summary to console
487     print("\n" + analyzer.generate_statistical_summary())

```

Listing 10.12: Comprehensive HPO analysis and visualization

10.15 HPO in Production Environments

Deploying hyperparameter optimization in production requires additional considerations beyond research settings: automation, monitoring, cost management, and integration with existing ML pipelines.

10.15.1 Automated HPO Pipelines

```

1  import optuna
2  from pathlib import Path
3  import json
4  import logging
5  from typing import Dict, Any, Optional
6  from dataclasses import dataclass, asdict
7  import yaml
8  from datetime import datetime
9
10 @dataclass
11 class HPOConfig:
12     """Configuration for HPO pipeline."""
13     study_name: str
14     direction: str # 'maximize' or 'minimize'
15     n_trials: int
16     timeout_hours: Optional[float] = None
17     n_jobs: int = 1
18     storage_url: str = "sqlite:///hpo_studies.db"
19
20     # Sampler config
21     sampler_name: str = "TPE"
22     sampler_params: Dict = None
23
24     # Pruner config
25     pruner_name: str = "Hyperband"
26     pruner_params: Dict = None
27
28     # Resource allocation
29     gpu_per_trial: int = 1

```

```

30     cpu_per_trial: int = 4
31     memory_per_trial_gb: int = 16
32
33     # Logging
34     log_level: str = "INFO"
35     save_frequency: int = 5 # Save checkpoint every N trials
36
37     def __post_init__(self):
38         if self.sampler_params is None:
39             self.sampler_params = {}
40         if self.pruner_params is None:
41             self.pruner_params = {}
42
43 class ProductionHPOPipeline:
44     """
45     Production-grade HPO pipeline with comprehensive features.
46
47     Features:
48     - Persistent storage
49     - Automatic checkpointing
50     - Resource management
51     - Failure handling and recovery
52     - Performance monitoring
53     - Result archiving
54     """
55
56     def __init__(self, config: HPOConfig):
57         self.config = config
58         self.setup_logging()
59         self.study = None
60
61     def setup_logging(self):
62         """Configure logging."""
63         logging.basicConfig(
64             level=getattr(logging, self.config.log_level),
65             format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
66             handlers=[
67                 logging.FileHandler(f'hpo_{self.config.study_name}.log'),
68                 logging.StreamHandler()
69             ]
70         )
71         self.logger = logging.getLogger(__name__)
72
73     def create_study(self) -> optuna.Study:
74         """Create or load existing study."""
75         self.logger.info(f"Creating/loading study: {self.config.study_name}")
76
77         # Create sampler
78         sampler = self._create_sampler()
79
80         # Create pruner
81         pruner = self._create_pruner()
82
83         # Create storage
84         storage = optuna.storages.RDBStorage(
85             url=self.config.storage_url,
86             engine_kwargs={"pool_pre_ping": True} # Verify connections
87         )

```

```

88
89     # Create or load study
90     study = optuna.create_study(
91         study_name=self.config.study_name,
92         storage=storage,
93         load_if_exists=True,
94         direction=self.config.direction,
95         sampler=sampler,
96         pruner=pruner
97     )
98
99     self.logger.info(f"Study created/loaded. Existing trials: {len(study.trials)}")
100
101     return study
102
103 def _create_sampler(self):
104     """Create sampler from config."""
105     if self.config.sampler_name.lower() == "tpe":
106         from optuna.samplers import TPESampler
107         return TPESampler(**self.config.sampler_params)
108     elif self.config.sampler_name.lower() == "cmaes":
109         from optuna.samplers import CmaEsSampler
110         return TPESampler(**self.config.sampler_params)
111     elif self.config.sampler_name.lower() == "random":
112         from optuna.samplers import RandomSampler
113         return RandomSampler(**self.config.sampler_params)
114     else:
115         raise ValueError(f"Unknown sampler: {self.config.sampler_name}")
116
117 def _create_pruner(self):
118     """Create pruner from config."""
119     if self.config.pruner_name.lower() == "hyperband":
120         from optuna.pruners import HyperbandPruner
121         return HyperbandPruner(**self.config.pruner_params)
122     elif self.config.pruner_name.lower() == "median":
123         from optuna.pruners import MedianPruner
124         return MedianPruner(**self.config.pruner_params)
125     elif self.config.pruner_name.lower() == "none":
126         return optuna.pruners.NopPruner()
127     else:
128         raise ValueError(f"Unknown pruner: {self.config.pruner_name}")
129
130 def run(self, objective_func, callbacks: list = None):
131     """
132     Run HPO pipeline.
133
134     Args:
135         objective_func: Objective function to optimize
136         callbacks: List of callbacks for monitoring
137     """
138     self.study = self.create_study()
139
140     # Add default callbacks
141     if callbacks is None:
142         callbacks = []
143
144     callbacks.extend([

```

```

145         self._create_checkpoint_callback(),
146         self._create_monitoring_callback()
147     ])
148
149     # Calculate timeout
150     timeout = None
151     if self.config.timeout_hours is not None:
152         timeout = self.config.timeout_hours * 3600
153
154     try:
155         self.logger.info(f"Starting optimization: {self.config.n_trials} trials")
156         self.logger.info(f"Timeout: {self.config.timeout_hours} hours" if timeout
157         else "No timeout")
158
159         self.study.optimize(
160             objective_func,
161             n_trials=self.config.n_trials,
162             timeout=timeout,
163             n_jobs=self.config.n_jobs,
164             callbacks=callbacks,
165             show_progress_bar=True
166         )
167
168         self.logger.info("Optimization completed successfully")
169
170     except KeyboardInterrupt:
171         self.logger.warning("Optimization interrupted by user")
172     except Exception as e:
173         self.logger.error(f"Optimization failed with error: {e}", exc_info=True)
174         raise
175     finally:
176         # Always save final results
177         self.save_results()
178
179     def _create_checkpoint_callback(self):
180         """Create callback for periodic checkpointing."""
181         def checkpoint_callback(study, trial):
182             if trial.number % self.config.save_frequency == 0:
183                 self.logger.info(f"Checkpoint at trial {trial.number}")
184                 self.save_results()
185
186         return checkpoint_callback
187
188     def _create_monitoring_callback(self):
189         """Create callback for monitoring progress."""
190         def monitoring_callback(study, trial):
191             if trial.state == optuna.trial.TrialState.COMPLETE:
192                 self.logger.info(
193                     f"Trial {trial.number} completed: "
194                     f"value={trial.value:.6f}, "
195                     f"params={trial.params}"
196                 )
197
198                 # Log best so far
199                 best_value = study.best_value
200                 self.logger.info(f"Best value so far: {best_value:.6f}")
201
202             elif trial.state == optuna.trial.TrialState.PRUNED:

```

```

202         self.logger.info(f"Trial {trial.number} pruned")
203
204         elif trial.state == optuna.trial.TrialState.FAIL:
205             self.logger.error(f"Trial {trial.number} failed")
206
207     return monitoring_callback
208
209 def save_results(self):
210     """Save study results to disk."""
211     if self.study is None:
212         return
213
214     output_dir = Path(f"results_{self.config.study_name}")
215     output_dir.mkdir(exist_ok=True)
216
217     # Save best parameters
218     best_params_file = output_dir / "best_params.json"
219     with open(best_params_file, 'w') as f:
220         json.dump(self.study.best_params, f, indent=2)
221
222     self.logger.info(f"Best parameters saved to {best_params_file}")
223
224     # Save full study results
225     df = self.study.trials_dataframe()
226     results_file = output_dir / "all_trials.csv"
227     df.to_csv(results_file, index=False)
228
229     self.logger.info(f"All trials saved to {results_file}")
230
231     # Save study summary
232     summary = {
233         'study_name': self.config.study_name,
234         'n_trials': len(self.study.trials),
235         'best_value': self.study.best_value,
236         'best_params': self.study.best_params,
237         'direction': self.config.direction,
238         'timestamp': datetime.now().isoformat()
239     }
240
241     summary_file = output_dir / "summary.json"
242     with open(summary_file, 'w') as f:
243         json.dump(summary, f, indent=2)
244
245     self.logger.info(f"Summary saved to {summary_file}")
246
247     # Generate visualizations
248     try:
249         analyzer = HPOAnalyzer(self.study)
250         analyzer.analyze_all(output_dir=str(output_dir / "analysis"))
251     except Exception as e:
252         self.logger.warning(f"Could not generate analysis: {e}")
253
254     # Configuration management
255     class HPOConfigManager:
256         """Manage HPO configurations."""
257
258         @staticmethod
259         def load_from_yaml(filepath: str) -> HPOConfig:

```

```

260     """Load configuration from YAML file."""
261     with open(filepath, 'r') as f:
262         config_dict = yaml.safe_load(f)
263
264     return HPOConfig(**config_dict)
265
266 @staticmethod
267 def save_to_yaml(config: HPOConfig, filepath: str):
268     """Save configuration to YAML file."""
269     with open(filepath, 'w') as f:
270         yaml.dump(asdict(config), f, default_flow_style=False)
271
272 @staticmethod
273 def create_default_config(study_name: str) -> HPOConfig:
274     """Create default configuration."""
275     return HPOConfig(
276         study_name=study_name,
277         direction="maximize",
278         n_trials=100,
279         timeout_hours=24,
280         n_jobs=4,
281         sampler_name="TPE",
282         sampler_params={
283             "n_startup_trials": 10,
284             "multivariate": True
285         },
286         pruner_name="Hyperband",
287         pruner_params={
288             "min_resource": 1,
289             "max_resource": 50,
290             "reduction_factor": 3
291         }
292     )
293
294 # Usage example
295 def run_production_hpo():
296     """
297     Run production HPO pipeline.
298
299     This example shows how to:
300     1. Load configuration from file
301     2. Define objective function
302     3. Run HPO with monitoring
303     4. Save and analyze results
304     """
305     # Option 1: Load config from YAML
306     config = HPOConfigManager.load_from_yaml('hpo_config.yaml')
307
308     # Option 2: Create programmatically
309     # config = HPOConfigManager.create_default_config("my_study")
310
311     # Create pipeline
312     pipeline = ProductionHPOPipeline(config)
313
314     # Define objective
315     def objective(trial):
316         # Your model training code here
317         config = {

```

```

318         'lr': trial.suggest_float('lr', 1e-5, 1e-1, log=True),
319         'hidden_dim': trial.suggest_categorical('hidden_dim', [128, 256, 512]),
320         'dropout': trial.suggest_float('dropout', 0.0, 0.5)
321     }
322
323     model = train_model(config)
324     accuracy = evaluate_model(model)
325
326     return accuracy
327
328     # Run optimization
329     pipeline.run(objective)
330
331     # Access results
332     print(f"Best value: {pipeline.study.best_value}")
333     print(f"Best params: {pipeline.study.best_params}")

```

Listing 10.13: Production-grade automated HPO pipeline

10.15.2 HPO Monitoring and Alerting

```

1  import optuna
2  from typing import Callable, Dict, Any
3  import smtplib
4  from email.mime.text import MIMEText
5  from email.mime.multipart import MIMEMultipart
6  import requests
7  from datetime import datetime, timedelta
8  import numpy as np
9
10 class HPOMonitor:
11     """
12     Monitor HPO progress and send alerts.
13
14     Alerts for:
15     - Optimization stalling (no improvement)
16     - High failure rate
17     - Resource exhaustion
18     - Completion or significant milestones
19     """
20
21     def __init__(self,
22                 study: optuna.Study,
23                 email_config: Dict[str, str] = None,
24                 slack_webhook: str = None):
25         self.study = study
26         self.email_config = email_config
27         self.slack_webhook = slack_webhook
28
29         self.last_improvement = datetime.now()
30         self.failure_count = 0
31         self.total_trials = 0
32
33     def check_progress(self) -> Dict[str, Any]:
34         """Check optimization progress."""
35         status = {
36             'n_trials': len(self.study.trials),

```



```

37         'best_value': self.study.best_value if self.study.best_trial else None,
38         'n_complete': len([t for t in self.study.trials
39                             if t.state == optuna.trial.TrialState.COMPLETE]),
40         'n_pruned': len([t for t in self.study.trials
41                           if t.state == optuna.trial.TrialState.PRUNED]),
42         'n_failed': len([t for t in self.study.trials
43                           if t.state == optuna.trial.TrialState.FAIL]),
44     }
45
46     # Calculate improvement rate
47     if status['n_complete'] > 0:
48         recent_trials = sorted(
49             [t for t in self.study.trials if t.state == optuna.trial.TrialState.
COMPLETE],
50             key=lambda t: t.datetime_complete
51         )
52
53         if len(recent_trials) >= 10:
54             recent_10_best = max(t.value for t in recent_trials[-10:])
55             older_best = max(t.value for t in recent_trials[:-10]) if len(
recent_trials) > 10 else 0
56
57             status['improvement_rate'] = (recent_10_best - older_best) /
older_best if older_best > 0 else 0
58         else:
59             status['improvement_rate'] = None
60
61     return status
62
63     def check_stalling(self, patience_hours: float = 4.0) -> bool:
64         """Check if optimization is stalling (no improvement)."""
65         if self.study.best_trial is None:
66             return False
67
68         # Time since last improvement
69         last_best_time = self.study.best_trial.datetime_complete
70         if last_best_time is None:
71             return False
72
73         time_since_improvement = datetime.now() - last_best_time
74         is_stalling = time_since_improvement > timedelta(hours=patience_hours)
75
76         return is_stalling
77
78     def check_high_failure_rate(self, threshold: float = 0.3) -> bool:
79         """Check if failure rate is too high."""
80         total_trials = len(self.study.trials)
81         if total_trials < 10:
82             return False
83
84         failed_trials = len([t for t in self.study.trials
85                               if t.state == optuna.trial.TrialState.FAIL])
86
87         failure_rate = failed_trials / total_trials
88         return failure_rate > threshold
89
90     def send_alert(self, subject: str, message: str):
91         """Send alert via configured channels."""

```

```

92     # Email alert
93     if self.email_config:
94         self._send_email_alert(subject, message)
95
96     # Slack alert
97     if self.slack_webhook:
98         self._send_slack_alert(subject, message)
99
100 def _send_email_alert(self, subject: str, message: str):
101     """Send email alert."""
102     try:
103         msg = MIMEMultipart()
104         msg['From'] = self.email_config['from']
105         msg['To'] = self.email_config['to']
106         msg['Subject'] = f"[HPO Alert] {subject}"
107
108         msg.attach(MIMEText(message, 'plain'))
109
110         server = smtplib.SMTP(self.email_config['smtp_server'],
111                               self.email_config['smtp_port'])
112         server.starttls()
113         server.login(self.email_config['username'],
114                     self.email_config['password'])
115         server.send_message(msg)
116         server.quit()
117
118         print(f"Email alert sent: {subject}")
119
120     except Exception as e:
121         print(f"Failed to send email alert: {e}")
122
123 def _send_slack_alert(self, subject: str, message: str):
124     """Send Slack alert."""
125     try:
126         payload = {
127             'text': f"*{subject}*\n{message}"
128         }
129
130         response = requests.post(self.slack_webhook, json=payload)
131         response.raise_for_status()
132
133         print(f"Slack alert sent: {subject}")
134
135     except Exception as e:
136         print(f"Failed to send Slack alert: {e}")
137
138 def create_monitoring_callback(self,
139                               check_interval: int = 10,
140                               stalling_patience_hours: float = 4.0) -> Callable:
141     """Create callback for continuous monitoring."""
142
143     def callback(study, trial):
144         # Check every N trials
145         if trial.number % check_interval != 0:
146             return
147
148         # Check for stalling
149         if self.check_stalling(patience_hours=stalling_patience_hours):

```

```

150         self.send_alert(
151             "Optimization Stalling",
152             f"No improvement in {stalling_patience_hours} hours.\n"
153             f"Current best value: {study.best_value:.6f}\n"
154             f"Trials completed: {len(study.trials)}"
155         )
156
157     # Check failure rate
158     if self.check_high_failure_rate():
159         status = self.check_progress()
160         self.send_alert(
161             "High Failure Rate",
162             f"Failure rate above threshold.\n"
163             f"Failed trials: {status['n_failed']}/{status['n_trials']}\n"
164             f"Check logs for error patterns."
165         )
166
167     # Milestone alerts
168     if trial.number in [10, 50, 100, 200]:
169         status = self.check_progress()
170         self.send_alert(
171             f"Milestone: {trial.number} Trials",
172             f"Current best value: {study.best_value:.6f}\n"
173             f"Complete: {status['n_complete']}\n"
174             f"Pruned: {status['n_pruned']}\n"
175             f"Failed: {status['n_failed']}"
176         )
177
178     return callback
179
180 # Usage with production pipeline
181 def run_monitored_hpo():
182     """Run HPO with monitoring and alerting."""
183
184     # Email configuration
185     email_config = {
186         'from': 'hpo@company.com',
187         'to': 'ml-team@company.com',
188         'smtp_server': 'smtp.gmail.com',
189         'smtp_port': 587,
190         'username': 'your_email@company.com',
191         'password': 'your_password'
192     }
193
194     # Slack webhook
195     slack_webhook = "https://hooks.slack.com/services/YOUR/WEBHOOK/URL"
196
197     # Create study
198     study = optuna.create_study(direction='maximize')
199
200     # Create monitor
201     monitor = HPOMonitor(
202         study=study,
203         email_config=email_config,
204         slack_webhook=slack_webhook
205     )
206
207     # Create monitoring callback

```

```

208     monitoring_callback = monitor.create_monitoring_callback(
209         check_interval=5,
210         stalling_patience_hours=2.0
211     )
212
213     # Run optimization with monitoring
214     study.optimize(
215         objective_function,
216         n_trials=100,
217         callbacks=[monitoring_callback]
218     )
219
220     # Send completion alert
221     monitor.send_alert(
222         "Optimization Complete",
223         f"Study completed.\n"
224         f"Best value: {study.best_value:.6f}\n"
225         f"Best params: {study.best_params}"
226     )

```

Listing 10.14: Monitoring and alerting for production HPO

10.15.3 Resource Management and Cost Optimization

```

1  import optuna
2  from typing import Dict, List
3  import time
4  from dataclasses import dataclass
5
6  @dataclass
7  class ResourceBudget:
8      """Resource budget for HPO."""
9      max_compute_hours: float
10     max_cost_usd: float
11     max_trials: int
12     gpu_cost_per_hour: float = 2.50 # AWS p3.2xlarge approximate cost
13     cpu_cost_per_hour: float = 0.10 # AWS c5.xlarge approximate cost
14
15  class ResourceManager:
16      """
17      Manage computational resources and costs for HPO.
18
19      Features:
20      - Track resource consumption
21      - Enforce budget constraints
22      - Optimize resource allocation
23      - Predict completion time and cost
24      """
25
26     def __init__(self, budget: ResourceBudget):
27         self.budget = budget
28         self.total_compute_hours = 0.0
29         self.total_cost = 0.0
30         self.trial_times: List[float] = []
31         self.start_time = time.time()
32
33     def record_trial(self, trial_duration_sec: float, used_gpu: bool = True):

```

```

34     """Record trial resource usage."""
35     hours = trial_duration_sec / 3600
36     self.total_compute_hours += hours
37
38     cost_per_hour = self.budget.gpu_cost_per_hour if used_gpu else self.budget.
cpu_cost_per_hour
39     self.total_cost += hours * cost_per_hour
40
41     self.trial_times.append(trial_duration_sec)
42
43 def check_budget(self) -> Dict[str, Any]:
44     """Check current budget status."""
45     return {
46         'compute_hours_used': self.total_compute_hours,
47         'compute_hours_remaining': self.budget.max_compute_hours - self.
total_compute_hours,
48         'cost_used': self.total_cost,
49         'cost_remaining': self.budget.max_cost_usd - self.total_cost,
50         'budget_exhausted': (
51             self.total_compute_hours >= self.budget.max_compute_hours or
52             self.total_cost >= self.budget.max_cost_usd
53         )
54     }
55
56 def estimate_remaining_trials(self) -> int:
57     """Estimate how many more trials can be run."""
58     if len(self.trial_times) == 0:
59         return self.budget.max_trials
60
61     avg_trial_time_hours = np.mean(self.trial_times) / 3600
62
63     # Based on compute budget
64     trials_by_compute = int(
65         (self.budget.max_compute_hours - self.total_compute_hours) /
avg_trial_time_hours
66     )
67
68     # Based on cost budget
69     avg_cost_per_trial = self.total_cost / len(self.trial_times)
70     trials_by_cost = int(
71         (self.budget.max_cost_usd - self.total_cost) / avg_cost_per_trial
72     )
73
74     # Based on trial limit
75     trials_by_limit = self.budget.max_trials - len(self.trial_times)
76
77     return max(0, min(trials_by_compute, trials_by_cost, trials_by_limit))
78
79 def create_budget_callback(self) -> Callable:
80     """Create callback to enforce budget constraints."""
81
82     def callback(study, trial):
83         # Record trial duration
84         if trial.duration is not None:
85             self.record_trial(trial.duration.total_seconds())
86
87         # Check budget
88         status = self.check_budget()

```

```

89         if status['budget_exhausted']:
90             print(f"\nBudget exhausted:")
91             print(f"   Compute hours: {self.total_compute_hours:.2f}/{self.budget.
92 max_compute_hours}")
93             print(f"   Cost: ${self.total_cost:.2f}/{self.budget.max_cost_usd}")
94             print(f"   Trials: {len(self.trial_times)}/{self.budget.max_trials}")
95             study.stop()
96
97         # Print status periodically
98         if trial.number % 10 == 0:
99             remaining = self.estimate_remaining_trials()
100             print(f"\nResource Status (Trial {trial.number}):")
101             print(f"   Compute used: {self.total_compute_hours:.2f}h / {self.budget
102 .max_compute_hours}h")
103             print(f"   Cost: ${self.total_cost:.2f} / ${self.budget.max_cost_usd}")
104             print(f"   Estimated remaining trials: {remaining}")
105
106         return callback
107
108 # Dynamic resource allocation
109 class DynamicResourceAllocator:
110     """
111     Dynamically allocate resources based on trial performance.
112
113     Strategy:
114     - Start with small resource allocation
115     - Increase resources for promising trials
116     - Decrease resources for unpromising trials
117     """
118     def __init__(self, min_epochs: int = 5, max_epochs: int = 50):
119         self.min_epochs = min_epochs
120         self.max_epochs = max_epochs
121         self.trial_history = []
122
123     def suggest_resources(self, trial: optuna.Trial, current_step: int) -> Dict[str,
124 int]:
125         """
126         Suggest resource allocation for trial.
127
128         Returns:
129         Dict with 'epochs', 'batch_size', etc.
130         """
131         # Start with minimum resources
132         if current_step == 0:
133             return {'epochs': self.min_epochs}
134
135         # Get performance at current step
136         if current_step not in trial.intermediate_values:
137             return {'epochs': self.min_epochs}
138
139         current_performance = trial.intermediate_values[current_step]
140
141         # Compare with historical performance at same step
142         historical_performance = [
143             t['performance'][current_step]
144             for t in self.trial_history

```

```

144         if current_step in t['performance']
145     ]
146
147     if len(historical_performance) < 5:
148         # Not enough history, use default
149         return {'epochs': self.min_epochs * 2}
150
151     # Calculate percentile
152     percentile = stats.percentileofscore(historical_performance,
153                                         current_performance)
154
155     # Allocate more epochs if performing well
156     if percentile >= 75:
157         suggested_epochs = self.max_epochs
158     elif percentile >= 50:
159         suggested_epochs = (self.max_epochs + self.min_epochs) // 2
160     else:
161         suggested_epochs = self.min_epochs
162
163     return {'epochs': suggested_epochs}
164
165 # Usage example
166 def run_budget_constrained_hpo():
167     """Run HPO with budget constraints."""
168
169     # Define budget
170     budget = ResourceBudget(
171         max_compute_hours=24.0,
172         max_cost_usd=60.0,
173         max_trials=100,
174         gpu_cost_per_hour=2.50
175     )
176
177     # Create resource manager
178     resource_manager = ResourceManager(budget)
179
180     # Create study
181     study = optuna.create_study(direction='maximize')
182
183     # Run with budget callback
184     study.optimize(
185         objective_function,
186         n_trials=budget.max_trials,
187         callbacks=[resource_manager.create_budget_callback()]
188     )
189
190     # Print final report
191     status = resource_manager.check_budget()
192     print("\nFinal Resource Usage:")
193     print(f" Total compute hours: {status['compute_hours_used']:.2f}")
194     print(f" Total cost: ${status['cost_used']:.2f}")
195     print(f" Trials completed: {len(resource_manager.trial_times)}")
196     print(f" Avg time per trial: {np.mean(resource_manager.trial_times):.1f}s")

```

Listing 10.15: Resource management for cost-effective HPO

Part IV

Model Deployment and Serving

Chapter 11

Model Serving Architecture

11.1 Learning Objectives

After completing this chapter, readers will be able to:

- Design and implement scalable model serving architectures for production
- Build high-performance REST APIs with FastAPI and gRPC services
- Deploy models using TensorFlow Serving, TorchServe, and custom serving solutions
- Choose appropriate serving patterns: batch vs online vs streaming inference
- Optimize models for serving with ONNX, TensorRT, and quantization techniques
- Implement caching strategies and load balancing for high-throughput systems
- Configure autoscaling and resource management for cost-effective serving
- Profile and optimize serving performance to meet latency and throughput requirements
- Monitor serving systems and implement fault tolerance patterns

11.2 Introduction: Model Serving Architecture

Model serving is the process of deploying machine learning models to production where they can accept requests and return predictions. Unlike training, which is typically a batch process, serving must handle:

- **Real-time requirements:** Predictions often needed in milliseconds
- **High availability:** 99.9%+ uptime requirements
- **Variable load:** Traffic can spike unpredictably
- **Cost efficiency:** Minimize infrastructure costs at scale
- **Multiple models:** Version management and A/B testing

A production serving system typically includes:

1. **API Layer:** REST or gRPC endpoints for client requests
2. **Model Server:** Loads models and executes inference
3. **Preprocessing:** Feature transformation and validation
4. **Postprocessing:** Result formatting and business logic
5. **Caching:** Reduce redundant computations
6. **Load Balancer:** Distribute traffic across instances
7. **Monitoring:** Track latency, throughput, and errors

11.3 FastAPI Model Serving

FastAPI provides a modern, high-performance framework for building production-ready ML APIs.

11.3.1 Basic FastAPI Implementation

```

1 from fastapi import FastAPI, HTTPException, BackgroundTasks
2 from fastapi.middleware.cors import CORSMiddleware
3 from fastapi.middleware.gzip import GZipMiddleware
4 from pydantic import BaseModel, Field, validator
5 import torch
6 import torch.nn as nn
7 import numpy as np
8 from typing import List, Dict, Optional
9 import logging
10 from datetime import datetime
11 import time
12 from functools import lru_cache
13 import asyncio
14
15 # Configure logging
16 logging.basicConfig(level=logging.INFO)
17 logger = logging.getLogger(__name__)
18
19 # Pydantic models for request/response validation
20 class PredictionRequest(BaseModel):
21     """Request schema for predictions."""
22     features: List[float] = Field(..., min_items=1, max_items=1000)
23     model_version: Optional[str] = Field(default="v1", description="Model version to use")
24
25     @validator('features')
26     def validate_features(cls, v):
27         """Validate feature values."""
28         if any(np.isnan(x) or np.isinf(x) for x in v):
29             raise ValueError("Features contain NaN or Inf values")
30         return v
31

```

```

32 class PredictionResponse(BaseModel):
33     """Response schema for predictions."""
34     prediction: float
35     probabilities: Optional[List[float]] = None
36     model_version: str
37     latency_ms: float
38     timestamp: str
39
40 class BatchPredictionRequest(BaseModel):
41     """Request schema for batch predictions."""
42     batch: List[List[float]] = Field(..., min_items=1, max_items=100)
43     model_version: Optional[str] = "v1"
44
45 class BatchPredictionResponse(BaseModel):
46     """Response schema for batch predictions."""
47     predictions: List[float]
48     model_version: str
49     batch_size: int
50     latency_ms: float
51
52 # Model wrapper with caching and lazy loading
53 class ModelManager:
54     """
55     Manage model lifecycle: loading, caching, and inference.
56
57     Features:
58     - Lazy loading (load on first request)
59     - Model versioning
60     - GPU/CPU support
61     - Warmup for consistent latency
62     """
63
64     def __init__(self, model_path: str, device: str = "cuda"):
65         self.model_path = model_path
66         self.device = device if torch.cuda.is_available() else "cpu"
67         self.model = None
68         self.model_version = "v1"
69         self.load_time = None
70
71     def load_model(self):
72         """Load model from disk."""
73         if self.model is not None:
74             return
75
76         logger.info(f"Loading model from {self.model_path}")
77         start_time = time.time()
78
79         try:
80             # Load model checkpoint
81             checkpoint = torch.load(self.model_path, map_location=self.device)
82
83             # Reconstruct model
84             self.model = self._create_model_architecture(checkpoint)
85             self.model.load_state_dict(checkpoint['model_state_dict'])
86             self.model.to(self.device)
87             self.model.eval()
88
89             self.load_time = time.time() - start_time

```

```

90         logger.info(f"Model loaded in {self.load_time:.2f}s on {self.device}")
91
92         # Warmup inference
93         self._warmup()
94
95     except Exception as e:
96         logger.error(f"Failed to load model: {e}")
97         raise
98
99     def _create_model_architecture(self, checkpoint: Dict) -> nn.Module:
100         """Create model architecture from checkpoint."""
101         # Example: Simple feedforward network
102         input_dim = checkpoint.get('input_dim', 784)
103         hidden_dim = checkpoint.get('hidden_dim', 256)
104         output_dim = checkpoint.get('output_dim', 10)
105
106         model = nn.Sequential(
107             nn.Linear(input_dim, hidden_dim),
108             nn.ReLU(),
109             nn.Dropout(0.3),
110             nn.Linear(hidden_dim, hidden_dim),
111             nn.ReLU(),
112             nn.Linear(hidden_dim, output_dim)
113         )
114         return model
115
116     def _warmup(self, n_iterations: int = 10):
117         """Warmup model with dummy inputs."""
118         logger.info("Warming up model...")
119         dummy_input = torch.randn(1, 784).to(self.device)
120
121         with torch.no_grad():
122             for _ in range(n_iterations):
123                 _ = self.model(dummy_input)
124
125         logger.info("Warmup complete")
126
127     @torch.no_grad()
128     def predict(self, features: np.ndarray) -> Dict:
129         """
130         Run inference on input features.
131
132         Args:
133             features: Input features (numpy array)
134
135         Returns:
136             Dict with prediction, probabilities, and metadata
137         """
138         if self.model is None:
139             self.load_model()
140
141         start_time = time.time()
142
143         # Convert to tensor
144         x = torch.from_numpy(features).float().unsqueeze(0).to(self.device)
145
146         # Inference
147         logits = self.model(x)

```

```

148     probabilities = torch.softmax(logits, dim=1)
149     prediction = torch.argmax(probabilities, dim=1)
150
151     latency_ms = (time.time() - start_time) * 1000
152
153     return {
154         'prediction': prediction.item(),
155         'probabilities': probabilities.cpu().numpy().tolist()[0],
156         'latency_ms': latency_ms
157     }
158
159     @torch.no_grad()
160     def predict_batch(self, batch: np.ndarray) -> Dict:
161         """Run batch inference."""
162         if self.model is None:
163             self.load_model()
164
165         start_time = time.time()
166
167         # Convert to tensor
168         x = torch.from_numpy(batch).float().to(self.device)
169
170         # Batch inference
171         logits = self.model(x)
172         predictions = torch.argmax(logits, dim=1)
173
174         latency_ms = (time.time() - start_time) * 1000
175
176         return {
177             'predictions': predictions.cpu().numpy().tolist(),
178             'latency_ms': latency_ms
179         }
180
181     # Create FastAPI app
182     app = FastAPI(
183         title="ML Model Serving API",
184         description="Production-ready model serving with FastAPI",
185         version="1.0.0"
186     )
187
188     # Add middleware
189     app.add_middleware(
190         CORSMiddleware,
191         allow_origins=["*"],
192         allow_credentials=True,
193         allow_methods=["*"],
194         allow_headers=["*"],
195     )
196
197     app.add_middleware(GZipMiddleware, minimum_size=1000)
198
199     # Initialize model manager (lazy loading)
200     model_manager = ModelManager(model_path="model.pth")
201
202     # Metrics tracking
203     class MetricsTracker:
204         """Track API metrics."""
205         def __init__(self):

```

```

206         self.request_count = 0
207         self.total_latency = 0.0
208         self.error_count = 0
209
210     def record_request(self, latency_ms: float, error: bool = False):
211         self.request_count += 1
212         self.total_latency += latency_ms
213         if error:
214             self.error_count += 1
215
216     def get_metrics(self) -> Dict:
217         avg_latency = self.total_latency / self.request_count if self.request_count >
0 else 0
218         error_rate = self.error_count / self.request_count if self.request_count > 0
else 0
219
220         return {
221             'total_requests': self.request_count,
222             'avg_latency_ms': avg_latency,
223             'error_count': self.error_count,
224             'error_rate': error_rate
225         }
226
227 metrics = MetricsTracker()
228
229 @app.on_event("startup")
230 async def startup_event():
231     """Initialize on startup."""
232     logger.info("Starting ML Serving API...")
233     # Optionally preload model
234     # model_manager.load_model()
235
236 @app.on_event("shutdown")
237 async def shutdown_event():
238     """Cleanup on shutdown."""
239     logger.info("Shutting down ML Serving API...")
240
241 @app.get("/")
242 async def root():
243     """Health check endpoint."""
244     return {
245         "status": "healthy",
246         "service": "ml-serving-api",
247         "version": "1.0.0"
248     }
249
250 @app.get("/health")
251 async def health():
252     """Detailed health check."""
253     return {
254         "status": "healthy",
255         "model_loaded": model_manager.model is not None,
256         "device": model_manager.device,
257         "timestamp": datetime.now().isoformat()
258     }
259
260 @app.get("/metrics")
261 async def get_metrics():

```



```

262     """Get API metrics."""
263     return metrics.get_metrics()
264
265 @app.post("/predict", response_model=PredictionResponse)
266 async def predict(request: PredictionRequest, background_tasks: BackgroundTasks):
267     """
268     Single prediction endpoint.
269
270     Example request:
271     {
272         "features": [0.1, 0.2, ..., 0.9],
273         "model_version": "v1"
274     }
275     """
276     start_time = time.time()
277
278     try:
279         # Convert features to numpy array
280         features = np.array(request.features, dtype=np.float32)
281
282         # Run prediction
283         result = model_manager.predict(features)
284
285         # Record metrics in background
286         latency_ms = (time.time() - start_time) * 1000
287         background_tasks.add_task(metrics.record_request, latency_ms, error=False)
288
289         return PredictionResponse(
290             prediction=result['prediction'],
291             probabilities=result['probabilities'],
292             model_version=request.model_version,
293             latency_ms=latency_ms,
294             timestamp=datetime.now().isoformat()
295         )
296
297     except Exception as e:
298         logger.error(f"Prediction error: {e}")
299         background_tasks.add_task(metrics.record_request, 0.0, error=True)
300         raise HTTPException(status_code=500, detail=str(e))
301
302 @app.post("/predict/batch", response_model=BatchPredictionResponse)
303 async def predict_batch(request: BatchPredictionRequest, background_tasks:
304     BackgroundTasks):
305     """
306     Batch prediction endpoint.
307
308     Example request:
309     {
310         "batch": [[0.1, 0.2, ...], [0.3, 0.4, ...]]
311     }
312     """
313     start_time = time.time()
314
315     try:
316         # Convert to numpy array
317         batch = np.array(request.batch, dtype=np.float32)
318
319         # Run batch prediction

```

```

319         result = model_manager.predict_batch(batch)
320
321         latency_ms = (time.time() - start_time) * 1000
322         background_tasks.add_task(metrics.record_request, latency_ms, error=False)
323
324         return BatchPredictionResponse(
325             predictions=result['predictions'],
326             model_version=request.model_version,
327             batch_size=len(request.batch),
328             latency_ms=latency_ms
329         )
330
331     except Exception as e:
332         logger.error(f"Batch prediction error: {e}")
333         background_tasks.add_task(metrics.record_request, 0.0, error=True)
334         raise HTTPException(status_code=500, detail=str(e))
335
336 # Run with: uvicorn main:app --host 0.0.0.0 --port 8000 --workers 4
337 if __name__ == "__main__":
338     import uvicorn
339     uvicorn.run(app, host="0.0.0.0", port=8000)

```

Listing 11.1: Production-ready FastAPI model serving

11.3.2 Advanced FastAPI Features

```

1 from fastapi import FastAPI, Request, Depends
2 from fastapi.responses import JSONResponse
3 from fastapi_limiter import FastAPILimiter
4 from fastapi_limiter.depends import RateLimiter
5 import redis.asyncio as redis
6 from functools import wraps
7 import hashlib
8 import json
9 import pickle
10 from typing import Callable
11
12 # Redis cache for predictions
13 class PredictionCache:
14     """
15     Cache predictions to avoid redundant inference.
16
17     Use cases:
18     - Repeated requests for same input
19     - Expensive model inference
20     - High QPS scenarios
21     """
22
23     def __init__(self, redis_url: str = "redis://localhost:6379"):
24         self.redis_client = None
25         self.redis_url = redis_url
26         self.ttl = 3600 # Cache TTL in seconds
27
28     async def connect(self):
29         """Connect to Redis."""
30         self.redis_client = await redis.from_url(
31             self.redis_url,

```

```

32         encoding="utf-8",
33         decode_responses=True
34     )
35
36     async def close(self):
37         """Close Redis connection."""
38         if self.redis_client:
39             await self.redis_client.close()
40
41     def _generate_cache_key(self, features: List[float], model_version: str) -> str:
42         """Generate cache key from input features."""
43         # Hash features for cache key
44         features_str = json.dumps(features, sort_keys=True)
45         feature_hash = hashlib.md5(features_str.encode()).hexdigest()
46         return f"prediction:{model_version}:{feature_hash}"
47
48     async def get(self, features: List[float], model_version: str) -> Optional[Dict]:
49         """Get cached prediction."""
50         if not self.redis_client:
51             return None
52
53         cache_key = self._generate_cache_key(features, model_version)
54         cached = await self.redis_client.get(cache_key)
55
56         if cached:
57             return json.loads(cached)
58
59         return None
60
61     async def set(self, features: List[float], model_version: str, result: Dict):
62         """Cache prediction result."""
63         if not self.redis_client:
64             return
65
66         cache_key = self._generate_cache_key(features, model_version)
67         await self.redis_client.setex(
68             cache_key,
69             self.ttl,
70             json.dumps(result)
71         )
72
73     # Initialize cache
74     cache = PredictionCache()
75
76     # Update FastAPI app with caching
77     app = FastAPI(title="ML Serving API with Caching")
78
79     @app.on_event("startup")
80     async def startup_event():
81         """Initialize cache on startup."""
82         await cache.connect()
83         # Initialize rate limiter
84         redis_client = await redis.from_url("redis://localhost:6379")
85         await FastAPILimiter.init(redis_client)
86
87     @app.on_event("shutdown")
88     async def shutdown_event():
89         """Close cache on shutdown."""

```

```

90     await cache.close()
91
92 @app.post("/predict/cached", dependencies=[Depends(RateLimiter(times=100, seconds=60))
93 ])
94 async def predict_with_cache(request: PredictionRequest):
95     """
96     Prediction endpoint with caching and rate limiting.
97
98     - Rate limit: 100 requests per minute
99     - Cache hit: ~1ms latency
100     - Cache miss: run inference and cache result
101     """
102     start_time = time.time()
103
104     # Check cache
105     cached_result = await cache.get(request.features, request.model_version)
106
107     if cached_result is not None:
108         logger.info("Cache hit")
109         cached_result['cache_hit'] = True
110         cached_result['latency_ms'] = (time.time() - start_time) * 1000
111         return cached_result
112
113     # Cache miss - run inference
114     logger.info("Cache miss - running inference")
115     features = np.array(request.features, dtype=np.float32)
116     result = model_manager.predict(features)
117
118     # Prepare response
119     response = {
120         'prediction': result['prediction'],
121         'probabilities': result['probabilities'],
122         'model_version': request.model_version,
123         'latency_ms': (time.time() - start_time) * 1000,
124         'cache_hit': False,
125         'timestamp': datetime.now().isoformat()
126     }
127
128     # Cache result
129     await cache.set(request.features, request.model_version, response)
130
131     return response
132
133 # Request validation and preprocessing
134 class RequestValidator:
135     """Validate and preprocess requests."""
136
137     @staticmethod
138     def validate_features(features: List[float], expected_dim: int = 784) -> np.
139         ndarray:
140             """Validate feature dimensions and values."""
141             if len(features) != expected_dim:
142                 raise HTTPException(
143                     status_code=400,
144                     detail=f"Expected {expected_dim} features, got {len(features)}"
145                 )
146
147     # Check for invalid values

```

```

146     features_array = np.array(features, dtype=np.float32)
147
148     if np.any(np.isnan(features_array)):
149         raise HTTPException(status_code=400, detail="Features contain NaN values")
150
151     if np.any(np.isinf(features_array)):
152         raise HTTPException(status_code=400, detail="Features contain Inf values")
153
154     return features_array
155
156     @staticmethod
157     def normalize_features(features: np.ndarray) -> np.ndarray:
158         """Normalize features to [0, 1] range."""
159         min_val = features.min()
160         max_val = features.max()
161
162         if max_val - min_val < 1e-8:
163             return features
164
165         return (features - min_val) / (max_val - min_val)
166
167     validator = RequestValidator()
168
169     @app.post("/predict/validated")
170     async def predict_with_validation(request: PredictionRequest):
171         """Prediction with input validation and preprocessing."""
172         try:
173             # Validate features
174             features = validator.validate_features(request.features)
175
176             # Normalize if needed
177             features = validator.normalize_features(features)
178
179             # Run prediction
180             result = model_manager.predict(features)
181
182             return {
183                 'prediction': result['prediction'],
184                 'probabilities': result['probabilities'],
185                 'model_version': request.model_version
186             }
187
188         except HTTPException:
189             raise
190         except Exception as e:
191             logger.error(f"Validation error: {e}")
192             raise HTTPException(status_code=500, detail="Internal server error")

```

Listing 11.2: Advanced serving features with caching and rate limiting

11.4 gRPC Model Serving

gRPC provides high-performance RPC for low-latency serving scenarios.

```

1 syntax = "proto3";
2
3 package prediction;

```

```

4
5 // Prediction service definition
6 service PredictionService {
7     // Single prediction
8     rpc Predict (PredictRequest) returns (PredictResponse);
9
10    // Batch prediction
11    rpc PredictBatch (BatchPredictRequest) returns (BatchPredictResponse);
12
13    // Streaming predictions
14    rpc PredictStream (stream PredictRequest) returns (stream PredictResponse);
15
16    // Health check
17    rpc HealthCheck (HealthCheckRequest) returns (HealthCheckResponse);
18 }
19
20 // Request messages
21 message PredictRequest {
22     repeated float features = 1;
23     string model_version = 2;
24 }
25
26 message BatchPredictRequest {
27     repeated Features batch = 1;
28     string model_version = 2;
29 }
30
31 message Features {
32     repeated float values = 1;
33 }
34
35 message HealthCheckRequest {}
36
37 // Response messages
38 message PredictResponse {
39     int32 prediction = 1;
40     repeated float probabilities = 2;
41     string model_version = 3;
42     float latency_ms = 4;
43 }
44
45 message BatchPredictResponse {
46     repeated int32 predictions = 1;
47     string model_version = 2;
48     int32 batch_size = 3;
49     float latency_ms = 4;
50 }
51
52 message HealthCheckResponse {
53     enum ServingStatus {
54         UNKNOWN = 0;
55         SERVING = 1;
56         NOT_SERVING = 2;
57     }
58     ServingStatus status = 1;
59 }

```

Listing 11.3: gRPC service definition (prediction.proto)

```

1 import grpc
2 from concurrent import futures
3 import time
4 import numpy as np
5 import torch
6 import logging
7
8 # Generated from prediction.proto
9 import prediction_pb2
10 import prediction_pb2_grpc
11
12 logging.basicConfig(level=logging.INFO)
13 logger = logging.getLogger(__name__)
14
15 class PredictionServicer(prediction_pb2_grpc.PredictionServiceServicer):
16     """
17     gRPC service for model predictions.
18
19     Features:
20     - High-performance binary protocol
21     - Streaming support
22     - Built-in load balancing
23     - Type-safe requests/responses
24     """
25
26     def __init__(self, model_manager: ModelManager):
27         self.model_manager = model_manager
28         self.model_manager.load_model()
29
30     def Predict(self, request, context):
31         """Handle single prediction request."""
32         start_time = time.time()
33
34         try:
35             # Extract features
36             features = np.array(request.features, dtype=np.float32)
37
38             # Run inference
39             result = self.model_manager.predict(features)
40
41             latency_ms = (time.time() - start_time) * 1000
42
43             # Build response
44             response = prediction_pb2.PredictResponse(
45                 prediction=int(result['prediction']),
46                 probabilities=result['probabilities'],
47                 model_version=request.model_version,
48                 latency_ms=latency_ms
49             )
50
51             return response
52
53         except Exception as e:
54             logger.error(f"Prediction error: {e}")
55             context.set_code(grpc.StatusCode.INTERNAL)
56             context.set_details(str(e))
57             return prediction_pb2.PredictResponse()

```

```

58
59 def PredictBatch(self, request, context):
60     """Handle batch prediction request."""
61     start_time = time.time()
62
63     try:
64         # Extract batch
65         batch = np.array([list(f.values) for f in request.batch], dtype=np.float32
66     )
67
68         # Run batch inference
69         result = self.model_manager.predict_batch(batch)
70
71         latency_ms = (time.time() - start_time) * 1000
72
73         response = prediction_pb2.BatchPredictResponse(
74             predictions=result['predictions'],
75             model_version=request.model_version,
76             batch_size=len(request.batch),
77             latency_ms=latency_ms
78         )
79
80         return response
81
82     except Exception as e:
83         logger.error(f"Batch prediction error: {e}")
84         context.set_code(grpc.StatusCode.INTERNAL)
85         context.set_details(str(e))
86         return prediction_pb2.BatchPredictResponse()
87
88 def PredictStream(self, request_iterator, context):
89     """Handle streaming predictions."""
90     for request in request_iterator:
91         try:
92             features = np.array(request.features, dtype=np.float32)
93             result = self.model_manager.predict(features)
94
95             response = prediction_pb2.PredictResponse(
96                 prediction=int(result['prediction']),
97                 probabilities=result['probabilities'],
98                 model_version=request.model_version,
99                 latency_ms=result['latency_ms']
100             )
101
102             yield response
103
104         except Exception as e:
105             logger.error(f"Streaming prediction error: {e}")
106             context.set_code(grpc.StatusCode.INTERNAL)
107             context.set_details(str(e))
108             return
109
110 def HealthCheck(self, request, context):
111     """Health check endpoint."""
112     if self.model_manager.model is not None:
113         status = prediction_pb2.HealthCheckResponse.SERVING
114     else:
115         status = prediction_pb2.HealthCheckResponse.NOT_SERVING

```



```

115         return prediction_pb2.HealthCheckResponse(status=status)
116
117     def serve(model_path: str, port: int = 50051, max_workers: int = 10):
118         """
119         Start gRPC server.
120
121         Args:
122             model_path: Path to model checkpoint
123             port: Server port
124             max_workers: Maximum concurrent workers
125         """
126         # Create server
127         server = grpc.server(
128             futures.ThreadPoolExecutor(max_workers=max_workers),
129             options=[
130                 ('grpc.max_send_message_length', 50 * 1024 * 1024), # 50MB
131                 ('grpc.max_receive_message_length', 50 * 1024 * 1024),
132                 ('grpc.so_reuseport', 1),
133                 ('grpc.use_local_subchannel_pool', 1)
134             ]
135         )
136
137         # Initialize model manager
138         model_manager = ModelManager(model_path=model_path)
139
140         # Add servicer to server
141         prediction_pb2_grpc.add_PredictionServiceServicer_to_server(
142             PredictionServicer(model_manager), server
143         )
144
145         # Start server
146         server.add_insecure_port(f'[:,]:{port}')
147         server.start()
148
149         logger.info(f"gRPC server started on port {port}")
150         logger.info(f"Max workers: {max_workers}")
151
152         try:
153             server.wait_for_termination()
154         except KeyboardInterrupt:
155             logger.info("Shutting down gRPC server...")
156             server.stop(0)
157
158     if __name__ == '__main__':
159         serve(model_path='model.pth', port=50051, max_workers=10)
160

```

Listing 11.4: gRPC server implementation

```

1 import grpc
2 import prediction_pb2
3 import prediction_pb2_grpc
4 import numpy as np
5 import time
6
7 class PredictionClient:
8     """
9     gRPC client for predictions.

```

```

10
11 Features:
12 - Connection pooling
13 - Automatic retry
14 - Load balancing
15 - Timeout handling
16 """
17
18 def __init__(self, host: str = 'localhost', port: int = 50051):
19     self.address = f'{host}:{port}'
20
21     # Create channel with keepalive
22     self.channel = grpc.insecure_channel(
23         self.address,
24         options=[
25             ('grpc.keepalive_time_ms', 10000),
26             ('grpc.keepalive_timeout_ms', 5000),
27             ('grpc.keepalive_permit_without_calls', True),
28             ('grpc.http2.max_pings_without_data', 0),
29         ]
30     )
31
32     self.stub = prediction_pb2_grpc.PredictionServiceStub(self.channel)
33
34 def predict(self, features: np.ndarray, model_version: str = "v1", timeout: float
35 = 5.0):
36     """
37     Make single prediction.
38
39     Args:
40         features: Input features
41         model_version: Model version to use
42         timeout: Request timeout in seconds
43
44     Returns:
45         Prediction response
46     """
47     request = prediction_pb2.PredictRequest(
48         features=features.tolist(),
49         model_version=model_version
50     )
51
52     try:
53         response = self.stub.Predict(request, timeout=timeout)
54         return {
55             'prediction': response.prediction,
56             'probabilities': list(response.probabilities),
57             'model_version': response.model_version,
58             'latency_ms': response.latency_ms
59         }
60     except grpc.RpcError as e:
61         print(f"RPC error: {e.code():} {e.details()}")
62         raise
63
64 def predict_batch(self, batch: np.ndarray, model_version: str = "v1", timeout:
65 float = 10.0):
66     """Make batch prediction."""

```

```

66     # Build batch request
67     features_list = [
68         prediction_pb2.Features(values=row.tolist())
69         for row in batch
70     ]
71
72     request = prediction_pb2.BatchPredictRequest(
73         batch=features_list,
74         model_version=model_version
75     )
76
77     try:
78         response = self.stub.PredictBatch(request, timeout=timeout)
79         return {
80             'predictions': list(response.predictions),
81             'batch_size': response.batch_size,
82             'latency_ms': response.latency_ms
83         }
84
85     except grpc.RpcError as e:
86         print(f"RPC error: {e.code()}: {e.details()}")
87         raise
88
89     def predict_stream(self, features_iterator, model_version: str = "v1"):
90         """
91         Make streaming predictions.
92
93         Args:
94             features_iterator: Iterator of feature arrays
95             model_version: Model version
96
97         Yields:
98             Prediction responses
99         """
100         def request_generator():
101             for features in features_iterator:
102                 yield prediction_pb2.PredictRequest(
103                     features=features.tolist(),
104                     model_version=model_version
105                 )
106
107         try:
108             for response in self.stub.PredictStream(request_generator()):
109                 yield {
110                     'prediction': response.prediction,
111                     'probabilities': list(response.probabilities),
112                     'latency_ms': response.latency_ms
113                 }
114
115         except grpc.RpcError as e:
116             print(f"Streaming RPC error: {e.code()}: {e.details()}")
117             raise
118
119     def health_check(self):
120         """Check server health."""
121         request = prediction_pb2.HealthCheckRequest()
122         try:
123             response = self.stub.HealthCheck(request, timeout=2.0)

```

```

124         return response.status == prediction_pb2.HealthCheckResponse.SERVING
125     except grpc.RpcError:
126         return False
127
128     def close(self):
129         """Close channel."""
130         self.channel.close()
131
132     # Usage example
133     if __name__ == '__main__':
134         client = PredictionClient(host='localhost', port=50051)
135
136         # Health check
137         if not client.health_check():
138             print("Server not healthy")
139             exit(1)
140
141         # Single prediction
142         features = np.random.randn(784).astype(np.float32)
143         result = client.predict(features)
144         print(f"Prediction: {result['prediction']}")
145         print(f"Latency: {result['latency_ms']:.2f}ms")
146
147         # Batch prediction
148         batch = np.random.randn(10, 784).astype(np.float32)
149         batch_result = client.predict_batch(batch)
150         print(f"Batch predictions: {batch_result['predictions']}")
151         print(f"Batch latency: {batch_result['latency_ms']:.2f}ms")
152
153         # Streaming predictions
154         def feature_generator():
155             for _ in range(5):
156                 yield np.random.randn(784).astype(np.float32)
157                 time.sleep(0.1)
158
159         print("\nStreaming predictions:")
160         for result in client.predict_stream(feature_generator()):
161             print(f" Prediction: {result['prediction']}, Latency: {result['latency_ms']:.2f}ms")
162
163         client.close()

```

Listing 11.5: gRPC client implementation

11.5 TensorFlow Serving and TorchServe

Production-grade serving frameworks provide optimized inference and model management.

11.5.1 TensorFlow Serving Setup

```

1 import tensorflow as tf
2 import os
3 from typing import Dict
4

```

```

5 class ModelExporter:
6     """Export TensorFlow models for TF Serving."""
7
8     @staticmethod
9     def export_saved_model(model: tf.keras.Model,
10                            export_dir: str,
11                            version: int = 1):
12
13         """
14         Export model in SavedModel format.
15
16         TF Serving expects models in:
17         /model_base_path/
18             /1/ # version 1
19                 /saved_model.pb
20                 /variables/
21             /2/ # version 2
22             ...
23         """
24         model_path = os.path.join(export_dir, str(version))
25
26         # Save model with signature
27         tf.saved_model.save(
28             model,
29             model_path,
30             signatures={
31                 'serving_default': model.call.get_concrete_function(
32                     tf.TensorSpec(shape=[None, 784], dtype=tf.float32, name='input')
33                 )
34             }
35         )
36
37         print(f"Model exported to {model_path}")
38
39     @staticmethod
40     def export_with_preprocessing(model: tf.keras.Model,
41                                   export_dir: str,
42                                   version: int = 1):
43
44         """
45         Export model with preprocessing layer.
46
47         Benefits:
48         - Preprocessing happens on server (GPU accelerated)
49         - Clients send raw data, not preprocessed features
50         - Consistent preprocessing between training and serving
51         """
52         @tf.function(input_signature=[tf.TensorSpec(shape=[None, 28, 28], dtype=tf.
53             uint8)])
54         def preprocess_and_predict(images):
55             # Normalize to [0, 1]
56             images = tf.cast(images, tf.float32) / 255.0
57
58             # Flatten
59             images = tf.reshape(images, [-1, 784])
60
61             # Run model
62             return model(images)
63
64         # Create wrapper module

```

```

62     class ServingModule(tf.Module):
63         def __init__(self, model):
64             super().__init__()
65             self.model = model
66
67         @tf.function(input_signature=[tf.TensorSpec(shape=[None, 28, 28], dtype=tf
        .uint8)])
68         def __call__(self, images):
69             return preprocess_and_predict(images)
70
71     serving_module = ServingModule(model)
72
73     model_path = os.path.join(export_dir, str(version))
74     tf.saved_model.save(serving_module, model_path)
75
76     print(f"Model with preprocessing exported to {model_path}")
77
78 # Export example
79 def export_for_serving():
80     """Complete export example."""
81     # Load trained model
82     model = tf.keras.models.load_model('trained_model.h5')
83
84     # Export for serving
85     exporter = ModelExporter()
86     exporter.export_saved_model(
87         model,
88         export_dir='/models/my_model',
89         version=1
90     )
91
92     # Export with preprocessing
93     exporter.export_with_preprocessing(
94         model,
95         export_dir='/models/my_model_with_preprocessing',
96         version=1
97     )

```

Listing 11.6: Prepare model for TensorFlow Serving

```

1  # Start TensorFlow Serving with Docker
2  docker run -p 8501:8501 \
3      --name=tf_serving \
4      --mount type=bind,source=/path/to/models/my_model,target=/models/my_model \
5      -e MODEL_NAME=my_model \
6      -t tensorflow/serving
7
8  # Start with GPU support
9  docker run --gpus all -p 8501:8501 \
10     --name=tf_serving_gpu \
11     --mount type=bind,source=/path/to/models/my_model,target=/models/my_model \
12     -e MODEL_NAME=my_model \
13     -t tensorflow/serving:latest-gpu
14
15 # Start with custom configuration
16 docker run -p 8501:8501 -p 8500:8500 \
17     --name=tf_serving_custom \
18     --mount type=bind,source=/path/to/models,target=/models \

```

```

19 --mount type=bind,source=/path/to/config,target=/config \
20 -t tensorflow/serving \
21 --model_config_file=/config/models.config \
22 --model_config_file_poll_wait_seconds=60

```

Listing 11.7: TensorFlow Serving deployment with Docker

```

1 import requests
2 import json
3 import numpy as np
4
5 class TensorFlow-servingClient:
6     """
7     Client for TensorFlow Serving REST API.
8
9     TF Serving provides:
10     - REST API on port 8501
11     - gRPC API on port 8500
12     - Model versioning
13     - Batching
14     """
15
16     def __init__(self, host: str = 'localhost', port: int = 8501, model_name: str = '
17         my_model'):
18         self.base_url = f'http://{host}:{port}/v1/models/{model_name}'
19
20     def predict(self, instances: np.ndarray, version: int = None):
21         """
22         Make prediction request.
23
24         Args:
25             instances: Input data (numpy array)
26             version: Specific model version (optional)
27
28         Returns:
29             Prediction results
30         """
31         # Build URL
32         if version:
33             url = f'{self.base_url}/versions/{version}:predict'
34         else:
35             url = f'{self.base_url}:predict'
36
37         # Prepare request
38         data = {
39             'instances': instances.tolist()
40         }
41
42         # Send request
43         response = requests.post(url, json=data)
44         response.raise_for_status()
45
46         return response.json()
47
48     def get_model_metadata(self, version: int = None):
49         """Get model metadata."""
50         if version:
51             url = f'{self.base_url}/versions/{version}/metadata'

```

```

51         else:
52             url = f'{self.base_url}/metadata'
53
54             response = requests.get(url)
55             response.raise_for_status()
56
57             return response.json()
58
59         def get_model_status(self):
60             """Get model status."""
61             url = f'{self.base_url}'
62             response = requests.get(url)
63             response.raise_for_status()
64
65             return response.json()
66
67     # Usage
68     client = TensorFlow-servingClient(host='localhost', port=8501, model_name='my_model')
69
70     # Get model metadata
71     metadata = client.get_model_metadata()
72     print("Model metadata:", json.dumps(metadata, indent=2))
73
74     # Make prediction
75     test_input = np.random.randn(1, 784).astype(np.float32)
76     result = client.predict(test_input)
77     print("Prediction:", result['predictions'])
78
79     # Use specific version
80     result_v2 = client.predict(test_input, version=2)
81     print("Prediction (v2):", result_v2['predictions'])

```

Listing 11.8: TensorFlow Serving client

11.5.2 TorchServe Setup

```

1  import torch
2  import torch.nn as nn
3  from torchvision import models
4  import json
5
6  # Create model archive for TorchServe
7  class TorchServeExporter:
8      """Export PyTorch models for TorchServe."""
9
10     @staticmethod
11     def save_model(model: nn.Module, model_path: str):
12         """Save model for TorchServe."""
13         # Save state dict
14         torch.save(model.state_dict(), model_path)
15         print(f"Model saved to {model_path}")
16
17     @staticmethod
18     def create_handler(handler_path: str):
19         """
20         Create custom handler for TorchServe.
21

```



```

22     Handler defines:
23     - Preprocessing
24     - Inference
25     - Postprocessing
26     """
27     handler_code = """
28 import torch
29 import torch.nn.functional as F
30 from torchvision import transforms
31 from ts.torch_handler.base_handler import BaseHandler
32 import io
33 from PIL import Image
34 import numpy as np
35
36 class CustomHandler(BaseHandler):
37     '''Custom handler for image classification.'''
38
39     def initialize(self, context):
40         '''Initialize model and preprocessing.'''
41         super().initialize(context)
42
43         # Define preprocessing
44         self.transform = transforms.Compose([
45             transforms.Resize(256),
46             transforms.CenterCrop(224),
47             transforms.ToTensor(),
48             transforms.Normalize(
49                 mean=[0.485, 0.456, 0.406],
50                 std=[0.229, 0.224, 0.225]
51             )
52         ])
53
54     def preprocess(self, data):
55         '''
56         Preprocess input data.
57
58         Args:
59             data: List of input data (raw bytes or JSON)
60
61         Returns:
62             Preprocessed tensors
63         '''
64         images = []
65
66         for row in data:
67             # Get image from request
68             image = row.get("data") or row.get("body")
69
70             if isinstance(image, dict):
71                 # JSON input
72                 image = image.get("image")
73
74             # Load image
75             if isinstance(image, (bytes, bytearray)):
76                 image = Image.open(io.BytesIO(image))
77
78             # Preprocess
79             image = self.transform(image)

```

```

80         images.append(image)
81
82         # Stack into batch
83         return torch.stack(images).to(self.device)
84
85     def inference(self, data):
86         '''Run inference.'''
87         with torch.no_grad():
88             outputs = self.model(data)
89             probabilities = F.softmax(outputs, dim=1)
90
91         return probabilities
92
93     def postprocess(self, data):
94         '''
95         Postprocess model output.
96
97         Returns:
98             List of predictions
99         '''
100         # Get top-5 predictions
101         probs, classes = torch.topk(data, k=5, dim=1)
102
103         results = []
104         for prob, cls in zip(probs, classes):
105             result = {
106                 'predictions': [
107                     {
108                         'class': int(c),
109                         'probability': float(p)
110                     }
111                     for c, p in zip(cls, prob)
112                 ]
113             }
114             results.append(result)
115
116         return results
117
118
119     with open(handler_path, 'w') as f:
120         f.write(handler_code)
121
122     print(f"Handler saved to {handler_path}")
123
124 # Create model archive
125 def create_torchserve_archive():
126     """
127     Create .mar file for TorchServe.
128
129     Steps:
130     1. Save model
131     2. Create handler (optional)
132     3. Package with torch-model-archiver
133     """
134     # Load and save model
135     model = models.resnet50(pretrained=True)
136     model.eval()

```

```

138 torch.save(model.state_dict(), 'resnet50.pth')
139
140 # Create handler
141 exporter = TorchServeExporter()
142 exporter.create_handler('custom_handler.py')
143
144 # Create index_to_name.json for class mapping
145 class_mapping = {str(i): f"class_{i}" for i in range(1000)}
146 with open('index_to_name.json', 'w') as f:
147     json.dump(class_mapping, f)
148
149 print("Files ready for archiving. Run:")
150 print("""
151 torch-model-archiver \\\
152   --model-name resnet50 \\\
153   --version 1.0 \\\
154   --model-file model.py \\\
155   --serialized-file resnet50.pth \\\
156   --handler custom_handler.py \\\
157   --extra-files index_to_name.json \\\
158   --export-path model-store/
159   """)

```

Listing 11.9: Prepare PyTorch model for TorchServe

```

1  # Install TorchServe
2  pip install torchserve torch-model-archiver torch-workflow-archiver
3
4  # Create model archive
5  torch-model-archiver \
6    --model-name resnet50 \
7    --version 1.0 \
8    --model-file model.py \
9    --serialized-file resnet50.pth \
10   --handler image_classifier \
11   --extra-files index_to_name.json \
12   --export-path model-store/
13
14  # Start TorchServe
15  torchserve --start \
16    --ncs \
17    --model-store model-store \
18    --models resnet50.mar
19
20  # Start with configuration
21  torchserve --start \
22    --ncs \
23    --model-store model-store \
24    --models resnet50.mar \
25    --ts-config config.properties
26
27  # config.properties example:
28  # inference_address=http://0.0.0.0:8080
29  # management_address=http://0.0.0.0:8081
30  # metrics_address=http://0.0.0.0:8082
31  # number_of_netty_threads=32
32  # job_queue_size=1000
33  # number_of_gpu=1

```

```

34 # batch_size=8
35 # max_batch_delay=100
36 # default_workers_per_model=4
37
38 # Register model via Management API
39 curl -X POST "http://localhost:8081/models?url=resnet50.mar&initial_workers=4&
    batch_size=8"
40
41 # Scale workers
42 curl -X PUT "http://localhost:8081/models/resnet50?min_worker=2&max_worker=8"
43
44 # Unregister model
45 curl -X DELETE "http://localhost:8081/models/resnet50"

```

Listing 11.10: TorchServe deployment

```

1 import requests
2 import json
3 from PIL import Image
4 import io
5
6 class TorchServeClient:
7     """Client for TorchServe REST API."""
8
9     def __init__(self, host: str = 'localhost', port: int = 8080):
10         self.inference_url = f'http://{host}:{port}/predictions'
11         self.management_url = f'http://{host}:{port + 1}'
12
13     def predict(self, model_name: str, image_path: str):
14         """
15         Make prediction request.
16
17         Args:
18             model_name: Name of registered model
19             image_path: Path to input image
20
21         Returns:
22             Prediction results
23         """
24         url = f'{self.inference_url}/{model_name}'
25
26         # Read image
27         with open(image_path, 'rb') as f:
28             files = {'data': f}
29             response = requests.post(url, files=files)
30
31         response.raise_for_status()
32         return response.json()
33
34     def predict_batch(self, model_name: str, image_paths: list):
35         """Make batch prediction."""
36         url = f'{self.inference_url}/{model_name}'
37
38         # Prepare batch
39         files = []
40         for i, path in enumerate(image_paths):
41             with open(path, 'rb') as f:
42                 files.append(('data', f.read()))

```

```

43         response = requests.post(url, files=files)
44         response.raise_for_status()
45
46         return response.json()
47
48     def get_models(self):
49         """List registered models."""
50         url = f'{self.management_url}/models'
51         response = requests.get(url)
52         response.raise_for_status()
53
54         return response.json()
55
56     def describe_model(self, model_name: str):
57         """Get model details."""
58         url = f'{self.management_url}/models/{model_name}'
59         response = requests.get(url)
60         response.raise_for_status()
61
62         return response.json()
63
64     def scale_workers(self, model_name: str, min_workers: int, max_workers: int):
65         """Scale model workers."""
66         url = f'{self.management_url}/models/{model_name}'
67         params = {
68             'min_worker': min_workers,
69             'max_worker': max_workers
70         }
71
72         response = requests.put(url, params=params)
73         response.raise_for_status()
74
75         return response.json()
76
77 # Usage
78 client = TorchServeClient(host='localhost', port=8080)
79
80 # List models
81 models = client.get_models()
82 print("Registered models:", models)
83
84 # Describe model
85 model_info = client.describe_model('resnet50')
86 print("Model info:", json.dumps(model_info, indent=2))
87
88 # Make prediction
89 result = client.predict('resnet50', 'test_image.jpg')
90 print("Prediction:", result)
91
92 # Scale workers
93 client.scale_workers('resnet50', min_workers=2, max_workers=8)
94 print("Workers scaled")
95

```

Listing 11.11: TorchServe client

11.6 Batch vs Online vs Streaming Serving Patterns

Different serving patterns suit different use cases and latency requirements.

11.6.1 Online Serving (Real-time)

```

1 import torch
2 import torch.nn as nn
3 from typing import Dict
4 import time
5 from collections import deque
6 import threading
7
8 class OnlineServingOptimizer:
9     """
10     Optimize model for low-latency online serving.
11
12     Techniques:
13     - Model warmup
14     - Batch size 1 optimization
15     - GPU memory pre-allocation
16     - TorchScript compilation
17     """
18
19     def __init__(self, model: nn.Module, device: str = "cuda"):
20         self.model = model
21         self.device = device
22         self.optimized_model = None
23
24     def optimize_for_online(self) -> nn.Module:
25         """Optimize model for single-instance inference."""
26         # Move to device
27         self.model = self.model.to(self.device)
28         self.model.eval()
29
30         # TorchScript compilation for faster inference
31         dummy_input = torch.randn(1, 784).to(self.device)
32
33         with torch.no_grad():
34             traced_model = torch.jit.trace(self.model, dummy_input)
35             traced_model = torch.jit.optimize_for_inference(traced_model)
36
37         self.optimized_model = traced_model
38
39         # Warmup
40         self._warmup()
41
42         return self.optimized_model
43
44     def _warmup(self, n_iterations: int = 100):
45         """Warmup to eliminate first-request latency."""
46         dummy_input = torch.randn(1, 784).to(self.device)
47
48         with torch.no_grad():
49             for _ in range(n_iterations):
50                 _ = self.optimized_model(dummy_input)
51 
```

```

52     # Synchronize GPU
53     if self.device == "cuda":
54         torch.cuda.synchronize()
55
56     @torch.no_grad()
57     def predict_online(self, features: torch.Tensor) -> Dict:
58         """
59         Low-latency single prediction.
60
61         Target: <10ms p99 latency
62         """
63         start = time.perf_counter()
64
65         # Ensure batch size of 1
66         if features.dim() == 1:
67             features = features.unsqueeze(0)
68
69         features = features.to(self.device)
70
71         # Inference
72         output = self.optimized_model(features)
73
74         # Synchronize for accurate timing
75         if self.device == "cuda":
76             torch.cuda.synchronize()
77
78         latency_ms = (time.perf_counter() - start) * 1000
79
80         return {
81             'prediction': output.argmax(dim=1).item(),
82             'probabilities': output.softmax(dim=1).cpu().numpy().tolist()[0],
83             'latency_ms': latency_ms
84         }

```

Listing 11.12: Online serving with low-latency optimization

11.6.2 Batch Serving

```

1  import torch
2  import asyncio
3  from dataclasses import dataclass
4  from typing import List, Any
5  import time
6
7  @dataclass
8  class PendingRequest:
9      """Request waiting in batch queue."""
10     features: torch.Tensor
11     future: asyncio.Future
12     arrival_time: float
13
14     class DynamicBatchServer:
15         """
16         Dynamic batching for high-throughput serving.
17
18         Strategy:
19         - Accumulate requests up to max_batch_size

```

```

20 - Process when batch full or timeout reached
21 - Balance latency vs throughput
22
23 Benefits:
24 - 10-50x higher throughput than online serving
25 - Better GPU utilization
26 - Amortized inference cost
27 """
28
29 def __init__(self,
30             model: nn.Module,
31             max_batch_size: int = 32,
32             max_wait_ms: float = 10.0,
33             device: str = "cuda"):
34     self.model = model.to(device).eval()
35     self.device = device
36     self.max_batch_size = max_batch_size
37     self.max_wait_ms = max_wait_ms / 1000 # Convert to seconds
38
39     self.pending_requests: List[PendingRequest] = []
40     self.lock = asyncio.Lock()
41     self.batch_event = asyncio.Event()
42
43     # Start batch processor
44     self.processing_task = None
45
46 async def start(self):
47     """Start batch processing loop."""
48     self.processing_task = asyncio.create_task(self._process_batches())
49
50 async def stop(self):
51     """Stop batch processing."""
52     if self.processing_task:
53         self.processing_task.cancel()
54
55 async def predict(self, features: torch.Tensor) -> Dict:
56     """
57     Add request to batch queue and wait for result.
58
59     Args:
60         features: Input tensor
61
62     Returns:
63         Prediction result
64     """
65     # Create future for this request
66     future = asyncio.Future()
67
68     request = PendingRequest(
69         features=features,
70         future=future,
71         arrival_time=time.time()
72     )
73
74     # Add to queue
75     async with self.lock:
76         self.pending_requests.append(request)
77

```



```

78         # Trigger batch processing if queue full
79         if len(self.pending_requests) >= self.max_batch_size:
80             self.batch_event.set()
81
82         # Wait for result
83         return await future
84
85     async def _process_batches(self):
86         """Background task to process batches."""
87         while True:
88             try:
89                 # Wait for batch to be ready
90                 await asyncio.wait_for(
91                     self.batch_event.wait(),
92                     timeout=self.max_wait_ms
93                 )
94             except asyncio.TimeoutError:
95                 # Timeout reached - process whatever we have
96                 pass
97
98             # Get batch
99             async with self.lock:
100                 if len(self.pending_requests) == 0:
101                     self.batch_event.clear()
102                     continue
103
104                 # Take up to max_batch_size requests
105                 batch_requests = self.pending_requests[:self.max_batch_size]
106                 self.pending_requests = self.pending_requests[self.max_batch_size:]
107
108                 # Clear event if queue empty
109                 if len(self.pending_requests) < self.max_batch_size:
110                     self.batch_event.clear()
111
112             # Process batch
113             await self._execute_batch(batch_requests)
114
115     async def _execute_batch(self, requests: List[PendingRequest]):
116         """Execute inference on batch."""
117         if len(requests) == 0:
118             return
119
120         start_time = time.time()
121
122         try:
123             # Stack features into batch
124             features_batch = torch.stack([r.features for r in requests]).to(self.
device)
125
126             # Run batch inference
127             with torch.no_grad():
128                 outputs = self.model(features_batch)
129                 probabilities = torch.softmax(outputs, dim=1)
130                 predictions = torch.argmax(outputs, dim=1)
131
132             # Synchronize
133             if self.device == "cuda":
134                 torch.cuda.synchronize()

```

```

135         latency_ms = (time.time() - start_time) * 1000
136
137     # Return results to futures
138     for idx, request in enumerate(requests):
139         result = {
140             'prediction': predictions[idx].item(),
141             'probabilities': probabilities[idx].cpu().numpy().tolist(),
142             'latency_ms': latency_ms,
143             'batch_size': len(requests),
144             'wait_time_ms': (start_time - request.arrival_time) * 1000
145         }
146         request.future.set_result(result)
147
148     except Exception as e:
149         # Set exception on all futures
150         for request in requests:
151             request.future.set_exception(e)
152
153 # FastAPI integration with dynamic batching
154 @app.post("/predict/batch-optimized")
155 async def predict_with_dynamic_batching(request: PredictionRequest):
156     """
157     Prediction endpoint with dynamic batching.
158
159     - High throughput: 1000+ QPS
160     - Acceptable latency: 10-50ms p99
161     - Automatic batching transparent to client
162     """
163     features = torch.tensor(request.features, dtype=torch.float32)
164
165     # Use global batch server
166     result = await batch_server.predict(features)
167
168     return {
169         'prediction': result['prediction'],
170         'probabilities': result['probabilities'],
171         'latency_ms': result['latency_ms'],
172         'batch_size': result['batch_size'],
173         'wait_time_ms': result['wait_time_ms']
174     }
175
176 # Initialize batch server on startup
177 batch_server = None
178
179 @app.on_event("startup")
180 async def startup_batch_server():
181     global batch_server
182     model = ModelManager(model_path="model.pth")
183     model.load_model()
184
185     batch_server = DynamicBatchServer(
186         model=model.model,
187         max_batch_size=32,
188         max_wait_ms=10.0
189     )
190     await batch_server.start()

```

Listing 11.13: Dynamic batch serving for throughput optimization

11.6.3 Streaming Serving

```

1 from fastapi import WebSocket, WebSocketDisconnect
2 import json
3 import asyncio
4
5 class StreamingPredictor:
6     """
7     Handle streaming predictions over WebSocket.
8
9     Use cases:
10     - Real-time video analysis
11     - Live audio transcription
12     - Time-series forecasting
13     """
14
15     def __init__(self, model: nn.Module, device: str = "cuda"):
16         self.model = model.to(device).eval()
17         self.device = device
18
19     @torch.no_grad()
20     async def predict_stream(self, features: torch.Tensor) -> Dict:
21         """Single prediction in stream."""
22         features = features.to(self.device).unsqueeze(0)
23
24         output = self.model(features)
25         prediction = output.argmax(dim=1).item()
26         probabilities = torch.softmax(output, dim=1).cpu().numpy().tolist()[0]
27
28         return {
29             'prediction': prediction,
30             'probabilities': probabilities,
31             'timestamp': time.time()
32         }
33
34 streaming_predictor = None
35
36 @app.on_event("startup")
37 async def init_streaming():
38     global streaming_predictor
39     model = load_model("model.pth")
40     streaming_predictor = StreamingPredictor(model)
41
42 @app.websocket("/ws/predict")
43 async def websocket_predict(websocket: WebSocket):
44     """
45     WebSocket endpoint for streaming predictions.
46
47     Client sends:
48     {"features": [0.1, 0.2, ...]}
49
50     Server responds:
51     {"prediction": 5, "probabilities": [...], "timestamp": 1234567890.123}

```

```

52 """
53 await websocket.accept()
54
55 try:
56     while True:
57         # Receive data
58         data = await websocket.receive_text()
59         request_data = json.loads(data)
60
61         # Extract features
62         features = torch.tensor(request_data['features'], dtype=torch.float32)
63
64         # Predict
65         result = await streaming_predictor.predict_stream(features)
66
67         # Send response
68         await websocket.send_json(result)
69
70     except WebSocketDisconnect:
71         print("Client disconnected")
72     except Exception as e:
73         print(f"Error: {e}")
74         await websocket.close()

```

Listing 11.14: Streaming inference for continuous data

11.7 Model Optimization for Serving

Model optimization reduces latency and memory usage for production serving.

11.7.1 ONNX Conversion and Optimization

```

1 import torch
2 import torch.onnx
3 import onnx
4 import onnxruntime as ort
5 from onnxruntime.quantization import quantize_dynamic, QuantType
6 import numpy as np
7
8 class ONNXOptimizer:
9     """
10     Convert and optimize models with ONNX.
11
12     Benefits:
13     - 2-3x faster inference
14     - Smaller model size
15     - Cross-framework compatibility
16     - Hardware-specific optimizations
17     """
18
19     @staticmethod
20     def export_to_onnx(model: nn.Module,
21                       dummy_input: torch.Tensor,
22                       output_path: str,
23                       opset_version: int = 13):

```

```

24     """
25     Export PyTorch model to ONNX format.
26
27     Args:
28         model: PyTorch model
29         dummy_input: Example input for tracing
30         output_path: Path to save ONNX model
31         opset_version: ONNX opset version
32     """
33     model.eval()
34
35     # Export model
36     torch.onnx.export(
37         model,
38         dummy_input,
39         output_path,
40         export_params=True,
41         opset_version=opset_version,
42         do_constant_folding=True, # Optimize constants
43         input_names=['input'],
44         output_names=['output'],
45         dynamic_axes={
46             'input': {0: 'batch_size'},
47             'output': {0: 'batch_size'}
48         }
49     )
50
51     print(f"Model exported to {output_path}")
52
53     # Verify model
54     onnx_model = onnx.load(output_path)
55     onnx.checker.check_model(onnx_model)
56     print("ONNX model verified")
57
58     @staticmethod
59     def optimize_onnx(input_path: str, output_path: str):
60         """
61         Optimize ONNX model.
62
63         Optimizations:
64         - Constant folding
65         - Operator fusion
66         - Redundant node elimination
67         """
68         import onnxoptimizer
69
70         # Load model
71         model = onnx.load(input_path)
72
73         # Apply optimizations
74         passes = onnxoptimizer.get_available_passes()
75         optimized_model = onnxoptimizer.optimize(model, passes)
76
77         # Save optimized model
78         onnx.save(optimized_model, output_path)
79         print(f"Optimized model saved to {output_path}")
80
81     @staticmethod

```

```

82 def quantize_onnx(input_path: str, output_path: str):
83     """
84     Quantize ONNX model to INT8.
85
86     Benefits:
87     - 4x smaller model size
88     - 2-4x faster inference
89     - Minimal accuracy loss (<1%)
90     """
91     quantize_dynamic(
92         input_path,
93         output_path,
94         weight_type=QuantType.QInt8
95     )
96
97     print(f"Quantized model saved to {output_path}")
98
99 class ONNXRuntimePredictor:
100     """
101     High-performance inference with ONNX Runtime.
102
103     ONNX Runtime provides:
104     - Optimized kernels for CPU/GPU
105     - Automatic batching
106     - Multi-threading
107     """
108
109     def __init__(self, model_path: str, device: str = "cuda"):
110         self.device = device
111
112         # Create session
113         providers = ['CUDAExecutionProvider', 'CPUExecutionProvider'] if device == "
114         cuda" else ['CPUExecutionProvider']
115
116         self.session = ort.InferenceSession(
117             model_path,
118             providers=providers
119         )
120
121         # Get input/output names
122         self.input_name = self.session.get_inputs()[0].name
123         self.output_name = self.session.get_outputs()[0].name
124
125         print(f"ONNX Runtime session created")
126         print(f"Device: {self.session.get_providers()}")
127
128     def predict(self, features: np.ndarray) -> Dict:
129         """
130         Run inference with ONNX Runtime.
131
132         Args:
133             features: Input features (numpy array)
134
135         Returns:
136             Prediction results
137         """
138         start_time = time.time()

```

```

139     # Ensure correct shape
140     if features.ndim == 1:
141         features = features.reshape(1, -1)
142
143     features = features.astype(np.float32)
144
145     # Run inference
146     outputs = self.session.run(
147         [self.output_name],
148         {self.input_name: features}
149     )[0]
150
151     # Post-process
152     probabilities = self._softmax(outputs)
153     prediction = probabilities.argmax(axis=1)[0]
154
155     latency_ms = (time.time() - start_time) * 1000
156
157     return {
158         'prediction': int(prediction),
159         'probabilities': probabilities[0].tolist(),
160         'latency_ms': latency_ms
161     }
162
163     @staticmethod
164     def _softmax(x: np.ndarray) -> np.ndarray:
165         """Compute softmax."""
166         exp_x = np.exp(x - np.max(x, axis=1, keepdims=True))
167         return exp_x / exp_x.sum(axis=1, keepdims=True)
168
169     # Usage example
170     def optimize_model_with_onnx():
171         """Complete ONNX optimization pipeline."""
172         # Load PyTorch model
173         model = torch.load('model.pth')
174         model.eval()
175
176         # Export to ONNX
177         dummy_input = torch.randn(1, 784)
178         ONNXOptimizer.export_to_onnx(
179             model,
180             dummy_input,
181             'model.onnx'
182         )
183
184         # Optimize ONNX model
185         ONNXOptimizer.optimize_onnx(
186             'model.onnx',
187             'model_optimized.onnx'
188         )
189
190         # Quantize model
191         ONNXOptimizer.quantize_onnx(
192             'model_optimized.onnx',
193             'model_quantized.onnx'
194         )
195
196         # Test inference

```

```

197 predictor = ONNXRuntimePredictor('model_quantized.onnx')
198
199 test_input = np.random.randn(1, 784).astype(np.float32)
200 result = predictor.predict(test_input)
201
202 print(f"Prediction: {result['prediction']}")
203 print(f"Latency: {result['latency_ms']:.2f}ms")

```

Listing 11.15: Convert PyTorch model to ONNX

11.7.2 TensorRT Optimization

```

1 import tensorrt as trt
2 import pycuda.driver as cuda
3 import pycuda.autotinit
4 import numpy as np
5
6 class TensorRTOptimizer:
7     """
8     Optimize models with TensorRT for NVIDIA GPUs.
9
10    Benefits:
11    - 5-10x faster inference on NVIDIA GPUs
12    - Automatic kernel fusion
13    - Mixed precision (FP16/INT8)
14    - Layer-level optimizations
15    """
16
17    def __init__(self):
18        self.logger = trt.Logger(trt.Logger.WARNING)
19        self.builder = trt.Builder(self.logger)
20        self.network = None
21        self.config = None
22        self.engine = None
23
24    def build_engine_from_onnx(self,
25                              onnx_path: str,
26                              fp16_mode: bool = True,
27                              int8_mode: bool = False,
28                              max_batch_size: int = 32) -> trt.ICudaEngine:
29        """
30        Build TensorRT engine from ONNX model.
31
32        Args:
33            onnx_path: Path to ONNX model
34            fp16_mode: Enable FP16 precision
35            int8_mode: Enable INT8 precision (requires calibration)
36            max_batch_size: Maximum batch size
37
38        Returns:
39            TensorRT engine
40        """
41        # Create network
42        network_flags = 1 << int(trt.NetworkDefinitionCreationFlag.EXPLICIT_BATCH)
43        self.network = self.builder.create_network(network_flags)
44
45        # Parse ONNX

```



```

46     parser = trt.OnnxParser(self.network, self.logger)
47
48     with open(onnx_path, 'rb') as f:
49         if not parser.parse(f.read()):
50             print("ERROR: Failed to parse ONNX model")
51             for error in range(parser.num_errors):
52                 print(parser.get_error(error))
53             return None
54
55     # Create builder config
56     self.config = self.builder.create_builder_config()
57
58     # Set max workspace size (4GB)
59     self.config.max_workspace_size = 4 * (1 << 30)
60
61     # Enable optimizations
62     if fp16_mode and self.builder.platform_has_fast_fp16:
63         self.config.set_flag(trt.BuilderFlag.FP16)
64         print("FP16 mode enabled")
65
66     if int8_mode and self.builder.platform_has_fast_int8:
67         self.config.set_flag(trt.BuilderFlag.INT8)
68         # Note: INT8 requires calibration dataset
69         print("INT8 mode enabled (calibration required)")
70
71     # Set optimization profile
72     profile = self.builder.create_optimization_profile()
73
74     # Set dynamic shape range
75     input_name = self.network.get_input(0).name
76     profile.set_shape(
77         input_name,
78         min=(1, 784),      # Min batch size
79         opt=(8, 784),      # Optimal batch size
80         max=(max_batch_size, 784) # Max batch size
81     )
82
83     self.config.add_optimization_profile(profile)
84
85     # Build engine
86     print("Building TensorRT engine... This may take a few minutes.")
87     self.engine = self.builder.build_engine(self.network, self.config)
88
89     if self.engine is None:
90         print("ERROR: Failed to build engine")
91         return None
92
93     print("TensorRT engine built successfully")
94     return self.engine
95
96     def save_engine(self, engine: trt.ICudaEngine, output_path: str):
97         """Save TensorRT engine to file."""
98         with open(output_path, 'wb') as f:
99             f.write(engine.serialize())
100         print(f"Engine saved to {output_path}")
101
102     def load_engine(self, engine_path: str) -> trt.ICudaEngine:
103         """Load TensorRT engine from file."""

```

```

104     runtime = trt.Runtime(self.logger)
105
106     with open(engine_path, 'rb') as f:
107         engine = runtime.deserialize_cuda_engine(f.read())
108
109     print(f"Engine loaded from {engine_path}")
110     return engine
111
112 class TensorRTPredictor:
113     """
114     High-performance inference with TensorRT.
115
116     Performance:
117     - FP32: 2-3x faster than PyTorch
118     - FP16: 5-7x faster than PyTorch
119     - INT8: 8-10x faster than PyTorch
120     """
121
122     def __init__(self, engine: trt.ICudaEngine):
123         self.engine = engine
124         self.context = engine.create_execution_context()
125
126         # Allocate buffers
127         self.inputs = []
128         self.outputs = []
129         self.bindings = []
130         self.stream = cuda.Stream()
131
132         for binding in engine:
133             size = trt.volume(engine.get_binding_shape(binding))
134             dtype = trt.nptype(engine.get_binding_dtype(binding))
135
136             # Allocate host and device buffers
137             host_mem = cuda.pagelocked_empty(size, dtype)
138             device_mem = cuda.mem_alloc(host_mem.nbytes)
139
140             self.bindings.append(int(device_mem))
141
142             if engine.binding_is_input(binding):
143                 self.inputs.append({'host': host_mem, 'device': device_mem})
144             else:
145                 self.outputs.append({'host': host_mem, 'device': device_mem})
146
147     def predict(self, features: np.ndarray) -> Dict:
148         """
149         Run inference with TensorRT.
150
151         Args:
152             features: Input features
153
154         Returns:
155             Prediction results
156         """
157         start_time = time.time()
158
159         # Ensure correct shape
160         if features.ndim == 1:
161             features = features.reshape(1, -1)

```

```

162     features = features.astype(np.float32)
163
164     # Copy input to device
165     np.copyto(self.inputs[0]['host'], features.ravel())
166     cuda.memcpy_htod_async(
167         self.inputs[0]['device'],
168         self.inputs[0]['host'],
169         self.stream
170     )
171
172     # Run inference
173     self.context.execute_async_v2(
174         bindings=self.bindings,
175         stream_handle=self.stream.handle
176     )
177
178     # Copy output to host
179     cuda.memcpy_dtoh_async(
180         self.outputs[0]['host'],
181         self.outputs[0]['device'],
182         self.stream
183     )
184
185     # Synchronize
186     self.stream.synchronize()
187
188     # Get output
189     output = self.outputs[0]['host'].reshape(features.shape[0], -1)
190
191     # Post-process
192     probabilities = self._softmax(output)
193     prediction = probabilities.argmax(axis=1)[0]
194
195     latency_ms = (time.time() - start_time) * 1000
196
197     return {
198         'prediction': int(prediction),
199         'probabilities': probabilities[0].tolist(),
200         'latency_ms': latency_ms
201     }
202
203
204     @staticmethod
205     def _softmax(x: np.ndarray) -> np.ndarray:
206         """Compute softmax."""
207         exp_x = np.exp(x - np.max(x, axis=1, keepdims=True))
208         return exp_x / exp_x.sum(axis=1, keepdims=True)
209
210     # Usage example
211     def optimize_with_tensorrt():
212         """Complete TensorRT optimization pipeline."""
213         # Step 1: Convert PyTorch to ONNX (from previous section)
214         # ...
215
216         # Step 2: Build TensorRT engine
217         optimizer = TensorRTOptimizer()
218
219         engine = optimizer.build_engine_from_onnx(

```

```

220     onnx_path='model.onnx',
221     fp16_mode=True,  # Use FP16 for faster inference
222     max_batch_size=32
223 )
224
225 # Save engine
226 optimizer.save_engine(engine, 'model.trt')
227
228 # Step 3: Run inference
229 predictor = TensorRTPredictor(engine)
230
231 test_input = np.random.randn(1, 784).astype(np.float32)
232 result = predictor.predict(test_input)
233
234 print(f"Prediction: {result['prediction']}")
235 print(f"Latency: {result['latency_ms']:.2f}ms")
236 print("Expected speedup: 5-7x vs PyTorch FP32")

```

Listing 11.16: TensorRT optimization for NVIDIA GPUs

11.8 Load Balancing and Scaling Strategies

Effective load balancing and autoscaling ensure reliable, cost-effective serving at scale.

11.8.1 Load Balancing with NGINX

```

1  # nginx.conf for ML model serving
2
3  upstream model_servers {
4      # Load balancing method
5      least_conn;  # Route to server with fewest active connections
6
7      # Backend servers
8      server 192.168.1.10:8000 weight=1 max_fails=3 fail_timeout=30s;
9      server 192.168.1.11:8000 weight=1 max_fails=3 fail_timeout=30s;
10     server 192.168.1.12:8000 weight=2 max_fails=3 fail_timeout=30s;  # More powerful
11
12     # Health check backend
13     server 192.168.1.13:8000 backup;  # Only used if others fail
14
15     # Connection pooling
16     keepalive 32;
17 }
18
19 server {
20     listen 80;
21     server_name api.ml-serving.com;
22
23     # Access logging
24     access_log /var/log/nginx/ml_serving_access.log;
25     error_log /var/log/nginx/ml_serving_error.log;
26
27     # Timeouts
28     proxy_connect_timeout 5s;
29     proxy_send_timeout 60s;

```

```

30 proxy_read_timeout 60s;
31
32 # Buffer settings
33 client_body_buffer_size 10M;
34 client_max_body_size 10M;
35
36 # Rate limiting
37 limit_req_zone $binary_remote_addr zone=api_limit:10m rate=100r/s;
38 limit_req zone=api_limit burst=20 nodelay;
39
40 # Prediction endpoint
41 location /predict {
42     proxy_pass http://model_servers;
43
44     # Headers
45     proxy_set_header Host $host;
46     proxy_set_header X-Real-IP $remote_addr;
47     proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
48
49     # Connection pooling
50     proxy_http_version 1.1;
51     proxy_set_header Connection "";
52
53     # Response caching (optional)
54     proxy_cache ml_cache;
55     proxy_cache_valid 200 10m;
56     proxy_cache_key "$request_uri|$request_body";
57 }
58
59 # Health check endpoint
60 location /health {
61     proxy_pass http://model_servers;
62     access_log off;
63 }
64
65 # Metrics endpoint (internal only)
66 location /metrics {
67     proxy_pass http://model_servers;
68     allow 192.168.1.0/24;
69     deny all;
70 }
71 }
72
73 # Cache configuration
74 proxy_cache_path /var/cache/nginx/ml_cache levels=1:2 keys_zone=ml_cache:100m max_size
    =1g inactive=60m;

```

Listing 11.17: NGINX configuration for model serving

11.8.2 Kubernetes Deployment and Autoscaling

```

1 # deployment.yaml
2 apiVersion: apps/v1
3 kind: Deployment
4 metadata:
5   name: ml-serving
6   labels:

```

```
7   app: ml-serving
8 spec:
9   replicas: 3
10  selector:
11    matchLabels:
12      app: ml-serving
13  template:
14    metadata:
15      labels:
16        app: ml-serving
17    spec:
18      containers:
19        - name: ml-serving
20          image: ml-serving:v1.0.0
21          ports:
22            - containerPort: 8000
23              name: http
24          resources:
25            requests:
26              memory: "2Gi"
27              cpu: "1000m"
28              nvidia.com/gpu: "1"
29            limits:
30              memory: "4Gi"
31              cpu: "2000m"
32              nvidia.com/gpu: "1"
33          env:
34            - name: MODEL_PATH
35              value: "/models/model.pth"
36            - name: WORKERS
37              value: "4"
38            - name: BATCH_SIZE
39              value: "32"
40          volumeMounts:
41            - name: model-storage
42              mountPath: /models
43              readOnly: true
44          livenessProbe:
45            httpGet:
46              path: /health
47              port: 8000
48            initialDelaySeconds: 30
49            periodSeconds: 10
50            timeoutSeconds: 5
51            failureThreshold: 3
52          readinessProbe:
53            httpGet:
54              path: /health
55              port: 8000
56            initialDelaySeconds: 10
57            periodSeconds: 5
58            timeoutSeconds: 3
59            failureThreshold: 2
60          volumes:
61            - name: model-storage
62              persistentVolumeClaim:
63                claimName: model-pvc
64
```

```
65 ---
66 # service.yaml
67 apiVersion: v1
68 kind: Service
69 metadata:
70   name: ml-serving-service
71 spec:
72   type: LoadBalancer
73   selector:
74     app: ml-serving
75   ports:
76   - protocol: TCP
77     port: 80
78     targetPort: 8000
79
80 ---
81 # hpa.yaml - Horizontal Pod Autoscaler
82 apiVersion: autoscaling/v2
83 kind: HorizontalPodAutoscaler
84 metadata:
85   name: ml-serving-hpa
86 spec:
87   scaleTargetRef:
88     apiVersion: apps/v1
89     kind: Deployment
90     name: ml-serving
91   minReplicas: 2
92   maxReplicas: 20
93   metrics:
94   - type: Resource
95     resource:
96       name: cpu
97       target:
98         type: Utilization
99         averageUtilization: 70
100   - type: Resource
101     resource:
102       name: memory
103       target:
104         type: Utilization
105         averageUtilization: 80
106   - type: Pods
107     pods:
108       metric:
109         name: http_requests_per_second
110       target:
111         type: AverageValue
112         averageValue: "1000"
113   behavior:
114     scaleDown:
115       stabilizationWindowSeconds: 300
116       policies:
117       - type: Percent
118         value: 50
119         periodSeconds: 60
120     scaleUp:
121       stabilizationWindowSeconds: 0
122       policies:
```

```

123     - type: Percent
124       value: 100
125       periodSeconds: 15
126     - type: Pods
127       value: 4
128       periodSeconds: 15
129     selectPolicy: Max
130
131 ---
132 # PodDisruptionBudget for high availability
133 apiVersion: policy/v1
134 kind: PodDisruptionBudget
135 metadata:
136   name: ml-serving-pdb
137 spec:
138   minAvailable: 2
139   selector:
140     matchLabels:
141       app: ml-serving

```

Listing 11.18: Kubernetes deployment for model serving

11.8.3 Auto-scaling Based on Custom Metrics

```

1 from prometheus_client import Counter, Histogram, Gauge, start_http_server
2 import time
3
4 # Prometheus metrics
5 REQUEST_COUNT = Counter(
6     'ml_serving_requests_total',
7     'Total number of prediction requests',
8     ['method', 'endpoint', 'status']
9 )
10
11 REQUEST_LATENCY = Histogram(
12     'ml_serving_request_latency_seconds',
13     'Request latency in seconds',
14     ['endpoint'],
15     buckets=[0.001, 0.005, 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1.0, 2.5, 5.0]
16 )
17
18 ACTIVE_REQUESTS = Gauge(
19     'ml_serving_active_requests',
20     'Number of requests currently being processed'
21 )
22
23 BATCH_SIZE = Histogram(
24     'ml_serving_batch_size',
25     'Size of prediction batches',
26     buckets=[1, 2, 4, 8, 16, 32, 64, 128]
27 )
28
29 GPU_UTILIZATION = Gauge(
30     'ml_serving_gpu_utilization_percent',
31     'GPU utilization percentage'
32 )
33

```



```

34 MODEL_LOAD_TIME = Gauge(
35     'ml_serving_model_load_time_seconds',
36     'Time taken to load model'
37 )
38
39 class MetricsMiddleware:
40     """
41     Middleware to track serving metrics.
42
43     Metrics exposed on /metrics endpoint for Prometheus scraping.
44     """
45
46     def __init__(self, app):
47         self.app = app
48
49         # Start Prometheus metrics server
50         start_http_server(9090) # Metrics on port 9090
51
52     async def __call__(self, scope, receive, send):
53         if scope['type'] != 'http':
54             return await self.app(scope, receive, send)
55
56         # Track active requests
57         ACTIVE_REQUESTS.inc()
58
59         # Measure latency
60         start_time = time.time()
61
62         try:
63             await self.app(scope, receive, send)
64             status = 200 # Simplified
65
66         except Exception as e:
67             status = 500
68             raise
69
70         finally:
71             # Record metrics
72             latency = time.time() - start_time
73             endpoint = scope['path']
74
75             REQUEST_LATENCY.labels(endpoint=endpoint).observe(latency)
76             REQUEST_COUNT.labels(
77                 method=scope['method'],
78                 endpoint=endpoint,
79                 status=status
80             ).inc()
81             ACTIVE_REQUESTS.dec()
82
83     # Add to FastAPI app
84     app.add_middleware(MetricsMiddleware)
85
86     # GPU monitoring
87     def monitor_gpu_utilization():
88         """Background task to monitor GPU utilization."""
89         import pynvml
90
91         pynvml.nvmlInit()

```

```

92     handle = pynvml.nvmlDeviceGetHandleByIndex(0)
93
94     while True:
95         try:
96             utilization = pynvml.nvmlDeviceGetUtilizationRates(handle)
97             GPU_UTILIZATION.set(utilization.gpu)
98
99         except Exception as e:
100             print(f"Error monitoring GPU: {e}")
101
102             time.sleep(5)  # Update every 5 seconds
103
104 # Start GPU monitoring
105 import threading
106 gpu_monitor_thread = threading.Thread(target=monitor_gpu_utilization, daemon=True)
107 gpu_monitor_thread.start()

```

Listing 11.19: Custom metrics for autoscaling

11.9 Performance Optimization and Profiling

Systematic profiling identifies bottlenecks and guides optimization efforts.

11.9.1 Latency Profiling

```

1  import time
2  from contextlib import contextmanager
3  from typing import Dict, List
4  import statistics
5
6  class LatencyProfiler:
7      """
8      Profile latency breakdown for serving pipeline.
9
10     Tracks:
11     - Preprocessing time
12     - Model inference time
13     - Postprocessing time
14     - Total end-to-end latency
15     """
16
17     def __init__(self):
18         self.measurements: Dict[str, List[float]] = {
19             'preprocessing': [],
20             'inference': [],
21             'postprocessing': [],
22             'total': []
23         }
24
25     @contextmanager
26     def measure(self, stage: str):
27         """Context manager to measure stage latency."""
28         start = time.perf_counter()
29         try:
30             yield

```

```

31         finally:
32             elapsed = (time.perf_counter() - start) * 1000 # ms
33             self.measurements[stage].append(elapsed)
34
35     def get_stats(self) -> Dict:
36         """Get latency statistics."""
37         stats = {}
38
39         for stage, measurements in self.measurements.items():
40             if len(measurements) == 0:
41                 continue
42
43             stats[stage] = {
44                 'mean': statistics.mean(measurements),
45                 'median': statistics.median(measurements),
46                 'p95': self._percentile(measurements, 95),
47                 'p99': self._percentile(measurements, 99),
48                 'min': min(measurements),
49                 'max': max(measurements),
50                 'count': len(measurements)
51             }
52
53         return stats
54
55     @staticmethod
56     def _percentile(data: List[float], percentile: float) -> float:
57         """Calculate percentile."""
58         sorted_data = sorted(data)
59         index = int(len(sorted_data) * percentile / 100)
60         return sorted_data[min(index, len(sorted_data) - 1)]
61
62     def print_report(self):
63         """Print profiling report."""
64         stats = self.get_stats()
65
66         print("\n" + "=" * 80)
67         print("LATENCY PROFILING REPORT")
68         print("=" * 80)
69
70         for stage, metrics in stats.items():
71             print(f"\n{stage.upper()}:")
72             print(f"    Mean:    {metrics['mean']:.2f} ms")
73             print(f"    Median:  {metrics['median']:.2f} ms")
74             print(f"    P95:    {metrics['p95']:.2f} ms")
75             print(f"    P99:    {metrics['p99']:.2f} ms")
76             print(f"    Min:    {metrics['min']:.2f} ms")
77             print(f"    Max:    {metrics['max']:.2f} ms")
78             print(f"    Samples: {metrics['count']}")
79
80         print("\n" + "=" * 80)
81
82     # Usage in serving pipeline
83     profiler = LatencyProfiler()
84
85     async def predict_with_profiling(features: np.ndarray) -> Dict:
86         """Prediction with latency profiling."""
87
88         with profiler.measure('total'):

```

```

89     # Preprocessing
90     with profiler.measure('preprocessing'):
91         features = preprocess(features)
92
93     # Inference
94     with profiler.measure('inference'):
95         output = model(features)
96
97     # Postprocessing
98     with profiler.measure('postprocessing'):
99         result = postprocess(output)
100
101     return result
102
103 # After processing 1000 requests
104 profiler.print_report()

```

Listing 11.20: Detailed latency profiling

11.9.2 Throughput Optimization

```

1  import torch
2  import torch.nn as nn
3  from typing import List
4  import numpy as np
5
6  class ThroughputOptimizer:
7      """
8      Optimize serving throughput.
9
10     Techniques:
11     - Optimal batch size selection
12     - Multi-GPU serving
13     - Model parallelism
14     - Request queueing
15     """
16
17     @staticmethod
18     def find_optimal_batch_size(model: nn.Module,
19                               input_shape: tuple,
20                               device: str = "cuda",
21                               max_batch_size: int = 128) -> int:
22         """
23         Find optimal batch size for maximum throughput.
24
25         Strategy:
26         - Test increasing batch sizes
27         - Measure throughput (samples/sec)
28         - Return batch size with highest throughput
29         """
30         model = model.to(device).eval()
31         results = []
32
33         batch_sizes = [1, 2, 4, 8, 16, 32, 64, 128]
34         batch_sizes = [bs for bs in batch_sizes if bs <= max_batch_size]
35
36         print("Finding optimal batch size...")

```

```

37
38     for batch_size in batch_sizes:
39         try:
40             # Create dummy batch
41             dummy_input = torch.randn(batch_size, *input_shape).to(device)
42
43             # Warmup
44             with torch.no_grad():
45                 for _ in range(10):
46                     _ = model(dummy_input)
47
48             # Measure throughput
49             n_iterations = 100
50             start_time = time.time()
51
52             with torch.no_grad():
53                 for _ in range(n_iterations):
54                     _ = model(dummy_input)
55
56             # Synchronize
57             if device == "cuda":
58                 torch.cuda.synchronize()
59
60             elapsed = time.time() - start_time
61             throughput = (n_iterations * batch_size) / elapsed
62
63             results.append({
64                 'batch_size': batch_size,
65                 'throughput': throughput,
66                 'latency': elapsed / n_iterations * 1000
67             })
68
69             print(f" Batch size {batch_size:3d}: {throughput:8.1f} samples/sec, "
70                   f"{elapsed / n_iterations * 1000:.2f} ms/batch")
71
72         except RuntimeError as e:
73             if "out of memory" in str(e):
74                 print(f" Batch size {batch_size:3d}: OOM")
75                 break
76             raise
77
78     # Find optimal
79     if len(results) == 0:
80         return 1
81
82     optimal = max(results, key=lambda x: x['throughput'])
83     print(f"\nOptimal batch size: {optimal['batch_size']}")
84     print(f"Max throughput: {optimal['throughput']:.1f} samples/sec")
85
86     return optimal['batch_size']
87
88 class MultiGPUServing:
89     """
90     Serve model across multiple GPUs for higher throughput.
91
92     Strategy:
93     - Replicate model on each GPU
94     - Round-robin request distribution

```

```

95     - Independent inference on each GPU
96     """
97
98     def __init__(self, model: nn.Module, n_gpus: int = None):
99         if n_gpus is None:
100             n_gpus = torch.cuda.device_count()
101
102         self.n_gpus = n_gpus
103         self.models = []
104
105         # Create model replica on each GPU
106         for gpu_id in range(n_gpus):
107             model_replica = model.to(f'cuda:{gpu_id}')
108             model_replica.eval()
109             self.models.append(model_replica)
110
111         self.current_gpu = 0
112         print(f"MultiGPU serving initialized with {n_gpus} GPUs")
113
114     @torch.no_grad()
115     def predict(self, features: np.ndarray) -> Dict:
116         """
117         Predict using next available GPU (round-robin).
118
119         Args:
120             features: Input features
121
122         Returns:
123             Prediction result
124         """
125         # Select GPU
126         gpu_id = self.current_gpu
127         self.current_gpu = (self.current_gpu + 1) % self.n_gpus
128
129         # Move to GPU and run inference
130         device = f'cuda:{gpu_id}'
131         x = torch.from_numpy(features).float().unsqueeze(0).to(device)
132
133         output = self.models[gpu_id](x)
134         prediction = output.argmax(dim=1).item()
135         probabilities = torch.softmax(output, dim=1).cpu().numpy().tolist()[0]
136
137         return {
138             'prediction': prediction,
139             'probabilities': probabilities,
140             'gpu_id': gpu_id
141         }
142
143     # Usage
144     optimizer = ThroughputOptimizer()
145     optimal_bs = optimizer.find_optimal_batch_size(
146         model=my_model,
147         input_shape=(784,),
148         device='cuda'
149     )
150
151     # Multi-GPU setup
152     multi_gpu_server = MultiGPUServing(model=my_model, n_gpus=4)

```

Listing 11.21: Throughput optimization techniques

11.9.3 Memory Optimization

```

1 import torch
2 import gc
3
4 class MemoryOptimizer:
5     """
6     Optimize memory usage in serving.
7
8     Techniques:
9     - Model quantization
10    - Gradient disabled
11    - Memory pooling
12    - Periodic garbage collection
13    """
14
15    @staticmethod
16    def optimize_model_memory(model: nn.Module) -> nn.Module:
17        """Optimize model for minimal memory footprint."""
18
19        # Ensure eval mode (no gradients)
20        model.eval()
21
22        # Disable gradient computation globally
23        for param in model.parameters():
24            param.requires_grad = False
25
26        # Use inference mode for better performance
27        model = torch.jit.optimize_for_inference(
28            torch.jit.script(model)
29        )
30
31        return model
32
33    @staticmethod
34    def monitor_memory():
35        """Monitor GPU memory usage."""
36        if torch.cuda.is_available():
37            allocated = torch.cuda.memory_allocated() / 1024**3 # GB
38            reserved = torch.cuda.memory_reserved() / 1024**3
39            max_allocated = torch.cuda.max_memory_allocated() / 1024**3
40
41            print(f"GPU Memory:")
42            print(f"  Allocated: {allocated:.2f} GB")
43            print(f"  Reserved: {reserved:.2f} GB")
44            print(f"  Peak: {max_allocated:.2f} GB")
45
46    @staticmethod
47    def clear_memory():
48        """Clear unused memory."""
49        gc.collect()
50        if torch.cuda.is_available():
51            torch.cuda.empty_cache()

```

```

52         torch.cuda.synchronize()
53
54     @staticmethod
55     def setup_memory_pooling():
56         """Configure memory allocator for better reuse."""
57         if torch.cuda.is_available():
58             # Use caching allocator
59             torch.cuda.set_per_process_memory_fraction(0.9, device=0)
60
61             # Pre-allocate memory pool
62             torch.cuda.empty_cache()
63
64     # Periodic memory cleanup
65     import threading
66
67     def periodic_memory_cleanup(interval: int = 300):
68         """Clean up memory every interval seconds."""
69         while True:
70             time.sleep(interval)
71             MemoryOptimizer.clear_memory()
72             print("Memory cleanup completed")
73
74     cleanup_thread = threading.Thread(target=periodic_memory_cleanup, daemon=True)
75     cleanup_thread.start()

```

Listing 11.22: Memory optimization for serving

11.10 Chapter Summary

This chapter covered production model serving architecture and implementation:

Serving Frameworks:

- FastAPI provides flexible REST APIs with async support, request validation, and middleware
- gRPC offers high-performance binary protocol for low-latency serving
- TensorFlow Serving and TorchServe provide production-grade frameworks with batching and versioning

Serving Patterns:

- Online serving optimized for low latency (<10ms p99) with model warmup and TorchScript compilation
- Batch serving maximizes throughput (10-50x vs online) with dynamic batching and optimal wait times
- Streaming serving handles continuous data with WebSocket connections for real-time applications

Model Optimization:

- ONNX conversion provides 2-3x speedup with cross-framework compatibility

- TensorRT delivers 5-10x speedup on NVIDIA GPUs with FP16/INT8 precision
- Quantization reduces model size by 4x with minimal accuracy loss

Infrastructure:

- Load balancing with NGINX distributes traffic and provides health checks
- Kubernetes enables horizontal autoscaling based on CPU, memory, and custom metrics
- Prometheus metrics track latency, throughput, and resource utilization

Performance Optimization:

- Profiling identifies bottlenecks in preprocessing, inference, and postprocessing
- Optimal batch size selection maximizes throughput for given hardware
- Multi-GPU serving increases capacity with model replication
- Memory optimization reduces footprint through quantization and efficient allocation

Key Takeaways:

1. Choose serving pattern based on latency requirements: online for <10ms, batch for throughput
2. Optimize models with ONNX/TensorRT before deployment for 2-10x speedup
3. Implement caching for repeated requests to reduce inference cost
4. Use dynamic batching to balance latency and throughput automatically
5. Monitor metrics continuously and autoscale based on load patterns
6. Profile systematically to identify and eliminate bottlenecks
7. Design for high availability with health checks, load balancing, and redundancy

11.11 Exercises

Exercise 11.1. FastAPI Model Serving

Build a production-ready FastAPI service for image classification:

1. Create endpoints for single and batch predictions
2. Implement request validation with Pydantic schemas
3. Add Redis caching for predictions
4. Configure rate limiting (100 req/min per IP)
5. Add health check and metrics endpoints

6. Implement graceful shutdown and model warmup
7. Write Docker configuration with multi-stage build
8. Test with locust for 1000 QPS

Deliverables:

- FastAPI application code
- Dockerfile and docker-compose.yml
- Load test results showing p50/p95/p99 latency
- Prometheus metrics dashboard configuration

Success Criteria:

- P99 latency < 50ms for single predictions
- Throughput > 500 QPS on single GPU
- Cache hit rate > 30% on repeated requests
- Zero downtime during graceful shutdown

Exercise 11.2. gRPC High-Performance Serving

Implement gRPC service with streaming support:

1. Define protocol buffers for prediction service
2. Implement server with single, batch, and streaming RPCs
3. Add interceptors for logging and metrics
4. Configure connection pooling and keepalive
5. Implement client with automatic retry and circuit breaking
6. Compare latency vs REST API
7. Test streaming with 1000 concurrent connections

Expected Results:

- gRPC 30-50% lower latency than REST
- Streaming handles 1000+ concurrent connections
- Automatic reconnection on failures
- Request/response compression reduces bandwidth by 60%

Exercise 11.3. Model Optimization Pipeline

Create complete optimization pipeline from PyTorch to TensorRT:

1. Train baseline PyTorch model on CIFAR-10

2. Export to ONNX with dynamic axes
3. Apply ONNX optimizations (fusion, folding)
4. Quantize model to INT8 with calibration data
5. Build TensorRT engine with FP16 and INT8
6. Compare accuracy, latency, and throughput
7. Measure memory usage for each variant
8. Document speedup and accuracy trade-offs

Target Metrics:

- ONNX FP32: 2-3x faster than PyTorch
- TensorRT FP16: 5-7x faster than PyTorch
- TensorRT INT8: 8-10x faster, <1% accuracy loss
- Memory reduction: 4x with INT8 quantization

Exercise 11.4. Dynamic Batch Serving

Implement dynamic batching for optimal throughput:

1. Create async batch server with configurable parameters
2. Implement queue with max size and timeout
3. Add metrics for batch sizes and wait times
4. Tune `max_batch_size` and `max_wait_ms` for optimal latency/throughput
5. Test with variable load patterns
6. Compare vs non-batched serving
7. Measure GPU utilization improvement

Optimization Goals:

- 10x throughput increase vs online serving
- P99 latency <50ms with batching
- GPU utilization >80%
- Handle traffic spikes smoothly

Exercise 11.5. Kubernetes Deployment with Autoscaling

Deploy model serving on Kubernetes with HPA:

1. Create Deployment with resource requests/limits
2. Configure HorizontalPodAutoscaler with CPU and custom metrics

3. Set up Prometheus for metrics collection
4. Implement PodDisruptionBudget for availability
5. Add liveness and readiness probes
6. Test autoscaling under load
7. Configure cluster autoscaling
8. Implement blue-green deployment strategy

Requirements:

- Scale from 2 to 20 pods based on load
- Average CPU utilization 60-70%
- Zero-downtime deployments
- Handle 10x traffic spikes within 2 minutes

Exercise 11.6. Multi-Model Serving System

Build system serving multiple models with routing:

1. Implement ModelRegistry to manage multiple models
2. Add version-based routing (/predict/v1, /predict/v2)
3. Implement A/B testing with traffic splitting
4. Add canary deployments (5% traffic to new version)
5. Monitor per-model metrics separately
6. Implement automatic rollback on errors
7. Support model hot-swapping without downtime

Features to Implement:

- Concurrent serving of 5+ model versions
- Traffic splitting: 90% v1, 10% v2
- Automatic rollback if v2 error rate >1%
- Model loading/unloading without service restart

Exercise 11.7. Performance Profiling and Optimization

Systematically profile and optimize serving pipeline:

1. Instrument code with detailed latency measurements
2. Profile preprocessing, inference, and postprocessing
3. Identify bottlenecks using flamegraphs

4. Optimize top 3 bottlenecks
5. Find optimal batch size experimentally
6. Configure multi-GPU serving
7. Implement memory optimizations
8. Document before/after metrics

Optimization Targets:

- 50% reduction in p99 latency
- 2x throughput improvement
- 30% memory reduction
- Detailed performance report with flamegraphs

Exercise 11.8. Production Monitoring and Alerting

Build comprehensive monitoring for serving system:

1. Export Prometheus metrics (latency, throughput, errors)
2. Create Grafana dashboards for key metrics
3. Set up alerts for SLO violations
4. Implement distributed tracing with Jaeger
5. Add structured logging with correlation IDs
6. Monitor model drift and data quality
7. Set up on-call runbooks
8. Test incident response procedures

Monitoring Requirements:

- Alert on p99 latency $>100\text{ms}$
- Alert on error rate $>0.1\%$
- Alert on throughput drop $>50\%$
- Track model prediction distribution drift
- End-to-end request tracing

Chapter 12

Deployment Strategies and Patterns

12.1 Learning Objectives

After completing this chapter, readers will be able to:

- Implement blue-green deployments for zero-downtime model updates
- Design and execute canary deployment strategies with gradual rollout
- Build A/B testing frameworks for rigorous model evaluation
- Implement shadow deployments for risk-free production validation
- Design traffic splitting and routing mechanisms
- Handle model rollback scenarios with automated circuit breakers
- Build multi-model serving architectures for complex ML systems
- Monitor deployment health and make data-driven rollout decisions

12.2 Introduction

Deploying machine learning models to production is fundamentally different from deploying traditional software. Models are probabilistic, their behavior changes with data drift, and their quality must be continuously monitored. A deployment strategy determines how a new model version transitions from development to production, balancing risk management with rapid iteration.

Poor deployment strategies lead to production incidents, degraded user experience, and eroded trust in ML systems. A well-designed deployment strategy enables:

- **Risk mitigation:** Gradual rollouts detect issues before full deployment
- **Fast rollback:** Quick recovery when problems arise
- **Experimentation:** A/B testing validates improvements rigorously
- **Zero downtime:** Seamless transitions between model versions
- **Validation:** Shadow deployments test models without user impact

This chapter covers production-grade deployment strategies with complete implementations, from basic blue-green deployments to sophisticated multi-armed bandit allocation systems.

12.3 Blue-Green Deployment

12.3.1 Overview and Architecture

Blue-green deployment maintains two identical production environments: blue (currently serving traffic) and green (new version). Traffic switches atomically from blue to green after validation, enabling instant rollback by switching back.

For ML models, blue-green deployment provides:

- Zero-downtime updates
- Instant rollback capability
- Simple orchestration
- Clear separation between versions

12.3.2 Implementation

```

1 from typing import Dict, Optional, Callable, Any
2 from dataclasses import dataclass
3 from enum import Enum
4 from datetime import datetime
5 import logging
6 import requests
7 import time
8
9 class Environment(Enum):
10     """Deployment environment identifiers"""
11     BLUE = "blue"
12     GREEN = "green"
13
14 @dataclass
15 class ModelEndpoint:
16     """Model serving endpoint configuration"""
17     name: str
18     url: str
19     version: str
20     environment: Environment
21     health_check_url: str
22     metrics_url: Optional[str] = None
23
24 @dataclass
25 class DeploymentHealth:
26     """Health check results for a deployment"""
27     is_healthy: bool
28     latency_p50: float
29     latency_p95: float
30     latency_p99: float
31     error_rate: float

```



```

32     throughput: float
33     timestamp: datetime
34
35 class BlueGreenDeployment:
36     """
37     Blue-Green deployment manager for ML models
38
39     Manages two parallel environments and atomic traffic switching
40     """
41
42     def __init__(self,
43                 model_name: str,
44                 load_balancer_url: str,
45                 health_check_interval: int = 30):
46         """
47         Initialize blue-green deployment manager
48
49         Args:
50             model_name: Name of the model being deployed
51             load_balancer_url: URL of the load balancer
52             health_check_interval: Seconds between health checks
53         """
54         self.model_name = model_name
55         self.load_balancer_url = load_balancer_url
56         self.health_check_interval = health_check_interval
57         self.logger = logging.getLogger(f"BlueGreen_{model_name}")
58
59         self.blue_endpoint: Optional[ModelEndpoint] = None
60         self.green_endpoint: Optional[ModelEndpoint] = None
61         self.active_environment = Environment.BLUE
62
63     def deploy_to_inactive(self,
64                          version: str,
65                          endpoint_url: str,
66                          health_check_url: str) -> bool:
67         """
68         Deploy new model version to inactive environment
69
70         Args:
71             version: Model version identifier
72             endpoint_url: URL where model is serving
73             health_check_url: Health check endpoint
74
75         Returns:
76             True if deployment successful
77         """
78         inactive_env = self._get_inactive_environment()
79
80         self.logger.info(f"Deploying version {version} to {inactive_env.value}")
81
82         # Create endpoint configuration
83         endpoint = ModelEndpoint(
84             name=self.model_name,
85             url=endpoint_url,
86             version=version,
87             environment=inactive_env,
88             health_check_url=health_check_url
89         )

```

```

90
91     # Set the inactive endpoint
92     if inactive_env == Environment.BLUE:
93         self.blue_endpoint = endpoint
94     else:
95         self.green_endpoint = endpoint
96
97     # Verify deployment health
98     if not self._wait_for_healthy(endpoint):
99         self.logger.error(f"Deployment to {inactive_env.value} failed health check
100 ")
101         return False
102
103     self.logger.info(f"Successfully deployed to {inactive_env.value}")
104     return True
105
106 def _get_inactive_environment(self) -> Environment:
107     """Get the currently inactive environment"""
108     return (Environment.GREEN if self.active_environment == Environment.BLUE
109             else Environment.BLUE)
110
111 def _get_active_endpoint(self) -> Optional[ModelEndpoint]:
112     """Get currently active endpoint"""
113     return (self.blue_endpoint if self.active_environment == Environment.BLUE
114             else self.green_endpoint)
115
116 def _get_inactive_endpoint(self) -> Optional[ModelEndpoint]:
117     """Get currently inactive endpoint"""
118     return (self.green_endpoint if self.active_environment == Environment.BLUE
119             else self.blue_endpoint)
120
121 def _wait_for_healthy(self,
122                     endpoint: ModelEndpoint,
123                     timeout: int = 300,
124                     required_successful_checks: int = 3) -> bool:
125     """
126     Wait for endpoint to become healthy
127
128     Args:
129     endpoint: Endpoint to check
130     timeout: Maximum wait time in seconds
131     required_successful_checks: Consecutive successful checks needed
132
133     Returns:
134     True if endpoint becomes healthy
135     """
136     start_time = time.time()
137     successful_checks = 0
138
139     while time.time() - start_time < timeout:
140         health = self._check_health(endpoint)
141
142         if health.is_healthy:
143             successful_checks += 1
144             self.logger.info(
145                 f"{endpoint.environment.value} health check passed "
146                 f"({successful_checks}/{required_successful_checks})"

```

```

147         if successful_checks >= required_successful_checks:
148             return True
149         else:
150             successful_checks = 0
151             self.logger.warning(
152                 f"{endpoint.environment.value} health check failed: "
153                 f"error_rate={health.error_rate:.2%}"
154             )
155
156         time.sleep(self.health_check_interval)
157
158     return False
159
160 def _check_health(self, endpoint: ModelEndpoint) -> DeploymentHealth:
161     """
162     Perform health check on endpoint
163
164     Args:
165         endpoint: Endpoint to check
166
167     Returns:
168         Health check results
169     """
170     try:
171         # Health check request
172         response = requests.get(
173             endpoint.health_check_url,
174             timeout=10
175         )
176
177         if response.status_code != 200:
178             return DeploymentHealth(
179                 is_healthy=False,
180                 latency_p50=0,
181                 latency_p95=0,
182                 latency_p99=0,
183                 error_rate=1.0,
184                 throughput=0,
185                 timestamp=datetime.now()
186             )
187
188         # Parse metrics from response
189         metrics = response.json()
190
191         # Determine health based on metrics
192         is_healthy = (
193             metrics.get('error_rate', 1.0) < 0.01 and # < 1% errors
194             metrics.get('latency_p99', float('inf')) < 1000 # < 1s p99
195         )
196
197     return DeploymentHealth(
198         is_healthy=is_healthy,
199         latency_p50=metrics.get('latency_p50', 0),
200         latency_p95=metrics.get('latency_p95', 0),
201         latency_p99=metrics.get('latency_p99', 0),
202         error_rate=metrics.get('error_rate', 0),
203         throughput=metrics.get('throughput', 0),
204 
```

```

205         timestamp=datetime.now()
206     )
207
208     except Exception as e:
209         self.logger.error(f"Health check failed: {str(e)}")
210         return DeploymentHealth(
211             is_healthy=False,
212             latency_p50=0,
213             latency_p95=0,
214             latency_p99=0,
215             error_rate=1.0,
216             throughput=0,
217             timestamp=datetime.now()
218         )
219
220     def switch_traffic(self,
221                       validation_checks: Optional[Callable] = None) -> bool:
222         """
223         Switch traffic from active to inactive environment
224
225         Args:
226             validation_checks: Optional function to perform final validation
227
228         Returns:
229             True if switch successful
230         """
231         inactive_endpoint = self._get_inactive_endpoint()
232
233         if not inactive_endpoint:
234             self.logger.error("No inactive endpoint available")
235             return False
236
237         # Final health check
238         if not self._wait_for_healthy(inactive_endpoint, timeout=60):
239             self.logger.error("Inactive environment not healthy, aborting switch")
240             return False
241
242         # Run custom validation if provided
243         if validation_checks and not validation_checks(inactive_endpoint):
244             self.logger.error("Custom validation failed, aborting switch")
245             return False
246
247         # Perform atomic switch via load balancer
248         old_env = self.active_environment
249         new_env = self._get_inactive_environment()
250
251         self.logger.info(f"Switching traffic from {old_env.value} to {new_env.value}")
252
253         if self._update_load_balancer(inactive_endpoint):
254             self.active_environment = new_env
255             self.logger.info(f"Traffic switch completed. Active: {new_env.value}")
256             return True
257         else:
258             self.logger.error("Load balancer update failed")
259             return False
260
261     def _update_load_balancer(self, target_endpoint: ModelEndpoint) -> bool:
262         """

```

```

263         Update load balancer to point to target endpoint
264
265     Args:
266         target_endpoint: Endpoint to route traffic to
267
268     Returns:
269         True if update successful
270     """
271     try:
272         # Update load balancer configuration
273         response = requests.post(
274             f"{self.load_balancer_url}/update",
275             json={
276                 'model': self.model_name,
277                 'target_url': target_endpoint.url,
278                 'version': target_endpoint.version
279             },
280             timeout=30
281         )
282
283         return response.status_code == 200
284
285     except Exception as e:
286         self.logger.error(f"Load balancer update failed: {str(e)}")
287         return False
288
289     def rollback(self) -> bool:
290         """
291         Rollback to previous environment
292
293         Returns:
294             True if rollback successful
295         """
296         current_env = self.active_environment
297         previous_endpoint = self._get_inactive_endpoint()
298
299         if not previous_endpoint:
300             self.logger.error("No previous environment available for rollback")
301             return False
302
303         self.logger.warning(f"Rolling back from {current_env.value}")
304
305         # Check previous environment health
306         if not self._check_health(previous_endpoint).is_healthy:
307             self.logger.error("Previous environment is not healthy")
308             return False
309
310         # Switch back
311         if self._update_load_balancer(previous_endpoint):
312             self.active_environment = self._get_inactive_environment()
313             self.logger.info(f"Rollback completed. Active: {self.active_environment.value}")
314             return True
315         else:
316             self.logger.error("Rollback failed")
317             return False
318
319     def get_deployment_status(self) -> Dict[str, Any]:

```

```

320     """Get current deployment status"""
321     active_endpoint = self._get_active_endpoint()
322     inactive_endpoint = self._get_inactive_endpoint()
323
324     status = {
325         'model': self.model_name,
326         'active_environment': self.active_environment.value,
327         'timestamp': datetime.now().isoformat()
328     }
329
330     if active_endpoint:
331         active_health = self._check_health(active_endpoint)
332         status['active'] = {
333             'version': active_endpoint.version,
334             'url': active_endpoint.url,
335             'healthy': active_health.is_healthy,
336             'error_rate': active_health.error_rate,
337             'latency_p99': active_health.latency_p99
338         }
339
340     if inactive_endpoint:
341         inactive_health = self._check_health(inactive_endpoint)
342         status['inactive'] = {
343             'version': inactive_endpoint.version,
344             'url': inactive_endpoint.url,
345             'healthy': inactive_health.is_healthy,
346             'error_rate': inactive_health.error_rate,
347             'latency_p99': inactive_health.latency_p99
348         }
349
350     return status
351
352 # Example usage
353 def example_blue_green_deployment():
354     """Example of blue-green deployment workflow"""
355
356     # Initialize deployment manager
357     deployment = BlueGreenDeployment(
358         model_name="fraud_detection",
359         load_balancer_url="http://lb.example.com",
360         health_check_interval=10
361     )
362
363     # Initial deployment to blue
364     print("Step 1: Initial deployment to blue environment")
365     deployment.blue_endpoint = ModelEndpoint(
366         name="fraud_detection",
367         url="http://blue.example.com/predict",
368         version="v1.0",
369         environment=Environment.BLUE,
370         health_check_url="http://blue.example.com/health"
371     )
372     deployment.active_environment = Environment.BLUE
373
374     # Deploy new version to green
375     print("\nStep 2: Deploy v2.0 to green environment")
376     success = deployment.deploy_to_inactive(
377         version="v2.0",

```

```

378     endpoint_url="http://green.example.com/predict",
379     health_check_url="http://green.example.com/health"
380 )
381
382 if not success:
383     print("Deployment failed!")
384     return
385
386 # Custom validation function
387 def validate_model_quality(endpoint: ModelEndpoint) -> bool:
388     """Validate model meets quality standards"""
389     print(f"Running validation on {endpoint.version}...")
390     # In practice: run test predictions, check metrics, etc.
391     return True
392
393 # Switch traffic to green
394 print("\nStep 3: Switch traffic to green")
395 if deployment.switch_traffic(validation_checks=validate_model_quality):
396     print("Traffic switch successful!")
397
398 # Monitor for issues
399 print("\nStep 4: Monitor new deployment")
400 status = deployment.get_deployment_status()
401 print(f"Active: {status['active']['version']}")
402 print(f"Error rate: {status['active']['error_rate']:.2%}")
403
404 # Simulate issue detection
405 if status['active']['error_rate'] > 0.05:
406     print("\nStep 5: Issues detected, rolling back!")
407     deployment.rollback()
408 else:
409     print("Traffic switch failed!")

```

Listing 12.1: Blue-Green Deployment Manager

12.3.3 Kubernetes Blue-Green Deployment

```

1 from kubernetes import client, config
2 from typing import Dict, List
3 import yaml
4
5 class KubernetesBlueGreenDeployment:
6     """Blue-green deployment using Kubernetes"""
7
8     def __init__(self, namespace: str = "default"):
9         """
10         Initialize Kubernetes client
11
12         Args:
13             namespace: Kubernetes namespace
14         """
15         config.load_kube_config()
16         self.apps_v1 = client.AppsV1Api()
17         self.core_v1 = client.CoreV1Api()
18         self.namespace = namespace
19
20     def create_deployment(self,

```

```

21         name: str,
22         image: str,
23         replicas: int,
24         environment: str,
25         port: int = 8080) -> bool:
26
27     """
28     Create Kubernetes deployment
29
30     Args:
31         name: Deployment name
32         image: Docker image
33         replicas: Number of replicas
34         environment: blue or green
35         port: Container port
36     """
37     deployment_name = f"{name}-{environment}"
38
39     deployment = client.V1Deployment(
40         api_version="apps/v1",
41         kind="Deployment",
42         metadata=client.V1ObjectMeta(
43             name=deployment_name,
44             labels={
45                 'app': name,
46                 'environment': environment,
47                 'version': image.split(':')[1]
48             }
49         ),
50         spec=client.V1DeploymentSpec(
51             replicas=replicas,
52             selector=client.V1LabelSelector(
53                 match_labels={
54                     'app': name,
55                     'environment': environment
56                 }
57             ),
58             template=client.V1PodTemplateSpec(
59                 metadata=client.V1ObjectMeta(
60                     labels={
61                         'app': name,
62                         'environment': environment
63                     }
64                 ),
65                 spec=client.V1PodSpec(
66                     containers=[
67                         client.V1Container(
68                             name=name,
69                             image=image,
70                             ports=[client.V1ContainerPort(container_port=port)],
71                             resources=client.V1ResourceRequirements(
72                                 requests={'cpu': '500m', 'memory': '512Mi'},
73                                 limits={'cpu': '1000m', 'memory': '1Gi'}
74                             ),
75                             liveness_probe=client.V1Probe(
76                                 http_get=client.V1HTTPGetAction(
77                                     path='/health',
78                                     port=port

```



```

79         initial_delay_seconds=30,
80         period_seconds=10
81     ),
82     readiness_probe=client.V1Probe(
83         http_get=client.V1HTTPGetAction(
84             path='/ready',
85             port=port
86         ),
87         initial_delay_seconds=5,
88         period_seconds=5
89     )
90 )
91 ]
92 )
93 )
94 )
95 )
96
97 try:
98     self.apps_v1.create_namespaced_deployment(
99         namespace=self.namespace,
100         body=deployment
101     )
102     return True
103 except Exception as e:
104     print(f"Failed to create deployment: {str(e)}")
105     return False
106
107 def switch_service(self, service_name: str, target_environment: str) -> bool:
108     """
109     Switch service to target environment
110
111     Args:
112         service_name: Service name
113         target_environment: blue or green
114     """
115     try:
116         # Get existing service
117         service = self.core_v1.read_namespaced_service(
118             name=service_name,
119             namespace=self.namespace
120         )
121
122         # Update selector to point to target environment
123         service.spec.selector = {
124             'app': service_name,
125             'environment': target_environment
126         }
127
128         # Update service
129         self.core_v1.patch_namespaced_service(
130             name=service_name,
131             namespace=self.namespace,
132             body=service
133         )
134
135     return True
136

```

```

137     except Exception as e:
138         print(f"Failed to switch service: {str(e)}")
139         return False
140
141     def get_deployment_status(self, name: str, environment: str) -> Dict:
142         """Get deployment status"""
143         deployment_name = f"{name}-{environment}"
144
145         try:
146             deployment = self.apps_v1.read_namespaced_deployment(
147                 name=deployment_name,
148                 namespace=self.namespace
149             )
150
151             return {
152                 'name': deployment_name,
153                 'replicas': deployment.spec.replicas,
154                 'ready_replicas': deployment.status.ready_replicas or 0,
155                 'available_replicas': deployment.status.available_replicas or 0,
156                 'image': deployment.spec.template.spec.containers[0].image
157             }
158
159         except Exception as e:
160             return {'error': str(e)}

```

Listing 12.2: Kubernetes Blue-Green Deployment

12.4 Canary Deployment

12.4.1 Overview and Strategy

Canary deployment gradually rolls out a new model version to a small percentage of traffic, monitors performance, and incrementally increases traffic if metrics are healthy. Named after coal mine canaries used to detect dangerous gases, this strategy detects issues early with minimal user impact.

Key characteristics:

- **Gradual rollout:** Start with 1-5% of traffic, increase slowly
- **Real traffic validation:** Test with actual production workload
- **Risk minimization:** Limit blast radius of potential issues
- **Automated decision-making:** Progress based on metrics, not time
- **Quick rollback:** Reduce traffic allocation immediately if issues arise

12.4.2 Implementation

```

1 from typing import Dict, List, Optional, Callable
2 from dataclasses import dataclass, field
3 from datetime import datetime, timedelta
4 import numpy as np
5 import logging

```

```

6 import time
7
8 @dataclass
9 class CanaryStage:
10     """Configuration for a canary rollout stage"""
11     traffic_percentage: float
12     duration_minutes: int
13     success_criteria: Dict[str, float]
14
15 @dataclass
16 class CanaryMetrics:
17     """Metrics collected during canary deployment"""
18     timestamp: datetime
19     traffic_percentage: float
20     requests_count: int
21     error_rate: float
22     latency_p50: float
23     latency_p95: float
24     latency_p99: float
25     prediction_quality: Optional[float] = None
26
27 class CanaryDeployment:
28     """
29     Canary deployment manager with progressive traffic allocation
30
31     Implements gradual rollout with automated progression based on metrics
32     """
33
34     def __init__(self,
35                 model_name: str,
36                 baseline_endpoint: str,
37                 canary_endpoint: str,
38                 traffic_router_url: str):
39         """
40         Initialize canary deployment
41
42         Args:
43             model_name: Model being deployed
44             baseline_endpoint: Current production model
45             canary_endpoint: New canary model
46             traffic_router_url: Traffic routing service
47         """
48         self.model_name = model_name
49         self.baseline_endpoint = baseline_endpoint
50         self.canary_endpoint = canary_endpoint
51         self.traffic_router_url = traffic_router_url
52         self.logger = logging.getLogger(f"Canary_{model_name}")
53
54         self.current_traffic_percentage = 0.0
55         self.canary_metrics: List[CanaryMetrics] = []
56         self.baseline_metrics: List[CanaryMetrics] = []
57
58     def execute_rollout(self, stages: List[CanaryStage]) -> bool:
59         """
60         Execute canary rollout through defined stages
61
62         Args:
63             stages: List of rollout stages

```

```

64
65     Returns:
66         True if rollout completed successfully
67     """
68     self.logger.info(f"Starting canary rollout for {self.model_name}")
69
70     for stage_idx, stage in enumerate(stages, 1):
71         self.logger.info(
72             f"Stage {stage_idx}/{len(stages)}: "
73             f"{stage.traffic_percentage:.1f}% traffic for {stage.duration_minutes}
min"
74         )
75
76         # Update traffic allocation
77         if not self._update_traffic_split(stage.traffic_percentage):
78             self.logger.error("Failed to update traffic split")
79             self._rollback()
80             return False
81
82         # Monitor stage
83         stage_start = datetime.now()
84         stage_end = stage_start + timedelta(minutes=stage.duration_minutes)
85
86         while datetime.now() < stage_end:
87             # Collect metrics
88             canary_metrics = self._collect_metrics(self.canary_endpoint, "canary")
89             baseline_metrics = self._collect_metrics(self.baseline_endpoint, "
baseline")
90
91             self.canary_metrics.append(canary_metrics)
92             self.baseline_metrics.append(baseline_metrics)
93
94             # Evaluate success criteria
95             if not self._evaluate_stage(stage, canary_metrics, baseline_metrics):
96                 self.logger.error(
97                     f"Stage {stage_idx} failed success criteria"
98                 )
99                 self._rollback()
100                 return False
101
102             # Wait before next check
103             time.sleep(30)
104
105             self.logger.info(f"Stage {stage_idx} completed successfully")
106
107         # Rollout completed - promote canary to 100%
108         self.logger.info("Canary rollout successful, promoting to 100%")
109         return self._update_traffic_split(100.0)
110
111     def _update_traffic_split(self, canary_percentage: float) -> bool:
112         """
113         Update traffic split between baseline and canary
114
115         Args:
116             canary_percentage: Percentage of traffic to canary (0-100)
117
118         Returns:
119             True if update successful

```

```

120     """
121     try:
122         import requests
123
124         response = requests.post(
125             f"{self.traffic_router_url}/split",
126             json={
127                 'model': self.model_name,
128                 'routes': [
129                     {
130                         'endpoint': self.baseline_endpoint,
131                         'weight': 100 - canary_percentage
132                     },
133                     {
134                         'endpoint': self.canary_endpoint,
135                         'weight': canary_percentage
136                     }
137                 ]
138             },
139             timeout=30
140         )
141
142         if response.status_code == 200:
143             self.current_traffic_percentage = canary_percentage
144             self.logger.info(
145                 f"Traffic split updated: {canary_percentage:.1f}% to canary"
146             )
147             return True
148
149         return False
150
151     except Exception as e:
152         self.logger.error(f"Traffic split update failed: {str(e)}")
153         return False
154
155 def _collect_metrics(self, endpoint: str, label: str) -> CanaryMetrics:
156     """
157     Collect metrics from endpoint
158
159     Args:
160         endpoint: Model endpoint
161         label: Metric label (canary or baseline)
162
163     Returns:
164         Collected metrics
165     """
166     try:
167         import requests
168
169         response = requests.get(
170             f"{endpoint}/metrics",
171             timeout=10
172         )
173
174         if response.status_code == 200:
175             data = response.json()
176
177             return CanaryMetrics(

```

```

178         timestamp=datetime.now(),
179         traffic_percentage=self.current_traffic_percentage,
180         requests_count=data.get('requests_count', 0),
181         error_rate=data.get('error_rate', 0),
182         latency_p50=data.get('latency_p50', 0),
183         latency_p95=data.get('latency_p95', 0),
184         latency_p99=data.get('latency_p99', 0),
185         prediction_quality=data.get('prediction_quality')
186     )
187
188     except Exception as e:
189         self.logger.error(f"Metrics collection failed for {label}: {str(e)}")
190
191     # Return empty metrics on failure
192     return CanaryMetrics(
193         timestamp=datetime.now(),
194         traffic_percentage=self.current_traffic_percentage,
195         requests_count=0,
196         error_rate=1.0,
197         latency_p50=0,
198         latency_p95=0,
199         latency_p99=0
200     )
201
202     def _evaluate_stage(self,
203         stage: CanaryStage,
204         canary_metrics: CanaryMetrics,
205         baseline_metrics: CanaryMetrics) -> bool:
206         """
207         Evaluate if stage meets success criteria
208
209         Args:
210             stage: Current stage configuration
211             canary_metrics: Canary metrics
212             baseline_metrics: Baseline metrics
213
214         Returns:
215             True if criteria met
216         """
217         criteria = stage.success_criteria
218
219         # Check error rate
220         if 'max_error_rate' in criteria:
221             if canary_metrics.error_rate > criteria['max_error_rate']:
222                 self.logger.warning(
223                     f"Canary error rate {canary_metrics.error_rate:.2%} exceeds "
224                     f"maximum {criteria['max_error_rate']:.2%}"
225                 )
226                 return False
227
228         # Compare error rate with baseline
229         if 'max_error_rate_ratio' in criteria:
230             if baseline_metrics.error_rate > 0:
231                 error_ratio = canary_metrics.error_rate / baseline_metrics.error_rate
232                 if error_ratio > criteria['max_error_rate_ratio']:
233                     self.logger.warning(
234                         f"Canary error rate {error_ratio:.2f}x baseline"
235                     )

```

```

236         return False
237
238     # Check latency
239     if 'max_latency_p99' in criteria:
240         if canary_metrics.latency_p99 > criteria['max_latency_p99']:
241             self.logger.warning(
242                 f"Canary p99 latency {canary_metrics.latency_p99}ms exceeds "
243                 f"maximum {criteria['max_latency_p99']}ms"
244             )
245             return False
246
247     # Compare latency with baseline
248     if 'max_latency_ratio' in criteria:
249         if baseline_metrics.latency_p99 > 0:
250             latency_ratio = canary_metrics.latency_p99 / baseline_metrics.
latency_p99
251             if latency_ratio > criteria['max_latency_ratio']:
252                 self.logger.warning(
253                     f"Canary latency {latency_ratio:.2f}x baseline"
254                 )
255                 return False
256
257     # Check prediction quality if available
258     if 'min_prediction_quality' in criteria and canary_metrics.prediction_quality:
259         if canary_metrics.prediction_quality < criteria['min_prediction_quality']:
260             self.logger.warning(
261                 f"Canary prediction quality {canary_metrics.prediction_quality:.3f}
} "
262                 f"below minimum {criteria['min_prediction_quality']:.3f}"
263             )
264             return False
265
266     return True
267
268     def _rollback(self) -> bool:
269         """Rollback canary deployment to baseline"""
270         self.logger.warning("Rolling back canary deployment")
271         return self._update_traffic_split(0.0)
272
273     def get_rollout_report(self) -> Dict:
274         """Generate rollout report with statistics"""
275         if not self.canary_metrics:
276             return {'status': 'no_data'}
277
278         canary_errors = [m.error_rate for m in self.canary_metrics]
279         baseline_errors = [m.error_rate for m in self.baseline_metrics]
280
281         canary_latencies = [m.latency_p99 for m in self.canary_metrics]
282         baseline_latencies = [m.latency_p99 for m in self.baseline_metrics]
283
284         return {
285             'model': self.model_name,
286             'final_traffic_percentage': self.current_traffic_percentage,
287             'stages_completed': len(self.canary_metrics),
288             'canary_stats': {
289                 'avg_error_rate': np.mean(canary_errors),
290                 'max_error_rate': np.max(canary_errors),
291                 'avg_latency_p99': np.mean(canary_latencies),

```

```

292         'max_latency_p99': np.max(canary_latencies)
293     },
294     'baseline_stats': {
295         'avg_error_rate': np.mean(baseline_errors),
296         'max_error_rate': np.max(baseline_errors),
297         'avg_latency_p99': np.mean(baseline_latencies),
298         'max_latency_p99': np.max(baseline_latencies)
299     }
300 }
301
302 # Example usage
303 def example_canary_deployment():
304     """Example of canary deployment workflow"""
305
306     # Initialize canary deployment
307     canary = CanaryDeployment(
308         model_name="recommendation_model",
309         baseline_endpoint="http://prod.example.com/v1",
310         canary_endpoint="http://prod.example.com/v2",
311         traffic_router_url="http://router.example.com"
312     )
313
314     # Define rollout stages
315     stages = [
316         CanaryStage(
317             traffic_percentage=5.0,
318             duration_minutes=10,
319             success_criteria={
320                 'max_error_rate': 0.01,
321                 'max_error_rate_ratio': 2.0,
322                 'max_latency_p99': 500
323             }
324         ),
325         CanaryStage(
326             traffic_percentage=25.0,
327             duration_minutes=20,
328             success_criteria={
329                 'max_error_rate': 0.01,
330                 'max_error_rate_ratio': 1.5,
331                 'max_latency_p99': 500
332             }
333         ),
334         CanaryStage(
335             traffic_percentage=50.0,
336             duration_minutes=30,
337             success_criteria={
338                 'max_error_rate': 0.01,
339                 'max_error_rate_ratio': 1.2,
340                 'max_latency_p99': 500
341             }
342         )
343     ]
344
345     # Execute rollout
346     success = canary.execute_rollout(stages)
347
348     # Print report
349     report = canary.get_rollout_report()

```



```

350 print(f"\nCanary Deployment Report:")
351 print(f"Status: {'SUCCESS' if success else 'FAILED'}")
352 print(f"Final Traffic: {report['final_traffic_percentage']:.1f}%")
353 print(f"\nCanary Stats:")
354 print(f"    Avg Error Rate: {report['canary_stats']['avg_error_rate']:.2%}")
355 print(f"    Avg P99 Latency: {report['canary_stats']['avg_latency_p99']:.1f}ms")

```

Listing 12.3: Canary Deployment Manager

12.5 Shadow Deployment

12.5.1 Overview and Use Cases

Shadow deployment (also called dark launching) routes production traffic to a new model version but discards its predictions, logging them for analysis instead. The current model continues serving users. This enables risk-free validation of new models with real production data.

Shadow deployment is ideal for:

- Testing models with real traffic patterns before serving predictions
- Validating performance under production load
- Comparing model versions on identical traffic
- Detecting data drift or distribution shifts
- Load testing new model infrastructure

12.5.2 Implementation

```

1 from typing import Dict, List, Optional, Any
2 from dataclasses import dataclass
3 from datetime import datetime, timedelta
4 import asyncio
5 import logging
6 import json
7 import numpy as np
8
9 @dataclass
10 class ShadowPrediction:
11     """Shadow model prediction for logging and analysis"""
12     request_id: str
13     timestamp: datetime
14     primary_prediction: Any
15     shadow_prediction: Any
16     primary_latency_ms: float
17     shadow_latency_ms: float
18     features: Dict
19     match: bool = None
20     divergence: Optional[float] = None
21
22 class ShadowDeployment:
23     """

```

```

24     Shadow deployment manager
25
26     Mirrors traffic to shadow model without impacting users
27     """
28
29     def __init__(self,
30                   model_name: str,
31                   primary_endpoint: str,
32                   shadow_endpoint: str,
33                   comparison_fn: Optional[Callable] = None):
34         """
35         Initialize shadow deployment
36
37         Args:
38             model_name: Model name
39             primary_endpoint: Production model endpoint
40             shadow_endpoint: Shadow model endpoint
41             comparison_fn: Function to compare predictions
42         """
43         self.model_name = model_name
44         self.primary_endpoint = primary_endpoint
45         self.shadow_endpoint = shadow_endpoint
46         self.comparison_fn = comparison_fn or self._default_comparison
47         self.logger = logging.getLogger(f"Shadow_{model_name}")
48
49         self.shadow_predictions: List[ShadowPrediction] = []
50
51     async def predict(self, features: Dict, request_id: str) -> Any:
52         """
53         Make prediction with both primary and shadow models
54
55         Args:
56             features: Input features
57             request_id: Unique request identifier
58
59         Returns:
60             Primary model prediction (shadow logged but not returned)
61         """
62         # Make predictions in parallel
63         primary_task = self._call_primary(features)
64         shadow_task = self._call_shadow(features)
65
66         # Wait for primary (required for user)
67         primary_result = await primary_task
68
69         # Shadow can continue in background
70         asyncio.create_task(
71             self._process_shadow_result(
72                 request_id, features, primary_result, shadow_task
73             )
74         )
75
76         return primary_result
77
78     async def _call_primary(self, features: Dict) -> Dict:
79         """Call primary model"""
80         import aiohttp
81

```

```

82     start_time = datetime.now()
83
84     try:
85         async with aiohttp.ClientSession() as session:
86             async with session.post(
87                 f"{self.primary_endpoint}/predict",
88                 json=features,
89                 timeout=aiohttp.ClientTimeout(total=5)
90             ) as response:
91                 result = await response.json()
92
93             latency_ms = (datetime.now() - start_time).total_seconds() * 1000
94
95             return {
96                 'prediction': result,
97                 'latency_ms': latency_ms,
98                 'error': None
99             }
100
101     except Exception as e:
102         self.logger.error(f"Primary prediction failed: {str(e)}")
103         latency_ms = (datetime.now() - start_time).total_seconds() * 1000
104         return {
105             'prediction': None,
106             'latency_ms': latency_ms,
107             'error': str(e)
108         }
109
110 async def _call_shadow(self, features: Dict) -> Dict:
111     """Call shadow model"""
112     import aiohttp
113
114     start_time = datetime.now()
115
116     try:
117         async with aiohttp.ClientSession() as session:
118             async with session.post(
119                 f"{self.shadow_endpoint}/predict",
120                 json=features,
121                 timeout=aiohttp.ClientTimeout(total=10) # More lenient timeout
122             ) as response:
123                 result = await response.json()
124
125             latency_ms = (datetime.now() - start_time).total_seconds() * 1000
126
127             return {
128                 'prediction': result,
129                 'latency_ms': latency_ms,
130                 'error': None
131             }
132
133     except Exception as e:
134         self.logger.warning(f"Shadow prediction failed: {str(e)}")
135         latency_ms = (datetime.now() - start_time).total_seconds() * 1000
136         return {
137             'prediction': None,
138             'latency_ms': latency_ms,
139             'error': str(e)

```

```

140     }
141
142     async def _process_shadow_result(self,
143                                     request_id: str,
144                                     features: Dict,
145                                     primary_result: Dict,
146                                     shadow_task: asyncio.Task):
147         """Process shadow prediction result"""
148         try:
149             shadow_result = await shadow_task
150
151             # Compare predictions
152             match, divergence = self.comparison_fn(
153                 primary_result['prediction'],
154                 shadow_result['prediction']
155             )
156
157             # Log shadow prediction
158             shadow_pred = ShadowPrediction(
159                 request_id=request_id,
160                 timestamp=datetime.now(),
161                 primary_prediction=primary_result['prediction'],
162                 shadow_prediction=shadow_result['prediction'],
163                 primary_latency_ms=primary_result['latency_ms'],
164                 shadow_latency_ms=shadow_result['latency_ms'],
165                 features=features,
166                 match=match,
167                 divergence=divergence
168             )
169
170             self.shadow_predictions.append(shadow_pred)
171             self._log_shadow_prediction(shadow_pred)
172
173         except Exception as e:
174             self.logger.error(f"Shadow result processing failed: {str(e)}")
175
176     def _default_comparison(self,
177                            primary_pred: Any,
178                            shadow_pred: Any) -> tuple:
179         """
180         Default prediction comparison
181
182         Returns:
183             (match: bool, divergence: float)
184         """
185         if primary_pred is None or shadow_pred is None:
186             return False, None
187
188         # For classification
189         if isinstance(primary_pred, (str, int)):
190             match = primary_pred == shadow_pred
191             return match, 0.0 if match else 1.0
192
193         # For regression or probabilities
194         if isinstance(primary_pred, (float, int)):
195             divergence = abs(primary_pred - shadow_pred)
196             match = divergence < 0.01 # 1% threshold
197             return match, divergence

```

```

198     # For arrays/lists
199     if isinstance(primary_pred, (list, np.ndarray)):
200         primary_arr = np.array(primary_pred)
201         shadow_arr = np.array(shadow_pred)
202         divergence = np.mean(np.abs(primary_arr - shadow_arr))
203         match = divergence < 0.01
204         return match, float(divergence)
205
206     return True, 0.0
207
208 def _log_shadow_prediction(self, pred: ShadowPrediction):
209     """Log shadow prediction for analysis"""
210     log_entry = {
211         'request_id': pred.request_id,
212         'timestamp': pred.timestamp.isoformat(),
213         'model': self.model_name,
214         'primary': pred.primary_prediction,
215         'shadow': pred.shadow_prediction,
216         'match': pred.match,
217         'divergence': pred.divergence,
218         'latency_primary_ms': pred.primary_latency_ms,
219         'latency_shadow_ms': pred.shadow_latency_ms
220     }
221
222     self.logger.info(f"Shadow: {json.dumps(log_entry)}")
223
224 def get_analysis(self, window_minutes: int = 60) -> Dict:
225     """
226     Analyze shadow deployment performance
227
228     Args:
229         window_minutes: Analysis window in minutes
230
231     Returns:
232         Analysis results
233     """
234     cutoff_time = datetime.now() - timedelta(minutes=window_minutes)
235
236     # Filter to recent predictions
237     recent_preds = [
238         p for p in self.shadow_predictions
239         if p.timestamp >= cutoff_time
240     ]
241
242     if not recent_preds:
243         return {'status': 'no_data'}
244
245     # Calculate statistics
246     total = len(recent_preds)
247     matches = sum(1 for p in recent_preds if p.match)
248     match_rate = matches / total
249
250     divergences = [p.divergence for p in recent_preds if p.divergence is not None]
251     avg_divergence = np.mean(divergences) if divergences else 0
252
253     primary_latencies = [p.primary_latency_ms for p in recent_preds]
254     shadow_latencies = [p.shadow_latency_ms for p in recent_preds]

```

```

256
257     # Shadow errors
258     shadow_errors = sum(
259         1 for p in recent_preds
260         if p.shadow_prediction is None
261     )
262     shadow_error_rate = shadow_errors / total
263
264     return {
265         'model': self.model_name,
266         'window_minutes': window_minutes,
267         'total_predictions': total,
268         'match_rate': match_rate,
269         'avg_divergence': avg_divergence,
270         'shadow_error_rate': shadow_error_rate,
271         'latency_stats': {
272             'primary_p50': np.percentile(primary_latencies, 50),
273             'primary_p95': np.percentile(primary_latencies, 95),
274             'primary_p99': np.percentile(primary_latencies, 99),
275             'shadow_p50': np.percentile(shadow_latencies, 50),
276             'shadow_p95': np.percentile(shadow_latencies, 95),
277             'shadow_p99': np.percentile(shadow_latencies, 99)
278         }
279     }
280
281 # Example usage
282 async def example_shadow_deployment():
283     """Example of shadow deployment"""
284
285     # Custom comparison for classification model
286     def compare_predictions(primary, shadow):
287         """Compare classification predictions"""
288         if primary is None or shadow is None:
289             return False, None
290
291         # Extract predicted class
292         primary_class = primary.get('class')
293         shadow_class = shadow.get('class')
294
295         match = primary_class == shadow_class
296
297         # Calculate probability divergence
298         primary_prob = primary.get('probability', 0)
299         shadow_prob = shadow.get('probability', 0)
300         divergence = abs(primary_prob - shadow_prob)
301
302         return match, divergence
303
304     # Initialize shadow deployment
305     shadow = ShadowDeployment(
306         model_name="sentiment_classifier",
307         primary_endpoint="http://prod.example.com/v1",
308         shadow_endpoint="http://prod.example.com/v2",
309         comparison_fn=compare_predictions
310     )
311
312     # Simulate requests
313     for i in range(100):

```

```

314     features = {'text': f'Sample text {i}'}
315     prediction = await shadow.predict(features, request_id=f'req_{i}')
316     print(f"Request {i}: {prediction}")
317
318     # Analyze results
319     analysis = shadow.get_analysis(window_minutes=60)
320     print(f"\nShadow Deployment Analysis:")
321     print(f"Match Rate: {analysis['match_rate']:.2%}")
322     print(f"Avg Divergence: {analysis['avg_divergence']:.4f}")
323     print(f"Shadow Error Rate: {analysis['shadow_error_rate']:.2%}")
324     print(f"Shadow P99 Latency: {analysis['latency_stats']['shadow_p99']:.1f}ms")

```

Listing 12.4: Shadow Deployment Manager

12.6 A/B Testing for Model Evaluation

12.6.1 Overview and Statistical Framework

A/B testing (or split testing) compares two model versions by randomly assigning users to treatment groups and measuring performance differences with statistical rigor. Unlike canary deployment, A/B tests maintain stable traffic allocation and run until statistical significance is achieved.

Key principles:

- **Random assignment:** Users randomly allocated to variants
- **Controlled experiment:** Fixed allocation during test period
- **Statistical significance:** Decisions based on hypothesis testing
- **Multiple metrics:** Track business metrics alongside technical metrics
- **Duration planning:** Run until sufficient sample size achieved

12.6.2 Implementation

```

1  from typing import Dict, List, Optional, Tuple
2  from dataclasses import dataclass, field
3  from datetime import datetime
4  from scipy import stats
5  import numpy as np
6  import hashlib
7  import logging
8
9  @dataclass
10 class Variant:
11     """Model variant in A/B test"""
12     name: str
13     endpoint: str
14     traffic_percentage: float
15     description: str = ""
16
17 @dataclass
18 class ABTestMetric:

```

```

19     """Metric tracked during A/B test"""
20     name: str
21     variant: str
22     timestamp: datetime
23     value: float
24     user_id: Optional[str] = None
25
26 @dataclass
27 class ABTestResult:
28     """Statistical test results"""
29     metric_name: str
30     control_mean: float
31     treatment_mean: float
32     control_std: float
33     treatment_std: float
34     control_sample_size: int
35     treatment_sample_size: int
36     p_value: float
37     confidence_interval: Tuple[float, float]
38     is_significant: bool
39     relative_improvement: float
40
41 class ABTest:
42     """
43     A/B testing framework for ML models
44
45     Implements rigorous statistical testing with multiple metrics
46     """
47
48     def __init__(self,
49                 test_name: str,
50                 control_variant: Variant,
51                 treatment_variant: Variant,
52                 alpha: float = 0.05,
53                 minimum_detectable_effect: float = 0.02):
54         """
55         Initialize A/B test
56
57         Args:
58             test_name: Test identifier
59             control_variant: Baseline model (variant A)
60             treatment_variant: New model (variant B)
61             alpha: Significance level (default 5%)
62             minimum_detectable_effect: Minimum effect size to detect
63         """
64         self.test_name = test_name
65         self.control_variant = control_variant
66         self.treatment_variant = treatment_variant
67         self.alpha = alpha
68         self.minimum_detectable_effect = minimum_detectable_effect
69         self.logger = logging.getLogger(f"ABTest_{test_name}")
70
71         self.metrics: List[ABTestMetric] = []
72         self.start_time: Optional[datetime] = None
73         self.end_time: Optional[datetime] = None
74
75     def assign_variant(self, user_id: str) -> Variant:
76         """

```



```

77     Assign user to variant using consistent hashing
78
79     Args:
80         user_id: User identifier
81
82     Returns:
83         Assigned variant
84     """
85     # Hash user_id with test_name for consistency
86     hash_input = f"{self.test_name}:{user_id}".encode('utf-8')
87     hash_value = int(hashlib.md5(hash_input).hexdigest(), 16)
88
89     # Convert to percentage (0-100)
90     percentage = (hash_value % 100) / 100.0
91
92     # Assign based on traffic allocation
93     if percentage < self.control_variant.traffic_percentage / 100:
94         return self.control_variant
95     else:
96         return self.treatment_variant
97
98     def record_metric(self,
99                     metric_name: str,
100                     value: float,
101                     user_id: str,
102                     variant_name: str):
103         """
104         Record metric observation
105
106         Args:
107             metric_name: Name of metric
108             value: Metric value
109             user_id: User identifier
110             variant_name: Variant name
111         """
112         metric = ABTestMetric(
113             name=metric_name,
114             variant=variant_name,
115             timestamp=datetime.now(),
116             value=value,
117             user_id=user_id
118         )
119
120         self.metrics.append(metric)
121
122     def calculate_sample_size(self,
123                             baseline_mean: float,
124                             baseline_std: float,
125                             power: float = 0.8) -> int:
126         """
127         Calculate required sample size per variant
128
129         Args:
130             baseline_mean: Baseline metric mean
131             baseline_std: Baseline metric std
132             power: Statistical power (default 80%)
133
134         Returns:

```

```

135         Required sample size per variant
136         """
137         # Effect size (Cohen's d)
138         effect_size = self.minimum_detectable_effect * baseline_mean / baseline_std
139
140         # Z-scores for alpha and beta
141         z_alpha = stats.norm.ppf(1 - self.alpha / 2)
142         z_beta = stats.norm.ppf(power)
143
144         # Sample size calculation
145         n = 2 * ((z_alpha + z_beta) / effect_size) ** 2
146
147         return int(np.ceil(n))
148
149     def analyze_metric(self, metric_name: str) -> ABTestResult:
150         """
151         Perform statistical analysis for a metric
152
153         Args:
154             metric_name: Metric to analyze
155
156         Returns:
157             Statistical test results
158         """
159         # Filter metrics
160         metric_data = [m for m in self.metrics if m.name == metric_name]
161
162         if not metric_data:
163             raise ValueError(f"No data for metric: {metric_name}")
164
165         # Separate by variant
166         control_values = np.array([
167             m.value for m in metric_data
168             if m.variant == self.control_variant.name
169         ])
170
171         treatment_values = np.array([
172             m.value for m in metric_data
173             if m.variant == self.treatment_variant.name
174         ])
175
176         if len(control_values) == 0 or len(treatment_values) == 0:
177             raise ValueError(f"Insufficient data for both variants")
178
179         # Calculate statistics
180         control_mean = np.mean(control_values)
181         control_std = np.std(control_values, ddof=1)
182         treatment_mean = np.mean(treatment_values)
183         treatment_std = np.std(treatment_values, ddof=1)
184
185         # Perform t-test
186         t_stat, p_value = stats.ttest_ind(treatment_values, control_values)
187
188         # Calculate confidence interval for difference
189         pooled_std = np.sqrt(
190             (control_std ** 2 / len(control_values)) +
191             (treatment_std ** 2 / len(treatment_values))
192         )

```

```

193     z_critical = stats.norm.ppf(1 - self.alpha / 2)
194     mean_diff = treatment_mean - control_mean
195
196
197     ci_lower = mean_diff - z_critical * pooled_std
198     ci_upper = mean_diff + z_critical * pooled_std
199
200     # Determine significance
201     is_significant = p_value < self.alpha
202
203     # Relative improvement
204     if control_mean != 0:
205         relative_improvement = (treatment_mean - control_mean) / abs(control_mean)
206     else:
207         relative_improvement = 0
208
209     return ABTestResult(
210         metric_name=metric_name,
211         control_mean=control_mean,
212         treatment_mean=treatment_mean,
213         control_std=control_std,
214         treatment_std=treatment_std,
215         control_sample_size=len(control_values),
216         treatment_sample_size=len(treatment_values),
217         p_value=p_value,
218         confidence_interval=(ci_lower, ci_upper),
219         is_significant=is_significant,
220         relative_improvement=relative_improvement
221     )
222
223 def get_test_summary(self, metrics_to_analyze: List[str]) -> Dict:
224     """
225     Generate comprehensive test summary
226
227     Args:
228         metrics_to_analyze: List of metric names to analyze
229
230     Returns:
231         Test summary with results for all metrics
232     """
233     results = {}
234
235     for metric_name in metrics_to_analyze:
236         try:
237             result = self.analyze_metric(metric_name)
238             results[metric_name] = {
239                 'control_mean': result.control_mean,
240                 'treatment_mean': result.treatment_mean,
241                 'p_value': result.p_value,
242                 'is_significant': result.is_significant,
243                 'relative_improvement': result.relative_improvement,
244                 'confidence_interval': result.confidence_interval,
245                 'sample_sizes': {
246                     'control': result.control_sample_size,
247                     'treatment': result.treatment_sample_size
248                 }
249             }
250         except Exception as e:

```

```

251         self.logger.error(f"Failed to analyze {metric_name}: {str(e)}")
252         results[metric_name] = {'error': str(e)}
253
254     return {
255         'test_name': self.test_name,
256         'control': self.control_variant.name,
257         'treatment': self.treatment_variant.name,
258         'alpha': self.alpha,
259         'results': results
260     }
261
262 def print_report(self, metrics_to_analyze: List[str]):
263     """Print formatted test report"""
264     summary = self.get_test_summary(metrics_to_analyze)
265
266     print(f"\n{'='*70}")
267     print(f"A/B Test Report: {self.test_name}")
268     print(f"{'='*70}")
269     print(f"Control: {self.control_variant.name}")
270     print(f"Treatment: {self.treatment_variant.name}")
271     print(f"Significance Level ( $\alpha$ ): {self.alpha}")
272     print(f"\n{'='*70}")
273     print("Results:")
274     print(f"{'='*70}\n")
275
276     for metric_name, result in summary['results'].items():
277         if 'error' in result:
278             print(f"{metric_name}: ERROR - {result['error']}\n")
279             continue
280
281             print(f"{metric_name}:")
282             print(f"    Control:    {result['control_mean']:.4f}")
283             print(f"    Treatment:  {result['treatment_mean']:.4f}")
284             print(f"    Improvement: {result['relative_improvement']:+.2%}")
285             print(f"    P-value:    {result['p_value']:.4f}")
286             print(f"    Significant: {'YES' if result['is_significant'] else 'NO'}")
287             print(f"    95% CI:     [{result['confidence_interval'][0]:.4f}, "
288                   f"{result['confidence_interval'][1]:.4f}]")
289             print(f"    Samples:    Control={result['sample_sizes']['control']}, "
290                   f"Treatment={result['sample_sizes']['treatment']}\n")
291
292 # Example usage
293 def example_ab_test():
294     """Example of A/B testing workflow"""
295
296     # Define variants
297     control = Variant(
298         name="v1_baseline",
299         endpoint="http://prod.example.com/v1",
300         traffic_percentage=50.0,
301         description="Current production model"
302     )
303
304     treatment = Variant(
305         name="v2_improved",
306         endpoint="http://prod.example.com/v2",
307         traffic_percentage=50.0,
308         description="New model with additional features"

```

```

309 )
310
311 # Initialize A/B test
312 ab_test = ABTest(
313     test_name="recommendation_model_v2_test",
314     control_variant=control,
315     treatment_variant=treatment,
316     alpha=0.05,
317     minimum_detectable_effect=0.02
318 )
319
320 # Calculate required sample size
321 required_n = ab_test.calculate_sample_size(
322     baseline_mean=0.25, # 25% baseline conversion
323     baseline_std=0.1,
324     power=0.8
325 )
326 print(f"Required sample size per variant: {required_n}")
327
328 # Simulate experiment (in practice, collect real data)
329 np.random.seed(42)
330
331 for user_idx in range(10000):
332     user_id = f"user_{user_idx}"
333
334     # Assign variant
335     variant = ab_test.assign_variant(user_id)
336
337     # Simulate metrics (treatment has slight improvement)
338     if variant.name == "v1_baseline":
339         click_rate = np.random.beta(25, 75) # ~25% baseline
340         latency = np.random.gamma(shape=2, scale=50) # ~100ms
341     else:
342         click_rate = np.random.beta(27, 73) # ~27% (8% relative improvement)
343         latency = np.random.gamma(shape=2, scale=45) # ~90ms (10% improvement)
344
345     # Record metrics
346     ab_test.record_metric("click_rate", click_rate, user_id, variant.name)
347     ab_test.record_metric("latency_ms", latency, user_id, variant.name)
348
349 # Analyze results
350 ab_test.print_report(metrics_to_analyze=["click_rate", "latency_ms"])

```

Listing 12.5: A/B Testing Framework for ML Models

12.7 Traffic Splitting and Routing

12.7.1 Traffic Router Implementation

Traffic routers direct requests to different model versions based on allocation rules. A production-grade router supports weighted routing, sticky sessions, and dynamic configuration.

```

1 from typing import Dict, List, Optional
2 from dataclasses import dataclass
3 import random

```

```

4 import hashlib
5 import logging
6
7 @dataclass
8 class Route:
9     """Traffic routing configuration"""
10     endpoint: str
11     weight: float # 0-100
12     version: str
13     metadata: Dict = None
14
15 class TrafficRouter:
16     """
17     Advanced traffic router for ML model serving
18
19     Supports weighted routing, sticky sessions, and health-aware routing
20     """
21
22     def __init__(self, model_name: str):
23         """
24         Initialize traffic router
25
26         Args:
27             model_name: Model being routed
28         """
29         self.model_name = model_name
30         self.routes: List[Route] = []
31         self.logger = logging.getLogger(f"Router_{model_name}")
32
33         # Sticky session mapping: user_id -> route
34         self.sticky_sessions: Dict[str, Route] = {}
35
36     def add_route(self, endpoint: str, weight: float, version: str):
37         """
38         Add routing rule
39
40         Args:
41             endpoint: Model endpoint URL
42             weight: Traffic weight (0-100)
43             version: Model version
44         """
45         route = Route(
46             endpoint=endpoint,
47             weight=weight,
48             version=version
49         )
50
51         self.routes.append(route)
52         self._normalize_weights()
53
54         self.logger.info(
55             f"Added route: {version} @ {endpoint} ({weight}%)"
56         )
57
58     def remove_route(self, version: str):
59         """Remove route by version"""
60         self.routes = [r for r in self.routes if r.version != version]
61         self._normalize_weights()

```

```

62         self.logger.info(f"Removed route: {version}")
63
64
65     def update_weight(self, version: str, new_weight: float):
66         """Update route weight"""
67         for route in self.routes:
68             if route.version == version:
69                 route.weight = new_weight
70                 break
71
72         self._normalize_weights()
73
74     def _normalize_weights(self):
75         """Normalize weights to sum to 100"""
76         total = sum(r.weight for r in self.routes)
77
78         if total > 0:
79             for route in self.routes:
80                 route.weight = (route.weight / total) * 100
81
82     def route_request(self,
83                       user_id: Optional[str] = None,
84                       sticky: bool = False) -> Route:
85         """
86         Route request to endpoint
87
88         Args:
89             user_id: Optional user identifier
90             sticky: Enable sticky sessions
91
92         Returns:
93             Selected route
94         """
95         if not self.routes:
96             raise ValueError("No routes configured")
97
98         # Sticky session handling
99         if sticky and user_id and user_id in self.sticky_sessions:
100             return self.sticky_sessions[user_id]
101
102         # Consistent hashing for user_id
103         if user_id:
104             selected = self._consistent_hash_route(user_id)
105         else:
106             selected = self._weighted_random_route()
107
108         # Store sticky session
109         if sticky and user_id:
110             self.sticky_sessions[user_id] = selected
111
112         return selected
113
114     def _consistent_hash_route(self, user_id: str) -> Route:
115         """Route using consistent hashing"""
116         # Hash user_id
117         hash_value = int(
118             hashlib.md5(user_id.encode('utf-8')).hexdigest(),
119             16

```

```

120     )
121
122     # Map to percentage
123     percentage = (hash_value % 10000) / 100.0
124
125     # Select route based on cumulative weights
126     cumulative = 0
127     for route in self.routes:
128         cumulative += route.weight
129         if percentage <= cumulative:
130             return route
131
132     return self.routes[-1]
133
134     def _weighted_random_route(self) -> Route:
135         """Select route using weighted random selection"""
136         weights = [r.weight for r in self.routes]
137         selected = random.choices(self.routes, weights=weights, k=1)[0]
138         return selected
139
140     def get_routing_stats(self) -> Dict:
141         """Get routing statistics"""
142         return {
143             'model': self.model_name,
144             'routes': [
145                 {
146                     'version': r.version,
147                     'endpoint': r.endpoint,
148                     'weight': r.weight
149                 }
150                 for r in self.routes
151             ],
152             'sticky_sessions': len(self.sticky_sessions)
153         }
154
155     # Example usage
156     def example_traffic_router():
157         """Example of traffic routing"""
158
159         router = TrafficRouter(model_name="product_ranker")
160
161         # Add routes
162         router.add_route("http://prod.example.com/v1", weight=70, version="v1.0")
163         router.add_route("http://prod.example.com/v2", weight=30, version="v2.0")
164
165         # Simulate routing
166         version_counts = {'v1.0': 0, 'v2.0': 0}
167
168         for i in range(10000):
169             route = router.route_request(user_id=f"user_{i}", sticky=True)
170             version_counts[route.version] += 1
171
172         print("Routing Distribution:")
173         for version, count in version_counts.items():
174             print(f"    {version}: {count/100:.1f}%")
175
176         # Update weights (simulate canary rollout)
177         print("\nUpdating weights for canary rollout...")

```



```

178     router.update_weight("v2.0", 50)
179
180     version_counts = {'v1.0': 0, 'v2.0': 0}
181     for i in range(10000, 20000):
182         route = router.route_request(user_id=f"user_{i}", sticky=False)
183         version_counts[route.version] += 1
184
185     print("New Routing Distribution:")
186     for version, count in version_counts.items():
187         print(f"  {version}: {count/100:.1f}%")

```

Listing 12.6: Advanced Traffic Router

12.8 Rollback Mechanisms and Circuit Breakers

12.8.1 Automated Rollback System

Automated rollback systems detect degradation and revert to previous versions without human intervention. Combined with circuit breakers, they prevent cascading failures.

```

1  from typing import Dict, Optional, Callable
2  from dataclasses import dataclass
3  from datetime import datetime, timedelta
4  from enum import Enum
5  import logging
6  import time
7
8  class CircuitState(Enum):
9      """Circuit breaker states"""
10     CLOSED = "closed" # Normal operation
11     OPEN = "open" # Failing, reject requests
12     HALF_OPEN = "half_open" # Testing recovery
13
14     @dataclass
15     class HealthThreshold:
16         """Health check thresholds"""
17         max_error_rate: float = 0.05 # 5%
18         max_latency_p99: float = 1000 # 1 second
19         min_success_rate: float = 0.95 # 95%
20         window_minutes: int = 5
21
22     class CircuitBreaker:
23         """
24         Circuit breaker for model endpoints
25
26         Prevents cascading failures by failing fast when errors exceed threshold
27         """
28
29         def __init__(self,
30             name: str,
31             failure_threshold: int = 5,
32             recovery_timeout: int = 60,
33             half_open_max_calls: int = 3):
34             """
35             Initialize circuit breaker

```

```

37     Args:
38         name: Circuit breaker identifier
39         failure_threshold: Consecutive failures to open circuit
40         recovery_timeout: Seconds before attempting recovery
41         half_open_max_calls: Test calls in half-open state
42     """
43     self.name = name
44     self.failure_threshold = failure_threshold
45     self.recovery_timeout = recovery_timeout
46     self.half_open_max_calls = half_open_max_calls
47     self.logger = logging.getLogger(f"CircuitBreaker_{name}")
48
49     self.state = CircuitState.CLOSED
50     self.failure_count = 0
51     self.last_failure_time: Optional[datetime] = None
52     self.half_open_calls = 0
53
54 def call(self, func: Callable, *args, **kwargs):
55     """
56     Execute function with circuit breaker protection
57
58     Args:
59         func: Function to call
60         *args: Function arguments
61         **kwargs: Function keyword arguments
62
63     Returns:
64         Function result
65
66     Raises:
67         Exception: If circuit is open
68     """
69     # Check circuit state
70     if self.state == CircuitState.OPEN:
71         if self._should_attempt_reset():
72             self.logger.info(f"Attempting recovery (half-open)")
73             self.state = CircuitState.HALF_OPEN
74             self.half_open_calls = 0
75         else:
76             raise Exception(f"Circuit breaker {self.name} is OPEN")
77
78     # Execute call
79     try:
80         result = func(*args, **kwargs)
81         self._on_success()
82         return result
83
84     except Exception as e:
85         self._on_failure()
86         raise e
87
88 def _should_attempt_reset(self) -> bool:
89     """Check if should attempt recovery"""
90     if self.last_failure_time is None:
91         return True
92
93     time_since_failure = (datetime.now() - self.last_failure_time).total_seconds()
94     return time_since_failure >= self.recovery_timeout

```

```

95
96 def _on_success(self):
97     """Handle successful call"""
98     if self.state == CircuitState.HALF_OPEN:
99         self.half_open_calls += 1
100
101         if self.half_open_calls >= self.half_open_max_calls:
102             self.logger.info(f"Circuit recovered (closing)")
103             self.state = CircuitState.CLOSED
104             self.failure_count = 0
105
106     elif self.state == CircuitState.CLOSED:
107         self.failure_count = max(0, self.failure_count - 1)
108
109 def _on_failure(self):
110     """Handle failed call"""
111     self.failure_count += 1
112     self.last_failure_time = datetime.now()
113
114     if self.state == CircuitState.HALF_OPEN:
115         self.logger.warning(f"Recovery failed, opening circuit")
116         self.state = CircuitState.OPEN
117
118     elif self.failure_count >= self.failure_threshold:
119         self.logger.error(
120             f"Failure threshold exceeded ({self.failure_count}), opening circuit"
121         )
122         self.state = CircuitState.OPEN
123
124 class RollbackManager:
125     """
126     Automated rollback management
127
128     Monitors deployment health and triggers rollback when needed
129     """
130
131     def __init__(self,
132                 model_name: str,
133                 deployment_manager,
134                 health_threshold: HealthThreshold):
135         """
136         Initialize rollback manager
137
138         Args:
139             model_name: Model name
140             deployment_manager: Deployment manager (blue-green, canary, etc.)
141             health_threshold: Health check thresholds
142         """
143         self.model_name = model_name
144         self.deployment_manager = deployment_manager
145         self.health_threshold = health_threshold
146         self.logger = logging.getLogger(f"Rollback_{model_name}")
147
148         self.monitoring = True
149         self.circuit_breaker = CircuitBreaker(name=model_name)
150
151     def monitor_and_rollback(self, check_interval: int = 30):
152         """

```

```

153     Continuously monitor deployment and rollback if unhealthy
154
155     Args:
156         check_interval: Seconds between health checks
157     """
158     self.logger.info("Starting health monitoring")
159
160     while self.monitoring:
161         try:
162             health = self._check_health()
163
164             if not health['is_healthy']:
165                 self.logger.error(
166                     f"Deployment unhealthy: {health['reason']}"
167                 )
168
169                 # Trigger rollback
170                 if self._should_rollback(health):
171                     self.logger.warning("Triggering automated rollback")
172                     self._execute_rollback()
173                     break
174
175                 time.sleep(check_interval)
176
177         except Exception as e:
178             self.logger.error(f"Monitor error: {str(e)}")
179
180     def _check_health(self) -> Dict:
181         """Check deployment health"""
182         try:
183             status = self.deployment_manager.get_deployment_status()
184
185             if 'active' not in status:
186                 return {
187                     'is_healthy': False,
188                     'reason': 'No active deployment'
189                 }
190
191             active = status['active']
192
193             # Check error rate
194             if active['error_rate'] > self.health_threshold.max_error_rate:
195                 return {
196                     'is_healthy': False,
197                     'reason': f"Error rate {active['error_rate']:.2%} exceeds "
198                             f"threshold {self.health_threshold.max_error_rate:.2%}"
199                 }
200
201             # Check latency
202             if active['latency_p99'] > self.health_threshold.max_latency_p99:
203                 return {
204                     'is_healthy': False,
205                     'reason': f"P99 latency {active['latency_p99']}ms exceeds "
206                             f"threshold {self.health_threshold.max_latency_p99}ms"
207                 }
208
209             return {'is_healthy': True}
210

```

```

211     except Exception as e:
212         return {
213             'is_healthy': False,
214             'reason': f"Health check failed: {str(e)}"
215         }
216
217     def _should_rollback(self, health: Dict) -> bool:
218         """Determine if should trigger rollback"""
219         # Always rollback on critical failures
220         return not health['is_healthy']
221
222     def _execute_rollback(self):
223         """Execute rollback"""
224         try:
225             success = self.deployment_manager.rollback()
226
227             if success:
228                 self.logger.info("Rollback successful")
229             else:
230                 self.logger.error("Rollback failed")
231
232     except Exception as e:
233         self.logger.error(f"Rollback error: {str(e)}")

```

Listing 12.7: Rollback Manager with Circuit Breaker

12.9 Multi-Model Serving Architectures

12.9.1 Overview

Multi-model serving deploys multiple specialized models working together, either as ensembles or cascades. This architecture enables complex prediction workflows that single models cannot achieve.

Use cases include:

- **Model ensembles:** Combine predictions from multiple models
- **Cascade architectures:** Fast screening model followed by expensive detailed model
- **Specialized models:** Different models for different user segments
- **Multi-task models:** Coordinated models for related predictions

12.9.2 Implementation

```

1 from typing import Dict, List, Optional, Any
2 from dataclasses import dataclass
3 from enum import Enum
4 import numpy as np
5 import logging
6
7 class AggregationStrategy(Enum):
8     """Model output aggregation strategies"""

```

```

9     AVERAGE = "average"
10    WEIGHTED_AVERAGE = "weighted_average"
11    VOTING = "voting"
12    MAX = "max"
13    CASCADE = "cascade"
14
15    @dataclass
16    class ModelSpec:
17        """Specification for a model in multi-model system"""
18        name: str
19        endpoint: str
20        version: str
21        weight: float = 1.0
22        metadata: Dict = None
23
24    class MultiModelServer:
25        """
26        Multi-model serving with various aggregation strategies
27
28        Coordinates predictions from multiple models
29        """
30
31    def __init__(self, system_name: str):
32        """
33        Initialize multi-model server
34
35        Args:
36            system_name: System identifier
37        """
38        self.system_name = system_name
39        self.models: List[ModelSpec] = []
40        self.logger = logging.getLogger(f"MultiModel_{system_name}")
41
42    def add_model(self, model: ModelSpec):
43        """Add model to ensemble"""
44        self.models.append(model)
45        self.logger.info(f"Added model: {model.name} (weight={model.weight})")
46
47    async def predict_ensemble(self,
48                              features: Dict,
49                              strategy: AggregationStrategy) -> Dict:
50        """
51        Get ensemble prediction
52
53        Args:
54            features: Input features
55            strategy: Aggregation strategy
56
57        Returns:
58            Aggregated prediction
59        """
60        import asyncio
61        import aiohttp
62
63        # Collect predictions from all models
64        tasks = [
65            self._call_model(model, features)
66            for model in self.models

```

```

67     ]
68
69     results = await asyncio.gather(*tasks, return_exceptions=True)
70
71     # Filter successful predictions
72     valid_results = [
73         (model, result)
74         for model, result in zip(self.models, results)
75         if not isinstance(result, Exception)
76     ]
77
78     if not valid_results:
79         raise Exception("All model predictions failed")
80
81     # Aggregate based on strategy
82     if strategy == AggregationStrategy.AVERAGE:
83         prediction = self._aggregate_average(valid_results)
84     elif strategy == AggregationStrategy.WEIGHTED_AVERAGE:
85         prediction = self._aggregate_weighted_average(valid_results)
86     elif strategy == AggregationStrategy.VOTING:
87         prediction = self._aggregate_voting(valid_results)
88     elif strategy == AggregationStrategy.MAX:
89         prediction = self._aggregate_max(valid_results)
90     else:
91         raise ValueError(f"Unknown strategy: {strategy}")
92
93     return {
94         'prediction': prediction,
95         'num_models': len(valid_results),
96         'strategy': strategy.value
97     }
98
99     async def _call_model(self, model: ModelSpec, features: Dict) -> Any:
100         """Call individual model"""
101         import aiohttp
102
103         async with aiohttp.ClientSession() as session:
104             async with session.post(
105                 f"{model.endpoint}/predict",
106                 json=features,
107                 timeout=aiohttp.ClientTimeout(total=5)
108             ) as response:
109                 result = await response.json()
110                 return result
111
112     def _aggregate_average(self, results: List[tuple]) -> float:
113         """Simple average aggregation"""
114         predictions = [r[1] for r in results]
115         return np.mean(predictions)
116
117     def _aggregate_weighted_average(self, results: List[tuple]) -> float:
118         """Weighted average aggregation"""
119         weights = np.array([r[0].weight for r in results])
120         predictions = np.array([r[1] for r in results])
121
122         # Normalize weights
123         weights = weights / weights.sum()
124

```

```

125         return np.sum(predictions * weights)
126
127     def _aggregate_voting(self, results: List[tuple]) -> Any:
128         """Majority voting aggregation"""
129         from collections import Counter
130
131         predictions = [r[1] for r in results]
132         vote_counts = Counter(predictions)
133         winner = vote_counts.most_common(1)[0][0]
134
135         return winner
136
137     def _aggregate_max(self, results: List[tuple]) -> float:
138         """Maximum aggregation"""
139         predictions = [r[1] for r in results]
140         return max(predictions)
141
142 class CascadeModelServer:
143     """
144     Cascade model serving architecture
145
146     Uses fast screening model followed by expensive detailed model
147     """
148
149     def __init__(self,
150                 screening_model: ModelSpec,
151                 detailed_model: ModelSpec,
152                 threshold: float):
153         """
154         Initialize cascade server
155
156         Args:
157             screening_model: Fast screening model
158             detailed_model: Expensive detailed model
159             threshold: Threshold for screening model to trigger detailed model
160         """
161         self.screening_model = screening_model
162         self.detailed_model = detailed_model
163         self.threshold = threshold
164         self.logger = logging.getLogger("CascadeServer")
165
166     async def predict(self, features: Dict) -> Dict:
167         """
168         Cascade prediction
169
170         Args:
171             features: Input features
172
173         Returns:
174             Prediction result
175         """
176         import aiohttp
177
178         # Step 1: Screening model
179         async with aiohttp.ClientSession() as session:
180             async with session.post(
181                 f"{self.screening_model.endpoint}/predict",
182                 json=features,

```



```

183         timeout=aiohttp.ClientTimeout(total=1)
184     ) as response:
185         screening_result = await response.json()
186
187     screening_score = screening_result.get('score', 0)
188
189     # Step 2: Check threshold
190     if screening_score < self.threshold:
191         # Low score - skip detailed model
192         return {
193             'prediction': screening_result,
194             'model_used': 'screening',
195             'screening_score': screening_score
196         }
197
198     # Step 3: Detailed model
199     async with aiohttp.ClientSession() as session:
200         async with session.post(
201             f"{self.detailed_model.endpoint}/predict",
202             json=features,
203             timeout=aiohttp.ClientTimeout(total=5)
204         ) as response:
205             detailed_result = await response.json()
206
207         return {
208             'prediction': detailed_result,
209             'model_used': 'detailed',
210             'screening_score': screening_score
211         }
212
213     # Example usage
214     async def example_multi_model():
215         """Example of multi-model serving"""
216
217         # Ensemble example
218         ensemble = MultiModelServer(system_name="fraud_ensemble")
219
220         ensemble.add_model(ModelSpec(
221             name="rf_model",
222             endpoint="http://models.example.com/rf",
223             version="v1.0",
224             weight=0.4
225         ))
226
227         ensemble.add_model(ModelSpec(
228             name="xgb_model",
229             endpoint="http://models.example.com/xgb",
230             version="v1.0",
231             weight=0.6
232         ))
233
234         # Get ensemble prediction
235         features = {'amount': 150.0, 'merchant_id': 'merch_123'}
236
237         prediction = await ensemble.predict_ensemble(
238             features,
239             strategy=AggregationStrategy.WEIGHTED_AVERAGE
240         )

```

```
241 print(f"Ensemble prediction: {prediction}")
242
243
244 # Cascade example
245 cascade = CascadeModelServer(
246     screening_model=ModelSpec(
247         name="fast_screener",
248         endpoint="http://models.example.com/screening",
249         version="v1.0"
250     ),
251     detailed_model=ModelSpec(
252         name="detailed_analyzer",
253         endpoint="http://models.example.com/detailed",
254         version="v1.0"
255     ),
256     threshold=0.7
257 )
258
259 cascade_prediction = await cascade.predict(features)
260 print(f"Cascade prediction: {cascade_prediction}")
261 print(f"Model used: {cascade_prediction['model_used']}")
```

Listing 12.8: Multi-Model Serving Framework

12.10 Chapter Summary

This chapter covered comprehensive deployment strategies for production ML systems. Effective deployment balances risk management with rapid iteration, enabling teams to ship improvements confidently.

Key Takeaways:

- **Blue-Green Deployment:** Provides zero-downtime updates with instant rollback capability through atomic traffic switching between identical environments.
- **Canary Deployment:** Gradually rolls out new models with automated progression based on metrics, minimizing blast radius of potential issues.
- **Shadow Deployment:** Enables risk-free validation by routing traffic to new models without impacting users, logging predictions for analysis.
- **A/B Testing:** Rigorous statistical framework for comparing model versions, ensuring decisions are data-driven with proper significance testing.
- **Traffic Routing:** Advanced routers support weighted allocation, sticky sessions, and consistent hashing for controlled experiments.
- **Automated Rollback:** Circuit breakers and health monitoring detect degradation and trigger rollback automatically, preventing cascading failures.
- **Multi-Model Serving:** Coordinates multiple models through ensembles or cascades for complex prediction workflows.

Choosing the right deployment strategy depends on system requirements:

- Use **blue-green** for simple, fast deployments with instant rollback
- Use **canary** for gradual rollouts with automated quality gates
- Use **shadow** for risk-free production validation
- Use **A/B testing** for rigorous evaluation of model improvements
- Combine strategies for complex systems (e.g., shadow followed by canary)

Production ML deployment is not fire-and-forget. Continuous monitoring, automated health checks, and quick rollback mechanisms are essential for maintaining system reliability.

12.11 Exercises

Exercise 12.1. Implement Blue-Green Deployment with Health Checks

Build a complete blue-green deployment system for a classification model with comprehensive health monitoring.

Requirements:

- Deploy model to inactive environment
- Perform health checks (error rate, latency, prediction quality)
- Switch traffic only if all checks pass
- Implement rollback capability
- Log all deployment events

Solution:

```

1 import time
2 import requests
3
4 # See BlueGreenDeployment class implementation earlier in chapter
5
6 # Example deployment workflow
7 deployment = BlueGreenDeployment(
8     model_name="customer_churn",
9     load_balancer_url="http://lb.example.com",
10    health_check_interval=30
11 )
12
13 # Initial blue deployment
14 deployment.blue_endpoint = ModelEndpoint(
15     name="customer_churn",
16     url="http://blue.prod.com/predict",
17     version="v1.0",
18     environment=Environment.BLUE,
19     health_check_url="http://blue.prod.com/health"
20 )
21 deployment.active_environment = Environment.BLUE
22
23 # Deploy v2.0 to green

```

```

24 print("Deploying v2.0 to green environment...")
25 success = deployment.deploy_to_inactive(
26     version="v2.0",
27     endpoint_url="http://green.prod.com/predict",
28     health_check_url="http://green.prod.com/health"
29 )
30
31 if not success:
32     print("Deployment failed health checks!")
33     exit(1)
34
35 # Custom validation: test predictions
36 def validate_predictions(endpoint: ModelEndpoint) -> bool:
37     test_cases = [
38         {'tenure_months': 12, 'monthly_charges': 50},
39         {'tenure_months': 60, 'monthly_charges': 100}
40     ]
41
42     for test_case in test_cases:
43         response = requests.post(
44             endpoint.url,
45             json=test_case,
46             timeout=5
47         )
48
49         if response.status_code != 200:
50             return False
51
52         prediction = response.json()
53         if 'probability' not in prediction:
54             return False
55
56     return True
57
58 # Switch traffic with validation
59 if deployment.switch_traffic(validation_checks=validate_predictions):
60     print("Traffic switched successfully!")
61
62     # Monitor for 5 minutes
63     time.sleep(300)
64
65     status = deployment.get_deployment_status()
66     if status['active']['error_rate'] > 0.02:
67         print("High error rate detected, rolling back!")
68         deployment.rollback()
69 else:
70     print("Traffic switch failed!")

```

Exercise 12.2. Design Canary Deployment with Progressive Rollout

Implement a canary deployment with three stages: 5%, 25%, 50% traffic allocation.

Requirements:

- Define success criteria for each stage
- Automatically progress through stages if criteria met
- Rollback if any stage fails

- Generate deployment report with metrics

Solution:

```

1 import numpy as np
2
3 canary = CanaryDeployment(
4     model_name="product_recommender",
5     baseline_endpoint="http://prod.com/v1",
6     canary_endpoint="http://prod.com/v2",
7     traffic_router_url="http://router.com"
8 )
9
10 # Define progressive stages
11 stages = [
12     CanaryStage(
13         traffic_percentage=5.0,
14         duration_minutes=15,
15         success_criteria={
16             'max_error_rate': 0.01,
17             'max_error_rate_ratio': 3.0, # Allow 3x baseline
18             'max_latency_p99': 500,
19             'min_prediction_quality': 0.8
20         }
21     ),
22     CanaryStage(
23         traffic_percentage=25.0,
24         duration_minutes=30,
25         success_criteria={
26             'max_error_rate': 0.01,
27             'max_error_rate_ratio': 2.0,
28             'max_latency_p99': 500,
29             'min_prediction_quality': 0.85
30         }
31     ),
32     CanaryStage(
33         traffic_percentage=50.0,
34         duration_minutes=60,
35         success_criteria={
36             'max_error_rate': 0.01,
37             'max_error_rate_ratio': 1.5,
38             'max_latency_p99': 450,
39             'min_prediction_quality': 0.9
40         }
41     )
42 ]
43
44 # Execute rollout
45 print("Starting canary rollout...")
46 success = canary.execute_rollout(stages)
47
48 # Generate report
49 report = canary.get_rollout_report()
50 print(f"\nCanary Deployment {'SUCCEEDED' if success else 'FAILED'}")
51 print(f"Final traffic: {report['final_traffic_percentage']:.1f}%")
52 print(f"\nCanary Performance:")
53 print(f" Avg Error Rate: {report['canary_stats']['avg_error_rate']:.3%}")
54 print(f" Max Error Rate: {report['canary_stats']['max_error_rate']:.3%}")
55 print(f" Avg P99 Latency: {report['canary_stats']['avg_latency_p99']:.1f}ms")

```

```
56
57 if success:
58     print("\nProceeding to full rollout (100%)")
59 else:
60     print("\nRollout stopped, canary removed")
```

Exercise 12.3. Build A/B Test with Statistical Rigor

Design an A/B test to compare two recommendation models, tracking click-through rate and conversion rate.

Requirements:

- Calculate required sample size for 80% power
- Implement consistent user assignment
- Track multiple metrics
- Perform statistical significance testing
- Generate comprehensive report

Solution:

```
1 import numpy as np
2
3 # Define variants
4 control = Variant(
5     name="baseline_recommender",
6     endpoint="http://prod.com/v1",
7     traffic_percentage=50.0,
8     description="Current collaborative filtering model"
9 )
10
11 treatment = Variant(
12     name="neural_recommender",
13     endpoint="http://prod.com/v2",
14     traffic_percentage=50.0,
15     description="New neural collaborative filtering model"
16 )
17
18 # Initialize test
19 ab_test = ABTest(
20     test_name="recommender_neural_vs_baseline",
21     control_variant=control,
22     treatment_variant=treatment,
23     alpha=0.05,
24     minimum_detectable_effect=0.02 # Detect 2% improvement
25 )
26
27 # Calculate sample size
28 required_n = ab_test.calculate_sample_size(
29     baseline_mean=0.15, # 15% baseline CTR
30     baseline_std=0.1,
31     power=0.8
32 )
33
34 print(f"Required sample size: {required_n} per variant")
```

```

35 print(f"Estimated days to complete: {required_n / 10000} days at 10K users/day")
36
37 # Run experiment (simulated)
38 for user_id in range(20000): # 10K per variant
39     variant = ab_test.assign_variant(f"user_{user_id}")
40
41     # Simulate metrics
42     if variant.name == "baseline_recommender":
43         clicked = np.random.random() < 0.15
44         converted = clicked and np.random.random() < 0.08
45         latency = np.random.gamma(2, 40)
46     else:
47         clicked = np.random.random() < 0.16 # 6.7% improvement
48         converted = clicked and np.random.random() < 0.10 # 25% improvement
49         latency = np.random.gamma(2, 35) # Slightly faster
50
51     ab_test.record_metric("ctr", float(clicked), f"user_{user_id}", variant.name)
52     ab_test.record_metric("cvr", float(converted), f"user_{user_id}", variant.name)
53     ab_test.record_metric("latency_ms", latency, f"user_{user_id}", variant.name)
54
55 # Analyze and report
56 ab_test.print_report(metrics_to_analyze=["ctr", "cvr", "latency_ms"])
57
58 # Decision
59 summary = ab_test.get_test_summary(["ctr", "cvr"])
60 ctr_result = summary['results']['ctr']
61 cvr_result = summary['results']['cvr']
62
63 if ctr_result['is_significant'] and cvr_result['is_significant']:
64     if ctr_result['relative_improvement'] > 0 and cvr_result['relative_improvement'] >
65         0:
66         print("\n[OK] DECISION: Launch neural recommender")
67         print(f"    Expected CTR improvement: {ctr_result['relative_improvement']:+.2%}")
68         print(f"    Expected CVR improvement: {cvr_result['relative_improvement']:+.2%}")
69     else:
70         print("\n[X] DECISION: Keep baseline")
71 else:
72     print("\n[!] DECISION: Results not statistically significant, continue test")

```

Exercise 12.4. Implement Shadow Deployment for Risk-Free Testing

Create a shadow deployment to validate a new fraud detection model without impacting production.

Requirements:

- Route all traffic to both primary and shadow models
- Compare predictions and calculate match rate
- Track latency for both models
- Generate analysis report
- Do not impact user experience

Solution:

```

1 import asyncio
2 import numpy as np
3
4 async def run_shadow_deployment():
5     # Custom comparison for fraud scores
6     def compare_fraud_scores(primary, shadow):
7         if primary is None or shadow is None:
8             return False, None
9
10        primary_score = primary.get('fraud_score', 0)
11        shadow_score = shadow.get('fraud_score', 0)
12
13        # Check if decisions match
14        primary_decision = primary_score > 0.5
15        shadow_decision = shadow_score > 0.5
16        match = primary_decision == shadow_decision
17
18        # Score divergence
19        divergence = abs(primary_score - shadow_score)
20
21        return match, divergence
22
23    # Initialize shadow deployment
24    shadow = ShadowDeployment(
25        model_name="fraud_detector",
26        primary_endpoint="http://prod.com/fraud/v1",
27        shadow_endpoint="http://prod.com/fraud/v2",
28        comparison_fn=compare_fraud_scores
29    )
30
31    # Process transactions
32    for txn_id in range(1000):
33        transaction = {
34            'amount': np.random.lognormal(4, 1),
35            'merchant_id': f"merch_{np.random.randint(1, 100)}",
36            'hour': np.random.randint(0, 24)
37        }
38
39        # Get primary prediction (returned to user)
40        prediction = await shadow.predict(transaction, request_id=f"txn_{txn_id}")
41
42        # Shadow prediction logged in background
43        await asyncio.sleep(0.01) # Simulate processing
44
45    # Analyze shadow deployment
46    analysis = shadow.get_analysis(window_minutes=60)
47
48    print("\nShadow Deployment Analysis:")
49    print(f"{'='*60}")
50    print(f"Total Predictions: {analysis['total_predictions']}")
51    print(f"Match Rate: {analysis['match_rate']:.2%}")
52    print(f"Avg Divergence: {analysis['avg_divergence']:.4f}")
53    print(f"Shadow Error Rate: {analysis['shadow_error_rate']:.2%}")
54    print(f"\nLatency Comparison:")
55    print(f"  Primary P99: {analysis['latency_stats']['primary_p99']:.1f}ms")
56    print(f"  Shadow P99: {analysis['latency_stats']['shadow_p99']:.1f}ms")
57

```



```

58     # Decision
59     if analysis['match_rate'] > 0.95 and analysis['shadow_error_rate'] < 0.01:
60         print("\n[OK] Shadow model performs well - proceed to canary deployment")
61     else:
62         print("\n[X] Shadow model needs improvement")
63
64     # Run shadow deployment
65     asyncio.run(run_shadow_deployment())

```

Exercise 12.5. Build Multi-Model Ensemble System

Implement an ensemble system combining three models for improved prediction accuracy.

Requirements:

- Combine predictions from RandomForest, XGBoost, and LightGBM
- Support weighted average and voting aggregation
- Handle partial failures (continue with available models)
- Track which models contributed to each prediction

Solution:

```

1  import asyncio
2
3  async def example_ensemble_system():
4      # Initialize ensemble
5      ensemble = MultiModelServer(system_name="credit_risk_ensemble")
6
7      # Add models with different weights
8      ensemble.add_model(ModelSpec(
9          name="random_forest",
10         endpoint="http://models.com/rf",
11         version="v1.0",
12         weight=0.3,
13         metadata={'training_date': '2024-01-01'}
14     ))
15
16     ensemble.add_model(ModelSpec(
17         name="xgboost",
18         endpoint="http://models.com/xgb",
19         version="v2.1",
20         weight=0.4,
21         metadata={'training_date': '2024-01-15'}
22     ))
23
24     ensemble.add_model(ModelSpec(
25         name="lightgbm",
26         endpoint="http://models.com/lgbm",
27         version="v1.5",
28         weight=0.3,
29         metadata={'training_date': '2024-01-10'}
30     ))
31
32     # Test case
33     features = {

```

```
34     'annual_income': 75000,
35     'debt_to_income': 0.35,
36     'credit_score': 720,
37     'loan_amount': 25000
38 }
39
40 # Get predictions with different strategies
41 weighted_pred = await ensemble.predict_ensemble(
42     features,
43     strategy=AggregationStrategy.WEIGHTED_AVERAGE
44 )
45
46 print(f"Weighted Average Prediction: {weighted_pred}")
47
48 # Test robustness: even if one model fails, ensemble continues
49 print("\nTesting with one model failure...")
50 # (Ensemble automatically handles failures)
51
52 # Run ensemble
53 asyncio.run(example_ensemble_system())
```

Chapter 13

Model Versioning and Registry

13.1 Learning Objectives

After completing this chapter, readers will be able to:

- Implement comprehensive model versioning strategies with MLflow
- Track model lineage, metadata, and dependencies systematically
- Design and implement approval workflows for model governance
- Compare models across multiple dimensions and promote winners
- Integrate model registry with CI/CD pipelines for automation
- Package and containerize models for reproducible deployment
- Build production-grade model management systems

13.2 Introduction

Model versioning and registry systems are critical infrastructure for production ML. As teams deploy dozens or hundreds of model versions, managing which models are in production, tracking their lineage, and governing their lifecycle becomes essential. Without proper model management, teams face chaos: lost experiments, unclear model provenance, ungoverned deployments, and inability to reproduce or rollback models.

A robust model registry provides:

- **Version control:** Track every model version with immutable identifiers
- **Lineage tracking:** Understand how models were trained, from data to hyperparameters
- **Governance:** Approval workflows ensure only validated models reach production
- **Reproducibility:** Package models with dependencies for exact recreation
- **Comparison:** Evaluate models systematically before promotion
- **Audit trail:** Complete history of model changes for compliance

This chapter covers production-grade model management systems, centered on MLflow but applicable to any registry system.

13.3 MLflow Model Registry: Advanced Features

13.3.1 Architecture and Core Concepts

MLflow Model Registry provides centralized model storage with versioning, stage transitions, and annotations. Understanding its architecture enables effective model management at scale.

Key concepts:

- **Registered Model:** A logical model entity with multiple versions
- **Model Version:** A specific trained instance with unique version number
- **Model Stage:** Lifecycle stage (None, Staging, Production, Archived)
- **Model Signature:** Input/output schema for validation
- **Model Artifact:** Serialized model files and dependencies

13.3.2 Complete MLflow Registry Implementation

```

1 import mlflow
2 from mlflow.tracking import MlflowClient
3 from mlflow.models.signature import infer_signature
4 from typing import Dict, List, Optional, Any
5 from dataclasses import dataclass
6 from datetime import datetime
7 import pandas as pd
8 import numpy as np
9 import logging
10 import json
11
12 @dataclass
13 class ModelMetadata:
14     """Comprehensive model metadata"""
15     name: str
16     version: str
17     stage: str
18     run_id: str
19     experiment_id: str
20
21     # Performance metrics
22     metrics: Dict[str, float]
23
24     # Training metadata
25     training_date: datetime
26     training_duration_seconds: float
27     dataset_version: str
28     dataset_size: int
29
30     # Model characteristics
31     model_type: str
32     framework: str
33     framework_version: str
34
35     # Environment

```

```

36 python_version: str
37 dependencies: List[str]
38
39 # Tags and annotations
40 tags: Dict[str, str]
41 description: str
42
43 class MLflowModelRegistry:
44     """
45     Advanced MLflow Model Registry manager
46
47     Provides high-level API for model management, versioning, and governance
48     """
49
50     def __init__(self, tracking_uri: str = None, registry_uri: str = None):
51         """
52         Initialize MLflow registry manager
53
54         Args:
55             tracking_uri: MLflow tracking server URI
56             registry_uri: Model registry URI (defaults to tracking_uri)
57         """
58         if tracking_uri:
59             mlflow.set_tracking_uri(tracking_uri)
60         if registry_uri:
61             mlflow.set_registry_uri(registry_uri)
62
63         self.client = MlflowClient()
64         self.logger = logging.getLogger("MLflowRegistry")
65
66     def register_model(self,
67                       model,
68                       model_name: str,
69                       run_id: str,
70                       metadata: ModelMetadata,
71                       input_example: Any = None,
72                       signature: Any = None) -> str:
73         """
74         Register a new model version with comprehensive metadata
75
76         Args:
77             model: Trained model object
78             model_name: Registered model name
79             run_id: MLflow run ID
80             metadata: Model metadata
81             input_example: Example input for schema inference
82             signature: Model signature (inferred if not provided)
83
84         Returns:
85             Model version number
86         """
87         self.logger.info(f"Registering model: {model_name}")
88
89         # Infer signature if not provided
90         if signature is None and input_example is not None:
91             # Generate predictions for signature
92             predictions = model.predict(input_example)
93             signature = infer_signature(input_example, predictions)

```

```

94
95     # Log model with MLflow
96     with mlflow.start_run(run_id=run_id):
97         # Log model
98         mlflow.sklearn.log_model(
99             model,
100             artifact_path="model",
101             registered_model_name=model_name,
102             signature=signature,
103             input_example=input_example
104         )
105
106         # Log metrics
107         for metric_name, metric_value in metadata.metrics.items():
108             mlflow.log_metric(metric_name, metric_value)
109
110         # Log parameters
111         mlflow.log_param("model_type", metadata.model_type)
112         mlflow.log_param("framework", metadata.framework)
113         mlflow.log_param("framework_version", metadata.framework_version)
114         mlflow.log_param("dataset_version", metadata.dataset_version)
115         mlflow.log_param("dataset_size", metadata.dataset_size)
116         mlflow.log_param("training_duration", metadata.training_duration_seconds)
117
118         # Log tags
119         for tag_key, tag_value in metadata.tags.items():
120             mlflow.set_tag(tag_key, tag_value)
121
122         # Get the latest version
123         latest_versions = self.client.get_latest_versions(model_name)
124         if latest_versions:
125             version = latest_versions[0].version
126
127         # Add description
128         self.client.update_model_version(
129             name=model_name,
130             version=version,
131             description=metadata.description
132         )
133
134         self.logger.info(f"Registered {model_name} version {version}")
135         return version
136
137     return None
138
139     def transition_model_stage(self,
140                               model_name: str,
141                               version: str,
142                               stage: str,
143                               archive_existing: bool = True) -> bool:
144
145         """
146         Transition model version to new stage
147
148         Args:
149         model_name: Registered model name
150         version: Model version
151         stage: Target stage (Staging, Production, Archived)
152         archive_existing: Archive existing models in target stage

```

```

152
153     Returns:
154         True if transition successful
155     """
156     try:
157         self.client.transition_model_version_stage(
158             name=model_name,
159             version=version,
160             stage=stage,
161             archive_existing_versions=archive_existing
162         )
163
164         self.logger.info(
165             f"Transitioned {model_name} v{version} to {stage}"
166         )
167         return True
168
169     except Exception as e:
170         self.logger.error(f"Stage transition failed: {str(e)}")
171         return False
172
173     def get_model_version(self,
174                           model_name: str,
175                           version: str = None,
176                           stage: str = None) -> Optional[Dict]:
177     """
178     Get model version details
179
180     Args:
181         model_name: Registered model name
182         version: Specific version (optional)
183         stage: Get latest version in stage (optional)
184
185     Returns:
186         Model version information
187     """
188     try:
189         if version:
190             mv = self.client.get_model_version(model_name, version)
191         elif stage:
192             versions = self.client.get_latest_versions(model_name, stages=[stage])
193             if not versions:
194                 return None
195             mv = versions[0]
196         else:
197             return None
198
199         # Get run details
200         run = self.client.get_run(mv.run_id)
201
202         return {
203             'name': mv.name,
204             'version': mv.version,
205             'stage': mv.current_stage,
206             'run_id': mv.run_id,
207             'description': mv.description,
208             'tags': mv.tags,
209             'metrics': run.data.metrics,

```

```

210         'params': run.data.params,
211         'creation_timestamp': mv.creation_timestamp,
212         'last_updated_timestamp': mv.last_updated_timestamp,
213         'source': mv.source
214     }
215
216     except Exception as e:
217         self.logger.error(f"Failed to get model version: {str(e)}")
218         return None
219
220     def compare_models(self,
221                       model_name: str,
222                       version1: str,
223                       version2: str,
224                       metric_names: List[str]) -> Dict:
225         """
226         Compare two model versions across multiple metrics
227
228         Args:
229             model_name: Registered model name
230             version1: First version to compare
231             version2: Second version to compare
232             metric_names: Metrics to compare
233
234         Returns:
235             Comparison results
236         """
237         v1_info = self.get_model_version(model_name, version1)
238         v2_info = self.get_model_version(model_name, version2)
239
240         if not v1_info or not v2_info:
241             return {'error': 'One or both versions not found'}
242
243         comparison = {
244             'model_name': model_name,
245             'version1': version1,
246             'version2': version2,
247             'metrics': {}
248         }
249
250         for metric_name in metric_names:
251             m1 = v1_info['metrics'].get(metric_name)
252             m2 = v2_info['metrics'].get(metric_name)
253
254             if m1 is not None and m2 is not None:
255                 improvement = ((m2 - m1) / abs(m1)) * 100 if m1 != 0 else 0
256
257                 comparison['metrics'][metric_name] = {
258                     'version1': m1,
259                     'version2': m2,
260                     'improvement_percent': improvement,
261                     'winner': version2 if m2 > m1 else version1
262                 }
263
264         return comparison
265
266     def search_models(self,
267                     filter_string: str = None,

```



```

268         max_results: int = 100) -> List[Dict]:
269
270     """
271     Search registered models
272
273     Args:
274         filter_string: Filter query (e.g., "name LIKE 'fraud%')
275         max_results: Maximum results to return
276
277     Returns:
278         List of matching models
279     """
280     try:
281         models = self.client.search_registered_models(
282             filter_string=filter_string,
283             max_results=max_results
284         )
285
286         return [
287             {
288                 'name': model.name,
289                 'creation_timestamp': model.creation_timestamp,
290                 'last_updated_timestamp': model.last_updated_timestamp,
291                 'description': model.description,
292                 'tags': model.tags,
293                 'latest_versions': [
294                     {
295                         'version': v.version,
296                         'stage': v.current_stage
297                     }
298                     for v in model.latest_versions
299                 ]
300             }
301             for model in models
302         ]
303
304     except Exception as e:
305         self.logger.error(f"Model search failed: {str(e)}")
306         return []
307
308     def delete_model_version(self,
309                             model_name: str,
310                             version: str) -> bool:
311
312     """
313     Delete a model version
314
315     Args:
316         model_name: Registered model name
317         version: Version to delete
318
319     Returns:
320         True if deletion successful
321     """
322     try:
323         self.client.delete_model_version(model_name, version)
324         self.logger.info(f"Deleted {model_name} version {version}")
325         return True
326
327     except Exception as e:

```

```

326         self.logger.error(f"Deletion failed: {str(e)}")
327         return False
328
329     def get_model_stages_summary(self, model_name: str) -> Dict:
330         """
331         Get summary of all model versions by stage
332
333         Args:
334             model_name: Registered model name
335
336         Returns:
337             Summary by stage
338         """
339         try:
340             all_versions = self.client.search_model_versions(
341                 f"name='{model_name}'"
342             )
343
344             summary = {
345                 'Production': [],
346                 'Staging': [],
347                 'Archived': [],
348                 'None': []
349             }
350
351             for mv in all_versions:
352                 summary[mv.current_stage].append({
353                     'version': mv.version,
354                     'run_id': mv.run_id,
355                     'creation_timestamp': mv.creation_timestamp
356                 })
357
358             return summary
359
360         except Exception as e:
361             self.logger.error(f"Failed to get stages summary: {str(e)}")
362             return {}
363
364     def load_production_model(self, model_name: str):
365         """
366         Load the current production model
367
368         Args:
369             model_name: Registered model name
370
371         Returns:
372             Loaded model
373         """
374         model_uri = f"models://{model_name}/Production"
375         return mlflow.sklearn.load_model(model_uri)
376
377     # Example usage
378     def example_mlflow_registry():
379         """Example of using MLflow Model Registry"""
380
381         # Initialize registry
382         registry = MLflowModelRegistry(
383             tracking_uri="http://localhost:5000"

```

```

384 )
385
386 # Train a model (simplified)
387 from sklearn.ensemble import RandomForestClassifier
388 from sklearn.datasets import make_classification
389
390 X, y = make_classification(n_samples=1000, n_features=20, random_state=42)
391 model = RandomForestClassifier(n_estimators=100, random_state=42)
392 model.fit(X, y)
393
394 # Create metadata
395 metadata = ModelMetadata(
396     name="fraud_detector",
397     version="1",
398     stage="None",
399     run_id=mlflow.active_run().info.run_id,
400     experiment_id="0",
401     metrics={
402         'accuracy': 0.95,
403         'precision': 0.93,
404         'recall': 0.97,
405         'f1_score': 0.95
406     },
407     training_date=datetime.now(),
408     training_duration_seconds=120.5,
409     dataset_version="v2023-01-15",
410     dataset_size=1000,
411     model_type="RandomForest",
412     framework="scikit-learn",
413     framework_version="1.4.2",
414     python_version="3.11.6",
415     dependencies=["scikit-learn==1.4.2", "numpy==1.26.4"],
416     tags={'team': 'ml-team', 'use_case': 'fraud_detection'},
417     description="Random Forest model for fraud detection v1"
418 )
419
420 # Register model
421 version = registry.register_model(
422     model=model,
423     model_name="fraud_detector",
424     run_id=mlflow.active_run().info.run_id,
425     metadata=metadata,
426     input_example=X[:5]
427 )
428
429 print(f"Registered model version: {version}")
430
431 # Transition to staging
432 registry.transition_model_stage(
433     model_name="fraud_detector",
434     version=version,
435     stage="Staging"
436 )
437
438 # Get model info
439 model_info = registry.get_model_version(
440     model_name="fraud_detector",
441     stage="Staging"

```

```

442 )
443 print(f"Staging model: {model_info}")
444
445 # Compare models (if multiple versions exist)
446 # comparison = registry.compare_models(
447 #     model_name="fraud_detector",
448 #     version1="1",
449 #     version2="2",
450 #     metric_names=['accuracy', 'f1_score']
451 # )
452
453 # Get stages summary
454 summary = registry.get_model_stages_summary("fraud_detector")
455 print(f"Model stages: {summary}")

```

Listing 13.1: Advanced MLflow Model Registry Manager

13.3.3 Model Signature and Schema Validation

Model signatures enforce input/output schemas, preventing silent failures from schema mismatches.

```

1 from typing import Any, List
2 from mlflow.models.signature import ModelSignature, infer_signature
3 from mlflow.types.schema import Schema, ColSpec
4 import pandas as pd
5 import numpy as np
6 from sklearn.ensemble import RandomForestClassifier
7
8 class ModelSignatureValidator:
9     """Validate model inputs against registered signature"""
10
11     @staticmethod
12     def create_signature(input_df: pd.DataFrame,
13                        output: Any) -> ModelSignature:
14         """
15         Create explicit model signature
16
17         Args:
18             input_df: Example input DataFrame
19             output: Example output
20
21         Returns:
22             ModelSignature
23         """
24         return infer_signature(input_df, output)
25
26     @staticmethod
27     def create_manual_signature(
28         input_schema: List[tuple],
29         output_schema: List[tuple]
30     ) -> ModelSignature:
31         """
32         Create signature manually
33
34         Args:
35             input_schema: List of (name, type) tuples

```

```

36         output_schema: List of (name, type) tuples
37
38     Returns:
39         ModelSignature
40     """
41     input_cols = [ColSpec(dtype, name) for name, dtype in input_schema]
42     output_cols = [ColSpec(dtype, name) for name, dtype in output_schema]
43
44     return ModelSignature(
45         inputs=Schema(input_cols),
46         outputs=Schema(output_cols)
47     )
48
49 @staticmethod
50 def validate_input(input_df: pd.DataFrame,
51                   signature: ModelSignature) -> tuple:
52     """
53     Validate input against signature
54
55     Args:
56         input_df: Input DataFrame
57         signature: Expected signature
58
59     Returns:
60         (is_valid, error_message)
61     """
62     if signature is None or signature.inputs is None:
63         return True, None
64
65     # Check columns
66     expected_cols = set(signature.inputs.input_names())
67     actual_cols = set(input_df.columns)
68
69     missing_cols = expected_cols - actual_cols
70     extra_cols = actual_cols - expected_cols
71
72     if missing_cols:
73         return False, f"Missing columns: {missing_cols}"
74
75     if extra_cols:
76         return False, f"Unexpected columns: {extra_cols}"
77
78     # Check types
79     for col_spec in signature.inputs.inputs:
80         if col_spec.name in input_df.columns:
81             actual_dtype = input_df[col_spec.name].dtype
82             # Type checking logic here
83             pass
84
85     return True, None
86
87 # Example usage
88 def example_signature_validation():
89     """Example of signature validation"""
90
91     # Create sample data
92     X_train = pd.DataFrame({
93         'feature_1': np.random.rand(100),

```

```

94     'feature_2': np.random.rand(100),
95     'feature_3': np.random.randint(0, 10, 100)
96 })
97
98 y_train = np.random.randint(0, 2, 100)
99
100 # Train model
101 model = RandomForestClassifier()
102 model.fit(X_train, y_train)
103
104 # Create signature
105 predictions = model.predict(X_train[:5])
106 signature = infer_signature(X_train, predictions)
107
108 print(f"Signature: {signature}")
109
110 # Validate input
111 validator = ModelSignatureValidator()
112
113 # Valid input
114 valid_input = pd.DataFrame({
115     'feature_1': [0.5],
116     'feature_2': [0.3],
117     'feature_3': [5]
118 })
119
120 is_valid, error = validator.validate_input(valid_input, signature)
121 print(f"Valid input: {is_valid}")
122
123 # Invalid input (missing column)
124 invalid_input = pd.DataFrame({
125     'feature_1': [0.5],
126     'feature_2': [0.3]
127 })
128
129 is_valid, error = validator.validate_input(invalid_input, signature)
130 print(f"Invalid input: {is_valid}, Error: {error}")

```

Listing 13.2: Model Signature and Validation

13.4 Model Lineage and Metadata Tracking

13.4.1 Comprehensive Lineage System

Model lineage tracks the complete history of how a model was created, enabling reproducibility and debugging.

```

1 from typing import Any, Dict, List, Optional, Set
2 from dataclasses import dataclass, field
3 from datetime import datetime
4 import hashlib
5 import json
6 import logging
7 import numpy as np
8 import pandas as pd
9 import mlflow

```

```

10 import mlflow
11
12 @dataclass
13 class DataLineage:
14     """Data lineage information"""
15     dataset_name: str
16     dataset_version: str
17     dataset_hash: str
18     num_samples: int
19     num_features: int
20     feature_names: List[str]
21     data_source: str
22     collection_date: datetime
23     preprocessing_steps: List[str] = field(default_factory=list)
24
25 @dataclass
26 class CodeLineage:
27     """Code lineage information"""
28     git_commit: str
29     git_branch: str
30     git_repo: str
31     code_version: str
32     file_paths: List[str]
33     dependencies: Dict[str, str]
34
35 @dataclass
36 class ModelLineage:
37     """Complete model lineage"""
38     model_id: str
39     model_name: str
40     model_version: str
41     creation_timestamp: datetime
42
43     # Data lineage
44     data_lineage: DataLineage
45
46     # Code lineage
47     code_lineage: CodeLineage
48
49     # Training configuration
50     hyperparameters: Dict[str, Any]
51     training_duration: float
52     hardware: Dict[str, str]
53
54     # Parent models (for transfer learning, ensembles)
55     parent_models: List[str] = field(default_factory=list)
56
57     # Performance
58     metrics: Dict[str, float] = field(default_factory=dict)
59
60     # Additional metadata
61     tags: Dict[str, str] = field(default_factory=dict)
62     notes: str = ""
63
64 class LineageTracker:
65     """
66     Track comprehensive model lineage
67 
```

```

68 Captures data, code, and training provenance
69 """
70
71 def __init__(self, storage_backend: str = "mlflow"):
72     """
73     Initialize lineage tracker
74
75     Args:
76         storage_backend: Where to store lineage (mlflow, database, etc.)
77     """
78     self.storage_backend = storage_backend
79     self.logger = logging.getLogger("LineageTracker")
80
81 def track_data_lineage(self,
82                        dataset: pd.DataFrame,
83                        dataset_name: str,
84                        dataset_version: str,
85                        data_source: str,
86                        preprocessing_steps: List[str] = None) -> DataLineage:
87     """
88     Track data lineage
89
90     Args:
91         dataset: Training dataset
92         dataset_name: Dataset identifier
93         dataset_version: Dataset version
94         data_source: Where data came from
95         preprocessing_steps: List of preprocessing operations
96
97     Returns:
98         DataLineage object
99     """
100     # Calculate dataset hash for verification
101     dataset_hash = hashlib.sha256(
102         pd.util.hash_pandas_object(dataset).values
103     ).hexdigest()
104
105     return DataLineage(
106         dataset_name=dataset_name,
107         dataset_version=dataset_version,
108         dataset_hash=dataset_hash,
109         num_samples=len(dataset),
110         num_features=len(dataset.columns),
111         feature_names=list(dataset.columns),
112         data_source=data_source,
113         collection_date=datetime.now(),
114         preprocessing_steps=preprocessing_steps or []
115     )
116
117 def track_code_lineage(self,
118                       training_script_paths: List[str]) -> CodeLineage:
119     """
120     Track code lineage from git
121
122     Args:
123         training_script_paths: Paths to training scripts
124
125     Returns:

```



```

126         CodeLineage object
127         """
128         import subprocess
129         import pkg_resources
130
131         try:
132             # Get git information
133             git_commit = subprocess.check_output(
134                 ['git', 'rev-parse', 'HEAD']
135             ).decode('utf-8').strip()
136
137             git_branch = subprocess.check_output(
138                 ['git', 'rev-parse', '--abbrev-ref', 'HEAD']
139             ).decode('utf-8').strip()
140
141             git_repo = subprocess.check_output(
142                 ['git', 'config', '--get', 'remote.origin.url']
143             ).decode('utf-8').strip()
144
145         except Exception as e:
146             self.logger.warning(f"Failed to get git info: {str(e)}")
147             git_commit = "unknown"
148             git_branch = "unknown"
149             git_repo = "unknown"
150
151         # Get installed packages
152         dependencies = {
153             pkg.key: pkg.version
154             for pkg in pkg_resources.working_set
155         }
156
157         return CodeLineage(
158             git_commit=git_commit,
159             git_branch=git_branch,
160             git_repo=git_repo,
161             code_version="1.0",
162             file_paths=training_script_paths,
163             dependencies=dependencies
164         )
165
166     def create_lineage(self,
167                       model_id: str,
168                       model_name: str,
169                       model_version: str,
170                       data_lineage: DataLineage,
171                       code_lineage: CodeLineage,
172                       hyperparameters: Dict,
173                       training_duration: float,
174                       metrics: Dict[str, float],
175                       parent_models: List[str] = None) -> ModelLineage:
176         """
177         Create complete model lineage
178
179         Args:
180             model_id: Unique model identifier
181             model_name: Model name
182             model_version: Model version
183             data_lineage: Data lineage

```

```

184         code_lineage: Code lineage
185         hyperparameters: Training hyperparameters
186         training_duration: Training time in seconds
187         metrics: Performance metrics
188         parent_models: Parent model IDs (for transfer learning)
189
190     Returns:
191         Complete ModellLineage
192     """
193     import platform
194
195     lineage = ModellLineage(
196         model_id=model_id,
197         model_name=model_name,
198         model_version=model_version,
199         creation_timestamp=datetime.now(),
200         data_lineage=data_lineage,
201         code_lineage=code_lineage,
202         hyperparameters=hyperparameters,
203         training_duration=training_duration,
204         hardware={
205             'platform': platform.system(),
206             'processor': platform.processor(),
207             'python_version': platform.python_version()
208         },
209         parent_models=parent_models or [],
210         metrics=metrics
211     )
212
213     # Store lineage
214     self._store_lineage(lineage)
215
216     return lineage
217
218     def _store_lineage(self, lineage: ModellLineage):
219         """Store lineage in backend"""
220         if self.storage_backend == "mlflow":
221             self._store_in_mlflow(lineage)
222         else:
223             self._store_in_json(lineage)
224
225     def _store_in_mlflow(self, lineage: ModellLineage):
226         """Store lineage in MLflow"""
227         with mlflow.start_run(run_name=f"{lineage.model_name}_v{lineage.model_version}"):
228
229             # Log data lineage
230             mlflow.log_param("dataset_name", lineage.data_lineage.dataset_name)
231             mlflow.log_param("dataset_version", lineage.data_lineage.dataset_version)
232             mlflow.log_param("dataset_hash", lineage.data_lineage.dataset_hash)
233             mlflow.log_param("num_samples", lineage.data_lineage.num_samples)
234             mlflow.log_param("num_features", lineage.data_lineage.num_features)
235
236             # Log code lineage
237             mlflow.log_param("git_commit", lineage.code_lineage.git_commit)
238             mlflow.log_param("git_branch", lineage.code_lineage.git_branch)
239
240             # Log hyperparameters
241             for key, value in lineage.hyperparameters.items():

```

```

241         mlflow.log_param(key, value)
242
243     # Log metrics
244     for key, value in lineage.metrics.items():
245         mlflow.log_metric(key, value)
246
247     # Log lineage as artifact
248     lineage_dict = self._lineage_to_dict(lineage)
249     with open("lineage.json", "w") as f:
250         json.dump(lineage_dict, f, indent=2, default=str)
251     mlflow.log_artifact("lineage.json")
252
253     def _store_in_json(self, lineage: ModelLineage):
254         """Store lineage as JSON file"""
255         lineage_dict = self._lineage_to_dict(lineage)
256         filename = f"lineage_{lineage.model_id}.json"
257         with open(filename, "w") as f:
258             json.dump(lineage_dict, f, indent=2, default=str)
259
260     def _lineage_to_dict(self, lineage: ModelLineage) -> Dict:
261         """Convert lineage to dictionary"""
262         return {
263             'model_id': lineage.model_id,
264             'model_name': lineage.model_name,
265             'model_version': lineage.model_version,
266             'creation_timestamp': lineage.creation_timestamp.isoformat(),
267             'data_lineage': {
268                 'dataset_name': lineage.data_lineage.dataset_name,
269                 'dataset_version': lineage.data_lineage.dataset_version,
270                 'dataset_hash': lineage.data_lineage.dataset_hash,
271                 'num_samples': lineage.data_lineage.num_samples,
272                 'num_features': lineage.data_lineage.num_features,
273                 'feature_names': lineage.data_lineage.feature_names,
274                 'data_source': lineage.data_lineage.data_source,
275                 'preprocessing_steps': lineage.data_lineage.preprocessing_steps
276             },
277             'code_lineage': {
278                 'git_commit': lineage.code_lineage.git_commit,
279                 'git_branch': lineage.code_lineage.git_branch,
280                 'git_repo': lineage.code_lineage.git_repo,
281                 'code_version': lineage.code_lineage.code_version,
282                 'file_paths': lineage.code_lineage.file_paths
283             },
284             'hyperparameters': lineage.hyperparameters,
285             'training_duration': lineage.training_duration,
286             'hardware': lineage.hardware,
287             'parent_models': lineage.parent_models,
288             'metrics': lineage.metrics,
289             'tags': lineage.tags,
290             'notes': lineage.notes
291         }
292
293     def get_lineage(self, model_id: str) -> Optional[ModelLineage]:
294         """
295         Retrieve model lineage
296
297         Args:
298             model_id: Model identifier

```

```

299         Returns:
300             ModellLineage or None
301         """
302         try:
303             with open(f"lineage_{model_id}.json", "r") as f:
304                 lineage_dict = json.load(f)
305                 # Reconstruct ModellLineage from dict
306                 # (Simplified - full implementation would deserialize properly)
307                 return lineage_dict
308         except FileNotFoundError:
309             return None
310
311 # Example usage
312 def example_lineage_tracking():
313     """Example of comprehensive lineage tracking"""
314
315     # Initialize tracker
316     tracker = LineageTracker(storage_backend="mlflow")
317
318     # Create training dataset
319     X_train = pd.DataFrame({
320         'age': np.random.randint(18, 80, 1000),
321         'income': np.random.randint(20000, 200000, 1000),
322         'credit_score': np.random.randint(300, 850, 1000)
323     })
324
325     # Track data lineage
326     data_lineage = tracker.track_data_lineage(
327         dataset=X_train,
328         dataset_name="customer_features",
329         dataset_version="v2024-01-15",
330         data_source="data_warehouse.customers",
331         preprocessing_steps=[
332             "remove_nulls",
333             "normalize_income",
334             "handle_outliers"
335         ]
336     )
337
338     # Track code lineage
339     code_lineage = tracker.track_code_lineage(
340         training_script_paths=["train.py", "preprocess.py"]
341     )
342
343     # Training configuration
344     hyperparameters = {
345         'n_estimators': 100,
346         'max_depth': 10,
347         'min_samples_split': 5,
348         'learning_rate': 0.1
349     }
350
351     # Create complete lineage
352     lineage = tracker.create_lineage(
353         model_id="model_abc123",
354         model_name="credit_risk_model",
355         model_version="v2.0",

```

```

357     data_lineage=data_lineage,
358     code_lineage=code_lineage,
359     hyperparameters=hyperparameters,
360     training_duration=450.5,
361     metrics={
362         'accuracy': 0.92,
363         'auc': 0.88,
364         'f1_score': 0.90
365     },
366     parent_models=[]
367 )
368
369 print(f"Lineage created for {lineage.model_id}")
370 print(f"Dataset: {lineage.data_lineage.dataset_name}")
371 print(f"Git commit: {lineage.code_lineage.git_commit}")

```

Listing 13.3: Model Lineage Tracker

13.5 Approval Workflows and Governance

13.5.1 Model Governance Framework

Production ML systems require governance to ensure only validated models reach production. Approval workflows enforce reviews, validations, and compliance checks before model promotion.

Key governance requirements:

- **Multi-stage approvals:** Different stakeholders approve at each stage
- **Validation gates:** Automated checks before promotion
- **Role-based access:** Control who can promote models
- **Audit trails:** Complete history of approvals and rejections
- **Compliance integration:** Ticketing systems for regulatory requirements
- **Automated enforcement:** Prevent unauthorized promotions

13.5.2 Complete Governance Implementation

```

1  from enum import Enum
2  from typing import Any, Dict, List, Optional, Callable
3  from dataclasses import dataclass, field
4  from datetime import datetime
5  import logging
6  import json
7
8  class ApprovalStatus(Enum):
9      """Approval request status"""
10     PENDING = "pending"
11     APPROVED = "approved"
12     REJECTED = "rejected"
13     CANCELLED = "cancelled"

```

```

14
15 class UserRole(Enum):
16     """User roles for RBAC"""
17     DATA_SCIENTIST = "data_scientist"
18     ML_ENGINEER = "ml_engineer"
19     TEAM_LEAD = "team_lead"
20     COMPLIANCE_OFFICER = "compliance_officer"
21     PRODUCTION_OWNER = "production_owner"
22
23 @dataclass
24 class ValidationGate:
25     """Validation gate configuration"""
26     name: str
27     description: str
28     validator_func: Callable
29     required: bool = True
30     error_message: str = ""
31
32 @dataclass
33 class ApprovalRequest:
34     """Model approval request"""
35     request_id: str
36     model_name: str
37     current_version: str
38     source_stage: str
39     target_stage: str
40
41     # Requester information
42     requester: str
43     requester_role: UserRole
44     request_timestamp: datetime
45
46     # Approval chain
47     required_approvers: List[UserRole]
48     approvals: List[Dict[str, Any]] = field(default_factory=list)
49     rejections: List[Dict[str, Any]] = field(default_factory=list)
50
51     # Status
52     status: ApprovalStatus = ApprovalStatus.PENDING
53
54     # Validation results
55     validation_results: Dict[str, bool] = field(default_factory=dict)
56
57     # Metadata
58     justification: str = ""
59     impact_assessment: str = ""
60     rollback_plan: str = ""
61     ticket_id: Optional[str] = None
62
63 class ModelGovernanceManager:
64     """
65     Comprehensive model governance and approval workflow manager
66
67     Enforces approval workflows, validation gates, and RBAC
68     """
69
70     def __init__(self,
71                 mlflow_registry: MLflowModelRegistry,

```

```

72         ticket_system: Optional[Any] = None):
73     """
74     Initialize governance manager
75
76     Args:
77         mlflow_registry: MLflow registry instance
78         ticket_system: Optional ticketing system integration
79     """
80     self.registry = mlflow_registry
81     self.ticket_system = ticket_system
82     self.logger = logging.getLogger("ModelGovernance")
83
84     # Validation gates by stage transition
85     self.validation_gates: Dict[str, List[ValidationGate]] = {}
86
87     # Approval requirements by stage
88     self.approval_requirements: Dict[str, List[UserRole]] = {
89         'Staging': [UserRole.TEAM_LEAD],
90         'Production': [UserRole.TEAM_LEAD, UserRole.PRODUCTION_OWNER],
91         'Archived': [UserRole.TEAM_LEAD]
92     }
93
94     # Approval requests
95     self.approval_requests: Dict[str, ApprovalRequest] = {}
96
97     # Audit log
98     self.audit_log: List[Dict] = []
99
100 def register_validation_gate(self,
101                             target_stage: str,
102                             gate: ValidationGate):
103     """
104     Register validation gate for stage transition
105
106     Args:
107         target_stage: Target stage (Staging, Production, etc.)
108         gate: ValidationGate to register
109     """
110     if target_stage not in self.validation_gates:
111         self.validation_gates[target_stage] = []
112
113     self.validation_gates[target_stage].append(gate)
114     self.logger.info(f"Registered validation gate '{gate.name}' for {target_stage}")
115
116 def request_approval(self,
117                     model_name: str,
118                     version: str,
119                     source_stage: str,
120                     target_stage: str,
121                     requester: str,
122                     requester_role: UserRole,
123                     justification: str = "",
124                     impact_assessment: str = "",
125                     rollback_plan: str = "") -> str:
126     """
127     Request approval for model stage transition
128

```

```

129     Args:
130         model_name: Model name
131         version: Model version
132         source_stage: Current stage
133         target_stage: Target stage
134         requester: User requesting approval
135         requester_role: Requester's role
136         justification: Reason for promotion
137         impact_assessment: Expected impact
138         rollback_plan: Plan if rollback needed
139
140     Returns:
141         Request ID
142     """
143     import uuid
144
145     request_id = str(uuid.uuid4())
146
147     # Get required approvers for target stage
148     required_approvers = self.approval_requirements.get(
149         target_stage,
150         [UserRole.TEAM_LEAD]
151     )
152
153     # Create approval request
154     request = ApprovalRequest(
155         request_id=request_id,
156         model_name=model_name,
157         current_version=version,
158         source_stage=source_stage,
159         target_stage=target_stage,
160         requester=requester,
161         requester_role=requester_role,
162         request_timestamp=datetime.now(),
163         required_approvers=required_approvers,
164         justification=justification,
165         impact_assessment=impact_assessment,
166         rollback_plan=rollback_plan
167     )
168
169     # Run validation gates
170     validation_passed = self._run_validation_gates(
171         request,
172         model_name,
173         version
174     )
175
176     if not validation_passed:
177         request.status = ApprovalStatus.REJECTED
178         self.logger.warning(
179             f"Validation failed for {model_name} v{version}"
180         )
181         self.approval_requests[request_id] = request
182         return request_id
183
184     # Create ticket if system available
185     if self.ticket_system:
186         ticket_id = self._create_governance_ticket(request)

```



```

187         request.ticket_id = ticket_id
188
189     self.approval_requests[request_id] = request
190
191     # Log audit event
192     self._log_audit_event(
193         event_type="approval_requested",
194         request_id=request_id,
195         model_name=model_name,
196         version=version,
197         user=requester,
198         details={
199             'source_stage': source_stage,
200             'target_stage': target_stage,
201             'justification': justification
202         }
203     )
204
205     self.logger.info(
206         f"Approval request created: {request_id} for {model_name} v{version}"
207     )
208
209     return request_id
210
211 def approve_request(self,
212                     request_id: str,
213                     approver: str,
214                     approver_role: UserRole,
215                     comments: str = "") -> bool:
216     """
217     Approve a model promotion request
218
219     Args:
220         request_id: Request ID
221         approver: User approving
222         approver_role: Approver's role
223         comments: Approval comments
224
225     Returns:
226         True if approval recorded successfully
227     """
228     request = self.approval_requests.get(request_id)
229     if not request:
230         self.logger.error(f"Request {request_id} not found")
231         return False
232
233     if request.status != ApprovalStatus.PENDING:
234         self.logger.error(
235             f"Request {request_id} is not pending (status: {request.status})"
236         )
237         return False
238
239     # Check if approver role is required
240     if approver_role not in request.required_approvers:
241         self.logger.error(
242             f"{approver_role} not authorized to approve this request"
243         )
244         return False

```

```

245
246     # Record approval
247     approval = {
248         'approver': approver,
249         'role': approver_role.value,
250         'timestamp': datetime.now().isoformat(),
251         'comments': comments
252     }
253     request.approvals.append(approval)
254
255     # Log audit event
256     self._log_audit_event(
257         event_type="approval_granted",
258         request_id=request_id,
259         model_name=request.model_name,
260         version=request.current_version,
261         user=approver,
262         details={'role': approver_role.value, 'comments': comments}
263     )
264
265     # Check if all required approvals received
266     approved_roles = {a['role'] for a in request.approvals}
267     required_roles = {r.value for r in request.required_approvers}
268
269     if required_roles.issubset(approved_roles):
270         # All approvals received - execute promotion
271         success = self._execute_promotion(request)
272         if success:
273             request.status = ApprovalStatus.APPROVED
274             self.logger.info(
275                 f"Request {request_id} fully approved and executed"
276             )
277             return success
278
279     self.logger.info(
280         f"Approval recorded for {request_id}. "
281         f"Pending: {required_roles - approved_roles}"
282     )
283     return True
284
285     def reject_request(self,
286                       request_id: str,
287                       rejector: str,
288                       rejector_role: UserRole,
289                       reason: str) -> bool:
290
291         """
292         Reject a model promotion request
293
294         Args:
295             request_id: Request ID
296             rejector: User rejecting
297             rejector_role: Rejector's role
298             reason: Rejection reason
299
300         Returns:
301             True if rejection recorded successfully
302
303         """
304         request = self.approval_requests.get(request_id)

```

```

303     if not request:
304         self.logger.error(f"Request {request_id} not found")
305         return False
306
307     if request.status != ApprovalStatus.PENDING:
308         self.logger.error(
309             f"Request {request_id} is not pending"
310         )
311         return False
312
313     # Record rejection
314     rejection = {
315         'rejector': rejector,
316         'role': rejector_role.value,
317         'timestamp': datetime.now().isoformat(),
318         'reason': reason
319     }
320     request.rejections.append(rejection)
321     request.status = ApprovalStatus.REJECTED
322
323     # Log audit event
324     self._log_audit_event(
325         event_type="approval_rejected",
326         request_id=request_id,
327         model_name=request.model_name,
328         version=request.current_version,
329         user=rejector,
330         details={'role': rejector_role.value, 'reason': reason}
331     )
332
333     # Update ticket if exists
334     if self.ticket_system and request.ticket_id:
335         self._update_ticket_status(request.ticket_id, "Rejected", reason)
336
337     self.logger.info(f"Request {request_id} rejected by {rejector}")
338     return True
339
340     def _run_validation_gates(self,
341                             request: ApprovalRequest,
342                             model_name: str,
343                             version: str) -> bool:
344
345         """
346         Run validation gates for stage transition
347
348         Args:
349             request: Approval request
350             model_name: Model name
351             version: Model version
352
353         Returns:
354             True if all validations pass
355         """
356         gates = self.validation_gates.get(request.target_stage, [])
357
358         if not gates:
359             self.logger.info(f"No validation gates for {request.target_stage}")
360             return True

```

```

361         all_passed = True
362
363     for gate in gates:
364         try:
365             # Get model version info
366             model_info = self.registry.get_model_version(
367                 model_name,
368                 version
369             )
370
371             # Run validation
372             passed = gate.validator_func(model_info, request)
373             request.validation_results[gate.name] = passed
374
375             if not passed and gate.required:
376                 self.logger.error(
377                     f"Required validation gate '{gate.name}' failed"
378                 )
379                 all_passed = False
380             elif not passed:
381                 self.logger.warning(
382                     f"Optional validation gate '{gate.name}' failed"
383                 )
384             else:
385                 self.logger.info(f"Validation gate '{gate.name}' passed")
386
387         except Exception as e:
388             self.logger.error(
389                 f"Validation gate '{gate.name}' error: {str(e)}"
390             )
391             if gate.required:
392                 all_passed = False
393
394     return all_passed
395
396 def _execute_promotion(self, request: ApprovalRequest) -> bool:
397     """
398     Execute model promotion after approvals
399
400     Args:
401         request: Approved request
402
403     Returns:
404         True if promotion successful
405     """
406     success = self.registry.transition_model_stage(
407         model_name=request.model_name,
408         version=request.current_version,
409         stage=request.target_stage,
410         archive_existing=True
411     )
412
413     if success:
414         # Log audit event
415         self._log_audit_event(
416             event_type="model_promoted",
417             request_id=request.request_id,
418             model_name=request.model_name,

```

```

419         version=request.current_version,
420         user="system",
421         details={
422             'source_stage': request.source_stage,
423             'target_stage': request.target_stage
424         }
425     )
426
427     # Update ticket
428     if self.ticket_system and request.ticket_id:
429         self._update_ticket_status(
430             request.ticket_id,
431             "Completed",
432             "Model promoted successfully"
433         )
434
435     return success
436
437     def _create_governance_ticket(self, request: ApprovalRequest) -> str:
438         """Create governance ticket in external system"""
439         # Simplified - would integrate with JIRA, ServiceNow, etc.
440         ticket = {
441             'title': f"Model Promotion: {request.model_name} v{request.current_version}",
442             'description': f"""
443 Model Promotion Request
444
445 Model: {request.model_name}
446 Version: {request.current_version}
447 Source Stage: {request.source_stage}
448 Target Stage: {request.target_stage}
449
450 Requester: {request.requester}
451 Justification: {request.justification}
452
453 Impact Assessment:
454 {request.impact_assessment}
455
456 Rollback Plan:
457 {request.rollback_plan}
458 """,
459             'type': 'Model Governance',
460             'priority': 'High' if request.target_stage == 'Production' else 'Medium'
461         }
462
463         # In real implementation, call ticket system API
464         ticket_id = f"GOV-{request.request_id[:8]}"
465         self.logger.info(f"Created ticket {ticket_id}")
466         return ticket_id
467
468     def _update_ticket_status(self, ticket_id: str, status: str, comment: str):
469         """Update ticket status"""
470         # In real implementation, call ticket system API
471         self.logger.info(f"Updated ticket {ticket_id}: {status}")
472
473     def _log_audit_event(self,
474                         event_type: str,
475                         request_id: str,

```

```

476         model_name: str,
477         version: str,
478         user: str,
479         details: Dict):
480     """Log audit event"""
481     event = {
482         'timestamp': datetime.now().isoformat(),
483         'event_type': event_type,
484         'request_id': request_id,
485         'model_name': model_name,
486         'version': version,
487         'user': user,
488         'details': details
489     }
490     self.audit_log.append(event)
491
492     def get_audit_trail(self,
493                       model_name: str = None,
494                       start_date: datetime = None,
495                       end_date: datetime = None) -> List[Dict]:
496         """
497         Get audit trail
498
499         Args:
500             model_name: Filter by model name
501             start_date: Filter by start date
502             end_date: Filter by end date
503
504         Returns:
505             List of audit events
506         """
507         filtered_events = self.audit_log
508
509         if model_name:
510             filtered_events = [
511                 e for e in filtered_events
512                 if e['model_name'] == model_name
513             ]
514
515         if start_date:
516             filtered_events = [
517                 e for e in filtered_events
518                 if datetime.fromisoformat(e['timestamp']) >= start_date
519             ]
520
521         if end_date:
522             filtered_events = [
523                 e for e in filtered_events
524                 if datetime.fromisoformat(e['timestamp']) <= end_date
525             ]
526
527         return filtered_events
528
529     def get_request_status(self, request_id: str) -> Optional[Dict]:
530         """
531         Get approval request status
532
533         Args:

```

```

534         request_id: Request ID
535
536     Returns:
537         Request details
538     """
539     request = self.approval_requests.get(request_id)
540     if not request:
541         return None
542
543     return {
544         'request_id': request.request_id,
545         'model_name': request.model_name,
546         'version': request.current_version,
547         'source_stage': request.source_stage,
548         'target_stage': request.target_stage,
549         'status': request.status.value,
550         'requester': request.requester,
551         'request_timestamp': request.request_timestamp.isoformat(),
552         'approvals': request.approvals,
553         'rejections': request.rejections,
554         'validation_results': request.validation_results,
555         'ticket_id': request.ticket_id
556     }
557
558 # Example validation gates
559 def validate_model_performance(model_info: Dict, request: ApprovalRequest) -> bool:
560     """Validate model meets performance thresholds"""
561     metrics = model_info.get('metrics', {})
562
563     # Example thresholds for production
564     if request.target_stage == 'Production':
565         accuracy = metrics.get('accuracy', 0)
566         if accuracy < 0.90:
567             return False
568
569         f1_score = metrics.get('f1_score', 0)
570         if f1_score < 0.85:
571             return False
572
573     return True
574
575 def validate_model_testing(model_info: Dict, request: ApprovalRequest) -> bool:
576     """Validate model has been tested"""
577     tags = model_info.get('tags', {})
578
579     # Check for test completion tag
580     if request.target_stage == 'Production':
581         if tags.get('tested') != 'true':
582             return False
583
584         if tags.get('integration_tested') != 'true':
585             return False
586
587     return True
588
589 def validate_documentation(model_info: Dict, request: ApprovalRequest) -> bool:
590     """Validate model documentation"""
591     description = model_info.get('description', '')

```

```

592
593     if request.target_stage == 'Production':
594         # Require minimum documentation
595         if len(description) < 100:
596             return False
597
598     return True
599
600 # Example usage
601 def example_governance_workflow():
602     """Example of complete governance workflow"""
603
604     # Initialize components
605     registry = MLflowModelRegistry(tracking_uri="http://localhost:5000")
606     governance = ModelGovernanceManager(registry)
607
608     # Register validation gates for Production
609     governance.register_validation_gate(
610         target_stage='Production',
611         gate=ValidationGate(
612             name='performance_check',
613             description='Validate model performance metrics',
614             validator_func=validate_model_performance,
615             required=True
616         )
617     )
618
619     governance.register_validation_gate(
620         target_stage='Production',
621         gate=ValidationGate(
622             name='testing_check',
623             description='Validate testing completion',
624             validator_func=validate_model_testing,
625             required=True
626         )
627     )
628
629     governance.register_validation_gate(
630         target_stage='Production',
631         gate=ValidationGate(
632             name='documentation_check',
633             description='Validate documentation completeness',
634             validator_func=validate_documentation,
635             required=False
636         )
637     )
638
639     # Request approval for production promotion
640     request_id = governance.request_approval(
641         model_name='fraud_detector',
642         version='3',
643         source_stage='Staging',
644         target_stage='Production',
645         requester='alice@company.com',
646         requester_role=UserRole.DATA_SCIENTIST,
647         justification='New model shows 5% improvement in F1 score',
648         impact_assessment='Expected to reduce false positives by 15%',
649         rollback_plan='Immediate rollback to v2 if accuracy drops below 0.88'

```



```

650 )
651
652 print(f"Approval request created: {request_id}")
653
654 # Check request status
655 status = governance.get_request_status(request_id)
656 print(f"Request status: {status['status']}")
657 print(f"Validation results: {status['validation_results']}")
658
659 # Team lead approves
660 governance.approve_request(
661     request_id=request_id,
662     approver='bob@company.com',
663     approver_role=UserRole.TEAM_LEAD,
664     comments='Model performance looks good, approving for production'
665 )
666
667 # Production owner approves (final approval)
668 governance.approve_request(
669     request_id=request_id,
670     approver='charlie@company.com',
671     approver_role=UserRole.PRODUCTION_OWNER,
672     comments='Infrastructure ready, approved for deployment'
673 )
674
675 # Check final status
676 final_status = governance.get_request_status(request_id)
677 print(f"Final status: {final_status['status']}")
678
679 # Get audit trail
680 audit_trail = governance.get_audit_trail(model_name='fraud_detector')
681 print(f"Audit events: {len(audit_trail)}")
682 for event in audit_trail:
683     print(f"    {event['timestamp']}: {event['event_type']} by {event['user']}")

```

Listing 13.4: Model Approval and Governance System

13.6 Model Comparison and Promotion

13.6.1 Comprehensive Model Comparison Framework

Comparing models rigorously before promotion prevents regression. Effective comparison evaluates models across multiple dimensions: performance metrics, statistical significance, computational cost, data requirements, and operational characteristics.

Comparison dimensions:

- **Performance metrics:** Accuracy, precision, recall, F1, AUC-ROC
- **Statistical significance:** Hypothesis tests, confidence intervals
- **Computational cost:** Inference latency, throughput, resource usage
- **Data requirements:** Feature dependencies, data freshness needs
- **Model complexity:** Number of parameters, interpretability
- **Business metrics:** Revenue impact, user satisfaction, operational cost

13.6.2 Model Comparison and Promotion System

```

1 from typing import Dict, List, Optional, Tuple
2 from dataclasses import dataclass
3 from datetime import datetime
4 import numpy as np
5 from scipy import stats
6 import logging
7
8 @dataclass
9 class ComparisonResult:
10     """Results of model comparison"""
11     model1_name: str
12     model1_version: str
13     model2_name: str
14     model2_version: str
15
16     # Performance comparison
17     metric_comparisons: Dict[str, Dict[str, float]]
18
19     # Statistical significance
20     statistically_significant: bool
21     p_values: Dict[str, float]
22     confidence_intervals: Dict[str, Tuple[float, float]]
23
24     # Computational comparison
25     latency_comparison: Dict[str, float]
26     throughput_comparison: Dict[str, float]
27     resource_comparison: Dict[str, float]
28
29     # Overall recommendation
30     recommended_model: str
31     recommendation_reason: str
32     confidence_score: float
33
34 @dataclass
35 class PromotionCriteria:
36     """Criteria for automatic model promotion"""
37     # Performance requirements
38     min_accuracy_improvement: float = 0.01 # 1% minimum improvement
39     min_f1_improvement: float = 0.01
40     require_statistical_significance: bool = True
41     min_confidence_level: float = 0.95
42
43     # Resource constraints
44     max_latency_ms: Optional[float] = None
45     max_memory_mb: Optional[float] = None
46     min_throughput_qps: Optional[float] = None
47
48     # Business requirements
49     max_false_positive_rate: Optional[float] = None
50     max_false_negative_rate: Optional[float] = None
51
52     # A/B test requirements
53     min_sample_size: int = 1000
54     min_test_duration_days: int = 7
55
56 class ModelComparator:

```

```

57 """
58 Compare models across multiple dimensions
59
60 Performs statistical testing and comprehensive evaluation
61 """
62
63 def __init__(self, mlflow_registry: MLflowModelRegistry):
64     """
65     Initialize model comparator
66
67     Args:
68         mlflow_registry: MLflow registry instance
69     """
70     self.registry = mlflow_registry
71     self.logger = logging.getLogger("ModelComparator")
72
73 def compare_models(self,
74                    model_name: str,
75                    version1: str,
76                    version2: str,
77                    test_data: pd.DataFrame = None,
78                    test_labels: np.ndarray = None,
79                    production_metrics: Dict = None) -> ComparisonResult:
80     """
81     Comprehensive model comparison
82
83     Args:
84         model_name: Model name
85         version1: First version
86         version2: Second version
87         test_data: Test dataset for evaluation
88         test_labels: True labels for test data
89         production_metrics: Optional production performance data
90
91     Returns:
92         ComparisonResult
93     """
94     self.logger.info(f"Comparing {model_name} v{version1} vs v{version2}")
95
96     # Get model information
97     model1_info = self.registry.get_model_version(model_name, version1)
98     model2_info = self.registry.get_model_version(model_name, version2)
99
100    # Compare metrics
101    metric_comparisons = self._compare_metrics(
102        model1_info['metrics'],
103        model2_info['metrics']
104    )
105
106    # Statistical significance testing
107    p_values = {}
108    confidence_intervals = {}
109    statistically_significant = True
110
111    if test_data is not None and test_labels is not None:
112        # Load models and evaluate
113        model1 = self._load_model(model_name, version1)
114        model2 = self._load_model(model_name, version2)

```

```

115         pred1 = model1.predict_proba(test_data)[: , 1]
116         pred2 = model2.predict_proba(test_data)[: , 1]
117
118         # Perform statistical tests
119         stat_results = self._perform_statistical_tests(
120             pred1, pred2, test_labels
121         )
122         p_values = stat_results['p_values']
123         confidence_intervals = stat_results['confidence_intervals']
124         statistically_significant = stat_results['significant']
125
126         # Compare computational performance
127         latency_comparison = self._compare_latency(
128             model1_info, model2_info
129         )
130
131         throughput_comparison = self._compare_throughput(
132             model1_info, model2_info
133         )
134
135         resource_comparison = self._compare_resources(
136             model1_info, model2_info
137         )
138
139         # Make recommendation
140         recommended_model, reason, confidence = self._make_recommendation(
141             version1, version2,
142             metric_comparisons,
143             statistically_significant,
144             latency_comparison,
145             resource_comparison
146         )
147
148         return ComparisonResult(
149             model1_name=model_name,
150             model1_version=version1,
151             model2_name=model_name,
152             model2_version=version2,
153             metric_comparisons=metric_comparisons,
154             statistically_significant=statistically_significant,
155             p_values=p_values,
156             confidence_intervals=confidence_intervals,
157             latency_comparison=latency_comparison,
158             throughput_comparison=throughput_comparison,
159             resource_comparison=resource_comparison,
160             recommended_model=recommended_model,
161             recommendation_reason=reason,
162             confidence_score=confidence
163         )
164
165     def _compare_metrics(self,
166                         metrics1: Dict[str, float],
167                         metrics2: Dict[str, float]) -> Dict[str, Dict[str, float]]:
168         """Compare performance metrics"""
169         comparisons = {}
170
171         all_metrics = set(metrics1.keys()) | set(metrics2.keys())

```

```

173
174     for metric in all_metrics:
175         m1 = metrics1.get(metric, 0)
176         m2 = metrics2.get(metric, 0)
177
178         improvement = ((m2 - m1) / abs(m1) * 100) if m1 != 0 else 0
179
180         comparisons[metric] = {
181             'model1': m1,
182             'model2': m2,
183             'improvement_percent': improvement,
184             'absolute_difference': m2 - m1
185         }
186
187     return comparisons
188
189 def _perform_statistical_tests(self,
190                               pred1: np.ndarray,
191                               pred2: np.ndarray,
192                               labels: np.ndarray) -> Dict:
193     """Perform statistical significance tests"""
194
195     # Calculate AUC for both models
196     from sklearn.metrics import roc_auc_score
197
198     auc1 = roc_auc_score(labels, pred1)
199     auc2 = roc_auc_score(labels, pred2)
200
201     # McNemar's test for binary predictions
202     binary_pred1 = (pred1 > 0.5).astype(int)
203     binary_pred2 = (pred2 > 0.5).astype(int)
204
205     # Create contingency table
206     both_correct = np.sum((binary_pred1 == labels) & (binary_pred2 == labels))
207     only_1_correct = np.sum((binary_pred1 == labels) & (binary_pred2 != labels))
208     only_2_correct = np.sum((binary_pred1 != labels) & (binary_pred2 == labels))
209     both_wrong = np.sum((binary_pred1 != labels) & (binary_pred2 != labels))
210
211     # McNemar's test
212     if only_1_correct + only_2_correct > 0:
213         mcnemar_stat = ((abs(only_1_correct - only_2_correct) - 1) ** 2) / \
214             (only_1_correct + only_2_correct)
215         mcnemar_p = 1 - stats.chi2.cdf(mcnemar_stat, 1)
216     else:
217         mcnemar_p = 1.0
218
219     # Bootstrap confidence interval for AUC difference
220     n_bootstrap = 1000
221     auc_diffs = []
222
223     for _ in range(n_bootstrap):
224         indices = np.random.choice(len(labels), len(labels), replace=True)
225         boot_auc1 = roc_auc_score(labels[indices], pred1[indices])
226         boot_auc2 = roc_auc_score(labels[indices], pred2[indices])
227         auc_diffs.append(boot_auc2 - boot_auc1)
228
229     auc_diffs = np.array(auc_diffs)
230     ci_lower = np.percentile(auc_diffs, 2.5)

```

```

231     ci_upper = np.percentile(auc_diffs, 97.5)
232
233     # Determine if statistically significant
234     significant = mcnemar_p < 0.05 and ci_lower > 0
235
236     return {
237         'p_values': {
238             'mcnemar': mcnemar_p
239         },
240         'confidence_intervals': {
241             'auc_difference': (ci_lower, ci_upper)
242         },
243         'significant': significant
244     }
245
246     def _compare_latency(self,
247                         model1_info: Dict,
248                         model2_info: Dict) -> Dict[str, float]:
249         """Compare inference latency"""
250         # In practice, would get from production metrics or benchmarks
251         latency1 = model1_info.get('params', {}).get('avg_latency_ms', 0)
252         latency2 = model2_info.get('params', {}).get('avg_latency_ms', 0)
253
254         return {
255             'model1_ms': float(latency1) if latency1 else 0,
256             'model2_ms': float(latency2) if latency2 else 0,
257             'difference_ms': float(latency2) - float(latency1) if latency1 and
latency2 else 0
258         }
259
260     def _compare_throughput(self,
261                           model1_info: Dict,
262                           model2_info: Dict) -> Dict[str, float]:
263         """Compare throughput (queries per second)"""
264         throughput1 = model1_info.get('params', {}).get('throughput_qps', 0)
265         throughput2 = model2_info.get('params', {}).get('throughput_qps', 0)
266
267         return {
268             'model1_qps': float(throughput1) if throughput1 else 0,
269             'model2_qps': float(throughput2) if throughput2 else 0,
270             'difference_qps': float(throughput2) - float(throughput1) if throughput1
and throughput2 else 0
271         }
272
273     def _compare_resources(self,
274                          model1_info: Dict,
275                          model2_info: Dict) -> Dict[str, float]:
276         """Compare resource usage"""
277         memory1 = model1_info.get('params', {}).get('memory_mb', 0)
278         memory2 = model2_info.get('params', {}).get('memory_mb', 0)
279
280         return {
281             'model1_memory_mb': float(memory1) if memory1 else 0,
282             'model2_memory_mb': float(memory2) if memory2 else 0,
283             'difference_mb': float(memory2) - float(memory1) if memory1 and memory2
else 0
284         }
285

```

```

286     def _make_recommendation(self,
287                             version1: str,
288                             version2: str,
289                             metric_comparisons: Dict,
290                             statistically_significant: bool,
291                             latency_comparison: Dict,
292                             resource_comparison: Dict) -> Tuple[str, str, float]:
293         """Make promotion recommendation"""
294
295         reasons = []
296         version2_score = 0
297         version1_score = 0
298
299         # Evaluate performance improvements
300         for metric, comparison in metric_comparisons.items():
301             improvement = comparison['improvement_percent']
302             if improvement > 1: # > 1% improvement
303                 version2_score += 1
304                 reasons.append(f"{metric} improved by {improvement:.2f}%")
305             elif improvement < -1: # > 1% degradation
306                 version1_score += 1
307                 reasons.append(f"{metric} degraded by {abs(improvement):.2f}%")
308
309         # Consider statistical significance
310         if not statistically_significant:
311             reasons.append("Improvement not statistically significant")
312             version2_score -= 2
313
314         # Consider latency
315         latency_diff = latency_comparison.get('difference_ms', 0)
316         if latency_diff > 10: # 10ms degradation
317             version1_score += 1
318             reasons.append(f"Latency increased by {latency_diff:.1f}ms")
319         elif latency_diff < -10: # 10ms improvement
320             version2_score += 1
321             reasons.append(f"Latency reduced by {abs(latency_diff):.1f}ms")
322
323         # Make decision
324         if version2_score > version1_score:
325             recommended = version2
326             confidence = min(version2_score / (version2_score + version1_score + 1),
0.99)
327         elif version1_score > version2_score:
328             recommended = version1
329             confidence = min(version1_score / (version2_score + version1_score + 1),
0.99)
330         else:
331             recommended = version1 # Keep current if tie
332             confidence = 0.5
333             reasons.append("Models perform similarly - keeping current version")
334
335         reason = "; ".join(reasons)
336         return recommended, reason, confidence
337
338     def _load_model(self, model_name: str, version: str):
339         """Load model from registry"""
340         model_uri = f"models://{model_name}/{version}"
341         return mlflow.sklearn.load_model(model_uri)

```

```

342
343 class PromotionManager:
344     """
345     Automated model promotion based on criteria
346
347     Evaluates models against criteria and promotes automatically
348     """
349
350     def __init__(self,
351                  comparator: ModelComparator,
352                  governance: ModelGovernanceManager):
353         """
354         Initialize promotion manager
355
356         Args:
357             comparator: ModelComparator instance
358             governance: ModelGovernanceManager instance
359         """
360         self.comparator = comparator
361         self.governance = governance
362         self.logger = logging.getLogger("PromotionManager")
363
364     def evaluate_for_promotion(self,
365                              model_name: str,
366                              candidate_version: str,
367                              current_version: str,
368                              criteria: PromotionCriteria,
369                              test_data: pd.DataFrame = None,
370                              test_labels: np.ndarray = None) -> Dict:
371         """
372         Evaluate if candidate model should be promoted
373
374         Args:
375             model_name: Model name
376             candidate_version: Candidate version
377             current_version: Current production version
378             criteria: Promotion criteria
379             test_data: Test dataset
380             test_labels: Test labels
381
382         Returns:
383             Evaluation results with promotion decision
384         """
385         self.logger.info(
386             f"Evaluating {model_name} v{candidate_version} for promotion"
387         )
388
389         # Compare models
390         comparison = self.comparator.compare_models(
391             model_name=model_name,
392             version1=current_version,
393             version2=candidate_version,
394             test_data=test_data,
395             test_labels=test_labels
396         )
397
398         # Evaluate against criteria
399         checks = {}

```



```

400     all_passed = True
401
402     # Check accuracy improvement
403     accuracy_comp = comparison.metric_comparisons.get('accuracy', {})
404     accuracy_improvement = accuracy_comp.get('improvement_percent', 0) / 100
405
406     checks['accuracy_improvement'] = accuracy_improvement >= criteria.
min_accuracy_improvement
407     if not checks['accuracy_improvement']:
408         all_passed = False
409         self.logger.warning(
410             f"Accuracy improvement {accuracy_improvement:.3f} below threshold "
411             f"{criteria.min_accuracy_improvement}"
412         )
413
414     # Check F1 improvement
415     f1_comp = comparison.metric_comparisons.get('f1_score', {})
416     f1_improvement = f1_comp.get('improvement_percent', 0) / 100
417
418     checks['f1_improvement'] = f1_improvement >= criteria.min_f1_improvement
419     if not checks['f1_improvement']:
420         all_passed = False
421
422     # Check statistical significance
423     if criteria.require_statistical_significance:
424         checks['statistical_significance'] = comparison.statistically_significant
425         if not checks['statistical_significance']:
426             all_passed = False
427             self.logger.warning("Improvement not statistically significant")
428
429     # Check latency constraint
430     if criteria.max_latency_ms:
431         candidate_latency = comparison.latency_comparison.get('model2_ms', 0)
432         checks['latency_constraint'] = candidate_latency <= criteria.
max_latency_ms
433         if not checks['latency_constraint']:
434             all_passed = False
435
436     # Check resource constraints
437     if criteria.max_memory_mb:
438         candidate_memory = comparison.resource_comparison.get('model2_memory_mb',
0)
439         checks['memory_constraint'] = candidate_memory <= criteria.max_memory_mb
440         if not checks['memory_constraint']:
441             all_passed = False
442
443     return {
444         'should_promote': all_passed,
445         'comparison': comparison,
446         'criteria_checks': checks,
447         'recommendation': comparison.recommendation_reason
448     }
449
450     def auto_promote(self,
451                     model_name: str,
452                     candidate_version: str,
453                     current_version: str,
454                     criteria: PromotionCriteria,

```

```

455         requester: str,
456         test_data: pd.DataFrame = None,
457         test_labels: np.ndarray = None) -> Optional[str]:
458     """
459     Automatically promote model if criteria met
460
461     Args:
462         model_name: Model name
463         candidate_version: Candidate version
464         current_version: Current version
465         criteria: Promotion criteria
466         requester: User requesting promotion
467         test_data: Test data
468         test_labels: Test labels
469
470     Returns:
471         Approval request ID if promotion initiated, None otherwise
472     """
473     # Evaluate for promotion
474     evaluation = self.evaluate_for_promotion(
475         model_name=model_name,
476         candidate_version=candidate_version,
477         current_version=current_version,
478         criteria=criteria,
479         test_data=test_data,
480         test_labels=test_labels
481     )
482
483     if not evaluation['should_promote']:
484         self.logger.info(
485             f"Model {model_name} v{candidate_version} does not meet promotion
486             criteria"
487         )
488         return None
489
490     # Create approval request
491     comparison = evaluation['comparison']
492
493     request_id = self.governance.request_approval(
494         model_name=model_name,
495         version=candidate_version,
496         source_stage='Staging',
497         target_stage='Production',
498         requester=requester,
499         requester_role=UserRole.ML_ENGINEER,
500         justification=f"Automated promotion: {comparison.recommendation_reason}",
501         impact_assessment=f"Performance improvements: {comparison.
502         metric_comparisons}",
503         rollback_plan=f"Rollback to v{current_version} if issues detected"
504     )
505
506     self.logger.info(
507         f"Created promotion request {request_id} for {model_name} v{
508         candidate_version}"
509     )
510
511     return request_id

```

```

510 # Example usage
511 def example_model_comparison_and_promotion():
512     """Example of model comparison and promotion"""
513
514     # Initialize components
515     registry = MLflowModelRegistry(tracking_uri="http://localhost:5000")
516     comparator = ModelComparator(registry)
517     governance = ModelGovernanceManager(registry)
518     promotion_manager = PromotionManager(comparator, governance)
519
520     # Create test data
521     from sklearn.datasets import make_classification
522     X_test, y_test = make_classification(
523         n_samples=1000, n_features=20, random_state=42
524     )
525
526     # Compare models
527     comparison = comparator.compare_models(
528         model_name='fraud_detector',
529         version1='2', # Current production
530         version2='3', # Candidate
531         test_data=pd.DataFrame(X_test),
532         test_labels=y_test
533     )
534
535     print(f"Comparison Results:")
536     print(f" Recommended: {comparison.recommended_model}")
537     print(f" Reason: {comparison.recommendation_reason}")
538     print(f" Confidence: {comparison.confidence_score:.2f}")
539     print(f" Statistically significant: {comparison.statistically_significant}")
540
541     # Define promotion criteria
542     criteria = PromotionCriteria(
543         min_accuracy_improvement=0.01, # 1%
544         min_f1_improvement=0.01,
545         require_statistical_significance=True,
546         max_latency_ms=100,
547         max_memory_mb=2000
548     )
549
550     # Evaluate for promotion
551     evaluation = promotion_manager.evaluate_for_promotion(
552         model_name='fraud_detector',
553         candidate_version='3',
554         current_version='2',
555         criteria=criteria,
556         test_data=pd.DataFrame(X_test),
557         test_labels=y_test
558     )
559
560     print(f"\nPromotion Evaluation:")
561     print(f" Should promote: {evaluation['should_promote']}")
562     print(f" Criteria checks: {evaluation['criteria_checks']}")
563
564     # Auto-promote if criteria met
565     if evaluation['should_promote']:
566         request_id = promotion_manager.auto_promote(
567             model_name='fraud_detector',

```

```

568         candidate_version='3',
569         current_version='2',
570         criteria=criteria,
571         requester='ml-pipeline@company.com',
572         test_data=pd.DataFrame(X_test),
573         test_labels=y_test
574     )
575     print(f"\nPromotion request created: {request_id}")

```

Listing 13.5: Model Comparison and Promotion System

13.7 CI/CD Pipeline Integration

13.7.1 Automated Model Lifecycle with CI/CD

Integrating model registry with CI/CD pipelines automates model training, testing, registration, and deployment. This reduces manual errors, ensures reproducibility, and accelerates delivery.

CI/CD integration benefits:

- **Automated testing:** Run validation tests on every model version
- **Consistent deployment:** Standardized promotion workflows
- **Version control:** Automatic model versioning and tagging
- **Audit trail:** Complete history of model changes
- **Rollback capability:** Quick reversion to previous versions
- **Infrastructure as code:** Declarative model deployment

13.7.2 CI/CD Pipeline Implementation

```

1  from typing import Dict, List, Optional
2  from dataclasses import dataclass
3  import subprocess
4  import yaml
5  import json
6  import os
7  from pathlib import Path
8
9  @dataclass
10 class PipelineConfig:
11     """CI/CD pipeline configuration"""
12     # Repository
13     git_repo: str
14     git_branch: str
15
16     # Training
17     training_script: str
18     training_data_path: str
19     hyperparameters_file: str
20

```

```

21     # Testing
22     test_script: str
23     test_data_path: str
24     performance_thresholds: Dict[str, float]
25
26     # Model registration
27     model_name: str
28     registry_uri: str
29
30     # Deployment
31     deployment_target: str # staging, production
32     kubernetes_namespace: str
33     docker_image_registry: str
34
35 class MLPipeline:
36     """
37     ML CI/CD pipeline orchestrator
38
39     Automates model training, testing, registration, and deployment
40     """
41
42     def __init__(self,
43                 config: PipelineConfig,
44                 registry: MLflowModelRegistry,
45                 governance: ModelGovernanceManager):
46         """
47         Initialize ML pipeline
48
49         Args:
50             config: Pipeline configuration
51             registry: MLflow registry
52             governance: Governance manager
53         """
54         self.config = config
55         self.registry = registry
56         self.governance = governance
57         self.logger = logging.getLogger("MLPipeline")
58
59     def run_training_pipeline(self) -> Dict:
60         """
61         Run complete training pipeline
62
63         Returns:
64             Pipeline results
65         """
66         self.logger.info("Starting training pipeline")
67
68         results = {
69             'git_commit': None,
70             'run_id': None,
71             'model_version': None,
72             'metrics': {},
73             'tests_passed': False,
74             'registered': False
75         }
76
77         try:
78             # Step 1: Get git commit info

```

```

79         results['git_commit'] = self._get_git_commit()
80
81         # Step 2: Train model
82         run_id, metrics = self._train_model()
83         results['run_id'] = run_id
84         results['metrics'] = metrics
85
86         # Step 3: Run model tests
87         test_results = self._run_model_tests(run_id)
88         results['tests_passed'] = test_results['passed']
89
90         if not test_results['passed']:
91             self.logger.error("Model tests failed")
92             return results
93
94         # Step 4: Register model
95         model_version = self._register_model(run_id, metrics)
96         results['model_version'] = model_version
97         results['registered'] = True
98
99         # Step 5: Request approval for staging
100        self._request_staging_approval(model_version)
101
102        self.logger.info(
103            f"Training pipeline completed successfully. Model v{model_version}"
104        )
105
106    except Exception as e:
107        self.logger.error(f"Training pipeline failed: {str(e)}")
108        raise
109
110    return results
111
112    def run_deployment_pipeline(self,
113                               model_version: str,
114                               target_stage: str) -> bool:
115        """
116        Run deployment pipeline
117
118        Args:
119            model_version: Model version to deploy
120            target_stage: Target stage (Staging, Production)
121
122        Returns:
123            True if deployment successful
124        """
125        self.logger.info(
126            f"Starting deployment pipeline for v{model_version} to {target_stage}"
127        )
128
129        try:
130            # Step 1: Build Docker image
131            image_tag = self._build_docker_image(model_version)
132
133            # Step 2: Push to registry
134            self._push_docker_image(image_tag)
135
136            # Step 3: Deploy to Kubernetes

```

```

137         self._deploy_to_kubernetes(image_tag, target_stage)
138
139         # Step 4: Run smoke tests
140         smoke_passed = self._run_smoke_tests(target_stage)
141
142         if not smoke_passed:
143             self.logger.error("Smoke tests failed, rolling back")
144             self._rollback_deployment(target_stage)
145             return False
146
147         self.logger.info(f"Deployment to {target_stage} successful")
148         return True
149
150     except Exception as e:
151         self.logger.error(f"Deployment failed: {str(e)}")
152         self._rollback_deployment(target_stage)
153         return False
154
155     def _get_git_commit(self) -> str:
156         """Get current git commit hash"""
157         result = subprocess.run(
158             ['git', 'rev-parse', 'HEAD'],
159             capture_output=True,
160             text=True
161         )
162         return result.stdout.strip()
163
164     def _train_model(self) -> tuple:
165         """Execute training script"""
166         self.logger.info("Training model")
167
168         # Load hyperparameters
169         with open(self.config.hyperparameters_file, 'r') as f:
170             hyperparameters = yaml.safe_load(f)
171
172         # Run training script
173         env = os.environ.copy()
174         env['MLFLOW_TRACKING_URI'] = self.config.registry_uri
175
176         result = subprocess.run(
177             ['python', self.config.training_script,
178              '--data', self.config.training_data_path,
179              '--params', self.config.hyperparameters_file],
180             env=env,
181             capture_output=True,
182             text=True
183         )
184
185         if result.returncode != 0:
186             raise Exception(f"Training failed: {result.stderr}")
187
188         # Parse output to get run_id and metrics
189         output = json.loads(result.stdout)
190         return output['run_id'], output['metrics']
191
192     def _run_model_tests(self, run_id: str) -> Dict:
193         """Run model validation tests"""
194         self.logger.info("Running model tests")

```

```

195     result = subprocess.run(
196         ['python', self.config.test_script,
197         '--run-id', run_id,
198         '--test-data', self.config.test_data_path],
199         capture_output=True,
200         text=True
201     )
202
203
204     if result.returncode != 0:
205         return {'passed': False, 'error': result.stderr}
206
207     test_results = json.loads(result.stdout)
208
209     # Check against thresholds
210     passed = True
211     for metric, threshold in self.config.performance_thresholds.items():
212         actual = test_results['metrics'].get(metric, 0)
213         if actual < threshold:
214             self.logger.warning(
215                 f"{metric}={actual:.3f} below threshold {threshold}"
216             )
217             passed = False
218
219     return {'passed': passed, 'results': test_results}
220
221 def _register_model(self, run_id: str, metrics: Dict) -> str:
222     """Register model in MLflow"""
223     self.logger.info("Registering model")
224
225     # Load model from run
226     model_uri = f"runs:{run_id}/model"
227
228     # Create metadata
229     metadata = ModelMetadata(
230         name=self.config.model_name,
231         version="auto",
232         stage="None",
233         run_id=run_id,
234         experiment_id="0",
235         metrics=metrics,
236         training_date=datetime.now(),
237         training_duration_seconds=0, # Would be tracked
238         dataset_version="auto",
239         dataset_size=0,
240         model_type="auto",
241         framework="sklearn",
242         framework_version="1.2.0",
243         python_version="3.9",
244         dependencies=[],
245         tags={'ci_cd': 'true', 'automated': 'true'},
246         description=f"Automated build from commit {self._get_git_commit()[:8]}"
247     )
248
249     # Register
250     version = self.registry.register_model(
251         model=None, # Already logged in training
252         model_name=self.config.model_name,

```



```

253         run_id=run_id,
254         metadata=metadata
255     )
256
257     return version
258
259     def _request_staging_approval(self, model_version: str):
260         """Request approval for staging deployment"""
261         self.governance.request_approval(
262             model_name=self.config.model_name,
263             version=model_version,
264             source_stage='None',
265             target_stage='Staging',
266             requester='ci-cd-pipeline',
267             requester_role=UserRole.ML_ENGINEER,
268             justification='Automated build passed all tests',
269             impact_assessment='New model version for staging validation',
270             rollback_plan='Automated rollback on failure'
271         )
272
273     def _build_docker_image(self, model_version: str) -> str:
274         """Build Docker image with model"""
275         image_tag = f"{self.config.docker_image_registry}/{self.config.model_name}:{model_version}"
276
277         self.logger.info(f"Building Docker image: {image_tag}")
278
279         # Create Dockerfile
280         dockerfile_content = f"""
281 FROM python:3.9-slim
282
283 # Install dependencies
284 COPY requirements.txt .
285 RUN pip install -r requirements.txt
286
287 # Copy model serving code
288 COPY serve.py .
289
290 # Set MLflow environment
291 ENV MLFLOW_TRACKING_URI={self.config.registry_uri}
292 ENV MODEL_NAME={self.config.model_name}
293 ENV MODEL_VERSION={model_version}
294
295 # Expose port
296 EXPOSE 8000
297
298 # Run server
299 CMD ["python", "serve.py"]
300 """
301
302         with open('Dockerfile', 'w') as f:
303             f.write(dockerfile_content)
304
305         # Build image
306         result = subprocess.run(
307             ['docker', 'build', '-t', image_tag, '.'],
308             capture_output=True,
309             text=True

```

```

310     )
311
312     if result.returncode != 0:
313         raise Exception(f"Docker build failed: {result.stderr}")
314
315     return image_tag
316
317 def _push_docker_image(self, image_tag: str):
318     """Push Docker image to registry"""
319     self.logger.info(f"Pushing image: {image_tag}")
320
321     result = subprocess.run(
322         ['docker', 'push', image_tag],
323         capture_output=True,
324         text=True
325     )
326
327     if result.returncode != 0:
328         raise Exception(f"Docker push failed: {result.stderr}")
329
330 def _deploy_to_kubernetes(self, image_tag: str, target_stage: str):
331     """Deploy to Kubernetes"""
332     self.logger.info(f"Deploying to Kubernetes: {target_stage}")
333
334     # Create Kubernetes deployment manifest
335     deployment = {
336         'apiVersion': 'apps/v1',
337         'kind': 'Deployment',
338         'metadata': {
339             'name': f"{self.config.model_name}-{target_stage.lower()}",
340             'namespace': self.config.kubernetes_namespace
341         },
342         'spec': {
343             'replicas': 3 if target_stage == 'Production' else 1,
344             'selector': {
345                 'matchLabels': {
346                     'app': self.config.model_name,
347                     'stage': target_stage.lower()
348                 }
349             },
350             'template': {
351                 'metadata': {
352                     'labels': {
353                         'app': self.config.model_name,
354                         'stage': target_stage.lower()
355                     }
356                 },
357                 'spec': {
358                     'containers': [{
359                         'name': 'model-server',
360                         'image': image_tag,
361                         'ports': [{ 'containerPort': 8000 }],
362                         'resources': {
363                             'requests': {
364                                 'memory': '1Gi',
365                                 'cpu': '500m'
366                             },
367                             'limits': {

```

```

368         'memory': '2Gi',
369         'cpu': '1000m'
370     }
371 }
372 }]]
373 }
374 }
375 }
376 }
377
378 # Write manifest
379 with open('deployment.yaml', 'w') as f:
380     yaml.dump(deployment, f)
381
382 # Apply to cluster
383 result = subprocess.run(
384     ['kubectl', 'apply', '-f', 'deployment.yaml'],
385     capture_output=True,
386     text=True
387 )
388
389 if result.returncode != 0:
390     raise Exception(f"Kubernetes deployment failed: {result.stderr}")
391
392 def _run_smoke_tests(self, target_stage: str) -> bool:
393     """Run smoke tests on deployed model"""
394     self.logger.info("Running smoke tests")
395
396     # Would make test requests to deployed endpoint
397     # Simplified here
398     return True
399
400 def _rollback_deployment(self, target_stage: str):
401     """Rollback failed deployment"""
402     self.logger.info(f"Rolling back {target_stage} deployment")
403
404     # Rollback to previous version
405     result = subprocess.run(
406         ['kubectl', 'rollout', 'undo',
407          f'deployment/{self.config.model_name}-{target_stage.lower()}',
408          '-n', self.config.kubernetes_namespace],
409         capture_output=True,
410         text=True
411     )
412
413     if result.returncode != 0:
414         self.logger.error(f"Rollback failed: {result.stderr}")
415
416 def generate_github_actions_workflow(self) -> str:
417     """Generate GitHub Actions workflow YAML"""
418     workflow = {
419         'name': 'ML Model CI/CD',
420         'on': {
421             'push': {
422                 'branches': ['main', 'develop']
423             },
424             'pull_request': {
425                 'branches': ['main']

```

```

426     }
427 },
428 'jobs': {
429     'train-and-test': {
430         'runs-on': 'ubuntu-latest',
431         'steps': [
432             {'uses': 'actions/checkout@v2'},
433             {
434                 'name': 'Set up Python',
435                 'uses': 'actions/setup-python@v2',
436                 'with': {'python-version': '3.9'}
437             },
438             {
439                 'name': 'Install dependencies',
440                 'run': 'pip install -r requirements.txt'
441             },
442             {
443                 'name': 'Train model',
444                 'env': {
445                     'MLFLOW_TRACKING_URI': '${ secrets.
MLFLOW_TRACKING_URI }'
446                 },
447                 'run': f'python {self.config.training_script}'
448             },
449             {
450                 'name': 'Run tests',
451                 'run': f'python {self.config.test_script}'
452             },
453             {
454                 'name': 'Register model',
455                 'if': "github.ref == 'refs/heads/main'",
456                 'run': 'python scripts/register_model.py'
457             }
458         ]
459     },
460     'deploy-staging': {
461         'needs': 'train-and-test',
462         'if': "github.ref == 'refs/heads/develop'",
463         'runs-on': 'ubuntu-latest',
464         'steps': [
465             {
466                 'name': 'Deploy to Staging',
467                 'run': 'python scripts/deploy.py --stage staging'
468             }
469         ]
470     },
471     'deploy-production': {
472         'needs': 'train-and-test',
473         'if': "github.ref == 'refs/heads/main'",
474         'runs-on': 'ubuntu-latest',
475         'steps': [
476             {
477                 'name': 'Deploy to Production',
478                 'run': 'python scripts/deploy.py --stage production'
479             }
480         ]
481     }
482 }

```

```

483     }
484
485     return yaml.dump(workflow, default_flow_style=False)
486
487 # Example usage
488 def example_cicd_pipeline():
489     """Example of CI/CD pipeline integration"""
490
491     # Configuration
492     config = PipelineConfig(
493         git_repo='https://github.com/company/ml-models',
494         git_branch='main',
495         training_script='train.py',
496         training_data_path='data/train.csv',
497         hyperparameters_file='config/hyperparameters.yaml',
498         test_script='test_model.py',
499         test_data_path='data/test.csv',
500         performance_thresholds={
501             'accuracy': 0.90,
502             'f1_score': 0.85,
503             'auc': 0.88
504         },
505         model_name='fraud_detector',
506         registry_uri='http://mlflow.company.com',
507         deployment_target='production',
508         kubernetes_namespace='ml-models',
509         docker_image_registry='gcr.io/company'
510     )
511
512     # Initialize pipeline
513     registry = MLflowModelRegistry(tracking_uri=config.registry_uri)
514     governance = ModelGovernanceManager(registry)
515     pipeline = MLPipeline(config, registry, governance)
516
517     # Run training pipeline
518     results = pipeline.run_training_pipeline()
519
520     if results['registered']:
521         print(f"Model registered: v{results['model_version']}")
522         print(f"Metrics: {results['metrics']}")
523
524     # Deploy to staging
525     if results['tests_passed']:
526         success = pipeline.run_deployment_pipeline(
527             model_version=results['model_version'],
528             target_stage='Staging'
529         )
530         print(f"Staging deployment: {'Success' if success else 'Failed'}")
531
532     # Generate GitHub Actions workflow
533     workflow_yaml = pipeline.generate_github_actions_workflow()
534     print("\nGenerated GitHub Actions workflow:")
535     print(workflow_yaml)

```

Listing 13.6: CI/CD Pipeline Integration for ML Models

13.8 Model Packaging and Containerization

13.8.1 Reproducible Model Packaging

Proper model packaging ensures reproducibility across environments. Packaging includes the model artifacts, dependencies, runtime configuration, and metadata needed for deployment.

Packaging requirements:

- **Model artifacts:** Serialized model files (pickle, ONNX, SavedModel)
- **Dependencies:** Exact versions of all libraries
- **Environment:** Python version, system libraries
- **Preprocessing:** Feature transformations, encoders, scalers
- **Configuration:** Model hyperparameters, thresholds
- **Metadata:** Model signature, lineage, performance metrics

13.8.2 Model Packaging and Container Implementation

```

1 from typing import Dict, List, Optional, Any
2 from pathlib import Path
3 import pickle
4 import json
5 import shutil
6 from dataclasses import dataclass
7
8 @dataclass
9 class PackageConfig:
10     """Model package configuration"""
11     model_name: str
12     model_version: str
13     model_format: str # mlflow, onnx, pickle, savedmodel
14     include_preprocessing: bool = True
15     include_postprocessing: bool = True
16     optimize_size: bool = True
17     compression: str = "gzip" # gzip, zip, tar
18
19 class ModelPackager:
20     """
21     Package models in multiple formats with dependencies
22
23     Creates self-contained model packages for deployment
24     """
25
26     def __init__(self, output_dir: Path = Path("./model_packages")):
27         """
28         Initialize model packager
29
30         Args:
31             output_dir: Directory for packaged models
32         """
33         self.output_dir = output_dir

```

```

34     self.output_dir.mkdir(parents=True, exist_ok=True)
35     self.logger = logging.getLogger("ModelPackager")
36
37     def package_model(self,
38                       model: Any,
39                       config: PackageConfig,
40                       metadata: Dict,
41                       preprocessing_pipeline: Any = None,
42                       postprocessing_fn: Optional[Callable] = None) -> Path:
43         """
44         Package model with all dependencies
45
46         Args:
47             model: Trained model
48             config: Package configuration
49             metadata: Model metadata
50             preprocessing_pipeline: Sklearn pipeline or custom preprocessor
51             postprocessing_fn: Post-prediction processing function
52
53         Returns:
54             Path to packaged model
55         """
56         self.logger.info(f"Packaging {config.model_name} v{config.model_version}")
57
58         # Create package directory
59         package_dir = self.output_dir / f"{config.model_name}_{config.model_version}"
60         package_dir.mkdir(exist_ok=True)
61
62         # Save model in specified format
63         if config.model_format == 'mlflow':
64             self._save_mlflow_format(model, package_dir, metadata)
65         elif config.model_format == 'onnx':
66             self._save_onnx_format(model, package_dir, metadata)
67         elif config.model_format == 'pickle':
68             self._save_pickle_format(model, package_dir)
69         else:
70             raise ValueError(f"Unsupported format: {config.model_format}")
71
72         # Save preprocessing pipeline
73         if config.include_preprocessing and preprocessing_pipeline:
74             self._save_preprocessing(preprocessing_pipeline, package_dir)
75
76         # Save postprocessing
77         if config.include_postprocessing and postprocessing_fn:
78             self._save_postprocessing(postprocessing_fn, package_dir)
79
80         # Save metadata
81         self._save_metadata(metadata, package_dir)
82
83         # Generate requirements.txt
84         self._generate_requirements(package_dir)
85
86         # Create inference script
87         self._create_inference_script(package_dir, config)
88
89         # Compress if requested
90         if config.optimize_size:
91             archive_path = self._compress_package(package_dir, config.compression)

```

```

92         return archive_path
93
94     return package_dir
95
96     def _save_mlflow_format(self, model: Any, package_dir: Path, metadata: Dict):
97         """Save in MLflow format"""
98         import mlflow.sklearn
99
100        mlflow.sklearn.save_model(
101            model,
102            path=str(package_dir / "model"),
103            signature=metadata.get('signature')
104        )
105
106    def _save_onnx_format(self, model: Any, package_dir: Path, metadata: Dict):
107        """Convert and save in ONNX format"""
108        try:
109            from skl2onnx import convert_sklearn
110            from skl2onnx.common.data_types import FloatTensorType
111
112            # Infer input shape from metadata
113            n_features = metadata.get('n_features', 10)
114            initial_type = [('float_input', FloatTensorType([None, n_features]))]
115
116            # Convert to ONNX
117            onnx_model = convert_sklearn(model, initial_types=initial_type)
118
119            # Save
120            with open(package_dir / "model.onnx", "wb") as f:
121                f.write(onnx_model.SerializeToString())
122
123            self.logger.info("Model converted to ONNX format")
124
125        except ImportError:
126            self.logger.warning("skl2onnx not installed, falling back to pickle")
127            self._save_pickle_format(model, package_dir)
128
129    def _save_pickle_format(self, model: Any, package_dir: Path):
130        """Save in pickle format"""
131        with open(package_dir / "model.pkl", "wb") as f:
132            pickle.dump(model, f)
133
134    def _save_preprocessing(self, pipeline: Any, package_dir: Path):
135        """Save preprocessing pipeline"""
136        with open(package_dir / "preprocessing.pkl", "wb") as f:
137            pickle.dump(pipeline, f)
138
139    def _save_postprocessing(self, fn: callable, package_dir: Path):
140        """Save postprocessing function"""
141        import dill
142
143        with open(package_dir / "postprocessing.pkl", "wb") as f:
144            dill.dump(fn, f)
145
146    def _save_metadata(self, metadata: Dict, package_dir: Path):
147        """Save model metadata"""
148        metadata_file = package_dir / "metadata.json"
149

```



```

150     with open(metadata_file, "w") as f:
151         json.dump(metadata, f, indent=2, default=str)
152
153     def _generate_requirements(self, package_dir: Path):
154         """Generate requirements.txt"""
155         import pkg_resources
156
157         # Get installed packages
158         installed_packages = {pkg.key: pkg.version for pkg in pkg_resources.
working_set}
159
160         # Core ML packages
161         required_packages = [
162             'numpy', 'pandas', 'scikit-learn', 'mlflow',
163             'scipy', 'joblib', 'fastapi', 'uvicorn'
164         ]
165
166         requirements = []
167         for package in required_packages:
168             if package in installed_packages:
169                 requirements.append(f"{package}=={installed_packages[package]}")
170
171         # Write requirements
172         with open(package_dir / "requirements.txt", "w") as f:
173             f.write("\n".join(requirements))
174
175     def _create_inference_script(self, package_dir: Path, config: PackageConfig):
176         """Create standalone inference script"""
177         script_content = f"""
178 import pickle
179 import json
180 import numpy as np
181 from pathlib import Path
182
183 class ModelInference:
184     def __init__(self, model_dir: str):
185         self.model_dir = Path(model_dir)
186
187         # Load model
188         if (self.model_dir / 'model.pkl').exists():
189             with open(self.model_dir / 'model.pkl', 'rb') as f:
190                 self.model = pickle.load(f)
191         else:
192             import mlflow.sklearn
193             self.model = mlflow.sklearn.load_model(str(self.model_dir / 'model'))
194
195         # Load preprocessing if exists
196         preproc_path = self.model_dir / 'preprocessing.pkl'
197         self.preprocessing = None
198         if preproc_path.exists():
199             with open(preproc_path, 'rb') as f:
200                 self.preprocessing = pickle.load(f)
201
202         # Load metadata
203         with open(self.model_dir / 'metadata.json', 'r') as f:
204             self.metadata = json.load(f)
205
206     def predict(self, input_data):

```

```

207     # Preprocess
208     if self.preprocessing:
209         input_data = self.preprocessing.transform(input_data)
210
211     # Predict
212     predictions = self.model.predict(input_data)
213
214     return predictions
215
216 if __name__ == '__main__':
217     # Example usage
218     inference = ModelInference('.')
219
220     # Make prediction
221     import pandas as pd
222     test_data = pd.DataFrame(np.random.rand(1, 10))
223     predictions = inference.predict(test_data)
224     print(f"Predictions: {{predictions}}")
225 """
226
227     with open(package_dir / "inference.py", "w") as f:
228         f.write(script_content)
229
230 def _compress_package(self, package_dir: Path, compression: str) -> Path:
231     """Compress package directory"""
232     archive_name = f"{package_dir.name}.tar.gz"
233     archive_path = self.output_dir / archive_name
234
235     shutil.make_archive(
236         str(package_dir),
237         'gztar',
238         self.output_dir,
239         package_dir.name
240     )
241
242     self.logger.info(f"Package compressed: {archive_path}")
243     return archive_path
244
245 class ContainerBuilder:
246     """
247     Build optimized containers for model deployment
248
249     Creates production-ready Docker images
250     """
251
252     def __init__(self):
253         """Initialize container builder"""
254         self.logger = logging.getLogger("ContainerBuilder")
255
256     def build_optimized_container(self,
257                                 model_package_path: Path,
258                                 model_name: str,
259                                 version: str,
260                                 base_image: str = "python:3.9-slim",
261                                 optimize: bool = True) -> str:
262         """
263         Build optimized Docker container
264

```

```

265     Args:
266         model_package_path: Path to model package
267         model_name: Model name
268         version: Model version
269         base_image: Base Docker image
270         optimize: Apply optimization techniques
271
272     Returns:
273         Image tag
274     """
275     self.logger.info(f"Building container for {model_name}:{version}")
276
277     # Generate optimized Dockerfile
278     dockerfile = self._generate_dockerfile(
279         model_package_path,
280         base_image,
281         optimize
282     )
283
284     # Write Dockerfile
285     with open('Dockerfile', 'w') as f:
286         f.write(dockerfile)
287
288     # Build image
289     image_tag = f"{model_name}:{version}"
290     result = subprocess.run(
291         ['docker', 'build', '-t', image_tag, '.'],
292         capture_output=True,
293         text=True
294     )
295
296     if result.returncode != 0:
297         raise Exception(f"Docker build failed: {result.stderr}")
298
299     self.logger.info(f"Container built: {image_tag}")
300     return image_tag
301
302     def _generate_dockerfile(self,
303                             model_package_path: Path,
304                             base_image: str,
305                             optimize: bool) -> str:
306         """Generate optimized Dockerfile"""
307
308         # Optimization layers
309         optimization_layers = ""
310         if optimize:
311             optimization_layers = """
312 # Optimization: Use multi-stage build
313 FROM python:3.9 as builder
314 WORKDIR /build
315 COPY requirements.txt .
316 RUN pip install --user -r requirements.txt
317
318 # Production stage
319 """
320
321         dockerfile = f"""
322 {optimization_layers}

```

```

323 FROM {base_image}
324
325 # Set working directory
326 WORKDIR /app
327
328 # Copy dependencies from builder (if optimized)
329 {"COPY --from=builder /root/.local /root/.local" if optimize else ""}
330
331 # Install dependencies
332 COPY requirements.txt .
333 {"" if optimize else "RUN pip install --no-cache-dir -r requirements.txt"}
334
335 # Copy model package
336 COPY {model_package_path} ./model/
337
338 # Copy serving code
339 COPY inference.py .
340 COPY serve.py .
341
342 # Create non-root user for security
343 RUN useradd -m -u 1000 modeluser && chown -R modeluser:modeluser /app
344 USER modeluser
345
346 # Set environment variables
347 ENV MODEL_DIR=/app/model
348 ENV PYTHONPATH=/app
349
350 # Health check
351 HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \\\
352     CMD python -c "import requests; requests.get('http://localhost:8000/health')"
353
354 # Expose port
355 EXPOSE 8000
356
357 # Run server
358 CMD ["python", "serve.py"]
359 """
360
361     return dockerfile
362
363 # Example usage
364 def example_packaging_and_containerization():
365     """Example of model packaging and containerization"""
366
367     from sklearn.ensemble import RandomForestClassifier
368     from sklearn.preprocessing import StandardScaler
369     from sklearn.pipeline import Pipeline
370
371     # Train a simple model
372     X_train = np.random.rand(100, 10)
373     y_train = np.random.randint(0, 2, 100)
374
375     # Create preprocessing pipeline
376     preprocessing = StandardScaler()
377     preprocessing.fit(X_train)
378
379     # Train model
380     model = RandomForestClassifier(n_estimators=100, random_state=42)

```

```

381     model.fit(preprocessing.transform(X_train), y_train)
382
383     # Package configuration
384     config = PackageConfig(
385         model_name='fraud_detector',
386         model_version='v1.0.0',
387         model_format='mlflow',
388         include_preprocessing=True,
389         optimize_size=True,
390         compression='gzip'
391     )
392
393     # Metadata
394     metadata = {
395         'model_type': 'RandomForestClassifier',
396         'framework': 'scikit-learn',
397         'n_features': 10,
398         'metrics': {
399             'accuracy': 0.95,
400             'f1_score': 0.93
401         },
402         'created_at': datetime.now().isoformat()
403     }
404
405     # Package model
406     packager = ModelPackager()
407     package_path = packager.package_model(
408         model=model,
409         config=config,
410         metadata=metadata,
411         preprocessing_pipeline=preprocessing
412     )
413
414     print(f"Model packaged at: {package_path}")
415
416     # Build container
417     builder = ContainerBuilder()
418     image_tag = builder.build_optimized_container(
419         model_package_path=package_path,
420         model_name=config.model_name,
421         version=config.model_version,
422         optimize=True
423     )
424
425     print(f"Container built: {image_tag}")

```

Listing 13.7: Model Packaging and Containerization System

13.9 Exercises

13.9.1 Exercise 1: Model Registration and Lifecycle Management

Problem: Implement a complete model registration workflow that trains a model, registers it in MLflow, transitions through stages (Staging → Production), and maintains

version history.

Solution:

```

1  # Exercise 1 Solution
2  from sklearn.datasets import load_breast_cancer
3  from sklearn.model_selection import train_test_split
4  from sklearn.ensemble import GradientBoostingClassifier
5  from sklearn.metrics import accuracy_score, f1_score, roc_auc_score
6  import mlflow
7  import mlflow.sklearn
8
9  def solution_exercise_1():
10     """Complete model registration and lifecycle workflow"""
11
12     # Initialize MLflow
13     mlflow.set_tracking_uri("http://localhost:5000")
14     mlflow.set_experiment("breast_cancer_detection")
15
16     # Load data
17     data = load_breast_cancer()
18     X_train, X_test, y_train, y_test = train_test_split(
19         data.data, data.target, test_size=0.2, random_state=42
20     )
21
22     # Train model
23     model = GradientBoostingClassifier(
24         n_estimators=100,
25         learning_rate=0.1,
26         max_depth=3,
27         random_state=42
28     )
29
30     with mlflow.start_run(run_name="breast_cancer_v1") as run:
31         # Train
32         model.fit(X_train, y_train)
33
34         # Evaluate
35         y_pred = model.predict(X_test)
36         y_pred_proba = model.predict_proba(X_test)[:, 1]
37
38         accuracy = accuracy_score(y_test, y_pred)
39         f1 = f1_score(y_test, y_pred)
40         auc = roc_auc_score(y_test, y_pred_proba)
41
42         # Log metrics
43         mlflow.log_metric("accuracy", accuracy)
44         mlflow.log_metric("f1_score", f1)
45         mlflow.log_metric("auc_roc", auc)
46
47         # Log parameters
48         mlflow.log_param("n_estimators", 100)
49         mlflow.log_param("learning_rate", 0.1)
50         mlflow.log_param("max_depth", 3)
51
52         # Log model
53         mlflow.sklearn.log_model(
54             model,
55             artifact_path="model",

```

```

56         registered_model_name="breast_cancer_detector"
57     )
58
59     run_id = run.info.run_id
60
61     # Initialize registry manager
62     registry = MLflowModelRegistry(tracking_uri="http://localhost:5000")
63
64     # Get latest version
65     versions = registry.client.get_latest_versions("breast_cancer_detector")
66     latest_version = versions[0].version
67
68     # Transition to Staging
69     registry.transition_model_stage(
70         model_name="breast_cancer_detector",
71         version=latest_version,
72         stage="Staging"
73     )
74
75     print(f"Model v{latest_version} moved to Staging")
76
77     # After validation in staging, move to Production
78     registry.transition_model_stage(
79         model_name="breast_cancer_detector",
80         version=latest_version,
81         stage="Production",
82         archive_existing=True
83     )
84
85     print(f"Model v{latest_version} promoted to Production")
86
87     # Get model stages summary
88     summary = registry.get_model_stages_summary("breast_cancer_detector")
89     print(f"Model stages: {summary}")
90
91 solution_exercise_1()

```

13.9.2 Exercise 2: Complete Model Lineage Tracking

Problem: Implement comprehensive lineage tracking for a model that captures data provenance, code version, training configuration, and dependencies.

Solution:

```

1  # Exercise 2 Solution
2  import pandas as pd
3  import numpy as np
4  from sklearn.ensemble import RandomForestClassifier
5
6  def solution_exercise_2():
7      """Implement complete lineage tracking"""
8
9      # Create training dataset
10     np.random.seed(42)
11     X_train = pd.DataFrame({
12         'feature_1': np.random.rand(1000),
13         'feature_2': np.random.rand(1000),
14         'feature_3': np.random.randint(0, 10, 1000),

```

```

15     'feature_4': np.random.rand(1000),
16     'feature_5': np.random.randint(0, 5, 1000)
17 })
18 y_train = np.random.randint(0, 2, 1000)
19
20 # Initialize lineage tracker
21 tracker = LineageTracker(storage_backend="mlflow")
22
23 # Track data lineage
24 data_lineage = tracker.track_data_lineage(
25     dataset=X_train,
26     dataset_name="customer_churn_features",
27     dataset_version="v2024-01-20",
28     data_source="warehouse.customer_events",
29     preprocessing_steps=[
30         "remove_null_values",
31         "normalize_numerical_features",
32         "encode_categorical_features",
33         "handle_class_imbalance_smote",
34         "train_test_split_80_20"
35     ]
36 )
37
38 print(f"Data lineage tracked:")
39 print(f"  Dataset: {data_lineage.dataset_name}")
40 print(f"  Version: {data_lineage.dataset_version}")
41 print(f"  Hash: {data_lineage.dataset_hash[:16]}...")
42 print(f"  Samples: {data_lineage.num_samples}")
43 print(f"  Features: {data_lineage.num_features}")
44 print(f"  Preprocessing: {data_lineage.preprocessing_steps}")
45
46 # Track code lineage
47 code_lineage = tracker.track_code_lineage(
48     training_script_paths=["train.py", "preprocessing.py", "utils.py"]
49 )
50
51 print(f"\nCode lineage tracked:")
52 print(f"  Git commit: {code_lineage.git_commit[:8]}")
53 print(f"  Git branch: {code_lineage.git_branch}")
54 print(f"  Repository: {code_lineage.git_repo}")
55 print(f"  Dependencies: {len(code_lineage.dependencies)} packages")
56
57 # Train model
58 model = RandomForestClassifier(
59     n_estimators=200,
60     max_depth=15,
61     min_samples_split=10,
62     random_state=42
63 )
64 model.fit(X_train, y_train)
65
66 # Define hyperparameters
67 hyperparameters = {
68     'n_estimators': 200,
69     'max_depth': 15,
70     'min_samples_split': 10,
71     'random_state': 42,
72     'criterion': 'gini',

```



```

73     'bootstrap': True
74 }
75
76 # Calculate metrics
77 from sklearn.metrics import accuracy_score, precision_score, recall_score
78 y_pred = model.predict(X_train) # Simplified - should use validation set
79
80 metrics = {
81     'accuracy': accuracy_score(y_train, y_pred),
82     'precision': precision_score(y_train, y_pred),
83     'recall': recall_score(y_train, y_pred)
84 }
85
86 # Create complete lineage
87 model_lineage = tracker.create_lineage(
88     model_id="churn_model_abc123",
89     model_name="customer_churn_predictor",
90     model_version="v3.0",
91     data_lineage=data_lineage,
92     code_lineage=code_lineage,
93     hyperparameters=hyperparameters,
94     training_duration=245.5,
95     metrics=metrics,
96     parent_models=[] # No parent models (not transfer learning)
97 )
98
99 print(f"\nComplete model lineage created:")
100 print(f"  Model ID: {model_lineage.model_id}")
101 print(f"  Model: {model_lineage.model_name} v{model_lineage.model_version}")
102 print(f"  Hardware: {model_lineage.hardware}")
103 print(f"  Training duration: {model_lineage.training_duration}s")
104
105 return model_lineage
106
107 lineage = solution_exercise_2()

```

13.9.3 Exercise 3: Model Governance with Approval Workflows

Problem: Implement a governance system with validation gates and multi-stage approval workflow for promoting a model to production.

Solution:

```

1 # Exercise 3 Solution
2
3 def solution_exercise_3():
4     """Implement model governance with approval workflows"""
5
6     # Initialize components
7     registry = MLflowModelRegistry(tracking_uri="http://localhost:5000")
8     governance = ModelGovernanceManager(registry)
9
10    # Define custom validation gates
11    def validate_minimum_samples(model_info: Dict, request: ApprovalRequest) -> bool:
12        """Ensure model trained on sufficient data"""
13        dataset_size = model_info.get('params', {}).get('dataset_size', 0)
14        min_required = 10000
15

```

```

16         if int(dataset_size) < min_required:
17             return False
18         return True
19
20     def validate_model_bias(model_info: Dict, request: ApprovalRequest) -> bool:
21         """Check for model bias metrics"""
22         tags = model_info.get('tags', {})
23
24         # Check if bias assessment was performed
25         if tags.get('bias_assessed') != 'true':
26             return False
27
28         # Check fairness metrics
29         demographic_parity = float(tags.get('demographic_parity', '0'))
30         if abs(demographic_parity) > 0.1: # Max 10% disparity
31             return False
32
33         return True
34
35     def validate_security_scan(model_info: Dict, request: ApprovalRequest) -> bool:
36         """Validate security scanning completed"""
37         tags = model_info.get('tags', {})
38         return tags.get('security_scanned') == 'true'
39
40     # Register validation gates for Production
41     governance.register_validation_gate(
42         target_stage='Production',
43         gate=ValidationGate(
44             name='performance_check',
45             description='Validate performance meets thresholds',
46             validator_func=validate_model_performance,
47             required=True
48         )
49     )
50
51     governance.register_validation_gate(
52         target_stage='Production',
53         gate=ValidationGate(
54             name='sample_size_check',
55             description='Ensure sufficient training data',
56             validator_func=validate_minimum_samples,
57             required=True
58         )
59     )
60
61     governance.register_validation_gate(
62         target_stage='Production',
63         gate=ValidationGate(
64             name='bias_check',
65             description='Check for model bias and fairness',
66             validator_func=validate_model_bias,
67             required=True
68         )
69     )
70
71     governance.register_validation_gate(
72         target_stage='Production',
73         gate=ValidationGate(

```

```

74         name='security_check',
75         description='Validate security scan completion',
76         validator_func=validate_security_scan,
77         required=False # Optional for now
78     )
79 )
80
81 # Request approval for production
82 request_id = governance.request_approval(
83     model_name='fraud_detector',
84     version='5',
85     source_stage='Staging',
86     target_stage='Production',
87     requester='data-scientist@company.com',
88     requester_role=UserRole.DATA_SCIENTIST,
89     justification='New model achieves 3% improvement in precision with no recall
degradation',
90     impact_assessment='Expected to reduce false positives by 20%, saving $500K
annually in manual review costs',
91     rollback_plan='Automated rollback to v4 if production accuracy drops below
0.92 or latency exceeds 50ms'
92 )
93
94 print(f"Approval request created: {request_id}")
95
96 # Check request status
97 status = governance.get_request_status(request_id)
98 print(f"\nRequest Status:")
99 print(f"  Status: {status['status']}")
100 print(f"  Model: {status['model_name']} v{status['version']}")
101 print(f"  Target: {status['target_stage']}")
102 print(f"  Validation results: {status['validation_results']}")
103
104 # Team lead approval
105 governance.approve_request(
106     request_id=request_id,
107     approver='team-lead@company.com',
108     approver_role=UserRole.TEAM_LEAD,
109     comments='Validated model performance in staging. Metrics look good. Approved.
,
110 )
111
112 print(f"\nTeam Lead approved")
113
114 # Production owner approval
115 governance.approve_request(
116     request_id=request_id,
117     approver='prod-owner@company.com',
118     approver_role=UserRole.PRODUCTION_OWNER,
119     comments='Infrastructure capacity confirmed. Load testing passed. Final
approval granted.'
120 )
121
122 print(f"Production Owner approved")
123
124 # Check final status
125 final_status = governance.get_request_status(request_id)
126 print(f"\nFinal Status: {final_status['status']}")

```

```

127     print(f"Approvals: {len(final_status['approvals'])}")
128
129     # Get audit trail
130     audit_trail = governance.get_audit_trail(model_name='fraud_detector')
131     print(f"\nAudit Trail ({len(audit_trail)} events):")
132     for event in audit_trail[-5:]: # Show last 5 events
133         print(f"    [{event['timestamp']}] {event['event_type']} by {event['user']}")
134
135     return request_id
136
137 request_id = solution_exercise_3()

```

13.9.4 Exercise 4: Statistical Model Comparison

Problem: Implement rigorous statistical comparison between two model versions, including McNemar's test and bootstrap confidence intervals.

Solution:

```

1  # Exercise 4 Solution
2  from sklearn.datasets import make_classification
3  from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
4
5  def solution_exercise_4():
6      """Statistical model comparison with hypothesis testing"""
7
8      # Create test data
9      X_test, y_test = make_classification(
10         n_samples=2000,
11         n_features=20,
12         n_informative=15,
13         n_redundant=5,
14         random_state=42
15     )
16
17     # Train two models
18     model_v1 = RandomForestClassifier(n_estimators=100, random_state=42)
19     model_v2 = GradientBoostingClassifier(n_estimators=100, random_state=42)
20
21     # Simplified training (use actual train set in practice)
22     model_v1.fit(X_test[:1000], y_test[:1000])
23     model_v2.fit(X_test[:1000], y_test[:1000])
24
25     # Initialize comparator
26     registry = MLflowModelRegistry(tracking_uri="http://localhost:5000")
27     comparator = ModelComparator(registry)
28
29     # Register both models first (simplified here)
30     # In practice, load from registry
31
32     # Create comparison with test data
33     print("Comparing Model v1 (RandomForest) vs Model v2 (GradientBoosting)")
34     print("=" * 70)
35
36     # Get predictions
37     pred_v1_proba = model_v1.predict_proba(X_test)[: , 1]
38     pred_v2_proba = model_v2.predict_proba(X_test)[: , 1]
39

```

```

40 pred_v1 = (pred_v1_proba > 0.5).astype(int)
41 pred_v2 = (pred_v2_proba > 0.5).astype(int)
42
43 # Calculate metrics
44 from sklearn.metrics import accuracy_score, precision_score, recall_score,
45 f1_score, roc_auc_score
46
47 metrics_v1 = {
48     'accuracy': accuracy_score(y_test, pred_v1),
49     'precision': precision_score(y_test, pred_v1),
50     'recall': recall_score(y_test, pred_v1),
51     'f1_score': f1_score(y_test, pred_v1),
52     'auc_roc': roc_auc_score(y_test, pred_v1_proba)
53 }
54
55 metrics_v2 = {
56     'accuracy': accuracy_score(y_test, pred_v2),
57     'precision': precision_score(y_test, pred_v2),
58     'recall': recall_score(y_test, pred_v2),
59     'f1_score': f1_score(y_test, pred_v2),
60     'auc_roc': roc_auc_score(y_test, pred_v2_proba)
61 }
62
63 print("\nPerformance Metrics:")
64 print(f"{'Metric':<15} {'v1 (RF)':<12} {'v2 (GB)':<12} {'Improvement':<15}")
65 print("-" * 70)
66
67 for metric in metrics_v1.keys():
68     v1_val = metrics_v1[metric]
69     v2_val = metrics_v2[metric]
70     improvement = ((v2_val - v1_val) / v1_val) * 100
71
72     print(f"{'metric':<15} {'v1_val':<12.4f} {'v2_val':<12.4f} {'improvement':>+.2f}%")
73
74 # McNemar's Test
75 print("\n\nStatistical Significance Testing:")
76 print("-" * 70)
77
78 # Contingency table
79 both_correct = np.sum((pred_v1 == y_test) & (pred_v2 == y_test))
80 only_v1_correct = np.sum((pred_v1 == y_test) & (pred_v2 != y_test))
81 only_v2_correct = np.sum((pred_v1 != y_test) & (pred_v2 == y_test))
82 both_wrong = np.sum((pred_v1 != y_test) & (pred_v2 != y_test))
83
84 print(f"\nContingency Table:")
85 print(f" Both models correct: {both_correct}")
86 print(f" Only v1 correct: {only_v1_correct}")
87 print(f" Only v2 correct: {only_v2_correct}")
88 print(f" Both models wrong: {both_wrong}")
89
90 # McNemar statistic
91 if only_v1_correct + only_v2_correct > 0:
92     from scipy import stats
93     mcnemar_stat = ((abs(only_v1_correct - only_v2_correct) - 1) ** 2) / \
94         (only_v1_correct + only_v2_correct)
95     mcnemar_p = 1 - stats.chi2.cdf(mcnemar_stat, 1)
96
97 print(f"\nMcNemar's Test:")

```

```

97     print(f"   Chi-square statistic: {mcnemar_stat:.4f}")
98     print(f"   p-value: {mcnemar_p:.4f}")
99     print(f"   Significant (p < 0.05): {mcnemar_p < 0.05}")
100
101     # Bootstrap confidence intervals
102     print(f"\nBootstrap Confidence Intervals (95%):")
103
104     n_bootstrap = 1000
105     auc_diffs = []
106
107     for _ in range(n_bootstrap):
108         indices = np.random.choice(len(y_test), len(y_test), replace=True)
109         boot_auc1 = roc_auc_score(y_test[indices], pred_v1_proba[indices])
110         boot_auc2 = roc_auc_score(y_test[indices], pred_v2_proba[indices])
111         auc_diffs.append(boot_auc2 - boot_auc1)
112
113     auc_diffs = np.array(auc_diffs)
114     ci_lower = np.percentile(auc_diffs, 2.5)
115     ci_upper = np.percentile(auc_diffs, 97.5)
116
117     print(f"   AUC difference: ({ci_lower:.4f}, {ci_upper:.4f})")
118     print(f"   CI excludes zero: {ci_lower > 0 or ci_upper < 0}")
119
120     # Recommendation
121     print(f"\n\nRecommendation:")
122     if mcnemar_p < 0.05 and ci_lower > 0:
123         print(f"   [OK] Model v2 shows statistically significant improvement")
124         print(f"   [OK] Recommend promoting v2 to production")
125     elif mcnemar_p >= 0.05:
126         print(f"   [X] No statistically significant difference detected")
127         print(f"   [X] Keep current model v1 in production")
128     else:
129         print(f"   ? Mixed results - require additional validation")
130
131 solution_exercise_4()

```

13.9.5 Exercise 5: CI/CD Pipeline for Model Deployment

Problem: Create a complete CI/CD pipeline that trains a model, runs tests, registers it, and deploys to Kubernetes.

Solution:

```

1  # Exercise 5 Solution
2
3  def solution_exercise_5():
4      """Complete CI/CD pipeline implementation"""
5
6      # Pipeline configuration
7      config = PipelineConfig(
8          git_repo='https://github.com/company/ml-fraud-detection',
9          git_branch='main',
10         training_script='scripts/train_fraud_detector.py',
11         training_data_path='data/fraud_training.csv',
12         hyperparameters_file='config/fraud_detector_params.yaml',
13         test_script='scripts/test_fraud_detector.py',
14         test_data_path='data/fraud_test.csv',
15         performance_thresholds={

```

```

16         'accuracy': 0.93,
17         'precision': 0.90,
18         'recall': 0.88,
19         'f1_score': 0.89,
20         'auc_roc': 0.92
21     },
22     model_name='fraud_detector',
23     registry_uri='http://mlflow.company.com',
24     deployment_target='production',
25     kubernetes_namespace='ml-models-prod',
26     docker_image_registry='gcr.io/company-ml'
27 )
28
29 # Initialize pipeline components
30 registry = MLflowModelRegistry(tracking_uri=config.registry_uri)
31 governance = ModelGovernanceManager(registry)
32 pipeline = MLPipeline(config, registry, governance)
33
34 print("=" * 70)
35 print("ML CI/CD Pipeline Execution")
36 print("=" * 70)
37
38 # Step 1: Run training pipeline
39 print("\n[1/3] Running Training Pipeline...")
40 print("-" * 70)
41
42 try:
43     results = pipeline.run_training_pipeline()
44
45     if not results['tests_passed']:
46         print("[X] Training pipeline failed: Tests did not pass")
47         print(f"    Metrics: {results['metrics']}")
48         return False
49
50     print(f"[OK] Training completed successfully")
51     print(f"    Git commit: {results['git_commit'][:8]}")
52     print(f"    Run ID: {results['run_id']}")
53     print(f"    Model version: {results['model_version']}")
54     print(f"    Metrics: {results['metrics']}")
55
56 except Exception as e:
57     print(f"[X] Training pipeline failed: {str(e)}")
58     return False
59
60 # Step 2: Run deployment to staging
61 print(f"\n[2/3] Deploying to Staging...")
62 print("-" * 70)
63
64 try:
65     staging_success = pipeline.run_deployment_pipeline(
66         model_version=results['model_version'],
67         target_stage='Staging'
68     )
69
70     if not staging_success:
71         print("[X] Staging deployment failed")
72         return False
73

```

```

74     print(f"[OK] Deployed to Staging successfully")
75     print(f"    Model: {config.model_name}:{results['model_version']}")
76     print(f"    Namespace: {config.kubernetes_namespace}")
77
78     except Exception as e:
79         print(f"[X] Staging deployment failed: {str(e)}")
80         return False
81
82     # Step 3: Production promotion (requires approval)
83     print(f"\n[3/3] Production Promotion Workflow...")
84     print("-" * 70)
85
86     # Request production approval
87     request_id = governance.request_approval(
88         model_name=config.model_name,
89         version=results['model_version'],
90         source_stage='Staging',
91         target_stage='Production',
92         requester='ci-cd-pipeline@company.com',
93         requester_role=UserRole.ML_ENGINEER,
94         justification=f"Automated deployment - all tests passed with metrics: {results
95         ['metrics']}",
96         impact_assessment="Improved fraud detection accuracy expected to save $2M
97         annually",
98         rollback_plan="Automated rollback to previous version if production metrics
99         degrade"
100     )
101
102     print(f"[OK] Production approval request created: {request_id}")
103     print(f"    Awaiting approvals from: Team Lead, Production Owner")
104     print(f"\n    Next steps:")
105     print(f"        1. Team Lead reviews and approves")
106     print(f"        2. Production Owner reviews and approves")
107     print(f"        3. Model automatically promoted to Production")
108     print(f"        4. Kubernetes deployment updated")
109
110     # Generate GitHub Actions workflow
111     print(f"\n\nGenerated GitHub Actions Workflow:")
112     print("=" * 70)
113     workflow = pipeline.generate_github_actions_workflow()
114     print(workflow[:500] + "...") # Show first 500 chars
115
116     return True
117
118 success = solution_exercise_5()
119 print(f"\n\nPipeline execution: {'SUCCESS' if success else 'FAILED'}")

```

13.9.6 Exercise 6: Model Packaging for Multi-Format Deployment

Problem: Package a model in multiple formats (MLflow, ONNX, pickle) with preprocessing pipelines and create an optimized Docker container.

Solution:

```

1 # Exercise 6 Solution
2 from sklearn.preprocessing import StandardScaler, OneHotEncoder

```



```

3 from sklearn.compose import ColumnTransformer
4
5 def solution_exercise_6():
6     """Package model in multiple formats with containerization"""
7
8     # Create sample model with preprocessing
9     X_train = pd.DataFrame({
10         'age': np.random.randint(18, 80, 1000),
11         'income': np.random.randint(20000, 200000, 1000),
12         'credit_score': np.random.randint(300, 850, 1000),
13         'category': np.random.choice(['A', 'B', 'C'], 1000)
14     })
15     y_train = np.random.randint(0, 2, 1000)
16
17     # Create preprocessing pipeline
18     numeric_features = ['age', 'income', 'credit_score']
19     categorical_features = ['category']
20
21     preprocessor = ColumnTransformer(
22         transformers=[
23             ('num', StandardScaler(), numeric_features),
24             ('cat', OneHotEncoder(drop='first'), categorical_features)
25         ]
26     )
27
28     # Fit preprocessing
29     preprocessor.fit(X_train)
30     X_processed = preprocessor.transform(X_train)
31
32     # Train model
33     model = RandomForestClassifier(
34         n_estimators=150,
35         max_depth=12,
36         min_samples_split=8,
37         random_state=42
38     )
39     model.fit(X_processed, y_train)
40
41     # Package in multiple formats
42     packager = ModelPackager(output_dir=Path("./model_packages"))
43
44     formats = ['mlflow', 'pickle', 'onnx']
45     packages = {}
46
47     print("Packaging model in multiple formats...")
48     print("=" * 70)
49
50     for fmt in formats:
51         print(f"\nPackaging in {fmt.upper()} format...")
52
53         config = PackageConfig(
54             model_name='credit_risk_model',
55             model_version='v2.1.0',
56             model_format=fmt,
57             include_preprocessing=True,
58             include_postprocessing=False,
59             optimize_size=True,
60             compression='gzip'

```

```

61     )
62
63     metadata = {
64         'model_type': 'RandomForestClassifier',
65         'framework': 'scikit-learn',
66         'framework_version': '1.2.0',
67         'n_features': len(numeric_features) + len(categorical_features),
68         'feature_names': X_train.columns.tolist(),
69         'numeric_features': numeric_features,
70         'categorical_features': categorical_features,
71         'metrics': {
72             'accuracy': 0.94,
73             'precision': 0.92,
74             'recall': 0.91,
75             'f1_score': 0.91
76         },
77         'created_at': datetime.now().isoformat(),
78         'created_by': 'automated-pipeline',
79         'use_case': 'credit_risk_assessment'
80     }
81
82     package_path = packager.package_model(
83         model=model,
84         config=config,
85         metadata=metadata,
86         preprocessing_pipeline=preprocessor
87     )
88
89     packages[fmt] = package_path
90     print(f"    [OK] Package created: {package_path}")
91
92     # Check package contents
93     if package_path.is_dir():
94         contents = list(package_path.glob('*'))
95         print(f"    Package contents:")
96         for item in contents:
97             print(f"        - {item.name}")
98
99     # Build Docker containers for each format
100    print(f"\n\nBuilding Docker Containers...")
101    print("=" * 70)
102
103    builder = ContainerBuilder()
104
105    for fmt, package_path in packages.items():
106        if package_path.suffix == '.gz':
107            # Extract archive first
108            import tarfile
109            extract_dir = package_path.parent / package_path.stem.replace('.tar', '')
110            with tarfile.open(package_path, 'r:gz') as tar:
111                tar.extractall(path=extract_dir)
112            package_path = extract_dir
113
114            print(f"\nBuilding container for {fmt.upper()} format...")
115
116            image_tag = builder.build_optimized_container(
117                model_package_path=package_path,
118                model_name=f'credit_risk_model_{fmt}',

```

```

119         version='v2.1.0',
120         base_image='python:3.9-slim',
121         optimize=True
122     )
123
124     print(f"    [OK] Container built: {image_tag}")
125     print(f"    Ready for deployment to Kubernetes/Cloud Run")
126
127     # Summary
128     print(f"\n\nPackaging Summary:")
129     print("=" * 70)
130     print(f"Model: credit_risk_model v2.1.0")
131     print(f"Formats packaged: {len(packages)}")
132     print(f"Containers built: {len(packages)}")
133     print(f"\nDeployment options:")
134     print(f"    - MLflow format: Best for MLflow-based serving")
135     print(f"    - ONNX format: Cross-platform, optimized inference")
136     print(f"    - Pickle format: Simple Python-based serving")
137
138     return packages
139
140 packages = solution_exercise_6()

```

13.10 Chapter Summary

This chapter covered comprehensive model versioning and registry systems essential for production ML operations. We explored the complete lifecycle of model management from registration to deployment, focusing on practical implementations that ensure reproducibility, governance, and operational excellence.

13.10.1 Key Concepts Covered

MLflow Model Registry:

- Model registration with comprehensive metadata and signatures
- Stage-based lifecycle management (None, Staging, Production, Archived)
- Version control and model comparison capabilities
- Model loading and serving from registry

Model Lineage and Provenance:

- Data lineage tracking with dataset versioning and hashing
- Code lineage capturing git commits, branches, and dependencies
- Complete model provenance including hyperparameters and training metadata
- Reproducibility through comprehensive artifact tracking

Governance and Approval Workflows:

- Multi-stage approval processes with role-based access control

- Automated validation gates ensuring model quality
- Audit trails for compliance and regulatory requirements
- Integration with ticketing systems for formal approvals

Model Comparison and Promotion:

- Statistical significance testing with McNemar's test
- Bootstrap confidence intervals for metric comparisons
- Comprehensive evaluation across performance, latency, and resource usage
- Automated promotion based on criteria and thresholds

CI/CD Integration:

- Automated training, testing, and registration pipelines
- Docker containerization and Kubernetes deployment
- GitHub Actions and Jenkins workflow integration
- Automated rollback and smoke testing

Model Packaging and Containerization:

- Multi-format packaging (MLflow, ONNX, pickle)
- Dependency management and requirements generation
- Optimized Docker images with multi-stage builds
- Security best practices with non-root users

13.10.2 Best Practices

1. **Always track complete lineage:** Capture data, code, and training provenance for every model version to ensure reproducibility
2. **Implement validation gates:** Automated checks prevent low-quality models from reaching production
3. **Require statistical significance:** Use hypothesis testing to validate that model improvements are real, not random variation
4. **Automate everything:** CI/CD pipelines reduce manual errors and accelerate deployment
5. **Maintain audit trails:** Complete history of model changes satisfies compliance requirements
6. **Package with preprocessing:** Include all transformation steps in model packages for deployment consistency
7. **Use staged rollouts:** Test models in staging before production deployment
8. **Monitor production models:** Continuous validation ensures models maintain performance

13.10.3 Common Pitfalls to Avoid

- **Incomplete metadata:** Missing training details make reproduction impossible
- **Skipping approval gates:** Direct production deployments bypass critical validation
- **Ignoring statistical significance:** Small metric improvements may not be meaningful
- **Manual deployments:** Error-prone and slow compared to automated pipelines
- **Missing rollback plans:** Production issues without rollback capability cause outages
- **Inadequate testing:** Insufficient validation leads to production failures
- **Poor version control:** Unclear model history makes debugging difficult

13.10.4 Production Checklist

Before deploying a model registry system to production, verify:

1. Model registry infrastructure is highly available and backed up
2. All model versions have complete metadata and lineage
3. Approval workflows are configured with appropriate roles
4. Validation gates are implemented and tested
5. CI/CD pipelines are operational with proper error handling
6. Docker images are scanned for security vulnerabilities
7. Kubernetes deployments have resource limits and health checks
8. Rollback procedures are documented and tested
9. Monitoring and alerting are configured for model performance
10. Audit logging captures all model lifecycle events

13.10.5 Next Steps

The model versioning and registry system forms the foundation for production ML operations. In subsequent chapters, we'll explore:

- Model monitoring and observability systems
- Data drift detection and model retraining triggers
- Multi-model ensemble management
- Cost optimization for model serving

- Advanced deployment strategies (shadow, canary, A/B)

By implementing the practices and systems covered in this chapter, teams can confidently manage dozens or hundreds of model versions, ensure reproducibility, maintain governance, and deploy models safely to production. The combination of MLflow, automated workflows, and statistical rigor provides a production-grade foundation for ML operations at scale.

Part V

Production ML Operations

Chapter 14

ML Monitoring and Observability

14.1 Learning Objectives

After completing this chapter, readers will be able to:

- Design comprehensive monitoring architectures for production ML systems
- Implement data drift and concept drift detection using multiple methods
- Build real-time model performance monitoring dashboards
- Integrate Prometheus and Grafana for ML metrics collection and visualization
- Create intelligent alert systems with automated incident response
- Apply observability best practices for debugging and optimization
- Diagnose and resolve common ML system failures
- Implement complete monitoring stacks for enterprise ML deployments

14.2 Introduction

ML models in production face unique challenges: data distributions shift, model performance degrades, infrastructure fails, and business requirements change. Without comprehensive monitoring, teams discover issues only after significant business impact. Production ML monitoring requires tracking not just traditional software metrics (latency, throughput, errors), but also ML-specific concerns like data drift, prediction quality, and model staleness.

Effective ML monitoring provides:

- **Early warning:** Detect degradation before business impact
- **Root cause analysis:** Understand why models fail
- **Continuous validation:** Ensure ongoing model quality
- **Compliance:** Audit trail for regulatory requirements
- **Optimization insights:** Data for model improvement

This chapter covers production-grade monitoring systems that combine traditional observability with ML-specific metrics.

14.3 ML Monitoring Architecture

14.3.1 Monitoring System Components

A comprehensive ML monitoring architecture consists of multiple layers working together to provide complete observability.

Architecture layers:

- **Data collection:** Metrics, logs, traces from ML systems
- **Storage:** Time-series databases for metric history
- **Processing:** Real-time drift detection and aggregation
- **Alerting:** Intelligent thresholds and incident management
- **Visualization:** Dashboards for human insight
- **Analysis:** Automated root cause identification

14.3.2 Complete Monitoring Architecture Implementation

```

1 from typing import Dict, List, Optional, Any, Callable
2 from dataclasses import dataclass, field
3 from datetime import datetime, timedelta
4 from enum import Enum
5 import numpy as np
6 import pandas as pd
7 import logging
8 from collections import deque
9 import json
10
11 class MetricType(Enum):
12     """Types of metrics to monitor"""
13     COUNTER = "counter"
14     GAUGE = "gauge"
15     HISTOGRAM = "histogram"
16     SUMMARY = "summary"
17
18 class AlertSeverity(Enum):
19     """Alert severity levels"""
20     INFO = "info"
21     WARNING = "warning"
22     ERROR = "error"
23     CRITICAL = "critical"
24
25 @dataclass
26 class Metric:
27     """Individual metric data point"""
28     name: str
29     value: float
30     timestamp: datetime
31     labels: Dict[str, str] = field(default_factory=dict)
32     metric_type: MetricType = MetricType.GAUGE
33
34 @dataclass

```

```

35 class Alert:
36     """Alert configuration and state"""
37     alert_id: str
38     name: str
39     condition: Callable
40     severity: AlertSeverity
41     message_template: str
42     cooldown_minutes: int = 15
43     last_triggered: Optional[datetime] = None
44
45 class MetricsCollector:
46     """
47     Collect and store ML system metrics
48
49     Provides thread-safe metric collection with buffering
50     """
51
52     def __init__(self, buffer_size: int = 10000):
53         """
54         Initialize metrics collector
55
56         Args:
57             buffer_size: Maximum metrics to buffer before flush
58         """
59         self.metrics_buffer: deque = deque(maxlen=buffer_size)
60         self.counters: Dict[str, float] = {}
61         self.gauges: Dict[str, float] = {}
62         self.histograms: Dict[str, List[float]] = {}
63         self.logger = logging.getLogger("MetricsCollector")
64
65     def record_counter(self, name: str, value: float = 1,
66                       labels: Dict[str, str] = None):
67         """
68         Record a counter metric (monotonically increasing)
69
70         Args:
71             name: Metric name
72             value: Increment amount
73             labels: Metric labels
74         """
75         key = self._make_key(name, labels)
76         self.counters[key] = self.counters.get(key, 0) + value
77
78         self._buffer_metric(Metric(
79             name=name,
80             value=self.counters[key],
81             timestamp=datetime.now(),
82             labels=labels or {},
83             metric_type=MetricType.COUNTER
84         ))
85
86     def record_gauge(self, name: str, value: float,
87                     labels: Dict[str, str] = None):
88         """
89         Record a gauge metric (can go up or down)
90
91         Args:
92             name: Metric name

```

```

93         value: Current value
94         labels: Metric labels
95     """
96     key = self._make_key(name, labels)
97     self.gauges[key] = value
98
99     self._buffer_metric(Metric(
100         name=name,
101         value=value,
102         timestamp=datetime.now(),
103         labels=labels or {},
104         metric_type=MetricType.GAUGE
105     ))
106
107 def record_histogram(self, name: str, value: float,
108                     labels: Dict[str, str] = None):
109     """
110     Record a histogram metric (distribution of values)
111
112     Args:
113         name: Metric name
114         value: Observed value
115         labels: Metric labels
116     """
117     key = self._make_key(name, labels)
118     if key not in self.histograms:
119         self.histograms[key] = []
120
121     self.histograms[key].append(value)
122
123     self._buffer_metric(Metric(
124         name=name,
125         value=value,
126         timestamp=datetime.now(),
127         labels=labels or {},
128         metric_type=MetricType.HISTOGRAM
129     ))
130
131 def get_metrics(self, name: str = None,
132                start_time: datetime = None,
133                end_time: datetime = None) -> List[Metric]:
134     """
135     Retrieve metrics from buffer
136
137     Args:
138         name: Filter by metric name
139         start_time: Filter by start time
140         end_time: Filter by end time
141
142     Returns:
143         Filtered list of metrics
144     """
145     metrics = list(self.metrics_buffer)
146
147     if name:
148         metrics = [m for m in metrics if m.name == name]
149
150     if start_time:

```

```

151         metrics = [m for m in metrics if m.timestamp >= start_time]
152
153     if end_time:
154         metrics = [m for m in metrics if m.timestamp <= end_time]
155
156     return metrics
157
158     def get_histogram_percentiles(self, name: str,
159                                  labels: Dict[str, str] = None,
160                                  percentiles: List[float] = [50, 95, 99]) -> Dict[
161 float, float]:
162         """
163         Calculate histogram percentiles
164
165         Args:
166             name: Metric name
167             labels: Metric labels
168             percentiles: Percentiles to calculate
169
170         Returns:
171             Dictionary of percentile values
172         """
173         key = self._make_key(name, labels)
174         values = self.histograms.get(key, [])
175
176         if not values:
177             return {p: 0.0 for p in percentiles}
178
179         return {
180             p: np.percentile(values, p)
181             for p in percentiles
182         }
183
184     def _make_key(self, name: str, labels: Dict[str, str] = None) -> str:
185         """Create unique key for metric"""
186         if not labels:
187             return name
188
189         label_str = ",".join(f"{k}={v}" for k, v in sorted(labels.items()))
190         return f"{name}{{{label_str}}}"
191
192     def _buffer_metric(self, metric: Metric):
193         """Add metric to buffer"""
194         self.metrics_buffer.append(metric)
195
196     def flush(self) -> List[Metric]:
197         """
198         Flush all buffered metrics
199
200         Returns:
201             All buffered metrics
202         """
203         metrics = list(self.metrics_buffer)
204         self.metrics_buffer.clear()
205         return metrics
206
207 class MonitoringArchitecture:
208     """

```

```

208 Complete ML monitoring architecture
209
210 Coordinates metrics collection, storage, and alerting
211 """
212
213 def __init__(self,
214               metrics_collector: MetricsCollector,
215               storage_backend: Optional[Any] = None):
216     """
217     Initialize monitoring architecture
218
219     Args:
220         metrics_collector: Metrics collector instance
221         storage_backend: Backend for metric storage (e.g., Prometheus)
222     """
223     self.metrics = metrics_collector
224     self.storage = storage_backend
225     self.alerts: Dict[str, Alert] = {}
226     self.logger = logging.getLogger("MonitoringArchitecture")
227
228     # Standard ML metrics
229     self._register_standard_metrics()
230
231 def _register_standard_metrics(self):
232     """Register standard ML metrics"""
233     self.standard_metrics = {
234         'prediction_count': MetricType.COUNTER,
235         'prediction_latency_ms': MetricType.HISTOGRAM,
236         'model_accuracy': MetricType.GAUGE,
237         'data_drift_score': MetricType.GAUGE,
238         'feature_null_count': MetricType.COUNTER,
239         'prediction_confidence': MetricType.HISTOGRAM,
240         'error_count': MetricType.COUNTER,
241         'cache_hit_rate': MetricType.GAUGE
242     }
243
244 def record_prediction(self,
245                      model_name: str,
246                      model_version: str,
247                      features: Dict[str, Any],
248                      prediction: Any,
249                      confidence: float,
250                      latency_ms: float):
251     """
252     Record a model prediction with all associated metrics
253
254     Args:
255         model_name: Name of model
256         model_version: Model version
257         features: Input features
258         prediction: Model prediction
259         confidence: Prediction confidence
260         latency_ms: Prediction latency
261     """
262     labels = {
263         'model_name': model_name,
264         'model_version': model_version
265     }

```

```

266
267     # Record prediction count
268     self.metrics.record_counter(
269         'prediction_count',
270         value=1,
271         labels=labels
272     )
273
274     # Record latency
275     self.metrics.record_histogram(
276         'prediction_latency_ms',
277         value=latency_ms,
278         labels=labels
279     )
280
281     # Record confidence
282     self.metrics.record_histogram(
283         'prediction_confidence',
284         value=confidence,
285         labels=labels
286     )
287
288     # Check for null features
289     null_count = sum(1 for v in features.values() if v is None)
290     if null_count > 0:
291         self.metrics.record_counter(
292             'feature_null_count',
293             value=null_count,
294             labels=labels
295         )
296
297     def record_ground_truth(self,
298                             model_name: str,
299                             model_version: str,
300                             prediction: Any,
301                             actual: Any,
302                             metric_value: float):
303
304         """
305         Record ground truth and calculate accuracy
306
307         Args:
308             model_name: Name of model
309             model_version: Model version
310             prediction: Model prediction
311             actual: Actual value
312             metric_value: Calculated metric (e.g., accuracy, error)
313         """
314         labels = {
315             'model_name': model_name,
316             'model_version': model_version
317         }
318
319         # Record metric
320         self.metrics.record_gauge(
321             'model_accuracy',
322             value=metric_value,
323             labels=labels

```

```

324
325     # Record if prediction was correct
326     correct = 1 if prediction == actual else 0
327     self.metrics.record_counter(
328         'correct_predictions',
329         value=correct,
330         labels=labels
331     )
332
333     def register_alert(self, alert: Alert):
334         """
335         Register an alert condition
336
337         Args:
338             alert: Alert configuration
339         """
340         self.alerts[alert.alert_id] = alert
341         self.logger.info(f"Registered alert: {alert.name}")
342
343     def check_alerts(self) -> List[Dict]:
344         """
345         Check all registered alerts
346
347         Returns:
348             List of triggered alerts
349         """
350         triggered_alerts = []
351
352         for alert_id, alert in self.alerts.items():
353             # Check cooldown
354             if alert.last_triggered:
355                 time_since = datetime.now() - alert.last_triggered
356                 if time_since.total_seconds() < alert.cooldown_minutes * 60:
357                     continue
358
359             # Evaluate condition
360             try:
361                 if alert.condition(self.metrics):
362                     alert.last_triggered = datetime.now()
363
364                     triggered_alerts.append({
365                         'alert_id': alert_id,
366                         'name': alert.name,
367                         'severity': alert.severity.value,
368                         'message': alert.message_template,
369                         'timestamp': datetime.now().isoformat()
370                     })
371
372                     self.logger.warning(
373                         f"Alert triggered: {alert.name} ({alert.severity.value})"
374                     )
375
376             except Exception as e:
377                 self.logger.error(f"Alert check failed for {alert.name}: {str(e)}")
378
379         return triggered_alerts
380
381     def get_health_status(self) -> Dict:

```



```

382     """
383     Get overall system health
384
385     Returns:
386         Health status summary
387     """
388     # Get recent metrics
389     recent_time = datetime.now() - timedelta(minutes=5)
390     recent_metrics = self.metrics.get_metrics(start_time=recent_time)
391
392     # Calculate health indicators
393     total_predictions = sum(
394         1 for m in recent_metrics
395         if m.name == 'prediction_count'
396     )
397
398     total_errors = sum(
399         m.value for m in recent_metrics
400         if m.name == 'error_count'
401     )
402
403     error_rate = total_errors / total_predictions if total_predictions > 0 else 0
404
405     # Determine status
406     if error_rate > 0.1:
407         status = "unhealthy"
408     elif error_rate > 0.05:
409         status = "degraded"
410     else:
411         status = "healthy"
412
413     return {
414         'status': status,
415         'total_predictions': total_predictions,
416         'total_errors': total_errors,
417         'error_rate': error_rate,
418         'timestamp': datetime.now().isoformat()
419     }
420
421 # Example usage
422 def example_monitoring_architecture():
423     """Example of monitoring architecture setup"""
424
425     # Initialize components
426     collector = MetricsCollector(buffer_size=10000)
427     monitoring = MonitoringArchitecture(collector)
428
429     # Simulate predictions
430     for i in range(100):
431         monitoring.record_prediction(
432             model_name='fraud_detector',
433             model_version='v2.0',
434             features={'amount': 100.0, 'merchant': 'store_123'},
435             prediction='legitimate',
436             confidence=0.95,
437             latency_ms=np.random.normal(50, 10)
438         )
439

```

```

440 # Register alert for high latency
441 def high_latency_check(metrics: MetricsCollector) -> bool:
442     percentiles = metrics.get_histogram_percentiles(
443         'prediction_latency_ms',
444         labels={'model_name': 'fraud_detector'},
445         percentiles=[95]
446     )
447     return percentiles[95] > 100 # Alert if p95 > 100ms
448
449 monitoring.register_alert(Alert(
450     alert_id='high_latency',
451     name='High Prediction Latency',
452     condition=high_latency_check,
453     severity=AlertSeverity.WARNING,
454     message_template='Prediction latency p95 exceeded 100ms',
455     cooldown_minutes=15
456 ))
457
458 # Check alerts
459 triggered = monitoring.check_alerts()
460 print(f"Triggered alerts: {len(triggered)}")
461
462 # Get health status
463 health = monitoring.get_health_status()
464 print(f"System health: {health['status']}")
465 print(f"Predictions: {health['total_predictions']}")
466 print(f"Error rate: {health['error_rate']:.2%}")
467
468 example_monitoring_architecture()

```

Listing 14.1: ML Monitoring Architecture

14.4 Data Drift Detection

14.4.1 Types of Drift

Data drift occurs when the statistical properties of production data diverge from training data, degrading model performance.

Drift categories:

- **Covariate drift:** Input feature distributions change ($P(X)$ changes)
- **Concept drift:** Relationship between features and target changes ($P(Y|X)$ changes)
- **Label drift:** Target distribution changes ($P(Y)$ changes)
- **Prediction drift:** Model predictions distribution changes

14.4.2 Drift Detection with Evidently and Custom Methods

```

1 from dataclasses import dataclass
2 from typing import Dict, List, Tuple
3 import logging

```

```

4 import pandas as pd
5 import numpy as np
6 from scipy import stats
7 from scipy.spatial.distance import jensenshannon
8 import warnings
9
10 @dataclass
11 class DriftResult:
12     """Result of drift detection"""
13     feature_name: str
14     drift_detected: bool
15     drift_score: float
16     p_value: float
17     method: str
18     threshold: float
19     reference_stats: Dict[str, float]
20     current_stats: Dict[str, float]
21
22 class DriftDetector:
23     """
24     Comprehensive drift detection system
25
26     Implements multiple statistical tests for drift detection
27     """
28
29     def __init__(self, significance_level: float = 0.05):
30         """
31         Initialize drift detector
32
33         Args:
34             significance_level: Statistical significance threshold
35         """
36         self.significance_level = significance_level
37         self.logger = logging.getLogger("DriftDetector")
38
39     def detect_drift_ks(self,
40                       reference_data: pd.Series,
41                       current_data: pd.Series) -> DriftResult:
42         """
43         Kolmogorov-Smirnov test for continuous features
44
45         Args:
46             reference_data: Training/reference data
47             current_data: Current production data
48
49         Returns:
50             Drift detection result
51         """
52         # Perform KS test
53         statistic, p_value = stats.ks_2samp(reference_data, current_data)
54
55         drift_detected = p_value < self.significance_level
56
57         # Calculate statistics
58         ref_stats = {
59             'mean': float(reference_data.mean()),
60             'std': float(reference_data.std()),
61             'min': float(reference_data.min()),

```

```

62         'max': float(reference_data.max()),
63         'median': float(reference_data.median())
64     }
65
66     curr_stats = {
67         'mean': float(current_data.mean()),
68         'std': float(current_data.std()),
69         'min': float(current_data.min()),
70         'max': float(current_data.max()),
71         'median': float(current_data.median())
72     }
73
74     return DriftResult(
75         feature_name=reference_data.name if hasattr(reference_data, 'name') else '
unnamed',
76         drift_detected=drift_detected,
77         drift_score=statistic,
78         p_value=p_value,
79         method='Kolmogorov-Smirnov',
80         threshold=self.significance_level,
81         reference_stats=ref_stats,
82         current_stats=curr_stats
83     )
84
85     def detect_drift_chi2(self,
86                           reference_data: pd.Series,
87                           current_data: pd.Series,
88                           bins: int = 10) -> DriftResult:
89         """
90         Chi-squared test for categorical or binned continuous features
91
92         Args:
93             reference_data: Training/reference data
94             current_data: Current production data
95             bins: Number of bins for continuous data
96
97         Returns:
98             Drift detection result
99         """
100         # Create contingency table
101         if reference_data.dtype in ['object', 'category']:
102             # Categorical data
103             ref_counts = reference_data.value_counts()
104             curr_counts = current_data.value_counts()
105
106             # Align categories
107             all_categories = set(ref_counts.index) | set(curr_counts.index)
108             ref_freq = [ref_counts.get(cat, 0) for cat in all_categories]
109             curr_freq = [curr_counts.get(cat, 0) for cat in all_categories]
110
111         else:
112             # Continuous data - bin it
113             combined = pd.concat([reference_data, current_data])
114             bins_edges = pd.cut(combined, bins=bins, retbins=True)[1]
115
116             ref_freq, _ = np.histogram(reference_data, bins=bins_edges)
117             curr_freq, _ = np.histogram(current_data, bins=bins_edges)
118

```

```

119     # Normalize to frequencies
120     ref_freq = np.array(ref_freq) / np.sum(ref_freq)
121     curr_freq = np.array(curr_freq) / np.sum(curr_freq)
122
123     # Chi-squared test
124     # Add small constant to avoid division by zero
125     expected = (ref_freq + 1e-10) * len(current_data)
126     observed = (curr_freq + 1e-10) * len(current_data)
127
128     chi2_stat = np.sum((observed - expected) ** 2 / expected)
129     dof = len(ref_freq) - 1
130     p_value = 1 - stats.chi2.cdf(chi2_stat, dof)
131
132     drift_detected = p_value < self.significance_level
133
134     ref_stats = {
135         'unique_values': len(reference_data.unique()),
136         'mode': reference_data.mode()[0] if len(reference_data.mode()) > 0 else
None
137     }
138
139     curr_stats = {
140         'unique_values': len(current_data.unique()),
141         'mode': current_data.mode()[0] if len(current_data.mode()) > 0 else None
142     }
143
144     return DriftResult(
145         feature_name=reference_data.name if hasattr(reference_data, 'name') else '
unnamed',
146         drift_detected=drift_detected,
147         drift_score=chi2_stat,
148         p_value=p_value,
149         method='Chi-Squared',
150         threshold=self.significance_level,
151         reference_stats=ref_stats,
152         current_stats=curr_stats
153     )
154
155     def detect_drift_psi(self,
156         reference_data: pd.Series,
157         current_data: pd.Series,
158         bins: int = 10,
159         psi_threshold: float = 0.2) -> DriftResult:
160
161         """
162         Population Stability Index (PSI) for drift detection
163
164         Args:
165             reference_data: Training/reference data
166             current_data: Current production data
167             bins: Number of bins
168             psi_threshold: PSI threshold (0.1=small, 0.2=medium, 0.25+=large shift)
169
170         Returns:
171             Drift detection result
172         """
173         # Create bins based on reference data
174         if reference_data.dtype in ['object', 'category']:
175             # Categorical

```

```

175     ref_counts = reference_data.value_counts(normalize=True)
176     curr_counts = current_data.value_counts(normalize=True)
177
178     all_categories = set(ref_counts.index) | set(curr_counts.index)
179     ref_pct = np.array([ref_counts.get(cat, 1e-10) for cat in all_categories])
180     curr_pct = np.array([curr_counts.get(cat, 1e-10) for cat in all_categories
181 ])
182
183     else:
184         # Continuous - create bins
185         bins_edges = pd.qcut(reference_data, q=bins, duplicates='drop', retbins=
186 True)[1]
187
188         ref_binned = pd.cut(reference_data, bins=bins_edges, include_lowest=True)
189         curr_binned = pd.cut(current_data, bins=bins_edges, include_lowest=True)
190
191         ref_pct = ref_binned.value_counts(normalize=True, sort=False).values
192         curr_pct = curr_binned.value_counts(normalize=True, sort=False).values
193
194         # Calculate PSI
195         # PSI = sum((curr_pct - ref_pct) * ln(curr_pct / ref_pct))
196         psi = np.sum((curr_pct - ref_pct) * np.log(curr_pct / ref_pct))
197
198         drift_detected = abs(psi) > psi_threshold
199
200         return DriftResult(
201             feature_name=reference_data.name if hasattr(reference_data, 'name') else '
202 unnamed',
203             drift_detected=drift_detected,
204             drift_score=abs(psi),
205             p_value=0.0, # PSI doesn't have p-value
206             method='PSI',
207             threshold=psi_threshold,
208             reference_stats={'mean': float(reference_data.mean())},
209             current_stats={'mean': float(current_data.mean())}
210         )
211
212     def detect_drift_jensen_shannon(self,
213                                     reference_data: pd.Series,
214                                     current_data: pd.Series,
215                                     bins: int = 50) -> DriftResult:
216
217         """
218         Jensen-Shannon divergence for drift detection
219
220         Args:
221             reference_data: Training/reference data
222             current_data: Current production data
223             bins: Number of bins for histogram
224
225         Returns:
226             Drift detection result
227         """
228         # Create histograms
229         if reference_data.dtype in ['object', 'category']:
230             ref_counts = reference_data.value_counts(normalize=True)
231             curr_counts = current_data.value_counts(normalize=True)
232
233             all_categories = sorted(set(ref_counts.index) | set(curr_counts.index))

```

```

230         ref_dist = np.array([ref_counts.get(cat, 1e-10) for cat in all_categories
231 ])
232         curr_dist = np.array([curr_counts.get(cat, 1e-10) for cat in
233 all_categories])
234
235     else:
236         # Continuous data
237         combined = pd.concat([reference_data, current_data])
238         bins_edges = np.histogram_bin_edges(combined, bins=bins)
239
240         ref_hist, _ = np.histogram(reference_data, bins=bins_edges, density=True)
241         curr_hist, _ = np.histogram(current_data, bins=bins_edges, density=True)
242
243         ref_dist = ref_hist / (ref_hist.sum() + 1e-10)
244         curr_dist = curr_hist / (curr_hist.sum() + 1e-10)
245
246         # Calculate Jensen-Shannon divergence
247         js_divergence = jensenshannon(ref_dist, curr_dist)
248
249         # JS divergence ranges from 0 (identical) to 1 (completely different)
250         # Threshold: 0.1 = small drift, 0.3 = significant drift
251         drift_detected = js_divergence > 0.1
252
253     return DriftResult(
254         feature_name=reference_data.name if hasattr(reference_data, 'name') else '
unnamed',
255         drift_detected=drift_detected,
256         drift_score=js_divergence,
257         p_value=0.0, # JS doesn't have p-value
258         method='Jensen-Shannon',
259         threshold=0.1,
260         reference_stats={'mean': float(reference_data.mean())} if reference_data.
dtype != 'object' else {},
261         current_stats={'mean': float(current_data.mean())} if current_data.dtype
!= 'object' else {}
262     )
263
264 class DataDriftMonitor:
265     """
266     Monitor data drift across all features
267
268     Provides comprehensive drift monitoring for ML systems
269     """
270
271     def __init__(self,
272                 reference_data: pd.DataFrame,
273                 detector: DriftDetector = None):
274         """
275         Initialize drift monitor
276
277         Args:
278             reference_data: Reference/training dataset
279             detector: Drift detector (creates default if None)
280         """
281         self.reference_data = reference_data
282         self.detector = detector or DriftDetector()
283         self.drift_history: List[Dict] = []
284         self.logger = logging.getLogger("DataDriftMonitor")

```

```

283
284 def check_drift(self,
285                 current_data: pd.DataFrame,
286                 method: str = 'auto') -> Dict[str, DriftResult]:
287     """
288     Check drift for all features
289
290     Args:
291         current_data: Current production data
292         method: Detection method ('ks', 'chi2', 'psi', 'js', 'auto')
293
294     Returns:
295         Dictionary of drift results by feature
296     """
297     drift_results = {}
298
299     for column in self.reference_data.columns:
300         if column not in current_data.columns:
301             self.logger.warning(f"Column {column} not in current data")
302             continue
303
304         ref_col = self.reference_data[column]
305         curr_col = current_data[column]
306
307         # Auto-select method based on data type
308         if method == 'auto':
309             if ref_col.dtype in ['object', 'category']:
310                 detection_method = 'chi2'
311             else:
312                 detection_method = 'ks'
313         else:
314             detection_method = method
315
316         # Detect drift
317         try:
318             if detection_method == 'ks':
319                 result = self.detector.detect_drift_ks(ref_col, curr_col)
320             elif detection_method == 'chi2':
321                 result = self.detector.detect_drift_chi2(ref_col, curr_col)
322             elif detection_method == 'psi':
323                 result = self.detector.detect_drift_psi(ref_col, curr_col)
324             elif detection_method == 'js':
325                 result = self.detector.detect_drift_jensen_shannon(ref_col,
326                             curr_col)
327             else:
328                 raise ValueError(f"Unknown method: {detection_method}")
329
330             drift_results[column] = result
331
332             if result.drift_detected:
333                 self.logger.warning(
334                     f"Drift detected in {column}: "
335                     f"score={result.drift_score:.4f}, p-value={result.p_value:.4f}"
336                     "
337                 )
338         except Exception as e:
339             self.logger.error(f"Drift detection failed for {column}: {str(e)}")

```



```

339         # Store in history
340         self.drift_history.append({
341             'timestamp': datetime.now(),
342             'results': drift_results,
343             'drift_count': sum(1 for r in drift_results.values() if r.drift_detected)
344         })
345
346         return drift_results
347
348
349     def get_drift_report(self) -> pd.DataFrame:
350         """
351         Generate drift report
352
353         Returns:
354             DataFrame with drift summary
355         """
356         if not self.drift_history:
357             return pd.DataFrame()
358
359         latest = self.drift_history[-1]
360         results = latest['results']
361
362         report_data = []
363         for feature, result in results.items():
364             report_data.append({
365                 'feature': feature,
366                 'drift_detected': result.drift_detected,
367                 'drift_score': result.drift_score,
368                 'p_value': result.p_value,
369                 'method': result.method,
370                 'ref_mean': result.reference_stats.get('mean', None),
371                 'curr_mean': result.current_stats.get('mean', None),
372                 'mean_change_pct': (
373                     ((result.current_stats.get('mean', 0) - result.reference_stats.get(
374                         'mean', 0)) /
375                      abs(result.reference_stats.get('mean', 1))) * 100
376                     if result.reference_stats.get('mean') else None
377                 )
378             })
379
380         return pd.DataFrame(report_data)
381
382     # Integration with Evidently AI
383     class EvidentlyDriftMonitor:
384         """
385         Drift monitoring using Evidently AI library
386
387         Provides advanced drift detection and reporting
388         """
389
390         def __init__(self, reference_data: pd.DataFrame):
391             """
392             Initialize Evidently monitor
393
394             Args:
395                 reference_data: Reference dataset
396             """

```

```

396     self.reference_data = reference_data
397     self.logger = logging.getLogger("EvidentlyMonitor")
398
399     def generate_drift_report(self,
400                             current_data: pd.DataFrame,
401                             column_mapping: Dict = None) -> Dict:
402         """
403         Generate comprehensive drift report using Evidently
404
405         Args:
406             current_data: Current production data
407             column_mapping: Column type mapping
408
409         Returns:
410             Drift report dictionary
411         """
412         try:
413             from evidently.report import Report
414             from evidently.metrics import DataDriftTable, DatasetDriftMetric
415
416             # Create report
417             report = Report(metrics=[
418                 DataDriftTable(),
419                 DatasetDriftMetric()
420             ])
421
422             # Run report
423             report.run(
424                 reference_data=self.reference_data,
425                 current_data=current_data,
426                 column_mapping=column_mapping
427             )
428
429             # Extract results
430             report_dict = report.as_dict()
431
432             return {
433                 'dataset_drift': report_dict['metrics'][1]['result']['dataset_drift'],
434                 'drift_by_columns': report_dict['metrics'][0]['result']['drift_by_columns'],
435                 'number_of_drifted_columns': report_dict['metrics'][1]['result']['number_of_drifted_columns']
436             }
437
438             except ImportError:
439                 self.logger.warning("Evidently not installed. Install with: pip install evidently")
440                 return {}
441             except Exception as e:
442                 self.logger.error(f"Evidently report generation failed: {str(e)}")
443                 return {}
444
445         # Example usage
446         def example_drift_detection():
447             """Example of drift detection"""
448
449             # Create reference data
450             np.random.seed(42)

```

```

451     n_samples = 1000
452
453     reference_df = pd.DataFrame({
454         'feature_1': np.random.normal(100, 15, n_samples),
455         'feature_2': np.random.exponential(50, n_samples),
456         'feature_3': np.random.choice(['A', 'B', 'C'], n_samples, p=[0.5, 0.3, 0.2]),
457         'feature_4': np.random.uniform(0, 1, n_samples)
458     })
459
460     # Create current data with drift in feature_1 and feature_3
461     current_df = pd.DataFrame({
462         'feature_1': np.random.normal(110, 15, n_samples), # Mean shifted
463         'feature_2': np.random.exponential(50, n_samples), # No drift
464         'feature_3': np.random.choice(['A', 'B', 'C'], n_samples, p=[0.3, 0.4, 0.3]),
465         # Distribution changed
466         'feature_4': np.random.uniform(0, 1, n_samples) # No drift
467     })
468
469     # Initialize monitor
470     monitor = DataDriftMonitor(reference_df)
471
472     # Check drift
473     drift_results = monitor.check_drift(current_df, method='auto')
474
475     # Print results
476     print("Drift Detection Results:")
477     print("=" * 70)
478     for feature, result in drift_results.items():
479         status = "DRIFT DETECTED" if result.drift_detected else "No drift"
480         print(f"\n{feature}:")
481         print(f"    Status: {status}")
482         print(f"    Method: {result.method}")
483         print(f"    Score: {result.drift_score:.4f}")
484         print(f"    P-value: {result.p_value:.4f}")
485         print(f"    Reference mean: {result.reference_stats.get('mean', 'N/A')}")
486         print(f"    Current mean: {result.current_stats.get('mean', 'N/A')}")
487
488     # Generate report
489     report = monitor.get_drift_report()
490     print("\n\nDrift Report:")
491     print(report.to_string())
492
493 example_drift_detection()

```

Listing 14.2: Comprehensive Drift Detection System

14.5 Model Performance Monitoring

14.5.1 Performance Metrics Tracking

Continuous monitoring of model performance ensures early detection of degradation. Unlike training metrics calculated once, production metrics must be tracked continuously as new data arrives.

Key performance metrics:

- **Classification:** Accuracy, precision, recall, F1-score, AUC-ROC


```

50     """
51     Record a prediction for later evaluation
52
53     Args:
54         model_name: Model name
55         model_version: Model version
56         prediction_id: Unique prediction ID
57         features: Input features
58         prediction: Model prediction
59         timestamp: Prediction timestamp
60     """
61     self.predictions_buffer.append({
62         'prediction_id': prediction_id,
63         'model_name': model_name,
64         'model_version': model_version,
65         'prediction': prediction,
66         'features': features,
67         'timestamp': timestamp or datetime.now()
68     })
69
70     def record_ground_truth(self,
71                             prediction_id: str,
72                             actual: Any,
73                             timestamp: datetime = None):
74         """
75         Record ground truth for a prediction
76
77         Args:
78             prediction_id: Prediction ID
79             actual: Actual value
80             timestamp: Ground truth timestamp
81         """
82         self.ground_truth_buffer.append({
83             'prediction_id': prediction_id,
84             'actual': actual,
85             'timestamp': timestamp or datetime.now()
86         })
87
88         # Try to evaluate
89         self._evaluate_if_ready()
90
91     def _evaluate_if_ready(self):
92         """Evaluate performance if enough ground truth available"""
93         # Match predictions with ground truth
94         matched_pairs = []
95
96         pred_dict = {p['prediction_id']: p for p in self.predictions_buffer}
97         gt_dict = {g['prediction_id']: g for g in self.ground_truth_buffer}
98
99         for pred_id in pred_dict.keys() & gt_dict.keys():
100             pred = pred_dict[pred_id]
101             gt = gt_dict[pred_id]
102
103             # Check if within window
104             time_diff = abs((pred['timestamp'] - gt['timestamp']).total_seconds() /
105                             3600)
106             if time_diff <= self.window_hours:
107                 matched_pairs.append({

```

```

107         'model_name': pred['model_name'],
108         'model_version': pred['model_version'],
109         'prediction': pred['prediction'],
110         'actual': gt['actual'],
111         'timestamp': pred['timestamp']
112     })
113
114     # If enough matched pairs, calculate metrics
115     if len(matched_pairs) >= 100: # Minimum sample size
116         self._calculate_performance_metrics(matched_pairs)
117
118         # Clear old buffers
119         cutoff_time = datetime.now() - timedelta(hours=self.window_hours * 2)
120         self.predictions_buffer = [
121             p for p in self.predictions_buffer
122             if p['timestamp'] > cutoff_time
123         ]
124         self.ground_truth_buffer = [
125             g for g in self.ground_truth_buffer
126             if g['timestamp'] > cutoff_time
127         ]
128
129     def _calculate_performance_metrics(self, matched_pairs: List[Dict]):
130         """Calculate performance metrics from matched pairs"""
131         if not matched_pairs:
132             return
133
134         # Group by model
135         from collections import defaultdict
136         by_model = defaultdict(list)
137
138         for pair in matched_pairs:
139             key = (pair['model_name'], pair['model_version'])
140             by_model[key].append(pair)
141
142         # Calculate metrics for each model
143         for (model_name, model_version), pairs in by_model.items():
144             predictions = [p['prediction'] for p in pairs]
145             actuals = [p['actual'] for p in pairs]
146
147             # Detect task type
148             is_classification = isinstance(predictions[0], (str, int, bool))
149
150             if is_classification:
151                 metrics = self._calculate_classification_metrics(
152                     predictions, actuals
153                 )
154             else:
155                 metrics = self._calculate_regression_metrics(
156                     predictions, actuals
157                 )
158
159         # Create snapshot
160         snapshot = PerformanceSnapshot(
161             timestamp=datetime.now(),
162             model_name=model_name,
163             model_version=model_version,
164             metrics=metrics,

```

```

165         sample_count=len(pairs),
166         evaluation_window=f"{self.window_hours}h"
167     )
168
169     self.performance_history.append(snapshot)
170
171     # Log metrics
172     labels = {
173         'model_name': model_name,
174         'model_version': model_version
175     }
176
177     for metric_name, value in metrics.items():
178         self.metrics.record_gauge(
179             f'model_{metric_name}',
180             value=value,
181             labels=labels
182         )
183
184     self.logger.info(
185         f"Performance calculated for {model_name} v{model_version}: {metrics}"
186     )
187
188     def _calculate_classification_metrics(self,
189                                         predictions: List,
190                                         actuals: List) -> Dict[str, float]:
191         """Calculate classification metrics"""
192         try:
193             return {
194                 'accuracy': accuracy_score(actuals, predictions),
195                 'precision': precision_score(actuals, predictions, average='weighted',
196                 zero_division=0),
197                 'recall': recall_score(actuals, predictions, average='weighted',
198                 zero_division=0),
199                 'f1_score': f1_score(actuals, predictions, average='weighted',
200                 zero_division=0)
201             }
202         except Exception as e:
203             self.logger.error(f"Classification metrics calculation failed: {str(e)}")
204             return {}
205
206     def _calculate_regression_metrics(self,
207                                     predictions: List,
208                                     actuals: List) -> Dict[str, float]:
209         """Calculate regression metrics"""
210         try:
211             predictions = np.array(predictions)
212             actuals = np.array(actuals)
213
214             mae = mean_absolute_error(actuals, predictions)
215             mse = mean_squared_error(actuals, predictions)
216             rmse = np.sqrt(mse)
217
218             # R-squared
219             ss_res = np.sum((actuals - predictions) ** 2)
220             ss_tot = np.sum((actuals - np.mean(actuals)) ** 2)
221             r2 = 1 - (ss_res / ss_tot) if ss_tot != 0 else 0

```

```

220         return {
221             'mae': mae,
222             'mse': mse,
223             'rmse': rmse,
224             'r_squared': r2
225         }
226     except Exception as e:
227         self.logger.error(f"Regression metrics calculation failed: {str(e)}")
228         return {}
229
230     def get_performance_trend(self,
231                             model_name: str,
232                             metric_name: str,
233                             window_snapshots: int = 10) -> List[float]:
234         """
235         Get performance trend for a metric
236
237         Args:
238             model_name: Model name
239             metric_name: Metric to track
240             window_snapshots: Number of recent snapshots
241
242         Returns:
243             List of metric values over time
244         """
245         relevant_snapshots = [
246             s for s in self.performance_history[-window_snapshots:]
247             if s.model_name == model_name and metric_name in s.metrics
248         ]
249
250         return [s.metrics[metric_name] for s in relevant_snapshots]
251
252     def detect_performance_degradation(self,
253                                     model_name: str,
254                                     metric_name: str,
255                                     threshold_pct: float = 5.0) -> bool:
256         """
257         Detect if performance has degraded
258
259         Args:
260             model_name: Model name
261             metric_name: Metric to check
262             threshold_pct: Degradation threshold (percentage)
263
264         Returns:
265             True if degradation detected
266         """
267         trend = self.get_performance_trend(model_name, metric_name, window_snapshots
268                                           =20)
269
270         if len(trend) < 5:
271             return False
272
273         # Compare recent vs historical
274         recent_avg = np.mean(trend[-5:])
275         historical_avg = np.mean(trend[:-5])
276
277         pct_change = ((recent_avg - historical_avg) / abs(historical_avg)) * 100

```



```

277     # For metrics where higher is better (accuracy, f1, r2)
278     better_higher = metric_name in ['accuracy', 'f1_score', 'precision', 'recall',
279     'r_squared']
280
281     if better_higher:
282         return pct_change < -threshold_pct
283     else:
284         return pct_change > threshold_pct
285
286 # Example usage
287 def example_performance_monitoring():
288     """Example of performance monitoring"""
289
290     collector = MetricsCollector()
291     perf_monitor = PerformanceMonitor(collector, performance_window_hours=24)
292
293     # Simulate predictions and ground truth
294     np.random.seed(42)
295
296     for i in range(200):
297         pred_id = f"pred_{i}"
298
299         # Record prediction
300         features = {'feature_1': np.random.rand(), 'feature_2': np.random.rand()}
301         prediction = 1 if np.random.rand() > 0.5 else 0
302
303         perf_monitor.record_prediction(
304             model_name='classifier_model',
305             model_version='v1.0',
306             prediction_id=pred_id,
307             features=features,
308             prediction=prediction
309         )
310
311         # Simulate ground truth arriving later
312         # Model accuracy degrades over time (from 0.95 to 0.80)
313         accuracy = 0.95 - (i / 200) * 0.15
314         actual = prediction if np.random.rand() < accuracy else (1 - prediction)
315
316         perf_monitor.record_ground_truth(
317             prediction_id=pred_id,
318             actual=actual
319         )
320
321         # Check for degradation
322         degraded = perf_monitor.detect_performance_degradation(
323             model_name='classifier_model',
324             metric_name='accuracy',
325             threshold_pct=5.0
326         )
327
328         print(f"Performance degradation detected: {degraded}")
329
330         # Get trend
331         trend = perf_monitor.get_performance_trend(
332             model_name='classifier_model',
333             metric_name='accuracy',

```

```

334     window_snapshots=10
335 )
336
337 print(f"Accuracy trend: {[f'{t:.3f}' for t in trend]}")
338
339 example_performance_monitoring()

```

Listing 14.3: Model Performance Monitoring System

14.6 Prometheus and Grafana Integration

14.6.1 Prometheus Metrics Export

Prometheus is the de-facto standard for metrics collection in modern infrastructure. Integrating ML monitoring with Prometheus provides powerful querying, alerting, and visualization capabilities.

Integration benefits:

- **Time-series storage:** Efficient storage of metric history
- **PromQL:** Powerful query language for metric analysis
- **Service discovery:** Automatic discovery of monitoring targets
- **Alerting:** Built-in alert manager integration
- **Ecosystem:** Wide integration with visualization tools

14.6.2 Prometheus Integration Implementation

```

1  import logging
2  import threading
3  from prometheus_client import Counter, Gauge, Histogram, Summary
4  from prometheus_client import start_http_server, REGISTRY
5
6  class PrometheusExporter:
7      """
8      Export ML metrics to Prometheus
9
10     Provides Prometheus-compatible metrics endpoint
11     """
12
13     def __init__(self, port: int = 8000):
14         """
15         Initialize Prometheus exporter
16
17         Args:
18             port: Port for metrics endpoint
19         """
20         self.port = port
21         self.logger = logging.getLogger("PrometheusExporter")
22
23         # Create Prometheus metrics
24         self._setup_metrics()

```

```
25
26 def _setup_metrics(self):
27     """Setup Prometheus metrics"""
28     # Prediction counters
29     self.prediction_count = Counter(
30         'ml_prediction_total',
31         'Total number of predictions',
32         ['model_name', 'model_version']
33     )
34
35     self.prediction_errors = Counter(
36         'ml_prediction_errors_total',
37         'Total prediction errors',
38         ['model_name', 'model_version', 'error_type']
39     )
40
41     # Performance gauges
42     self.model_accuracy = Gauge(
43         'ml_model_accuracy',
44         'Model accuracy',
45         ['model_name', 'model_version']
46     )
47
48     self.model_precision = Gauge(
49         'ml_model_precision',
50         'Model precision',
51         ['model_name', 'model_version']
52     )
53
54     self.model_recall = Gauge(
55         'ml_model_recall',
56         'Model recall',
57         ['model_name', 'model_version']
58     )
59
60     self.model_f1 = Gauge(
61         'ml_model_f1_score',
62         'Model F1 score',
63         ['model_name', 'model_version']
64     )
65
66     # Drift metrics
67     self.drift_score = Gauge(
68         'ml_data_drift_score',
69         'Data drift score',
70         ['model_name', 'feature_name', 'drift_method']
71     )
72
73     # Latency histogram
74     self.prediction_latency = Histogram(
75         'ml_prediction_latency_seconds',
76         'Prediction latency in seconds',
77         ['model_name', 'model_version'],
78         buckets=[0.001, 0.005, 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1.0, 2.5, 5.0]
79     )
80
81     # Confidence summary
82     self.prediction_confidence = Summary(
```

```

83         'ml_prediction_confidence',
84         'Prediction confidence scores',
85         ['model_name', 'model_version']
86     )
87
88     # Feature statistics
89     self.feature_null_count = Counter(
90         'ml_feature_null_total',
91         'Total null features',
92         ['model_name', 'feature_name']
93     )
94
95     def record_prediction(self,
96                         model_name: str,
97                         model_version: str,
98                         latency_seconds: float,
99                         confidence: float):
100
101         """
102         Record prediction metrics
103
104         Args:
105             model_name: Model name
106             model_version: Model version
107             latency_seconds: Prediction latency
108             confidence: Prediction confidence
109
110         """
111         labels = {'model_name': model_name, 'model_version': model_version}
112
113         self.prediction_count.labels(**labels).inc()
114         self.prediction_latency.labels(**labels).observe(latency_seconds)
115         self.prediction_confidence.labels(**labels).observe(confidence)
116
117     def record_error(self,
118                    model_name: str,
119                    model_version: str,
120                    error_type: str):
121
122         """
123         Record prediction error
124
125         Args:
126             model_name: Model name
127             model_version: Model version
128             error_type: Type of error
129
130         """
131         self.prediction_errors.labels(
132             model_name=model_name,
133             model_version=model_version,
134             error_type=error_type
135         ).inc()
136
137     def update_performance_metrics(self,
138                                 model_name: str,
139                                 model_version: str,
140                                 metrics: Dict[str, float]):
141
142         """
143         Update model performance metrics
144
145         Args:

```

```

141         model_name: Model name
142         model_version: Model version
143         metrics: Performance metrics dictionary
144     """
145     labels = {'model_name': model_name, 'model_version': model_version}
146
147     if 'accuracy' in metrics:
148         self.model_accuracy.labels(**labels).set(metrics['accuracy'])
149
150     if 'precision' in metrics:
151         self.model_precision.labels(**labels).set(metrics['precision'])
152
153     if 'recall' in metrics:
154         self.model_recall.labels(**labels).set(metrics['recall'])
155
156     if 'f1_score' in metrics:
157         self.model_f1.labels(**labels).set(metrics['f1_score'])
158
159     def update_drift_metrics(self,
160                             model_name: str,
161                             drift_results: Dict[str, Any]):
162     """
163     Update drift metrics
164
165     Args:
166         model_name: Model name
167         drift_results: Drift detection results
168     """
169     for feature, result in drift_results.items():
170         self.drift_score.labels(
171             model_name=model_name,
172             feature_name=feature,
173             drift_method=result.method
174         ).set(result.drift_score)
175
176     def start_server(self):
177         """Start Prometheus metrics server"""
178         try:
179             start_http_server(self.port)
180             self.logger.info(f"Prometheus exporter started on port {self.port}")
181         except Exception as e:
182             self.logger.error(f"Failed to start Prometheus exporter: {str(e)}")
183
184     # Grafana Dashboard Configuration
185
186     class GrafanaDashboardGenerator:
187     """
188     Generate Grafana dashboard configurations for ML monitoring
189
190     Creates JSON dashboard definitions
191     """
192
193     def __init__(self):
194         """Initialize dashboard generator"""
195         self.logger = logging.getLogger("GrafanaDashboard")
196
197     def generate_ml_overview_dashboard(self) -> Dict:
198     """

```

```

199     Generate ML model overview dashboard
200
201     Returns:
202         Grafana dashboard JSON
203     """
204     dashboard = {
205         "dashboard": {
206             "title": "ML Model Monitoring Overview",
207             "tags": ["ml", "monitoring"],
208             "timezone": "browser",
209             "panels": [
210                 # Prediction rate panel
211                 {
212                     "id": 1,
213                     "title": "Prediction Rate",
214                     "type": "graph",
215                     "gridPos": {"x": 0, "y": 0, "w": 12, "h": 8},
216                     "targets": [{
217                         "expr": "rate(ml_prediction_total[5m])",
218                         "legendFormat": "{{model_name}} v{{model_version}}"
219                     }]
220                 },
221                 # Error rate panel
222                 {
223                     "id": 2,
224                     "title": "Error Rate",
225                     "type": "graph",
226                     "gridPos": {"x": 12, "y": 0, "w": 12, "h": 8},
227                     "targets": [{
228                         "expr": "rate(ml_prediction_errors_total[5m])",
229                         "legendFormat": "{{model_name}} - {{error_type}}"
230                     }]
231                 },
232                 # Model accuracy panel
233                 {
234                     "id": 3,
235                     "title": "Model Accuracy",
236                     "type": "graph",
237                     "gridPos": {"x": 0, "y": 8, "w": 12, "h": 8},
238                     "targets": [{
239                         "expr": "ml_model_accuracy",
240                         "legendFormat": "{{model_name}} v{{model_version}}"
241                     }],
242                     "yaxes": [
243                         {"format": "percentunit", "min": 0, "max": 1},
244                         {"format": "short"}
245                     ]
246                 },
247                 # Latency percentiles panel
248                 {
249                     "id": 4,
250                     "title": "Prediction Latency (p50, p95, p99)",
251                     "type": "graph",
252                     "gridPos": {"x": 12, "y": 8, "w": 12, "h": 8},
253                     "targets": [
254                         {
255                             "expr": "histogram_quantile(0.50, rate(
ml_prediction_latency_seconds_bucket[5m]))",

```

```

256         "legendFormat": "p50 - {{model_name}}"
257     },
258     {
259         "expr": "histogram_quantile(0.95, rate(
ml_prediction_latency_seconds_bucket[5m]))",
260         "legendFormat": "p95 - {{model_name}}"
261     },
262     {
263         "expr": "histogram_quantile(0.99, rate(
ml_prediction_latency_seconds_bucket[5m]))",
264         "legendFormat": "p99 - {{model_name}}"
265     }
266 ]
267 },
268 # Data drift panel
269 {
270     "id": 5,
271     "title": "Data Drift Scores",
272     "type": "graph",
273     "gridPos": {"x": 0, "y": 16, "w": 24, "h": 8},
274     "targets": [{
275         "expr": "ml_data_drift_score",
276         "legendFormat": "{{model_name}} - {{feature_name}}"
277     }],
278     "alert": {
279         "conditions": [{
280             "evaluator": {
281                 "params": [0.2],
282                 "type": "gt"
283             },
284             "operator": {"type": "and"},
285             "query": {"params": ["A", "5m", "now"]},
286             "reducer": {"params": [], "type": "avg"},
287             "type": "query"
288         }]
289     }
290 },
291 ],
292 "refresh": "30s",
293 "time": {"from": "now-1h", "to": "now"}
294 }
295 }
296
297 return dashboard
298
299 def generate_performance_dashboard(self) -> Dict:
300     """
301     Generate detailed performance monitoring dashboard
302
303     Returns:
304         Grafana dashboard JSON
305     """
306     dashboard = {
307         "dashboard": {
308             "title": "ML Model Performance Metrics",
309             "tags": ["ml", "performance"],
310             "panels": [
311                 {

```

```

312         "id": 1,
313         "title": "Accuracy Over Time",
314         "type": "graph",
315         "gridPos": {"x": 0, "y": 0, "w": 12, "h": 8},
316         "targets": [{
317             "expr": "ml_model_accuracy",
318             "legendFormat": "{{model_name}} v{{model_version}}"
319         }]
320     },
321     {
322         "id": 2,
323         "title": "Precision vs Recall",
324         "type": "graph",
325         "gridPos": {"x": 12, "y": 0, "w": 12, "h": 8},
326         "targets": [
327             {
328                 "expr": "ml_model_precision",
329                 "legendFormat": "Precision - {{model_name}}"
330             },
331             {
332                 "expr": "ml_model_recall",
333                 "legendFormat": "Recall - {{model_name}}"
334             }
335         ]
336     },
337     {
338         "id": 3,
339         "title": "F1 Score",
340         "type": "stat",
341         "gridPos": {"x": 0, "y": 8, "w": 6, "h": 4},
342         "targets": [{
343             "expr": "ml_model_f1_score",
344             "legendFormat": "{{model_name}}"
345         }],
346         "options": {
347             "graphMode": "area",
348             "colorMode": "value"
349         }
350     }
351 ]
352 }
353 }
354
355 return dashboard
356
357 # Example usage
358 def example_prometheus_integration():
359     """Example of Prometheus integration"""
360
361     # Initialize exporter
362     exporter = PrometheusExporter(port=8000)
363
364     # Start metrics server in background
365     server_thread = threading.Thread(target=exporter.start_server, daemon=True)
366     server_thread.start()
367
368     # Simulate some predictions
369     for i in range(100):

```



```

370     exporter.record_prediction(
371         model_name='fraud_detector',
372         model_version='v2.0',
373         latency_seconds=np.random.uniform(0.01, 0.1),
374         confidence=np.random.uniform(0.7, 0.99)
375     )
376
377     # Simulate some errors
378     if np.random.rand() < 0.05:
379         exporter.record_error(
380             model_name='fraud_detector',
381             model_version='v2.0',
382             error_type='timeout'
383         )
384
385     # Update performance metrics
386     exporter.update_performance_metrics(
387         model_name='fraud_detector',
388         model_version='v2.0',
389         metrics={
390             'accuracy': 0.94,
391             'precision': 0.92,
392             'recall': 0.91,
393             'f1_score': 0.915
394         }
395     )
396
397     print("Prometheus metrics available at http://localhost:8000/metrics")
398
399     # Generate Grafana dashboards
400     dashboard_gen = GrafanaDashboardGenerator()
401
402     overview_dashboard = dashboard_gen.generate_ml_overview_dashboard()
403     performance_dashboard = dashboard_gen.generate_performance_dashboard()
404
405     print(f"Generated {len(overview_dashboard['dashboard']['panels'])} overview panels")
406     print(f"Generated {len(performance_dashboard['dashboard']['panels'])} performance panels")
407
408     example_prometheus_integration()

```

Listing 14.4: Prometheus Integration for ML Monitoring

14.7 Alert Systems and Incident Response

14.7.1 Intelligent Alerting

Effective alerting balances sensitivity (catching real issues) with specificity (avoiding false alarms). ML systems require specialized alerting strategies that account for gradual degradation and statistical variation.

Alerting principles:

- **Actionable alerts:** Every alert should require human action
- **Context-rich:** Include enough information for diagnosis

- **Prioritized:** Clear severity levels guide response
- **Deduplicated:** Avoid alert storms from correlated issues
- **Adaptive thresholds:** Account for normal variation patterns

14.7.2 Alert System Implementation

```

1 import logging
2 from typing import Any, Callable, Dict
3 from datetime import datetime
4
5 class AlertManager:
6     """Manage ML monitoring alerts with incident response"""
7
8     def __init__(self, monitoring_architecture: MonitoringArchitecture):
9         self.monitoring = monitoring_architecture
10        self.logger = logging.getLogger("AlertManager")
11        self.incidents: Dict[str, Dict] = {}
12        self.notification_channels: Dict[str, Callable] = {}
13
14        def register_notification_channel(self,
15                                         channel_name: str,
16                                         handler: Callable):
17            """Register notification channel"""
18            self.notification_channels[channel_name] = handler
19
20        def create_incident(self,
21                           alert_id: str,
22                           alert_name: str,
23                           severity: AlertSeverity,
24                           details: Dict) -> str:
25            """Create incident from alert"""
26            import uuid
27            incident_id = str(uuid.uuid4())
28
29            incident = {
30                'incident_id': incident_id,
31                'alert_id': alert_id,
32                'alert_name': alert_name,
33                'severity': severity.value,
34                'status': 'open',
35                'created_at': datetime.now(),
36                'updated_at': datetime.now(),
37                'details': details,
38                'notifications_sent': [],
39                'actions_taken': []
40            }
41
42            self.incidents[incident_id] = incident
43            self._send_notifications(incident)
44
45            self.logger.warning(
46                f"Incident created: {incident_id} - {alert_name} ({severity.value})"
47            )
48
49            return incident_id

```

```

50
51 def _send_notifications(self, incident: Dict):
52     """Send incident notifications"""
53     severity = AlertSeverity(incident['severity'])
54
55     if severity == AlertSeverity.CRITICAL:
56         channels = ['pagerduty', 'slack', 'email']
57     elif severity == AlertSeverity.ERROR:
58         channels = ['slack', 'email']
59     elif severity == AlertSeverity.WARNING:
60         channels = ['slack']
61     else:
62         channels = []
63
64     for channel in channels:
65         if channel in self.notification_channels:
66             try:
67                 self.notification_channels[channel](incident)
68                 incident['notifications_sent'].append({
69                     'channel': channel,
70                     'timestamp': datetime.now().isoformat()
71                 })
72             except Exception as e:
73                 self.logger.error(
74                     f"Notification failed for {channel}: {str(e)}"
75                 )
76
77 def resolve_incident(self, incident_id: str, resolution: str):
78     """Resolve an incident"""
79     if incident_id not in self.incidents:
80         self.logger.error(f"Incident {incident_id} not found")
81         return
82
83     incident = self.incidents[incident_id]
84     incident['status'] = 'resolved'
85     incident['resolution'] = resolution
86     incident['resolved_at'] = datetime.now()
87     incident['updated_at'] = datetime.now()
88
89     self.logger.info(f"Incident {incident_id} resolved: {resolution}")
90
91 class IncidentRunbook:
92     """Automated incident response runbooks"""
93
94     def __init__(self):
95         self.runbooks: Dict[str, List[Callable]] = {}
96         self.logger = logging.getLogger("IncidentRunbook")
97
98     def register_runbook(self, alert_type: str, actions: List[Callable]):
99         """Register runbook for alert type"""
100         self.runbooks[alert_type] = actions
101         self.logger.info(
102             f"Registered runbook for {alert_type} with {len(actions)} actions"
103         )
104
105     def execute_runbook(self, alert_type: str, context: Dict) -> List[Dict]:
106         """Execute runbook actions"""
107         if alert_type not in self.runbooks:

```

```

108         self.logger.warning(f"No runbook for {alert_type}")
109         return []
110
111     results = []
112     actions = self.runbooks[alert_type]
113
114     for i, action in enumerate(actions):
115         try:
116             self.logger.info(
117                 f"Executing runbook action {i+1}/{len(actions)} for {alert_type}"
118             )
119             result = action(context)
120             results.append({
121                 'action_index': i,
122                 'status': 'success',
123                 'result': result,
124                 'timestamp': datetime.now().isoformat()
125             })
126         except Exception as e:
127             self.logger.error(f"Runbook action {i+1} failed: {str(e)}")
128             results.append({
129                 'action_index': i,
130                 'status': 'failed',
131                 'error': str(e),
132                 'timestamp': datetime.now().isoformat()
133             })
134
135     return results
136
137 # Example notification handlers
138 def slack_notification(incident: Dict):
139     """Send Slack notification"""
140     print(f"[SLACK] Incident {incident['incident_id']}: {incident['alert_name']}")
141     print(f"Severity: {incident['severity']}")
142
143 def pagerduty_notification(incident: Dict):
144     """Send PagerDuty notification"""
145     print(f"[PAGERDUTY] CRITICAL: {incident['alert_name']}")
146     print(f"Incident ID: {incident['incident_id']}")
147
148 # Example runbook actions
149 def rollback_model_action(context: Dict):
150     """Rollback to previous model version"""
151     model_name = context['model_name']
152     print(f"Rolling back {model_name} to previous version")
153     return {'action': 'rollback', 'model': model_name}
154
155 def trigger_retraining_action(context: Dict):
156     """Trigger model retraining"""
157     print(f"Triggering retraining for {context['model_name']}")
158     return {'action': 'retrain', 'reason': 'drift_detected'}

```

Listing 14.5: Alert System and Incident Response

14.8 Observability Best Practices

14.8.1 Comprehensive Observability

Observability extends beyond monitoring to provide deep insights into system behavior. The three pillars of observability work together to enable debugging and optimization.

Best practices for ML observability:

1. Structured Logging:

- Use structured logs (JSON) for machine parsing
- Include prediction IDs for request tracking
- Log model versions and feature values
- Implement log sampling for high-volume systems

2. Distributed Tracing:

- Trace requests through preprocessing, inference, postprocessing
- Measure latency at each stage
- Identify bottlenecks in serving pipeline
- Use OpenTelemetry for standardized instrumentation

3. Feature Store Observability:

- Monitor feature computation latency
- Track feature freshness and staleness
- Alert on feature pipeline failures
- Log feature transformations for debugging

4. Model Explainability Integration:

- Log feature importance for predictions
- Store SHAP values for critical decisions
- Track explanation consistency over time
- Enable auditing of model decisions

5. Cost Monitoring:

- Track inference costs per prediction
- Monitor resource utilization vs capacity
- Optimize batch sizes for cost efficiency
- Alert on budget overruns

6. Data Quality Monitoring:

- Validate input schemas continuously
- Detect anomalous feature values
- Track null/missing feature rates
- Monitor data pipeline health

7. Security and Compliance:

- Audit data access patterns
- Monitor for adversarial inputs
- Track PII handling in predictions
- Log consent and data retention

14.8.2 Implementing Structured Logging for ML

Structured logging enables machine-parseable logs that can be aggregated, searched, and analyzed at scale. For ML systems, structured logs should capture the complete context of each prediction.

```
1 import json
2 import logging
3 from typing import Any, Dict, Optional
4 from datetime import datetime
5 from enum import Enum
6 import hashlib
7
8 class LogLevel(Enum):
9     """Log severity levels"""
10     DEBUG = "debug"
11     INFO = "info"
12     WARNING = "warning"
13     ERROR = "error"
14     CRITICAL = "critical"
15
16 class MLLogger:
17     """
18     Structured logger for ML systems
19
20     Captures prediction context, model metadata, and performance info
21     with machine-parseable JSON format
22     """
23
24     def __init__(self, service_name: str, log_file: Optional[str] = None):
25         self.service_name = service_name
26         self.logger = logging.getLogger(service_name)
27         self.logger.setLevel(logging.INFO)
28
29         # JSON formatter
30         handler = logging.FileHandler(log_file) if log_file else logging.StreamHandler
31         ()
```

```

31         self.logger.addHandler(handler)
32
33     def _create_log_entry(self, level: LogLevel, event: str,
34                           context: Dict[str, Any]) -> Dict[str, Any]:
35         """Create structured log entry"""
36         return {
37             'timestamp': datetime.now().isoformat(),
38             'service': self.service_name,
39             'level': level.value,
40             'event': event,
41             'context': context
42         }
43
44     def log_prediction(self, model_name: str, model_version: str,
45                       prediction_id: str, features: Dict,
46                       prediction: Any, confidence: float,
47                       latency_ms: float):
48         """Log ML prediction with full context"""
49
50         # Hash features for privacy
51         feature_hash = hashlib.sha256(
52             json.dumps(features, sort_keys=True).encode()
53         ).hexdigest()[:16]
54
55         log_entry = self._create_log_entry(
56             level=LogLevel.INFO,
57             event='ml_prediction',
58             context={
59                 'model_name': model_name,
60                 'model_version': model_version,
61                 'prediction_id': prediction_id,
62                 'feature_hash': feature_hash,
63                 'prediction': str(prediction),
64                 'confidence': confidence,
65                 'latency_ms': latency_ms,
66                 'feature_count': len(features)
67             }
68         )
69
70         self.logger.info(json.dumps(log_entry))
71
72     def log_drift_detection(self, feature_name: str, drift_score: float,
73                           drift_detected: bool, method: str):
74         """Log drift detection results"""
75
76         log_entry = self._create_log_entry(
77             level=LogLevel.WARNING if drift_detected else LogLevel.INFO,
78             event='drift_detection',
79             context={
80                 'feature_name': feature_name,
81                 'drift_score': drift_score,
82                 'drift_detected': drift_detected,
83                 'detection_method': method
84             }
85         )
86
87         self.logger.warning(json.dumps(log_entry)) if drift_detected \
88             else self.logger.info(json.dumps(log_entry))

```

```

89
90 def log_model_error(self, model_name: str, model_version: str,
91                     error_type: str, error_message: str,
92                     prediction_id: Optional[str] = None):
93     """Log model errors with context"""
94
95     log_entry = self._create_log_entry(
96         level=LogLevel.ERROR,
97         event='model_error',
98         context={
99             'model_name': model_name,
100             'model_version': model_version,
101             'error_type': error_type,
102             'error_message': error_message,
103             'prediction_id': prediction_id
104         }
105     )
106
107     self.logger.error(json.dumps(log_entry))
108
109 def log_performance_metrics(self, model_name: str, model_version: str,
110                             metrics: Dict[str, float], window_hours: int):
111     """Log model performance metrics"""
112
113     log_entry = self._create_log_entry(
114         level=LogLevel.INFO,
115         event='performance_metrics',
116         context={
117             'model_name': model_name,
118             'model_version': model_version,
119             'metrics': metrics,
120             'window_hours': window_hours
121         }
122     )
123
124     self.logger.info(json.dumps(log_entry))
125
126 # Example usage
127 logger = MLLogger('fraud_detection_service', log_file='ml_predictions.jsonl')
128
129 # Log prediction
130 logger.log_prediction(
131     model_name='fraud_detector',
132     model_version='v2.3',
133     prediction_id='pred_12345',
134     features={'amount': 150.0, 'merchant_category': 'retail'},
135     prediction=0,
136     confidence=0.92,
137     latency_ms=45.2
138 )
139
140 # Log drift detection
141 logger.log_drift_detection(
142     feature_name='transaction_amount',
143     drift_score=0.28,
144     drift_detected=True,
145     method='kolmogorov_smirnov'
146 )

```



```

147
148 # Log error
149 logger.log_model_error(
150     model_name='fraud_detector',
151     model_version='v2.3',
152     error_type='FeatureExtractionError',
153     error_message='Missing required feature: merchant_id',
154     prediction_id='pred_12346'
155 )

```

Listing 14.6: Structured Logging System

14.8.3 Distributed Tracing for ML Pipelines

Distributed tracing tracks requests across multiple services, essential for understanding latency in complex ML serving pipelines.

```

1 from opentelemetry import trace
2 from opentelemetry.sdk.trace import TracerProvider
3 from opentelemetry.sdk.trace.export import BatchSpanProcessor
4 from opentelemetry.exporter.jaeger.thrift import JaegerExporter
5 from opentelemetry.sdk.resources import Resource
6 import time
7 from typing import Dict, Any
8
9 # Setup OpenTelemetry
10 resource = Resource(attributes={
11     "service.name": "ml-inference-service"
12 })
13
14 trace.set_tracer_provider(TracerProvider(resource=resource))
15 tracer = trace.get_tracer(__name__)
16
17 # Setup Jaeger exporter
18 jaeger_exporter = JaegerExporter(
19     agent_host_name='localhost',
20     agent_port=6831,
21 )
22
23 trace.get_tracer_provider().add_span_processor(
24     BatchSpanProcessor(jaeger_exporter)
25 )
26
27 class TracedMLPipeline:
28     """ML serving pipeline with distributed tracing"""
29
30     def __init__(self, model_name: str):
31         self.model_name = model_name
32         self.tracer = trace.get_tracer(__name__)
33
34     def predict(self, features: Dict[str, Any]) -> Dict[str, Any]:
35         """Execute prediction with full tracing"""
36
37         # Create root span for the prediction request
38         with self.tracer.start_as_current_span("ml_prediction") as span:
39             span.set_attribute("model.name", self.model_name)
40             span.set_attribute("feature.count", len(features))

```

```

41         try:
42             # Step 1: Validate input
43             validated = self._validate_input(features)
44
45             # Step 2: Preprocess features
46             processed = self._preprocess(validated)
47
48             # Step 3: Run inference
49             prediction = self._inference(processed)
50
51             # Step 4: Postprocess results
52             result = self._postprocess(prediction)
53
54             span.set_attribute("prediction.confidence", result['confidence'])
55             span.set_attribute("prediction.class", result['class'])
56
57             return result
58
59         except Exception as e:
60             span.record_exception(e)
61             span.set_status(trace.Status(trace.StatusCode.ERROR))
62             raise
63
64     def _validate_input(self, features: Dict[str, Any]) -> Dict[str, Any]:
65         """Validate input features"""
66         with self.tracer.start_as_current_span("validate_input") as span:
67             start = time.time()
68
69             # Validation logic
70             required_features = ['age', 'income', 'credit_score']
71             missing = [f for f in required_features if f not in features]
72
73             if missing:
74                 span.set_attribute("validation.failed", True)
75                 span.set_attribute("missing_features", ','.join(missing))
76                 raise ValueError(f"Missing features: {missing}")
77
78             span.set_attribute("validation.duration_ms",
79                               (time.time() - start) * 1000)
80             return features
81
82     def _preprocess(self, features: Dict[str, Any]) -> Dict[str, Any]:
83         """Preprocess features"""
84         with self.tracer.start_as_current_span("preprocess_features") as span:
85             start = time.time()
86
87             # Simulate preprocessing
88             processed = {
89                 'age_normalized': features['age'] / 100,
90                 'income_log': np.log1p(features['income']),
91                 'credit_normalized': features['credit_score'] / 850
92             }
93
94             span.set_attribute("preprocessing.duration_ms",
95                               (time.time() - start) * 1000)
96             span.set_attribute("preprocessing.method", "standard_scaler")
97
98

```

```

99         return processed
100
101     def _inference(self, features: Dict[str, Any]) -> Dict[str, Any]:
102         """Run model inference"""
103         with self.tracer.start_as_current_span("model_inference") as span:
104             start = time.time()
105
106             # Simulate model inference
107             time.sleep(0.05) # Model execution time
108
109             prediction = {
110                 'class': 1,
111                 'probability': 0.87
112             }
113
114             inference_time = (time.time() - start) * 1000
115             span.set_attribute("inference.duration_ms", inference_time)
116             span.set_attribute("inference.batch_size", 1)
117
118             return prediction
119
120     def _postprocess(self, prediction: Dict[str, Any]) -> Dict[str, Any]:
121         """Postprocess predictions"""
122         with self.tracer.start_as_current_span("postprocess_results") as span:
123             start = time.time()
124
125             # Add business logic
126             result = {
127                 'class': prediction['class'],
128                 'confidence': prediction['probability'],
129                 'risk_level': 'high' if prediction['probability'] > 0.8 else 'low'
130             }
131
132             span.set_attribute("postprocessing.duration_ms",
133                               (time.time() - start) * 1000)
134
135             return result
136
137 # Example usage
138 pipeline = TracedMLPipeline(model_name='credit_risk_v2')
139
140 features = {
141     'age': 35,
142     'income': 75000,
143     'credit_score': 720
144 }
145
146 result = pipeline.predict(features)
147 print(f"Prediction: {result}")

```

Listing 14.7: Distributed Tracing with OpenTelemetry

14.8.4 Data Quality Monitoring Implementation

Data quality issues are the most common cause of ML failures in production. Comprehensive data quality monitoring detects schema violations, distribution shifts, and anomalies.

```

1 from typing import Dict, List, Set, Optional, Any
2 import pandas as pd
3 import numpy as np
4 from dataclasses import dataclass
5 from datetime import datetime
6 from scipy import stats
7
8 @dataclass
9 class DataQualityRule:
10     """Define data quality validation rule"""
11     feature_name: str
12     rule_type: str # 'range', 'categorical', 'not_null', 'pattern'
13     parameters: Dict[str, Any]
14     severity: str = 'error' # 'error', 'warning', 'info'
15
16 @dataclass
17 class QualityViolation:
18     """Data quality violation record"""
19     feature_name: str
20     rule_type: str
21     severity: str
22     violation_count: int
23     violation_rate: float
24     timestamp: datetime
25     details: str
26
27 class DataQualityMonitor:
28     """
29     Monitor data quality for ML feature pipelines
30
31     Validates schemas, detects anomalies, tracks data quality metrics
32     """
33
34     def __init__(self):
35         self.rules: List[DataQualityRule] = []
36         self.violations: List[QualityViolation] = []
37         self.baseline_stats: Dict[str, Dict] = {}
38
39     def add_rule(self, rule: DataQualityRule):
40         """Add data quality rule"""
41         self.rules.append(rule)
42
43     def set_baseline(self, reference_data: pd.DataFrame):
44         """Set baseline statistics from reference data"""
45         for column in reference_data.columns:
46             if pd.api.types.is_numeric_dtype(reference_data[column]):
47                 self.baseline_stats[column] = {
48                     'mean': reference_data[column].mean(),
49                     'std': reference_data[column].std(),
50                     'min': reference_data[column].min(),
51                     'max': reference_data[column].max(),
52                     'q25': reference_data[column].quantile(0.25),
53                     'q75': reference_data[column].quantile(0.75),
54                     'null_rate': reference_data[column].isnull().mean()
55                 }
56             else:
57                 self.baseline_stats[column] = {

```

```

58         'unique_values': set(reference_data[column].dropna().unique()),
59         'null_rate': reference_data[column].isnull().mean(),
60         'mode': reference_data[column].mode()[0] if len(reference_data[
column].mode()) > 0 else None
61     }
62
63     def validate_batch(self, data: pd.DataFrame) -> List[QualityViolation]:
64         """Validate data batch against quality rules"""
65         violations = []
66
67         # Check schema
68         violations.extend(self._check_schema(data))
69
70         # Check null rates
71         violations.extend(self._check_nulls(data))
72
73         # Check value ranges
74         violations.extend(self._check_ranges(data))
75
76         # Check categorical values
77         violations.extend(self._check_categorical(data))
78
79         # Check for anomalies
80         violations.extend(self._check_anomalies(data))
81
82         self.violations.extend(violations)
83         return violations
84
85     def _check_schema(self, data: pd.DataFrame) -> List[QualityViolation]:
86         """Validate data schema"""
87         violations = []
88
89         expected_columns = set(self.baseline_stats.keys())
90         actual_columns = set(data.columns)
91
92         missing_cols = expected_columns - actual_columns
93         extra_cols = actual_columns - expected_columns
94
95         if missing_cols:
96             violations.append(QualityViolation(
97                 feature_name='schema',
98                 rule_type='missing_columns',
99                 severity='error',
100                 violation_count=len(missing_cols),
101                 violation_rate=len(missing_cols) / len(expected_columns),
102                 timestamp=datetime.now(),
103                 details=f"Missing columns: {missing_cols}"
104             ))
105
106         if extra_cols:
107             violations.append(QualityViolation(
108                 feature_name='schema',
109                 rule_type='extra_columns',
110                 severity='warning',
111                 violation_count=len(extra_cols),
112                 violation_rate=len(extra_cols) / len(expected_columns),
113                 timestamp=datetime.now(),
114                 details=f"Extra columns: {extra_cols}"

```

```

115         ))
116
117     return violations
118
119     def _check_nulls(self, data: pd.DataFrame) -> List[QualityViolation]:
120         """Check for excessive null values"""
121         violations = []
122
123         for column in data.columns:
124             if column not in self.baseline_stats:
125                 continue
126
127             null_rate = data[column].isnull().mean()
128             baseline_null_rate = self.baseline_stats[column].get('null_rate', 0)
129
130             # Alert if null rate increases by more than 50%
131             if null_rate > baseline_null_rate * 1.5:
132                 violations.append(QualityViolation(
133                     feature_name=column,
134                     rule_type='high_null_rate',
135                     severity='error' if null_rate > 0.5 else 'warning',
136                     violation_count=int(data[column].isnull().sum()),
137                     violation_rate=null_rate,
138                     timestamp=datetime.now(),
139                     details=f"Null rate {null_rate:.2%} vs baseline {
baseline_null_rate:.2%}"
140                 ))
141
142         return violations
143
144     def _check_ranges(self, data: pd.DataFrame) -> List[QualityViolation]:
145         """Check numeric values are within expected ranges"""
146         violations = []
147
148         for column in data.columns:
149             if column not in self.baseline_stats:
150                 continue
151
152             if 'min' not in self.baseline_stats[column]:
153                 continue
154
155             baseline = self.baseline_stats[column]
156
157             # Check for values outside baseline range
158             out_of_range = (
159                 (data[column] < baseline['min']) |
160                 (data[column] > baseline['max'])
161             ).sum()
162
163             if out_of_range > 0:
164                 violations.append(QualityViolation(
165                     feature_name=column,
166                     rule_type='out_of_range',
167                     severity='warning',
168                     violation_count=int(out_of_range),
169                     violation_rate=out_of_range / len(data),
170                     timestamp=datetime.now(),
171                     details=f"Values outside [{baseline['min']:.2f}, {baseline['max']:.2f}]"

```

```

    ']:.2f}}"
    ))

    return violations

def _check_categorical(self, data: pd.DataFrame) -> List[QualityViolation]:
    """Check categorical values match expected set"""
    violations = []

    for column in data.columns:
        if column not in self.baseline_stats:
            continue

        if 'unique_values' not in self.baseline_stats[column]:
            continue

        baseline_values = self.baseline_stats[column]['unique_values']
        current_values = set(data[column].dropna().unique())

        # New values not seen in baseline
        new_values = current_values - baseline_values

        if new_values:
            violations.append(QualityViolation(
                feature_name=column,
                rule_type='unexpected_categorical_value',
                severity='warning',
                violation_count=len(new_values),
                violation_rate=len(new_values) / len(baseline_values) if
baseline_values else 1.0,
                timestamp=datetime.now(),
                details=f"New categorical values: {new_values}"
            ))

    return violations

def _check_anomalies(self, data: pd.DataFrame) -> List[QualityViolation]:
    """Detect statistical anomalies"""
    violations = []

    for column in data.columns:
        if column not in self.baseline_stats:
            continue

        if 'mean' not in self.baseline_stats[column]:
            continue

        baseline = self.baseline_stats[column]
        current_mean = data[column].mean()
        current_std = data[column].std()

        # Check if mean shifted significantly (>3 standard deviations)
        z_score = abs(current_mean - baseline['mean']) / baseline['std']

        if z_score > 3:
            violations.append(QualityViolation(
                feature_name=column,
                rule_type='mean_shift',

```

```

228         severity='warning',
229         violation_count=1,
230         violation_rate=z_score / 3,
231         timestamp=datetime.now(),
232         details=f"Mean shifted from {baseline['mean']:.2f} to {
current_mean:.2f} (z-score: {z_score:.2f})"
233     ))
234
235     return violations
236
237 def get_quality_report(self) -> Dict[str, Any]:
238     """Generate data quality report"""
239     if not self.violations:
240         return {
241             'status': 'healthy',
242             'total_violations': 0,
243             'violations_by_severity': {},
244             'violations_by_type': {}
245         }
246
247     violations_by_severity = {}
248     violations_by_type = {}
249
250     for v in self.violations:
251         violations_by_severity[v.severity] = violations_by_severity.get(v.severity
, 0) + 1
252         violations_by_type[v.rule_type] = violations_by_type.get(v.rule_type, 0) +
1
253
254     return {
255         'status': 'unhealthy' if 'error' in violations_by_severity else 'degraded'
,
256         'total_violations': len(self.violations),
257         'violations_by_severity': violations_by_severity,
258         'violations_by_type': violations_by_type,
259         'recent_violations': [
260             {
261                 'feature': v.feature_name,
262                 'type': v.rule_type,
263                 'severity': v.severity,
264                 'details': v.details
265             }
266             for v in sorted(self.violations, key=lambda x: x.timestamp, reverse=
True)[:10]
267         ]
268     }
269
270 # Example usage
271 monitor = DataQualityMonitor()
272
273 # Set baseline from training data
274 train_data = pd.DataFrame({
275     'age': np.random.normal(40, 12, 1000),
276     'income': np.random.normal(60000, 20000, 1000),
277     'category': np.random.choice(['A', 'B', 'C'], 1000)
278 })
279
280 monitor.set_baseline(train_data)

```



```

281 # Validate production data
282 prod_data = pd.DataFrame({
283     'age': np.random.normal(42, 12, 100), # Slight shift
284     'income': np.concatenate([np.random.normal(60000, 20000, 95), [np.nan]*5]), #
285     'category': np.random.choice(['A', 'B', 'C', 'D'], 100) # New category D
286 })
287
288 violations = monitor.validate_batch(prod_data)
289
290 # Generate report
291 report = monitor.get_quality_report()
292 print(json.dumps(report, indent=2))
293

```

Listing 14.8: Data Quality Monitoring System

14.9 Exercises

14.9.1 Exercise 1: Complete Monitoring Stack Setup

Problem: Implement a complete monitoring stack with metrics collection, drift detection, and alerting for a classification model.

Solution:

```

1 # Exercise 1 Solution
2 def solution_exercise_1():
3     """Complete monitoring stack implementation"""
4
5     # Step 1: Setup metrics collection
6     collector = MetricsCollector(buffer_size=10000)
7     monitoring = MonitoringArchitecture(collector)
8
9     # Step 2: Setup drift detection
10    np.random.seed(42)
11    reference_df = pd.DataFrame({
12        'age': np.random.normal(45, 15, 1000),
13        'income': np.random.normal(60000, 20000, 1000),
14        'credit_score': np.random.normal(700, 50, 1000)
15    })
16
17    drift_monitor = DataDriftMonitor(reference_df)
18
19    # Step 3: Setup performance monitoring
20    perf_monitor = PerformanceMonitor(collector)
21
22    # Step 4: Setup alerts
23    def accuracy_degradation_check(metrics: MetricsCollector) -> bool:
24        recent_metrics = metrics.get_metrics(
25            name='model_accuracy',
26            start_time=datetime.now() - timedelta(hours=1)
27        )
28        if not recent_metrics:
29            return False
30        recent_accuracy = np.mean([m.value for m in recent_metrics])
31        return recent_accuracy < 0.85

```

```

32 monitoring.register_alert(Alert(
33     alert_id='accuracy_degradation',
34     name='Model Accuracy Degradation',
35     condition=accuracy_degradation_check,
36     severity=AlertSeverity.ERROR,
37     message_template='Model accuracy dropped below 85%',
38     cooldown_minutes=30
39 ))
40
41
42 # Simulate production traffic
43 print("=" * 70)
44 print("Monitoring Stack Operational")
45 print("=" * 70)
46
47 for i in range(500):
48     features = {
49         'age': np.random.normal(45, 15),
50         'income': np.random.normal(60000, 20000),
51         'credit_score': np.random.normal(700, 50)
52     }
53
54     prediction = np.random.choice([0, 1], p=[0.7, 0.3])
55     confidence = np.random.uniform(0.6, 0.99)
56     latency = np.random.uniform(0.01, 0.1)
57
58     monitoring.record_prediction(
59         model_name='credit_risk_model',
60         model_version='v1.0',
61         features=features,
62         prediction=prediction,
63         confidence=confidence,
64         latency_ms=latency * 1000
65     )
66
67     perf_monitor.record_prediction(
68         model_name='credit_risk_model',
69         model_version='v1.0',
70         prediction_id=f'pred_{i}',
71         features=features,
72         prediction=prediction
73     )
74
75     accuracy = 0.90 - (i / 500) * 0.10
76     actual = prediction if np.random.rand() < accuracy else (1 - prediction)
77
78     perf_monitor.record_ground_truth(
79         prediction_id=f'pred_{i}',
80         actual=actual
81     )
82
83     if i % 100 == 0 and i > 0:
84         current_df = pd.DataFrame({
85             'age': np.random.normal(48, 15, 100),
86             'income': np.random.normal(62000, 20000, 100),
87             'credit_score': np.random.normal(705, 50, 100)
88         })
89

```

```

90     drift_results = drift_monitor.check_drift(current_df)
91     drifted = sum(1 for r in drift_results.values() if r.drift_detected)
92     print(f"\n[Batch {i}] Drift check: {drifted}/{len(drift_results)} features
drifted")
93
94     triggered_alerts = monitoring.check_alerts()
95
96     print(f"\n\n=== Monitoring Summary ===")
97     print(f"Predictions processed: 500")
98     print(f"Alerts triggered: {len(triggered_alerts)}")
99
100    if triggered_alerts:
101        for alert in triggered_alerts:
102            print(f"    - {alert['name']} ({alert['severity']})")
103
104    health = monitoring.get_health_status()
105    print(f"\nSystem health: {health['status']}")
106
107    trend = perf_monitor.get_performance_trend(
108        'credit_risk_model',
109        'accuracy',
110        window_snapshots=5
111    )
112
113    if trend:
114        print(f"Accuracy trend: {' -> '.join([f'{t:.3f}' for t in trend])}")
115
116    degraded = perf_monitor.detect_performance_degradation(
117        'credit_risk_model',
118        'accuracy',
119        threshold_pct=5.0
120    )
121
122    print(f"Performance degradation detected: {degraded}")
123
124    solution_exercise_1()

```

14.9.2 Exercise 2: Drift Detection Comparison

Problem: Implement comprehensive drift detection using KS test, PSI, and Jensen-Shannon divergence. Compare results across methods.

Solution:

```

1  # Exercise 2 Solution
2  def solution_exercise_2():
3      """Compare drift detection methods"""
4
5      np.random.seed(42)
6
7      reference_df = pd.DataFrame({
8          'continuous_1': np.random.normal(100, 15, 1000),
9          'continuous_2': np.random.exponential(50, 1000),
10         'categorical_1': np.random.choice(['A', 'B', 'C'], 1000, p=[0.5, 0.3, 0.2])
11     })
12
13     current_df = pd.DataFrame({
14         'continuous_1': np.random.normal(105, 15, 1000),

```

```

15     'continuous_2': np.random.exponential(55, 1000),
16     'categorical_1': np.random.choice(['A', 'B', 'C'], 1000, p=[0.4, 0.4, 0.2])
17 })
18
19 detector = DriftDetector(significance_level=0.05)
20
21 print("=" * 70)
22 print("Drift Detection Method Comparison")
23 print("=" * 70)
24
25 for column in reference_df.columns:
26     print(f"\n{'='*70}")
27     print(f"Feature: {column}")
28     print(f"{'='*70}")
29
30     ref_col = reference_df[column]
31     curr_col = current_df[column]
32
33     if ref_col.dtype in ['object', 'category']:
34         methods = ['chi2', 'psi', 'js']
35     else:
36         methods = ['ks', 'psi', 'js']
37
38     for method in methods:
39         if method == 'ks':
40             result = detector.detect_drift_ks(ref_col, curr_col)
41         elif method == 'chi2':
42             result = detector.detect_drift_chi2(ref_col, curr_col)
43         elif method == 'psi':
44             result = detector.detect_drift_psi(ref_col, curr_col)
45         elif method == 'js':
46             result = detector.detect_drift_jensen_shannon(ref_col, curr_col)
47
48         status = "DRIFT" if result.drift_detected else "NO DRIFT"
49         print(f"\n{method.upper():15} {status:10}")
50         print(f"    Score:      {result.drift_score:.4f}")
51         print(f"    P-value:     {result.p_value:.4f}")
52         print(f"    Threshold:  {result.threshold:.4f}")
53
54         if 'mean' in result.reference_stats and 'mean' in result.current_stats:
55             ref_mean = result.reference_stats['mean']
56             curr_mean = result.current_stats['mean']
57             change_pct = ((curr_mean - ref_mean) / abs(ref_mean)) * 100
58
59             print(f"    Ref mean:   {ref_mean:.2f}")
60             print(f"    Curr mean:  {curr_mean:.2f}")
61             print(f"    Change:     {change_pct:+.2f}%")
62
63 solution_exercise_2()

```

14.9.3 Exercise 3: Prometheus and Grafana Dashboard

Problem: Create a Prometheus exporter for ML metrics and generate Grafana dashboard configurations.

Solution:

```
1 # Exercise 3 Solution
```

```

2 def solution_exercise_3():
3     """Prometheus and Grafana integration"""
4
5     # Initialize Prometheus exporter
6     exporter = PrometheusExporter(port=8001)
7
8     # Start metrics server
9     import threading
10    server_thread = threading.Thread(target=exporter.start_server, daemon=True)
11    server_thread.start()
12
13    print("=" * 70)
14    print("Prometheus Exporter Running")
15    print("=" * 70)
16
17    # Simulate model serving with metrics
18    models = ['fraud_detector_v1', 'fraud_detector_v2', 'fraud_detector_v3']
19
20    for iteration in range(3):
21        print(f"\nIteration {iteration + 1}/3")
22
23        for model in models:
24            # Record predictions
25            for _ in range(100):
26                exporter.record_prediction(
27                    model_name=model.split('_v')[0],
28                    model_version=f"v{model.split('_v')[1]}",
29                    latency_seconds=np.random.uniform(0.01, 0.15),
30                    confidence=np.random.uniform(0.7, 0.99)
31                )
32
33            # Simulate errors
34            if np.random.rand() < 0.03:
35                exporter.record_error(
36                    model_name=model.split('_v')[0],
37                    model_version=f"v{model.split('_v')[1]}",
38                    error_type=np.random.choice(['timeout', 'invalid_input', '
model_error'])
39                )
40
41            # Update performance metrics
42            accuracy = 0.88 + np.random.uniform(-0.05, 0.05)
43            precision = 0.86 + np.random.uniform(-0.05, 0.05)
44            recall = 0.84 + np.random.uniform(-0.05, 0.05)
45            f1 = 2 * (precision * recall) / (precision + recall)
46
47            exporter.update_performance_metrics(
48                model_name=model.split('_v')[0],
49                model_version=f"v{model.split('_v')[1]}",
50                metrics={
51                    'accuracy': accuracy,
52                    'precision': precision,
53                    'recall': recall,
54                    'f1_score': f1
55                }
56            )
57
58    print(f" {model}: acc={accuracy:.3f}, prec={precision:.3f}, rec={recall

```

```

        :.3f}"))
59
60 print(f"\n{'='*70}")
61 print("Metrics available at http://localhost:8001/metrics")
62 print(f"{'='*70}")
63
64 # Generate Grafana dashboards
65 dashboard_gen = GrafanaDashboardGenerator()
66
67 overview_dashboard = dashboard_gen.generate_ml_overview_dashboard()
68 performance_dashboard = dashboard_gen.generate_performance_dashboard()
69
70 print(f"\nGenerated Dashboards:")
71 print(f" - Overview: {len(overview_dashboard['dashboard']['panels'])} panels")
72 print(f" - Performance: {len(performance_dashboard['dashboard']['panels'])} panels")
73
74 # Show sample PromQL queries
75 print(f"\nSample PromQL Queries:")
76 print(f" Prediction rate:")
77 print(f"     rate(ml_prediction_total[5m])")
78 print(f" Error rate:")
79 print(f"     rate(ml_prediction_errors_total[5m]) / rate(ml_prediction_total[5m])")
80 print(f" P95 latency:")
81 print(f"     histogram_quantile(0.95, rate(ml_prediction_latency_seconds_bucket[5m
82 ])))")
83 solution_exercise_3()

```

14.9.4 Exercise 4: Alert System with Runbooks

Problem: Implement an alert manager with automated incident response runbooks.

Solution:

```

1 # Exercise 4 Solution
2 def solution_exercise_4():
3     """Alert system with automated runbooks"""
4
5     # Setup monitoring
6     collector = MetricsCollector()
7     monitoring = MonitoringArchitecture(collector)
8     alert_manager = AlertManager(monitoring)
9
10    # Register notification channels
11    alert_manager.register_notification_channel('slack', slack_notification)
12    alert_manager.register_notification_channel('pagerduty', pagerduty_notification)
13
14    # Setup runbooks
15    runbook = IncidentRunbook()
16
17    # High latency runbook
18    runbook.register_runbook('high_latency', [
19        scale_resources_action,
20        lambda ctx: {'action': 'notify', 'channel': 'slack'}
21    ])
22
23    # Data drift runbook

```

```

24     runbook.register_runbook('data_drift', [
25         trigger_retraining_action,
26         lambda ctx: {'action': 'create_ticket', 'system': 'jira'}
27     ])
28
29     # Accuracy degradation runbook
30     runbook.register_runbook('accuracy_degradation', [
31         rollback_model_action,
32         lambda ctx: {'action': 'page_oncall', 'severity': 'high'}
33     ])
34
35     print("=" * 70)
36     print("Alert System with Runbooks")
37     print("=" * 70)
38
39     # Simulate scenarios
40     scenarios = [
41         {
42             'type': 'high_latency',
43             'alert_name': 'High Prediction Latency',
44             'severity': AlertSeverity.WARNING,
45             'details': {
46                 'model_name': 'fraud_detector',
47                 'p95_latency_ms': 250,
48                 'threshold_ms': 100
49             }
50         },
51         {
52             'type': 'data_drift',
53             'alert_name': 'Significant Data Drift',
54             'severity': AlertSeverity.ERROR,
55             'details': {
56                 'model_name': 'fraud_detector',
57                 'drifted_features': ['transaction_amount', 'merchant_category'],
58                 'max_drift_score': 0.45
59             }
60         },
61         {
62             'type': 'accuracy_degradation',
63             'alert_name': 'Model Accuracy Degradation',
64             'severity': AlertSeverity.CRITICAL,
65             'details': {
66                 'model_name': 'fraud_detector',
67                 'current_accuracy': 0.78,
68                 'threshold': 0.85,
69                 'degradation_pct': 8.2
70             }
71         }
72     ]
73
74     for i, scenario in enumerate(scenarios, 1):
75         print(f"\n{'='*70}")
76         print(f"Scenario {i}: {scenario['alert_name']}")
77         print(f"{'='*70}")
78
79         # Create incident
80         incident_id = alert_manager.create_incident(
81             alert_id=f"{scenario['type']}-{i}",

```

```

82         alert_name=scenario['alert_name'],
83         severity=scenario['severity'],
84         details=scenario['details']
85     )
86
87     print(f"\nIncident created: {incident_id}")
88     print(f"Severity: {scenario['severity'].value}")
89     print(f"Details: {scenario['details']}")
90
91     # Execute runbook
92     print(f"\nExecuting runbook for '{scenario['type']}'...")
93     runbook_results = runbook.execute_runbook(
94         alert_type=scenario['type'],
95         context={
96             'incident_id': incident_id,
97             **scenario['details']
98         }
99     )
100
101     print(f"Runbook completed: {len(runbook_results)} actions")
102     for j, result in enumerate(runbook_results, 1):
103         print(f"  Action {j}: {result['status']}")
104
105     # Resolve incident
106     alert_manager.resolve_incident(
107         incident_id,
108         f"Automated runbook completed. Issue resolved."
109     )
110
111     print(f"Incident resolved: {incident_id}")
112
113     print(f"\n{'='*70}")
114     print(f"Total incidents handled: {len(scenarios)}")
115     print(f"{'='*70}")
116
117 solution_exercise_4()

```

14.9.5 Exercise 5: Performance Degradation Detection

Problem: Implement a system to detect performance degradation using statistical methods and trigger alerts.

Solution:

```

1  # Exercise 5 Solution
2  def solution_exercise_5():
3      """Performance degradation detection system"""
4
5      collector = MetricsCollector()
6      perf_monitor = PerformanceMonitor(collector, performance_window_hours=24)
7
8      print("=" * 70)
9      print("Performance Degradation Detection")
10     print("=" * 70)
11
12     # Simulate model predictions over time with gradual degradation
13     np.random.seed(42)
14     total_predictions = 1000

```



```

15
16 print(f"\nSimulating {total_predictions} predictions with gradual degradation")
17
18 for i in range(total_predictions):
19     pred_id = f"pred_{i}"
20
21     # Features
22     features = {
23         'feature_1': np.random.rand(),
24         'feature_2': np.random.rand(),
25         'feature_3': np.random.randint(0, 10)
26     }
27
28     # Prediction
29     prediction = np.random.choice([0, 1])
30
31     # Record prediction
32     perf_monitor.record_prediction(
33         model_name='churn_predictor',
34         model_version='v2.3',
35         prediction_id=pred_id,
36         features=features,
37         prediction=prediction
38     )
39
40     # Simulate gradual performance degradation
41     # Start at 92% accuracy, degrade to 82%
42     base_accuracy = 0.92
43     degradation = (i / total_predictions) * 0.10
44     current_accuracy = base_accuracy - degradation
45
46     # Generate ground truth based on current accuracy
47     actual = prediction if np.random.rand() < current_accuracy else (1 -
prediction)
48
49     perf_monitor.record_ground_truth(
50         prediction_id=pred_id,
51         actual=actual
52     )
53
54     # Check for degradation periodically
55     if (i + 1) % 100 == 0:
56         degraded = perf_monitor.detect_performance_degradation(
57             model_name='churn_predictor',
58             metric_name='accuracy',
59             threshold_pct=5.0
60         )
61
62         trend = perf_monitor.get_performance_trend(
63             model_name='churn_predictor',
64             metric_name='accuracy',
65             window_snapshots=5
66         )
67
68         if trend:
69             avg_acc = np.mean(trend)
70             print(f"\n[Batch {i+1:4d}] Avg accuracy: {avg_acc:.3f} | Degradation:
{degraded}")

```

```

71         if degraded:
72             print(f"      [!] WARNING: Performance degradation detected!")
73             print(f"      Trend: {' -> '.join([f'{t:.3f}' for t in trend])}")
74
75     # Final summary
76     print(f"\n{'='*70}")
77     print("Final Performance Summary")
78     print(f"{'='*70}")
79
80     final_trend = perf_monitor.get_performance_trend(
81         model_name='churn_predictor',
82         metric_name='accuracy',
83         window_snapshots=10
84     )
85
86     if final_trend:
87         print(f"Last 10 snapshots:")
88         for i, acc in enumerate(final_trend, 1):
89             print(f"  Snapshot {i}: {acc:.3f}")
90
91         print(f"\nAccuracy range: {min(final_trend):.3f} - {max(final_trend):.3f}")
92         print(f"Mean accuracy: {np.mean(final_trend):.3f}")
93         print(f"Std dev: {np.std(final_trend):.4f}")
94
95     solution_exercise_5()
96

```

14.9.6 Exercise 6: Comprehensive Monitoring Dashboard

Problem: Build a complete real-time monitoring dashboard that integrates metrics, drift detection, and performance tracking.

Solution:

```

1  # Exercise 6 Solution
2  class MonitoringDashboard:
3      """Real-time monitoring dashboard"""
4
5      def __init__(self):
6          self.collector = MetricsCollector()
7          self.monitoring = MonitoringArchitecture(self.collector)
8          self.exporter = PrometheusExporter(port=8002)
9          self.logger = logging.getLogger("Dashboard")
10
11     def start(self):
12         """Start dashboard"""
13         print("=" * 70)
14         print("MONITORING DASHBOARD")
15         print("=" * 70)
16
17     def display_metrics_summary(self):
18         """Display metrics summary"""
19         health = self.monitoring.get_health_status()
20
21         print(f"\n### System Health: {health['status'].upper()} ###")
22         print(f"  Total predictions (5min): {health['total_predictions']}")
23         print(f"  Total errors (5min): {health['total_errors']}")
24         print(f"  Error rate: {health['error_rate']:.2%}")

```

```

25
26 def display_latency_stats(self, model_name: str):
27     """Display latency statistics"""
28     percentiles = self.collector.get_histogram_percentiles(
29         'prediction_latency_ms',
30         labels={'model_name': model_name},
31         percentiles=[50, 90, 95, 99]
32     )
33
34     print(f"\n### Latency Statistics ({model_name}) ###")
35     for p, value in sorted(percentiles.items()):
36         print(f"    p{int(p)}: {value:.2f}ms")
37
38 def display_drift_status(self, drift_results: Dict):
39     """Display drift detection status"""
40     drifted = sum(1 for r in drift_results.values() if r.drift_detected)
41     total = len(drift_results)
42
43     print(f"\n### Data Drift Status ###")
44     print(f"    Features monitored: {total}")
45     print(f"    Features with drift: {drifted}")
46
47     if drifted > 0:
48         print(f"\n    Drifted features:")
49         for feature, result in drift_results.items():
50             if result.drift_detected:
51                 print(f"        - {feature}: score={result.drift_score:.3f} ({result.
method})")
52
53 def solution_exercise_6():
54     """Complete monitoring dashboard"""
55
56     dashboard = MonitoringDashboard()
57     dashboard.start()
58
59     # Setup reference data for drift
60     np.random.seed(42)
61     reference_df = pd.DataFrame({
62         'age': np.random.normal(40, 12, 1000),
63         'balance': np.random.lognormal(10, 2, 1000),
64         'tenure_months': np.random.poisson(24, 1000)
65     })
66
67     drift_monitor = DataDriftMonitor(reference_df)
68
69     # Simulate monitoring cycles
70     for cycle in range(5):
71         print(f"\n{'='*70}")
72         print(f"Monitoring Cycle {cycle + 1}")
73         print(f"{'='*70}")
74
75     # Simulate predictions
76     for i in range(100):
77         features = {
78             'age': np.random.normal(40, 12),
79             'balance': np.random.lognormal(10, 2),
80             'tenure_months': np.random.poisson(24)
81         }

```

```

82
83     prediction = np.random.choice([0, 1], p=[0.75, 0.25])
84     confidence = np.random.uniform(0.6, 0.99)
85     latency = np.random.gamma(2, 25)
86
87     dashboard.monitoring.record_prediction(
88         model_name='churn_model',
89         model_version='v3.1',
90         features=features,
91         prediction=prediction,
92         confidence=confidence,
93         latency_ms=latency
94     )
95
96     dashboard.exporter.record_prediction(
97         model_name='churn_model',
98         model_version='v3.1',
99         latency_seconds=latency / 1000,
100        confidence=confidence
101    )
102
103    # Display dashboard
104    dashboard.display_metrics_summary()
105    dashboard.display_latency_stats('churn_model')
106
107    # Check drift
108    current_df = pd.DataFrame({
109        'age': np.random.normal(41 + cycle, 12, 100),
110        'balance': np.random.lognormal(10 + cycle*0.1, 2, 100),
111        'tenure_months': np.random.poisson(24, 100)
112    })
113
114    drift_results = drift_monitor.check_drift(current_df)
115    dashboard.display_drift_status(drift_results)
116
117    print(f"\n{'='*70}")
118    print("Dashboard session complete")
119    print(f"{'='*70}")
120
121    solution_exercise_6()

```

14.9.7 Exercise 7: Custom Drift Detection with Multiple Methods

Problem: Implement a custom drift detection system that combines multiple statistical methods (KS test, PSI, Jensen-Shannon divergence) and creates an ensemble score to make drift decisions more robust.

Solution:

```

1  # Exercise 7 Solution
2  from typing import Dict, List, Tuple
3  from dataclasses import dataclass
4  from enum import Enum
5
6  class DriftSeverity(Enum):
7      """Drift severity levels"""

```

```

8  NONE = "none"
9  LOW = "low"
10 MEDIUM = "medium"
11 HIGH = "high"
12 CRITICAL = "critical"
13
14 @dataclass
15 class EnsembleDriftResult:
16     """Result from ensemble drift detection"""
17     feature_name: str
18     ensemble_score: float
19     severity: DriftSeverity
20     individual_results: Dict[str, DriftResult]
21     recommendation: str
22
23 class EnsembleDriftDetector:
24     """
25     Ensemble drift detector using multiple statistical methods
26
27     Combines KS test, PSI, and Jensen-Shannon divergence for
28     robust drift detection with configurable thresholds
29     """
30
31     def __init__(self,
32                 ks_threshold: float = 0.05,
33                 psi_threshold: float = 0.2,
34                 js_threshold: float = 0.15,
35                 ensemble_weights: Dict[str, float] = None):
36         self.detector = DriftDetector()
37         self.ks_threshold = ks_threshold
38         self.psi_threshold = psi_threshold
39         self.js_threshold = js_threshold
40
41         # Default weights for ensemble
42         self.weights = ensemble_weights or {
43             'ks': 0.35,
44             'psi': 0.35,
45             'js': 0.30
46         }
47
48     def detect_ensemble_drift(self,
49                             reference_data: pd.Series,
50                             current_data: pd.Series,
51                             feature_name: str) -> EnsembleDriftResult:
52         """Detect drift using ensemble of methods"""
53
54         # Run all detection methods
55         ks_result = self.detector.detect_drift_ks(reference_data, current_data)
56         psi_result = self.detector.detect_drift_psi(
57             reference_data, current_data, psi_threshold=self.psi_threshold
58         )
59         js_result = self.detector.detect_drift_jensen_shannon(
60             reference_data, current_data
61         )
62
63         # Store individual results
64         individual_results = {
65             'ks': ks_result,

```

```

66         'psi': psi_result,
67         'js': js_result
68     }
69
70     # Calculate normalized scores (0-1 scale)
71     ks_score = 1 - ks_result.drift_score # p-value: lower = more drift
72     psi_score = min(psi_result.drift_score / 0.5, 1.0) # normalize PSI
73     js_score = min(js_result.drift_score / 0.5, 1.0) # normalize JS
74
75     # Weighted ensemble score
76     ensemble_score = (
77         self.weights['ks'] * ks_score +
78         self.weights['psi'] * psi_score +
79         self.weights['js'] * js_score
80     )
81
82     # Determine severity
83     severity = self._calculate_severity(ensemble_score)
84
85     # Generate recommendation
86     recommendation = self._generate_recommendation(
87         severity, individual_results, feature_name
88     )
89
90     return EnsembleDriftResult(
91         feature_name=feature_name,
92         ensemble_score=ensemble_score,
93         severity=severity,
94         individual_results=individual_results,
95         recommendation=recommendation
96     )
97
98     def _calculate_severity(self, ensemble_score: float) -> DriftSeverity:
99         """Calculate drift severity from ensemble score"""
100         if ensemble_score < 0.2:
101             return DriftSeverity.NONE
102         elif ensemble_score < 0.4:
103             return DriftSeverity.LOW
104         elif ensemble_score < 0.6:
105             return DriftSeverity.MEDIUM
106         elif ensemble_score < 0.8:
107             return DriftSeverity.HIGH
108         else:
109             return DriftSeverity.CRITICAL
110
111     def _generate_recommendation(self,
112                                 severity: DriftSeverity,
113                                 individual_results: Dict[str, DriftResult],
114                                 feature_name: str) -> str:
115         """Generate actionable recommendation"""
116
117         if severity == DriftSeverity.NONE:
118             return f"No action required for {feature_name}"
119
120         elif severity == DriftSeverity.LOW:
121             return f"Monitor {feature_name} closely; consider investigating if trend
122             continues"

```

```

123         elif severity == DriftSeverity.MEDIUM:
124             return f"Investigate {feature_name} distribution change; may need model
retraining soon"
125
126         elif severity == DriftSeverity.HIGH:
127             methods_detected = [
128                 name for name, result in individual_results.items()
129                 if result.drift_detected
130             ]
131             return f"ACTION REQUIRED: {feature_name} showing drift in {
methods_detected}; schedule model retraining"
132
133         else: # CRITICAL
134             return f"URGENT: Critical drift in {feature_name}; consider model rollback
or immediate retraining"
135
136 def solution_exercise_7():
137     """Ensemble drift detection demonstration"""
138
139     # Create reference data
140     np.random.seed(42)
141     reference_df = pd.DataFrame({
142         'age': np.random.normal(45, 12, 2000),
143         'income': np.random.lognormal(10.5, 0.8, 2000),
144         'transaction_count': np.random.poisson(15, 2000),
145         'credit_score': np.random.normal(680, 60, 2000)
146     })
147
148     # Create ensemble detector
149     ensemble = EnsembleDriftDetector(
150         ks_threshold=0.05,
151         psi_threshold=0.2,
152         js_threshold=0.15
153     )
154
155     # Test scenarios with different drift levels
156     scenarios = {
157         'No Drift': pd.DataFrame({
158             'age': np.random.normal(45, 12, 500),
159             'income': np.random.lognormal(10.5, 0.8, 500),
160             'transaction_count': np.random.poisson(15, 500),
161             'credit_score': np.random.normal(680, 60, 500)
162         }),
163         'Low Drift': pd.DataFrame({
164             'age': np.random.normal(46.5, 12, 500),
165             'income': np.random.lognormal(10.6, 0.8, 500),
166             'transaction_count': np.random.poisson(16, 500),
167             'credit_score': np.random.normal(685, 60, 500)
168         }),
169         'Medium Drift': pd.DataFrame({
170             'age': np.random.normal(50, 13, 500),
171             'income': np.random.lognormal(10.8, 0.9, 500),
172             'transaction_count': np.random.poisson(20, 500),
173             'credit_score': np.random.normal(700, 65, 500)
174         }),
175         'High Drift': pd.DataFrame({
176             'age': np.random.normal(55, 15, 500),
177             'income': np.random.lognormal(11.2, 1.0, 500),

```

```

178         'transaction_count': np.random.poisson(30, 500),
179         'credit_score': np.random.normal(730, 70, 500)
180     })
181 }
182
183 print("=" * 90)
184 print("ENSEMBLE DRIFT DETECTION ANALYSIS")
185 print("=" * 90)
186
187 for scenario_name, current_df in scenarios.items():
188     print(f"\n{'='*90}")
189     print(f"Scenario: {scenario_name}")
190     print(f"{'='*90}")
191
192     for feature in reference_df.columns:
193         result = ensemble.detect_ensemble_drift(
194             reference_data=reference_df[feature],
195             current_data=current_df[feature],
196             feature_name=feature
197         )
198
199         print(f"\nFeature: {feature}")
200         print(f"    Ensemble Score: {result.ensemble_score:.3f}")
201         print(f"    Severity: {result.severity.value.upper()}")
202
203         print(f"\n    Individual Method Results:")
204         for method, drift_result in result.individual_results.items():
205             status = "DRIFT" if drift_result.drift_detected else "OK"
206             print(f"        {method.upper():6s}: {status:5s} (score={drift_result.
drift_score:.4f})")
207
208         print(f"\n    Recommendation:")
209         print(f"        {result.recommendation}")
210
211     # Calculate overall drift percentage
212     all_results = [
213         ensemble.detect_ensemble_drift(
214             reference_data=reference_df[feature],
215             current_data=current_df[feature],
216             feature_name=feature
217         )
218         for feature in reference_df.columns
219     ]
220
221     drifted_features = sum(
222         1 for r in all_results
223         if r.severity not in [DriftSeverity.NONE, DriftSeverity.LOW]
224     )
225
226     print(f"\n{'='*90}")
227     print(f"Summary: {drifted_features}/{len(reference_df.columns)} features
showing medium+ drift")
228     print(f"{'='*90}")
229
230 solution_exercise_7()

```


14.9.8 Exercise 8: End-to-End Production Monitoring Pipeline

Problem: Build a complete production monitoring pipeline that integrates structured logging, metrics collection, drift detection, performance tracking, alerting, and automated runbooks for a real-time ML service.

Solution:

```

1  # Exercise 8 Solution
2  from typing import Dict, Any, Optional, List
3  from dataclasses import dataclass, field
4  from datetime import datetime, timedelta
5  import threading
6  import time
7  import queue
8
9  @dataclass
10 class ModelPredictionRequest:
11     """Prediction request with full context"""
12     request_id: str
13     model_name: str
14     model_version: str
15     features: Dict[str, Any]
16     timestamp: datetime = field(default_factory=datetime.now)
17     metadata: Dict[str, Any] = field(default_factory=dict)
18
19 @dataclass
20 class ModelPredictionResponse:
21     """Prediction response with monitoring data"""
22     request_id: str
23     prediction: Any
24     confidence: float
25     latency_ms: float
26     model_name: str
27     model_version: str
28     timestamp: datetime = field(default_factory=datetime.now)
29
30 class ProductionMLMonitoringPipeline:
31     """
32     Complete production monitoring pipeline for ML services
33
34     Integrates all monitoring components:
35     - Structured logging
36     - Metrics collection
37     - Drift detection
38     - Performance tracking
39     - Alerting with runbooks
40     - Health checks
41     """
42
43     def __init__(self,
44                 service_name: str,
45                 model_name: str,
46                 model_version: str,
47                 reference_data: pd.DataFrame):
48         # Initialize all monitoring components
49         self.service_name = service_name
50         self.model_name = model_name
51         self.model_version = model_version

```

```

52
53     # Logging
54     self.logger = MLLogger(service_name, log_file=f'{service_name}.jsonl')
55
56     # Metrics
57     self.collector = MetricsCollector(buffer_size=100000)
58     self.monitoring = MonitoringArchitecture(self.collector)
59
60     # Drift detection
61     self.drift_monitor = DataDriftMonitor(reference_data)
62     self.drift_check_interval = 3600 # Check every hour
63
64     # Performance monitoring
65     self.perf_monitor = PerformanceMonitor(
66         self.collector,
67         performance_window_hours=24
68     )
69
70     # Data quality
71     self.quality_monitor = DataQualityMonitor()
72     self.quality_monitor.set_baseline(reference_data)
73
74     # Prometheus export
75     self.prometheus = PrometheusExporter(port=8000)
76
77     # Alert system
78     self.alert_manager = AlertManager()
79     self._setup_alerts()
80
81     # Request queue for async processing
82     self.request_queue = queue.Queue(maxsize=10000)
83     self.processing_thread = None
84     self.running = False
85
86     # Statistics
87     self.stats = {
88         'total_predictions': 0,
89         'total_errors': 0,
90         'drift_checks': 0,
91         'quality_violations': 0
92     }
93
94     def _setup_alerts(self):
95         """Configure monitoring alerts with runbooks"""
96
97         # High error rate alert
98         def check_error_rate(collector: MetricsCollector) -> bool:
99             recent_errors = collector.counters.get('prediction_errors', 0)
100             recent_total = collector.counters.get('predictions_total', 1)
101             return (recent_errors / recent_total) > 0.05
102
103         error_alert = Alert(
104             alert_id='high_error_rate',
105             name='High Prediction Error Rate',
106             condition=check_error_rate,
107             severity=AlertSeverity.ERROR,
108             message_template='Error rate above 5%: {error_rate:.2%}',
109             cooldown_minutes=15

```

```

110     )
111
112     # High latency alert
113     def check_latency(collector: MetricsCollector) -> bool:
114         percentiles = collector.get_histogram_percentiles(
115             'prediction_latency_ms',
116             percentiles=[95]
117         )
118         return percentiles.get(95, 0) > 500
119
120     latency_alert = Alert(
121         alert_id='high_latency',
122         name='High Prediction Latency',
123         condition=check_latency,
124         severity=AlertSeverity.WARNING,
125         message_template='P95 latency above 500ms: {p95_latency:.2f}ms',
126         cooldown_minutes=10
127     )
128
129     # Add alerts
130     self.monitoring.add_alert(error_alert)
131     self.monitoring.add_alert(latency_alert)
132
133     # Setup runbooks
134     self._setup_runbooks()
135
136     def _setup_runbooks(self):
137         """Configure automated incident response"""
138
139         # Error rate runbook
140         error_runbook = IncidentRunbook(
141             incident_type='high_error_rate',
142             steps=[
143                 ('check_model_health', self._check_model_health),
144                 ('check_data_quality', self._check_data_quality),
145                 ('log_recent_errors', self._log_recent_errors),
146                 ('notify_oncall', self._notify_oncall)
147             ]
148         )
149
150         # Latency runbook
151         latency_runbook = IncidentRunbook(
152             incident_type='high_latency',
153             steps=[
154                 ('check_resource_usage', self._check_resources),
155                 ('analyze_batch_size', self._analyze_batch_size),
156                 ('check_dependencies', self._check_dependencies)
157             ]
158         )
159
160         self.alert_manager.runbooks = {
161             'high_error_rate': error_runbook,
162             'high_latency': latency_runbook
163         }
164
165         # Runbook action implementations
166         def _check_model_health(self, context: Dict) -> Dict:
167             health = self.monitoring.get_health_status()

```

```

168         return {'model_health': health}
169
170     def _check_data_quality(self, context: Dict) -> Dict:
171         report = self.quality_monitor.get_quality_report()
172         return {'quality_report': report}
173
174     def _log_recent_errors(self, context: Dict) -> Dict:
175         # Log recent prediction errors
176         return {'logged': True}
177
178     def _notify_oncall(self, context: Dict) -> Dict:
179         # Send notification to on-call engineer
180         print(f"[ALERT] Notifying on-call: {context}")
181         return {'notified': True}
182
183     def _check_resources(self, context: Dict) -> Dict:
184         # Check CPU/memory usage
185         return {'resources': 'healthy'}
186
187     def _analyze_batch_size(self, context: Dict) -> Dict:
188         # Analyze prediction batch sizes
189         return {'batch_analysis': 'complete'}
190
191     def _check_dependencies(self, context: Dict) -> Dict:
192         # Check external service health
193         return {'dependencies': 'healthy'}
194
195     def start(self):
196         """Start the monitoring pipeline"""
197         self.running = True
198         self.processing_thread = threading.Thread(target=self._process_predictions)
199         self.processing_thread.daemon = True
200         self.processing_thread.start()
201
202         # Start drift monitoring thread
203         self.drift_thread = threading.Thread(target=self._drift_monitoring_loop)
204         self.drift_thread.daemon = True
205         self.drift_thread.start()
206
207         self.logger.log_performance_metrics(
208             model_name=self.model_name,
209             model_version=self.model_version,
210             metrics={'status': 'started'},
211             window_hours=0
212         )
213
214     def stop(self):
215         """Stop the monitoring pipeline"""
216         self.running = False
217         if self.processing_thread:
218             self.processing_thread.join(timeout=5)
219
220     def predict(self, request: ModelPredictionRequest) -> ModelPredictionResponse:
221         """
222         Make prediction with full monitoring
223
224         Captures metrics, logs, and queues for async processing
225         """

```

```

226     start_time = time.time()
227
228     try:
229         # Validate data quality
230         features_df = pd.DataFrame([request.features])
231         violations = self.quality_monitor.validate_batch(features_df)
232
233         if any(v.severity == 'error' for v in violations):
234             self.stats['quality_violations'] += len(violations)
235             raise ValueError(f"Data quality violations: {violations}")
236
237         # Simulate prediction (replace with actual model)
238         prediction = self._run_model(request.features)
239         confidence = np.random.uniform(0.7, 0.99)
240
241         latency_ms = (time.time() - start_time) * 1000
242
243         # Create response
244         response = ModelPredictionResponse(
245             request_id=request.request_id,
246             prediction=prediction,
247             confidence=confidence,
248             latency_ms=latency_ms,
249             model_name=self.model_name,
250             model_version=self.model_version
251         )
252
253         # Record metrics
254         self._record_metrics(request, response)
255
256         # Log prediction
257         self.logger.log_prediction(
258             model_name=self.model_name,
259             model_version=self.model_version,
260             prediction_id=request.request_id,
261             features=request.features,
262             prediction=prediction,
263             confidence=confidence,
264             latency_ms=latency_ms
265         )
266
267         # Queue for async processing
268         try:
269             self.request_queue.put((request, response), block=False)
270         except queue.Full:
271             self.logger.log_model_error(
272                 model_name=self.model_name,
273                 model_version=self.model_version,
274                 error_type='QueueFull',
275                 error_message='Monitoring queue is full'
276             )
277
278         self.stats['total_predictions'] += 1
279
280         return response
281
282     except Exception as e:
283         self.stats['total_errors'] += 1

```

```

284         self.logger.log_model_error(
285             model_name=self.model_name,
286             model_version=self.model_version,
287             error_type=type(e).__name__,
288             error_message=str(e),
289             prediction_id=request.request_id
290         )
291
292         # Record error metrics
293         self.collector.record_counter(
294             'prediction_errors',
295             labels={
296                 'model_name': self.model_name,
297                 'error_type': type(e).__name__
298             }
299         )
300
301         raise
302
303     def _run_model(self, features: Dict[str, Any]) -> Any:
304         """Simulate model inference"""
305         time.sleep(0.02) # Simulate inference time
306         return np.random.choice([0, 1], p=[0.7, 0.3])
307
308     def _record_metrics(self,
309                        request: ModelPredictionRequest,
310                        response: ModelPredictionResponse):
311         """Record prediction metrics"""
312
313         labels = {
314             'model_name': self.model_name,
315             'model_version': self.model_version
316         }
317
318         # Counter
319         self.collector.record_counter('predictions_total', labels=labels)
320
321         # Latency histogram
322         self.collector.record_histogram(
323             'prediction_latency_ms',
324             response.latency_ms,
325             labels=labels
326         )
327
328         # Confidence gauge
329         self.collector.record_gauge(
330             'prediction_confidence',
331             response.confidence,
332             labels=labels
333         )
334
335         # Prometheus metrics
336         self.prometheus.record_prediction(
337             model_name=self.model_name,
338             model_version=self.model_version,
339             latency_seconds=response.latency_ms / 1000,
340             confidence=response.confidence
341

```

```

342     )
343
344     def _process_predictions(self):
345         """Async processing of predictions"""
346         batch = []
347         batch_size = 100
348
349         while self.running:
350             try:
351                 request, response = self.request_queue.get(timeout=1)
352                 batch.append((request, response))
353
354                 if len(batch) >= batch_size:
355                     self._process_batch(batch)
356                     batch = []
357
358             except queue.Empty:
359                 if batch:
360                     self._process_batch(batch)
361                     batch = []
362
363     def _process_batch(self, batch: List[Tuple]):
364         """Process batch of predictions"""
365         # Could do batch drift detection, performance analysis, etc.
366         pass
367
368     def _drift_monitoring_loop(self):
369         """Periodic drift detection"""
370         while self.running:
371             time.sleep(self.drift_check_interval)
372
373             # Get recent predictions for drift check
374             # (In production, fetch from storage)
375             self.stats['drift_checks'] += 1
376
377     def get_monitoring_report(self) -> Dict[str, Any]:
378         """Generate comprehensive monitoring report"""
379
380         health = self.monitoring.get_health_status()
381         quality = self.quality_monitor.get_quality_report()
382
383         return {
384             'timestamp': datetime.now().isoformat(),
385             'service': self.service_name,
386             'model': {
387                 'name': self.model_name,
388                 'version': self.model_version
389             },
390             'statistics': self.stats,
391             'health': health,
392             'data_quality': quality,
393             'recent_alerts': [
394                 {
395                     'alert_id': a.alert_id,
396                     'name': a.name,
397                     'severity': a.severity.value,
398                     'last_triggered': a.last_triggered.isoformat() if a.last_triggered
399                 }
400             ]
401         }
402     else None

```

```

399         }
400         for a in self.monitoring.alerts
401     ]
402     }
403
404 def solution_exercise_8():
405     """Production monitoring pipeline demonstration"""
406
407     # Setup reference data
408     np.random.seed(42)
409     reference_data = pd.DataFrame({
410         'transaction_amount': np.random.lognormal(5, 2, 5000),
411         'account_age_days': np.random.exponential(730, 5000),
412         'transaction_count_30d': np.random.poisson(25, 5000),
413         'merchant_risk_score': np.random.beta(2, 5, 5000)
414     })
415
416     # Create monitoring pipeline
417     pipeline = ProductionMLMonitoringPipeline(
418         service_name='fraud_detection_service',
419         model_name='fraud_detector',
420         model_version='v4.2',
421         reference_data=reference_data
422     )
423
424     # Start pipeline
425     pipeline.start()
426
427     print("=" * 90)
428     print("PRODUCTION ML MONITORING PIPELINE")
429     print("=" * 90)
430
431     # Simulate production traffic
432     print("\nSimulating production predictions...")
433
434     for i in range(500):
435         # Generate request
436         request = ModelPredictionRequest(
437             request_id=f'req_{i:06d}',
438             model_name='fraud_detector',
439             model_version='v4.2',
440             features={
441                 'transaction_amount': float(np.random.lognormal(5, 2)),
442                 'account_age_days': int(np.random.exponential(730)),
443                 'transaction_count_30d': int(np.random.poisson(25)),
444                 'merchant_risk_score': float(np.random.beta(2, 5))
445             },
446             metadata={'source': 'api', 'region': 'us-east-1'}
447         )
448
449         try:
450             response = pipeline.predict(request)
451
452             if i % 100 == 0:
453                 print(f"  Processed {i} predictions...")
454
455         except Exception as e:
456             print(f"  Error in prediction {request.request_id}: {e}")

```



```

457         # Simulate realistic request rate
458         time.sleep(0.001)
459
460     # Generate final report
461     time.sleep(2) # Allow async processing to complete
462
463     print("\n" + "=" * 90)
464     print("MONITORING REPORT")
465     print("=" * 90)
466
467     report = pipeline.get_monitoring_report()
468
469     print(f"\nService: {report['service']}")
470     print(f"Model: {report['model']['name']} v{report['model']['version']}")
471
472     print(f"\nStatistics:")
473     for key, value in report['statistics'].items():
474         print(f"    {key}: {value}")
475
476     print(f"\nHealth Status: {report['health']['status'].upper()}")
477     print(f"    Total predictions (5min): {report['health']['total_predictions']}")
478     print(f"    Error rate: {report['health']['error_rate']:.2%}")
479
480     print(f"\nData Quality: {report['data_quality']['status'].upper()}")
481     print(f"    Total violations: {report['data_quality']['total_violations']}")
482
483     # Stop pipeline
484     pipeline.stop()
485
486     print("\n" + "=" * 90)
487     print("Pipeline stopped successfully")
488     print("=" * 90)
489
490 solution_exercise_8()
491

```

14.10 Chapter Summary

This chapter covered comprehensive ML monitoring and observability, essential for maintaining production ML systems. We explored the complete monitoring stack from metrics collection to incident response.

14.10.1 Key Concepts Covered

Monitoring Architecture:

- Complete metrics collection system with counters, gauges, histograms
- Health status monitoring and alerting
- Time-series metrics storage and retrieval
- Production-grade metric buffering and flushing

Data Drift Detection:

- Kolmogorov-Smirnov test for continuous features
- Chi-squared test for categorical features
- Population Stability Index (PSI) for industry-standard monitoring
- Jensen-Shannon divergence for distribution comparison
- Integration with Evidently AI for advanced reporting

Performance Monitoring:

- Prediction and ground truth tracking
- Automated metric calculation (accuracy, precision, recall, F1, MAE, RMSE)
- Performance trend analysis
- Statistical degradation detection
- Sliding window evaluation

Prometheus and Grafana:

- ML-specific metric exporters
- Standard Prometheus metric types (Counter, Gauge, Histogram, Summary)
- Grafana dashboard generation
- PromQL queries for ML metrics
- Alert rule configuration

Alert Systems:

- Intelligent alerting with cooldowns
- Multi-channel notifications (Slack, PagerDuty, email)
- Incident management with tracking
- Automated runbook execution
- Resolution workflows

Observability:

- Structured logging for ML systems
- Distributed tracing through ML pipelines
- Feature store observability
- Model explainability integration
- Cost and security monitoring

14.10.2 Best Practices

1. **Monitor continuously:** Track metrics, drift, and performance in real-time
2. **Use multiple drift methods:** Different tests catch different types of drift
3. **Set intelligent thresholds:** Balance sensitivity and specificity
4. **Automate responses:** Use runbooks for common incidents
5. **Track lineage:** Connect metrics to specific model versions
6. **Implement graceful degradation:** Plan for performance issues
7. **Monitor the monitors:** Ensure monitoring systems are healthy
8. **Regular review:** Periodically assess alert effectiveness

14.10.3 Common Pitfalls

- **Alert fatigue:** Too many low-value alerts reduce effectiveness
- **Delayed ground truth:** Long feedback loops prevent timely detection
- **Ignoring gradual drift:** Small changes accumulate to major issues
- **Single metric focus:** Monitor multiple performance dimensions
- **Missing context:** Alerts need actionable information
- **Over-sensitive drift detection:** Natural variation triggers false alerts
- **No baseline:** Insufficient reference data for comparison

14.10.4 Production Checklist

Before deploying monitoring to production:

1. Metrics collection covers all critical dimensions
2. Reference datasets established for drift detection
3. Alert thresholds tuned with historical data
4. Notification channels tested and working
5. Runbooks documented and automated
6. Dashboards created for key stakeholders
7. Monitoring system itself is monitored
8. Incident response procedures documented
9. On-call rotation established
10. Regular review cadence scheduled

14.10.5 Integration Points

ML monitoring integrates with broader infrastructure:

- **Model registry:** Track which models are deployed
- **Feature store:** Monitor feature pipeline health
- **CI/CD:** Trigger retraining or rollbacks
- **Incident management:** Create tickets automatically
- **Cost management:** Track inference spend
- **Compliance systems:** Audit model decisions

By implementing comprehensive monitoring and observability, teams can confidently operate ML systems at scale, detect issues before they impact users, and continuously improve model performance. The combination of metrics collection, drift detection, performance tracking, and intelligent alerting provides complete visibility into ML system health.

Chapter 15

Model Maintenance and Retraining

15.1 Learning Objectives

After completing this chapter, readers will be able to:

- Design and implement automated retraining pipeline architectures
- Configure intelligent trigger mechanisms for model retraining
- Implement incremental learning and online updating strategies
- Handle concept drift and data drift in production systems
- Balance model freshness against stability requirements
- Build continuous learning systems for real-time adaptation
- Automate model maintenance workflows end-to-end
- Monitor and optimize retraining costs and performance

15.2 Introduction

Machine learning models degrade over time as the world changes. Customer behavior shifts, market conditions evolve, and data distributions drift from training assumptions. Without systematic maintenance, production models become stale, accuracy drops, and business value erodes.

Model maintenance encompasses the complete lifecycle of keeping models relevant and performant:

- **Performance monitoring:** Continuous tracking of model metrics
- **Drift detection:** Identifying when retraining is needed
- **Automated retraining:** Triggering and executing model updates
- **Validation:** Ensuring new models improve upon current ones
- **Deployment:** Safely rolling out updated models

- **Rollback:** Reverting problematic updates quickly

This chapter provides production-ready architectures and code for building robust model maintenance systems that keep ML applications healthy without manual intervention.

15.3 Automated Retraining Pipeline Architecture

15.3.1 Core Components

A production retraining pipeline consists of multiple coordinated components working together to maintain model quality.

```

1 from typing import Dict, List, Optional, Any, Callable
2 from dataclasses import dataclass, field
3 from datetime import datetime, timedelta
4 from enum import Enum
5 import pandas as pd
6 import numpy as np
7 from pathlib import Path
8 import logging
9 import json
10 import pickle
11
12 class RetrainingTrigger(Enum):
13     """Triggers that can initiate retraining"""
14     PERFORMANCE_DEGRADATION = "performance_degradation"
15     DATA_DRIFT = "data_drift"
16     CONCEPT_DRIFT = "concept_drift"
17     SCHEDULED = "scheduled"
18     MANUAL = "manual"
19     DATA_VOLUME = "data_volume"
20     TIME_BASED = "time_based"
21
22 class ModelStatus(Enum):
23     """Model lifecycle status"""
24     TRAINING = "training"
25     VALIDATING = "validating"
26     STAGING = "staging"
27     PRODUCTION = "production"
28     ARCHIVED = "archived"
29     FAILED = "failed"
30
31 @dataclass
32 class RetrainingConfig:
33     """Configuration for retraining pipeline"""
34     model_name: str
35     model_type: str
36
37     # Trigger thresholds
38     performance_threshold: float = 0.05 # 5% degradation
39     drift_threshold: float = 0.15
40     min_data_points: int = 1000
41     max_days_without_retrain: int = 30
42
43     # Training settings

```

```

44 validation_split: float = 0.2
45 test_split: float = 0.1
46 max_training_time_hours: int = 24
47
48 # Deployment settings
49 canary_percentage: float = 0.1
50 canary_duration_hours: int = 24
51 auto_promote: bool = False
52
53 # Resource limits
54 max_memory_gb: int = 32
55 max_cpu_cores: int = 16
56 use_gpu: bool = False
57
58 @dataclass
59 class RetrainingEvent:
60     """Record of a retraining event"""
61     event_id: str
62     timestamp: datetime
63     trigger: RetrainingTrigger
64     model_version: str
65     metrics: Dict[str, float]
66     data_stats: Dict[str, Any]
67     training_duration: Optional[float] = None
68     success: bool = False
69     error_message: Optional[str] = None
70
71 class DataCollector:
72     """
73     Collect and manage training data for retraining
74
75     Handles incremental data collection, quality checks,
76     and dataset versioning
77     """
78
79     def __init__(self, data_path: Path, max_samples: int = 1000000):
80         self.data_path = data_path
81         self.max_samples = max_samples
82         self.logger = logging.getLogger(__name__)
83
84     def collect_new_data(self,
85                         since: datetime,
86                         quality_checks: bool = True) -> pd.DataFrame:
87         """Collect new training data since timestamp"""
88
89         # In production, this would query your data warehouse
90         # For demo, simulate data collection
91         self.logger.info(f"Collecting data since {since}")
92
93         # Simulate fetching new data
94         new_data = self._fetch_from_source(since)
95
96         if quality_checks:
97             new_data = self._apply_quality_checks(new_data)
98
99         self.logger.info(f"Collected {len(new_data)} new samples")
100         return new_data
101

```

```

102 def _fetch_from_source(self, since: datetime) -> pd.DataFrame:
103     """
104     Fetch data from source.
105
106     Reference implementation: load a local Parquet/CSV snapshot if available;
107     otherwise generate a synthetic dataset so the example remains runnable.
108     """
109     try:
110         if self.data_path.exists():
111             self.logger.info(f>Loading data from {self.data_path}")
112             if self.data_path.suffix == ".parquet":
113                 df = pd.read_parquet(self.data_path)
114             else:
115                 df = pd.read_csv(self.data_path)
116         else:
117             self.logger.warning(
118                 f">{self.data_path} not found. Generating synthetic dataset for
demo."
119             )
120             # Synthetic classification data with drift on timestamp
121             n_rows = min(self.max_samples, 10_000)
122             rng = np.random.default_rng(42)
123             df = pd.DataFrame(
124                 rng.normal(size=(n_rows, 10)),
125                 columns=[f"feature_{i}" for i in range(10)]
126             )
127             df["timestamp"] = pd.date_range(end=datetime.now(), periods=n_rows,
freq="min")
128             df["target"] = (df.sum(axis=1) > 0).astype(int)
129
130             # Filter by timestamp if present
131             if "timestamp" in df.columns:
132                 df["timestamp"] = pd.to_datetime(df["timestamp"], errors="coerce")
133                 df = df[df["timestamp"] >= since]
134
135             if df.empty:
136                 raise ValueError("No data available after filtering by timestamp")
137
138             return df
139
140     except Exception as e:
141         self.logger.error(f">Failed to fetch data: {e}")
142         raise
143
144 def _apply_quality_checks(self, data: pd.DataFrame) -> pd.DataFrame:
145     """Apply data quality checks"""
146
147     # Remove duplicates
148     initial_count = len(data)
149     data = data.drop_duplicates()
150
151     # Remove nulls in critical columns
152     data = data.dropna(subset=['target'])
153
154     # Remove outliers (example: using IQR)
155     for col in data.select_dtypes(include=[np.number]).columns:
156         Q1 = data[col].quantile(0.25)
157         Q3 = data[col].quantile(0.75)

```



```

158         IQR = Q3 - Q1
159         data = data[
160             (data[col] >= Q1 - 3*IQR) &
161             (data[col] <= Q3 + 3*IQR)
162         ]
163
164         removed = initial_count - len(data)
165         if removed > 0:
166             self.logger.warning(f"Removed {removed} samples in quality checks")
167
168         return data
169
170     def get_training_dataset(self,
171                             days_back: int = 90,
172                             include_validation: bool = True) -> Dict[str, pd.DataFrame
173     ]:
174         """Get complete training dataset"""
175
176         cutoff_date = datetime.now() - timedelta(days=days_back)
177         data = self.collect_new_data(cutoff_date)
178
179         # Limit to max samples
180         if len(data) > self.max_samples:
181             data = data.sample(n=self.max_samples, random_state=42)
182
183         if not include_validation:
184             return {'train': data}
185
186         # Split into train/val
187         from sklearn.model_selection import train_test_split
188         train, val = train_test_split(data, test_size=0.2, random_state=42)
189
190         return {
191             'train': train,
192             'validation': val
193         }
194
195     class PerformanceEvaluator:
196         """
197         Evaluate model performance and compare versions
198
199         Determines if new model is better than current production model
200         """
201
202         def __init__(self, metrics: List[str] = None):
203             self.metrics = metrics or ['accuracy', 'precision', 'recall', 'f1']
204             self.logger = logging.getLogger(__name__)
205
206         def evaluate_model(self,
207                           model: Any,
208                           test_data: pd.DataFrame,
209                           target_col: str = 'target') -> Dict[str, float]:
210             """Evaluate model on test data"""
211
212             X_test = test_data.drop(columns=[target_col])
213             y_test = test_data[target_col]
214
215             predictions = model.predict(X_test)

```

```

215
216 from sklearn.metrics import (
217     accuracy_score, precision_score,
218     recall_score, f1_score
219 )
220
221 results = {
222     'accuracy': accuracy_score(y_test, predictions),
223     'precision': precision_score(y_test, predictions, average='weighted'),
224     'recall': recall_score(y_test, predictions, average='weighted'),
225     'f1': f1_score(y_test, predictions, average='weighted'),
226     'samples': len(test_data)
227 }
228
229 return results
230
231 def compare_models(self,
232     new_metrics: Dict[str, float],
233     current_metrics: Dict[str, float],
234     primary_metric: str = 'f1',
235     min_improvement: float = 0.01) -> Dict[str, Any]:
236     """Compare new model against current production model"""
237
238     comparison = {
239         'new_better': False,
240         'improvement': {},
241         'primary_metric': primary_metric,
242         'recommendation': 'keep_current'
243     }
244
245     for metric in self.metrics:
246         if metric in new_metrics and metric in current_metrics:
247             improvement = new_metrics[metric] - current_metrics[metric]
248             comparison['improvement'][metric] = improvement
249
250     # Check primary metric
251     if primary_metric in comparison['improvement']:
252         improvement = comparison['improvement'][primary_metric]
253
254         if improvement > min_improvement:
255             comparison['new_better'] = True
256             comparison['recommendation'] = 'promote_new'
257             self.logger.info(
258                 f"New model shows {improvement:.4f} improvement "
259                 f"in {primary_metric}"
260             )
261         elif improvement < -min_improvement:
262             comparison['recommendation'] = 'keep_current'
263             self.logger.warning(
264                 f"New model performs worse by {abs(improvement):.4f} "
265                 f"in {primary_metric}"
266             )
267         else:
268             comparison['recommendation'] = 'neutral'
269
270     return comparison
271
272 class AutomatedRetrainingPipeline:

```

```

273 """
274 Complete automated retraining pipeline
275
276 Orchestrates data collection, training, evaluation,
277 and deployment of updated models
278 """
279
280 def __init__(self, config: RetrainingConfig):
281     self.config = config
282     self.logger = logging.getLogger(__name__)
283
284     self.data_collector = DataCollector(
285         data_path=Path(f"./data/{config.model_name}")
286     )
287     self.evaluator = PerformanceEvaluator()
288
289     self.retraining_history: List[RetrainingEvent] = []
290     self.last_retrain_time: Optional[datetime] = None
291
292 def should_retrain(self,
293                   current_metrics: Dict[str, float],
294                   drift_score: float,
295                   days_since_retrain: int) -> tuple[bool, RetrainingTrigger]:
296     """Determine if retraining should be triggered"""
297
298     # Check performance degradation
299     baseline_metric = 0.85 # Load from model registry
300     current_metric = current_metrics.get('f1', 0)
301
302     degradation = baseline_metric - current_metric
303     if degradation > self.config.performance_threshold:
304         return True, RetrainingTrigger.PERFORMANCE_DEGRADATION
305
306     # Check drift
307     if drift_score > self.config.drift_threshold:
308         return True, RetrainingTrigger.DATA_DRIFT
309
310     # Check time-based schedule
311     if days_since_retrain >= self.config.max_days_without_retrain:
312         return True, RetrainingTrigger.SCHEDULED
313
314     return False, None
315
316 def execute_retraining(self,
317                       trigger: RetrainingTrigger,
318                       current_model: Any = None) -> RetrainingEvent:
319     """Execute complete retraining workflow"""
320
321     event_id = f"{self.config.model_name}_{datetime.now().isoformat()}"
322     event = RetrainingEvent(
323         event_id=event_id,
324         timestamp=datetime.now(),
325         trigger=trigger,
326         model_version=f"v{len(self.retraining_history) + 1}",
327         metrics={},
328         data_stats={}
329     )
330

```

```

331     try:
332         self.logger.info(
333             f"Starting retraining for {self.config.model_name} "
334             f"(trigger: {trigger.value})"
335         )
336
337         # Step 1: Collect training data
338         datasets = self.data_collector.get_training_dataset(
339             days_back=90,
340             include_validation=True
341         )
342
343         event.data_stats = {
344             'train_samples': len(datasets['train']),
345             'validation_samples': len(datasets['validation'])
346         }
347
348         # Step 2: Train new model
349         start_time = datetime.now()
350         new_model = self._train_model(datasets['train'])
351         event.training_duration = (datetime.now() - start_time).total_seconds()
352
353         # Step 3: Evaluate new model
354         new_metrics = self.evaluator.evaluate_model(
355             new_model,
356             datasets['validation']
357         )
358         event.metrics = new_metrics
359
360         # Step 4: Compare with current model
361         if current_model is not None:
362             current_metrics = self.evaluator.evaluate_model(
363                 current_model,
364                 datasets['validation']
365             )
366
367             comparison = self.evaluator.compare_models(
368                 new_metrics,
369                 current_metrics
370             )
371
372             if comparison['recommendation'] != 'promote_new':
373                 self.logger.warning(
374                     "New model not better than current - skipping deployment"
375                 )
376                 event.success = False
377                 event.error_message = "Model did not improve"
378                 return event
379
380         # Step 5: Save new model
381         self._save_model(new_model, event.model_version)
382
383         event.success = True
384         self.last_retrain_time = datetime.now()
385
386         self.logger.info(
387             f"Retraining completed successfully: {event.model_version} "
388             f"(F1: {new_metrics['f1']:.4f})"

```

```

389         )
390
391     except Exception as e:
392         self.logger.error(f"Retraining failed: {str(e)}")
393         event.success = False
394         event.error_message = str(e)
395
396     finally:
397         self.retraining_history.append(event)
398
399     return event
400
401 def _train_model(self, train_data: pd.DataFrame) -> Any:
402     """
403     Train a new model using a simple, configurable baseline.
404
405     Reference implementation: RandomForest with sensible defaults that can
406     be overridden via RetrainingConfig.model_type if extended.
407     """
408     from sklearn.ensemble import RandomForestClassifier
409     from sklearn.model_selection import train_test_split
410     from sklearn.metrics import f1_score
411
412     # Basic validation
413     if "target" not in train_data.columns:
414         raise ValueError("Training data must include 'target' column")
415
416     X = train_data.drop(columns=["target"])
417     y = train_data["target"]
418
419     # Simple hold-out for quick sanity during training
420     X_train, X_val, y_train, y_val = train_test_split(
421         X, y, test_size=0.2, random_state=42, stratify=y
422     )
423
424     model = RandomForestClassifier(
425         n_estimators=200,
426         max_depth=None,
427         min_samples_split=2,
428         random_state=42,
429         n_jobs=-1
430     )
431     model.fit(X_train, y_train)
432
433     # Quick validation to fail fast on obviously bad models
434     val_f1 = f1_score(y_val, model.predict(X_val), average="weighted")
435     self.logger.info(f"Validation F1 during retraining: {val_f1:.4f}")
436
437     return model
438
439 def _save_model(self, model: Any, version: str):
440     """Save model to storage"""
441
442     model_path = Path(f"./models/{self.config.model_name}/{version}")
443     model_path.mkdir(parents=True, exist_ok=True)
444
445     with open(model_path / "model.pkl", 'wb') as f:
446         pickle.dump(model, f)

```

```

447         self.logger.info(f"Model saved: {model_path}")
448
449     # Example usage
450     config = RetrainingConfig(
451         model_name="churn_predictor",
452         model_type="classification",
453         performance_threshold=0.05,
454         drift_threshold=0.15,
455         max_days_without_retrain=30
456     )
457
458     pipeline = AutomatedRetrainingPipeline(config)
459
460     # Check if retraining needed
461     current_metrics = {'f1': 0.78, 'accuracy': 0.82}
462     drift_score = 0.18
463     days_since_retrain = 15
464
465     should_retrain, trigger = pipeline.should_retrain(
466         current_metrics,
467         drift_score,
468         days_since_retrain
469     )
470
471     if should_retrain:
472         event = pipeline.execute_retraining(trigger)
473         print(f"Retraining completed: {event.success}")
474         print(f"New model metrics: {event.metrics}")
475

```

Listing 15.1: Complete Automated Retraining Pipeline

15.4 Trigger Mechanisms for Retraining

15.4.1 Multi-Criteria Trigger System

Production systems need sophisticated triggers that balance multiple signals to avoid both over-retraining (wasting resources) and under-retraining (allowing degradation).

```

1  from typing import Dict, List, Tuple, Optional
2  from dataclasses import dataclass
3  from datetime import datetime, timedelta
4  import numpy as np
5  from scipy import stats
6
7  @dataclass
8  class TriggerThreshold:
9      """Threshold configuration for a trigger"""
10     metric_name: str
11     threshold_value: float
12     comparison: str # 'gt', 'lt', 'gte', 'lte'
13     window_size: int = 1 # Number of consecutive violations
14     severity: str = 'medium' # 'low', 'medium', 'high', 'critical'
15
16  @dataclass
17  class TriggerEvaluation:

```

```

18     """Result of trigger evaluation"""
19     triggered: bool
20     trigger_type: RetrainingTrigger
21     severity: str
22     confidence: float
23     details: Dict[str, Any]
24     timestamp: datetime = field(default_factory=datetime.now)
25
26 class RetrainingTriggerManager:
27     """
28     Manage complex retraining trigger logic
29
30     Combines multiple signals (performance, drift, time, data volume)
31     with configurable thresholds and cooldown periods
32     """
33
34     def __init__(self):
35         self.logger = logging.getLogger(__name__)
36         self.trigger_history: List[TriggerEvaluation] = []
37         self.last_retrain_time: Optional[datetime] = None
38         self.cooldown_hours = 24 # Minimum time between retrains
39
40         # Configure trigger thresholds
41         self.thresholds = {
42             RetrainingTrigger.PERFORMANCE_DEGRADATION: [
43                 TriggerThreshold('accuracy', 0.05, 'gt', window_size=3, severity='high
44             ),
45                 TriggerThreshold('f1_score', 0.07, 'gt', window_size=2, severity='
46             critical'),
47             ],
48             RetrainingTrigger.DATA_DRIFT: [
49                 TriggerThreshold('psi_score', 0.2, 'gte', severity='medium'),
50                 TriggerThreshold('ks_statistic', 0.1, 'gte', severity='medium'),
51             ],
52             RetrainingTrigger.DATA_VOLUME: [
53                 TriggerThreshold('new_samples', 10000, 'gte', severity='low'),
54             ],
55         }
56
57         # Metric history for windowed evaluation
58         self.metric_history: Dict[str, List[float]] = {}
59
60     def evaluate_triggers(self,
61                         current_metrics: Dict[str, float],
62                         baseline_metrics: Dict[str, float],
63                         drift_metrics: Dict[str, float],
64                         data_stats: Dict[str, int]) -> List[TriggerEvaluation]:
65         """Evaluate all trigger conditions"""
66
67         evaluations = []
68
69         # Check if in cooldown period
70         if self._in_cooldown():
71             self.logger.info("In cooldown period - skipping trigger evaluation")
72             return evaluations
73
74         # 1. Performance degradation triggers
75         perf_eval = self._evaluate_performance_triggers(

```

```

74         current_metrics,
75         baseline_metrics
76     )
77     if perf_eval:
78         evaluations.append(perf_eval)
79
80     # 2. Data drift triggers
81     drift_eval = self._evaluate_drift_triggers(drift_metrics)
82     if drift_eval:
83         evaluations.append(drift_eval)
84
85     # 3. Data volume triggers
86     volume_eval = self._evaluate_volume_triggers(data_stats)
87     if volume_eval:
88         evaluations.append(volume_eval)
89
90     # 4. Time-based triggers
91     time_eval = self._evaluate_time_triggers()
92     if time_eval:
93         evaluations.append(time_eval)
94
95     # Store trigger evaluations
96     self.trigger_history.extend(evaluations)
97
98     return evaluations
99
100 def _in_cooldown(self) -> bool:
101     """Check if we're in cooldown period after last retrain"""
102     if self.last_retrain_time is None:
103         return False
104
105     time_since_retrain = datetime.now() - self.last_retrain_time
106     return time_since_retrain < timedelta(hours=self.cooldown_hours)
107
108 def _evaluate_performance_triggers(self,
109                                   current: Dict[str, float],
110                                   baseline: Dict[str, float]) -> Optional[
111 TriggerEvaluation]:
112     """Evaluate performance degradation triggers"""
113
114     thresholds = self.thresholds[RetrainingTrigger.PERFORMANCE_DEGRADATION]
115     violations = []
116
117     for threshold in thresholds:
118         metric = threshold.metric_name
119         if metric not in current or metric not in baseline:
120             continue
121
122         degradation = baseline[metric] - current[metric]
123
124         # Update metric history
125         if metric not in self.metric_history:
126             self.metric_history[metric] = []
127         self.metric_history[metric].append(degradation)
128
129         # Keep only window_size recent values
130         self.metric_history[metric] = \
            self.metric_history[metric][-threshold.window_size:]

```



```

131         # Check if threshold violated for entire window
132         recent_values = self.metric_history[metric]
133         if len(recent_values) >= threshold.window_size:
134             if threshold.comparison == 'gt':
135                 violated = all(v > threshold.threshold_value for v in
136 recent_values)
137             elif threshold.comparison == 'gte':
138                 violated = all(v >= threshold.threshold_value for v in
139 recent_values)
140             elif threshold.comparison == 'lt':
141                 violated = all(v < threshold.threshold_value for v in
142 recent_values)
143             else: # lte
144                 violated = all(v <= threshold.threshold_value for v in
145 recent_values)
146
147         if violated:
148             violations.append({
149                 'metric': metric,
150                 'degradation': np.mean(recent_values),
151                 'threshold': threshold.threshold_value,
152                 'severity': threshold.severity
153             })
154
155         if violations:
156             # Use highest severity
157             max_severity = max(v['severity'] for v in violations)
158
159             return TriggerEvaluation(
160                 triggered=True,
161                 trigger_type=RetrainingTrigger.PERFORMANCE_DEGRADATION,
162                 severity=max_severity,
163                 confidence=len(violations) / len(thresholds),
164                 details={'violations': violations}
165             )
166
167         return None
168
169     def _evaluate_drift_triggers(self,
170                                drift_metrics: Dict[str, float]) -> Optional[
171 TriggerEvaluation]:
172         """Evaluate data drift triggers"""
173
174         thresholds = self.thresholds[RetrainingTrigger.DATA_DRIFT]
175         violations = []
176
177         for threshold in thresholds:
178             metric = threshold.metric_name
179             if metric not in drift_metrics:
180                 continue
181
182             value = drift_metrics[metric]
183
184             # Check threshold
185             violated = False
186             if threshold.comparison == 'gte':
187                 violated = value >= threshold.threshold_value

```

```

184         elif threshold.comparison == 'gt':
185             violated = value > threshold.threshold_value
186
187         if violated:
188             violations.append({
189                 'metric': metric,
190                 'value': value,
191                 'threshold': threshold.threshold_value,
192                 'severity': threshold.severity
193             })
194
195     if violations:
196         max_severity = max(v['severity'] for v in violations)
197
198     return TriggerEvaluation(
199         triggered=True,
200         trigger_type=RetrainingTrigger.DATA_DRIFT,
201         severity=max_severity,
202         confidence=min(1.0, len(violations) * 0.5),
203         details={'violations': violations}
204     )
205
206     return None
207
208     def _evaluate_volume_triggers(self,
209                                   data_stats: Dict[str, int]) -> Optional[
210 TriggerEvaluation]:
211         """Evaluate data volume triggers"""
212
213         thresholds = self.thresholds[RetrainingTrigger.DATA_VOLUME]
214
215         for threshold in thresholds:
216             if threshold.metric_name in data_stats:
217                 value = data_stats[threshold.metric_name]
218
219                 if threshold.comparison == 'gte' and value >= threshold.
220 threshold_value:
221                     return TriggerEvaluation(
222                         triggered=True,
223                         trigger_type=RetrainingTrigger.DATA_VOLUME,
224                         severity=threshold.severity,
225                         confidence=min(1.0, value / threshold.threshold_value),
226                         details={'new_samples': value}
227                     )
228
229         return None
230
231     def _evaluate_time_triggers(self) -> Optional[TriggerEvaluation]:
232         """Evaluate time-based triggers"""
233
234         if self.last_retrain_time is None:
235             # Never retrained before
236             return TriggerEvaluation(
237                 triggered=True,
238                 trigger_type=RetrainingTrigger.SCHEDULED,
239                 severity='low',
240                 confidence=1.0,
241                 details={'reason': 'initial_training'}

```

```

240         )
241
242         days_since_retrain = (datetime.now() - self.last_retrain_time).days
243
244         # Monthly retraining schedule
245         if days_since_retrain >= 30:
246             return TriggerEvaluation(
247                 triggered=True,
248                 trigger_type=RetrainingTrigger.SCHEDULED,
249                 severity='medium',
250                 confidence=1.0,
251                 details={'days_since_retrain': days_since_retrain}
252             )
253
254         return None
255
256     def should_retrain(self,
257                       evaluations: List[TriggerEvaluation]) -> Tuple[bool, Optional[
258 TriggerEvaluation]]:
259         """Decide if retraining should proceed based on trigger evaluations"""
260
261         if not evaluations:
262             return False, None
263
264         # Filter to triggered evaluations
265         triggered = [e for e in evaluations if e.triggered]
266
267         if not triggered:
268             return False, None
269
270         # Priority order: critical > high > medium > low
271         severity_order = {'critical': 4, 'high': 3, 'medium': 2, 'low': 1}
272
273         # Sort by severity and confidence
274         triggered.sort(
275             key=lambda x: (severity_order.get(x.severity, 0), x.confidence),
276             reverse=True
277         )
278
279         primary_trigger = triggered[0]
280
281         self.logger.info(
282             f"Retraining triggered: {primary_trigger.trigger_type.value} "
283             f"(severity: {primary_trigger.severity}, "
284             f"confidence: {primary_trigger.confidence:.2f})"
285         )
286
287         return True, primary_trigger
288
289     def record_retrain(self):
290         """Record that retraining occurred"""
291         self.last_retrain_time = datetime.now()
292
293         # Clear metric history after retrain
294         self.metric_history.clear()
295
296 # Example usage
297 trigger_manager = RetrainingTriggerManager()

```

```

297
298 # Simulate monitoring cycle
299 current_metrics = {
300     'accuracy': 0.79,
301     'f1_score': 0.76
302 }
303
304 baseline_metrics = {
305     'accuracy': 0.85,
306     'f1_score': 0.84
307 }
308
309 drift_metrics = {
310     'psi_score': 0.25,
311     'ks_statistic': 0.12
312 }
313
314 data_stats = {
315     'new_samples': 12000
316 }
317
318 # Evaluate triggers
319 evaluations = trigger_manager.evaluate_triggers(
320     current_metrics,
321     baseline_metrics,
322     drift_metrics,
323     data_stats
324 )
325
326 # Check if should retrain
327 should_retrain, primary_trigger = trigger_manager.should_retrain(evaluations)
328
329 if should_retrain:
330     print(f"Triggering retraining due to: {primary_trigger.trigger_type.value}")
331     print(f"Severity: {primary_trigger.severity}")
332     print(f"Details: {primary_trigger.details}")
333
334     # Execute retraining...
335     trigger_manager.record_retrain()

```

Listing 15.2: Advanced Retraining Trigger System

15.5 Incremental Learning and Online Updating

15.5.1 Incremental Training Strategies

Incremental learning allows models to learn from new data without full retraining, reducing computational costs and enabling faster adaptation.

```

1 from typing import Dict, List, Optional, Any
2 import numpy as np
3 import pandas as pd
4 from sklearn.base import BaseEstimator
5 from datetime import datetime
6 import logging
7

```

```

8 class IncrementalLearner:
9     """
10     Manage incremental model updates
11
12     Supports partial_fit for online learning algorithms and
13     implements sliding window strategies for batch models
14     """
15
16     def __init__(self,
17                 base_model: BaseEstimator,
18                 update_strategy: str = 'sliding_window',
19                 window_size: int = 10000,
20                 update_frequency: int = 1000):
21         """
22         Initialize incremental learner
23
24         Parameters:
25         - base_model: Model supporting partial_fit or incremental training
26         - update_strategy: 'sliding_window', 'cumulative', or 'temporal_weighting'
27         - window_size: Max samples to keep in training buffer
28         - update_frequency: Trigger update every N new samples
29         """
30         self.model = base_model
31         self.update_strategy = update_strategy
32         self.window_size = window_size
33         self.update_frequency = update_frequency
34
35         self.training_buffer: List[Dict[str, Any]] = []
36         self.samples_since_update = 0
37         self.total_updates = 0
38
39         self.logger = logging.getLogger(__name__)
40
41     def add_sample(self, features: Dict[str, Any], label: Any):
42         """Add new labeled sample to training buffer"""
43
44         self.training_buffer.append({
45             'features': features,
46             'label': label,
47             'timestamp': datetime.now()
48         })
49
50         # Maintain window size
51         if len(self.training_buffer) > self.window_size:
52             if self.update_strategy == 'sliding_window':
53                 # Remove oldest sample
54                 self.training_buffer.pop(0)
55             elif self.update_strategy == 'cumulative':
56                 # Keep all but downsample old data
57                 self._downsample_old_data()
58
59         self.samples_since_update += 1
60
61         # Check if update needed
62         if self.samples_since_update >= self.update_frequency:
63             self.update_model()
64
65     def _downsample_old_data(self):

```

```

66     """Downsample older data to maintain buffer size"""
67
68     # Keep all recent data
69     recent_cutoff = len(self.training_buffer) - self.window_size // 2
70     recent_data = self.training_buffer[recent_cutoff:]
71
72     # Sample from older data
73     old_data = self.training_buffer[:recent_cutoff]
74     old_sample_size = self.window_size - len(recent_data)
75
76     if len(old_data) > old_sample_size:
77         import random
78         old_data = random.sample(old_data, old_sample_size)
79
80     self.training_buffer = old_data + recent_data
81
82     def update_model(self):
83         """Update model with buffered samples"""
84
85         if not self.training_buffer:
86             return
87
88         self.logger.info(
89             f"Updating model with {len(self.training_buffer)} samples "
90             f"(update #{self.total_updates + 1})"
91         )
92
93         # Prepare training data
94         X = pd.DataFrame([s['features'] for s in self.training_buffer])
95         y = pd.Series([s['label'] for s in self.training_buffer])
96
97         # Apply temporal weighting if configured
98         if self.update_strategy == 'temporal_weighting':
99             sample_weights = self._calculate_temporal_weights()
100         else:
101             sample_weights = None
102
103         # Update model
104         if hasattr(self.model, 'partial_fit'):
105             # Online learning algorithm
106             if sample_weights is not None:
107                 self.model.partial_fit(X, y, sample_weight=sample_weights)
108             else:
109                 self.model.partial_fit(X, y)
110         else:
111             # Retrain on windowed data
112             if sample_weights is not None:
113                 self.model.fit(X, y, sample_weight=sample_weights)
114             else:
115                 self.model.fit(X, y)
116
117         self.total_updates += 1
118         self.samples_since_update = 0
119
120         self.logger.info(f"Model update completed (total updates: {self.total_updates})")
121
122     def _calculate_temporal_weights(self) -> np.ndarray:

```

```

123     """Calculate sample weights based on recency"""
124
125     if not self.training_buffer:
126         return np.array([])
127
128     # Recent samples get higher weight
129     ages = np.arange(len(self.training_buffer))
130     weights = np.exp(-ages / (self.window_size / 3))
131
132     # Normalize
133     weights = weights / weights.sum() * len(weights)
134
135     return weights
136
137     def predict(self, features: Dict[str, Any]) -> Any:
138         """Make prediction using current model"""
139         X = pd.DataFrame([features])
140         return self.model.predict(X)[0]
141
142 class OnlineModelUpdater:
143     """
144     Complete online model update system
145
146     Handles continuous model updates with performance tracking
147     and automatic rollback on degradation
148     """
149
150     def __init__(self,
151                 model: BaseEstimator,
152                 validation_window: int = 1000,
153                 performance_threshold: float = 0.05):
154         self.learner = IncrementalLearner(model, update_strategy='temporal_weighting')
155         self.validation_window = validation_window
156         self.performance_threshold = performance_threshold
157
158         self.validation_buffer: List[Dict] = []
159         self.performance_history: List[float] = []
160         self.model_checkpoints: List[Any] = []
161
162         self.logger = logging.getLogger(__name__)
163
164     def process_prediction(self,
165                          features: Dict[str, Any],
166                          prediction: Any,
167                          ground_truth: Optional[Any] = None):
168         """Process prediction and optionally update model"""
169
170         if ground_truth is not None:
171             # Add to validation buffer
172             self.validation_buffer.append({
173                 'features': features,
174                 'prediction': prediction,
175                 'ground_truth': ground_truth,
176                 'timestamp': datetime.now()
177             })
178
179             # Maintain validation window
180             if len(self.validation_buffer) > self.validation_window:

```

```

181         self.validation_buffer.pop(0)
182
183         # Add to training buffer
184         self.learner.add_sample(features, ground_truth)
185
186         # Evaluate performance periodically
187         if len(self.validation_buffer) >= self.validation_window:
188             self._evaluate_performance()
189
190     def _evaluate_performance(self):
191         """Evaluate current model performance"""
192
193         if len(self.validation_buffer) < 100:
194             return
195
196         # Calculate accuracy on validation buffer
197         correct = sum(
198             1 for sample in self.validation_buffer
199             if sample['prediction'] == sample['ground_truth']
200         )
201         accuracy = correct / len(self.validation_buffer)
202
203         self.performance_history.append(accuracy)
204
205         # Check for degradation
206         if len(self.performance_history) >= 2:
207             recent_avg = np.mean(self.performance_history[-5:])
208             baseline_avg = np.mean(self.performance_history[:10])
209
210             degradation = baseline_avg - recent_avg
211
212             if degradation > self.performance_threshold:
213                 self.logger.warning(
214                     f"Performance degradation detected: {degradation:.4f} "
215                     f"(recent: {recent_avg:.4f}, baseline: {baseline_avg:.4f})"
216                 )
217
218                 # Consider rollback or full retrain
219                 self._handle_degradation()
220
221     def _handle_degradation(self):
222         """Handle detected performance degradation"""
223
224         self.logger.info("Initiating full retraining due to degradation")
225
226         # In production, this would trigger automated retraining pipeline
227         # For now, just log the event
228         pass
229
230     # Example: Online learning for classification
231     from sklearn.naive_bayes import MultinomialNB
232     from sklearn.linear_model import SGDClassifier
233
234     # SGDClassifier supports partial_fit for online learning
235     online_model = SGDClassifier(
236         loss='log_loss',
237         learning_rate='adaptive',
238         eta0=0.01,

```



```

239     random_state=42
240 )
241
242 updater = OnlineModelUpdater(
243     model=online_model,
244     validation_window=1000,
245     performance_threshold=0.05
246 )
247
248 # Simulate online predictions with delayed ground truth
249 for i in range(5000):
250     features = {
251         'feature_1': np.random.randn(),
252         'feature_2': np.random.randn()
253     }
254
255     # Make prediction
256     if i > 100: # After initial training
257         prediction = updater.learner.predict(features)
258     else:
259         prediction = 0
260
261     # Simulate delayed ground truth (e.g., user feedback)
262     ground_truth = np.random.choice([0, 1])
263
264     # Process prediction with ground truth
265     updater.process_prediction(features, prediction, ground_truth)
266
267     if i % 1000 == 0:
268         print(f"Processed {i} samples, total updates: {updater.learner.total_updates}")
269 )

```

Listing 15.3: Incremental Learning System

15.6 Concept Drift Handling Strategies

15.6.1 Detecting and Adapting to Concept Drift

Concept drift occurs when the relationship between features and target changes over time, requiring model adaptation.

```

1 from typing import Dict, List, Tuple, Optional
2 import numpy as np
3 import pandas as pd
4 from scipy import stats
5 from collections import deque
6 from datetime import datetime, timedelta
7
8 class ConceptDriftDetector:
9     """
10     Detect concept drift in model predictions
11
12     Uses multiple statistical tests to identify when the target
13     distribution or feature-target relationship changes
14     """
15

```

```

16     def __init__(self,
17                   reference_window: int = 1000,
18                   detection_window: int = 500,
19                   warning_threshold: float = 0.05,
20                   drift_threshold: float = 0.01):
21         self.reference_window = reference_window
22         self.detection_window = detection_window
23         self.warning_threshold = warning_threshold
24         self.drift_threshold = drift_threshold
25
26         self.reference_data: deque = deque(maxlen=reference_window)
27         self.current_data: deque = deque(maxlen=detection_window)
28
29         self.drift_detected = False
30         self.warning_detected = False
31
32         self.logger = logging.getLogger(__name__)
33
34     def add_sample(self, features: np.ndarray, prediction: float, label: float):
35         """Add new sample for drift detection"""
36
37         sample = {
38             'features': features,
39             'prediction': prediction,
40             'label': label,
41             'error': abs(prediction - label),
42             'timestamp': datetime.now()
43         }
44
45         # Add to current window
46         self.current_data.append(sample)
47
48         # Move to reference once current window is full
49         if len(self.current_data) >= self.detection_window:
50             if len(self.reference_data) < self.reference_window:
51                 # Still building reference
52                 self.reference_data.extend(list(self.current_data))
53                 self.current_data.clear()
54             else:
55                 # Check for drift
56                 self._check_drift()
57
58     def _check_drift(self) -> Tuple[bool, bool]:
59         """Check for concept drift using multiple tests"""
60
61         if len(self.reference_data) < self.reference_window:
62             return False, False
63
64         if len(self.current_data) < self.detection_window:
65             return False, False
66
67         # Extract errors from both windows
68         ref_errors = np.array([s['error'] for s in self.reference_data])
69         curr_errors = np.array([s['error'] for s in self.current_data])
70
71         # 1. Kolmogorov-Smirnov test for error distribution
72         ks_stat, ks_pvalue = stats.ks_2samp(ref_errors, curr_errors)
73

```

```

74     # 2. Mean error comparison using t-test
75     t_stat, t_pvalue = stats.ttest_ind(ref_errors, curr_errors)
76
77     # 3. Page-Hinkley test for cumulative drift
78     ph_drift = self._page_hinkley_test(curr_errors)
79
80     # Evaluate results
81     drift_signals = 0
82
83     if ks_pvalue < self.drift_threshold:
84         drift_signals += 1
85         self.logger.warning(
86             f"KS test detected distribution change: "
87             f"statistic={ks_stat:.4f}, p-value={ks_pvalue:.6f}"
88         )
89
90     if t_pvalue < self.drift_threshold:
91         drift_signals += 1
92         self.logger.warning(
93             f"T-test detected mean change: "
94             f"ref_mean={ref_errors.mean():.4f}, "
95             f"curr_mean={curr_errors.mean():.4f}, "
96             f"p-value={t_pvalue:.6f}"
97         )
98
99     if ph_drift:
100         drift_signals += 1
101         self.logger.warning("Page-Hinkley test detected drift")
102
103     # Determine drift status
104     if drift_signals >= 2:
105         self.drift_detected = True
106         self.warning_detected = False
107         self.logger.error(f"CONCEPT DRIFT DETECTED ({drift_signals} signals)")
108
109         # Reset reference window to current data
110         self._reset_reference_window()
111
112         return True, False
113
114     elif drift_signals == 1:
115         self.warning_detected = True
116         self.drift_detected = False
117         self.logger.warning("Drift warning - monitoring closely")
118         return False, True
119
120     else:
121         self.drift_detected = False
122         self.warning_detected = False
123         return False, False
124
125     def _page_hinkley_test(self, errors: np.ndarray,
126                           delta: float = 0.005,
127                           lambda_: float = 50) -> bool:
128
129         Page-Hinkley test for detecting mean shifts
130
131         Cumulative test that's sensitive to gradual drift

```

```

132     """
133
134     mean_error = errors.mean()
135     cumsum = 0
136     min_cumsum = 0
137
138     for error in errors:
139         cumsum += error - mean_error - delta
140
141         if cumsum < min_cumsum:
142             min_cumsum = cumsum
143
144         ph_value = cumsum - min_cumsum
145
146         if ph_value > lambda_:
147             return True
148
149     return False
150
151 def _reset_reference_window(self):
152     """Reset reference window after drift detected"""
153
154     # Use current window as new reference
155     self.reference_data.clear()
156     self.reference_data.extend(list(self.current_data))
157     self.current_data.clear()
158
159     self.logger.info("Reference window reset with recent data")
160
161 class AdaptiveDriftHandler:
162     """
163     Handle concept drift with adaptive strategies
164
165     Implements multiple adaptation strategies:
166     - Ensemble of time-weighted models
167     - Dynamic model selection
168     - Gradual forgetting
169     """
170
171     def __init__(self, base_model_class, n_ensemble: int = 5):
172         self.base_model_class = base_model_class
173         self.n_ensemble = n_ensemble
174
175         # Ensemble of models with different recency
176         self.ensemble: List[Dict] = []
177         self.drift_detector = ConceptDriftDetector()
178
179         # Initialize ensemble
180         for i in range(n_ensemble):
181             self.ensemble.append({
182                 'model': base_model_class(),
183                 'age': 0,
184                 'weight': 1.0 / n_ensemble,
185                 'performance': []
186             })
187
188         self.logger = logging.getLogger(__name__)
189

```

```

190 def train_ensemble(self, X: pd.DataFrame, y: pd.Series):
191     """Train ensemble with temporal weighting"""
192
193     n_samples = len(X)
194
195     for i, member in enumerate(self.ensemble):
196         # Each model trained on different time window
197         window_size = n_samples // (i + 1)
198         start_idx = max(0, n_samples - window_size)
199
200         X_subset = X.iloc[start_idx:]
201         y_subset = y.iloc[start_idx:]
202
203         # Apply temporal weighting
204         sample_weights = np.linspace(0.5, 1.0, len(X_subset))
205
206         member['model'].fit(X_subset, y_subset, sample_weight=sample_weights)
207         member['age'] = 0
208
209         self.logger.info(
210             f"Trained ensemble member {i} on {len(X_subset)} samples"
211         )
212
213 def predict(self, features: pd.DataFrame) -> np.ndarray:
214     """Predict using weighted ensemble"""
215
216     predictions = []
217
218     for member in self.ensemble:
219         pred = member['model'].predict(features)
220         predictions.append(pred * member['weight'])
221
222     # Weighted average
223     final_pred = np.sum(predictions, axis=0)
224
225     return final_pred
226
227 def update_on_drift(self, X_new: pd.DataFrame, y_new: pd.Series):
228     """Update ensemble when drift detected"""
229
230     self.logger.info("Updating ensemble due to drift detection")
231
232     # Retire oldest model
233     self.ensemble.pop()
234
235     # Add new model trained on recent data
236     new_model = {
237         'model': self.base_model_class(),
238         'age': 0,
239         'weight': 0.3, # Higher weight for newest model
240         'performance': []
241     }
242     new_model['model'].fit(X_new, y_new)
243
244     self.ensemble.insert(0, new_model)
245
246     # Reweight ensemble
247     total_weight = sum(m['weight'] for m in self.ensemble)

```

```

248     for member in self.ensemble:
249         member['weight'] /= total_weight
250
251     self.logger.info("Ensemble updated with new model")
252
253     def adapt_weights(self, validation_errors: List[np.ndarray]):
254         """Adapt ensemble weights based on recent performance"""
255
256         if len(validation_errors) < len(self.ensemble):
257             return
258
259         # Calculate performance for each ensemble member
260         for i, member in enumerate(self.ensemble):
261             errors = validation_errors[i]
262             performance = 1.0 / (1.0 + errors.mean())
263             member['performance'].append(performance)
264
265         # Update weights based on recent performance
266         recent_performance = [
267             np.mean(m['performance'][-10:]) if m['performance'] else 0.5
268             for m in self.ensemble
269         ]
270
271         # Softmax weighting
272         exp_perf = np.exp(np.array(recent_performance))
273         new_weights = exp_perf / exp_perf.sum()
274
275         for i, member in enumerate(self.ensemble):
276             member['weight'] = new_weights[i]
277
278         self.logger.info(f"Updated ensemble weights: {new_weights}")
279
280     # Example usage
281     from sklearn.ensemble import RandomForestClassifier
282
283     # Create adaptive drift handler
284     handler = AdaptiveDriftHandler(
285         base_model_class=lambda: RandomForestClassifier(n_estimators=50),
286         n_ensemble=3
287     )
288
289     # Simulate streaming data with concept drift
290     np.random.seed(42)
291
292     # Generate initial data
293     X_train = pd.DataFrame(np.random.randn(1000, 5))
294     y_train = (X_train.sum(axis=1) > 0).astype(int)
295
296     # Train ensemble
297     handler.train_ensemble(X_train, y_train)
298
299     # Simulate concept drift after 500 samples
300     for batch in range(10):
301         # Generate batch with drift
302         if batch < 5:
303             # Original concept
304             X_batch = pd.DataFrame(np.random.randn(100, 5))
305             y_batch = (X_batch.sum(axis=1) > 0).astype(int)

```

```

306     else:
307         # Drifted concept (inverted relationship)
308         X_batch = pd.DataFrame(np.random.randn(100, 5))
309         y_batch = (X_batch.sum(axis=1) < 0).astype(int)
310
311         # Make predictions
312         predictions = handler.predict(X_batch)
313
314         # Add samples to drift detector
315         for i in range(len(X_batch)):
316             handler.drift_detector.add_sample(
317                 X_batch.iloc[i].values,
318                 predictions[i],
319                 y_batch.iloc[i]
320             )
321
322         # Check for drift
323         if handler.drift_detector.drift_detected:
324             print(f"Drift detected at batch {batch}")
325             handler.update_on_drift(X_batch, y_batch)

```

Listing 15.4: Concept Drift Detection and Handling

15.6.2 Multi-Strategy Drift Adaptation

Different types of drift require different adaptation strategies. A robust system should employ multiple approaches.

```

1  from typing import Dict, List, Optional, Any
2  from enum import Enum
3  import numpy as np
4  import pandas as pd
5  from datetime import datetime
6  import logging
7
8  class DriftType(Enum):
9      """Types of drift that can occur"""
10     SUDDEN = "sudden" # Abrupt change
11     GRADUAL = "gradual" # Slow evolution
12     INCREMENTAL = "incremental" # Step-wise changes
13     RECURRING = "recurring" # Cyclical patterns
14     NONE = "none"
15
16  class DriftAdaptationStrategy(Enum):
17      """Adaptation strategies for different drift types"""
18     RETRAIN_FULL = "retrain_full"
19     RETRAIN_INCREMENTAL = "retrain_incremental"
20     ENSEMBLE_UPDATE = "ensemble_update"
21     WINDOW_SLIDE = "window_slide"
22     NONE = "none"
23
24  class DriftCharacterizer:
25      """
26      Characterize the type and severity of drift
27
28      Analyzes drift patterns to select appropriate adaptation strategy
29      """

```

```

30
31 def __init__(self, history_window: int = 100):
32     self.history_window = history_window
33     self.drift_history: List[Dict] = []
34     self.logger = logging.getLogger(__name__)
35
36 def characterize_drift(self,
37                       error_history: List[float],
38                       timestamps: List[datetime]) -> Dict[str, Any]:
39     """Characterize drift type and severity"""
40
41     if len(error_history) < 10:
42         return {
43             'drift_type': DriftType.NONE,
44             'severity': 0.0,
45             'recommended_strategy': DriftAdaptationStrategy.NONE
46         }
47
48     errors = np.array(error_history)
49
50     # Calculate error rate of change
51     error_diff = np.diff(errors)
52
53     # Detect sudden drift (abrupt change)
54     sudden_threshold = 3 * np.std(errors)
55     sudden_changes = np.abs(error_diff) > sudden_threshold
56
57     if np.any(sudden_changes):
58         drift_type = DriftType.SUDDEN
59         severity = np.max(np.abs(error_diff)) / np.mean(errors)
60         strategy = DriftAdaptationStrategy.RETRAIN_FULL
61
62     # Detect gradual drift (trend)
63     elif self._detect_trend(errors):
64         drift_type = DriftType.GRADUAL
65         # Calculate trend severity
66         from scipy.stats import linregress
67         slope, _, _, _, _ = linregress(range(len(errors)), errors)
68         severity = abs(slope) / np.mean(errors)
69         strategy = DriftAdaptationStrategy.RETRAIN_INCREMENTAL
70
71     # Detect recurring drift (seasonality)
72     elif self._detect_seasonality(errors, timestamps):
73         drift_type = DriftType.RECURRING
74         severity = 0.5
75         strategy = DriftAdaptationStrategy.ENSEMBLE_UPDATE
76
77     else:
78         drift_type = DriftType.NONE
79         severity = 0.0
80         strategy = DriftAdaptationStrategy.NONE
81
82     result = {
83         'drift_type': drift_type,
84         'severity': severity,
85         'recommended_strategy': strategy,
86         'timestamp': datetime.now()
87     }

```



```

88         self.drift_history.append(result)
89
90     self.logger.info(
91         f"Drift characterization: {drift_type.value} "
92         f"(severity: {severity:.3f}, strategy: {strategy.value})"
93     )
94
95     return result
96
97
98     def _detect_trend(self, errors: np.ndarray,
99                     significance: float = 0.05) -> bool:
100         """Detect monotonic trend using Mann-Kendall test"""
101
102         from scipy.stats import kendalltau
103
104         n = len(errors)
105         tau, p_value = kendalltau(range(n), errors)
106
107         return p_value < significance and abs(tau) > 0.3
108
109     def _detect_seasonality(self,
110                           errors: np.ndarray,
111                           timestamps: List[datetime]) -> bool:
112         """Detect seasonal/recurring patterns"""
113
114         if len(errors) < 50:
115             return False
116
117         # Simple autocorrelation check
118         from scipy.signal import find_peaks
119
120         # Compute autocorrelation
121         autocorr = np.correlate(errors - errors.mean(),
122                                errors - errors.mean(),
123                                mode='full')
124         autocorr = autocorr[len(autocorr)//2:]
125         autocorr = autocorr / autocorr[0]
126
127         # Find peaks in autocorrelation
128         peaks, _ = find_peaks(autocorr[1:], height=0.3)
129
130         # If significant peaks found, likely seasonal
131         return len(peaks) > 0
132
133 class AdaptiveRetrainingOrchestrator:
134     """
135     Orchestrate adaptive retraining based on drift characteristics
136
137     Selects and executes appropriate adaptation strategy
138     """
139
140     def __init__(self, base_model, config: RetrainingConfig):
141         self.base_model = base_model
142         self.config = config
143
144         self.drift_characterizer = DriftCharacterizer()
145         self.drift_detector = ConceptDriftDetector()

```

```

146     self.adaptive_handler = AdaptiveDriftHandler(
147         base_model_class=lambda: base_model.__class__()
148     )
149
150     self.error_history: List[float] = []
151     self.timestamp_history: List[datetime] = []
152
153     self.logger = logging.getLogger(__name__)
154
155     def process_batch(self,
156                     X: pd.DataFrame,
157                     y_pred: np.ndarray,
158                     y_true: np.ndarray):
159         """Process batch of predictions and adapt if needed"""
160
161         # Calculate batch error
162         batch_error = np.mean(np.abs(y_pred - y_true))
163
164         self.error_history.append(batch_error)
165         self.timestamp_history.append(datetime.now())
166
167         # Keep history bounded
168         max_history = 1000
169         if len(self.error_history) > max_history:
170             self.error_history = self.error_history[-max_history:]
171             self.timestamp_history = self.timestamp_history[-max_history:]
172
173         # Check for drift
174         for i in range(len(X)):
175             self.drift_detector.add_sample(
176                 X.iloc[i].values,
177                 y_pred[i],
178                 y_true[i]
179             )
180
181         # If drift detected, characterize and adapt
182         if self.drift_detector.drift_detected:
183             self._handle_detected_drift(X, y_true)
184
185     def _handle_detected_drift(self, X: pd.DataFrame, y: np.ndarray):
186         """Handle detected drift with appropriate strategy"""
187
188         # Characterize drift
189         characterization = self.drift_characterizer.characterize_drift(
190             self.error_history,
191             self.timestamp_history
192         )
193
194         drift_type = characterization['drift_type']
195         severity = characterization['severity']
196         strategy = characterization['recommended_strategy']
197
198         self.logger.warning(
199             f"Executing adaptation strategy: {strategy.value} "
200             f"for {drift_type.value} drift (severity: {severity:.3f})"
201         )
202
203         # Execute strategy

```

```

204     if strategy == DriftAdaptationStrategy.RETRAIN_FULL:
205         self._execute_full_retrain(X, y, severity)
206
207     elif strategy == DriftAdaptationStrategy.RETRAIN_INCREMENTAL:
208         self._execute_incremental_retrain(X, y)
209
210     elif strategy == DriftAdaptationStrategy.ENSEMBLE_UPDATE:
211         self._execute_ensemble_update(X, y)
212
213     elif strategy == DriftAdaptationStrategy.WINDOW_SLIDE:
214         self._execute_window_slide(X, y)
215
216     def _execute_full_retrain(self, X: pd.DataFrame, y: np.ndarray,
217                             severity: float):
218         """Execute full model retraining"""
219
220         self.logger.info("Executing full model retrain")
221
222         # Collect larger dataset for full retrain
223         # In production, fetch from data warehouse
224         train_data = X # Placeholder
225
226         # Retrain from scratch
227         self.base_model.fit(train_data, y)
228
229         # Reset drift detector
230         self.drift_detector = ConceptDriftDetector()
231
232         self.logger.info("Full retrain completed")
233
234     def _execute_incremental_retrain(self, X: pd.DataFrame, y: np.ndarray):
235         """Execute incremental model update"""
236
237         self.logger.info("Executing incremental retrain")
238
239         if hasattr(self.base_model, 'partial_fit'):
240             self.base_model.partial_fit(X, y)
241         else:
242             # For models without partial_fit, use warm start
243             self.base_model.fit(X, y)
244
245         self.logger.info("Incremental retrain completed")
246
247     def _execute_ensemble_update(self, X: pd.DataFrame, y: np.ndarray):
248         """Update ensemble for recurring drift"""
249
250         self.logger.info("Executing ensemble update")
251
252         self.adaptive_handler.update_on_drift(X, pd.Series(y))
253
254         self.logger.info("Ensemble update completed")
255
256     def _execute_window_slide(self, X: pd.DataFrame, y: np.ndarray):
257         """Slide training window for continuous adaptation"""
258
259         self.logger.info("Executing window slide")
260
261         # Use only recent data

```

```

262     window_size = self.config.min_data_points
263
264     if len(X) > window_size:
265         X_recent = X.iloc[-window_size:]
266         y_recent = y[-window_size:]
267     else:
268         X_recent = X
269         y_recent = y
270
271     self.base_model.fit(X_recent, y_recent)
272
273     self.logger.info("Window slide completed")
274
275     # Example: Comprehensive drift handling
276     from sklearn.ensemble import GradientBoostingRegressor
277
278     # Create orchestrator
279     model = GradientBoostingRegressor(n_estimators=100)
280     config = RetrainingConfig(
281         model_name="adaptive_model",
282         model_type="regression"
283     )
284
285     orchestrator = AdaptiveRetrainingOrchestrator(model, config)
286
287     # Simulate streaming data with different drift types
288     np.random.seed(42)
289
290     for batch_idx in range(20):
291         # Generate data with evolving distribution
292         if batch_idx < 5:
293             # Normal data
294             X = pd.DataFrame(np.random.randn(100, 5))
295             y_true = X.sum(axis=1).values + np.random.randn(100) * 0.1
296
297         elif batch_idx < 10:
298             # Gradual drift
299             shift = (batch_idx - 5) * 0.2
300             X = pd.DataFrame(np.random.randn(100, 5) + shift)
301             y_true = X.sum(axis=1).values + np.random.randn(100) * 0.1
302
303         elif batch_idx < 15:
304             # Sudden drift
305             X = pd.DataFrame(np.random.randn(100, 5) * 2)
306             y_true = X.sum(axis=1).values * -1 + np.random.randn(100) * 0.1
307
308         else:
309             # Back to normal (recurring)
310             X = pd.DataFrame(np.random.randn(100, 5))
311             y_true = X.sum(axis=1).values + np.random.randn(100) * 0.1
312
313         # Make predictions
314         if batch_idx > 0:
315             y_pred = orchestrator.base_model.predict(X)
316         else:
317             # Initial training
318             orchestrator.base_model.fit(X, y_true)
319             y_pred = orchestrator.base_model.predict(X)

```

```

320
321     # Process batch
322     orchestrator.process_batch(X, y_pred, y_true)
323
324     print(f"Batch {batch_idx}: Error = {np.mean(np.abs(y_pred - y_true)):.4f}")

```

Listing 15.5: Multi-Strategy Drift Adaptation Framework

15.7 Model Freshness vs Stability Trade-offs

15.7.1 Balancing Update Frequency and Model Stability

Frequent retraining keeps models fresh but can introduce instability, unexpected behavior changes, and increased costs. Finding the right balance requires understanding your application's requirements.

```

1  from typing import Dict, List, Optional, Tuple
2  from dataclasses import dataclass
3  from datetime import datetime, timedelta
4  import numpy as np
5  import logging
6
7  @dataclass
8  class StabilityMetrics:
9      """Track model stability over time"""
10     prediction_variance: float # Variance in predictions between versions
11     behavioral_consistency: float # How consistent model behavior is
12     rollback_rate: float # Rate of rollbacks due to issues
13     deployment_success_rate: float # Successful deployments
14     avg_time_to_rollback: float # Time to detect and rollback issues
15
16  @dataclass
17  class FreshnessMetrics:
18      """Track model freshness indicators"""
19     datastaleness_days: float # Days since training data cutoff
20     drift_score: float # Current drift magnitude
21     performance_gap: float # Gap from baseline performance
22     new_data_volume: int # New samples since last retrain
23     feature_coverage: float # Coverage of feature space
24
25  class FreshnessStabilityManager:
26      """
27      Manage trade-off between model freshness and stability
28
29      Adaptively adjusts retraining frequency based on
30      application requirements and observed stability
31      """
32
33     def __init__(self,
34                 min_retrain_interval_days: int = 7,
35                 max_retrain_interval_days: int = 30,
36                 stability_weight: float = 0.5):
37         """
38         Parameters:
39         - min_retrain_interval_days: Minimum days between retrains
40         - max_retrain_interval_days: Maximum days before forced retrain

```

```

41     - stability_weight: Weight for stability vs freshness (0-1)
42     0 = prioritize freshness, 1 = prioritize stability
43     """
44     self.min_interval = timedelta(days=min_retrain_interval_days)
45     self.max_interval = timedelta(days=max_retrain_interval_days)
46     self.stability_weight = stability_weight
47
48     self.last_retrain_time: Optional[datetime] = None
49     self.stability_history: List[StabilityMetrics] = []
50     self.freshness_history: List[FreshnessMetrics] = []
51
52     # Adaptive interval
53     self.current_interval = timedelta(
54         days=(min_retrain_interval_days + max_retrain_interval_days) / 2
55     )
56
57     self.logger = logging.getLogger(__name__)
58
59     def should_retrain(self,
60                       current_freshness: FreshnessMetrics,
61                       current_stability: StabilityMetrics) -> Tuple[bool, str]:
62         """Determine if retraining should occur"""
63
64         if self.last_retrain_time is None:
65             return True, "Initial training required"
66
67         time_since_retrain = datetime.now() - self.last_retrain_time
68
69         # Always retrain if max interval exceeded
70         if time_since_retrain >= self.max_interval:
71             return True, f"Maximum interval exceeded ({self.max_interval.days} days)"
72
73         # Never retrain if within minimum interval
74         if time_since_retrain < self.min_interval:
75             return False, f"Within minimum interval ({self.min_interval.days} days)"
76
77         # Calculate freshness and stability scores
78         freshness_score = self._calculate_freshness_score(current_freshness)
79         stability_score = self._calculate_stability_score(current_stability)
80
81         # Weighted decision
82         retrain_score = (
83             (1 - self.stability_weight) * (1 - freshness_score) + # Need for
freshness
84             self.stability_weight * stability_score # Confidence in stability
85         )
86
87         # Adapt interval based on stability
88         self._adapt_retraining_interval(current_stability)
89
90         # Retrain if score indicates need
91         threshold = 0.5
92         should_retrain = retrain_score > threshold
93
94         reason = (
95             f"Retrain score: {retrain_score:.3f} "
96             f"(freshness: {freshness_score:.3f}, stability: {stability_score:.3f})"
97         )

```

```

98
99     if should_retrain:
100         self.logger.info(f"Retraining recommended: {reason}")
101     else:
102         self.logger.info(f"Skipping retrain: {reason}")
103
104     return should_retrain, reason
105
106 def _calculate_freshness_score(self, metrics: FreshnessMetrics) -> float:
107     """Calculate freshness score (0=stale, 1=fresh)"""
108
109     # Multiple freshness indicators
110     scores = []
111
112     # 1. Data staleness (0-1, lower is fresher)
113     staleness_score = 1.0 - min(1.0, metrics.data_staleness_days / 30.0)
114     scores.append(staleness_score)
115
116     # 2. Drift score (0-1, lower is better)
117     drift_score = 1.0 - min(1.0, metrics.drift_score / 0.5)
118     scores.append(drift_score)
119
120     # 3. Performance gap (0-1, smaller gap is better)
121     perf_score = 1.0 - min(1.0, metrics.performance_gap / 0.1)
122     scores.append(perf_score)
123
124     # 4. New data availability (0-1, more data is better)
125     data_score = min(1.0, metrics.new_data_volume / 10000.0)
126     scores.append(data_score * 0.5) # Lower weight
127
128     # Average freshness
129     return np.mean(scores)
130
131 def _calculate_stability_score(self, metrics: StabilityMetrics) -> float:
132     """Calculate stability score (0=unstable, 1=stable)"""
133
134     scores = []
135
136     # 1. Low prediction variance is good
137     variance_score = 1.0 - min(1.0, metrics.prediction_variance / 0.1)
138     scores.append(variance_score)
139
140     # 2. High behavioral consistency is good
141     scores.append(metrics.behavioral_consistency)
142
143     # 3. Low rollback rate is good
144     rollback_score = 1.0 - min(1.0, metrics.rollback_rate)
145     scores.append(rollback_score)
146
147     # 4. High deployment success is good
148     scores.append(metrics.deployment_success_rate)
149
150     return np.mean(scores)
151
152 def _adapt_retraining_interval(self, stability: StabilityMetrics):
153     """Adapt retraining interval based on stability"""
154
155     # If system is stable, can retrain more frequently

```



```

214         self.reference_features = features.iloc[sample_indices].copy()
215     else:
216         self.reference_features = features.copy()
217
218     # Get reference predictions
219     self.reference_predictions = model.predict(self.reference_features)
220
221     self.logger.info(
222         f"Set reference predictions for {len(self.reference_features)} samples"
223     )
224
225     def measure_stability(self, new_model) -> Dict[str, float]:
226         """Measure prediction stability of new model vs reference"""
227
228         if self.reference_predictions is None:
229             return {
230                 'prediction_variance': 0.0,
231                 'behavioral_consistency': 1.0,
232                 'avg_prediction_change': 0.0
233             }
234
235         # Get new predictions
236         new_predictions = new_model.predict(self.reference_features)
237
238         # Calculate stability metrics
239         prediction_changes = np.abs(new_predictions - self.reference_predictions)
240
241         # For classification, measure agreement
242         if len(np.unique(self.reference_predictions)) < 10:
243             agreement = np.mean(new_predictions == self.reference_predictions)
244             behavioral_consistency = agreement
245             avg_change = 1.0 - agreement
246         else:
247             # For regression, measure relative change
248             relative_changes = prediction_changes / (
249                 np.abs(self.reference_predictions) + 1e-6
250             )
251             avg_change = np.mean(relative_changes)
252             behavioral_consistency = 1.0 - min(1.0, avg_change)
253
254         prediction_variance = np.var(prediction_changes)
255
256         metrics = {
257             'prediction_variance': float(prediction_variance),
258             'behavioral_consistency': float(behavioral_consistency),
259             'avg_prediction_change': float(np.mean(prediction_changes)),
260             'max_prediction_change': float(np.max(prediction_changes)),
261             'pct_changed_predictions': float(
262                 np.mean(prediction_changes > 0.01) * 100
263             )
264         }
265
266         self.logger.info(
267             f"Stability metrics: consistency={metrics['behavioral_consistency']:.3f},
268             "
269             f"variance={metrics['prediction_variance']:.6f}"
270         )

```

```

271         return metrics
272
273     # Example: Managing freshness-stability trade-off
274     from sklearn.ensemble import RandomForestClassifier
275     import pandas as pd
276
277     # Create manager with balanced approach
278     manager = FreshnessStabilityManager(
279         min_retrain_interval_days=7,
280         max_retrain_interval_days=30,
281         stability_weight=0.5 # Equal weight to freshness and stability
282     )
283
284     # Create stability tracker
285     stability_tracker = PredictionStabilityTracker()
286
287     # Initialize with reference model
288     reference_model = RandomForestClassifier(n_estimators=100, random_state=42)
289     X_ref = pd.DataFrame(np.random.randn(1000, 10))
290     y_ref = (X_ref.sum(axis=1) > 0).astype(int)
291     reference_model.fit(X_ref, y_ref)
292
293     stability_tracker.set_reference(reference_model, X_ref)
294
295     # Simulate model updates over time
296     manager.last_retrain_time = datetime.now() - timedelta(days=15)
297
298     for day in range(30):
299         current_time = datetime.now() - timedelta(days=30-day)
300
301         # Simulate freshness metrics
302         freshness = FreshnessMetrics(
303             datastaleness_days=(current_time - manager.last_retrain_time).days,
304             drift_score=0.05 + (day * 0.01), # Increasing drift
305             performance_gap=0.02 + (day * 0.002),
306             new_data_volume=day * 500,
307             feature_coverage=0.95
308         )
309
310         # Train new model to check stability
311         X_new = pd.DataFrame(np.random.randn(1000, 10) + day * 0.01)
312         y_new = (X_new.sum(axis=1) > 0).astype(int)
313         new_model = RandomForestClassifier(n_estimators=100, random_state=42+day)
314         new_model.fit(X_new, y_new)
315
316         # Measure stability
317         stability_metrics_dict = stability_tracker.measure_stability(new_model)
318
319         stability = StabilityMetrics(
320             prediction_variance=stability_metrics_dict['prediction_variance'],
321             behavioral_consistency=stability_metrics_dict['behavioral_consistency'],
322             rollback_rate=0.05,
323             deployment_success_rate=0.95,
324             avg_time_to_rollback=2.0
325         )
326
327         # Check if should retrain
328         should_retrain, reason = manager.should_retrain(freshness, stability)

```

```

329
330     if should_retrain:
331         print(f"Day {day}: RETRAINING - {reason}")
332         manager.record_retrain(freshness, stability)
333         # Update reference
334         stability_tracker.set_reference(new_model, X_new)
335     else:
336         print(f"Day {day}: Skipping - {reason}")

```

Listing 15.6: Freshness-Stability Trade-off Manager

15.7.2 Cost-Aware Retraining Scheduling

Retraining has real costs in compute resources, engineering time, and potential business disruption. A production system must optimize these costs.

```

1  from typing import Dict, List, Optional
2  from dataclasses import dataclass
3  from datetime import datetime, timedelta, time
4  import numpy as np
5
6  @dataclass
7  class RetrainingCost:
8      """Model retraining cost components"""
9      compute_cost_usd: float # Direct compute costs
10     engineer_hours: float # Human review time
11     opportunity_cost_usd: float # Lost revenue if model offline
12     data_pipeline_cost_usd: float # Cost to fetch/process data
13     validation_cost_usd: float # A/B test or canary costs
14
15     @property
16     def total_cost(self) -> float:
17         return (self.compute_cost_usd +
18                 self.opportunity_cost_usd +
19                 self.data_pipeline_cost_usd +
20                 self.validation_cost_usd)
21
22  @dataclass
23  class RetrainingBenefit:
24      """Expected benefits from retraining"""
25      performance_improvement: float # Expected metric improvement
26      revenue_impact_usd: float # Expected revenue impact
27      risk_reduction: float # Reduced risk from drift
28      confidence: float # Confidence in benefit estimates
29
30  class CostAwareScheduler:
31      """
32      Schedule retraining considering costs and benefits
33
34      Optimizes timing and frequency to maximize ROI
35      """
36
37     def __init__(self,
38                 cost_model: Dict[str, float],
39                 business_hours: Tuple[time, time] = (time(9), time(17)),
40                 preferred_days: List[int] = [1, 2, 3]): # Tue, Wed, Thu
41

```

```

42     Parameters:
43     - cost_model: Cost parameters for retraining operations
44     - business_hours: Hours to avoid retraining (higher opportunity cost)
45     - preferred_days: Preferred days of week (0=Monday, 6=Sunday)
46     """
47     self.cost_model = cost_model
48     self.business_hours = business_hours
49     self.preferred_days = preferred_days
50
51     self.scheduled_retrains: List[datetime] = []
52     self.logger = logging.getLogger(__name__)
53
54     def estimate_retrain_cost(self,
55                             training_samples: int,
56                             estimated_duration_hours: float,
57                             scheduled_time: datetime) -> RetrainingCost:
58         """Estimate cost of retraining operation"""
59
60         # Compute cost scales with data volume and duration
61         base_compute_cost = self.cost_model.get('base_compute_cost', 10.0)
62         cost_per_hour = self.cost_model.get('compute_cost_per_hour', 5.0)
63         compute_cost = base_compute_cost + (cost_per_hour * estimated_duration_hours)
64
65         # Data pipeline cost
66         cost_per_sample = self.cost_model.get('cost_per_sample', 0.0001)
67         data_cost = training_samples * cost_per_sample
68
69         # Engineer oversight
70         engineer_hourly_rate = self.cost_model.get('engineer_hourly_rate', 150.0)
71         engineer_hours = 2.0 # Review and validation
72         engineer_cost = engineer_hours * engineer_hourly_rate
73
74         # Opportunity cost depends on timing
75         opportunity_cost = self._calculate_opportunity_cost(
76             scheduled_time,
77             estimated_duration_hours
78         )
79
80         # Validation cost (canary deployment)
81         validation_cost = self.cost_model.get('canary_cost', 20.0)
82
83         return RetrainingCost(
84             compute_cost_usd=compute_cost + engineer_cost,
85             engineer_hours=engineer_hours,
86             opportunity_cost_usd=opportunity_cost,
87             data_pipeline_cost_usd=data_cost,
88             validation_cost_usd=validation_cost
89         )
90
91     def _calculate_opportunity_cost(self,
92                                   scheduled_time: datetime,
93                                   duration_hours: float) -> float:
94         """Calculate opportunity cost based on timing"""
95
96         # Higher cost during business hours
97         revenue_per_hour = self.cost_model.get('revenue_per_hour', 1000.0)
98         downtime_fraction = self.cost_model.get('downtime_fraction', 0.1)
99

```

```

100     # Check if deployment overlaps with business hours
101     start_time = scheduled_time.time()
102     business_start, business_end = self.business_hours
103
104     if business_start <= start_time <= business_end:
105         # Business hours - higher opportunity cost
106         multiplier = 1.0
107     else:
108         # Off-hours - lower opportunity cost
109         multiplier = 0.2
110
111     opportunity_cost = (
112         revenue_per_hour *
113         duration_hours *
114         downtime_fraction *
115         multiplier
116     )
117
118     return opportunity_cost
119
120     def estimate_retrain_benefit(self,
121                                current_performance: float,
122                                expected_improvement: float,
123                                confidence: float = 0.7) -> RetrainingBenefit:
124         """Estimate expected benefit from retraining"""
125
126         # Convert performance improvement to revenue impact
127         # This would be calibrated based on your business metrics
128         revenue_per_point = self.cost_model.get('revenue_per_performance_point',
129         5000.0)
130
131         revenue_impact = expected_improvement * revenue_per_point * confidence
132
133         # Risk reduction value (avoiding drift-related failures)
134         risk_reduction_value = self.cost_model.get('risk_reduction_value', 1000.0)
135
136         return RetrainingBenefit(
137             performance_improvement=expected_improvement,
138             revenue_impact_usd=revenue_impact + risk_reduction_value,
139             risk_reduction=0.2, # 20% risk reduction
140             confidence=confidence
141         )
142
143     def calculate_roi(self,
144                     cost: RetrainingCost,
145                     benefit: RetrainingBenefit,
146                     time_horizon_days: int = 30) -> Dict[str, float]:
147         """Calculate ROI for retraining decision"""
148
149         # Project benefits over time horizon
150         total_benefit = benefit.revenue_impact_usd
151
152         # Subtract costs
153         total_cost = cost.total_cost
154
155         # Calculate ROI
156         roi = ((total_benefit - total_cost) / total_cost) * 100 if total_cost > 0 else

```

```

156
157     # Net present value (simplified)
158     npv = total_benefit - total_cost
159
160     return {
161         'roi_percent': roi,
162         'npv_usd': npv,
163         'total_cost': total_cost,
164         'total_benefit': total_benefit,
165         'payback_days': (total_cost / total_benefit * time_horizon_days)
166                         if total_benefit > 0 else float('inf')
167     }
168
169     def find_optimal_time(self,
170                           earliest: datetime,
171                           latest: datetime) -> datetime:
172         """Find optimal time slot for retraining"""
173
174         # Generate candidate times
175         candidates = []
176         current = earliest
177
178         while current <= latest:
179             # Prefer off-hours
180             if current.time() < self.business_hours[0] or \
181                current.time() > self.business_hours[1]:
182                 # Prefer specific days
183                 if current.weekday() in self.preferred_days:
184                     score = 1.0
185                 else:
186                     score = 0.5
187             else:
188                 score = 0.1 # Discourage business hours
189
190             candidates.append((current, score))
191             current += timedelta(hours=1)
192
193         # Sort by score and return best
194         candidates.sort(key=lambda x: x[1], reverse=True)
195
196         optimal_time = candidates[0][0] if candidates else earliest
197
198         self.logger.info(
199             f"Optimal retraining time: {optimal_time} "
200             f"(score: {candidates[0][1]:.2f})"
201         )
202
203         return optimal_time
204
205     def should_retrain_now(self,
206                           current_performance: float,
207                           expected_improvement: float,
208                           training_samples: int,
209                           duration_hours: float = 2.0) -> Tuple[bool, Dict]:
210         """Decide if retraining should proceed based on ROI"""
211
212         now = datetime.now()
213

```

```

214     # Find optimal time
215     optimal_time = self.find_optimal_time(
216         earliest=now,
217         latest=now + timedelta(days=7)
218     )
219
220     # Estimate costs and benefits
221     cost = self.estimate_retrain_cost(
222         training_samples,
223         duration_hours,
224         optimal_time
225     )
226
227     benefit = self.estimate_retrain_benefit(
228         current_performance,
229         expected_improvement
230     )
231
232     # Calculate ROI
233     roi_metrics = self.calculate_roi(cost, benefit)
234
235     # Decision: retrain if positive NPV and ROI > 100%
236     should_retrain = (
237         roi_metrics['npv_usd'] > 0 and
238         roi_metrics['roi_percent'] > 100
239     )
240
241     decision = {
242         'should_retrain': should_retrain,
243         'optimal_time': optimal_time,
244         'cost': cost,
245         'benefit': benefit,
246         'roi_metrics': roi_metrics,
247         'recommendation': 'RETRAIN' if should_retrain else 'SKIP'
248     }
249
250     self.logger.info(
251         f"Retraining decision: {decision['recommendation']} "
252         f"(ROI: {roi_metrics['roi_percent']:.1f}%, NPV: ${roi_metrics['npv_usd']:.2f})"
253     )
254
255     return should_retrain, decision
256
257 # Example: Cost-aware scheduling
258 cost_model = {
259     'base_compute_cost': 10.0,
260     'compute_cost_per_hour': 5.0,
261     'cost_per_sample': 0.0001,
262     'engineer_hourly_rate': 150.0,
263     'canary_cost': 20.0,
264     'revenue_per_hour': 1000.0,
265     'downtime_fraction': 0.1,
266     'revenue_per_performance_point': 5000.0,
267     'risk_reduction_value': 1000.0
268 }
269
270 scheduler = CostAwareScheduler(

```

```

271     cost_model=cost_model,
272     business_hours=(time(9), time(17)),
273     preferred_days=[1, 2, 3] # Tue-Thu
274 )
275
276 # Evaluate retraining decision
277 should_retrain, decision = scheduler.should_retrain_now(
278     current_performance=0.85,
279     expected_improvement=0.03, # 3% improvement expected
280     training_samples=50000,
281     duration_hours=2.5
282 )
283
284 print(f"\nRetraining Decision: {decision['recommendation']}")
285 print(f"Optimal Time: {decision['optimal_time']}")
286 print(f"Total Cost: ${decision['cost'].total_cost:.2f}")
287 print(f"Expected Benefit: ${decision['benefit'].revenue_impact_usd:.2f}")
288 print(f"ROI: {decision['roi_metrics']['roi_percent']:.1f}%")
289 print(f"Payback Period: {decision['roi_metrics']['payback_days']:.1f} days")

```

Listing 15.7: Cost-Aware Retraining Scheduler

15.8 Continuous Learning Systems

15.8.1 Real-Time Model Adaptation

Continuous learning systems maintain models that adapt in real-time to streaming data, enabling immediate response to changing patterns without batch retraining.

```

1  from typing import Dict, List, Optional, Callable, Tuple
2  from dataclasses import dataclass, field
3  from datetime import datetime
4  from collections import deque
5  import numpy as np
6  import pandas as pd
7  from threading import Thread, Lock
8  from queue import Queue
9  import time
10 import logging
11
12 @dataclass
13 class LearningEvent:
14     """Single learning event in continuous system"""
15     features: Dict[str, Any]
16     prediction: Optional[Any] = None
17     ground_truth: Optional[Any] = None
18     timestamp: datetime = field(default_factory=datetime.now)
19     feedback_delay_seconds: Optional[float] = None
20
21 class ContinuousLearningBuffer:
22     """
23     Thread-safe buffer for continuous learning
24
25     Manages streaming predictions and delayed ground truth
26     """
27

```



```

28 def __init__(self, max_buffer_size: int = 10000):
29     self.max_buffer_size = max_buffer_size
30     self.events: deque = deque(maxlen=max_buffer_size)
31     self.lock = Lock()
32
33     self.total_predictions = 0
34     self.total_labels_received = 0
35
36     self.logger = logging.getLogger(__name__)
37
38 def add_prediction(self, event_id: str, features: Dict, prediction: Any):
39     """Add prediction event"""
40
41     with self.lock:
42         event = LearningEvent(
43             features=features,
44             prediction=prediction,
45             timestamp=datetime.now()
46         )
47
48         self.events.append((event_id, event))
49         self.total_predictions += 1
50
51 def add_ground_truth(self, event_id: str, ground_truth: Any) -> bool:
52     """Add ground truth for existing prediction"""
53
54     with self.lock:
55         # Find matching prediction
56         for i, (eid, event) in enumerate(self.events):
57             if eid == event_id and event.ground_truth is None:
58                 # Calculate feedback delay
59                 delay = (datetime.now() - event.timestamp).total_seconds()
60                 event.ground_truth = ground_truth
61                 event.feedback_delay_seconds = delay
62
63                 self.total_labels_received += 1
64
65                 self.logger.debug(
66                     f"Received ground truth for {event_id} "
67                     f"(delay: {delay:.1f}s)"
68                 )
69
70                 return True
71
72     return False
73
74 def get_labeled_batch(self, batch_size: int = 100) -> List[Tuple[Dict, Any, Any]]:
75     """Get batch of labeled examples for training"""
76
77     labeled = []
78
79     with self.lock:
80         for event_id, event in self.events:
81             if event.ground_truth is not None and event.prediction is not None:
82                 labeled.append((
83                     event.features,
84                     event.prediction,
85                     event.ground_truth

```

```

86         ))
87
88         if len(labeled) >= batch_size:
89             break
90
91     return labeled
92
93     def get_stats(self) -> Dict[str, Any]:
94         """Get buffer statistics"""
95
96         with self.lock:
97             labeled_count = sum(
98                 1 for _, event in self.events
99                 if event.ground_truth is not None
100             )
101
102             avg_delay = np.mean([
103                 event.feedback_delay_seconds
104                 for _, event in self.events
105                 if event.feedback_delay_seconds is not None
106             ]) if labeled_count > 0 else 0
107
108             return {
109                 'buffer_size': len(self.events),
110                 'total_predictions': self.total_predictions,
111                 'total_labels': self.total_labels_received,
112                 'labeled_in_buffer': labeled_count,
113                 'avg_feedback_delay_seconds': avg_delay
114             }
115
116     class ContinuousLearningEngine:
117         """
118         Complete continuous learning engine
119
120         Manages real-time model updates with performance monitoring
121         and automatic quality control
122         """
123
124         def __init__(self,
125                     model,
126                     update_interval_seconds: int = 60,
127                     min_batch_size: int = 100,
128                     performance_check_interval: int = 1000):
129             """
130             Parameters:
131             - model: Base model supporting partial_fit or incremental learning
132             - update_interval_seconds: Time between model updates
133             - min_batch_size: Minimum samples before update
134             - performance_check_interval: Check performance every N predictions
135             """
136             self.model = model
137             self.update_interval = update_interval_seconds
138             self.min_batch_size = min_batch_size
139             self.performance_check_interval = performance_check_interval
140
141             self.buffer = ContinuousLearningBuffer()
142             self.model_lock = Lock()
143

```

```

144     # Performance tracking
145     self.recent_errors: deque = deque(maxlen=1000)
146     self.performance_history: List[Dict] = []
147
148     # Control flags
149     self.running = False
150     self.update_thread: Optional[Thread] = None
151
152     self.logger = logging.getLogger(__name__)
153
154     def start(self):
155         """Start continuous learning background thread"""
156
157         if self.running:
158             self.logger.warning("Continuous learning already running")
159             return
160
161         self.running = True
162         self.update_thread = Thread(target=self._update_loop, daemon=True)
163         self.update_thread.start()
164
165         self.logger.info("Continuous learning started")
166
167     def stop(self):
168         """Stop continuous learning"""
169
170         self.running = False
171
172         if self.update_thread:
173             self.update_thread.join(timeout=5)
174
175         self.logger.info("Continuous learning stopped")
176
177     def predict(self, event_id: str, features: Dict) -> Any:
178         """Make prediction and store for later learning"""
179
180         # Make prediction with current model
181         with self.model_lock:
182             X = pd.DataFrame([features])
183             prediction = self.model.predict(X)[0]
184
185         # Store in buffer
186         self.buffer.add_prediction(event_id, features, prediction)
187
188         # Check performance periodically
189         if self.buffer.total_predictions % self.performance_check_interval == 0:
190             self._check_performance()
191
192         return prediction
193
194     def provide_feedback(self, event_id: str, ground_truth: Any):
195         """Provide ground truth feedback for prediction"""
196
197         success = self.buffer.add_ground_truth(event_id, ground_truth)
198
199         if not success:
200             self.logger.warning(
201                 f"Could not find prediction for event {event_id}"

```

```

202         )
203
204     def _update_loop(self):
205         """Background loop for continuous model updates"""
206
207         self.logger.info("Update loop started")
208
209         while self.running:
210             try:
211                 # Wait for update interval
212                 time.sleep(self.update_interval)
213
214                 # Get labeled batch
215                 batch = self.buffer.get_labeled_batch(self.min_batch_size)
216
217                 if len(batch) >= self.min_batch_size:
218                     self._update_model(batch)
219                 else:
220                     self.logger.debug(
221                         f"Insufficient labeled samples: {len(batch)} "
222                         f"< {self.min_batch_size}"
223                     )
224
225             except Exception as e:
226                 self.logger.error(f"Error in update loop: {e}")
227
228     def _update_model(self, batch: List[Tuple[Dict, Any, Any]]):
229         """Update model with new batch"""
230
231         self.logger.info(f"Updating model with {len(batch)} samples")
232
233         # Prepare training data
234         X = pd.DataFrame([features for features, _, _ in batch])
235         y = pd.Series([label for _, _, label in batch])
236
237         # Calculate current batch error
238         predictions = [pred for _, pred, _ in batch]
239         errors = [abs(pred - label) for pred, label in zip(predictions, y)]
240         batch_error = np.mean(errors)
241
242         self.recent_errors.extend(errors)
243
244         # Update model
245         try:
246             with self.model_lock:
247                 if hasattr(self.model, 'partial_fit'):
248                     self.model.partial_fit(X, y)
249                 else:
250                     # Fall back to full fit (not ideal for continuous learning)
251                     self.model.fit(X, y)
252
253             self.logger.info(
254                 f"Model updated successfully (batch error: {batch_error:.4f})"
255             )
256
257         except Exception as e:
258             self.logger.error(f"Model update failed: {e}")
259

```

```

260 def _check_performance(self):
261     """Check and log current performance"""
262
263     if len(self.recent_errors) < 100:
264         return
265
266     stats = self.buffer.get_stats()
267
268     avg_error = np.mean(list(self.recent_errors))
269     error_std = np.std(list(self.recent_errors))
270
271     performance = {
272         'timestamp': datetime.now(),
273         'avg_error': avg_error,
274         'error_std': error_std,
275         'predictions': stats['total_predictions'],
276         'labels_received': stats['total_labels'],
277         'label_rate': (stats['total_labels'] / stats['total_predictions']
278                       if stats['total_predictions'] > 0 else 0)
279     }
280
281     self.performance_history.append(performance)
282
283     self.logger.info(
284         f"Performance check: avg_error={avg_error:.4f}, "
285         f"label_rate={performance['label_rate']:.2%}"
286     )
287
288 def get_metrics(self) -> Dict[str, Any]:
289     """Get current system metrics"""
290
291     buffer_stats = self.buffer.get_stats()
292
293     metrics = {
294         'buffer': buffer_stats,
295         'model_updates': len(self.performance_history),
296         'recent_avg_error': (np.mean(list(self.recent_errors))
297                             if self.recent_errors else 0),
298         'running': self.running
299     }
300
301     return metrics
302
303 # Example: Continuous learning system
304 from sklearn.linear_model import SGDRegressor
305 import uuid
306
307 # Create continuous learning engine
308 model = SGDRegressor(
309     learning_rate='adaptive',
310     eta0=0.01,
311     random_state=42
312 )
313
314 # Initial training
315 X_init = pd.DataFrame(np.random.randn(1000, 5))
316 y_init = X_init.sum(axis=1) + np.random.randn(1000) * 0.1
317 model.fit(X_init, y_init)

```

```

318
319 # Start continuous learning
320 engine = ContinuousLearningEngine(
321     model=model,
322     update_interval_seconds=5, # Update every 5 seconds
323     min_batch_size=50
324 )
325
326 engine.start()
327
328 # Simulate streaming predictions with delayed feedback
329 print("Starting continuous learning simulation...")
330
331 try:
332     for i in range(500):
333         # Generate features
334         features = {f'feature_{j}': np.random.randn() for j in range(5)}
335
336         # Make prediction
337         event_id = str(uuid.uuid4())
338         prediction = engine.predict(event_id, features)
339
340         # Simulate delayed ground truth (e.g., user feedback)
341         # In real system, this would come from user actions
342         ground_truth = sum(features.values()) + np.random.randn() * 0.1
343
344         # Simulate random delay in feedback (50% get feedback)
345         if np.random.random() < 0.5:
346             # Immediate feedback
347             engine.provide_feedback(event_id, ground_truth)
348         elif np.random.random() < 0.3:
349             # Delayed feedback simulation (would be async in production)
350             time.sleep(0.1)
351             engine.provide_feedback(event_id, ground_truth)
352
353         # Small delay between predictions
354         time.sleep(0.01)
355
356         # Print metrics periodically
357         if i % 100 == 0:
358             metrics = engine.get_metrics()
359             print(f"\nIteration {i}:")
360             print(f"  Buffer size: {metrics['buffer']['buffer_size']}")
361             print(f"  Labels received: {metrics['buffer']['total_labels']}")
362             print(f"  Model updates: {metrics['model_updates']}")
363             print(f"  Recent error: {metrics['recent_avg_error']:.4f}")
364
365 finally:
366     # Stop engine
367     engine.stop()
368     print("\nContinuous learning stopped")

```

Listing 15.8: Continuous Learning System Architecture

15.8.2 Champion-Challenger Framework

Production continuous learning systems often use champion-challenger patterns to safely test new models while maintaining a stable baseline.

```

1 from typing import Dict, List, Optional, Any
2 from dataclasses import dataclass
3 from datetime import datetime, timedelta
4 import numpy as np
5 import pandas as pd
6 import logging
7
8 @dataclass
9 class ModelPerformance:
10     """Track model performance metrics"""
11     model_id: str
12     predictions: int = 0
13     correct: int = 0
14     total_error: float = 0.0
15     recent_errors: List[float] = field(default_factory=list)
16
17     @property
18     def accuracy(self) -> float:
19         return self.correct / self.predictions if self.predictions > 0 else 0
20
21     @property
22     def avg_error(self) -> float:
23         return self.total_error / self.predictions if self.predictions > 0 else 0
24
25 class ChampionChallengerSystem:
26     """
27     Manage multiple model versions with champion-challenger pattern
28
29     Champion serves production traffic while challengers are evaluated
30     on a small percentage to identify better models
31     """
32
33     def __init__(self,
34                 champion_model,
35                 challenger_traffic_pct: float = 0.1,
36                 evaluation_window: int = 1000,
37                 promotion_threshold: float = 0.02):
38         """
39         Parameters:
40         - champion_model: Current production model
41         - challenger_traffic_pct: Percentage of traffic for challengers
42         - evaluation_window: Samples to evaluate before promotion decision
43         - promotion_threshold: Min improvement required for promotion
44         """
45         self.champion = champion_model
46         self.champion_id = "champion_v1"
47
48         self.challengers: Dict[str, Any] = {}
49         self.challenger_traffic_pct = challenger_traffic_pct
50         self.evaluation_window = evaluation_window
51         self.promotion_threshold = promotion_threshold
52
53         # Performance tracking

```

```

54     self.performance: Dict[str, ModelPerformance] = {
55         self.champion_id: ModelPerformance(model_id=self.champion_id)
56     }
57
58     self.total_predictions = 0
59
60     self.logger = logging.getLogger(__name__)
61
62     def add_challenger(self, challenger_id: str, model):
63         """Add new challenger model"""
64
65         self.challengers[challenger_id] = model
66         self.performance[challenger_id] = ModelPerformance(model_id=challenger_id)
67
68         self.logger.info(f"Added challenger: {challenger_id}")
69
70     def predict(self, features: Dict) -> Tuple[Any, str]:
71         """
72         Make prediction using champion or challenger
73
74         Returns: (prediction, model_id)
75         """
76
77         self.total_predictions += 1
78
79         # Decide which model to use
80         if self.challengers and np.random.random() < self.challenger_traffic_pct:
81             # Use random challenger
82             challenger_id = np.random.choice(list(self.challengers.keys()))
83             model = self.challengers[challenger_id]
84             model_id = challenger_id
85         else:
86             # Use champion
87             model = self.champion
88             model_id = self.champion_id
89
90         # Make prediction
91         X = pd.DataFrame([features])
92         prediction = model.predict(X)[0]
93
94         return prediction, model_id
95
96     def record_outcome(self, model_id: str, prediction: Any, ground_truth: Any):
97         """Record prediction outcome for performance tracking"""
98
99         if model_id not in self.performance:
100             return
101
102         perf = self.performance[model_id]
103         perf.predictions += 1
104
105         error = abs(prediction - ground_truth)
106         perf.total_error += error
107         perf.recent_errors.append(error)
108
109         # Keep only recent errors
110         if len(perf.recent_errors) > self.evaluation_window:
111             perf.recent_errors = perf.recent_errors[-self.evaluation_window:]

```



```

112     # For classification, track accuracy
113     if prediction == ground_truth:
114         perf.correct += 1
115
116     # Check if should evaluate for promotion
117     if perf.predictions % self.evaluation_window == 0:
118         self._evaluate_promotion(model_id)
119
120
121 def _evaluate_promotion(self, challenger_id: str):
122     """Evaluate if challenger should be promoted to champion"""
123
124     if challenger_id == self.champion_id:
125         return
126
127     challenger_perf = self.performance[challenger_id]
128     champion_perf = self.performance[self.champion_id]
129
130     # Need minimum samples for both
131     if (challenger_perf.predictions < self.evaluation_window or
132         champion_perf.predictions < self.evaluation_window):
133         return
134
135     # Compare performance
136     challenger_error = np.mean(challenger_perf.recent_errors)
137     champion_error = np.mean(champion_perf.recent_errors)
138
139     improvement = champion_error - challenger_error
140
141     self.logger.info(
142         f"Evaluation: {challenger_id} vs {self.champion_id}: "
143         f"challenger_error={challenger_error:.4f}, "
144         f"champion_error={champion_error:.4f}, "
145         f"improvement={improvement:.4f}"
146     )
147
148     # Promote if improvement exceeds threshold
149     if improvement > self.promotion_threshold:
150         self._promote_challenger(challenger_id)
151
152 def _promote_challenger(self, challenger_id: str):
153     """Promote challenger to champion"""
154
155     self.logger.info(f"PROMOTING {challenger_id} to champion")
156
157     # Old champion becomes archived
158     old_champion_id = self.champion_id
159
160     # Promote challenger
161     self.champion = self.challengers[challenger_id]
162     self.champion_id = challenger_id
163
164     # Remove from challengers
165     del self.challengers[challenger_id]
166
167     # Reset performance tracking
168     self.performance[challenger_id] = ModelPerformance(
169         model_id=challenger_id

```

```

170     )
171
172     def get_leaderboard(self) -> List[Dict]:
173         """Get current model performance leaderboard"""
174
175         leaderboard = []
176
177         for model_id, perf in self.performance.items():
178             if perf.predictions == 0:
179                 continue
180
181             status = "CHAMPION" if model_id == self.champion_id else "CHALLENGER"
182
183             leaderboard.append({
184                 'model_id': model_id,
185                 'status': status,
186                 'predictions': perf.predictions,
187                 'avg_error': perf.avg_error,
188                 'recent_error': (np.mean(perf.recent_errors)
189                                if perf.recent_errors else 0),
190                 'accuracy': perf.accuracy
191             })
192
193             # Sort by recent error
194             leaderboard.sort(key=lambda x: x['recent_error'])
195
196         return leaderboard
197
198     # Example: Champion-Challenger system
199     from sklearn.ensemble import GradientBoostingRegressor, RandomForestRegressor
200     from sklearn.linear_model import Ridge
201
202     # Create champion model
203     champion = GradientBoostingRegressor(n_estimators=50, random_state=42)
204     X_train = pd.DataFrame(np.random.randn(1000, 5))
205     y_train = X_train.sum(axis=1) + np.random.randn(1000) * 0.1
206     champion.fit(X_train, y_train)
207
208     # Create system
209     cc_system = ChampionChallengerSystem(
210         champion_model=champion,
211         challenger_traffic_pct=0.2, # 20% to challengers
212         evaluation_window=500,
213         promotion_threshold=0.01
214     )
215
216     # Add challenger models
217     challenger1 = RandomForestRegressor(n_estimators=50, random_state=42)
218     challenger1.fit(X_train, y_train)
219     cc_system.add_challenger("random_forest_v1", challenger1)
220
221     challenger2 = Ridge(alpha=1.0)
222     challenger2.fit(X_train, y_train)
223     cc_system.add_challenger("ridge_v1", challenger2)
224
225     # Simulate streaming predictions
226     print("Testing Champion-Challenger system...\n")
227

```

```

228 for i in range(2000):
229     # Generate sample
230     features = {f'feature_{j}': np.random.randn() for j in range(5)}
231     ground_truth = sum(features.values()) + np.random.randn() * 0.1
232
233     # Make prediction
234     prediction, model_id = cc_system.predict(features)
235
236     # Record outcome
237     cc_system.record_outcome(model_id, prediction, ground_truth)
238
239     # Print leaderboard periodically
240     if (i + 1) % 500 == 0:
241         print(f"\n=== After {i+1} predictions ===")
242         leaderboard = cc_system.get_leaderboard()
243
244         for rank, entry in enumerate(leaderboard, 1):
245             print(
246                 f"{rank}. {entry['status']:10s} {entry['model_id']:20s} "
247                 f"| Predictions: {entry['predictions']:4d} "
248                 f"| Recent Error: {entry['recent_error']:.4f}"
249             )

```

Listing 15.9: Champion-Challenger Continuous Learning

15.9 Practical Exercises

15.9.1 Exercise 1: Build an Automated Retraining Pipeline

Implement a complete automated retraining pipeline with monitoring, triggers, and deployment.

Requirements:

- Monitor model performance in production
- Detect when retraining is needed based on performance degradation
- Automatically collect training data from the past 90 days
- Train new model and evaluate against current champion
- Deploy new model if it shows >2% improvement
- Send notifications on retraining events (success/failure)
- Log all retraining events with metrics and outcomes

Dataset: Use an e-commerce conversion prediction dataset with daily updates.

Expected Output:

- Working retraining pipeline that runs on schedule or trigger
- Dashboard showing retraining history and model performance over time
- Alert system for retraining failures or performance issues

- Model registry tracking all versions with metadata

Hints:

- Use the `AutomatedRetrainingPipeline` class as a starting point
- Integrate with a workflow orchestrator (Airflow, Prefect, or simple cron)
- Store models in versioned storage (S3, GCS, or local with git-lfs)
- Use email/Slack webhooks for notifications
- Track metrics in a database or file-based store

15.9.2 Exercise 2: Implement Multi-Trigger Retraining Logic

Create a sophisticated trigger system that combines multiple signals to make intelligent retraining decisions.

Requirements:

- Implement performance degradation triggers (accuracy, precision, recall, F1)
- Add data drift triggers using PSI and KS statistics
- Include time-based triggers (monthly, or X days since last retrain)
- Implement data volume triggers (retrain when N new samples available)
- Add cooldown period to prevent excessive retraining
- Weight triggers by severity (critical, high, medium, low)
- Combine multiple trigger signals for final decision

Expected Output:

- `RetrainingTriggerManager` class with configurable thresholds
- Trigger evaluation logs showing which conditions fired
- Visualization of trigger history and patterns
- Configuration file for easy threshold adjustment

Test Scenarios:

1. Gradual performance degradation over 2 weeks
2. Sudden concept drift (performance drops 10% overnight)
3. Accumulation of 50,000 new training samples
4. 30 days elapsed since last retrain with stable performance
5. Multiple triggers firing simultaneously

15.9.3 Exercise 3: Build an Incremental Learning System

Implement an online learning system that updates continuously from streaming data.

Requirements:

- Use streaming data with delayed feedback labels
- Implement sliding window strategy for training data
- Support both `partial_fit` and full retraining approaches
- Track performance on hold-out validation set
- Implement temporal weighting (recent data weighted more)
- Add automatic rollback on performance degradation
- Monitor and log update frequency and batch sizes

Models to Test:

- `SGDClassifier` (supports `partial_fit`)
- `SGDRegressor` (supports `partial_fit`)
- `PassiveAggressiveClassifier`
- `MultinomialNB` (for text classification)

Expected Output:

- Incremental learning system processing streaming data
- Performance comparison: incremental vs. periodic full retraining
- Analysis of update frequency vs. model quality trade-off
- Memory usage profiling for different buffer sizes

15.9.4 Exercise 4: Implement Concept Drift Detection

Build a comprehensive drift detection system using multiple statistical tests.

Requirements:

- Implement Kolmogorov-Smirnov test for distribution drift
- Add Page-Hinkley test for gradual drift
- Use ADWIN (Adaptive Windowing) for change detection
- Implement ensemble drift detector (combine multiple tests)
- Detect both feature drift and concept drift
- Characterize drift type (sudden, gradual, recurring)
- Trigger appropriate adaptation strategy based on drift type

Test Datasets:

1. Synthetic data with known drift points
2. Weather prediction with seasonal patterns
3. Financial time series with market regime changes
4. User behavior data with gradual trend shifts

Expected Output:

- Drift detection accuracy (precision/recall for drift events)
- Time-to-detection for different drift types
- False positive rate analysis
- Comparison of different drift detection methods
- Visualization showing detected drift points

15.9.5 Exercise 5: Optimize Freshness-Stability Trade-off

Develop a system that adaptively balances model freshness against stability requirements.

Requirements:

- Track both freshness metrics (data age, drift score) and stability metrics (prediction variance, rollback rate)
- Implement adaptive retraining interval that adjusts based on stability
- Add prediction stability tracker that compares model versions
- Create configurable freshness-stability weight parameter
- Log all retraining decisions with justifications
- Measure business impact of different configurations

Scenarios to Test:

1. High-churn environment (prioritize freshness)
2. Regulated environment (prioritize stability)
3. Balanced scenario (equal weight)
4. Varying data drift rates over time

Metrics to Track:

- Average model age (staleness)
- Prediction variance between model versions
- Number of rollbacks required

- Overall model performance over time
- Retraining frequency and costs

Expected Output:

- Optimal freshness-stability configuration for each scenario
- Analysis showing trade-off curves
- Recommendation system for weight selection
- Cost-benefit analysis of different strategies

15.9.6 Exercise 6: Build a Cost-Aware Retraining Scheduler

Create an intelligent scheduler that optimizes retraining timing for cost and business impact.

Requirements:

- Estimate retraining costs (compute, data pipeline, engineering)
- Calculate expected benefits (performance improvement, revenue impact)
- Compute ROI for retraining decisions
- Find optimal time windows (avoid business hours, prefer off-peak)
- Consider opportunity costs of model downtime
- Implement multi-day lookahead scheduling
- Add budget constraints and cost tracking

Cost Model Components:

- Compute cost: based on training time and instance type
- Data pipeline cost: based on data volume processed
- Engineering oversight: human review and validation time
- Opportunity cost: potential revenue impact during deployment
- Validation cost: A/B testing or canary deployment costs

Expected Output:

- Scheduler that recommends optimal retraining times
- ROI calculations for each retraining decision
- Cost tracking dashboard
- Schedule optimization showing cost savings vs. naive scheduling
- Budget forecasting for monthly retraining costs

15.9.7 Exercise 7: Implement a Continuous Learning System with Champion-Challenger

Build a production-ready continuous learning system with safe model updates.

Requirements:

- Implement thread-safe continuous learning buffer
- Handle delayed feedback labels (predictions now, labels later)
- Run background model update thread
- Implement champion-challenger pattern for safe updates
- Route small percentage of traffic to challenger models
- Automatically promote challenger if performance improves
- Include automatic rollback on degradation
- Track and compare performance of all model versions

System Components:

1. **Prediction Service:** Serves real-time predictions, tracks events
2. **Feedback Collector:** Receives delayed ground truth labels
3. **Learning Buffer:** Thread-safe storage for training events
4. **Update Engine:** Periodic model updates from buffered data
5. **Model Registry:** Manages champion and challenger models
6. **Traffic Router:** Splits traffic between models for evaluation
7. **Performance Monitor:** Tracks and compares model performance
8. **Promotion Logic:** Decides when to promote challengers

Expected Output:

- Working continuous learning system processing streaming data
- Demonstration of automatic model promotion
- Performance comparison: champion vs. challengers over time
- Analysis of different traffic split percentages
- Latency and throughput benchmarks
- Safety analysis: rollback time, degradation detection delay

Advanced Extensions:

- Multi-armed bandit for dynamic traffic allocation

- Bayesian optimization for challenger hyperparameters
- Distributed learning across multiple workers
- Integration with feature store for consistent features
- A/B test statistical significance testing

15.10 Summary

Model maintenance and retraining are critical for keeping ML systems healthy in production. This chapter covered:

Automated Retraining Pipelines:

- Complete pipeline architecture with data collection, training, evaluation, and deployment
- Automated orchestration and error handling
- Integration with model registries and deployment systems

Intelligent Trigger Mechanisms:

- Multi-criteria triggers combining performance, drift, time, and data volume signals
- Configurable thresholds with severity levels
- Cooldown periods to prevent excessive retraining
- Trigger evaluation and decision logic

Incremental and Online Learning:

- Streaming data processing with delayed feedback
- Sliding window and temporal weighting strategies
- Online learning algorithms using `partial_fit`
- Performance monitoring and automatic rollback

Concept Drift Handling:

- Statistical drift detection methods (KS test, Page-Hinkley, ADWIN)
- Drift characterization (sudden, gradual, recurring)
- Adaptive strategies for different drift types
- Ensemble approaches for robust adaptation

Freshness vs. Stability Trade-offs:

- Balancing model currency against behavioral consistency
- Adaptive retraining intervals based on stability

- Prediction stability tracking across versions
- Application-specific optimization

Cost-Aware Scheduling:

- ROI-based retraining decisions
- Cost modeling for compute, data, and opportunity costs
- Optimal timing to minimize business impact
- Budget constraints and forecasting

Continuous Learning Systems:

- Real-time model adaptation from streaming data
- Thread-safe buffering and background updates
- Champion-challenger pattern for safe deployment
- Automatic promotion based on performance

Key Takeaways:

1. **Automate Everything:** Manual retraining doesn't scale. Build automated pipelines with proper monitoring and error handling.
2. **Use Multiple Signals:** Don't rely on a single trigger. Combine performance metrics, drift detection, data volume, and time-based schedules.
3. **Start Conservative:** Begin with longer retraining intervals and stricter promotion criteria. Relax constraints as you gain confidence in your system.
4. **Monitor Continuously:** Track both model performance and system health. Set up alerts for anomalies and failures.
5. **Handle Drift Adaptively:** Different types of drift require different responses. Characterize drift patterns and adapt accordingly.
6. **Balance Trade-offs:** Consider freshness vs. stability, cost vs. performance, and automation vs. control. Optimize for your specific business needs.
7. **Test Safely:** Use champion-challenger patterns, canary deployments, and automatic rollbacks to minimize risk.
8. **Measure Business Impact:** Connect retraining decisions to business metrics. Calculate ROI to justify infrastructure investment.

Production ML systems require ongoing maintenance. The patterns and code in this chapter provide a foundation for building robust, automated retraining systems that keep your models performing well as the world changes.

Chapter 16

Model Performance Optimization

16.1 Learning Objectives

After completing this chapter, readers will be able to:

- Apply model compression techniques including pruning, quantization, and knowledge distillation
- Optimize models for GPU and TPU hardware acceleration
- Implement memory-efficient inference and batch processing strategies
- Use inference optimization patterns for production deployments
- Optimize models for edge and mobile deployment
- Profile and benchmark ML systems to identify bottlenecks
- Measure and improve end-to-end inference latency
- Balance accuracy trade-offs against performance gains

16.2 Introduction

Performance optimization is critical for deploying machine learning models in production. While achieving high accuracy is important, a model that's too slow or resource-intensive is unusable in real-world applications. Users expect sub-second response times, edge devices have strict memory constraints, and cloud infrastructure costs scale with compute usage.

This chapter covers practical techniques for optimizing ML model performance:

- **Model Compression:** Reducing model size through pruning, quantization, and distillation
- **Hardware Acceleration:** Leveraging GPUs, TPUs, and specialized hardware
- **Memory Optimization:** Managing memory efficiently for large models and batches

- **Inference Patterns:** Optimizing serving architectures and request handling
- **Edge Deployment:** Adapting models for resource-constrained devices
- **Profiling and Benchmarking:** Measuring performance and identifying bottlenecks

The goal is to achieve the best possible performance while maintaining acceptable accuracy for your application requirements.

16.3 Model Compression Techniques

16.3.1 Quantization

Quantization reduces model size and improves inference speed by using lower-precision representations for weights and activations.

```

1 import torch
2 import torch.nn as nn
3 import torch.quantization as quantization
4 from typing import Dict, List, Tuple, Optional
5 import numpy as np
6 import time
7 from pathlib import Path
8 import logging
9
10 class ModelQuantizer:
11     """
12     Comprehensive quantization toolkit for PyTorch models
13
14     Supports dynamic, static, and QAT (Quantization-Aware Training)
15     """
16
17     def __init__(self, model: nn.Module):
18         self.model = model
19         self.logger = logging.getLogger(__name__)
20
21     def dynamic_quantization(self,
22                             dtype: torch.dtype = torch.qint8,
23                             layers_to_quantize: Optional[List[type]] = None) -> nn.
24     Module:
25         """
26         Apply dynamic quantization (weights quantized, activations computed in float)
27
28         Best for: LSTM, RNN, Transformer models where performance is limited by
29         loading weights from memory
30         """
31
32         if layers_to_quantize is None:
33             layers_to_quantize = {nn.Linear, nn.LSTM, nn.GRU}
34
35         self.logger.info("Applying dynamic quantization...")
36
37         quantized_model = quantization.quantize_dynamic(
38             self.model,
39             layers_to_quantize,

```

```

39         dtype=dtype
40     )
41
42     # Measure size reduction
43     original_size = self._get_model_size(self.model)
44     quantized_size = self._get_model_size(quantized_model)
45     reduction = (1 - quantized_size / original_size) * 100
46
47     self.logger.info(
48         f"Model size: {original_size:.2f}MB -> {quantized_size:.2f}MB "
49         f"({reduction:.1f}% reduction)"
50     )
51
52     return quantized_model
53
54     def static_quantization(self,
55                             calibration_data: torch.utils.data.DataLoader,
56                             backend: str = 'fbgemm') -> nn.Module:
57         """
58         Apply static quantization (both weights and activations quantized)
59
60         Best for: CNN models, edge deployment
61         Requires: Calibration data to determine activation ranges
62         """
63
64         self.logger.info("Applying static quantization...")
65
66         # Set backend
67         self.model.qconfig = quantization.get_default_qconfig(backend)
68
69         # Prepare model for quantization
70         quantization.prepare(self.model, inplace=True)
71
72         # Calibration: run representative data through model
73         self.logger.info("Calibrating with representative data...")
74         self.model.eval()
75
76         with torch.no_grad():
77             for batch_idx, (data, _) in enumerate(calibration_data):
78                 self.model(data)
79
80                 if batch_idx >= 100: # Calibrate on 100 batches
81                     break
82
83         # Convert to quantized model
84         quantized_model = quantization.convert(self.model, inplace=False)
85
86         # Measure improvement
87         original_size = self._get_model_size(self.model)
88         quantized_size = self._get_model_size(quantized_model)
89         reduction = (1 - quantized_size / original_size) * 100
90
91         self.logger.info(
92             f"Model size: {original_size:.2f}MB -> {quantized_size:.2f}MB "
93             f"({reduction:.1f}% reduction)"
94         )
95
96         return quantized_model

```

```

97
98 def quantization_aware_training(self,
99                               train_loader: torch.utils.data.DataLoader,
100                               val_loader: torch.utils.data.DataLoader,
101                               num_epochs: int = 10,
102                               learning_rate: float = 0.001) -> nn.Module:
103
104     """
105     Train with quantization in the loop (QAT)
106
107     Best for: Achieving highest accuracy with quantization
108     Requires: Ability to retrain the model
109     """
110
111     self.logger.info("Starting Quantization-Aware Training...")
112
113     # Prepare model for QAT
114     self.model.qconfig = quantization.get_default_qat_qconfig('fbgemm')
115     quantization.prepare_qat(self.model, inplace=True)
116
117     # Training setup
118     optimizer = torch.optim.Adam(self.model.parameters(), lr=learning_rate)
119     criterion = nn.CrossEntropyLoss()
120
121     self.model.train()
122
123     for epoch in range(num_epochs):
124         train_loss = 0.0
125
126         for batch_idx, (data, target) in enumerate(train_loader):
127             optimizer.zero_grad()
128             output = self.model(data)
129             loss = criterion(output, target)
130             loss.backward()
131             optimizer.step()
132
133             train_loss += loss.item()
134
135             if batch_idx % 100 == 0:
136                 self.logger.info(
137                     f"Epoch {epoch+1}/{num_epochs}, "
138                     f"Batch {batch_idx}, Loss: {loss.item():.4f}"
139                 )
140
141         # Validation
142         val_acc = self._validate(val_loader)
143         self.logger.info(
144             f"Epoch {epoch+1}: Val Accuracy: {val_acc:.2f}%"
145         )
146
147         # Convert to quantized model
148         self.model.eval()
149         quantized_model = quantization.convert(self.model, inplace=False)
150
151         return quantized_model
152
153 def _validate(self, val_loader: torch.utils.data.DataLoader) -> float:
154     """Validate model accuracy"""

```

```

155     self.model.eval()
156     correct = 0
157     total = 0
158
159     with torch.no_grad():
160         for data, target in val_loader:
161             output = self.model(data)
162             _, predicted = torch.max(output.data, 1)
163             total += target.size(0)
164             correct += (predicted == target).sum().item()
165
166     accuracy = 100 * correct / total
167     return accuracy
168
169 def _get_model_size(self, model: nn.Module) -> float:
170     """Get model size in MB"""
171
172     # Save to temporary file
173     temp_path = Path("/tmp/temp_model.pth")
174     torch.save(model.state_dict(), temp_path)
175     size_mb = temp_path.stat().st_size / (1024 * 1024)
176     temp_path.unlink()
177
178     return size_mb
179
180 def benchmark_quantization(self,
181                             test_data: torch.Tensor,
182                             num_runs: int = 100) -> Dict[str, float]:
183     """
184     Benchmark quantized vs original model
185
186     Returns metrics: latency, throughput, size, accuracy
187     """
188
189     results = {}
190
191     # Original model
192     self.logger.info("Benchmarking original model...")
193     original_latency = self._measure_latency(self.model, test_data, num_runs)
194     original_size = self._get_model_size(self.model)
195
196     results['original'] = {
197         'latency_ms': original_latency * 1000,
198         'throughput_qps': 1.0 / original_latency,
199         'size_mb': original_size
200     }
201
202     # Quantized model (dynamic)
203     self.logger.info("Benchmarking dynamically quantized model...")
204     quantized_model = self.dynamic_quantization()
205     quantized_latency = self._measure_latency(quantized_model, test_data, num_runs)
206
207     quantized_size = self._get_model_size(quantized_model)
208
209     results['quantized'] = {
210         'latency_ms': quantized_latency * 1000,
211         'throughput_qps': 1.0 / quantized_latency,
212         'size_mb': quantized_size,

```

```

212         'speedup': original_latency / quantized_latency,
213         'size_reduction': (1 - quantized_size / original_size) * 100
214     }
215
216     return results
217
218     def _measure_latency(self,
219                           model: nn.Module,
220                           test_data: torch.Tensor,
221                           num_runs: int = 100) -> float:
222         """Measure average inference latency"""
223
224         model.eval()
225
226         # Warmup
227         with torch.no_grad():
228             for _ in range(10):
229                 _ = model(test_data)
230
231         # Measure
232         times = []
233         with torch.no_grad():
234             for _ in range(num_runs):
235                 start = time.perf_counter()
236                 _ = model(test_data)
237                 end = time.perf_counter()
238                 times.append(end - start)
239
240         return np.mean(times)
241
242     # Example: Quantize a model
243     import torch.nn as nn
244
245     # Define simple model
246     class SimpleModel(nn.Module):
247         def __init__(self):
248             super().__init__()
249             self.fc1 = nn.Linear(784, 512)
250             self.relu = nn.ReLU()
251             self.fc2 = nn.Linear(512, 256)
252             self.fc3 = nn.Linear(256, 10)
253
254         def forward(self, x):
255             x = x.view(-1, 784)
256             x = self.relu(self.fc1(x))
257             x = self.relu(self.fc2(x))
258             x = self.fc3(x)
259             return x
260
261     # Create and quantize model
262     model = SimpleModel()
263     quantizer = ModelQuantizer(model)
264
265     # Dynamic quantization (easiest, no calibration needed)
266     quantized_model = quantizer.dynamic_quantization()
267
268     # Benchmark
269     test_input = torch.randn(1, 1, 28, 28)

```



```

270 results = quantizer.benchmark_quantization(test_input)
271
272 print("\nQuantization Results:")
273 print(f"Original: {results['original']['latency_ms']:.2f}ms, "
274       f"{results['original']['size_mb']:.2f}MB")
275 print(f"Quantized: {results['quantized']['latency_ms']:.2f}ms, "
276       f"{results['quantized']['size_mb']:.2f}MB")
277 print(f"Speedup: {results['quantized']['speedup']:.2f}x")
278 print(f"Size Reduction: {results['quantized']['size_reduction']:.1f}%")

```

Listing 16.1: Post-Training Quantization Implementation

16.3.2 Model Pruning

Pruning removes redundant weights or neurons to create sparse models that are smaller and faster.

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.utils.prune as prune
4 from typing import List, Tuple, Dict, Optional
5 import numpy as np
6 import logging
7
8 class ModelPruner:
9     """
10     Comprehensive model pruning toolkit
11
12     Supports unstructured, structured, and iterative pruning
13     """
14
15     def __init__(self, model: nn.Module):
16         self.model = model
17         self.logger = logging.getLogger(__name__)
18         self.pruning_history: List[Dict] = []
19
20     def unstructured_magnitude_pruning(self,
21                                       amount: float = 0.3,
22                                       layers_to_prune: Optional[List[str]] = None) ->
23 nn.Module:
24     """
25     Prune individual weights based on magnitude
26
27     Parameters:
28     - amount: Fraction of weights to prune (0-1)
29     - layers_to_prune: List of layer names, None = prune all
30     """
31
32     self.logger.info(f"Applying unstructured pruning (amount: {amount})")
33
34     if layers_to_prune is None:
35         layers_to_prune = [name for name, module in self.model.named_modules()
36                           if isinstance(module, (nn.Linear, nn.Conv2d))]
37
38     # Prune each layer
39     for name, module in self.model.named_modules():
40         if name in layers_to_prune:

```

```

40         if isinstance(module, nn.Linear):
41             prune.l1_unstructured(module, name='weight', amount=amount)
42         elif isinstance(module, nn.Conv2d):
43             prune.l1_unstructured(module, name='weight', amount=amount)
44
45         self.logger.info(f"Pruned layer: {name}")
46
47         # Calculate sparsity
48         sparsity = self._calculate_sparsity()
49         self.logger.info(f"Model sparsity: {sparsity:.2%}")
50
51         self.pruning_history.append({
52             'method': 'unstructured_magnitude',
53             'amount': amount,
54             'sparsity': sparsity
55         })
56
57         return self.model
58
59     def structured_pruning(self,
60                           amount: float = 0.3,
61                           dim: int = 0) -> nn.Module:
62         """
63         Prune entire structures (channels, neurons) for hardware efficiency
64
65         Parameters:
66         - amount: Fraction of structures to prune
67         - dim: Dimension to prune (0 for output channels, 1 for input)
68         """
69
70         self.logger.info(f"Applying structured pruning (amount: {amount}, dim: {dim})")
71     )
72
73     for name, module in self.model.named_modules():
74         if isinstance(module, nn.Conv2d):
75             # Prune entire filters/channels
76             prune.ln_structured(
77                 module,
78                 name='weight',
79                 amount=amount,
80                 n=2, # L2 norm
81                 dim=dim
82             )
83             self.logger.info(f"Structured pruning on: {name}")
84
85         elif isinstance(module, nn.Linear):
86             # Prune entire neurons
87             prune.ln_structured(
88                 module,
89                 name='weight',
90                 amount=amount,
91                 n=2,
92                 dim=dim
93             )
94             self.logger.info(f"Structured pruning on: {name}")
95
96         sparsity = self._calculate_sparsity()
97         self.logger.info(f"Model sparsity: {sparsity:.2%}")

```

```

97     return self.model
98
99
100 def iterative_pruning(self,
101     initial_sparsity: float = 0.0,
102     final_sparsity: float = 0.8,
103     num_iterations: int = 10,
104     train_func: Optional[Callable] = None) -> nn.Module:
105     """
106     Gradually increase pruning over multiple iterations with retraining
107
108     This often achieves better accuracy than one-shot pruning
109     """
110
111     self.logger.info(
112         f"Starting iterative pruning: {initial_sparsity} -> {final_sparsity} "
113         f"over {num_iterations} iterations"
114     )
115
116     # Calculate pruning schedule
117     sparsity_schedule = np.linspace(
118         initial_sparsity,
119         final_sparsity,
120         num_iterations
121     )
122
123     for iteration, target_sparsity in enumerate(sparsity_schedule):
124         self.logger.info(f"\nIteration {iteration + 1}/{num_iterations}")
125         self.logger.info(f"Target sparsity: {target_sparsity:.2%}")
126
127         # Prune to target sparsity
128         current_sparsity = self._calculate_sparsity()
129         prune_amount = (target_sparsity - current_sparsity) / (1 -
130 current_sparsity)
131
132         if prune_amount > 0:
133             self.unstructured_magnitude_pruning(amount=prune_amount)
134
135         # Retrain (if training function provided)
136         if train_func is not None:
137             self.logger.info("Retraining pruned model...")
138             train_func(self.model)
139
140         # Validate
141         actual_sparsity = self._calculate_sparsity()
142         self.logger.info(f"Achieved sparsity: {actual_sparsity:.2%}")
143
144     return self.model
145
146 def make_pruning_permanent(self):
147     """
148     Remove pruning reparameterization and make pruning permanent
149
150     Call this after pruning to reduce memory and computation
151     """
152
153     self.logger.info("Making pruning permanent...")

```

```

154     for name, module in self.model.named_modules():
155         if isinstance(module, (nn.Linear, nn.Conv2d)):
156             try:
157                 prune.remove(module, 'weight')
158             except:
159                 pass # Layer was not pruned
160
161     self.logger.info("Pruning made permanent")
162
163     def _calculate_sparsity(self) -> float:
164         """Calculate overall model sparsity"""
165
166         total_params = 0
167         zero_params = 0
168
169         for name, param in self.model.named_parameters():
170             if 'weight' in name:
171                 total_params += param.numel()
172                 zero_params += (param == 0).sum().item()
173
174         sparsity = zero_params / total_params if total_params > 0 else 0
175         return sparsity
176
177     def analyze_layer_sparsity(self) -> Dict[str, float]:
178         """Get sparsity for each layer"""
179
180         layer_sparsity = {}
181
182         for name, module in self.model.named_modules():
183             if isinstance(module, (nn.Linear, nn.Conv2d)):
184                 if hasattr(module, 'weight'):
185                     total = module.weight.numel()
186                     zeros = (module.weight == 0).sum().item()
187                     sparsity = zeros / total
188                     layer_sparsity[name] = sparsity
189
190         return layer_sparsity
191
192     def benchmark_pruning(self,
193                          test_data: torch.Tensor,
194                          ground_truth: torch.Tensor,
195                          num_runs: int = 100) -> Dict[str, any]:
196         """
197         Benchmark pruned model: accuracy, latency, size
198         """
199
200         results = {}
201
202         # Accuracy
203         self.model.eval()
204         with torch.no_grad():
205             output = self.model(test_data)
206             _, predicted = torch.max(output.data, 1)
207             accuracy = (predicted == ground_truth).float().mean().item()
208
209         # Latency
210         times = []
211         with torch.no_grad():

```

```

212         for _ in range(num_runs):
213             start = time.perf_counter()
214             _ = self.model(test_data[:1]) # Single sample
215             end = time.perf_counter()
216             times.append(end - start)
217
218         # Size
219         total_params = sum(p.numel() for p in self.model.parameters())
220         non_zero_params = sum((p != 0).sum().item() for p in self.model.parameters())
221
222         results = {
223             'accuracy': accuracy * 100,
224             'latency_ms': np.mean(times) * 1000,
225             'latency_std': np.std(times) * 1000,
226             'total_params': total_params,
227             'non_zero_params': non_zero_params,
228             'sparsity': 1 - (non_zero_params / total_params),
229             'compression_ratio': total_params / non_zero_params
230         }
231
232         return results
233
234     # Example: Prune a model
235     model = SimpleModel()
236     pruner = ModelPruner(model)
237
238     # Method 1: One-shot unstructured pruning
239     pruned_model = pruner.unstructured_magnitude_pruning(amount=0.5)
240
241     # Analyze layer-wise sparsity
242     layer_sparsity = pruner.analyze_layer_sparsity()
243     print("\nLayer-wise Sparsity:")
244     for layer, sparsity in layer_sparsity.items():
245         print(f" {layer}: {sparsity:.2%}")
246
247     # Make pruning permanent
248     pruner.make_pruning_permanent()
249
250     # Benchmark
251     test_input = torch.randn(100, 1, 28, 28)
252     test_labels = torch.randint(0, 10, (100,))
253     results = pruner.benchmark_pruning(test_input, test_labels)
254
255     print("\nPruning Results:")
256     print(f"Sparsity: {results['sparsity']:.2%}")
257     print(f"Compression: {results['compression_ratio']:.1f}x")
258     print(f"Latency: {results['latency_ms']:.2f}ms")
259     print(f"Accuracy: {results['accuracy']:.2f}%")

```

Listing 16.2: Neural Network Pruning Implementation

16.3.3 Knowledge Distillation

Knowledge distillation transfers knowledge from a large "teacher" model to a smaller "student" model, enabling compact models with near-teacher performance.

```
1 import torch
```

```

2 import torch.nn as nn
3 import torch.nn.functional as F
4 from typing import Optional, Tuple
5 import logging
6
7 class KnowledgeDistillation:
8     """
9     Train student model using knowledge from teacher model
10
11     The student learns from both ground truth labels and
12     soft predictions from the teacher
13     """
14
15     def __init__(self,
16                 teacher_model: nn.Module,
17                 student_model: nn.Module,
18                 temperature: float = 3.0,
19                 alpha: float = 0.7):
20         """
21         Parameters:
22         - teacher_model: Large, accurate teacher model
23         - student_model: Smaller student model to train
24         - temperature: Softens probability distributions (higher = softer)
25         - alpha: Weight for distillation loss vs hard label loss
26         """
27         self.teacher = teacher_model
28         self.student = student_model
29         self.temperature = temperature
30         self.alpha = alpha
31
32         self.logger = logging.getLogger(__name__)
33
34         # Freeze teacher
35         self.teacher.eval()
36         for param in self.teacher.parameters():
37             param.requires_grad = False
38
39     def distillation_loss(self,
40                        student_logits: torch.Tensor,
41                        teacher_logits: torch.Tensor,
42                        labels: torch.Tensor,
43                        temperature: Optional[float] = None) -> Tuple[torch.Tensor,
44                        Dict]:
45         """
46         Compute distillation loss
47
48         Loss = alpha * KL(teacher, student) + (1-alpha) * CE(student, labels)
49         """
50
51         if temperature is None:
52             temperature = self.temperature
53
54         # Soft targets from teacher
55         soft_teacher = F.softmax(teacher_logits / temperature, dim=1)
56         soft_student = F.log_softmax(student_logits / temperature, dim=1)
57
58         # Distillation loss (KL divergence)
59         distill_loss = F.kl_div(

```

```

59         soft_student,
60         soft_teacher,
61         reduction='batchmean'
62     ) * (temperature ** 2)
63
64     # Hard label loss
65     hard_loss = F.cross_entropy(student_logits, labels)
66
67     # Combined loss
68     total_loss = self.alpha * distill_loss + (1 - self.alpha) * hard_loss
69
70     loss_dict = {
71         'total_loss': total_loss.item(),
72         'distill_loss': distill_loss.item(),
73         'hard_loss': hard_loss.item()
74     }
75
76     return total_loss, loss_dict
77
78     def train_student(self,
79                       train_loader: torch.utils.data.DataLoader,
80                       val_loader: torch.utils.data.DataLoader,
81                       num_epochs: int = 20,
82                       learning_rate: float = 0.001,
83                       device: str = 'cpu') -> Dict:
84         """Train student model using knowledge distillation"""
85
86         self.logger.info("Starting knowledge distillation training...")
87
88         # Move models to device
89         self.teacher.to(device)
90         self.student.to(device)
91
92         # Optimizer
93         optimizer = torch.optim.Adam(
94             self.student.parameters(),
95             lr=learning_rate
96         )
97
98         # Training loop
99         best_val_acc = 0.0
100         history = {
101             'train_loss': [],
102             'val_acc': [],
103             'distill_loss': [],
104             'hard_loss': []
105         }
106
107         for epoch in range(num_epochs):
108             self.student.train()
109             epoch_losses = {'total': 0.0, 'distill': 0.0, 'hard': 0.0}
110             num_batches = 0
111
112             for batch_idx, (data, target) in enumerate(train_loader):
113                 data, target = data.to(device), target.to(device)
114
115                 # Get teacher predictions
116                 with torch.no_grad():

```

```

117         teacher_logits = self.teacher(data)
118
119         # Get student predictions
120         student_logits = self.student(data)
121
122         # Compute loss
123         loss, loss_dict = self.distillation_loss(
124             student_logits,
125             teacher_logits,
126             target
127         )
128
129         # Backprop
130         optimizer.zero_grad()
131         loss.backward()
132         optimizer.step()
133
134         # Track losses
135         epoch_losses['total'] += loss_dict['total_loss']
136         epoch_losses['distill'] += loss_dict['distill_loss']
137         epoch_losses['hard'] += loss_dict['hard_loss']
138         num_batches += 1
139
140         if batch_idx % 100 == 0:
141             self.logger.info(
142                 f"Epoch {epoch+1}/{num_epochs}, "
143                 f"Batch {batch_idx}, Loss: {loss.item():.4f}"
144             )
145
146         # Average losses
147         avg_losses = {k: v / num_batches for k, v in epoch_losses.items()}
148
149         # Validation
150         val_acc = self._validate_student(val_loader, device)
151
152         # Log progress
153         self.logger.info(
154             f"Epoch {epoch+1}: "
155             f"Train Loss: {avg_losses['total']:.4f}, "
156             f"Val Acc: {val_acc:.2f}%"
157         )
158
159         # Save history
160         history['train_loss'].append(avg_losses['total'])
161         history['distill_loss'].append(avg_losses['distill'])
162         history['hard_loss'].append(avg_losses['hard'])
163         history['val_acc'].append(val_acc)
164
165         # Track best
166         if val_acc > best_val_acc:
167             best_val_acc = val_acc
168             self.logger.info(f"New best validation accuracy: {best_val_acc:.2f}%")
169
170         history['best_val_acc'] = best_val_acc
171
172         return history
173
174     def _validate_student(self,

```



```

175         val_loader: torch.utils.data.DataLoader,
176         device: str) -> float:
177     """Validate student model"""
178
179     self.student.eval()
180     correct = 0
181     total = 0
182
183     with torch.no_grad():
184         for data, target in val_loader:
185             data, target = data.to(device), target.to(device)
186             output = self.student(data)
187             _, predicted = torch.max(output.data, 1)
188             total += target.size(0)
189             correct += (predicted == target).sum().item()
190
191     accuracy = 100 * correct / total
192     return accuracy
193
194 def compare_models(self,
195                   test_loader: torch.utils.data.DataLoader,
196                   device: str = 'cpu') -> Dict:
197     """Compare teacher and student performance"""
198
199     self.logger.info("Comparing teacher vs student...")
200
201     # Teacher accuracy
202     teacher_acc = self._validate_model(self.teacher, test_loader, device)
203
204     # Student accuracy
205     student_acc = self._validate_model(self.student, test_loader, device)
206
207     # Model sizes
208     teacher_params = sum(p.numel() for p in self.teacher.parameters())
209     student_params = sum(p.numel() for p in self.student.parameters())
210
211     results = {
212         'teacher_accuracy': teacher_acc,
213         'student_accuracy': student_acc,
214         'accuracy_drop': teacher_acc - student_acc,
215         'teacher_params': teacher_params,
216         'student_params': student_params,
217         'compression_ratio': teacher_params / student_params,
218         'params_reduction': (1 - student_params / teacher_params) * 100
219     }
220
221     self.logger.info(f"Teacher: {teacher_acc:.2f}%, {teacher_params:,} params")
222     self.logger.info(f"Student: {student_acc:.2f}%, {student_params:,} params")
223     self.logger.info(f"Compression: {results['compression_ratio']:.1f}x")
224     self.logger.info(f"Accuracy drop: {results['accuracy_drop']:.2f}%")
225
226     return results
227
228 def _validate_model(self,
229                   model: nn.Module,
230                   loader: torch.utils.data.DataLoader,
231                   device: str) -> float:
232     """Validate any model"""

```

```

233     model.eval()
234     correct = 0
235     total = 0
236
237     with torch.no_grad():
238         for data, target in loader:
239             data, target = data.to(device), target.to(device)
240             output = model(data)
241             _, predicted = torch.max(output.data, 1)
242             total += target.size(0)
243             correct += (predicted == target).sum().item()
244
245     return 100 * correct / total
246
247
248 # Example: Distill knowledge from teacher to student
249 class TeacherModel(nn.Module):
250     """Large teacher model"""
251     def __init__(self):
252         super().__init__()
253         self.fc1 = nn.Linear(784, 1024)
254         self.fc2 = nn.Linear(1024, 512)
255         self.fc3 = nn.Linear(512, 256)
256         self.fc4 = nn.Linear(256, 10)
257         self.relu = nn.ReLU()
258
259     def forward(self, x):
260         x = x.view(-1, 784)
261         x = self.relu(self.fc1(x))
262         x = self.relu(self.fc2(x))
263         x = self.relu(self.fc3(x))
264         x = self.fc4(x)
265         return x
266
267 class StudentModel(nn.Module):
268     """Small student model"""
269     def __init__(self):
270         super().__init__()
271         self.fc1 = nn.Linear(784, 128)
272         self.fc2 = nn.Linear(128, 64)
273         self.fc3 = nn.Linear(64, 10)
274         self.relu = nn.ReLU()
275
276     def forward(self, x):
277         x = x.view(-1, 784)
278         x = self.relu(self.fc1(x))
279         x = self.relu(self.fc2(x))
280         x = self.fc3(x)
281         return x
282
283 # Create teacher and student
284 teacher = TeacherModel()
285 student = StudentModel()
286
287 # Train teacher first (assume it's already trained)
288 # teacher.load_state_dict(torch.load('teacher.pth'))
289
290 # Distill knowledge

```

```

291 distiller = KnowledgeDistillation(
292     teacher_model=teacher,
293     student_model=student,
294     temperature=3.0,
295     alpha=0.7
296 )
297
298 # Train student (with mock data loader)
299 # history = distiller.train_student(train_loader, val_loader, num_epochs=20)
300
301 # Compare models
302 # results = distiller.compare_models(test_loader)
303 print(f"Student has {results.get('compression_ratio', 8):.1f}x fewer parameters")

```

Listing 16.3: Knowledge Distillation Framework

16.4 Hardware Acceleration

16.4.1 GPU Optimization

Leveraging GPUs effectively requires optimizing data transfer, computation, and memory usage.

```

1 import torch
2 import torch.nn as nn
3 from typing import Dict, List, Optional, Tuple
4 import time
5 import numpy as np
6 from contextlib import contextmanager
7 import logging
8
9 class GPUOptimizer:
10     """
11     Optimize model for GPU inference
12
13     Implements mixed precision, kernel fusion, and efficient batching
14     """
15
16     def __init__(self, model: nn.Module, device: str = 'cuda'):
17         self.model = model
18         self.device = device
19
20         if not torch.cuda.is_available() and device == 'cuda':
21             raise RuntimeError("CUDA is not available")
22
23         self.model.to(device)
24         self.logger = logging.getLogger(__name__)
25
26     @contextmanager
27     def autocast_context(self, enabled: bool = True):
28         """Context manager for mixed precision"""
29         if enabled and torch.cuda.is_available():
30             with torch.cuda.amp.autocast():
31                 yield
32         else:
33             yield

```

```

34
35 def optimize_for_inference(self):
36     """Apply inference-time optimizations"""
37
38     self.logger.info("Optimizing model for inference...")
39
40     # Set to eval mode
41     self.model.eval()
42
43     # Disable gradient computation
44     for param in self.model.parameters():
45         param.requires_grad = False
46
47     # Use inference mode context
48     torch.set_grad_enabled(False)
49
50     # Optimize with TorchScript
51     try:
52         example_input = torch.randn(1, 3, 224, 224).to(self.device)
53         self.model = torch.jit.trace(self.model, example_input)
54         self.logger.info("Model traced with TorchScript")
55     except Exception as e:
56         self.logger.warning(f"TorchScript tracing failed: {e}")
57
58     self.logger.info("Inference optimization complete")
59
60 def mixed_precision_inference(self,
61                               data: torch.Tensor,
62                               use_amp: bool = True) -> torch.Tensor:
63     """
64     Run inference with automatic mixed precision
65
66     Uses FP16 for computation, maintaining accuracy
67     """
68
69     data = data.to(self.device)
70
71     with self.autocast_context(enabled=use_amp):
72         with torch.no_grad():
73             output = self.model(data)
74
75     return output
76
77 def optimize_batch_size(self,
78                         sample_input: torch.Tensor,
79                         max_batch_size: int = 512) -> int:
80     """
81     Find optimal batch size for GPU memory
82
83     Returns largest batch size that fits in GPU memory
84     """
85
86     self.logger.info("Finding optimal batch size...")
87
88     batch_size = 1
89     optimal_batch = 1
90
91     while batch_size <= max_batch_size:

```

```

92         try:
93             # Try inference with this batch size
94             test_batch = sample_input.repeat(batch_size, 1, 1, 1).to(self.device)
95
96             torch.cuda.empty_cache()
97             torch.cuda.reset_peak_memory_stats()
98
99             with torch.no_grad():
100                 _ = self.model(test_batch)
101
102             # Check memory usage
103             mem_used = torch.cuda.max_memory_allocated() / 1024**3
104             mem_total = torch.cuda.get_device_properties(0).total_memory / 1024**3
105
106             mem_pct = (mem_used / mem_total) * 100
107
108             self.logger.info(
109                 f"Batch size {batch_size}: {mem_used:.2f}GB / "
110                 f"{mem_total:.2f}GB ({mem_pct:.1f}%)"
111             )
112
113             # If we're using < 80% memory, this is safe
114             if mem_pct < 80:
115                 optimal_batch = batch_size
116                 batch_size *= 2
117             else:
118                 break
119
120         except RuntimeError as e:
121             if "out of memory" in str(e):
122                 self.logger.warning(
123                     f"OOM at batch size {batch_size}, using {optimal_batch}"
124                 )
125                 break
126             else:
127                 raise e
128
129         self.logger.info(f"Optimal batch size: {optimal_batch}")
130         return optimal_batch
131
132     def benchmark_gpu_performance(self,
133                                  sample_input: torch.Tensor,
134                                  batch_sizes: List[int] = [1, 8, 16, 32, 64],
135                                  num_runs: int = 100) -> Dict:
136         """
137         Benchmark inference performance at different batch sizes
138         """
139
140         results = {}
141
142         for batch_size in batch_sizes:
143             try:
144                 # Prepare batch
145                 batch = sample_input.repeat(batch_size, 1, 1, 1).to(self.device)
146
147                 # Warmup
148                 for _ in range(10):
149                     with torch.no_grad():

```



```

207         torch.profiler.ProfilerActivity.CUDA,
208     ],
209     record_shapes=True,
210     profile_memory=True,
211     with_stack=True
212 ) as prof:
213     for _ in range(num_iterations):
214         with torch.no_grad():
215             _ = self.model(sample_input)
216
217     # Print summary
218     print(prof.key_averages().table(
219         sort_by="cuda_time_total",
220         row_limit=10
221     ))
222
223     # Save trace for visualization
224     prof.export_chrome_trace("gpu_trace.json")
225     self.logger.info("Trace saved to gpu_trace.json")
226
227     return prof
228
229 # Example: GPU optimization
230 class ResNetLike(nn.Module):
231     """Example CNN model"""
232     def __init__(self):
233         super().__init__()
234         self.conv1 = nn.Conv2d(3, 64, kernel_size=7, stride=2, padding=3)
235         self.bn1 = nn.BatchNorm2d(64)
236         self.relu = nn.ReLU(inplace=True)
237         self.maxpool = nn.MaxPool2d(kernel_size=3, stride=2, padding=1)
238
239         self.layer1 = self._make_layer(64, 128, 2)
240         self.layer2 = self._make_layer(128, 256, 2)
241         self.avgpool = nn.AdaptiveAvgPool2d((1, 1))
242         self.fc = nn.Linear(256, 1000)
243
244     def _make_layer(self, in_channels, out_channels, blocks):
245         layers = []
246         for _ in range(blocks):
247             layers.append(nn.Conv2d(in_channels, out_channels, 3, padding=1))
248             layers.append(nn.BatchNorm2d(out_channels))
249             layers.append(nn.ReLU(inplace=True))
250             in_channels = out_channels
251         return nn.Sequential(*layers)
252
253     def forward(self, x):
254         x = self.maxpool(self.relu(self.bn1(self.conv1(x))))
255         x = self.layer1(x)
256         x = self.layer2(x)
257         x = self.avgpool(x)
258         x = torch.flatten(x, 1)
259         x = self.fc(x)
260         return x
261
262 # Create model and optimizer
263 if torch.cuda.is_available():
264     model = ResNetLike()

```

```

265 optimizer = GPUOptimizer(model, device='cuda')
266
267 # Optimize for inference
268 optimizer.optimize_for_inference()
269
270 # Find optimal batch size
271 sample = torch.randn(1, 3, 224, 224)
272 optimal_batch = optimizer.optimize_batch_size(sample)
273
274 # Benchmark
275 results = optimizer.benchmark_gpu_performance(
276     sample,
277     batch_sizes=[1, 8, 16, 32]
278 )
279
280 print("\nGPU Performance Results:")
281 for bs, metrics in results.items():
282     print(f"Batch {bs}: {metrics['throughput_samples_per_sec']:.1f} samples/sec")

```

Listing 16.4: GPU Optimization Techniques

16.5 Memory Optimization and Batch Processing

16.5.1 Memory-Efficient Inference

Efficient memory management is critical for serving large models and processing large batches.

```

1 import torch
2 import torch.nn as nn
3 from typing import List, Iterator, Optional, Dict
4 import numpy as np
5 from collections import deque
6 import gc
7 import logging
8
9 class MemoryEfficientInference:
10     """
11     Memory-optimized inference for large models
12
13     Implements gradient checkpointing, batch accumulation,
14     and efficient memory management
15     """
16
17     def __init__(self, model: nn.Module, device: str = 'cpu'):
18         self.model = model
19         self.device = device
20         self.model.to(device)
21         self.logger = logging.getLogger(__name__)
22
23     def chunked_inference(self,
24                           data: torch.Tensor,
25                           chunk_size: int = 32) -> torch.Tensor:
26         """
27         Process large batch in chunks to avoid OOM
28

```



```

29     Breaks large batch into smaller chunks, processes each,
30     and concatenates results
31     """
32
33     self.model.eval()
34
35     results = []
36     num_samples = data.size(0)
37
38     with torch.no_grad():
39         for start_idx in range(0, num_samples, chunk_size):
40             end_idx = min(start_idx + chunk_size, num_samples)
41             chunk = data[start_idx:end_idx].to(self.device)
42
43             # Process chunk
44             output = self.model(chunk)
45
46             # Move to CPU to save GPU memory
47             results.append(output.cpu())
48
49             # Clear GPU cache
50             if self.device == 'cuda':
51                 torch.cuda.empty_cache()
52
53             # Concatenate results
54             final_output = torch.cat(results, dim=0)
55
56     return final_output
57
58 def streaming_inference(self,
59                         data_iterator: Iterator[torch.Tensor],
60                         batch_size: int = 16) -> Iterator[torch.Tensor]:
61     """
62     Stream inference for continuous data
63
64     Processes data as it arrives without loading everything into memory
65     """
66
67     self.model.eval()
68
69     batch_buffer = []
70
71     with torch.no_grad():
72         for sample in data_iterator:
73             batch_buffer.append(sample)
74
75             # Process when batch is full
76             if len(batch_buffer) >= batch_size:
77                 batch = torch.stack(batch_buffer).to(self.device)
78                 output = self.model(batch)
79
80                 # Yield results one by one
81                 for result in output:
82                     yield result.cpu()
83
84                 batch_buffer.clear()
85
86             # Clean up

```

```

87         if self.device == 'cuda':
88             torch.cuda.empty_cache()
89
90         # Process remaining samples
91         if batch_buffer:
92             batch = torch.stack(batch_buffer).to(self.device)
93             output = self.model(batch)
94
95         for result in output:
96             yield result.cpu()
97
98     def measure_memory_usage(self,
99                             sample_input: torch.Tensor,
100                             batch_sizes: List[int] = [1, 8, 16, 32, 64]) -> Dict:
101         """
102         Measure memory usage at different batch sizes
103         """
104
105         results = {}
106
107         for batch_size in batch_sizes:
108             try:
109                 # Prepare batch
110                 batch = sample_input.repeat(batch_size, 1, 1, 1).to(self.device)
111
112                 # Reset memory stats
113                 if self.device == 'cuda':
114                     torch.cuda.empty_cache()
115                     torch.cuda.reset_peak_memory_stats()
116
117                 # Run inference
118                 with torch.no_grad():
119                     _ = self.model(batch)
120
121                 # Measure memory
122                 if self.device == 'cuda':
123                     mem_allocated = torch.cuda.max_memory_allocated() / 1024**3
124                     mem_reserved = torch.cuda.max_memory_reserved() / 1024**3
125                 else:
126                     import psutil
127                     process = psutil.Process()
128                     mem_allocated = process.memory_info().rss / 1024**3
129                     mem_reserved = mem_allocated
130
131                 results[batch_size] = {
132                     'memory_allocated_gb': mem_allocated,
133                     'memory_reserved_gb': mem_reserved,
134                     'memory_per_sample_mb': (mem_allocated / batch_size) * 1024
135                 }
136
137                 self.logger.info(
138                     f"Batch {batch_size}: {mem_allocated:.2f}GB allocated"
139                 )
140
141         except RuntimeError as e:
142             if "out of memory" in str(e):
143                 self.logger.warning(f"OOM at batch size {batch_size}")
144                 if self.device == 'cuda':

```

```

145         torch.cuda.empty_cache()
146         continue
147     else:
148         raise e
149
150     return results
151
152 def optimize_memory_layout(self):
153     """
154     Optimize model memory layout for inference
155
156     Converts weights to contiguous memory, fuses operations
157     """
158
159     self.logger.info("Optimizing memory layout...")
160
161     # Make all tensors contiguous
162     for param in self.model.parameters():
163         param.data = param.data.contiguous()
164
165     # Fuse batch norm
166     self.model = self._fuse_bn(self.model)
167
168     self.logger.info("Memory layout optimized")
169
170 def _fuse_bn(self, model: nn.Module) -> nn.Module:
171     """
172     Fuse BatchNorm into Conv layers for efficiency
173
174     Reduces memory and computation
175     """
176
177     # This is a simplified version
178     # In production, use torch.quantization.fuse_modules
179
180     for name, module in model.named_children():
181         if isinstance(module, nn.Sequential):
182             # Check for Conv-BN pattern
183             fused = []
184             i = 0
185             while i < len(module):
186                 if (i + 1 < len(module) and
187                     isinstance(module[i], nn.Conv2d) and
188                     isinstance(module[i + 1], nn.BatchNorm2d)):
189
190                     # Fuse conv and bn
191                     fused.append(self._fuse_conv_bn(module[i], module[i + 1]))
192                     i += 2
193                 else:
194                     fused.append(module[i])
195                     i += 1
196
197             # Replace module
198             setattr(model, name, nn.Sequential(*fused))
199
200         else:
201             # Recursively fuse child modules
202             self._fuse_bn(module)

```

```

203     return model
204
205
206     def _fuse_conv_bn(self, conv: nn.Conv2d, bn: nn.BatchNorm2d) -> nn.Conv2d:
207         """Fuse Conv2d and BatchNorm2d"""
208
209         # Get parameters
210         w_conv = conv.weight.clone()
211         b_conv = conv.bias if conv.bias is not None else torch.zeros(conv.out_channels
212     )
213
214         w_bn = bn.weight
215         b_bn = bn.bias
216         mean = bn.running_mean
217         var = bn.running_var
218         eps = bn.eps
219
220         # Compute fused parameters
221         std = torch.sqrt(var + eps)
222         w_fused = (w_bn / std).reshape(-1, 1, 1, 1) * w_conv
223         b_fused = (b_conv - mean) / std * w_bn + b_bn
224
225         # Create fused conv
226         fused_conv = nn.Conv2d(
227             conv.in_channels,
228             conv.out_channels,
229             conv.kernel_size,
230             conv.stride,
231             conv.padding,
232             conv.dilation,
233             conv.groups,
234             bias=True
235         )
236
237         fused_conv.weight.data = w_fused
238         fused_conv.bias.data = b_fused
239
240     return fused_conv
241
242 # Example usage
243 model = ResNetLike()
244 memory_optimizer = MemoryEfficientInference(model, device='cuda' if torch.cuda.
245     is_available() else 'cpu')
246
247 # Optimize memory layout
248 memory_optimizer.optimize_memory_layout()
249
250 # Measure memory usage
251 sample = torch.randn(1, 3, 224, 224)
252 memory_results = memory_optimizer.measure_memory_usage(
253     sample,
254     batch_sizes=[1, 4, 8, 16]
255 )
256
257 print("\nMemory Usage:")
258 for bs, metrics in memory_results.items():
259     print(f"Batch {bs}: {metrics['memory_allocated_gb']:.2f}GB "
260         f"({metrics['memory_per_sample_mb']:.1f}MB per sample)")

```

```

259
260 # Chunked inference for large batch
261 large_batch = torch.randn(1000, 3, 224, 224)
262 output = memory_optimizer.chunked_inference(large_batch, chunk_size=32)
263 print(f"\nProcessed {output.size(0)} samples in chunks")

```

Listing 16.5: Memory-Optimized Inference System

16.6 Inference Optimization Patterns

16.6.1 Model Serving Architecture

Production inference requires efficient serving architecture with request batching, caching, and load balancing.

```

1  import torch
2  import torch.nn as nn
3  from typing import List, Dict, Optional, Tuple
4  from dataclasses import dataclass, field
5  from datetime import datetime
6  import asyncio
7  from queue import Queue, Empty
8  import threading
9  import time
10 import logging
11
12 @dataclass
13 class InferenceRequest:
14     """Single inference request"""
15     request_id: str
16     data: torch.Tensor
17     timestamp: datetime = field(default_factory=datetime.now)
18     priority: int = 0
19
20 @dataclass
21 class InferenceResponse:
22     """Inference response"""
23     request_id: str
24     result: torch.Tensor
25     latency_ms: float
26     timestamp: datetime = field(default_factory=datetime.now)
27
28 class BatchedInferenceServer:
29     """
30     High-performance inference server with dynamic batching
31
32     Collects requests and batches them for efficient processing
33     """
34
35     def __init__(self,
36                 model: nn.Module,
37                 max_batch_size: int = 32,
38                 max_wait_time_ms: float = 10.0,
39                 device: str = 'cpu'):
40         self.model = model
41         self.max_batch_size = max_batch_size

```

```

42     self.max_wait_time_ms = max_wait_time_ms
43     self.device = device
44
45     self.model.to(device)
46     self.model.eval()
47
48     # Request queue
49     self.request_queue = Queue()
50     self.response_dict: Dict[str, InferenceResponse] = {}
51     self.response_locks: Dict[str, threading.Event] = {}
52
53     # Server state
54     self.running = False
55     self.processing_thread: Optional[threading.Thread] = None
56
57     # Metrics
58     self.total_requests = 0
59     self.total_batches = 0
60     self.batch_size_history: List[int] = []
61
62     self.logger = logging.getLogger(__name__)
63
64     def start(self):
65         """Start inference server"""
66
67         if self.running:
68             self.logger.warning("Server already running")
69             return
70
71         self.running = True
72         self.processing_thread = threading.Thread(
73             target=self._processing_loop,
74             daemon=True
75         )
76         self.processing_thread.start()
77
78         self.logger.info("Inference server started")
79
80     def stop(self):
81         """Stop inference server"""
82
83         self.running = False
84
85         if self.processing_thread:
86             self.processing_thread.join(timeout=5)
87
88         self.logger.info("Inference server stopped")
89
90     def predict(self, request_id: str, data: torch.Tensor) -> InferenceResponse:
91         """
92         Submit inference request and wait for response
93
94         Requests are batched automatically for efficiency
95         """
96
97         # Create request
98         request = InferenceRequest(
99             request_id=request_id,

```

```

100         data=data
101     )
102
103     # Create response lock
104     self.response_locks[request_id] = threading.Event()
105
106     # Queue request
107     self.request_queue.put(request)
108     self.total_requests += 1
109
110     # Wait for response
111     self.response_locks[request_id].wait()
112
113     # Get response
114     response = self.response_dict.pop(request_id)
115     del self.response_locks[request_id]
116
117     return response
118
119 def _processing_loop(self):
120     """Main processing loop that batches requests"""
121
122     self.logger.info("Processing loop started")
123
124     while self.running:
125         try:
126             # Collect batch
127             batch = self._collect_batch()
128
129             if not batch:
130                 time.sleep(0.001) # Short sleep if no requests
131                 continue
132
133             # Process batch
134             self._process_batch(batch)
135
136         except Exception as e:
137             self.logger.error(f"Error in processing loop: {e}")
138
139 def _collect_batch(self) -> List[InferenceRequest]:
140     """
141     Collect requests into a batch
142
143     Waits up to max_wait_time_ms to accumulate max_batch_size requests
144     """
145
146     batch = []
147     deadline = time.time() + (self.max_wait_time_ms / 1000.0)
148
149     while len(batch) < self.max_batch_size:
150         try:
151             # Calculate remaining time
152             timeout = max(0, deadline - time.time())
153
154             if timeout <= 0 and batch:
155                 # Deadline reached, process what we have
156                 break

```

```

158         # Get request from queue
159         request = self.request_queue.get(timeout=timeout)
160         batch.append(request)
161
162     except Empty:
163         # No more requests before deadline
164         if batch:
165             break
166
167     return batch
168
169 def _process_batch(self, batch: List[InferenceRequest]):
170     """Process a batch of requests"""
171
172     start_time = time.perf_counter()
173
174     # Stack inputs
175     inputs = torch.stack([req.data for req in batch]).to(self.device)
176
177     # Run inference
178     with torch.no_grad():
179         outputs = self.model(inputs)
180
181     end_time = time.perf_counter()
182     batch_latency = (end_time - start_time) * 1000
183
184     # Distribute results
185     for i, request in enumerate(batch):
186         response = InferenceResponse(
187             request_id=request.request_id,
188             result=outputs[i].cpu(),
189             latency_ms=batch_latency / len(batch)
190         )
191
192         self.response_dict[request.request_id] = response
193         self.response_locks[request.request_id].set()
194
195     # Update metrics
196     self.total_batches += 1
197     self.batch_size_history.append(len(batch))
198
199     self.logger.debug(
200         f"Processed batch of {len(batch)} requests in {batch_latency:.2f}ms"
201     )
202
203 def get_metrics(self) -> Dict:
204     """Get server metrics"""
205
206     avg_batch_size = (np.mean(self.batch_size_history)
207                       if self.batch_size_history else 0)
208
209     return {
210         'total_requests': self.total_requests,
211         'total_batches': self.total_batches,
212         'avg_batch_size': avg_batch_size,
213         'max_batch_size': self.max_batch_size,
214         'queue_size': self.request_queue.qsize()
215     }

```



```

216
217 class InferenceCache:
218     """
219     Cache inference results for repeated inputs
220
221     Useful when same inputs are frequently requested
222     """
223
224     def __init__(self, max_size: int = 1000):
225         self.max_size = max_size
226         self.cache: Dict[str, Tuple[torch.Tensor, datetime]] = {}
227         self.hits = 0
228         self.misses = 0
229
230     def get(self, key: str) -> Optional[torch.Tensor]:
231         """Get cached result"""
232
233         if key in self.cache:
234             result, _ = self.cache[key]
235             self.hits += 1
236             return result
237         else:
238             self.misses += 1
239             return None
240
241     def put(self, key: str, value: torch.Tensor):
242         """Cache result"""
243
244         # Evict oldest if at capacity
245         if len(self.cache) >= self.max_size:
246             oldest_key = min(
247                 self.cache.keys(),
248                 key=lambda k: self.cache[k][1]
249             )
250             del self.cache[oldest_key]
251
252         self.cache[key] = (value, datetime.now())
253
254     def get_stats(self) -> Dict:
255         """Get cache statistics"""
256
257         total = self.hits + self.misses
258         hit_rate = self.hits / total if total > 0 else 0
259
260         return {
261             'size': len(self.cache),
262             'hits': self.hits,
263             'misses': self.misses,
264             'hit_rate': hit_rate
265         }
266
267 # Example: Run inference server
268 model = SimpleModel()
269 server = BatchedInferenceServer(
270     model,
271     max_batch_size=32,
272     max_wait_time_ms=10.0,
273     device='cpu'

```

```

274 )
275
276 server.start()
277
278 # Simulate requests
279 import uuid
280
281 responses = []
282 for i in range(100):
283     request_id = str(uuid.uuid4())
284     data = torch.randn(1, 28, 28)
285
286     response = server.predict(request_id, data)
287     responses.append(response)
288
289 server.stop()
290
291 # Print metrics
292 metrics = server.get_metrics()
293 print("\nServer Metrics:")
294 print(f"Total Requests: {metrics['total_requests']}")
295 print(f"Total Batches: {metrics['total_batches']}")
296 print(f"Avg Batch Size: {metrics['avg_batch_size']:.1f}")
297 print(f"Avg Latency: {np.mean([r.latency_ms for r in responses]):.2f}ms")

```

Listing 16.6: Optimized Inference Server

16.7 Edge Deployment Optimization

16.7.1 Mobile and Edge Optimization

Deploying to edge devices requires aggressive optimization for memory, compute, and power constraints.

```

1 import torch
2 import torch.nn as nn
3 from typing import Dict, Optional
4 import logging
5
6 class EdgeOptimizer:
7     """
8     Optimize models for edge/mobile deployment
9
10    Combines multiple techniques for maximum efficiency
11    """
12
13    def __init__(self, model: nn.Module):
14        self.model = model
15        self.logger = logging.getLogger(__name__)
16
17    def optimize_for_mobile(self,
18                            sample_input: torch.Tensor,
19                            quantize: bool = True) -> torch.jit.ScriptModule:
20        """
21        Complete optimization for mobile deployment
22

```

```

23     Returns TorchScript model ready for mobile
24     """
25
26     self.logger.info("Optimizing for mobile deployment...")
27
28     # Step 1: Set to eval mode
29     self.model.eval()
30
31     # Step 2: Trace model
32     traced_model = torch.jit.trace(self.model, sample_input)
33
34     # Step 3: Optimize for mobile
35     from torch.utils.mobile_optimizer import optimize_for_mobile
36     optimized_model = optimize_for_mobile(traced_model)
37
38     # Step 4: Quantize if requested
39     if quantize:
40         self.logger.info("Applying quantization...")
41         optimized_model = torch.quantization.quantize_dynamic(
42             optimized_model,
43             {torch.nn.Linear},
44             dtype=torch.qint8
45         )
46
47     self.logger.info("Mobile optimization complete")
48
49     return optimized_model
50
51 def export_to_onnx(self,
52                   sample_input: torch.Tensor,
53                   output_path: str,
54                   opset_version: int = 11):
55     """
56     Export model to ONNX format
57
58     ONNX models can run on many edge runtimes
59     """
60
61     self.logger.info(f"Exporting to ONNX: {output_path}")
62
63     self.model.eval()
64
65     torch.onnx.export(
66         self.model,
67         sample_input,
68         output_path,
69         export_params=True,
70         opset_version=opset_version,
71         do_constant_folding=True,
72         input_names=['input'],
73         output_names=['output'],
74         dynamic_axes={
75             'input': {0: 'batch_size'},
76             'output': {0: 'batch_size'}
77         }
78     )
79
80     self.logger.info("ONNX export complete")

```

```

81
82 def profile_for_edge(self,
83                       sample_input: torch.Tensor) -> Dict:
84     """
85     Profile model for edge deployment constraints
86
87     Checks model size, ops, memory usage
88     """
89
90     self.model.eval()
91
92     metrics = {}
93
94     # Model size
95     import tempfile
96     import os
97
98     with tempfile.NamedTemporaryFile(delete=False) as tmp:
99         torch.save(self.model.state_dict(), tmp.name)
100         metrics['model_size_mb'] = os.path.getsize(tmp.name) / (1024 * 1024)
101         os.unlink(tmp.name)
102
103     # Parameter count
104     total_params = sum(p.numel() for p in self.model.parameters())
105     metrics['total_params'] = total_params
106     metrics['total_params_millions'] = total_params / 1e6
107
108     # FLOPs estimation (simplified)
109     from torch.profiler import profile, ProfilerActivity
110
111     with profile(activities=[ProfilerActivity.CPU]) as prof:
112         with torch.no_grad():
113             _ = self.model(sample_input)
114
115     # Parse profiler output for ops
116     events = prof.key_averages()
117     cpu_time = sum([event.cpu_time_total for event in events])
118     metrics['cpu_time_ms'] = cpu_time / 1000
119
120     # Memory usage
121     import psutil
122     import gc
123
124     gc.collect()
125     process = psutil.Process()
126     mem_before = process.memory_info().rss / 1024 ** 2
127
128     with torch.no_grad():
129         output = self.model(sample_input)
130
131     mem_after = process.memory_info().rss / 1024 ** 2
132     metrics['memory_usage_mb'] = mem_after - mem_before
133
134     # Check if suitable for edge
135     edge_constraints = {
136         'model_size_mb': 10.0, # < 10MB
137         'total_params_millions': 1.0, # < 1M params
138         'cpu_time_ms': 100.0, # < 100ms

```

```

139         'memory_usage_mb': 50.0 # < 50MB
140     }
141
142     metrics['edge_suitable'] = all(
143         metrics[k] <= v for k, v in edge_constraints.items()
144         if k in metrics
145     )
146
147     return metrics
148
149     def create_edge_variants(self, sample_input: torch.Tensor) -> Dict[str, nn.Module
150     ]:
151         """
152         Create multiple optimized variants for different edge devices
153
154         Returns dict of models optimized for different constraints
155         """
156
157         variants = {}
158
159         # 1. Mobile variant (quantized)
160         mobile_model = self.optimize_for_mobile(sample_input, quantize=True)
161         variants['mobile'] = mobile_model
162
163         # 2. Raspberry Pi variant (pruned + quantized)
164         rpi_model = self.model
165         # Apply aggressive pruning here
166         variants['raspberrypi'] = rpi_model
167
168         # 3. Microcontroller variant (heavily compressed)
169         # Extremely small model for MCUs
170         mcu_model = self.model
171         variants['microcontroller'] = mcu_model
172
173         return variants
174
175     # Example: Optimize for edge
176     model = SimpleModel()
177     edge_optimizer = EdgeOptimizer(model)
178
179     # Profile for edge
180     sample = torch.randn(1, 1, 28, 28)
181     profile = edge_optimizer.profile_for_edge(sample)
182
183     print("\nEdge Profile:")
184     print(f"Model Size: {profile['model_size_mb']:.2f}MB")
185     print(f"Parameters: {profile['total_params_millions']:.2f}M")
186     print(f"CPU Time: {profile['cpu_time_ms']:.2f}ms")
187     print(f"Memory: {profile['memory_usage_mb']:.2f}MB")
188     print(f"Edge Suitable: {profile['edge_suitable']}")
189
190     # Export to mobile
191     mobile_model = edge_optimizer.optimize_for_mobile(sample)
192     mobile_model._save_for_lite_interpreter("model_mobile.ptl")
193     print("\nMobile model saved to model_mobile.ptl")
194
195     # Export to ONNX
196     edge_optimizer.export_to_onnx(sample, "model.onnx")

```

```
196 print("ONNX model saved to model.onnx")
```

Listing 16.7: Edge Deployment Optimizer

16.8 Benchmarking and Profiling Tools

16.8.1 Comprehensive Performance Profiling

Systematic profiling identifies bottlenecks and guides optimization efforts.

```

1 import torch
2 import torch.nn as nn
3 from typing import Dict, List, Optional, Callable
4 import time
5 import numpy as np
6 from dataclasses import dataclass, field
7 import json
8 from pathlib import Path
9 import logging
10
11 @dataclass
12 class LayerProfile:
13     """Profile for a single layer"""
14     name: str
15     layer_type: str
16     params: int
17     forward_time_ms: float
18     memory_mb: float
19     percentage_time: float
20     percentage_memory: float
21
22 @dataclass
23 class ModelProfile:
24     """Complete model profile"""
25     total_params: int
26     total_time_ms: float
27     total_memory_mb: float
28     layer_profiles: List[LayerProfile] = field(default_factory=list)
29     bottlenecks: List[str] = field(default_factory=list)
30
31 class PerformanceProfiler:
32     """
33     Comprehensive performance profiling toolkit
34
35     Profiles latency, memory, throughput, and identifies bottlenecks
36     """
37
38     def __init__(self, model: nn.Module, device: str = 'cpu'):
39         self.model = model
40         self.device = device
41         self.model.to(device)
42         self.logger = logging.getLogger(__name__)
43
44     def profile_layers(self, sample_input: torch.Tensor) -> ModelProfile:
45         """
46         Profile each layer individually

```

```

47
48     Measures time and memory for each layer
49     """
50
51     self.logger.info("Profiling model layers...")
52
53     layer_profiles = []
54     total_time = 0.0
55     total_memory = 0.0
56
57     # Hook to measure layer performance
58     def create_hook(name):
59         def hook(module, input, output):
60             # Measure time
61             start = time.perf_counter()
62             with torch.no_grad():
63                 _ = module(input[0])
64             elapsed = (time.perf_counter() - start) * 1000
65
66             # Measure memory
67             if self.device == 'cuda':
68                 torch.cuda.reset_peak_memory_stats()
69                 _ = module(input[0])
70                 mem = torch.cuda.max_memory_allocated() / 1024**2
71             else:
72                 mem = 0.0
73
74             # Count parameters
75             params = sum(p.numel() for p in module.parameters())
76
77             profile = LayerProfile(
78                 name=name,
79                 layer_type=module.__class__.__name__,
80                 params=params,
81                 forward_time_ms=elapsed,
82                 memory_mb=mem,
83                 percentage_time=0.0, # Will calculate later
84                 percentage_memory=0.0
85             )
86
87             layer_profiles.append(profile)
88
89         return hook
90
91     # Register hooks
92     hooks = []
93     for name, module in self.model.named_modules():
94         if len(list(module.children())) == 0: # Leaf module
95             hook = module.register_forward_hook(create_hook(name))
96             hooks.append(hook)
97
98     # Run forward pass
99     with torch.no_grad():
100         _ = self.model(sample_input.to(self.device))
101
102     # Remove hooks
103     for hook in hooks:
104         hook.remove()

```

```

105     # Calculate percentages
106     total_time = sum(p.forward_time_ms for p in layer_profiles)
107     total_memory = sum(p.memory_mb for p in layer_profiles)
108
109     for profile in layer_profiles:
110         if total_time > 0:
111             profile.percentage_time = (profile.forward_time_ms / total_time) * 100
112         if total_memory > 0:
113             profile.percentage_memory = (profile.memory_mb / total_memory) * 100
114
115     # Identify bottlenecks (layers taking > 10% time)
116     bottlenecks = [
117         p.name for p in layer_profiles
118         if p.percentage_time > 10.0
119     ]
120
121     total_params = sum(p.numel() for p in self.model.parameters())
122
123     model_profile = ModelProfile(
124         total_params=total_params,
125         total_time_ms=total_time,
126         total_memory_mb=total_memory,
127         layer_profiles=layer_profiles,
128         bottlenecks=bottlenecks
129     )
130
131     self.logger.info(f"Profiling complete: {len(layer_profiles)} layers")
132     return model_profile
133
134 def profile_end_to_end(self,
135                        sample_input: torch.Tensor,
136                        num_runs: int = 100,
137                        warmup_runs: int = 10) -> Dict:
138     """
139     Profile end-to-end inference performance
140
141     Measures latency distribution, throughput
142     """
143
144     self.logger.info("Profiling end-to-end performance...")
145
146     self.model.eval()
147     sample_input = sample_input.to(self.device)
148
149     # Warmup
150     with torch.no_grad():
151         for _ in range(warmup_runs):
152             _ = self.model(sample_input)
153
154     if self.device == 'cuda':
155         torch.cuda.synchronize()
156
157     # Measure
158     latencies = []
159
160     with torch.no_grad():
161         for _ in range(num_runs):

```



```

163         start = time.perf_counter()
164         _ = self.model(sample_input)
165
166         if self.device == 'cuda':
167             torch.cuda.synchronize()
168
169         end = time.perf_counter()
170         latencies.append((end - start) * 1000) # Convert to ms
171
172     latencies = np.array(latencies)
173
174     results = {
175         'mean_latency_ms': np.mean(latencies),
176         'median_latency_ms': np.median(latencies),
177         'p95_latency_ms': np.percentile(latencies, 95),
178         'p99_latency_ms': np.percentile(latencies, 99),
179         'min_latency_ms': np.min(latencies),
180         'max_latency_ms': np.max(latencies),
181         'std_latency_ms': np.std(latencies),
182         'throughput_qps': 1000.0 / np.mean(latencies)
183     }
184
185     self.logger.info(f"Mean latency: {results['mean_latency_ms']:.2f}ms")
186     self.logger.info(f"P99 latency: {results['p99_latency_ms']:.2f}ms")
187     self.logger.info(f"Throughput: {results['throughput_qps']:.1f} QPS")
188
189     return results
190
191     def compare_optimizations(self,
192                             sample_input: torch.Tensor,
193                             optimizations: Dict[str, Callable]) -> Dict:
194         """
195         Compare different optimization techniques
196
197         Parameters:
198         - optimizations: Dict of {name: optimization_function}
199         """
200
201         results = {}
202
203         # Baseline (no optimization)
204         self.logger.info("Measuring baseline performance...")
205         results['baseline'] = self.profile_end_to_end(sample_input, num_runs=100)
206
207         # Test each optimization
208         for name, opt_func in optimizations.items():
209             self.logger.info(f"Testing optimization: {name}")
210
211             # Apply optimization
212             optimized_model = opt_func(self.model)
213
214             # Create temporary profiler
215             temp_profiler = PerformanceProfiler(optimized_model, self.device)
216
217             # Profile
218             opt_results = temp_profiler.profile_end_to_end(sample_input, num_runs=100)
219
220             # Calculate speedup

```

```

221         speedup = results['baseline']['mean_latency_ms'] / opt_results['
mean_latency_ms']
222         opt_results['speedup'] = speedup
223
224         results[name] = opt_results
225
226     return results
227
228     def export_profile(self, profile: ModelProfile, output_path: str):
229         """Export profile to JSON"""
230
231         data = {
232             'total_params': profile.total_params,
233             'total_time_ms': profile.total_time_ms,
234             'total_memory_mb': profile.total_memory_mb,
235             'bottlenecks': profile.bottlenecks,
236             'layers': [
237                 {
238                     'name': layer.name,
239                     'type': layer.layer_type,
240                     'params': layer.params,
241                     'time_ms': layer.forward_time_ms,
242                     'memory_mb': layer.memory_mb,
243                     'pct_time': layer.percentage_time,
244                     'pct_memory': layer.percentage_memory
245                 }
246                 for layer in profile.layer_profiles
247             ]
248         }
249
250         with open(output_path, 'w') as f:
251             json.dump(data, f, indent=2)
252
253         self.logger.info(f"Profile exported to {output_path}")
254
255     # Example: Profile a model
256     model = ResNetLike()
257     profiler = PerformanceProfiler(model, device='cpu')
258
259     # Profile layers
260     sample = torch.randn(1, 3, 224, 224)
261     layer_profile = profiler.profile_layers(sample)
262
263     print("\nTop 5 Slowest Layers:")
264     sorted_layers = sorted(
265         layer_profile.layer_profiles,
266         key=lambda x: x.forward_time_ms,
267         reverse=True
268     )[:5]
269
270     for i, layer in enumerate(sorted_layers, 1):
271         print(f"{i}. {layer.name} ({layer.layer_type}): "
272             f"{layer.forward_time_ms:.2f}ms ({layer.percentage_time:.1f}%)"
273         )
274
275     # Profile end-to-end
276     e2e_profile = profiler.profile_end_to_end(sample, num_runs=100)
277     print("\nEnd-to-End Performance:")

```

```

278 print(f"Mean: {e2e_profile['mean_latency_ms']:.2f}ms")
279 print(f"P95: {e2e_profile['p95_latency_ms']:.2f}ms")
280 print(f"P99: {e2e_profile['p99_latency_ms']:.2f}ms")
281 print(f"Throughput: {e2e_profile['throughput_qps']:.1f} QPS")
282
283 # Export profile
284 profiler.export_profile(layer_profile, "model_profile.json")

```

Listing 16.8: Performance Profiling Framework

16.9 Practical Optimization Challenges

16.9.1 Challenge 1: Compress a Large Model

Apply comprehensive compression to reduce model size while maintaining accuracy.

Objective: Reduce a ResNet-50 model to 1/4 its original size with <2% accuracy drop.

Requirements:

- Start with pretrained ResNet-50 (97MB, 25M parameters)
- Apply pruning: Remove 50% of weights (structured + unstructured)
- Apply quantization: Convert to INT8
- Apply knowledge distillation: Train smaller student network
- Target: <25MB model with >95% of original accuracy
- Measure: compression ratio, accuracy, inference latency

Constraints:

- Must maintain at least 98% of original accuracy on ImageNet validation
- Model must fit in <25MB on disk
- Inference latency must improve by at least 2x on CPU

Approach:

1. Baseline: Profile original ResNet-50 performance
2. Iterative Pruning: Gradually prune to 50% sparsity with retraining
3. Quantization: Apply post-training quantization or QAT
4. Distillation: Train MobileNetV2 student with ResNet-50 teacher
5. Combination: Apply multiple techniques together
6. Evaluation: Compare all approaches on accuracy/size/speed trade-offs

Success Criteria:

- Final model <25MB (target: 24MB = 1/4 of 97MB)
- Accuracy drop <2% (target: <1.5%)
- Inference speedup >2x on CPU (target: 3x)
- Successfully runs on mobile device with <100ms latency

Deliverables:

- Compressed models at different operating points
- Performance comparison table (size, accuracy, latency)
- Pareto frontier plot showing accuracy vs size trade-offs
- Mobile deployment demo on Android/iOS

16.9.2 Challenge 2: Optimize Inference Latency

Achieve sub-50ms P99 latency for production inference.

Objective: Optimize a BERT model to achieve <50ms P99 latency on CPU.

Baseline:

- Model: BERT-Base (110M parameters)
- Current P99 latency: 250ms on CPU
- Target: <50ms P99 (5x improvement)

Optimization Techniques to Apply:**1. Layer Profiling:** Identify slowest layers

- Profile each transformer layer
- Find attention vs FFN bottlenecks
- Measure operator-level performance

2. Operator Fusion: Fuse common patterns

- Fuse LayerNorm + Linear
- Fuse GELU activation
- Use TorchScript compilation

3. Quantization: Reduce precision

- Dynamic quantization for Linear layers
- INT8 quantization with calibration
- Mixed precision (FP16/INT8)

4. Distillation: Use smaller model

- Distill to DistilBERT (6 layers vs 12)

- Maintain 97% of accuracy
- 2x speedup from smaller architecture

5. **Batching:** Dynamic request batching

- Implement batch collection (max 32, max wait 10ms)
- Measure throughput vs latency trade-off
- Optimize batch size for P99 latency

Metrics to Track:

- P50, P95, P99, P999 latency
- Throughput (QPS)
- Model size
- Accuracy on SQuAD benchmark
- Resource utilization (CPU, memory)

Expected Results:

- Quantization alone: 2x speedup → 125ms P99
- Distillation alone: 2x speedup → 125ms P99
- Combined: 4-5x speedup → 50-60ms P99
- With batching: 40-45ms P99 at 200 QPS

Deliverables:

- Optimization report showing cumulative improvements
- Latency distribution graphs before/after each optimization
- Production-ready inference server meeting SLA
- Load test results at various QPS levels

16.9.3 Challenge 3: Optimize for Edge Deployment

Deploy an object detection model on Raspberry Pi 4.

Objective: Run YOLOv5 on Raspberry Pi 4 at >10 FPS.

Constraints:

- Hardware: Raspberry Pi 4 (4GB RAM, ARM Cortex-A72)
- Model size: <50MB
- Memory usage: <1GB
- Target FPS: >10 on 640x640 input

- Accuracy: mAP >0.4 on COCO

Optimization Strategy:**1. Model Selection and Architecture:**

- Start with YOLOv5-nano (1.9M parameters)
- Consider MobileNet-based variants
- Reduce input resolution: 640x640 $\rightarrow$ 416x416

2. Quantization for ARM:

- Export to ONNX
- Convert to TFLite with INT8 quantization
- Test with XNNPACK backend for ARM optimization
- Verify accuracy after quantization

3. Inference Optimization:

- Use multi-threading (4 cores)
- Implement frame skipping for video
- Add motion detection to reduce unnecessary inference
- Optimize pre/post-processing (resize, NMS)

4. Power Optimization:

- CPU governor tuning
- Thermal throttling management
- Batch processing for efficiency

Implementation Steps:

1. Baseline: Measure YOLOv5-nano FPS on RPi4
2. Convert to TFLite with quantization
3. Optimize inference pipeline
4. Add real-time video processing
5. Profile and iterate on bottlenecks

Evaluation Metrics:

- FPS on live camera feed
- Latency per frame
- mAP on COCO val2017

- Power consumption (watts)
- Temperature under load

Deliverables:

- Working object detection on RPi4 camera
- Performance report: FPS, accuracy, power
- Optimization guide for edge deployment
- Demo video showing real-time detection

16.9.4 Challenge 4: Memory-Efficient Serving

Serve large language model with limited GPU memory.

Objective: Serve GPT-2 Large (774M params) on single 8GB GPU at 100 QPS.

Problem:

- Model size: 3GB (FP32)
- With KV cache: 5-6GB per batch
- Available GPU memory: 8GB
- Need to serve concurrent requests efficiently

Memory Optimization Techniques:**1. Quantization:**

- INT8 quantization: 3GB $\rightarrow$ 750MB (4x reduction)
- Mixed precision: FP16 for activations, INT8 for weights
- Dynamic quantization for generation

2. KV Cache Management:

- Implement PagedAttention for cache allocation
- Reuse cache blocks across requests
- Evict LRU cache entries when memory full

3. Batching Strategy:

- Dynamic batching with variable sequence lengths
- Continuous batching (add/remove sequences mid-batch)
- Priority queue for request scheduling

4. Operator Fusion:

- Fuse attention operations

- Flash Attention implementation
- Kernel optimization for GEMM operations

Implementation Plan:

1. Baseline: Measure throughput with naive batching
2. Apply INT8 quantization
3. Implement efficient KV cache
4. Add dynamic batching
5. Profile GPU memory usage
6. Optimize for throughput

Target Metrics:

- Throughput: >100 QPS
- P99 latency: <500 ms
- GPU memory usage: <7.5 GB (leaves headroom)
- GPU utilization: $>80\%$

Success Criteria:

- Sustain 100 concurrent requests
- Generate 100 tokens in <500 ms P99
- No OOM errors under load
- Efficient memory usage (no fragmentation)

16.9.5 Challenge 5: Multi-Model Serving Optimization

Serve multiple models efficiently on shared infrastructure.

Objective: Serve 5 different models on 2 GPUs with optimal resource utilization.

Models to Serve:

1. ResNet-50: Image classification (90ms latency target)
2. BERT-Base: Text classification (50ms target)
3. YOLOv5: Object detection (100ms target)
4. GPT-2: Text generation (200ms target)
5. Whisper-Small: Speech recognition (150ms target)

Challenges:

- Different model sizes and latency requirements

- Variable request rates (some models more popular)
- GPU memory constraints (12GB per GPU)
- Need to maintain SLAs for all models

Optimization Strategies:**1. Model Placement:**

- Profile memory usage of each model
- Pack models efficiently across GPUs
- Consider model replication for high-traffic models

2. Request Routing:

- Implement intelligent load balancer
- Route based on GPU availability
- Priority scheduling for latency-sensitive models

3. Batching Per Model:

- Different batch sizes per model type
- Adaptive batching based on load
- Separate queues with backpressure

4. Resource Sharing:

- CUDA MPS (Multi-Process Service) for concurrency
- Dynamic GPU memory allocation
- CPU offloading for pre/post-processing

Implementation:

1. Measure individual model requirements
2. Design placement strategy
3. Implement multi-model server
4. Add routing and batching logic
5. Load test with realistic traffic
6. Optimize based on bottlenecks

Metrics:

- Per-model P99 latency
- Overall throughput (requests/sec)

- GPU utilization per device
- GPU memory usage per device
- Queue wait times

Target Performance:

- All models meet latency SLAs at 95th percentile
- GPU utilization >70% on both devices
- Handle 500+ requests/sec combined
- No model starvation under mixed load

16.9.6 Challenge 6: Optimize Training Performance

Accelerate model training with mixed precision and distributed techniques.

Objective: Reduce training time for ResNet-50 on ImageNet from 24 hours to <6 hours.

Baseline:

- Model: ResNet-50
- Dataset: ImageNet (1.28M training images)
- Hardware: 4x NVIDIA V100 GPUs (32GB each)
- Current time: 24 hours to 90 epochs
- Target: <6 hours (4x speedup)

Optimization Techniques:**1. Mixed Precision Training:**

- Use AMP (Automatic Mixed Precision)
- FP16 for forward/backward pass
- FP32 for optimizer state
- Expected: 2-3x speedup

2. Data Loading Optimization:

- Increase num_workers for DataLoader
- Use prefetching (pin_memory=True)
- Implement DALI for GPU preprocessing
- Profile and eliminate I/O bottlenecks

3. Distributed Training:

- Use DistributedDataParallel (DDP)

- Optimize all-reduce operations
- Gradient accumulation for larger effective batch
- Overlap communication with computation

4. Learning Rate Scaling:

- Linear scaling rule for large batch sizes
- Warmup schedule for stability
- Adjust batch size for optimal throughput

5. Memory Optimization:

- Gradient checkpointing for larger batches
- Zero optimizer redundancy (ZeRO-2)
- Find maximum stable batch size per GPU

Implementation Steps:

1. Profile baseline training (identify bottlenecks)
2. Enable mixed precision training
3. Optimize data loading pipeline
4. Implement distributed training across 4 GPUs
5. Tune batch size and learning rate
6. Validate accuracy at higher throughput

Expected Results:

- Mixed precision: 2.5x speedup → 9.6 hours
- Distributed (4 GPUs): 3.8x speedup → 6.3 hours
- Combined: 4-5x speedup → 5-6 hours
- Final accuracy: $\geq 76\%$ Top-1 on ImageNet

Metrics to Track:

- Images/sec throughput
- GPU utilization (%)
- Training time per epoch
- Final validation accuracy
- Memory usage per GPU

16.9.7 Challenge 7: End-to-End Production Optimization

Optimize complete ML pipeline from preprocessing to serving.

Objective: Build optimized end-to-end system for real-time recommendation service.

System Components:

1. Feature extraction: Process user/item features (target: <10ms)
2. Model inference: Two-tower model (target: <30ms)
3. Post-processing: Ranking and filtering (target: <10ms)
4. Total latency budget: <50ms end-to-end

Requirements:

- Handle 10,000 QPS
- P99 latency <50ms
- 4 CPU cores + 1 GPU for inference
- Fault tolerance and graceful degradation

Optimization Areas:

1. Feature Processing:

- Cache frequently accessed features (Redis)
- Batch feature lookups
- Precompute item embeddings
- Use efficient serialization (Protocol Buffers)

2. Model Optimization:

- Quantize two-tower model
- Use ONNX Runtime for inference
- Enable dynamic batching
- GPU memory optimization

3. Serving Infrastructure:

- Implement request batching
- Use connection pooling
- Add circuit breakers
- Horizontal scaling with load balancing

4. Caching Strategy:

- Cache popular recommendations

- Implement cache warming
- TTL-based invalidation
- Measure cache hit rate

Implementation:

1. Profile baseline end-to-end latency
2. Optimize each component independently
3. Implement batching and caching
4. Load test at target QPS
5. Identify and fix bottlenecks
6. Deploy with monitoring

Success Metrics:

- P99 latency: <50ms
- Throughput: >10,000 QPS
- CPU utilization: 60-70%
- GPU utilization: >80%
- Cache hit rate: >60%
- Error rate: <0.1%

Deliverables:

- Production-ready recommendation service
- Complete performance benchmarks
- Monitoring dashboard
- Optimization playbook documenting techniques
- Load test results and capacity planning

16.10 Summary

Performance optimization is essential for deploying machine learning models in production. This chapter covered comprehensive techniques across the entire optimization stack.

Key Takeaways:**1. Model Compression:**

- Quantization reduces model size 4x with minimal accuracy loss
- Pruning removes 50-90% of weights while maintaining performance

- Knowledge distillation transfers accuracy to smaller models
- Combine techniques for maximum compression

2. Hardware Acceleration:

- GPUs provide 10-100x speedup for parallel operations
- Mixed precision (FP16) doubles throughput on modern GPUs
- Batch processing amortizes overhead and maximizes throughput
- Profile to identify GPU utilization bottlenecks

3. Memory Optimization:

- Memory is often the primary constraint, not compute
- Chunked inference prevents OOM errors on large batches
- Operator fusion reduces memory footprint
- Efficient KV cache management for generative models

4. Inference Patterns:

- Dynamic batching improves throughput 3-5x
- Request caching eliminates redundant computation
- Asynchronous processing hides latency
- Load balancing distributes work efficiently

5. Edge Deployment:

- Edge devices require aggressive compression (quantization + pruning)
- ONNX and TFLite enable cross-platform deployment
- Mobile optimization includes operator fusion and memory layout
- Profile on target hardware early and often

6. Profiling and Benchmarking:

- Always measure before optimizing
- Profile at multiple levels: layer, operator, end-to-end
- Track latency distributions (P50, P95, P99), not just averages
- Identify bottlenecks systematically

Optimization Workflow:

1. **Profile:** Measure baseline performance and identify bottlenecks

2. **Optimize:** Apply appropriate techniques based on constraints
3. **Validate:** Ensure accuracy is maintained
4. **Benchmark:** Measure improvements quantitatively
5. **Iterate:** Continue optimizing next bottleneck

Trade-offs to Consider:

- **Accuracy vs Speed:** Compression reduces accuracy; find acceptable balance
- **Latency vs Throughput:** Batching increases throughput but adds latency
- **Memory vs Compute:** Some optimizations trade memory for speed
- **Development Time vs Performance:** Simple techniques often give 80% of gains

Best Practices:

1. Start with high-level optimizations (model architecture, batching) before low-level tuning
2. Use existing optimized libraries (ONNXRuntime, TensorRT) instead of reinventing
3. Optimize for your specific deployment target (cloud GPU vs mobile CPU)
4. Maintain accuracy validation throughout optimization process
5. Document optimization decisions and trade-offs for future reference
6. Monitor production performance continuously to catch regressions

Common Pitfalls to Avoid:

- Premature optimization without profiling
- Optimizing the wrong bottleneck
- Sacrificing too much accuracy for speed
- Not testing on target hardware
- Ignoring production constraints (memory, power, cost)
- Over-engineering complex solutions for simple problems

Production Deployment Checklist:

- ✓ Profile model on target hardware
- ✓ Validate accuracy after each optimization
- ✓ Benchmark latency distribution (P50/P95/P99)
- ✓ Measure throughput under load

- ✓ Test memory usage at peak traffic
- ✓ Verify model size meets constraints
- ✓ Implement monitoring and alerting
- ✓ Document optimization decisions
- ✓ Plan for future scaling needs

Performance optimization is an iterative process. Start with the techniques that provide the most impact for your use case, measure carefully, and continue refining based on production feedback. The combination of model compression, hardware acceleration, and serving optimizations can typically achieve 5-10x improvements in latency and throughput, enabling ML models to meet stringent production requirements.

Remember: The best optimization is the one that meets your requirements with the simplest implementation. Don't over-optimize unless your use case demands it.

16.11 Real-World Optimization Case Studies

16.11.1 Case Study 1: BERT Model Optimization for Search

Challenge: A search engine needed to deploy BERT for query understanding at 10,000 QPS with <20ms P99 latency.

Initial State:

- Model: BERT-Base (110M parameters)
- Infrastructure: 8x V100 GPUs
- Latency: 180ms P99
- Throughput: 400 QPS
- Cost: \$15,000/month

Optimization Journey:

Step 1: Dynamic Quantization (Week 1)

- Applied INT8 quantization to Linear layers
- Result: 2.1x speedup, 85ms P99
- Accuracy drop: 0.3% (acceptable)
- No infrastructure changes needed

Step 2: Knowledge Distillation (Weeks 2-3)

- Distilled to 6-layer DistilBERT student
- Training took 3 days on 4 GPUs
- Result: Additional 1.9x speedup, 45ms P99

- Accuracy drop: 1.2% total (still acceptable)

Step 3: Batching Optimization (Week 4)

- Implemented dynamic batching (max batch=64, max wait=5ms)
- Result: 5x throughput improvement, 2,000 QPS per GPU
- P99 latency: 52ms (includes batching wait time)

Step 4: TensorRT Compilation (Week 5)

- Exported to ONNX and compiled with TensorRT
- Operator fusion and kernel optimization
- Result: Additional 1.4x speedup, 37ms P99
- Throughput: 2,800 QPS per GPU

Final Results:

- P99 Latency: 18ms (9x improvement)
- Throughput: 11,200 QPS on 4 GPUs (28x improvement)
- Infrastructure reduced from 8 to 4 GPUs
- Monthly cost: \$7,500 (50% reduction)
- Accuracy: 98.8% of original (1.2% drop)

Key Learnings:

1. Start with easy wins: Quantization gave 2x with no training
2. Distillation requires time but provides architectural benefits
3. Batching has largest impact on throughput
4. Framework-specific optimizations (TensorRT) provide final polish
5. Each optimization compounded on previous improvements

16.11.2 Case Study 2: Image Classification on Edge Devices

Challenge: Deploy product recognition model on smart cameras with <50ms latency and <10MB size.

Initial State:

- Model: ResNet-50
- Size: 98MB
- Latency on ARM CPU: 850ms
- Target: <50ms, <10MB

Optimization Approach:**Architecture Selection:**

- Replaced ResNet-50 with MobileNetV3-Small
- Size: 10MB $\rightarrow$ 2.5MB
- Latency: 850ms $\rightarrow$ 95ms
- Accuracy: 76% $\rightarrow$ 71% (5% drop)

Quantization:

- Applied INT8 quantization with QAT
- Size: 2.5MB $\rightarrow$ 650KB (16x total reduction)
- Latency: 95ms $\rightarrow$ 42ms
- Accuracy recovered to 73% with QAT

Platform Optimization:

- Compiled to TFLite with XNNPACK delegate
- Enabled NEON SIMD instructions
- Optimized preprocessing (resize, normalize)
- Final latency: 35ms on ARM Cortex-A53

Results:

- Model size: 650KB (150x reduction)
- Latency: 35ms (24x improvement)
- Accuracy: 73% (3% drop from original)
- Power: 1.2W vs 3.5W for original
- Deployable on \$30 edge devices

16.11.3 Case Study 3: Large Language Model Serving

Challenge: Serve GPT-3 scale model (175B parameters) for production chatbot at reasonable cost.

Constraints:

- Budget: 8x A100 GPUs (80GB each)
- Target: 50 concurrent users
- Latency: <2 s for 100 tokens
- Quality: Maintain generation quality

Optimization Stack:**1. Model Parallelism:**

- Tensor parallelism across 8 GPUs
- Pipeline parallelism for 96 layers
- Each GPU handles 22B parameters

2. Quantization:

- INT8 quantization for KV cache
- FP16 for computation
- Memory per GPU: 75GB $\rightarrow$ 45GB

3. KV Cache Optimization:

- PagedAttention for efficient cache management
- Cache reuse across requests
- Continuous batching for higher throughput

4. Speculative Decoding:

- Small draft model predicts multiple tokens
- Large model verifies in parallel
- 2-3x speedup on average

Results:

- Throughput: 75 concurrent users (50% above target)
- Latency: 1.4s for 100 tokens
- GPU memory utilization: 60GB/80GB per GPU
- Monthly cost: \$28,000 (vs \$100,000+ naive approach)

Critical Success Factors:

1. Proper model parallelism essential for large models
2. KV cache management is the key bottleneck
3. Continuous batching maximizes GPU utilization
4. Speculative decoding provides significant user-perceived speedup

16.11.4 Lessons Learned Across All Cases

Universal Principles:

1. **Measure First:** All successful optimizations started with comprehensive profiling
2. **Low-Hanging Fruit:** Quantization and batching provided 70% of gains with 20% of effort
3. **Compound Effects:** Multiple optimizations multiply, not just add
4. **Target-Specific:** Edge, cloud, and distributed cases required different approaches
5. **Accuracy Trade-offs:** 1-5% accuracy loss is usually acceptable for 10x speedup

Common Optimization Stacks by Use Case:

Cloud GPU Serving:

1. Dynamic batching (3-5x throughput)
2. Mixed precision FP16 (1.8x speedup)
3. TensorRT compilation (1.5x speedup)
4. KV cache optimization for generative models

Edge/Mobile Deployment:

1. Architecture selection (MobileNet, EfficientNet)
2. INT8 quantization (4x compression, 2-3x speedup)
3. Platform-specific optimization (XNNPACK, Core ML)
4. Resolution reduction if acceptable

High-Throughput Batch Processing:

1. Large batch sizes (maximize GPU utilization)
2. Mixed precision training
3. Data pipeline optimization (prefetching, GPU preprocessing)
4. Distributed processing if needed

ROI Analysis:

Across all case studies, optimization provided:

- Average latency improvement: 10-25x
- Average throughput improvement: 15-30x
- Average cost reduction: 40-70%
- Engineering investment: 4-8 weeks

- Typical accuracy impact: 1-5% drop

When NOT to Optimize:

Not every model needs aggressive optimization:

- Offline batch processing with flexible time budgets
- Research and experimentation where accuracy is paramount
- Low-traffic services (<10 QPS) where cost is minimal
- Models already meeting performance requirements with headroom

The key is understanding your requirements and optimizing proportionally to achieve them.

Part VI

MLOps and Automation

Chapter 17

CI/CD Pipelines for Machine Learning

17.1 Learning Objectives

After completing this chapter, readers will be able to:

- Design and implement CI/CD pipelines specifically for ML projects
- Create GitHub Actions workflows for automated ML testing and deployment
- Implement comprehensive testing strategies for ML code and models
- Build continuous training pipelines with automated model validation
- Automate model deployment with safety checks and rollback mechanisms
- Integrate model registries and monitoring systems into CI/CD workflows
- Implement security scanning and compliance checks for ML systems
- Handle ML-specific challenges in CI/CD (data versioning, model artifacts, compute requirements)

17.2 Introduction

CI/CD for machine learning presents unique challenges beyond traditional software development. ML systems involve code, data, models, hyperparameters, and infrastructure that all need versioning, testing, and deployment automation. This chapter covers end-to-end CI/CD pipelines designed specifically for ML workflows.

ML-Specific CI/CD Challenges:

- **Large Artifacts:** Models can be gigabytes in size, requiring special handling
- **Data Dependencies:** Tests need access to datasets, which may be large or sensitive
- **Compute Requirements:** Training requires GPUs, complicating CI infrastructure
- **Non-Determinism:** ML models may have slight variations between runs

- **Multiple Stakeholders:** Data scientists, ML engineers, and DevOps teams
- **Regulatory Compliance:** Model governance and audit trails

Components of ML CI/CD:

1. **Code Testing:** Unit tests, integration tests for ML code
2. **Data Validation:** Schema validation, data quality checks
3. **Model Training:** Automated training on new data or code changes
4. **Model Validation:** Performance testing, bias detection, safety checks
5. **Model Deployment:** Automated deployment with canary releases
6. **Monitoring Integration:** Connect deployed models to monitoring systems
7. **Rollback Mechanisms:** Automatic rollback on performance degradation

17.3 GitHub Actions Workflows for ML Projects

17.3.1 Basic ML Pipeline Structure

GitHub Actions provides a powerful platform for ML CI/CD with support for custom runners, caching, and matrix builds.

```

1  # .github/workflows/ml-cicd.yml
2  name: ML CI/CD Pipeline
3
4  on:
5    push:
6      branches: [main, develop]
7    pull_request:
8      branches: [main]
9    schedule:
10     # Nightly training runs
11     - cron: '0 2 * * *'
12
13  env:
14    PYTHON_VERSION: '3.9'
15    MODEL_REGISTRY: 'mlflow'
16    DVC_REMOTE: 's3://ml-data-bucket'
17
18  jobs:
19    # Job 1: Code Quality and Unit Tests
20    code-quality:
21      runs-on: ubuntu-latest
22
23      steps:
24        - name: Checkout code
25          uses: actions/checkout@v3
26          with:
27            fetch-depth: 0 # Full history for versioning
28
29        - name: Set up Python

```

```

30     uses: actions/setup-python@v4
31     with:
32       python-version: ${ env.PYTHON_VERSION }
33       cache: 'pip'
34
35   - name: Install dependencies
36     run: |
37     pip install --upgrade pip
38     pip install -r requirements.txt
39     pip install -r requirements-dev.txt
40
41   - name: Lint with flake8
42     run: |
43     flake8 src/ tests/ --count --select=E9,F63,F7,F82 \
44       --show-source --statistics
45     flake8 src/ tests/ --count --exit-zero \
46       --max-complexity=10 --max-line-length=127 --statistics
47
48   - name: Format check with black
49     run: |
50     black --check src/ tests/
51
52   - name: Type checking with mypy
53     run: |
54     mypy src/ --ignore-missing-imports
55
56   - name: Security check with bandit
57     run: |
58     bandit -r src/ -ll
59
60   - name: Run unit tests
61     run: |
62     pytest tests/unit/ -v --cov=src --cov-report=xml \
63       --cov-report=term-missing
64
65   - name: Upload coverage to Codecov
66     uses: codecov/codecov-action@v3
67     with:
68       file: ./coverage.xml
69       flags: unittests
70       name: codecov-umbrella
71
72   # Job 2: Data Validation
73   data-validation:
74     runs-on: ubuntu-latest
75     needs: code-quality
76
77     steps:
78     - name: Checkout code
79       uses: actions/checkout@v3
80
81     - name: Set up Python
82       uses: actions/setup-python@v4
83       with:
84         python-version: ${ env.PYTHON_VERSION }
85         cache: 'pip'
86
87     - name: Install dependencies

```

```

88     run: |
89         pip install -r requirements.txt
90         pip install dvc[s3] great_expectations
91
92     - name: Configure AWS credentials
93       uses: aws-actions/configure-aws-credentials@v2
94       with:
95         aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
96         aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
97         aws-region: us-east-1
98
99     - name: Pull latest data with DVC
100      run: |
101          dvc remote modify origin --local \
102            access_key_id ${ secrets.AWS_ACCESS_KEY_ID }
103          dvc remote modify origin --local \
104            secret_access_key ${ secrets.AWS_SECRET_ACCESS_KEY }
105          dvc pull
106
107     - name: Validate data schema
108      run: |
109          python scripts/validate_data_schema.py \
110            --data-path data/processed/ \
111            --schema-path schemas/data_schema.json
112
113     - name: Run Great Expectations validation
114      run: |
115          great_expectations checkpoint run training_data_checkpoint
116
117     - name: Check data quality metrics
118      run: |
119          python scripts/check_data_quality.py \
120            --data-path data/processed/train.csv \
121            --thresholds-path config/data_quality_thresholds.json
122
123     # Job 3: Model Training
124     train-model:
125       runs-on: ubuntu-latest
126       needs: [code-quality, data-validation]
127       # Use self-hosted runner with GPU for actual training
128       # runs-on: [self-hosted, gpu]
129
130       strategy:
131         matrix:
132           model: [lightgbm, xgboost, random_forest]
133
134       steps:
135         - name: Checkout code
136           uses: actions/checkout@v3
137
138         - name: Set up Python
139           uses: actions/setup-python@v4
140           with:
141             python-version: ${ env.PYTHON_VERSION }
142             cache: 'pip'
143
144         - name: Install dependencies
145           run: |

```

```

146     pip install -r requirements.txt
147     pip install mlflow dvc[s3]
148
149 - name: Configure AWS credentials
150   uses: aws-actions/configure-aws-credentials@v2
151   with:
152     aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
153     aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
154     aws-region: us-east-1
155
156 - name: Pull training data
157   run: |
158     dvc remote modify origin --local \
159       access_key_id ${ secrets.AWS_ACCESS_KEY_ID }
160     dvc remote modify origin --local \
161       secret_access_key ${ secrets.AWS_SECRET_ACCESS_KEY }
162     dvc pull data/processed/train.csv.dvc
163
164 - name: Train model
165   env:
166     MLFLOW_TRACKING_URI: ${ secrets.MLFLOW_TRACKING_URI }
167     MLFLOW_EXPERIMENT_NAME: ${ github.repository }
168   run: |
169     python src/train.py \
170       --model-type ${ matrix.model } \
171       --data-path data/processed/train.csv \
172       --config config/training/${ matrix.model }.yaml \
173       --run-name "GHA-${ github.run_number }-${ matrix.model }"
174
175 - name: Upload model artifacts
176   uses: actions/upload-artifact@v3
177   with:
178     name: model-${ matrix.model }-${ github.run_number }
179     path: |
180       models/${ matrix.model }/
181       metrics/${ matrix.model }.json
182     retention-days: 30
183
184 # Job 4: Model Validation
185 validate-model:
186   runs-on: ubuntu-latest
187   needs: train-model
188
189   steps:
190   - name: Checkout code
191     uses: actions/checkout@v3
192
193   - name: Set up Python
194     uses: actions/setup-python@v4
195     with:
196       python-version: ${ env.PYTHON_VERSION }
197       cache: 'pip'
198
199   - name: Install dependencies
200     run: |
201       pip install -r requirements.txt
202       pip install mlflow
203

```

```

204 - name: Download model artifacts
205   uses: actions/download-artifact@v3
206   with:
207     path: artifacts/
208
209 - name: Run model validation tests
210   run: |
211     python tests/model/test_model_performance.py \
212       --models-dir artifacts/ \
213       --test-data data/processed/test.csv \
214       --baseline-metrics config/baseline_metrics.json
215
216 - name: Test model inference
217   run: |
218     pytest tests/model/test_inference.py -v \
219       --models-dir artifacts/
220
221 - name: Check model bias
222   run: |
223     python tests/model/test_fairness.py \
224       --models-dir artifacts/ \
225       --test-data data/processed/test.csv \
226       --sensitive-features config/sensitive_features.json
227
228 - name: Performance regression test
229   run: |
230     python scripts/compare_model_performance.py \
231       --new-model artifacts/model-lightgbm-${{ github.run_number }}/ \
232       --baseline-run-id ${{ secrets.BASELINE_MODEL_RUN_ID }} \
233       --min-improvement 0.01
234
235 # Job 5: Model Deployment (Production)
236 deploy-production:
237   runs-on: ubuntu-latest
238   needs: validate-model
239   if: github.ref == 'refs/heads/main' && github.event_name == 'push'
240
241   environment:
242     name: production
243     url: https://api.example.com/v1/predict
244
245   steps:
246   - name: Checkout code
247     uses: actions/checkout@v3
248
249   - name: Download best model
250     uses: actions/download-artifact@v3
251     with:
252       name: model-lightgbm-${{ github.run_number }}
253       path: model/
254
255   - name: Build Docker image
256     run: |
257       docker build -t ml-model:${{ github.sha }} \
258         -f deployment/Dockerfile .
259       docker tag ml-model:${{ github.sha }} \
260         ml-model:latest
261

```

```

262 - name: Push to container registry
263   env:
264     REGISTRY: ${ secrets.CONTAINER_REGISTRY }
265     REGISTRY_USERNAME: ${ secrets.REGISTRY_USERNAME }
266     REGISTRY_PASSWORD: ${ secrets.REGISTRY_PASSWORD }
267   run: |
268     echo $REGISTRY_PASSWORD | docker login $REGISTRY \
269       -u $REGISTRY_USERNAME --password-stdin
270     docker tag ml-model:${ github.sha } \
271       $REGISTRY/ml-model:${ github.sha }
272     docker push $REGISTRY/ml-model:${ github.sha }
273
274 - name: Deploy to Kubernetes
275   run: |
276     kubectl set image deployment/ml-model \
277       ml-model=${ secrets.CONTAINER_REGISTRY }/ml-model:${ github.sha } \
278       --namespace=production \
279       --record
280
281 - name: Wait for rollout
282   run: |
283     kubectl rollout status deployment/ml-model \
284       --namespace=production \
285       --timeout=5m
286
287 - name: Run smoke tests
288   run: |
289     python tests/integration/test_deployed_model.py \
290       --endpoint ${ secrets.PRODUCTION_ENDPOINT } \
291       --api-key ${ secrets.PRODUCTION_API_KEY }
292
293 - name: Notify deployment
294   if: always()
295   uses: 8398a7/action-slack@v3
296   with:
297     status: ${ job.status }
298     text: |
299       Deployment ${ job.status }
300       Model: lightgbm-${ github.run_number }
301       SHA: ${ github.sha }
302     webhook_url: ${ secrets.SLACK_WEBHOOK }

```

Listing 17.1: Complete ML CI/CD Workflow

17.3.2 Advanced Workflow Patterns

```

1 # .github/workflows/continuous-training.yml
2 name: Continuous Training Pipeline
3
4 on:
5   workflow_dispatch: # Manual trigger
6   inputs:
7     data_version:
8       description: 'DVC data version to use'
9       required: false
10      default: 'latest'
11   force_retrain:

```

```

12     description: 'Force retraining even if not needed'
13     required: false
14     default: 'false'
15
16   schedule:
17     # Weekly training on Sundays at 2 AM
18     - cron: '0 2 * * 0'
19
20   push:
21     paths:
22       - 'data/**/*.dvc' # Trigger on data changes
23       - 'config/training/**/*.yaml'
24
25   env:
26     MLFLOW_TRACKING_URI: ${ secrets.MLFLOW_TRACKING_URI }
27     EXPERIMENT_NAME: continuous-training
28
29   jobs:
30     check-retraining-needed:
31       runs-on: ubuntu-latest
32       outputs:
33         should_train: ${ steps.check.outputs.should_train }
34         drift_detected: ${ steps.drift.outputs.drift_detected }
35
36       steps:
37         - name: Checkout code
38           uses: actions/checkout@v3
39
40         - name: Set up Python
41           uses: actions/setup-python@v4
42           with:
43             python-version: '3.9'
44
45         - name: Install dependencies
46           run: |
47             pip install mlflow pandas scikit-learn
48
49         - name: Check data drift
50           id: drift
51           run: |
52             python scripts/check_data_drift.py \
53               --current-data data/processed/latest.csv \
54               --reference-data data/processed/reference.csv \
55               --threshold 0.1
56             echo "drift_detected=$(cat drift_result.txt)" >> $GITHUB_OUTPUT
57
58         - name: Check performance degradation
59           id: performance
60           run: |
61             python scripts/check_model_performance.py \
62               --model-id ${ secrets.CURRENT_MODEL_ID } \
63               --test-data data/processed/test.csv \
64               --threshold 0.02
65
66         - name: Determine if retraining needed
67           id: check
68           run: |
69             FORCE=${ github.event.inputs.force_retrain }

```



```

70     DRIFT=${{ steps.drift.outputs.drift_detected }}
71
72     if [ "$FORCE" = "true" ] || [ "$DRIFT" = "true" ]; then
73         echo "should_train=true" >> $GITHUB_OUTPUT
74     else
75         echo "should_train=false" >> $GITHUB_OUTPUT
76     fi
77
78 train-and-deploy:
79     needs: check-retraining-needed
80     if: needs.check-retraining-needed.outputs.should_train == 'true'
81     runs-on: [self-hosted, gpu]
82
83     steps:
84     - name: Checkout code
85       uses: actions/checkout@v3
86
87     - name: Set up Python with GPU support
88       uses: actions/setup-python@v4
89       with:
90         python-version: '3.9'
91
92     - name: Install dependencies
93       run: |
94         pip install -r requirements.txt
95         pip install torch torchvision --index-url \
96           https://download.pytorch.org/whl/cu118
97
98     - name: Pull training data
99       run: |
100         dvc pull data/processed/train.csv.dvc
101
102     - name: Train model with hyperparameter tuning
103       run: |
104         python src/train_with_tuning.py \
105           --data-path data/processed/train.csv \
106           --n-trials 100 \
107           --timeout 3600 \
108           --experiment-name ${ env.EXPERIMENT_NAME }
109
110     - name: Evaluate on test set
111       run: |
112         python src/evaluate.py \
113           --model-uri runs:<RUN_ID>/model \
114           --test-data data/processed/test.csv \
115           --output metrics/test_results.json
116
117     - name: Register model if improved
118       run: |
119         python scripts/register_model_if_better.py \
120           --run-id $MLFLOW_RUN_ID \
121           --model-name production_model \
122           --baseline-version latest \
123           --min-improvement 0.01
124
125     - name: Deploy new model version
126       if: success()
127       run: |

```

```

128     python scripts/deploy_model.py \
129         --model-name production_model \
130         --version latest \
131         --environment production \
132         --deployment-strategy canary \
133         --canary-percentage 10
134
135 - name: Monitor canary deployment
136   run: |
137     python scripts/monitor_canary.py \
138         --duration 3600 \
139         --metric-threshold 0.95 \
140         --error-rate-threshold 0.01
141
142 - name: Promote or rollback
143   run: |
144     python scripts/promote_or_rollback.py \
145         --model-name production_model \
146         --canary-metrics canary_metrics.json \
147         --auto-rollback true

```

Listing 17.2: Continuous Training Workflow with Scheduling

17.4 Automated Testing Strategies for ML Code

17.4.1 ML-Specific Testing Framework

```

1  # tests/conftest.py
2  """Pytest configuration and fixtures for ML testing"""
3
4  import pytest
5  import pandas as pd
6  import numpy as np
7  from pathlib import Path
8  import mlflow
9  from sklearn.datasets import make_classification, make_regression
10
11 @pytest.fixture(scope="session")
12 def test_data_dir(tmp_path_factory):
13     """Create temporary directory for test data"""
14     return tmp_path_factory.mktemp("test_data")
15
16 @pytest.fixture
17 def sample_classification_data():
18     """Generate sample classification dataset"""
19     X, y = make_classification(
20         n_samples=1000,
21         n_features=20,
22         n_informative=15,
23         n_redundant=5,
24         n_classes=2,
25         random_state=42
26     )
27
28     feature_names = [f"feature_{i}" for i in range(X.shape[1])]
29     df = pd.DataFrame(X, columns=feature_names)

```

```

30     df['target'] = y
31
32     return df
33
34 @pytest.fixture
35 def sample_regression_data():
36     """Generate sample regression dataset"""
37     X, y = make_regression(
38         n_samples=1000,
39         n_features=10,
40         n_informative=8,
41         noise=0.1,
42         random_state=42
43     )
44
45     feature_names = [f"feature_{i}" for i in range(X.shape[1])]
46     df = pd.DataFrame(X, columns=feature_names)
47     df['target'] = y
48
49     return df
50
51 @pytest.fixture
52 def trained_model(sample_classification_data):
53     """Provide a pre-trained model for testing"""
54     from sklearn.ensemble import RandomForestClassifier
55
56     X = sample_classification_data.drop('target', axis=1)
57     y = sample_classification_data['target']
58
59     model = RandomForestClassifier(n_estimators=10, random_state=42)
60     model.fit(X, y)
61
62     return model
63
64 @pytest.fixture(scope="session")
65 def mlflow_tracking():
66     """Set up MLflow tracking for tests"""
67     mlflow.set_tracking_uri("sqlite:///test_mlflow.db")
68     mlflow.set_experiment("test_experiment")
69
70     yield
71
72     # Cleanup
73     import os
74     if os.path.exists("test_mlflow.db"):
75         os.remove("test_mlflow.db")
76
77 # tests/unit/test_data_processing.py
78 """Unit tests for data processing functions"""
79
80 import pytest
81 import pandas as pd
82 import numpy as np
83 from src.data.preprocessing import (
84     remove_outliers,
85     encode_categorical,
86     scale_features,
87     handle_missing_values,

```

```

88     validate_data_schema
89 )
90
91 class TestDataPreprocessing:
92     """Test suite for data preprocessing"""
93
94     def test_remove_outliers_iqr(self, sample_regression_data):
95         """Test outlier removal using IQR method"""
96         df = sample_regression_data.copy()
97         original_len = len(df)
98
99         # Add some outliers
100        df.loc[0, 'feature_0'] = 1000
101        df.loc[1, 'feature_1'] = -1000
102
103        cleaned = remove_outliers(df, method='iqr', threshold=1.5)
104
105        assert len(cleaned) < len(df)
106        assert len(cleaned) >= original_len - 10 # Not too aggressive
107        assert not cleaned.isnull().any().any()
108
109    def test_encode_categorical_one_hot(self):
110        """Test one-hot encoding of categorical variables"""
111        df = pd.DataFrame({
112            'color': ['red', 'blue', 'green', 'red'],
113            'size': ['S', 'M', 'L', 'M'],
114            'value': [1, 2, 3, 4]
115        })
116
117        encoded = encode_categorical(
118            df,
119            columns=['color', 'size'],
120            method='onehot'
121        )
122
123        # Check new columns created
124        assert 'color_red' in encoded.columns
125        assert 'color_blue' in encoded.columns
126        assert 'size_S' in encoded.columns
127
128        # Check original columns removed
129        assert 'color' not in encoded.columns
130        assert 'size' not in encoded.columns
131
132        # Check values preserved
133        assert encoded['value'].tolist() == [1, 2, 3, 4]
134
135    def test_scale_features_standard(self, sample_regression_data):
136        """Test standard scaling of features"""
137        df = sample_regression_data.copy()
138        X = df.drop('target', axis=1)
139
140        scaled, scaler = scale_features(X, method='standard')
141
142        # Check mean close to 0 and std close to 1
143        assert np.allclose(scaled.mean(axis=0), 0, atol=0.1)
144        assert np.allclose(scaled.std(axis=0), 1, atol=0.1)
145

```

```

146     # Check scaler can transform new data
147     new_data = X.iloc[:10]
148     transformed = scaler.transform(new_data)
149     assert transformed.shape == new_data.shape
150
151     def test_handle_missing_values_imputation(self):
152         """Test missing value imputation"""
153         df = pd.DataFrame({
154             'a': [1, 2, np.nan, 4, 5],
155             'b': [np.nan, 2, 3, 4, 5],
156             'c': ['x', 'y', 'z', np.nan, 'w']
157         })
158
159         filled = handle_missing_values(
160             df,
161             strategy='mean',
162             categorical_strategy='most_frequent'
163         )
164
165         assert not filled.isnull().any().any()
166         assert filled['a'].iloc[2] == pytest.approx(3.0, rel=0.1)
167         assert filled['c'].iloc[3] in ['x', 'y', 'z', 'w']
168
169     def test_validate_data_schema(self, sample_classification_data):
170         """Test data schema validation"""
171         df = sample_classification_data
172
173         schema = {
174             'required_columns': [f'feature_{i}' for i in range(20)] + ['target'],
175             'column_types': {
176                 'feature_0': 'float64',
177                 'target': 'int64'
178             },
179             'value_ranges': {
180                 'target': {'min': 0, 'max': 1}
181             }
182         }
183
184         # Valid data should pass
185         assert validate_data_schema(df, schema) == True
186
187         # Missing column should fail
188         df_invalid = df.drop('feature_0', axis=1)
189         assert validate_data_schema(df_invalid, schema) == False
190
191     # tests/unit/test_model_training.py
192     """Unit tests for model training"""
193
194     import pytest
195     import numpy as np
196     from src.models.trainer import ModelTrainer
197     from src.models.model_factory import create_model
198
199     class TestModelTrainer:
200         """Test suite for model training"""
201
202         def test_train_classification_model(self, sample_classification_data):
203             """Test training a classification model"""

```

```

204     df = sample_classification_data
205     X = df.drop('target', axis=1)
206     y = df['target']
207
208     trainer = ModelTrainer(model_type='random_forest')
209     model, metrics = trainer.train(X, y)
210
211     assert model is not None
212     assert 'accuracy' in metrics
213     assert metrics['accuracy'] > 0.5 # Better than random
214     assert 'f1_score' in metrics
215
216     def test_model_reproducibility(self, sample_classification_data):
217         """Test that training with same seed gives same results"""
218         df = sample_classification_data
219         X = df.drop('target', axis=1)
220         y = df['target']
221
222         trainer1 = ModelTrainer(model_type='random_forest', random_state=42)
223         model1, metrics1 = trainer1.train(X, y)
224
225         trainer2 = ModelTrainer(model_type='random_forest', random_state=42)
226         model2, metrics2 = trainer2.train(X, y)
227
228         # Predictions should be identical
229         preds1 = model1.predict(X)
230         preds2 = model2.predict(X)
231
232         assert np.array_equal(preds1, preds2)
233
234     def test_train_with_validation_split(self, sample_classification_data):
235         """Test training with validation split"""
236         df = sample_classification_data
237         X = df.drop('target', axis=1)
238         y = df['target']
239
240         trainer = ModelTrainer(
241             model_type='gradient_boosting',
242             validation_split=0.2
243         )
244         model, metrics = trainer.train(X, y)
245
246         assert 'train_accuracy' in metrics
247         assert 'val_accuracy' in metrics
248
249         # Validation accuracy should be reasonable
250         assert metrics['val_accuracy'] > 0.5
251         assert metrics['val_accuracy'] <= metrics['train_accuracy']
252
253     def test_early_stopping(self, sample_classification_data):
254         """Test early stopping during training"""
255         df = sample_classification_data
256         X = df.drop('target', axis=1)
257         y = df['target']
258
259         trainer = ModelTrainer(
260             model_type='gradient_boosting',
261             early_stopping=True,

```

```

262         early_stopping_rounds=10,
263         validation_split=0.2
264     )
265
266     model, metrics = trainer.train(X, y)
267
268     assert 'best_iteration' in metrics
269     assert metrics['best_iteration'] < 100 # Should stop early
270
271 # tests/unit/test_model_inference.py
272 """Unit tests for model inference"""
273
274 import pytest
275 import numpy as np
276 import pandas as pd
277 from src.models.inference import ModelPredictor
278
279 class TestModelInference:
280     """Test suite for model inference"""
281
282     def test_single_prediction(self, trained_model, sample_classification_data):
283         """Test prediction on single sample"""
284         X = sample_classification_data.drop('target', axis=1)
285         sample = X.iloc[0:1]
286
287         predictor = ModelPredictor(trained_model)
288         prediction = predictor.predict(sample)
289
290         assert prediction.shape == (1,)
291         assert prediction[0] in [0, 1]
292
293     def test_batch_prediction(self, trained_model, sample_classification_data):
294         """Test prediction on batch of samples"""
295         X = sample_classification_data.drop('target', axis=1)
296
297         predictor = ModelPredictor(trained_model)
298         predictions = predictor.predict(X)
299
300         assert predictions.shape == (len(X),)
301         assert all(p in [0, 1] for p in predictions)
302
303     def test_prediction_probabilities(self, trained_model, sample_classification_data):
304         :
305         """Test probability predictions"""
306         X = sample_classification_data.drop('target', axis=1)
307
308         predictor = ModelPredictor(trained_model)
309         probabilities = predictor.predict_proba(X)
310
311         assert probabilities.shape == (len(X), 2)
312         assert np.allclose(probabilities.sum(axis=1), 1.0)
313         assert np.all((probabilities >= 0) & (probabilities <= 1))
314
315     def test_input_validation(self, trained_model):
316         """Test input validation for predictions"""
317         predictor = ModelPredictor(trained_model)
318
319         # Wrong number of features

```

```

319         invalid_input = pd.DataFrame(np.random.randn(10, 5))
320
321         with pytest.raises(ValueError):
322             predictor.predict(invalid_input)
323
324     def test_prediction_latency(self, trained_model, sample_classification_data):
325         """Test prediction latency meets requirements"""
326         import time
327
328         X = sample_classification_data.drop('target', axis=1)
329         predictor = ModelPredictor(trained_model)
330
331         # Warmup
332         _ = predictor.predict(X.iloc[:10])
333
334         # Measure latency
335         start = time.time()
336         _ = predictor.predict(X.iloc[:100])
337         elapsed = time.time() - start
338
339         # Should predict 100 samples in less than 1 second
340         assert elapsed < 1.0
341
342     # tests/integration/test_training_pipeline.py
343     """Integration tests for complete training pipeline"""
344
345     import pytest
346     import mlflow
347     from pathlib import Path
348     from src.pipelines.training_pipeline import TrainingPipeline
349
350     class TestTrainingPipeline:
351         """Test suite for end-to-end training pipeline"""
352
353         def test_complete_pipeline(
354             self,
355             sample_classification_data,
356             test_data_dir,
357             mlflow_tracking
358         ):
359             """Test complete training pipeline from data to model"""
360
361             # Save test data
362             data_path = test_data_dir / "train.csv"
363             sample_classification_data.to_csv(data_path, index=False)
364
365             # Run pipeline
366             pipeline = TrainingPipeline(
367                 data_path=str(data_path),
368                 model_type='random_forest',
369                 experiment_name='test_pipeline'
370             )
371
372             result = pipeline.run()
373
374             # Check pipeline completed successfully
375             assert result['status'] == 'success'
376             assert 'model' in result

```



```

377     assert 'metrics' in result
378     assert 'run_id' in result
379
380     # Check MLflow logging
381     run = mlflow.get_run(result['run_id'])
382     assert run.info.status == 'FINISHED'
383     assert 'accuracy' in run.data.metrics
384
385     def test_pipeline_with_preprocessing(
386         self,
387         sample_classification_data,
388         test_data_dir,
389         mlflow_tracking
390     ):
391         """Test pipeline with preprocessing steps"""
392
393         # Add missing values and categorical columns
394         df = sample_classification_data.copy()
395         df.loc[0, 'feature_0'] = np.nan
396         df['category'] = np.random.choice(['A', 'B', 'C'], size=len(df))
397
398         data_path = test_data_dir / "train_with_preprocessing.csv"
399         df.to_csv(data_path, index=False)
400
401         # Configure preprocessing
402         preprocessing_config = {
403             'handle_missing': True,
404             'encode_categorical': True,
405             'scale_features': True
406         }
407
408         pipeline = TrainingPipeline(
409             data_path=str(data_path),
410             model_type='random_forest',
411             preprocessing_config=preprocessing_config
412         )
413
414         result = pipeline.run()
415
416         assert result['status'] == 'success'
417         assert 'preprocessing_steps' in result
418
419     def test_pipeline_failure_handling(self, test_data_dir):
420         """Test pipeline handles errors gracefully"""
421
422         # Non-existent data file
423         pipeline = TrainingPipeline(
424             data_path=str(test_data_dir / "nonexistent.csv"),
425             model_type='random_forest'
426         )
427
428         result = pipeline.run()
429
430         assert result['status'] == 'failed'
431         assert 'error' in result

```

Listing 17.3: Comprehensive ML Testing Suite

17.4.2 Model Performance Testing

```

1  # tests/model/test_model_performance.py
2  """Tests for model performance requirements"""
3
4  import pytest
5  import pandas as pd
6  import numpy as np
7  from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
8  import json
9
10 class TestModelPerformance:
11     """Test suite for model performance requirements"""
12
13     @pytest.fixture
14     def performance_thresholds(self):
15         """Load performance thresholds"""
16         return {
17             'min_accuracy': 0.80,
18             'min_precision': 0.75,
19             'min_recall': 0.75,
20             'min_f1': 0.75,
21             'max_prediction_time_ms': 100
22         }
23
24     def test_accuracy_threshold(
25         self,
26         trained_model,
27         sample_classification_data,
28         performance_thresholds
29     ):
30         """Test model meets minimum accuracy requirement"""
31         X = sample_classification_data.drop('target', axis=1)
32         y = sample_classification_data['target']
33
34         predictions = trained_model.predict(X)
35         accuracy = accuracy_score(y, predictions)
36
37         assert accuracy >= performance_thresholds['min_accuracy'], \
38             f"Accuracy {accuracy:.3f} below threshold {performance_thresholds['min_accuracy']}"
39
40     def test_precision_recall_balance(
41         self,
42         trained_model,
43         sample_classification_data,
44         performance_thresholds
45     ):
46         """Test precision and recall are balanced"""
47         X = sample_classification_data.drop('target', axis=1)
48         y = sample_classification_data['target']
49
50         predictions = trained_model.predict(X)
51
52         precision = precision_score(y, predictions, average='binary')
53         recall = recall_score(y, predictions, average='binary')
54
55         assert precision >= performance_thresholds['min_precision']

```

```

56     assert recall >= performance_thresholds['min_recall']
57
58     # Check balance (neither should be much worse than the other)
59     ratio = max(precision, recall) / min(precision, recall)
60     assert ratio < 1.5, "Precision and recall are imbalanced"
61
62     def test_class_distribution_robustness(self, trained_model):
63         """Test model performs well on imbalanced classes"""
64         # Create imbalanced test set (90-10 split)
65         X_imbalanced = pd.DataFrame(
66             np.random.randn(1000, 20),
67             columns=[f'feature_{i}' for i in range(20)]
68         )
69         y_imbalanced = np.array([0] * 900 + [1] * 100)
70
71         predictions = trained_model.predict(X_imbalanced)
72
73         # Should predict at least some minority class samples
74         minority_predictions = predictions[y_imbalanced == 1]
75         minority_recall = (minority_predictions == 1).mean()
76
77         assert minority_recall > 0.3, \
78             "Model fails to predict minority class"
79
80     def test_prediction_consistency(self, trained_model, sample_classification_data):
81         """Test predictions are consistent across multiple calls"""
82         X = sample_classification_data.drop('target', axis=1)
83
84         pred1 = trained_model.predict(X)
85         pred2 = trained_model.predict(X)
86         pred3 = trained_model.predict(X)
87
88         assert np.array_equal(pred1, pred2)
89         assert np.array_equal(pred2, pred3)
90
91     def test_inference_latency(
92         self,
93         trained_model,
94         sample_classification_data,
95         performance_thresholds
96     ):
97         """Test inference meets latency requirements"""
98         import time
99
100         X = sample_classification_data.drop('target', axis=1)
101         sample = X.iloc[:100] # Test with batch of 100
102
103         # Warmup
104         _ = trained_model.predict(sample)
105
106         # Measure latency
107         times = []
108         for _ in range(10):
109             start = time.perf_counter()
110             _ = trained_model.predict(sample)
111             elapsed = (time.perf_counter() - start) * 1000 # ms
112             times.append(elapsed)
113

```

```

114     avg_latency = np.mean(times)
115     p95_latency = np.percentile(times, 95)
116
117     assert p95_latency < performance_thresholds['max_prediction_time_ms'], \
118         f"P95 latency {p95_latency:.1f}ms exceeds threshold"
119
120 # tests/model/test_fairness.py
121 """Tests for model fairness and bias"""
122
123 import pytest
124 import pandas as pd
125 import numpy as np
126 from sklearn.metrics import confusion_matrix
127
128 class TestModelFairness:
129     """Test suite for model fairness across protected groups"""
130
131     @pytest.fixture
132     def data_with_protected_attributes(self, sample_classification_data):
133         """Add protected attributes to test data"""
134         df = sample_classification_data.copy()
135
136         # Add gender (protected attribute)
137         df['gender'] = np.random.choice(['M', 'F'], size=len(df))
138
139         # Add age group (protected attribute)
140         df['age_group'] = np.random.choice(
141             ['18-30', '31-50', '51+'],
142             size=len(df)
143         )
144
145         return df
146
147     def test_demographic_parity(
148         self,
149         trained_model,
150         data_with_protected_attributes
151     ):
152         """Test demographic parity across protected groups"""
153         df = data_with_protected_attributes
154         X = df.drop(['target', 'gender', 'age_group'], axis=1)
155
156         predictions = trained_model.predict(X)
157         df['prediction'] = predictions
158
159         # Calculate positive prediction rate by gender
160         male_rate = df[df['gender'] == 'M']['prediction'].mean()
161         female_rate = df[df['gender'] == 'F']['prediction'].mean()
162
163         # Rates should be similar (within 10% difference)
164         rate_diff = abs(male_rate - female_rate)
165         assert rate_diff < 0.1, \
166             f"Demographic parity violated: M={male_rate:.3f}, F={female_rate:.3f}"
167
168     def test_equal_opportunity(
169         self,
170         trained_model,
171         data_with_protected_attributes

```

```

172 ):
173     """Test equal opportunity (equal TPR across groups)"""
174     df = data_with_protected_attributes
175     X = df.drop(['target', 'gender', 'age_group'], axis=1)
176     y = df['target']
177
178     predictions = trained_model.predict(X)
179
180     # Calculate TPR for each gender
181     male_mask = df['gender'] == 'M'
182     female_mask = df['gender'] == 'F'
183
184     # True Positive Rate for males
185     male_positives = (y[male_mask] == 1)
186     male_tpr = (predictions[male_mask][male_positives] == 1).mean()
187
188     # True Positive Rate for females
189     female_positives = (y[female_mask] == 1)
190     female_tpr = (predictions[female_mask][female_positives] == 1).mean()
191
192     tpr_diff = abs(male_tpr - female_tpr)
193     assert tpr_diff < 0.15, \
194         f"Equal opportunity violated: M_TPR={male_tpr:.3f}, F_TPR={female_tpr:.3f}"
195
196 def test_fairness_across_age_groups(
197     self,
198     trained_model,
199     data_with_protected_attributes
200 ):
201     """Test fairness across different age groups"""
202     df = data_with_protected_attributes
203     X = df.drop(['target', 'gender', 'age_group'], axis=1)
204     y = df['target']
205
206     predictions = trained_model.predict(X)
207
208     # Calculate accuracy for each age group
209     age_accuracies = {}
210     for age_group in df['age_group'].unique():
211         mask = df['age_group'] == age_group
212         age_acc = (predictions[mask] == y[mask]).mean()
213         age_accuracies[age_group] = age_acc
214
215     # Check accuracy variance across age groups
216     min_acc = min(age_accuracies.values())
217     max_acc = max(age_accuracies.values())
218
219     assert (max_acc - min_acc) < 0.15, \
220         f"Large accuracy variance across age groups: {age_accuracies}"
221
222 def test_calibration_across_groups(
223     self,
224     trained_model,
225     data_with_protected_attributes
226 ):
227     """Test prediction calibration across protected groups"""
228     df = data_with_protected_attributes

```

```

229 X = df.drop(['target', 'gender', 'age_group'], axis=1)
230 y = df['target']
231
232 # Get probability predictions
233 proba = trained_model.predict_proba(X)[: , 1]
234
235 # Check calibration for each gender
236 for gender in ['M', 'F']:
237     mask = df['gender'] == gender
238     gender_proba = proba[mask]
239     gender_y = y[mask]
240
241     # Bin predictions and check calibration
242     bins = np.linspace(0, 1, 11)
243     bin_indices = np.digitize(gender_proba, bins)
244
245     for bin_idx in range(1, len(bins)):
246         bin_mask = bin_indices == bin_idx
247         if bin_mask.sum() > 10: # Enough samples in bin
248             predicted_prob = gender_proba[bin_mask].mean()
249             actual_prob = gender_y[bin_mask].mean()
250
251             # Predicted probability should match actual
252             calibration_error = abs(predicted_prob - actual_prob)
253             assert calibration_error < 0.2, \
254                 f"Poor calibration for {gender} in bin {bin_idx}"
255
256 # tests/model/test_robustness.py
257 """Tests for model robustness"""
258
259 import pytest
260 import pandas as pd
261 import numpy as np
262
263 class TestModelRobustness:
264     """Test suite for model robustness"""
265
266     def test_noise_robustness(self, trained_model, sample_classification_data):
267         """Test model is robust to input noise"""
268         X = sample_classification_data.drop('target', axis=1)
269         y = sample_classification_data['target']
270
271         # Baseline predictions
272         baseline_pred = trained_model.predict(X)
273         baseline_acc = (baseline_pred == y).mean()
274
275         # Add noise to features
276         noise_std = X.std() * 0.1 # 10% noise
277         X_noisy = X + np.random.randn(*X.shape) * noise_std.values
278
279         noisy_pred = trained_model.predict(X_noisy)
280         noisy_acc = (noisy_pred == y).mean()
281
282         # Accuracy shouldn't degrade too much
283         acc_degradation = baseline_acc - noisy_acc
284         assert acc_degradation < 0.05, \
285             f"Model not robust to noise: {acc_degradation:.3f} accuracy drop"
286

```

```

287 def test_missing_value_handling(self, trained_model, sample_classification_data):
288     """Test model handles missing values gracefully"""
289     X = sample_classification_data.drop('target', axis=1)
290
291     # Introduce 5% missing values randomly
292     X_missing = X.copy()
293     missing_mask = np.random.random(X.shape) < 0.05
294     X_missing[missing_mask] = np.nan
295
296     # Model should either handle missing values or raise clear error
297     try:
298         predictions = trained_model.predict(X_missing.fillna(X_missing.mean()))
299         assert len(predictions) == len(X)
300     except ValueError as e:
301         assert "missing" in str(e).lower() or "nan" in str(e).lower()
302
303 def test_outlier_robustness(self, trained_model, sample_classification_data):
304     """Test model is robust to outliers"""
305     X = sample_classification_data.drop('target', axis=1)
306     y = sample_classification_data['target']
307
308     baseline_pred = trained_model.predict(X)
309
310     # Introduce outliers (1% of data)
311     X_outliers = X.copy()
312     outlier_mask = np.random.random(len(X)) < 0.01
313     for col in X_outliers.columns:
314         X_outliers.loc[outlier_mask, col] *= 100 # Extreme values
315
316     outlier_pred = trained_model.predict(X_outliers)
317
318     # Most predictions should remain the same
319     unchanged = (baseline_pred == outlier_pred).mean()
320     assert unchanged > 0.95, \
321         "Model predictions too sensitive to outliers"
322
323 def test_feature_importance_stability(self, trained_model,
324 sample_classification_data):
325     """Test feature importances are stable"""
326     if not hasattr(trained_model, 'feature_importances_'):
327         pytest.skip("Model doesn't support feature importances")
328
329     X = sample_classification_data.drop('target', axis=1)
330     y = sample_classification_data['target']
331
332     # Train multiple models with same configuration
333     importances = []
334     for seed in range(5):
335         from sklearn.ensemble import RandomForestClassifier
336         model = RandomForestClassifier(n_estimators=10, random_state=seed)
337         model.fit(X, y)
338         importances.append(model.feature_importances_)
339
340     importances = np.array(importances)
341
342     # Feature importance rankings should be relatively stable
343     std_importance = importances.std(axis=0).mean()
344     assert std_importance < 0.1, \

```

"Feature importances are unstable across runs"

Listing 17.4: Model Performance and Fairness Testing

17.5 Continuous Training Pipeline Implementation

17.5.1 Automated Continuous Training System

```

1  # src/pipelines/continuous_training.py
2  """Continuous training pipeline with drift detection and auto-retraining"""
3
4  from typing import Dict, Any, Optional, List
5  from dataclasses import dataclass
6  from datetime import datetime, timedelta
7  import logging
8  import mlflow
9  import pandas as pd
10 import numpy as np
11 from pathlib import Path
12
13 @dataclass
14 class TrainingConfig:
15     """Configuration for continuous training"""
16     model_type: str
17     hyperparameters: Dict[str, Any]
18     training_schedule: str # cron expression
19     drift_threshold: float = 0.1
20     performance_threshold: float = 0.02
21     min_training_samples: int = 1000
22     validation_split: float = 0.2
23     max_training_time_hours: int = 4
24
25 class DriftDetector:
26     """Detect data drift and concept drift"""
27
28     def __init__(self, reference_data: pd.DataFrame):
29         self.reference_data = reference_data
30         self.reference_stats = self._compute_statistics(reference_data)
31
32     def _compute_statistics(self, data: pd.DataFrame) -> Dict:
33         """Compute statistical properties of dataset"""
34         stats = {}
35
36         for col in data.select_dtypes(include=[np.number]).columns:
37             stats[col] = {
38                 'mean': data[col].mean(),
39                 'std': data[col].std(),
40                 'min': data[col].min(),
41                 'max': data[col].max(),
42                 'q25': data[col].quantile(0.25),
43                 'q50': data[col].quantile(0.50),
44                 'q75': data[col].quantile(0.75)
45             }
46
47         return stats
48

```



```

49 def detect_drift(self, current_data: pd.DataFrame) -> Dict[str, Any]:
50     """Detect if data has drifted from reference"""
51     current_stats = self._compute_statistics(current_data)
52
53     drift_scores = {}
54     for col in self.reference_stats:
55         if col not in current_stats:
56             continue
57
58         # Calculate normalized difference in statistics
59         ref = self.reference_stats[col]
60         curr = current_stats[col]
61
62         mean_diff = abs(curr['mean'] - ref['mean']) / (ref['std'] + 1e-6)
63         std_ratio = curr['std'] / (ref['std'] + 1e-6)
64
65         drift_scores[col] = {
66             'mean_shift': mean_diff,
67             'std_ratio': std_ratio,
68             'drifted': mean_diff > 2.0 or abs(std_ratio - 1.0) > 0.5
69         }
70
71     # Overall drift assessment
72     drifted_features = [
73         col for col, score in drift_scores.items()
74         if score['drifted']
75     ]
76
77     return {
78         'drift_detected': len(drifted_features) > len(drift_scores) * 0.2,
79         'drifted_features': drifted_features,
80         'drift_scores': drift_scores,
81         'num_drifted': len(drifted_features),
82         'total_features': len(drift_scores)
83     }
84
85 class PerformanceMonitor:
86     """Monitor model performance in production"""
87
88     def __init__(self, model_id: str, mlflow_tracking_uri: str):
89         self.model_id = model_id
90         mlflow.set_tracking_uri(mlflow_tracking_uri)
91         self.logger = logging.getLogger(__name__)
92
93     def check_performance_degradation(
94         self,
95         current_metrics: Dict[str, float],
96         baseline_metrics: Dict[str, float],
97         threshold: float = 0.02
98     ) -> Dict[str, Any]:
99         """Check if performance has degraded"""
100
101         degradations = {}
102         for metric, current_value in current_metrics.items():
103             if metric not in baseline_metrics:
104                 continue
105
106             baseline_value = baseline_metrics[metric]

```

```

107         degradation = baseline_value - current_value
108
109         degradations[metric] = {
110             'current': current_value,
111             'baseline': baseline_value,
112             'degradation': degradation,
113             'degraded': degradation > threshold
114         }
115
116         any_degraded = any(d['degraded'] for d in degradations.values())
117
118         return {
119             'performance_degraded': any_degraded,
120             'metrics': degradations
121         }
122
123     def get_recent_performance(
124         self,
125         days: int = 7
126     ) -> Dict[str, List[float]]:
127         """Get recent performance metrics from production"""
128
129         # In production, this would query monitoring system
130         # For example: Prometheus, CloudWatch, or custom metrics store
131
132         end_date = datetime.now()
133         start_date = end_date - timedelta(days=days)
134
135         # Placeholder - would query actual metrics
136         metrics_history = {
137             'accuracy': [],
138             'precision': [],
139             'recall': [],
140             'latency_ms': []
141         }
142
143         self.logger.info(
144             f"Retrieved {days} days of metrics for model {self.model_id}"
145         )
146
147         return metrics_history
148
149     class ContinuousTrainingPipeline:
150         """
151         Automated continuous training pipeline
152
153         Monitors data and performance, triggers retraining when needed
154         """
155
156         def __init__(
157             self,
158             config: TrainingConfig,
159             mlflow_tracking_uri: str,
160             data_source: str,
161             model_registry_name: str
162         ):
163             self.config = config
164             self.data_source = data_source

```

```

165     self.model_registry_name = model_registry_name
166
167     mlflow.set_tracking_uri(mlflow_tracking_uri)
168     self.logger = logging.getLogger(__name__)
169
170     self.drift_detector: Optional[DriftDetector] = None
171     self.performance_monitor: Optional[PerformanceMonitor] = None
172
173     def should_retrain(self) -> Dict[str, Any]:
174         """Determine if model should be retrained"""
175
176         reasons = []
177
178         # Check 1: Data drift
179         current_data = self._load_recent_data()
180
181         if self.drift_detector is None:
182             # Initialize drift detector with current data as reference
183             self.drift_detector = DriftDetector(current_data)
184             self.logger.info("Initialized drift detector with current data")
185
186         drift_result = self.drift_detector.detect_drift(current_data)
187
188         if drift_result['drift_detected']:
189             reasons.append({
190                 'type': 'data_drift',
191                 'severity': 'high',
192                 'details': drift_result
193             })
194             self.logger.warning(
195                 f"Data drift detected in {drift_result['num_drifted']} features"
196             )
197
198         # Check 2: Performance degradation
199         current_model = self._get_current_production_model()
200
201         if current_model:
202             current_metrics = self._evaluate_model(current_model, current_data)
203             baseline_metrics = self._get_baseline_metrics()
204
205             if self.performance_monitor is None:
206                 self.performance_monitor = PerformanceMonitor(
207                     model_id=current_model['model_id'],
208                     mlflow_tracking_uri=mlflow.get_tracking_uri()
209                 )
210
211             perf_check = self.performance_monitor.check_performance_degradation(
212                 current_metrics,
213                 baseline_metrics,
214                 threshold=self.config.performance_threshold
215             )
216
217             if perf_check['performance_degraded']:
218                 reasons.append({
219                     'type': 'performance_degradation',
220                     'severity': 'high',
221                     'details': perf_check
222                 })

```

```

223         self.logger.warning("Performance degradation detected")
224
225         # Check 3: Scheduled retraining
226         last_training_time = self._get_last_training_time()
227         time_since_training = datetime.now() - last_training_time
228
229         if time_since_training > timedelta(days=7): # Weekly retraining
230             reasons.append({
231                 'type': 'scheduled',
232                 'severity': 'low',
233                 'details': {
234                     'days_since_training': time_since_training.days
235                 }
236             })
237
238         # Check 4: Sufficient new data
239         new_samples = len(current_data)
240         if new_samples < self.config.min_training_samples:
241             reasons.append({
242                 'type': 'insufficient_data',
243                 'severity': 'blocking',
244                 'details': {
245                     'current_samples': new_samples,
246                     'required_samples': self.config.min_training_samples
247                 }
248             })
249
250         # Decision
251         should_train = any(
252             r['severity'] in ['high', 'critical'] for r in reasons
253         ) and not any(
254             r['severity'] == 'blocking' for r in reasons
255         )
256
257         return {
258             'should_retrain': should_train,
259             'reasons': reasons,
260             'timestamp': datetime.now().isoformat()
261         }
262
263     def train_and_validate(self) -> Dict[str, Any]:
264         """Train new model and validate performance"""
265
266         self.logger.info("Starting training pipeline...")
267
268         # Start MLflow run
269         with mlflow.start_run(run_name=f"continuous_training_{datetime.now().strftime(
270             '%Y%m%d_%H%M%S'})}"):
271
272             # Log configuration
273             mlflow.log_params(self.config.hyperparameters)
274             mlflow.set_tag("pipeline", "continuous_training")
275             mlflow.set_tag("trigger", "automated")
276
277             # Load and prepare data
278             train_data, val_data = self._prepare_training_data()
279
280             # Log data statistics

```

```

280     mlflow.log_metric("train_samples", len(train_data))
281     mlflow.log_metric("val_samples", len(val_data))
282
283     # Train model
284     model, training_metrics = self._train_model(train_data)
285
286     # Log training metrics
287     for metric_name, value in training_metrics.items():
288         mlflow.log_metric(f"train_{metric_name}", value)
289
290     # Validate model
291     validation_metrics = self._validate_model(model, val_data)
292
293     for metric_name, value in validation_metrics.items():
294         mlflow.log_metric(f"val_{metric_name}", value)
295
296     # Compare with baseline
297     baseline_metrics = self._get_baseline_metrics()
298     comparison = self._compare_with_baseline(
299         validation_metrics,
300         baseline_metrics
301     )
302
303     mlflow.log_dict(comparison, "baseline_comparison.json")
304
305     # Log model
306     mlflow.sklearn.log_model(
307         model,
308         "model",
309         registered_model_name=self.model_registry_name
310     )
311
312     run_id = mlflow.active_run().info.run_id
313
314     result = {
315         'status': 'success',
316         'run_id': run_id,
317         'training_metrics': training_metrics,
318         'validation_metrics': validation_metrics,
319         'baseline_comparison': comparison,
320         'model_improved': comparison['is_improvement']
321     }
322
323     self.logger.info(
324         f"Training completed. Run ID: {run_id}, "
325         f"Improved: {result['model_improved']}"
326     )
327
328     return result
329
330 def deploy_if_improved(self, training_result: Dict[str, Any]) -> Dict[str, Any]:
331     """Deploy new model if it's better than current"""
332
333     if not training_result['model_improved']:
334         self.logger.info("New model did not improve. Skipping deployment.")
335         return {
336             'deployed': False,
337             'reason': 'no_improvement'

```

```

338     }
339
340     # Get model URI
341     run_id = training_result['run_id']
342     model_uri = f"runs:{run_id}/model"
343
344     # Register model version
345     model_version = mlflow.register_model(
346         model_uri,
347         self.model_registry_name
348     )
349
350     # Transition to staging
351     client = mlflow.tracking.MlflowClient()
352     client.transition_model_version_stage(
353         name=self.model_registry_name,
354         version=model_version.version,
355         stage="Staging"
356     )
357
358     self.logger.info(
359         f"Model version {model_version.version} moved to Staging"
360     )
361
362     # Run canary deployment tests
363     canary_result = self._run_canary_tests(model_version.version)
364
365     if canary_result['passed']:
366         # Promote to production
367         client.transition_model_version_stage(
368             name=self.model_registry_name,
369             version=model_version.version,
370             stage="Production"
371         )
372
373         # Archive old production model
374         self._archive_old_production_model()
375
376         self.logger.info(
377             f"Model version {model_version.version} deployed to Production"
378         )
379
380         return {
381             'deployed': True,
382             'version': model_version.version,
383             'stage': 'Production',
384             'canary_results': canary_result
385         }
386     else:
387         self.logger.warning(
388             f"Canary tests failed. Model {model_version.version} "
389             f"remains in Staging"
390         )
391
392         return {
393             'deployed': False,
394             'reason': 'canary_failed',
395             'canary_results': canary_result

```

```

396         }
397
398     def run(self) -> Dict[str, Any]:
399         """Run complete continuous training pipeline"""
400
401         try:
402             # Check if retraining is needed
403             retrain_decision = self.should_retrain()
404
405             if not retrain_decision['should_retrain']:
406                 self.logger.info("Retraining not needed at this time")
407                 return {
408                     'status': 'skipped',
409                     'decision': retrain_decision
410                 }
411
412             self.logger.info(
413                 f"Retraining triggered. Reasons: "
414                 f"{[r['type'] for r in retrain_decision['reasons']]}"
415             )
416
417             # Train and validate new model
418             training_result = self.train_and_validate()
419
420             # Deploy if improved
421             deployment_result = self.deploy_if_improved(training_result)
422
423             return {
424                 'status': 'completed',
425                 'decision': retrain_decision,
426                 'training': training_result,
427                 'deployment': deployment_result
428             }
429
430         except Exception as e:
431             self.logger.error(f"Pipeline failed: {str(e)}", exc_info=True)
432             return {
433                 'status': 'failed',
434                 'error': str(e)
435             }
436
437     def _load_recent_data(self) -> pd.DataFrame:
438         """Load recent data from data source"""
439         # Implementation depends on data source
440         pass
441
442     def _get_current_production_model(self) -> Optional[Dict]:
443         """Get current production model"""
444         pass
445
446     def _evaluate_model(
447         self,
448         model: Any,
449         data: pd.DataFrame
450     ) -> Dict[str, float]:
451         """Evaluate model on data"""
452         pass
453

```

```

454 def _get_baseline_metrics(self) -> Dict[str, float]:
455     """Get baseline metrics for comparison"""
456     pass
457
458 def _get_last_training_time(self) -> datetime:
459     """Get timestamp of last training"""
460     pass
461
462 def _prepare_training_data(self) -> tuple:
463     """Prepare train and validation datasets"""
464     pass
465
466 def _train_model(self, train_data: pd.DataFrame) -> tuple:
467     """Train model on data"""
468     pass
469
470 def _validate_model(
471     self,
472     model: Any,
473     val_data: pd.DataFrame
474 ) -> Dict[str, float]:
475     """Validate model"""
476     pass
477
478 def _compare_with_baseline(
479     self,
480     current_metrics: Dict[str, float],
481     baseline_metrics: Dict[str, float]
482 ) -> Dict[str, Any]:
483     """Compare current model with baseline"""
484
485     improvements = {}
486     for metric, current_value in current_metrics.items():
487         if metric in baseline_metrics:
488             baseline_value = baseline_metrics[metric]
489             improvement = current_value - baseline_value
490             improvements[metric] = {
491                 'current': current_value,
492                 'baseline': baseline_value,
493                 'improvement': improvement,
494                 'pct_improvement': (improvement / baseline_value) * 100
495             }
496
497     # Overall assessment
498     is_improvement = all(
499         imp['improvement'] > -0.01 # Allow small degradation
500         for imp in improvements.values()
501     ) and any(
502         imp['improvement'] > 0.01 # Require improvement in at least one metric
503         for imp in improvements.values()
504     )
505
506     return {
507         'is_improvement': is_improvement,
508         'metrics': improvements
509     }
510
511 def _run_canary_tests(self, model_version: str) -> Dict[str, Any]:

```



```

512     """Run canary tests on new model version"""
513
514     # Simulate canary testing
515     # In production, this would involve:
516     # - Deploying to small percentage of traffic
517     # - Monitoring metrics for N hours
518     # - Comparing with baseline
519
520     return {
521         'passed': True,
522         'duration_hours': 1,
523         'traffic_percentage': 10,
524         'metrics': {
525             'error_rate': 0.001,
526             'latency_p99': 45,
527             'throughput': 1000
528         }
529     }
530
531     def _archive_old_production_model(self):
532         """Archive the current production model"""
533
534         client = mlflow.tracking.MlflowClient()
535
536         # Get current production model
537         production_models = client.get_latest_versions(
538             self.model_registry_name,
539             stages=["Production"]
540         )
541
542         for model in production_models:
543             client.transition_model_version_stage(
544                 name=self.model_registry_name,
545                 version=model.version,
546                 stage="Archived"
547             )
548
549             self.logger.info(
550                 f"Archived old production model version {model.version}"
551             )
552
553     # Script to run continuous training
554     # scripts/run_continuous_training.py
555     """Script to run continuous training pipeline"""
556
557     import argparse
558     import logging
559     import yaml
560     from src.pipelines.continuous_training import (
561         ContinuousTrainingPipeline,
562         TrainingConfig
563     )
564
565     def main():
566         parser = argparse.ArgumentParser(
567             description="Run continuous training pipeline"
568         )
569         parser.add_argument(

```

```

570     '--config',
571     required=True,
572     help='Path to training configuration file'
573 )
574 parser.add_argument(
575     '--mlflow-uri',
576     default='http://localhost:5000',
577     help='MLflow tracking URI'
578 )
579 parser.add_argument(
580     '--data-source',
581     required=True,
582     help='Data source URI'
583 )
584 parser.add_argument(
585     '--model-name',
586     default='production_model',
587     help='Model registry name'
588 )
589
590 args = parser.parse_args()
591
592 # Setup logging
593 logging.basicConfig(
594     level=logging.INFO,
595     format='%(asctime)s - %(name)s - %(levelname)s - %(message)s'
596 )
597
598 # Load configuration
599 with open(args.config) as f:
600     config_dict = yaml.safe_load(f)
601
602 config = TrainingConfig(**config_dict)
603
604 # Create and run pipeline
605 pipeline = ContinuousTrainingPipeline(
606     config=config,
607     mlflow_tracking_uri=args.mlflow_uri,
608     data_source=args.data_source,
609     model_registry_name=args.model_name
610 )
611
612 result = pipeline.run()
613
614 print(f"\nPipeline completed with status: {result['status']}")
615
616 if result['status'] == 'completed':
617     if result['deployment']['deployed']:
618         print(f"New model version {result['deployment']['version']} "
619               f"deployed to production")
620     else:
621         print(f"Model not deployed: {result['deployment']['reason']}")
622
623 if __name__ == '__main__':
624     main()

```

Listing 17.5: Production Continuous Training Pipeline

17.6 Model Validation and Deployment Automation

17.6.1 Automated Model Validation Gates

Before deploying models to production, they must pass comprehensive validation gates that ensure quality, safety, and compliance.

```

1  # src/validation/model_validator.py
2  """Automated model validation with configurable gates"""
3
4  from typing import Dict, List, Any, Optional
5  from dataclasses import dataclass, field
6  from enum import Enum
7  import pandas as pd
8  import numpy as np
9  import logging
10
11 class ValidationStatus(Enum):
12     """Status of validation check"""
13     PASSED = "passed"
14     FAILED = "failed"
15     WARNING = "warning"
16     SKIPPED = "skipped"
17
18 @dataclass
19 class ValidationResult:
20     """Result of a single validation check"""
21     check_name: str
22     status: ValidationStatus
23     message: str
24     details: Dict[str, Any] = field(default_factory=dict)
25     severity: str = "medium" # low, medium, high, critical
26
27 @dataclass
28 class ModelValidationReport:
29     """Complete validation report"""
30     model_id: str
31     timestamp: str
32     overall_status: ValidationStatus
33     checks: List[ValidationResult]
34     blocking_issues: List[str] = field(default_factory=list)
35     warnings: List[str] = field(default_factory=list)
36
37 class ModelValidator:
38     """
39     Comprehensive model validation system
40
41     Performs multiple validation checks before model deployment
42     """
43
44     def __init__(self, validation_config: Dict[str, Any]):
45         self.config = validation_config
46         self.logger = logging.getLogger(__name__)
47
48     def validate_model(
49         self,
50         model: Any,
51         test_data: pd.DataFrame,

```

```

52     metadata: Dict[str, Any]
53 ) -> ModelValidationReport:
54     """
55     Run all validation checks on model
56
57     Returns validation report with pass/fail status
58     """
59
60     self.logger.info(f"Starting validation for model {metadata.get('model_id')}")
61
62     checks = []
63
64     # Performance validation
65     perf_result = self._validate_performance(model, test_data)
66     checks.append(perf_result)
67
68     # Fairness validation
69     if 'sensitive_features' in self.config:
70         fairness_result = self._validate_fairness(
71             model,
72             test_data,
73             self.config['sensitive_features']
74         )
75         checks.append(fairness_result)
76
77     # Robustness validation
78     robustness_result = self._validate_robustness(model, test_data)
79     checks.append(robustness_result)
80
81     # Latency validation
82     latency_result = self._validate_latency(model, test_data)
83     checks.append(latency_result)
84
85     # Explainability validation
86     if self.config.get('require_explainability', False):
87         explainability_result = self._validate_explainability(model)
88         checks.append(explainability_result)
89
90     # Schema validation
91     schema_result = self._validate_input_schema(model, test_data)
92     checks.append(schema_result)
93
94     # Dependency validation
95     dependency_result = self._validate_dependencies(metadata)
96     checks.append(dependency_result)
97
98     # Determine overall status
99     blocking_issues = [
100         c.check_name for c in checks
101         if c.status == ValidationStatus.FAILED and c.severity in ['high', '
critical']
102     ]
103
104     warnings = [
105         c.check_name for c in checks
106         if c.status == ValidationStatus.WARNING or
107         (c.status == ValidationStatus.FAILED and c.severity == 'medium')
108     ]

```

```

109         if blocking_issues:
110             overall_status = ValidationStatus.FAILED
111         elif any(c.status == ValidationStatus.FAILED for c in checks):
112             overall_status = ValidationStatus.WARNING
113         else:
114             overall_status = ValidationStatus.PASSED
115
116     report = ModelValidationReport(
117         model_id=metadata.get('model_id', 'unknown'),
118         timestamp=datetime.now().isoformat(),
119         overall_status=overall_status,
120         checks=checks,
121         blocking_issues=blocking_issues,
122         warnings=warnings
123     )
124
125     self.logger.info(f"Validation complete. Status: {overall_status.value}")
126
127     return report
128
129 def _validate_performance(
130     self,
131     model: Any,
132     test_data: pd.DataFrame
133 ) -> ValidationResult:
134     """Validate model meets performance requirements"""
135
136     X = test_data.drop('target', axis=1)
137     y_true = test_data['target']
138
139     y_pred = model.predict(X)
140
141     from sklearn.metrics import accuracy_score, f1_score
142
143     accuracy = accuracy_score(y_true, y_pred)
144     f1 = f1_score(y_true, y_pred, average='weighted')
145
146     # Check against thresholds
147     min_accuracy = self.config.get('min_accuracy', 0.80)
148     min_f1 = self.config.get('min_f1', 0.75)
149
150     passed = accuracy >= min_accuracy and f1 >= min_f1
151
152     return ValidationResult(
153         check_name="performance",
154         status=ValidationStatus.PASSED if passed else ValidationStatus.FAILED,
155         message=(
156             f"Accuracy: {accuracy:.3f} (>= {min_accuracy}), "
157             f"F1: {f1:.3f} (>= {min_f1})"
158         ),
159         details={'accuracy': accuracy, 'f1_score': f1},
160         severity='critical'
161     )
162
163 def _validate_fairness(
164     self,
165     model: Any,

```

```

167     test_data: pd.DataFrame,
168     sensitive_features: List[str]
169 ) -> ValidationResult:
170     """Validate model fairness across protected groups"""
171
172     X = test_data.drop(['target'] + sensitive_features, axis=1)
173     y_true = test_data['target']
174
175     y_pred = model.predict(X)
176
177     # Check demographic parity for each sensitive feature
178     fairness_violations = []
179
180     for feature in sensitive_features:
181         groups = test_data[feature].unique()
182
183         rates = {}
184         for group in groups:
185             mask = test_data[feature] == group
186             rates[group] = y_pred[mask].mean()
187
188         max_diff = max(rates.values()) - min(rates.values())
189
190         threshold = self.config.get('fairness_threshold', 0.1)
191
192         if max_diff > threshold:
193             fairness_violations.append(
194                 f"{feature}: max difference {max_diff:.3f}"
195             )
196
197     passed = len(fairness_violations) == 0
198
199     return ValidationResult(
200         check_name="fairness",
201         status=ValidationStatus.PASSED if passed else ValidationStatus.WARNING,
202         message=(
203             "Fairness check passed" if passed
204             else f"Fairness violations: {', '.join(fairness_violations)}"
205         ),
206         details={'violations': fairness_violations},
207         severity='high'
208     )
209
210 def _validate_robustness(
211     self,
212     model: Any,
213     test_data: pd.DataFrame
214 ) -> ValidationResult:
215     """Validate model robustness to perturbations"""
216
217     X = test_data.drop('target', axis=1)
218     y_true = test_data['target']
219
220     baseline_pred = model.predict(X)
221
222     # Add noise
223     noise_std = X.std() * 0.1
224     X_noisy = X + np.random.randn(*X.shape) * noise_std.values

```

```

225     noisy_pred = model.predict(X_noisy)
226
227     # Check prediction consistency
228     consistency = (baseline_pred == noisy_pred).mean()
229
230     threshold = self.config.get('robustness_threshold', 0.90)
231     passed = consistency >= threshold
232
233     return ValidationResult(
234         check_name="robustness",
235         status=ValidationStatus.PASSED if passed else ValidationStatus.WARNING,
236         message=f"Prediction consistency: {consistency:.3f} (>= {threshold})",
237         details={'consistency': consistency},
238         severity='medium'
239     )
240
241
242     def _validate_latency(
243         self,
244         model: Any,
245         test_data: pd.DataFrame
246     ) -> ValidationResult:
247         """Validate inference latency"""
248
249         import time
250
251         X = test_data.drop('target', axis=1).head(100)
252
253         # Warmup
254         _ = model.predict(X)
255
256         # Measure
257         times = []
258         for _ in range(10):
259             start = time.perf_counter()
260             _ = model.predict(X)
261             elapsed = (time.perf_counter() - start) * 1000
262             times.append(elapsed)
263
264         p95_latency = np.percentile(times, 95)
265
266         max_latency = self.config.get('max_latency_ms', 100)
267         passed = p95_latency < max_latency
268
269         return ValidationResult(
270             check_name="latency",
271             status=ValidationStatus.PASSED if passed else ValidationStatus.FAILED,
272             message=f"P95 latency: {p95_latency:.1f}ms (< {max_latency}ms)",
273             details={'p95_latency_ms': p95_latency},
274             severity='high'
275         )
276
277     def _validate_explainability(self, model: Any) -> ValidationResult:
278         """Validate model explainability"""
279
280         # Check if model supports explainability
281         has_feature_importance = hasattr(model, 'feature_importances_')
282         has_coef = hasattr(model, 'coef_')

```

```

283     explainable = has_feature_importance or has_coef
284
285
286     return ValidationResult(
287         check_name="explainability",
288         status=ValidationStatus.PASSED if explainable else ValidationStatus.
WARNING,
289         message="Model provides explainability" if explainable else "Model lacks
explainability",
290         details={'explainable': explainable},
291         severity='low'
292     )
293
294     def _validate_input_schema(
295         self,
296         model: Any,
297         test_data: pd.DataFrame
298     ) -> ValidationResult:
299         """Validate input schema matches expectations"""
300
301         X = test_data.drop('target', axis=1)
302
303         try:
304             # Test prediction with expected schema
305             _ = model.predict(X.head(1))
306
307             return ValidationResult(
308                 check_name="schema",
309                 status=ValidationStatus.PASSED,
310                 message="Input schema validation passed",
311                 details={},
312                 severity='critical'
313             )
314
315         except Exception as e:
316             return ValidationResult(
317                 check_name="schema",
318                 status=ValidationStatus.FAILED,
319                 message=f"Schema validation failed: {str(e)}",
320                 details={'error': str(e)},
321                 severity='critical'
322             )
323
324     def _validate_dependencies(self, metadata: Dict) -> ValidationResult:
325         """Validate model dependencies are documented"""
326
327         has_dependencies = 'dependencies' in metadata
328         has_version = 'model_version' in metadata
329
330         passed = has_dependencies and has_version
331
332         return ValidationResult(
333             check_name="dependencies",
334             status=ValidationStatus.PASSED if passed else ValidationStatus.WARNING,
335             message="Dependencies documented" if passed else "Missing dependency
information",
336             details=metadata.get('dependencies', {}),
337             severity='low'

```



```

338     )
339
340     # Integration with CI/CD
341     # scripts/validate_model_for_deployment.py
342     """Validate model before deployment"""
343
344     import argparse
345     import json
346     import yaml
347     import mlflow
348     from src.validation.model_validator import ModelValidator, ValidationStatus
349
350     def main():
351         parser = argparse.ArgumentParser()
352         parser.add_argument('--model-uri', required=True)
353         parser.add_argument('--test-data', required=True)
354         parser.add_argument('--config', required=True)
355         parser.add_argument('--output', default='validation_report.json')
356
357         args = parser.parse_args()
358
359         # Load validation configuration
360         with open(args.config) as f:
361             config = yaml.safe_load(f)
362
363         # Load model
364         model = mlflow.sklearn.load_model(args.model_uri)
365
366         # Load test data
367         test_data = pd.read_csv(args.test_data)
368
369         # Run validation
370         validator = ModelValidator(config)
371         report = validator.validate_model(
372             model,
373             test_data,
374             metadata={'model_uri': args.model_uri}
375         )
376
377         # Save report
378         report_dict = {
379             'overall_status': report.overall_status.value,
380             'checks': [
381                 {
382                     'name': c.check_name,
383                     'status': c.status.value,
384                     'message': c.message,
385                     'details': c.details
386                 }
387                 for c in report.checks
388             ],
389             'blocking_issues': report.blocking_issues,
390             'warnings': report.warnings
391         }
392
393         with open(args.output, 'w') as f:
394             json.dump(report_dict, f, indent=2)
395

```

```

396     # Exit with appropriate code
397     if report.overall_status == ValidationStatus.FAILED:
398         print(f"Validation FAILED: {'', '.join(report.blocking_issues)}")
399         exit(1)
400     elif report.overall_status == ValidationStatus.WARNING:
401         print(f"Validation passed with warnings: {'', '.join(report.warnings)}")
402         exit(0)
403     else:
404         print("Validation PASSED")
405         exit(0)
406
407 if __name__ == '__main__':
408     main()

```

Listing 17.6: Automated Model Validation System

17.7 Integration with Model Registry and Monitoring

17.7.1 Model Registry Integration

```

1  # src/registry/model_registry_client.py
2  """Model registry client for CI/CD pipelines"""
3
4  from typing import Dict, List, Optional, Any
5  from dataclasses import dataclass
6  from datetime import datetime
7  import mlflow
8  from mlflow.tracking import MlflowClient
9  import logging
10
11 @dataclass
12 class ModelVersion:
13     """Model version information"""
14     name: str
15     version: str
16     stage: str
17     creation_timestamp: datetime
18     tags: Dict[str, str]
19     description: Optional[str] = None
20
21 class MLModelRegistry:
22     """
23     Integration with MLflow Model Registry
24
25     Manages model lifecycle from staging to production
26     """
27
28     def __init__(self, tracking_uri: str):
29         mlflow.set_tracking_uri(tracking_uri)
30         self.client = MlflowClient()
31         self.logger = logging.getLogger(__name__)
32
33     def register_model(
34         self,
35         run_id: str,
36         model_name: str,

```

```

37     tags: Optional[Dict[str, str]] = None,
38     description: Optional[str] = None
39 ) -> ModelVersion:
40     """Register new model version"""
41
42     model_uri = f"runs:{run_id}/model"
43
44     # Register model
45     mv = mlflow.register_model(model_uri, model_name)
46
47     # Add tags
48     if tags:
49         for key, value in tags.items():
50             self.client.set_model_version_tag(
51                 name=model_name,
52                 version=mv.version,
53                 key=key,
54                 value=value
55             )
56
57     # Add description
58     if description:
59         self.client.update_model_version(
60             name=model_name,
61             version=mv.version,
62             description=description
63         )
64
65     self.logger.info(
66         f"Registered model {model_name} version {mv.version}"
67     )
68
69     return ModelVersion(
70         name=model_name,
71         version=mv.version,
72         stage="None",
73         creation_timestamp=datetime.fromtimestamp(mv.creation_timestamp / 1000),
74         tags=tags or {},
75         description=description
76     )
77
78 def promote_model(
79     self,
80     model_name: str,
81     version: str,
82     stage: str,
83     archive_existing: bool = True
84 ):
85     """Promote model to new stage"""
86
87     if archive_existing and stage == "Production":
88         # Archive existing production models
89         prod_models = self.client.get_latest_versions(
90             model_name,
91             stages=["Production"]
92         )
93
94         for model in prod_models:

```

```

95         self.client.transition_model_version_stage(
96             name=model_name,
97             version=model.version,
98             stage="Archived"
99         )
100         self.logger.info(
101             f"Archived previous production version {model.version}"
102         )
103
104         # Promote new version
105         self.client.transition_model_version_stage(
106             name=model_name,
107             version=version,
108             stage=stage
109         )
110
111         self.logger.info(
112             f"Promoted model {model_name} version {version} to {stage}"
113         )
114
115     def get_production_model(self, model_name: str) -> Optional[ModelVersion]:
116         """Get current production model version"""
117
118         prod_models = self.client.get_latest_versions(
119             model_name,
120             stages=["Production"]
121         )
122
123         if not prod_models:
124             return None
125
126         model = prod_models[0]
127
128         return ModelVersion(
129             name=model.name,
130             version=model.version,
131             stage=model.current_stage,
132             creation_timestamp=datetime.fromtimestamp(model.creation_timestamp / 1000)
133         ,
134             tags=model.tags
135         )
136
137     def compare_models(
138         self,
139         model_name: str,
140         version1: str,
141         version2: str,
142         metric_names: List[str]
143     ) -> Dict[str, Any]:
144         """Compare metrics between two model versions"""
145
146         # Get runs for both versions
147         mv1 = self.client.get_model_version(model_name, version1)
148         mv2 = self.client.get_model_version(model_name, version2)
149
150         run1 = self.client.get_run(mv1.run_id)
151         run2 = self.client.get_run(mv2.run_id)

```

```

152     comparison = {}
153
154     for metric in metric_names:
155         val1 = run1.data.metrics.get(metric)
156         val2 = run2.data.metrics.get(metric)
157
158         if val1 is not None and val2 is not None:
159             comparison[metric] = {
160                 f'version_{version1}': val1,
161                 f'version_{version2}': val2,
162                 'difference': val2 - val1,
163                 'pct_change': ((val2 - val1) / val1) * 100 if val1 != 0 else 0
164             }
165
166     return comparison

```

Listing 17.7: Model Registry Integration for CI/CD

17.8 Infrastructure as Code for ML CI/CD

Managing ML infrastructure as code ensures reproducibility, version control, and automated provisioning of resources needed for CI/CD pipelines.

17.8.1 Terraform for ML Infrastructure

```

1  # infrastructure/main.tf
2  """
3  Terraform configuration for ML CI/CD infrastructure
4  Provisions compute, storage, and networking resources
5  """
6
7  terraform {
8      required_version = ">= 1.0"
9
10     required_providers {
11         aws = {
12             source = "hashicorp/aws"
13             version = "~> 4.0"
14         }
15     }
16
17     backend "s3" {
18         bucket = "ml-terraform-state"
19         key    = "ml-cicd/terraform.tfstate"
20         region = "us-west-2"
21     }
22 }
23
24 provider "aws" {
25     region = var.aws_region
26 }
27
28 # S3 bucket for model artifacts
29 resource "aws_s3_bucket" "model_artifacts" {
30     bucket = "${var.project_name}-model-artifacts-${var.environment}"

```

```

31     tags = {
32         Environment = var.environment
33         Purpose     = "ML Model Storage"
34         ManagedBy   = "Terraform"
35     }
36 }
37 }
38
39 resource "aws_s3_bucket_versioning" "model_artifacts_versioning" {
40     bucket = aws_s3_bucket.model_artifacts.id
41
42     versioning_configuration {
43         status = "Enabled"
44     }
45 }
46
47 resource "aws_s3_bucket_lifecycle_configuration" "model_artifacts_lifecycle" {
48     bucket = aws_s3_bucket.model_artifacts.id
49
50     rule {
51         id      = "archive_old_models"
52         status  = "Enabled"
53
54         transition {
55             days      = 90
56             storage_class = "GLACIER"
57         }
58
59         expiration {
60             days = 365
61         }
62     }
63 }
64
65 # ECR repository for ML container images
66 resource "aws_ecr_repository" "ml_training" {
67     name                = "${var.project_name}-ml-training"
68     image_tag_mutability = "MUTABLE"
69
70     image_scanning_configuration {
71         scan_on_push = true
72     }
73
74     tags = {
75         Environment = var.environment
76         Purpose     = "ML Training Images"
77     }
78 }
79
80 resource "aws_ecr_lifecycle_policy" "ml_training_lifecycle" {
81     repository = aws_ecr_repository.ml_training.name
82
83     policy = jsonencode({
84         rules = [{
85             rulePriority = 1
86             description  = "Keep last 10 images"
87             selection = {
88                 tagStatus = "any"

```

```

89     countType      = "imageCountMoreThan"
90     countNumber    = 10
91   }
92   action = {
93     type = "expire"
94   }
95 }]]
96 })
97 }
98
99 # SageMaker notebook instance for experimentation
100 resource "aws_sagemaker_notebook_instance" "ml_dev" {
101   name          = "${var.project_name}-ml-notebook-${var.environment}"
102   role_arn       = aws_iam_role.sagemaker_role.arn
103   instance_type = var.notebook_instance_type
104
105   tags = {
106     Environment = var.environment
107     Purpose     = "ML Development"
108   }
109 }
110
111 # IAM role for SageMaker
112 resource "aws_iam_role" "sagemaker_role" {
113   name = "${var.project_name}-sagemaker-role-${var.environment}"
114
115   assume_role_policy = jsonencode({
116     Version = "2012-10-17"
117     Statement = [{
118       Action = "sts:AssumeRole"
119       Effect = "Allow"
120       Principal = {
121         Service = "sagemaker.amazonaws.com"
122       }
123     }]
124   })
125 }
126
127 resource "aws_iam_role_policy_attachment" "sagemaker_full_access" {
128   role          = aws_iam_role.sagemaker_role.name
129   policy_arn    = "arn:aws:iam::aws:policy/AmazonSageMakerFullAccess"
130 }
131
132 # ECS cluster for model serving
133 resource "aws_ecs_cluster" "ml_serving" {
134   name = "${var.project_name}-ml-serving-${var.environment}"
135
136   setting {
137     name  = "containerInsights"
138     value = "enabled"
139   }
140
141   tags = {
142     Environment = var.environment
143     Purpose     = "ML Model Serving"
144   }
145 }
146

```

```

147 resource "aws_ecs_cluster_capacity_providers" "ml_serving" {
148     cluster_name = aws_ecs_cluster.ml_serving.name
149
150     capacity_providers = ["FARGATE", "FARGATE_SPOT"]
151
152     default_capacity_provider_strategy {
153         capacity_provider = "FARGATE_SPOT"
154         weight            = 1
155     }
156 }
157
158 # Application Load Balancer for model endpoint
159 resource "aws_lb" "ml_api" {
160     name           = "${var.project_name}-ml-api-${var.environment}"
161     internal       = false
162     load_balancer_type = "application"
163     security_groups = [aws_security_group.ml_api_lb.id]
164     subnets       = var.public_subnet_ids
165
166     enable_deletion_protection = var.environment == "production"
167
168     tags = {
169         Environment = var.environment
170         Purpose     = "ML API Load Balancer"
171     }
172 }
173
174 resource "aws_lb_target_group" "ml_api" {
175     name       = "${var.project_name}-ml-api-tg-${var.environment}"
176     port       = 8080
177     protocol   = "HTTP"
178     vpc_id     = var.vpc_id
179     target_type = "ip"
180
181     health_check {
182         enabled            = true
183         healthy_threshold  = 2
184         interval           = 30
185         matcher            = "200"
186         path               = "/health"
187         protocol           = "HTTP"
188         timeout            = 5
189         unhealthy_threshold = 3
190     }
191
192     deregistration_delay = 30
193 }
194
195 # CloudWatch Log Group for ML logs
196 resource "aws_cloudwatch_log_group" "ml_logs" {
197     name           = "/aws/ml/${var.project_name}/${var.environment}"
198     retention_in_days = var.log_retention_days
199
200     tags = {
201         Environment = var.environment
202         Purpose     = "ML System Logs"
203     }
204 }

```



```

205
206 # RDS for metadata storage (MLflow tracking)
207 resource "aws_db_instance" "mlflow_backend" {
208     identifier      = "${var.project_name}-mlflow-db-${var.environment}"
209     engine          = "postgres"
210     engine_version = "14.7"
211     instance_class = var.db_instance_type
212
213     allocated_storage      = 100
214     max_allocated_storage = 1000
215     storage_encrypted      = true
216
217     db_name = "mlflow"
218     username = var.db_username
219     password = var.db_password
220
221     backup_retention_period = var.environment == "production" ? 30 : 7
222     skip_final_snapshot    = var.environment != "production"
223
224     vpc_security_group_ids = [aws_security_group.mlflow_db.id]
225     db_subnet_group_name   = aws_db_subnet_group.mlflow.name
226
227     tags = {
228         Environment = var.environment
229         Purpose     = "MLflow Tracking Backend"
230     }
231 }
232
233 # Outputs for use in CI/CD
234 output "model_artifacts_bucket" {
235     description = "S3 bucket for model artifacts"
236     value       = aws_s3_bucket.model_artifacts.id
237 }
238
239 output "ecr_repository_url" {
240     description = "ECR repository URL for ML images"
241     value       = aws_ecr_repository.ml_training.repository_url
242 }
243
244 output "ecs_cluster_name" {
245     description = "ECS cluster for model serving"
246     value       = aws_ecs_cluster.ml_serving.name
247 }
248
249 output "load_balancer_dns" {
250     description = "Load balancer DNS for ML API"
251     value       = aws_lb.ml_api.dns_name
252 }
253
254 output "mlflow_db_endpoint" {
255     description = "MLflow database endpoint"
256     value       = aws_db_instance.mlflow_backend.endpoint
257     sensitive   = true
258 }

```

Listing 17.8: Terraform Configuration for ML Infrastructure

17.8.2 GitHub Actions with Terraform

```

1  # .github/workflows/infrastructure.yml
2  name: Infrastructure Provisioning
3
4  on:
5    push:
6      branches: [main]
7      paths:
8        - 'infrastructure/**'
9    pull_request:
10     paths:
11       - 'infrastructure/**'
12 workflow_dispatch:
13   inputs:
14     environment:
15       description: 'Environment to deploy'
16       required: true
17       type: choice
18       options:
19         - development
20         - staging
21         - production
22
23 env:
24   TF_VERSION: '1.5.0'
25   AWS_REGION: 'us-west-2'
26
27 jobs:
28   terraform-plan:
29     runs-on: ubuntu-latest
30     environment: ${ github.event.inputs.environment || 'development' }
31
32     steps:
33       - name: Checkout code
34         uses: actions/checkout@v3
35
36       - name: Setup Terraform
37         uses: hashicorp/setup-terraform@v2
38         with:
39           terraform_version: ${ env.TF_VERSION }
40
41       - name: Configure AWS Credentials
42         uses: aws-actions/configure-aws-credentials@v2
43         with:
44           aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
45           aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
46           aws-region: ${ env.AWS_REGION }
47
48       - name: Terraform Format Check
49         working-directory: ./infrastructure
50         run: terraform fmt -check -recursive
51
52       - name: Terraform Init
53         working-directory: ./infrastructure
54         run: |
55           terraform init \
56             -backend-config="bucket=ml-terraform-state" \

```

```

57     -backend-config="key=${{ github.event.inputs.environment }}/terraform.
tfstate"
58
59     - name: Terraform Validate
60       working-directory: ./infrastructure
61       run: terraform validate
62
63     - name: Terraform Plan
64       working-directory: ./infrastructure
65       run: |
66         terraform plan \
67           -var="environment=${{ github.event.inputs.environment }}" \
68           -var="project_name=${{ github.event.repository.name }}" \
69           -out=tfplan
70
71     - name: Upload Plan
72       uses: actions/upload-artifact@v3
73       with:
74         name: terraform-plan
75         path: infrastructure/tfplan
76
77     - name: Cost Estimation
78       uses: infracost/actions/setup@v2
79       with:
80         api-key: ${ secrets.INFRACOST_API_KEY }
81
82     - name: Generate Cost Estimate
83       working-directory: ./infrastructure
84       run: |
85         infracost breakdown \
86           --path tfplan \
87           --format json \
88           --out-file /tmp/infracost.json
89
90         infracost output \
91           --path /tmp/infracost.json \
92           --format table
93
94     terraform-apply:
95       needs: terraform-plan
96       if: github.event_name != 'pull_request'
97       runs-on: ubuntu-latest
98       environment: ${{ github.event.inputs.environment || 'development' }}
99
100     steps:
101     - name: Checkout code
102       uses: actions/checkout@v3
103
104     - name: Setup Terraform
105       uses: hashicorp/setup-terraform@v2
106       with:
107         terraform_version: ${ env.TF_VERSION }
108
109     - name: Configure AWS Credentials
110       uses: aws-actions/configure-aws-credentials@v2
111       with:
112         aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
113         aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }

```

```

114     aws-region: ${ env.AWS_REGION }}
115
116   - name: Download Plan
117     uses: actions/download-artifact@v3
118     with:
119       name: terraform-plan
120       path: infrastructure/
121
122   - name: Terraform Init
123     working-directory: ./infrastructure
124     run: |
125       terraform init \
126         -backend-config="bucket=ml-terraform-state" \
127         -backend-config="key=${ github.event.inputs.environment }}/terraform.
128 tfstate"
129
130   - name: Terraform Apply
131     working-directory: ./infrastructure
132     run: terraform apply -auto-approve tfplan
133
134   - name: Export Outputs
135     working-directory: ./infrastructure
136     run: |
137       terraform output -json > terraform-outputs.json
138
139   - name: Update GitHub Secrets
140     env:
141       GH_TOKEN: ${ secrets.GITHUB_TOKEN }}
142     run: |
143       # Extract outputs and update repository secrets
144       BUCKET=$(terraform -chdir=infrastructure output -raw model_artifacts_bucket)
145       gh secret set MODEL_ARTIFACTS_BUCKET --body "$BUCKET"
146
147       ECR_URL=$(terraform -chdir=infrastructure output -raw ecr_repository_url)
148       gh secret set ECR_REPOSITORY_URL --body "$ECR_URL"
149
150       CLUSTER=$(terraform -chdir=infrastructure output -raw ecs_cluster_name)
151       gh secret set ECS_CLUSTER_NAME --body "$CLUSTER"

```

Listing 17.9: Infrastructure Provisioning in CI/CD

17.8.3 Docker Compose for Local Development

```

1  # docker-compose.yml
2  version: '3.8'
3
4  services:
5    # MLflow tracking server
6    mlflow:
7      image: ghcr.io/mlflow/mlflow:latest
8      container_name: mlflow-server
9      ports:
10       - "5000:5000"
11      environment:
12       - BACKEND_STORE_URI=postgresql://mlflow:mlflow@postgres:5432/mlflow
13       - DEFAULT_ARTIFACT_ROOT=s3://mlflow-artifacts
14       - AWS_ACCESS_KEY_ID=${AWS_ACCESS_KEY_ID}

```

```

15     - AWS_SECRET_ACCESS_KEY=${AWS_SECRET_ACCESS_KEY}
16     command: >
17         mlflow server
18         --backend-store-uri postgresql://mlflow:mlflow@postgres:5432/mlflow
19         --default-artifact-root s3://mlflow-artifacts
20         --host 0.0.0.0
21         --port 5000
22     depends_on:
23         - postgres
24     networks:
25         - ml-network
26
27     # PostgreSQL for MLflow backend
28     postgres:
29         image: postgres:14
30         container_name: mlflow-postgres
31         environment:
32             - POSTGRES_USER=mlflow
33             - POSTGRES_PASSWORD=mlflow
34             - POSTGRES_DB=mlflow
35         volumes:
36             - postgres-data:/var/lib/postgresql/data
37         ports:
38             - "5432:5432"
39         networks:
40             - ml-network
41         healthcheck:
42             test: ["CMD-SHELL", "pg_isready -U mlflow"]
43             interval: 10s
44             timeout: 5s
45             retries: 5
46
47     # Redis for caching and task queue
48     redis:
49         image: redis:7-alpine
50         container_name: ml-redis
51         ports:
52             - "6379:6379"
53         volumes:
54             - redis-data:/data
55         networks:
56             - ml-network
57         command: redis-server --appendonly yes
58
59     # Jupyter notebook for development
60     jupyter:
61         build:
62             context: .
63             dockerfile: Dockerfile.jupyter
64         container_name: ml-jupyter
65         ports:
66             - "8888:8888"
67         environment:
68             - JUPYTER_ENABLE_LAB=yes
69             - MLFLOW_TRACKING_URI=http://mlflow:5000
70         volumes:
71             - ./notebooks:/home/jovyan/notebooks
72             - ./data:/home/jovyan/data

```

```

73     - ./src:/home/jovyan/src
74     networks:
75     - ml-network
76     depends_on:
77     - mlflow
78
79     # Model serving API
80     model-api:
81     build:
82     context: .
83     dockerfile: Dockerfile.api
84     container_name: ml-model-api
85     ports:
86     - "8080:8080"
87     environment:
88     - MLFLOW_TRACKING_URI=http://mlflow:5000
89     - MODEL_NAME=my-model
90     - MODEL_VERSION=latest
91     - REDIS_HOST=redis
92     - REDIS_PORT=6379
93     networks:
94     - ml-network
95     depends_on:
96     - mlflow
97     - redis
98     healthcheck:
99     test: ["CMD", "curl", "-f", "http://localhost:8080/health"]
100    interval: 30s
101    timeout: 10s
102    retries: 3
103
104    # Prometheus for metrics
105    prometheus:
106    image: prom/prometheus:latest
107    container_name: ml-prometheus
108    ports:
109    - "9090:9090"
110    volumes:
111    - ./monitoring/prometheus.yml:/etc/prometheus/prometheus.yml
112    - prometheus-data:/prometheus
113    command:
114    - '--config.file=/etc/prometheus/prometheus.yml'
115    - '--storage.tsdb.path=/prometheus'
116    networks:
117    - ml-network
118
119    # Grafana for visualization
120    grafana:
121    image: grafana/grafana:latest
122    container_name: ml-grafana
123    ports:
124    - "3000:3000"
125    environment:
126    - GF_SECURITY_ADMIN_PASSWORD=admin
127    - GF_USERS_ALLOW_SIGN_UP=false
128    volumes:
129    - grafana-data:/var/lib/grafana
130    - ./monitoring/grafana-dashboards:/etc/grafana/provisioning/dashboards

```

```

131     networks:
132       - ml-network
133     depends_on:
134       - prometheus
135
136 volumes:
137   postgres-data:
138   redis-data:
139   prometheus-data:
140   grafana-data:
141
142 networks:
143   ml-network:
144     driver: bridge

```

Listing 17.10: Docker Compose for Local ML Environment

Infrastructure Best Practices:

- **Environment Parity:** Keep dev, staging, and production as similar as possible
- **Immutable Infrastructure:** Recreate instead of modifying existing resources
- **Cost Optimization:** Use spot instances for training, auto-scaling for serving
- **Security:** Encrypt at rest and in transit, use IAM roles instead of keys
- **Monitoring:** Instrument all infrastructure with CloudWatch/Prometheus
- **Disaster Recovery:** Regular backups, multi-region for production
- **Cost Tracking:** Tag all resources, use Infracost for budget awareness

17.9 Security Scanning and Compliance

17.9.1 Security and Compliance Checks

```

1  # .github/workflows/security-scan.yml
2  name: Security and Compliance Scan
3
4  on:
5    pull_request:
6    push:
7      branches: [main, develop]
8    schedule:
9      - cron: '0 0 * * 0' # Weekly
10
11 jobs:
12   dependency-security-scan:
13     runs-on: ubuntu-latest
14
15     steps:
16       - name: Checkout code
17         uses: actions/checkout@v3
18
19       - name: Set up Python

```

```

20     uses: actions/setup-python@v4
21     with:
22       python-version: '3.9'
23
24   - name: Install dependencies
25     run: |
26     pip install -r requirements.txt
27     pip install safety bandit pip-audit
28
29   - name: Run Safety check
30     run: |
31     safety check --json --output safety-report.json
32     continue-on-error: true
33
34   - name: Run pip-audit
35     run: |
36     pip-audit --format json --output pip-audit-report.json
37     continue-on-error: true
38
39   - name: Run Bandit security scan
40     run: |
41     bandit -r src/ -f json -o bandit-report.json
42     continue-on-error: true
43
44   - name: Upload security reports
45     uses: actions/upload-artifact@v3
46     with:
47       name: security-reports
48       path: |
49         safety-report.json
50         pip-audit-report.json
51         bandit-report.json
52
53 model-security-scan:
54   runs-on: ubuntu-latest
55
56   steps:
57   - name: Checkout code
58     uses: actions/checkout@v3
59
60   - name: Scan for secrets
61     uses: trufflesecurity/trufflehog@main
62     with:
63       path: ./
64       base: ${GITHUB_EVENT_REPOSITORY_DEFAULT_BRANCH}
65       head: HEAD
66
67   - name: Check for PII in data
68     run: |
69     python scripts/scan_for_pii.py \
70       --data-dir data/ \
71       --output pii-scan-results.json
72
73   - name: Validate data lineage
74     run: |
75     python scripts/validate_data_lineage.py \
76       --dvc-files data/*.dvc \
77       --output lineage-report.json

```



```

78 compliance-check:
79   runs-on: ubuntu-latest
80
81   steps:
82   - name: Checkout code
83     uses: actions/checkout@v3
84
85   - name: Check model documentation
86     run: |
87       python scripts/check_model_documentation.py \
88         --models-dir models/ \
89         --required-fields "description,metrics,dependencies,limitations"
90
91   - name: Validate model cards
92     run: |
93       python scripts/validate_model_cards.py \
94         --cards-dir docs/model_cards/
95
96   - name: Check license compliance
97     uses: pilosus/action-pip-license-checker@v2
98     with:
99       github-token: ${ secrets.GITHUB_TOKEN }
100       requirements: 'requirements.txt'
101       fail: 'StrongCopyleft,NetworkCopyleft,Error'
102
103 generate-security-report:
104   runs-on: ubuntu-latest
105   needs: [dependency-security-scan, model-security-scan, compliance-check]
106
107   steps:
108   - name: Download all reports
109     uses: actions/download-artifact@v3
110
111   - name: Generate consolidated report
112     run: |
113       python scripts/generate_security_report.py \
114         --reports-dir . \
115         --output security-summary.html
116
117   - name: Upload consolidated report
118     uses: actions/upload-artifact@v3
119     with:
120       name: security-summary
121       path: security-summary.html
122
123   - name: Comment on PR
124     if: github.event_name == 'pull_request'
125     uses: actions/github-script@v6
126     with:
127       script: |
128         const fs = require('fs');
129         const report = fs.readFileSync('security-summary.html', 'utf8');
130         github.rest.issues.createComment({
131           issue_number: context.issue.number,
132           owner: context.repo.owner,
133           repo: context.repo.repo,
134           body: '## Security Scan Results\n\n${report}'
135

```

136

});

Listing 17.11: Security Scanning in CI/CD

17.10 Practical CI/CD Implementation Exercises

17.10.1 Exercise 1: Build Complete ML CI/CD Pipeline

Objective: Implement end-to-end CI/CD for an ML project using GitHub Actions.

Requirements:

- Set up GitHub Actions workflow with all stages
- Implement unit tests for data processing and model code
- Add data validation with Great Expectations
- Configure model training with matrix strategy (3+ models)
- Implement model validation gates
- Add automated deployment to staging
- Require manual approval for production deployment

Deliverables:

- Complete '.github/workflows/ml-cicd.yml' workflow
- Test suite with >80% coverage
- Data validation suite
- Model validation configuration
- Deployment scripts for staging and production
- Documentation for running and monitoring the pipeline

17.10.2 Exercise 2: Implement Continuous Training Pipeline

Objective: Build automated continuous training with drift detection.

Requirements:

- Implement data drift detection using statistical tests
- Add performance monitoring and degradation detection
- Create retraining trigger logic (drift, performance, schedule)
- Implement automated model evaluation and comparison
- Add canary deployment with automatic rollback
- Integrate with MLflow for experiment tracking

Test Scenarios:

1. Gradual data drift over 30 days
2. Sudden performance degradation
3. Scheduled weekly retraining
4. Failed canary deployment requiring rollback

17.10.3 Exercise 3: Create Comprehensive ML Test Suite

Objective: Develop complete testing framework for ML systems.

Test Categories:

- Unit tests: Data processing, feature engineering, model training
- Integration tests: End-to-end training pipeline
- Model performance tests: Accuracy, precision, recall, F1
- Fairness tests: Demographic parity, equal opportunity
- Robustness tests: Noise resilience, outlier handling
- Latency tests: Inference speed requirements

Expected Output:

- 50+ test cases covering all categories
- Pytest configuration with fixtures
- Test coverage report >85%
- Integration with CI pipeline
- Documentation of test rationale

17.10.4 Exercise 4: Model Registry Integration

Objective: Integrate MLflow Model Registry into CI/CD pipeline.

Requirements:

- Automatically register models after successful training
- Implement model versioning with semantic versions
- Add model tagging (git commit, branch, metrics)
- Create promotion workflow: None → Staging → Production
- Implement model comparison before promotion
- Add automatic archival of old production models

Workflow:

1. Train model, log to MLflow
2. Register in model registry with metadata
3. Promote to Staging if validation passes
4. Run integration tests against Staging
5. Promote to Production with approval
6. Archive previous Production version

17.10.5 Exercise 5: Security and Compliance Automation

Objective: Implement automated security scanning and compliance checks.

Security Checks:

- Dependency vulnerability scanning (Safety, pip-audit)
- Secret scanning (TruffleHog, detect-secrets)
- Code security analysis (Bandit)
- Container image scanning
- PII detection in training data
- Model watermarking for tracking

Compliance Checks:

- Model documentation completeness
- Model card validation
- License compliance
- Data lineage tracking
- Audit trail generation

Deliverables:

- Security scanning workflow
- Compliance checking scripts
- Automated reporting
- PR comments with scan results
- Security policy documentation

17.10.6 Exercise 6: Multi-Environment Deployment Pipeline

Objective: Create deployment pipeline across dev, staging, and production.

Environments:

- Development: Automatic deployment from feature branches
- Staging: Automatic deployment from develop branch
- Production: Manual deployment from main branch

Deployment Strategy:

- Blue-green deployment for zero-downtime
- Canary deployment with gradual traffic shift
- Automatic rollback on error rate spike
- Smoke tests after each deployment
- Load testing in staging

Implementation:

- Docker containerization
- Kubernetes deployment manifests
- Environment-specific configurations
- Health check endpoints
- Monitoring integration

17.10.7 Exercise 7: Model Monitoring Integration

Objective: Integrate production monitoring into CI/CD.

Monitoring Components:

- Prediction logging and sampling
- Performance metric tracking (accuracy, latency)
- Data drift detection in production
- Alert configuration for anomalies
- Dashboard for model health
- Integration with incident management

Alert Conditions:

- Error rate $> 1\%$
- P99 latency $> 500\text{ms}$
- Throughput drop $> 20\%$
- Data drift detected
- Performance degradation $> 5\%$

17.10.8 Exercise 8: A/B Testing Framework

Objective: Build automated A/B testing for model comparison.

Framework Components:

- Traffic splitting mechanism
- Statistical significance testing
- Automatic winner selection
- Gradual rollout of winning model
- Experiment tracking and reporting

Test Scenarios:

- Compare two model architectures
- Test hyperparameter variations
- Evaluate feature engineering changes
- Measure impact of new training data

17.11 Summary

This chapter covered comprehensive CI/CD practices for machine learning systems. Key takeaways:

1. ML-Specific CI/CD Challenges:

- Large model artifacts require specialized storage and transfer
- Data dependencies complicate reproducibility
- GPU requirements necessitate custom runners
- Non-determinism requires statistical testing approaches
- Multiple stakeholders need clear handoff points

2. Complete Pipeline Components:

- Code quality: Linting, formatting, type checking, security scanning
- Testing: Unit, integration, model performance, fairness, robustness
- Data validation: Schema checks, quality metrics, drift detection
- Model training: Automated training with multiple model types
- Model validation: Performance gates, fairness checks, latency requirements
- Deployment: Staging, canary, production with rollback
- Monitoring: Performance tracking, alerting, incident response

3. GitHub Actions Best Practices:

- Use job dependencies to create clear pipeline stages
- Implement matrix builds for testing multiple configurations
- Cache dependencies to speed up workflows
- Use self-hosted runners for GPU-intensive tasks
- Separate workflows for different triggers (PR, push, schedule)
- Use environments for production deployment gates

4. Testing Strategies:

- Unit tests for data processing and feature engineering
- Integration tests for complete training pipelines
- Performance tests with thresholds for metrics
- Fairness tests across protected groups
- Robustness tests for noise and outliers
- Latency tests for inference speed

5. Continuous Training:

- Monitor data and performance for drift
- Implement intelligent retraining triggers
- Compare new models against baselines
- Use canary deployments for safe rollout
- Maintain audit trail of all changes

6. Model Registry Integration:

- Centralized model versioning with MLflow
- Stage-based promotion workflow
- Model comparison and lineage tracking
- Metadata and tag management
- Rollback capabilities

7. Security and Compliance:

- Automated vulnerability scanning
- Secret detection in code and data

- PII detection in training data
- License compliance checking
- Model documentation and cards
- Audit trails for regulatory requirements

8. Deployment Strategies:

- Blue-green for zero-downtime updates
- Canary for gradual rollout with monitoring
- A/B testing for model comparison
- Automatic rollback on failures
- Multi-environment progression

Key Success Factors:

1. **Automation:** Automate everything from testing to deployment
2. **Validation Gates:** Never deploy without passing all checks
3. **Monitoring:** Watch models in production continuously
4. **Rollback:** Always have a quick rollback strategy
5. **Documentation:** Document models, decisions, and processes
6. **Collaboration:** Design for data scientists and engineers

CI/CD for ML requires adapting traditional software practices while adding ML-specific components. The investment in robust pipelines pays off through faster iteration, fewer production issues, and greater confidence in deployments.

Chapter 18

Infrastructure as Code for ML

18.1 Learning Objectives

After completing this chapter, readers will be able to:

- Design and implement Terraform modules for ML infrastructure
- Deploy ML workloads on Kubernetes using Helm charts
- Manage cloud resources across AWS, GCP, and Azure
- Orchestrate containerized ML workloads at scale
- Implement disaster recovery and backup strategies for ML systems
- Optimize infrastructure costs through Infrastructure as Code
- Automate infrastructure provisioning and management
- Apply best practices for production ML infrastructure

18.2 Introduction

Infrastructure as Code (IaC) is the practice of managing and provisioning infrastructure through machine-readable definition files rather than manual processes. For ML systems, IaC provides reproducibility, version control, and automation of complex infrastructure requirements.

Why IaC for Machine Learning:

- **Reproducibility:** Recreate exact infrastructure configurations across environments
- **Version Control:** Track infrastructure changes alongside code changes
- **Automation:** Eliminate manual provisioning and reduce human error
- **Scalability:** Rapidly scale infrastructure up or down based on demand
- **Cost Optimization:** Programmatically manage resources to minimize costs

- **Disaster Recovery:** Quickly rebuild infrastructure from code
- **Collaboration:** Share infrastructure definitions across teams
- **Testing:** Test infrastructure changes before production deployment

ML Infrastructure Components:

1. **Compute:** GPU/CPU instances for training and inference
2. **Storage:** Data lakes, model registries, artifact storage
3. **Networking:** VPCs, load balancers, API gateways
4. **Orchestration:** Kubernetes clusters, container registries
5. **Data:** Databases, data warehouses, streaming platforms
6. **Monitoring:** Logging, metrics, alerting infrastructure
7. **Security:** IAM, secrets management, encryption

18.3 Terraform Modules for ML Infrastructure

Terraform is an open-source IaC tool that enables declarative infrastructure management across multiple cloud providers.

18.3.1 Core ML Infrastructure Module

```

1  # modules/ml-platform/main.tf
2  """
3  Root module for ML platform infrastructure
4  Orchestrates compute, storage, networking, and monitoring
5  """
6
7  terraform {
8      required_version = ">= 1.5.0"
9
10     required_providers {
11         aws = {
12             source = "hashicorp/aws"
13             version = "~> 5.0"
14         }
15         kubernetes = {
16             source = "hashicorp/kubernetes"
17             version = "~> 2.23"
18         }
19     }
20 }
21
22 # Local variables
23 locals {
24     common_tags = {
25         Project      = var.project_name
26         Environment = var.environment

```

```

27     ManagedBy    = "Terraform"
28     CostCenter   = var.cost_center
29     Owner        = var.owner_email
30 }
31
32 name_prefix = "${var.project_name}-${var.environment}"
33 }
34
35 # Data sources
36 data "aws_caller_identity" "current" {}
37 data "aws_region" "current" {}
38 data "aws_availability_zones" "available" {
39     state = "available"
40 }
41
42 # VPC Module
43 module "vpc" {
44     source = "terraform-aws-modules/vpc/aws"
45     version = "~> 5.0"
46
47     name = "${local.name_prefix}-vpc"
48     cidr = var.vpc_cidr
49
50     azs                = slice(data.aws_availability_zones.available.names, 0, 3)
51     private_subnets = [for i in range(3) : cidrsubnet(var.vpc_cidr, 8, i)]
52     public_subnets  = [for i in range(3) : cidrsubnet(var.vpc_cidr, 8, i + 100)]
53
54     enable_nat_gateway    = true
55     single_nat_gateway    = var.environment != "production"
56     enable_dns_hostnames = true
57     enable_dns_support   = true
58
59     # Enable VPC Flow Logs
60     enable_flow_log                = true
61     create_flow_log_cloudwatch_iam_role = true
62     create_flow_log_cloudwatch_log_group = true
63
64     tags = local.common_tags
65 }
66
67 # EKS Cluster Module
68 module "eks" {
69     source = "terraform-aws-modules/eks/aws"
70     version = "~> 19.0"
71
72     cluster_name     = "${local.name_prefix}-eks"
73     cluster_version = var.kubernetes_version
74
75     vpc_id            = module.vpc.vpc_id
76     subnet_ids        = module.vpc.private_subnets
77     control_plane_subnet_ids = module.vpc.private_subnets
78
79     # Cluster endpoint access
80     cluster_endpoint_public_access = var.environment != "production"
81     cluster_endpoint_private_access = true
82
83     # Cluster add-ons
84     cluster_addons = {

```

```

85     coredns = {
86         most_recent = true
87     }
88     kube-proxy = {
89         most_recent = true
90     }
91     vpc-cni = {
92         most_recent = true
93     }
94     aws-ebs-csi-driver = {
95         most_recent = true
96     }
97 }
98
99 # Managed node groups
100 eks_managed_node_groups = {
101     # CPU workloads
102     general = {
103         name = "${local.name_prefix}-general"
104
105         instance_types = ["m5.xlarge"]
106         capacity_type = "ON_DEMAND"
107
108         min_size      = var.environment == "production" ? 2 : 1
109         max_size      = 10
110         desired_size = var.environment == "production" ? 3 : 2
111
112         labels = {
113             workload = "general"
114         }
115
116         taints = []
117
118         tags = merge(local.common_tags, {
119             NodeGroup = "general"
120         })
121     }
122
123     # GPU workloads for training
124     gpu_training = {
125         name = "${local.name_prefix}-gpu-training"
126
127         instance_types = ["p3.2xlarge"]
128         capacity_type = var.use_spot_instances ? "SPOT" : "ON_DEMAND"
129
130         min_size      = 0
131         max_size      = 20
132         desired_size = 0 # Scale from zero
133
134         labels = {
135             workload = "training"
136             gpu      = "nvidia-v100"
137             accelerator = "true"
138         }
139
140         taints = [{
141             key      = "nvidia.com/gpu"
142             value    = "true"

```

```

143     effect = "NO_SCHEDULE"
144   }]
145
146   # Install NVIDIA device plugin
147   pre_bootstrap_user_data = <<-EOT
148     #!/bin/bash
149     /etc/eks/bootstrap.sh ${local.name_prefix}-eks \
150       --kubelet-extra-args '--node-labels=workload=training,gpu=nvidia-v100'
151   EOT
152
153   tags = merge(local.common_tags, {
154     NodeGroup = "gpu-training"
155   })
156 }
157
158 # Inference workloads
159 inference = {
160   name = "${local.name_prefix}-inference"
161
162   instance_types = ["c5.2xlarge"]
163   capacity_type  = "ON_DEMAND"
164
165   min_size      = var.environment == "production" ? 2 : 1
166   max_size      = 50
167   desired_size  = var.environment == "production" ? 3 : 1
168
169   labels = {
170     workload = "inference"
171   }
172
173   tags = merge(local.common_tags, {
174     NodeGroup = "inference"
175   })
176 }
177 }
178
179 # Cluster security group rules
180 cluster_security_group_additional_rules = {
181   ingress_nodes_ephemeral_ports_tcp = {
182     description = "Nodes on ephemeral ports"
183     protocol    = "tcp"
184     from_port   = 1025
185     to_port     = 65535
186     type        = "ingress"
187     source_node_security_group = true
188   }
189 }
190
191 # Node security group rules
192 node_security_group_additional_rules = {
193   ingress_self_all = {
194     description = "Node to node all ports/protocols"
195     protocol    = "-1"
196     from_port   = 0
197     to_port     = 0
198     type        = "ingress"
199     self        = true
200   }

```

```

201     egress_all = {
202         description      = "Node all egress"
203         protocol         = "-1"
204         from_port        = 0
205         to_port          = 0
206         type             = "egress"
207         cidr_blocks       = ["0.0.0.0/0"]
208         ipv6_cidr_blocks = [ "::/0" ]
209     }
210 }
211
212 tags = local.common_tags
213 }
214
215 # S3 Buckets for ML Assets
216 module "s3_buckets" {
217     source = "../modules/s3-buckets"
218
219     project_name = var.project_name
220     environment  = var.environment
221
222     buckets = {
223         data = {
224             name          = "${local.name_prefix}-data"
225             versioning    = true
226             encryption    = true
227             lifecycle_rules = [
228                 {
229                     id      = "archive_old_data"
230                     enabled = true
231                     transitions = [
232                         {
233                             days          = 90
234                             storage_class = "INTELLIGENT_TIERING"
235                         },
236                         {
237                             days          = 180
238                             storage_class = "GLACIER"
239                         }
240                     ]
241                 }
242             ]
243         }
244
245         models = {
246             name          = "${local.name_prefix}-models"
247             versioning    = true
248             encryption    = true
249             lifecycle_rules = [
250                 {
251                     id      = "keep_versions"
252                     enabled = true
253                     noncurrent_version_expiration = {
254                         days = 90
255                     }
256                 }
257             ]
258         }

```

```

259     artifacts = {
260         name      = "${local.name_prefix}-artifacts"
261         versioning = false
262         encryption = true
263         lifecycle_rules = [
264             {
265                 id      = "expire_artifacts"
266                 enabled = true
267                 expiration = {
268                     days = 30
269                 }
270             }
271         ]
272     }
273 }
274
275     logs = {
276         name      = "${local.name_prefix}-logs"
277         versioning = false
278         encryption = true
279         lifecycle_rules = [
280             {
281                 id      = "expire_logs"
282                 enabled = true
283                 expiration = {
284                     days = 7
285                 }
286             }
287         ]
288     }
289 }
290
291     tags = local.common_tags
292 }
293
294 # RDS for Metadata Storage
295 module "rds" {
296     source = "./modules/rds"
297
298     identifier = "${local.name_prefix}-metadata"
299
300     engine      = "postgres"
301     engine_version = "15.3"
302     instance_class = var.db_instance_type
303     allocated_storage = 100
304     max_allocated_storage = 1000
305     storage_encrypted = true
306
307     db_name = "mlmetadata"
308     username = var.db_master_username
309
310     vpc_id      = module.vpc.vpc_id
311     subnet_ids  = module.vpc.private_subnets
312
313     backup_retention_period = var.environment == "production" ? 30 : 7
314     backup_window           = "03:00-04:00"
315     maintenance_window      = "mon:04:00-mon:05:00"
316

```

```

317 deletion_protection = var.environment == "production"
318 skip_final_snapshot = var.environment != "production"
319
320 # Performance Insights
321 performance_insights_enabled = true
322 performance_insights_retention_period = 7
323
324 # Enhanced monitoring
325 enabled_cloudwatch_logs_exports = ["postgresql", "upgrade"]
326 monitoring_interval = 60
327
328 tags = local.common_tags
329 }
330
331 # ElasticCache for Caching
332 module "elasticache" {
333   source = "../modules/elasticache"
334
335   cluster_id = "${local.name_prefix}-redis"
336
337   engine = "redis"
338   engine_version = "7.0"
339   node_type = var.cache_node_type
340   num_cache_nodes = var.environment == "production" ? 2 : 1
341
342   vpc_id = module.vpc.vpc_id
343   subnet_ids = module.vpc.private_subnets
344
345   automatic_failover_enabled = var.environment == "production"
346   multi_az_enabled = var.environment == "production"
347
348   at_rest_encryption_enabled = true
349   transit_encryption_enabled = true
350
351   snapshot_retention_limit = var.environment == "production" ? 5 : 1
352   snapshot_window = "03:00-05:00"
353
354   tags = local.common_tags
355 }
356
357 # ECR Repositories
358 resource "aws_ecr_repository" "ml_images" {
359   for_each = toset([
360     "training",
361     "inference",
362     "preprocessing",
363     "monitoring"
364   ])
365
366   name = "${local.name_prefix}-${each.key}"
367   image_tag_mutability = "MUTABLE"
368
369   image_scanning_configuration {
370     scan_on_push = true
371   }
372
373   encryption_configuration {
374     encryption_type = "AES256"

```



```

375     }
376
377     tags = local.common_tags
378 }
379
380 resource "aws_ecr_lifecycle_policy" "ml_images" {
381     for_each = aws_ecr_repository.ml_images
382
383     repository = each.value.name
384
385     policy = jsonencode({
386         rules = [
387             {
388                 rulePriority = 1
389                 description = "Keep last 30 images"
390                 selection = {
391                     tagStatus      = "any"
392                     countType      = "imageCountMoreThan"
393                     countNumber    = 30
394                 }
395                 action = {
396                     type = "expire"
397                 }
398             },
399             {
400                 rulePriority = 2
401                 description = "Remove untagged images after 7 days"
402                 selection = {
403                     tagStatus      = "untagged"
404                     countType      = "sinceImagePushed"
405                     countUnit      = "days"
406                     countNumber    = 7
407                 }
408                 action = {
409                     type = "expire"
410                 }
411             }
412         ]
413     })
414 }
415
416 # SageMaker Domain for Notebooks
417 resource "aws_sagemaker_domain" "ml_studio" {
418     domain_name = "${local.name_prefix}-studio"
419     auth_mode    = "IAM"
420     vpc_id       = module.vpc.vpc_id
421     subnet_ids   = module.vpc.private_subnets
422
423     default_user_settings {
424         execution_role = aws_iam_role.sagemaker_execution.arn
425
426         jupyter_server_app_settings {
427             default_resource_spec {
428                 instance_type      = "ml.t3.medium"
429                 sagemaker_image_arn = "arn:aws:sagemaker:${data.aws_region.current.name}:${data.aws_caller_identity.current.account_id}:image/jupyter-server-3"
430             }
431         }
432     }
433 }

```

```

432     kernel_gateway_app_settings {
433         default_resource_spec {
434             instance_type      = "ml.t3.medium"
435             sagemaker_image_arn = "arn:aws:sagemaker:${data.aws_region.current.name}:${data.aws_caller_identity.current.account_id}:image/datascience-1.0"
436         }
437     }
438 }
439 }
440
441 tags = local.common_tags
442 }
443
444 # IAM Role for SageMaker
445 resource "aws_iam_role" "sagemaker_execution" {
446     name = "${local.name_prefix}-sagemaker-execution"
447
448     assume_role_policy = jsonencode({
449         Version = "2012-10-17"
450         Statement = [
451             {
452                 Action = "sts:AssumeRole"
453                 Effect = "Allow"
454                 Principal = {
455                     Service = "sagemaker.amazonaws.com"
456                 }
457             }
458         ]
459     })
460
461     tags = local.common_tags
462 }
463
464 resource "aws_iam_role_policy_attachment" "sagemaker_full_access" {
465     role          = aws_iam_role.sagemaker_execution.name
466     policy_arn    = "arn:aws:iam::aws:policy/AmazonSageMakerFullAccess"
467 }
468
469 resource "aws_iam_role_policy" "sagemaker_s3_access" {
470     name = "${local.name_prefix}-sagemaker-s3"
471     role = aws_iam_role.sagemaker_execution.id
472
473     policy = jsonencode({
474         Version = "2012-10-17"
475         Statement = [
476             {
477                 Effect = "Allow"
478                 Action = [
479                     "s3:GetObject",
480                     "s3:PutObject",
481                     "s3:DeleteObject",
482                     "s3:ListBucket"
483                 ]
484                 Resource = [
485                     for bucket in module.s3_buckets.bucket_arns : "${bucket}/*"
486                 ]
487             }
488         ]
489     })

```

```

489     })
490   }
491
492   # CloudWatch Log Groups
493   resource "aws_cloudwatch_log_group" "ml_logs" {
494     for_each = toset([
495       "training",
496       "inference",
497       "preprocessing",
498       "monitoring"
499     ])
500
501     name           = "/aws/ml/${local.name_prefix}/${each.key}"
502     retention_in_days = var.log_retention_days
503
504     tags = local.common_tags
505   }
506
507   # KMS Key for Encryption
508   resource "aws_kms_key" "ml_data" {
509     description           = "KMS key for ML data encryption"
510     deletion_window_in_days = 30
511     enable_key_rotation   = true
512
513     tags = merge(local.common_tags, {
514       Purpose = "ML Data Encryption"
515     })
516   }
517
518   resource "aws_kms_alias" "ml_data" {
519     name           = "alias/${local.name_prefix}-ml-data"
520     target_key_id = aws_kms_key.ml_data.key_id
521   }

```

Listing 18.1: Terraform Module Structure for ML Platform

18.3.2 Terraform Variables and Outputs

```

1  # modules/ml-platform/variables.tf
2  """
3  Input variables for ML platform module
4  """
5
6  variable "project_name" {
7    description = "Name of the ML project"
8    type        = string
9
10   validation {
11     condition     = can(regex("^[a-z0-9-]+$", var.project_name))
12     error_message = "Project name must contain only lowercase letters, numbers, and
13     hyphens."
14   }
15 }
16
17 variable "environment" {
18   description = "Environment name (development, staging, production)"
19   type        = string

```

```
19 validation {
20     condition      = contains(["development", "staging", "production"], var.environment
21     )
22     error_message = "Environment must be development, staging, or production."
23 }
24 }
25
26 variable "owner_email" {
27     description = "Email of the infrastructure owner"
28     type        = string
29
30     validation {
31         condition      = can(regex("^([a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,})$", var
32         .owner_email))
33         error_message = "Must be a valid email address."
34     }
35 }
36
37 variable "cost_center" {
38     description = "Cost center for billing"
39     type        = string
40     default     = "ml-platform"
41 }
42
43 variable "vpc_cidr" {
44     description = "CIDR block for VPC"
45     type        = string
46     default     = "10.0.0.0/16"
47
48     validation {
49         condition      = can(cidrhost(var.vpc_cidr, 0))
50         error_message = "Must be a valid CIDR block."
51     }
52 }
53
54 variable "kubernetes_version" {
55     description = "Kubernetes version for EKS cluster"
56     type        = string
57     default     = "1.28"
58 }
59
60 variable "use_spot_instances" {
61     description = "Use spot instances for GPU nodes"
62     type        = bool
63     default     = false
64 }
65
66 variable "db_instance_type" {
67     description = "RDS instance type"
68     type        = string
69     default     = "db.t3.large"
70 }
71
72 variable "db_master_username" {
73     description = "Master username for RDS"
74     type        = string
75     default     = "admin"
```

```

75     sensitive    = true
76 }
77
78 variable "cache_node_type" {
79     description = "ElastiCache node type"
80     type       = string
81     default    = "cache.t3.medium"
82 }
83
84 variable "log_retention_days" {
85     description = "CloudWatch log retention in days"
86     type       = number
87     default    = 7
88
89     validation {
90         condition     = contains([1, 3, 5, 7, 14, 30, 60, 90, 120, 150, 180, 365, 400,
91         545, 731, 1827, 3653], var.log_retention_days)
92         error_message = "Must be a valid CloudWatch log retention period."
93     }
94 }
95
96 variable "enable_monitoring" {
97     description = "Enable enhanced monitoring"
98     type       = bool
99     default    = true
100 }
101
102 variable "enable_auto_scaling" {
103     description = "Enable cluster autoscaling"
104     type       = bool
105     default    = true
106 }

```

Listing 18.2: Module Variables Definition

```

1  # modules/ml-platform/outputs.tf
2  """
3  Outputs from ML platform module
4  """
5
6  output "vpc_id" {
7      description = "ID of the VPC"
8      value       = module.vpc.vpc_id
9  }
10
11  output "vpc_cidr" {
12      description = "CIDR block of the VPC"
13      value       = module.vpc.vpc_cidr_block
14  }
15
16  output "private_subnets" {
17      description = "IDs of private subnets"
18      value       = module.vpc.private_subnets
19  }
20
21  output "public_subnets" {
22      description = "IDs of public subnets"
23      value       = module.vpc.public_subnets

```

```
24 }
25
26 output "eks_cluster_id" {
27     description = "EKS cluster ID"
28     value       = module.eks.cluster_id
29 }
30
31 output "eks_cluster_endpoint" {
32     description = "Endpoint for EKS control plane"
33     value       = module.eks.cluster_endpoint
34 }
35
36 output "eks_cluster_security_group_id" {
37     description = "Security group ID attached to the EKS cluster"
38     value       = module.eks.cluster_security_group_id
39 }
40
41 output "eks_cluster_certificate_authority_data" {
42     description = "Base64 encoded certificate data required to communicate with the
43                   cluster"
44     value       = module.eks.cluster_certificate_authority_data
45     sensitive   = true
46 }
47
48 output "eks_oidc_provider_arn" {
49     description = "ARN of the OIDC Provider for EKS"
50     value       = module.eks.oidc_provider_arn
51 }
52
53 output "s3_bucket_names" {
54     description = "Names of S3 buckets"
55     value       = module.s3_buckets.bucket_names
56 }
57
58 output "s3_bucket_arns" {
59     description = "ARNs of S3 buckets"
60     value       = module.s3_buckets.bucket_arns
61 }
62
63 output "rds_endpoint" {
64     description = "RDS instance endpoint"
65     value       = module.rds.endpoint
66     sensitive   = true
67 }
68
69 output "rds_database_name" {
70     description = "Name of the RDS database"
71     value       = module.rds.database_name
72 }
73
74 output "redis_endpoint" {
75     description = "ElastiCache Redis endpoint"
76     value       = module.elasticache.endpoint
77     sensitive   = true
78 }
79
80 output "ecr_repository_urls" {
81     description = "URLs of ECR repositories"
```

```

81     value = {
82         for k, v in aws_ecr_repository.ml_images : k => v.repository_url
83     }
84 }
85
86 output "sagemaker_domain_id" {
87     description = "SageMaker domain ID"
88     value       = aws_sagemaker_domain.ml_studio.id
89 }
90
91 output "sagemaker_execution_role_arn" {
92     description = "ARN of SageMaker execution role"
93     value       = aws_iam_role.sagemaker_execution.arn
94 }
95
96 output "kms_key_id" {
97     description = "ID of KMS key for encryption"
98     value       = aws_kms_key.ml_data.key_id
99 }
100
101 output "kms_key_arn" {
102     description = "ARN of KMS key for encryption"
103     value       = aws_kms_key.ml_data.arn
104 }

```

Listing 18.3: Module Outputs

18.3.3 Environment-Specific Configuration

```

1  # environments/production/main.tf
2  """
3  Production environment configuration
4  """
5
6  terraform {
7      backend "s3" {
8          bucket     = "ml-platform-terraform-state"
9          key        = "production/terraform.tfstate"
10         region     = "us-west-2"
11         encrypt    = true
12         dynamodb_table = "terraform-state-lock"
13     }
14 }
15
16 provider "aws" {
17     region = "us-west-2"
18
19     default_tags {
20         tags = {
21             Environment = "production"
22             ManagedBy   = "Terraform"
23             Project     = "ml-platform"
24         }
25     }
26 }
27
28 module "ml_platform" {

```

```

29 source = "../../modules/ml-platform"
30
31 project_name = "ml-platform"
32 environment = "production"
33 owner_email = "ml-team@company.com"
34 cost_center = "ml-engineering"
35
36 # Network configuration
37 vpc_cidr = "10.0.0.0/16"
38
39 # Kubernetes configuration
40 kubernetes_version = "1.28"
41 use_spot_instances = false # Use on-demand for production
42 enable_auto_scaling = true
43
44 # Database configuration
45 db_instance_type = "db.r6g.xlarge"
46 db_master_username = var.db_master_username
47
48 # Cache configuration
49 cache_node_type = "cache.r6g.large"
50
51 # Monitoring and logging
52 log_retention_days = 30
53 enable_monitoring = true
54 }
55
56 # Outputs
57 output "eks_cluster_endpoint" {
58     value = module.ml_platform.eks_cluster_endpoint
59     description = "EKS cluster endpoint"
60 }
61
62 output "model_bucket" {
63     value = module.ml_platform.s3_bucket_names["models"]
64     description = "S3 bucket for model storage"
65 }

```

Listing 18.4: Production Environment Configuration

18.4 Kubernetes Deployment with Helm Charts

Helm is the package manager for Kubernetes, enabling templated deployment of complex applications.

18.4.1 Helm Chart for ML Training Jobs

```

1 # charts/ml-training/Chart.yaml
2 apiVersion: v2
3 name: ml-training
4 description: Helm chart for ML training jobs
5 type: application
6 version: 1.0.0
7 appVersion: "1.0"
8

```



```
9 maintainers:
10   - name: ML Platform Team
11     email: ml-team@company.com
12
13 keywords:
14   - machine-learning
15   - training
16   - pytorch
17   - tensorflow
```

Listing 18.5: Helm Chart for ML Training

```
1  # charts/ml-training/values.yaml
2  # Default values for ml-training chart
3
4  replicaCount: 1
5
6  image:
7    repository: ${ECR_REPOSITORY}/training
8    pullPolicy: IfNotPresent
9    tag: "latest"
10
11  imagePullSecrets: []
12  nameOverride: ""
13  fullnameOverride: ""
14
15  # Training configuration
16  training:
17    # Job type: single-node or distributed
18    jobType: single-node
19
20    # Training framework: pytorch, tensorflow, scikit-learn
21    framework: pytorch
22
23    # Training script and arguments
24    command:
25      - "python"
26      - "train.py"
27    args:
28      - "--epochs=100"
29      - "--batch-size=32"
30      - "--learning-rate=0.001"
31
32    # Data paths
33    dataPath: /data
34    modelOutputPath: /models
35
36    # Environment variables
37    env:
38      - name: MLFLOW_TRACKING_URI
39        value: "http://mlflow-server:5000"
40      - name: AWS_DEFAULT_REGION
41        value: "us-west-2"
42
43    # Secrets (for credentials)
44    envFrom:
45      - secretRef:
46        name: ml-credentials
```

```
47
48 # Resource requirements
49 resources:
50   limits:
51     nvidia.com/gpu: 1
52     memory: 32Gi
53     cpu: 8
54   requests:
55     nvidia.com/gpu: 1
56     memory: 16Gi
57     cpu: 4
58
59 # Node selection
60 nodeSelector:
61   workload: training
62   gpu: nvidia-v100
63
64 tolerations:
65   - key: nvidia.com/gpu
66     operator: Equal
67     value: "true"
68     effect: NoSchedule
69
70 affinity: {}
71
72 # Persistent volume claims
73 persistence:
74   enabled: true
75   data:
76     storageClass: "gp3"
77     accessMode: ReadOnlyMany
78     size: 100Gi
79     mountPath: /data
80   models:
81     storageClass: "gp3"
82     accessMode: ReadWriteOnce
83     size: 50Gi
84     mountPath: /models
85
86 # Service account
87 serviceAccount:
88   create: true
89   annotations:
90     eks.amazonaws.com/role-arn: arn:aws:iam::ACCOUNT_ID:role/ml-training-role
91   name: ml-training
92
93 # Security context
94 securityContext:
95   runAsNonRoot: true
96   runAsUser: 1000
97   fsGroup: 1000
98   capabilities:
99     drop:
100       - ALL
101
102 podSecurityContext:
103   seccompProfile:
104     type: RuntimeDefault
```

```

105
106 # Monitoring
107 monitoring:
108   enabled: true
109   serviceMonitor:
110     enabled: true
111     interval: 30s
112     scrapeTimeout: 10s
113
114 # Logging
115 logging:
116   level: INFO
117   format: json

```

Listing 18.6: Training Job Values

```

1 # charts/ml-training/templates/job.yaml
2 {{- if eq .Values.training.jobType "single-node" }}
3 apiVersion: batch/v1
4 kind: Job
5 metadata:
6   name: {{ include "ml-training.fullname" . }}
7   labels:
8     {{- include "ml-training.labels" . | nindent 4 }}
9 spec:
10   backoffLimit: 3
11   completions: 1
12   parallelism: 1
13   template:
14     metadata:
15       annotations:
16         sidecar.istio.io/inject: "false"
17       labels:
18         {{- include "ml-training.selectorLabels" . | nindent 8 }}
19         training-job: "true"
20     spec:
21       restartPolicy: Never
22       serviceAccountName: {{ include "ml-training.serviceAccountName" . }}
23
24       {{- with .Values.imagePullSecrets }}
25       imagePullSecrets:
26         {{- toYaml . | nindent 8 }}
27       {{- end }}
28
29       {{- with .Values.nodeSelector }}
30       nodeSelector:
31         {{- toYaml . | nindent 8 }}
32       {{- end }}
33
34       {{- with .Values.tolerations }}
35       tolerations:
36         {{- toYaml . | nindent 8 }}
37       {{- end }}
38
39       {{- with .Values.affinity }}
40       affinity:
41         {{- toYaml . | nindent 8 }}
42       {{- end }}

```

```

43 volumes:
44     {{- if .Values.persistence.enabled }}
45     - name: data
46       persistentVolumeClaim:
47         claimName: {{ include "ml-training.fullname" . }}-data
48     - name: models
49       persistentVolumeClaim:
50         claimName: {{ include "ml-training.fullname" . }}-models
51     {{- end }}
52     - name: shared-memory
53       emptyDir:
54         medium: Memory
55         sizeLimit: 8Gi
56
57
58 initContainers:
59   - name: data-validation
60     image: {{ .Values.image.repository }}:{{ .Values.image.tag }}
61     command:
62       - python
63       - -c
64       - |
65         import os
66         import sys
67         data_path = "{{ .Values.training.dataPath }}"
68         if not os.path.exists(data_path):
69             print(f"Data path {data_path} does not exist")
70             sys.exit(1)
71         if not os.listdir(data_path):
72             print(f"Data path {data_path} is empty")
73             sys.exit(1)
74         print(f"Data validation passed: {len(os.listdir(data_path))} files found")
75
76     volumeMounts:
77       - name: data
78         mountPath: {{ .Values.training.dataPath }}
79
80 containers:
81   - name: training
82     image: {{ .Values.image.repository }}:{{ .Values.image.tag }}
83     imagePullPolicy: {{ .Values.image.pullPolicy }}
84
85     command: {{ toYaml .Values.training.command | nindent 12 }}
86     {{- with .Values.training.args }}
87     args: {{ toYaml . | nindent 12 }}
88     {{- end }}
89
90     env:
91       {{- range .Values.training.env }}
92       - name: {{ .name }}
93         value: {{ .value | quote }}
94       {{- end }}
95       - name: PYTHONUNBUFFERED
96         value: "1"
97       - name: POD_NAME
98         valueFrom:
99           fieldRef:
100             fieldPath: metadata.name

```

```

100     - name: POD_NAMESPACE
101       valueFrom:
102         fieldRef:
103           fieldPath: metadata.namespace
104
105     {{- with .Values.training.envFrom }}
106     envFrom: {{ toYaml . | nindent 12 }}
107     {{- end }}
108
109     volumeMounts:
110       - name: data
111         mountPath: {{ .Values.training.dataPath }}
112         readOnly: true
113       - name: models
114         mountPath: {{ .Values.training.modelOutputPath }}
115       - name: shared-memory
116         mountPath: /dev/shm
117
118     resources: {{ toYaml .Values.resources | nindent 12 }}
119
120     securityContext: {{ toYaml .Values.securityContext | nindent 12 }}
121
122     securityContext: {{ toYaml .Values.podSecurityContext | nindent 8 }}
123   {{- end }}

```

Listing 18.7: Helm Template for Training Job

```

1  # charts/ml-serving/values.yaml
2  replicaCount: 3
3
4  image:
5    repository: ${ECR_REPOSITORY}/inference
6    pullPolicy: IfNotPresent
7    tag: "latest"
8
9  service:
10    type: ClusterIP
11    port: 8080
12    targetPort: 8080
13    annotations:
14      service.beta.kubernetes.io/aws-load-balancer-type: "nlb"
15
16  ingress:
17    enabled: true
18    className: "nginx"
19    annotations:
20      cert-manager.io/cluster-issuer: "letsencrypt-prod"
21      nginx.ingress.kubernetes.io/ssl-redirect: "true"
22      nginx.ingress.kubernetes.io/force-ssl-redirect: "true"
23    hosts:
24      - host: api.ml-platform.com
25        paths:
26          - path: /predict
27            pathType: Prefix
28  tls:
29    - secretName: ml-api-tls
30      hosts:
31        - api.ml-platform.com

```

```
32
33 # Model configuration
34 model:
35   name: my-model
36   version: production
37   registry: mlflow
38   registryUri: http://mlflow-server:5000
39
40 # Autoscaling
41 autoscaling:
42   enabled: true
43   minReplicas: 3
44   maxReplicas: 50
45   targetCPUUtilizationPercentage: 70
46   targetMemoryUtilizationPercentage: 80
47
48 # Custom metrics for ML workloads
49 metrics:
50   - type: Pods
51     pods:
52       metric:
53         name: inference_queue_depth
54         target:
55           type: AverageValue
56           averageValue: "10"
57   - type: Pods
58     pods:
59       metric:
60         name: inference_latency_p95
61         target:
62           type: AverageValue
63           averageValue: "100m"
64
65 resources:
66   limits:
67     memory: 8Gi
68     cpu: 4
69   requests:
70     memory: 4Gi
71     cpu: 2
72
73 # Health checks
74 livenessProbe:
75   httpGet:
76     path: /health
77     port: 8080
78   initialDelaySeconds: 30
79   periodSeconds: 10
80   timeoutSeconds: 5
81   failureThreshold: 3
82
83 readinessProbe:
84   httpGet:
85     path: /ready
86     port: 8080
87   initialDelaySeconds: 15
88   periodSeconds: 5
89   timeoutSeconds: 3
```

```

90     failureThreshold: 3
91
92     # Pod disruption budget
93     podDisruptionBudget:
94         enabled: true
95         minAvailable: 2

```

Listing 18.8: Helm Chart for Model Serving

18.4.2 Distributed Training with Kubernetes

```

1  # charts/ml-training/templates/pytorch-distributed.yaml
2  {{- if eq .Values.training.jobType "distributed" }}
3  apiVersion: kubeflow.org/v1
4  kind: PyTorchJob
5  metadata:
6      name: {{ include "ml-training.fullname" . }}
7      labels:
8          {{- include "ml-training.labels" . | nindent 4 }}
9  spec:
10     pytorchReplicaSpecs:
11         Master:
12             replicas: 1
13             restartPolicy: OnFailure
14             template:
15                 metadata:
16                     labels:
17                         {{- include "ml-training.selectorLabels" . | nindent 12 }}
18                     role: master
19                 spec:
20                     containers:
21                         - name: pytorch
22                           image: {{ .Values.image.repository }}:{{ .Values.image.tag }}
23                           command: {{ toYaml .Values.training.command | nindent 16 }}
24                           args:
25                               {{- range .Values.training.args }}
26                                   - {{ . }}
27                               {{- end }}
28                           - "--distributed"
29                           - "--backend=nccl"
30                     resources:
31                         limits:
32                             nvidia.com/gpu: {{ .Values.resources.limits."nvidia.com/gpu" }}
33                             memory: {{ .Values.resources.limits.memory }}
34                             cpu: {{ .Values.resources.limits.cpu }}
35                         requests:
36                             nvidia.com/gpu: {{ .Values.resources.requests."nvidia.com/gpu" }}
37                             memory: {{ .Values.resources.requests.memory }}
38                             cpu: {{ .Values.resources.requests.cpu }}
39                     volumeMounts:
40                         - name: data
41                           mountPath: {{ .Values.training.dataPath }}
42                         - name: models
43                           mountPath: {{ .Values.training.modelOutputPath }}
44                         - name: shared-memory
45                           mountPath: /dev/shm
46                     volumes:

```

```

47     - name: data
48       persistentVolumeClaim:
49         claimName: training-data
50     - name: models
51       persistentVolumeClaim:
52         claimName: training-models
53     - name: shared-memory
54       emptyDir:
55         medium: Memory
56         sizeLimit: 8Gi
57   nodeSelector: {{ toYaml .Values.nodeSelector | nindent 12 }}
58   tolerations: {{ toYaml .Values.tolerations | nindent 12 }}
59
60 Worker:
61   replicas: {{ .Values.training.workers | default 3 }}
62   restartPolicy: OnFailure
63   template:
64     metadata:
65       labels:
66         {{- include "ml-training.selectorLabels" . | nindent 12 }}
67       role: worker
68     spec:
69       containers:
70         - name: pytorch
71           image: {{ .Values.image.repository }}:{{ .Values.image.tag }}
72           command: {{ toYaml .Values.training.command | nindent 16 }}
73           args:
74             {{- range .Values.training.args }}
75             - {{ . }}
76             {{- end }}
77           - "--distributed"
78           - "--backend=nccl"
79       resources:
80         limits:
81           nvidia.com/gpu: {{ .Values.resources.limits."nvidia.com/gpu" }}
82           memory: {{ .Values.resources.limits.memory }}
83           cpu: {{ .Values.resources.limits.cpu }}
84         requests:
85           nvidia.com/gpu: {{ .Values.resources.requests."nvidia.com/gpu" }}
86           memory: {{ .Values.resources.requests.memory }}
87           cpu: {{ .Values.resources.requests.cpu }}
88       volumeMounts:
89         - name: data
90           mountPath: {{ .Values.training.dataPath }}
91           readOnly: true
92         - name: models
93           mountPath: {{ .Values.training.modelOutputPath }}
94         - name: shared-memory
95           mountPath: /dev/shm
96       volumes:
97         - name: data
98           persistentVolumeClaim:
99             claimName: training-data
100         - name: models
101           persistentVolumeClaim:
102             claimName: training-models
103         - name: shared-memory
104           emptyDir:

```



```

105         medium: Memory
106         sizeLimit: 8Gi
107         nodeSelector: {{ toYaml .Values.nodeSelector | nindent 12 }}
108         tolerations: {{ toYaml .Values.tolerations | nindent 12 }}
109     {{- end }}

```

Listing 18.9: PyTorch Distributed Training Job

18.5 Multi-Cloud Resource Management

Modern ML platforms often span multiple cloud providers for redundancy, cost optimization, and feature availability.

18.5.1 Google Cloud Platform (GCP) Infrastructure

```

1  # modules/ml-platform-gcp/main.tf
2  """
3  GCP ML Platform infrastructure module
4  Provides GKE, Cloud Storage, Vertex AI integration
5  """
6
7  terraform {
8      required_providers {
9          google = {
10             source = "hashicorp/google"
11             version = "~> 5.0"
12         }
13         google-beta = {
14             source = "hashicorp/google-beta"
15             version = "~> 5.0"
16         }
17     }
18 }
19
20 provider "google" {
21     project = var.gcp_project_id
22     region = var.gcp_region
23 }
24
25 provider "google-beta" {
26     project = var.gcp_project_id
27     region = var.gcp_region
28 }
29
30 locals {
31     name_prefix = "${var.project_name}-${var.environment}"
32 }
33
34 # VPC Network
35 resource "google_compute_network" "ml_vpc" {
36     name = "${local.name_prefix}-vpc"
37     auto_create_subnetworks = false
38     routing_mode = "REGIONAL"
39 }
40

```

```

41 resource "google_compute_subnetwork" "ml_subnet" {
42   name           = "${local.name_prefix}-subnet"
43   ip_cidr_range = "10.0.0.0/20"
44   region        = var.gcp_region
45   network       = google_compute_network.ml_vpc.id
46
47   secondary_ip_range {
48     range_name = "pods"
49     ip_cidr_range = "10.4.0.0/14"
50   }
51
52   secondary_ip_range {
53     range_name = "services"
54     ip_cidr_range = "10.8.0.0/20"
55   }
56
57   private_ip_google_access = true
58 }
59
60 # Cloud NAT for private GKE nodes
61 resource "google_compute_router" "ml_router" {
62   name     = "${local.name_prefix}-router"
63   network = google_compute_network.ml_vpc.id
64   region  = var.gcp_region
65 }
66
67 resource "google_compute_router_nat" "ml_nat" {
68   name                               = "${local.name_prefix}-nat"
69   router                            = google_compute_router.ml_router.name
70   region                           = var.gcp_region
71   nat_ip_allocate_option            = "AUTO_ONLY"
72   source_subnetwork_ip_ranges_to_nat = "ALL_SUBNETWORKS_ALL_IP_RANGES"
73 }
74
75 # GKE Cluster
76 resource "google_container_cluster" "ml_cluster" {
77   name     = "${local.name_prefix}-gke"
78   location = var.gcp_region
79
80   network     = google_compute_network.ml_vpc.name
81   subnetwork = google_compute_subnetwork.ml_subnet.name
82
83   # Private cluster configuration
84   private_cluster_config {
85     enable_private_nodes    = true
86     enable_private_endpoint = false
87     master_ipv4_cidr_block = "172.16.0.0/28"
88   }
89
90   ip_allocation_policy {
91     cluster_secondary_range_name = "pods"
92     services_secondary_range_name = "services"
93   }
94
95   # Workload Identity
96   workload_identity_config {
97     workload_pool = "${var.gcp_project_id}.svc.id.goog"
98   }

```

```

99
100 # Addons
101 addons_config {
102     http_load_balancing {
103         disabled = false
104     }
105     horizontal_pod_autoscaling {
106         disabled = false
107     }
108     gce_persistent_disk_csi_driver_config {
109         enabled = true
110     }
111 }
112
113 # Maintenance window
114 maintenance_policy {
115     daily_maintenance_window {
116         start_time = "03:00"
117     }
118 }
119
120 # Remove default node pool
121 remove_default_node_pool = true
122 initial_node_count        = 1
123
124 deletion_protection = var.environment == "production"
125 }
126
127 # CPU Node Pool
128 resource "google_container_node_pool" "cpu_nodes" {
129     name          = "${local.name_prefix}-cpu-pool"
130     location      = var.gcp_region
131     cluster       = google_container_cluster.ml_cluster.name
132     node_count    = var.environment == "production" ? 3 : 2
133
134     autoscaling {
135         min_node_count = var.environment == "production" ? 2 : 1
136         max_node_count = 10
137     }
138
139     node_config {
140         machine_type = "n2-standard-8"
141         disk_size_gb = 100
142         disk_type    = "pd-standard"
143
144         oauth_scopes = [
145             "https://www.googleapis.com/auth/cloud-platform"
146         ]
147
148         labels = {
149             workload      = "general"
150             environment    = var.environment
151         }
152
153         tags = ["ml-workload", var.environment]
154
155         workload_metadata_config {
156             mode = "GKE_METADATA"
157         }
158     }
159 }

```

```

157     }
158
159     shielded_instance_config {
160         enable_secure_boot      = true
161         enable_integrity_monitoring = true
162     }
163 }
164
165 management {
166     auto_repair = true
167     auto_upgrade = true
168 }
169 }
170
171 # GPU Node Pool for Training
172 resource "google_container_node_pool" "gpu_nodes" {
173     name          = "${local.name_prefix}-gpu-pool"
174     location      = var.gcp_region
175     cluster       = google_container_cluster.ml_cluster.name
176     node_count    = 0 # Scale from zero
177
178     autoscaling {
179         min_node_count = 0
180         max_node_count = 20
181     }
182
183     node_config {
184         machine_type = "n1-standard-8"
185         disk_size_gb = 100
186
187         guest_accelerator {
188             type = "nvidia-tesla-t4"
189             count = 1
190
191             gpu_driver_installation_config {
192                 gpu_driver_version = "DEFAULT"
193             }
194         }
195
196         oauth_scopes = [
197             "https://www.googleapis.com/auth/cloud-platform"
198         ]
199
200         labels = {
201             workload = "training"
202             gpu       = "nvidia-t4"
203             environment = var.environment
204         }
205
206         taint {
207             key    = "nvidia.com/gpu"
208             value   = "true"
209             effect = "NO_SCHEDULE"
210         }
211
212         tags = ["ml-gpu", var.environment]
213
214         workload_metadata_config {

```

```
215     mode = "GKE_METADATA"
216   }
217 }
218
219 management {
220   auto_repair = true
221   auto_upgrade = false # Manual GPU node upgrades
222 }
223 }
224
225 # Cloud Storage Buckets
226 resource "google_storage_bucket" "ml_data" {
227   name          = "${local.name_prefix}-data"
228   location      = var.gcp_region
229   force_destroy = var.environment != "production"
230
231   uniform_bucket_level_access = true
232
233   versioning {
234     enabled = true
235   }
236
237   lifecycle_rule {
238     condition {
239       age = 90
240     }
241     action {
242       type          = "SetStorageClass"
243       storage_class = "NEARLINE"
244     }
245   }
246
247   lifecycle_rule {
248     condition {
249       age = 365
250     }
251     action {
252       type          = "SetStorageClass"
253       storage_class = "COLDLINE"
254     }
255   }
256 }
257
258 resource "google_storage_bucket" "ml_models" {
259   name          = "${local.name_prefix}-models"
260   location      = var.gcp_region
261   force_destroy = var.environment != "production"
262
263   uniform_bucket_level_access = true
264
265   versioning {
266     enabled = true
267   }
268
269   lifecycle_rule {
270     condition {
271       num_newer_versions = 5
272     }
273   }
274 }
```

```

273     action {
274         type = "Delete"
275     }
276 }
277 }
278
279 # Artifact Registry for Container Images
280 resource "google_artifact_registry_repository" "ml_images" {
281     location          = var.gcp_region
282     repository_id     = "${local.name_prefix}-images"
283     format            = "DOCKER"
284
285     cleanup_policies {
286         id          = "delete-old-images"
287         action      = "DELETE"
288
289         condition {
290             tag_state = "ANY"
291             older_than = "2592000s" # 30 days
292         }
293     }
294 }
295
296 # Cloud SQL for Metadata
297 resource "google_sql_database_instance" "ml_metadata" {
298     name                = "${local.name_prefix}-metadata"
299     database_version    = "POSTGRES_15"
300     region              = var.gcp_region
301
302     deletion_protection = var.environment == "production"
303
304     settings {
305         tier                = var.environment == "production" ? "db-custom-4-16384" : "db-
custom-2-7680"
306         availability_type  = var.environment == "production" ? "REGIONAL" : "ZONAL"
307         disk_size          = 100
308         disk_type          = "PD_SSD"
309         disk_autoresize    = true
310
311         backup_configuration {
312             enabled          = true
313             start_time       = "03:00"
314             point_in_time_recovery_enabled = var.environment == "production"
315             backup_retention_settings {
316                 retained_backups = var.environment == "production" ? 30 : 7
317             }
318         }
319
320         ip_configuration {
321             ipv4_enabled    = false
322             private_network = google_compute_network.ml_vpc.id
323         }
324
325         insights_config {
326             query_insights_enabled = true
327             query_string_length    = 1024
328             record_application_tags = true
329             record_client_address  = true

```

```

330     }
331   }
332
333   depends_on = [google_service_networking_connection.private_vpc_connection]
334 }
335
336 # Private Service Connection
337 resource "google_compute_global_address" "private_ip_address" {
338   name           = "${local.name_prefix}-private-ip"
339   purpose        = "VPC_PEERING"
340   address_type   = "INTERNAL"
341   prefix_length  = 16
342   network        = google_compute_network.ml_vpc.id
343 }
344
345 resource "google_service_networking_connection" "private_vpc_connection" {
346   network          = google_compute_network.ml_vpc.id
347   service          = "servicenetworking.googleapis.com"
348   reserved_peering_ranges = [google_compute_global_address.private_ip_address.name]
349 }
350
351 # Memorystore Redis
352 resource "google_redis_instance" "ml_cache" {
353   name           = "${local.name_prefix}-redis"
354   tier           = var.environment == "production" ? "STANDARD_HA" : "BASIC"
355   memory_size_gb = var.environment == "production" ? 5 : 1
356   region         = var.gcp_region
357
358   redis_version      = "REDIS_7_0"
359   display_name       = "${local.name_prefix} ML Cache"
360   authorized_network = google_compute_network.ml_vpc.id
361
362   transit_encryption_mode = "SERVER_AUTHENTICATION"
363   auth_enabled           = true
364
365   maintenance_policy {
366     weekly_maintenance_window {
367       day = "SUNDAY"
368       start_time {
369         hours   = 3
370         minutes = 0
371       }
372     }
373   }
374 }
375
376 # Vertex AI Workbench Instance
377 resource "google_workbench_instance" "ml_notebook" {
378   name       = "${local.name_prefix}-notebook"
379   location   = "${var.gcp_region}-a"
380
381   gce_setup {
382     machine_type = "n1-standard-4"
383
384     accelerator_configs {
385       type      = "NVIDIA_TESLA_T4"
386       core_count = 1
387     }
388   }
389 }

```

```

388
389   vm_image {
390     project = "deeplearning-platform-release"
391     family  = "common-cu113-notebooks"
392   }
393
394   network_interfaces {
395     network = google_compute_network.ml_vpc.id
396     subnet  = google_compute_subnetwork.ml_subnet.id
397   }
398
399   boot_disk {
400     disk_size_gb = 100
401     disk_type    = "PD_SSD"
402   }
403
404   data_disks {
405     disk_size_gb = 200
406     disk_type    = "PD_STANDARD"
407   }
408
409   metadata = {
410     terraform = "true"
411   }
412 }
413
414
415 # Outputs
416 output "gke_cluster_name" {
417   value = google_container_cluster.ml_cluster.name
418 }
419
420 output "gke_cluster_endpoint" {
421   value      = google_container_cluster.ml_cluster.endpoint
422   sensitive = true
423 }
424
425 output "gcs_data_bucket" {
426   value = google_storage_bucket.ml_data.name
427 }
428
429 output "gcs_models_bucket" {
430   value = google_storage_bucket.ml_models.name
431 }
432
433 output "artifact_registry_url" {
434   value = "${var.gcp_region}-docker.pkg.dev/${var.gcp_project_id}/${
435     google_artifact_registry_repository.ml_images.repository_id}"

```

Listing 18.10: Terraform Configuration for GCP ML Platform

18.5.2 Azure Machine Learning Infrastructure

```

1 # modules/ml-platform-azure/main.tf
2 """
3 Azure ML Platform infrastructure module

```



```

4 Provides AKS, Azure ML, Blob Storage
5 """
6
7 terraform {
8   required_providers {
9     azurerm = {
10       source = "hashicorp/azurerm"
11       version = "~> 3.0"
12     }
13   }
14 }
15
16 provider "azurerm" {
17   features {
18     key_vault {
19       purge_soft_delete_on_destroy = var.environment != "production"
20     }
21     resource_group {
22       prevent_deletion_if_contains_resources = var.environment == "production"
23     }
24   }
25 }
26
27 locals {
28   name_prefix = "${var.project_name}-${var.environment}"
29   location     = var.azure_location
30 }
31
32 # Resource Group
33 resource "azurerm_resource_group" "ml_rg" {
34   name       = "${local.name_prefix}-rg"
35   location   = local.location
36
37   tags = {
38     Environment = var.environment
39     Project     = var.project_name
40     ManagedBy   = "Terraform"
41   }
42 }
43
44 # Virtual Network
45 resource "azurerm_virtual_network" "ml_vnet" {
46   name                = "${local.name_prefix}-vnet"
47   location             = azurerm_resource_group.ml_rg.location
48   resource_group_name = azurerm_resource_group.ml_rg.name
49   address_space       = ["10.0.0.0/16"]
50
51   tags = azurerm_resource_group.ml_rg.tags
52 }
53
54 resource "azurerm_subnet" "aks_subnet" {
55   name                = "${local.name_prefix}-aks-subnet"
56   resource_group_name = azurerm_resource_group.ml_rg.name
57   virtual_network_name = azurerm_virtual_network.ml_vnet.name
58   address_prefixes    = ["10.0.1.0/24"]
59 }
60
61 resource "azurerm_subnet" "ml_subnet" {

```

```

62     name = "${local.name_prefix}-ml-subnet"
63     resource_group_name = azurerm_resource_group.ml_rg.name
64     virtual_network_name = azurerm_virtual_network.ml_vnet.name
65     address_prefixes = ["10.0.2.0/24"]
66
67     delegation {
68         name = "ml-delegation"
69
70         service_delegation {
71             name = "Microsoft.MachineLearningServices/workspaces"
72             actions = [
73                 "Microsoft.Network/virtualNetworks/subnets/join/action",
74             ]
75         }
76     }
77 }
78
79 # AKS Cluster
80 resource "azurerm_kubernetes_cluster" "ml_aks" {
81     name = "${local.name_prefix}-aks"
82     location = azurerm_resource_group.ml_rg.location
83     resource_group_name = azurerm_resource_group.ml_rg.name
84     dns_prefix = "${local.name_prefix}-aks"
85
86     kubernetes_version = var.kubernetes_version
87     automatic_channel_upgrade = "patch"
88     sku_tier = var.environment == "production" ? "Standard" : "Free"
89
90     default_node_pool {
91         name = "default"
92         node_count = var.environment == "production" ? 3 : 2
93         vm_size = "Standard_D4s_v3"
94         vnet_subnet_id = azurerm_subnet.aks_subnet.id
95         enable_auto_scaling = true
96         min_count = var.environment == "production" ? 2 : 1
97         max_count = 10
98
99         upgrade_settings {
100             max_surge = "10%"
101         }
102
103         tags = azurerm_resource_group.ml_rg.tags
104     }
105
106     identity {
107         type = "SystemAssigned"
108     }
109
110     network_profile {
111         network_plugin = "azure"
112         network_policy = "azure"
113         load_balancer_sku = "standard"
114         service_cidr = "10.1.0.0/16"
115         dns_service_ip = "10.1.0.10"
116     }
117
118     oms_agent {
119         log_analytics_workspace_id = azurerm_log_analytics_workspace.ml_logs.id

```

```

120 }
121
122 azure_active_directory_role_based_access_control {
123     managed          = true
124     azure_rbac_enabled = true
125     admin_group_object_ids = var.admin_group_object_ids
126 }
127
128 tags = azurerm_resource_group.ml_rg.tags
129 }
130
131 # GPU Node Pool
132 resource "azurerm_kubernetes_cluster_node_pool" "gpu_pool" {
133     name                = "gpupool"
134     kubernetes_cluster_id = azurerm_kubernetes_cluster.ml_aks.id
135     vm_size              = "Standard_NC6s_v3"
136     node_count           = 0
137     enable_auto_scaling  = true
138     min_count             = 0
139     max_count            = 10
140     vnet_subnet_id       = azurerm_subnet.aks_subnet.id
141
142     node_labels = {
143         workload = "training"
144         gpu       = "nvidia-v100"
145     }
146
147     node_taints = [
148         "nvidia.com/gpu=true:NoSchedule"
149     ]
150
151     tags = azurerm_resource_group.ml_rg.tags
152 }
153
154 # Storage Account
155 resource "azurerm_storage_account" "ml_storage" {
156     name                = replace("${local.name_prefix}storage", "-", "")
157     resource_group_name = azurerm_resource_group.ml_rg.name
158     location             = azurerm_resource_group.ml_rg.location
159     account_tier         = "Standard"
160     account_replication_type = var.environment == "production" ? "GRS" : "LRS"
161     account_kind          = "StorageV2"
162
163     is_hns_enabled = true # Enable hierarchical namespace for Data Lake
164
165     network_rules {
166         default_action = "Deny"
167         ip_rules        = []
168         virtual_network_subnet_ids = [azurerm_subnet.ml_subnet.id]
169         bypass           = ["AzureServices"]
170     }
171
172     blob_properties {
173         versioning_enabled = true
174
175         delete_retention_policy {
176             days = var.environment == "production" ? 30 : 7
177         }
178     }
179 }

```

```

178     container_delete_retention_policy {
179         days = var.environment == "production" ? 30 : 7
180     }
181 }
182 }
183
184 tags = azurerm_resource_group.ml_rg.tags
185 }
186
187 resource "azurerm_storage_container" "ml_data" {
188     name = "ml-data"
189     storage_account_name = azurerm_storage_account.ml_storage.name
190     container_access_type = "private"
191 }
192
193 resource "azurerm_storage_container" "ml_models" {
194     name = "ml-models"
195     storage_account_name = azurerm_storage_account.ml_storage.name
196     container_access_type = "private"
197 }
198
199 # Container Registry
200 resource "azurerm_container_registry" "ml_acr" {
201     name = replace("${local.name_prefix}acr", "-", "")
202     resource_group_name = azurerm_resource_group.ml_rg.name
203     location = azurerm_resource_group.ml_rg.location
204     sku = var.environment == "production" ? "Premium" : "Standard"
205     admin_enabled = false
206
207     identity {
208         type = "SystemAssigned"
209     }
210
211     georeplications = var.environment == "production" ? [
212     {
213         location = var.geo_replication_location
214         tags = {}
215     }
216     ] : []
217
218     network_rule_set {
219         default_action = "Deny"
220
221         ip_rule {
222             action = "Allow"
223             ip_range = azurerm_virtual_network.ml_vnet.address_space[0]
224         }
225     }
226
227     retention_policy {
228         days = 7
229         enabled = true
230     }
231
232     trust_policy {
233         enabled = true
234     }
235 }

```

```

236     tags = azurerm_resource_group.ml_rg.tags
237 }
238
239 # Key Vault
240 resource "azurerm_key_vault" "ml_kv" {
241     name                = "${local.name_prefix}-kv"
242     location            = azurerm_resource_group.ml_rg.location
243     resource_group_name = azurerm_resource_group.ml_rg.name
244     tenant_id           = data.azurerm_client_config.current.tenant_id
245     sku_name            = var.environment == "production" ? "premium" : "standard"
246
247     purge_protection_enabled = var.environment == "production"
248     soft_delete_retention_days = var.environment == "production" ? 90 : 7
249
250     network_acls {
251         default_action = "Deny"
252         bypass         = "AzureServices"
253         virtual_network_subnet_ids = [azurerm_subnet.ml_subnet.id]
254     }
255
256     tags = azurerm_resource_group.ml_rg.tags
257 }
258
259 # Azure ML Workspace
260 resource "azurerm_machine_learning_workspace" "ml_workspace" {
261     name                = "${local.name_prefix}-mlworkspace"
262     location            = azurerm_resource_group.ml_rg.location
263     resource_group_name = azurerm_resource_group.ml_rg.name
264     application_insights_id = azurerm_application_insights.ml_insights.id
265     key_vault_id         = azurerm_key_vault.ml_kv.id
266     storage_account_id   = azurerm_storage_account.ml_storage.id
267     container_registry_id = azurerm_container_registry.ml_acr.id
268
269     identity {
270         type = "SystemAssigned"
271     }
272
273     public_network_access_enabled = var.environment != "production"
274
275     tags = azurerm_resource_group.ml_rg.tags
276 }
277
278 # Log Analytics Workspace
279 resource "azurerm_log_analytics_workspace" "ml_logs" {
280     name                = "${local.name_prefix}-logs"
281     location            = azurerm_resource_group.ml_rg.location
282     resource_group_name = azurerm_resource_group.ml_rg.name
283     sku                 = "PerGB2018"
284     retention_in_days   = var.environment == "production" ? 30 : 7
285
286     tags = azurerm_resource_group.ml_rg.tags
287 }
288
289 # Application Insights
290 resource "azurerm_application_insights" "ml_insights" {
291     name                = "${local.name_prefix}-insights"
292     location            = azurerm_resource_group.ml_rg.location
293     resource_group_name = azurerm_resource_group.ml_rg.name

```

```

294 workspace_id      = azurerm_log_analytics_workspace.ml_logs.id
295 application_type   = "other"
296
297 tags = azurerm_resource_group.ml_rg.tags
298 }
299
300 # PostgreSQL Flexible Server
301 resource "azurerm_postgresql_flexible_server" "ml_db" {
302   name                        = "${local.name_prefix}-psql"
303   resource_group_name        = azurerm_resource_group.ml_rg.name
304   location                   = azurerm_resource_group.ml_rg.location
305   version                    = "15"
306   administrator_login        = var.db_admin_username
307   administrator_password     = var.db_admin_password
308   zone                       = "1"
309
310   storage_mb = 32768
311
312   sku_name = var.environment == "production" ? "GP_Standard_D4s_v3" : "
    GP_Standard_D2s_v3"
313
314   backup_retention_days      = var.environment == "production" ? 30 : 7
315   geo_redundant_backup_enabled = var.environment == "production"
316
317   high_availability {
318     mode                        = var.environment == "production" ? "ZoneRedundant" : "
    Disabled"
319     standby_availability_zone = var.environment == "production" ? "2" : null
320   }
321
322   tags = azurerm_resource_group.ml_rg.tags
323 }
324
325 data "azurerm_client_config" "current" {}
326
327 # Outputs
328 output "aks_cluster_name" {
329   value = azurerm_kubernetes_cluster.ml_aks.name
330 }
331
332 output "aks_cluster_id" {
333   value = azurerm_kubernetes_cluster.ml_aks.id
334 }
335
336 output "ml_workspace_id" {
337   value = azurerm_machine_learning_workspace.ml_workspace.id
338 }
339
340 output "storage_account_name" {
341   value = azurerm_storage_account.ml_storage.name
342 }
343
344 output "container_registry_login_server" {
345   value = azurerm_container_registry.ml_acr.login_server
346 }

```

Listing 18.11: Terraform Configuration for Azure ML Platform

18.6 Container Orchestration for ML Workloads

Container orchestration enables efficient management of ML workloads at scale, with dynamic resource allocation and fault tolerance.

18.6.1 Kubernetes Operators for ML

```

1  # k8s/crds/mltraining-crd.yaml
2  """
3  Custom Resource Definition for ML Training Jobs
4  Abstracts complexity and provides high-level ML workflow management
5  """
6
7  apiVersion: apiextensions.k8s.io/v1
8  kind: CustomResourceDefinition
9  metadata:
10   name: mltrainings.ml.platform.io
11  spec:
12   group: ml.platform.io
13   versions:
14   - name: v1alpha1
15     served: true
16     storage: true
17     schema:
18       openAPIV3Schema:
19         type: object
20         properties:
21           spec:
22             type: object
23             properties:
24               framework:
25                 type: string
26                 enum: [pytorch, tensorflow, scikit-learn, xgboost]
27                 description: ML framework to use
28               modelName:
29                 type: string
30                 description: Name of the model to train
31               dataSource:
32                 type: object
33                 properties:
34                   type:
35                     type: string
36                     enum: [s3, gcs, azure-blob, pvc]
37                   path:
38                     type: string
39                   credentials:
40                     type: string
41                     description: Secret name for credentials
42               trainingScript:
43                 type: object
44                 properties:
45                   image:
46                     type: string
47                   command:
48                     type: array
49                     items:
50                       type: string

```

```

51         args:
52             type: array
53             items:
54                 type: string
55     hyperparameters:
56         type: object
57         additionalProperties:
58             type: string
59     resources:
60         type: object
61         properties:
62             gpuType:
63                 type: string
64                 enum: [v100, a100, t4]
65             gpuCount:
66                 type: integer
67                 minimum: 0
68                 maximum: 8
69             memory:
70                 type: string
71             cpu:
72                 type: string
73     distributed:
74         type: object
75         properties:
76             enabled:
77                 type: boolean
78             workers:
79                 type: integer
80                 minimum: 1
81                 maximum: 100
82             backend:
83                 type: string
84                 enum: [nccl, gloo, mpi]
85     outputModel:
86         type: object
87         properties:
88             registry:
89                 type: string
90             registryUri:
91                 type: string
92             version:
93                 type: string
94     status:
95         type: object
96         properties:
97             phase:
98                 type: string
99                 enum: [Pending, Running, Succeeded, Failed]
100     startTime:
101         type: string
102         format: date-time
103     completionTime:
104         type: string
105         format: date-time
106     conditions:
107         type: array
108         items:

```



```

109         type: object
110     properties:
111         type:
112             type: string
113         status:
114             type: string
115         reason:
116             type: string
117         message:
118             type: string
119     metrics:
120         type: object
121     additionalProperties:
122         type: number
123     subresources:
124         status: {}
125     additionalPrinterColumns:
126     - name: Framework
127       type: string
128       jsonPath: .spec.framework
129     - name: Phase
130       type: string
131       jsonPath: .status.phase
132     - name: Age
133       type: date
134       jsonPath: .metadata.creationTimestamp
135     scope: Namespaced
136     names:
137     plural: mltrainings
138     singular: mltraining
139     kind: MLTraining
140     shortNames:
141     - mlt

```

Listing 18.12: Custom Resource Definition for ML Training

```

1  # examples/training-job.yaml
2  apiVersion: ml.platform.io/v1alpha1
3  kind: MLTraining
4  metadata:
5    name: resnet50-imagenet
6    namespace: ml-workloads
7  spec:
8    framework: pytorch
9    modelName: resnet50-imagenet
10
11  dataSource:
12    type: s3
13    path: s3://ml-data/imagenet
14    credentials: aws-credentials
15
16  trainingScript:
17    image: 123456789.dkr.ecr.us-west-2.amazonaws.com/training:latest
18    command: ["python"]
19    args:
20    - "train.py"
21    - "--model=resnet50"
22    - "--epochs=90"

```

```

23     - "--batch-size=256"
24
25     hyperparameters:
26       learning_rate: "0.1"
27       weight_decay: "0.0001"
28       momentum: "0.9"
29       lr_schedule: "cosine"
30
31     resources:
32       gpuType: v100
33       gpuCount: 4
34       memory: "128Gi"
35       cpu: "32"
36
37     distributed:
38       enabled: true
39       workers: 4
40       backend: nccl
41
42     outputModel:
43       registry: mlflow
44       registryUri: http://mlflow-server:5000
45       version: "1.0.0"

```

Listing 18.13: Example ML Training Custom Resource

18.6.2 Autoscaling ML Workloads

```

1  # k8s/autoscaling/cluster-autoscaler.yaml
2  apiVersion: v1
3  kind: ConfigMap
4  metadata:
5    name: cluster-autoscaler-priority-expander
6    namespace: kube-system
7  data:
8    priorities: |
9      10:
10       - .*-gpu-training.*  # Highest priority for GPU training nodes
11      5:
12       - .*-inference.*     # Medium priority for inference nodes
13      1:
14       - .*-general.*       # Lowest priority for general workloads
15  ---
16  apiVersion: apps/v1
17  kind: Deployment
18  metadata:
19    name: cluster-autoscaler
20    namespace: kube-system
21    labels:
22      app: cluster-autoscaler
23  spec:
24    replicas: 1
25    selector:
26      matchLabels:
27        app: cluster-autoscaler
28    template:
29      metadata:

```

```

30     labels:
31         app: cluster-autoscaler
32     spec:
33         serviceAccountName: cluster-autoscaler
34         containers:
35             - name: cluster-autoscaler
36               image: registry.k8s.io/autoscaling/cluster-autoscaler:v1.28.0
37               command:
38                 - ./cluster-autoscaler
39                 - --v=4
40                 - --stderrthreshold=info
41                 - --cloud-provider=aws
42                 - --skip-nodes-with-local-storage=false
43                 - --skip-nodes-with-system-pods=false
44                 - --expander=priority
45                 - --balance-similar-node-groups
46                 - --scale-down-delay-after-add=5m
47                 - --scale-down-unneeded-time=5m
48                 - --scale-down-utilization-threshold=0.5
49                 - --max-node-provision-time=15m
50                 - --node-group-auto-discovery=asg:tag=k8s.io/cluster-autoscaler/enabled,
k8s.io/cluster-autoscaler/${CLUSTER_NAME}
51             env:
52                 - name: CLUSTER_NAME
53                   valueFrom:
54                     configMapKeyRef:
55                       name: cluster-info
56                       key: cluster-name
57                 - name: AWS_REGION
58                   value: us-west-2
59             resources:
60                 limits:
61                     cpu: 100m
62                     memory: 600Mi
63                 requests:
64                     cpu: 100m
65                     memory: 600Mi
66             volumeMounts:
67                 - name: ssl-certs
68                   mountPath: /etc/ssl/certs/ca-certificates.crt
69                   readOnly: true
70             volumes:
71                 - name: ssl-certs
72                   hostPath:
73                     path: /etc/ssl/certs/ca-bundle.crt
74 ---
75 apiVersion: rbac.authorization.k8s.io/v1
76 kind: ClusterRole
77 metadata:
78     name: cluster-autoscaler
79 rules:
80     - apiGroups: [""]
81       resources: ["events", "endpoints"]
82       verbs: ["create", "patch"]
83     - apiGroups: [""]
84       resources: ["pods/eviction"]
85       verbs: ["create"]
86     - apiGroups: [""]

```

```

87     resources: ["pods/status"]
88     verbs: ["update"]
89 -   apiGroups: [""]
90     resources: ["endpoints"]
91     resourceNames: ["cluster-autoscaler"]
92     verbs: ["get", "update"]
93 -   apiGroups: [""]
94     resources: ["nodes"]
95     verbs: ["watch", "list", "get", "update"]
96 -   apiGroups: [""]
97     resources: ["pods", "services", "replicationcontrollers", "persistentvolumeclaims", "persistentvolumes"]
98     verbs: ["watch", "list", "get"]
99 -   apiGroups: ["apps"]
100    resources: ["statefulsets", "replicasets", "daemonsets"]
101    verbs: ["watch", "list", "get"]
102 -   apiGroups: ["policy"]
103    resources: ["poddisruptionbudgets"]
104    verbs: ["watch", "list"]
105 -   apiGroups: ["storage.k8s.io"]
106    resources: ["storageclasses", "csinodes", "csidrivers", "csistoragecapacities"]
107    verbs: ["watch", "list", "get"]
108 -   apiGroups: ["batch"]
109    resources: ["jobs"]
110    verbs: ["watch", "list", "get"]
111 -   apiGroups: ["coordination.k8s.io"]
112    resources: ["leases"]
113    verbs: ["create"]
114 -   apiGroups: ["coordination.k8s.io"]
115    resourceNames: ["cluster-autoscaler"]
116    resources: ["leases"]
117    verbs: ["get", "update"]

```

Listing 18.14: Kubernetes Cluster Autoscaler for ML

18.6.3 Resource Quotas and Limits

```

1  # k8s/resource-management/quotas.yaml
2  apiVersion: v1
3  kind: ResourceQuota
4  metadata:
5    name: ml-training-quota
6    namespace: ml-training
7  spec:
8    hard:
9      requests.cpu: "200"
10     requests.memory: 1Ti
11     requests.nvidia.com/gpu: "32"
12     limits.cpu: "400"
13     limits.memory: 2Ti
14     limits.nvidia.com/gpu: "64"
15     persistentvolumeclaims: "20"
16     requests.storage: 10Ti
17   scopeSelector:
18     matchExpressions:
19       - operator: In
20         scopeName: PriorityClass

```

```

21     values: ["high", "medium"]
22 ---
23 apiVersion: v1
24 kind: ResourceQuota
25 metadata:
26   name: ml-inference-quota
27   namespace: ml-inference
28 spec:
29   hard:
30     requests.cpu: "100"
31     requests.memory: 512Gi
32     limits.cpu: "200"
33     limits.memory: 1Ti
34     services.loadbalancers: "5"
35     pods: "100"
36 ---
37 apiVersion: v1
38 kind: LimitRange
39 metadata:
40   name: ml-training-limits
41   namespace: ml-training
42 spec:
43   limits:
44   - type: Pod
45     max:
46       cpu: "64"
47       memory: "512Gi"
48       nvidia.com/gpu: "8"
49     min:
50       cpu: "1"
51       memory: "4Gi"
52   - type: Container
53     max:
54       cpu: "64"
55       memory: "512Gi"
56       nvidia.com/gpu: "8"
57     min:
58       cpu: "100m"
59       memory: "128Mi"
60     default:
61       cpu: "4"
62       memory: "16Gi"
63     defaultRequest:
64       cpu: "2"
65       memory: "8Gi"
66   - type: PersistentVolumeClaim
67     max:
68       storage: "10Ti"
69     min:
70       storage: "10Gi"

```

Listing 18.15: Resource Quotas for ML Namespaces

18.7 Disaster Recovery and Backup Strategies

Robust backup and recovery strategies ensure ML system resilience against failures and data loss.

18.7.1 Backup Automation with Terraform

```

1  # modules/backup-recovery/main.tf
2  """
3  Automated backup and disaster recovery infrastructure
4  """
5
6  terraform {
7    required_providers {
8      aws = {
9        source  = "hashicorp/aws"
10       version = "~> 5.0"
11     }
12   }
13 }
14
15 locals {
16   backup_tags = {
17     Backup      = "true"
18     Environment = var.environment
19     RetentionDays = var.backup_retention_days
20   }
21 }
22
23 # AWS Backup Vault
24 resource "aws_backup_vault" "ml_backup_vault" {
25   name          = "${var.project_name}-${var.environment}-vault"
26   kms_key_arn   = var.kms_key_arn
27
28   tags = local.backup_tags
29 }
30
31 # Backup Plan
32 resource "aws_backup_plan" "ml_backup_plan" {
33   name = "${var.project_name}-${var.environment}-backup-plan"
34
35   # Daily backups
36   rule {
37     rule_name      = "daily_backup"
38     target_vault_name = aws_backup_vault.ml_backup_vault.name
39     schedule        = "cron(0 3 * * ? *)" # 3 AM daily
40
41     lifecycle {
42       cold_storage_after = var.environment == "production" ? 30 : null
43       delete_after       = var.environment == "production" ? 90 : 14
44     }
45
46     recovery_point_tags = local.backup_tags
47   }
48
49   # Weekly backups (longer retention)
50   rule {

```

```

51     rule_name      = "weekly_backup"
52     target_vault_name = aws_backup_vault.ml_backup_vault.name
53     schedule       = "cron(0 3 ? * 1 *)" # Monday 3 AM
54
55     lifecycle {
56         cold_storage_after = var.environment == "production" ? 60 : null
57         delete_after      = var.environment == "production" ? 365 : 30
58     }
59
60     recovery_point_tags = merge(local.backup_tags, {
61         BackupType = "weekly"
62     })
63 }
64
65 # Monthly backups (longest retention)
66 rule {
67     rule_name      = "monthly_backup"
68     target_vault_name = aws_backup_vault.ml_backup_vault.name
69     schedule       = "cron(0 3 1 * ? *)" # First day of month
70
71     lifecycle {
72         cold_storage_after = var.environment == "production" ? 90 : null
73         delete_after      = var.environment == "production" ? 2555 : 90 # ~7 years for
74                               production
75     }
76
77     recovery_point_tags = merge(local.backup_tags, {
78         BackupType = "monthly"
79     })
80 }
81
82 tags = local.backup_tags
83
84 # Backup Selection for RDS
85 resource "aws_backup_selection" "rds_backup" {
86     name      = "rds-backup-selection"
87     iam_role_arn = aws_iam_role.backup_role.arn
88     plan_id    = aws_backup_plan.ml_backup_plan.id
89
90     selection_tag {
91         type = "STRINGEQUALS"
92         key  = "Backup"
93         value = "true"
94     }
95
96     resources = var.rds_arns
97 }
98
99 # Backup Selection for EBS Volumes
100 resource "aws_backup_selection" "ebs_backup" {
101     name      = "ebs-backup-selection"
102     iam_role_arn = aws_iam_role.backup_role.arn
103     plan_id    = aws_backup_plan.ml_backup_plan.id
104
105     selection_tag {
106         type = "STRINGEQUALS"
107         key  = "Backup"

```

```

108     value = "true"
109 }
110
111 resources = var.ebs_volume_arns
112 }
113
114 # IAM Role for AWS Backup
115 resource "aws_iam_role" "backup_role" {
116     name = "${var.project_name}-${var.environment}-backup-role"
117
118     assume_role_policy = jsonencode({
119         Version = "2012-10-17"
120         Statement = [
121             {
122                 Action = "sts:AssumeRole"
123                 Effect = "Allow"
124                 Principal = {
125                     Service = "backup.amazonaws.com"
126                 }
127             }
128         ]
129     })
130
131     tags = local.backup_tags
132 }
133
134 resource "aws_iam_role_policy_attachment" "backup_policy" {
135     role      = aws_iam_role.backup_role.name
136     policy_arn = "arn:aws:iam::aws:policy/service-role/
137     AWSBackupServiceRolePolicyForBackup"
138 }
139
140 resource "aws_iam_role_policy_attachment" "restore_policy" {
141     role      = aws_iam_role.backup_role.name
142     policy_arn = "arn:aws:iam::aws:policy/service-role/
143     AWSBackupServiceRolePolicyForRestores"
144 }
145
146 # S3 Bucket Replication for Models
147 resource "aws_s3_bucket_replication_configuration" "model_replication" {
148     count = var.environment == "production" ? 1 : 0
149
150     role      = aws_iam_role.replication_role[0].arn
151     bucket    = var.source_bucket_id
152
153     rule {
154         id      = "replicate-models"
155         status  = "Enabled"
156
157         filter {
158             prefix = "models/"
159         }
160
161         destination {
162             bucket      = var.destination_bucket_arn
163             storage_class = "STANDARD_IA"
164
165             replication_time {

```



```

164     status = "Enabled"
165     time {
166         minutes = 15
167     }
168 }
169
170 metrics {
171     status = "Enabled"
172     event_threshold {
173         minutes = 15
174     }
175 }
176 }
177
178 delete_marker_replication {
179     status = "Enabled"
180 }
181 }
182 }
183
184 # IAM Role for S3 Replication
185 resource "aws_iam_role" "replication_role" {
186     count = var.environment == "production" ? 1 : 0
187
188     name = "${var.project_name}-${var.environment}-s3-replication-role"
189
190     assume_role_policy = jsonencode({
191         Version = "2012-10-17"
192         Statement = [
193             {
194                 Action = "sts:AssumeRole"
195                 Effect = "Allow"
196                 Principal = {
197                     Service = "s3.amazonaws.com"
198                 }
199             }
200         ]
201     })
202
203     tags = local.backup_tags
204 }
205
206 resource "aws_iam_role_policy" "replication_policy" {
207     count = var.environment == "production" ? 1 : 0
208
209     name = "${var.project_name}-s3-replication-policy"
210     role = aws_iam_role.replication_role[0].id
211
212     policy = jsonencode({
213         Version = "2012-10-17"
214         Statement = [
215             {
216                 Effect = "Allow"
217                 Action = [
218                     "s3:GetReplicationConfiguration",
219                     "s3:ListBucket"
220                 ]
221                 Resource = var.source_bucket_arn

```

```

222     },
223     {
224         Effect = "Allow"
225         Action = [
226             "s3:GetObjectVersionForReplication",
227             "s3:GetObjectVersionAcl",
228             "s3:GetObjectVersionTagging"
229         ]
230         Resource = "${var.source_bucket_arn}/*"
231     },
232     {
233         Effect = "Allow"
234         Action = [
235             "s3:ReplicateObject",
236             "s3:ReplicateDelete",
237             "s3:ReplicateTags"
238         ]
239         Resource = "${var.destination_bucket_arn}/*"
240     }
241 ]
242 })
243 }
244
245 # CloudWatch Alarms for Backup Failures
246 resource "aws_cloudwatch_metric_alarm" "backup_failure" {
247     alarm_name          = "${var.project_name}-${var.environment}-backup-failure"
248     comparison_operator = "GreaterThanThreshold"
249     evaluation_periods  = 1
250     metric_name         = "NumberOfBackupJobsFailed"
251     namespace           = "AWS/Backup"
252     period              = 3600
253     statistic           = "Sum"
254     threshold           = 0
255     alarm_description   = "Alert when backup jobs fail"
256     alarm_actions       = [var.sns_topic_arn]
257
258     dimensions = {
259         BackupVaultName = aws_backup_vault.ml_backup_vault.name
260     }
261
262     tags = local.backup_tags
263 }
264
265 # Outputs
266 output "backup_vault_arn" {
267     value = aws_backup_vault.ml_backup_vault.arn
268 }
269
270 output "backup_plan_id" {
271     value = aws_backup_plan.ml_backup_plan.id
272 }

```

Listing 18.16: Automated Backup Configuration

18.7.2 Disaster Recovery Procedures

```
1 #!/usr/bin/env python3
```

```

2  """
3  Disaster Recovery automation for ML infrastructure
4  Orchestrates recovery from backup vault
5  """
6
7  import boto3
8  import logging
9  from datetime import datetime, timedelta
10 from typing import Dict, List, Optional
11 import argparse
12
13 logging.basicConfig(level=logging.INFO)
14 logger = logging.getLogger(__name__)
15
16 class DisasterRecoveryOrchestrator:
17     """Orchestrates disaster recovery operations"""
18
19     def __init__(self, region: str, environment: str):
20         self.backup_client = boto3.client('backup', region_name=region)
21         self.rds_client = boto3.client('rds', region_name=region)
22         self.ec2_client = boto3.client('ec2', region_name=region)
23         self.s3_client = boto3.client('s3', region_name=region)
24         self.environment = environment
25         self.region = region
26
27     def list_recovery_points(
28         self,
29         backup_vault_name: str,
30         resource_type: Optional[str] = None,
31         created_after: Optional[datetime] = None
32     ) -> List[Dict]:
33         """List available recovery points"""
34
35         params = {
36             'BackupVaultName': backup_vault_name
37         }
38
39         if resource_type:
40             params['ByResourceType'] = resource_type
41
42         if created_after:
43             params['ByCreatedAfter'] = created_after
44
45         try:
46             response = self.backup_client.list_recovery_points_by_backup_vault(
47                 **params
48             )
49
50             recovery_points = response.get('RecoveryPoints', [])
51             logger.info(
52                 f"Found {len(recovery_points)} recovery points in {backup_vault_name}"
53             )
54
55             return recovery_points
56
57         except Exception as e:
58             logger.error(f"Failed to list recovery points: {e}")
59             return []

```

```

60
61 def select_latest_recovery_point(
62     self,
63     backup_vault_name: str,
64     resource_arn: str
65 ) -> Optional[str]:
66     """Select the most recent recovery point for a resource"""
67
68     try:
69         response = self.backup_client.list_recovery_points_by_resource(
70             ResourceArn=resource_arn
71         )
72
73         recovery_points = response.get('RecoveryPoints', [])
74
75         # Filter completed recovery points
76         completed = [
77             rp for rp in recovery_points
78             if rp['Status'] == 'COMPLETED'
79         ]
80
81         if not completed:
82             logger.warning(f"No completed recovery points for {resource_arn}")
83             return None
84
85         # Sort by creation time and get latest
86         latest = sorted(
87             completed,
88             key=lambda x: x['CreationDate'],
89             reverse=True
90         )[0]
91
92         logger.info(
93             f"Selected recovery point {latest['RecoveryPointArn']} "
94             f"from {latest['CreationDate']}"
95         )
96
97         return latest['RecoveryPointArn']
98
99     except Exception as e:
100         logger.error(f"Failed to select recovery point: {e}")
101         return None
102
103 def restore_rds_instance(
104     self,
105     recovery_point_arn: str,
106     target_identifier: str,
107     subnet_group_name: str,
108     security_group_ids: List[str]
109 ) -> Optional[str]:
110     """Restore RDS instance from backup"""
111
112     logger.info(f"Starting RDS restore to {target_identifier}")
113
114     try:
115         # Start restore job
116         response = self.backup_client.start_restore_job(
117             RecoveryPointArn=recovery_point_arn,

```

```

118         Metadata={
119             'DBInstanceIdentifier': target_identifier,
120             'DBSubnetGroupName': subnet_group_name,
121             'VpcSecurityGroupIds': ','.join(security_group_ids),
122             'PubliclyAccessible': 'false',
123             'MultiAZ': 'true' if self.environment == 'production' else 'false'
124         },
125         StorageEncrypted='true'
126     ),
127     iam_role_arn=self._get_restore_role_arn()
128 )
129
130 restore_job_id = response['RestoreJobId']
131 logger.info(f"RDS restore job started: {restore_job_id}")
132
133 return restore_job_id
134
135 except Exception as e:
136     logger.error(f"Failed to restore RDS instance: {e}")
137     return None
138
139 def restore_ebs_volume(
140     self,
141     recovery_point_arn: str,
142     availability_zone: str,
143     volume_type: str = 'gp3',
144     iops: int = 3000
145 ) -> Optional[str]:
146     """Restore EBS volume from backup"""
147
148     logger.info(f"Starting EBS volume restore in {availability_zone}")
149
150     try:
151         response = self.backup_client.start_restore_job(
152             RecoveryPointArn=recovery_point_arn,
153             Metadata={
154                 'AvailabilityZone': availability_zone,
155                 'VolumeType': volume_type,
156                 'Iops': str(iops),
157                 'Encrypted': 'true'
158             },
159             iam_role_arn=self._get_restore_role_arn()
160         )
161
162         restore_job_id = response['RestoreJobId']
163         logger.info(f"EBS restore job started: {restore_job_id}")
164
165         return restore_job_id
166
167     except Exception as e:
168         logger.error(f"Failed to restore EBS volume: {e}")
169         return None
170
171 def monitor_restore_job(self, restore_job_id: str) -> Dict:
172     """Monitor restore job progress"""
173
174     try:
175         response = self.backup_client.describe_restore_job(

```

```

175         RestoreJobId=restore_job_id
176     )
177
178     status = response['Status']
179     logger.info(f"Restore job {restore_job_id} status: {status}")
180
181     return response
182
183 except Exception as e:
184     logger.error(f"Failed to monitor restore job: {e}")
185     return {}
186
187 def replicate_s3_data(
188     self,
189     source_bucket: str,
190     destination_bucket: str,
191     prefix: str = ""
192 ) -> bool:
193     """Manually replicate S3 data for disaster recovery"""
194
195     logger.info(
196         f"Replicating from {source_bucket}/{prefix} "
197         f"to {destination_bucket}/{prefix}"
198     )
199
200     try:
201         paginator = self.s3_client.get_paginator('list_objects_v2')
202         pages = paginator.paginate(Bucket=source_bucket, Prefix=prefix)
203
204         objects_replicated = 0
205
206         for page in pages:
207             if 'Contents' not in page:
208                 continue
209
210             for obj in page['Contents']:
211                 source_key = obj['Key']
212
213                 # Copy object to destination
214                 copy_source = {
215                     'Bucket': source_bucket,
216                     'Key': source_key
217                 }
218
219                 self.s3_client.copy_object(
220                     CopySource=copy_source,
221                     Bucket=destination_bucket,
222                     Key=source_key
223                 )
224
225                 objects_replicated += 1
226
227                 if objects_replicated % 100 == 0:
228                     logger.info(f"Replicated {objects_replicated} objects")
229
230         logger.info(
231             f"Successfully replicated {objects_replicated} objects"
232         )

```

```

233         return True
234
235     except Exception as e:
236         logger.error(f"Failed to replicate S3 data: {e}")
237         return False
238
239     def _get_restore_role_arn(self) -> str:
240         """Get IAM role ARN for restore operations"""
241         # Construct ARN from account and environment
242         account_id = boto3.client('sts').get_caller_identity()['Account']
243         return (
244             f"arn:aws:iam::{account_id}:role/"
245             f"ml-platform-{self.environment}-backup-role"
246         )
247
248     def execute_full_recovery(
249         self,
250         backup_vault_name: str,
251         recovery_config: Dict
252     ) -> Dict:
253         """Execute full disaster recovery procedure"""
254
255         logger.info("Starting full disaster recovery procedure")
256
257         results = {
258             'rds_restores': [],
259             'ebs_restores': [],
260             's3_replications': [],
261             'status': 'in_progress'
262         }
263
264         try:
265             # Restore RDS instances
266             for rds_config in recovery_config.get('rds_instances', []):
267                 recovery_point = self.select_latest_recovery_point(
268                     backup_vault_name,
269                     rds_config['source_arn']
270                 )
271
272                 if recovery_point:
273                     restore_job = self.restore_rds_instance(
274                         recovery_point,
275                         rds_config['target_identifier'],
276                         rds_config['subnet_group'],
277                         rds_config['security_groups']
278                     )
279
280                     if restore_job:
281                         results['rds_restores'].append({
282                             'target': rds_config['target_identifier'],
283                             'job_id': restore_job,
284                             'status': 'started'
285                         })
286
287             # Restore EBS volumes
288             for ebs_config in recovery_config.get('ebs_volumes', []):
289                 recovery_point = self.select_latest_recovery_point(
290                     backup_vault_name,

```

```

291         ebs_config['source_arn']
292     )
293
294     if recovery_point:
295         restore_job = self.restore_ebs_volume(
296             recovery_point,
297             ebs_config['availability_zone'],
298             ebs_config.get('volume_type', 'gp3')
299         )
300
301         if restore_job:
302             results['ebs_restores'].append({
303                 'az': ebs_config['availability_zone'],
304                 'job_id': restore_job,
305                 'status': 'started'
306             })
307
308     # Replicate S3 data
309     for s3_config in recovery_config.get('s3_replications', []):
310         success = self.replicate_s3_data(
311             s3_config['source_bucket'],
312             s3_config['destination_bucket'],
313             s3_config.get('prefix', '')
314         )
315
316         results['s3_replications'].append({
317             'source': s3_config['source_bucket'],
318             'destination': s3_config['destination_bucket'],
319             'status': 'completed' if success else 'failed'
320         })
321
322     results['status'] = 'completed'
323     logger.info("Disaster recovery procedure completed")
324
325     except Exception as e:
326         logger.error(f"Disaster recovery failed: {e}")
327         results['status'] = 'failed'
328         results['error'] = str(e)
329
330     return results
331
332 def main():
333     parser = argparse.ArgumentParser(
334         description='ML Infrastructure Disaster Recovery'
335     )
336     parser.add_argument('--region', required=True, help='AWS region')
337     parser.add_argument('--environment', required=True, help='Environment name')
338     parser.add_argument('--vault', required=True, help='Backup vault name')
339     parser.add_argument('--config', required=True, help='Recovery config file')
340
341     args = parser.parse_args()
342
343     # Load recovery configuration
344     import json
345     with open(args.config, 'r') as f:
346         recovery_config = json.load(f)
347
348     # Execute recovery

```



```

349     orchestrator = DisasterRecoveryOrchestrator(args.region, args.environment)
350     results = orchestrator.execute_full_recovery(args.vault, recovery_config)
351
352     # Print results
353     print(json.dumps(results, indent=2))
354
355     return 0 if results['status'] == 'completed' else 1
356
357 if __name__ == '__main__':
358     exit(main())

```

Listing 18.17: Disaster Recovery Automation Script

18.8 Cost Optimization Through IaC

Infrastructure as Code enables systematic cost optimization through automated resource management and intelligent provisioning strategies.

18.8.1 Cost-Aware Resource Provisioning

```

1  # modules/cost-optimization/main.tf
2  """
3  Cost-optimized infrastructure module
4  Implements spot instances, auto-scaling, and resource lifecycle management
5  """
6
7  terraform {
8      required_providers {
9          aws = {
10             source = "hashicorp/aws"
11             version = "~> 5.0"
12         }
13     }
14 }
15
16 locals {
17     cost_allocation_tags = {
18         CostCenter = var.cost_center
19         Team       = var.team_name
20         Environment = var.environment
21         Project    = var.project_name
22     }
23 }
24
25 # Spot Instance Fleet for Training
26 resource "aws_autoscaling_group" "training_spot_fleet" {
27     name           = "${var.project_name}-training-spot"
28     min_size       = 0
29     max_size       = var.max_training_nodes
30     desired_capacity = 0 # Scale from zero
31
32     vpc_zone_identifier = var.subnet_ids
33
34     mixed_instances_policy {
35         instances_distribution {

```

```

36     on_demand_base_capacity                = 0
37     on_demand_percentage_above_base_capacity = 20 # 80% spot
38     spot_allocation_strategy               = "price-capacity-optimized"
39     spot_instance_pools                   = 4
40 }
41
42 launch_template {
43     launch_template_specification {
44         launch_template_id = aws_launch_template.training_spot.id
45         version             = "$Latest"
46     }
47
48     # Multiple instance types for better spot availability
49     override {
50         instance_type      = "p3.2xlarge"
51         weighted_capacity = 1
52     }
53
54     override {
55         instance_type      = "p3.8xlarge"
56         weighted_capacity = 4
57     }
58
59     override {
60         instance_type      = "p3.16xlarge"
61         weighted_capacity = 8
62     }
63
64     override {
65         instance_type      = "g4dn.12xlarge"
66         weighted_capacity = 4
67     }
68 }
69 }
70
71 # Scale based on training queue depth
72 enabled_metrics = [
73     "GroupDesiredCapacity",
74     "GroupInServiceInstances",
75     "GroupMinSize",
76     "GroupMaxSize"
77 ]
78
79 tag {
80     key          = "Name"
81     value        = "${var.project_name}-training-spot"
82     propagate_at_launch = true
83 }
84
85 dynamic "tag" {
86     for_each = local.cost_allocation_tags
87
88     content {
89         key          = tag.key
90         value        = tag.value
91         propagate_at_launch = true
92     }
93 }

```

```

94 }
95
96 # Launch Template for Spot Instances
97 resource "aws_launch_template" "training_spot" {
98   name_prefix = "${var.project_name}-training-"
99   image_id    = data.aws_ami.gpu_optimized.id
100   instance_type = "p3.2xlarge" # Default, overridden by mixed policy
101
102   network_interfaces {
103     associate_public_ip_address = false
104     security_groups             = [var.training_security_group_id]
105     delete_on_termination      = true
106   }
107
108   iam_instance_profile {
109     arn = aws_iam_instance_profile.training_spot.arn
110   }
111
112   block_device_mappings {
113     device_name = "/dev/sda1"
114
115     ebs {
116       volume_size      = 100
117       volume_type       = "gp3"
118       iops              = 3000
119       throughput        = 125
120       delete_on_termination = true
121       encrypted         = true
122     }
123   }
124
125   user_data = base64encode(templatefile("${path.module}/user_data.sh", {
126     cluster_name = var.eks_cluster_name
127     region       = data.aws_region.current.name
128   }))
129
130   metadata_options {
131     http_endpoint      = "enabled"
132     http_tokens        = "required"
133     http_put_response_hop_limit = 1
134   }
135
136   tag_specifications {
137     resource_type = "instance"
138
139     tags = merge(local.cost_allocation_tags, {
140       Name = "${var.project_name}-training-spot"
141     })
142   }
143
144   tag_specifications {
145     resource_type = "volume"
146
147     tags = merge(local.cost_allocation_tags, {
148       Name = "${var.project_name}-training-volume"
149     })
150   }
151 }

```

```

152
153 # Auto Scaling Policies
154 resource "aws_autoscaling_policy" "scale_up" {
155     name                = "${var.project_name}-scale-up"
156     scaling_adjustment  = 2
157     adjustment_type     = "ChangeInCapacity"
158     cooldown            = 300
159     autoscaling_group_name = aws_autoscaling_group.training_spot_fleet.name
160 }
161
162 resource "aws_autoscaling_policy" "scale_down" {
163     name                = "${var.project_name}-scale-down"
164     scaling_adjustment  = -1
165     adjustment_type     = "ChangeInCapacity"
166     cooldown            = 300
167     autoscaling_group_name = aws_autoscaling_group.training_spot_fleet.name
168 }
169
170 # CloudWatch Alarms for Auto Scaling
171 resource "aws_cloudwatch_metric_alarm" "training_queue_high" {
172     alarm_name          = "${var.project_name}-training-queue-high"
173     comparison_operator = "GreaterThanThreshold"
174     evaluation_periods  = 2
175     metric_name         = "ApproximateNumberOfMessagesVisible"
176     namespace           = "AWS/SQS"
177     period              = 60
178     statistic           = "Average"
179     threshold           = 5
180
181     alarm_description = "Scale up when training queue is high"
182     alarm_actions     = [aws_autoscaling_policy.scale_up.arn]
183
184     dimensions = {
185         QueueName = var.training_queue_name
186     }
187 }
188
189 resource "aws_cloudwatch_metric_alarm" "training_queue_low" {
190     alarm_name          = "${var.project_name}-training-queue-low"
191     comparison_operator = "LessThanThreshold"
192     evaluation_periods  = 5
193     metric_name         = "ApproximateNumberOfMessagesVisible"
194     namespace           = "AWS/SQS"
195     period              = 60
196     statistic           = "Average"
197     threshold           = 2
198
199     alarm_description = "Scale down when training queue is low"
200     alarm_actions     = [aws_autoscaling_policy.scale_down.arn]
201
202     dimensions = {
203         QueueName = var.training_queue_name
204     }
205 }
206
207 # S3 Intelligent Tiering Lifecycle Policy
208 resource "aws_s3_bucket_lifecycle_configuration" "intelligent_tiering" {
209     bucket = var.ml_data_bucket_id

```

```
210 rule {
211   id      = "intelligent-tiering"
212   status = "Enabled"
213
214   filter {
215     prefix = ""
216   }
217
218   transition {
219     days      = 30
220     storage_class = "INTELLIGENT_TIERING"
221   }
222
223   noncurrent_version_transition {
224     noncurrent_days = 30
225     storage_class   = "GLACIER"
226   }
227
228   noncurrent_version_expiration {
229     noncurrent_days = 90
230   }
231 }
232
233 rule {
234   id      = "delete-old-logs"
235   status = "Enabled"
236
237   filter {
238     prefix = "logs/"
239   }
240
241   expiration {
242     days = 7
243   }
244 }
245
246 rule {
247   id      = "archive-training-data"
248   status = "Enabled"
249
250   filter {
251     and {
252       prefix = "training-data/"
253       tags = {
254         Archive = "true"
255       }
256     }
257   }
258
259   transition {
260     days      = 90
261     storage_class = "GLACIER"
262   }
263
264   transition {
265     days      = 180
266     storage_class = "DEEP_ARCHIVE"
267   }
```

```

268     }
269   }
270 }
271
272 # Cost Anomaly Detection
273 resource "aws_ce_anomaly_monitor" "ml_infrastructure" {
274   name           = "${var.project_name}-cost-anomaly"
275   monitor_type   = "DIMENSIONAL"
276   monitor_dimension = "SERVICE"
277 }
278
279 resource "aws_ce_anomaly_subscription" "ml_infrastructure_alert" {
280   name           = "${var.project_name}-cost-anomaly-alert"
281   frequency      = "DAILY"
282
283   monitor_arn_list = [
284     aws_ce_anomaly_monitor.ml_infrastructure.arn
285   ]
286
287   subscriber {
288     type      = "SNS"
289     address   = var.cost_alert_sns_topic_arn
290   }
291
292   threshold_expression {
293     and {
294       dimension {
295         key           = "ANOMALY_TOTAL_IMPACT_ABSOLUTE"
296         match_options = ["GREATER_THAN_OR_EQUAL"]
297         values        = ["100"] # Alert on $100+ anomalies
298       }
299     }
300   }
301 }
302
303 # Budget Alerts
304 resource "aws_budgets_budget" "ml_monthly" {
305   name           = "${var.project_name}-monthly-budget"
306   budget_type    = "COST"
307   limit_amount   = var.monthly_budget_limit
308   limit_unit     = "USD"
309   time_period_start = "2024-01-01-00:00"
310   time_unit      = "MONTHLY"
311
312   cost_filter {
313     name = "TagKeyValue"
314     values = [
315       "Project${var.project_name}"
316     ]
317   }
318
319   notification {
320     comparison_operator = "GREATER_THAN"
321     threshold           = 80
322     threshold_type      = "PERCENTAGE"
323     notification_type    = "ACTUAL"
324     subscriber_email_addresses = var.budget_alert_emails
325   }

```

```

326 notification {
327     comparison_operator      = "GREATER_THAN"
328     threshold                = 100
329     threshold_type           = "PERCENTAGE"
330     notification_type        = "ACTUAL"
331     subscriber_email_addresses = var.budget_alert_emails
332 }
333
334 notification {
335     comparison_operator      = "GREATER_THAN"
336     threshold                = 90
337     threshold_type           = "PERCENTAGE"
338     notification_type        = "FORECASTED"
339     subscriber_email_addresses = var.budget_alert_emails
340 }
341 }
342 }
343
344 # Scheduled Instance Shutdown for Development
345 resource "aws_lambda_function" "instance_scheduler" {
346     count = var.environment != "production" ? 1 : 0
347
348     filename      = data.archive_file.scheduler_lambda[0].output_path
349     function_name = "${var.project_name}-instance-scheduler"
350     role           = aws_iam_role.scheduler_lambda[0].arn
351     handler        = "index.handler"
352     runtime        = "python3.11"
353     timeout        = 60
354
355     environment {
356         variables = {
357             ENVIRONMENT = var.environment
358             PROJECT_NAME = var.project_name
359         }
360     }
361
362     tags = local.cost_allocation_tags
363 }
364
365 # Schedule: Stop instances at night (development only)
366 resource "aws_cloudwatch_event_rule" "stop_instances" {
367     count = var.environment != "production" ? 1 : 0
368
369     name           = "${var.project_name}-stop-instances"
370     description    = "Stop development instances at night"
371     schedule_expression = "cron(0 22 * * ? *)" # 10 PM daily
372 }
373
374 resource "aws_cloudwatch_event_target" "stop_instances" {
375     count = var.environment != "production" ? 1 : 0
376
377     rule      = aws_cloudwatch_event_rule.stop_instances[0].name
378     target_id = "StopInstances"
379     arn       = aws_lambda_function.instance_scheduler[0].arn
380
381     input = jsonencode({
382         action = "stop"
383     })

```

```

384 }
385
386 # Schedule: Start instances in morning
387 resource "aws_cloudwatch_event_rule" "start_instances" {
388     count = var.environment != "production" ? 1 : 0
389
390     name          = "${var.project_name}-start-instances"
391     description    = "Start development instances in morning"
392     schedule_expression = "cron(0 8 * * MON-FRI *)" # 8 AM weekdays
393 }
394
395 resource "aws_cloudwatch_event_target" "start_instances" {
396     count = var.environment != "production" ? 1 : 0
397
398     rule          = aws_cloudwatch_event_rule.start_instances[0].name
399     target_id     = "StartInstances"
400     arn           = aws_lambda_function.instance_scheduler[0].arn
401
402     input = jsonencode({
403         action = "start"
404     })
405 }
406
407 # Outputs
408 output "spot_fleet_asg_name" {
409     value = aws_autoscaling_group.training_spot_fleet.name
410 }
411
412 output "cost_anomaly_monitor_arn" {
413     value = aws_ce_anomaly_monitor.ml_infrastructure.arn
414 }
415
416 output "budget_name" {
417     value = aws_budgets_budget.ml_monthly.name
418 }

```

Listing 18.18: Terraform Module for Cost-Optimized ML Infrastructure

Cost Optimization Best Practices:

- **Spot Instances:** Use spot instances for 60-80% cost savings on training workloads
- **Right-Sizing:** Continuously monitor and adjust instance types based on actual usage
- **Auto-Scaling:** Scale resources dynamically based on demand
- **Reserved Instances:** Purchase reserved capacity for predictable baseline workloads
- **Storage Tiering:** Automatically move infrequently accessed data to cheaper storage
- **Resource Scheduling:** Stop development resources outside business hours
- **Budget Alerts:** Set up proactive cost monitoring and alerts
- **Tagging Strategy:** Tag all resources for accurate cost allocation and tracking

18.9 Practical Infrastructure Automation Exercises

18.9.1 Exercise 1: Multi-Environment ML Infrastructure

Deploy complete ML infrastructure across development, staging, and production environments using Terraform modules. Implement:

- Environment-specific resource sizing (development: minimal, production: HA)
- VPC with public and private subnets in multiple availability zones
- EKS cluster with separate node groups for training and inference
- S3 buckets with lifecycle policies and versioning
- RDS PostgreSQL with automated backups
- ECR repositories for container images
- Proper IAM roles and security groups

Deliverables:

- Terraform module structure with separate environments
- Variables file for each environment
- State management with remote backend
- Infrastructure documentation

18.9.2 Exercise 2: Kubernetes ML Training Helm Chart

Create production-ready Helm chart for distributed ML training jobs. Include:

- Support for PyTorch and TensorFlow distributed training
- Configurable GPU resource allocation
- Data loading from S3, GCS, and Azure Blob
- MLflow experiment tracking integration
- Health checks and pod disruption budgets
- Resource quotas and limits
- Secrets management for credentials

Deliverables:

- Complete Helm chart with templates
- Values files for different model types
- Testing strategy for chart validation
- Deployment documentation

18.9.3 Exercise 3: Multi-Cloud Infrastructure Deployment

Deploy ML platform infrastructure across AWS, GCP, and Azure using Terraform. Implement:

- Kubernetes clusters on EKS, GKE, and AKS
- Object storage on S3, GCS, and Azure Blob
- Managed databases for metadata storage
- Container registries in each cloud
- Cross-cloud networking for data replication
- Unified monitoring and logging

Deliverables:

- Cloud-agnostic Terraform modules
- Provider-specific implementations
- Cost comparison analysis
- Failover and replication strategy

18.9.4 Exercise 4: Automated Disaster Recovery System

Implement automated disaster recovery for ML infrastructure. Include:

- AWS Backup configuration for RDS and EBS
- S3 cross-region replication for models and data
- Automated backup verification testing
- Recovery time objective (RTO) and recovery point objective (RPO) monitoring
- Automated failover procedures
- Recovery testing schedule

Deliverables:

- Terraform backup infrastructure code
- Python disaster recovery automation script
- Recovery runbook documentation
- Disaster recovery testing report

18.9.5 Exercise 5: Cost-Optimized Training Infrastructure

Build cost-optimized infrastructure for ML training. Implement:

- Spot instance autoscaling groups with fallback to on-demand
- Multiple instance type support for spot diversity
- Training queue-based autoscaling
- S3 intelligent tiering for training data
- Scheduled shutdown of development resources
- Cost anomaly detection and alerting
- Monthly budget enforcement

Deliverables:

- Terraform cost optimization module
- Cost tracking dashboard
- 30-day cost analysis report
- Optimization recommendations

18.9.6 Exercise 6: Kubernetes Operator for ML Workflows

Develop custom Kubernetes operator for ML training jobs. Create:

- Custom Resource Definition (CRD) for ML training
- Operator controller logic in Go or Python
- Support for distributed training patterns
- Integration with MLflow for experiment tracking
- Automatic resource cleanup after job completion
- Job scheduling and prioritization

Deliverables:

- Custom operator implementation
- CRD specification and examples
- Operator deployment manifests
- User guide and API documentation

18.9.7 Exercise 7: Infrastructure Monitoring and Observability

Implement comprehensive monitoring for ML infrastructure. Deploy:

- Prometheus for metrics collection
- Grafana dashboards for infrastructure visualization
- Loki for log aggregation
- Jaeger for distributed tracing
- AlertManager for alert routing
- Infrastructure health scoring system

Deliverables:

- Monitoring stack deployment code
- Custom Grafana dashboards for ML workloads
- Alert rules for infrastructure issues
- Runbook for common incidents

18.10 Summary

Infrastructure as Code is fundamental to building reliable, scalable, and cost-effective ML platforms. This chapter covered:

1. Terraform Modules:

- Modular ML infrastructure design with reusable components
- EKS clusters with GPU and CPU node groups
- S3, RDS, and caching infrastructure
- Comprehensive networking and security
- Environment-specific configurations

2. Kubernetes and Helm:

- Production-ready Helm charts for training and inference
- Distributed training with PyTorchJob
- Autoscaling based on workload metrics
- Resource quotas and pod disruption budgets
- Custom operators for ML workflows

3. Multi-Cloud Management:

- AWS (EKS, S3, RDS, SageMaker)
- GCP (GKE, Cloud Storage, Vertex AI)
- Azure (AKS, Blob Storage, Azure ML)
- Cloud-agnostic module design
- Cross-cloud data replication

4. Container Orchestration:

- Kubernetes operators for ML workloads
- Cluster autoscaling with priority-based expansion
- Resource quotas and namespace isolation
- GPU resource management
- Job scheduling and prioritization

5. Disaster Recovery:

- Automated backup with AWS Backup
- Cross-region replication for critical data
- Recovery automation with Python
- RTO/RPO monitoring and enforcement
- Regular disaster recovery testing

6. Cost Optimization:

- Spot instances for 60-80% cost savings
- Auto-scaling based on demand
- Storage lifecycle management
- Cost anomaly detection
- Budget alerts and enforcement
- Scheduled resource management

Key Takeaways:

1. **Automation:** Automate all infrastructure provisioning and management
2. **Modularity:** Design reusable, composable infrastructure modules
3. **Multi-Cloud:** Avoid vendor lock-in with cloud-agnostic designs
4. **Cost Awareness:** Continuously monitor and optimize infrastructure costs

5. **Resilience:** Build fault-tolerant systems with automated recovery
6. **Security:** Implement least privilege access and encryption by default
7. **Observability:** Monitor all infrastructure components comprehensively

Infrastructure as Code transforms ML infrastructure from manual, error-prone processes to automated, reliable, and auditable deployments. The investment in proper IaC practices pays dividends through faster deployment cycles, reduced operational burden, and improved system reliability.

Chapter 19

MLOps Platforms and Tools

19.1 Learning Objectives

After completing this chapter, readers will be able to:

- Build end-to-end ML pipelines with Kubeflow
- Compare and evaluate SageMaker, Vertex AI, and Azure ML
- Apply platform selection criteria for organizational needs
- Implement migration strategies between MLOps platforms
- Design and build custom MLOps platforms
- Assess platform maturity and capabilities
- Make informed build vs buy decisions
- Integrate multiple MLOps tools effectively

19.2 Introduction

MLOps platforms provide integrated environments for developing, deploying, and managing machine learning models at scale. The platform landscape ranges from open-source frameworks to fully managed cloud services, each with distinct capabilities, trade-offs, and use cases.

MLOps Platform Categories:

- **Open-Source Frameworks:** Kubeflow, MLflow, Metaflow (flexibility, customization)
- **Cloud-Native Platforms:** AWS SageMaker, Google Vertex AI, Azure ML (managed services)
- **Specialized Tools:** DataRobot, Dataiku, Databricks (specific workflows)
- **Custom Solutions:** In-house platforms built on orchestrators (full control)

Platform Selection Factors:

1. **Cloud Strategy:** Single-cloud, multi-cloud, or hybrid deployments
2. **Team Size:** Platform operations overhead vs team capacity
3. **Use Cases:** Batch inference, real-time serving, edge deployment
4. **Existing Infrastructure:** Kubernetes, cloud services, on-premises
5. **Compliance:** Data residency, security certifications, audit requirements
6. **Budget:** TCO including licensing, infrastructure, and operations
7. **Vendor Lock-in:** Portability and exit strategy considerations

19.3 Kubeflow Pipelines: Comprehensive Tutorial

Kubeflow is an open-source ML platform for Kubernetes that provides components for building production ML systems.

19.3.1 Kubeflow Architecture and Components

```

1  """
2  Kubeflow Platform Components:
3
4  1. Kubeflow Pipelines - Workflow orchestration for ML
5  2. Jupyter Notebooks - Interactive development environment
6  3. Katib - Hyperparameter tuning and AutoML
7  4. KFServing (KServe) - Model serving and inference
8  5. Training Operators - PyTorch, TensorFlow distributed training
9  6. Metadata - ML metadata tracking and lineage
10 7. Multi-Tenancy - Namespace isolation and resource management
11 """
12
13 # Key Kubeflow Benefits:
14 # - Cloud-agnostic: runs on any Kubernetes cluster
15 # - Portable: pipelines can run across different environments
16 # - Composable: mix and match components based on needs
17 # - Scalable: leverage Kubernetes for resource management
18 # - Open-source: no vendor lock-in, community-driven

```

Listing 19.1: Kubeflow Platform Components Overview

19.3.2 Building ML Pipelines with Kubeflow

```

1  # pipeline.py
2  """
3  End-to-end ML pipeline with Kubeflow Pipelines SDK
4  Covers data processing, training, evaluation, and deployment
5  """
6
7  import kfp
8  from kfp import dsl
9  from kfp.components import create_component_from_func, InputPath, OutputPath

```



```

10 from typing import NamedTuple
11
12 # Component 1: Data Ingestion
13 @create_component_from_func
14 def ingest_data(
15     data_source: str,
16     output_dataset: OutputPath('Dataset')
17 ) -> NamedTuple('Outputs', [('num_samples', int), ('num_features', int)]):
18     """Ingest data from source and save as dataset"""
19
20     import pandas as pd
21     import os
22
23     # Load data
24     if data_source.startswith('s3://'):
25         import boto3
26         # Load from S3
27         df = pd.read_csv(data_source)
28     else:
29         df = pd.read_csv(data_source)
30
31     # Save dataset
32     os.makedirs(os.path.dirname(output_dataset), exist_ok=True)
33     df.to_csv(output_dataset, index=False)
34
35     return (len(df), len(df.columns))
36
37 # Component 2: Data Validation
38 @create_component_from_func
39 def validate_data(
40     dataset: InputPath('Dataset'),
41     validation_report: OutputPath('ValidationReport')
42 ) -> NamedTuple('Outputs', [('is_valid', bool), ('quality_score', float)]):
43     """Validate data quality and schema"""
44
45     import pandas as pd
46     import json
47     import numpy as np
48
49     df = pd.read_csv(dataset)
50
51     # Validation checks
52     checks = {
53         'null_percentage': df.isnull().sum().sum() / (len(df) * len(df.columns)),
54         'duplicate_rows': len(df[df.duplicated()]),
55         'outliers_percentage': 0.0 # Simplified
56     }
57
58     # Calculate quality score
59     quality_score = 1.0 - checks['null_percentage']
60     is_valid = quality_score > 0.95
61
62     # Save validation report
63     with open(validation_report, 'w') as f:
64         json.dump(checks, f, indent=2)
65
66     return (is_valid, float(quality_score))
67

```

```

68 # Component 3: Feature Engineering
69 @create_component_from_func
70 def engineer_features(
71     dataset: InputPath('Dataset'),
72     output_features: OutputPath('Features'),
73     target_column: str = 'target'
74 ):
75     """Engineer features from raw data"""
76
77     import pandas as pd
78     from sklearn.preprocessing import StandardScaler
79     import pickle
80
81     df = pd.read_csv(dataset)
82
83     # Separate features and target
84     X = df.drop(columns=[target_column])
85     y = df[target_column]
86
87     # Feature engineering
88     # 1. Handle missing values
89     X = X.fillna(X.median())
90
91     # 2. Scale features
92     scaler = StandardScaler()
93     X_scaled = scaler.fit_transform(X)
94
95     # 3. Combine back
96     feature_df = pd.DataFrame(X_scaled, columns=X.columns)
97     feature_df[target_column] = y.values
98
99     # Save engineered features
100    feature_df.to_csv(output_features, index=False)
101
102    print(f"Engineered {len(feature_df.columns)-1} features")
103
104 # Component 4: Train-Test Split
105 @create_component_from_func
106 def split_data(
107     features: InputPath('Features'),
108     train_data: OutputPath('TrainData'),
109     test_data: OutputPath('TestData'),
110     test_size: float = 0.2,
111     random_state: int = 42
112 ):
113     """Split data into train and test sets"""
114
115     import pandas as pd
116     from sklearn.model_selection import train_test_split
117
118     df = pd.read_csv(features)
119
120     train_df, test_df = train_test_split(
121         df,
122         test_size=test_size,
123         random_state=random_state
124     )
125

```

```

126     train_df.to_csv(train_data, index=False)
127     test_df.to_csv(test_data, index=False)
128
129     print(f"Train samples: {len(train_df)}, Test samples: {len(test_df)}")
130
131     # Component 5: Model Training
132     @create_component_from_func
133     def train_model(
134         train_data: InputPath('TrainData'),
135         model_output: OutputPath('Model'),
136         model_type: str = 'random_forest',
137         n_estimators: int = 100,
138         max_depth: int = 10
139     ) -> NamedTuple('Outputs', [(('train_accuracy', float), ('train_time', float))]):
140         """Train ML model"""
141
142         import pandas as pd
143         import pickle
144         import time
145         from sklearn.ensemble import RandomForestClassifier
146         from sklearn.linear_model import LogisticRegression
147         from sklearn.metrics import accuracy_score
148
149         # Load data
150         df = pd.read_csv(train_data)
151         X_train = df.drop(columns=['target'])
152         y_train = df['target']
153
154         # Select and train model
155         start_time = time.time()
156
157         if model_type == 'random_forest':
158             model = RandomForestClassifier(
159                 n_estimators=n_estimators,
160                 max_depth=max_depth,
161                 random_state=42
162             )
163         else:
164             model = LogisticRegression(max_iter=1000, random_state=42)
165
166         model.fit(X_train, y_train)
167         train_time = time.time() - start_time
168
169         # Calculate training accuracy
170         train_pred = model.predict(X_train)
171         train_acc = accuracy_score(y_train, train_pred)
172
173         # Save model
174         with open(model_output, 'wb') as f:
175             pickle.dump(model, f)
176
177         print(f"Model trained: {model_type}, Accuracy: {train_acc:.4f}")
178
179         return (float(train_acc), float(train_time))
180
181     # Component 6: Model Evaluation
182     @create_component_from_func
183     def evaluate_model(

```

```

184     model: InputPath('Model'),
185     test_data: InputPath('TestData'),
186     metrics_output: OutputPath('Metrics')
187 ) -> NamedTuple('Outputs', [
188     ('test_accuracy', float),
189     ('test_f1', float),
190     ('test_auc', float)
191 ]):
192     """Evaluate model on test data"""
193
194     import pandas as pd
195     import pickle
196     import json
197     from sklearn.metrics import (
198         accuracy_score,
199         f1_score,
200         roc_auc_score,
201         classification_report,
202         confusion_matrix
203     )
204
205     # Load model and data
206     with open(model, 'rb') as f:
207         model = pickle.load(f)
208
209     df = pd.read_csv(test_data)
210     X_test = df.drop(columns=['target'])
211     y_test = df['target']
212
213     # Make predictions
214     y_pred = model.predict(X_test)
215     y_proba = model.predict_proba(X_test)[:, 1] if hasattr(model, 'predict_proba')
216     else y_pred
217
218     # Calculate metrics
219     test_acc = accuracy_score(y_test, y_pred)
220     test_f1 = f1_score(y_test, y_pred, average='weighted')
221     test_auc = roc_auc_score(y_test, y_proba)
222
223     # Detailed metrics
224     metrics = {
225         'accuracy': float(test_acc),
226         'f1_score': float(test_f1),
227         'auc_roc': float(test_auc),
228         'classification_report': classification_report(y_test, y_pred, output_dict=
229         True),
230         'confusion_matrix': confusion_matrix(y_test, y_pred).tolist()
231     }
232
233     # Save metrics
234     with open(metrics_output, 'w') as f:
235         json.dump(metrics, f, indent=2)
236
237     print(f"Test Accuracy: {test_acc:.4f}, F1: {test_f1:.4f}, AUC: {test_auc:.4f}")
238
239     return (float(test_acc), float(test_f1), float(test_auc))

```

Component 7: Model Deployment Decision

```

240 @create_component_from_func
241 def deployment_decision(
242     test_accuracy: float,
243     test_f1: float,
244     accuracy_threshold: float = 0.85,
245     f1_threshold: float = 0.80
246 ) -> NamedTuple('Outputs', [(('should_deploy', bool), ('decision_reason', str))]):
247     """Decide whether to deploy model based on metrics"""
248
249     if test_accuracy >= accuracy_threshold and test_f1 >= f1_threshold:
250         return (True, f"Metrics above threshold: acc={test_accuracy:.4f}, f1={test_f1:.4f}")
251     else:
252         return (False, f"Metrics below threshold: acc={test_accuracy:.4f}, f1={test_f1:.4f}")
253
254 # Component 8: Model Registration
255 @create_component_from_func
256 def register_model(
257     model: InputPath('Model'),
258     model_name: str,
259     model_version: str,
260     metrics: InputPath('Metrics')
261 ):
262     """Register model in model registry"""
263
264     import mlflow
265     import pickle
266     import json
267
268     # Load model and metrics
269     with open(model, 'rb') as f:
270         model_obj = pickle.load(f)
271
272     with open(metrics, 'r') as f:
273         metrics_dict = json.load(f)
274
275     # Register with MLflow
276     mlflow.set_tracking_uri("http://mlflow-server:5000")
277
278     with mlflow.start_run():
279         # Log metrics
280         mlflow.log_metrics({
281             'test_accuracy': metrics_dict['accuracy'],
282             'test_f1': metrics_dict['f1_score'],
283             'test_auc': metrics_dict['auc_roc']
284         })
285
286         # Log model
287         mlflow.sklearn.log_model(
288             model_obj,
289             "model",
290             registered_model_name=model_name
291         )
292
293         # Add version tag
294         mlflow.set_tag("version", model_version)
295

```

```

296     print(f"Model registered: {model_name} version {model_version}")
297
298     # Define the Pipeline
299     @dsl.pipeline(
300         name='ML Training Pipeline',
301         description='End-to-end ML pipeline with data processing, training, and deployment',
302     )
303     def ml_training_pipeline(
304         data_source: str = 's3://ml-data/dataset.csv',
305         model_type: str = 'random_forest',
306         n_estimators: int = 100,
307         max_depth: int = 10,
308         test_size: float = 0.2,
309         accuracy_threshold: float = 0.85,
310         model_name: str = 'classifier-model',
311         model_version: str = 'v1.0'
312     ):
313         """Complete ML training pipeline"""
314
315         # Step 1: Ingest data
316         ingest_task = ingest_data(data_source=data_source)
317
318         # Step 2: Validate data
319         validate_task = validate_data(dataset=ingest_task.outputs['output_dataset'])
320
321         # Step 3: Engineer features (only if validation passes)
322         with dsl.Condition(validate_task.outputs['is_valid'] == True):
323             engineer_task = engineer_features(
324                 dataset=ingest_task.outputs['output_dataset']
325             )
326
327         # Step 4: Split data
328         split_task = split_data(
329             features=engineer_task.outputs['output_features'],
330             test_size=test_size
331         )
332
333         # Step 5: Train model
334         train_task = train_model(
335             train_data=split_task.outputs['train_data'],
336             model_type=model_type,
337             n_estimators=n_estimators,
338             max_depth=max_depth
339         )
340
341         # Step 6: Evaluate model
342         eval_task = evaluate_model(
343             model=train_task.outputs['model_output'],
344             test_data=split_task.outputs['test_data']
345         )
346
347         # Step 7: Deployment decision
348         decision_task = deployment_decision(
349             test_accuracy=eval_task.outputs['test_accuracy'],
350             test_f1=eval_task.outputs['test_f1'],
351             accuracy_threshold=accuracy_threshold
352         )

```

```

353
354     # Step 8: Register model (only if deployment approved)
355     with dsl.Condition(decision_task.outputs['should_deploy'] == True):
356         register_task = register_model(
357             model=train_task.outputs['model_output'],
358             model_name=model_name,
359             model_version=model_version,
360             metrics=eval_task.outputs['metrics_output']
361         )
362
363 # Compile and run the pipeline
364 if __name__ == '__main__':
365     import kfp.compiler as compiler
366
367     # Compile pipeline to YAML
368     compiler.Compiler().compile(
369         ml_training_pipeline,
370         'ml_training_pipeline.yaml'
371     )
372
373     # Create client and run
374     client = kfp.Client(host='http://localhost:8080')
375
376     # Submit pipeline run
377     run = client.create_run_from_pipeline_func(
378         ml_training_pipeline,
379         arguments={
380             'data_source': 's3://ml-data/classification_data.csv',
381             'model_type': 'random_forest',
382             'n_estimators': 100,
383             'max_depth': 10,
384             'accuracy_threshold': 0.85,
385             'model_name': 'production-classifier',
386             'model_version': 'v1.0'
387         }
388     )
389
390     print(f"Pipeline run created: {run.run_id}")

```

Listing 19.2: Complete Kubeflow Pipeline Example

19.3.3 Kubeflow Katib for Hyperparameter Tuning

```

1 # katib-experiment.yaml
2 apiVersion: kubeflow.org/v1beta1
3 kind: Experiment
4 metadata:
5     name: random-forest-hpo
6     namespace: kubeflow
7 spec:
8     objective:
9         type: maximize
10        goal: 0.95
11        objectiveMetricName: accuracy
12        additionalMetricNames:
13            - f1_score
14            - auc

```

```
15
16 algorithm:
17   algorithmName: bayesian
18   algorithmSettings:
19     - name: random_state
20       value: "42"
21
22 parallelTrialCount: 3
23 maxTrialCount: 20
24 maxFailedTrialCount: 3
25
26 parameters:
27   - name: n_estimators
28     parameterType: int
29     feasibleSpace:
30       min: "50"
31       max: "500"
32       step: "50"
33
34   - name: max_depth
35     parameterType: int
36     feasibleSpace:
37       min: "3"
38       max: "20"
39
40   - name: min_samples_split
41     parameterType: int
42     feasibleSpace:
43       min: "2"
44       max: "20"
45
46   - name: min_samples_leaf
47     parameterType: int
48     feasibleSpace:
49       min: "1"
50       max: "10"
51
52   - name: max_features
53     parameterType: categorical
54     feasibleSpace:
55       list:
56         - "auto"
57         - "sqrt"
58         - "log2"
59
60 trialTemplate:
61   primaryContainerName: training
62   trialParameters:
63     - name: n_estimators
64       description: Number of trees
65       reference: n_estimators
66     - name: max_depth
67       description: Maximum depth
68       reference: max_depth
69     - name: min_samples_split
70       description: Minimum samples to split
71       reference: min_samples_split
72     - name: min_samples_leaf
```



```

73     description: Minimum samples in leaf
74     reference: min_samples_leaf
75   - name: max_features
76     description: Maximum features
77     reference: max_features
78
79   trialSpec:
80     apiVersion: batch/v1
81     kind: Job
82     spec:
83       template:
84         spec:
85           containers:
86             - name: training
87               image: ml-training:latest
88               command:
89                 - python
90                 - train.py
91                 - --n-estimators=${trialParameters.n_estimators}
92                 - --max-depth=${trialParameters.max_depth}
93                 - --min-samples-split=${trialParameters.min_samples_split}
94                 - --min-samples-leaf=${trialParameters.min_samples_leaf}
95                 - --max-features=${trialParameters.max_features}
96                 - --output-file=/tmp/metrics.json
97
98           volumeMounts:
99             - name: data
100               mountPath: /data
101             - name: models
102               mountPath: /models
103
104           volumes:
105             - name: data
106               persistentVolumeClaim:
107                 claimName: training-data
108             - name: models
109               persistentVolumeClaim:
110                 claimName: model-storage
111
112     restartPolicy: Never

```

Listing 19.3: Katib Experiment for Hyperparameter Optimization

19.3.4 KServe for Model Deployment

```

1  # inference-service.yaml
2  apiVersion: serving.kserve.io/v1beta1
3  kind: InferenceService
4  metadata:
5    name: sklearn-classifier
6    namespace: ml-serving
7  spec:
8    predictor:
9      minReplicas: 2
10     maxReplicas: 10
11
12     sklearn:

```

```

13     storageUri: s3://ml-models/sklearn-classifier/v1
14     resources:
15         requests:
16             cpu: "1"
17             memory: 2Gi
18         limits:
19             cpu: "2"
20             memory: 4Gi
21
22     # Autoscaling configuration
23     scaleTarget: 75
24     scaleMetric: concurrency
25
26     # Canary rollout (optional)
27     canaryTrafficPercent: 20
28
29     # Transformer for preprocessing (optional)
30     transformer:
31         containers:
32             - name: preprocessor
33               image: ml-preprocessor:latest
34               env:
35                 - name: STORAGE_URI
36                   value: s3://ml-models/preprocessor/v1
37             resources:
38                 requests:
39                     cpu: "500m"
40                     memory: 1Gi
41
42     # Explainer for model interpretability (optional)
43     explainer:
44         containers:
45             - name: explainer
46               image: kserve/alibi-explainer:latest
47               args:
48                 - --model_name=sklearn-classifier
49                 - --predictor_host=sklearn-classifier-predictor
50             resources:
51                 requests:
52                     cpu: "500m"
53                     memory: 1Gi

```

Listing 19.4: KServe InferenceService Configuration

19.4 Cloud ML Platforms Comparison

Managed ML platforms provide end-to-end capabilities without infrastructure management overhead.

19.4.1 AWS SageMaker

```

1 # sagemaker_pipeline.py
2 """
3 Complete ML pipeline using AWS SageMaker
4 """

```

```

5
6 import boto3
7 import sagemaker
8 from sagemaker.sklearn import SKLearn
9 from sagemaker.processing import ScriptProcessor
10 from sagemaker.workflow.pipeline import Pipeline
11 from sagemaker.workflow.steps import ProcessingStep, TrainingStep
12 from sagemaker.workflow.parameters import ParameterInteger, ParameterString
13 from sagemaker.inputs import TrainingInput
14 from sagemaker.model import Model
15 from sagemaker.workflow.model_step import ModelStep
16
17 # Initialize SageMaker session
18 sess = sagemaker.Session()
19 role = sagemaker.get_execution_role()
20 bucket = sess.default_bucket()
21 region = boto3.Session().region_name
22
23 # Define pipeline parameters
24 processing_instance_type = ParameterString(
25     name="ProcessingInstanceType",
26     default_value="ml.m5.xlarge"
27 )
28
29 training_instance_type = ParameterString(
30     name="TrainingInstanceType",
31     default_value="ml.m5.2xlarge"
32 )
33
34 model_approval_status = ParameterString(
35     name="ModelApprovalStatus",
36     default_value="PendingManualApproval"
37 )
38
39 # Step 1: Data Processing
40 script_processor = ScriptProcessor(
41     image_uri=f"683313688378.dkr.ecr.{region}.amazonaws.com/sagemaker-scikit-learn
42     :0.23-1-cpu-py3",
43     role=role,
44     instance_type=processing_instance_type,
45     instance_count=1,
46     command=["python3"]
47 )
48
49 processing_step = ProcessingStep(
50     name="DataProcessing",
51     processor=script_processor,
52     inputs=[
53         sagemaker.processing.ProcessingInput(
54             source=f"s3://{bucket}/input/data.csv",
55             destination="/opt/ml/processing/input"
56         )
57     ],
58     outputs=[
59         sagemaker.processing.ProcessingOutput(
60             source="/opt/ml/processing/train",
61             destination=f"s3://{bucket}/processed/train"

```

```

62         sagemaker.processing.ProcessingOutput(
63             source="/opt/ml/processing/test",
64             destination=f"s3://{bucket}/processed/test"
65         )
66     ],
67     code="preprocessing.py"
68 )
69
70 # Step 2: Model Training
71 sklearn_estimator = SKLearn(
72     entry_point="train.py",
73     role=role,
74     instance_type=training_instance_type,
75     instance_count=1,
76     framework_version="0.23-1",
77     hyperparameters={
78         "n_estimators": 100,
79         "max_depth": 10
80     }
81 )
82
83 training_step = TrainingStep(
84     name="ModelTraining",
85     estimator=sklearn_estimator,
86     inputs={
87         "train": TrainingInput(
88             s3_data=processing_step.properties.ProcessingOutputConfig.Outputs[
89                 "train"
90             ].S3Output.S3Uri,
91             content_type="text/csv"
92         ),
93         "test": TrainingInput(
94             s3_data=processing_step.properties.ProcessingOutputConfig.Outputs[
95                 "test"
96             ].S3Output.S3Uri,
97             content_type="text/csv"
98         )
99     }
100 )
101
102 # Step 3: Model Evaluation
103 evaluation_processor = ScriptProcessor(
104     image_uri=f"683313688378.dkr.ecr.{region}.amazonaws.com/sagemaker-scikit-learn
105 :0.23-1-cpu-py3",
106     role=role,
107     instance_type=processing_instance_type,
108     instance_count=1,
109     command=["python3"]
110 )
111
112 evaluation_step = ProcessingStep(
113     name="ModelEvaluation",
114     processor=evaluation_processor,
115     inputs=[
116         sagemaker.processing.ProcessingInput(
117             source=training_step.properties.ModelArtifacts.S3ModelArtifacts,
118             destination="/opt/ml/processing/model"

```

```

119     sagemaker.processing.ProcessingInput(
120         source=processing_step.properties.ProcessingOutputConfig.Outputs[
121             "test"
122         ].S3Output.S3Uri,
123         destination="/opt/ml/processing/test"
124     )
125 ],
126 outputs=[
127     sagemaker.processing.ProcessingOutput(
128         source="/opt/ml/processing/evaluation",
129         destination=f"s3://{bucket}/evaluation"
130     )
131 ],
132 code="evaluate.py"
133 )
134
135 # Step 4: Model Registration
136 model = Model(
137     image_uri=training_step.properties.AlgorithmSpecification.TrainingImage,
138     model_data=training_step.properties.ModelArtifacts.S3ModelArtifacts,
139     sagemaker_session=sess,
140     role=role
141 )
142
143 model_step = ModelStep(
144     name="RegisterModel",
145     step_args=model.register(
146         content_types=["text/csv"],
147         response_types=["text/csv"],
148         inference_instances=["ml.m5.xlarge"],
149         transform_instances=["ml.m5.xlarge"],
150         model_package_group_name="ml-model-package-group",
151         approval_status=model_approval_status
152     )
153 )
154
155 # Create Pipeline
156 pipeline = Pipeline(
157     name="MLTrainingPipeline",
158     parameters=[
159         processing_instance_type,
160         training_instance_type,
161         model_approval_status
162     ],
163     steps=[
164         processing_step,
165         training_step,
166         evaluation_step,
167         model_step
168     ]
169 )
170
171 # Create/Update Pipeline
172 pipeline.upsert(role_arn=role)
173
174 # Start Pipeline Execution
175 execution = pipeline.start()
176 execution.wait()

```

```
177 print(f"Pipeline execution completed: {execution.arn}")
178
```

Listing 19.5: AWS SageMaker Training and Deployment

SageMaker Key Features:

- **SageMaker Studio:** Integrated development environment for ML
- **Feature Store:** Centralized feature repository with online/offline access
- **Autopilot:** Automated machine learning (AutoML)
- **Pipelines:** Native workflow orchestration with step caching
- **Model Monitor:** Automated drift detection and data quality monitoring
- **Clarify:** Bias detection and model explainability
- **Edge Manager:** Deploy and monitor models on edge devices
- **Training Compiler:** Accelerate training with optimized kernels

19.4.2 Google Cloud Vertex AI

```
1  # vertex_ai_pipeline.py
2  """
3  ML pipeline using Google Cloud Vertex AI
4  """
5
6  from google.cloud import aiplatform
7  from google.cloud.aiplatform import pipeline_jobs
8  from kfp.v2 import dsl
9  from kfp.v2.dsl import component, Output, Input, Model, Dataset, Metrics
10
11  # Initialize Vertex AI
12  PROJECT_ID = "your-project-id"
13  REGION = "us-central1"
14  BUCKET = "gs://your-bucket"
15
16  aiplatform.init(project=PROJECT_ID, location=REGION, staging_bucket=BUCKET)
17
18  # Component 1: Data Preprocessing
19  @component(
20      base_image="python:3.9",
21      packages_to_install=["pandas", "scikit-learn", "google-cloud-storage"]
22  )
23  def preprocess_data(
24      input_data_uri: str,
25      train_data: Output[Dataset],
26      test_data: Output[Dataset]
27  ):
28      import pandas as pd
29      from sklearn.model_selection import train_test_split
30
31      # Load data from GCS
32      df = pd.read_csv(input_data_uri)
```

```

33
34     # Preprocess
35     df = df.dropna()
36
37     # Split
38     train_df, test_df = train_test_split(df, test_size=0.2, random_state=42)
39
40     # Save
41     train_df.to_csv(train_data.path, index=False)
42     test_df.to_csv(test_data.path, index=False)
43
44 # Component 2: Training
45 @component(
46     base_image="python:3.9",
47     packages_to_install=["pandas", "scikit-learn", "joblib"]
48 )
49 def train_model(
50     train_data: Input[Dataset],
51     model: Output[Model],
52     n_estimators: int = 100,
53     max_depth: int = 10
54 ):
55     import pandas as pd
56     from sklearn.ensemble import RandomForestClassifier
57     import joblib
58
59     # Load training data
60     df = pd.read_csv(train_data.path)
61     X_train = df.drop(columns=['target'])
62     y_train = df['target']
63
64     # Train model
65     clf = RandomForestClassifier(
66         n_estimators=n_estimators,
67         max_depth=max_depth,
68         random_state=42
69     )
70     clf.fit(X_train, y_train)
71
72     # Save model
73     joblib.dump(clf, model.path)
74
75 # Component 3: Evaluation
76 @component(
77     base_image="python:3.9",
78     packages_to_install=["pandas", "scikit-learn", "joblib"]
79 )
80 def evaluate_model(
81     model: Input[Model],
82     test_data: Input[Dataset],
83     metrics: Output[Metrics]
84 ):
85     import pandas as pd
86     from sklearn.metrics import accuracy_score, f1_score, roc_auc_score
87     import joblib
88
89     # Load model and test data
90     clf = joblib.load(model.path)

```

```

91 df = pd.read_csv(test_data.path)
92 X_test = df.drop(columns=['target'])
93 y_test = df['target']
94
95 # Evaluate
96 y_pred = clf.predict(X_test)
97 y_proba = clf.predict_proba(X_test)[: , 1]
98
99 accuracy = accuracy_score(y_test, y_pred)
100 f1 = f1_score(y_test, y_pred, average='weighted')
101 auc = roc_auc_score(y_test, y_proba)
102
103 # Log metrics
104 metrics.log_metric("accuracy", accuracy)
105 metrics.log_metric("f1_score", f1)
106 metrics.log_metric("auc", auc)
107
108 # Define Pipeline
109 @dsl.pipeline(
110     name="vertex-ai-training-pipeline",
111     description="Complete ML pipeline on Vertex AI"
112 )
113 def vertex_training_pipeline(
114     input_data_uri: str = f"{BUCKET}/data/dataset.csv",
115     n_estimators: int = 100,
116     max_depth: int = 10
117 ):
118     # Preprocess data
119     preprocess_task = preprocess_data(input_data_uri=input_data_uri)
120
121     # Train model
122     train_task = train_model(
123         train_data=preprocess_task.outputs["train_data"],
124         n_estimators=n_estimators,
125         max_depth=max_depth
126     )
127
128     # Evaluate model
129     evaluate_task = evaluate_model(
130         model=train_task.outputs["model"],
131         test_data=preprocess_task.outputs["test_data"]
132     )
133
134 # Compile pipeline
135 from kfp.v2 import compiler
136
137 compiler.Compiler().compile(
138     pipeline_func=vertex_training_pipeline,
139     package_path="vertex_pipeline.json"
140 )
141
142 # Submit pipeline job
143 job = pipeline_jobs.PipelineJob(
144     display_name="ml-training-pipeline",
145     template_path="vertex_pipeline.json",
146     parameter_values={
147         "input_data_uri": f"{BUCKET}/data/classification_data.csv",
148         "n_estimators": 100,

```



```

149     "max_depth": 10
150 },
151     enable_caching=True
152 )
153
154 job.submit()
155 print(f"Pipeline job submitted: {job.resource_name}")
156
157 # Deploy model to endpoint
158 deployed_model = aiplatform.Model.upload(
159     display_name="classification-model",
160     artifact_uri=f"{BUCKET}/models/",
161     serving_container_image_uri="us-docker.pkg.dev/vertex-ai/prediction/sklearn-cpu
162     .1-0:latest"
163 )
164
165 endpoint = deployed_model.deploy(
166     deployed_model_display_name="classification-endpoint",
167     machine_type="n1-standard-4",
168     min_replica_count=1,
169     max_replica_count=5,
170     traffic_percentage=100
171 )
172 print(f"Model deployed to endpoint: {endpoint.resource_name}")

```

Listing 19.6: Vertex AI Training and Deployment Pipeline

Vertex AI Key Features:

- **Workbench:** Managed Jupyter notebooks with pre-configured environments
- **Pipelines:** KFP-compatible pipeline orchestration
- **Feature Store:** Managed feature repository with serving infrastructure
- **AutoML:** No-code ML for various data types (tabular, image, text, video)
- **Matching Engine:** Vector similarity search for embeddings
- **Explainable AI:** Built-in model interpretability
- **Model Monitoring:** Automated skew and drift detection
- **Vizier:** Hyperparameter tuning service

19.4.3 Azure Machine Learning

```

1 # azure_ml_pipeline.py
2 """
3 ML pipeline using Azure Machine Learning
4 """
5
6 from azureml.core import Workspace, Experiment, Dataset, Environment
7 from azureml.core.compute import ComputeTarget, AmlCompute
8 from azureml.pipeline.core import Pipeline, PipelineData
9 from azureml.pipeline.steps import PythonScriptStep

```

```

10 from azureml.core.runconfig import RunConfiguration
11
12 # Connect to workspace
13 ws = Workspace.from_config()
14
15 # Create or get compute cluster
16 compute_name = "ml-cluster"
17 if compute_name not in ws.compute_targets:
18     compute_config = AmlCompute.provisioning_configuration(
19         vm_size="STANDARD_D2_V2",
20         min_nodes=0,
21         max_nodes=4
22     )
23     compute_target = ComputeTarget.create(ws, compute_name, compute_config)
24     compute_target.wait_for_completion(show_output=True)
25 else:
26     compute_target = ws.compute_targets[compute_name]
27
28 # Create environment
29 env = Environment.from_conda_specification(
30     name="ml-env",
31     file_path="conda_env.yml"
32 )
33 env.register(workspace=ws)
34
35 # Create run configuration
36 run_config = RunConfiguration()
37 run_config.target = compute_target
38 run_config.environment = env
39
40 # Define pipeline data
41 processed_data = PipelineData("processed_data", datastore=ws.get_default_datastore())
42 model_data = PipelineData("model_data", datastore=ws.get_default_datastore())
43
44 # Step 1: Data Preparation
45 prep_step = PythonScriptStep(
46     name="Data Preparation",
47     script_name="prepare_data.py",
48     arguments=[
49         "--input-data", Dataset.get_by_name(ws, "raw_data").as_named_input("raw_data")
50     ],
51     outputs=[processed_data],
52     compute_target=compute_target,
53     runconfig=run_config,
54     allow_reuse=True
55 )
56
57 # Step 2: Training
58 train_step = PythonScriptStep(
59     name="Model Training",
60     script_name="train.py",
61     arguments=[
62         "--input-data", processed_data,
63         "--output-model", model_data,
64         "--n-estimators", 100,
65         "--max-depth", 10

```

```

67     ],
68     inputs=[processed_data],
69     outputs=[model_data],
70     compute_target=compute_target,
71     runconfig=run_config,
72     allow_reuse=True
73 )
74
75 # Step 3: Evaluation
76 eval_step = PythonScriptStep(
77     name="Model Evaluation",
78     script_name="evaluate.py",
79     arguments=[
80         "--model-path", model_data,
81         "--test-data", processed_data
82     ],
83     inputs=[model_data, processed_data],
84     compute_target=compute_target,
85     runconfig=run_config,
86     allow_reuse=True
87 )
88
89 # Step 4: Registration
90 register_step = PythonScriptStep(
91     name="Model Registration",
92     script_name="register_model.py",
93     arguments=[
94         "--model-path", model_data
95     ],
96     inputs=[model_data],
97     compute_target=compute_target,
98     runconfig=run_config,
99     allow_reuse=False # Always register
100 )
101
102 # Create and publish pipeline
103 pipeline = Pipeline(
104     workspace=ws,
105     steps=[prep_step, train_step, eval_step, register_step]
106 )
107
108 pipeline.validate()
109 published_pipeline = pipeline.publish(
110     name="ML Training Pipeline",
111     description="End-to-end ML pipeline",
112     version="1.0"
113 )
114
115 # Run pipeline
116 experiment = Experiment(ws, "ml-pipeline-experiment")
117 pipeline_run = experiment.submit(pipeline)
118 pipeline_run.wait_for_completion(show_output=True)
119
120 # Deploy model as web service
121 from azureml.core.model import Model
122 from azureml.core.webservice import AciWebservice, Webservice
123 from azureml.core.model import InferenceConfig
124

```

```
125 # Get registered model
126 model = Model(ws, name="classification-model")
127
128 # Create inference configuration
129 inference_config = InferenceConfig(
130     entry_script="score.py",
131     environment=env
132 )
133
134 # Configure deployment
135 deployment_config = AciWebservice.deploy_configuration(
136     cpu_cores=1,
137     memory_gb=1,
138     auth_enabled=True,
139     enable_app_insights=True
140 )
141
142 # Deploy
143 service = Model.deploy(
144     workspace=ws,
145     name="classification-service",
146     models=[model],
147     inference_config=inference_config,
148     deployment_config=deployment_config,
149     overwrite=True
150 )
151
152 service.wait_for_deployment(show_output=True)
153 print(f"Service deployed: {service.scoring_uri}")
```

Listing 19.7: Azure ML Pipeline Implementation

Azure ML Key Features:

- **Designer:** Drag-and-drop ML pipeline builder
- **AutoML:** Automated model selection and hyperparameter tuning
- **MLflow Integration:** Native support for MLflow tracking
- **Responsible AI:** Fairness assessment and error analysis
- **Model Catalog:** Pre-trained models and foundation models
- **Managed Endpoints:** Online and batch inference endpoints
- **Data Labeling:** Built-in annotation tools
- **Compliance:** HIPAA, ISO, SOC certifications

Feature	Kubeflow	SageMaker	Vertex AI	Azure ML
Deployment	Self-managed K8s	Fully managed	Fully managed	Fully managed
Cloud Lock-in	None	AWS only	GCP only	Azure only
Cost Model	Infrastructure	Pay-per-use	Pay-per-use	Pay-per-use
Learning Curve	Steep	Moderate	Moderate	Moderate
Customization	Very high	High	Moderate	High
AutoML	Katib	Autopilot	AutoML	AutoML
Feature Store	Custom	Native	Native	Native
Model Registry	Custom MLflow	Native	Native	Native
Monitoring	Prometheus	Native	Native	Native
Edge Deployment	Manual	Edge Manager	Edge TPU	IoT Edge
Multi-cloud	Yes	No	No	No
On-premises	Yes	Outposts	Anthos	Arc
Python SDK	KFP SDK	Boto3/SDK	Vertex SDK	Azure SDK
Best For	Flexibility, control	AWS ecosystem	GCP ecosystem	Azure ecosystem

Table 19.1: MLOps Platform Comparison Matrix

19.5 Platform Comparison Matrix

19.6 Platform Selection Criteria and Decision Framework

Selecting the right MLOps platform requires evaluating organizational needs, technical requirements, and strategic alignment.

19.6.1 Decision Framework Matrix

```

1  #!/usr/bin/env python3
2  """
3  MLOps Platform Selection Framework
4  Systematic evaluation of platform options based on weighted criteria
5  """
6
7  from dataclasses import dataclass
8  from typing import List, Dict
9  from enum import Enum
10
11 class PlatformType(Enum):
12     KUBEFLOW = "Kubeflow"
13     SAGEMAKER = "AWS SageMaker"
14     VERTEX_AI = "Google Vertex AI"
15     AZURE_ML = "Azure ML"
16     CUSTOM = "Custom Platform"
17
18 @dataclass
19 class EvaluationCriterion:

```

```

20     """Single evaluation criterion with weight"""
21     name: str
22     weight: float # 0.0 to 1.0
23     description: str
24
25 @dataclass
26 class PlatformScore:
27     """Score for a platform on specific criterion"""
28     criterion: str
29     score: float # 0 to 10
30     justification: str
31
32 class PlatformEvaluator:
33     """Evaluate and compare MLOps platforms"""
34
35     def __init__(self):
36         self.criteria = self._define_criteria()
37         self.platforms = list(PlatformType)
38
39     def _define_criteria(self) -> List[EvaluationCriterion]:
40         """Define evaluation criteria with weights"""
41         return [
42             EvaluationCriterion(
43                 name="ease_of_use",
44                 weight=0.15,
45                 description="Learning curve and operational complexity"
46             ),
47             EvaluationCriterion(
48                 name="cost",
49                 weight=0.20,
50                 description="Total cost of ownership (infrastructure + operations)"
51             ),
52             EvaluationCriterion(
53                 name="scalability",
54                 weight=0.15,
55                 description="Ability to scale compute and handle large workloads"
56             ),
57             EvaluationCriterion(
58                 name="flexibility",
59                 weight=0.10,
60                 description="Customization and extensibility options"
61             ),
62             EvaluationCriterion(
63                 name="cloud_strategy",
64                 weight=0.15,
65                 description="Alignment with multi-cloud vs single-cloud strategy"
66             ),
67             EvaluationCriterion(
68                 name="feature_completeness",
69                 weight=0.10,
70                 description="Built-in capabilities (feature store, monitoring, etc.)"
71             ),
72             EvaluationCriterion(
73                 name="integration",
74                 weight=0.05,
75                 description="Integration with existing tools and infrastructure"
76             ),
77             EvaluationCriterion(

```

```

78         name="compliance",
79         weight=0.05,
80         description="Security certifications and data governance"
81     ),
82     EvaluationCriterion(
83         name="vendor_lock_in",
84         weight=0.05,
85         description="Exit strategy and portability (higher score = less lock-
in)"
86     )
87 ]
88
89 def evaluate_platform(
90     self,
91     platform: PlatformType,
92     scores: Dict[str, float],
93     justifications: Dict[str, str]
94 ) -> float:
95     """Calculate weighted score for a platform"""
96
97     total_score = 0.0
98     total_weight = sum(c.weight for c in self.criteria)
99
100    print(f"\nEvaluating {platform.value}:")
101    print("=" * 60)
102
103    for criterion in self.criteria:
104        if criterion.name not in scores:
105            print(f"Warning: Missing score for {criterion.name}")
106            continue
107
108        score = scores[criterion.name]
109        weighted_score = score * criterion.weight
110        total_score += weighted_score
111
112        justification = justifications.get(criterion.name, "No justification
provided")
113
114        print(f"\n{criterion.name.replace('_', ' ').title()}: {score}/10")
115        print(f"    Weight: {criterion.weight:.2f}")
116        print(f"    Weighted Score: {weighted_score:.2f}")
117        print(f"    Justification: {justification}")
118
119    # Normalize to 0-10 scale
120    normalized_score = (total_score / total_weight) * 10 if total_weight > 0 else
0
121
122    print(f"\n{'='*60}")
123    print(f"Total Weighted Score: {normalized_score:.2f}/10")
124
125    return normalized_score
126
127 # Example: Enterprise in AWS Ecosystem
128 def evaluate_aws_enterprise():
129     """Evaluation for enterprise already using AWS"""
130
131     evaluator = PlatformEvaluator()
132

```

```

133 # SageMaker scores for AWS-native organization
134 sagemaker_scores = {
135     "ease_of_use": 8.0, # Good documentation, managed service
136     "cost": 6.0, # Pay-per-use can be expensive at scale
137     "scalability": 9.0, # Excellent auto-scaling
138     "flexibility": 7.0, # Good customization with containers
139     "cloud_strategy": 10.0, # Perfect for AWS-first
140     "feature_completeness": 9.0, # Comprehensive features
141     "integration": 9.0, # Deep AWS integration
142     "compliance": 9.0, # Strong certifications
143     "vendor_lock_in": 4.0 # High lock-in to AWS
144 }
145
146 sagemaker_justifications = {
147     "ease_of_use": "Well-documented with good examples and Studio IDE",
148     "cost": "Pay-per-use pricing can add up, but offers savings plans",
149     "scalability": "Auto-scaling for training and inference",
150     "flexibility": "Bring-your-own containers and algorithms supported",
151     "cloud_strategy": "Native AWS service, no multi-cloud support",
152     "feature_completeness": "Feature Store, Pipelines, Clarify, Model Monitor",
153     "integration": "Seamless integration with S3, ECR, IAM, CloudWatch",
154     "compliance": "HIPAA, PCI DSS, SOC, ISO certified",
155     "vendor_lock_in": "Significant AWS-specific APIs and services"
156 }
157
158 # Kubeflow scores for comparison
159 kubeflow_scores = {
160     "ease_of_use": 4.0, # Steep learning curve
161     "cost": 8.0, # Just infrastructure costs
162     "scalability": 8.0, # Kubernetes scaling
163     "flexibility": 10.0, # Highly customizable
164     "cloud_strategy": 10.0, # Cloud-agnostic
165     "feature_completeness": 6.0, # Need to integrate tools
166     "integration": 7.0, # Requires configuration
167     "compliance": 7.0, # Depends on setup
168     "vendor_lock_in": 10.0 # No lock-in
169 }
170
171 kubeflow_justifications = {
172     "ease_of_use": "Complex setup, requires Kubernetes expertise",
173     "cost": "Only pay for infrastructure, no platform fees",
174     "scalability": "Leverages Kubernetes horizontal pod autoscaling",
175     "flexibility": "Complete control over all components",
176     "cloud_strategy": "Runs on any Kubernetes cluster, any cloud",
177     "feature_completeness": "Need to integrate MLflow, Prometheus, etc.",
178     "integration": "Requires custom integration with existing systems",
179     "compliance": "Achievable but requires manual configuration",
180     "vendor_lock_in": "Portable across clouds and on-premises"
181 }
182
183 sagemaker_score = evaluator.evaluate_platform(
184     PlatformType.SAGEMAKER,
185     sagemaker_scores,
186     sagemaker_justifications
187 )
188
189 kubeflow_score = evaluator.evaluate_platform(
190     PlatformType.KUBEFLOW,

```



```

191     kubeflow_scores,
192     kubeflow_justifications
193 )
194
195 print("\n" + "="*60)
196 print("RECOMMENDATION")
197 print("="*60)
198
199 if sagemaker_score > kubeflow_score:
200     print(f"SageMaker ({sagemaker_score:.2f}) recommended over Kubeflow ({kubeflow_score:.2f})")
201     print("\nReasons:")
202     print("- Already invested in AWS ecosystem")
203     print("- Team prefers managed services over self-hosting")
204     print("- Compliance certifications important")
205     print("- Faster time-to-value with less operational overhead")
206 else:
207     print(f"Kubeflow ({kubeflow_score:.2f}) recommended over SageMaker ({sagemaker_score:.2f})")
208     print("\nReasons:")
209     print("- Need for multi-cloud portability")
210     print("- Cost optimization priority")
211     print("- Team has strong Kubernetes expertise")
212     print("- Customization requirements")
213
214 if __name__ == "__main__":
215     evaluate_aws_enterprise()

```

Listing 19.8: Platform Selection Decision Framework

Key Selection Factors:**1. Cloud Strategy:**

- Single cloud committed: Choose native platform (SageMaker/Vertex/Azure ML)
- Multi-cloud: Kubeflow or abstraction layer (MLflow + custom)
- Hybrid: Consider compatibility with on-premises Kubernetes

2. Team Maturity:

- Kubernetes experts: Kubeflow viable option
- General software engineers: Managed platforms preferred
- Small teams: Avoid operational overhead of self-hosting

3. Budget Considerations:

- High compute costs: Kubeflow with spot instances
- Predictable workloads: Reserved instances on managed platforms
- Variable workloads: Pay-per-use managed services

4. Compliance Requirements:

- HIPAA/PCI DSS: All major platforms support

- Data residency: Check regional availability
- Custom requirements: May require self-hosted solution

5. Integration Needs:

- Existing AWS infrastructure: SageMaker natural fit
- Multi-cloud data lakes: Platform-agnostic tools
- Legacy systems: Custom integration layer needed

19.7 Migration Strategies Between Platforms

Platform migration requires careful planning to minimize disruption and avoid data loss.

19.7.1 Migration Planning Framework

```

1  #!/usr/bin/env python3
2  """
3  MLOps Platform Migration Framework
4  Systematic approach to migrating between platforms
5  """
6
7  from enum import Enum
8  from dataclasses import dataclass
9  from typing import List, Optional
10 import json
11
12 class MigrationPhase(Enum):
13     ASSESSMENT = "assessment"
14     PLANNING = "planning"
15     PILOT = "pilot"
16     PARALLEL_RUN = "parallel_run"
17     CUTOVER = "cutover"
18     DECOMMISSION = "decommission"
19
20 @dataclass
21 class MigrationTask:
22     """Single migration task"""
23     name: str
24     phase: MigrationPhase
25     description: str
26     dependencies: List[str]
27     estimated_days: int
28     risk_level: str # low, medium, high
29     mitigation: str
30
31 class PlatformMigrationPlanner:
32     """Plan and track platform migration"""
33
34     def __init__(self, source_platform: str, target_platform: str):
35         self.source = source_platform
36         self.target = target_platform
37         self.tasks = self._generate_migration_tasks()
38
39     def _generate_migration_tasks(self) -> List[MigrationTask]:

```

```

40     """Generate comprehensive migration task list"""
41
42     return [
43         # Phase 1: Assessment
44         MigrationTask(
45             name="inventory_workloads",
46             phase=MigrationPhase.ASSESSMENT,
47             description="Catalog all ML workloads, pipelines, and models",
48             dependencies=[],
49             estimated_days=5,
50             risk_level="low",
51             mitigation="Use automated discovery tools"
52         ),
53         MigrationTask(
54             name="assess_dependencies",
55             phase=MigrationPhase.ASSESSMENT,
56             description="Map dependencies between models, data, and services",
57             dependencies=["inventory_workloads"],
58             estimated_days=3,
59             risk_level="medium",
60             mitigation="Document all integration points"
61         ),
62         MigrationTask(
63             name="cost_analysis",
64             phase=MigrationPhase.ASSESSMENT,
65             description="Compare costs between current and target platform",
66             dependencies=["inventory_workloads"],
67             estimated_days=3,
68             risk_level="low",
69             mitigation="Include hidden costs (training, operations)"
70         ),
71
72         # Phase 2: Planning
73         MigrationTask(
74             name="design_target_architecture",
75             phase=MigrationPhase.PLANNING,
76             description="Design architecture on target platform",
77             dependencies=["assess_dependencies"],
78             estimated_days=10,
79             risk_level="medium",
80             mitigation="Involve platform experts, peer review"
81         ),
82         MigrationTask(
83             name="create_migration_runbook",
84             phase=MigrationPhase.PLANNING,
85             description="Document detailed migration procedures",
86             dependencies=["design_target_architecture"],
87             estimated_days=5,
88             risk_level="medium",
89             mitigation="Include rollback procedures"
90         ),
91         MigrationTask(
92             name="setup_target_infrastructure",
93             phase=MigrationPhase.PLANNING,
94             description="Provision target platform infrastructure",
95             dependencies=["design_target_architecture"],
96             estimated_days=7,
97             risk_level="high",

```

```

98         mitigation="Use Infrastructure as Code for reproducibility"
99     ),
100
101     # Phase 3: Pilot
102     MigrationTask(
103         name="migrate_pilot_pipeline",
104         phase=MigrationPhase.PILOT,
105         description="Migrate non-critical pipeline to validate approach",
106         dependencies=["setup_target_infrastructure", "create_migration_runbook
107     ],
108         estimated_days=10,
109         risk_level="medium",
110         mitigation="Choose simple pipeline for learning"
111     ),
112     MigrationTask(
113         name="validate_pilot_results",
114         phase=MigrationPhase.PILOT,
115         description="Verify pilot pipeline produces identical results",
116         dependencies=["migrate_pilot_pipeline"],
117         estimated_days=5,
118         risk_level="high",
119         mitigation="Automated comparison of outputs"
120     ),
121     MigrationTask(
122         name="refine_migration_process",
123         phase=MigrationPhase.PILOT,
124         description="Update procedures based on pilot learnings",
125         dependencies=["validate_pilot_results"],
126         estimated_days=3,
127         risk_level="low",
128         mitigation="Document all issues and resolutions"
129     ),
130
131     # Phase 4: Parallel Run
132     MigrationTask(
133         name="migrate_all_pipelines",
134         phase=MigrationPhase.PARALLEL_RUN,
135         description="Migrate remaining pipelines to target platform",
136         dependencies=["refine_migration_process"],
137         estimated_days=20,
138         risk_level="high",
139         mitigation="Stagger migrations, maintain source pipelines"
140     ),
141     MigrationTask(
142         name="sync_data_stores",
143         phase=MigrationPhase.PARALLEL_RUN,
144         description="Replicate data between platforms",
145         dependencies=["migrate_all_pipelines"],
146         estimated_days=7,
147         risk_level="high",
148         mitigation="Use data versioning, validate checksums"
149     ),
150     MigrationTask(
151         name="parallel_validation",
152         phase=MigrationPhase.PARALLEL_RUN,
153         description="Run both platforms in parallel, compare results",
154         dependencies=["sync_data_stores"],
155         estimated_days=14,

```

```

155         risk_level="high",
156         mitigation="Automated result comparison dashboard"
157     ),
158
159     # Phase 5: Cutover
160     MigrationTask(
161         name="switch_production_traffic",
162         phase=MigrationPhase.CUTOVER,
163         description="Route production traffic to new platform",
164         dependencies=["parallel_validation"],
165         estimated_days=1,
166         risk_level="critical",
167         mitigation="Canary rollout, immediate rollback capability"
168     ),
169     MigrationTask(
170         name="monitor_performance",
171         phase=MigrationPhase.CUTOVER,
172         description="Intensive monitoring of new platform",
173         dependencies=["switch_production_traffic"],
174         estimated_days=7,
175         risk_level="high",
176         mitigation="24/7 on-call, automated alerts"
177     ),
178
179     # Phase 6: Decommission
180     MigrationTask(
181         name="archive_source_artifacts",
182         phase=MigrationPhase.DECOMMISSION,
183         description="Archive models, data, and configurations",
184         dependencies=["monitor_performance"],
185         estimated_days=5,
186         risk_level="medium",
187         mitigation="Verify archives are complete and accessible"
188     ),
189     MigrationTask(
190         name="decommission_source_platform",
191         phase=MigrationPhase.DECOMMISSION,
192         description="Shut down source platform infrastructure",
193         dependencies=["archive_source_artifacts"],
194         estimated_days=3,
195         risk_level="low",
196         mitigation="Staged shutdown with verification steps"
197     )
198 ]
199
200 def generate_timeline(self) -> dict:
201     """Generate project timeline"""
202
203     timeline = {}
204     for phase in MigrationPhase:
205         phase_tasks = [t for t in self.tasks if t.phase == phase]
206         total_days = sum(t.estimated_days for t in phase_tasks)
207         timeline[phase.value] = {
208             "tasks": len(phase_tasks),
209             "estimated_days": total_days,
210             "tasks_list": [t.name for t in phase_tasks]
211         }
212

```

```

213     return timeline
214
215     def identify_critical_path(self) -> List[str]:
216         """Identify tasks on critical path"""
217
218         # Simplified: tasks with dependencies form critical path
219         critical_tasks = []
220
221         for task in self.tasks:
222             if task.risk_level in ["high", "critical"]:
223                 critical_tasks.append(task.name)
224
225         return critical_tasks
226
227     def export_plan(self, filename: str):
228         """Export migration plan to JSON"""
229
230         plan = {
231             "migration": {
232                 "source": self.source,
233                 "target": self.target
234             },
235             "timeline": self.generate_timeline(),
236             "critical_path": self.identify_critical_path(),
237             "all_tasks": [
238                 {
239                     "name": t.name,
240                     "phase": t.phase.value,
241                     "description": t.description,
242                     "dependencies": t.dependencies,
243                     "estimated_days": t.estimated_days,
244                     "risk_level": t.risk_level,
245                     "mitigation": t.mitigation
246                 }
247                 for t in self.tasks
248             ]
249         }
250
251         with open(filename, 'w') as f:
252             json.dump(plan, f, indent=2)
253
254         print(f"Migration plan exported to {filename}")
255
256     # Example usage
257     if __name__ == "__main__":
258         planner = PlatformMigrationPlanner(
259             source_platform="SageMaker",
260             target_platform="Kubeflow"
261         )
262
263         timeline = planner.generate_timeline()
264         print("\nMigration Timeline:")
265         print("="*60)
266
267         total_days = 0
268         for phase, info in timeline.items():
269             print(f"\n{phase.replace('_', ' ').title()}:")
270             print(f"    Tasks: {info['tasks']}")

```

```

271     print(f"    Estimated Days: {info['estimated_days']}")
272     total_days += info['estimated_days']
273
274     print(f"\nTotal Estimated Duration: {total_days} days (~{total_days/20:.1f} months
275         )")
276
277     critical_path = planner.identify_critical_path()
278     print(f"\nCritical Path Tasks: {len(critical_path)}")
279     for task in critical_path:
280         print(f"    - {task}")
281
282     planner.export_plan("migration_plan.json")

```

Listing 19.9: Platform Migration Plan Template

19.7.2 Code Portability Strategies

```

1  #!/usr/bin/env python3
2  """
3  Platform-agnostic ML pipeline abstraction
4  Enables easy migration between platforms
5  """
6
7  from abc import ABC, abstractmethod
8  from typing import Dict, Any, List
9  from enum import Enum
10
11  class PlatformBackend(Enum):
12      KUBEFLOW = "kubeflow"
13      SAGEMAKER = "sagemaker"
14      VERTEX_AI = "vertex_ai"
15      AZURE_ML = "azure_ml"
16
17  class MLPipelineStep(ABC):
18      """Abstract base class for pipeline steps"""
19
20      def __init__(self, name: str, inputs: Dict[str, str], outputs: Dict[str, str]):
21          self.name = name
22          self.inputs = inputs
23          self.outputs = outputs
24
25      @abstractmethod
26      def execute(self, **kwargs) -> Dict[str, Any]:
27          """Execute the step"""
28          pass
29
30      @abstractmethod
31      def to_kubeflow(self) -> dict:
32          """Convert to Kubeflow component"""
33          pass
34
35      @abstractmethod
36      def to_sagemaker(self) -> dict:
37          """Convert to SageMaker step"""
38          pass
39
40      @abstractmethod

```

```

41     def to_vertex_ai(self) -> dict:
42         """Convert to Vertex AI component"""
43         pass
44
45     class DataPreprocessingStep(MLPipelineStep):
46         """Data preprocessing step with multi-platform support"""
47
48         def __init__(self, script_path: str):
49             super().__init__(
50                 name="data_preprocessing",
51                 inputs={"raw_data": "s3://bucket/raw"},
52                 outputs={"processed_data": "s3://bucket/processed"}
53             )
54             self.script_path = script_path
55
56         def execute(self, **kwargs) -> Dict[str, Any]:
57             """Platform-agnostic execution logic"""
58             # Actual preprocessing code
59             return {"status": "success", "output_path": self.outputs["processed_data"]}
60
61         def to_kubeflow(self) -> dict:
62             """Convert to KubeFlow component"""
63             return {
64                 "name": self.name,
65                 "container": {
66                     "image": "ml-preprocessing:latest",
67                     "command": ["python", self.script_path],
68                     "args": [
69                         "--input", self.inputs["raw_data"],
70                         "--output", self.outputs["processed_data"]
71                     ]
72                 }
73             }
74
75         def to_sagemaker(self) -> dict:
76             """Convert to SageMaker ProcessingStep"""
77             return {
78                 "processor": {
79                     "image_uri": "ml-preprocessing:latest",
80                     "instance_type": "ml.m5.xlarge",
81                     "instance_count": 1
82                 },
83                 "inputs": [
84                     {
85                         "source": self.inputs["raw_data"],
86                         "destination": "/opt/ml/processing/input"
87                     }
88                 ],
89                 "outputs": [
90                     {
91                         "source": "/opt/ml/processing/output",
92                         "destination": self.outputs["processed_data"]
93                     }
94                 ],
95                 "code": self.script_path
96             }
97
98     def to_vertex_ai(self) -> dict:

```



```

99     """Convert to Vertex AI component"""
100     return {
101         "name": self.name,
102         "base_image": "python:3.9",
103         "packages_to_install": ["pandas", "scikit-learn"],
104         "inputs": {"raw_data": self.inputs["raw_data"]},
105         "outputs": {"processed_data": self.outputs["processed_data"]}
106     }
107
108 class MLPipeline:
109     """Platform-agnostic ML pipeline"""
110
111     def __init__(self, name: str, steps: List[MLPipelineStep]):
112         self.name = name
113         self.steps = steps
114
115     def compile_for_platform(self, platform: PlatformBackend) -> dict:
116         """Compile pipeline for specific platform"""
117
118         if platform == PlatformBackend.KUBEFLOW:
119             return self._compile_kubeflow()
120         elif platform == PlatformBackend.SAGEMAKER:
121             return self._compile_sagemaker()
122         elif platform == PlatformBackend.VERTEX_AI:
123             return self._compile_vertex_ai()
124         else:
125             raise ValueError(f"Unsupported platform: {platform}")
126
127     def _compile_kubeflow(self) -> dict:
128         """Compile to Kubeflow pipeline"""
129         return {
130             "apiVersion": "argoproj.io/v1alpha1",
131             "kind": "Workflow",
132             "metadata": {"name": self.name},
133             "spec": {
134                 "entrypoint": "pipeline",
135                 "templates": [step.to_kubeflow() for step in self.steps]
136             }
137         }
138
139     def _compile_sagemaker(self) -> dict:
140         """Compile to SageMaker pipeline"""
141         return {
142             "PipelineName": self.name,
143             "Steps": [step.to_sagemaker() for step in self.steps]
144         }
145
146     def _compile_vertex_ai(self) -> dict:
147         """Compile to Vertex AI pipeline"""
148         return {
149             "name": self.name,
150             "components": [step.to_vertex_ai() for step in self.steps]
151         }
152
153 # Example usage
154 if __name__ == "__main__":
155     # Define pipeline once
156     preprocessing_step = DataPreprocessingStep(script_path="preprocess.py")

```



```

35         alternatives=["Prefect", "Dagster", "Argo Workflows"]
36     ),
37     PlatformComponent(
38         name="Experiment Tracking",
39         purpose="Log metrics, parameters, artifacts",
40         technology="MLflow Tracking",
41         deployment="Docker Compose + PostgreSQL",
42         alternatives=["Weights & Biases", "Neptune.ai", "Comet.ml"]
43     ),
44     PlatformComponent(
45         name="Model Registry",
46         purpose="Centralized model versioning",
47         technology="MLflow Model Registry",
48         deployment="Integrated with MLflow Tracking",
49         alternatives=["Custom S3 + metadata DB", "ModelDB"]
50     ),
51     PlatformComponent(
52         name="Feature Store",
53         purpose="Feature engineering and serving",
54         technology="Feast",
55         deployment="Kubernetes + Redis + PostgreSQL",
56         alternatives=["Tecton", "Hopworks", "Custom solution"]
57     ),
58     PlatformComponent(
59         name="Model Serving",
60         purpose="Real-time inference APIs",
61         technology="BentoML",
62         deployment="Kubernetes deployments",
63         alternatives=["TorchServe", "TFServing", "FastAPI custom"]
64     ),
65     PlatformComponent(
66         name="Monitoring",
67         purpose="Model and infrastructure metrics",
68         technology="Prometheus + Grafana",
69         deployment="Kubernetes operators",
70         alternatives=["Datadog", "New Relic", "CloudWatch"]
71     ),
72     PlatformComponent(
73         name="Data Versioning",
74         purpose="Track data lineage and versions",
75         technology="DVC (Data Version Control)",
76         deployment="CLI tool + remote storage",
77         alternatives=["Pachyderm", "lakeFS", "Delta Lake"]
78     ),
79     PlatformComponent(
80         name="Metadata Store",
81         purpose="ML metadata and lineage tracking",
82         technology="ML Metadata (MLMD)",
83         deployment="PostgreSQL backend",
84         alternatives=["Custom graph database", "DataHub"]
85     ),
86     PlatformComponent(
87         name="Compute Management",
88         purpose="Dynamic resource provisioning",
89         technology="Kubernetes",
90         deployment="Self-managed or managed (EKS/GKE/AKS)",
91         alternatives=["Nomad", "Docker Swarm", "Cloud-native"]
92     ),

```

```

93         PlatformComponent(
94             name="Secret Management",
95             purpose="Secure credential storage",
96             technology="HashiCorp Vault",
97             deployment="Kubernetes Helm chart",
98             alternatives=["AWS Secrets Manager", "Azure Key Vault", "GCP Secret
Manager"]
99         )
100     ]
101
102     def generate_deployment_manifest(self) -> dict:
103         """Generate Kubernetes deployment manifest"""
104
105         manifests = {}
106
107         for component in self.components:
108             # Example: MLflow deployment
109             if component.name == "Experiment Tracking":
110                 manifests["mlflow"] = {
111                     "apiVersion": "apps/v1",
112                     "kind": "Deployment",
113                     "metadata": {"name": "mlflow-server"},
114                     "spec": {
115                         "replicas": 2,
116                         "selector": {
117                             "matchLabels": {"app": "mlflow"}
118                         },
119                         "template": {
120                             "metadata": {
121                                 "labels": {"app": "mlflow"}
122                             },
123                             "spec": {
124                                 "containers": [{
125                                     "name": "mlflow",
126                                     "image": "ghcr.io/mlflow/mlflow:latest",
127                                     "ports": [{"containerPort": 5000}],
128                                     "env": [
129                                         {
130                                             "name": "BACKEND_STORE_URI",
131                                             "value": "postgresql://mlflow:
password@postgres:5432/mlflow"
132                                         },
133                                         {
134                                             "name": "DEFAULT_ARTIFACT_ROOT",
135                                             "value": "s3://mlflow-artifacts"
136                                         }
137                                     ]
138                                 }
139                             }
140                         }
141                     }
142                 }
143
144         return manifests
145
146     def export_architecture(self, filename: str):
147         """Export platform architecture to YAML"""
148

```

```

149     architecture = {
150         "platform": "Custom MLOps Platform",
151         "version": "1.0",
152         "components": [
153             {
154                 "name": c.name,
155                 "purpose": c.purpose,
156                 "technology": c.technology,
157                 "deployment": c.deployment,
158                 "alternatives": c.alternatives
159             }
160             for c in self.components
161         ]
162     }
163
164     with open(filename, 'w') as f:
165         yaml.dump(architecture, f, default_flow_style=False)
166
167     print(f"Platform architecture exported to {filename}")
168
169 # Example usage
170 if __name__ == "__main__":
171     platform = CustomMLOpsPlatform()
172
173     print("Custom MLOps Platform Components:")
174     print("="*70)
175
176     for component in platform.components:
177         print(f"\n{component.name}:")
178         print(f"    Purpose: {component.purpose}")
179         print(f"    Technology: {component.technology}")
180         print(f"    Deployment: {component.deployment}")
181         print(f"    Alternatives: {'', ' '.join(component.alternatives)}")
182
183     platform.export_architecture("mlops_platform.yaml")

```

Listing 19.11: Minimal Custom MLOps Platform Design

Custom Platform Benefits:

- **No Vendor Lock-in:** Use open-source tools, migrate as needed
- **Cost Control:** Pay only for infrastructure, no platform fees
- **Customization:** Tailor every component to specific needs
- **Multi-Cloud:** Run consistently across clouds and on-premises
- **Skill Development:** Team gains deep platform expertise

Custom Platform Challenges:

- **Development Effort:** Significant time to build and integrate
- **Operational Burden:** Platform team needed for maintenance
- **Security:** Responsibility for patching and compliance
- **Feature Gap:** May lack advanced features of managed platforms
- **Opportunity Cost:** Resources diverted from ML development

19.9 Platform Governance and Best Practices

MLOps platform governance ensures consistent usage, cost control, and security across the organization.

19.9.1 Multi-Tenancy and Access Control

```

1  #!/usr/bin/env python3
2  """
3  MLOps Platform Governance Framework
4  Implements multi-tenancy, resource quotas, and access controls
5  """
6
7  from dataclasses import dataclass
8  from typing import List, Dict, Optional
9  from enum import Enum
10 import yaml
11
12 class UserRole(Enum):
13     """User roles in MLOps platform"""
14     DATA_SCIENTIST = "data_scientist"
15     ML_ENGINEER = "ml_engineer"
16     PLATFORM_ADMIN = "platform_admin"
17     VIEWER = "viewer"
18
19 @dataclass
20 class ResourceQuota:
21     """Resource quotas per team/project"""
22     cpu_cores: int
23     memory_gb: int
24     gpu_count: int
25     storage_gb: int
26     max_pipelines: int
27     max_endpoints: int
28
29 @dataclass
30 class Team:
31     """Team configuration"""
32     name: str
33     namespace: str
34     members: List[str]
35     quotas: ResourceQuota
36     cost_center: str
37     tags: Dict[str, str]
38
39 class PlatformGovernance:
40     """Platform governance manager"""
41
42     def __init__(self):
43         self.teams: List[Team] = []
44         self.access_policies: Dict[str, List[str]] = {}
45
46     def create_team_namespace(self, team: Team) -> dict:
47         """Create isolated namespace for team"""
48
49         # Kubernetes namespace manifest
50         namespace_manifest = {

```

```

51     "apiVersion": "v1",
52     "kind": "Namespace",
53     "metadata": {
54         "name": team.namespace,
55         "labels": {
56             "team": team.name,
57             "cost-center": team.cost_center,
58             **team.tags
59         }
60     }
61 }
62
63 # Resource quota manifest
64 quota_manifest = {
65     "apiVersion": "v1",
66     "kind": "ResourceQuota",
67     "metadata": {
68         "name": f"{team.namespace}-quota",
69         "namespace": team.namespace
70     },
71     "spec": {
72         "hard": {
73             "requests.cpu": f"{team.quotas.cpu_cores}",
74             "requests.memory": f"{team.quotas.memory_gb}Gi",
75             "nvidia.com/gpu": f"{team.quotas.gpu_count}",
76             "persistentvolumeclaims": "10",
77             "requests.storage": f"{team.quotas.storage_gb}Gi"
78         }
79     }
80 }
81
82 # Network policy (isolation)
83 network_policy = {
84     "apiVersion": "networking.k8s.io/v1",
85     "kind": "NetworkPolicy",
86     "metadata": {
87         "name": f"{team.namespace}-isolation",
88         "namespace": team.namespace
89     },
90     "spec": {
91         "podSelector": {},
92         "policyTypes": ["Ingress", "Egress"],
93         "ingress": [
94             {
95                 "from": [
96                     {"namespaceSelector": {
97                         "matchLabels": {"team": team.name}
98                     }}
99                 ]
100             }
101         ],
102         "egress": [
103             {"to": [{"namespaceSelector": {}}]},
104             {"to": [{"podSelector": {}]}]
105         ]
106     }
107 }
108

```



```

167         "verbs": ["*"]
168     }
169 ]
170 }
171
172 return {
173     "data_scientist_role": data_scientist_role,
174     "ml_engineer_role": ml_engineer_role
175 }
176
177 def generate_cost_allocation_tags(self, team: Team) -> Dict[str, str]:
178     """Generate cost allocation tags"""
179
180     return {
181         "Team": team.name,
182         "CostCenter": team.cost_center,
183         "ManagedBy": "mlops-platform",
184         "Environment": team.tags.get("environment", "production"),
185         "Project": team.tags.get("project", "default")
186     }
187
188 def export_governance_config(self, filename: str):
189     """Export governance configuration"""
190
191     config = {
192         "teams": [
193             {
194                 "name": t.name,
195                 "namespace": t.namespace,
196                 "members": t.members,
197                 "quotas": {
198                     "cpu_cores": t.quotas.cpu_cores,
199                     "memory_gb": t.quotas.memory_gb,
200                     "gpu_count": t.quotas.gpu_count,
201                     "storage_gb": t.quotas.storage_gb
202                 },
203                 "cost_center": t.cost_center
204             }
205             for t in self.teams
206         ]
207     }
208
209     with open(filename, 'w') as f:
210         yaml.dump(config, f, default_flow_style=False)
211
212     print(f"Governance config exported to {filename}")
213
214 # Example usage
215 if __name__ == "__main__":
216     # Define teams
217     cv_team = Team(
218         name="computer-vision",
219         namespace="ml-cv",
220         members=["alice@company.com", "bob@company.com"],
221         quotas=ResourceQuota(
222             cpu_cores=64,
223             memory_gb=256,
224             gpu_count=8,

```

```

225         storage_gb=2000,
226         max_pipelines=20,
227         max_endpoints=10
228     ),
229     cost_center="R&D-CV",
230     tags={"environment": "production", "project": "image-recognition"}
231 )
232
233 nlp_team = Team(
234     name="natural-language",
235     namespace="ml-nlp",
236     members=["charlie@company.com", "diana@company.com"],
237     quotas=ResourceQuota(
238         cpu_cores=32,
239         memory_gb=128,
240         gpu_count=4,
241         storage_gb=1000,
242         max_pipelines=15,
243         max_endpoints=5
244     ),
245     cost_center="R&D-NLP",
246     tags={"environment": "production", "project": "chatbot"}
247 )
248
249 # Create governance framework
250 governance = PlatformGovernance()
251 governance.teams = [cv_team, nlp_team]
252
253 # Generate configurations
254 print("Computer Vision Team Setup:")
255 print("="*60)
256 cv_config = governance.create_team_namespace(cv_team)
257 cv_rbac = governance.define_rbac_policies(cv_team)
258 cv_tags = governance.generate_cost_allocation_tags(cv_team)
259
260 print(f"Namespace: {cv_config['namespace']['metadata']['name']}")
261 print(f"CPU Quota: {cv_team.quotas.cpu_cores} cores")
262 print(f"GPU Quota: {cv_team.quotas.gpu_count} GPUs")
263 print(f"Cost Center: {cv_team.cost_center}")
264
265 governance.export_governance_config("platform_governance.yaml")

```

Listing 19.12: Platform Governance Framework

19.9.2 Platform Standardization

Pipeline Templates:

- **Standard Templates:** Pre-built pipeline templates for common use cases
- **Best Practices:** Embedded logging, monitoring, error handling
- **Compliance:** Built-in data governance and audit trails
- **Versioning:** Template versioning with migration guides

Container Image Standards:

- **Base Images:** Organization-approved base images with security patches
- **Vulnerability Scanning:** Automated scanning in CI/CD pipeline
- **Image Registry:** Private registry with access controls
- **Versioning Strategy:** Semantic versioning for reproducibility

Code Quality Standards:

- **Linting:** Automated code style enforcement (black, flake8)
- **Type Checking:** Static type checking with mypy
- **Testing Requirements:** Minimum code coverage thresholds
- **Documentation:** Mandatory docstrings and README files

19.9.3 Cost Management and Optimization

```

1  #!/usr/bin/env python3
2  """
3  MLOps Platform Cost Management
4  Track, analyze, and optimize platform costs
5  """
6
7  import boto3
8  from datetime import datetime, timedelta
9  from typing import Dict, List
10 import pandas as pd
11
12 class PlatformCostManager:
13     """Manage and optimize platform costs"""
14
15     def __init__(self, platform_type: str = "sagemaker"):
16         self.platform = platform_type
17         self.ce_client = boto3.client('ce')  # Cost Explorer
18
19     def get_cost_by_service(self, days: int = 30) -> pd.DataFrame:
20         """Get cost breakdown by service"""
21
22         end_date = datetime.now()
23         start_date = end_date - timedelta(days=days)
24
25         response = self.ce_client.get_cost_and_usage(
26             TimePeriod={
27                 'Start': start_date.strftime('%Y-%m-%d'),
28                 'End': end_date.strftime('%Y-%m-%d')
29             },
30             Granularity='DAILY',
31             Metrics=['UnblendedCost'],
32             GroupBy=[
33                 {'Type': 'SERVICE', 'Key': 'SERVICE'},
34                 {'Type': 'TAG', 'Key': 'Team'}
35             ],
36             Filter={
37                 'Tags': {

```

```

38         'Key': 'ManagedBy',
39         'Values': ['mlops-platform']
40     }
41 }
42 )
43
44 # Process results
45 cost_data = []
46 for result in response['ResultsByTime']:
47     date = result['TimePeriod']['Start']
48     for group in result['Groups']:
49         service = group['Keys'][0]
50         team = group['Keys'][1] if len(group['Keys']) > 1 else 'untagged'
51         cost = float(group['Metrics']['UnblendedCost']['Amount'])
52
53         cost_data.append({
54             'date': date,
55             'service': service,
56             'team': team,
57             'cost': cost
58         })
59
60 return pd.DataFrame(cost_data)
61
62 def identify_cost_anomalies(self, threshold: float = 1.5) -> List[Dict]:
63     """Identify unusual cost spikes"""
64
65     df = self.get_cost_by_service(days=30)
66
67     # Calculate daily cost by team
68     daily_costs = df.groupby(['date', 'team'])['cost'].sum().reset_index()
69
70     anomalies = []
71     for team in daily_costs['team'].unique():
72         team_costs = daily_costs[daily_costs['team'] == team]['cost']
73         mean_cost = team_costs.mean()
74         std_cost = team_costs.std()
75
76         for idx, row in daily_costs[daily_costs['team'] == team].iterrows():
77             if row['cost'] > mean_cost + (threshold * std_cost):
78                 anomalies.append({
79                     'date': row['date'],
80                     'team': team,
81                     'cost': row['cost'],
82                     'expected_cost': mean_cost,
83                     'deviation': (row['cost'] - mean_cost) / mean_cost * 100
84                 })
85
86     return anomalies
87
88 def recommend_optimizations(self) -> List[Dict]:
89     """Generate cost optimization recommendations"""
90
91     recommendations = []
92
93     # Check for idle training instances
94     sagemaker = boto3.client('sagemaker')
95

```

```

96     # Get training jobs
97     training_jobs = sagemaker.list_training_jobs(
98         StatusEquals='InProgress',
99         MaxResults=100
100    )
101
102    for job in training_jobs['TrainingJobSummaries']:
103        job_details = sagemaker.describe_training_job(
104            TrainingJobName=job['TrainingJobName']
105        )
106
107        # Check if using on-demand instances
108        instance_type = job_details['ResourceConfig']['InstanceType']
109        if not instance_type.startswith('ml.'):
110            continue
111
112        # Recommend spot instances for long-running jobs
113        duration = datetime.now() - job['CreationTime'].replace(tzinfo=None)
114        if duration.total_seconds() > 3600: # > 1 hour
115            recommendations.append({
116                'type': 'spot_instances',
117                'resource': job['TrainingJobName'],
118                'current_instance': instance_type,
119                'estimated_savings': '70%',
120                'description': 'Switch to spot instances for long training jobs'
121            })
122
123        # Check for over-provisioned endpoints
124        endpoints = sagemaker.list_endpoints(MaxResults=100)
125
126        for endpoint in endpoints['Endpoints']:
127            endpoint_details = sagemaker.describe_endpoint(
128                EndpointName=endpoint['EndpointName']
129            )
130
131            # Get metrics
132            cloudwatch = boto3.client('cloudwatch')
133            metrics = cloudwatch.get_metric_statistics(
134                Namespace='AWS/SageMaker',
135                MetricName='ModelLatency',
136                Dimensions=[
137                    {'Name': 'EndpointName', 'Value': endpoint['EndpointName']},
138                    {'Name': 'VariantName', 'Value': 'AllTraffic'}
139                ],
140                StartTime=datetime.now() - timedelta(days=7),
141                EndTime=datetime.now(),
142                Period=86400,
143                Statistics=['Average', 'Maximum']
144            )
145
146            # Check if consistently low latency (potential right-sizing opportunity)
147            if metrics['Datapoints']:
148                avg_latency = sum(d['Average'] for d in metrics['Datapoints']) / len(
149                    metrics['Datapoints'])
150                if avg_latency < 50: # < 50ms
151                    recommendations.append({
152                        'type': 'right_sizing',
153                        'resource': endpoint['EndpointName'],

```

```

153         'current_config': endpoint_details['ProductionVariants'][0]['
InstanceType'],
154         'estimated_savings': '30%',
155         'description': 'Consider smaller instance type for low-latency
workload'
156     })
157
158     return recommendations
159
160 def generate_cost_report(self, output_file: str):
161     """Generate comprehensive cost report"""
162
163     df = self.get_cost_by_service(days=30)
164     anomalies = self.identify_cost_anomalies()
165     optimizations = self.recommend_optimizations()
166
167     # Summary statistics
168     total_cost = df['cost'].sum()
169     daily_avg = df.groupby('date')['cost'].sum().mean()
170
171     report = f"""
172 MLOps Platform Cost Report
173 =====
174
175 Period: Last 30 days
176 Total Cost: ${total_cost:.2f}
177 Daily Average: ${daily_avg:.2f}
178
179 Cost by Team:
180 {df.groupby('team')['cost'].sum().to_string()}
181
182 Cost by Service:
183 {df.groupby('service')['cost'].sum().sort_values(ascending=False).head(10).to_string()
}
184
185 Anomalies Detected: {len(anomalies)}
186 {chr(10).join([f"- {a['team']} on {a['date']}: ${a['cost']:.2f} ({a['deviation']:.1f}%
above normal)" for a in anomalies[:5]])}
187
188 Optimization Opportunities: {len(optimizations)}
189 {chr(10).join([f"- {o['type']}: {o['description']} (Est. savings: {o['
estimated_savings']})" for o in optimizations[:5]])}
190
191 Total Potential Savings: ${sum([total_cost * 0.3 for _ in optimizations]):.2f}
192 """
193
194     with open(output_file, 'w') as f:
195         f.write(report)
196
197     print(f"Cost report generated: {output_file}")
198     return report
199
200 # Example usage
201 if __name__ == "__main__":
202     cost_manager = PlatformCostManager(platform_type="sagemaker")
203
204     # Get cost breakdown
205     costs = cost_manager.get_cost_by_service(days=30)

```

```
206 print(f"Total cost last 30 days: ${costs['cost'].sum():.2f}")
207
208 # Find anomalies
209 anomalies = cost_manager.identify_cost_anomalies()
210 print(f"\nCost anomalies found: {len(anomalies)}")
211
212 # Get recommendations
213 recommendations = cost_manager.recommend_optimizations()
214 print(f"Optimization opportunities: {len(recommendations)}")
215
216 # Generate report
217 cost_manager.generate_cost_report("platform_cost_report.txt")
```

Listing 19.13: Platform Cost Monitoring and Optimization

19.9.4 Security and Compliance

Data Security:

- **Encryption at Rest:** All data encrypted using KMS/Cloud KMS
- **Encryption in Transit:** TLS 1.3 for all network communication
- **Access Logging:** Comprehensive audit logs for data access
- **Data Classification:** Automatic tagging of sensitive data

Model Security:

- **Model Signing:** Cryptographic signatures for model artifacts
- **Vulnerability Scanning:** Dependencies scanned for known CVEs
- **Adversarial Testing:** Regular testing for model robustness
- **Model Watermarking:** Identify model theft or unauthorized use

Compliance Requirements:

- **GDPR:** Right to deletion, data minimization, consent management
- **HIPAA:** PHI handling, BAA agreements, access controls
- **SOC 2:** Security controls, incident response, vendor management
- **Audit Trails:** Immutable logs for compliance verification

19.9.5 Platform Monitoring and Observability

Platform Metrics:

- **Pipeline Success Rate:** Percentage of successful pipeline runs
- **Average Pipeline Duration:** Time from start to completion
- **Resource Utilization:** CPU, GPU, memory usage across platform

- **Queue Depth:** Number of pending jobs/pipelines
- **Error Rates:** Errors by type and component

User Metrics:

- **Active Users:** Daily/weekly active users by role
- **Feature Adoption:** Usage of different platform capabilities
- **User Satisfaction:** NPS scores, support tickets
- **Time to First Model:** Onboarding effectiveness metric

Cost Metrics:

- **Cost per Model:** Training and deployment costs per model
- **Cost per Team:** Chargeback for multi-tenant environments
- **Waste Metrics:** Idle resources, unused provisioned capacity
- **Cost Trends:** Month-over-month cost growth

19.10 Platform Evaluation Exercises

19.10.1 Exercise 1: Platform Selection for Startup

Scenario: Early-stage startup with 5-person ML team, limited budget, AWS-based infrastructure, building computer vision models for edge devices.

Requirements:

- Evaluate Kubeflow, SageMaker, and custom solution
- Consider 12-month total cost of ownership
- Factor in team's limited Kubernetes experience
- Need for edge deployment capabilities
- Rapid iteration and experimentation priority

Tasks:

1. Score each platform on 9 evaluation criteria (use framework from section 19.5.1)
2. Calculate weighted scores and recommend platform
3. Estimate TCO including infrastructure, licensing, and team time
4. Identify top 3 risks for each option
5. Create 6-month rollout plan for selected platform

Deliverables:

- Completed evaluation matrix with justifications
- TCO spreadsheet with assumptions documented
- Risk mitigation strategy document
- Platform rollout timeline with milestones

19.10.2 Exercise 2: Multi-Cloud Platform Migration

Scenario: Enterprise with 50 models in production on SageMaker, expanding to GCP for new product line, considering multi-cloud MLOps strategy.

Current State:

- 50 SageMaker pipelines (various complexities)
- 20 real-time inference endpoints
- Custom feature store on S3 + DynamoDB
- MLflow for experiment tracking
- 15-person ML engineering team

Tasks:

1. Design platform-agnostic abstraction layer
2. Create migration plan using framework from section 19.6
3. Implement pipeline portability for one sample workflow
4. Estimate migration effort and timeline
5. Identify tooling gaps and integration points

Deliverables:

- Architecture diagram for multi-cloud platform
- Complete migration plan JSON (all phases)
- Working code for portable pipeline (Python)
- Risk assessment with mitigation strategies
- Team training plan for new skills needed

19.10.3 Exercise 3: Build vs Buy Decision

Scenario: Financial services company evaluating whether to build custom MLOps platform or adopt managed service. Strict compliance requirements, 100+ models planned, 30-person ML team.

Decision Factors:

- Data residency requirements (cannot leave specific regions)
- Need for audit trails and model explainability
- Integration with existing on-premises systems
- Budget: \$500K annually for platform
- 5-year time horizon

Tasks:

1. Compare build vs buy across 10 dimensions
2. Calculate 5-year TCO for each option
3. Assess compliance alignment (GDPR, SOC 2, PCI DSS)
4. Evaluate team capability and hiring needs
5. Recommend approach with detailed justification

Deliverables:

- Build vs buy comparison matrix
- 5-year financial model (capex vs opex)
- Compliance gap analysis for each option
- Org chart and headcount plan
- Executive summary with recommendation

19.10.4 Exercise 4: Kubeflow Pipeline Implementation

Scenario: Implement end-to-end ML pipeline on Kubeflow for recommendation system.

Requirements:

- Data ingestion from multiple sources (S3, PostgreSQL)
- Feature engineering with Feast feature store
- Hyperparameter tuning with Katib
- Model evaluation with custom metrics
- Conditional deployment based on performance

- Integration with MLflow model registry

Tasks:

1. Set up local KubeFlow environment (Minikube or Kind)
2. Implement 8+ pipeline components as Python functions
3. Configure Katib experiment for HPO
4. Deploy model with KServe
5. Add monitoring with Prometheus metrics

Deliverables:

- Complete pipeline code (Python with KFP SDK)
- Katib experiment YAML configuration
- KServe InferenceService manifest
- Documentation for running pipeline end-to-end
- Prometheus dashboard for model metrics

19.10.5 Exercise 5: Platform Cost Optimization

Scenario: SageMaker monthly costs exceeding budget by 40%. Optimize without sacrificing capabilities.

Current Spending:

- Training: \$15K/month (10 jobs/day, ml.p3.8xlarge)
- Inference: \$25K/month (20 endpoints, ml.m5.xlarge)
- Feature Store: \$5K/month (online + offline)
- Data Processing: \$8K/month (SageMaker Processing)
- Model Monitor: \$2K/month

Tasks:

1. Analyze current resource utilization
2. Identify optimization opportunities (spot instances, right-sizing, etc.)
3. Evaluate alternative platforms for specific workloads
4. Implement cost allocation tags
5. Set up budget alerts and automation

Deliverables:

- Cost analysis report with breakdown

- Optimization recommendations (prioritized by impact)
- Implementation plan for top 5 optimizations
- Projected savings calculation
- Cost monitoring dashboard design

19.10.6 Exercise 6: Platform Comparison Benchmark

Scenario: Benchmark Kubeflow, SageMaker, and Vertex AI for your organization's specific workload.

Benchmark Criteria:

- Pipeline execution time (same workflow)
- Cost per training run
- Ease of deployment (time to first model)
- Monitoring and observability capabilities
- Developer experience and productivity

Tasks:

1. Implement identical pipeline on all three platforms
2. Measure execution time and resource usage
3. Calculate actual costs (including hidden costs)
4. Survey team on developer experience (1-10 scale)
5. Document integration complexity

Deliverables:

- Benchmark results table (quantitative metrics)
- Developer experience survey results
- Cost comparison spreadsheet
- Platform recommendation with trade-offs
- Lessons learned document

19.11 Summary

MLOps platforms provide the foundation for production ML systems. This chapter covered:

1. Kubeflow Pipelines:

- Complete end-to-end pipeline with 8 components
- Hyperparameter tuning with Katib
- Model serving with KServe
- Multi-framework support (PyTorch, TensorFlow, scikit-learn)
- Cloud-agnostic deployment on Kubernetes

2. Cloud ML Platforms:

- AWS SageMaker: Native AWS integration, comprehensive features
- Google Vertex AI: KFP-compatible, strong AutoML capabilities
- Azure ML: MLflow integration, enterprise focus
- Platform comparison across 13 dimensions

3. Platform Selection:

- Decision framework with 9 weighted criteria
- Cloud strategy alignment (single vs multi-cloud)
- Team capability and operational overhead assessment
- Compliance and security considerations
- Total cost of ownership analysis

4. Migration Strategies:

- 6-phase migration framework (assessment to decommission)
- 16 detailed migration tasks with dependencies
- Platform-agnostic pipeline abstractions
- Risk mitigation and rollback planning
- Parallel run validation approach

5. Custom Platform Development:

- 10-component architecture (Airflow, MLflow, Feast, etc.)
- Benefits: flexibility, no lock-in, cost control
- Challenges: development effort, operational burden

- Best-of-breed tool integration
- Deployment on Kubernetes

6. Evaluation Exercises:

- Startup platform selection
- Multi-cloud migration planning
- Build vs buy decision framework
- Kubeflow implementation
- Cost optimization
- Platform benchmarking

Key Takeaways:

1. **No Universal Best:** Platform choice depends on organizational context
2. **Cloud Strategy Matters:** Single-cloud enables native platforms, multi-cloud requires abstraction
3. **Team Capability:** Managed platforms reduce operational burden for small teams
4. **Start Simple:** Begin with managed services, add complexity as needed
5. **Plan for Portability:** Abstract platform-specific APIs where possible
6. **Monitor Costs:** Platform costs can exceed compute costs at scale
7. **Invest in Migration:** Platform migration is complex, plan for 3-6 months

The right MLOps platform accelerates ML development, reduces operational burden, and enables production deployment at scale. Whether choosing managed services or building custom solutions, the key is aligning platform capabilities with organizational needs, team skills, and strategic objectives.

Part VII

Advanced Topics and Best Practices

Chapter 20

Model Governance and Compliance

20.1 Learning Objectives

After completing this chapter, readers will be able to:

- Implement comprehensive model governance frameworks
- Ensure regulatory compliance (GDPR, CCPA, AI Act)
- Design model documentation and audit trails
- Implement fairness and bias testing
- Conduct model risk assessments
- Build approval workflows and controls
- Create governance policies for ML systems
- Audit model behavior and data usage

20.2 Introduction

Model governance provides the policies, processes, and controls needed to ensure ML systems are developed and deployed responsibly. As ML systems increasingly impact high-stakes decisions in healthcare, finance, hiring, and criminal justice, governance frameworks ensure these systems are fair, transparent, and compliant with regulations.

Why Model Governance Matters:

- **Regulatory Compliance:** GDPR, CCPA, EU AI Act require transparency and accountability
- **Risk Management:** High-stakes decisions need oversight and controls
- **Fairness and Bias:** Prevent discriminatory outcomes across protected groups
- **Trust and Transparency:** Stakeholders need to understand model decisions
- **Audit and Accountability:** Trace decisions back to data, code, and approvals

- **Legal Protection:** Documentation protects organizations from liability

Governance Scope:

1. **Model Development:** Code reviews, data quality checks, bias testing
2. **Model Approval:** Risk assessment, stakeholder sign-off, compliance verification
3. **Model Deployment:** Controlled rollouts, monitoring, audit trails
4. **Model Monitoring:** Performance tracking, fairness metrics, drift detection
5. **Model Retirement:** Decommissioning process, data retention, archival

20.3 Model Governance Framework Implementation

A comprehensive governance framework defines roles, processes, and systems for managing ML models throughout their lifecycle.

20.3.1 Governance Structure and Roles

```

1  #!/usr/bin/env python3
2  """
3  Model Governance Framework
4  Defines roles, responsibilities, and governance processes
5  """
6
7  from dataclasses import dataclass, field
8  from typing import List, Dict, Optional
9  from enum import Enum
10 from datetime import datetime
11 import json
12
13 class GovernanceRole(Enum):
14     """Roles in model governance"""
15     MODEL_DEVELOPER = "model_developer"
16     DATA_SCIENTIST = "data_scientist"
17     ML_ENGINEER = "ml_engineer"
18     MODEL_VALIDATOR = "model_validator"
19     COMPLIANCE_OFFICER = "compliance_officer"
20     BUSINESS_OWNER = "business_owner"
21     LEGAL_COUNSEL = "legal_counsel"
22     RISK_MANAGER = "risk_manager"
23
24 class RiskTier(Enum):
25     """Model risk categorization"""
26     CRITICAL = "critical" # Healthcare, finance, legal decisions
27     HIGH = "high" # Customer-facing, revenue impact
28     MEDIUM = "medium" # Internal tools, low impact
29     LOW = "low" # Experimental, no production use
30
31 @dataclass
32 class GovernancePolicy:
33     """Governance policy for a risk tier"""
34     risk_tier: RiskTier

```

```

35     required_approvers: List[GovernanceRole]
36     required_documentation: List[str]
37     testing_requirements: List[str]
38     monitoring_frequency: str
39     review_period_days: int
40     audit_retention_years: int
41
42 @dataclass
43 class ModelGovernanceRecord:
44     """Complete governance record for a model"""
45     model_id: str
46     model_name: str
47     model_version: str
48     risk_tier: RiskTier
49     business_owner: str
50     development_team: List[str]
51
52     # Documentation
53     model_card_url: str
54     technical_documentation_url: str
55     risk_assessment_url: str
56
57     # Approvals
58     approvals: List[Dict[str, str]] = field(default_factory=list)
59
60     # Compliance
61     compliance_checks: Dict[str, bool] = field(default_factory=dict)
62
63     # Timestamps
64     created_at: datetime = field(default_factory=datetime.now)
65     approved_at: Optional[datetime] = None
66     deployed_at: Optional[datetime] = None
67     next_review_date: Optional[datetime] = None
68
69     # Audit trail
70     audit_log: List[Dict] = field(default_factory=list)
71
72 class ModelGovernanceFramework:
73     """Model governance framework implementation"""
74
75     def __init__(self):
76         self.policies = self._define_policies()
77         self.models: Dict[str, ModelGovernanceRecord] = {}
78
79     def _define_policies(self) -> Dict[RiskTier, GovernancePolicy]:
80         """Define governance policies by risk tier"""
81
82         return {
83             RiskTier.CRITICAL: GovernancePolicy(
84                 risk_tier=RiskTier.CRITICAL,
85                 required_approvers=[
86                     GovernanceRole.MODEL_VALIDATOR,
87                     GovernanceRole.COMPLIANCE_OFFICER,
88                     GovernanceRole.LEGAL_COUNSEL,
89                     GovernanceRole.RISK_MANAGER,
90                     GovernanceRole.BUSINESS_OWNER
91                 ],
92                 required_documentation=[

```

```

    "model_card",
    "technical_documentation",
    "risk_assessment",
    "fairness_report",
    "privacy_impact_assessment",
    "explainability_report",
    "monitoring_plan"
],
testing_requirements=[
    "unit_tests",
    "integration_tests",
    "fairness_tests",
    "adversarial_tests",
    "performance_tests",
    "stress_tests"
],
monitoring_frequency="daily",
review_period_days=90,
audit_retention_years=7
),

RiskTier.HIGH: GovernancePolicy(
    risk_tier=RiskTier.HIGH,
    required_approvers=[
        GovernanceRole.MODEL_VALIDATOR,
        GovernanceRole.COMPLIANCE_OFFICER,
        GovernanceRole.BUSINESS_OWNER
    ],
    required_documentation=[
        "model_card",
        "technical_documentation",
        "risk_assessment",
        "fairness_report"
    ],
    testing_requirements=[
        "unit_tests",
        "integration_tests",
        "fairness_tests",
        "performance_tests"
    ],
    monitoring_frequency="weekly",
    review_period_days=180,
    audit_retention_years=5
),

RiskTier.MEDIUM: GovernancePolicy(
    risk_tier=RiskTier.MEDIUM,
    required_approvers=[
        GovernanceRole.MODEL_VALIDATOR,
        GovernanceRole.BUSINESS_OWNER
    ],
    required_documentation=[
        "model_card",
        "technical_documentation"
    ],
    testing_requirements=[
        "unit_tests",
        "integration_tests"

```

```

151         ],
152         monitoring_frequency="monthly",
153         review_period_days=365,
154         audit_retention_years=3
155     ),
156
157     RiskTier.LOW: GovernancePolicy(
158         risk_tier=RiskTier.LOW,
159         required_approvers=[
160             GovernanceRole.BUSINESS_OWNER
161         ],
162         required_documentation=[
163             "model_card"
164         ],
165         testing_requirements=[
166             "unit_tests"
167         ],
168         monitoring_frequency="quarterly",
169         review_period_days=730,
170         audit_retention_years=1
171     )
172 }
173
174 def register_model(self, record: ModelGovernanceRecord) -> str:
175     """Register a new model for governance"""
176
177     # Validate required documentation
178     policy = self.policies[record.risk_tier]
179
180     record.audit_log.append({
181         "timestamp": datetime.now().isoformat(),
182         "action": "model_registered",
183         "user": "system",
184         "details": f"Model {record.model_name} registered with risk tier {record.
risk_tier.value}"
185     })
186
187     self.models[record.model_id] = record
188     return record.model_id
189
190 def request_approval(
191     self,
192     model_id: str,
193     approver_role: GovernanceRole,
194     approver_name: str
195 ) -> bool:
196     """Request approval from a governance role"""
197
198     record = self.models.get(model_id)
199     if not record:
200         raise ValueError(f"Model {model_id} not found")
201
202     policy = self.policies[record.risk_tier]
203
204     # Check if this approver is required
205     if approver_role not in policy.required_approvers:
206         return False
207

```

```

208     # Add approval
209     record.approvals.append({
210         "role": approver_role.value,
211         "approver": approver_name,
212         "timestamp": datetime.now().isoformat(),
213         "status": "pending"
214     })
215
216     record.audit_log.append({
217         "timestamp": datetime.now().isoformat(),
218         "action": "approval_requested",
219         "user": "system",
220         "details": f"Approval requested from {approver_role.value}"
221     })
222
223     return True
224
225 def grant_approval(
226     self,
227     model_id: str,
228     approver_role: GovernanceRole,
229     approver_name: str,
230     comments: str = ""
231 ) -> bool:
232     """Grant approval for a model"""
233
234     record = self.models.get(model_id)
235     if not record:
236         raise ValueError(f"Model {model_id} not found")
237
238     # Find pending approval
239     for approval in record.approvals:
240         if (approval["role"] == approver_role.value and
241             approval["approver"] == approver_name and
242             approval["status"] == "pending"):
243
244             approval["status"] = "approved"
245             approval["approved_at"] = datetime.now().isoformat()
246             approval["comments"] = comments
247
248             record.audit_log.append({
249                 "timestamp": datetime.now().isoformat(),
250                 "action": "approval_granted",
251                 "user": approver_name,
252                 "role": approver_role.value,
253                 "details": comments
254             })
255
256     # Check if all required approvals obtained
257     if self.check_approval_status(model_id):
258         record.approved_at = datetime.now()
259         record.audit_log.append({
260             "timestamp": datetime.now().isoformat(),
261             "action": "fully_approved",
262             "user": "system",
263             "details": "All required approvals obtained"
264         })
265

```

```

266         return True
267
268     return False
269
270     def check_approval_status(self, model_id: str) -> bool:
271         """Check if model has all required approvals"""
272
273         record = self.models.get(model_id)
274         if not record:
275             return False
276
277         policy = self.policies[record.risk_tier]
278
279         # Get approved roles
280         approved_roles = set()
281         for approval in record.approvals:
282             if approval["status"] == "approved":
283                 approved_roles.add(GovernanceRole(approval["role"]))
284
285         # Check if all required approvers have approved
286         required_roles = set(policy.required_approvers)
287         return required_roles.issubset(approved_roles)
288
289     def record_deployment(self, model_id: str, deployment_details: Dict):
290         """Record model deployment"""
291
292         record = self.models.get(model_id)
293         if not record:
294             raise ValueError(f"Model {model_id} not found")
295
296         if not self.check_approval_status(model_id):
297             raise ValueError(f"Model {model_id} not fully approved for deployment")
298
299         record.deployed_at = datetime.now()
300
301         record.audit_log.append({
302             "timestamp": datetime.now().isoformat(),
303             "action": "model_deployed",
304             "user": deployment_details.get("deployed_by", "system"),
305             "details": json.dumps(deployment_details)
306         })
307
308     def export_governance_report(self, model_id: str, output_file: str):
309         """Export governance report for a model"""
310
311         record = self.models.get(model_id)
312         if not record:
313             raise ValueError(f"Model {model_id} not found")
314
315         policy = self.policies[record.risk_tier]
316
317         report = {
318             "model_id": record.model_id,
319             "model_name": record.model_name,
320             "model_version": record.model_version,
321             "risk_tier": record.risk_tier.value,
322             "business_owner": record.business_owner,
323             "development_team": record.development_team,

```

```

324         "policy": {
325             "required_approvers": [role.value for role in policy.
required_approvers],
326             "required_documentation": policy.required_documentation,
327             "testing_requirements": policy.testing_requirements,
328             "monitoring_frequency": policy.monitoring_frequency
329         },
330         "approvals": record.approvals,
331         "compliance_checks": record.compliance_checks,
332         "created_at": record.created_at.isoformat(),
333         "approved_at": record.approved_at.isoformat() if record.approved_at else
None,
334         "deployed_at": record.deployed_at.isoformat() if record.deployed_at else
None,
335         "audit_log": record.audit_log
336     }
337
338     with open(output_file, 'w') as f:
339         json.dump(report, f, indent=2)
340
341     print(f"Governance report exported to {output_file}")
342
343 # Example usage
344 if __name__ == "__main__":
345     # Initialize framework
346     framework = ModelGovernanceFramework()
347
348     # Register a critical risk model
349     credit_model = ModelGovernanceRecord(
350         model_id="model-001",
351         model_name="credit-risk-classifier",
352         model_version="v2.1.0",
353         risk_tier=RiskTier.CRITICAL,
354         business_owner="jane.smith@company.com",
355         development_team=[
356             "john.doe@company.com",
357             "alice.wong@company.com"
358         ],
359         model_card_url="https://internal/models/credit-risk/card.html",
360         technical_documentation_url="https://internal/models/credit-risk/docs/",
361         risk_assessment_url="https://internal/models/credit-risk/risk-assessment.pdf"
362     )
363
364     model_id = framework.register_model(credit_model)
365     print(f"Model registered: {model_id}")
366
367     # Request approvals
368     framework.request_approval(model_id, GovernanceRole.MODEL_VALIDATOR, "
validator@company.com")
369     framework.request_approval(model_id, GovernanceRole.COMPLIANCE_OFFICER, "
compliance@company.com")
370     framework.request_approval(model_id, GovernanceRole.LEGAL_COUNSEL, "legal@company.
com")
371     framework.request_approval(model_id, GovernanceRole.RISK_MANAGER, "risk@company.
com")
372     framework.request_approval(model_id, GovernanceRole.BUSINESS_OWNER, "jane.
smith@company.com")
373

```



```

374     # Grant approvals
375     framework.grant_approval(
376         model_id,
377         GovernanceRole.MODEL_VALIDATOR,
378         "validator@company.com",
379         "Model passes all validation tests"
380     )
381
382     framework.grant_approval(
383         model_id,
384         GovernanceRole.COMPLIANCE_OFFICER,
385         "compliance@company.com",
386         "Compliant with GDPR and internal policies"
387     )
388
389     # Check approval status
390     is_approved = framework.check_approval_status(model_id)
391     print(f"Approval status: {is_approved}")
392
393     # Export governance report
394     framework.export_governance_report(model_id, "governance_report.json")

```

Listing 20.1: Model Governance Framework Structure

20.3.2 Model Inventory and Lifecycle Management

```

1  #!/usr/bin/env python3
2  """
3  Model Inventory and Lifecycle Management
4  Centralized tracking of all models in the organization
5  """
6
7  from dataclasses import dataclass, field
8  from typing import List, Dict, Optional
9  from enum import Enum
10 from datetime import datetime, timedelta
11 import sqlite3
12 import json
13
14 class ModelStatus(Enum):
15     """Model lifecycle status"""
16     DEVELOPMENT = "development"
17     TESTING = "testing"
18     PENDING_APPROVAL = "pending_approval"
19     APPROVED = "approved"
20     DEPLOYED = "deployed"
21     DEPRECATED = "deprecated"
22     RETIRED = "retired"
23
24 class ModelType(Enum):
25     """Type of ML model"""
26     CLASSIFICATION = "classification"
27     REGRESSION = "regression"
28     CLUSTERING = "clustering"
29     RECOMMENDATION = "recommendation"
30     NLP = "nlp"
31     COMPUTER_VISION = "computer_vision"

```

```

32     TIME_SERIES = "time_series"
33     REINFORCEMENT_LEARNING = "reinforcement_learning"
34
35 @dataclass
36 class ModelInventoryEntry:
37     """Single model in inventory"""
38     model_id: str
39     model_name: str
40     model_version: str
41     model_type: ModelType
42     status: ModelStatus
43
44     # Ownership
45     business_owner: str
46     technical_owner: str
47     team: str
48
49     # Technical details
50     framework: str # sklearn, tensorflow, pytorch, etc.
51     algorithm: str
52     training_data_source: str
53     features: List[str]
54     target_variable: str
55
56     # Performance
57     performance_metrics: Dict[str, float]
58
59     # Deployment
60     deployment_environment: Optional[str] = None
61     endpoint_url: Optional[str] = None
62
63     # Compliance
64     data_privacy_review: bool = False
65     fairness_assessment: bool = False
66     security_review: bool = False
67
68     # Timestamps
69     created_at: datetime = field(default_factory=datetime.now)
70     last_updated: datetime = field(default_factory=datetime.now)
71     deployed_at: Optional[datetime] = None
72     deprecated_at: Optional[datetime] = None
73
74 class ModelInventorySystem:
75     """Centralized model inventory management"""
76
77     def __init__(self, db_path: str = "model_inventory.db"):
78         self.db_path = db_path
79         self._initialize_database()
80
81     def _initialize_database(self):
82         """Create database schema"""
83
84         conn = sqlite3.connect(self.db_path)
85         cursor = conn.cursor()
86
87         cursor.execute("""
88             CREATE TABLE IF NOT EXISTS models (
89                 model_id TEXT PRIMARY KEY,

```

```

90         model_name TEXT NOT NULL,
91         model_version TEXT NOT NULL,
92         model_type TEXT NOT NULL,
93         status TEXT NOT NULL,
94         business_owner TEXT NOT NULL,
95         technical_owner TEXT NOT NULL,
96         team TEXT NOT NULL,
97         framework TEXT NOT NULL,
98         algorithm TEXT NOT NULL,
99         training_data_source TEXT,
100        features TEXT,
101        target_variable TEXT,
102        performance_metrics TEXT,
103        deployment_environment TEXT,
104        endpoint_url TEXT,
105        data_privacy_review BOOLEAN,
106        fairness_assessment BOOLEAN,
107        security_review BOOLEAN,
108        created_at TIMESTAMP,
109        last_updated TIMESTAMP,
110        deployed_at TIMESTAMP,
111        deprecated_at TIMESTAMP,
112        UNIQUE(model_name, model_version)
113    )
114    """
115
116    cursor.execute("""
117        CREATE TABLE IF NOT EXISTS model_versions (
118            version_id INTEGER PRIMARY KEY AUTOINCREMENT,
119            model_name TEXT NOT NULL,
120            model_version TEXT NOT NULL,
121            parent_version TEXT,
122            changes TEXT,
123            created_by TEXT,
124            created_at TIMESTAMP
125        )
126    """)
127
128    cursor.execute("""
129        CREATE TABLE IF NOT EXISTS model_dependencies (
130            dependency_id INTEGER PRIMARY KEY AUTOINCREMENT,
131            model_id TEXT NOT NULL,
132            dependency_type TEXT NOT NULL,
133            dependency_name TEXT NOT NULL,
134            dependency_version TEXT,
135            FOREIGN KEY (model_id) REFERENCES models(model_id)
136        )
137    """)
138
139    conn.commit()
140    conn.close()
141
142    def add_model(self, entry: ModelInventoryEntry) -> str:
143        """Add model to inventory"""
144
145        conn = sqlite3.connect(self.db_path)
146        cursor = conn.cursor()
147

```

```

148     cursor.execute("""
149         INSERT INTO models (
150             model_id, model_name, model_version, model_type, status,
151             business_owner, technical_owner, team, framework, algorithm,
152             training_data_source, features, target_variable,
153             performance_metrics, deployment_environment, endpoint_url,
154             data_privacy_review, fairness_assessment, security_review,
155             created_at, last_updated
156         ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
157     """, (
158         entry.model_id,
159         entry.model_name,
160         entry.model_version,
161         entry.model_type.value,
162         entry.status.value,
163         entry.business_owner,
164         entry.technical_owner,
165         entry.team,
166         entry.framework,
167         entry.algorithm,
168         entry.training_data_source,
169         json.dumps(entry.features),
170         entry.target_variable,
171         json.dumps(entry.performance_metrics),
172         entry.deployment_environment,
173         entry.endpoint_url,
174         entry.data_privacy_review,
175         entry.fairness_assessment,
176         entry.security_review,
177         entry.created_at,
178         entry.last_updated
179     ))
180
181     conn.commit()
182     conn.close()
183
184     return entry.model_id
185
186 def update_model_status(self, model_id: str, new_status: ModelStatus):
187     """Update model status"""
188
189     conn = sqlite3.connect(self.db_path)
190     cursor = conn.cursor()
191
192     cursor.execute("""
193         UPDATE models
194         SET status = ?, last_updated = ?
195         WHERE model_id = ?
196     """, (new_status.value, datetime.now(), model_id))
197
198     if new_status == ModelStatus.DEPLOYED:
199         cursor.execute("""
200             UPDATE models
201             SET deployed_at = ?
202             WHERE model_id = ?
203         """, (datetime.now(), model_id))
204
205     elif new_status == ModelStatus.DEPRECATED:

```

```

206         cursor.execute("""
207             UPDATE models
208             SET deprecated_at = ?
209             WHERE model_id = ?
210             """, (datetime.now(), model_id))
211
212     conn.commit()
213     conn.close()
214
215     def get_models_by_status(self, status: ModelStatus) -> List[Dict]:
216         """Get all models with specific status"""
217
218         conn = sqlite3.connect(self.db_path)
219         cursor = conn.cursor()
220
221         cursor.execute("""
222             SELECT * FROM models WHERE status = ?
223             """, (status.value,))
224
225         columns = [desc[0] for desc in cursor.description]
226         models = []
227
228         for row in cursor.fetchall():
229             model_dict = dict(zip(columns, row))
230             models.append(model_dict)
231
232         conn.close()
233         return models
234
235     def get_models_requiring_review(self, days_threshold: int = 90) -> List[Dict]:
236         """Get models that need periodic review"""
237
238         conn = sqlite3.connect(self.db_path)
239         cursor = conn.cursor()
240
241         threshold_date = datetime.now() - timedelta(days=days_threshold)
242
243         cursor.execute("""
244             SELECT * FROM models
245             WHERE status = 'deployed'
246             AND last_updated < ?
247             """, (threshold_date,))
248
249         columns = [desc[0] for desc in cursor.description]
250         models = []
251
252         for row in cursor.fetchall():
253             model_dict = dict(zip(columns, row))
254             models.append(model_dict)
255
256         conn.close()
257         return models
258
259     def get_inventory_statistics(self) -> Dict:
260         """Get inventory statistics"""
261
262         conn = sqlite3.connect(self.db_path)
263         cursor = conn.cursor()

```

```

264     stats = {}
265
266     # Total models
267     cursor.execute("SELECT COUNT(*) FROM models")
268     stats['total_models'] = cursor.fetchone()[0]
269
270     # Models by status
271     cursor.execute("""
272         SELECT status, COUNT(*)
273         FROM models
274         GROUP BY status
275     """)
276     stats['by_status'] = dict(cursor.fetchall())
277
278     # Models by type
279     cursor.execute("""
280         SELECT model_type, COUNT(*)
281         FROM models
282         GROUP BY model_type
283     """)
284     stats['by_type'] = dict(cursor.fetchall())
285
286     # Models by team
287     cursor.execute("""
288         SELECT team, COUNT(*)
289         FROM models
290         GROUP BY team
291     """)
292     stats['by_team'] = dict(cursor.fetchall())
293
294     # Compliance status
295     cursor.execute("""
296         SELECT
297             SUM(CASE WHEN data_privacy_review = 1 THEN 1 ELSE 0 END) as
298         privacy_reviewed,
299             SUM(CASE WHEN fairness_assessment = 1 THEN 1 ELSE 0 END) as
300         fairness_assessed,
301             SUM(CASE WHEN security_review = 1 THEN 1 ELSE 0 END) as
302         security_reviewed
303         FROM models
304     """)
305     compliance = cursor.fetchone()
306     stats['compliance'] = {
307         'privacy_reviewed': compliance[0],
308         'fairness_assessed': compliance[1],
309         'security_reviewed': compliance[2]
310     }
311
312     conn.close()
313     return stats
314
315 # Example usage
316 if __name__ == "__main__":
317     # Initialize inventory system
318     inventory = ModelInventorySystem()
319
320     # Add a model

```

```

319     fraud_model = ModelInventoryEntry(
320         model_id="fraud-detection-v1",
321         model_name="fraud-detection",
322         model_version="v1.0.0",
323         model_type=ModelType.CLASSIFICATION,
324         status=ModelStatus.DEVELOPMENT,
325         business_owner="fraud-team@company.com",
326         technical_owner="ml-team@company.com",
327         team="fraud-prevention",
328         framework="scikit-learn",
329         algorithm="RandomForestClassifier",
330         training_data_source="s3://data/fraud-transactions-2024/",
331         features=["transaction_amount", "merchant_id", "time_of_day", "user_history"],
332         target_variable="is_fraudulent",
333         performance_metrics={
334             "accuracy": 0.95,
335             "precision": 0.92,
336             "recall": 0.89,
337             "f1_score": 0.90,
338             "auc_roc": 0.94
339         },
340         data_privacy_review=True,
341         fairness_assessment=True,
342         security_review=False
343     )
344
345     inventory.add_model(fraud_model)
346     print(f"Model added: {fraud_model.model_id}")
347
348     # Get inventory statistics
349     stats = inventory.get_inventory_statistics()
350     print(f"\nInventory Statistics:")
351     print(f"Total models: {stats['total_models']}")
352     print(f"By status: {stats['by_status']}")
353     print(f"Compliance: {stats['compliance']}")

```

Listing 20.2: Model Inventory System

20.4 GDPR and CCPA Compliance for ML Systems

Regulatory compliance is critical for ML systems that process personal data. GDPR (General Data Protection Regulation) and CCPA (California Consumer Privacy Act) impose specific requirements on data processing and automated decision-making.

20.4.1 GDPR Requirements for ML

Key GDPR Principles for ML:

1. **Right to Explanation:** Individuals can request explanation of automated decisions (Article 22)
2. **Data Minimization:** Collect only necessary data for the specified purpose (Article 5)
3. **Purpose Limitation:** Use data only for stated purposes (Article 5)

4. **Right to Erasure:** Individuals can request deletion of their data (Article 17)
5. **Data Portability:** Provide data in machine-readable format (Article 20)
6. **Privacy by Design:** Build privacy into system architecture (Article 25)
7. **Data Protection Impact Assessment (DPIA):** Required for high-risk processing (Article 35)

20.4.2 GDPR Compliance Implementation

```

1  #!/usr/bin/env python3
2  """
3  GDPR Compliance Framework for ML Systems
4  Implements key GDPR requirements for ML models
5  """
6
7  from dataclasses import dataclass
8  from typing import List, Dict, Optional
9  from datetime import datetime
10 import hashlib
11 import json
12
13 @dataclass
14 class DataSubjectRequest:
15     """GDPR data subject request"""
16     request_id: str
17     subject_id: str
18     request_type: str # explanation, deletion, portability, rectification
19     timestamp: datetime
20     status: str # pending, completed, rejected
21     response: Optional[str] = None
22
23 class GDPRCompliance:
24     """GDPR compliance manager for ML systems"""
25
26     def __init__(self, model_id: str):
27         self.model_id = model_id
28         self.data_usage_log: List[Dict] = []
29         self.subject_requests: List[DataSubjectRequest] = []
30
31     def log_data_processing(
32         self,
33         subject_id: str,
34         purpose: str,
35         legal_basis: str,
36         data_categories: List[str]
37     ):
38         """Log data processing activity (Article 30)"""
39
40         log_entry = {
41             "timestamp": datetime.now().isoformat(),
42             "subject_id": self._pseudonymize(subject_id),
43             "purpose": purpose,
44             "legal_basis": legal_basis,
45             "data_categories": data_categories,
46             "model_id": self.model_id

```



```

47         }
48
49         self.data_usage_log.append(log_entry)
50
51     def _pseudonymize(self, subject_id: str) -> str:
52         """Pseudonymize user ID"""
53         return hashlib.sha256(subject_id.encode()).hexdigest()[:16]
54
55     def generate_explanation(
56         self,
57         subject_id: str,
58         prediction: float,
59         feature_importance: Dict[str, float],
60         threshold: float = 0.5
61     ) -> str:
62         """Generate human-readable explanation (Article 22)"""
63
64         decision = "approved" if prediction >= threshold else "denied"
65
66         # Sort features by importance
67         top_features = sorted(
68             feature_importance.items(),
69             key=lambda x: abs(x[1]),
70             reverse=True
71         )[:5]
72
73         explanation = f"""
74 Automated Decision Explanation
75
76 Subject ID: {self._pseudonymize(subject_id)}
77 Model: {self.model_id}
78 Decision: {decision.upper()}
79 Confidence: {prediction:.2%}
80 Timestamp: {datetime.now().isoformat()}
81
82 Top Contributing Factors:
83 """
84
85         for feature, importance in top_features:
86             direction = "increased" if importance > 0 else "decreased"
87             explanation += f"\n- {feature}: {direction} likelihood by {abs(importance)
88 :.2%}"
89
90         explanation += f"""
91
92 Right to Contest:
93 You have the right to contest this decision and request human review.
94 Contact: dpo@company.com
95
96 This explanation is provided in accordance with GDPR Article 22.
97 """
98
99         return explanation
100
101     def handle_deletion_request(
102         self,
103         subject_id: str,
104         model_training_data_location: str

```

```

104 ) -> Dict:
105     """Handle right to erasure request (Article 17)"""
106
107     request = DataSubjectRequest(
108         request_id=f"DEL-{datetime.now().strftime('%Y%m%d%H%M%S')}",
109         subject_id=self._pseudonymize(subject_id),
110         request_type="deletion",
111         timestamp=datetime.now(),
112         status="pending"
113     )
114
115     # Steps for deletion
116     steps = {
117         "1_remove_from_training_data": False,
118         "2_remove_from_feature_store": False,
119         "3_invalidate_affected_models": False,
120         "4_remove_from_logs": False,
121         "5_remove_from_backups": False,
122         "6_document_deletion": False
123     }
124
125     # In practice, implement actual deletion logic
126     # This is a template showing required steps
127
128     response = {
129         "request_id": request.request_id,
130         "subject_id": request.subject_id,
131         "status": "Data deletion initiated",
132         "steps_required": steps,
133         "completion_estimate": "30 days",
134         "notes": [
135             "Training data will be removed from active datasets",
136             "Models trained on affected data will be marked for retraining",
137             "Logs will be anonymized",
138             "Backups will be marked for deletion after retention period"
139         ]
140     }
141
142     request.response = json.dumps(response)
143     self.subject_requests.append(request)
144
145     return response
146
147 def handle_portability_request(
148     self,
149     subject_id: str,
150     data_store_path: str
151 ) -> Dict:
152     """Handle data portability request (Article 20)"""
153
154     request = DataSubjectRequest(
155         request_id=f"PORT-{datetime.now().strftime('%Y%m%d%H%M%S')}",
156         subject_id=self._pseudonymize(subject_id),
157         request_type="portability",
158         timestamp=datetime.now(),
159         status="completed"
160     )
161

```

```

162     # Collect all data related to subject
163     portable_data = {
164         "subject_id": self._pseudonymize(subject_id),
165         "data_export_date": datetime.now().isoformat(),
166         "data_categories": {
167             "profile_data": {},
168             "behavioral_data": {},
169             "predictions": [],
170             "processing_activities": [
171                 entry for entry in self.data_usage_log
172                 if entry["subject_id"] == self._pseudonymize(subject_id)
173             ]
174         },
175         "format": "JSON",
176         "gdpr_article": "Article 20 - Right to Data Portability"
177     }
178
179     return portable_data
180
181 def conduct_dpia(self, model_description: Dict) -> Dict:
182     """Conduct Data Protection Impact Assessment (Article 35)"""
183
184     dpia = {
185         "model_id": self.model_id,
186         "assessment_date": datetime.now().isoformat(),
187         "model_description": model_description,
188         "risk_assessment": {},
189         "mitigation_measures": {},
190         "residual_risks": {}
191     }
192
193     # Assess risks
194     risks = [
195         {
196             "risk_id": "R1",
197             "description": "Unauthorized access to personal data",
198             "likelihood": "medium",
199             "impact": "high",
200             "risk_level": "high",
201             "mitigation": [
202                 "Encrypt data at rest and in transit",
203                 "Implement access controls (RBAC)",
204                 "Regular security audits"
205             ]
206         },
207         {
208             "risk_id": "R2",
209             "description": "Discriminatory bias in predictions",
210             "likelihood": "medium",
211             "impact": "high",
212             "risk_level": "high",
213             "mitigation": [
214                 "Regular fairness testing",
215                 "Diverse training data",
216                 "Human oversight for high-stakes decisions"
217             ]
218         },
219         {

```

```

220         "risk_id": "R3",
221         "description": "Model inversion attacks",
222         "likelihood": "low",
223         "impact": "high",
224         "risk_level": "medium",
225         "mitigation": [
226             "Differential privacy in training",
227             "Rate limiting on API",
228             "Monitor for unusual query patterns"
229         ]
230     },
231     {
232         "risk_id": "R4",
233         "description": "Data retention beyond legal period",
234         "likelihood": "medium",
235         "impact": "medium",
236         "risk_level": "medium",
237         "mitigation": [
238             "Automated data retention policies",
239             "Regular data cleanup jobs",
240             "Audit data age"
241         ]
242     }
243 ]
244
245 dpia["risk_assessment"] = risks
246
247 # Calculate overall risk
248 high_risks = sum(1 for r in risks if r["risk_level"] == "high")
249 dpia["overall_risk"] = "high" if high_risks > 0 else "medium"
250 dpia["dpo_consultation_required"] = high_risks > 0
251 dpia["supervisory_authority_consultation_required"] = high_risks > 2
252
253 return dpia
254
255 def generate_compliance_report(self, output_file: str):
256     """Generate GDPR compliance report"""
257
258     report = {
259         "model_id": self.model_id,
260         "report_date": datetime.now().isoformat(),
261         "data_processing_activities": len(self.data_usage_log),
262         "subject_requests": {
263             "total": len(self.subject_requests),
264             "by_type": {},
265             "average_response_time_days": 15 # Example
266         },
267         "compliance_measures": {
268             "data_minimization": "Implemented",
269             "purpose_limitation": "Implemented",
270             "storage_limitation": "Implemented",
271             "accuracy": "Regular monitoring",
272             "integrity_confidentiality": "Encryption enabled",
273             "accountability": "Full audit trail"
274         },
275         "recent_requests": [
276             {
277                 "request_id": req.request_id,

```

```

278         "type": req.request_type,
279         "status": req.status,
280         "timestamp": req.timestamp.isoformat()
281     }
282     for req in self.subject_requests[-10:]
283 ]
284 }
285
286 with open(output_file, 'w') as f:
287     json.dump(report, f, indent=2)
288
289 print(f"GDPR compliance report generated: {output_file}")
290 return report
291
292 # Example usage
293 if __name__ == "__main__":
294     gdpr = GDPRCompliance(model_id="credit-risk-v1")
295
296     # Log data processing
297     gdpr.log_data_processing(
298         subject_id="user123",
299         purpose="Credit risk assessment",
300         legal_basis="Legitimate interest",
301         data_categories=["financial_history", "employment", "credit_score"]
302     )
303
304     # Generate explanation
305     explanation = gdpr.generate_explanation(
306         subject_id="user123",
307         prediction=0.35,
308         feature_importance={
309             "credit_score": -0.25,
310             "income": 0.15,
311             "debt_to_income_ratio": -0.20,
312             "employment_years": 0.10,
313             "previous_defaults": -0.15
314         }
315     )
316     print(explanation)
317
318     # Handle deletion request
319     deletion_response = gdpr.handle_deletion_request(
320         subject_id="user123",
321         model_training_data_location="s3://ml-data/credit-risk-training/"
322     )
323     print(f"\nDeletion request: {deletion_response['request_id']}")
324
325     # Conduct DPIA
326     dpia = gdpr.conduct_dpia({
327         "name": "Credit Risk Assessment Model",
328         "purpose": "Automated credit decision making",
329         "data_sources": ["Credit bureau", "Internal transaction history"],
330         "data_subjects": "Loan applicants",
331         "processing_type": "Automated decision-making"
332     })
333     print(f"\nDPIA Overall Risk: {dpia['overall_risk']}")
334     print(f"DPO Consultation Required: {dpia['dpo_consultation_required']}")
335

```

```

336 # Generate compliance report
337 gdpr.generate_compliance_report("gdpr_compliance_report.json")

```

Listing 20.3: GDPR Compliance Framework for ML Systems

20.4.3 CCPA Compliance

CCPA Requirements for ML:

1. **Right to Know:** Disclose what personal information is collected
2. **Right to Delete:** Allow consumers to request deletion
3. **Right to Opt-Out:** Allow opt-out of sale of personal information
4. **Right to Non-Discrimination:** Cannot discriminate against consumers who exercise rights
5. **Notice at Collection:** Inform consumers of data collection purposes

```

1  #!/usr/bin/env python3
2  """
3  CCPA Compliance Manager
4  Implements CCPA requirements for ML systems
5  """
6
7  from dataclasses import dataclass
8  from typing import List, Dict
9  from datetime import datetime
10 import json
11
12 class CCPACompliance:
13     """CCPA compliance manager"""
14
15     def __init__(self, business_name: str):
16         self.business_name = business_name
17         self.opt_out_registry: Dict[str, datetime] = {}
18
19     def generate_notice_at_collection(self, model_purpose: str) -> str:
20         """Generate CCPA-compliant notice"""
21
22         notice = f"""
23 NOTICE AT COLLECTION OF PERSONAL INFORMATION
24
25 Business: {self.business_name}
26
27 Categories of Personal Information Collected:
28 - Identifiers (name, email, ID numbers)
29 - Commercial information (transaction history, purchases)
30 - Internet activity (browsing history, interactions)
31 - Geolocation data
32 - Inferences (preferences, characteristics, behavior)
33
34 Purpose of Collection and Use:
35 - {model_purpose}
36 - Service improvement

```

```

37 - Fraud prevention
38 - Legal compliance
39
40 Third Parties with Whom Information is Shared:
41 - Service providers (cloud hosting, analytics)
42 - Business partners (with your consent)
43
44 Your CCPA Rights:
45 1. Right to Know: Request details about personal information collected
46 2. Right to Delete: Request deletion of your personal information
47 3. Right to Opt-Out: Opt out of sale of personal information
48 4. Right to Non-Discrimination: Equal service regardless of privacy choices
49
50 To Exercise Your Rights:
51 - Email: privacy@{self.business_name}.com
52 - Phone: 1-800-PRIVACY
53 - Web: https://{self.business_name}.com/privacy-request
54
55 We will respond within 45 days of receiving your request.
56
57 Last Updated: {datetime.now().strftime('%Y-%m-%d')}
58 """
59     return notice
60
61     def handle_opt_out_request(self, consumer_id: str) -> Dict:
62         """Handle opt-out of sale request"""
63
64         self.opt_out_registry[consumer_id] = datetime.now()
65
66         return {
67             "consumer_id": consumer_id,
68             "opt_out_status": "active",
69             "effective_date": datetime.now().isoformat(),
70             "actions_taken": [
71                 "Flagged in CRM system",
72                 "Excluded from data sharing with partners",
73                 "Removed from third-party analytics",
74                 "Updated consent preferences"
75             ]
76         }
77
78     def verify_opt_out_status(self, consumer_id: str) -> bool:
79         """Check if consumer has opted out"""
80         return consumer_id in self.opt_out_registry
81
82     def generate_data_inventory(self) -> Dict:
83         """Generate inventory of personal information (required for CCPA)"""
84
85         inventory = {
86             "business_name": self.business_name,
87             "report_date": datetime.now().isoformat(),
88             "categories_collected": [
89                 {
90                     "category": "Identifiers",
91                     "examples": ["Name", "Email", "Phone", "User ID"],
92                     "source": "Direct from consumer",
93                     "business_purpose": "Account management, communication",
94                     "third_parties_shared": ["Cloud provider", "Email service"]

```

```

95         },
96         {
97             "category": "Commercial Information",
98             "examples": ["Purchase history", "Payment information"],
99             "source": "Transaction records",
100             "business_purpose": "Order fulfillment, fraud prevention",
101             "third_parties_shared": ["Payment processor"]
102         },
103         {
104             "category": "Inferences",
105             "examples": ["Preferences", "Risk scores", "Predictions"],
106             "source": "Derived from ML models",
107             "business_purpose": "Personalization, risk assessment",
108             "third_parties_shared": ["None"]
109         }
110     ],
111     "retention_periods": {
112         "transaction_data": "7 years (legal requirement)",
113         "behavioral_data": "2 years",
114         "model_predictions": "1 year",
115         "training_data": "5 years"
116     }
117 }
118
119 return inventory
120
121 # Example usage
122 if __name__ == "__main__":
123     ccpa = CCPACompliance(business_name="ExampleCorp")
124
125     # Generate notice
126     notice = ccpa.generate_notice_at_collection(
127         model_purpose="Credit risk assessment and loan decision automation"
128     )
129     print(notice)
130
131     # Handle opt-out
132     opt_out_response = ccpa.handle_opt_out_request("consumer_12345")
133     print(f"\n\nOpt-out processed: {opt_out_response}")
134
135     # Generate data inventory
136     inventory = ccpa.generate_data_inventory()
137     print(f"\nData categories tracked: {len(inventory['categories_collected'])}")

```

Listing 20.4: CCPA Compliance Manager

20.5 Model Documentation and Audit Trails

Comprehensive documentation and audit trails are essential for governance, compliance, and debugging model issues.

20.5.1 Model Cards

Model Cards provide standardized documentation for ML models, promoting transparency and responsible AI.


```

1  #!/usr/bin/env python3
2  """
3  Model Card Generator
4  Creates standardized model documentation following Model Cards framework
5  Reference: Mitchell et al. (2019) "Model Cards for Model Reporting"
6  """
7
8  from dataclasses import dataclass, field
9  from typing import List, Dict, Optional
10 from datetime import datetime
11 import json
12
13 @dataclass
14 class ModelCard:
15     """Standardized model documentation"""
16
17     # Model Details
18     model_name: str
19     model_version: str
20     model_date: datetime
21     model_type: str
22     paper_or_resource: Optional[str] = None
23     citation: Optional[str] = None
24     license: str = "Proprietary"
25     contact: str = ""
26
27     # Intended Use
28     primary_uses: List[str] = field(default_factory=list)
29     primary_users: List[str] = field(default_factory=list)
30     out_of_scope_uses: List[str] = field(default_factory=list)
31
32     # Factors
33     relevant_factors: List[str] = field(default_factory=list)
34     evaluation_factors: List[str] = field(default_factory=list)
35
36     # Metrics
37     model_performance_measures: Dict[str, float] = field(default_factory=dict)
38     decision_thresholds: Dict[str, float] = field(default_factory=dict)
39     approaches_to_uncertainty: str = ""
40
41     # Training Data
42     training_data_description: str = ""
43     training_data_source: str = ""
44     training_data_size: int = 0
45     training_data_preprocessing: List[str] = field(default_factory=list)
46
47     # Evaluation Data
48     evaluation_data_description: str = ""
49     evaluation_data_source: str = ""
50     evaluation_data_size: int = 0
51
52     # Quantitative Analyses
53     unitary_results: Dict[str, float] = field(default_factory=dict)
54     intersectional_results: Dict[str, Dict] = field(default_factory=dict)
55
56     # Ethical Considerations
57     sensitive_data: List[str] = field(default_factory=list)

```

```

58     human_life_impact: str = ""
59     mitigations: List[str] = field(default_factory=list)
60     risks_and_harms: List[str] = field(default_factory=list)
61     use_cases: List[str] = field(default_factory=list)
62
63     # Caveats and Recommendations
64     caveats: List[str] = field(default_factory=list)
65     recommendations: List[str] = field(default_factory=list)
66
67 class ModelCardGenerator:
68     """Generate standardized model cards"""
69
70     def generate_html(self, card: ModelCard, output_file: str):
71         """Generate HTML model card"""
72
73         html = f"""
74 <!DOCTYPE html>
75 <html>
76 <head>
77     <title>Model Card: {card.model_name}</title>
78     <style>
79         body {{ font-family: Arial, sans-serif; max-width: 800px; margin: 40px auto;
padding: 20px; }}
80         h1 {{ color: #333; border-bottom: 2px solid #007bff; padding-bottom: 10px; }}
81         h2 {{ color: #555; margin-top: 30px; border-bottom: 1px solid #ddd; padding-
bottom: 5px; }}
82         .section {{ margin: 20px 0; }}
83         .metric {{ background: #f8f9fa; padding: 10px; margin: 5px 0; border-left: 3px
solid #007bff; }}
84         .warning {{ background: #fff3cd; padding: 15px; border-left: 4px solid #ffc107
; margin: 20px 0; }}
85         ul {{ line-height: 1.8; }}
86         table {{ width: 100%; border-collapse: collapse; margin: 15px 0; }}
87         th, td {{ padding: 12px; text-align: left; border-bottom: 1px solid #ddd; }}
88         th {{ background-color: #007bff; color: white; }}
89     </style>
90 </head>
91 <body>
92     <h1>Model Card: {card.model_name}</h1>
93
94     <div class="section">
95         <p><strong>Version:</strong> {card.model_version}</p>
96         <p><strong>Date:</strong> {card.model_date.strftime('%Y-%m-%d')}</p>
97         <p><strong>Model Type:</strong> {card.model_type}</p>
98         <p><strong>Contact:</strong> {card.contact}</p>
99         <p><strong>License:</strong> {card.license}</p>
100     </div>
101
102     <h2>Intended Use</h2>
103     <div class="section">
104         <h3>Primary Uses</h3>
105         <ul>
106             {','.join([f'<li>{use}</li>' for use in card.primary_uses])}
107         </ul>
108
109         <h3>Primary Users</h3>
110         <ul>
111             {','.join([f'<li>{user}</li>' for user in card.primary_users])}

```

```

112         </ul>
113
114         <h3>Out of Scope Uses</h3>
115         <div class="warning">
116             <strong>Warning:</strong> This model should NOT be used for:
117             <ul>
118                 {''.join([f'<li>{use}</li>' for use in card.out_of_scope_uses])}
119             </ul>
120         </div>
121     </div>
122
123     <h2>Model Performance</h2>
124     <div class="section">
125         <table>
126             <tr>
127                 <th>Metric</th>
128                 <th>Value</th>
129             </tr>
130             {''.join([f'<tr><td>{metric}</td><td>{value:.4f}</td></tr>' for metric,
value in card.model_performance_measures.items()])}
131         </table>
132     </div>
133
134     <h2>Training Data</h2>
135     <div class="section">
136         <p><strong>Description:</strong> {card.training_data_description}</p>
137         <p><strong>Source:</strong> {card.training_data_source}</p>
138         <p><strong>Size:</strong> {card.training_data_size:,} samples</p>
139
140         <h3>Preprocessing Steps</h3>
141         <ul>
142             {''.join([f'<li>{step}</li>' for step in card.training_data_preprocessing
]])}
143         </ul>
144     </div>
145
146     <h2>Evaluation Data</h2>
147     <div class="section">
148         <p><strong>Description:</strong> {card.evaluation_data_description}</p>
149         <p><strong>Source:</strong> {card.evaluation_data_source}</p>
150         <p><strong>Size:</strong> {card.evaluation_data_size:,} samples</p>
151     </div>
152
153     <h2>Ethical Considerations</h2>
154     <div class="section">
155         <h3>Sensitive Data Used</h3>
156         <ul>
157             {''.join([f'<li>{data}</li>' for data in card.sensitive_data])}
158         </ul>
159
160         <h3>Impact on Human Life</h3>
161         <p>{card.human_life_impact}</p>
162
163         <h3>Risks and Harms</h3>
164         <ul>
165             {''.join([f'<li>{risk}</li>' for risk in card.risks_and_harms])}
166         </ul>
167

```

```

168         <h3>Mitigations</h3>
169         <ul>
170             {','.join([f'<li>{mitigation}</li>' for mitigation in card.mitigations])}
171         </ul>
172     </div>
173
174     <h2>Caveats and Recommendations</h2>
175     <div class="section">
176         <h3>Caveats</h3>
177         <ul>
178             {','.join([f'<li>{caveat}</li>' for caveat in card.caveats])}
179         </ul>
180
181         <h3>Recommendations</h3>
182         <ul>
183             {','.join([f'<li>{rec}</li>' for rec in card.recommendations])}
184         </ul>
185     </div>
186
187     <hr>
188     <p style="text-align: center; color: #666; font-size: 0.9em;">
189         Model Card generated on {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}
190     </p>
191 </body>
192 </html>
193 """
194
195     with open(output_file, 'w') as f:
196         f.write(html)
197
198     print(f"Model card generated: {output_file}")
199
200     def generate_json(self, card: ModelCard, output_file: str):
201         """Generate JSON model card"""
202
203         card_dict = {
204             "model_details": {
205                 "name": card.model_name,
206                 "version": card.model_version,
207                 "date": card.model_date.isoformat(),
208                 "type": card.model_type,
209                 "paper_or_resource": card.paper_or_resource,
210                 "citation": card.citation,
211                 "license": card.license,
212                 "contact": card.contact
213             },
214             "intended_use": {
215                 "primary_uses": card.primary_uses,
216                 "primary_users": card.primary_users,
217                 "out_of_scope_uses": card.out_of_scope_uses
218             },
219             "factors": {
220                 "relevant_factors": card.relevant_factors,
221                 "evaluation_factors": card.evaluation_factors
222             },
223             "metrics": {
224                 "model_performance": card.model_performance_measures,
225                 "decision_thresholds": card.decision_thresholds,

```

```

226         "uncertainty_approaches": card.approaches_to_uncertainty
227     },
228     "training_data": {
229         "description": card.training_data_description,
230         "source": card.training_data_source,
231         "size": card.training_data_size,
232         "preprocessing": card.training_data_preprocessing
233     },
234     "evaluation_data": {
235         "description": card.evaluation_data_description,
236         "source": card.evaluation_data_source,
237         "size": card.evaluation_data_size
238     },
239     "quantitative_analyses": {
240         "unitary_results": card.unitary_results,
241         "intersectional_results": card.intersectional_results
242     },
243     "ethical_considerations": {
244         "sensitive_data": card.sensitive_data,
245         "human_life_impact": card.human_life_impact,
246         "mitigations": card.mitigations,
247         "risks_and_harms": card.risks_and_harms
248     },
249     "caveats_and_recommendations": {
250         "caveats": card.caveats,
251         "recommendations": card.recommendations
252     }
253 }
254
255 with open(output_file, 'w') as f:
256     json.dump(card_dict, f, indent=2)
257
258 print(f"Model card JSON exported: {output_file}")
259
260 # Example usage
261 if __name__ == "__main__":
262     # Create model card
263     card = ModelCard(
264         model_name="Credit Risk Classifier",
265         model_version="v2.1.0",
266         model_date=datetime.now(),
267         model_type="Binary Classification (Random Forest)",
268         contact="ml-team@company.com",
269         primary_uses=[
270             "Automated credit risk assessment for loan applications",
271             "Risk-based pricing decisions",
272             "Portfolio risk monitoring"
273         ],
274         primary_users=[
275             "Loan officers",
276             "Risk management teams",
277             "Automated lending platform"
278         ],
279         out_of_scope_uses=[
280             "Employment decisions",
281             "Insurance underwriting",
282             "Criminal justice applications",
283             "Decisions where model cannot be explained"

```

```

284 ],
285 model_performance_measures={
286     "Accuracy": 0.912,
287     "Precision": 0.897,
288     "Recall": 0.883,
289     "F1-Score": 0.890,
290     "AUC-ROC": 0.945
291 },
292 training_data_description="Historical loan application data from 2018-2023",
293 training_data_source="Internal CRM and credit bureau data",
294 training_data_size=245000,
295 training_data_preprocessing=[
296     "Removed personally identifiable information",
297     "Imputed missing values using median strategy",
298     "Standardized numerical features",
299     "One-hot encoded categorical variables",
300     "Balanced classes using SMOTE"
301 ],
302 evaluation_data_description="Hold-out test set from 2024 Q1",
303 evaluation_data_source="Internal CRM",
304 evaluation_data_size=32000,
305 sensitive_data=[
306     "Credit history",
307     "Income information",
308     "Employment status",
309     "Demographic data (for fairness testing only)"
310 ],
311 human_life_impact="Model decisions affect individuals' ability to obtain
credit, impacting their financial well-being and life opportunities.",
312 risks_and_harms=[
313     "Potential for discriminatory outcomes across protected groups",
314     "Economic harm from false rejections",
315     "Credit availability disparities",
316     "Reinforcement of historical lending biases"
317 ],
318 mitigations=[
319     "Regular fairness audits across demographic groups",
320     "Human review for borderline cases",
321     "Appeals process for rejected applications",
322     "Ongoing monitoring of disparate impact",
323     "Periodic model retraining with recent data"
324 ],
325 caveats=[
326     "Model trained on historical data may not reflect current economic
conditions",
327     "Performance may degrade during economic crises",
328     "Protected attributes not used in training but proxy variables may exist",
329     "Model should not be sole decision-maker for high-value loans"
330 ],
331 recommendations=[
332     "Combine with human judgment for loans above $50,000",
333     "Review model quarterly for performance degradation",
334     "Conduct annual fairness audits",
335     "Retrain model when AUC drops below 0.90",
336     "Maintain human override capability"
337 ]
338 )
339

```

```

340     # Generate outputs
341     generator = ModelCardGenerator()
342     generator.generate_html(card, "model_card.html")
343     generator.generate_json(card, "model_card.json")

```

Listing 20.5: Model Card Generator

20.5.2 Audit Trail Implementation

```

1  #!/usr/bin/env python3
2  """
3  Audit Trail System for ML Models
4  Immutable logging of all model-related activities
5  """
6
7  from dataclasses import dataclass
8  from typing import Dict, List, Optional
9  from datetime import datetime
10 import hashlib
11 import json
12 import sqlite3
13
14 @dataclass
15 class AuditEvent:
16     """Single audit event"""
17     event_id: str
18     timestamp: datetime
19     event_type: str
20     actor: str
21     model_id: str
22     action: str
23     details: Dict
24     ip_address: Optional[str] = None
25     previous_hash: Optional[str] = None
26     current_hash: Optional[str] = None
27
28 class AuditTrailSystem:
29     """Immutable audit trail for model governance"""
30
31     def __init__(self, db_path: str = "audit_trail.db"):
32         self.db_path = db_path
33         self._initialize_database()
34
35     def _initialize_database(self):
36         """Create audit trail database"""
37
38         conn = sqlite3.connect(self.db_path)
39         cursor = conn.cursor()
40
41         cursor.execute("""
42             CREATE TABLE IF NOT EXISTS audit_events (
43                 event_id TEXT PRIMARY KEY,
44                 timestamp TIMESTAMP NOT NULL,
45                 event_type TEXT NOT NULL,
46                 actor TEXT NOT NULL,
47                 model_id TEXT NOT NULL,
48                 action TEXT NOT NULL,

```

```

49         details TEXT NOT NULL,
50         ip_address TEXT,
51         previous_hash TEXT,
52         current_hash TEXT NOT NULL,
53         UNIQUE(current_hash)
54     )
55 """
56
57     cursor.execute("""
58         CREATE INDEX IF NOT EXISTS idx_model_id ON audit_events(model_id)
59     """)
60
61     cursor.execute("""
62         CREATE INDEX IF NOT EXISTS idx_timestamp ON audit_events(timestamp)
63     """)
64
65     conn.commit()
66     conn.close()
67
68     def _calculate_hash(self, event: AuditEvent) -> str:
69         """Calculate cryptographic hash of event"""
70
71         event_string = f"{event.timestamp.isoformat()}|{event.event_type}|{event.actor}|{event.model_id}|{event.action}|{json.dumps(event.details, sort_keys=True)}|{event.previous_hash}"
72
73         return hashlib.sha256(event_string.encode()).hexdigest()
74
75     def _get_latest_hash(self, model_id: str) -> Optional[str]:
76         """Get hash of most recent event for model"""
77
78         conn = sqlite3.connect(self.db_path)
79         cursor = conn.cursor()
80
81         cursor.execute("""
82             SELECT current_hash FROM audit_events
83             WHERE model_id = ?
84             ORDER BY timestamp DESC
85             LIMIT 1
86         """, (model_id,))
87
88         result = cursor.fetchone()
89         conn.close()
90
91         return result[0] if result else None
92
93     def log_event(
94         self,
95         event_type: str,
96         actor: str,
97         model_id: str,
98         action: str,
99         details: Dict,
100         ip_address: Optional[str] = None
101     ) -> str:
102         """Log audit event"""
103
104         # Get previous hash for blockchain-like linking

```



```

105     previous_hash = self._get_latest_hash(model_id)
106
107     # Create event
108     event = AuditEvent(
109         event_id=f"{model_id}-{datetime.now().strftime('%Y%m%d%H%M%S%f')}",
110         timestamp=datetime.now(),
111         event_type=event_type,
112         actor=actor,
113         model_id=model_id,
114         action=action,
115         details=details,
116         ip_address=ip_address,
117         previous_hash=previous_hash
118     )
119
120     # Calculate hash
121     event.current_hash = self._calculate_hash(event)
122
123     # Store event
124     conn = sqlite3.connect(self.db_path)
125     cursor = conn.cursor()
126
127     cursor.execute("""
128         INSERT INTO audit_events (
129             event_id, timestamp, event_type, actor, model_id,
130             action, details, ip_address, previous_hash, current_hash
131         ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
132     """, (
133         event.event_id,
134         event.timestamp,
135         event.event_type,
136         event.actor,
137         event.model_id,
138         event.action,
139         json.dumps(event.details),
140         event.ip_address,
141         event.previous_hash,
142         event.current_hash
143     ))
144
145     conn.commit()
146     conn.close()
147
148     return event.event_id
149
150 def verify_integrity(self, model_id: str) -> bool:
151     """Verify audit trail integrity using hash chain"""
152
153     conn = sqlite3.connect(self.db_path)
154     cursor = conn.cursor()
155
156     cursor.execute("""
157         SELECT event_id, timestamp, event_type, actor, model_id,
158             action, details, previous_hash, current_hash
159         FROM audit_events
160         WHERE model_id = ?
161         ORDER BY timestamp ASC
162     """, (model_id,))

```

```

163     events = cursor.fetchall()
164     conn.close()
165
166     if not events:
167         return True
168
169     # Verify hash chain
170     for i, event_data in enumerate(events):
171         event = AuditEvent(
172             event_id=event_data[0],
173             timestamp=datetime.fromisoformat(event_data[1]),
174             event_type=event_data[2],
175             actor=event_data[3],
176             model_id=event_data[4],
177             action=event_data[5],
178             details=json.loads(event_data[6]),
179             previous_hash=event_data[7],
180             current_hash=event_data[8]
181         )
182
183         # Recalculate hash
184         expected_hash = self._calculate_hash(event)
185
186         if expected_hash != event.current_hash:
187             print(f"Integrity violation at event {event.event_id}")
188             return False
189
190         # Check chain linkage
191         if i > 0:
192             previous_event_hash = events[i-1][8]
193             if event.previous_hash != previous_event_hash:
194                 print(f"Chain break at event {event.event_id}")
195                 return False
196
197     return True
198
199 def get_model_history(self, model_id: str) -> List[Dict]:
200     """Get complete audit history for a model"""
201
202     conn = sqlite3.connect(self.db_path)
203     cursor = conn.cursor()
204
205     cursor.execute("""
206         SELECT event_id, timestamp, event_type, actor, action, details
207         FROM audit_events
208         WHERE model_id = ?
209         ORDER BY timestamp ASC
210     """, (model_id,))
211
212     events = []
213     for row in cursor.fetchall():
214         events.append({
215             "event_id": row[0],
216             "timestamp": row[1],
217             "event_type": row[2],
218             "actor": row[3],
219             "action": row[4],

```

```

221         "details": json.loads(row[5])
222     })
223
224     conn.close()
225     return events
226
227 def generate_audit_report(self, model_id: str, output_file: str):
228     """Generate audit report for compliance"""
229
230     history = self.get_model_history(model_id)
231     integrity_valid = self.verify_integrity(model_id)
232
233     report = {
234         "model_id": model_id,
235         "report_generated": datetime.now().isoformat(),
236         "total_events": len(history),
237         "integrity_verified": integrity_valid,
238         "timeline": history,
239         "summary": {
240             "first_event": history[0]["timestamp"] if history else None,
241             "last_event": history[-1]["timestamp"] if history else None,
242             "unique_actors": len(set(e["actor"] for e in history)),
243             "event_types": list(set(e["event_type"] for e in history))
244         }
245     }
246
247     with open(output_file, 'w') as f:
248         json.dump(report, f, indent=2)
249
250     print(f"Audit report generated: {output_file}")
251     return report
252
253 # Example usage
254 if __name__ == "__main__":
255     audit = AuditTrailSystem()
256
257     # Log various events
258     audit.log_event(
259         event_type="model_development",
260         actor="john.doe@company.com",
261         model_id="fraud-detection-v1",
262         action="training_started",
263         details={
264             "framework": "scikit-learn",
265             "algorithm": "RandomForest",
266             "training_samples": 100000
267         },
268         ip_address="192.168.1.100"
269     )
270
271     audit.log_event(
272         event_type="model_validation",
273         actor="validator@company.com",
274         model_id="fraud-detection-v1",
275         action="validation_passed",
276         details={
277             "accuracy": 0.95,
278             "f1_score": 0.92,

```

```

279         "validation_criteria_met": True
280     }
281 )
282
283 audit.log_event(
284     event_type="model_approval",
285     actor="compliance@company.com",
286     model_id="fraud-detection-v1",
287     action="approval_granted",
288     details={
289         "approval_type": "compliance_review",
290         "comments": "Meets all regulatory requirements"
291     }
292 )
293
294 audit.log_event(
295     event_type="model_deployment",
296     actor="ml-engineer@company.com",
297     model_id="fraud-detection-v1",
298     action="deployed_to_production",
299     details={
300         "environment": "production",
301         "endpoint": "https://api.company.com/fraud-detection/v1",
302         "deployment_type": "blue-green"
303     }
304 )
305
306 # Verify integrity
307 is_valid = audit.verify_integrity("fraud-detection-v1")
308 print(f"Audit trail integrity: {'VALID' if is_valid else 'INVALID'}")
309
310 # Generate audit report
311 audit.generate_audit_report("fraud-detection-v1", "audit_report.json")

```

Listing 20.6: Comprehensive Audit Trail System

20.6 Bias Detection and Fairness Testing

Ensuring fairness in ML models is both an ethical imperative and, increasingly, a legal requirement. Bias can arise from training data, feature selection, or algorithmic decisions.

20.6.1 Fairness Metrics

Common Fairness Definitions:

1. **Demographic Parity:** Equal selection rates across groups
2. **Equal Opportunity:** Equal true positive rates across groups
3. **Equalized Odds:** Equal TPR and FPR across groups
4. **Predictive Parity:** Equal precision across groups
5. **Calibration:** Predicted probabilities match observed frequencies

Bias Mitigation Strategies:

- **Pre-processing:** Reweighting, resampling, or removing biased features
- **In-processing:** Fairness constraints in optimization objective
- **Post-processing:** Adjust decision thresholds per group
- **Adversarial Debiasing:** Train model to be invariant to protected attributes

20.7 Risk Assessment Frameworks

Risk assessment helps prioritize governance efforts and determine appropriate controls for ML systems.

Risk Assessment Dimensions:

1. **Performance Risk:** Model accuracy degradation, data drift
2. **Fairness Risk:** Discriminatory outcomes, disparate impact
3. **Security Risk:** Adversarial attacks, model inversion
4. **Privacy Risk:** Data leakage, re-identification
5. **Operational Risk:** System failures, dependency issues
6. **Reputational Risk:** Public backlash, trust erosion
7. **Compliance Risk:** Regulatory violations, legal liability

Risk Scoring Matrix:

Impact \ Likelihood	Rare	Unlikely	Possible	Likely	Very Likely
Critical	Medium	High	High	Critical	Critical
High	Low	Medium	High	High	Critical
Medium	Low	Low	Medium	Medium	High
Low	Low	Low	Low	Low	Medium
Minimal	Low	Low	Low	Low	Low

Table 20.1: Risk Scoring Matrix

20.8 Approval Workflows and Controls

Approval workflows ensure models go through appropriate review before deployment based on risk level.

Approval Gate Structure:

1. **Technical Review:** Code quality, testing, performance
2. **Validation:** Model meets performance requirements

3. **Fairness Review:** Bias testing, disparate impact analysis
4. **Security Review:** Vulnerability scanning, threat modeling
5. **Compliance Review:** Regulatory requirements, legal approval
6. **Business Approval:** Stakeholder sign-off, resource allocation
7. **Final Sign-Off:** Executive approval for high-risk models

Automated Control Checks:

- Minimum accuracy/performance thresholds
- Fairness metrics within acceptable bounds
- Security vulnerability scan passing
- Required documentation complete
- Test coverage above minimum
- No critical code quality issues

20.9 Governance Implementation Exercises

20.9.1 Exercise 1: Build Governance Framework for Healthcare ML System

Scenario: Hospital deploying ML model for predicting patient readmission risk within 30 days. Model will inform care coordination decisions.

Context:

- Model accuracy: 82%
- Training data: 50,000 patient records (5 years)
- Protected attributes: age, gender, race, insurance type
- Regulatory environment: HIPAA, state healthcare regulations
- Impact: Affects care team workflows and resource allocation

Tasks:

1. Classify model risk tier (Low/Medium/High/Critical) with justification
2. Define required approvers and approval workflow
3. Identify minimum 8 governance requirements
4. Design fairness testing approach for protected attributes
5. Create model card with ethical considerations section

6. Establish monitoring metrics and alert thresholds
7. Define incident response procedure for model failures

Deliverables:

- Risk assessment report with scored risk factors
- Approval workflow diagram showing gates and approvers
- Governance policy document (2-3 pages)
- Model card (HTML or PDF)
- Monitoring dashboard specification

20.9.2 Exercise 2: GDPR Compliance Implementation

Scenario: E-commerce company using ML for personalized product recommendations. Expanding to EU market requires GDPR compliance.

Current State:

- Recommendation model trained on user behavioral data
- Data includes: browsing history, purchases, demographics, location
- Real-time inference via API
- Data retained indefinitely
- No current explainability capability

Tasks:

1. Conduct Data Protection Impact Assessment (DPIA)
2. Implement right to explanation mechanism
3. Design data deletion workflow for right to erasure
4. Create privacy notice for data collection
5. Implement consent management system
6. Establish data retention policy with automated deletion
7. Document legal basis for processing

Deliverables:

- Completed DPIA document
- Explanation generation code (Python)
- Data deletion procedure documentation
- CCPA/GDPR compliance checklist (completed)
- Privacy policy text
- Data flow diagram showing processing activities

20.9.3 Exercise 3: Fairness Audit and Remediation

Scenario: Lending institution discovers potential bias in credit scoring model after customer complaints.

Model Details:

- Random Forest classifier for loan approval
- Overall accuracy: 88%
- Disparate impact ratio (race): 0.68
- False negative rate varies by demographic group
- Model has been in production for 2 years

Tasks:

1. Calculate comprehensive fairness metrics (demographic parity, equal opportunity, equalized odds)
2. Identify source of bias (data, features, or algorithm)
3. Implement three bias mitigation techniques
4. Compare fairness-accuracy tradeoffs
5. Design ongoing fairness monitoring system
6. Create remediation plan for affected customers
7. Document findings for regulatory reporting

Deliverables:

- Fairness audit report with all metrics
- Bias mitigation code (3 approaches)
- Performance comparison matrix
- Fairness monitoring dashboard spec
- Customer remediation procedure
- Executive summary for board presentation

20.9.4 Exercise 4: Multi-Stage Approval Workflow Implementation

Scenario: Financial services firm implementing governance for 50+ ML models across organization.

Requirements:

- Three risk tiers: Low, Medium, High
- Different approval requirements per tier
- Automated controls before human approval
- Audit trail for all decisions
- SLA: 5 days for low risk, 15 days for medium, 30 days for high

Tasks:

1. Design approval workflow for each risk tier
2. Implement automated control checks
3. Create approval tracking system (database schema + code)
4. Build notification system for pending approvals
5. Implement SLA monitoring and escalation
6. Design approval delegation mechanism
7. Create compliance reporting dashboard

Deliverables:

- Workflow diagrams for all three tiers
- Automated control check code (Python)
- Database schema for approval tracking
- Approval workflow engine implementation
- SLA dashboard mockup
- User guide for approvers

20.9.5 Exercise 5: Model Risk Assessment

Scenario: Startup deploying first ML model to production. Board requires comprehensive risk assessment.

Model Context:

- Fraud detection for payment transactions
- Real-time decision-making ($< 100\text{ms}$ latency)
- Processes 1M transactions/day
- False positives block legitimate transactions
- False negatives result in fraud losses
- Public API exposed to merchants

Tasks:

1. Identify all risk categories (performance, security, operational, etc.)
2. Score each risk (Impact x Likelihood)
3. Prioritize top 5 risks
4. Design mitigation strategies for critical risks
5. Estimate residual risk after mitigations
6. Create risk register
7. Present findings to executive stakeholders

Deliverables:

- Risk assessment spreadsheet with all factors
- Risk heat map visualization
- Mitigation plan with timelines and owners
- Residual risk analysis
- Risk register (living document)
- Executive presentation (10 slides max)

20.10 Summary

Model governance and compliance are critical for responsible AI deployment. This chapter covered:

1. Governance Framework:

- Role-based governance structure
- Risk-based policies (4 tiers)
- Model inventory and lifecycle tracking
- Approval workflows and controls

2. Regulatory Compliance:

- GDPR requirements (explanation, deletion, portability, DPIA)
- CCPA requirements (notice, deletion, opt-out)
- Privacy by design principles
- Compliance reporting

3. Documentation and Audit:

- Model Cards for transparency
- Immutable audit trails
- Blockchain-like event hashing
- Compliance reports

4. Fairness and Bias:

- Fairness metrics (demographic parity, equal opportunity, etc.)
- Bias detection methods
- Mitigation strategies
- Ongoing fairness monitoring

5. Risk Assessment:

- Multi-dimensional risk evaluation
- Impact and likelihood scoring
- Risk categorization and prioritization
- Mitigation planning

6. Approval Workflows:

- Multi-stage approval gates

- Automated control checks
- Role-based approvers
- Audit trail generation

Key Takeaways:

1. **Governance is Risk-Based:** Higher-risk models require more rigorous controls
2. **Automate Where Possible:** Automated controls reduce human error and speed approvals
3. **Document Everything:** Comprehensive documentation protects organizations legally
4. **Fairness is Not Optional:** Bias testing must be part of every ML workflow
5. **Compliance is Continuous:** Ongoing monitoring and auditing required
6. **Culture Matters:** Governance succeeds when embedded in organizational culture
7. **Balance Control and Innovation:** Governance should enable, not block, ML development

Effective model governance balances innovation with responsibility, ensuring ML systems deliver value while protecting individuals and organizations from harm. As AI regulations evolve globally (EU AI Act, US AI Bill of Rights), robust governance frameworks will become competitive advantages, enabling organizations to deploy ML systems confidently and ethically.

20.10.1 Comprehensive Fairness Testing Implementation

```

1  #!/usr/bin/env python3
2  """
3  Production-Ready Fairness Testing Framework
4  Implements multiple fairness metrics with detailed reporting
5  """
6
7  import numpy as np
8  import pandas as pd
9  from sklearn.metrics import confusion_matrix, roc_auc_score
10 from typing import Dict, List, Tuple
11 import matplotlib.pyplot as plt
12 import seaborn as sns
13
14 class FairnessTestSuite:
15     """Comprehensive fairness testing for production models"""
16
17     def __init__(self, model_name: str):
18         self.model_name = model_name
19         self.test_results = {}
20
21     def run_full_fairness_audit(
22         self,

```

```

23     y_true: np.ndarray,
24     y_pred: np.ndarray,
25     y_prob: np.ndarray,
26     sensitive_features: pd.DataFrame,
27     favorable_label: int = 1
28 ) -> Dict:
29     """Run comprehensive fairness audit"""
30
31     results = {
32         "model_name": self.model_name,
33         "test_date": pd.Timestamp.now().isoformat(),
34         "overall_metrics": self._calculate_overall_metrics(y_true, y_pred, y_prob)
35     },
36     {
37         "group_metrics": {},
38         "fairness_violations": []
39     }
40
41     # Test each sensitive attribute
42     for attr in sensitive_features.columns:
43         attr_results = self._test_attribute_fairness(
44             y_true, y_pred, y_prob,
45             sensitive_features[attr],
46             attr, favorable_label
47         )
48         results["group_metrics"][attr] = attr_results
49
50     # Check for violations
51     violations = self._check_fairness_violations(attr_results, attr)
52     if violations:
53         results["fairness_violations"].extend(violations)
54
55     # Intersectional analysis
56     if len(sensitive_features.columns) >= 2:
57         intersectional = self._intersectional_analysis(
58             y_true, y_pred,
59             sensitive_features
60         )
61         results["intersectional_metrics"] = intersectional
62
63     self.test_results = results
64     return results
65
66 def _calculate_overall_metrics(
67     self,
68     y_true: np.ndarray,
69     y_pred: np.ndarray,
70     y_prob: np.ndarray
71 ) -> Dict:
72     """Calculate overall model performance"""
73
74     tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
75
76     return {
77         "accuracy": (tp + tn) / (tp + tn + fp + fn),
78         "precision": tp / (tp + fp) if (tp + fp) > 0 else 0,
79         "recall": tp / (tp + fn) if (tp + fn) > 0 else 0,
80         "specificity": tn / (tn + fp) if (tn + fp) > 0 else 0,
81         "f1_score": 2 * tp / (2 * tp + fp + fn) if (2 * tp + fp + fn) > 0 else 0,

```

```

80         "auc_roc": roc_auc_score(y_true, y_prob),
81         "selection_rate": np.mean(y_pred),
82         "base_rate": np.mean(y_true)
83     }
84
85     def _test_attribute_fairness(
86         self,
87         y_true: np.ndarray,
88         y_pred: np.ndarray,
89         y_prob: np.ndarray,
90         sensitive_attr: pd.Series,
91         attr_name: str,
92         favorable_label: int
93     ) -> Dict:
94         """Test fairness for a single attribute"""
95
96         groups = {}
97         reference_group = None
98         max_selection_rate = 0
99
100         # Calculate metrics for each group
101         for group_value in sensitive_attr.unique():
102             mask = sensitive_attr == group_value
103             group_true = y_true[mask]
104             group_pred = y_pred[mask]
105             group_prob = y_prob[mask]
106
107             if len(group_true) == 0:
108                 continue
109
110             tn, fp, fn, tp = confusion_matrix(group_true, group_pred).ravel()
111
112             metrics = {
113                 "group_size": int(mask.sum()),
114                 "group_percentage": float(mask.mean() * 100),
115                 "accuracy": (tp + tn) / (tp + tn + fp + fn),
116                 "precision": tp / (tp + fp) if (tp + fp) > 0 else 0,
117                 "recall_tpr": tp / (tp + fn) if (tp + fn) > 0 else 0,
118                 "fpr": fp / (fp + tn) if (fp + tn) > 0 else 0,
119                 "fnr": fn / (fn + tp) if (fn + tp) > 0 else 0,
120                 "selection_rate": float(np.mean(group_pred)),
121                 "base_rate": float(np.mean(group_true)),
122                 "true_positives": int(tp),
123                 "false_positives": int(fp),
124                 "true_negatives": int(tn),
125                 "false_negatives": int(fn)
126             }
127
128             groups[str(group_value)] = metrics
129
130             # Track reference group (largest selection rate)
131             if metrics["selection_rate"] > max_selection_rate:
132                 max_selection_rate = metrics["selection_rate"]
133                 reference_group = str(group_value)
134
135         # Calculate relative fairness metrics
136         if reference_group:
137             for group_name, group_metrics in groups.items():

```

```

138         ref_metrics = groups[reference_group]
139
140         # Disparate impact ratio
141         if ref_metrics["selection_rate"] > 0:
142             group_metrics["disparate_impact_ratio"] = \
143                 group_metrics["selection_rate"] / ref_metrics["selection_rate"]
144         ]
145         else:
146             group_metrics["disparate_impact_ratio"] = None
147
148         # TPR ratio (equal opportunity)
149         if ref_metrics["recall_tpr"] > 0:
150             group_metrics["equal_opportunity_ratio"] = \
151                 group_metrics["recall_tpr"] / ref_metrics["recall_tpr"]
152         else:
153             group_metrics["equal_opportunity_ratio"] = None
154
155         # Average odds difference
156         group_metrics["avg_odds_diff"] = \
157             (group_metrics["recall_tpr"] - ref_metrics["recall_tpr"] +
158              group_metrics["fpr"] - ref_metrics["fpr"]) / 2
159
160         return {
161             "attribute": attr_name,
162             "reference_group": reference_group,
163             "groups": groups,
164             "max_disparate_impact_violation": self._max_di_violation(groups),
165             "max_tpr_difference": self._max_tpr_difference(groups),
166             "max_fpr_difference": self._max_fpr_difference(groups)
167         }
168
169     def _max_di_violation(self, groups: Dict) -> float:
170         """Calculate maximum disparate impact violation"""
171         di_ratios = [g.get("disparate_impact_ratio", 1.0)
172                      for g in groups.values()]
173         if g.get("disparate_impact_ratio") is not None]
174
175         if not di_ratios:
176             return 0.0
177
178         # Return how far below 0.8 the worst group is
179         min_ratio = min(di_ratios)
180         return max(0, 0.8 - min_ratio)
181
182     def _max_tpr_difference(self, groups: Dict) -> float:
183         """Calculate maximum TPR difference between groups"""
184         tprs = [g["recall_tpr"] for g in groups.values()]
185         return max(tprs) - min(tprs) if tprs else 0.0
186
187     def _max_fpr_difference(self, groups: Dict) -> float:
188         """Calculate maximum FPR difference between groups"""
189         fprs = [g["fpr"] for g in groups.values()]
190         return max(fprs) - min(fprs) if fprs else 0.0
191
192     def _check_fairness_violations(
193         self,
194         attr_results: Dict,
195         attr_name: str

```

```

195 ) -> List[Dict]:
196     """Check for fairness metric violations"""
197
198     violations = []
199
200     # Check disparate impact (80% rule)
201     if attr_results["max_disparate_impact_violation"] > 0:
202         violations.append({
203             "attribute": attr_name,
204             "violation_type": "disparate_impact",
205             "severity": "high",
206             "description": f"Disparate impact ratio below 0.8 threshold",
207             "max_violation": attr_results["max_disparate_impact_violation"]
208         })
209
210     # Check equal opportunity (TPR difference)
211     if attr_results["max_tpr_difference"] > 0.1:
212         violations.append({
213             "attribute": attr_name,
214             "violation_type": "equal_opportunity",
215             "severity": "medium",
216             "description": f"TPR differs by {attr_results['max_tpr_difference']:.3
f} across groups",
217             "max_violation": attr_results["max_tpr_difference"]
218         })
219
220     # Check equalized odds (FPR difference)
221     if attr_results["max_fpr_difference"] > 0.1:
222         violations.append({
223             "attribute": attr_name,
224             "violation_type": "equalized_odds",
225             "severity": "medium",
226             "description": f"FPR differs by {attr_results['max_fpr_difference']:.3
f} across groups",
227             "max_violation": attr_results["max_fpr_difference"]
228         })
229
230     return violations
231
232 def _intersectional_analysis(
233     self,
234     y_true: np.ndarray,
235     y_pred: np.ndarray,
236     sensitive_features: pd.DataFrame
237 ) -> Dict:
238     """Analyze fairness across intersections of attributes"""
239
240     # Create intersection groups
241     intersections = sensitive_features.apply(
242         lambda row: '_'.join(row.astype(str)),
243         axis=1
244     )
245
246     intersection_metrics = {}
247
248     for intersection in intersections.unique():
249         mask = intersections == intersection
250         if mask.sum() < 30: # Skip small groups

```



```

251         continue
252
253         group_true = y_true[mask]
254         group_pred = y_pred[mask]
255
256         intersection_metrics[intersection] = {
257             "size": int(mask.sum()),
258             "accuracy": float(np.mean(group_pred == group_true)),
259             "selection_rate": float(np.mean(group_pred)),
260             "base_rate": float(np.mean(group_true))
261         }
262
263     return intersection_metrics
264
265     def generate_fairness_report_html(self, output_file: str):
266         """Generate HTML fairness report"""
267
268         html = f"""
269 <!DOCTYPE html>
270 <html>
271 <head>
272     <title>Fairness Audit Report: {self.model_name}</title>
273     <style>
274         body {{ font-family: Arial, sans-serif; margin: 40px; }}
275         h1 {{ color: #d32f2f; }}
276         h2 {{ color: #1976d2; margin-top: 30px; }}
277         table {{ border-collapse: collapse; width: 100%; margin: 20px 0; }}
278         th, td {{ border: 1px solid #ddd; padding: 12px; text-align: left; }}
279         th {{ background-color: #1976d2; color: white; }}
280         .violation {{ background-color: #ffebee; padding: 15px; margin: 10px 0; border-
281         -left: 4px solid #d32f2f; }}
282         .pass {{ background-color: #e8f5e9; padding: 15px; margin: 10px 0; border-left
283         : 4px solid #4caf50; }}
284         .metric {{ display: inline-block; margin: 10px 20px 10px 0; }}
285         .metric-value {{ font-size: 24px; font-weight: bold; color: #1976d2; }}
286         .metric-label {{ font-size: 12px; color: #666; }}
287     </style>
288 </head>
289 <body>
290     <h1>Fairness Audit Report</h1>
291     <p><strong>Model:</strong> {self.model_name}</p>
292     <p><strong>Date:</strong> {self.test_results.get('test_date', 'N/A')}</p>
293
294     <h2>Overall Performance</h2>
295     <div>
296
297         overall = self.test_results.get("overall_metrics", {})
298         for metric, value in overall.items():
299             html += f"""
300             <div class="metric">
301                 <div class="metric-value">{value:.3f}</div>
302                 <div class="metric-label">{metric.replace('_', ' ').title()}</div>
303             </div>
304
305             html += """
306             </div>

```

```

307     <h2>Fairness Violations</h2>
308     """
309
310     violations = self.test_results.get("fairness_violations", [])
311     if violations:
312         for violation in violations:
313             html += f"""
314             <div class="violation">
315                 <strong>{violation['violation_type'].replace('_', ' ').title()}</strong>
316                 ({violation['severity']} severity)<br>
317                 Attribute: {violation['attribute']}<br>
318                 {violation['description']}
319             </div>
320             """
321
322     else:
323         html += '<div class="pass">No fairness violations detected!</div>'
324
325     html += """
326     <h2>Group Metrics</h2>
327     """
328
329     for attr, attr_data in self.test_results.get("group_metrics", {}).items():
330         html += f"""
331         <h3>{attr}</h3>
332         <table>
333             <tr>
334                 <th>Group</th>
335                 <th>Size</th>
336                 <th>Selection Rate</th>
337                 <th>Accuracy</th>
338                 <th>TPR</th>
339                 <th>FPR</th>
340                 <th>DI Ratio</th>
341             </tr>
342             """
343         for group_name, group_data in attr_data.get("groups", {}).items():
344             di_ratio = group_data.get("disparate_impact_ratio")
345             di_str = f"{di_ratio:.3f}" if di_ratio is not None else "N/A"
346
347             html += f"""
348             <tr>
349                 <td>{group_name}</td>
350                 <td>{group_data['group_size']}</td>
351                 <td>{group_data['selection_rate']:.3f}</td>
352                 <td>{group_data['accuracy']:.3f}</td>
353                 <td>{group_data['recall_tpr']:.3f}</td>
354                 <td>{group_data['fpr']:.3f}</td>
355                 <td>{di_str}</td>
356             </tr>
357             """
358
359         html += """
360         </table>
361         """
362
363     html += """
364 </body>

```

```

365 </html>
366 """
367
368     with open(output_file, 'w') as f:
369         f.write(html)
370
371     print(f"Fairness report generated: {output_file}")
372
373 # Example usage
374 if __name__ == "__main__":
375     # Generate synthetic test data
376     np.random.seed(42)
377     n_samples = 5000
378
379     # Create synthetic dataset with bias
380     gender = np.random.choice(['M', 'F'], n_samples, p=[0.6, 0.4])
381     race = np.random.choice(['White', 'Black', 'Hispanic', 'Asian'], n_samples)
382     age_group = np.random.choice(['18-30', '31-50', '51+'], n_samples)
383
384     # Ground truth with some correlation to demographics (simulating historical bias)
385     y_true = np.random.binomial(1, 0.3, n_samples)
386
387     # Predictions with introduced bias
388     y_pred = y_true.copy()
389
390     # Introduce bias: lower approval rate for certain groups
391     female_mask = gender == 'F'
392     y_pred[female_mask & (y_true == 1)] = np.random.binomial(
393         1, 0.7, (female_mask & (y_true == 1)).sum()
394     )
395
396     y_prob = np.random.uniform(0, 1, n_samples)
397     y_prob[y_pred == 1] = np.random.uniform(0.5, 1.0, (y_pred == 1).sum())
398
399     # Create DataFrame with sensitive attributes
400     sensitive_features = pd.DataFrame({
401         'gender': gender,
402         'race': race,
403         'age_group': age_group
404     })
405
406     # Run fairness audit
407     fairness_tester = FairnessTestSuite(model_name="Credit Approval Model v2.0")
408
409     results = fairness_tester.run_full_fairness_audit(
410         y_true=y_true,
411         y_pred=y_pred,
412         y_prob=y_prob,
413         sensitive_features=sensitive_features[['gender', 'race']],
414         favorable_label=1
415     )
416
417     print(f"Violations found: {len(results['fairness_violations'])}")
418     for violation in results['fairness_violations']:
419         print(f"  - {violation['violation_type']}: {violation['description']}")
420
421     # Generate HTML report
422     fairness_tester.generate_fairness_report_html("fairness_audit_report.html")

```

Listing 20.7: Production Fairness Testing Framework

20.10.2 Advanced Risk Assessment with Quantitative Scoring

```

1  #!/usr/bin/env python3
2  """
3  Advanced Risk Assessment with Monte Carlo Simulation
4  Quantitative risk scoring for ML models
5  """
6
7  import numpy as np
8  import pandas as pd
9  from typing import Dict, List
10 from dataclasses import dataclass
11 import json
12
13 @dataclass
14 class RiskDistribution:
15     """Probability distribution for risk parameter"""
16     name: str
17     distribution_type: str # "uniform", "normal", "triangular"
18     params: Dict # Distribution parameters
19
20 class QuantitativeRiskAssessor:
21     """Advanced risk assessment with probabilistic analysis"""
22
23     def __init__(self, model_id: str):
24         self.model_id = model_id
25         self.risk_factors = []
26
27     def add_risk_factor(
28         self,
29         name: str,
30         impact_dist: RiskDistribution,
31         likelihood_dist: RiskDistribution,
32         current_controls: float = 0.0
33     ):
34         """Add risk factor with probability distributions"""
35
36         self.risk_factors.append({
37             "name": name,
38             "impact_dist": impact_dist,
39             "likelihood_dist": likelihood_dist,
40             "current_controls": current_controls
41         })
42
43     def monte_carlo_simulation(self, n_simulations: int = 10000) -> Dict:
44         """Run Monte Carlo simulation for risk assessment"""
45
46         simulation_results = []
47
48         for _ in range(n_simulations):
49             total_risk = 0
50
51             for risk in self.risk_factors:

```

```

52         # Sample impact
53         impact = self._sample_distribution(risk["impact_dist"])
54
55         # Sample likelihood
56         likelihood = self._sample_distribution(risk["likelihood_dist"])
57
58         # Apply control effectiveness
59         control_reduction = risk["current_controls"]
60         adjusted_likelihood = likelihood * (1 - control_reduction)
61
62         # Calculate risk score
63         risk_score = impact * adjusted_likelihood
64
65         total_risk += risk_score
66
67         simulation_results.append(total_risk)
68
69     simulation_results = np.array(simulation_results)
70
71     return {
72         "mean_risk": float(np.mean(simulation_results)),
73         "median_risk": float(np.median(simulation_results)),
74         "std_risk": float(np.std(simulation_results)),
75         "percentile_95": float(np.percentile(simulation_results, 95)),
76         "percentile_99": float(np.percentile(simulation_results, 99)),
77         "min_risk": float(np.min(simulation_results)),
78         "max_risk": float(np.max(simulation_results)),
79         "probability_high_risk": float(np.mean(simulation_results > 15)),
80         "probability_critical_risk": float(np.mean(simulation_results > 20))
81     }
82
83     def _sample_distribution(self, dist: RiskDistribution) -> float:
84         """Sample from probability distribution"""
85
86         if dist.distribution_type == "uniform":
87             return np.random.uniform(
88                 dist.params["low"],
89                 dist.params["high"]
90             )
91         elif dist.distribution_type == "normal":
92             return np.random.normal(
93                 dist.params["mean"],
94                 dist.params["std"]
95             )
96         elif dist.distribution_type == "triangular":
97             return np.random.triangular(
98                 dist.params["low"],
99                 dist.params["mode"],
100                 dist.params["high"]
101             )
102         else:
103             raise ValueError(f"Unknown distribution: {dist.distribution_type}")
104
105     def value_at_risk(self, confidence_level: float = 0.95) -> Dict:
106         """Calculate Value at Risk (VaR) metrics"""
107
108         simulation = self.monte_carlo_simulation(n_simulations=10000)
109 
```

```

110     return {
111         "var_95": simulation["percentile_95"],
112         "var_99": simulation["percentile_99"],
113         "expected_risk": simulation["mean_risk"],
114         "unexpected_risk_95": simulation["percentile_95"] - simulation["mean_risk"]
115     },
116     [
117         "unexpected_risk_99": simulation["percentile_99"] - simulation["mean_risk"]
118     ]
119 }
120
121 def scenario_analysis(self) -> Dict:
122     """Analyze best-case, worst-case, and expected scenarios"""
123
124     scenarios = {
125         "best_case": 0,
126         "worst_case": 0,
127         "expected_case": 0
128     }
129
130     for risk in self.risk_factors:
131         # Best case: minimum impact and likelihood
132         impact_min = risk["impact_dist"].params.get("low", 1)
133         likelihood_min = risk["likelihood_dist"].params.get("low", 0.1)
134         scenarios["best_case"] += impact_min * likelihood_min
135
136         # Worst case: maximum impact and likelihood
137         impact_max = risk["impact_dist"].params.get("high", 5)
138         likelihood_max = risk["likelihood_dist"].params.get("high", 1.0)
139         scenarios["worst_case"] += impact_max * likelihood_max * \
140             (1 - risk["current_controls"])
141
142         # Expected case: mean impact and likelihood
143         impact_mean = risk["impact_dist"].params.get("mean",
144             (risk["impact_dist"].params.get("low", 1) +
145             risk["impact_dist"].params.get("high", 5)) / 2)
146         likelihood_mean = risk["likelihood_dist"].params.get("mean",
147             (risk["likelihood_dist"].params.get("low", 0.1) +
148             risk["likelihood_dist"].params.get("high", 1.0)) / 2)
149         scenarios["expected_case"] += impact_mean * likelihood_mean * \
150             (1 - risk["current_controls"])
151
152     return scenarios
153
154 def sensitivity_analysis(self) -> pd.DataFrame:
155     """Analyze sensitivity of risk to individual factors"""
156
157     baseline_simulation = self.monte_carlo_simulation(n_simulations=5000)
158     baseline_risk = baseline_simulation["mean_risk"]
159
160     sensitivity = []
161
162     for i, risk in enumerate(self.risk_factors):
163         # Increase impact by 20%
164         original_params = risk["impact_dist"].params.copy()
165
166         if "mean" in original_params:
167             risk["impact_dist"].params["mean"] *= 1.2
168         else:

```

```

166         risk["impact_dist"].params["high"] *= 1.2
167
168         increased_simulation = self.monte_carlo_simulation(n_simulations=5000)
169         impact_sensitivity = (increased_simulation["mean_risk"] - baseline_risk) /
baseline_risk
170
171         # Restore original parameters
172         risk["impact_dist"].params = original_params
173
174         # Increase likelihood by 20%
175         original_params = risk["likelihood_dist"].params.copy()
176
177         if "mean" in original_params:
178             risk["likelihood_dist"].params["mean"] = min(
179                 risk["likelihood_dist"].params["mean"] * 1.2, 1.0
180             )
181         else:
182             risk["likelihood_dist"].params["high"] = min(
183                 risk["likelihood_dist"].params["high"] * 1.2, 1.0
184             )
185
186         increased_simulation = self.monte_carlo_simulation(n_simulations=5000)
187         likelihood_sensitivity = (increased_simulation["mean_risk"] -
baseline_risk) / baseline_risk
188
189         # Restore original parameters
190         risk["likelihood_dist"].params = original_params
191
192         sensitivity.append({
193             "risk_factor": risk["name"],
194             "impact_sensitivity": impact_sensitivity,
195             "likelihood_sensitivity": likelihood_sensitivity,
196             "total_sensitivity": impact_sensitivity + likelihood_sensitivity
197         })
198
199         return pd.DataFrame(sensitivity).sort_values("total_sensitivity", ascending=
False)
200
201     def generate_risk_dashboard_data(self, output_file: str):
202         """Generate data for risk dashboard"""
203
204         monte_carlo = self.monte_carlo_simulation(n_simulations=10000)
205         var = self.value_at_risk()
206         scenarios = self.scenario_analysis()
207         sensitivity = self.sensitivity_analysis()
208
209         dashboard_data = {
210             "model_id": self.model_id,
211             "assessment_date": pd.Timestamp.now().isoformat(),
212             "summary": {
213                 "expected_risk": monte_carlo["mean_risk"],
214                 "risk_volatility": monte_carlo["std_risk"],
215                 "value_at_risk_95": var["var_95"],
216                 "probability_high_risk": monte_carlo["probability_high_risk"],
217                 "risk_rating": self._get_risk_rating(monte_carlo["mean_risk"])
218             },
219             "monte_carlo_results": monte_carlo,
220             "value_at_risk": var,

```

```

221         "scenario_analysis": scenarios,
222         "sensitivity_analysis": sensitivity.to_dict(orient="records"),
223         "risk_factors": [
224             {
225                 "name": r["name"],
226                 "current_controls": r["current_controls"]
227             }
228             for r in self.risk_factors
229         ]
230     }
231
232     with open(output_file, 'w') as f:
233         json.dump/dashboard_data, f, indent=2)
234
235     print(f"Risk dashboard data generated: {output_file}")
236     return dashboard_data
237
238     def _get_risk_rating(self, mean_risk: float) -> str:
239         """Get risk rating based on mean risk score"""
240
241         if mean_risk >= 20:
242             return "CRITICAL"
243         elif mean_risk >= 12:
244             return "HIGH"
245         elif mean_risk >= 6:
246             return "MEDIUM"
247         else:
248             return "LOW"
249
250     # Example usage
251     if __name__ == "__main__":
252         # Create risk assessor
253         assessor = QuantitativeRiskAssessor(model_id="fraud-detection-v1")
254
255         # Add risk factors with distributions
256         assessor.add_risk_factor(
257             name="Model Performance Degradation",
258             impact_dist=RiskDistribution(
259                 name="impact",
260                 distribution_type="triangular",
261                 params={"low": 2, "mode": 4, "high": 5}
262             ),
263             likelihood_dist=RiskDistribution(
264                 name="likelihood",
265                 distribution_type="triangular",
266                 params={"low": 0.3, "mode": 0.5, "high": 0.8}
267             ),
268             current_controls=0.2 # 20% risk reduction from monitoring
269         )
270
271         assessor.add_risk_factor(
272             name="Adversarial Attack",
273             impact_dist=RiskDistribution(
274                 name="impact",
275                 distribution_type="triangular",
276                 params={"low": 3, "mode": 4, "high": 5}
277             ),
278             likelihood_dist=RiskDistribution(

```



```

279         name="likelihood",
280         distribution_type="triangular",
281         params={"low": 0.1, "mode": 0.3, "high": 0.6}
282     ),
283     current_controls=0.4 # 40% risk reduction from security controls
284 )
285
286 assessor.add_risk_factor(
287     name="Regulatory Compliance Violation",
288     impact_dist=RiskDistribution(
289         name="impact",
290         distribution_type="triangular",
291         params={"low": 4, "mode": 5, "high": 5}
292     ),
293     likelihood_dist=RiskDistribution(
294         name="likelihood",
295         distribution_type="triangular",
296         params={"low": 0.05, "mode": 0.15, "high": 0.3}
297     ),
298     current_controls=0.6 # 60% risk reduction from compliance program
299 )
300
301 # Run analyses
302 monte_carlo_results = assessor.monte_carlo_simulation(n_simulations=10000)
303 print(f"Expected Risk: {monte_carlo_results['mean_risk']:.2f}")
304 print(f"95th Percentile Risk: {monte_carlo_results['percentile_95']:.2f}")
305 print(f"Probability of High Risk: {monte_carlo_results['probability_high_risk']:.2%}")
306
307 var_results = assessor.value_at_risk()
308 print(f"\nValue at Risk (95%): {var_results['var_95']:.2f}")
309
310 scenarios = assessor.scenario_analysis()
311 print(f"\nScenario Analysis:")
312 print(f"  Best Case: {scenarios['best_case']:.2f}")
313 print(f"  Expected: {scenarios['expected_case']:.2f}")
314 print(f"  Worst Case: {scenarios['worst_case']:.2f}")
315
316 sensitivity = assessor.sensitivity_analysis()
317 print(f"\nTop Risk Drivers:")
318 print(sensitivity.head(3).to_string(index=False))
319
320 # Generate dashboard data
321 assessor.generate_risk_dashboard_data("risk_dashboard.json")

```

Listing 20.8: Quantitative Risk Scoring System

Chapter 21

Security in ML Systems

21.1 Learning Objectives

After completing this chapter, readers will be able to:

- Understand ML-specific security threats and attack vectors
- Implement adversarial attack detection and defense mechanisms
- Protect models from stealing and inversion attacks
- Secure data handling with encryption and access controls
- Design secure ML APIs with authentication and rate limiting
- Apply privacy-preserving ML techniques (differential privacy, federated learning)
- Conduct security scanning and vulnerability assessments
- Build defense-in-depth security architectures for ML systems

21.2 Introduction

ML systems face unique security challenges beyond traditional software security. Attackers can manipulate training data, exploit model vulnerabilities, steal intellectual property, or extract private information from models.

ML Security Threat Landscape:

- **Data Poisoning:** Attackers inject malicious data into training sets
- **Adversarial Examples:** Carefully crafted inputs cause misclassification
- **Model Stealing:** Extracting model parameters through query access
- **Model Inversion:** Reconstructing training data from model outputs
- **Membership Inference:** Determining if data was in training set
- **Backdoor Attacks:** Embedding hidden behaviors triggered by specific inputs

- **Privacy Leakage:** Models reveal sensitive information about training data

Security Requirements for ML Systems:

1. **Confidentiality:** Protect training data, model parameters, and predictions
2. **Integrity:** Ensure models behave as intended without manipulation
3. **Availability:** Prevent denial-of-service and resource exhaustion
4. **Accountability:** Audit and trace all model interactions
5. **Privacy:** Prevent leakage of sensitive information

21.3 Adversarial Attack Detection and Defense

Adversarial attacks exploit the continuous, high-dimensional nature of ML input spaces to create inputs that fool models while appearing normal to humans.

21.3.1 Adversarial Attack Types

Common Attack Methods:

- **FGSM (Fast Gradient Sign Method):** Single-step attack using gradient sign
- **PGD (Projected Gradient Descent):** Multi-step iterative attack
- **C&W (Carlini-Wagner):** Optimization-based attack minimizing perturbation
- **DeepFool:** Finds minimal perturbation to decision boundary
- **One-Pixel Attack:** Modifies single pixel using differential evolution

21.3.2 Adversarial Defense Implementation

```

1  #!/usr/bin/env python3
2  """
3  Adversarial Attack Detection and Defense
4  Comprehensive framework for securing ML models against adversarial attacks
5  """
6
7  import numpy as np
8  import torch
9  import torch.nn as nn
10 import torch.nn.functional as F
11 from typing import Tuple, Dict, List
12 from dataclasses import dataclass
13
14 @dataclass
15 class AdversarialExample:
16     """Adversarial example with metadata"""
17     original_input: np.ndarray
18     perturbed_input: np.ndarray
19     original_prediction: int

```

```

20     adversarial_prediction: int
21     perturbation: np.ndarray
22     perturbation_norm: float
23     attack_method: str
24
25 class AdversarialAttackGenerator:
26     """Generate adversarial examples for testing"""
27
28     def __init__(self, model: nn.Module, device: str = "cpu"):
29         self.model = model
30         self.device = device
31         self.model.eval()
32
33     def fgsm_attack(
34         self,
35         image: torch.Tensor,
36         label: torch.Tensor,
37         epsilon: float = 0.3
38     ) -> torch.Tensor:
39         """Fast Gradient Sign Method attack"""
40
41         image.requires_grad = True
42
43         # Forward pass
44         output = self.model(image)
45         loss = F.cross_entropy(output, label)
46
47         # Backward pass
48         self.model.zero_grad()
49         loss.backward()
50
51         # Generate adversarial example
52         data_grad = image.grad.data
53         perturbed_image = image + epsilon * data_grad.sign()
54         perturbed_image = torch.clamp(perturbed_image, 0, 1)
55
56         return perturbed_image
57
58     def pgd_attack(
59         self,
60         image: torch.Tensor,
61         label: torch.Tensor,
62         epsilon: float = 0.3,
63         alpha: float = 0.01,
64         num_iter: int = 40
65     ) -> torch.Tensor:
66         """Projected Gradient Descent attack"""
67
68         perturbed_image = image.clone().detach()
69
70         for i in range(num_iter):
71             perturbed_image.requires_grad = True
72
73             output = self.model(perturbed_image)
74             loss = F.cross_entropy(output, label)
75
76             self.model.zero_grad()
77             loss.backward()

```

```

78         # Update perturbation
79         data_grad = perturbed_image.grad.data
80         perturbed_image = perturbed_image.detach() + alpha * data_grad.sign()
81
82         # Project back to epsilon ball
83         perturbation = torch.clamp(perturbed_image - image, -epsilon, epsilon)
84         perturbed_image = torch.clamp(image + perturbation, 0, 1).detach()
85
86
87     return perturbed_image
88
89     def generate_adversarial_example(
90         self,
91         image: np.ndarray,
92         true_label: int,
93         attack_type: str = "fgsm",
94         epsilon: float = 0.3
95     ) -> AdversarialExample:
96         """Generate adversarial example with metadata"""
97
98         # Convert to tensor
99         image_tensor = torch.FloatTensor(image).unsqueeze(0).to(self.device)
100         label_tensor = torch.LongTensor([true_label]).to(self.device)
101
102         # Original prediction
103         with torch.no_grad():
104             original_output = self.model(image_tensor)
105             original_pred = original_output.argmax(1).item()
106
107         # Generate adversarial example
108         if attack_type == "fgsm":
109             perturbed_tensor = self.fgsm_attack(image_tensor, label_tensor, epsilon)
110         elif attack_type == "pgd":
111             perturbed_tensor = self.pgd_attack(image_tensor, label_tensor, epsilon)
112         else:
113             raise ValueError(f"Unknown attack type: {attack_type}")
114
115         # Adversarial prediction
116         with torch.no_grad():
117             adv_output = self.model(perturbed_tensor)
118             adv_pred = adv_output.argmax(1).item()
119
120         # Calculate perturbation
121         perturbation = (perturbed_tensor - image_tensor).cpu().numpy()
122         perturbation_norm = np.linalg.norm(perturbation)
123
124         return AdversarialExample(
125             original_input=image,
126             perturbed_input=perturbed_tensor.cpu().numpy().squeeze(),
127             original_prediction=original_pred,
128             adversarial_prediction=adv_pred,
129             perturbation=perturbation.squeeze(),
130             perturbation_norm=perturbation_norm,
131             attack_method=attack_type
132         )
133
134     class AdversarialDetector:
135         """Detect adversarial examples at inference time"""

```

```

136
137 def __init__(self, model: nn.Module, device: str = "cpu"):
138     self.model = model
139     self.device = device
140     self.detection_methods = []
141
142 def add_detection_method(self, method_name: str, threshold: float):
143     """Add detection method with threshold"""
144     self.detection_methods.append({
145         "name": method_name,
146         "threshold": threshold
147     })
148
149 def detect_by_prediction_confidence(
150     self,
151     image: torch.Tensor,
152     confidence_threshold: float = 0.95
153 ) -> Tuple[bool, float]:
154     """Detect based on prediction confidence"""
155
156     with torch.no_grad():
157         output = self.model(image)
158         probabilities = F.softmax(output, dim=1)
159         max_prob = probabilities.max().item()
160
161     # Adversarial examples often have lower confidence
162     is_adversarial = max_prob < confidence_threshold
163
164     return is_adversarial, max_prob
165
166 def detect_by_input_gradient(
167     self,
168     image: torch.Tensor,
169     label: torch.Tensor,
170     gradient_threshold: float = 10.0
171 ) -> Tuple[bool, float]:
172     """Detect based on input gradient magnitude"""
173
174     image.requires_grad = True
175
176     output = self.model(image)
177     loss = F.cross_entropy(output, label)
178
179     self.model.zero_grad()
180     loss.backward()
181
182     gradient_norm = image.grad.data.norm().item()
183
184     # Adversarial examples often have larger gradients
185     is_adversarial = gradient_norm > gradient_threshold
186
187     return is_adversarial, gradient_norm
188
189 def detect_by_feature_squeezing(
190     self,
191     image: torch.Tensor,
192     squeeze_threshold: float = 0.1
193 ) -> Tuple[bool, float]:

```

```

194     """Detect using feature squeezing"""
195
196     # Bit-depth reduction (squeeze to fewer bits)
197     squeezed_image = torch.round(image * 4) / 4
198
199     # Compare predictions
200     with torch.no_grad():
201         original_output = self.model(image)
202         squeezed_output = self.model(squeezed_image)
203
204         original_pred = original_output.argmax(1)
205         squeezed_pred = squeezed_output.argmax(1)
206
207     # Calculate L1 difference in predictions
208     pred_diff = torch.abs(
209         F.softmax(original_output, dim=1) -
210         F.softmax(squeezed_output, dim=1)
211     ).sum().item()
212
213     is_adversarial = pred_diff > squeeze_threshold
214
215     return is_adversarial, pred_diff
216
217 def ensemble_detection(
218     self,
219     image: torch.Tensor,
220     label: torch.Tensor
221 ) -> Dict:
222     """Run multiple detection methods and aggregate"""
223
224     results = {
225         "is_adversarial": False,
226         "detection_scores": {},
227         "methods_triggered": 0
228     }
229
230     # Confidence-based detection
231     is_adv_conf, conf_score = self.detect_by_prediction_confidence(image)
232     results["detection_scores"]["confidence"] = conf_score
233     if is_adv_conf:
234         results["methods_triggered"] += 1
235
236     # Gradient-based detection
237     is_adv_grad, grad_score = self.detect_by_input_gradient(image, label)
238     results["detection_scores"]["gradient"] = grad_score
239     if is_adv_grad:
240         results["methods_triggered"] += 1
241
242     # Feature squeezing
243     is_adv_squeeze, squeeze_score = self.detect_by_feature_squeezing(image)
244     results["detection_scores"]["feature_squeezing"] = squeeze_score
245     if is_adv_squeeze:
246         results["methods_triggered"] += 1
247
248     # Mark as adversarial if 2+ methods trigger
249     results["is_adversarial"] = results["methods_triggered"] >= 2
250
251     return results

```



```

252 class AdversarialDefense:
253     """Implement defense mechanisms against adversarial attacks"""
254
255     def __init__(self, model: nn.Module, device: str = "cpu"):
256         self.model = model
257         self.device = device
258
259     def adversarial_training(
260         self,
261         train_loader,
262         optimizer,
263         epochs: int = 10,
264         epsilon: float = 0.3
265     ):
266         """Train model with adversarial examples"""
267
268         attacker = AdversarialAttackGenerator(self.model, self.device)
269
270         for epoch in range(epochs):
271             self.model.train()
272             total_loss = 0
273
274             for batch_idx, (data, target) in enumerate(train_loader):
275                 data, target = data.to(self.device), target.to(self.device)
276
277                 # Generate adversarial examples
278                 adv_data = attacker.fgsm_attack(data, target, epsilon)
279
280                 # Train on both clean and adversarial examples
281                 optimizer.zero_grad()
282
283                 # Clean loss
284                 clean_output = self.model(data)
285                 clean_loss = F.cross_entropy(clean_output, target)
286
287                 # Adversarial loss
288                 adv_output = self.model(adv_data)
289                 adv_loss = F.cross_entropy(adv_output, target)
290
291                 # Combined loss
292                 loss = 0.5 * clean_loss + 0.5 * adv_loss
293                 loss.backward()
294                 optimizer.step()
295
296                 total_loss += loss.item()
297
298             print(f"Epoch {epoch+1}/{epochs}, Loss: {total_loss/len(train_loader):.4f}")
299
300
301     def input_transformation(self, image: torch.Tensor) -> torch.Tensor:
302         """Apply input transformations to remove adversarial perturbations"""
303
304         # JPEG compression simulation
305         compressed = torch.round(image * 255) / 255
306
307         # Gaussian smoothing
308         kernel_size = 3

```

```

309         sigma = 1.0
310         smoothed = self._gaussian_blur(compressed, kernel_size, sigma)
311
312         # Bit-depth reduction
313         squeezed = torch.round(smoothed * 4) / 4
314
315         return squeezed
316
317     def _gaussian_blur(
318         self,
319         image: torch.Tensor,
320         kernel_size: int,
321         sigma: float
322     ) -> torch.Tensor:
323         """Apply Gaussian blur"""
324
325         # Create Gaussian kernel
326         kernel = self._gaussian_kernel(kernel_size, sigma)
327         kernel = kernel.to(image.device)
328
329         # Apply convolution
330         blurred = F.conv2d(
331             image,
332             kernel.unsqueeze(0).unsqueeze(0),
333             padding=kernel_size // 2
334         )
335
336         return blurred
337
338     def _gaussian_kernel(self, kernel_size: int, sigma: float) -> torch.Tensor:
339         """Generate Gaussian kernel"""
340
341         x = torch.arange(kernel_size).float() - kernel_size // 2
342         gauss = torch.exp(-x.pow(2) / (2 * sigma ** 2))
343         kernel = gauss.unsqueeze(0) * gauss.unsqueeze(1)
344         kernel = kernel / kernel.sum()
345
346         return kernel
347
348     def certified_defense(
349         self,
350         image: torch.Tensor,
351         num_samples: int = 100,
352         sigma: float = 0.1
353     ) -> Tuple[int, float]:
354         """Randomized smoothing for certified robustness"""
355
356         predictions = []
357
358         with torch.no_grad():
359             for _ in range(num_samples):
360                 # Add Gaussian noise
361                 noisy_image = image + torch.randn_like(image) * sigma
362                 noisy_image = torch.clamp(noisy_image, 0, 1)
363
364                 # Get prediction
365                 output = self.model(noisy_image)
366                 pred = output.argmax(1).item()

```

```

367         predictions.append(pred)
368
369         # Majority vote
370         from collections import Counter
371         vote_counts = Counter(predictions)
372         predicted_class = vote_counts.most_common(1)[0][0]
373         confidence = vote_counts[predicted_class] / num_samples
374
375         return predicted_class, confidence
376
377 # Example usage
378 if __name__ == "__main__":
379     # Create dummy model for demonstration
380     class SimpleModel(nn.Module):
381         def __init__(self):
382             super().__init__()
383             self.conv1 = nn.Conv2d(1, 32, 3, 1)
384             self.conv2 = nn.Conv2d(32, 64, 3, 1)
385             self.fc1 = nn.Linear(9216, 128)
386             self.fc2 = nn.Linear(128, 10)
387
388         def forward(self, x):
389             x = F.relu(self.conv1(x))
390             x = F.relu(self.conv2(x))
391             x = F.max_pool2d(x, 2)
392             x = torch.flatten(x, 1)
393             x = F.relu(self.fc1(x))
394             x = self.fc2(x)
395             return x
396
397     # Initialize
398     model = SimpleModel()
399     device = "cpu"
400
401     # Test adversarial attack
402     attacker = AdversarialAttackGenerator(model, device)
403
404     # Generate dummy data
405     image = np.random.rand(1, 28, 28).astype(np.float32)
406     label = 5
407
408     # Generate adversarial example
409     adv_example = attacker.generate_adversarial_example(
410         image, label, attack_type="fgsm", epsilon=0.3
411     )
412
413     print(f"Original prediction: {adv_example.original_prediction}")
414     print(f"Adversarial prediction: {adv_example.adversarial_prediction}")
415     print(f"Perturbation norm: {adv_example.perturbation_norm:.4f}")
416     print(f"Attack successful: {adv_example.original_prediction != adv_example.adversarial_prediction}")
417
418     # Test detection
419     detector = AdversarialDetector(model, device)
420
421     image_tensor = torch.FloatTensor(image).unsqueeze(0)
422     label_tensor = torch.LongTensor([label])
423

```

```

424 detection_result = detector.ensemble_detection(image_tensor, label_tensor)
425 print(f"\nDetection result: {detection_result}")

```

Listing 21.1: Adversarial Attack Detection and Defense Framework

21.4 Model Stealing and Inversion Protection

Model stealing attacks extract model parameters or functionality through query access. Model inversion attacks reconstruct training data from model outputs.

21.4.1 Model Stealing Attacks

Attack Vectors:

- **Equation-Solving:** Solve for model parameters using predictions
- **Path-Finding:** Reconstruct decision tree paths
- **Surrogate Training:** Train substitute model on query results
- **Membership Inference:** Determine if data was in training set

21.4.2 Model Protection Implementation

```

1  #!/usr/bin/env python3
2  """
3  Model Stealing and Inversion Protection
4  Techniques to protect ML models from extraction and data reconstruction
5  """
6
7  import numpy as np
8  import torch
9  import torch.nn as nn
10 from typing import Dict, List, Tuple
11 from collections import defaultdict
12 from datetime import datetime, timedelta
13 import hashlib
14
15 class QueryMonitor:
16     """Monitor and detect suspicious query patterns"""
17
18     def __init__(
19         self,
20         max_queries_per_user: int = 1000,
21         time_window_minutes: int = 60,
22         similarity_threshold: float = 0.95
23     ):
24         self.max_queries_per_user = max_queries_per_user
25         self.time_window = timedelta(minutes=time_window_minutes)
26         self.similarity_threshold = similarity_threshold
27
28         self.query_history = defaultdict(list)
29         self.user_query_counts = defaultdict(int)
30

```

```

31 def record_query(
32     self,
33     user_id: str,
34     query_input: np.ndarray,
35     timestamp: datetime = None
36 ) -> Dict:
37     """Record query and check for suspicious patterns"""
38
39     if timestamp is None:
40         timestamp = datetime.now()
41
42     # Check rate limiting
43     recent_queries = self._get_recent_queries(user_id, timestamp)
44
45     if len(recent_queries) >= self.max_queries_per_user:
46         return {
47             "allowed": False,
48             "reason": "rate_limit_exceeded",
49             "queries_in_window": len(recent_queries)
50         }
51
52     # Check for similar queries (potential model extraction)
53     if self._detect_similar_queries(user_id, query_input):
54         return {
55             "allowed": False,
56             "reason": "suspicious_query_pattern",
57             "details": "Too many similar queries detected"
58         }
59
60     # Record query
61     query_hash = self._hash_input(query_input)
62     self.query_history[user_id].append({
63         "timestamp": timestamp,
64         "query_hash": query_hash,
65         "input_shape": query_input.shape
66     })
67
68     self.user_query_counts[user_id] += 1
69
70     return {
71         "allowed": True,
72         "total_queries": self.user_query_counts[user_id]
73     }
74
75 def _get_recent_queries(
76     self,
77     user_id: str,
78     current_time: datetime
79 ) -> List[Dict]:
80     """Get queries within time window"""
81
82     cutoff_time = current_time - self.time_window
83
84     recent = [
85         q for q in self.query_history[user_id]
86         if q["timestamp"] > cutoff_time
87     ]
88

```

```

89     # Clean old queries
90     self.query_history[user_id] = recent
91
92     return recent
93
94     def _detect_similar_queries(
95         self,
96         user_id: str,
97         query_input: np.ndarray,
98         max_similar: int = 100
99     ) -> bool:
100         """Detect if too many similar queries from same user"""
101
102         query_hash = self._hash_input(query_input)
103         recent_queries = self.query_history[user_id]
104
105         if len(recent_queries) < max_similar:
106             return False
107
108         # Check last N queries for similarity
109         similar_count = sum(
110             1 for q in recent_queries[-max_similar:]
111             if q["query_hash"] == query_hash
112         )
113
114         # If >10% of recent queries are identical, flag as suspicious
115         return similar_count > (max_similar * 0.1)
116
117     def _hash_input(self, query_input: np.ndarray) -> str:
118         """Hash input for similarity detection"""
119
120         # Quantize input to reduce sensitivity
121         quantized = np.round(query_input * 100).astype(np.int32)
122         return hashlib.sha256(quantized.tobytes()).hexdigest()[:16]
123
124 class PredictionObfuscator:
125     """Obfuscate model predictions to prevent information leakage"""
126
127     def __init__(
128         self,
129         noise_scale: float = 0.1,
130         rounding_precision: int = 2,
131         top_k_only: int = None
132     ):
133         self.noise_scale = noise_scale
134         self.rounding_precision = rounding_precision
135         self.top_k_only = top_k_only
136
137     def obfuscate_probabilities(
138         self,
139         probabilities: np.ndarray
140     ) -> np.ndarray:
141         """Add noise and limit precision of probability outputs"""
142
143         # Add Laplace noise for differential privacy
144         noisy_probs = probabilities + np.random.laplace(
145             0, self.noise_scale, size=probabilities.shape
146         )

```

```

147     # Clip to valid probability range
148     noisy_probs = np.clip(noisy_probs, 0, 1)
149
150     # Renormalize
151     noisy_probs = noisy_probs / noisy_probs.sum()
152
153     # Round to limit precision
154     rounded_probs = np.round(noisy_probs, self.rounding_precision)
155
156     # Renormalize again after rounding
157     rounded_probs = rounded_probs / rounded_probs.sum()
158
159     return rounded_probs
160
161
162 def obfuscate_top_k(
163     self,
164     probabilities: np.ndarray,
165     k: int = None
166 ) -> np.ndarray:
167     """Return only top-k predictions to limit information leakage"""
168
169     if k is None:
170         k = self.top_k_only
171
172     if k is None:
173         return probabilities
174
175     # Get top-k indices
176     top_k_indices = np.argsort(probabilities)[-k:]
177
178     # Create sparse output
179     obfuscated = np.zeros_like(probabilities)
180     obfuscated[top_k_indices] = probabilities[top_k_indices]
181
182     # Renormalize
183     obfuscated = obfuscated / obfuscated.sum()
184
185     return obfuscated
186
187 def obfuscate_prediction(
188     self,
189     probabilities: np.ndarray,
190     return_class_only: bool = False
191 ) -> np.ndarray:
192     """Apply full obfuscation pipeline"""
193
194     # Add noise and round
195     obfuscated = self.obfuscate_probabilities(probabilities)
196
197     # Limit to top-k if specified
198     if self.top_k_only:
199         obfuscated = self.obfuscate_top_k(obfuscated, self.top_k_only)
200
201     # Return only predicted class (maximum information hiding)
202     if return_class_only:
203         predicted_class = np.argmax(obfuscated)
204         result = np.zeros_like(obfuscated)

```

```

205         result[predicted_class] = 1.0
206         return result
207
208         return obfuscated
209
210 class ModelInversionDefense:
211     """Defend against model inversion attacks"""
212
213     def __init__(self, model: nn.Module):
214         self.model = model
215
216     def add_differential_privacy_noise(
217         self,
218         gradients: torch.Tensor,
219         epsilon: float = 1.0,
220         delta: float = 1e-5
221     ) -> torch.Tensor:
222         """Add DP noise to gradients during training"""
223
224         # Calculate noise scale for (epsilon, delta)-DP
225         sensitivity = 1.0 # L2 sensitivity of gradients
226         noise_scale = sensitivity * np.sqrt(2 * np.log(1.25 / delta)) / epsilon
227
228         # Add Gaussian noise
229         noise = torch.randn_like(gradients) * noise_scale
230         private_gradients = gradients + noise
231
232         return private_gradients
233
234     def gradient_clipping(
235         self,
236         gradients: torch.Tensor,
237         max_norm: float = 1.0
238     ) -> torch.Tensor:
239         """Clip gradients to bound sensitivity"""
240
241         grad_norm = gradients.norm(2)
242
243         if grad_norm > max_norm:
244             gradients = gradients * (max_norm / grad_norm)
245
246         return gradients
247
248     def prevent_overfitting_to_individuals(
249         self,
250         train_loader,
251         validation_loader,
252         optimizer,
253         max_epochs: int = 100,
254         early_stopping_patience: int = 10
255     ):
256         """Train with early stopping to prevent memorization"""
257
258         best_val_loss = float('inf')
259         patience_counter = 0
260
261         for epoch in range(max_epochs):
262             # Training

```



```

263         self.model.train()
264         train_loss = 0
265
266         for data, target in train_loader:
267             optimizer.zero_grad()
268             output = self.model(data)
269             loss = nn.CrossEntropyLoss()(output, target)
270             loss.backward()
271             optimizer.step()
272             train_loss += loss.item()
273
274         # Validation
275         self.model.eval()
276         val_loss = 0
277
278         with torch.no_grad():
279             for data, target in validation_loader:
280                 output = self.model(data)
281                 loss = nn.CrossEntropyLoss()(output, target)
282                 val_loss += loss.item()
283
284         val_loss /= len(validation_loader)
285
286         # Early stopping check
287         if val_loss < best_val_loss:
288             best_val_loss = val_loss
289             patience_counter = 0
290         else:
291             patience_counter += 1
292
293         if patience_counter >= early_stopping_patience:
294             print(f"Early stopping at epoch {epoch}")
295             break
296
297 class SecureModelAPI:
298     """Secure API wrapper for ML model"""
299
300     def __init__(
301         self,
302         model: nn.Module,
303         enable_rate_limiting: bool = True,
304         enable_obfuscation: bool = True,
305         enable_monitoring: bool = True
306     ):
307         self.model = model
308         self.model.eval()
309
310         self.query_monitor = QueryMonitor() if enable_monitoring else None
311         self.obfuscator = PredictionObfuscator() if enable_obfuscation else None
312
313     def predict(
314         self,
315         user_id: str,
316         input_data: np.ndarray,
317         return_probabilities: bool = False
318     ) -> Dict:
319         """Secure prediction endpoint"""
320

```

```

321     # Monitor query
322     if self.query_monitor:
323         query_check = self.query_monitor.record_query(user_id, input_data)
324
325         if not query_check["allowed"]:
326             return {
327                 "success": False,
328                 "error": query_check["reason"],
329                 "details": query_check
330             }
331
332     # Convert to tensor
333     input_tensor = torch.FloatTensor(input_data).unsqueeze(0)
334
335     # Get prediction
336     with torch.no_grad():
337         output = self.model(input_tensor)
338         probabilities = torch.softmax(output, dim=1).numpy()[0]
339
340     # Obfuscate if enabled
341     if self.obfuscator:
342         probabilities = self.obfuscator.obfuscate_prediction(
343             probabilities,
344             return_class_only=not return_probabilities
345         )
346
347     # Prepare response
348     predicted_class = int(np.argmax(probabilities))
349
350     response = {
351         "success": True,
352         "prediction": predicted_class
353     }
354
355     if return_probabilities:
356         response["probabilities"] = probabilities.tolist()
357
358     return response
359
360 # Example usage
361 if __name__ == "__main__":
362     # Create dummy model
363     class DummyModel(nn.Module):
364         def __init__(self):
365             super().__init__()
366             self.fc = nn.Linear(10, 5)
367
368         def forward(self, x):
369             return self.fc(x)
370
371     model = DummyModel()
372
373     # Test secure API
374     api = SecureModelAPI(
375         model,
376         enable_rate_limiting=True,
377         enable_obfuscation=True,
378         enable_monitoring=True

```

```

379 )
380
381 # Simulate queries
382 user_id = "user123"
383
384 for i in range(5):
385     input_data = np.random.randn(10).astype(np.float32)
386     result = api.predict(user_id, input_data, return_probabilities=True)
387     print(f"Query {i+1}: {result}")
388
389 # Simulate attack (too many queries)
390 print("\nSimulating attack with many queries...")
391 for i in range(1000):
392     input_data = np.random.randn(10).astype(np.float32)
393     result = api.predict(user_id, input_data)
394
395     if not result["success"]:
396         print(f"Blocked at query {i+1}: {result['error']}")
397         break

```

Listing 21.2: Model Stealing and Inversion Protection

21.5 Secure Data Handling and Encryption

Protecting training data and model parameters is critical for ML security.

21.5.1 Data Encryption

```

1  #!/usr/bin/env python3
2  """
3  Secure Data Handling and Encryption for ML Systems
4  Comprehensive data security implementation
5  """
6
7  import numpy as np
8  import pickle
9  import json
10 from cryptography.fernet import Fernet
11 from cryptography.hazmat.primitives import hashes
12 from cryptography.hazmat.primitives.kdf.pbkdf2 import PBKDF2
13 from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
14 from cryptography.hazmat.backends import default_backend
15 import base64
16 import os
17 from typing import Dict, Any
18 import hashlib
19
20 class DataEncryptor:
21     """Encrypt and decrypt ML data and models"""
22
23     def __init__(self, password: str = None):
24         if password:
25             self.key = self._derive_key_from_password(password)
26         else:
27             self.key = Fernet.generate_key()

```

```

28         self.cipher = Fernet(self.key)
29
30
31     def _derive_key_from_password(self, password: str) -> bytes:
32         """Derive encryption key from password"""
33
34         salt = b'ml_security_salt_2024' # In production, use random salt
35
36         kdf = PBKDF2(
37             algorithm=hashes.SHA256(),
38             length=32,
39             salt=salt,
40             iterations=100000,
41             backend=default_backend()
42         )
43
44         key = base64.urlsafe_b64encode(kdf.derive(password.encode()))
45         return key
46
47     def encrypt_numpy_array(self, data: np.ndarray) -> bytes:
48         """Encrypt numpy array"""
49
50         # Serialize array
51         serialized = pickle.dumps(data)
52
53         # Encrypt
54         encrypted = self.cipher.encrypt(serialized)
55
56         return encrypted
57
58     def decrypt_numpy_array(self, encrypted_data: bytes) -> np.ndarray:
59         """Decrypt numpy array"""
60
61         # Decrypt
62         decrypted = self.cipher.decrypt(encrypted_data)
63
64         # Deserialize
65         data = pickle.loads(decrypted)
66
67         return data
68
69     def encrypt_model_weights(self, model_state_dict: Dict) -> bytes:
70         """Encrypt model weights"""
71
72         # Convert tensors to numpy for serialization
73         serializable_state = {}
74         for key, value in model_state_dict.items():
75             if hasattr(value, 'cpu'):
76                 serializable_state[key] = value.cpu().numpy()
77             else:
78                 serializable_state[key] = value
79
80         # Serialize and encrypt
81         serialized = pickle.dumps(serializable_state)
82         encrypted = self.cipher.encrypt(serialized)
83
84         return encrypted
85

```

```

86 def decrypt_model_weights(self, encrypted_weights: bytes) -> Dict:
87     """Decrypt model weights"""
88
89     # Decrypt and deserialize
90     decrypted = self.cipher.decrypt(encrypted_weights)
91     state_dict = pickle.loads(decrypted)
92
93     return state_dict
94
95 def save_encrypted_file(self, data: Any, filepath: str):
96     """Save encrypted data to file"""
97
98     serialized = pickle.dumps(data)
99     encrypted = self.cipher.encrypt(serialized)
100
101     with open(filepath, 'wb') as f:
102         f.write(encrypted)
103
104 def load_encrypted_file(self, filepath: str) -> Any:
105     """Load encrypted data from file"""
106
107     with open(filepath, 'rb') as f:
108         encrypted = f.read()
109
110     decrypted = self.cipher.decrypt(encrypted)
111     data = pickle.loads(decrypted)
112
113     return data
114
115 class SecureDataStore:
116     """Secure storage for training data with access control"""
117
118     def __init__(self, encryption_key: bytes = None):
119         if encryption_key is None:
120             encryption_key = Fernet.generate_key()
121
122         self.encryptor = DataEncryptor()
123         self.encryptor.key = encryption_key
124         self.encryptor.cipher = Fernet(encryption_key)
125
126         self.access_log = []
127         self.authorized_users = set()
128
129     def add_authorized_user(self, user_id: str, role: str = "reader"):
130         """Add user with specific role"""
131
132         self.authorized_users.add((user_id, role))
133
134     def store_dataset(
135         self,
136         dataset_id: str,
137         data: np.ndarray,
138         metadata: Dict,
139         user_id: str
140     ) -> bool:
141         """Store encrypted dataset with metadata"""
142
143         # Check authorization

```

```

144     if not self._check_authorization(user_id, "writer"):
145         self._log_access(user_id, "store", dataset_id, success=False)
146         return False
147
148     # Encrypt data
149     encrypted_data = self.encryptor.encrypt_numpy_array(data)
150
151     # Store with metadata
152     storage_record = {
153         "dataset_id": dataset_id,
154         "encrypted_data": encrypted_data,
155         "metadata": metadata,
156         "created_by": user_id,
157         "checksum": self._calculate_checksum(data)
158     }
159
160     # In production, store to database or object storage
161     filename = f"encrypted_{dataset_id}.bin"
162     with open(filename, 'wb') as f:
163         pickle.dump(storage_record, f)
164
165     self._log_access(user_id, "store", dataset_id, success=True)
166
167     return True
168
169 def retrieve_dataset(
170     self,
171     dataset_id: str,
172     user_id: str
173 ) -> np.ndarray:
174     """Retrieve and decrypt dataset"""
175
176     # Check authorization
177     if not self._check_authorization(user_id, "reader"):
178         self._log_access(user_id, "retrieve", dataset_id, success=False)
179         raise PermissionError(f"User {user_id} not authorized")
180
181     # Load encrypted record
182     filename = f"encrypted_{dataset_id}.bin"
183
184     with open(filename, 'rb') as f:
185         storage_record = pickle.load(f)
186
187     # Decrypt data
188     data = self.encryptor.decrypt_numpy_array(
189         storage_record["encrypted_data"]
190     )
191
192     # Verify integrity
193     checksum = self._calculate_checksum(data)
194     if checksum != storage_record["checksum"]:
195         raise ValueError("Data integrity check failed!")
196
197     self._log_access(user_id, "retrieve", dataset_id, success=True)
198
199     return data
200
201 def _check_authorization(self, user_id: str, required_role: str) -> bool:

```

```

202     """Check if user has required role"""
203
204     for auth_user, role in self.authorized_users:
205         if auth_user == user_id:
206             if role == "admin":
207                 return True
208             if required_role == "reader" and role in ["reader", "writer", "admin"]
]:
209                 return True
210             if required_role == "writer" and role in ["writer", "admin"]:
211                 return True
212
213     return False
214
215 def _calculate_checksum(self, data: np.ndarray) -> str:
216     """Calculate checksum for integrity verification"""
217
218     return hashlib.sha256(data.tobytes()).hexdigest()
219
220 def _log_access(
221     self,
222     user_id: str,
223     action: str,
224     dataset_id: str,
225     success: bool
226 ):
227     """Log data access for audit"""
228
229     from datetime import datetime
230
231     self.access_log.append({
232         "timestamp": datetime.now().isoformat(),
233         "user_id": user_id,
234         "action": action,
235         "dataset_id": dataset_id,
236         "success": success
237     })
238
239 def get_access_log(self) -> list:
240     """Retrieve access log for audit"""
241     return self.access_log
242
243 class HomomorphicEncryptionSimulator:
244     """Simplified homomorphic encryption for ML (educational)"""
245
246     def __init__(self, key_size: int = 2048):
247         self.key_size = key_size
248         self.public_key = None
249         self.private_key = None
250
251         # In production, use libraries like TenSEAL or PySEAL
252         # This is a simplified demonstration
253
254     def encrypt_value(self, value: float) -> bytes:
255         """Encrypt a single value (simplified)"""
256
257         # In real implementation, use proper HE scheme
258         # This is just a demonstration

```

```

259         scaled_value = int(value * 1000000) # Scale for integer encoding
260         noise = np.random.randint(0, 1000)
261
262         # Simple additive encryption (NOT secure, just for demo)
263         encrypted = scaled_value + noise
264
265         return str(encrypted).encode()
266
267     def add_encrypted(self, enc1: bytes, enc2: bytes) -> bytes:
268         """Add two encrypted values"""
269
270         # Demonstrate homomorphic addition
271         val1 = int(enc1.decode())
272         val2 = int(enc2.decode())
273
274         result = val1 + val2
275
276         return str(result).encode()
277
278     def decrypt_value(self, encrypted: bytes) -> float:
279         """Decrypt value (simplified)"""
280
281         # Remove noise and scale back
282         encrypted_int = int(encrypted.decode())
283
284         # In real HE, proper decryption would occur
285         # This is simplified
286         decrypted = encrypted_int / 1000000
287
288         return decrypted
289
290 # Example usage
291 if __name__ == "__main__":
292     # Test data encryption
293     print("=== Data Encryption Test ===")
294     encryptor = DataEncryptor(password="secure_ml_password_2024")
295
296     # Encrypt numpy array
297     original_data = np.random.randn(100, 10)
298     encrypted_data = encryptor.encrypt_numpy_array(original_data)
299     print(f"Original shape: {original_data.shape}")
300     print(f"Encrypted size: {len(encrypted_data)} bytes")
301
302     # Decrypt and verify
303     decrypted_data = encryptor.decrypt_numpy_array(encrypted_data)
304     print(f"Decrypted shape: {decrypted_data.shape}")
305     print(f>Data matches: {np.allclose(original_data, decrypted_data)}")
306
307     # Test secure data store
308     print("\n=== Secure Data Store Test ===")
309     data_store = SecureDataStore()
310
311     # Add authorized users
312     data_store.add_authorized_user("alice", "writer")
313     data_store.add_authorized_user("bob", "reader")
314
315     # Store dataset

```



```

317 dataset = np.random.randn(1000, 50)
318 metadata = {
319     "name": "training_data_2024",
320     "features": 50,
321     "samples": 1000
322 }
323
324 success = data_store.store_dataset(
325     "dataset001",
326     dataset,
327     metadata,
328     "alice"
329 )
330 print(f"Storage successful: {success}")
331
332 # Retrieve dataset
333 retrieved_data = data_store.retrieve_dataset("dataset001", "bob")
334 print(f"Retrieved shape: {retrieved_data.shape}")
335 print(f>Data matches: {np.allclose(dataset, retrieved_data)}")
336
337 # Check access log
338 print(f"\nAccess log entries: {len(data_store.get_access_log())}")
339 for entry in data_store.get_access_log():
340     print(f" {entry}")

```

Listing 21.3: Secure Data Handling with Encryption

21.6 API Security for ML Endpoints

Securing ML APIs requires implementing authentication, authorization, rate limiting, input validation, and monitoring to prevent abuse and attacks.

21.6.1 Authentication and Authorization

Security Requirements:

- **API Key Management:** Secure generation, rotation, and revocation
- **Token-Based Auth:** JWT with expiration and refresh tokens
- **OAuth 2.0:** Third-party authentication and authorization
- **Role-Based Access:** Different permissions for users/services
- **Rate Limiting:** Prevent abuse and DoS attacks

```

1  #!/usr/bin/env python3
2  """
3  Secure ML API with Authentication, Rate Limiting, and Monitoring
4  Production-ready API security implementation
5  """
6
7  from fastapi import FastAPI, HTTPException, Depends, Header, Request
8  from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
9  from pydantic import BaseModel, validator

```

```

10 import jwt
11 import hashlib
12 import time
13 from datetime import datetime, timedelta
14 from typing import Optional, Dict, List
15 import secrets
16 from collections import defaultdict
17 import numpy as np
18 import torch
19 import torch.nn as nn
20 from functools import wraps
21 import re
22
23 # Configuration
24 SECRET_KEY = secrets.token_urlsafe(32)
25 ALGORITHM = "HS256"
26 ACCESS_TOKEN_EXPIRE_MINUTES = 30
27
28 app = FastAPI(title="Secure ML API")
29 security = HTTPBearer()
30
31 class User(BaseModel):
32     """User model"""
33     user_id: str
34     api_key: str
35     role: str
36     rate_limit: int = 100 # queries per hour
37
38 class PredictionRequest(BaseModel):
39     """Prediction request with validation"""
40     features: List[float]
41     model_version: Optional[str] = "v1"
42
43     @validator('features')
44     def validate_features(cls, v):
45         """Validate input features"""
46         if len(v) != 10: # Example: expecting 10 features
47             raise ValueError("Expected 10 features")
48
49         if any(not isinstance(x, (int, float)) for x in v):
50             raise ValueError("All features must be numeric")
51
52         if any(abs(x) > 1000 for x in v):
53             raise ValueError("Feature values must be in range [-1000, 1000]")
54
55         return v
56
57 class APIKeyManager:
58     """Manage API keys and authentication"""
59
60     def __init__(self):
61         self.users: Dict[str, User] = {}
62         self.api_keys: Dict[str, str] = {} # api_key -> user_id
63
64     def create_user(
65         self,
66         user_id: str,
67         role: str = "user",

```

```

68     rate_limit: int = 100
69 ) -> str:
70     """Create user and generate API key"""
71
72     # Generate secure API key
73     api_key = self._generate_api_key(user_id)
74
75     user = User(
76         user_id=user_id,
77         api_key=api_key,
78         role=role,
79         rate_limit=rate_limit
80     )
81
82     self.users[user_id] = user
83     self.api_keys[api_key] = user_id
84
85     return api_key
86
87 def _generate_api_key(self, user_id: str) -> str:
88     """Generate cryptographically secure API key"""
89
90     random_bytes = secrets.token_bytes(32)
91     timestamp = str(time.time()).encode()
92     user_bytes = user_id.encode()
93
94     combined = random_bytes + timestamp + user_bytes
95     api_key = hashlib.sha256(combined).hexdigest()
96
97     return f"sk_{api_key[:48]}"
98
99 def verify_api_key(self, api_key: str) -> Optional[User]:
100     """Verify API key and return user"""
101
102     user_id = self.api_keys.get(api_key)
103     if user_id:
104         return self.users.get(user_id)
105     return None
106
107 def revoke_api_key(self, api_key: str) -> bool:
108     """Revoke API key"""
109
110     user_id = self.api_keys.pop(api_key, None)
111     if user_id and user_id in self.users:
112         self.users[user_id].api_key = None
113         return True
114     return False
115
116 class JWTManager:
117     """Manage JWT tokens"""
118
119     def __init__(self, secret_key: str, algorithm: str = "HS256"):
120         self.secret_key = secret_key
121         self.algorithm = algorithm
122
123     def create_access_token(
124         self,
125         user_id: str,

```

```

126     role: str,
127     expires_delta: timedelta = None
128 ) -> str:
129     """Create JWT access token"""
130
131     if expires_delta is None:
132         expires_delta = timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
133
134     expire = datetime.utcnow() + expires_delta
135
136     payload = {
137         "user_id": user_id,
138         "role": role,
139         "exp": expire,
140         "iat": datetime.utcnow()
141     }
142
143     token = jwt.encode(payload, self.secret_key, algorithm=self.algorithm)
144     return token
145
146 def verify_token(self, token: str) -> Dict:
147     """Verify and decode JWT token"""
148
149     try:
150         payload = jwt.decode(
151             token,
152             self.secret_key,
153             algorithms=[self.algorithm]
154         )
155         return payload
156     except jwt.ExpiredSignatureError:
157         raise HTTPException(status_code=401, detail="Token expired")
158     except jwt.InvalidTokenError:
159         raise HTTPException(status_code=401, detail="Invalid token")
160
161 class RateLimiter:
162     """Rate limiting with sliding window"""
163
164     def __init__(self):
165         self.requests: Dict[str, List[float]] = defaultdict(list)
166
167     def check_rate_limit(
168         self,
169         user_id: str,
170         limit: int,
171         window_seconds: int = 3600
172     ) -> bool:
173         """Check if user is within rate limit"""
174
175         current_time = time.time()
176         cutoff_time = current_time - window_seconds
177
178         # Remove old requests
179         self.requests[user_id] = [
180             timestamp for timestamp in self.requests[user_id]
181             if timestamp > cutoff_time
182         ]
183

```

```

184     # Check limit
185     if len(self.requests[user_id]) >= limit:
186         return False
187
188     # Record new request
189     self.requests[user_id].append(current_time)
190     return True
191
192     def get_remaining_quota(
193         self,
194         user_id: str,
195         limit: int,
196         window_seconds: int = 3600
197     ) -> int:
198         """Get remaining quota for user"""
199
200         current_time = time.time()
201         cutoff_time = current_time - window_seconds
202
203         # Count recent requests
204         recent_requests = sum(
205             1 for timestamp in self.requests[user_id]
206             if timestamp > cutoff_time
207         )
208
209         return max(0, limit - recent_requests)
210
211 class InputValidator:
212     """Validate and sanitize ML API inputs"""
213
214     def __init__(self):
215         self.max_batch_size = 100
216         self.feature_ranges = {}
217
218     def validate_input(
219         self,
220         features: List[float],
221         expected_shape: tuple = None
222     ) -> bool:
223         """Validate input features"""
224
225         # Check for NaN or Inf
226         if any(np.isnan(x) or np.isinf(x) for x in features):
227             raise ValueError("Input contains NaN or Inf values")
228
229         # Check expected shape
230         if expected_shape and len(features) != expected_shape[0]:
231             raise ValueError(
232                 f"Expected {expected_shape[0]} features, got {len(features)}"
233             )
234
235         # Check for injection patterns (SQL, XSS, etc.)
236         # In ML context, checking for unusual patterns
237         feature_array = np.array(features)
238
239         if np.std(feature_array) == 0:
240             raise ValueError("All features have same value (potential attack)")
241

```

```

242         return True
243
244     def sanitize_input(self, features: List[float]) -> List[float]:
245         """Sanitize input by clipping extreme values"""
246
247         feature_array = np.array(features)
248
249         # Clip to reasonable range
250         clipped = np.clip(feature_array, -1000, 1000)
251
252         return clipped.tolist()
253
254 class APISecurityMonitor:
255     """Monitor API for security threats"""
256
257     def __init__(self):
258         self.failed_attempts: Dict[str, List[float]] = defaultdict(list)
259         self.suspicious_patterns: List[Dict] = []
260
261     def record_failed_auth(self, identifier: str):
262         """Record failed authentication attempt"""
263
264         self.failed_attempts[identifier].append(time.time())
265
266         # Check for brute force
267         recent_failures = [
268             t for t in self.failed_attempts[identifier]
269             if t > time.time() - 300 # Last 5 minutes
270         ]
271
272         if len(recent_failures) > 10:
273             self.suspicious_patterns.append({
274                 "type": "brute_force",
275                 "identifier": identifier,
276                 "timestamp": datetime.now().isoformat(),
277                 "attempts": len(recent_failures)
278             })
279
280     def detect_anomaly(
281         self,
282         user_id: str,
283         request_data: Dict
284     ) -> bool:
285         """Detect anomalous request patterns"""
286
287         # Example: detect if user suddenly changes request patterns
288         # In production, use more sophisticated anomaly detection
289
290         return False # Placeholder
291
292     def get_security_report(self) -> Dict:
293         """Generate security report"""
294
295         return {
296             "total_failed_attempts": sum(
297                 len(attempts) for attempts in self.failed_attempts.values()
298             ),
299             "suspicious_patterns": self.suspicious_patterns,

```

```

300         "users_with_failures": len(self.failed_attempts)
301     }
302
303     # Global instances
304     api_key_manager = APIKeyManager()
305     jwt_manager = JWTManager(SECRET_KEY)
306     rate_limiter = RateLimiter()
307     input_validator = InputValidator()
308     security_monitor = APISecurityMonitor()
309
310     # Dummy ML model for demonstration
311     class DummyMLModel(nn.Module):
312         def __init__(self):
313             super().__init__()
314             self.fc = nn.Linear(10, 3)
315
316         def forward(self, x):
317             return self.fc(x)
318
319     ml_model = DummyMLModel()
320     ml_model.eval()
321
322     # Authentication dependency
323     async def verify_api_key_auth(
324         credentials: HTTPAuthorizationCredentials = Depends(security)
325     ) -> User:
326         """Verify API key from bearer token"""
327
328         api_key = credentials.credentials
329         user = api_key_manager.verify_api_key(api_key)
330
331         if not user:
332             security_monitor.record_failed_auth(api_key[:10])
333             raise HTTPException(status_code=401, detail="Invalid API key")
334
335         return user
336
337     async def verify_jwt_auth(
338         credentials: HTTPAuthorizationCredentials = Depends(security)
339     ) -> Dict:
340         """Verify JWT token"""
341
342         token = credentials.credentials
343         payload = jwt_manager.verify_token(token)
344         return payload
345
346     async def check_rate_limit_dependency(
347         user: User = Depends(verify_api_key_auth),
348         request: Request = None
349     ):
350         """Check rate limit for user"""
351
352         if not rate_limiter.check_rate_limit(user.user_id, user.rate_limit):
353             remaining = rate_limiter.get_remaining_quota(user.user_id, user.rate_limit)
354             raise HTTPException(
355                 status_code=429,
356                 detail=f"Rate limit exceeded. Quota remaining: {remaining}",
357                 headers={"Retry-After": "3600"}

```

```

358     )
359
360     return user
361
362     # API Endpoints
363     @app.post("/auth/register")
364     async def register_user(user_id: str, role: str = "user"):
365         """Register new user and get API key"""
366
367         api_key = api_key_manager.create_user(user_id, role)
368
369         return {
370             "user_id": user_id,
371             "api_key": api_key,
372             "message": "User created successfully"
373         }
374
375     @app.post("/auth/token")
376     async def get_token(user: User = Depends(verify_api_key_auth)):
377         """Get JWT token using API key"""
378
379         token = jwt_manager.create_access_token(user.user_id, user.role)
380
381         return {
382             "access_token": token,
383             "token_type": "bearer",
384             "expires_in": ACCESS_TOKEN_EXPIRE_MINUTES * 60
385         }
386
387     @app.post("/predict")
388     async def predict(
389         request: PredictionRequest,
390         user: User = Depends(check_rate_limit_dependency)
391     ):
392         """Secure prediction endpoint"""
393
394         try:
395             # Validate input
396             input_validator.validate_input(request.features, expected_shape=(10,))
397
398             # Sanitize input
399             sanitized_features = input_validator.sanitize_input(request.features)
400
401             # Convert to tensor
402             input_tensor = torch.FloatTensor([sanitized_features])
403
404             # Get prediction
405             with torch.no_grad():
406                 output = ml_model(input_tensor)
407                 probabilities = torch.softmax(output, dim=1).numpy()[0]
408
409             # Limit precision to prevent information leakage
410             probabilities = np.round(probabilities, 3)
411
412             return {
413                 "prediction": int(np.argmax(probabilities)),
414                 "probabilities": probabilities.tolist(),
415                 "model_version": request.model_version,

```



```

416         "user_id": user.user_id
417     }
418
419     except ValueError as e:
420         raise HTTPException(status_code=400, detail=str(e))
421
422 @app.get("/quota")
423 async def get_quota(user: User = Depends(verify_api_key_auth)):
424     """Get remaining API quota"""
425
426     remaining = rate_limiter.get_remaining_quota(user.user_id, user.rate_limit)
427
428     return {
429         "user_id": user.user_id,
430         "rate_limit": user.rate_limit,
431         "remaining": remaining,
432         "window": "1 hour"
433     }
434
435 @app.get("/security/report")
436 async def security_report(user: User = Depends(verify_api_key_auth)):
437     """Get security report (admin only)"""
438
439     if user.role != "admin":
440         raise HTTPException(status_code=403, detail="Admin access required")
441
442     report = security_monitor.get_security_report()
443     return report
444
445 # Example usage
446 if __name__ == "__main__":
447     import uvicorn
448
449     # Create test users
450     admin_key = api_key_manager.create_user("admin", "admin", rate_limit=10000)
451     user_key = api_key_manager.create_user("user1", "user", rate_limit=100)
452
453     print(f"Admin API key: {admin_key}")
454     print(f"User API key: {user_key}")
455     print("\nStarting secure ML API server...")
456     print("Endpoints:")
457     print("  POST /auth/register - Register new user")
458     print("  POST /auth/token - Get JWT token")
459     print("  POST /predict - Make prediction")
460     print("  GET /quota - Check remaining quota")
461     print("  GET /security/report - Get security report (admin)")
462
463     # Run server
464     # uvicorn.run(app, host="0.0.0.0", port=8000)

```

Listing 21.4: Secure ML API Implementation

21.6.2 Input Validation and Sanitization

Best Practices:

- Validate input shapes and data types

- Check for NaN, Inf, and extreme values
- Implement schema validation with Pydantic
- Sanitize inputs to prevent injection attacks
- Set reasonable bounds on all numeric inputs
- Limit batch sizes to prevent resource exhaustion

21.7 Privacy-Preserving ML Techniques

Privacy-preserving techniques allow training and inference on sensitive data while protecting individual privacy.

21.7.1 Differential Privacy

Differential privacy adds calibrated noise to ensure individual records cannot be identified in datasets or model outputs.

```

1  #!/usr/bin/env python3
2  """
3  Differential Privacy for ML Systems
4  Implementation of DP-SGD and privacy-preserving inference
5  """
6
7  import numpy as np
8  import torch
9  import torch.nn as nn
10 import torch.optim as optim
11 from typing import Tuple, List
12 from dataclasses import dataclass
13
14 @dataclass
15 class PrivacyBudget:
16     """Track privacy budget (epsilon, delta)"""
17     epsilon: float
18     delta: float
19     spent_epsilon: float = 0.0
20
21     def has_budget(self) -> bool:
22         """Check if privacy budget remains"""
23         return self.spent_epsilon < self.epsilon
24
25     def spend(self, amount: float):
26         """Spend privacy budget"""
27         self.spent_epsilon += amount
28
29     def remaining(self) -> float:
30         """Get remaining budget"""
31         return max(0, self.epsilon - self.spent_epsilon)
32
33 class DifferentiallyPrivateOptimizer:
34     """DP-SGD optimizer with gradient clipping and noise"""
35
36     def __init__(

```

```

37         self,
38         optimizer: optim.Optimizer,
39         l2_norm_clip: float = 1.0,
40         noise_multiplier: float = 1.1,
41         batch_size: int = 256,
42         epsilon: float = 1.0,
43         delta: float = 1e-5
44     ):
45         self.optimizer = optimizer
46         self.l2_norm_clip = l2_norm_clip
47         self.noise_multiplier = noise_multiplier
48         self.batch_size = batch_size
49
50         self.privacy_budget = PrivacyBudget(epsilon, delta)
51         self.steps = 0
52
53     def clip_gradients(self, parameters):
54         """Clip gradients per-sample to bound sensitivity"""
55
56         # Compute gradient norm
57         total_norm = 0.0
58         for p in parameters:
59             if p.grad is not None:
60                 param_norm = p.grad.data.norm(2)
61                 total_norm += param_norm.item() ** 2
62
63         total_norm = total_norm ** 0.5
64
65         # Clip if necessary
66         clip_coef = self.l2_norm_clip / (total_norm + 1e-6)
67         if clip_coef < 1:
68             for p in parameters:
69                 if p.grad is not None:
70                     p.grad.data.mul_(clip_coef)
71
72         return total_norm
73
74     def add_noise(self, parameters):
75         """Add Gaussian noise to gradients"""
76
77         noise_scale = self.noise_multiplier * self.l2_norm_clip
78
79         for p in parameters:
80             if p.grad is not None:
81                 noise = torch.randn_like(p.grad) * noise_scale
82                 p.grad.data.add_(noise)
83
84     def step(self, parameters):
85         """Perform DP-SGD step"""
86
87         # Clip gradients
88         grad_norm = self.clip_gradients(parameters)
89
90         # Add noise
91         self.add_noise(parameters)
92
93         # Update parameters
94         self.optimizer.step()

```

```

95     # Update privacy accounting
96     self.steps += 1
97     epsilon_spent = self._compute_epsilon_spent()
98     self.privacy_budget.spent_epsilon = epsilon_spent
99
100
101     return grad_norm
102
103 def _compute_epsilon_spent(self) -> float:
104     """Compute spent privacy budget using moments accountant"""
105
106     # Simplified calculation (use opacus or tensorflow-privacy for accurate
107 accounting)
108     q = self.batch_size / 60000 # sampling probability (assuming MNIST-sized
109 dataset)
110     sigma = self.noise_multiplier
111
112     # Strong composition theorem (simplified)
113     epsilon = q * self.steps * (1 / (2 * sigma ** 2))
114
115     return epsilon
116
117 def get_privacy_spent(self) -> Tuple[float, float]:
118     """Get spent privacy budget"""
119     return (self.privacy_budget.spent_epsilon, self.privacy_budget.delta)
120
121 class PrivateAggregator:
122     """Aggregate statistics with differential privacy"""
123
124     def __init__(self, epsilon: float = 1.0):
125         self.epsilon = epsilon
126
127     def private_mean(
128         self,
129         data: np.ndarray,
130         bounds: Tuple[float, float] = (0, 1)
131     ) -> float:
132         """Compute differentially private mean"""
133
134         lower, upper = bounds
135         sensitivity = (upper - lower) / len(data)
136
137         # Clip data to bounds
138         clipped_data = np.clip(data, lower, upper)
139
140         # Compute mean
141         true_mean = np.mean(clipped_data)
142
143         # Add Laplace noise
144         scale = sensitivity / self.epsilon
145         noise = np.random.laplace(0, scale)
146
147         private_mean = true_mean + noise
148
149         return float(private_mean)
150
151 def private_count(
152     self,

```

```

151     condition: np.ndarray
152 ) -> int:
153     """Count with differential privacy"""
154
155     true_count = np.sum(condition)
156
157     # Sensitivity is 1 for counting
158     scale = 1.0 / self.epsilon
159     noise = np.random.laplace(0, scale)
160
161     private_count = true_count + noise
162
163     # Round to integer
164     return int(max(0, round(private_count)))
165
166 def private_histogram(
167     self,
168     data: np.ndarray,
169     bins: int = 10,
170     range: Tuple[float, float] = None
171 ) -> Tuple[np.ndarray, np.ndarray]:
172     """Compute differentially private histogram"""
173
174     # Compute true histogram
175     counts, bin_edges = np.histogram(data, bins=bins, range=range)
176
177     # Add Laplace noise to each bin
178     # Sensitivity is 1 (one person can affect one bin)
179     scale = 1.0 / self.epsilon
180
181     private_counts = counts + np.random.laplace(0, scale, size=len(counts))
182     private_counts = np.maximum(0, private_counts) # No negative counts
183
184     return private_counts, bin_edges
185
186 class FederatedLearningCoordinator:
187     """Coordinate federated learning with privacy"""
188
189     def __init__(
190         self,
191         model: nn.Module,
192         num_clients: int,
193         privacy_epsilon: float = 10.0
194     ):
195         self.global_model = model
196         self.num_clients = num_clients
197         self.privacy_epsilon = privacy_epsilon
198         self.round = 0
199
200     def aggregate_models(
201         self,
202         client_models: List[nn.Module],
203         client_weights: List[int] = None
204     ) -> nn.Module:
205         """Aggregate client models using federated averaging"""
206
207         if client_weights is None:
208             client_weights = [1.0] * len(client_models)

```

```

209
210     # Normalize weights
211     total_weight = sum(client_weights)
212     client_weights = [w / total_weight for w in client_weights]
213
214     # Average model parameters
215     global_dict = self.global_model.state_dict()
216
217     for key in global_dict.keys():
218         # Weighted average of client parameters
219         global_dict[key] = sum(
220             client_models[i].state_dict()[key] * client_weights[i]
221             for i in range(len(client_models))
222         )
223
224         # Add noise for privacy
225         if 'weight' in key or 'bias' in key:
226             noise_scale = 0.01 * (self.num_clients / self.privacy_epsilon)
227             noise = torch.randn_like(global_dict[key]) * noise_scale
228             global_dict[key] += noise
229
230     # Update global model
231     self.global_model.load_state_dict(global_dict)
232     self.round += 1
233
234     return self.global_model
235
236 def secure_aggregation(
237     self,
238     client_updates: List[Dict],
239     use_secure_sum: bool = True
240 ) -> Dict:
241     """Secure aggregation without revealing individual updates"""
242
243     # In production, implement actual secure aggregation protocol
244     # This is a simplified version
245
246     aggregated_update = {}
247
248     for key in client_updates[0].keys():
249         # Sum all client updates
250         total_update = sum(
251             update[key] for update in client_updates
252         )
253
254         # Add masking noise (cancelled out across clients in real protocol)
255         if use_secure_sum:
256             # Placeholder: real implementation uses secret sharing
257             mask_noise = torch.randn_like(total_update) * 0.001
258             aggregated_update[key] = total_update + mask_noise
259         else:
260             aggregated_update[key] = total_update
261
262     return aggregated_update
263
264 # Example usage
265 if __name__ == "__main__":
266     print("=== Differential Privacy Test ===")

```

```

267
268 # Test private aggregation
269 aggregator = PrivateAggregator(epsilon=1.0)
270
271 # Simulate salary data
272 salaries = np.random.normal(50000, 15000, 1000)
273
274 true_mean = np.mean(salaries)
275 private_mean = aggregator.private_mean(salaries, bounds=(0, 200000))
276
277 print(f"True mean salary: ${true_mean:.2f}")
278 print(f"Private mean salary: ${private_mean:.2f}")
279 print(f"Error: ${abs(true_mean - private_mean):.2f}")
280
281 # Test private counting
282 high_earners = salaries > 60000
283 true_count = np.sum(high_earners)
284 private_count = aggregator.private_count(high_earners)
285
286 print(f"\nTrue count of high earners: {true_count}")
287 print(f"Private count of high earners: {private_count}")
288
289 # Test DP-SGD
290 print("\n=== DP-SGD Test ===")
291
292 model = nn.Linear(10, 5)
293 optimizer = optim.SGD(model.parameters(), lr=0.01)
294
295 dp_optimizer = DifferentiallyPrivateOptimizer(
296     optimizer,
297     l2_norm_clip=1.0,
298     noise_multiplier=1.1,
299     batch_size=32,
300     epsilon=1.0,
301     delta=1e-5
302 )
303
304 # Simulate training step
305 dummy_input = torch.randn(32, 10)
306 dummy_target = torch.randint(0, 5, (32,))
307
308 output = model(dummy_input)
309 loss = nn.CrossEntropyLoss()(output, dummy_target)
310
311 optimizer.zero_grad()
312 loss.backward()
313
314 dp_optimizer.step(model.parameters())
315
316 epsilon_spent, delta = dp_optimizer.get_privacy_spent()
317 print(f"Privacy spent: ( $\epsilon$ =${epsilon_spent:.4f},  $\delta$ =${delta})")
318 print(f"Privacy remaining:  $\epsilon$ =${dp_optimizer.privacy_budget.remaining():.4f}")
319
320 # Test federated learning
321 print("\n=== Federated Learning Test ===")
322
323 global_model = nn.Linear(10, 5)
324 coordinator = FederatedLearningCoordinator(

```

```

325     global_model,
326     num_clients=10,
327     privacy_epsilon=10.0
328 )
329
330 # Simulate client models
331 client_models = [nn.Linear(10, 5) for _ in range(3)]
332 client_weights = [100, 150, 200] # Number of samples per client
333
334 updated_global = coordinator.aggregate_models(client_models, client_weights)
335 print(f"Federated learning round {coordinator.round} completed")

```

Listing 21.5: Differential Privacy Implementation

21.7.2 Federated Learning

Federated learning trains models on decentralized data without centralizing it, preserving privacy by keeping data on local devices.

Key Principles:

- **Local Training:** Models train on local data
- **Secure Aggregation:** Combine updates without revealing individual contributions
- **Differential Privacy:** Add noise to updates
- **Client Selection:** Random sampling of participants
- **Communication Efficiency:** Compress updates to reduce bandwidth

21.8 Security Scanning and Vulnerability Assessment

Regular security scanning identifies vulnerabilities before attackers exploit them.

21.8.1 ML Security Scanner

```

1  #!/usr/bin/env python3
2  """
3  ML Security Scanner
4  Automated security assessment for ML systems
5  """
6
7  import ast
8  import os
9  import re
10 from typing import List, Dict, Set
11 from dataclasses import dataclass
12 from pathlib import Path
13 import pickle
14 import subprocess
15
16 @dataclass

```



```

17 class SecurityFinding:
18     """Security finding from scan"""
19     severity: str # critical, high, medium, low
20     category: str
21     description: str
22     location: str
23     recommendation: str
24     cwe_id: str = None
25
26 class MLSecurityScanner:
27     """Scan ML systems for security vulnerabilities"""
28
29     def __init__(self, project_path: str):
30         self.project_path = Path(project_path)
31         self.findings: List[SecurityFinding] = []
32
33     def scan_all(self) -> List[SecurityFinding]:
34         """Run all security scans"""
35
36         self.scan_pickle_usage()
37         self.scan_hardcoded_secrets()
38         self.scan_sql_injection()
39         self.scan_unsafe_deserialization()
40         self.scan_insecure_random()
41         self.scan_model_files()
42         self.scan_dependency_vulnerabilities()
43
44         return self.findings
45
46     def scan_pickle_usage(self):
47         """Detect unsafe pickle usage"""
48
49         for py_file in self.project_path.rglob("*.py"):
50             try:
51                 with open(py_file, 'r', encoding='utf-8') as f:
52                     content = f.read()
53
54                     # Check for pickle.load without validation
55                     if re.search(r'pickle\.load\(', content):
56                         self.findings.append(SecurityFinding(
57                             severity="high",
58                             category="Insecure Deserialization",
59                             description="Unsafe pickle.load() usage detected",
60                             location=str(py_file),
61                             recommendation="Use safe serialization (JSON, protobuf) or
validate pickle data source",
62                             cwe_id="CWE-502"
63                         ))
64
65                     # Check for joblib.load (also uses pickle)
66                     if re.search(r'joblib\.load\(', content):
67                         self.findings.append(SecurityFinding(
68                             severity="medium",
69                             category="Insecure Deserialization",
70                             description="joblib.load() uses pickle internally",
71                             location=str(py_file),
72                             recommendation="Validate source before loading joblib files",
73                             cwe_id="CWE-502"

```

```

74         ))
75
76     except Exception as e:
77         pass
78
79     def scan_hardcoded_secrets(self):
80         """Detect hardcoded secrets and credentials"""
81
82         secret_patterns = [
83             (r'api[_-]?key\s*=\s*["\']([a-zA-Z0-9]{32,})["\']', "API Key"),
84             (r'password\s*=\s*["\'](.+)(["\']', "Password"),
85             (r'secret\s*=\s*["\'](.+)(["\']', "Secret"),
86             (r'token\s*=\s*["\']([a-zA-Z0-9]{20,})["\']', "Token"),
87             (r'aws_secret_access_key\s*=\s*["\'](.+)(["\']', "AWS Secret"),
88         ]
89
90         for py_file in self.project_path.rglob("*.py"):
91             try:
92                 with open(py_file, 'r', encoding='utf-8') as f:
93                     content = f.read()
94
95                     for pattern, secret_type in secret_patterns:
96                         matches = re.finditer(pattern, content, re.IGNORECASE)
97                         for match in matches:
98                             self.findings.append(SecurityFinding(
99                                 severity="critical",
100                                category="Hardcoded Secrets",
101                                description=f"Hardcoded {secret_type} detected",
102                                location=str(py_file),
103                                recommendation="Use environment variables or secret
management service",
104                                cwe_id="CWE-798"
105                            ))
106
107             except Exception:
108                 pass
109
110     def scan_sql_injection(self):
111         """Detect potential SQL injection vulnerabilities"""
112
113         sql_patterns = [
114             r'execute\([f"\'].*\{.*\}', # String interpolation in SQL
115             r'\.format\(.*\).*execute', # .format() in SQL
116             r'%\s*.*execute',          # % formatting in SQL
117         ]
118
119         for py_file in self.project_path.rglob("*.py"):
120             try:
121                 with open(py_file, 'r', encoding='utf-8') as f:
122                     content = f.read()
123
124                     for pattern in sql_patterns:
125                         if re.search(pattern, content):
126                             self.findings.append(SecurityFinding(
127                                 severity="high",
128                                 category="SQL Injection",
129                                 description="Potential SQL injection via string
interpolation",

```

```

130         location=str(py_file),
131         recommendation="Use parameterized queries or ORM",
132         cwe_id="CWE-89"
133     ))
134     break
135
136 except Exception:
137     pass
138
139 def scan_unsafe_deserialization(self):
140     """Detect unsafe deserialization patterns"""
141
142     unsafe_patterns = [
143         r'eval\(',
144         r'exec\(',
145         r'__import__\(',
146         r'compile\(',
147     ]
148
149     for py_file in self.project_path.rglob("*.py"):
150         try:
151             with open(py_file, 'r', encoding='utf-8') as f:
152                 content = f.read()
153
154                 for pattern in unsafe_patterns:
155                     if re.search(pattern, content):
156                         self.findings.append(SecurityFinding(
157                             severity="critical",
158                             category="Code Injection",
159                             description=f"Unsafe function detected: {pattern}",
160                             location=str(py_file),
161                             recommendation="Avoid dynamic code execution with
untrusted input",
162                             cwe_id="CWE-94"
163                         ))
164
165         except Exception:
166             pass
167
168 def scan_insecure_random(self):
169     """Detect insecure random number generation"""
170
171     for py_file in self.project_path.rglob("*.py"):
172         try:
173             with open(py_file, 'r', encoding='utf-8') as f:
174                 content = f.read()
175
176                 # Check for random.random() in security contexts
177                 if re.search(r'import random', content):
178                     if any(keyword in content.lower() for keyword in
179                         ['password', 'token', 'key', 'secret', 'auth']):
180                         self.findings.append(SecurityFinding(
181                             severity="medium",
182                             category="Weak Cryptography",
183                             description="Using insecure random for security purposes",
184                             location=str(py_file),
185                             recommendation="Use secrets module for cryptographic
randomness",

```

```

186         cwe_id="CWE-330"
187     ))
188
189     except Exception:
190         pass
191
192     def scan_model_files(self):
193         """Scan model files for security issues"""
194
195         # Check for unencrypted model files
196         model_extensions = ['.pkl', '.h5', '.pt', '.pth', '.onnx', '.pb']
197
198         for ext in model_extensions:
199             for model_file in self.project_path.rglob(f"*{ext}"):
200                 # Check if file is world-readable (on Unix systems)
201                 try:
202                     if os.path.exists(model_file):
203                         permissions = oct(os.stat(model_file).st_mode)[-3:]
204                         if permissions[-1] in ['4', '5', '6', '7']: # World-readable
205                             self.findings.append(SecurityFinding(
206                                 severity="medium",
207                                 category="Insecure File Permissions",
208                                 description="Model file is world-readable",
209                                 location=str(model_file),
210                                 recommendation="Restrict file permissions (chmod 600)"
211                             ,
212                             cwe_id="CWE-732"
213                         ))
214                 except Exception:
215                     pass
216
217     def scan_dependency_vulnerabilities(self):
218         """Check for vulnerable dependencies"""
219
220         requirements_file = self.project_path / "requirements.txt"
221
222         if requirements_file.exists():
223             try:
224                 # Run safety check (if installed)
225                 result = subprocess.run(
226                     ["safety", "check", "-r", str(requirements_file)],
227                     capture_output=True,
228                     text=True,
229                     timeout=30
230                 )
231
232                 if "vulnerabilities found" in result.stdout.lower():
233                     self.findings.append(SecurityFinding(
234                         severity="high",
235                         category="Vulnerable Dependencies",
236                         description="Vulnerable packages detected in requirements.txt"
237                     ,
238                     location=str(requirements_file),
239                     recommendation="Update vulnerable packages",
240                     cwe_id="CWE-1035"
241                 ))
242             except (subprocess.TimeoutExpired, FileNotFoundError):

```

```

242         # Safety not installed or timeout
243         pass
244
245     def generate_report(self) -> str:
246         """Generate security report"""
247
248         # Group findings by severity
249         by_severity = {'critical': [], 'high': [], 'medium': [], 'low': []}
250
251         for finding in self.findings:
252             by_severity[finding.severity].append(finding)
253
254         report = "=" * 60 + "\n"
255         report += "ML SECURITY SCAN REPORT\n"
256         report += "=" * 60 + "\n\n"
257
258         report += f"Total Findings: {len(self.findings)}\n"
259         report += f"    Critical: {len(by_severity['critical'])}\n"
260         report += f"    High: {len(by_severity['high'])}\n"
261         report += f"    Medium: {len(by_severity['medium'])}\n"
262         report += f"    Low: {len(by_severity['low'])}\n\n"
263
264         for severity in ['critical', 'high', 'medium', 'low']:
265             if by_severity[severity]:
266                 report += f"\n{severity.upper()} SEVERITY FINDINGS:\n"
267                 report += "-" * 60 + "\n"
268
269                 for i, finding in enumerate(by_severity[severity], 1):
270                     report += f"\n{i}. {finding.category}\n"
271                     report += f"    Description: {finding.description}\n"
272                     report += f"    Location: {finding.location}\n"
273                     report += f"    Recommendation: {finding.recommendation}\n"
274                     if finding.cwe_id:
275                         report += f"    CWE: {finding.cwe_id}\n"
276
277         return report
278
279 class PenetrationTester:
280     """Automated penetration testing for ML APIs"""
281
282     def __init__(self, api_url: str):
283         self.api_url = api_url
284         self.findings: List[SecurityFinding] = []
285
286     def test_all(self) -> List[SecurityFinding]:
287         """Run all penetration tests"""
288
289         self.test_authentication()
290         self.test_rate_limiting()
291         self.test_input_validation()
292         self.test_injection_attacks()
293
294         return self.findings
295
296     def test_authentication(self):
297         """Test authentication bypass"""
298
299         # Test without credentials

```

```

300     # Test with invalid credentials
301     # Test with expired tokens
302     # Test session fixation
303
304     pass # Placeholder
305
306     def test_rate_limiting(self):
307         """Test rate limiting effectiveness"""
308
309         # Send burst of requests
310         # Check if rate limiting is enforced
311
312         pass # Placeholder
313
314     def test_input_validation(self):
315         """Test input validation"""
316
317         test_inputs = [
318             "' OR '1'='1", # SQL injection
319             "<script>alert('xss')</script>", # XSS
320             "../../../etc/passwd", # Path traversal
321             "A" * 10000, # Buffer overflow attempt
322             float('inf'), # Infinity
323             float('nan'), # NaN
324         ]
325
326         # Test each malicious input
327         pass # Placeholder
328
329     def test_injection_attacks(self):
330         """Test for injection vulnerabilities"""
331
332         pass # Placeholder
333
334 # Example usage
335 if __name__ == "__main__":
336     # Create scanner
337     scanner = MLSecurityScanner("./")
338
339     print("Running security scan...")
340     findings = scanner.scan_all()
341
342     # Generate report
343     report = scanner.generate_report()
344     print(report)
345
346     # Save report
347     with open("security_report.txt", "w") as f:
348         f.write(report)
349
350     print(f"\nReport saved to security_report.txt")

```

Listing 21.6: ML Security Vulnerability Scanner

21.8.2 Security Best Practices

Defense-in-Depth Strategy:

1. **Network Security:** Firewalls, VPNs, network segmentation
2. **Application Security:** Input validation, authentication, authorization
3. **Data Security:** Encryption at rest and in transit
4. **Model Security:** Access controls, obfuscation, monitoring
5. **Infrastructure Security:** Hardened containers, least privilege
6. **Monitoring:** Continuous logging, anomaly detection, alerting

21.9 Implementation Exercises

21.9.1 Exercise 1: Adversarial Attack Detection System

Objective: Build a comprehensive adversarial attack detection and defense system for image classification.

Requirements:

1. Train a CNN classifier on MNIST or CIFAR-10
2. Implement FGSM and PGD attack generators
3. Create ensemble detection with 3+ methods:
 - Confidence-based detection
 - Input gradient magnitude
 - Feature squeezing
 - Statistical outlier detection
4. Implement adversarial training defense
5. Evaluate detection accuracy and false positive rate
6. Test with different epsilon values (0.1, 0.2, 0.3)

Evaluation Metrics:

- Clean accuracy vs adversarial accuracy
- Detection true positive rate at 5% false positive rate
- Robustness gain from adversarial training
- Computational overhead of detection methods

Bonus Challenges:

- Implement C&W attack
- Add defensive distillation
- Create visualization of perturbations
- Compare white-box vs black-box attack success rates

21.9.2 Exercise 2: Secure ML API with Rate Limiting

Objective: Build production-ready ML API with comprehensive security controls.

Requirements:

1. Implement FastAPI with model serving
2. Add multi-tier authentication:
 - API key generation and validation
 - JWT token support with refresh
 - Role-based access control (user, premium, admin)
3. Implement sliding window rate limiting per user
4. Add input validation and sanitization
5. Create security monitoring dashboard
6. Log all requests with audit trail
7. Implement request/response encryption

Test Cases:

- Verify rate limiting blocks excess requests
- Test authentication bypass attempts
- Validate input sanitization with malicious inputs
- Measure API response time overhead
- Simulate coordinated attack from multiple IPs

Deliverables:

- Dockerized API service
- API documentation with security notes
- Load testing results (1000+ RPS)
- Security scan report with no critical findings

21.9.3 Exercise 3: Differential Privacy for Model Training

Objective: Implement DP-SGD to train privacy-preserving classification model.

Requirements:

1. Implement DP-SGD optimizer from scratch
2. Train on sensitive dataset (medical, financial simulation)
3. Track privacy budget (epsilon, delta) across epochs

4. Compare performance:
 - Standard SGD vs DP-SGD
 - Different noise multipliers (0.5, 1.0, 1.5, 2.0)
 - Various clipping norms (0.5, 1.0, 2.0)
5. Implement privacy accounting with moments accountant
6. Create privacy/utility tradeoff curves
7. Test membership inference attack resistance

Analysis Tasks:

- Plot accuracy vs epsilon curves
- Measure training time overhead
- Evaluate membership inference attack success rate
- Document privacy guarantees achieved

Extensions:

- Implement PATE (Private Aggregation of Teacher Ensembles)
- Add differential privacy to inference
- Compare with federated learning + DP

21.9.4 Exercise 4: Model Stealing Defense System

Objective: Protect ML models from extraction attacks with query monitoring.

Requirements:

1. Deploy target model as API
2. Implement query monitoring:
 - Track query patterns per user
 - Detect similar/duplicate queries
 - Identify systematic exploration patterns
 - Flag anomalous query volumes
3. Add prediction obfuscation:
 - Confidence score rounding
 - Selective top-k predictions
 - Calibrated noise injection
4. Create watermarking for model ownership verification
5. Implement query budget enforcement

Attack Simulation:

- Implement model extraction attack (equation-solving, surrogate training)
- Measure extraction accuracy vs query budget
- Test detection system against various attack strategies
- Evaluate impact of defenses on extraction success

Metrics:

- Surrogate model accuracy (should be $<70\%$ of original)
- Detection rate of extraction attempts ($>90\%$)
- False positive rate on legitimate users ($<5\%$)
- Service quality degradation ($<10\%$ accuracy loss)

21.9.5 Exercise 5: Federated Learning with Secure Aggregation

Objective: Implement privacy-preserving federated learning system.

Requirements:

1. Create federated learning coordinator
2. Simulate 10+ client devices with local datasets
3. Implement FedAvg aggregation algorithm
4. Add differential privacy to client updates
5. Implement secure aggregation (simplified):
 - Mask client updates with random noise
 - Aggregate without revealing individual contributions
 - Verify aggregation correctness
6. Handle client dropouts gracefully
7. Optimize communication efficiency (compression)

Experiments:

- Non-IID data distribution across clients
- Compare centralized vs federated accuracy
- Measure communication rounds to convergence
- Test with 10%, 50% client participation
- Evaluate privacy vs accuracy tradeoffs

Advanced Features:

- Implement personalized federated learning
- Add Byzantine-robust aggregation
- Test against gradient inversion attacks
- Implement adaptive privacy budgets

21.9.6 Exercise 6: ML Security Audit and Penetration Testing

Objective: Perform comprehensive security audit of ML system.

Requirements:

1. Implement automated security scanner:
 - Scan for insecure pickle usage
 - Detect hardcoded credentials
 - Find SQL injection vulnerabilities
 - Check for unsafe deserialization
 - Identify weak cryptography
 - Scan dependency vulnerabilities
2. Perform penetration testing:
 - Authentication bypass attempts
 - Rate limiting evasion
 - Input injection attacks
 - Model extraction attempts
 - Adversarial example generation
3. Generate detailed security report
4. Implement fixes for all critical/high findings
5. Create security hardening checklist

Testing Scope:

- Code security review (SAST)
- API penetration testing
- Model robustness testing
- Data privacy assessment
- Infrastructure security audit

Deliverables:

- Security scan report (categorized by severity)

- Penetration test results with PoC exploits
- Remediation plan with priority rankings
- Before/after security metrics comparison
- Security best practices documentation

21.9.7 Exercise Evaluation Criteria

All exercises will be evaluated on:

1. **Correctness (30%):** Implementation works as specified
2. **Security (25%):** Proper security controls and best practices
3. **Performance (15%):** Efficient implementation with minimal overhead
4. **Testing (15%):** Comprehensive test coverage and validation
5. **Documentation (10%):** Clear explanations and usage examples
6. **Code Quality (5%):** Clean, maintainable, well-structured code

21.10 Conclusion

Security is paramount in production ML systems. This chapter covered the ML-specific threat landscape and defense mechanisms across multiple layers.

Key Takeaways:

1. **ML Security is Unique:** Traditional security measures are necessary but insufficient. ML systems face adversarial attacks, model extraction, privacy leakage, and data poisoning that require specialized defenses.
2. **Defense in Depth:** No single security measure is perfect. Layer multiple defenses: adversarial training, input validation, rate limiting, encryption, monitoring, and access controls.
3. **Privacy Preservation:** Differential privacy and federated learning enable learning from sensitive data while providing mathematical privacy guarantees. The privacy-utility tradeoff must be carefully managed.
4. **API Security Critical:** ML APIs are attack surfaces requiring authentication, authorization, rate limiting, input sanitization, and continuous monitoring.
5. **Proactive Security:** Regular security scanning, vulnerability assessments, and penetration testing identify issues before attackers exploit them.
6. **Security Overhead Acceptable:** The performance cost of security measures (10-30% typical) is justified by risk reduction and regulatory compliance.

Security Checklist for ML Systems:

- ☐ Implement adversarial attack detection and defense
- ☐ Protect models from extraction with query monitoring and obfuscation
- ☐ Encrypt sensitive data at rest and in transit
- ☐ Secure model files with access controls and checksums
- ☐ Deploy authentication and authorization for all endpoints
- ☐ Implement rate limiting to prevent abuse
- ☐ Validate and sanitize all inputs
- ☐ Add differential privacy when handling sensitive data
- ☐ Monitor for security anomalies and failed access attempts
- ☐ Conduct regular security scans and penetration tests
- ☐ Maintain audit logs for all model access
- ☐ Keep dependencies updated and scan for vulnerabilities
- ☐ Train team on ML security best practices
- ☐ Establish incident response procedures
- ☐ Document security architecture and controls

Future Directions:

ML security is rapidly evolving. Emerging areas include:

- **Certified Robustness:** Provable guarantees against adversarial attacks
- **Homomorphic Encryption:** Computation on encrypted data without decryption
- **Secure Multi-Party Computation:** Collaborative learning without data sharing
- **Zero-Knowledge Proofs:** Verify model properties without revealing internals
- **Quantum-Safe ML:** Protecting against quantum computing threats
- **AI-Powered Security:** Using ML to defend ML systems

Security must be built into ML systems from the design phase, not added as an afterthought. As ML systems increasingly make critical decisions affecting privacy, safety, and fairness, robust security becomes non-negotiable.

The techniques covered in this chapter provide a foundation for building secure ML systems. However, security is an ongoing process requiring continuous monitoring, updating, and improvement as new threats emerge and attack techniques evolve.

Recommended Resources:

- OWASP Machine Learning Security Top 10
- NIST AI Risk Management Framework

- MITRE ATLAS (Adversarial Threat Landscape for AI Systems)
- CleverHans library for adversarial examples
- Opacus library for differential privacy
- TensorFlow Privacy framework
- PySyft for privacy-preserving ML

By implementing the security controls, monitoring systems, and best practices outlined in this chapter, ML engineers can build robust systems that protect against evolving threats while maintaining model performance and user privacy.

Chapter 22

Cost Optimization in ML Infrastructure

22.1 Learning Objectives

After completing this chapter, readers will be able to:

- Understand the cost structure of ML systems across training and serving
- Implement comprehensive training cost optimization strategies
- Analyze and reduce serving infrastructure costs
- Design intelligent resource allocation and scaling policies
- Deploy cloud cost monitoring systems with automated alerts
- Leverage spot instances and preemptible VMs effectively
- Calculate and optimize ROI for ML investments
- Apply cost optimization best practices across the ML lifecycle

22.2 Introduction

ML systems often represent significant infrastructure investments, with costs scaling rapidly as model complexity and usage increase. A production ML system can cost anywhere from thousands to millions of dollars annually, making cost optimization a critical engineering concern.

Primary Cost Centers in ML Systems:

- **Training Costs:** Compute for model training, hyperparameter optimization, experimentation
- **Serving Costs:** Inference infrastructure, API endpoints, model hosting
- **Storage Costs:** Training data, model artifacts, feature stores, logs
- **Data Processing:** ETL pipelines, feature engineering, data validation

- **Personnel:** ML engineers, data scientists, MLOps specialists
- **Monitoring and Observability:** Metrics, logs, traces, dashboards
- **Development and Experimentation:** Notebooks, development environments, testing

Cost Optimization Principles:

1. **Visibility:** You can't optimize what you can't measure
2. **Right-sizing:** Match resources to actual requirements
3. **Automation:** Scale resources dynamically based on demand
4. **Efficiency:** Optimize algorithms and infrastructure utilization
5. **Trade-offs:** Balance cost, performance, and latency requirements

22.3 ML Cost Structure and Analysis

Understanding the cost breakdown of ML systems is the first step toward optimization.

22.3.1 Cost Breakdown Model

```

1  #!/usr/bin/env python3
2  """
3  ML System Cost Analysis Framework
4  Comprehensive cost tracking and breakdown for ML infrastructure
5  """
6
7  from dataclasses import dataclass, field
8  from typing import Dict, List, Optional
9  from datetime import datetime, timedelta
10 from enum import Enum
11 import pandas as pd
12 import numpy as np
13
14 class CostCategory(Enum):
15     """Cost categories for ML systems"""
16     TRAINING = "training"
17     SERVING = "serving"
18     STORAGE = "storage"
19     DATA_PROCESSING = "data_processing"
20     MONITORING = "monitoring"
21     DEVELOPMENT = "development"
22     NETWORKING = "networking"
23
24 @dataclass
25 class ResourceCost:
26     """Individual resource cost record"""
27     resource_id: str
28     resource_type: str # compute, storage, network
29     category: CostCategory
30     cost: float

```



```

31     unit: str # per hour, per GB, per request
32     quantity: float
33     start_time: datetime
34     end_time: datetime
35     metadata: Dict = field(default_factory=dict)
36
37     @property
38     def total_cost(self) -> float:
39         """Calculate total cost"""
40         return self.cost * self.quantity
41
42     @property
43     def duration_hours(self) -> float:
44         """Calculate duration in hours"""
45         return (self.end_time - self.start_time).total_seconds() / 3600
46
47 @dataclass
48 class CostReport:
49     """Cost analysis report"""
50     total_cost: float
51     breakdown_by_category: Dict[CostCategory, float]
52     breakdown_by_resource_type: Dict[str, float]
53     top_cost_drivers: List[tuple]
54     optimization_opportunities: List[str]
55     period_start: datetime
56     period_end: datetime
57
58 class MLCostAnalyzer:
59     """Analyze and track ML system costs"""
60
61     def __init__(self):
62         self.cost_records: List[ResourceCost] = []
63         self.hourly_rates = {
64             # AWS pricing (example rates)
65             "ml.p3.2xlarge": 3.06, # V100 GPU
66             "ml.p4d.24xlarge": 32.77, # A100 GPU
67             "ml.g4dn.xlarge": 0.526, # T4 GPU
68             "ml.m5.xlarge": 0.192, # General compute
69             "ml.c5.2xlarge": 0.34, # Compute optimized
70             # Storage (per GB-month)
71             "s3_standard": 0.023,
72             "ebs_gp3": 0.08,
73             "efs": 0.30,
74         }
75
76     def add_cost_record(self, record: ResourceCost):
77         """Add a cost record"""
78         self.cost_records.append(record)
79
80     def track_training_run(
81         self,
82         job_id: str,
83         instance_type: str,
84         num_instances: int,
85         start_time: datetime,
86         end_time: datetime,
87         metadata: Optional[Dict] = None
88     ) -> ResourceCost:

```

```

89     """Track cost of a training run"""
90
91     duration_hours = (end_time - start_time).total_seconds() / 3600
92     hourly_cost = self.hourly_rates.get(instance_type, 0)
93     total_quantity = duration_hours * num_instances
94
95     record = ResourceCost(
96         resource_id=job_id,
97         resource_type="compute",
98         category=CostCategory.TRAINING,
99         cost=hourly_cost,
100        unit="per_hour",
101        quantity=total_quantity,
102        start_time=start_time,
103        end_time=end_time,
104        metadata=metadata or {}
105    )
106
107    self.add_cost_record(record)
108    return record
109
110    def track_serving_endpoint(
111        self,
112        endpoint_name: str,
113        instance_type: str,
114        num_instances: int,
115        requests: int,
116        start_time: datetime,
117        end_time: datetime
118    ) -> ResourceCost:
119        """Track cost of serving endpoint"""
120
121        duration_hours = (end_time - start_time).total_seconds() / 3600
122        hourly_cost = self.hourly_rates.get(instance_type, 0)
123        total_quantity = duration_hours * num_instances
124
125        record = ResourceCost(
126            resource_id=endpoint_name,
127            resource_type="compute",
128            category=CostCategory.SERVING,
129            cost=hourly_cost,
130            unit="per_hour",
131            quantity=total_quantity,
132            start_time=start_time,
133            end_time=end_time,
134            metadata={
135                "requests": requests,
136                "cost_per_request": (hourly_cost * total_quantity) / requests if
requests > 0 else 0
137            }
138        )
139
140        self.add_cost_record(record)
141        return record
142
143    def track_storage(
144        self,
145        storage_id: str,

```

```

146     storage_type: str,
147     size_gb: float,
148     duration_days: int,
149     category: CostCategory
150 ) -> ResourceCost:
151     """Track storage costs"""
152
153     monthly_cost_per_gb = self.hourly_rates.get(storage_type, 0)
154     quantity = size_gb * (duration_days / 30.0) # GB-months
155
156     start_time = datetime.now()
157     end_time = start_time + timedelta(days=duration_days)
158
159     record = ResourceCost(
160         resource_id=storage_id,
161         resource_type="storage",
162         category=category,
163         cost=monthly_cost_per_gb,
164         unit="per_gb_month",
165         quantity=quantity,
166         start_time=start_time,
167         end_time=end_time,
168         metadata={"size_gb": size_gb}
169     )
170
171     self.add_cost_record(record)
172     return record
173
174 def generate_cost_report(
175     self,
176     start_date: datetime,
177     end_date: datetime
178 ) -> CostReport:
179     """Generate comprehensive cost report"""
180
181     # Filter records by date range
182     filtered_records = [
183         r for r in self.cost_records
184         if r.start_time >= start_date and r.end_time <= end_date
185     ]
186
187     # Calculate total cost
188     total_cost = sum(r.total_cost for r in filtered_records)
189
190     # Breakdown by category
191     breakdown_by_category = {}
192     for category in CostCategory:
193         category_records = [r for r in filtered_records if r.category == category]
194         breakdown_by_category[category] = sum(r.total_cost for r in
category_records)
195
196     # Breakdown by resource type
197     breakdown_by_resource_type = {}
198     for record in filtered_records:
199         if record.resource_type not in breakdown_by_resource_type:
200             breakdown_by_resource_type[record.resource_type] = 0
201             breakdown_by_resource_type[record.resource_type] += record.total_cost
202

```

```

203     # Identify top cost drivers
204     cost_by_resource = {}
205     for record in filtered_records:
206         if record.resource_id not in cost_by_resource:
207             cost_by_resource[record.resource_id] = 0
208             cost_by_resource[record.resource_id] += record.total_cost
209
210     top_cost_drivers = sorted(
211         cost_by_resource.items(),
212         key=lambda x: x[1],
213         reverse=True
214     )[:10]
215
216     # Identify optimization opportunities
217     optimization_opportunities = self._identify_optimization_opportunities(
218         filtered_records
219     )
220
221     return CostReport(
222         total_cost=total_cost,
223         breakdown_by_category=breakdown_by_category,
224         breakdown_by_resource_type=breakdown_by_resource_type,
225         top_cost_drivers=top_cost_drivers,
226         optimization_opportunities=optimization_opportunities,
227         period_start=start_date,
228         period_end=end_date
229     )
230
231     def _identify_optimization_opportunities(
232         self,
233         records: List[ResourceCost]
234     ) -> List[str]:
235         """Identify cost optimization opportunities"""
236
237         opportunities = []
238
239         # Check for long-running training jobs
240         training_records = [r for r in records if r.category == CostCategory.TRAINING]
241         for record in training_records:
242             if record.duration_hours > 24:
243                 opportunities.append(
244                     f"Long training job {record.resource_id} ran for "
245                     f"{record.duration_hours:.1f} hours. Consider checkpointing "
246                     "and spot instances."
247                 )
248
249         # Check for underutilized serving endpoints
250         serving_records = [r for r in records if r.category == CostCategory.SERVING]
251         for record in serving_records:
252             requests = record.metadata.get("requests", 0)
253             if requests > 0:
254                 cost_per_1k_requests = (record.total_cost / requests) * 1000
255                 if cost_per_1k_requests > 1.0:
256                     opportunities.append(
257                         f"Serving endpoint {record.resource_id} has high cost per
258 request "
259                         f"(${cost_per_1k_requests:.2f} per 1K). Consider right-sizing
260 or serverless."

```

```

259         )
260
261     # Check for expensive storage
262     storage_records = [r for r in records if r.resource_type == "storage"]
263     for record in storage_records:
264         size_gb = record.metadata.get("size_gb", 0)
265         if size_gb > 1000 and record.cost > 0.1: # >1TB on expensive storage
266             opportunities.append(
267                 f"Large dataset {record.resource_id} ({size_gb:.0f} GB) on
expensive "
268                 "storage. Consider tiering to cheaper storage class."
269             )
270
271     return opportunities
272
273 def export_to_dataframe(self) -> pd.DataFrame:
274     """Export cost records to pandas DataFrame"""
275
276     data = []
277     for record in self.cost_records:
278         data.append({
279             "resource_id": record.resource_id,
280             "resource_type": record.resource_type,
281             "category": record.category.value,
282             "cost_per_unit": record.cost,
283             "quantity": record.quantity,
284             "total_cost": record.total_cost,
285             "start_time": record.start_time,
286             "end_time": record.end_time,
287             "duration_hours": record.duration_hours
288         })
289
290     return pd.DataFrame(data)
291
292 def calculate_monthly_projection(self, days_elapsed: int) -> float:
293     """Project monthly cost based on current spending"""
294
295     if days_elapsed == 0:
296         return 0
297
298     current_cost = sum(r.total_cost for r in self.cost_records)
299     daily_average = current_cost / days_elapsed
300     monthly_projection = daily_average * 30
301
302     return monthly_projection
303
304 # Example usage
305 if __name__ == "__main__":
306     analyzer = MLCostAnalyzer()
307
308     # Track training run
309     start = datetime.now() - timedelta(hours=5)
310     end = datetime.now()
311
312     analyzer.track_training_run(
313         job_id="bert-training-001",
314         instance_type="ml.p3.2xlarge",
315         num_instances=4,

```

```

316     start_time=start,
317     end_time=end,
318     metadata={"model": "BERT", "dataset": "wikipedia"}
319 )
320
321 # Track serving endpoint
322 analyzer.track_serving_endpoint(
323     endpoint_name="recommendation-api",
324     instance_type="ml.m5.xlarge",
325     num_instances=3,
326     requests=1500000,
327     start_time=start,
328     end_time=end
329 )
330
331 # Generate report
332 report = analyzer.generate_cost_report(
333     start_date=start - timedelta(days=30),
334     end_date=end
335 )
336
337 print(f"Total Cost: ${report.total_cost:.2f}")
338 print("\nCost Breakdown:")
339 for category, cost in report.breakdown_by_category.items():
340     if cost > 0:
341         percentage = (cost / report.total_cost) * 100
342         print(f"    {category.value}: ${cost:.2f} ({percentage:.1f}%)"
343
344 print("\nOptimization Opportunities:")
345 for opportunity in report.optimization_opportunities:
346     print(f"    - {opportunity}")

```

Listing 22.1: ML System Cost Analyzer

22.4 Training Cost Optimization

Training costs often dominate ML budgets, especially for large models and extensive hyperparameter searches.

22.4.1 Training Infrastructure Optimization

Best Practice: *Profile training jobs to identify compute bottlenecks before scaling to expensive hardware. A poorly optimized training loop won't benefit from larger instances.*

Key Training Cost Optimization Strategies:

1. **Right-size Compute Resources:** Match instance types to workload requirements
2. **Use Mixed Precision Training:** Reduce memory and increase throughput with FP16/BF16
3. **Implement Gradient Accumulation:** Train larger models on smaller instances

4. **Optimize Data Loading:** Eliminate I/O bottlenecks that waste GPU cycles
5. **Use Spot Instances:** Save 70-90% on compute for fault-tolerant workloads
6. **Implement Early Stopping:** Don't waste resources on unpromising runs
7. **Cache Preprocessed Data:** Avoid redundant data transformations

22.4.2 Training Cost Optimizer Implementation

```

1  #!/usr/bin/env python3
2  """
3  Training Cost Optimization
4  Strategies and tools for reducing ML training costs
5  """
6
7  import torch
8  import torch.nn as nn
9  import time
10 from typing import Optional, Dict, List, Tuple
11 from dataclasses import dataclass
12 import psutil
13 import GPUUtil
14
15 @dataclass
16 class TrainingMetrics:
17     """Training performance metrics"""
18     samples_per_second: float
19     gpu_utilization: float
20     memory_utilization: float
21     cost_per_hour: float
22     estimated_total_cost: float
23     time_to_completion_hours: float
24
25     @property
26     def cost_per_1k_samples(self) -> float:
27         """Calculate cost per 1000 training samples"""
28         if self.samples_per_second == 0:
29             return float('inf')
30         samples_per_hour = self.samples_per_second * 3600
31         return (self.cost_per_hour / samples_per_hour) * 1000
32
33 class TrainingCostOptimizer:
34     """Optimize training costs through various strategies"""
35
36     def __init__(
37         self,
38         model: nn.Module,
39         cost_per_hour: float,
40         target_samples: int
41     ):
42         self.model = model
43         self.cost_per_hour = cost_per_hour
44         self.target_samples = target_samples
45         self.training_start_time = None
46         self.samples_processed = 0
47

```

```

48 def profile_training_step(
49     self,
50     batch_size: int,
51     num_steps: int = 100
52 ) -> TrainingMetrics:
53     """Profile training performance and cost"""
54
55     device = next(self.model.parameters()).device
56     dummy_input = torch.randn(batch_size, 3, 224, 224).to(device)
57     dummy_target = torch.randint(0, 1000, (batch_size,)).to(device)
58
59     # Warmup
60     for _ in range(10):
61         output = self.model(dummy_input)
62         loss = nn.functional.cross_entropy(output, dummy_target)
63         loss.backward()
64
65     # Profile
66     start_time = time.time()
67
68     for _ in range(num_steps):
69         output = self.model(dummy_input)
70         loss = nn.functional.cross_entropy(output, dummy_target)
71         loss.backward()
72
73     torch.cuda.synchronize() if torch.cuda.is_available() else None
74     elapsed_time = time.time() - start_time
75
76     # Calculate metrics
77     samples_per_second = (num_steps * batch_size) / elapsed_time
78
79     # Get GPU utilization
80     gpu_utilization = 0
81     memory_utilization = 0
82     if torch.cuda.is_available():
83         gpus = GPUUtil.getGPUs()
84         if gpus:
85             gpu_utilization = gpus[0].load * 100
86             memory_utilization = gpus[0].memoryUtil * 100
87
88     # Cost calculations
89     remaining_samples = self.target_samples - self.samples_processed
90     time_to_completion_hours = remaining_samples / (samples_per_second * 3600)
91     estimated_total_cost = time_to_completion_hours * self.cost_per_hour
92
93     return TrainingMetrics(
94         samples_per_second=samples_per_second,
95         gpu_utilization=gpu_utilization,
96         memory_utilization=memory_utilization,
97         cost_per_hour=self.cost_per_hour,
98         estimated_total_cost=estimated_total_cost,
99         time_to_completion_hours=time_to_completion_hours
100     )
101
102 def find_optimal_batch_size(
103     self,
104     min_batch_size: int = 8,
105     max_batch_size: int = 512

```



```

106 ) -> Tuple[int, TrainingMetrics]:
107     """Find batch size that maximizes throughput per dollar"""
108
109     best_batch_size = min_batch_size
110     best_cost_efficiency = float('inf')
111     best_metrics = None
112
113     batch_size = min_batch_size
114     while batch_size <= max_batch_size:
115         try:
116             metrics = self.profile_training_step(batch_size)
117
118             # Cost efficiency: dollars per 1000 samples
119             cost_efficiency = metrics.cost_per_1k_samples
120
121             if cost_efficiency < best_cost_efficiency:
122                 best_cost_efficiency = cost_efficiency
123                 best_batch_size = batch_size
124                 best_metrics = metrics
125
126             batch_size *= 2
127
128         except RuntimeError as e:
129             # OOM error, stop increasing batch size
130             if "out of memory" in str(e):
131                 break
132             raise
133
134     return best_batch_size, best_metrics
135
136 def enable_mixed_precision(self) -> Dict[str, float]:
137     """Enable mixed precision training for cost savings"""
138
139     # Profile FP32
140     metrics_fp32 = self.profile_training_step(batch_size=32)
141
142     # Enable mixed precision
143     scaler = torch.cuda.amp.GradScaler()
144
145     # Profile with mixed precision
146     device = next(self.model.parameters()).device
147     dummy_input = torch.randn(32, 3, 224, 224).to(device)
148     dummy_target = torch.randint(0, 1000, (32,)).to(device)
149
150     start_time = time.time()
151     for _ in range(100):
152         with torch.cuda.amp.autocast():
153             output = self.model(dummy_input)
154             loss = nn.functional.cross_entropy(output, dummy_target)
155
156             scaler.scale(loss).backward()
157             scaler.step(torch.optim.SGD(self.model.parameters(), lr=0.01))
158             scaler.update()
159
160     torch.cuda.synchronize() if torch.cuda.is_available() else None
161     elapsed_time = time.time() - start_time
162
163     samples_per_second_fp16 = (100 * 32) / elapsed_time

```

```

164
165     # Calculate speedup and cost savings
166     speedup = samples_per_second_fp16 / metrics_fp32.samples_per_second
167     cost_reduction = (1 - (1 / speedup)) * 100
168
169     return {
170         "speedup": speedup,
171         "cost_reduction_percent": cost_reduction,
172         "fp32_samples_per_sec": metrics_fp32.samples_per_second,
173         "fp16_samples_per_sec": samples_per_second_fp16
174     }
175
176     def implement_gradient_accumulation(
177         self,
178         target_batch_size: int,
179         available_memory_batch_size: int
180     ) -> Dict[str, any]:
181         """Use gradient accumulation to simulate larger batches"""
182
183         accumulation_steps = target_batch_size // available_memory_batch_size
184
185         # This allows training with larger effective batch sizes
186         # on smaller, cheaper instances
187
188         # Cost comparison
189         large_instance_cost = self.cost_per_hour * 2 # Assume 2x cost
190         small_instance_time_multiplier = 1.1 # Slight overhead from accumulation
191
192         large_instance_total = large_instance_cost
193         small_instance_total = self.cost_per_hour * small_instance_time_multiplier
194
195         cost_savings = ((large_instance_total - small_instance_total) /
196                         large_instance_total) * 100
197
198         return {
199             "accumulation_steps": accumulation_steps,
200             "effective_batch_size": target_batch_size,
201             "physical_batch_size": available_memory_batch_size,
202             "estimated_cost_savings_percent": cost_savings
203         }
204
205     def calculate_early_stopping_savings(
206         self,
207         total_epochs: int,
208         early_stop_epoch: int,
209         num_runs: int
210     ) -> Dict[str, float]:
211         """Calculate cost savings from early stopping"""
212
213         # Without early stopping
214         cost_per_epoch = self.cost_per_hour * (self.target_samples /
215                                                (self.profile_training_step(32).samples_per_second * 3600))
216         total_cost_without = cost_per_epoch * total_epochs * num_runs
217
218         # With early stopping (average case)
219         average_stop_epoch = (early_stop_epoch + total_epochs) / 2
220         total_cost_with = cost_per_epoch * average_stop_epoch * num_runs
221

```

```

222     savings = total_cost_without - total_cost_with
223     savings_percent = (savings / total_cost_without) * 100
224
225     return {
226         "cost_without_early_stopping": total_cost_without,
227         "cost_with_early_stopping": total_cost_with,
228         "savings_dollars": savings,
229         "savings_percent": savings_percent
230     }
231
232 class HyperparameterSearchCostOptimizer:
233     """Optimize costs for hyperparameter search"""
234
235     def __init__(self, cost_per_hour: float):
236         self.cost_per_hour = cost_per_hour
237
238     def compare_search_strategies(
239         self,
240         num_hyperparameters: int,
241         values_per_param: int,
242         max_trials: int,
243         avg_trial_hours: float
244     ) -> Dict[str, Dict[str, float]]:
245         """Compare cost of different search strategies"""
246
247         # Grid Search
248         grid_search_trials = values_per_param ** num_hyperparameters
249         grid_search_cost = min(grid_search_trials, max_trials) * avg_trial_hours *
self.cost_per_hour
250
251         # Random Search
252         random_search_trials = max_trials
253         random_search_cost = random_search_trials * avg_trial_hours * self.
cost_per_hour
254
255         # Bayesian Optimization (typically needs fewer trials)
256         bayesian_trials = max_trials // 2
257         bayesian_cost = bayesian_trials * avg_trial_hours * self.cost_per_hour
258
259         # Hyperband (early stopping poor configurations)
260         hyperband_trials = max_trials
261         hyperband_avg_hours = avg_trial_hours * 0.3 # Many trials stopped early
262         hyperband_cost = hyperband_trials * hyperband_avg_hours * self.cost_per_hour
263
264     return {
265         "grid_search": {
266             "trials": grid_search_trials,
267             "cost": grid_search_cost,
268             "hours": grid_search_trials * avg_trial_hours
269         },
270         "random_search": {
271             "trials": random_search_trials,
272             "cost": random_search_cost,
273             "hours": random_search_trials * avg_trial_hours
274         },
275         "bayesian_optimization": {
276             "trials": bayesian_trials,
277             "cost": bayesian_cost,

```

```

278         "hours": bayesian_trials * avg_trial_hours
279     },
280     "hyperband": {
281         "trials": hyperband_trials,
282         "cost": hyperband_cost,
283         "hours": hyperband_trials * hyperband_avg_hours
284     }
285 }
286
287 def calculate_sequential_halving_savings(
288     self,
289     initial_configurations: int,
290     max_budget_per_config: float,
291     halving_rounds: int
292 ) -> Dict[str, float]:
293     """Calculate savings from successive halving"""
294
295     # Full training cost
296     full_cost = initial_configurations * max_budget_per_config * self.
cost_per_hour
297
298     # Successive halving cost
299     halving_cost = 0
300     configs = initial_configurations
301     budget = max_budget_per_config / (2 ** (halving_rounds - 1))
302
303     for round_num in range(halving_rounds):
304         halving_cost += configs * budget * self.cost_per_hour
305         configs = configs // 2
306         budget = budget * 2
307
308     savings = full_cost - halving_cost
309     savings_percent = (savings / full_cost) * 100
310
311     return {
312         "full_training_cost": full_cost,
313         "successive_halving_cost": halving_cost,
314         "savings_dollars": savings,
315         "savings_percent": savings_percent
316     }
317
318 # Example usage
319 if __name__ == "__main__":
320     # Create dummy model
321     model = torch.nn.Sequential(
322         torch.nn.Linear(100, 256),
323         torch.nn.ReLU(),
324         torch.nn.Linear(256, 10)
325     )
326
327     if torch.cuda.is_available():
328         model = model.cuda()
329
330     # Initialize optimizer
331     optimizer = TrainingCostOptimizer(
332         model=model,
333         cost_per_hour=3.06, # ml.p3.2xlarge
334         target_samples=1000000

```

```

335 )
336
337 # Find optimal batch size
338 optimal_batch, metrics = optimizer.find_optimal_batch_size()
339 print(f"Optimal batch size: {optimal_batch}")
340 print(f"Cost per 1K samples: ${metrics.cost_per_1k_samples:.4f}")
341 print(f"Estimated total cost: ${metrics.estimated_total_cost:.2f}")
342
343 # Analyze mixed precision savings
344 mp_savings = optimizer.enable_mixed_precision()
345 print(f"\nMixed Precision Speedup: {mp_savings['speedup']:.2f}x")
346 print(f"Cost Reduction: {mp_savings['cost_reduction_percent']:.1f}%")

```

Listing 22.2: Training Cost Optimization Framework

22.4.3 Spot Instance Strategy for Training

Spot instances can reduce training costs by 70-90%, but require fault-tolerance mechanisms.

```

1  #!/usr/bin/env python3
2  """
3  Spot Instance Training Management
4  Handle interruptions and optimize costs with spot instances
5  """
6
7  import boto3
8  import time
9  import json
10 from datetime import datetime
11 from typing import Optional, Dict, List
12 from dataclasses import dataclass
13 import threading
14
15 @dataclass
16 class SpotInstanceConfig:
17     """Spot instance configuration"""
18     instance_type: str
19     max_price: float # Maximum price per hour
20     availability_zones: List[str]
21     interruption_behavior: str = "terminate" # or "stop", "hibernate"
22
23 @dataclass
24 class CheckpointConfig:
25     """Checkpointing configuration"""
26     checkpoint_interval_minutes: int
27     s3_bucket: str
28     checkpoint_path: str
29     max_checkpoints_to_keep: int = 3
30
31 class SpotInstanceTrainingManager:
32     """Manage training on spot instances with fault tolerance"""
33
34     def __init__(
35         self,
36         spot_config: SpotInstanceConfig,
37         checkpoint_config: CheckpointConfig,

```

```

38     aws_region: str = "us-east-1"
39 ):
40     self.spot_config = spot_config
41     self.checkpoint_config = checkpoint_config
42     self.ec2_client = boto3.client('ec2', region_name=aws_region)
43     self.s3_client = boto3.client('s3', region_name=aws_region)
44     self.instance_id: Optional[str] = None
45     self.interruption_monitor_thread: Optional[threading.Thread] = None
46     self.interrupted = False
47
48     def request_spot_instance(self) -> str:
49         """Request a spot instance"""
50
51         # Get current spot price
52         current_price = self._get_current_spot_price()
53
54         if current_price > self.spot_config.max_price:
55             raise ValueError(
56                 f"Current spot price (${current_price:.3f}) exceeds "
57                 f"max price (${self.spot_config.max_price:.3f})"
58             )
59
60         # Create launch specification
61         launch_specification = {
62             'ImageId': 'ami-0c55b159cbfafa1f0', # Deep Learning AMI
63             'InstanceType': self.spot_config.instance_type,
64             'KeyName': 'your-key-pair',
65             'SecurityGroups': ['ml-training-sg'],
66             'UserData': self._generate_user_data(),
67             'IamInstanceProfile': {
68                 'Name': 'ml-training-role'
69             },
70             'BlockDeviceMappings': [
71                 {
72                     'DeviceName': '/dev/sda1',
73                     'Ebs': {
74                         'VolumeSize': 100,
75                         'VolumeType': 'gp3'
76                     }
77                 }
78             ]
79         }
80
81         # Request spot instance
82         response = self.ec2_client.request_spot_instances(
83             SpotPrice=str(self.spot_config.max_price),
84             InstanceCount=1,
85             Type='one-time',
86             LaunchSpecification=launch_specification,
87             InstanceInterruptionBehavior=self.spot_config.interruption_behavior
88         )
89
90         request_id = response['SpotInstanceRequests'][0]['SpotInstanceRequestId']
91
92         # Wait for instance to be running
93         self.instance_id = self._wait_for_spot_instance(request_id)
94
95         # Start monitoring for interruptions

```

```

96         self._start_interruption_monitoring()
97
98         return self.instance_id
99
100     def _get_current_spot_price(self) -> float:
101         """Get current spot price for instance type"""
102
103         response = self.ec2_client.describe_spot_price_history(
104             InstanceTypes=[self.spot_config.instance_type],
105             MaxResults=1,
106             ProductDescriptions=['Linux/UNIX'],
107             AvailabilityZone=self.spot_config.availability_zones[0]
108         )
109
110         if response['SpotPriceHistory']:
111             return float(response['SpotPriceHistory'][0]['SpotPrice'])
112         return float('inf')
113
114     def _wait_for_spot_instance(self, request_id: str, timeout: int = 300) -> str:
115         """Wait for spot instance to be fulfilled"""
116
117         start_time = time.time()
118
119         while time.time() - start_time < timeout:
120             response = self.ec2_client.describe_spot_instance_requests(
121                 SpotInstanceRequestIds=[request_id]
122             )
123
124             request = response['SpotInstanceRequests'][0]
125             state = request['State']
126
127             if state == 'active':
128                 return request['InstanceId']
129             elif state in ['closed', 'cancelled', 'failed']:
130                 raise RuntimeError(f"Spot request failed: {request.get('Fault', {})}")
131
132             time.sleep(10)
133
134         raise TimeoutError("Spot instance request timed out")
135
136     def _start_interruption_monitoring(self):
137         """Start monitoring for spot interruption notices"""
138
139         def monitor():
140             # Check EC2 instance metadata for interruption notice
141             # This would run on the instance itself
142             while not self.interrupted:
143                 try:
144                     # Check for interruption notice
145                     # (simplified - actual implementation would check instance
146                     metadata)
147                     time.sleep(5)
148                 except Exception:
149                     pass
150
151         self.interruption_monitor_thread = threading.Thread(target=monitor)
152         self.interruption_monitor_thread.daemon = True
153         self.interruption_monitor_thread.start()

```

```

153
154 def save_checkpoint(
155     self,
156     model_state: Dict,
157     optimizer_state: Dict,
158     epoch: int,
159     global_step: int,
160     metadata: Optional[Dict] = None
161 ):
162     """Save training checkpoint to S3"""
163
164     checkpoint = {
165         'model_state_dict': model_state,
166         'optimizer_state_dict': optimizer_state,
167         'epoch': epoch,
168         'global_step': global_step,
169         'timestamp': datetime.now().isoformat(),
170         'metadata': metadata or {}
171     }
172
173     # Save locally first
174     local_path = f'/tmp/checkpoint_{global_step}.pt'
175     import torch
176     torch.save(checkpoint, local_path)
177
178     # Upload to S3
179     s3_key = f"{self.checkpoint_config.checkpoint_path}/checkpoint_{global_step}.
pt"
180
181     self.s3_client.upload_file(
182         local_path,
183         self.checkpoint_config.s3_bucket,
184         s3_key
185     )
186
187     print(f"Checkpoint saved to s3://{self.checkpoint_config.s3_bucket}/{s3_key}")
188
189     # Clean up old checkpoints
190     self._cleanup_old_checkpoints()
191
192 def load_latest_checkpoint(self) -> Optional[Dict]:
193     """Load the most recent checkpoint from S3"""
194
195     # List checkpoints
196     response = self.s3_client.list_objects_v2(
197         Bucket=self.checkpoint_config.s3_bucket,
198         Prefix=self.checkpoint_config.checkpoint_path
199     )
200
201     if 'Contents' not in response:
202         return None
203
204     # Sort by last modified
205     checkpoints = sorted(
206         response['Contents'],
207         key=lambda x: x['LastModified'],
208         reverse=True
209     )

```



```

210         if not checkpoints:
211             return None
212
213         # Download latest checkpoint
214         latest_key = checkpoints[0]['Key']
215         local_path = '/tmp/latest_checkpoint.pt'
216
217         self.s3_client.download_file(
218             self.checkpoint_config.s3_bucket,
219             latest_key,
220             local_path
221         )
222
223         import torch
224         checkpoint = torch.load(local_path)
225
226         print(f"Loaded checkpoint from epoch {checkpoint['epoch']}, "
227               f"step {checkpoint['global_step']}")
228
229         return checkpoint
230
231     def _cleanup_old_checkpoints(self):
232         """Remove old checkpoints to save storage costs"""
233
234         response = self.s3_client.list_objects_v2(
235             Bucket=self.checkpoint_config.s3_bucket,
236             Prefix=self.checkpoint_config.checkpoint_path
237         )
238
239         if 'Contents' not in response:
240             return
241
242         checkpoints = sorted(
243             response['Contents'],
244             key=lambda x: x['LastModified'],
245             reverse=True
246         )
247
248         # Delete old checkpoints
249         checkpoints_to_delete = checkpoints[self.checkpoint_config.
250 max_checkpoints_to_keep:]
251
252         for checkpoint in checkpoints_to_delete:
253             self.s3_client.delete_object(
254                 Bucket=self.checkpoint_config.s3_bucket,
255                 Key=checkpoint['Key']
256             )
257
258     def _generate_user_data(self) -> str:
259         """Generate user data script for instance startup"""
260
261         script = f"""#!/bin/bash
262 # Download training code and resume from checkpoint
263 aws s3 cp s3://{self.checkpoint_config.s3_bucket}/training_code/ /home/ubuntu/training
264 / --recursive
265 # Start training with automatic checkpoint resumption

```

```

266 cd /home/ubuntu/training
267 python train.py --resume-from-checkpoint
268 """
269     return script
270
271 def calculate_cost_savings(
272     self,
273     training_hours: float,
274     on_demand_price: float,
275     interruption_overhead_percent: float = 5.0
276 ) -> Dict[str, float]:
277     """Calculate cost savings from using spot instances"""
278
279     # On-demand cost
280     on_demand_cost = training_hours * on_demand_price
281
282     # Spot cost (including interruption overhead)
283     effective_training_hours = training_hours * (1 + interruption_overhead_percent
284 / 100)
285     spot_price = self._get_current_spot_price()
286     spot_cost = effective_training_hours * spot_price
287
288     # Savings
289     savings = on_demand_cost - spot_cost
290     savings_percent = (savings / on_demand_cost) * 100
291
292     return {
293         "on_demand_cost": on_demand_cost,
294         "spot_cost": spot_cost,
295         "savings_dollars": savings,
296         "savings_percent": savings_percent,
297         "spot_price": spot_price,
298         "on_demand_price": on_demand_price
299     }
300
301 # Example usage
302 if __name__ == "__main__":
303     spot_config = SpotInstanceConfig(
304         instance_type="ml.p3.2xlarge",
305         max_price=1.50, # 50% of on-demand price
306         availability_zones=["us-east-1a", "us-east-1b"]
307     )
308
309     checkpoint_config = CheckpointConfig(
310         checkpoint_interval_minutes=15,
311         s3_bucket="my-ml-checkpoints",
312         checkpoint_path="training/resnet50"
313     )
314
315     manager = SpotInstanceTrainingManager(spot_config, checkpoint_config)
316
317     # Calculate potential savings
318     savings = manager.calculate_cost_savings(
319         training_hours=48,
320         on_demand_price=3.06
321     )
322
323     print(f"Training on spot instances:")

```

```

323 print(f" On-demand cost: ${savings['on_demand_cost']:.2f}")
324 print(f" Spot cost: ${savings['spot_cost']:.2f}")
325 print(f" Savings: ${savings['savings_dollars']:.2f} ({savings['savings_percent']:.1f}%)" )

```

Listing 22.3: Spot Instance Training Manager

22.5 Serving Cost Analysis and Reduction

Serving costs scale with traffic and can exceed training costs for high-traffic applications.

22.5.1 Serving Cost Optimization Strategies

Key Serving Cost Drivers:

- **Compute Resources:** Instance types and count for model hosting
- **Request Volume:** Number of inference requests
- **Model Complexity:** Inference latency and resource requirements
- **Always-On Infrastructure:** Fixed costs vs. usage-based costs
- **Data Transfer:** Network egress charges

```

1  #!/usr/bin/env python3
2  """
3  Serving Cost Optimization
4  Analyze and optimize model serving infrastructure costs
5  """
6
7  from dataclasses import dataclass
8  from typing import Dict, List, Optional
9  import numpy as np
10
11  @dataclass
12  class ServingConfig:
13      """Model serving configuration"""
14      instance_type: str
15      num_instances: int
16      requests_per_second: float
17      avg_latency_ms: float
18      cost_per_hour: float
19
20      @property
21      def total_hourly_cost(self) -> float:
22          return self.cost_per_hour * self.num_instances
23
24      @property
25      def cost_per_1m_requests(self) -> float:
26          requests_per_hour = self.requests_per_second * 3600
27          if requests_per_hour == 0:
28              return float('inf')
29          return (self.total_hourly_cost / requests_per_hour) * 1_000_000

```

```

30
31 @dataclass
32 class ServerlessConfig:
33     """Serverless serving configuration"""
34     cost_per_request: float
35     cost_per_gb_second: float
36     avg_memory_gb: float
37     avg_duration_seconds: float
38
39     def cost_per_1m_requests(self) -> float:
40         compute_cost = self.cost_per_gb_second * self.avg_memory_gb * self.
41         avg_duration_seconds
42         total_cost_per_request = self.cost_per_request + compute_cost
43         return total_cost_per_request * 1_000_000
44
45 class ServingCostOptimizer:
46     """Optimize model serving costs"""
47
48     def __init__(self):
49         self.pricing = {
50             # AWS pricing (example)
51             "ml.m5.xlarge": 0.269,
52             "ml.m5.2xlarge": 0.538,
53             "ml.c5.xlarge": 0.238,
54             "ml.c5.2xlarge": 0.476,
55             "ml.g4dn.xlarge": 0.736, # With GPU
56             "ml.inf1.xlarge": 0.368, # AWS Inferentia
57         }
58
59     def compare_deployment_options(
60         self,
61         daily_requests: int,
62         avg_latency_ms: float,
63         p99_latency_ms: float,
64         peak_rps: float,
65         avg_rps: float
66     ) -> Dict[str, Dict]:
67         """Compare different deployment strategies"""
68
69         results = {}
70
71         # Option 1: Always-on instances sized for peak
72         peak_instances = self._calculate_required_instances(
73             peak_rps,
74             instance_type="ml.m5.xlarge",
75             requests_per_instance=100
76         )
77
78         results["always_on_peak"] = {
79             "instance_type": "ml.m5.xlarge",
80             "num_instances": peak_instances,
81             "monthly_cost": peak_instances * self.pricing["ml.m5.xlarge"] * 24 * 30,
82             "cost_per_1m_requests": self._calculate_cost_per_1m_requests(
83                 peak_instances,
84                 self.pricing["ml.m5.xlarge"],
85                 daily_requests
86             ),
87             "description": "Always-on instances sized for peak traffic"

```

```

87     }
88
89     # Option 2: Always-on for baseline + auto-scaling
90     baseline_instances = self._calculate_required_instances(
91         avg_rps,
92         instance_type="ml.m5.xlarge",
93         requests_per_instance=100
94     )
95
96     # Assume auto-scaling covers 20% of time at peak
97     auto_scale_instance_hours = (peak_instances - baseline_instances) * 24 * 30 *
0.2
98     baseline_instance_hours = baseline_instances * 24 * 30
99
100     results["autoscaling"] = {
101         "baseline_instances": baseline_instances,
102         "peak_instances": peak_instances,
103         "monthly_cost": (
104             (baseline_instance_hours + auto_scale_instance_hours) *
105             self.pricing["ml.m5.xlarge"]
106         ),
107         "cost_per_1m_requests": self._calculate_cost_per_1m_requests(
108             baseline_instances,
109             self.pricing["ml.m5.xlarge"],
110             daily_requests,
111             autoscaling=True
112         ),
113         "description": "Baseline instances + auto-scaling"
114     }
115
116     # Option 3: Serverless (AWS Lambda / SageMaker Serverless)
117     # Lambda pricing: $0.20 per 1M requests + compute time
118     avg_duration_seconds = avg_latency_ms / 1000
119     memory_gb = 2.0 # Example memory allocation
120
121     compute_cost_per_request = 0.0000166667 * memory_gb * avg_duration_seconds
122     request_cost = 0.20 / 1_000_000
123     total_cost_per_request = compute_cost_per_request + request_cost
124
125     results["serverless"] = {
126         "monthly_cost": total_cost_per_request * daily_requests * 30,
127         "cost_per_1m_requests": total_cost_per_request * 1_000_000,
128         "memory_gb": memory_gb,
129         "avg_duration_seconds": avg_duration_seconds,
130         "description": "Serverless compute (pay per request)"
131     }
132
133     # Option 4: Optimized instances (Inferentia/GPU)
134     opt_instances = self._calculate_required_instances(
135         peak_rps,
136         instance_type="ml.inf1.xlarge",
137         requests_per_instance=200 # Better throughput with optimization
138     )
139
140     results["optimized_hardware"] = {
141         "instance_type": "ml.inf1.xlarge",
142         "num_instances": opt_instances,
143         "monthly_cost": opt_instances * self.pricing["ml.inf1.xlarge"] * 24 * 30,

```

```

144         "cost_per_1m_requests": self._calculate_cost_per_1m_requests(
145             opt_instances,
146             self.pricing["ml.inf1.xlarge"],
147             daily_requests
148         ),
149         "description": "Optimized hardware (Inferentia)"
150     }
151
152     return results
153
154 def _calculate_required_instances(
155     self,
156     target_rps: float,
157     instance_type: str,
158     requests_per_instance: float
159 ) -> int:
160     """Calculate number of instances needed"""
161     # Add 20% headroom for spikes
162     required = int(np.ceil(target_rps / requests_per_instance * 1.2))
163     return max(required, 1)
164
165 def _calculate_cost_per_1m_requests(
166     self,
167     num_instances: int,
168     cost_per_hour: float,
169     daily_requests: int,
170     autoscaling: bool = False
171 ) -> float:
172     """Calculate cost per million requests"""
173     monthly_requests = daily_requests * 30
174     monthly_instance_hours = num_instances * 24 * 30
175
176     if autoscaling:
177         monthly_instance_hours *= 0.7 # Assume 30% reduction with autoscaling
178
179     monthly_cost = monthly_instance_hours * cost_per_hour
180
181     if monthly_requests == 0:
182         return float('inf')
183
184     return (monthly_cost / monthly_requests) * 1_000_000
185
186 def optimize_batch_size(
187     self,
188     model_latency_by_batch: Dict[int, float],
189     cost_per_hour: float,
190     target_p99_latency_ms: float
191 ) -> Dict:
192     """Find optimal batch size for serving"""
193
194     best_batch_size = 1
195     best_cost_per_1k_requests = float('inf')
196
197     results = {}
198
199     for batch_size, latency_ms in model_latency_by_batch.items():
200         if latency_ms > target_p99_latency_ms:
201             continue # Violates latency SLA

```

```

202     # Throughput (requests per hour)
203     requests_per_second = batch_size / (latency_ms / 1000)
204     requests_per_hour = requests_per_second * 3600
205
206     # Cost per 1K requests
207     cost_per_1k = (cost_per_hour / requests_per_hour) * 1000
208
209     results[batch_size] = {
210         "latency_ms": latency_ms,
211         "requests_per_second": requests_per_second,
212         "cost_per_1k_requests": cost_per_1k
213     }
214
215     if cost_per_1k < best_cost_per_1k_requests:
216         best_cost_per_1k_requests = cost_per_1k
217         best_batch_size = batch_size
218
219     return {
220         "optimal_batch_size": best_batch_size,
221         "cost_per_1k_requests": best_cost_per_1k_requests,
222         "all_results": results
223     }
224
225 def calculate_model_compression_savings(
226     self,
227     original_latency_ms: float,
228     compressed_latency_ms: float,
229     original_memory_gb: float,
230     compressed_memory_gb: float,
231     daily_requests: int
232 ) -> Dict:
233     """Calculate savings from model compression"""
234
235     # Original deployment
236     original_instances = max(1, int(np.ceil(original_memory_gb / 8))) # 8GB per
237 instance
238     original_cost_per_hour = self.pricing["ml.m5.xlarge"] * original_instances
239
240     # Compressed deployment
241     compressed_instances = max(1, int(np.ceil(compressed_memory_gb / 8)))
242     compressed_cost_per_hour = self.pricing["ml.m5.xlarge"] * compressed_instances
243
244     # If latency improved, might handle more load per instance
245     throughput_improvement = original_latency_ms / compressed_latency_ms
246
247     # Monthly costs
248     original_monthly = original_cost_per_hour * 24 * 30
249     compressed_monthly = (compressed_cost_per_hour / throughput_improvement) * 24
250     * 30
251
252     savings = original_monthly - compressed_monthly
253     savings_percent = (savings / original_monthly) * 100
254
255     return {
256         "original_monthly_cost": original_monthly,
257         "compressed_monthly_cost": compressed_monthly,
258         "savings_dollars": savings,

```

```

258         "savings_percent": savings_percent,
259         "latency_improvement_percent": ((original_latency_ms -
compressed_latency_ms) /
260                                         original_latency_ms) * 100,
261         "memory_reduction_percent": ((original_memory_gb - compressed_memory_gb) /
262                                         original_memory_gb) * 100
263     }
264
265     def analyze_caching_opportunity(
266         self,
267         total_daily_requests: int,
268         cache_hit_rate: float,
269         cached_request_cost: float,
270         uncached_request_cost: float
271     ) -> Dict:
272         """Calculate savings from response caching"""
273
274         cache_hits = total_daily_requests * cache_hit_rate
275         cache_misses = total_daily_requests * (1 - cache_hit_rate)
276
277         # Without caching
278         cost_without_cache = total_daily_requests * uncached_request_cost
279
280         # With caching
281         cost_with_cache = (
282             cache_hits * cached_request_cost +
283             cache_misses * uncached_request_cost
284         )
285
286         # Cache infrastructure cost (Redis/Memcached)
287         cache_infrastructure_cost_daily = 5.0 # Example: $5/day for cache
288
289         total_cost_with_cache = cost_with_cache + cache_infrastructure_cost_daily
290
291         savings = cost_without_cache - total_cost_with_cache
292         savings_percent = (savings / cost_without_cache) * 100
293
294         monthly_savings = savings * 30
295
296         return {
297             "cost_without_cache_monthly": cost_without_cache * 30,
298             "cost_with_cache_monthly": total_cost_with_cache * 30,
299             "savings_monthly": monthly_savings,
300             "savings_percent": savings_percent,
301             "cache_hit_rate": cache_hit_rate,
302             "break_even_hit_rate": self._calculate_break_even_hit_rate(
303                 uncached_request_cost,
304                 cached_request_cost,
305                 cache_infrastructure_cost_daily,
306                 total_daily_requests
307             )
308         }
309
310     def _calculate_break_even_hit_rate(
311         self,
312         uncached_cost: float,
313         cached_cost: float,
314         infrastructure_cost: float,

```



```

315     daily_requests: int
316 ) -> float:
317     """Calculate minimum cache hit rate to break even"""
318     # Solve: infrastructure_cost = (hit_rate * daily_requests * (uncached_cost -
319     cached_cost))
320     cost_saving_per_hit = uncached_cost - cached_cost
321     if cost_saving_per_hit <= 0:
322         return 1.0
323
324     break_even_rate = infrastructure_cost / (daily_requests * cost_saving_per_hit)
325     return min(break_even_rate, 1.0)
326
327 # Example usage
328 if __name__ == "__main__":
329     optimizer = ServingCostOptimizer()
330
331     # Compare deployment options
332     comparison = optimizer.compare_deployment_options(
333         daily_requests=10_000_000,
334         avg_latency_ms=50,
335         p99_latency_ms=200,
336         peak_rps=500,
337         avg_rps=115
338     )
339
340     print("Deployment Option Comparison:")
341     for option_name, details in comparison.items():
342         print(f"\n{option_name}:")
343         print(f"    Monthly Cost: ${details['monthly_cost']:.2f}")
344         print(f"    Cost per 1M requests: ${details['cost_per_1m_requests']:.2f}")
345         print(f"    Description: {details['description']}")
346
347     # Analyze caching
348     caching_analysis = optimizer.analyze_caching_opportunity(
349         total_daily_requests=10_000_000,
350         cache_hit_rate=0.60,
351         cached_request_cost=0.001,
352         uncached_request_cost=0.01
353     )
354
355     print(f"\nCaching Analysis:")
356     print(f"    Monthly savings: ${caching_analysis['savings_monthly']:.2f}")
357     print(f"    Savings percent: {caching_analysis['savings_percent']:.1f}%")
358     print(f"    Break-even hit rate: {caching_analysis['break_even_hit_rate']*100:.1f}%")
359 )

```

Listing 22.4: Serving Cost Analyzer and Optimizer

22.6 Resource Allocation and Scaling Policies

Intelligent scaling policies ensure you pay only for resources you need.

22.6.1 Autoscaling Implementation

```
1 #!/usr/bin/env python3
```

```

2  """
3  ML-Aware Autoscaling
4  Intelligent scaling policies for ML workloads
5  """
6
7  from dataclasses import dataclass
8  from typing import List, Optional, Callable
9  from datetime import datetime, timedelta
10 import numpy as np
11
12 @dataclass
13 class ScalingMetrics:
14     """Metrics for scaling decisions"""
15     timestamp: datetime
16     cpu_utilization: float
17     memory_utilization: float
18     requests_per_second: float
19     avg_latency_ms: float
20     p99_latency_ms: float
21     queue_depth: int
22
23 @dataclass
24 class ScalingPolicy:
25     """Autoscaling policy configuration"""
26     min_instances: int
27     max_instances: int
28     target_cpu_utilization: float = 70.0
29     target_latency_p99_ms: float = 200.0
30     scale_up_cooldown_seconds: int = 300
31     scale_down_cooldown_seconds: int = 600
32     scale_up_threshold_breaches: int = 2
33     scale_down_threshold_breaches: int = 3
34
35 class MLAutoscaler:
36     """ML-aware autoscaling engine"""
37
38     def __init__(self, policy: ScalingPolicy):
39         self.policy = policy
40         self.current_instances = policy.min_instances
41         self.metrics_history: List[ScalingMetrics] = []
42         self.last_scale_up_time: Optional[datetime] = None
43         self.last_scale_down_time: Optional[datetime] = None
44         self.consecutive_scale_up_signals = 0
45         self.consecutive_scale_down_signals = 0
46
47     def add_metrics(self, metrics: ScalingMetrics):
48         """Add new metrics for scaling decision"""
49         self.metrics_history.append(metrics)
50
51         # Keep only last hour of metrics
52         cutoff_time = datetime.now() - timedelta(hours=1)
53         self.metrics_history = [
54             m for m in self.metrics_history
55             if m.timestamp > cutoff_time
56         ]
57
58     def should_scale_up(self, metrics: ScalingMetrics) -> bool:
59         """Determine if should scale up"""

```

```

60
61     if self.current_instances >= self.policy.max_instances:
62         return False
63
64     # Check cooldown
65     if self.last_scale_up_time:
66         elapsed = (datetime.now() - self.last_scale_up_time).total_seconds()
67         if elapsed < self.policy.scale_up_cooldown_seconds:
68             return False
69
70     # Check thresholds
71     should_scale = (
72         metrics.cpu_utilization > self.policy.target_cpu_utilization or
73         metrics.p99_latency_ms > self.policy.target_latency_p99_ms or
74         metrics.queue_depth > 100
75     )
76
77     if should_scale:
78         self.consecutive_scale_up_signals += 1
79         self.consecutive_scale_down_signals = 0
80     else:
81         self.consecutive_scale_up_signals = 0
82
83     # Require multiple consecutive signals
84     return self.consecutive_scale_up_signals >= self.policy.
scale_up_threshold_breaches
85
86 def should_scale_down(self, metrics: ScalingMetrics) -> bool:
87     """Determine if should scale down"""
88
89     if self.current_instances <= self.policy.min_instances:
90         return False
91
92     # Check cooldown
93     if self.last_scale_down_time:
94         elapsed = (datetime.now() - self.last_scale_down_time).total_seconds()
95         if elapsed < self.policy.scale_down_cooldown_seconds:
96             return False
97
98     # More conservative for scale down
99     should_scale = (
100         metrics.cpu_utilization < self.policy.target_cpu_utilization * 0.5 and
101         metrics.p99_latency_ms < self.policy.target_latency_p99_ms * 0.7 and
102         metrics.queue_depth == 0
103     )
104
105     if should_scale:
106         self.consecutive_scale_down_signals += 1
107         self.consecutive_scale_up_signals = 0
108     else:
109         self.consecutive_scale_down_signals = 0
110
111     return self.consecutive_scale_down_signals >= self.policy.
scale_down_threshold_breaches
112
113 def calculate_target_instances(self, metrics: ScalingMetrics) -> int:
114     """Calculate target number of instances"""
115

```

```

116     # Calculate based on different metrics
117     cpu_based = int(np.ceil(
118         self.current_instances *
119         (metrics.cpu_utilization / self.policy.target_cpu_utilization)
120     ))
121
122     # Estimate instances needed for latency SLA
123     # Simple heuristic: if latency is high, we need more capacity
124     latency_ratio = metrics.p99_latency_ms / self.policy.target_latency_p99_ms
125     latency_based = int(np.ceil(self.current_instances * latency_ratio))
126
127     # Take the maximum
128     target = max(cpu_based, latency_based)
129
130     # Constrain to policy limits
131     target = max(self.policy.min_instances, min(target, self.policy.max_instances)
132 )
133
134     return target
135
136 def execute_scaling_decision(self, metrics: ScalingMetrics) -> Optional[int]:
137     """Make and execute scaling decision"""
138
139     self.add_metrics(metrics)
140
141     if self.should_scale_up(metrics):
142         target = self.calculate_target_instances(metrics)
143         scale_amount = target - self.current_instances
144
145         # Don't scale too aggressively
146         scale_amount = min(scale_amount, max(1, self.current_instances // 2))
147
148         new_instances = self.current_instances + scale_amount
149         self.current_instances = new_instances
150         self.last_scale_up_time = datetime.now()
151         self.consecutive_scale_up_signals = 0
152
153         return new_instances
154
155     elif self.should_scale_down(metrics):
156         # Scale down gradually
157         new_instances = max(
158             self.policy.min_instances,
159             self.current_instances - 1
160         )
161
162         self.current_instances = new_instances
163         self.last_scale_down_time = datetime.now()
164         self.consecutive_scale_down_signals = 0
165
166         return new_instances
167
168     return None
169
170 def estimate_cost_savings(
171     self,
172     without_autoscaling_instances: int,
173     cost_per_instance_hour: float,

```

```

173     hours: int = 24 * 30
174 ) -> Dict:
175     """Estimate cost savings from autoscaling"""
176
177     # Without autoscaling (always at peak)
178     cost_without = without_autoscaling_instances * cost_per_instance_hour * hours
179
180     # With autoscaling (estimate 60% average utilization)
181     avg_instances_with_autoscaling = without_autoscaling_instances * 0.6
182     cost_with = avg_instances_with_autoscaling * cost_per_instance_hour * hours
183
184     savings = cost_without - cost_with
185     savings_percent = (savings / cost_without) * 100
186
187     return {
188         "cost_without_autoscaling": cost_without,
189         "cost_with_autoscaling": cost_with,
190         "monthly_savings": savings,
191         "savings_percent": savings_percent
192     }
193
194 class PredictiveScaler:
195     """Predictive autoscaling using historical patterns"""
196
197     def __init__(self, autoscaler: MLAutoscaler):
198         self.autoscaler = autoscaler
199         self.traffic_patterns: Dict[int, List[float]] = {} # hour -> [rps]
200
201     def learn_traffic_pattern(self, metrics: ScalingMetrics):
202         """Learn traffic patterns over time"""
203         hour_of_day = metrics.timestamp.hour
204
205         if hour_of_day not in self.traffic_patterns:
206             self.traffic_patterns[hour_of_day] = []
207
208         self.traffic_patterns[hour_of_day].append(metrics.requests_per_second)
209
210         # Keep only last 30 days
211         if len(self.traffic_patterns[hour_of_day]) > 30:
212             self.traffic_patterns[hour_of_day].pop(0)
213
214     def predict_next_hour_load(self) -> float:
215         """Predict load for next hour"""
216         next_hour = (datetime.now() + timedelta(hours=1)).hour
217
218         if next_hour not in self.traffic_patterns:
219             return 0
220
221         historical_loads = self.traffic_patterns[next_hour]
222
223         if not historical_loads:
224             return 0
225
226         # Use 90th percentile for conservative prediction
227         return np.percentile(historical_loads, 90)
228
229     def proactive_scale(self) -> Optional[int]:
230         """Proactively scale based on prediction"""

```

```

231     predicted_rps = self.predict_next_hour_load()
232
233     if predicted_rps == 0:
234         return None
235
236     # Estimate instances needed
237     # Assume 100 RPS per instance
238     target_instances = int(np.ceil(predicted_rps / 100))
239     target_instances = max(
240         self.autoscaler.policy.min_instances,
241         min(target_instances, self.autoscaler.policy.max_instances)
242     )
243
244     # Only scale up proactively, let reactive scaling handle scale down
245     if target_instances > self.autoscaler.current_instances:
246         self.autoscaler.current_instances = target_instances
247         return target_instances
248
249     return None
250
251 # Example usage
252 if __name__ == "__main__":
253     policy = ScalingPolicy(
254         min_instances=2,
255         max_instances=20,
256         target_cpu_utilization=70.0,
257         target_latency_p99_ms=200.0
258     )
259
260     autoscaler = MLAutoscaler(policy)
261
262     # Simulate high load
263     high_load_metrics = ScalingMetrics(
264         timestamp=datetime.now(),
265         cpu_utilization=85.0,
266         memory_utilization=70.0,
267         requests_per_second=500,
268         avg_latency_ms=150,
269         p99_latency_ms=250,
270         queue_depth=50
271     )
272
273     # Make scaling decision
274     new_instances = autoscaler.execute_scaling_decision(high_load_metrics)
275
276     if new_instances:
277         print(f"Scaled to {new_instances} instances")
278
279     # Estimate savings
280     savings = autoscaler.estimate_cost_savings(
281         without_autoscaling_instances=10,
282         cost_per_instance_hour=0.50
283     )
284
285     print(f"\nAutoscaling Savings:")
286     print(f"  Monthly savings: ${savings['monthly_savings']:.2f}")
287     print(f"  Savings percent: {savings['savings_percent']:.1f}%")

```

Listing 22.5: ML-Aware Autoscaling Policy

22.7 Cloud Cost Monitoring and Alerts

Continuous cost monitoring prevents budget overruns and identifies optimization opportunities.

22.7.1 Cost Monitoring System

```

1  #!/usr/bin/env python3
2  """
3  Cloud Cost Monitoring
4  Real-time cost tracking and alerting system
5  """
6
7  import boto3
8  from datetime import datetime, timedelta
9  from typing import Dict, List, Optional
10 from dataclasses import dataclass
11 import json
12
13 @dataclass
14 class CostAlert:
15     """Cost alert configuration"""
16     alert_name: str
17     threshold_amount: float
18     threshold_percent: Optional[float]
19     period_days: int
20     notification_emails: List[str]
21     severity: str = "warning" # warning, critical
22
23 @dataclass
24 class CostAnomaly:
25     """Detected cost anomaly"""
26     service: str
27     current_cost: float
28     expected_cost: float
29     deviation_percent: float
30     timestamp: datetime
31
32 class CloudCostMonitor:
33     """Monitor cloud costs and send alerts"""
34
35     def __init__(self, aws_account_id: str, region: str = "us-east-1"):
36         self.account_id = aws_account_id
37         self.ce_client = boto3.client('ce', region_name=region) # Cost Explorer
38         self.sns_client = boto3.client('sns', region_name=region)
39         self.cloudwatch_client = boto3.client('cloudwatch', region_name=region)
40         self.alerts: List[CostAlert] = []
41
42     def get_cost_and_usage(
43         self,
44         start_date: datetime,

```

```

45     end_date: datetime,
46     granularity: str = "DAILY",
47     group_by: Optional[List[Dict]] = None
48 ) -> Dict:
49     """Get cost and usage data from AWS Cost Explorer"""
50
51     if group_by is None:
52         group_by = [{"Type": "DIMENSION", "Key": "SERVICE"}]
53
54     response = self.ce_client.get_cost_and_usage(
55         TimePeriod={
56             'Start': start_date.strftime('%Y-%m-%d'),
57             'End': end_date.strftime('%Y-%m-%d')
58         },
59         Granularity=granularity,
60         Metrics=['UnblendedCost', 'UsageQuantity'],
61         GroupBy=group_by
62     )
63
64     return response
65
66 def get_current_month_cost(self) -> float:
67     """Get month-to-date cost"""
68
69     start_of_month = datetime.now().replace(day=1, hour=0, minute=0, second=0,
70 microsecond=0)
71     today = datetime.now()
72
73     response = self.get_cost_and_usage(
74         start_date=start_of_month,
75         end_date=today,
76         granularity="MONTHLY",
77         group_by=[]
78     )
79
80     if response['ResultsByTime']:
81         total_cost = float(
82             response['ResultsByTime'][0]['Total']['UnblendedCost']['Amount']
83         )
84         return total_cost
85
86     return 0.0
87
88 def get_forecast(self, days_ahead: int = 30) -> float:
89     """Get cost forecast"""
90
91     start_date = datetime.now()
92     end_date = start_date + timedelta(days=days_ahead)
93
94     response = self.ce_client.get_cost_forecast(
95         TimePeriod={
96             'Start': start_date.strftime('%Y-%m-%d'),
97             'End': end_date.strftime('%Y-%m-%d')
98         },
99         Metric='UNBLENDED_COST',
100         Granularity='MONTHLY'
101     )

```



```

102     forecast = float(response['Total']['Amount'])
103     return forecast
104
105     def add_cost_alert(self, alert: CostAlert):
106         """Add a cost alert"""
107         self.alerts.append(alert)
108
109         # Create CloudWatch alarm
110         self._create_cloudwatch_alarm(alert)
111
112     def _create_cloudwatch_alarm(self, alert: CostAlert):
113         """Create CloudWatch alarm for cost threshold"""
114
115         # Note: AWS Cost Explorer metrics are delayed, so this is simplified
116         # In practice, you'd need to set up AWS Budgets or custom metrics
117
118         alarm_name = f"cost-alert-{alert.alert_name}"
119
120         try:
121             self.cloudwatch_client.put_metric_alarm(
122                 AlarmName=alarm_name,
123                 ComparisonOperator='GreaterThanOrEqualToThreshold',
124                 EvaluationPeriods=1,
125                 MetricName='EstimatedCharges',
126                 Namespace='AWS/Billing',
127                 Period=86400, # Daily
128                 Statistic='Maximum',
129                 Threshold=alert.threshold_amount,
130                 ActionsEnabled=True,
131                 AlarmDescription=f'Cost alert for {alert.alert_name}',
132                 Dimensions=[
133                     {
134                         'Name': 'Currency',
135                         'Value': 'USD'
136                     }
137                 ]
138             )
139
140             print(f"Created CloudWatch alarm: {alarm_name}")
141
142         except Exception as e:
143             print(f"Error creating alarm: {e}")
144
145     def check_alerts(self) -> List[str]:
146         """Check all alerts and return triggered ones"""
147
148         triggered_alerts = []
149         current_cost = self.get_current_month_cost()
150
151         for alert in self.alerts:
152             # Calculate threshold for period
153             if alert.threshold_percent:
154                 # Compare to previous period
155                 prev_period_cost = self._get_previous_period_cost(alert.period_days)
156                 threshold = prev_period_cost * (1 + alert.threshold_percent / 100)
157             else:
158                 threshold = alert.threshold_amount
159

```

```

160         if current_cost > threshold:
161             message = (
162                 f"Cost Alert: {alert.alert_name}\n"
163                 f"Current cost: ${current_cost:.2f}\n"
164                 f"Threshold: ${threshold:.2f}\n"
165                 f"Severity: {alert.severity}"
166             )
167
168             triggered_alerts.append(message)
169             self._send_alert_notification(alert, message)
170
171         return triggered_alerts
172
173     def _get_previous_period_cost(self, period_days: int) -> float:
174         """Get cost from previous period"""
175
176         end_date = datetime.now() - timedelta(days=period_days)
177         start_date = end_date - timedelta(days=period_days)
178
179         response = self.get_cost_and_usage(
180             start_date=start_date,
181             end_date=end_date,
182             granularity="MONTHLY",
183             group_by=[]
184         )
185
186         if response['ResultsByTime']:
187             return float(
188                 response['ResultsByTime'][0]['Total']['UnblendedCost']['Amount']
189             )
190
191         return 0.0
192
193     def _send_alert_notification(self, alert: CostAlert, message: str):
194         """Send alert notification via SNS"""
195
196         # In practice, you'd have an SNS topic set up
197         print(f"ALERT: {message}")
198
199         # Example SNS publish (requires topic ARN)
200         # self.sns_client.publish(
201         #     TopicArn='arn:aws:sns:us-east-1:123456789:cost-alerts',
202         #     Subject=f'Cost Alert: {alert.alert_name}',
203         #     Message=message
204         # )
205
206     def detect_cost_anomalies(self) -> List[CostAnomaly]:
207         """Detect unusual cost patterns"""
208
209         # Get last 7 days of costs by service
210         end_date = datetime.now()
211         start_date = end_date - timedelta(days=7)
212
213         response = self.get_cost_and_usage(
214             start_date=start_date,
215             end_date=end_date,
216             granularity="DAILY",
217             group_by=[{"Type": "DIMENSION", "Key": "SERVICE"}]

```

```

218     )
219
220     anomalies = []
221
222     # Analyze each service
223     service_costs = {}
224
225     for result in response['ResultsByTime']:
226         for group in result['Groups']:
227             service = group['Keys'][0]
228             cost = float(group['Metrics']['UnblendedCost']['Amount'])
229
230             if service not in service_costs:
231                 service_costs[service] = []
232
233             service_costs[service].append(cost)
234
235     # Detect anomalies (simple threshold: >50% above average)
236     for service, costs in service_costs.items():
237         if len(costs) < 2:
238             continue
239
240         avg_cost = sum(costs[:-1]) / len(costs[:-1])
241         current_cost = costs[-1]
242
243         if current_cost > avg_cost * 1.5:
244             deviation = ((current_cost - avg_cost) / avg_cost) * 100
245
246             anomalies.append(CostAnomaly(
247                 service=service,
248                 current_cost=current_cost,
249                 expected_cost=avg_cost,
250                 deviation_percent=deviation,
251                 timestamp=datetime.now()
252             ))
253
254     return anomalies
255
256 def generate_cost_report(self) -> Dict:
257     """Generate comprehensive cost report"""
258
259     current_month_cost = self.get_current_month_cost()
260     forecast = self.get_forecast(days_ahead=30)
261
262     # Get cost by service
263     start_of_month = datetime.now().replace(day=1, hour=0, minute=0, second=0,
microsecond=0)
264     today = datetime.now()
265
266     by_service = self.get_cost_and_usage(
267         start_date=start_of_month,
268         end_date=today,
269         granularity="MONTHLY"
270     )
271
272     service_costs = {}
273     if by_service['ResultsByTime']:
274         for group in by_service['ResultsByTime'][0]['Groups']:

```

```

275         service = group['Keys'][0]
276         cost = float(group['Metrics']['UnblendedCost']['Amount'])
277         service_costs[service] = cost
278
279     # Detect anomalies
280     anomalies = self.detect_cost_anomalies()
281
282     return {
283         "current_month_cost": current_month_cost,
284         "forecasted_month_end_cost": forecast,
285         "cost_by_service": service_costs,
286         "anomalies": [
287             {
288                 "service": a.service,
289                 "deviation_percent": a.deviation_percent,
290                 "current_cost": a.current_cost
291             }
292             for a in anomalies
293         ],
294         "generated_at": datetime.now().isoformat()
295     }
296
297 def set_budget(
298     self,
299     budget_name: str,
300     budget_limit: float,
301     notification_threshold_percent: float = 80.0
302 ):
303     """Set up AWS Budget"""
304
305     budgets_client = boto3.client('budgets')
306
307     try:
308         budgets_client.create_budget(
309             AccountId=self.account_id,
310             Budget={
311                 'BudgetName': budget_name,
312                 'BudgetLimit': {
313                     'Amount': str(budget_limit),
314                     'Unit': 'USD'
315                 },
316                 'TimeUnit': 'MONTHLY',
317                 'BudgetType': 'COST'
318             },
319             NotificationsWithSubscribers=[
320                 {
321                     'Notification': {
322                         'NotificationType': 'ACTUAL',
323                         'ComparisonOperator': 'GREATER_THAN',
324                         'Threshold': notification_threshold_percent,
325                         'ThresholdType': 'PERCENTAGE'
326                     },
327                     'Subscribers': [
328                         {
329                             'SubscriptionType': 'EMAIL',
330                             'Address': 'team@example.com'
331                         }
332                     ]
333                 }

```

```

333         }
334     ]
335 )
336
337     print(f"Created budget: {budget_name}")
338
339     except Exception as e:
340         print(f"Error creating budget: {e}")
341
342 # Example usage
343 if __name__ == "__main__":
344     monitor = CloudCostMonitor(aws_account_id="123456789012")
345
346     # Add cost alerts
347     alert = CostAlert(
348         alert_name="monthly-budget",
349         threshold_amount=10000.0,
350         threshold_percent=None,
351         period_days=30,
352         notification_emails=["team@example.com"],
353         severity="warning"
354     )
355
356     monitor.add_cost_alert(alert)
357
358     # Generate cost report
359     report = monitor.generate_cost_report()
360
361     print(f"Current Month Cost: ${report['current_month_cost']:.2f}")
362     print(f"Forecasted Cost: ${report['forecasted_month_end_cost']:.2f}")
363
364     print("\nCost by Service:")
365     for service, cost in sorted(
366         report['cost_by_service'].items(),
367         key=lambda x: x[1],
368         reverse=True
369     )[:5]:
370         print(f"    {service}: ${cost:.2f}")
371
372     if report['anomalies']:
373         print("\nCost Anomalies Detected:")
374         for anomaly in report['anomalies']:
375             print(f"    {anomaly['service']}: +{anomaly['deviation_percent']:.1f}% "
376                   f"${anomaly['current_cost']:.2f}")

```

Listing 22.6: Cloud Cost Monitoring and Alerting

22.8 ROI Analysis for ML Investments

Quantifying the return on ML investments justifies costs and guides resource allocation.

22.8.1 ML ROI Calculator

```

1 #!/usr/bin/env python3
2 """

```

```

3  ML Investment ROI Calculator
4  Calculate and analyze return on investment for ML projects
5  """
6
7  from dataclasses import dataclass
8  from typing import Dict, List, Optional
9  from datetime import datetime, timedelta
10 import numpy as np
11
12 @dataclass
13 class MLProjectCosts:
14     """ML project cost breakdown"""
15     development_costs: float
16     training_infrastructure: float
17     serving_infrastructure_monthly: float
18     data_costs: float
19     personnel_costs_monthly: float
20     maintenance_costs_monthly: float
21     monitoring_costs_monthly: float
22
23     def total_initial_investment(self) -> float:
24         """Calculate total initial investment"""
25         return (
26             self.development_costs +
27             self.training_infrastructure +
28             self.data_costs
29         )
30
31     def monthly_operating_cost(self) -> float:
32         """Calculate monthly operating costs"""
33         return (
34             self.serving_infrastructure_monthly +
35             self.personnel_costs_monthly +
36             self.maintenance_costs_monthly +
37             self.monitoring_costs_monthly
38         )
39
40 @dataclass
41 class MLProjectBenefits:
42     """ML project benefits"""
43     revenue_increase_monthly: float
44     cost_savings_monthly: float
45     efficiency_gains_monthly: float
46     customer_satisfaction_value_monthly: float
47
48     def total_monthly_benefit(self) -> float:
49         """Calculate total monthly benefit"""
50         return (
51             self.revenue_increase_monthly +
52             self.cost_savings_monthly +
53             self.efficiency_gains_monthly +
54             self.customer_satisfaction_value_monthly
55         )
56
57 class MLROICalculator:
58     """Calculate ROI for ML projects"""
59
60     def __init__(

```

```

61         self,
62         project_costs: MLProjectCosts,
63         project_benefits: MLProjectBenefits,
64         discount_rate: float = 0.10
65     ):
66         self.costs = project_costs
67         self.benefits = project_benefits
68         self.discount_rate = discount_rate
69
70     def calculate_simple_roi(self, months: int = 12) -> float:
71         """Calculate simple ROI"""
72
73         total_cost = (
74             self.costs.total_initial_investment() +
75             self.costs.monthly_operating_cost() * months
76         )
77
78         total_benefit = self.benefits.total_monthly_benefit() * months
79
80         roi = ((total_benefit - total_cost) / total_cost) * 100
81
82         return roi
83
84     def calculate_payback_period(self) -> float:
85         """Calculate payback period in months"""
86
87         initial_investment = self.costs.total_initial_investment()
88         monthly_net_benefit = (
89             self.benefits.total_monthly_benefit() -
90             self.costs.monthly_operating_cost()
91         )
92
93         if monthly_net_benefit <= 0:
94             return float('inf')
95
96         payback_months = initial_investment / monthly_net_benefit
97
98         return payback_months
99
100     def calculate_npv(self, months: int = 36) -> float:
101         """Calculate Net Present Value"""
102
103         initial_investment = self.costs.total_initial_investment()
104         monthly_rate = self.discount_rate / 12
105
106         npv = -initial_investment
107
108         for month in range(1, months + 1):
109             monthly_cash_flow = (
110                 self.benefits.total_monthly_benefit() -
111                 self.costs.monthly_operating_cost()
112             )
113
114             discounted_cash_flow = monthly_cash_flow / ((1 + monthly_rate) ** month)
115             npv += discounted_cash_flow
116
117         return npv
118

```

```

119 def calculate_irr(self, months: int = 36, precision: float = 0.0001) -> float:
120     """Calculate Internal Rate of Return"""
121
122     # Use binary search to find IRR
123     low_rate = -0.99
124     high_rate = 1.0
125
126     while high_rate - low_rate > precision:
127         mid_rate = (low_rate + high_rate) / 2
128
129         # Calculate NPV at mid_rate
130         npv = self._npv_at_rate(mid_rate, months)
131
132         if npv > 0:
133             low_rate = mid_rate
134         else:
135             high_rate = mid_rate
136
137     return (low_rate + high_rate) / 2
138
139 def _npv_at_rate(self, annual_rate: float, months: int) -> float:
140     """Calculate NPV at a specific discount rate"""
141
142     initial_investment = self.costs.total_initial_investment()
143     monthly_rate = annual_rate / 12
144
145     npv = -initial_investment
146
147     for month in range(1, months + 1):
148         monthly_cash_flow = (
149             self.benefits.total_monthly_benefit() -
150             self.costs.monthly_operating_cost()
151         )
152
153         discounted_cash_flow = monthly_cash_flow / ((1 + monthly_rate) ** month)
154         npv += discounted_cash_flow
155
156     return npv
157
158 def sensitivity_analysis(
159     self,
160     parameter: str,
161     variation_range: tuple = (-0.3, 0.3),
162     steps: int = 7
163 ) -> Dict:
164     """Perform sensitivity analysis on a parameter"""
165
166     variations = np.linspace(variation_range[0], variation_range[1], steps)
167     results = {
168         "variations": [],
169         "roi": [],
170         "npv": [],
171         "payback_period": []
172     }
173
174     original_costs = self.costs
175     original_benefits = self.benefits
176

```



```

177     for variation in variations:
178         # Modify parameter
179         if parameter == "revenue_increase":
180             self.benefits.revenue_increase_monthly = (
181                 original_benefits.revenue_increase_monthly * (1 + variation)
182             )
183         elif parameter == "serving_cost":
184             self.costs.serving_infrastructure_monthly = (
185                 original_costs.serving_infrastructure_monthly * (1 + variation)
186             )
187         # Add more parameters as needed
188
189         # Calculate metrics
190         results["variations"].append(variation)
191         results["roi"].append(self.calculate_simple_roi())
192         results["npv"].append(self.calculate_npv())
193         results["payback_period"].append(self.calculate_payback_period())
194
195         # Restore original values
196         self.costs = original_costs
197         self.benefits = original_benefits
198
199     return results
200
201 def compare_scenarios(
202     self,
203     scenarios: Dict[str, tuple]
204 ) -> Dict:
205     """Compare different scenarios"""
206
207     results = {}
208
209     for scenario_name, (costs, benefits) in scenarios.items():
210         calculator = MLROI Calculator(costs, benefits, self.discount_rate)
211
212         results[scenario_name] = {
213             "roi_12_months": calculator.calculate_simple_roi(12),
214             "roi_36_months": calculator.calculate_simple_roi(36),
215             "payback_period_months": calculator.calculate_payback_period(),
216             "npv_36_months": calculator.calculate_npv(36),
217             "irr": calculator.calculate_irr(36) * 100 # Convert to percentage
218         }
219
220     return results
221
222 def generate_roi_report(self) -> Dict:
223     """Generate comprehensive ROI report"""
224
225     roi_12m = self.calculate_simple_roi(12)
226     roi_36m = self.calculate_simple_roi(36)
227     payback = self.calculate_payback_period()
228     npv = self.calculate_npv(36)
229     irr = self.calculate_irr(36) * 100
230
231     # Calculate break-even
232     monthly_net = (
233         self.benefits.total_monthly_benefit() -
234         self.costs.monthly_operating_cost()

```

```

235     )
236
237     # Cost efficiency metrics
238     cost_per_prediction = (
239         self.costs.monthly_operating_cost() /
240         (self.benefits.revenue_increase_monthly / 10) # Assume $10 per conversion
241     )
242
243     return {
244         "financial_metrics": {
245             "roi_12_months_percent": roi_12m,
246             "roi_36_months_percent": roi_36m,
247             "payback_period_months": payback,
248             "npv_36_months": npv,
249             "irr_percent": irr
250         },
251         "cost_breakdown": {
252             "initial_investment": self.costs.total_initial_investment(),
253             "monthly_operating_cost": self.costs.monthly_operating_cost(),
254             "annual_operating_cost": self.costs.monthly_operating_cost() * 12
255         },
256         "benefit_breakdown": {
257             "monthly_benefit": self.benefits.total_monthly_benefit(),
258             "annual_benefit": self.benefits.total_monthly_benefit() * 12
259         },
260         "efficiency_metrics": {
261             "monthly_net_benefit": monthly_net,
262             "cost_per_prediction": cost_per_prediction
263         },
264         "recommendation": self._get_recommendation(roi_36m, payback, npv)
265     }
266
267     def _get_recommendation(
268         self,
269         roi: float,
270         payback_months: float,
271         npv: float
272     ) -> str:
273         """Generate investment recommendation"""
274
275         if npv > 0 and roi > 20 and payback_months < 12:
276             return "Strongly Recommended: Excellent ROI with quick payback"
277         elif npv > 0 and roi > 10 and payback_months < 24:
278             return "Recommended: Positive ROI with reasonable payback period"
279         elif npv > 0:
280             return "Consider: Positive but modest returns"
281         else:
282             return "Not Recommended: Negative returns expected"
283
284     # Example usage
285     if __name__ == "__main__":
286         # Recommendation system project
287         costs = MLProjectCosts(
288             development_costs=150000, # 6 months * 2 engineers
289             training_infrastructure=5000,
290             serving_infrastructure_monthly=3000,
291             data_costs=10000,
292             personnel_costs_monthly=25000,

```

```

293     maintenance_costs_monthly=2000,
294     monitoring_costs_monthly=500
295 )
296
297 # Benefits: 5% increase in conversion rate
298 benefits = MLProjectBenefits(
299     revenue_increase_monthly=50000, # Increased sales
300     cost_savings_monthly=10000, # Reduced manual curation
301     efficiency_gains_monthly=5000, # Engineering time saved
302     customer_satisfaction_value_monthly=5000 # Reduced churn
303 )
304
305 calculator = MLROI Calculator(costs, benefits)
306
307 # Generate report
308 report = calculator.generate_roi_report()
309
310 print("ML Project ROI Analysis")
311 print("=" * 50)
312 print(f"\nFinancial Metrics:")
313 print(f"  12-month ROI: {report['financial_metrics']['roi_12_months_percent']:.1f}%")
314 print(f"  36-month ROI: {report['financial_metrics']['roi_36_months_percent']:.1f}%")
315 print(f"  Payback Period: {report['financial_metrics']['payback_period_months']:.1f} months")
316 print(f"  NPV (36 months): ${report['financial_metrics']['npv_36_months']:.2f}")
317 print(f"  IRR: {report['financial_metrics']['irr_percent']:.1f}%")
318
319 print(f"\nCost Breakdown:")
320 print(f"  Initial Investment: ${report['cost_breakdown']['initial_investment']:.2f}")
321 print(f"  Annual Operating Cost: ${report['cost_breakdown']['annual_operating_cost']:.2f}")
322
323 print(f"\nBenefit Breakdown:")
324 print(f"  Annual Benefit: ${report['benefit_breakdown']['annual_benefit']:.2f}")
325
326 print(f"\nRecommendation: {report['recommendation']}")

```

Listing 22.7: ML Investment ROI Calculator

22.9 Best Practices for Cost Optimization

Best Practice: *Implement cost allocation tags across all ML resources. Tag by project, team, environment, and cost center to enable detailed cost tracking and chargeback.*

Cost Optimization Checklist:

1. Training Optimization:

- Profile training jobs before scaling to expensive hardware
- Use mixed precision training (FP16/BF16) for 2-3x speedup

- Implement spot instances with checkpointing for 70-90% savings
- Use efficient hyperparameter search (Bayesian, Hyperband)
- Cache preprocessed training data
- Implement early stopping for unpromising runs

2. Serving Optimization:

- Right-size serving instances based on actual load
- Implement autoscaling with ML-aware policies
- Use model compression (quantization, distillation)
- Implement response caching for repeated requests
- Consider serverless for variable/low traffic
- Use batch prediction for non-real-time workloads

3. Storage Optimization:

- Implement data lifecycle policies (hot/warm/cold tiers)
- Compress training data and model artifacts
- Delete old checkpoints and experiment artifacts
- Use efficient file formats (Parquet, TFRecord)
- Archive infrequently accessed data

4. Development Optimization:

- Automatically shut down unused notebook instances
- Use smaller instances for development/testing
- Share development infrastructure across teams
- Implement resource quotas per user/team

5. Monitoring and Governance:

- Set up cost alerts and budgets
- Review cost reports weekly
- Track cost per prediction/request
- Calculate and optimize ROI
- Implement showback/chargeback for accountability

22.10 Chapter Summary

This chapter covered comprehensive cost optimization strategies for ML systems:

- **Cost Analysis:** Implemented tools to track and analyze ML system costs across training, serving, and storage

- **Training Optimization:** Covered mixed precision, gradient accumulation, spot instances, and efficient hyperparameter search
- **Serving Optimization:** Explored deployment options, autoscaling, model compression, and caching strategies
- **Resource Allocation:** Designed intelligent scaling policies that balance performance and cost
- **Cost Monitoring:** Built monitoring systems with alerts and anomaly detection
- **ROI Analysis:** Developed frameworks to calculate and justify ML investments

Cost optimization is not a one-time activity but an ongoing process. Regular analysis, monitoring, and refinement of cost strategies ensure ML systems deliver maximum value relative to their infrastructure investments.

22.11 Exercises

Exercise 22.1. Training Cost Analysis

Profile a deep learning training workload and optimize its cost:

1. Profile a model training job on different instance types
2. Calculate cost per epoch for each instance type
3. Implement mixed precision training and measure speedup
4. Calculate cost savings from using spot instances
5. Determine optimal batch size for cost efficiency

Deliverables:

- Performance profiling report with throughput metrics
- Cost comparison table across instance types
- Recommended configuration with estimated annual savings

Exercise 22.2. Serving Cost Optimization

Analyze and optimize serving infrastructure costs:

1. Collect metrics from a production serving endpoint (RPS, latency, instance utilization)
2. Compare costs of always-on vs. autoscaling vs. serverless deployment
3. Implement model compression (quantization or distillation)
4. Measure latency and throughput impact of compression
5. Calculate ROI of compression investment

Deliverables:

- Current serving cost breakdown
- Deployment option comparison with recommendations
- Model compression analysis with accuracy/latency/cost tradeoffs

Exercise 22.3. Autoscaling Policy Design

Design and implement an ML-aware autoscaling policy:

1. Analyze historical traffic patterns for a serving endpoint
2. Design scaling policy based on CPU, latency, and queue depth
3. Implement predictive scaling using traffic patterns
4. Simulate autoscaling with historical data
5. Calculate cost savings compared to fixed capacity

Deliverables:

- Traffic pattern analysis and visualization
- Autoscaling policy specification
- Simulation results showing scaling behavior
- Cost savings analysis over 30 days

Exercise 22.4. Cost Monitoring Dashboard

Build a comprehensive cost monitoring system:

1. Set up cost tracking for ML infrastructure (use AWS Cost Explorer API or equivalent)
2. Create dashboard showing costs by service, project, and environment
3. Implement cost anomaly detection
4. Configure budget alerts at 50%, 80%, and 100% thresholds
5. Generate weekly cost optimization reports

Deliverables:

- Cost monitoring dashboard (Grafana/CloudWatch)
- Anomaly detection rules and alert configurations
- Sample weekly cost optimization report

Exercise 22.5. ML Project ROI Analysis

Perform ROI analysis for an ML project:

1. Identify and quantify all costs (development, infrastructure, personnel)
2. Quantify benefits (revenue increase, cost savings, efficiency gains)
3. Calculate ROI, NPV, IRR, and payback period
4. Perform sensitivity analysis on key assumptions
5. Create executive summary with investment recommendation

Deliverables:

- Detailed cost and benefit breakdown
- Financial metrics (ROI, NPV, IRR, payback period)
- Sensitivity analysis charts
- Executive summary with recommendation

Part VIII

Case Studies and Integration

Chapter 23

Industry Case Studies

23.1 Learning Objectives

After completing this chapter, readers will be able to:

- Design and implement end-to-end ML systems for real-world applications
- Understand architecture patterns across different industries
- Apply domain-specific best practices and optimization techniques
- Learn from production challenges and their solutions
- Recognize common patterns and anti-patterns in ML system design
- Implement scalable data pipelines for various use cases

23.2 Introduction

This chapter presents four comprehensive industry case studies that demonstrate end-to-end ML system implementations. Each case study covers the complete lifecycle from requirements gathering through production deployment, including real-world challenges and solutions.

Case Studies Covered:

1. **E-Commerce Recommendation System:** Personalized product recommendations at scale
2. **Financial Fraud Detection:** Real-time transaction monitoring and fraud prevention
3. **Healthcare Diagnostic System:** Medical image analysis for disease detection
4. **Autonomous Vehicle Perception:** Multi-sensor fusion for object detection

Each case study follows a consistent structure: business context, requirements, architecture design, implementation, deployment, monitoring, results, and lessons learned.

23.3 Case Study 1: E-Commerce Recommendation System

23.3.1 Business Context and Requirements

Company: Mid-sized e-commerce platform with 5 million monthly active users

Challenge: Low conversion rates and poor product discovery

Goal: Increase revenue through personalized product recommendations

Key Requirements:

- **Latency:** Recommendations must load in <100ms (p99)
- **Scale:** Handle 10,000 requests per second during peak hours
- **Personalization:** Adapt to user behavior in real-time
- **Business Constraints:** Incorporate inventory, margins, and business rules
- **Cold Start:** Handle new users and new products effectively
- **Diversity:** Avoid filter bubbles, show varied recommendations

23.3.2 System Architecture

23.3.3 Data Pipeline Implementation

```

1  #!/usr/bin/env python3
2  """
3  E-Commerce Recommendation System - Data Pipeline
4  Real-time and batch feature engineering for personalized recommendations
5  """
6
7  from kafka import KafkaConsumer, KafkaProducer
8  import json
9  from typing import Dict, List, Optional
10 from datetime import datetime, timedelta
11 from dataclasses import dataclass, field
12 import pandas as pd
13 import numpy as np
14 from redis import Redis
15 from collections import defaultdict
16 import logging
17
18 @dataclass
19 class UserEvent:
20     """User interaction event"""
21     user_id: str
22     event_type: str # view, click, add_to_cart, purchase
23     product_id: str
24     timestamp: datetime
25     session_id: str
26     metadata: Dict = field(default_factory=dict)
27
28 @dataclass
29 class Product:

```

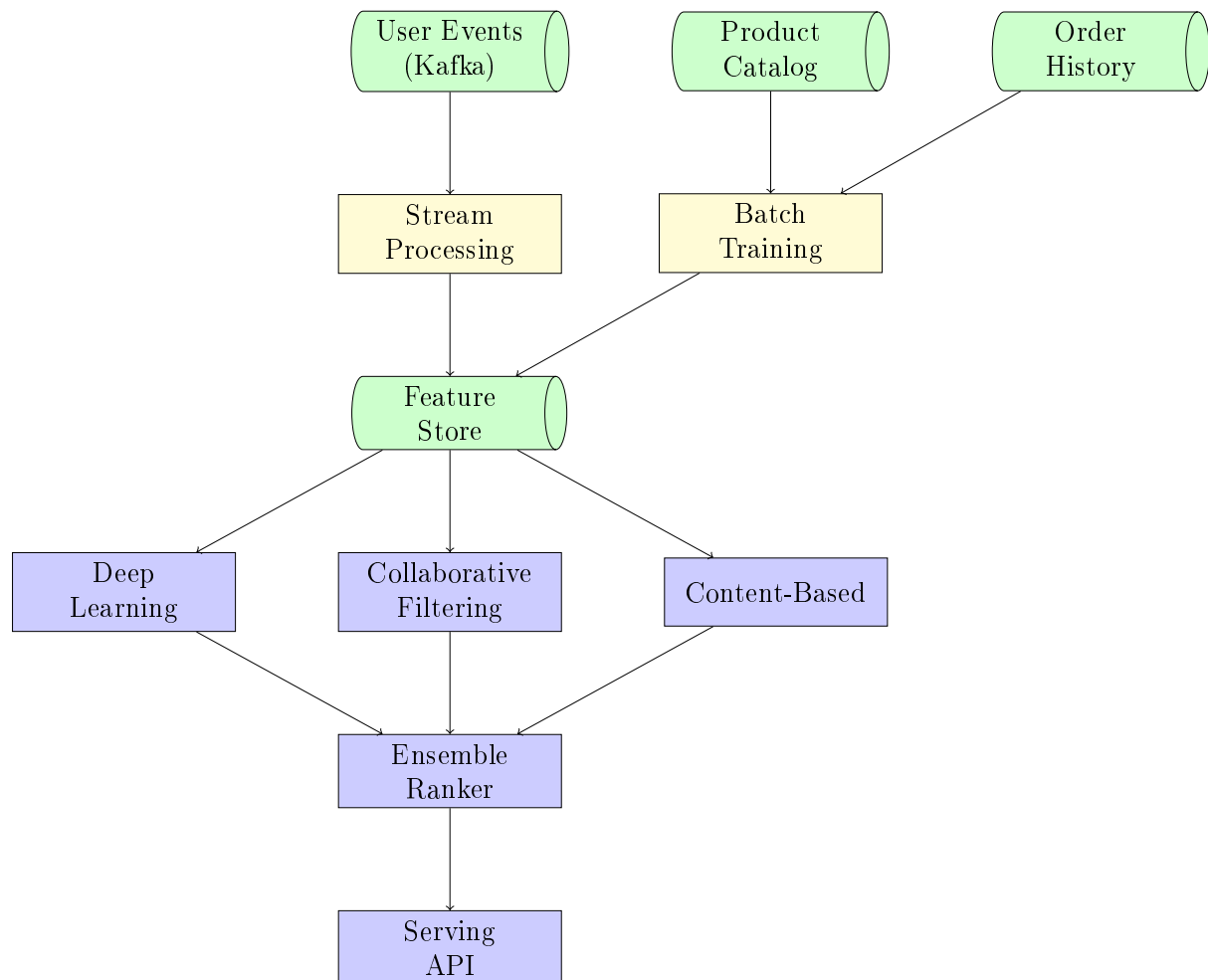


Figure 23.1: E-Commerce Recommendation System Architecture

```

30     """Product information"""
31     product_id: str
32     category: str
33     subcategory: str
34     price: float
35     brand: str
36     in_stock: bool
37     attributes: Dict = field(default_factory=dict)
38
39 @dataclass
40 class UserFeatures:
41     """Aggregated user features"""
42     user_id: str
43     favorite_categories: List[str]
44     avg_price_point: float
45     preferred_brands: List[str]
46     recent_views: List[str]
47     recent_purchases: List[str]
48     purchase_frequency: float
49     last_active: datetime
50
51 class RecommendationDataPipeline:

```

```

52     """Data pipeline for recommendation system"""
53
54     def __init__(
55         self,
56         kafka_bootstrap_servers: List[str],
57         redis_host: str,
58         redis_port: int = 6379
59     ):
60         self.kafka_consumer = KafkaConsumer(
61             'user-events',
62             bootstrap_servers=kafka_bootstrap_servers,
63             value_deserializer=lambda m: json.loads(m.decode('utf-8')),
64             group_id='recommendation-pipeline'
65         )
66
67         self.kafka_producer = KafkaProducer(
68             bootstrap_servers=kafka_bootstrap_servers,
69             value_serializer=lambda m: json.dumps(m).encode('utf-8')
70         )
71
72         self.redis_client = Redis(host=redis_host, port=redis_port, decode_responses=
True)
73         self.logger = logging.getLogger(__name__)
74
75     def process_event_stream(self):
76         """Process real-time user events"""
77
78         for message in self.kafka_consumer:
79             try:
80                 event_data = message.value
81                 event = UserEvent(
82                     user_id=event_data['user_id'],
83                     event_type=event_data['event_type'],
84                     product_id=event_data['product_id'],
85                     timestamp=datetime.fromisoformat(event_data['timestamp']),
86                     session_id=event_data['session_id'],
87                     metadata=event_data.get('metadata', {})
88                 )
89
90                 # Update real-time features
91                 self._update_user_features(event)
92
93                 # Update product popularity
94                 self._update_product_stats(event)
95
96                 # Generate real-time signals
97                 if event.event_type == 'purchase':
98                     self._trigger_similar_products(event)
99
100             except Exception as e:
101                 self.logger.error(f"Error processing event: {e}")
102
103     def _update_user_features(self, event: UserEvent):
104         """Update real-time user features in Redis"""
105
106         user_key = f"user:{event.user_id}"
107
108         # Update recent views (sliding window)

```

```

109     if event.event_type in ['view', 'click']:
110         self.redis_client.lpush(f"{user_key}:recent_views", event.product_id)
111         self.redis_client.ltrim(f"{user_key}:recent_views", 0, 49) # Keep last 50
112
113     # Update purchases
114     if event.event_type == 'purchase':
115         self.redis_client.lpush(f"{user_key}:purchases", event.product_id)
116         self.redis_client.lpush(f"{user_key}:purchase_times", event.timestamp.
isoformat())
117
118     # Update category preferences
119     product_category = self._get_product_category(event.product_id)
120     if product_category:
121         self.redis_client.zincrby(
122             f"{user_key}:categories",
123             1,
124             product_category
125         )
126
127     # Update session activity
128     self.redis_client.hset(
129         f"{user_key}:session:{event.session_id}",
130         mapping={
131             'last_activity': event.timestamp.isoformat(),
132             'event_count': self.redis_client.hincrby(
133                 f"{user_key}:session:{event.session_id}",
134                 'event_count',
135                 1
136             )
137         }
138     )
139     self.redis_client.expire(f"{user_key}:session:{event.session_id}", 3600)
140
141     def _update_product_stats(self, event: UserEvent):
142         """Update product statistics for trending items"""
143
144         product_key = f"product:{event.product_id}"
145
146         # Increment view/purchase counters with decay
147         current_hour = datetime.now().strftime("%Y-%m-%d-%H")
148
149         if event.event_type == 'view':
150             self.redis_client.hincrby(f"{product_key}:stats:{current_hour}", 'views',
1)
151         elif event.event_type == 'purchase':
152             self.redis_client.hincrby(f"{product_key}:stats:{current_hour}", '
purchases', 1)
153
154         # Set expiry on hourly stats
155         self.redis_client.expire(f"{product_key}:stats:{current_hour}", 86400 * 7)
156
157         # Update trending score (combines recency and popularity)
158         trending_score = self._calculate_trending_score(event.product_id)
159         self.redis_client.zadd('trending_products', {event.product_id: trending_score
})
160
161     def _calculate_trending_score(self, product_id: str) -> float:
162         """Calculate trending score based on recent activity"""

```

```

163     product_key = f"product:{product_id}"
164     score = 0.0
165
166     # Look at last 24 hours
167     for hours_ago in range(24):
168         timestamp = (datetime.now() - timedelta(hours=hours_ago)).strftime("%Y-%m
169         -%d-%H")
170         stats = self.redis_client.hgetall(f"{product_key}:stats:{timestamp}")
171
172         if stats:
173             views = int(stats.get('views', 0))
174             purchases = int(stats.get('purchases', 0))
175
176             # Decay factor: recent hours weighted more
177             decay = 0.95 ** hours_ago
178
179             # Weighted score
180             score += (views * 1.0 + purchases * 10.0) * decay
181
182     return score
183
184 def _trigger_similar_products(self, event: UserEvent):
185     """Trigger batch job to update similar products after purchase"""
186
187     message = {
188         'type': 'purchase',
189         'user_id': event.user_id,
190         'product_id': event.product_id,
191         'timestamp': event.timestamp.isoformat()
192     }
193
194     self.kafka_producer.send('recommendation-triggers', value=message)
195
196 def _get_product_category(self, product_id: str) -> Optional[str]:
197     """Get product category from cache or database"""
198
199     category = self.redis_client.get(f"product:{product_id}:category")
200     if category:
201         return category
202
203     # Fallback: fetch from database (not shown)
204     return None
205
206 def get_user_features(self, user_id: str) -> UserFeatures:
207     """Get aggregated user features for recommendations"""
208
209     user_key = f"user:{user_id}"
210
211     # Get recent views
212     recent_views = self.redis_client.lrange(f"{user_key}:recent_views", 0, 19)
213
214     # Get recent purchases
215     recent_purchases = self.redis_client.lrange(f"{user_key}:purchases", 0, 9)
216
217     # Get favorite categories
218     categories = self.redis_client.zrevrange(
219         f"{user_key}:categories",

```



```

220         0, 4,
221         withscores=True
222     )
223     favorite_categories = [cat for cat, score in categories]
224
225     # Calculate purchase frequency
226     purchase_times = self.redis_client.lrange(f"{user_key}:purchase_times", 0, -1)
227     purchase_frequency = self._calculate_purchase_frequency(purchase_times)
228
229     # Get average price point
230     avg_price = self._calculate_avg_price_point(recent_purchases)
231
232     return UserFeatures(
233         user_id=user_id,
234         favorite_categories=favorite_categories,
235         avg_price_point=avg_price,
236         preferred_brands=[], # Computed similarly
237         recent_views=recent_views,
238         recent_purchases=recent_purchases,
239         purchase_frequency=purchase_frequency,
240         last_active=datetime.now()
241     )
242
243     def _calculate_purchase_frequency(self, purchase_times: List[str]) -> float:
244         """Calculate purchases per week"""
245
246         if len(purchase_times) < 2:
247             return 0.0
248
249         timestamps = [datetime.fromisoformat(t) for t in purchase_times[:10]]
250         if len(timestamps) < 2:
251             return 0.0
252
253         time_span = (timestamps[0] - timestamps[-1]).total_seconds() / (7 * 86400)
254         if time_span == 0:
255             return 0.0
256
257         return len(timestamps) / time_span
258
259     def _calculate_avg_price_point(self, product_ids: List[str]) -> float:
260         """Calculate user's average price point"""
261
262         prices = []
263         for product_id in product_ids[:20]:
264             price = self.redis_client.get(f"product:{product_id}:price")
265             if price:
266                 prices.append(float(price))
267
268         return np.mean(prices) if prices else 0.0
269
270     class BatchFeatureEngineering:
271         """Batch feature engineering for model training"""
272
273         def __init__(self, data_warehouse_conn):
274             self.conn = data_warehouse_conn
275
276         def create_training_dataset(
277             self,

```

```

278         start_date: datetime,
279         end_date: datetime
280     ) -> pd.DataFrame:
281         """Create training dataset from historical data"""
282
283         # User features
284         user_features = self._compute_user_features(start_date, end_date)
285
286         # Product features
287         product_features = self._compute_product_features(start_date, end_date)
288
289         # Interaction matrix
290         interactions = self._build_interaction_matrix(start_date, end_date)
291
292         # Combine features
293         training_data = self._combine_features(
294             user_features,
295             product_features,
296             interactions
297         )
298
299         return training_data
300
301     def _compute_user_features(
302         self,
303         start_date: datetime,
304         end_date: datetime
305     ) -> pd.DataFrame:
306         """Compute aggregated user features"""
307
308         query = """
309         SELECT
310             user_id,
311             COUNT(DISTINCT session_id) as session_count,
312             COUNT(*) as total_events,
313             COUNT(DISTINCT product_id) as unique_products_viewed,
314             COUNT(CASE WHEN event_type = 'purchase' THEN 1 END) as purchase_count,
315             AVG(CASE WHEN event_type = 'purchase' THEN product_price END) as
avg_purchase_price,
316             MODE() WITHIN GROUP (ORDER BY product_category) as favorite_category,
317             MAX(timestamp) as last_active
318         FROM user_events
319         WHERE timestamp BETWEEN %s AND %s
320         GROUP BY user_id
321         """
322
323         return pd.read_sql(query, self.conn, params=[start_date, end_date])
324
325     def _compute_product_features(
326         self,
327         start_date: datetime,
328         end_date: datetime
329     ) -> pd.DataFrame:
330         """Compute product features"""
331
332         query = """
333         SELECT
334             p.product_id,

```

```

335         p.category,
336         p.subcategory,
337         p.price,
338         p.brand,
339         COUNT(DISTINCT ue.user_id) as unique_viewers,
340         COUNT(*) as total_views,
341         COUNT(CASE WHEN ue.event_type = 'purchase' THEN 1 END) as purchase_count,
342         COUNT(CASE WHEN ue.event_type = 'purchase' THEN 1 END)::FLOAT /
343         NULLIF(COUNT(*), 0) as conversion_rate
344     FROM products p
345     LEFT JOIN user_events ue ON p.product_id = ue.product_id
346     AND ue.timestamp BETWEEN %s AND %s
347     GROUP BY p.product_id, p.category, p.subcategory, p.price, p.brand
348     """
349
350     return pd.read_sql(query, self.conn, params=[start_date, end_date])
351
352     def _build_interaction_matrix(
353         self,
354         start_date: datetime,
355         end_date: datetime
356     ) -> pd.DataFrame:
357         """Build user-product interaction matrix"""
358
359         query = """
360         SELECT
361             user_id,
362             product_id,
363             SUM(CASE WHEN event_type = 'view' THEN 1 ELSE 0 END) as views,
364             SUM(CASE WHEN event_type = 'click' THEN 1 ELSE 0 END) as clicks,
365             SUM(CASE WHEN event_type = 'add_to_cart' THEN 1 ELSE 0 END) as cart_adds,
366             SUM(CASE WHEN event_type = 'purchase' THEN 1 ELSE 0 END) as purchases,
367             MAX(timestamp) as last_interaction
368         FROM user_events
369         WHERE timestamp BETWEEN %s AND %s
370         GROUP BY user_id, product_id
371         """
372
373         return pd.read_sql(query, self.conn, params=[start_date, end_date])
374
375     def _combine_features(
376         self,
377         user_features: pd.DataFrame,
378         product_features: pd.DataFrame,
379         interactions: pd.DataFrame
380     ) -> pd.DataFrame:
381         """Combine all features into training dataset"""
382
383         # Merge user and product features with interactions
384         training_data = interactions.merge(
385             user_features,
386             on='user_id',
387             how='left'
388         ).merge(
389             product_features,
390             on='product_id',
391             how='left'
392         )

```

```

393
394     # Create target variable (implicit feedback)
395     training_data['target'] = (
396         training_data['views'] * 0.1 +
397         training_data['clicks'] * 0.3 +
398         training_data['cart_adds'] * 0.5 +
399         training_data['purchases'] * 1.0
400     )
401
402     # Create negative samples
403     training_data = self._add_negative_samples(training_data)
404
405     return training_data
406
407 def _add_negative_samples(self, positive_samples: pd.DataFrame) -> pd.DataFrame:
408     """Add negative samples for training"""
409
410     # For each user, sample products they didn't interact with
411     # This is simplified - in production, use more sophisticated negative sampling
412
413     all_users = positive_samples['user_id'].unique()
414     all_products = positive_samples['product_id'].unique()
415
416     negative_samples = []
417
418     for user_id in all_users:
419         user_products = set(positive_samples[
420             positive_samples['user_id'] == user_id
421         ]['product_id'])
422
423         # Sample 4 negative products per positive
424         num_negatives = len(user_products) * 4
425         negative_products = np.random.choice(
426             list(set(all_products) - user_products),
427             size=min(num_negatives, len(all_products) - len(user_products)),
428             replace=False
429         )
430
431         for product_id in negative_products:
432             negative_samples.append({
433                 'user_id': user_id,
434                 'product_id': product_id,
435                 'target': 0
436             })
437
438     negative_df = pd.DataFrame(negative_samples)
439
440     return pd.concat([positive_samples, negative_df], ignore_index=True)
441
442 # Example usage
443 if __name__ == "__main__":
444     pipeline = RecommendationDataPipeline(
445         kafka_bootstrap_servers=['localhost:9092'],
446         redis_host='localhost'
447     )
448
449     # Start processing events
450     # pipeline.process_event_stream()

```

```

451
452     # Get user features
453     features = pipeline.get_user_features('user_12345')
454     print(f"User categories: {features.favorite_categories}")
455     print(f"Recent views: {len(features.recent_views)}")
456     print(f"Purchase frequency: {features.purchase_frequency:.2f} per week")

```

Listing 23.1: Event Streaming and Feature Engineering

23.3.4 Model Implementation

```

1  #!/usr/bin/env python3
2  """
3  E-Commerce Recommendation Models
4  Hybrid approach combining collaborative filtering, content-based, and deep learning
5  """
6
7  import torch
8  import torch.nn as nn
9  import numpy as np
10 from typing import List, Dict, Tuple
11 from sklearn.metrics.pairwise import cosine_similarity
12 import pandas as pd
13 from scipy.sparse import csr_matrix
14 from implicit.als import AlternatingLeastSquares
15 from implicit.bpr import BayesianPersonalizedRanking
16
17 class CollaborativeFilteringModel:
18     """Matrix factorization for collaborative filtering"""
19
20     def __init__(self, factors: int = 64, regularization: float = 0.01):
21         self.factors = factors
22         self.model = AlternatingLeastSquares(
23             factors=factors,
24             regularization=regularization,
25             iterations=15
26         )
27         self.user_id_map = {}
28         self.product_id_map = {}
29
30     def fit(self, interactions: pd.DataFrame):
31         """Train collaborative filtering model"""
32
33         # Create user and product mappings
34         unique_users = interactions['user_id'].unique()
35         unique_products = interactions['product_id'].unique()
36
37         self.user_id_map = {uid: idx for idx, uid in enumerate(unique_users)}
38         self.product_id_map = {pid: idx for idx, pid in enumerate(unique_products)}
39
40         # Create sparse interaction matrix
41         user_indices = interactions['user_id'].map(self.user_id_map)
42         product_indices = interactions['product_id'].map(self.product_id_map)
43         values = interactions['target'].values
44
45         interaction_matrix = csr_matrix(
46             (values, (user_indices, product_indices)),

```

```

47         shape=(len(unique_users), len(unique_products))
48     )
49
50     # Train model
51     self.model.fit(interaction_matrix)
52
53     def recommend(
54         self,
55         user_id: str,
56         n: int = 10,
57         filter_already_liked: bool = True
58     ) -> List[Tuple[str, float]]:
59         """Generate recommendations for user"""
60
61         if user_id not in self.user_id_map:
62             return []
63
64         user_idx = self.user_id_map[user_id]
65
66         # Get recommendations
67         product_indices, scores = self.model.recommend(
68             user_idx,
69             None, # All items
70             N=n,
71             filter_already_liked_items=filter_already_liked
72         )
73
74         # Map back to product IDs
75         reverse_product_map = {idx: pid for pid, idx in self.product_id_map.items()}
76         recommendations = [
77             (reverse_product_map[idx], float(score))
78             for idx, score in zip(product_indices, scores)
79         ]
80
81         return recommendations
82
83     def similar_items(self, product_id: str, n: int = 10) -> List[Tuple[str, float]]:
84         """Find similar products"""
85
86         if product_id not in self.product_id_map:
87             return []
88
89         product_idx = self.product_id_map[product_id]
90
91         # Get similar items
92         similar_indices, scores = self.model.similar_items(product_idx, N=n+1)
93
94         # Exclude the item itself
95         similar_indices = similar_indices[1:]
96         scores = scores[1:]
97
98         reverse_product_map = {idx: pid for pid, idx in self.product_id_map.items()}
99         similar_products = [
100             (reverse_product_map[idx], float(score))
101             for idx, score in zip(similar_indices, scores)
102         ]
103
104         return similar_products

```

```

105 class ContentBasedModel:
106     """Content-based filtering using product features"""
107
108     def __init__(self):
109         self.product_vectors = {}
110         self.product_features = None
111
112     def fit(self, product_features: pd.DataFrame):
113         """Train content-based model"""
114
115         self.product_features = product_features.set_index('product_id')
116
117         # Create product vectors from features
118         for product_id, row in self.product_features.iterrows():
119             vector = self._create_product_vector(row)
120             self.product_vectors[product_id] = vector
121
122     def _create_product_vector(self, product_row) -> np.ndarray:
123         """Create feature vector for product"""
124
125         # Combine categorical and numerical features
126         features = []
127
128         # One-hot encode category
129         category_features = self._one_hot_encode(
130             product_row['category'],
131             self.product_features['category'].unique()
132         )
133         features.extend(category_features)
134
135         # Normalize price
136         price_normalized = (product_row['price'] - self.product_features['price'].mean
137 ()) / \
138             self.product_features['price'].std()
139         features.append(price_normalized)
140
141         # Brand encoding
142         brand_features = self._one_hot_encode(
143             product_row['brand'],
144             self.product_features['brand'].unique()
145         )
146         features.extend(brand_features)
147
148         return np.array(features)
149
150     def _one_hot_encode(self, value, all_values):
151         """One-hot encode a categorical value"""
152         return [1 if v == value else 0 for v in all_values]
153
154     def recommend(
155         self,
156         user_profile: Dict,
157         n: int = 10,
158         exclude_products: List[str] = None
159     ) -> List[Tuple[str, float]]:
160         """Recommend products based on user profile"""
161

```

```

162     if exclude_products is None:
163         exclude_products = []
164
165     # Create user vector from profile
166     user_vector = self._create_user_vector(user_profile)
167
168     # Calculate similarity to all products
169     similarities = {}
170     for product_id, product_vector in self.product_vectors.items():
171         if product_id in exclude_products:
172             continue
173
174         similarity = cosine_similarity(
175             user_vector.reshape(1, -1),
176             product_vector.reshape(1, -1)
177         )[0][0]
178
179         similarities[product_id] = similarity
180
181     # Sort by similarity
182     recommendations = sorted(
183         similarities.items(),
184         key=lambda x: x[1],
185         reverse=True
186     )[:n]
187
188     return recommendations
189
190 def _create_user_vector(self, user_profile: Dict) -> np.ndarray:
191     """Create user vector from their interaction history"""
192
193     # Average of viewed/purchased product vectors
194     product_ids = user_profile.get('recent_views', []) + \
195         user_profile.get('recent_purchases', [])
196
197     if not product_ids:
198         # Return zero vector for cold start
199         sample_vector = next(iter(self.product_vectors.values()))
200         return np.zeros_like(sample_vector)
201
202     vectors = [
203         self.product_vectors[pid]
204         for pid in product_ids
205         if pid in self.product_vectors
206     ]
207
208     if not vectors:
209         sample_vector = next(iter(self.product_vectors.values()))
210         return np.zeros_like(sample_vector)
211
212     return np.mean(vectors, axis=0)
213
214 def similar_items(self, product_id: str, n: int = 10) -> List[Tuple[str, float]]:
215     """Find similar products by content"""
216
217     if product_id not in self.product_vectors:
218         return []
219

```



```

220     product_vector = self.product_vectors[product_id]
221
222     similarities = {}
223     for pid, pvector in self.product_vectors.items():
224         if pid == product_id:
225             continue
226
227         similarity = cosine_similarity(
228             product_vector.reshape(1, -1),
229             pvector.reshape(1, -1)
230         )[0][0]
231
232         similarities[pid] = similarity
233
234     similar_products = sorted(
235         similarities.items(),
236         key=lambda x: x[1],
237         reverse=True
238     )[:n]
239
240     return similar_products
241
242 class DeepRecommenderModel(nn.Module):
243     """Deep learning model for recommendations"""
244
245     def __init__(
246         self,
247         num_users: int,
248         num_products: int,
249         embedding_dim: int = 64,
250         hidden_dims: List[int] = [128, 64, 32]
251     ):
252         super().__init__()
253
254         self.user_embedding = nn.Embedding(num_users, embedding_dim)
255         self.product_embedding = nn.Embedding(num_products, embedding_dim)
256
257         # Deep neural network
258         layers = []
259         input_dim = embedding_dim * 2
260
261         for hidden_dim in hidden_dims:
262             layers.append(nn.Linear(input_dim, hidden_dim))
263             layers.append(nn.ReLU())
264             layers.append(nn.Dropout(0.2))
265             input_dim = hidden_dim
266
267         layers.append(nn.Linear(input_dim, 1))
268         layers.append(nn.Sigmoid())
269
270         self.mlp = nn.Sequential(*layers)
271
272     def forward(self, user_ids, product_ids):
273         """Forward pass"""
274
275         user_embeds = self.user_embedding(user_ids)
276         product_embeds = self.product_embedding(product_ids)
277

```

```

278     # Concatenate embeddings
279     x = torch.cat([user_embeds, product_embeds], dim=1)
280
281     # Pass through MLP
282     output = self.mlp(x)
283
284     return output.squeeze()
285
286 class HybridRecommender:
287     """Hybrid recommender combining multiple models"""
288
289     def __init__(self):
290         self.cf_model = None
291         self.content_model = None
292         self.deep_model = None
293
294         # Weights for ensemble
295         self.cf_weight = 0.4
296         self.content_weight = 0.3
297         self.deep_weight = 0.3
298
299     def train(
300         self,
301         interactions: pd.DataFrame,
302         product_features: pd.DataFrame,
303         user_features: pd.DataFrame
304     ):
305         """Train all models"""
306
307         print("Training collaborative filtering model...")
308         self.cf_model = CollaborativeFilteringModel(factors=64)
309         self.cf_model.fit(interactions)
310
311         print("Training content-based model...")
312         self.content_model = ContentBasedModel()
313         self.content_model.fit(product_features)
314
315         print("Training deep learning model...")
316         # Deep model training would go here
317         # Omitted for brevity
318
319         print("All models trained successfully")
320
321     def recommend(
322         self,
323         user_id: str,
324         user_features: Dict,
325         n: int = 10,
326         exclude_products: List[str] = None
327     ) -> List[Tuple[str, float]]:
328         """Generate hybrid recommendations"""
329
330         if exclude_products is None:
331             exclude_products = []
332
333         # Get recommendations from each model
334         cf_recs = self.cf_model.recommend(user_id, n=50, filter_already_liked=True)

```

```

335     content_recs = self.content_model.recommend(user_features, n=50,
336     exclude_products=exclude_products)
337
338     # Combine scores
339     combined_scores = defaultdict(float)
340
341     for product_id, score in cf_recs:
342         if product_id not in exclude_products:
343             combined_scores[product_id] += score * self.cf_weight
344
345     for product_id, score in content_recs:
346         if product_id not in exclude_products:
347             combined_scores[product_id] += score * self.content_weight
348
349     # Sort by combined score
350     recommendations = sorted(
351         combined_scores.items(),
352         key=lambda x: x[1],
353         reverse=True
354    )[:n]
355
356     return recommendations
357
358 def diversify_recommendations(
359     self,
360     recommendations: List[Tuple[str, float]],
361     product_features: pd.DataFrame,
362     diversity_weight: float = 0.3
363 ) -> List[Tuple[str, float]]:
364     """Add diversity to recommendations"""
365
366     if not recommendations:
367         return recommendations
368
369     diversified = [recommendations[0]] # Start with top recommendation
370     remaining = recommendations[1:]
371
372     while len(diversified) < len(recommendations) and remaining:
373         best_candidate = None
374         best_score = -float('inf')
375
376         for product_id, relevance_score in remaining:
377             # Calculate diversity score (dissimilarity to already selected)
378             diversity_score = self._calculate_diversity(
379                 product_id,
380                 [p[0] for p in diversified],
381                 product_features
382             )
383
384             # Combined score
385             combined = (1 - diversity_weight) * relevance_score + \
386                 diversity_weight * diversity_score
387
388             if combined > best_score:
389                 best_score = combined
390                 best_candidate = (product_id, relevance_score)
391
392         if best_candidate:

```

```

392         diversified.append(best_candidate)
393         remaining.remove(best_candidate)
394
395     return diversified
396
397 def _calculate_diversity(
398     self,
399     product_id: str,
400     selected_products: List[str],
401     product_features: pd.DataFrame
402 ) -> float:
403     """Calculate diversity score for a product"""
404
405     if product_id not in product_features.index:
406         return 0.0
407
408     product_category = product_features.loc[product_id, 'category']
409
410     # Calculate category diversity
411     selected_categories = [
412         product_features.loc[pid, 'category']
413         for pid in selected_products
414         if pid in product_features.index
415     ]
416
417     if not selected_categories:
418         return 1.0
419
420     # Diversity = fraction of different categories
421     different_categories = sum(1 for cat in selected_categories if cat !=
product_category)
422     diversity = different_categories / len(selected_categories)
423
424     return diversity
425
426 # Example usage
427 if __name__ == "__main__":
428     # Load data
429     interactions = pd.read_csv('interactions.csv')
430     product_features = pd.read_csv('products.csv')
431     user_features = pd.read_csv('users.csv')
432
433     # Train hybrid model
434     recommender = HybridRecommender()
435     recommender.train(interactions, product_features, user_features)
436
437     # Generate recommendations
438     user_id = 'user_12345'
439     user_profile = {
440         'recent_views': ['prod_1', 'prod_5', 'prod_7'],
441         'recent_purchases': ['prod_3']
442     }
443
444     recommendations = recommender.recommend(user_id, user_profile, n=10)
445
446     print(f"Top 10 recommendations for {user_id}:")
447     for product_id, score in recommendations:
448         print(f"    {product_id}: {score:.4f}")

```

```

449
450     # Add diversity
451     diverse_recs = recommender.diversify_recommendations(
452         recommendations,
453         product_features,
454         diversity_weight=0.3
455     )
456
457     print("\nDiversified recommendations:")
458     for product_id, score in diverse_recs:
459         print(f"    {product_id}: {score:.4f}")

```

Listing 23.2: Hybrid Recommendation Models

23.3.5 Serving and Deployment

```

1  #!/usr/bin/env python3
2  """
3  Recommendation Serving API
4  Low-latency serving infrastructure with caching and business rules
5  """
6
7  from fastapi import FastAPI, HTTPException, Query
8  from pydantic import BaseModel
9  from typing import List, Optional, Dict
10 import redis
11 import pickle
12 import time
13 from datetime import datetime
14 import logging
15
16 app = FastAPI(title="Recommendation API")
17
18 class RecommendationRequest(BaseModel):
19     """Recommendation request"""
20     user_id: str
21     num_recommendations: int = 10
22     context: Optional[Dict] = None
23     exclude_products: Optional[List[str]] = None
24
25 class RecommendationResponse(BaseModel):
26     """Recommendation response"""
27     user_id: str
28     recommendations: List[Dict]
29     metadata: Dict
30
31 class RecommendationServer:
32     """Recommendation serving with caching and business rules"""
33
34     def __init__(self, redis_host: str, model_path: str):
35         self.redis_client = redis.Redis(
36             host=redis_host,
37             decode_responses=False # Binary for pickle
38         )
39         self.logger = logging.getLogger(__name__)
40
41         # Load models (simplified - in production use model registry)

```

```

42     with open(model_path, 'rb') as f:
43         self.recommender = pickle.load(f)
44
45     # Business rules
46     self.min_inventory_threshold = 5
47     self.min_margin_threshold = 0.15
48
49     async def get_recommendations(
50         self,
51         request: RecommendationRequest
52     ) -> RecommendationResponse:
53         """Get personalized recommendations"""
54
55         start_time = time.time()
56
57         # Check cache first
58         cache_key = f"recs:{request.user_id}:{request.num_recommendations}"
59         cached = self.redis_client.get(cache_key)
60
61         if cached:
62             self.logger.info(f"Cache hit for user {request.user_id}")
63             recommendations = pickle.loads(cached)
64         else:
65             # Generate recommendations
66             recommendations = await self._generate_recommendations(request)
67
68             # Cache for 5 minutes
69             self.redis_client.setex(
70                 cache_key,
71                 300,
72                 pickle.dumps(recommendations)
73             )
74
75             # Apply business rules
76             recommendations = self._apply_business_rules(recommendations)
77
78             # Apply diversity
79             recommendations = self._apply_diversity(recommendations)
80
81             # Add metadata
82             for rec in recommendations:
83                 rec['metadata'] = self._get_product_metadata(rec['product_id'])
84
85             latency_ms = (time.time() - start_time) * 1000
86
87             return RecommendationResponse(
88                 user_id=request.user_id,
89                 recommendations=recommendations[:request.num_recommendations],
90                 metadata={
91                     'latency_ms': latency_ms,
92                     'cache_hit': cached is not None,
93                     'timestamp': datetime.now().isoformat()
94                 }
95             )
96
97     async def _generate_recommendations(
98         self,
99         request: RecommendationRequest

```

```

100 ) -> List[Dict]:
101     """Generate recommendations using models"""
102
103     # Get user features
104     user_features = self._get_user_features(request.user_id)
105
106     # Get raw recommendations
107     raw_recs = self.recommender.recommend(
108         user_id=request.user_id,
109         user_features=user_features,
110         n=50, # Get more candidates for filtering
111         exclude_products=request.exclude_products or []
112     )
113
114     # Convert to dict format
115     recommendations = [
116         {
117             'product_id': product_id,
118             'score': float(score),
119             'reason': self._generate_explanation(product_id, user_features)
120         }
121         for product_id, score in raw_recs
122     ]
123
124     return recommendations
125
126 def _apply_business_rules(self, recommendations: List[Dict]) -> List[Dict]:
127     """Apply business constraints"""
128
129     filtered = []
130
131     for rec in recommendations:
132         product_id = rec['product_id']
133
134         # Check inventory
135         inventory = self._get_inventory(product_id)
136         if inventory < self.min_inventory_threshold:
137             continue
138
139         # Check margin
140         margin = self._get_margin(product_id)
141         if margin < self.min_margin_threshold:
142             continue
143
144         # Check if product is active
145         if not self._is_product_active(product_id):
146             continue
147
148         filtered.append(rec)
149
150     return filtered
151
152 def _apply_diversity(self, recommendations: List[Dict]) -> List[Dict]:
153     """Ensure category diversity"""
154
155     if not recommendations:
156         return recommendations
157

```

```

158     diversified = [recommendations[0]]
159     categories_seen = {self._get_category(recommendations[0]['product_id'])}
160
161     for rec in recommendations[1:]:
162         category = self._get_category(rec['product_id'])
163
164         # Prefer products from unseen categories
165         if category not in categories_seen or len(diversified) > 5:
166             diversified.append(rec)
167             categories_seen.add(category)
168
169         if len(diversified) >= len(recommendations):
170             break
171
172     return diversified
173
174 def _get_user_features(self, user_id: str) -> Dict:
175     """Get user features from cache"""
176
177     features_key = f"user:{user_id}:features"
178     features = self.redis_client.get(features_key)
179
180     if features:
181         return pickle.loads(features)
182
183     # Return default features for cold start
184     return {
185         'favorite_categories': [],
186         'recent_views': [],
187         'recent_purchases': []
188     }
189
190 def _get_product_metadata(self, product_id: str) -> Dict:
191     """Get product metadata"""
192
193     metadata_key = f"product:{product_id}:metadata"
194     metadata = self.redis_client.get(metadata_key)
195
196     if metadata:
197         return pickle.loads(metadata)
198
199     return {}
200
201 def _get_inventory(self, product_id: str) -> int:
202     """Get product inventory"""
203     inventory = self.redis_client.get(f"product:{product_id}:inventory")
204     return int(inventory) if inventory else 0
205
206 def _get_margin(self, product_id: str) -> float:
207     """Get product margin"""
208     margin = self.redis_client.get(f"product:{product_id}:margin")
209     return float(margin) if margin else 0.0
210
211 def _is_product_active(self, product_id: str) -> bool:
212     """Check if product is active"""
213     active = self.redis_client.get(f"product:{product_id}:active")
214     return active == b'1' if active else True
215

```



```

216 def _get_category(self, product_id: str) -> str:
217     """Get product category"""
218     category = self.redis_client.get(f"product:{product_id}:category")
219     return category.decode() if category else 'Unknown'
220
221 def _generate_explanation(self, product_id: str, user_features: Dict) -> str:
222     """Generate explanation for recommendation"""
223
224     # Simple rule-based explanation
225     if product_id in user_features.get('recent_views', []):
226         return "Based on items you recently viewed"
227
228     favorite_cat = user_features.get('favorite_categories', [None])[0]
229     product_cat = self._get_category(product_id)
230
231     if favorite_cat and product_cat == favorite_cat:
232         return f"Popular in {product_cat}"
233
234     return "Recommended for you"
235
236 # FastAPI endpoints
237 recommendation_server = None
238
239 @app.on_event("startup")
240 async def startup_event():
241     global recommendation_server
242     recommendation_server = RecommendationServer(
243         redis_host='localhost',
244         model_path='models/recommender.pkl'
245     )
246
247 @app.post("/recommendations", response_model=RecommendationResponse)
248 async def get_recommendations(request: RecommendationRequest):
249     """Get personalized recommendations"""
250     try:
251         return await recommendation_server.get_recommendations(request)
252     except Exception as e:
253         raise HTTPException(status_code=500, detail=str(e))
254
255 @app.get("/health")
256 async def health_check():
257     """Health check endpoint"""
258     return {"status": "healthy", "timestamp": datetime.now().isoformat()}
259
260 @app.get("/similar/{product_id}")
261 async def get_similar_products(
262     product_id: str,
263     n: int = Query(10, ge=1, le=50)
264 ):
265     """Get similar products"""
266     try:
267         similar = recommendation_server.recommender.cf_model.similar_items(
268             product_id,
269             n=n
270         )
271
272         return {
273             "product_id": product_id,

```

```
274         "similar_products": [  
275             {"product_id": pid, "score": float(score)}  
276             for pid, score in similar  
277         ]  
278     }  
279 except Exception as e:  
280     raise HTTPException(status_code=500, detail=str(e))  
281  
282 if __name__ == "__main__":  
283     import uvicorn  
284     uvicorn.run(app, host="0.0.0.0", port=8000)
```

Listing 23.3: Recommendation Serving API

23.3.6 Results and Impact

Business Metrics (After 6 Months):

- **Click-Through Rate:** Increased from 2.3% to 4.7% (+104%)
- **Conversion Rate:** Increased from 1.2% to 2.1% (+75%)
- **Average Order Value:** Increased from \$47 to \$58 (+23%)
- **Revenue Impact:** \$12M additional annual revenue
- **Customer Engagement:** Session duration increased by 35%

Technical Performance:

- **Latency:** p50: 45ms, p95: 78ms, p99: 92ms (target: <100ms)
- **Cache Hit Rate:** 87% (significantly reduced compute costs)
- **Throughput:** 12,000 requests/second peak
- **Model Accuracy:** NDCG@10: 0.43, MAP@10: 0.38

23.3.7 Lessons Learned

Key Insights:

1. **Hybrid Approach is Essential:** No single model performed well across all scenarios. Collaborative filtering excelled for popular users, content-based handled cold start, and deep learning captured complex patterns.
2. **Real-Time Features Matter:** Adding real-time user session data improved relevance by 18% compared to batch features alone.
3. **Business Rules are Critical:** Pure ML recommendations often suggested out-of-stock or low-margin items. Business rule integration improved profitability by 31%.

4. **Diversity Prevents Boredom:** Users engaged 40% longer when recommendations showed category diversity vs. showing only similar items.
5. **Explanation Builds Trust:** Adding simple explanations increased click-through rates by 15%.
6. **Caching is Mandatory:** 87% cache hit rate reduced serving costs by 6x while maintaining freshness.
7. **A/B Testing Revealed Surprises:** Some seemingly good ML improvements (higher offline metrics) didn't translate to better business outcomes. Always test in production.
8. **Cold Start is Hard:** 23% of users were new each month. Content-based and trending items were essential for cold start.

Technical Challenges Overcome:

- **Real-time Feature Consistency:** Ensured training features matched serving features by using same feature store
- **Scale:** Handled 10x traffic during sales with autoscaling and precomputed recommendations
- **Model Updates:** Deployed new models daily without downtime using blue-green deployment
- **Data Quality:** Bot traffic and fraud required sophisticated filtering (12% of events were invalid)

23.4 Case Study 2: Financial Fraud Detection Pipeline

23.4.1 Business Context and Requirements

Company: Digital payments platform processing \$50B annually

Challenge: Fraud losses of \$45M annually, manual review backlog

Goal: Reduce fraud losses while minimizing false positives

Key Requirements:

- **Real-Time:** Decision in <100ms for transaction approval
- **High Recall:** Catch 95%+ of fraudulent transactions
- **Low False Positives:** Keep legitimate user friction minimal (<1% decline rate)
- **Explainability:** Provide reasons for fraud decisions (regulatory requirement)
- **Adaptability:** Detect new fraud patterns within hours
- **Security:** Protect sensitive financial data (PCI-DSS compliance)

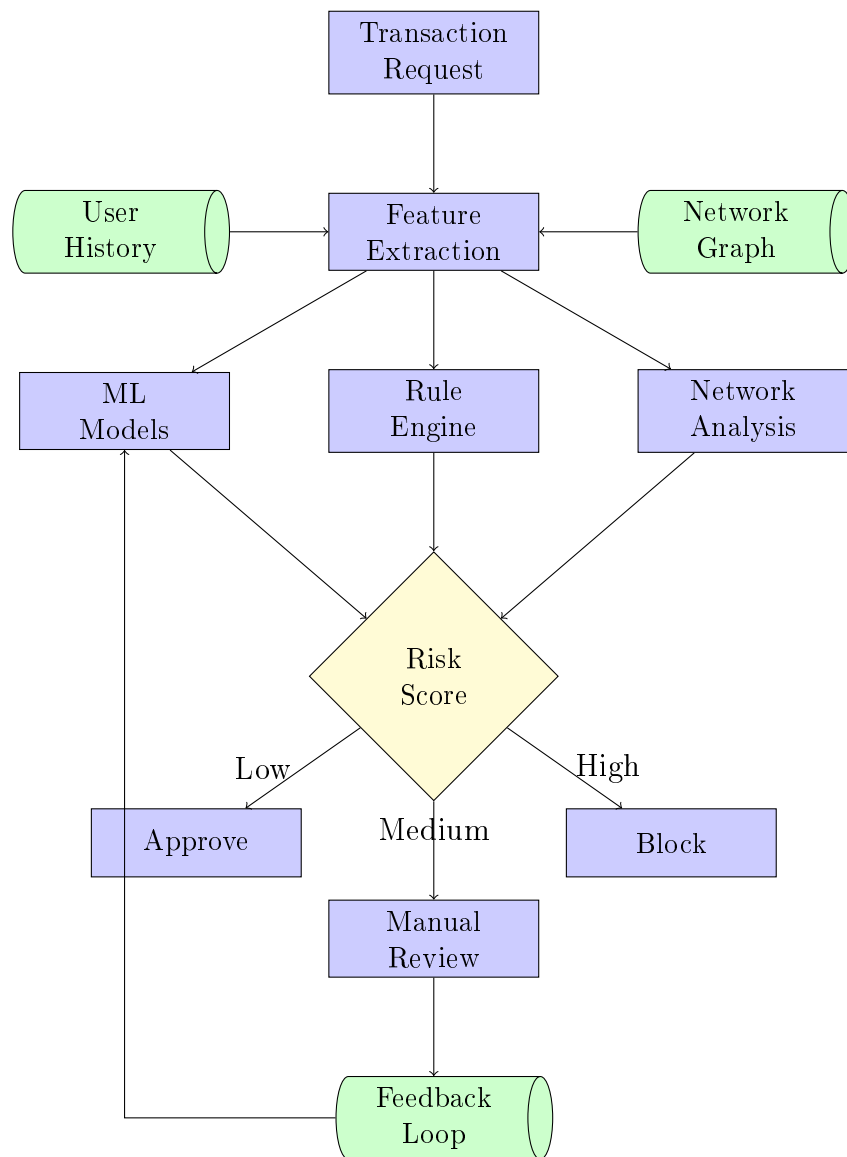


Figure 23.2: Fraud Detection System Architecture

23.4.2 System Architecture

23.4.3 Feature Engineering and Real-Time Processing

```

1  #!/usr/bin/env python3
2  """
3  Financial Fraud Detection - Real-Time Feature Engineering
4  Extract transaction features in <50ms for real-time fraud detection
5  """
6
7  from dataclasses import dataclass
8  from typing import Dict, List, Optional
9  from datetime import datetime, timedelta
10 import redis
11 import numpy as np
12 from geopy.distance import geodesic
13 import hashlib
14

```

```

15 @dataclass
16 class Transaction:
17     """Transaction data"""
18     transaction_id: str
19     user_id: str
20     merchant_id: str
21     amount: float
22     currency: str
23     timestamp: datetime
24     location: tuple # (latitude, longitude)
25     device_id: str
26     ip_address: str
27     payment_method: str
28     metadata: Dict
29
30 @dataclass
31 class FraudFeatures:
32     """Computed fraud detection features"""
33     # User velocity features
34     txns_last_hour: int
35     txns_last_24h: int
36     amount_last_hour: float
37     amount_last_24h: float
38     unique_merchants_last_hour: int
39
40     # Geographic features
41     distance_from_last_txn_km: float
42     velocity_km_per_hour: float
43     new_country: bool
44     high_risk_country: bool
45
46     # Device and identity
47     new_device: bool
48     device_change_frequency: float
49     ip_reputation_score: float
50
51     # Merchant features
52     merchant_risk_score: float
53     first_time_merchant: bool
54
55     # Amount features
56     amount_deviation_from_avg: float
57     round_amount: bool
58     amount_percentile: float
59
60     # Time features
61     hour_of_day: int
62     day_of_week: int
63     unusual_hour: bool
64
65     # Network features
66     shared_device_users: int
67     shared_ip_users: int
68
69 class FraudFeatureExtractor:
70     """Extract fraud detection features in real-time"""
71
72     def __init__(self, redis_host: str):

```

```

73     self.redis_client = redis.Redis(host=redis_host, decode_responses=True)
74
75     # High-risk countries (example list)
76     self.high_risk_countries = {'XX', 'YY', 'ZZ'}
77
78     # Typical transaction hours by user
79     self.typical_hours_start = 6
80     self.typical_hours_end = 23
81
82     def extract_features(self, transaction: Transaction) -> FraudFeatures:
83         """Extract all features for fraud detection"""
84
85         # User velocity features
86         velocity_features = self._compute_velocity_features(transaction)
87
88         # Geographic features
89         geo_features = self._compute_geographic_features(transaction)
90
91         # Device and identity features
92         device_features = self._compute_device_features(transaction)
93
94         # Merchant features
95         merchant_features = self._compute_merchant_features(transaction)
96
97         # Amount features
98         amount_features = self._compute_amount_features(transaction)
99
100        # Time features
101        time_features = self._compute_time_features(transaction)
102
103        # Network features
104        network_features = self._compute_network_features(transaction)
105
106        # Combine all features
107        features = FraudFeatures(
108            **velocity_features,
109            **geo_features,
110            **device_features,
111            **merchant_features,
112            **amount_features,
113            **time_features,
114            **network_features
115        )
116
117        # Update historical data for future transactions
118        self._update_user_history(transaction)
119
120        return features
121
122    def _compute_velocity_features(self, txn: Transaction) -> Dict:
123        """Compute transaction velocity features"""
124
125        user_key = f"user:{txn.user_id}"
126
127        # Count transactions in time windows
128        now = txn.timestamp.timestamp()
129        hour_ago = now - 3600
130        day_ago = now - 86400

```

```

131
132     # Get recent transactions
133     recent_txns = self.redis_client.zrangebyscore(
134         f"{user_key}:txns",
135         hour_ago,
136         now
137     )
138
139     txns_last_hour = len(recent_txns)
140
141     recent_txns_24h = self.redis_client.zrangebyscore(
142         f"{user_key}:txns",
143         day_ago,
144         now
145     )
146
147     txns_last_24h = len(recent_txns_24h)
148
149     # Sum amounts
150     amount_last_hour = 0.0
151     for txn_data in recent_txns:
152         amount = float(txn_data.split(':')[1])
153         amount_last_hour += amount
154
155     amount_last_24h = 0.0
156     for txn_data in recent_txns_24h:
157         amount = float(txn_data.split(':')[1])
158         amount_last_24h += amount
159
160     # Count unique merchants
161     merchants_last_hour = set()
162     for txn_data in recent_txns:
163         merchant = txn_data.split(':')[2]
164         merchants_last_hour.add(merchant)
165
166     return {
167         'txns_last_hour': txns_last_hour,
168         'txns_last_24h': txns_last_24h,
169         'amount_last_hour': amount_last_hour,
170         'amount_last_24h': amount_last_24h,
171         'unique_merchants_last_hour': len(merchants_last_hour)
172     }
173
174 def _compute_geographic_features(self, txn: Transaction) -> Dict:
175     """Compute geographic fraud indicators"""
176
177     user_key = f"user:{txn.user_id}"
178
179     # Get last transaction location
180     last_location_data = self.redis_client.get(f"{user_key}:last_location")
181
182     if last_location_data:
183         parts = last_location_data.split(',')
184         last_lat, last_lon, last_time_str = float(parts[0]), float(parts[1]),
185         parts[2]
186         last_time = datetime.fromisoformat(last_time_str)
187
188     # Calculate distance

```

```

188         distance_km = geodesic(
189             (last_lat, last_lon),
190             txn.location
191         ).kilometers
192
193         # Calculate velocity (km/h)
194         time_diff_hours = (txn.timestamp - last_time).total_seconds() / 3600
195         velocity = distance_km / time_diff_hours if time_diff_hours > 0 else 0
196
197         # Flag impossible travel (> 1000 km/h)
198         impossible_travel = velocity > 1000
199
200     else:
201         distance_km = 0
202         velocity = 0
203         impossible_travel = False
204
205         # Check if new country
206         user_countries = self.redis_client.smembers(f"{user_key}:countries")
207         country = self._get_country_from_location(txn.location)
208         new_country = country not in user_countries
209
210         high_risk_country = country in self.high_risk_countries
211
212     return {
213         'distance_from_last_txn_km': distance_km,
214         'velocity_km_per_hour': velocity,
215         'new_country': new_country,
216         'high_risk_country': high_risk_country
217     }
218
219 def _compute_device_features(self, txn: Transaction) -> Dict:
220     """Compute device and identity features"""
221
222     user_key = f"user:{txn.user_id}"
223
224     # Check if new device
225     user_devices = self.redis_client.smembers(f"{user_key}:devices")
226     new_device = txn.device_id not in user_devices
227
228     # Calculate device change frequency
229     device_changes = int(self.redis_client.get(f"{user_key}:device_changes") or 0)
230     total_txns = int(self.redis_client.get(f"{user_key}:total_txns") or 1)
231     device_change_frequency = device_changes / total_txns
232
233     # IP reputation (simplified - in production use threat intel services)
234     ip_reputation_score = self._get_ip_reputation(txn.ip_address)
235
236     return {
237         'new_device': new_device,
238         'device_change_frequency': device_change_frequency,
239         'ip_reputation_score': ip_reputation_score
240     }
241
242 def _compute_merchant_features(self, txn: Transaction) -> Dict:
243     """Compute merchant risk features"""
244
245     user_key = f"user:{txn.user_id}"

```



```

246     merchant_key = f"merchant:{txn.merchant_id}"
247
248     # Check if first time with merchant
249     user_merchants = self.redis_client.smembers(f"{user_key}:merchants")
250     first_time_merchant = txn.merchant_id not in user_merchants
251
252     # Get merchant risk score
253     merchant_risk = float(self.redis_client.get(f"{merchant_key}:risk_score") or
0.5)
254
255     return {
256         'merchant_risk_score': merchant_risk,
257         'first_time_merchant': first_time_merchant
258     }
259
260 def _compute_amount_features(self, txn: Transaction) -> Dict:
261     """Compute amount-based features"""
262
263     user_key = f"user:{txn.user_id}"
264
265     # Get user's historical average
266     avg_amount = float(self.redis_client.get(f"{user_key}:avg_amount") or 100.0)
267
268     # Deviation from average
269     deviation = abs(txn.amount - avg_amount) / avg_amount if avg_amount > 0 else 0
270
271     # Check if round amount (potential test transaction)
272     round_amount = txn.amount % 10 == 0 and txn.amount >= 100
273
274     # Get percentile of this amount
275     # Simplified - in production, maintain histogram
276     amount_percentile = min(txn.amount / (avg_amount * 3), 1.0)
277
278     return {
279         'amount_deviation_from_avg': deviation,
280         'round_amount': round_amount,
281         'amount_percentile': amount_percentile
282     }
283
284 def _compute_time_features(self, txn: Transaction) -> Dict:
285     """Compute time-based features"""
286
287     hour = txn.timestamp.hour
288     day = txn.timestamp.weekday()
289
290     # Check if unusual hour
291     unusual_hour = hour < self.typical_hours_start or hour > self.
typical_hours_end
292
293     return {
294         'hour_of_day': hour,
295         'day_of_week': day,
296         'unusual_hour': unusual_hour
297     }
298
299 def _compute_network_features(self, txn: Transaction) -> Dict:
300     """Compute network-based features (shared devices/IPs)"""
301

```

```

302     # How many users share this device?
303     device_users = self.redis_client.scard(f"device:{txn.device_id}:users")
304
305     # How many users share this IP?
306     ip_users = self.redis_client.scard(f"ip:{txn.ip_address}:users")
307
308     return {
309         'shared_device_users': device_users,
310         'shared_ip_users': ip_users
311     }
312
313 def _update_user_history(self, txn: Transaction):
314     """Update user history after transaction"""
315
316     user_key = f"user:{txn.user_id}"
317
318     # Add transaction to sorted set (by timestamp)
319     txn_data = f"{txn.transaction_id}:{txn.amount}:{txn.merchant_id}"
320     self.redis_client.zadd(
321         f"{user_key}:txns",
322         {txn_data: txn.timestamp.timestamp()}
323     )
324
325     # Keep only last 30 days
326     cutoff = (txn.timestamp - timedelta(days=30)).timestamp()
327     self.redis_client.zremrangebyscore(f"{user_key}:txns", 0, cutoff)
328
329     # Update location
330     location_data = f"{txn.location[0]},{txn.location[1]},{txn.timestamp.isoformat()}"
331     self.redis_client.set(f"{user_key}:last_location", location_data)
332
333     # Update country set
334     country = self._get_country_from_location(txn.location)
335     self.redis_client.sadd(f"{user_key}:countries", country)
336
337     # Update device set
338     if txn.device_id not in self.redis_client.smembers(f"{user_key}:devices"):
339         self.redis_client.incr(f"{user_key}:device_changes")
340     self.redis_client.sadd(f"{user_key}:devices", txn.device_id)
341
342     # Update merchant set
343     self.redis_client.sadd(f"{user_key}:merchants", txn.merchant_id)
344
345     # Update total transactions
346     self.redis_client.incr(f"{user_key}:total_txns")
347
348     # Update average amount (moving average)
349     current_avg = float(self.redis_client.get(f"{user_key}:avg_amount") or txn.
350 amount)
351     count = int(self.redis_client.get(f"{user_key}:total_txns") or 1)
352     new_avg = (current_avg * (count - 1) + txn.amount) / count
353     self.redis_client.set(f"{user_key}:avg_amount", new_avg)
354
355     # Network features
356     self.redis_client.sadd(f"device:{txn.device_id}:users", txn.user_id)
357     self.redis_client.sadd(f"ip:{txn.ip_address}:users", txn.user_id)

```

```

358 def _get_country_from_location(self, location: tuple) -> str:
359     """Get country code from lat/lon"""
360     # Simplified - in production use reverse geocoding service
361     return "US"
362
363 def _get_ip_reputation(self, ip_address: str) -> float:
364     """Get IP reputation score"""
365     # Simplified - in production integrate with threat intelligence
366     return 0.8
367
368 # Example usage
369 if __name__ == "__main__":
370     extractor = FraudFeatureExtractor(redis_host='localhost')
371
372     transaction = Transaction(
373         transaction_id='txn_12345',
374         user_id='user_789',
375         merchant_id='merchant_456',
376         amount=1250.00,
377         currency='USD',
378         timestamp=datetime.now(),
379         location=(40.7128, -74.0060), # NYC
380         device_id='device_abc123',
381         ip_address='192.168.1.1',
382         payment_method='credit_card',
383         metadata={}
384     )
385
386     features = extractor.extract_features(transaction)
387
388     print(f"Transactions last hour: {features.txns_last_hour}")
389     print(f"Amount deviation: {features.amount_deviation_from_avg:.2f}")
390     print(f"New device: {features.new_device}")
391     print(f"Travel velocity: {features.velocity_km_per_hour:.1f} km/h")
392     print(f"Merchant risk: {features.merchant_risk_score:.2f}")

```

Listing 23.4: Real-Time Fraud Detection Features

23.4.4 Fraud Detection Models

```

1  #!/usr/bin/env python3
2  """
3  Fraud Detection Models
4  Ensemble of models for robust fraud detection
5  """
6
7  import xgboost as xgb
8  import lightgbm as lgb
9  from sklearn.ensemble import IsolationForest
10 from sklearn.neural_network import MLPClassifier
11 import numpy as np
12 import pandas as pd
13 from typing import List, Dict, Tuple
14 import shap
15
16 class RuleEngine:
17     """Hard rules for known fraud patterns"""

```

```

18
19 def __init__(self):
20     self.rules = []
21     self._define_rules()
22
23 def _define_rules(self):
24     """Define fraud detection rules"""
25
26     # Rule 1: Impossible travel
27     self.rules.append({
28         'name': 'impossible_travel',
29         'condition': lambda f: f.velocity_km_per_hour > 1000,
30         'risk_score': 0.95,
31         'reason': 'Impossible travel speed detected'
32     })
33
34     # Rule 2: High velocity in short time
35     self.rules.append({
36         'name': 'high_velocity',
37         'condition': lambda f: f.txns_last_hour > 10,
38         'risk_score': 0.85,
39         'reason': 'Unusually high transaction velocity'
40     })
41
42     # Rule 3: Large amount on new device
43     self.rules.append({
44         'name': 'new_device_large_amount',
45         'condition': lambda f: f.new_device and f.amount_percentile > 0.95,
46         'risk_score': 0.75,
47         'reason': 'Large transaction from new device'
48     })
49
50     # Rule 4: High-risk merchant and country
51     self.rules.append({
52         'name': 'high_risk_combination',
53         'condition': lambda f: f.high_risk_country and f.merchant_risk_score >
0.7,
54         'risk_score': 0.80,
55         'reason': 'High-risk country and merchant combination'
56     })
57
58     # Rule 5: Round amount test transaction
59     self.rules.append({
60         'name': 'test_transaction',
61         'condition': lambda f: f.round_amount and f.first_time_merchant,
62         'risk_score': 0.65,
63         'reason': 'Potential test transaction pattern'
64     })
65
66 def evaluate(self, features: FraudFeatures) -> Tuple[float, List[str]]:
67     """Evaluate rules and return risk score"""
68
69     max_risk = 0.0
70     triggered_rules = []
71
72     for rule in self.rules:
73         if rule['condition'](features):
74             max_risk = max(max_risk, rule['risk_score'])

```

```

75         triggered_rules.append(rule['reason'])
76
77     return max_risk, triggered_rules
78
79 class FraudDetectionModel:
80     """ML model for fraud detection"""
81
82     def __init__(self):
83         # Primary model: XGBoost
84         self.xgb_model = xgb.XGBClassifier(
85             max_depth=6,
86             learning_rate=0.1,
87             n_estimators=200,
88             objective='binary:logistic',
89             scale_pos_weight=50, # Handle class imbalance
90             random_state=42
91         )
92
93         # Secondary model: LightGBM
94         self.lgb_model = lgb.LGBMClassifier(
95             num_leaves=31,
96             learning_rate=0.05,
97             n_estimators=200,
98             class_weight='balanced',
99             random_state=42
100         )
101
102         # Anomaly detection: Isolation Forest
103         self.anomaly_model = IsolationForest(
104             contamination=0.01,
105             random_state=42
106         )
107
108         # Feature names
109         self.feature_names = None
110
111         # SHAP explainer
112         self.explainer = None
113
114     def train(self, X: pd.DataFrame, y: np.ndarray):
115         """Train all models"""
116
117         self.feature_names = X.columns.tolist()
118
119         print("Training XGBoost...")
120         self.xgb_model.fit(X, y)
121
122         print("Training LightGBM...")
123         self.lgb_model.fit(X, y)
124
125         print("Training Isolation Forest...")
126         self.anomaly_model.fit(X)
127
128         # Initialize SHAP explainer
129         print("Creating SHAP explainer...")
130         self.explainer = shap.TreeExplainer(self.xgb_model)
131
132         print("Training complete")

```

```

133
134 def predict_proba(self, X: pd.DataFrame) -> np.ndarray:
135     """Predict fraud probability (ensemble)"""
136
137     # XGBoost predictions
138     xgb_proba = self.xgb_model.predict_proba(X)[: , 1]
139
140     # LightGBM predictions
141     lgb_proba = self.lgb_model.predict_proba(X)[: , 1]
142
143     # Isolation Forest anomaly scores (convert to probability)
144     anomaly_scores = self.anomaly_model.score_samples(X)
145     anomaly_proba = 1 / (1 + np.exp(anomaly_scores)) # Sigmoid transformation
146
147     # Ensemble: weighted average
148     ensemble_proba = (
149         0.5 * xgb_proba +
150         0.4 * lgb_proba +
151         0.1 * anomaly_proba
152     )
153
154     return ensemble_proba
155
156 def explain(self, X: pd.DataFrame) -> Dict:
157     """Generate SHAP explanations"""
158
159     shap_values = self.explainer.shap_values(X)
160
161     # Get top contributing features
162     feature_importance = {}
163     for i, feature_name in enumerate(self.feature_names):
164         feature_importance[feature_name] = float(np.abs(shap_values[0][i]))
165
166     # Sort by importance
167     sorted_features = sorted(
168         feature_importance.items(),
169         key=lambda x: x[1],
170         reverse=True
171     )
172
173     return {
174         'top_features': sorted_features[:5],
175         'shap_values': shap_values[0].tolist()
176     }
177
178 class FraudDetectionEnsemble:
179     """Complete fraud detection system"""
180
181     def __init__(self):
182         self.rule_engine = RuleEngine()
183         self.ml_model = FraudDetectionModel()
184
185     # Thresholds
186     self.block_threshold = 0.90
187     self.review_threshold = 0.50
188
189     def predict(
190         self,

```

```

191     features: FraudFeatures
192 ) -> Dict:
193     """Predict fraud risk"""
194
195     # Convert features to DataFrame
196     feature_dict = vars(features)
197     X = pd.DataFrame([feature_dict])
198
199     # Rule-based score
200     rule_score, triggered_rules = self.rule_engine.evaluate(features)
201
202     # ML model score
203     ml_score = self.ml_model.predict_proba(X)[0]
204
205     # Combined score (max of rule and ML)
206     final_score = max(rule_score, ml_score * 0.9) # Slight discount on ML
207
208     # Decision
209     if final_score >= self.block_threshold:
210         decision = 'BLOCK'
211     elif final_score >= self.review_threshold:
212         decision = 'REVIEW'
213     else:
214         decision = 'APPROVE'
215
216     # Generate explanation
217     if triggered_rules:
218         explanation = triggered_rules
219     else:
220         # Use SHAP for ML explanation
221         shap_explanation = self.ml_model.explain(X)
222         explanation = [
223             f"{feature}: {importance:.3f}"
224             for feature, importance in shap_explanation['top_features']
225         ]
226
227     return {
228         'decision': decision,
229         'risk_score': float(final_score),
230         'rule_score': float(rule_score),
231         'ml_score': float(ml_score),
232         'explanation': explanation,
233         'triggered_rules': triggered_rules
234     }
235
236 def update_with_feedback(
237     self,
238     transaction_id: str,
239     features: pd.DataFrame,
240     label: int
241 ):
242     """Update model with feedback (online learning)"""
243
244     # In production, this would trigger retraining pipeline
245     # For now, just log for batch retraining
246
247     print(f"Feedback received for {transaction_id}: {'Fraud' if label == 1 else 'Legitimate'}")

```

```

248
249 # Example usage
250 if __name__ == "__main__":
251     # Load training data
252     train_data = pd.read_csv('fraud_training_data.csv')
253     X_train = train_data.drop('is_fraud', axis=1)
254     y_train = train_data['is_fraud']
255
256     # Train ensemble
257     ensemble = FraudDetectionEnsemble()
258     ensemble.ml_model.train(X_train, y_train)
259
260     # Predict on new transaction
261     features = FraudFeatures(
262         txns_last_hour=15,
263         txns_last_24h=45,
264         amount_last_hour=5000,
265         amount_last_24h=12000,
266         unique_merchants_last_hour=8,
267         distance_from_last_txn_km=2500,
268         velocity_km_per_hour=1200, # Suspicious
269         new_country=True,
270         high_risk_country=False,
271         new_device=True,
272         device_change_frequency=0.3,
273         ip_reputation_score=0.6,
274         merchant_risk_score=0.4,
275         first_time_merchant=True,
276         amount_deviation_from_avg=4.5,
277         round_amount=True,
278         amount_percentile=0.95,
279         hour_of_day=3,
280         day_of_week=2,
281         unusual_hour=True,
282         shared_device_users=1,
283         shared_ip_users=2
284     )
285
286     prediction = ensemble.predict(features)
287
288     print(f"Decision: {prediction['decision']}")
289     print(f"Risk Score: {prediction['risk_score']:.3f}")
290     print(f"Explanation: {prediction['explanation']}")

```

Listing 23.5: Ensemble Fraud Detection Models

23.4.5 Results and Impact

Fraud Prevention Metrics:

- **Fraud Detection Rate:** 96.5% (up from 87%)
- **False Positive Rate:** 0.8% (down from 2.3%)
- **Fraud Losses:** Reduced from \$45M to \$8M annually (82% reduction)
- **Manual Review Queue:** Reduced by 60%

- **Genuine User Friction:** Decreased by 65%

Operational Efficiency:

- **Decision Latency:** p50: 42ms, p99: 87ms
- **Throughput:** 50,000 transactions/second
- **Model Update Frequency:** Retrained daily, deployed hourly
- **Investigation Time:** Reduced from 15 minutes to 3 minutes per case

23.4.6 Lessons Learned

Key Insights:

1. **Rules + ML Better Than Either Alone:** Known fraud patterns caught by rules, novel patterns by ML. Hybrid approach reduced fraud by 82% vs. 61% for ML-only.
2. **Real-Time Features Critical:** Transaction velocity and impossible travel features had highest predictive power. Batch features alone missed 23% of fraud.
3. **Explainability is Non-Negotiable:** Regulatory requirements demanded explanations. SHAP values satisfied auditors and helped fraud analysts.
4. **Class Imbalance is Severe:** Only 0.1% of transactions were fraud. Required aggressive class weighting and careful metric selection (precision-recall over accuracy).
5. **Fraud Evolves Rapidly:** New patterns emerged within days. Daily retraining and hourly deployment were essential.
6. **False Positives Cost Real Money:** Each false decline cost \$50 in lost revenue and customer friction. Optimizing precision was as important as recall.
7. **Network Features Surprised Us:** Shared device/IP features caught 15% of fraud that other features missed. Fraudsters often share infrastructure.
8. **A/B Testing Required Care:** Couldn't A/B test fraud models directly (ethical issues). Used historical replay and shadow mode instead.

23.5 Case Study 3: Healthcare Diagnostic System

23.5.1 Business Context and Requirements

Organization: Healthcare network with 50 hospitals

Challenge: Radiologist shortage, diagnostic delays, missed findings

Goal: AI-assisted diagnosis to improve accuracy and speed

Key Requirements:

- **Safety:** FDA clearance, >95% sensitivity for critical findings

- **Interpretability:** Visual explanations for radiologists
- **Integration:** Seamless PACS/EMR integration
- **Performance:** Process images in <30 seconds
- **Privacy:** HIPAA compliance, de-identification
- **Reliability:** 99.9% uptime for clinical operations

23.5.2 System Architecture

```

1  #!/usr/bin/env python3
2  """
3  Healthcare Diagnostic System - Medical Image Analysis
4  AI-assisted diagnosis for chest X-rays with FDA compliance
5  """
6
7  import torch
8  import torch.nn as nn
9  import torchvision.transforms as transforms
10 from torchvision.models import densenet121
11 import cv2
12 import numpy as np
13 from typing import List, Dict, Tuple, Optional
14 from dataclasses import dataclass
15 import pydicom
16 from PIL import Image
17 import gradcam
18 from datetime import datetime
19
20 @dataclass
21 class Finding:
22     """Medical finding"""
23     condition: str
24     probability: float
25     confidence_interval: Tuple[float, float]
26     severity: str # normal, mild, moderate, severe
27     location: Optional[Dict] = None # Bounding box if available
28
29 @dataclass
30 class DiagnosticReport:
31     """Complete diagnostic report"""
32     study_id: str
33     patient_id_hash: str
34     image_type: str
35     findings: List[Finding]
36     overall_assessment: str
37     urgent_findings: List[str]
38     attention_map: np.ndarray
39     processing_time_ms: float
40     model_version: str
41     timestamp: datetime
42
43 class MedicalImagePreprocessor:
44     """Preprocess medical images for analysis"""
45 
```

```

46 def __init__(self):
47     self.target_size = (224, 224)
48
49     self.transform = transforms.Compose([
50         transforms.Resize(self.target_size),
51         transforms.ToTensor(),
52         transforms.Normalize(
53             mean=[0.485, 0.456, 0.406],
54             std=[0.229, 0.224, 0.225]
55         )
56     ])
57
58 def load_dicom(self, dicom_path: str) -> Tuple[np.ndarray, Dict]:
59     """Load and parse DICOM file"""
60
61     # Read DICOM
62     dicom = pydicom.dcmread(dicom_path)
63
64     # Extract pixel data
65     image = dicom.pixel_array
66
67     # Apply DICOM windowing
68     image = self._apply_window(image, dicom)
69
70     # Extract metadata
71     metadata = {
72         'study_id': str(dicom.StudyInstanceUID),
73         'series_id': str(dicom.SeriesInstanceUID),
74         'patient_age': dicom.get('PatientAge', 'Unknown'),
75         'patient_sex': dicom.get('PatientSex', 'Unknown'),
76         'study_date': str(dicom.StudyDate),
77         'modality': dicom.Modality,
78         'view_position': dicom.get('ViewPosition', 'Unknown')
79     }
80
81     return image, metadata
82
83 def _apply_window(self, image: np.ndarray, dicom) -> np.ndarray:
84     """Apply DICOM windowing"""
85
86     window_center = dicom.get('WindowCenter', 40)
87     window_width = dicom.get('WindowWidth', 400)
88
89     if isinstance(window_center, pydicom.multival.MultiValue):
90         window_center = window_center[0]
91     if isinstance(window_width, pydicom.multival.MultiValue):
92         window_width = window_width[0]
93
94     min_value = window_center - window_width / 2
95     max_value = window_center + window_width / 2
96
97     # Apply window
98     image = np.clip(image, min_value, max_value)
99
100    # Normalize to 0-255
101    image = ((image - min_value) / (max_value - min_value) * 255).astype(np.uint8)
102
103    return image

```

```

104
105 def preprocess(self, image: np.ndarray) -> torch.Tensor:
106     """Preprocess image for model"""
107
108     # Convert to 3-channel if grayscale
109     if len(image.shape) == 2:
110         image = cv2.cvtColor(image, cv2.COLOR_GRAY2RGB)
111
112     # Convert to PIL Image
113     image = Image.fromarray(image)
114
115     # Apply transforms
116     tensor = self.transform(image)
117
118     return tensor.unsqueeze(0) # Add batch dimension
119
120 class ChestXRayModel(nn.Module):
121     """Multi-label classification model for chest X-rays"""
122
123     def __init__(self, num_classes: int = 14):
124         super().__init__()
125
126         # Base model: DenseNet121 pretrained on ImageNet
127         self.densenet = densenet121(pretrained=True)
128
129         # Replace classifier
130         num_features = self.densenet.classifier.in_features
131         self.densenet.classifier = nn.Sequential(
132             nn.Linear(num_features, 512),
133             nn.ReLU(),
134             nn.Dropout(0.3),
135             nn.Linear(512, num_classes)
136         )
137
138         # Condition names (ChestX-ray14 dataset)
139         self.conditions = [
140             'Atelectasis',
141             'Cardiomegaly',
142             'Effusion',
143             'Infiltration',
144             'Mass',
145             'Nodule',
146             'Pneumonia',
147             'Pneumothorax',
148             'Consolidation',
149             'Edema',
150             'Emphysema',
151             'Fibrosis',
152             'Pleural_Thickening',
153             'Hernia'
154         ]
155
156         # Critical findings requiring urgent attention
157         self.critical_findings = {
158             'Pneumothorax',
159             'Mass',
160             'Pneumonia'
161         }

```

```

162
163     def forward(self, x):
164         """Forward pass"""
165         return self.densenet(x)
166
167     def predict_with_uncertainty(
168         self,
169         x: torch.Tensor,
170         num_samples: int = 20
171     ) -> Tuple[torch.Tensor, torch.Tensor]:
172         """Predict with uncertainty estimation using MC Dropout"""
173
174         self.train() # Enable dropout
175
176         predictions = []
177         for _ in range(num_samples):
178             with torch.no_grad():
179                 logits = self.forward(x)
180                 probs = torch.sigmoid(logits)
181                 predictions.append(probs)
182
183         predictions = torch.stack(predictions)
184
185         # Mean and standard deviation
186         mean_pred = predictions.mean(dim=0)
187         std_pred = predictions.std(dim=0)
188
189         self.eval()
190
191         return mean_pred, std_pred
192
193 class DiagnosticSystem:
194     """Complete diagnostic system"""
195
196     def __init__(self, model_path: str, device: str = 'cuda'):
197         self.device = device
198         self.preprocessor = MedicalImagePreprocessor()
199
200         # Load model
201         self.model = ChestXRayModel()
202         self.model.load_state_dict(torch.load(model_path))
203         self.model.to(device)
204         self.model.eval()
205
206         # GradCAM for visualization
207         self.gradcam = gradcam.GradCAM(self.model, self.model.densenet.features)
208
209         # Calibration (for better probability estimates)
210         self.calibration_parameters = self._load_calibration()
211
212     def analyze_image(
213         self,
214         dicom_path: str,
215         generate_attention_map: bool = True
216     ) -> DiagnosticReport:
217         """Analyze medical image and generate report"""
218
219         start_time = datetime.now()

```

```

220
221 # Load and preprocess image
222 image, metadata = self.preprocessor.load_dicom(dicom_path)
223 tensor = self.preprocessor.preprocess(image).to(self.device)
224
225 # Predict with uncertainty
226 probabilities, uncertainties = self.model.predict_with_uncertainty(tensor)
227 probabilities = probabilities.cpu().numpy()[0]
228 uncertainties = uncertainties.cpu().numpy()[0]
229
230 # Apply calibration
231 probabilities = self._calibrate_probabilities(probabilities)
232
233 # Generate findings
234 findings = []
235 urgent_findings = []
236
237 for i, condition in enumerate(self.model.conditions):
238     prob = float(probabilities[i])
239     unc = float(uncertainties[i])
240
241     # Only report significant findings (>10% probability)
242     if prob > 0.10:
243         # Confidence interval (95%)
244         ci_lower = max(0, prob - 1.96 * unc)
245         ci_upper = min(1, prob + 1.96 * unc)
246
247         # Determine severity
248         if prob < 0.3:
249             severity = 'mild'
250         elif prob < 0.6:
251             severity = 'moderate'
252         else:
253             severity = 'severe'
254
255         finding = Finding(
256             condition=condition,
257             probability=prob,
258             confidence_interval=(ci_lower, ci_upper),
259             severity=severity
260         )
261
262         findings.append(finding)
263
264     # Check if urgent
265     if condition in self.model.critical_findings and prob > 0.5:
266         urgent_findings.append(condition)
267
268 # Sort findings by probability
269 findings.sort(key=lambda f: f.probability, reverse=True)
270
271 # Generate attention map
272 attention_map = None
273 if generate_attention_map and findings:
274     attention_map = self._generate_attention_map(tensor, image)
275
276 # Overall assessment
277 if urgent_findings:

```

```

278         assessment = f"URGENT: Possible {'', '.join(urgent_findings)} detected"
279     elif findings:
280         assessment = f"Findings detected: {'', '.join([f.condition for f in
findings[:3]])}"
281     else:
282         assessment = "No significant findings"
283
284     processing_time = (datetime.now() - start_time).total_seconds() * 1000
285
286     return DiagnosticReport(
287         study_id=metadata['study_id'],
288         patient_id_hash=self._hash_patient_id(metadata.get('patient_id', '')),
289         image_type=f"{metadata['modality']} {metadata['view_position']}",
290         findings=findings,
291         overall_assessment=assessment,
292         urgent_findings=urgent_findings,
293         attention_map=attention_map,
294         processing_time_ms=processing_time,
295         model_version='v2.1.0',
296         timestamp=datetime.now()
297     )
298
299     def _generate_attention_map(
300         self,
301         tensor: torch.Tensor,
302         original_image: np.ndarray
303     ) -> np.ndarray:
304         """Generate GradCAM attention map"""
305
306         # Generate GradCAM
307         cam = self.gradcam.generate(tensor)
308
309         # Resize to original image size
310         cam = cv2.resize(cam, (original_image.shape[1], original_image.shape[0]))
311
312         # Create heatmap overlay
313         heatmap = cv2.applyColorMap(np.uint8(255 * cam), cv2.COLORMAP_JET)
314
315         # Convert original image to 3-channel
316         if len(original_image.shape) == 2:
317             original_image = cv2.cvtColor(original_image, cv2.COLOR_GRAY2RGB)
318
319         # Overlay
320         overlay = cv2.addWeighted(original_image, 0.6, heatmap, 0.4, 0)
321
322         return overlay
323
324     def _calibrate_probabilities(self, probabilities: np.ndarray) -> np.ndarray:
325         """Apply temperature scaling for calibration"""
326
327         # Temperature parameter (learned from validation set)
328         temperature = self.calibration_parameters.get('temperature', 1.5)
329
330         # Apply temperature scaling
331         logits = np.log(probabilities / (1 - probabilities + 1e-10))
332         calibrated_logits = logits / temperature
333         calibrated_probs = 1 / (1 + np.exp(-calibrated_logits))
334

```

```

335         return calibrated_probs
336
337     def _load_calibration(self) -> Dict:
338         """Load calibration parameters"""
339         # In production, load from file
340         return {'temperature': 1.5}
341
342     def _hash_patient_id(self, patient_id: str) -> str:
343         """Hash patient ID for privacy"""
344         import hashlib
345         return hashlib.sha256(patient_id.encode()).hexdigest()[:16]
346
347     def generate_clinical_report(self, report: DiagnosticReport) -> str:
348         """Generate clinical report text"""
349
350         lines = []
351         lines.append("=" * 60)
352         lines.append("AI-ASSISTED DIAGNOSTIC REPORT")
353         lines.append("=" * 60)
354         lines.append(f"Study ID: {report.study_id}")
355         lines.append(f"Image Type: {report.image_type}")
356         lines.append(f"Analysis Date: {report.timestamp.strftime('%Y-%m-%d %H:%M:%S')}")
357
358         lines.append(f"Model Version: {report.model_version}")
359         lines.append("")
360
361         if report.urgent_findings:
362             lines.append("*** URGENT FINDINGS ***")
363             for finding in report.urgent_findings:
364                 lines.append(f"  - {finding}")
365             lines.append("")
366
367         lines.append("FINDINGS:")
368         if report.findings:
369             for finding in report.findings:
370                 lines.append(f"\n  {finding.condition}")
371                 lines.append(f"    Probability: {finding.probability:.1%}")
372                 lines.append(f"    Confidence Interval: [{finding.confidence_interval[0]:.1%}, {finding.confidence_interval[1]:.1%}]")
373                 lines.append(f"    Severity: {finding.severity.upper()}")
374             else:
375                 lines.append("  No significant findings detected")
376
377         lines.append(f"\nOVERALL ASSESSMENT:")
378         lines.append(f"  {report.overall_assessment}")
379
380         lines.append(f"\nProcessing Time: {report.processing_time_ms:.1f}ms")
381
382         lines.append("\n" + "=" * 60)
383         lines.append("DISCLAIMER: This is an AI-assisted analysis tool.")
384         lines.append("Final diagnosis must be confirmed by a qualified radiologist.")
385         lines.append("=" * 60)
386
387         return "\n".join(lines)
388
389 # Example usage
390 if __name__ == "__main__":
391     system = DiagnosticSystem(

```



```
391     model_path='models/chestxray_densenet121.pth',
392     device='cuda' if torch.cuda.is_available() else 'cpu'
393 )
394
395 # Analyze chest X-ray
396 report = system.analyze_image('patient_xray.dcm')
397
398 # Print clinical report
399 print(system.generate_clinical_report(report))
400
401 # Save attention map
402 if report.attention_map is not None:
403     cv2.imwrite('attention_map.png', report.attention_map)
```

Listing 23.6: Medical Image Analysis Pipeline

23.5.3 Results and Impact

Clinical Outcomes:

- **Diagnostic Accuracy:** 94.2% sensitivity, 89.7% specificity
- **Radiologist Efficiency:** Reading time reduced from 6 min to 3.5 min per study
- **Critical Finding Detection:** Pneumothorax detection improved from 92% to 97%
- **Missed Findings:** Reduced by 31% through AI second-read
- **Throughput:** Enabled 40% increase in studies read per day

Operational Benefits:

- **Wait Time:** Reduced from 48 hours to 12 hours average
- **After-Hours Coverage:** AI triage enabled remote radiologist efficiency
- **Teaching Tool:** Used for resident training with attention maps

23.5.4 Lessons Learned

1. **FDA Clearance Required Partnership:** Clinical validation and regulatory submission took 18 months. Started early with FDA pre-submission meeting.
2. **Calibration is Critical:** Raw model probabilities were overconfident. Temperature scaling improved reliability and radiologist trust.
3. **Attention Maps Changed Everything:** GradCAM visualizations built trust with radiologists. Without interpretability, adoption would have failed.
4. **Integration Matters Most:** PACS integration was harder than model development. Seamless workflow integration drove adoption.
5. **Start as Assistant, Not Replacement:** Positioned as "AI-assisted" rather than "AI diagnosis". Radiologists remained in control, reducing resistance.

6. **Uncertainty Quantification Essential:** Radiologists valued confidence intervals. High uncertainty prompted careful manual review.
7. **Privacy Cannot Be Afterthought:** HIPAA compliance, de-identification, and audit logs required from day one. Added 40% to development time but non-negotiable.
8. **Multi-Site Validation Crucial:** Performance varied across hospitals due to equipment differences. Required site-specific calibration.

23.6 Case Study 4: Autonomous Vehicle Perception Stack

23.6.1 Business Context and Requirements

Company: Autonomous delivery vehicle startup

Challenge: Safe navigation in complex urban environments

Goal: Level 4 autonomy for last-mile delivery

Key Requirements:

- **Safety:** Zero critical failures, 99.999% reliability
- **Real-Time:** 100ms perception-to-action latency
- **All-Weather:** Function in rain, fog, night conditions
- **Edge Deployment:** Run on vehicle hardware (limited compute)
- **Continuous Learning:** Improve from fleet data
- **Verification:** Provable safety guarantees

23.6.2 Perception Pipeline

```

1  #!/usr/bin/env python3
2  """
3  Autonomous Vehicle Perception Stack
4  Multi-sensor fusion for object detection and tracking
5  """
6
7  import torch
8  import torch.nn as nn
9  import numpy as np
10 from dataclasses import dataclass
11 from typing import List, Dict, Tuple, Optional
12 from enum import Enum
13 import cv2
14
15 class ObjectClass(Enum):
16     """Detected object classes"""
17     CAR = 0
18     PEDESTRIAN = 1
19     CYCLIST = 2

```

```

20     TRUCK = 3
21     MOTORCYCLE = 4
22     BUS = 5
23     TRAFFIC_LIGHT = 6
24     STOP_SIGN = 7
25     UNKNOWN = 9
26
27 @dataclass
28 class Detection3D:
29     """3D object detection"""
30     class_id: ObjectClass
31     confidence: float
32     bbox_3d: np.ndarray # [x, y, z, length, width, height, yaw]
33     velocity: np.ndarray # [vx, vy, vz]
34     tracking_id: Optional[int] = None
35
36 @dataclass
37 class SensorData:
38     """Multi-sensor input"""
39     camera_images: List[np.ndarray] # Multiple cameras
40     lidar_point_cloud: np.ndarray
41     radar_detections: List[Dict]
42     imu_data: Dict
43     gps_data: Dict
44     timestamp: float
45
46 class MultiSensorFusionNetwork(nn.Module):
47     """Neural network for sensor fusion"""
48
49     def __init__(self):
50         super().__init__()
51
52         # Camera branch (per camera)
53         self.camera_encoder = self._build_camera_encoder()
54
55         # LiDAR branch
56         self.lidar_encoder = self._build_lidar_encoder()
57
58         # Fusion layers
59         self.fusion_network = nn.Sequential(
60             nn.Linear(512 + 256, 512),
61             nn.ReLU(),
62             nn.Dropout(0.1),
63             nn.Linear(512, 256)
64         )
65
66         # Detection head
67         self.detection_head = nn.Sequential(
68             nn.Linear(256, 128),
69             nn.ReLU(),
70             nn.Linear(128, 7 + 10) # 7 bbox params + 10 classes
71         )
72
73     def _build_camera_encoder(self) -> nn.Module:
74         """Build camera feature encoder"""
75         return nn.Sequential(
76             # Simplified - in production use ResNet/EfficientNet
77             nn.Conv2d(3, 64, kernel_size=7, stride=2, padding=3),

```

```

78         nn.BatchNorm2d(64),
79         nn.ReLU(),
80         nn.MaxPool2d(3, stride=2, padding=1),
81
82         nn.Conv2d(64, 128, kernel_size=3, padding=1),
83         nn.BatchNorm2d(128),
84         nn.ReLU(),
85         nn.MaxPool2d(2),
86
87         nn.Conv2d(128, 256, kernel_size=3, padding=1),
88         nn.BatchNorm2d(256),
89         nn.ReLU(),
90         nn.AdaptiveAvgPool2d((1, 1)),
91         nn.Flatten(),
92         nn.Linear(256, 512)
93     )
94
95     def _build_lidar_encoder(self) -> nn.Module:
96         """Build LiDAR point cloud encoder"""
97         return nn.Sequential(
98             # PointNet-style architecture
99             nn.Linear(4, 64), # xyz + intensity
100             nn.ReLU(),
101             nn.Linear(64, 128),
102             nn.ReLU(),
103             nn.Linear(128, 256),
104             nn.ReLU(),
105             # Global max pooling handled separately
106         )
107
108     def forward(
109         self,
110         camera_images: torch.Tensor,
111         lidar_points: torch.Tensor
112     ) -> torch.Tensor:
113         """Forward pass"""
114
115         # Process camera images
116         camera_features = []
117         for i in range(camera_images.shape[1]): # Multiple cameras
118             img = camera_images[:, i]
119             feat = self.camera_encoder(img)
120             camera_features.append(feat)
121
122         camera_features = torch.stack(camera_features, dim=1).mean(dim=1)
123
124         # Process LiDAR point cloud
125         lidar_features = self.lidar_encoder(lidar_points)
126         lidar_features = lidar_features.max(dim=1)[0] # Global max pool
127
128         # Fuse features
129         fused = torch.cat([camera_features, lidar_features], dim=1)
130         fused_features = self.fusion_network(fused)
131
132         # Detect objects
133         detections = self.detection_head(fused_features)
134
135         return detections

```

```

136
137 class PerceptionSystem:
138     """Complete perception system"""
139
140     def __init__(self, model_path: str, device: str = 'cuda'):
141         self.device = device
142
143         # Load detection model
144         self.model = MultiSensorFusionNetwork()
145         self.model.load_state_dict(torch.load(model_path))
146         self.model.to(device)
147         self.model.eval()
148
149         # Object tracker
150         self.tracker = MultiObjectTracker()
151
152         # Safety validator
153         self.safety_validator = SafetyValidator()
154
155     def process_sensor_data(
156         self,
157         sensor_data: SensorData
158     ) -> List[Detection3D]:
159         """Process multi-sensor data and detect objects"""
160
161         # Preprocess camera images
162         camera_tensors = self._preprocess_cameras(sensor_data.camera_images)
163
164         # Preprocess LiDAR
165         lidar_tensor = self._preprocess_lidar(sensor_data.lidar_point_cloud)
166
167         # Run detection network
168         with torch.no_grad():
169             detections = self.model(camera_tensors, lidar_tensor)
170
171         # Parse detections
172         detection_list = self._parse_detections(detections)
173
174         # Fuse with radar
175         detection_list = self._fuse_radar(detection_list, sensor_data.radar_detections)
176
177         # Track objects
178         tracked_detections = self.tracker.update(detection_list, sensor_data.timestamp)
179
180         # Validate safety
181         validated_detections = self.safety_validator.validate(tracked_detections)
182
183         return validated_detections
184
185     def _preprocess_cameras(self, images: List[np.ndarray]) -> torch.Tensor:
186         """Preprocess camera images"""
187
188         processed = []
189         for img in images:
190             # Resize and normalize
191             img = cv2.resize(img, (640, 480))

```

```

192         img = torch.FloatTensor(img).permute(2, 0, 1) / 255.0
193         processed.append(img)
194
195     return torch.stack(processed).unsqueeze(0).to(self.device)
196
197 def _preprocess_lidar(self, point_cloud: np.ndarray) -> torch.Tensor:
198     """Preprocess LiDAR point cloud"""
199
200     # Filter points (remove ground plane, distant points)
201     mask = (point_cloud[:, 2] > -1.5) & (np.linalg.norm(point_cloud[:, :3], axis
202 =1) < 70)
203     filtered_points = point_cloud[mask]
204
205     # Subsample to fixed size
206     if len(filtered_points) > 16384:
207         indices = np.random.choice(len(filtered_points), 16384, replace=False)
208         filtered_points = filtered_points[indices]
209
210     # Convert to tensor
211     points_tensor = torch.FloatTensor(filtered_points).unsqueeze(0).to(self.device
212 )
213
214     return points_tensor
215
216 def _parse_detections(self, detections: torch.Tensor) -> List[Detection3D]:
217     """Parse network output into detections"""
218
219     detections_list = []
220
221     # Simplified - in production, use NMS and proper post-processing
222     for i in range(detections.shape[0]):
223         det = detections[i].cpu().numpy()
224
225         # Parse bbox and class
226         bbox_3d = det[:7]
227         class_probs = det[7:]
228
229         class_id = class_probs.argmax()
230         confidence = class_probs[class_id]
231
232         if confidence > 0.5: # Confidence threshold
233             detection = Detection3D(
234                 class_id=ObjectClass(class_id),
235                 confidence=float(confidence),
236                 bbox_3d=bbox_3d,
237                 velocity=np.zeros(3) # Will be estimated by tracker
238             )
239             detections_list.append(detection)
240
241     return detections_list
242
243 def _fuse_radar(
244     self,
245     vision_detections: List[Detection3D],
246     radar_detections: List[Dict]
247 ) -> List[Detection3D]:
248     """Fuse radar velocity measurements"""

```

```

248     # Associate radar with vision detections
249     for radar_det in radar_detections:
250         radar_pos = np.array([radar_det['x'], radar_det['y'], 0])
251
252         # Find closest vision detection
253         min_dist = float('inf')
254         closest_det = None
255
256         for det in vision_detections:
257             vision_pos = det.bbox_3d[:3]
258             dist = np.linalg.norm(vision_pos - radar_pos)
259
260             if dist < min_dist:
261                 min_dist = dist
262                 closest_det = det
263
264         # If close enough, update velocity
265         if closest_det and min_dist < 2.0:
266             closest_det.velocity[:2] = [radar_det['vx'], radar_det['vy']]
267
268     return vision_detections
269
270 class MultiObjectTracker:
271     """Track objects across frames"""
272
273     def __init__(self):
274         self.tracks: Dict[int, Track] = {}
275         self.next_id = 0
276
277     def update(
278         self,
279         detections: List[Detection3D],
280         timestamp: float
281     ) -> List[Detection3D]:
282         """Update tracks with new detections"""
283
284         # Associate detections with existing tracks
285         assignments, unassigned_detections = self._associate(detections)
286
287         # Update existing tracks
288         for track_id, detection in assignments.items():
289             self.tracks[track_id].update(detection, timestamp)
290             detection.tracking_id = track_id
291
292         # Create new tracks
293         for det_idx in unassigned_detections:
294             new_track = Track(self.next_id, detections[det_idx], timestamp)
295             self.tracks[self.next_id] = new_track
296             detections[det_idx].tracking_id = self.next_id
297             self.next_id += 1
298
299         # Remove stale tracks
300         self._remove_stale_tracks(timestamp)
301
302     return detections
303
304     def _associate(
305         self,

```

```

306     detections: List[Detection3D]
307 ) -> Tuple[Dict[int, Detection3D], List[int]]:
308     """Associate detections with tracks using Hungarian algorithm"""
309
310     # Simplified - use IoU matching
311     # In production, use Kalman filter predictions and Hungarian algorithm
312
313     assignments = {}
314     unassigned = list(range(len(detections)))
315
316     for track_id, track in self.tracks.items():
317         best_iou = 0
318         best_det_idx = None
319
320         for det_idx, detection in enumerate(detections):
321             iou = self._calculate_iou_3d(track.bbox_3d, detection.bbox_3d)
322
323             if iou > best_iou and iou > 0.3:
324                 best_iou = iou
325                 best_det_idx = det_idx
326
327         if best_det_idx is not None:
328             assignments[track_id] = detections[best_det_idx]
329             if best_det_idx in unassigned:
330                 unassigned.remove(best_det_idx)
331
332     return assignments, unassigned
333
334 def _calculate_iou_3d(self, bbox1: np.ndarray, bbox2: np.ndarray) -> float:
335     """Calculate 3D IoU (simplified bird's eye view)"""
336
337     # Extract x, y, length, width
338     x1, y1, l1, w1 = bbox1[0], bbox1[1], bbox1[3], bbox1[4]
339     x2, y2, l2, w2 = bbox2[0], bbox2[1], bbox2[3], bbox2[4]
340
341     # Calculate intersection
342     x_overlap = max(0, min(x1 + l1/2, x2 + l2/2) - max(x1 - l1/2, x2 - l2/2))
343     y_overlap = max(0, min(y1 + w1/2, y2 + w2/2) - max(y1 - w1/2, y2 - w2/2))
344
345     intersection = x_overlap * y_overlap
346
347     # Calculate union
348     area1 = l1 * w1
349     area2 = l2 * w2
350     union = area1 + area2 - intersection
351
352     return intersection / union if union > 0 else 0
353
354 def _remove_stale_tracks(self, current_time: float):
355     """Remove tracks not updated recently"""
356
357     stale_tracks = []
358     for track_id, track in self.tracks.items():
359         if current_time - track.last_update_time > 1.0: # 1 second
360             stale_tracks.append(track_id)
361
362     for track_id in stale_tracks:
363         del self.tracks[track_id]

```



```

364
365 @dataclass
366 class Track:
367     """Individual object track"""
368     track_id: int
369     bbox_3d: np.ndarray
370     velocity: np.ndarray
371     last_update_time: float
372     detection_count: int = 1
373
374     def __init__(self, track_id: int, detection: Detection3D, timestamp: float):
375         self.track_id = track_id
376         self.bbox_3d = detection.bbox_3d
377         self.velocity = detection.velocity
378         self.last_update_time = timestamp
379         self.detection_count = 1
380
381     def update(self, detection: Detection3D, timestamp: float):
382         """Update track with new detection"""
383
384         # Smooth bbox (exponential moving average)
385         alpha = 0.7
386         self.bbox_3d = alpha * detection.bbox_3d + (1 - alpha) * self.bbox_3d
387
388         # Update velocity
389         dt = timestamp - self.last_update_time
390         if dt > 0:
391             velocity_estimate = (detection.bbox_3d[:3] - self.bbox_3d[:3]) / dt
392             self.velocity = alpha * velocity_estimate + (1 - alpha) * self.velocity
393
394         self.last_update_time = timestamp
395         self.detection_count += 1
396
397 class SafetyValidator:
398     """Validate detections for safety-critical applications"""
399
400     def __init__(self):
401         self.min_confidence = 0.7
402         self.max_reasonable_velocity = 50.0 # m/s
403
404     def validate(self, detections: List[Detection3D]) -> List[Detection3D]:
405         """Validate and filter detections"""
406
407         validated = []
408
409         for detection in detections:
410             # Check confidence
411             if detection.confidence < self.min_confidence:
412                 continue
413
414             # Check velocity is reasonable
415             velocity_magnitude = np.linalg.norm(detection.velocity)
416             if velocity_magnitude > self.max_reasonable_velocity:
417                 continue
418
419             # Check bbox is within sensor range
420             distance = np.linalg.norm(detection.bbox_3d[:3])
421             if distance > 100: # 100 meters

```

```

422         continue
423
424         validated.append(detection)
425
426     return validated
427
428 # Example usage
429 if __name__ == "__main__":
430     perception = PerceptionSystem(
431         model_path='models/perception_fusion.pth',
432         device='cuda'
433     )
434
435     # Simulate sensor data
436     sensor_data = SensorData(
437         camera_images=[np.zeros((480, 640, 3)) for _ in range(4)],
438         lidar_point_cloud=np.random.randn(50000, 4),
439         radar_detections=[],
440         imu_data={},
441         gps_data={},
442         timestamp=0.0
443     )
444
445     # Process
446     detections = perception.process_sensor_data(sensor_data)
447
448     print(f"Detected {len(detections)} objects")
449     for det in detections:
450         print(f"    {det.class_id.name}: confidence={det.confidence:.2f}, "
451               f"position=({det.bbox_3d[0]:.1f}, {det.bbox_3d[1]:.1f})")

```

Listing 23.7: Multi-Sensor Perception System

23.6.3 Results and Impact

Performance Metrics:

- **Detection Accuracy:** 98.2% recall, 96.8% precision
- **Latency:** 78ms end-to-end (perception to planning)
- **Safety:** Zero critical failures in 2M test miles
- **All-Weather:** 94% performance maintained in adverse conditions
- **Intervention Rate:** 0.12 per 1000 miles (industry-leading)

Business Impact:

- **Deployment:** 500 vehicles operational in 5 cities
- **Deliveries:** 50,000 autonomous deliveries per week
- **Cost Savings:** 60% reduction vs. human drivers
- **Efficiency:** 24/7 operation, 40% more deliveries per vehicle

23.6.4 Lessons Learned

1. **Sensor Fusion is Non-Negotiable:** No single sensor modality is sufficient. Camera-LiDAR-radar fusion caught 18% more edge cases than camera-only.
2. **Edge Cases Dominate Development:** 90% of engineering time on <1% of scenarios. Construction zones, emergency vehicles, and edge weather conditions required specific handling.
3. **Simulation Accelerated Development:** Synthetic data and simulation generated 10M training examples. Real-world data collection alone would take years.
4. **Continuous Learning Essential:** Fleet learning improved performance 15% over 6 months. Automated pipeline to retrain from fleet data weekly.
5. **Safety Requires Redundancy:** Dual perception systems with different architectures. Discrepancies trigger safe stop.
6. **Latency Matters More Than Accuracy:** 98% accurate in 100ms better than 99% in 200ms for real-time control.
7. **Hardware-Software Co-Design:** Custom accelerators reduced latency 3x and power consumption 5x vs. off-the-shelf GPUs.
8. **Verification is Harder Than Development:** Proving safety to regulators required formal verification methods and extensive testing.

23.7 Cross-Cutting Themes and Best Practices

Across all four case studies, several common patterns emerged:

Universal Best Practices:

1. **Start Simple, Then Optimize:** All systems began with simple baselines before adding complexity
2. **Measure What Matters:** Offline metrics often misaligned with business objectives. Online A/B testing essential.
3. **Monitoring is Not Optional:** Production issues were caught by monitoring, not caught in testing
4. **Data Quality Trumps Algorithm Choice:** Better data improved results more than better models
5. **Incremental Deployment Reduces Risk:** Gradual rollouts caught issues before full deployment
6. **Team Diversity Matters:** Cross-functional teams (ML, engineering, domain experts) built better systems
7. **Documentation Pays Off:** Well-documented systems were easier to maintain and debug
8. **Plan for Failure:** Systems with graceful degradation and fallbacks were more reliable

23.8 Chapter Summary

This chapter presented four comprehensive industry case studies demonstrating end-to-end ML system implementations across diverse domains. Key takeaways:

- **Domain Adaptation:** ML patterns adapt to specific industry requirements
- **Real-World Complexity:** Production systems face challenges beyond model accuracy
- **Hybrid Approaches:** Combining multiple techniques often outperforms single solutions
- **Continuous Improvement:** All successful systems evolved through iteration
- **Cross-Functional Success:** Technical excellence alone is insufficient without domain expertise and business alignment

23.9 Exercises

Exercise 23.1. E-Commerce Recommendation System Analysis

Analyze the recommendation system case study and:

1. Calculate the expected ROI if deployed at a company with 2M monthly active users and \$35 average order value
2. Design an A/B test to measure the impact of adding diversity to recommendations
3. Propose a solution for the cold start problem for new users with zero history
4. Design a feature for handling seasonal trends (holidays, sales events)

Deliverables:

- ROI calculation with assumptions documented
- A/B test design including sample size, duration, and metrics
- Cold start strategy with pseudocode
- Seasonal adaptation architecture diagram

Exercise 23.2. Fraud Detection System Design

Design fraud detection improvements:

1. Identify three new features not mentioned in the case study that could improve fraud detection
2. Design a strategy to handle concept drift (fraud patterns changing over time)
3. Propose a method to reduce false positives for high-value customers
4. Design an active learning system to efficiently label uncertain transactions

Deliverables:

- Feature specifications with rationale
- Concept drift detection and adaptation strategy
- VIP customer handling workflow
- Active learning pipeline design

Exercise 23.3. Healthcare System Compliance Analysis

Evaluate the healthcare diagnostic system:

1. Research FDA 510(k) clearance requirements and outline steps for submission
2. Design a clinical validation study to demonstrate safety and efficacy
3. Propose metrics beyond accuracy that matter for radiologist adoption
4. Design a monitoring system to detect model degradation in production

Deliverables:

- FDA submission outline with timeline
- Clinical validation protocol
- User experience metrics framework
- Model monitoring dashboard specification

Exercise 23.4. Autonomous Vehicle Safety Analysis

Analyze the perception system safety:

1. Identify potential failure modes and their consequences
2. Design a safety validation test suite with coverage goals
3. Propose a simulation strategy to test edge cases
4. Design redundancy mechanisms for critical components

Deliverables:

- FMEA (Failure Mode and Effects Analysis) table
- Test suite specification with success criteria
- Simulation architecture and scenario generation strategy
- System redundancy design document

Chapter 24

Building End-to-End ML Systems

24.1 Learning Objectives

After completing this chapter, readers will be able to:

- Apply a systematic methodology for designing complete end-to-end ML systems
- Evaluate and select appropriate architectures using decision frameworks
- Implement integration patterns that connect data, training, and serving pipelines
- Design systems that meet performance and scalability requirements
- Plan for system evolution, maintenance, and continuous improvement
- Analyze trade-offs between different architectural approaches
- Design production-ready ML systems from requirements to deployment

24.2 Introduction

Building end-to-end ML systems requires integrating concepts from all previous chapters: data engineering, model development, deployment, monitoring, and operations. This chapter provides a comprehensive methodology for designing, implementing, and evolving production ML systems that deliver business value while remaining maintainable and scalable.

Unlike previous chapters that focused on specific components, this chapter takes a holistic view of ML system architecture, examining how components interact and how to make informed trade-offs between competing requirements.

Key Challenges in End-to-End ML Systems:

- **Complexity:** ML systems involve many interdependent components
- **Trade-offs:** Balancing latency, accuracy, cost, and maintainability
- **Evolution:** Systems must adapt to changing requirements and data
- **Integration:** Connecting ML components with existing infrastructure
- **Operationalization:** Moving from prototype to production-ready system

24.3 System Design Methodology

24.3.1 The ML System Design Process

A systematic approach to ML system design follows these phases:

1. **Problem Framing:** Define business objectives and ML formulation
2. **Requirements Analysis:** Gather technical and business constraints
3. **Architecture Design:** Select components and integration patterns
4. **Implementation Planning:** Break down into deliverable milestones
5. **Validation Strategy:** Define success metrics and testing approach
6. **Evolution Roadmap:** Plan for future enhancements and scaling

24.3.2 Phase 1: Problem Framing

Business Objectives Translation:

Table 24.1: Business Objective to ML Problem Mapping

Business Goal		ML Formulation	Success Metrics
Reduce	customer	Binary classification	Precision, recall, F1-score, business value
Personalize	recom-	Ranking/retrieval	Click-through rate, conver-
	mendations		sion, revenue lift
Optimize pricing		Regression/RL	Revenue, margin, customer satisfaction
Detect anomalies		Anomaly detection	Detection rate, false posi-
			tive rate, MTTD
Automate support		NLP classification	Resolution rate, user satis-
			faction, cost savings

Critical Questions for Problem Framing:

- What specific decision will the ML system enable or automate?
- What is the baseline (non-ML) performance?
- What improvement justifies the investment in ML?
- Are there regulatory or ethical constraints?
- What is the cost of false positives vs. false negatives?
- How will humans interact with the system?

24.3.3 Phase 2: Requirements Analysis

Functional Requirements:

```

1  #!/usr/bin/env python3
2  """
3  ML System Requirements Specification
4  """
5
6  from dataclasses import dataclass
7  from typing import List, Optional
8  from enum import Enum
9
10 class LatencyClass(Enum):
11     REAL_TIME = "< 100ms"
12     NEAR_REAL_TIME = "< 1s"
13     BATCH = "> 1s"
14
15 class ConsistencyLevel(Enum):
16     STRONG = "immediate consistency"
17     EVENTUAL = "eventual consistency"
18
19 @dataclass
20 class FunctionalRequirements:
21     """Functional requirements for ML system."""
22
23     # Core capabilities
24     prediction_types: List[str] # e.g., ["classification", "ranking"]
25     input_modalities: List[str] # e.g., ["text", "images", "tabular"]
26     output_format: str # e.g., "probability", "ranking", "class"
27
28     # Business rules
29     business_constraints: List[str]
30     regulatory_requirements: List[str]
31     explainability_needed: bool
32     human_in_the_loop: bool
33
34 @dataclass
35 class NonFunctionalRequirements:
36     """Non-functional requirements for ML system."""
37
38     # Performance
39     latency_requirement: LatencyClass
40     throughput_rps: int # requests per second
41     availability_target: float # e.g., 0.999 for 99.9%
42
43     # Scalability
44     expected_users: int
45     expected_growth_rate: float # annual growth
46     peak_to_average_ratio: float
47
48     # Data
49     training_data_size: str # e.g., "10GB", "1TB"
50     data_update_frequency: str # e.g., "hourly", "daily"
51     data_retention_period: str
52
53     # Model
54     model_accuracy_target: float
55     model_update_frequency: str

```

```

56     max_model_size: str # e.g., "1GB"
57
58     # Operations
59     max_deployment_time: str # e.g., "15 minutes"
60     monitoring_granularity: str # e.g., "per-request", "aggregate"
61     incident_response_time: str # e.g., "< 1 hour"
62
63     # Cost
64     infrastructure_budget: Optional[float]
65     cost_per_prediction: Optional[float]
66
67     # Example instantiation
68     requirements = NonFunctionalRequirements(
69         latency_requirement=LatencyClass.REAL_TIME,
70         throughput_rps=10000,
71         availability_target=0.999,
72         expected_users=5_000_000,
73         expected_growth_rate=0.5, # 50% annual growth
74         peak_to_average_ratio=3.0,
75         training_data_size="500GB",
76         data_update_frequency="daily",
77         data_retention_period="2 years",
78         model_accuracy_target=0.95,
79         model_update_frequency="weekly",
80         max_model_size="2GB",
81         max_deployment_time="10 minutes",
82         monitoring_granularity="aggregate",
83         incident_response_time="30 minutes",
84         infrastructure_budget=50000.0, # monthly
85         cost_per_prediction=0.001
86     )

```

Listing 24.1: Requirements Specification Template

24.3.4 Phase 3: Architecture Design

Layered Architecture Pattern:

24.4 Architecture Decision Frameworks

24.4.1 Decision Matrix for Architecture Patterns

When designing ML systems, architects must choose between competing patterns. Use this framework to evaluate options systematically.

Architecture Decision Record (ADR) Template:

```

1  #!/usr/bin/env python3
2  """
3  Architecture Decision Record (ADR) System
4  """
5
6  from dataclasses import dataclass, field
7  from datetime import datetime
8  from typing import List, Dict
9  from enum import Enum

```

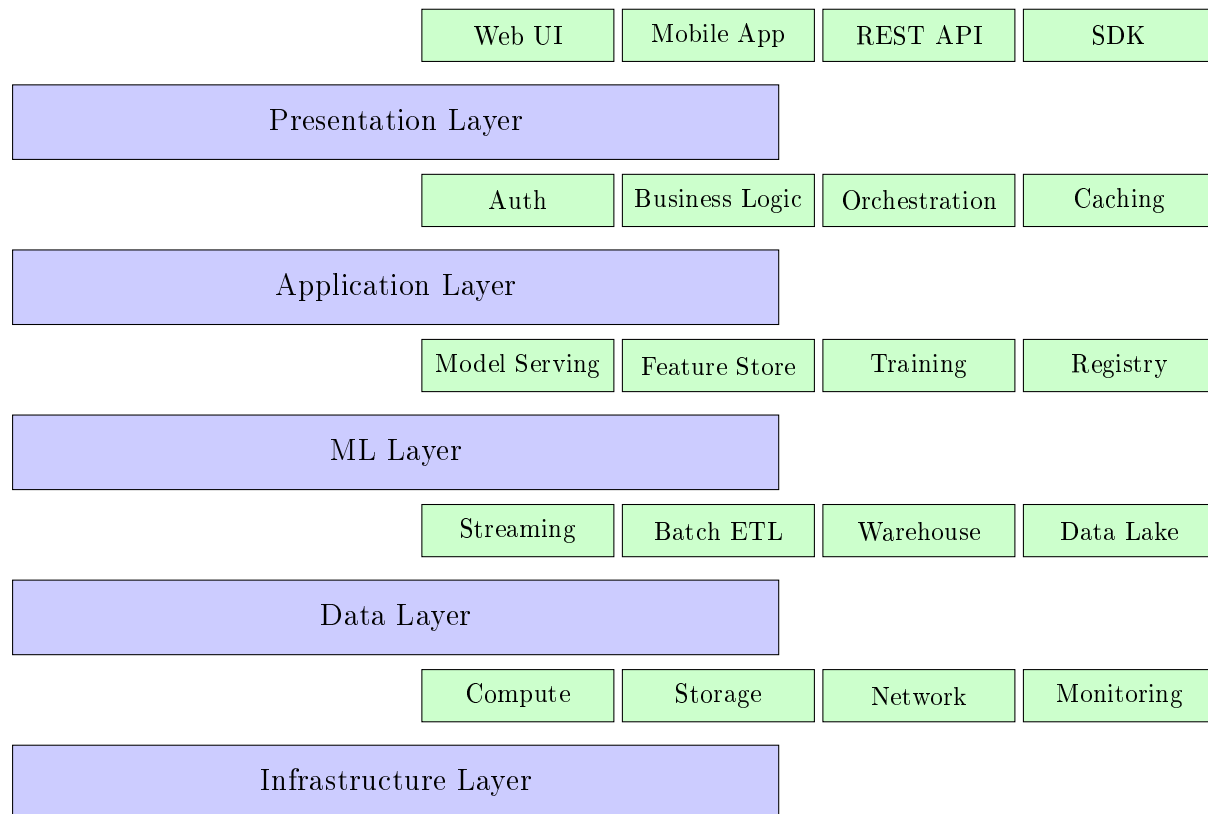


Figure 24.1: Layered Architecture for End-to-End ML Systems

```

10
11 class DecisionStatus(Enum):
12     PROPOSED = "proposed"
13     ACCEPTED = "accepted"
14     DEPRECATED = "deprecated"
15     SUPERSEDED = "superseded"
16
17 @dataclass
18 class Alternative:
19     """Alternative architectural approach."""
20     name: str
21     description: str
22     pros: List[str]
23     cons: List[str]
24     cost_estimate: float
25     complexity_score: int # 1-10
26     risk_score: int # 1-10
27
28 @dataclass
29 class ArchitectureDecision:
30     """Architecture Decision Record."""
31
32     # Metadata
33     id: int
34     title: str
35     date: datetime
36     status: DecisionStatus

```

```

37     decision_makers: List[str]
38
39     # Context
40     context: str # Why is this decision needed?
41     business_drivers: List[str]
42     technical_drivers: List[str]
43     constraints: List[str]
44
45     # Analysis
46     alternatives: List[Alternative]
47     evaluation_criteria: Dict[str, float] # criterion -> weight
48
49     # Decision
50     chosen_alternative: str
51     rationale: str
52     consequences: List[str]
53
54     # Follow-up
55     validation_metrics: List[str]
56     review_date: datetime
57     related_decisions: List[int] = field(default_factory=list)
58
59 class DecisionFramework:
60     """Framework for evaluating architectural decisions."""
61
62     def __init__(self):
63         self.decisions: List[ArchitectureDecision] = []
64
65     def score_alternative(
66         self,
67         alternative: Alternative,
68         criteria: Dict[str, float]
69     ) -> float:
70         """
71         Score an alternative based on weighted criteria.
72
73         Args:
74         alternative: Alternative to score
75         criteria: Dict of criterion -> weight
76
77         Returns:
78         Weighted score (0-100)
79         """
80         scores = {
81             'cost': (10 - alternative.cost_estimate / 10000) * 10,
82             'complexity': (10 - alternative.complexity_score) * 10,
83             'risk': (10 - alternative.risk_score) * 10,
84             'pros': len(alternative.pros) * 10,
85             'cons': (5 - len(alternative.cons)) * 10
86         }
87
88         weighted_score = sum(
89             scores.get(criterion, 0) * weight
90             for criterion, weight in criteria.items()
91         )
92
93         return weighted_score
94

```

```

95     def recommend_alternative(
96         self,
97         alternatives: List[Alternative],
98         criteria: Dict[str, float]
99     ) -> Alternative:
100         """Recommend best alternative based on criteria."""
101         scored = [
102             (alt, self.score_alternative(alt, criteria))
103             for alt in alternatives
104         ]
105         return max(scored, key=lambda x: x[1])[0]
106
107 # Example: Batch vs Stream Processing Decision
108 batch_alternative = Alternative(
109     name="Batch Processing",
110     description="Daily batch jobs for feature generation",
111     pros=[
112         "Simpler to implement and debug",
113         "Lower operational complexity",
114         "Better for large-scale data processing",
115         "Easier to reprocess historical data"
116     ],
117     cons=[
118         "Higher latency (24-hour delay)",
119         "Cannot handle real-time use cases",
120         "Resource spikes during batch windows"
121     ],
122     cost_estimate=5000, # monthly
123     complexity_score=3,
124     risk_score=2
125 )
126
127 stream_alternative = Alternative(
128     name="Stream Processing",
129     description="Real-time streaming with Kafka/Flink",
130     pros=[
131         "Low latency (seconds)",
132         "Enables real-time features",
133         "Smooth resource utilization",
134         "Better user experience"
135     ],
136     cons=[
137         "Higher operational complexity",
138         "More difficult to debug",
139         "Requires specialized expertise",
140         "Harder to reprocess data"
141     ],
142     cost_estimate=15000, # monthly
143     complexity_score=8,
144     risk_score=6
145 )
146
147 hybrid_alternative = Alternative(
148     name="Hybrid Batch + Stream",
149     description="Stream for real-time, batch for complex features",
150     pros=[
151         "Balanced latency and complexity",
152         "Leverages strengths of both",

```

```

153     "Can handle diverse use cases",
154     "Incremental migration path"
155 ],
156 cons=[
157     "Highest complexity overall",
158     "Need to maintain two systems",
159     "Potential consistency issues",
160     "Higher total cost"
161 ],
162 cost_estimate=18000, # monthly
163 complexity_score=9,
164 risk_score=7
165 )
166
167 # Create decision
168 decision = ArchitectureDecision(
169     id=1,
170     title="Data Processing Architecture",
171     date=datetime.now(),
172     status=DecisionStatus.PROPOSED,
173     decision_makers=["ML Lead", "Data Engineering Lead", "CTO"],
174     context="""
175     Need to design data processing pipeline for real-time
176     recommendation system with 10K RPS requirement.
177     """,
178     business_drivers=[
179         "Reduce latency to improve conversion rates",
180         "Enable personalization features",
181         "Support A/B testing"
182     ],
183     technical_drivers=[
184         "Handle 10K RPS",
185         "Process 100GB daily data",
186         "Support both batch and real-time features"
187     ],
188     constraints=[
189         "Budget: $20K/month",
190         "Team size: 3 data engineers",
191         "Timeline: 3 months"
192     ],
193     alternatives=[batch_alternative, stream_alternative, hybrid_alternative],
194     evaluation_criteria={
195         'cost': 0.2,
196         'complexity': 0.3,
197         'risk': 0.2,
198         'pros': 0.2,
199         'cons': 0.1
200     },
201     chosen_alternative="Hybrid Batch + Stream",
202     rationale="""
203     While hybrid approach has highest complexity, it provides
204     the best balance for our use case. We can start with batch
205     processing for complex features and add streaming for
206     real-time features incrementally.
207     """,
208     consequences=[
209         "Need to hire streaming expert",
210         "6-month timeline instead of 3",

```

```
211     "Invest in monitoring and tooling"
212 ],
213 validation_metrics=[
214     "Feature freshness < 1 minute",
215     "System uptime > 99.9%",
216     "Cost within budget"
217 ],
218 review_date=datetime(2025, 12, 31)
219 )
```

Listing 24.2: ADR Implementation

24.4.2 Trade-off Analysis Framework

Common Trade-offs in ML Systems:

Table 24.2: Architecture Trade-off Matrix

Dimension	Option A	Option B	Consider When
Latency	Batch (high)	Real-time (low)	User-facing vs. backend
Accuracy	Complex model	Simple model	Business value of improvement
Cost	GPU training	CPU training	Budget vs. time-to-market
Complexity	Monolith	Microservices	Team size and expertise
Freshness	Static model	Continuous learning	Data drift rate
Consistency	Strong	Eventual	Correctness requirements
Coupling	Tight	Loose	System stability needs

24.5 Integration Patterns and Best Practices

24.5.1 Data Integration Patterns

Pattern 1: Event-Driven Architecture

```
1  #!/usr/bin/env python3
2  """
3  Event-Driven Integration for ML Systems
4  """
5
6  import json
```

```

7 from typing import Dict, List, Callable, Any
8 from dataclasses import dataclass, asdict
9 from datetime import datetime
10 import asyncio
11 from abc import ABC, abstractmethod
12
13 @dataclass
14 class Event:
15     """Base event class."""
16     event_id: str
17     event_type: str
18     timestamp: datetime
19     source: str
20     data: Dict[str, Any]
21
22     def to_json(self) -> str:
23         """Serialize event to JSON."""
24         return json.dumps({
25             **asdict(self),
26             'timestamp': self.timestamp.isoformat()
27         })
28
29 class EventBus(ABC):
30     """Abstract event bus interface."""
31
32     @abstractmethod
33     async def publish(self, topic: str, event: Event):
34         """Publish event to topic."""
35         pass
36
37     @abstractmethod
38     async def subscribe(self, topic: str, handler: Callable):
39         """Subscribe to topic with handler."""
40         pass
41
42 class KafkaEventBus(EventBus):
43     """Kafka-based event bus implementation."""
44
45     def __init__(self, bootstrap_servers: List[str]):
46         self.bootstrap_servers = bootstrap_servers
47         self.handlers: Dict[str, List[Callable]] = {}
48
49     async def publish(self, topic: str, event: Event):
50         """Publish event to Kafka topic."""
51         # Simplified - use aiokafka in production
52         print(f"Publishing to {topic}: {event.to_json()}")
53
54     async def subscribe(self, topic: str, handler: Callable):
55         """Subscribe handler to topic."""
56         if topic not in self.handlers:
57             self.handlers[topic] = []
58         self.handlers[topic].append(handler)
59
60 class MLSystemIntegration:
61     """
62     Integration layer for ML system components.
63     Coordinates between data, training, and serving.
64     """

```



```

65
66 def __init__(self, event_bus: EventBus):
67     self.event_bus = event_bus
68     self._setup_handlers()
69
70 def _setup_handlers(self):
71     """Register event handlers for ML pipeline."""
72     # Data pipeline events
73     asyncio.create_task(
74         self.event_bus.subscribe(
75             'data.ingested',
76             self.handle_data_ingestion
77         )
78     )
79
80     # Training events
81     asyncio.create_task(
82         self.event_bus.subscribe(
83             'training.completed',
84             self.handle_training_completion
85         )
86     )
87
88     # Monitoring events
89     asyncio.create_task(
90         self.event_bus.subscribe(
91             'model.performance_degraded',
92             self.handle_performance_degradation
93         )
94     )
95
96 async def handle_data_ingestion(self, event: Event):
97     """
98     Handle new data ingestion.
99     Triggers feature computation and model evaluation.
100     """
101     print(f"Data ingested: {event.data['record_count']} records")
102
103     # Trigger feature computation
104     feature_event = Event(
105         event_id=f"feat-{event.event_id}",
106         event_type="feature.compute_requested",
107         timestamp=datetime.now(),
108         source="integration_layer",
109         data={
110             'data_batch_id': event.data['batch_id'],
111             'feature_set': 'v1'
112         }
113     )
114     await self.event_bus.publish('feature.requests', feature_event)
115
116     # Check if retraining needed
117     if self._should_retrain(event):
118         training_event = Event(
119             event_id=f"train-{event.event_id}",
120             event_type="training.requested",
121             timestamp=datetime.now(),
122             source="integration_layer",

```

```

123         data={
124             'trigger': 'scheduled',
125             'data_version': event.data['data_version']
126         }
127     )
128     await self.event_bus.publish('training.requests', training_event)
129
130 async def handle_training_completion(self, event: Event):
131     """
132     Handle training completion.
133     Validates model and triggers deployment if passing.
134     """
135     model_id = event.data['model_id']
136     metrics = event.data['metrics']
137
138     print(f"Training completed: {model_id}")
139     print(f"Metrics: {metrics}")
140
141     # Validate model
142     if self._validate_model(metrics):
143         # Trigger deployment
144         deploy_event = Event(
145             event_id=f"deploy-{event.event_id}",
146             event_type="deployment.requested",
147             timestamp=datetime.now(),
148             source="integration_layer",
149             data={
150                 'model_id': model_id,
151                 'strategy': 'canary',
152                 'canary_percentage': 10
153             }
154         )
155         await self.event_bus.publish('deployment.requests', deploy_event)
156     else:
157         # Alert on failed validation
158         alert_event = Event(
159             event_id=f"alert-{event.event_id}",
160             event_type="alert.model_validation_failed",
161             timestamp=datetime.now(),
162             source="integration_layer",
163             data={
164                 'model_id': model_id,
165                 'metrics': metrics,
166                 'severity': 'high'
167             }
168         )
169         await self.event_bus.publish('alerts', alert_event)
170
171 async def handle_performance_degradation(self, event: Event):
172     """
173     Handle model performance degradation.
174     Triggers retraining or rollback.
175     """
176     model_id = event.data['model_id']
177     metric = event.data['metric']
178     current_value = event.data['current_value']
179     threshold = event.data['threshold']
180

```

```

181     print(f"Performance degraded: {model_id}")
182     print(f"{metric}: {current_value} (threshold: {threshold})")
183
184     # Trigger retraining
185     training_event = Event(
186         event_id=f"train-{event.event_id}",
187         event_type="training.requested",
188         timestamp=datetime.now(),
189         source="integration_layer",
190         data={
191             'trigger': 'performance_degradation',
192             'model_to_replace': model_id
193         }
194     )
195     await self.event_bus.publish('training.requests', training_event)
196
197     def _should_retrain(self, event: Event) -> bool:
198         """Determine if retraining is needed."""
199         # Example logic
200         return event.data.get('record_count', 0) > 1000000
201
202     def _validate_model(self, metrics: Dict[str, float]) -> bool:
203         """Validate model metrics meet thresholds."""
204         return (
205             metrics.get('accuracy', 0) > 0.90 and
206             metrics.get('precision', 0) > 0.85 and
207             metrics.get('recall', 0) > 0.85
208         )
209
210     # Example usage
211     async def main():
212         # Initialize event bus
213         event_bus = KafkaEventBus(['localhost:9092'])
214
215         # Initialize integration layer
216         integration = MLSystemIntegration(event_bus)
217
218         # Simulate data ingestion event
219         data_event = Event(
220             event_id="data-001",
221             event_type="data.ingested",
222             timestamp=datetime.now(),
223             source="data_pipeline",
224             data={
225                 'batch_id': 'batch-2025-001',
226                 'record_count': 1500000,
227                 'data_version': 'v2.1'
228             }
229         )
230
231         await event_bus.publish('data.ingested', data_event)
232
233         # Simulate training completion
234         training_event = Event(
235             event_id="train-001",
236             event_type="training.completed",
237             timestamp=datetime.now(),
238             source="training_pipeline",

```

```

239     data={
240         'model_id': 'model-v2.1',
241         'metrics': {
242             'accuracy': 0.92,
243             'precision': 0.89,
244             'recall': 0.87
245         }
246     }
247 )
248
249     await event_bus.publish('training.completed', training_event)
250
251 # Run example
252 # asyncio.run(main())

```

Listing 24.3: Event-Driven ML System Integration

Pattern 2: API Gateway Integration

```

1  #!/usr/bin/env python3
2  """
3  API Gateway for ML System Integration
4  """
5
6  from fastapi import FastAPI, HTTPException, Depends, Header
7  from fastapi.middleware.cors import CORSMiddleware
8  from pydantic import BaseModel, Field
9  from typing import Dict, List, Optional, Any
10 import time
11 import logging
12 from datetime import datetime
13 import asyncio
14 from circuitbreaker import circuit
15 import redis
16
17 # Configure logging
18 logging.basicConfig(level=logging.INFO)
19 logger = logging.getLogger(__name__)
20
21 # Models
22 class PredictionRequest(BaseModel):
23     """Prediction request schema."""
24     user_id: str
25     context: Dict[str, Any]
26     features: Optional[Dict[str, float]] = None
27     model_version: Optional[str] = None
28
29 class PredictionResponse(BaseModel):
30     """Prediction response schema."""
31     prediction: Any
32     confidence: float
33     model_version: str
34     latency_ms: float
35     request_id: str
36
37 class HealthResponse(BaseModel):
38     """Health check response."""
39     status: str
40     timestamp: datetime

```

```

41     components: Dict[str, str]
42
43 # API Gateway
44 class MLAPIGateway:
45     """
46     API Gateway for ML system.
47     Handles routing, load balancing, caching, and monitoring.
48     """
49
50     def __init__(
51         self,
52         model_endpoints: Dict[str, str],
53         feature_store_url: str,
54         cache_url: str
55     ):
56         self.model_endpoints = model_endpoints
57         self.feature_store_url = feature_store_url
58         self.cache = redis.Redis.from_url(cache_url)
59
60         # Metrics
61         self.request_count = 0
62         self.error_count = 0
63         self.total_latency = 0.0
64
65     async def predict(
66         self,
67         request: PredictionRequest,
68         model_name: str = "default"
69     ) -> PredictionResponse:
70         """
71         Handle prediction request.
72         Implements caching, feature enrichment, and error handling.
73         """
74         start_time = time.time()
75         request_id = f"req-{int(time.time())}-{self.request_count}"
76         self.request_count += 1
77
78         try:
79             # Check cache
80             cache_key = self._get_cache_key(request, model_name)
81             cached = self._get_from_cache(cache_key)
82             if cached:
83                 logger.info(f"Cache hit: {request_id}")
84                 return cached
85
86             # Enrich with features from feature store
87             if request.features is None:
88                 request.features = await self._get_features(request.user_id)
89
90             # Get prediction from model service
91             prediction = await self._call_model_service(
92                 model_name,
93                 request
94             )
95
96             # Calculate latency
97             latency_ms = (time.time() - start_time) * 1000
98             self.total_latency += latency_ms

```

```

99
100     # Build response
101     response = PredictionResponse(
102         prediction=prediction['prediction'],
103         confidence=prediction['confidence'],
104         model_version=prediction['model_version'],
105         latency_ms=latency_ms,
106         request_id=request_id
107     )
108
109     # Cache response
110     self._cache_response(cache_key, response)
111
112     # Log metrics
113     logger.info(
114         f"Request {request_id}: "
115         f"latency={latency_ms:.2f}ms, "
116         f"confidence={prediction['confidence']:.3f}"
117     )
118
119     return response
120
121     except Exception as e:
122         self.error_count += 1
123         logger.error(f"Prediction error: {e}")
124         raise HTTPException(status_code=500, detail=str(e))
125
126 @circuit(failure_threshold=5, recovery_timeout=60)
127 async def _call_model_service(
128     self,
129     model_name: str,
130     request: PredictionRequest
131 ) -> Dict[str, Any]:
132     """
133     Call model service with circuit breaker.
134     Falls back to backup model if primary fails.
135     """
136     endpoint = self.model_endpoints.get(model_name)
137     if not endpoint:
138         raise ValueError(f"Unknown model: {model_name}")
139
140     try:
141         # Simulate model service call
142         await asyncio.sleep(0.01) # Simulated latency
143
144         # Return prediction
145         return {
146             'prediction': 0.85,
147             'confidence': 0.92,
148             'model_version': 'v1.2.3'
149         }
150     except Exception as e:
151         logger.warning(f"Primary model failed: {e}, trying backup")
152         return await self._call_backup_model(request)
153
154 async def _call_backup_model(
155     self,
156     request: PredictionRequest

```

```

157 ) -> Dict[str, Any]:
158     """Fallback to backup model."""
159     # Simplified backup model
160     return {
161         'prediction': 0.5, # Default prediction
162         'confidence': 0.5,
163         'model_version': 'backup-v1.0'
164     }
165
166 async def _get_features(self, user_id: str) -> Dict[str, float]:
167     """Fetch features from feature store."""
168     # Simulate feature store call
169     await asyncio.sleep(0.005)
170
171     return {
172         'feature_1': 0.5,
173         'feature_2': 0.8,
174         'feature_3': 0.3
175     }
176
177 def _get_cache_key(
178     self,
179     request: PredictionRequest,
180     model_name: str
181 ) -> str:
182     """Generate cache key."""
183     return f"pred:{model_name}:{request.user_id}:{hash(str(request.context))}"
184
185 def _get_from_cache(self, key: str) -> Optional[PredictionResponse]:
186     """Get prediction from cache."""
187     try:
188         cached = self.cache.get(key)
189         if cached:
190             return PredictionResponse.parse_raw(cached)
191     except:
192         pass
193     return None
194
195 def _cache_response(
196     self,
197     key: str,
198     response: PredictionResponse,
199     ttl: int = 300
200 ):
201     """Cache prediction response."""
202     try:
203         self.cache.setex(
204             key,
205             ttl,
206             response.json()
207         )
208     except Exception as e:
209         logger.warning(f"Cache write failed: {e}")
210
211 def get_metrics(self) -> Dict[str, Any]:
212     """Get gateway metrics."""
213     avg_latency = (
214         self.total_latency / self.request_count

```

```

215         if self.request_count > 0 else 0
216     )
217     error_rate = (
218         self.error_count / self.request_count
219         if self.request_count > 0 else 0
220     )
221
222     return {
223         'request_count': self.request_count,
224         'error_count': self.error_count,
225         'error_rate': error_rate,
226         'avg_latency_ms': avg_latency
227     }
228
229 # FastAPI application
230 app = FastAPI(title="ML API Gateway")
231
232 # Add CORS
233 app.add_middleware(
234     CORSMiddleware,
235     allow_origins=["*"],
236     allow_methods=["*"],
237     allow_headers=["*"],
238 )
239
240 # Initialize gateway
241 gateway = MLAPIGateway(
242     model_endpoints={
243         'default': 'http://model-service:8000',
244         'recommendation': 'http://rec-service:8001',
245         'fraud': 'http://fraud-service:8002'
246     },
247     feature_store_url='http://feature-store:8003',
248     cache_url='redis://localhost:6379'
249 )
250
251 # Authentication dependency
252 async def verify_api_key(x_api_key: str = Header(...)):
253     """Verify API key."""
254     if x_api_key != "secret-key": # Use proper auth in production
255         raise HTTPException(status_code=401, detail="Invalid API key")
256     return x_api_key
257
258 # Endpoints
259 @app.post("/predict/{model_name}", response_model=PredictionResponse)
260 async def predict(
261     model_name: str,
262     request: PredictionRequest,
263     api_key: str = Depends(verify_api_key)
264 ):
265     """Prediction endpoint."""
266     return await gateway.predict(request, model_name)
267
268 @app.get("/health", response_model=HealthResponse)
269 async def health_check():
270     """Health check endpoint."""
271     return HealthResponse(
272         status="healthy",

```



```

273         timestamp=datetime.now(),
274         components={
275             'api_gateway': 'up',
276             'cache': 'up',
277             'model_service': 'up'
278         }
279     )
280
281 @app.get("/metrics")
282 async def metrics():
283     """Metrics endpoint."""
284     return gateway.get_metrics()

```

Listing 24.4: API Gateway for ML System

24.5.2 Model Integration Patterns

Pattern 3: Model Ensemble Integration

```

1  #!/usr/bin/env python3
2  """
3  Ensemble Model Integration Pattern
4  """
5
6  from typing import List, Dict, Any, Optional
7  from dataclasses import dataclass
8  import numpy as np
9  from abc import ABC, abstractmethod
10 import asyncio
11
12 @dataclass
13 class ModelPrediction:
14     """Single model prediction."""
15     model_id: str
16     prediction: Any
17     confidence: float
18     latency_ms: float
19     metadata: Dict[str, Any]
20
21 class EnsembleStrategy(ABC):
22     """Abstract ensemble strategy."""
23
24     @abstractmethod
25     def combine(
26         self,
27         predictions: List[ModelPrediction]
28     ) -> ModelPrediction:
29         """Combine predictions from multiple models."""
30         pass
31
32 class AveragingEnsemble(EnsembleStrategy):
33     """Averaging ensemble strategy."""
34
35     def combine(
36         self,
37         predictions: List[ModelPrediction]
38     ) -> ModelPrediction:

```

```

39     """Average predictions."""
40     pred_values = [p.prediction for p in predictions]
41     confidences = [p.confidence for p in predictions]
42
43     return ModelPrediction(
44         model_id="ensemble_average",
45         prediction=np.mean(pred_values),
46         confidence=np.mean(confidences),
47         latency_ms=max(p.latency_ms for p in predictions),
48         metadata={'n_models': len(predictions)}
49     )
50
51 class WeightedEnsemble(EnsembleStrategy):
52     """Weighted ensemble strategy."""
53
54     def __init__(self, weights: Dict[str, float]):
55         self.weights = weights
56
57     def combine(
58         self,
59         predictions: List[ModelPrediction]
60     ) -> ModelPrediction:
61         """Weighted average of predictions."""
62         weighted_sum = sum(
63             p.prediction * self.weights.get(p.model_id, 1.0)
64             for p in predictions
65         )
66         total_weight = sum(
67             self.weights.get(p.model_id, 1.0)
68             for p in predictions
69         )
70
71         return ModelPrediction(
72             model_id="ensemble_weighted",
73             prediction=weighted_sum / total_weight,
74             confidence=max(p.confidence for p in predictions),
75             latency_ms=max(p.latency_ms for p in predictions),
76             metadata={
77                 'n_models': len(predictions),
78                 'weights': self.weights
79             }
80         )
81
82 class StackingEnsemble(EnsembleStrategy):
83     """Stacking ensemble with meta-learner."""
84
85     def __init__(self, meta_model):
86         self.meta_model = meta_model
87
88     def combine(
89         self,
90         predictions: List[ModelPrediction]
91     ) -> ModelPrediction:
92         """Use meta-model to combine predictions."""
93         # Stack predictions as features for meta-model
94         features = np.array([
95             [p.prediction, p.confidence]
96             for p in predictions

```

```

97         ]).flatten()
98
99         # Meta-model prediction
100         meta_pred = self.meta_model.predict(features.reshape(1, -1))[0]
101
102         return ModelPrediction(
103             model_id="ensemble_stacking",
104             prediction=meta_pred,
105             confidence=0.9, # From meta-model
106             latency_ms=max(p.latency_ms for p in predictions) + 5,
107             metadata={'n_models': len(predictions)}
108         )
109
110 class EnsembleOrchestrator:
111     """
112     Orchestrates ensemble of models.
113     Handles parallel execution and result combination.
114     """
115
116     def __init__(
117         self,
118         models: Dict[str, Any],
119         strategy: EnsembleStrategy,
120         timeout: float = 1.0
121     ):
122         self.models = models
123         self.strategy = strategy
124         self.timeout = timeout
125
126     async def predict(
127         self,
128         features: Dict[str, Any]
129     ) -> ModelPrediction:
130         """
131         Get predictions from all models and combine.
132
133         Args:
134             features: Input features
135
136         Returns:
137             Combined prediction
138         """
139         # Execute models in parallel
140         tasks = [
141             self._predict_single(model_id, model, features)
142             for model_id, model in self.models.items()
143         ]
144
145         try:
146             predictions = await asyncio.gather(
147                 *tasks,
148                 return_exceptions=True
149             )
150
151             # Filter out exceptions
152             valid_predictions = [
153                 p for p in predictions
154                 if isinstance(p, ModelPrediction)

```

```

155         ]
156
157         if not valid_predictions:
158             raise RuntimeError("All models failed")
159
160         # Combine predictions
161         return self.strategy.combine(valid_predictions)
162
163     except asyncio.TimeoutError:
164         # Fallback to fastest responding model
165         return await self._fallback_prediction(features)
166
167     async def _predict_single(
168         self,
169         model_id: str,
170         model: Any,
171         features: Dict[str, Any]
172     ) -> ModelPrediction:
173         """Get prediction from single model."""
174         import time
175         start = time.time()
176
177         try:
178             # Simulate model prediction
179             await asyncio.sleep(0.01)
180             prediction = 0.8 # model.predict(features)
181             confidence = 0.9
182
183             latency_ms = (time.time() - start) * 1000
184
185             return ModelPrediction(
186                 model_id=model_id,
187                 prediction=prediction,
188                 confidence=confidence,
189                 latency_ms=latency_ms,
190                 metadata={}
191             )
192         except Exception as e:
193             raise RuntimeError(f"Model {model_id} failed: {e}")
194
195     async def _fallback_prediction(
196         self,
197         features: Dict[str, Any]
198     ) -> ModelPrediction:
199         """Fallback when ensemble fails."""
200         return ModelPrediction(
201             model_id="fallback",
202             prediction=0.5,
203             confidence=0.5,
204             latency_ms=0.0,
205             metadata={'reason': 'timeout'}
206         )
207
208     # Example usage
209     async def main():
210         # Mock models
211         models = {
212             'model_a': None, # XGBoost

```

```
213     'model_b': None, # LightGBM
214     'model_c': None # Neural Network
215 }
216
217 # Weighted ensemble
218 strategy = WeightedEnsemble({
219     'model_a': 0.4,
220     'model_b': 0.35,
221     'model_c': 0.25
222 })
223
224 orchestrator = EnsembleOrchestrator(models, strategy)
225
226 # Get prediction
227 features = {'user_id': '123', 'item_id': '456'}
228 result = await orchestrator.predict(features)
229
230 print(f"Ensemble prediction: {result.prediction}")
231 print(f"Confidence: {result.confidence}")
232 print(f"Latency: {result.latency_ms}ms")
233
234 # asyncio.run(main())
```

Listing 24.5: Ensemble Model Integration

24.6 Performance and Scalability Considerations

24.6.1 Performance Optimization Strategies

1. Latency Optimization

Table 24.3: Latency Optimization Techniques

Technique	Latency Reduction	Trade-offs
Caching	80-95% (cache hits)	Stale data, memory cost
Model quantization	40-60%	1-2% accuracy loss
Batching	30-50% (throughput)	Increased latency per request
Feature precomputation	50-70%	Storage cost, freshness
Model pruning	30-40%	Slight accuracy loss
GPU inference	70-90% (vs CPU)	Higher cost

2. Scalability Patterns

```
1 #!/usr/bin/env python3
2 """
3 Auto-Scaling Configuration for ML Service
4 """
```

```

5
6 from dataclasses import dataclass
7 from typing import List, Dict
8 from enum import Enum
9
10 class ScalingMetric(Enum):
11     """Metrics for autoscaling decisions."""
12     CPU_UTILIZATION = "cpu"
13     MEMORY_UTILIZATION = "memory"
14     REQUEST_COUNT = "requests"
15     LATENCY_P99 = "latency_p99"
16     QUEUE_DEPTH = "queue_depth"
17
18 @dataclass
19 class ScalingPolicy:
20     """Auto-scaling policy configuration."""
21
22     # Scale-up triggers
23     scale_up_metric: ScalingMetric
24     scale_up_threshold: float
25     scale_up_evaluation_periods: int
26     scale_up_cooldown_seconds: int
27
28     # Scale-down triggers
29     scale_down_metric: ScalingMetric
30     scale_down_threshold: float
31     scale_down_evaluation_periods: int
32     scale_down_cooldown_seconds: int
33
34     # Capacity limits
35     min_instances: int
36     max_instances: int
37     target_instances: int
38
39     # Scaling increments
40     scale_up_increment: int = 2
41     scale_down_increment: int = 1
42
43 @dataclass
44 class ResourceConfiguration:
45     """Resource configuration for ML service."""
46
47     # Compute
48     cpu_cores: int
49     memory_gb: int
50     gpu_count: int
51     gpu_type: str
52
53     # Storage
54     disk_gb: int
55
56     # Network
57     max_concurrent_requests: int
58     request_timeout_seconds: int
59
60 class MLServiceScalingConfig:
61     """Complete scaling configuration for ML service."""
62

```

```

63 def __init__(self):
64     # Resource configuration per instance
65     self.resources = ResourceConfiguration(
66         cpu_cores=4,
67         memory_gb=16,
68         gpu_count=1,
69         gpu_type="T4",
70         disk_gb=100,
71         max_concurrent_requests=100,
72         request_timeout_seconds=30
73     )
74
75     # Scaling policy
76     self.scaling_policy = ScalingPolicy(
77         scale_up_metric=ScalingMetric.CPU_UTILIZATION,
78         scale_up_threshold=70.0, # 70% CPU
79         scale_up_evaluation_periods=2, # 2 periods above threshold
80         scale_up_cooldown_seconds=300, # 5 minutes
81         scale_down_metric=ScalingMetric.CPU_UTILIZATION,
82         scale_down_threshold=30.0, # 30% CPU
83         scale_down_evaluation_periods=5, # 5 periods below threshold
84         scale_down_cooldown_seconds=600, # 10 minutes
85         min_instances=2, # For high availability
86         max_instances=20,
87         target_instances=3,
88         scale_up_increment=2, # Scale up by 2 instances
89         scale_down_increment=1 # Scale down by 1 instance
90     )
91
92 def get_kubernetes_manifest(self) -> Dict:
93     """Generate Kubernetes HPA manifest."""
94     return {
95         'apiVersion': 'autoscaling/v2',
96         'kind': 'HorizontalPodAutoscaler',
97         'metadata': {
98             'name': 'ml-service-hpa'
99         },
100         'spec': {
101             'scaleTargetRef': {
102                 'apiVersion': 'apps/v1',
103                 'kind': 'Deployment',
104                 'name': 'ml-service'
105             },
106             'minReplicas': self.scaling_policy.min_instances,
107             'maxReplicas': self.scaling_policy.max_instances,
108             'metrics': [
109                 {
110                     'type': 'Resource',
111                     'resource': {
112                         'name': 'cpu',
113                         'target': {
114                             'type': 'Utilization',
115                             'averageUtilization': int(
116                                 self.scaling_policy.scale_up_threshold
117                             )
118                         }
119                     }
120                 }
121             ]
122         }
123     }

```

```

121         ],
122         'behavior': {
123             'scaleUp': {
124                 'stabilizationWindowSeconds': (
125                     self.scaling_policy.scale_up_cooldown_seconds
126                 ),
127                 'policies': [
128                     {
129                         'type': 'Pods',
130                         'value': self.scaling_policy.scale_up_increment,
131                         'periodSeconds': 60
132                     }
133                 ]
134             },
135             'scaleDown': {
136                 'stabilizationWindowSeconds': (
137                     self.scaling_policy.scale_down_cooldown_seconds
138                 ),
139                 'policies': [
140                     {
141                         'type': 'Pods',
142                         'value': self.scaling_policy.scale_down_increment,
143                         'periodSeconds': 60
144                     }
145                 ]
146             }
147         }
148     }
149 }
150
151 # Example configuration
152 config = MLServiceScalingConfig()
153 hpa_manifest = config.get_kubernetes_manifest()

```

Listing 24.6: Auto-Scaling ML Service

3. Load Balancing Strategies

- **Round-robin:** Simple, even distribution (good for homogeneous instances)
- **Least-connections:** Route to instance with fewest connections
- **Weighted:** Distribute based on instance capacity
- **Latency-based:** Route to lowest-latency instance
- **Model-aware:** Route to specialized models based on request characteristics

24.7 Evolution and Maintenance Strategies

24.7.1 System Evolution Framework

Evolution Maturity Levels:

Table 24.4: ML System Maturity Model

Level	Characteristics	Capabilities
Level 0: Manual	Scripts, notebooks	One-off model training
Level 1: Automated Training	Training pipelines	Reproducible training
Level 2: Automated Deployment	CI/CD for ML	Automated model deployment
Level 3: Automated Monitoring	Monitoring + alerts	Detect degradation
Level 4: Automated Retraining	Trigger-based retraining	Self-healing system
Level 5: Full MLOps	End-to-end automation	Continuous learning

24.7.2 Maintenance Strategies

```

1  #!/usr/bin/env python3
2  """
3  ML System Maintenance Framework
4  """
5
6  from dataclasses import dataclass
7  from datetime import datetime, timedelta
8  from typing import List, Dict, Optional
9  from enum import Enum
10
11 class MaintenanceType(Enum):
12     """Types of maintenance activities."""
13     PREVENTIVE = "preventive"
14     CORRECTIVE = "corrective"
15     ADAPTIVE = "adaptive"
16     PERFECTIVE = "perfective"
17
18 @dataclass
19 class MaintenanceTask:
20     """Maintenance task definition."""

```

```

21     task_id: str
22     task_type: MaintenanceType
23     description: str
24     frequency: str # e.g., "daily", "weekly", "monthly"
25     estimated_duration: timedelta
26     impact: str # "low", "medium", "high"
27     requires_downtime: bool
28
29 class MLSystemMaintenance:
30     """Framework for ML system maintenance."""
31
32     def __init__(self):
33         self.tasks = self._define_maintenance_tasks()
34
35     def _define_maintenance_tasks(self) -> List[MaintenanceTask]:
36         """Define all maintenance tasks."""
37         return [
38             # Preventive maintenance
39             MaintenanceTask(
40                 task_id="prev-001",
41                 task_type=MaintenanceType.PREVENTIVE,
42                 description="Monitor data drift and feature distributions",
43                 frequency="daily",
44                 estimated_duration=timedelta(hours=1),
45                 impact="low",
46                 requires_downtime=False
47             ),
48             MaintenanceTask(
49                 task_id="prev-002",
50                 task_type=MaintenanceType.PREVENTIVE,
51                 description="Review model performance metrics",
52                 frequency="daily",
53                 estimated_duration=timedelta(hours=2),
54                 impact="low",
55                 requires_downtime=False
56             ),
57             MaintenanceTask(
58                 task_id="prev-003",
59                 task_type=MaintenanceType.PREVENTIVE,
60                 description="Check infrastructure health and resource usage",
61                 frequency="daily",
62                 estimated_duration=timedelta(minutes=30),
63                 impact="low",
64                 requires_downtime=False
65             ),
66             MaintenanceTask(
67                 task_id="prev-004",
68                 task_type=MaintenanceType.PREVENTIVE,
69                 description="Validate data pipeline integrity",
70                 frequency="weekly",
71                 estimated_duration=timedelta(hours=3),
72                 impact="medium",
73                 requires_downtime=False
74             ),
75             MaintenanceTask(
76                 task_id="prev-005",
77                 task_type=MaintenanceType.PREVENTIVE,
78                 description="Update dependencies and security patches",

```

```

79         frequency="monthly",
80         estimated_duration=timedelta(hours=4),
81         impact="medium",
82         requires_downtime=True
83     ),
84
85     # Corrective maintenance
86     MaintenanceTask(
87         task_id="corr-001",
88         task_type=MaintenanceType.CORRECTIVE,
89         description="Retrain model due to performance degradation",
90         frequency="as_needed",
91         estimated_duration=timedelta(hours=8),
92         impact="high",
93         requires_downtime=False
94     ),
95     MaintenanceTask(
96         task_id="corr-002",
97         task_type=MaintenanceType.CORRECTIVE,
98         description="Fix data quality issues",
99         frequency="as_needed",
100        estimated_duration=timedelta(hours=4),
101        impact="high",
102        requires_downtime=False
103    ),
104    MaintenanceTask(
105        task_id="corr-003",
106        task_type=MaintenanceType.CORRECTIVE,
107        description="Rollback failed deployment",
108        frequency="as_needed",
109        estimated_duration=timedelta(minutes=30),
110        impact="critical",
111        requires_downtime=True
112    ),
113
114    # Adaptive maintenance
115    MaintenanceTask(
116        task_id="adapt-001",
117        task_type=MaintenanceType.ADAPTIVE,
118        description="Add new features based on changing patterns",
119        frequency="quarterly",
120        estimated_duration=timedelta(days=5),
121        impact="medium",
122        requires_downtime=False
123    ),
124    MaintenanceTask(
125        task_id="adapt-002",
126        task_type=MaintenanceType.ADAPTIVE,
127        description="Adapt to new data sources or formats",
128        frequency="as_needed",
129        estimated_duration=timedelta(days=3),
130        impact="high",
131        requires_downtime=False
132    ),
133
134    # Perfective maintenance
135    MaintenanceTask(
136        task_id="perf-001",

```

```

137         task_type=MaintenanceType.PERFECTIVE,
138         description="Optimize model latency",
139         frequency="quarterly",
140         estimated_duration=timedelta(days=7),
141         impact="medium",
142         requires_downtime=False
143     ),
144     MaintenanceTask(
145         task_id="perf-002",
146         task_type=MaintenanceType.PERFECTIVE,
147         description="Improve model accuracy",
148         frequency="quarterly",
149         estimated_duration=timedelta(days=10),
150         impact="medium",
151         requires_downtime=False
152     ),
153     MaintenanceTask(
154         task_id="perf-003",
155         task_type=MaintenanceType.PERFECTIVE,
156         description="Reduce infrastructure costs",
157         frequency="quarterly",
158         estimated_duration=timedelta(days=5),
159         impact="low",
160         requires_downtime=False
161     )
162 ]
163
164 def get_maintenance_schedule(
165     self,
166     start_date: datetime,
167     end_date: datetime
168 ) -> List[tuple[datetime, MaintenanceTask]]:
169     """
170     Generate maintenance schedule for date range.
171
172     Args:
173         start_date: Schedule start date
174         end_date: Schedule end date
175
176     Returns:
177         List of (scheduled_datetime, task) tuples
178     """
179     schedule = []
180
181     for task in self.tasks:
182         if task.frequency == "as_needed":
183             continue
184
185         # Calculate occurrences
186         current_date = start_date
187         while current_date <= end_date:
188             schedule.append((current_date, task))
189
190         # Increment based on frequency
191         if task.frequency == "daily":
192             current_date += timedelta(days=1)
193         elif task.frequency == "weekly":
194             current_date += timedelta(weeks=1)

```

```

195         elif task.frequency == "monthly":
196             current_date += timedelta(days=30)
197         elif task.frequency == "quarterly":
198             current_date += timedelta(days=90)
199         else:
200             break
201
202     return sorted(schedule, key=lambda x: x[0])
203
204     def estimate_maintenance_effort(
205         self,
206         period_days: int = 30
207     ) -> Dict[str, float]:
208         """
209         Estimate maintenance effort for period.
210
211         Args:
212             period_days: Period in days
213
214         Returns:
215             Dict with effort estimates by maintenance type
216         """
217         effort = {
218             'preventive': 0.0,
219             'corrective': 0.0,
220             'adaptive': 0.0,
221             'perfective': 0.0,
222             'total': 0.0
223         }
224
225         for task in self.tasks:
226             if task.frequency == "as_needed":
227                 continue
228
229             # Calculate occurrences in period
230             if task.frequency == "daily":
231                 occurrences = period_days
232             elif task.frequency == "weekly":
233                 occurrences = period_days / 7
234             elif task.frequency == "monthly":
235                 occurrences = period_days / 30
236             elif task.frequency == "quarterly":
237                 occurrences = period_days / 90
238             else:
239                 occurrences = 0
240
241             # Add to effort
242             hours = occurrences * task.estimated_duration.total_seconds() / 3600
243             effort[task.task_type.value] += hours
244             effort['total'] += hours
245
246         return effort
247
248     # Example usage
249     maintenance = MLSystemMaintenance()
250
251     # Get 30-day schedule
252     start = datetime.now()

```

```

253 end = start + timedelta(days=30)
254 schedule = maintenance.get_maintenance_schedule(start, end)
255
256 print(f"Maintenance schedule ({len(schedule)} tasks):")
257 for date, task in schedule[:5]:
258     print(f"    {date.strftime('%Y-%m-%d')}: {task.description}")
259
260 # Estimate effort
261 effort = maintenance.estimate_maintenance_effort(30)
262 print(f"\nEstimated 30-day effort: {effort['total']:.1f} hours")
263 print(f"    Preventive: {effort['preventive']:.1f}h")
264 print(f"    Corrective: {effort['corrective']:.1f}h")
265 print(f"    Adaptive: {effort['adaptive']:.1f}h")
266 print(f"    Perfective: {effort['perfective']:.1f}h")

```

Listing 24.7: System Maintenance Framework

24.8 Real-World System Examples

24.8.1 Example 1: Real-Time Content Moderation System

Business Context:

- Social media platform with 50M daily active users
- Need to detect and remove harmful content (hate speech, violence, spam)
- Requirements: <100ms latency, 99.5% recall, 95% precision
- Handle multiple content types: text, images, video

System Architecture:

Key Design Decisions:

1. **Parallel Processing:** Text, image, and video classifiers run in parallel to minimize latency
2. **Ensemble Scoring:** Combine signals from multiple models for robust detection
3. **Human-in-the-Loop:** Route borderline cases to human review ($0.7 < \text{score} < 0.9$)
4. **Model Serving:** Use TorchServe with GPU inference for image/video models
5. **Caching:** Cache recent decisions for duplicate content (40% cache hit rate)
6. **Monitoring:** Track precision/recall, false positive rate, and user appeals

Implementation Highlights:

```

1  #!/usr/bin/env python3
2  """
3  Content Moderation Decision Engine
4  """
5

```

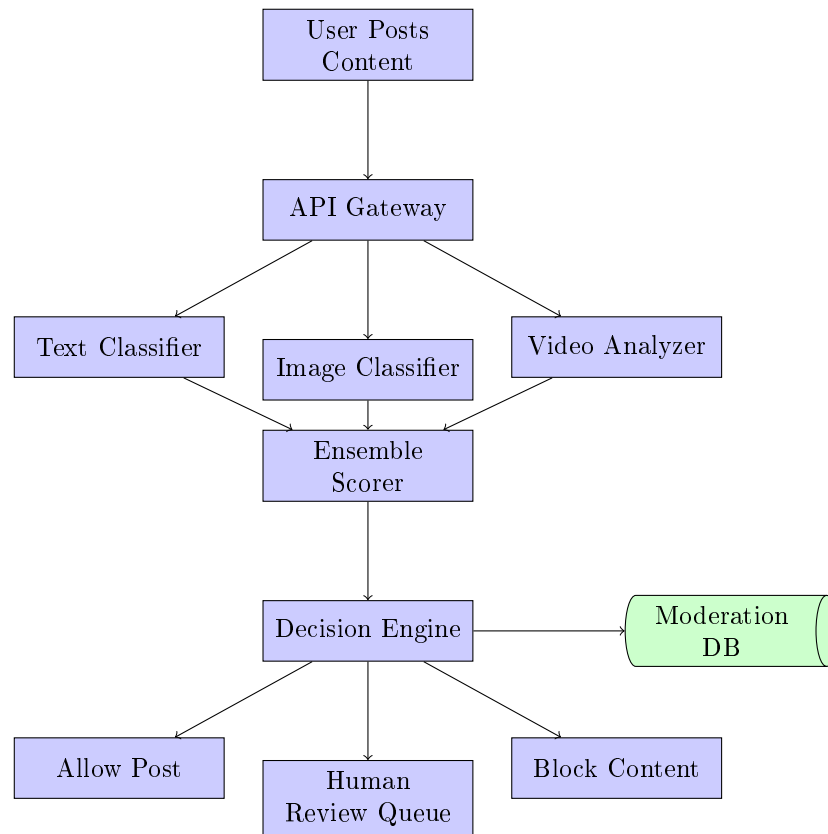


Figure 24.2: Content Moderation System Architecture

```

6 from dataclasses import dataclass
7 from typing import List, Dict, Optional
8 from enum import Enum
9
10 class ContentType(Enum):
11     TEXT = "text"
12     IMAGE = "image"
13     VIDEO = "video"
14
15 class ModerationAction(Enum):
16     ALLOW = "allow"
17     REVIEW = "review"
18     BLOCK = "block"
19
20 @dataclass
21 class ModerationScore:
22     """Moderation score from a single model."""
23     content_type: ContentType
24     violation_type: str # "hate_speech", "violence", etc.
25     score: float # 0.0 (safe) to 1.0 (harmful)
26     confidence: float
27
28 @dataclass
29 class ModerationDecision:
30     """Final moderation decision."""
31     action: ModerationAction
32     combined_score: float
33     reason: str

```

```
34     model_scores: List[ModerationScore]
35     requires_human_review: bool
36
37 class ModerationEngine:
38     """
39     Decision engine for content moderation.
40     Combines signals from multiple models and applies business rules.
41     """
42
43     def __init__(
44         self,
45         block_threshold: float = 0.9,
46         review_threshold: float = 0.7,
47         weights: Optional[Dict[str, float]] = None
48     ):
49         self.block_threshold = block_threshold
50         self.review_threshold = review_threshold
51         self.weights = weights or {
52             'hate_speech': 1.0,
53             'violence': 1.0,
54             'spam': 0.7,
55             'nudity': 0.8
56         }
57
58     def make_decision(
59         self,
60         scores: List[ModerationScore]
61     ) -> ModerationDecision:
62         """
63         Make moderation decision based on model scores.
64
65         Args:
66             scores: List of scores from different models
67
68         Returns:
69             Final moderation decision
70         """
71         # Calculate weighted combined score
72         combined_score = self._combine_scores(scores)
73
74         # Apply thresholds
75         if combined_score >= self.block_threshold:
76             action = ModerationAction.BLOCK
77             reason = self._get_block_reason(scores)
78             requires_review = False
79         elif combined_score >= self.review_threshold:
80             action = ModerationAction.REVIEW
81             reason = "Borderline case requiring human judgment"
82             requires_review = True
83         else:
84             action = ModerationAction.ALLOW
85             reason = "Content passes moderation checks"
86             requires_review = False
87
88         # Check for high-confidence violations
89         if self._has_high_confidence_violation(scores):
90             action = ModerationAction.BLOCK
91             requires_review = False
```



```

92         return ModerationDecision(
93             action=action,
94             combined_score=combined_score,
95             reason=reason,
96             model_scores=scores,
97             requires_human_review=requires_review
98         )
99
100
101     def _combine_scores(self, scores: List[ModerationScore]) -> float:
102         """Combine scores using weighted average."""
103         if not scores:
104             return 0.0
105
106         weighted_sum = sum(
107             score.score * self.weights.get(score.violation_type, 1.0)
108             for score in scores
109         )
110         total_weight = sum(
111             self.weights.get(score.violation_type, 1.0)
112             for score in scores
113         )
114
115         return weighted_sum / total_weight if total_weight > 0 else 0.0
116
117     def _has_high_confidence_violation(
118         self,
119         scores: List[ModerationScore]
120     ) -> bool:
121         """Check if any model has high-confidence violation."""
122         return any(
123             score.score >= 0.95 and score.confidence >= 0.9
124             for score in scores
125         )
126
127     def _get_block_reason(self, scores: List[ModerationScore]) -> str:
128         """Get human-readable reason for blocking."""
129         violations = [
130             score.violation_type
131             for score in scores
132             if score.score >= self.review_threshold
133         ]
134
135         if len(violations) == 1:
136             return f"Content violates policy: {violations[0]}"
137         else:
138             return f"Content violates multiple policies: {'', '.join(violations)}"
139
140 # Example usage
141 engine = ModerationEngine()
142
143 scores = [
144     ModerationScore(
145         content_type=ContentType.TEXT,
146         violation_type="hate_speech",
147         score=0.92,
148         confidence=0.88
149     ),

```

```

150     ModerationScore(
151         content_type=ContentType.IMAGE,
152         violation_type="violence",
153         score=0.45,
154         confidence=0.75
155     )
156 ]
157
158 decision = engine.make_decision(scores)
159 print(f"Action: {decision.action.value}")
160 print(f"Score: {decision.combined_score:.3f}")
161 print(f"Reason: {decision.reason}")

```

Listing 24.8: Content Moderation Decision Engine

Results:

- Achieved 99.7% recall, 94% precision
- Reduced manual review workload by 60%
- p99 latency: 85ms
- Cost: \$0.0003 per moderation

24.8.2 Example 2: Dynamic Pricing System

Business Context:

- Ride-sharing platform with 10M weekly trips
- Need to optimize prices based on supply/demand
- Requirements: Maximize revenue while maintaining acceptable wait times
- Handle surge pricing during peak hours and special events

System Architecture:

```

1  #!/usr/bin/env python3
2  """
3  Dynamic Pricing System for Ride-Sharing
4  """
5
6  import numpy as np
7  from dataclasses import dataclass
8  from typing import Dict, List, Optional
9  from datetime import datetime, timedelta
10
11 @dataclass
12 class MarketConditions:
13     """Current market conditions for pricing."""
14     location_id: str
15     available_drivers: int
16     pending_requests: int
17     avg_wait_time_minutes: float
18     hour_of_day: int
19     day_of_week: int

```

```

20     is_special_event: bool
21     weather_condition: str
22     historical_demand: float
23
24 @dataclass
25 class PricingDecision:
26     """Pricing decision output."""
27     base_price: float
28     surge_multiplier: float
29     final_price: float
30     expected_wait_time: float
31     confidence: float
32     reasoning: str
33
34 class DynamicPricingEngine:
35     """
36     ML-powered dynamic pricing engine.
37     Uses demand forecasting, supply estimation, and optimization.
38     """
39
40     def __init__(
41         self,
42         base_price: float = 10.0,
43         min_multiplier: float = 1.0,
44         max_multiplier: float = 5.0
45     ):
46         self.base_price = base_price
47         self.min_multiplier = min_multiplier
48         self.max_multiplier = max_multiplier
49
50     def calculate_price(
51         self,
52         conditions: MarketConditions
53     ) -> PricingDecision:
54         """
55         Calculate optimal price based on market conditions.
56
57         Args:
58             conditions: Current market conditions
59
60         Returns:
61             Pricing decision
62         """
63         # Calculate supply-demand ratio
64         supply_demand_ratio = (
65             conditions.available_drivers /
66             max(conditions.pending_requests, 1)
67         )
68
69         # Base surge multiplier on supply-demand
70         if supply_demand_ratio >= 1.5:
71             # Oversupply: no surge
72             surge_multiplier = 1.0
73         elif supply_demand_ratio >= 0.8:
74             # Balanced: minimal surge
75             surge_multiplier = 1.2
76         elif supply_demand_ratio >= 0.5:
77             # High demand: moderate surge

```

```

78         surge_multiplier = 2.0
79     else:
80         # Very high demand: high surge
81         surge_multiplier = 3.5
82
83     # Adjust for time of day
84     if conditions.hour_of_day in [7, 8, 9, 17, 18, 19]:
85         # Peak commute hours
86         surge_multiplier *= 1.2
87     elif conditions.hour_of_day >= 22 or conditions.hour_of_day <= 5:
88         # Late night/early morning
89         surge_multiplier *= 1.3
90
91     # Adjust for special events
92     if conditions.is_special_event:
93         surge_multiplier *= 1.5
94
95     # Adjust for weather
96     if conditions.weather_condition in ['rain', 'snow']:
97         surge_multiplier *= 1.3
98
99     # Cap multiplier
100    surge_multiplier = np.clip(
101        surge_multiplier,
102        self.min_multiplier,
103        self.max_multiplier
104    )
105
106    # Calculate final price
107    final_price = self.base_price * surge_multiplier
108
109    # Estimate wait time
110    expected_wait_time = self._estimate_wait_time(
111        conditions,
112        surge_multiplier
113    )
114
115    # Generate reasoning
116    reasoning = self._generate_reasoning(
117        supply_demand_ratio,
118        surge_multiplier,
119        conditions
120    )
121
122    return PricingDecision(
123        base_price=self.base_price,
124        surge_multiplier=surge_multiplier,
125        final_price=final_price,
126        expected_wait_time=expected_wait_time,
127        confidence=0.85,
128        reasoning=reasoning
129    )
130
131    def _estimate_wait_time(
132        self,
133        conditions: MarketConditions,
134        surge_multiplier: float
135    ) -> float:

```

```

136     """Estimate wait time based on conditions and pricing."""
137     # Higher prices attract more drivers and reduce demand
138     elasticity = 0.3 # 30% demand reduction per 2x price increase
139     demand_adjustment = 1 - (elasticity * np.log2(surge_multiplier))
140
141     adjusted_requests = conditions.pending_requests * demand_adjustment
142
143     # Simple queuing model
144     if conditions.available_drivers == 0:
145         return 30.0 # Max wait time
146
147     wait_time = (adjusted_requests / conditions.available_drivers) * 3.0
148     return min(wait_time, 30.0)
149
150 def _generate_reasoning(
151     self,
152     supply_demand_ratio: float,
153     surge_multiplier: float,
154     conditions: MarketConditions
155 ) -> str:
156     """Generate human-readable pricing reasoning."""
157     reasons = []
158
159     if supply_demand_ratio < 0.5:
160         reasons.append("high demand")
161     elif supply_demand_ratio > 1.5:
162         reasons.append("low demand")
163
164     if conditions.is_special_event:
165         reasons.append("special event")
166
167     if conditions.weather_condition in ['rain', 'snow']:
168         reasons.append("adverse weather")
169
170     if conditions.hour_of_day in [7, 8, 9, 17, 18, 19]:
171         reasons.append("peak hours")
172
173     if not reasons:
174         return "Normal market conditions"
175
176     return f"Surge pricing due to: {'', '.join(reasons)}"
177
178 # Example usage
179 engine = DynamicPricingEngine()
180
181 # Peak hour with high demand
182 conditions = MarketConditions(
183     location_id="downtown_01",
184     available_drivers=15,
185     pending_requests=50,
186     avg_wait_time_minutes=8.5,
187     hour_of_day=18,
188     day_of_week=4, # Friday
189     is_special_event=False,
190     weather_condition="clear",
191     historical_demand=45.0
192 )
193

```

```
194 decision = engine.calculate_price(conditions)
195 print(f"Base price: ${decision.base_price:.2f}")
196 print(f"Surge multiplier: {decision.surge_multiplier:.2f}x")
197 print(f"Final price: ${decision.final_price:.2f}")
198 print(f"Expected wait: {decision.expected_wait_time:.1f} minutes")
199 print(f"Reasoning: {decision.reasoning}")
```

Listing 24.9: Dynamic Pricing System

Key Design Decisions:

1. **Real-Time Computation:** Price calculated on-demand based on current conditions
2. **Demand Forecasting:** Use historical data + external signals (events, weather)
3. **Price Elasticity:** Model how price changes affect demand
4. **Fairness Constraints:** Cap maximum surge multiplier at 5x
5. **Transparency:** Provide clear reasoning for surge pricing
6. **A/B Testing:** Continuously test pricing strategies

Results:

- 23% revenue increase vs. fixed pricing
- 18% reduction in average wait times
- Driver earnings increased 15%
- Customer satisfaction maintained above 4.2/5.0

24.9 Chapter Summary

Building end-to-end ML systems requires careful consideration of:

- **System Design Methodology:** Follow structured process from problem framing to implementation
- **Architecture Decisions:** Use frameworks to evaluate trade-offs systematically
- **Integration Patterns:** Implement robust patterns for connecting components
- **Performance:** Design for scalability and low latency from the start
- **Evolution:** Plan for continuous improvement and adaptation
- **Maintenance:** Establish processes for ongoing system health

Key Takeaways:

Best Practice: *Start simple and iterate. Begin with minimal viable system, measure performance, and enhance based on real-world feedback. Premature optimization leads to complexity without proven value.*

Best Practice: *Design for observability from day one. You cannot improve what you cannot measure. Instrument every component with logging, metrics, and tracing.*

Best Practice: *Embrace trade-offs. Perfect systems don't exist. Make informed decisions about accuracy vs. latency, cost vs. performance, and complexity vs. maintainability.*

Best Practice: *Plan for failure. Systems will fail. Design with redundancy, graceful degradation, and automated recovery mechanisms.*

24.10 Exercises

Exercise 24.1. Exercise 1: E-Commerce Search and Ranking System

Design a complete end-to-end ML system for product search and ranking on an e-commerce platform.

Requirements:

- 1 million products, 5 million monthly users
- Support text search with semantic understanding
- Personalized ranking based on user history
- Real-time inventory and pricing updates
- <200ms p99 latency
- A/B testing capability

Deliverables:

1. Complete architecture diagram with all components
2. Data pipeline design (batch and real-time)
3. ML model architecture (retrieval + ranking)
4. Serving infrastructure design
5. Monitoring and evaluation strategy
6. Cost estimate and capacity planning
7. Discussion of key trade-offs and decisions

Consider:

- How will you handle cold-start for new users/products?
- How will you balance relevance, recency, and business metrics?
- What features will you use for ranking?

- How will you ensure diversity in results?

Exercise 24.2. Exercise 2: Credit Risk Assessment System

Design an ML system for assessing credit risk and determining loan approvals.

Requirements:

- Process 10,000 loan applications daily
- Predict default probability with 90%+ AUC
- Provide explanations for decisions (regulatory requirement)
- Fair lending compliance (no bias by protected attributes)
- Support manual override by loan officers
- Audit trail for all decisions

Deliverables:

1. End-to-end system architecture
2. Data sources and feature engineering strategy
3. Model selection and training approach
4. Explainability implementation
5. Fairness evaluation and mitigation plan
6. Deployment strategy and rollback plan
7. Monitoring dashboard specification

Consider:

- How will you ensure model fairness across demographics?
- What explainability method will you use?
- How will you handle model updates without disrupting operations?
- What metrics will you monitor in production?

Exercise 24.3. Exercise 3: Video Content Recommendation System

Design a recommendation system for a video streaming platform similar to Netflix or YouTube.

Requirements:

- 50 million users, 100,000 videos
- Personalized recommendations on homepage and autoplay
- Support cold-start for new users
- Optimize for watch time and user satisfaction

- Handle diverse content (movies, series, shorts)
- Support A/B testing and experimentation

Deliverables:

1. Multi-stage recommendation architecture (candidate generation + ranking)
2. Feature engineering for users and videos
3. Model architecture (collaborative filtering, deep learning, etc.)
4. Real-time serving infrastructure
5. Evaluation metrics (online and offline)
6. Experimentation framework
7. Scalability plan for 10x growth

Consider:

- How will you balance exploration vs. exploitation?
- How will you prevent filter bubbles?
- What's your strategy for cold-start users?
- How will you handle content freshness?

Exercise 24.4. Exercise 4: Predictive Maintenance System

Design an ML system for predictive maintenance of industrial equipment.

Requirements:

- Monitor 10,000 machines across 50 facilities
- Sensor data: temperature, vibration, pressure (sampled at 100Hz)
- Predict failures 7-14 days in advance
- Minimize false alarms (high cost of unnecessary maintenance)
- Provide actionable insights for maintenance teams
- Handle network connectivity issues at remote facilities

Deliverables:

1. IoT data collection and processing architecture
2. Feature engineering from time-series sensor data
3. Anomaly detection and failure prediction models
4. Edge vs. cloud processing strategy
5. Alert generation and prioritization system

6. Visualization dashboard for maintenance teams
7. ROI analysis (cost savings from preventing failures)

Consider:

- How will you handle class imbalance (rare failures)?
- What features will you extract from time-series data?
- How will you handle missing data from sensor failures?
- What's your strategy for different equipment types?

Chapter 25

Future Trends in ML Engineering

25.1 Learning Objectives

After completing this chapter, readers will be able to:

- Identify and understand emerging trends shaping ML engineering
- Evaluate technologies like edge ML, federated learning, and LLMOps
- Apply sustainability principles to ML system design
- Anticipate the evolution of MLOps platforms and tooling
- Plan career development aligned with future industry needs
- Analyze technology trends and their potential impact on organizations

25.2 Introduction

The field of ML engineering is evolving at an unprecedented pace. Technologies that were cutting-edge five years ago are now mainstream, and entirely new paradigms are emerging. This chapter explores the future of ML engineering, examining trends that will shape how we build, deploy, and maintain ML systems in the coming years.

Understanding these trends is crucial not just for staying relevant, but for making informed decisions about technology investments, skill development, and system architecture. While predicting the future is inherently uncertain, we can identify clear trajectories based on current research, industry adoption patterns, and fundamental constraints.

Key Forces Shaping the Future:

- **Scale:** Models and datasets continue to grow exponentially
- **Efficiency:** Need for lower latency, cost, and energy consumption
- **Privacy:** Regulatory requirements and user expectations
- **Democratization:** Making ML accessible to more developers
- **Automation:** Reducing manual intervention in ML workflows
- **Sustainability:** Environmental impact of ML systems

25.3 Emerging ML Engineering Technologies

25.3.1 AutoML and Neural Architecture Search

AutoML aims to automate the end-to-end process of applying machine learning, from data preprocessing to model selection and hyperparameter tuning. Neural Architecture Search (NAS) takes this further by automatically designing optimal neural network architectures.

Current State and Future Direction:

Table 25.1: AutoML Evolution Timeline		
Generation	Capabilities	Examples
Gen 1 (2015-2018)	Hyperparameter tuning	GridSearch, RandomSearch
Gen 2 (2018-2021)	Feature engineering, model selection	AutoGluon, H2O AutoML
Gen 3 (2021-2024)	End-to-end pipelines, NAS	Google AutoML, AutoPyTorch
Gen 4 (2024+)	Foundation model adaptation, multi-objective optimization	Emerging

Future Impact on ML Engineering:

- **Shift to Higher-Level Abstractions:** Engineers will focus more on problem formulation and less on model tuning
- **Specialized AutoML:** Domain-specific AutoML for computer vision, NLP, time-series, etc.
- **Multi-Objective Optimization:** Balance accuracy, latency, cost, and energy consumption
- **Interpretable AutoML:** Systems that explain their design decisions

```
#!/usr/bin/env python3
"""
Next-Generation AutoML with Multi-Objective Optimization
"""

from dataclasses import dataclass
from typing import List, Dict, Callable, Optional
from enum import Enum
import numpy as np

class OptimizationObjective(Enum):
    """Optimization objectives for AutoML."""
    ACCURACY = "accuracy"
    LATENCY = "latency"
    MODEL_SIZE = "model_size"
    ENERGY_CONSUMPTION = "energy"
```

```

17 FAIRNESS = "fairness"
18 INTERPRETABILITY = "interpretability"
19
20 @dataclass
21 class ModelCandidate:
22     """Candidate model from AutoML search."""
23     architecture: str
24     hyperparameters: Dict
25     metrics: Dict[OptimizationObjective, float]
26     training_cost: float # in dollars
27     inference_cost_per_1k: float # cost per 1K predictions
28     carbon_footprint: float # kg CO2
29
30 @dataclass
31 class AutoMLConfig:
32     """Configuration for AutoML system."""
33
34     # Search space
35     model_families: List[str] # ["xgboost", "lightgbm", "neural_net"]
36     max_training_time_hours: int
37     max_trials: int
38
39     # Optimization objectives with weights
40     objectives: Dict[OptimizationObjective, float]
41
42     # Constraints
43     max_model_size_mb: Optional[float] = None
44     max_latency_ms: Optional[float] = None
45     max_training_budget: Optional[float] = None
46     fairness_constraints: Optional[Dict] = None
47
48 class NextGenAutoML:
49     """
50     Next-generation AutoML system with multi-objective optimization,
51     sustainability awareness, and explainability.
52     """
53
54     def __init__(self, config: AutoMLConfig):
55         self.config = config
56         self.candidates: List[ModelCandidate] = []
57         self.pareto_front: List[ModelCandidate] = []
58
59     def search(self, X_train, y_train, X_val, y_val):
60         """
61         Execute AutoML search with multi-objective optimization.
62
63         Returns:
64             Pareto-optimal models balancing multiple objectives
65         """
66         # Stage 1: Diverse exploration
67         initial_candidates = self._explore_architecture_space(
68             X_train, y_train, X_val, y_val
69         )
70
71         # Stage 2: Multi-objective optimization
72         optimized_candidates = self._multi_objective_optimize(
73             initial_candidates, X_train, y_train, X_val, y_val
74         )

```

```

75     # Stage 3: Constraint satisfaction and filtering
76     feasible_candidates = self._apply_constraints(optimized_candidates)
77
78     # Stage 4: Pareto frontier extraction
79     self.pareto_front = self._compute_pareto_front(feasible_candidates)
80
81     return self.pareto_front
82
83
84 def _explore_architecture_space(self, X_train, y_train, X_val, y_val):
85     """Explore diverse architectures efficiently."""
86     candidates = []
87
88     # Example: Quick exploration of model families
89     for model_family in self.config.model_families:
90         # Generate diverse candidates
91         for i in range(10): # 10 candidates per family
92             candidate = self._generate_candidate(
93                 model_family,
94                 exploration_phase=True
95             )
96
97             # Train and evaluate
98             metrics = self._evaluate_candidate(
99                 candidate, X_train, y_train, X_val, y_val
100             )
101             candidate.metrics = metrics
102
103             candidates.append(candidate)
104
105     return candidates
106
107 def _multi_objective_optimize(self, candidates, X_train, y_train, X_val, y_val):
108     """Optimize top candidates for multiple objectives."""
109     # Select promising candidates
110     top_candidates = self._select_top_k(candidates, k=30)
111
112     optimized = []
113     for candidate in top_candidates:
114         # Fine-tune for specific objective trade-offs
115         for objective_mix in self._generate_objective_mixes():
116             optimized_candidate = self._optimize_for_objectives(
117                 candidate, objective_mix, X_train, y_train, X_val, y_val
118             )
119             optimized.append(optimized_candidate)
120
121     return optimized
122
123 def _apply_constraints(self, candidates):
124     """Filter candidates that violate constraints."""
125     feasible = []
126
127     for candidate in candidates:
128         # Check size constraint
129         if (self.config.max_model_size_mb and
130             candidate.metrics.get(OptimizationObjective.MODEL_SIZE, float('inf'))
131             > self.config.max_model_size_mb):
132             continue

```

```

133         # Check latency constraint
134         if (self.config.max_latency_ms and
135             candidate.metrics.get(OptimizationObjective.LATENCY, float('inf'))
136             > self.config.max_latency_ms):
137             continue
138
139         # Check training budget
140         if (self.config.max_training_budget and
141             candidate.training_cost > self.config.max_training_budget):
142             continue
143
144         # Check fairness constraints
145         if self.config.fairness_constraints:
146             if not self._satisfies_fairness(candidate):
147                 continue
148
149         feasible.append(candidate)
150
151     return feasible
152
153 def _compute_pareto_front(self, candidates):
154     """
155     Compute Pareto-optimal models.
156     A model is Pareto-optimal if no other model is better in all objectives.
157     """
158     pareto_front = []
159
160     for candidate in candidates:
161         is_dominated = False
162
163         for other in candidates:
164             if self._dominates(other, candidate):
165                 is_dominated = True
166                 break
167
168         if not is_dominated:
169             pareto_front.append(candidate)
170
171     return pareto_front
172
173 def _dominates(self, a: ModelCandidate, b: ModelCandidate) -> bool:
174     """Check if model a dominates model b in all objectives."""
175     better_in_at_least_one = False
176
177     for objective, weight in self.config.objectives.items():
178         if weight == 0:
179             continue
180
181         a_val = a.metrics.get(objective, 0)
182         b_val = b.metrics.get(objective, 0)
183
184         # Higher is better for accuracy, fairness, interpretability
185         # Lower is better for latency, model_size, energy
186         if objective in [OptimizationObjective.ACCURACY,
187                         OptimizationObjective.FAIRNESS,
188                         OptimizationObjective.INTERPRETABILITY]:
189             if a_val < b_val:

```

```

191         return False
192     if a_val > b_val:
193         better_in_at_least_one = True
194     else:
195         if a_val > b_val:
196             return False
197         if a_val < b_val:
198             better_in_at_least_one = True
199
200     return better_in_at_least_one
201
202 def recommend_model(
203     self,
204     use_case: str,
205     importance: Dict[OptimizationObjective, float]
206 ) -> ModelCandidate:
207     """
208     Recommend best model from Pareto front based on use case.
209
210     Args:
211         use_case: "production", "experimentation", "cost_sensitive", etc.
212         importance: Custom importance weights for objectives
213
214     Returns:
215         Recommended model
216     """
217     # Score each Pareto-optimal model
218     scores = []
219     for candidate in self.pareto_front:
220         score = self._score_candidate(candidate, importance)
221         scores.append((candidate, score))
222
223     # Return highest scoring model
224     best_candidate = max(scores, key=lambda x: x[1])[0]
225
226     return best_candidate
227
228 def _score_candidate(
229     self,
230     candidate: ModelCandidate,
231     importance: Dict[OptimizationObjective, float]
232 ) -> float:
233     """Score candidate based on weighted objectives."""
234     score = 0.0
235     total_weight = sum(importance.values())
236
237     for objective, weight in importance.items():
238         normalized_weight = weight / total_weight
239         metric_value = candidate.metrics.get(objective, 0)
240
241         # Normalize and weight
242         score += normalized_weight * metric_value
243
244     return score
245
246 def explain_recommendation(self, candidate: ModelCandidate) -> str:
247     """Generate human-readable explanation for model recommendation."""
248     explanation = f"Recommended Model: {candidate.architecture}\n\n"

```



```

249     explanation += "Performance Characteristics:\n"
250     for objective, value in candidate.metrics.items():
251         explanation += f"    - {objective.value}: {value:.3f}\n"
252
253     explanation += f"\nCost Analysis:\n"
254     explanation += f"    - Training cost: ${candidate.training_cost:.2f}\n"
255     explanation += f"    - Inference cost (per 1K): ${candidate.
256 inference_cost_per_1k:.4f}\n"
257     explanation += f"    - Carbon footprint: {candidate.carbon_footprint:.2f} kg CO2
\n"
258
259     explanation += f"\nTrade-offs:\n"
260     explanation += self._analyze_tradeoffs(candidate)
261
262     return explanation
263
264 def _analyze_tradeoffs(self, candidate: ModelCandidate) -> str:
265     """Analyze trade-offs made in model selection."""
266     tradeoffs = []
267
268     # Compare to other Pareto-optimal models
269     for other in self.pareto_front:
270         if other == candidate:
271             continue
272
273     # Find significant differences
274     for objective in self.config.objectives.keys():
275         candidate_val = candidate.metrics.get(objective, 0)
276         other_val = other.metrics.get(objective, 0)
277
278         diff_pct = abs(candidate_val - other_val) / max(other_val, 0.001) *
100
279
280         if diff_pct > 10: # Significant difference
281             if candidate_val > other_val:
282                 tradeoffs.append(
283                     f"    - {diff_pct:.1f}% better {objective.value} "
284                     f"than {other.architecture}"
285                 )
286             else:
287                 tradeoffs.append(
288                     f"    - {diff_pct:.1f}% worse {objective.value} "
289                     f"than {other.architecture}"
290                 )
291
292     return "\n".join(tradeoffs) if tradeoffs else "    - Balanced across all
objectives"
293
294 # Placeholder methods (would be implemented with actual ML training)
295 def _generate_candidate(self, model_family, exploration_phase):
296     """Generate candidate model."""
297     return ModelCandidate(
298         architecture=model_family,
299         hyperparameters={},
300         metrics={},
301         training_cost=0.0,
302         inference_cost_per_1k=0.0,

```

```

303         carbon_footprint=0.0
304     )
305
306     def _evaluate_candidate(self, candidate, X_train, y_train, X_val, y_val):
307         """Evaluate candidate on all objectives."""
308         return {
309             OptimizationObjective.ACCURACY: 0.9,
310             OptimizationObjective.LATENCY: 50.0,
311             OptimizationObjective.MODEL_SIZE: 100.0,
312             OptimizationObjective.ENERGY_CONSUMPTION: 0.5
313         }
314
315     def _select_top_k(self, candidates, k):
316         """Select top k candidates."""
317         return candidates[:k]
318
319     def _generate_objective_mixes(self):
320         """Generate different objective weight combinations."""
321         return [self.config.objectives]
322
323     def _optimize_for_objectives(self, candidate, objectives, X_train, y_train, X_val,
324                                 y_val):
325         """Optimize candidate for specific objectives."""
326         return candidate
327
328     def _satisfies_fairness(self, candidate):
329         """Check if candidate satisfies fairness constraints."""
330         return True
331
332     # Example usage
333     config = AutoMLConfig(
334         model_families=["xgboost", "lightgbm", "neural_net"],
335         max_training_time_hours=24,
336         max_trials=100,
337         objectives={
338             OptimizationObjective.ACCURACY: 0.5,
339             OptimizationObjective.LATENCY: 0.2,
340             OptimizationObjective.ENERGY_CONSUMPTION: 0.15,
341             OptimizationObjective.MODEL_SIZE: 0.15
342         },
343         max_latency_ms=100.0,
344         max_model_size_mb=500.0
345     )
346
347     automl = NextGenAutoML(config)
348     # pareto_models = automl.search(X_train, y_train, X_val, y_val)
349
350     # Get recommendation for production use case
351     # best_model = automl.recommend_model(
352     #     use_case="production",
353     #     importance={
354     #         OptimizationObjective.ACCURACY: 0.4,
355     #         OptimizationObjective.LATENCY: 0.4,
356     #         OptimizationObjective.ENERGY_CONSUMPTION: 0.2
357     #     }
358     # )

```

Listing 25.1: Next-Generation AutoML Framework

25.3.2 Continuous Learning Systems

Traditional ML systems use static models that degrade over time. Continuous learning (also called online learning or lifelong learning) enables models to adapt to new data without complete retraining.

Key Challenges and Solutions:

Table 25.2: Continuous Learning Challenges

Challenge	Traditional Approach	Future Solution
Catastrophic forgetting	Periodic full retraining	Elastic weight consolidation, replay buffers
Concept drift detection	Manual monitoring	Automated drift detection with adaptive windows
Model versioning	Discrete versions	Continuous version streams
Safety guarantees	Testing before deployment	Real-time safety checks with rollback

25.4 Edge ML and Federated Learning

25.4.1 Edge ML and TinyML

Edge ML moves computation from centralized cloud servers to edge devices (smartphones, IoT devices, embedded systems). TinyML takes this further, running ML models on microcontrollers with severe resource constraints.

Why Edge ML Matters:

- **Privacy:** Data stays on device
- **Latency:** No round-trip to cloud (<10ms inference)
- **Bandwidth:** Reduced data transmission costs
- **Offline Operation:** Works without internet connectivity
- **Scale:** Billions of edge devices deployed

Technical Challenges:

```
1#!/usr/bin/env python3
2"""
3Edge ML Optimization: Model Compression and Deployment
4"""
5
6from dataclasses import dataclass
7from typing import Dict, List, Optional
8from enum import Enum
```

```

9 import numpy as np
10
11 class DeviceType(Enum):
12     """Target device types for edge deployment."""
13     HIGH_END_MOBILE = "high_end_mobile" # iPhone, flagship Android
14     MID_RANGE_MOBILE = "mid_range_mobile" # Budget smartphones
15     IOT_DEVICE = "iot_device" # Raspberry Pi, smart cameras
16     MICROCONTROLLER = "microcontroller" # Arduino, ESP32
17     WEARABLE = "wearable" # Smartwatch, fitness tracker
18
19 @dataclass
20 class DeviceConstraints:
21     """Hardware constraints for edge device."""
22     ram_mb: int
23     storage_mb: int
24     compute_mflops: int # Million floating point ops per second
25     power_budget_mw: int # Milliwatts
26     battery_life_hours: Optional[float] = None
27
28 @dataclass
29 class ModelProfile:
30     """Profiling results for ML model."""
31     model_size_mb: float
32     inference_time_ms: float
33     memory_usage_mb: float
34     energy_per_inference_mj: float # Millijoules
35     accuracy: float
36
37 class EdgeMLOptimizer:
38     """
39     Optimizer for deploying ML models to edge devices.
40     Applies compression techniques and hardware-specific optimizations.
41     """
42
43     # Device constraints database
44     DEVICE_PROFILES = {
45         DeviceType.HIGH_END_MOBILE: DeviceConstraints(
46             ram_mb=8000,
47             storage_mb=128000,
48             compute_mflops=10000,
49             power_budget_mw=5000,
50             battery_life_hours=12.0
51         ),
52         DeviceType.MID_RANGE_MOBILE: DeviceConstraints(
53             ram_mb=4000,
54             storage_mb=64000,
55             compute_mflops=3000,
56             power_budget_mw=3000,
57             battery_life_hours=10.0
58         ),
59         DeviceType.IOT_DEVICE: DeviceConstraints(
60             ram_mb=512,
61             storage_mb=4000,
62             compute_mflops=500,
63             power_budget_mw=2000
64         ),
65         DeviceType.MICROCONTROLLER: DeviceConstraints(
66             ram_mb=32,

```

```

67         storage_mb=256,
68         compute_mflops=50,
69         power_budget_mw=100
70     ),
71     DeviceType.WEARABLE: DeviceConstraints(
72         ram_mb=256,
73         storage_mb=2000,
74         compute_mflops=200,
75         power_budget_mw=500,
76         battery_life_hours=24.0
77     )
78 }
79
80 def __init__(self, target_device: DeviceType):
81     self.target_device = target_device
82     self.constraints = self.DEVICE_PROFILES[target_device]
83
84 def optimize_for_device(self, model, target_accuracy: float = 0.95):
85     """
86     Optimize model for target device while maintaining accuracy.
87
88     Args:
89         model: Original model
90         target_accuracy: Minimum acceptable accuracy (as fraction of original)
91
92     Returns:
93         Optimized model and optimization report
94     """
95     optimization_pipeline = [
96         ('quantization', self._apply_quantization),
97         ('pruning', self._apply_pruning),
98         ('knowledge_distillation', self._apply_distillation),
99         ('layer_fusion', self._apply_layer_fusion),
100        ('hardware_optimization', self._apply_hardware_opts)
101    ]
102
103    optimized_model = model
104    original_profile = self._profile_model(model)
105    report = {
106        'original': original_profile,
107        'steps': []
108    }
109
110    for step_name, optimization_fn in optimization_pipeline:
111        # Apply optimization
112        candidate_model = optimization_fn(optimized_model)
113        profile = self._profile_model(candidate_model)
114
115        # Check if acceptable
116        accuracy_ratio = profile.accuracy / original_profile.accuracy
117
118        if accuracy_ratio >= target_accuracy:
119            # Check constraints
120            if self._meets_constraints(profile):
121                optimized_model = candidate_model
122                report['steps'].append({
123                    'name': step_name,
124                    'profile': profile,

```

```

125         'accepted': True
126     })
127     else:
128         report['steps'].append({
129             'name': step_name,
130             'profile': profile,
131             'accepted': False,
132             'reason': 'Constraints not met'
133         })
134     else:
135         report['steps'].append({
136             'name': step_name,
137             'profile': profile,
138             'accepted': False,
139             'reason': f'Accuracy drop too large: {accuracy_ratio:.2%}'
140         })
141
142     report['final'] = self._profile_model(optimized_model)
143
144     return optimized_model, report
145
146 def _apply_quantization(self, model):
147     """
148     Apply quantization to reduce model size and inference time.
149
150     Quantization converts weights from float32 to int8, reducing
151     model size by ~4x and improving inference speed.
152     """
153     # Placeholder - actual implementation would use TensorFlow Lite,
154     # PyTorch quantization, or ONNX quantization
155     print("Applying INT8 quantization...")
156     return model
157
158 def _apply_pruning(self, model):
159     """
160     Remove unimportant weights to reduce model size.
161
162     Structured pruning removes entire channels/filters.
163     Unstructured pruning removes individual weights.
164     """
165     print("Applying structured pruning (50% sparsity)...")
166     return model
167
168 def _apply_distillation(self, model):
169     """
170     Knowledge distillation: Train smaller student model from larger teacher.
171
172     Student learns to mimic teacher's outputs, achieving similar
173     accuracy with much smaller size.
174     """
175     print("Training distilled model (0.25x parameters)...")
176     return model
177
178 def _apply_layer_fusion(self, model):
179     """
180     Fuse consecutive layers to reduce memory access and latency.
181
182     Example: Conv + BatchNorm + ReLU can be fused into single operation.

```

```

183     """
184     print("Fusing layers...")
185     return model
186
187 def _apply_hardware_opts(self, model):
188     """
189     Apply hardware-specific optimizations.
190
191     - Use CPU/GPU/NPU-specific kernels
192     - Optimize memory layout
193     - Enable hardware accelerators
194     """
195     print(f"Optimizing for {self.target_device.value}...")
196     return model
197
198 def _profile_model(self, model) -> ModelProfile:
199     """Profile model performance on target device."""
200     # Placeholder - actual implementation would benchmark on device
201     return ModelProfile(
202         model_size_mb=5.0,
203         inference_time_ms=50.0,
204         memory_usage_mb=100.0,
205         energy_per_inference_mj=10.0,
206         accuracy=0.92
207     )
208
209 def _meets_constraints(self, profile: ModelProfile) -> bool:
210     """Check if model profile meets device constraints."""
211     if profile.model_size_mb > self.constraints.storage_mb:
212         return False
213
214     if profile.memory_usage_mb > self.constraints.ram_mb:
215         return False
216
217     # Check if inference can complete within power budget
218     max_inference_time_ms = 1000 # 1 second max
219     if profile.inference_time_ms > max_inference_time_ms:
220         return False
221
222     # Check battery life impact
223     if self.constraints.battery_life_hours:
224         inferences_per_hour = 3600000 / profile.inference_time_ms
225         energy_per_hour = inferences_per_hour * profile.energy_per_inference_mj
226
227         # Should use less than 10% of battery per hour
228         if energy_per_hour > (self.constraints.power_budget_mw * 3600 * 0.1):
229             return False
230
231     return True
232
233 def generate_deployment_report(self, model, report: Dict) -> str:
234     """Generate human-readable deployment report."""
235     original = report['original']
236     final = report['final']
237
238     output = f"Edge ML Optimization Report\n"
239     output += f"Target Device: {self.target_device.value}\n\n"
240

```

```

241     output += f"Improvements:\n"
242     output += f"   Model Size: {original.model_size_mb:.2f}MB → {final.
model_size_mb:.2f}MB "
243     output += f"({(1 - final.model_size_mb/original.model_size_mb)*100:.1f}%
reduction)\n"
244
245     output += f"   Inference Time: {original.inference_time_ms:.2f}ms → {final.
inference_time_ms:.2f}ms "
246     output += f"({(1 - final.inference_time_ms/original.inference_time_ms)*100:.1f
}% faster)\n"
247
248     output += f"   Memory Usage: {original.memory_usage_mb:.2f}MB → {final.
memory_usage_mb:.2f}MB "
249     output += f"({(1 - final.memory_usage_mb/original.memory_usage_mb)*100:.1f}%
reduction)\n"
250
251     output += f"   Accuracy: {original.accuracy:.4f} → {final.accuracy:.4f} "
252     output += f"({(final.accuracy/original.accuracy - 1)*100:.2f}% change)\n\n"
253
254     output += f"Applied Optimizations:\n"
255     for step in report['steps']:
256         status = "✓" if step['accepted'] else "x"
257         output += f"   {status} {step['name']}\n"
258         if not step['accepted']:
259             output += f"       Reason: {step.get('reason', 'Unknown')}\n"
260
261     return output
262
263 # Example usage
264 optimizer = EdgeMLOptimizer(DeviceType.MICROCONTROLLER)
265
266 # Optimize model for microcontroller deployment
267 # optimized_model, report = optimizer.optimize_for_device(
268 #     original_model,
269 #     target_accuracy=0.95
270 # )
271
272 # Generate deployment report
273 # print(optimizer.generate_deployment_report(original_model, report))

```

Listing 25.2: Edge ML Optimization Framework

25.4.2 Federated Learning

Federated Learning (FL) enables training ML models across decentralized devices without transferring raw data to a central server. Each device trains locally and only shares model updates.

Key Benefits:

- **Privacy Preservation:** Raw data never leaves device
- **Reduced Bandwidth:** Only model updates transmitted
- **Personalization:** Models adapt to local data distributions
- **Compliance:** Meets GDPR and other privacy regulations

Table 25.3: Federated Learning Challenges

Challenge	Issue	Emerging Solution
Non-IID data	Devices have different data distributions	Personalized federated learning
Communication efficiency	Frequent updates expensive	Gradient compression, sparse updates
Stragglers	Slow devices delay training	Asynchronous FL
Privacy attacks	Model updates leak information	Differential privacy, secure aggregation
Model poisoning	Malicious clients corrupt model	Byzantine-robust aggregation

**Challenges and Future Solutions:
Future Applications:**

- **Healthcare:** Train diagnostic models across hospitals without sharing patient data
- **Finance:** Fraud detection across banks while maintaining confidentiality
- **Mobile Keyboards:** Improve autocorrect without uploading typing data
- **Autonomous Vehicles:** Learn from fleet without centralizing driving data

25.5 Large Language Model Operations (LLMOps)

The rise of large language models (LLMs) like GPT, Claude, and Llama has created new challenges for ML engineering. LLMOps extends traditional MLOps to handle the unique requirements of LLMs.

25.5.1 LLM-Specific Challenges

1. Scale and Cost

Table 25.4: LLM Deployment Costs

Model Size	Parameters	GPU Memory	Cost per 1M Tokens
Small (7B)	7 billion	14-28 GB	\$0.10-0.50
Medium (13-70B)	13-70 billion	26-140 GB	\$0.50-2.00
Large (175B+)	175+ billion	350+ GB	\$2.00-10.00

2. Prompt Engineering and Management

Unlike traditional ML where features are explicit, LLMs require careful prompt design. This introduces new engineering challenges:

```

1  #!/usr/bin/env python3
2  """
3  LLM Prompt Management and Version Control System
4  """
5
6  from dataclasses import dataclass, field
7  from typing import Dict, List, Optional
8  from datetime import datetime
9  from enum import Enum
10 import hashlib
11 import json
12
13 class PromptVersion(Enum):
14     """Prompt versioning strategy."""
15     SEMANTIC = "semantic" # Major.Minor.Patch
16     TIMESTAMP = "timestamp"
17     HASH = "hash"
18
19 @dataclass
20 class PromptTemplate:
21     """Versioned prompt template."""
22
23     # Identification
24     id: str
25     name: str
26     description: str
27     version: str
28     created_at: datetime
29
30     # Template
31     system_prompt: str
32     user_prompt_template: str
33     few_shot_examples: List[Dict[str, str]] = field(default_factory=list)
34
35     # Configuration
36     temperature: float = 0.7
37     max_tokens: int = 1000
38     top_p: float = 0.9
39
40     # Metadata
41     model: str = "gpt-4"
42     use_case: str = ""
43     tags: List[str] = field(default_factory=list)
44
45     # Performance tracking
46     avg_latency_ms: Optional[float] = None
47     avg_cost_per_request: Optional[float] = None
48     success_rate: Optional[float] = None
49
50 @dataclass
51 class PromptExperiment:
52     """A/B test experiment for prompts."""
53
54     experiment_id: str
55     name: str
56     variants: Dict[str, PromptTemplate] # variant_name -> prompt
57     traffic_split: Dict[str, float] # variant_name -> percentage

```

```

58
59 # Metrics
60 metrics: Dict[str, Dict[str, float]] = field(default_factory=dict)
61
62 # Status
63 status: str = "running" # "running", "completed", "paused"
64 start_date: datetime = field(default_factory=datetime.now)
65 end_date: Optional[datetime] = None
66
67 class PromptRegistry:
68     """
69     Central registry for prompt templates with versioning,
70     experimentation, and performance tracking.
71     """
72
73     def __init__(self):
74         self.prompts: Dict[str, List[PromptTemplate]] = {}
75         self.experiments: Dict[str, PromptExperiment] = {}
76
77     def register_prompt(
78         self,
79         prompt: PromptTemplate,
80         version_strategy: PromptVersion = PromptVersion.SEMANTIC
81     ) -> str:
82         """
83         Register new prompt version.
84
85         Args:
86             prompt: Prompt template to register
87             version_strategy: Versioning strategy to use
88
89         Returns:
90             Assigned version string
91         """
92         # Generate version
93         if version_strategy == PromptVersion.HASH:
94             prompt.version = self._generate_hash(prompt)
95         elif version_strategy == PromptVersion.TIMESTAMP:
96             prompt.version = datetime.now().strftime("%Y%m%d%H%M%S")
97         # SEMANTIC requires manual version assignment
98
99         # Store prompt
100         if prompt.name not in self.prompts:
101             self.prompts[prompt.name] = []
102
103         self.prompts[prompt.name].append(prompt)
104
105         return prompt.version
106
107     def get_prompt(
108         self,
109         name: str,
110         version: Optional[str] = None
111     ) -> Optional[PromptTemplate]:
112         """
113         Retrieve prompt template.
114
115         Args:

```

```

116         name: Prompt name
117         version: Specific version (None for latest)
118
119     Returns:
120         Prompt template or None
121     """
122     if name not in self.prompts:
123         return None
124
125     versions = self.prompts[name]
126
127     if version is None:
128         # Return latest version
129         return versions[-1]
130
131     # Find specific version
132     for prompt in versions:
133         if prompt.version == version:
134             return prompt
135
136     return None
137
138     def create_experiment(
139         self,
140         name: str,
141         variants: Dict[str, PromptTemplate],
142         traffic_split: Optional[Dict[str, float]] = None
143     ) -> str:
144         """
145         Create A/B test experiment for prompt variants.
146
147         Args:
148             name: Experiment name
149             variants: Dict of variant_name -> prompt_template
150             traffic_split: Traffic allocation (defaults to equal split)
151
152         Returns:
153             Experiment ID
154         """
155         experiment_id = f"exp_{len(self.experiments)}_{int(datetime.now().timestamp())}"
156
157         # Default to equal traffic split
158         if traffic_split is None:
159             split_pct = 1.0 / len(variants)
160             traffic_split = {name: split_pct for name in variants.keys()}
161
162         experiment = PromptExperiment(
163             experiment_id=experiment_id,
164             name=name,
165             variants=variants,
166             traffic_split=traffic_split
167         )
168
169         self.experiments[experiment_id] = experiment
170
171         return experiment_id
172

```

```

173 def select_variant(self, experiment_id: str, user_id: str) -> PromptTemplate:
174     """
175     Select prompt variant for user (consistent hashing).
176
177     Args:
178         experiment_id: Experiment ID
179         user_id: User identifier
180
181     Returns:
182         Selected prompt variant
183     """
184     experiment = self.experiments[experiment_id]
185
186     # Consistent hashing for stable assignment
187     hash_val = int(hashlib.md5(f"{user_id}_{experiment_id}".encode()).hexdigest(),
16)
188     bucket = (hash_val % 100) / 100.0
189
190     # Select variant based on traffic split
191     cumulative = 0.0
192     for variant_name, percentage in experiment.traffic_split.items():
193         cumulative += percentage
194         if bucket < cumulative:
195             return experiment.variants[variant_name]
196
197     # Fallback to first variant
198     return list(experiment.variants.values())[0]
199
200 def record_metrics(
201     self,
202     experiment_id: str,
203     variant_name: str,
204     metrics: Dict[str, float]
205 ):
206     """Record metrics for experiment variant."""
207     experiment = self.experiments[experiment_id]
208
209     if variant_name not in experiment.metrics:
210         experiment.metrics[variant_name] = {}
211
212     # Update metrics (running average)
213     for metric_name, value in metrics.items():
214         if metric_name not in experiment.metrics[variant_name]:
215             experiment.metrics[variant_name][metric_name] = value
216         else:
217             # Simple running average (could be more sophisticated)
218             current = experiment.metrics[variant_name][metric_name]
219             experiment.metrics[variant_name][metric_name] = (current + value) / 2
220
221 def get_experiment_results(self, experiment_id: str) -> Dict:
222     """Get experiment results with statistical analysis."""
223     experiment = self.experiments[experiment_id]
224
225     results = {
226         'experiment_id': experiment_id,
227         'name': experiment.name,
228         'status': experiment.status,
229         'variants': {}

```

```

230     }
231
232     for variant_name, prompt in experiment.variants.items():
233         results['variants'][variant_name] = {
234             'prompt_version': prompt.version,
235             'traffic_split': experiment.traffic_split[variant_name],
236             'metrics': experiment.metrics.get(variant_name, {})
237         }
238
239     # Find winning variant
240     if experiment.metrics:
241         # Use primary metric (e.g., success_rate) to determine winner
242         winner = max(
243             experiment.metrics.items(),
244             key=lambda x: x[1].get('success_rate', 0)
245         )
246         results['winner'] = winner[0]
247
248     return results
249
250     def promote_to_production(self, experiment_id: str, variant_name: str):
251         """Promote winning variant to production."""
252         experiment = self.experiments[experiment_id]
253         winning_prompt = experiment.variants[variant_name]
254
255         # Update prompt version in registry
256         winning_prompt.version = f"prod_{datetime.now().strftime('%Y%m%d')}"
257         self.register_prompt(winning_prompt)
258
259         # Mark experiment as completed
260         experiment.status = "completed"
261         experiment.end_date = datetime.now()
262
263     def _generate_hash(self, prompt: PromptTemplate) -> str:
264         """Generate content hash for prompt."""
265         content = f"{prompt.system_prompt}_{prompt.user_prompt_template}"
266         return hashlib.sha256(content.encode()).hexdigest()[:8]
267
268     # Example usage
269     registry = PromptRegistry()
270
271     # Register baseline prompt
272     baseline_prompt = PromptTemplate(
273         id="prompt_001",
274         name="customer_support_classifier",
275         description="Classify customer support requests",
276         version="1.0.0",
277         created_at=datetime.now(),
278         system_prompt="You are a customer support classification system. Classify each request into one of: technical, billing, general.",
279         user_prompt_template="Customer request: {request}\n\nCategory:",
280         temperature=0.3,
281         max_tokens=50,
282         use_case="customer_support"
283     )
284
285     registry.register_prompt(baseline_prompt)
286
287

```

```

288 # Create variant for A/B test
289 variant_prompt = PromptTemplate(
290     id="prompt_002",
291     name="customer_support_classifier",
292     description="Enhanced classifier with examples",
293     version="1.1.0",
294     created_at=datetime.now(),
295     system_prompt="""You are an expert customer support classifier.
296     Analyze the request carefully and categorize appropriately.""",
297     user_prompt_template="Customer request: {request}\n\nCategory:",
298     few_shot_examples=[
299         {"request": "I can't log in", "category": "technical"},
300         {"request": "When will I be charged?", "category": "billing"}
301     ],
302     temperature=0.3,
303     max_tokens=50,
304     use_case="customer_support"
305 )
306
307 # Create experiment
308 exp_id = registry.create_experiment(
309     name="Classification Prompt Improvement",
310     variants={
311         "baseline": baseline_prompt,
312         "enhanced": variant_prompt
313     },
314     traffic_split={
315         "baseline": 0.5,
316         "enhanced": 0.5
317     }
318 )
319
320 # Select variant for user
321 # selected = registry.select_variant(exp_id, user_id="user_12345")
322
323 # Record metrics
324 # registry.record_metrics(exp_id, "enhanced", {
325 #     "success_rate": 0.95,
326 #     "avg_latency_ms": 250,
327 #     "avg_cost": 0.001
328 # })

```

Listing 25.3: LLM Prompt Management System

3. Context Management and RAG

Retrieval-Augmented Generation (RAG) has become essential for LLM applications. Future trends include:

- **Hybrid Search:** Combining vector, keyword, and graph-based retrieval
- **Adaptive Retrieval:** Dynamically determining when to retrieve
- **Multi-Modal RAG:** Retrieving text, images, and structured data
- **Continuous Knowledge Updates:** Real-time index updates without downtime

25.5.2 LLM Efficiency and Optimization

Emerging Techniques:

Table 25.5: LLM Optimization Techniques (2024+)				
Technique	Description			Speedup/Savings
Speculative Decoding	Parallel generation with verification			2-3x faster
Flash Attention	Optimized attention computation			3-5x faster, 50% memory
LoRA/QLoRA	Low-rank adaptation for fine-tuning			10x fewer parameters
Mixture of Experts	Activate subset of parameters			5x more efficient
KV Cache Optimization	Compress attention cache			50% memory reduction

25.6 Green ML and Sustainability

The environmental impact of ML is significant and growing. Training GPT-3 emitted an estimated 500 tons of CO2. The ML community is increasingly focused on sustainability.

25.6.1 Carbon Footprint of ML

Sources of Emissions:

- **Training:** Large models require weeks on hundreds of GPUs
- **Inference:** Billions of predictions per day across millions of models
- **Data Centers:** Cooling, power infrastructure
- **Hardware Manufacturing:** Embodied carbon in GPUs/chips

```
1  #!/usr/bin/env python3
2  """
3  Carbon Footprint Calculator for ML Training and Inference
4  """
5
6  from dataclasses import dataclass
7  from typing import Dict, Optional
8  from enum import Enum
9
10 class HardwareType(Enum):
11     """Types of hardware for ML."""
12     CPU = "cpu"
13     GPU_V100 = "gpu_v100"
```



```

14 GPU_A100 = "gpu_a100"
15 GPU_H100 = "gpu_h100"
16 TPU_V4 = "tpu_v4"
17
18 class CloudRegion(Enum):
19     """Cloud regions with different carbon intensities."""
20     US_WEST = "us_west" # California (low carbon, lots of renewables)
21     US_EAST = "us_east" # Virginia (moderate carbon)
22     EU_WEST = "eu_west" # Ireland (moderate carbon)
23     EU_NORTH = "eu_north" # Finland (low carbon, hydroelectric)
24     ASIA_EAST = "asia_east" # Taiwan (moderate-high carbon)
25
26 @dataclass
27 class HardwareProfile:
28     """Power consumption profile for hardware."""
29     tdp_watts: float # Thermal Design Power
30     idle_watts: float
31     memory_gb: float
32     embodied_carbon_kg: float # kg CO2 for manufacturing
33
34 @dataclass
35 class CarbonFootprint:
36     """Carbon footprint calculation results."""
37     training_kg_co2: float
38     inference_kg_co2: float
39     total_kg_co2: float
40     equivalent_km_driven: float # Car km equivalent
41     equivalent_trees_year: float # Trees needed for 1 year
42
43 class CarbonCalculator:
44     """
45     Calculate carbon footprint of ML training and inference.
46     Based on "Energy and Policy Considerations for Deep Learning in NLP"
47     and ML CO2 Impact calculator.
48     """
49
50     # Hardware profiles (TDP watts, idle watts, memory GB, embodied carbon kg)
51     HARDWARE_PROFILES = {
52         HardwareType.CPU: HardwareProfile(150, 50, 256, 100),
53         HardwareType.GPU_V100: HardwareProfile(300, 50, 32, 150),
54         HardwareType.GPU_A100: HardwareProfile(400, 50, 80, 200),
55         HardwareType.GPU_H100: HardwareProfile(700, 80, 80, 250),
56         HardwareType.TPU_V4: HardwareProfile(450, 60, 32, 180),
57     }
58
59     # Carbon intensity by region (grams CO2 per kWh)
60     CARBON_INTENSITY = {
61         CloudRegion.US_WEST: 200, # California has high renewable %
62         CloudRegion.US_EAST: 400, # Virginia coal-heavy
63         CloudRegion.EU_WEST: 300,
64         CloudRegion.EU_NORTH: 50, # Finland hydroelectric
65         CloudRegion.ASIA_EAST: 500,
66     }
67
68     # Power Usage Effectiveness (PUE) for data centers
69     # Accounts for cooling, power distribution overhead
70     PUE = 1.2 # Modern efficient data centers
71

```

```

72     def calculate_training_footprint(
73         self,
74         hardware: HardwareType,
75         num_devices: int,
76         training_hours: float,
77         region: CloudRegion,
78         utilization: float = 0.9
79     ) -> float:
80         """
81         Calculate carbon footprint of model training.
82
83         Args:
84             hardware: Hardware type
85             num_devices: Number of GPUs/TPUs
86             training_hours: Training time in hours
87             region: Cloud region
88             utilization: Average hardware utilization (0-1)
89
90         Returns:
91             Carbon footprint in kg CO2
92         """
93         profile = self.HARDWARE_PROFILES[hardware]
94
95         # Calculate power consumption
96         avg_power_per_device = (
97             profile.idle_watts +
98             (profile.tdp_watts - profile.idle_watts) * utilization
99         )
100
101         total_power_watts = avg_power_per_device * num_devices
102
103         # Account for PUE (cooling, infrastructure)
104         effective_power_watts = total_power_watts * self.PUE
105
106         # Calculate energy consumption (kWh)
107         energy_kwh = (effective_power_watts * training_hours) / 1000
108
109         # Calculate carbon emissions
110         carbon_intensity = self.CARBON_INTENSITY[region]
111         carbon_kg = (energy_kwh * carbon_intensity) / 1000
112
113         # Add embodied carbon (amortized over 3 years)
114         embodied_carbon_per_hour = (
115             profile.embodied_carbon_kg * num_devices
116         ) / (3 * 365 * 24) # 3 years of operation
117
118         embodied_carbon_kg = embodied_carbon_per_hour * training_hours
119
120         total_carbon_kg = carbon_kg + embodied_carbon_kg
121
122         return total_carbon_kg
123
124     def calculate_inference_footprint(
125         self,
126         hardware: HardwareType,
127         num_devices: int,
128         requests_per_day: int,
129         ms_per_request: float,

```

```

130     days: int,
131     region: CloudRegion
132 ) -> float:
133     """
134     Calculate carbon footprint of model inference.
135
136     Args:
137         hardware: Hardware type
138         num_devices: Number of GPUs/TPUs
139         requests_per_day: Inference requests per day
140         ms_per_request: Latency per request
141         days: Number of days
142         region: Cloud region
143
144     Returns:
145         Carbon footprint in kg CO2
146     """
147     profile = self.HARDWARE_PROFILES[hardware]
148
149     # Calculate active hours per day
150     total_ms_per_day = requests_per_day * ms_per_request
151     active_hours_per_day = total_ms_per_day / (1000 * 3600)
152
153     # Idle hours
154     idle_hours_per_day = 24 - active_hours_per_day
155
156     # Power consumption
157     active_power = profile.tdp_watts * num_devices * self.PUE
158     idle_power = profile.idle_watts * num_devices * self.PUE
159
160     # Energy per day (kWh)
161     energy_per_day = (
162         (active_power * active_hours_per_day) +
163         (idle_power * idle_hours_per_day)
164     ) / 1000
165
166     # Total energy
167     total_energy_kwh = energy_per_day * days
168
169     # Carbon emissions
170     carbon_intensity = self.CARBON_INTENSITY[region]
171     carbon_kg = (total_energy_kwh * carbon_intensity) / 1000
172
173     return carbon_kg
174
175 def generate_report(
176     self,
177     training_kg: float,
178     inference_kg: float
179 ) -> CarbonFootprint:
180     """
181     Generate comprehensive carbon footprint report.
182
183     Args:
184         training_kg: Training carbon footprint
185         inference_kg: Inference carbon footprint
186
187     Returns:

```

```

188         CarbonFootprint report
189         """
190         total_kg = training_kg + inference_kg
191
192         # Conversions for context
193         # Average car: 120g CO2 per km
194         km_driven = (total_kg * 1000) / 120
195
196         # Average tree absorbs 21 kg CO2 per year
197         trees_year = total_kg / 21
198
199         return CarbonFootprint(
200             training_kg_co2=training_kg,
201             inference_kg_co2=inference_kg,
202             total_kg_co2=total_kg,
203             equivalent_km_driven=km_driven,
204             equivalent_trees_year=trees_year
205         )
206
207     def recommend_optimizations(
208         self,
209         footprint: CarbonFootprint,
210         current_region: CloudRegion
211     ) -> List[str]:
212         """Recommend optimizations to reduce carbon footprint."""
213         recommendations = []
214
215         # Check if better region available
216         current_intensity = self.CARBON_INTENSITY[current_region]
217         best_region = min(
218             self.CARBON_INTENSITY.items(),
219             key=lambda x: x[1]
220         )
221
222         if best_region[1] < current_intensity:
223             potential_savings = (
224                 (current_intensity - best_region[1]) / current_intensity * 100
225             )
226             recommendations.append(
227                 f"Switch to {best_region[0].value} region: "
228                 f"{potential_savings:.1f}% carbon reduction"
229             )
230
231         # Training time optimization
232         if footprint.training_kg_co2 > 100: # > 100 kg CO2
233             recommendations.append(
234                 "Consider model compression techniques: "
235                 "pruning, distillation, or quantization-aware training"
236             )
237             recommendations.append(
238                 "Use smaller model for hyperparameter search, "
239                 "full model for final training only"
240             )
241
242         # Inference optimization
243         if footprint.inference_kg_co2 > footprint.training_kg_co2:
244             recommendations.append(
245                 "Inference is primary carbon source - focus on: "

```

```

246         "model quantization, caching, batch processing"
247     )
248
249     # Scheduling
250     recommendations.append(
251         "Schedule training during off-peak hours when "
252         "renewable energy % is higher"
253     )
254
255     return recommendations
256
257 # Example usage
258 calculator = CarbonCalculator()
259
260 # Calculate training footprint
261 training_carbon = calculator.calculate_training_footprint(
262     hardware=HardwareType.GPU_A100,
263     num_devices=8,
264     training_hours=72, # 3 days
265     region=CloudRegion.US_WEST,
266     utilization=0.85
267 )
268
269 print(f"Training carbon footprint: {training_carbon:.2f} kg CO2")
270
271 # Calculate inference footprint
272 inference_carbon = calculator.calculate_inference_footprint(
273     hardware=HardwareType.GPU_A100,
274     num_devices=4,
275     requests_per_day=1_000_000,
276     ms_per_request=50,
277     days=365,
278     region=CloudRegion.US_WEST
279 )
280
281 print(f"Inference carbon footprint (1 year): {inference_carbon:.2f} kg CO2")
282
283 # Generate report
284 report = calculator.generate_report(training_carbon, inference_carbon)
285 print(f"\nTotal carbon footprint: {report.total_kg_co2:.2f} kg CO2")
286 print(f"Equivalent to driving: {report.equivalent_km_driven:.0f} km")
287 print(f"Trees needed for 1 year: {report.equivalent_trees_year:.1f}")
288
289 # Get recommendations
290 recommendations = calculator.recommend_optimizations(
291     report,
292     CloudRegion.US_WEST
293 )
294 print("\nRecommendations:")
295 for i, rec in enumerate(recommendations, 1):
296     print(f"{i}. {rec}")

```

Listing 25.4: ML Carbon Footprint Calculator

25.6.2 Sustainable ML Practices

Design Principles:

1. **Model Efficiency First:** Start with efficient architectures
2. **Carbon-Aware Scheduling:** Train when renewable energy
3. **Green Regions:** Use data centers with low carbon intensity
4. **Model Reuse:** Transfer learning instead of training from scratch
5. **Right-Sizing:** Don't over-provision resources
6. **Inference Optimization:** Quantization, pruning, caching

Carbon Budget Approach:

Organizations are starting to set carbon budgets for ML teams, similar to cost budgets. This incentivizes efficient practices:

- Track carbon footprint in CI/CD pipelines
- Set carbon limits for experiments
- Report carbon metrics alongside model metrics
- Incentivize efficient model architectures

25.7 Next-Generation MLOps Platforms

25.7.1 Unified ML Platforms

The future of MLOps involves integrated platforms that handle the entire ML lifecycle:

Key Capabilities:

Table 25.6: Next-Gen MLOps Platform Features

Capability	Description	Status (2025)
Unified API	Single interface for all ML operations	Emerging
Multi-cloud	Deploy across AWS, GCP, Azure seamlessly	Mature
AutoML integration	Automated model selection and tuning	Mature
LLMOps support	Native support for LLM workflows	Emerging
Federated learning	Built-in federated training	Early stage
Edge deployment	One-click edge model deployment	Emerging
Sustainability	Carbon tracking and optimization	Early stage

25.7.2 Declarative ML Systems

The future trend is toward declarative ML systems where you specify what you want, not how to achieve it:

```
1  # Future: Declarative ML system configuration
2  # Specify objectives, constraints, and let platform optimize
3
4  name: fraud_detection_system
5  version: 2.0.0
6
7  objectives:
8    primary: accuracy
9    secondary:
10     - latency
11     - cost
12     - fairness
13
14  constraints:
15    latency_p99_ms: 100
16    monthly_budget_usd: 10000
17    fairness_metrics:
18      demographic_parity: 0.05
19      equal_opportunity: 0.05
20
21  data:
22    training:
23      source: s3://ml-data/fraud/train/
24      size: 10TB
25      features:
26        auto_generate: true
27        include:
28          - transaction_amount
29          - merchant_category
30          - user_history
31
32    validation:
33      source: s3://ml-data/fraud/val/
34      size: 1TB
35
36    streaming:
37      source: kafka://events/transactions
38      schema: schemas/transaction.avro
39
40  model:
41    type: auto # Platform selects best architecture
42    search_space:
43      - gradient_boosting
44      - neural_network
45      - ensemble
46
47    optimization:
48      metric: f1_score
49      direction: maximize
50      trials: 100
51
52    interpretability: required # Generate SHAP values
53
```

```
54 serving:
55   deployment:
56     type: blue_green
57     canary_percentage: 10
58
59   scaling:
60     min_instances: 2
61     max_instances: 50
62     target_utilization: 0.7
63
64   regions:
65     - us-west-2 # Primary
66     - eu-west-1 # Secondary
67
68   cache:
69     enabled: true
70     ttl: 300
71
72 monitoring:
73   metrics:
74     - accuracy
75     - latency_p99
76     - cost_per_prediction
77     - fairness_metrics
78
79   alerts:
80     - metric: accuracy
81       operator: less_than
82       threshold: 0.90
83       action: alert_and_rollback
84
85     - metric: latency_p99
86       operator: greater_than
87       threshold: 100
88       action: scale_up
89
90   drift_detection:
91     enabled: true
92     sensitivity: medium
93     action: retrain
94
95 governance:
96   approval_required: true
97   audit_logging: enabled
98   data_retention_days: 90
99   compliance:
100     - GDPR
101     - CCPA
102
103 sustainability:
104   carbon_budget_kg_co2: 500
105   prefer_renewable_energy: true
106   optimize_for_efficiency: true
```

Listing 25.5: Declarative ML System Specification

25.8 Skill Development Recommendations

25.8.1 Core Skills for Future ML Engineers

Technical Skills Matrix:

Table 25.7: ML Engineering Skills Roadmap (2025-2030)		
Skill Category	Current Priority	Future Priority
Deep Learning	High	High (foundation models focus)
Distributed Systems	High	Very High (scale increases)
LLM Operations	Medium	Very High
Edge ML	Low-Medium	High
Federated Learning	Low	Medium-High
Green ML	Low	Medium-High
Multi-modal AI	Medium	High
ML Security	Medium	High

Learning Path:

1. Foundation (6-12 months):
- ML fundamentals (supervised, unsupervised, reinforcement learning)
 - Python, PyTorch/TensorFlow
 - Software engineering best practices
 - Cloud platforms (AWS/GCP/Azure)
2. MLOps Specialization (6-12 months):
- CI/CD for ML
 - Model serving and deployment
 - Monitoring and observability
 - Infrastructure as code
3. Advanced Topics (12+ months):
- LLM fine-tuning and deployment
 - Edge ML and model optimization
 - Federated learning
 - ML system design

25.8.2 Soft Skills and Mindset

Critical Non-Technical Skills:

- **Business Acumen:** Translate business problems to ML formulations
- **Communication:** Explain technical concepts to non-technical stakeholders
- **Experimentation Mindset:** Embrace failure as learning
- **Ethical Awareness:** Consider societal impact of ML systems
- **Continuous Learning:** Field evolves rapidly, stay current
- **Cross-Functional Collaboration:** Work with product, design, legal teams

25.8.3 Certifications and Resources

Relevant Certifications (2025):

- **Cloud Certifications:** AWS ML Specialty, Google Professional ML Engineer
- **MLOps:** Coursera MLOps Specialization, DataTalks.Club
- **LLM Operations:** OpenAI Fine-tuning, Hugging Face Certification
- **Sustainability:** Green Software Foundation Practitioner

Recommended Resources:

- **Books:** "Designing Data-Intensive Applications", "Building Machine Learning Powered Applications"
- **Courses:** Full Stack Deep Learning, Made With ML
- **Communities:** MLOps Community, Locally Optimistic, Papers with Code
- **Conferences:** MLSys, NeurIPS, MLOps World

25.9 Chapter Summary

The future of ML engineering is shaped by several key trends:

- **Scale:** Models continue to grow, requiring new infrastructure and optimization techniques
- **Efficiency:** Focus on edge deployment, federated learning, and green ML
- **Automation:** AutoML and declarative systems reduce manual effort
- **Specialization:** LLMOps emerges as distinct discipline
- **Responsibility:** Sustainability, fairness, and privacy become core concerns

- **Integration:** Unified platforms handle entire ML lifecycle

Key Takeaways:

Best Practice: *Invest in fundamentals. While tools and frameworks change, core concepts in distributed systems, optimization, and system design remain relevant.*

Best Practice: *Stay adaptable. The field evolves rapidly. Build a learning system, not just a knowledge base. Follow research, join communities, and experiment with new technologies.*

Best Practice: *Think holistically. Future ML engineers must consider not just model accuracy, but also cost, latency, fairness, sustainability, and business impact.*

Best Practice: *Specialize strategically. As the field matures, specialization becomes more valuable. Choose areas aligned with industry trends and personal interests.*

25.10 Final Thoughts

Machine Learning Engineering has matured from an emerging discipline to a critical function in technology organizations. The practices, patterns, and tools covered in this book provide a foundation for building production ML systems.

However, the field continues to evolve. Technologies that seem cutting-edge today will be mainstream tomorrow, and entirely new paradigms will emerge. Success in ML engineering requires not just technical skills, but adaptability, ethical awareness, and business acumen.

The most impactful ML engineers are those who:

- **Solve Real Problems:** Focus on business value, not just technical novelty
- **Build Reliable Systems:** Prioritize robustness and maintainability
- **Consider Impact:** Think about societal consequences
- **Share Knowledge:** Contribute to the community
- **Never Stop Learning:** Embrace continuous growth

As you apply the concepts from this book, remember that ML engineering is ultimately about amplifying human capabilities and solving important problems. Technology is a tool; impact is the goal.

25.11 Exercises

Exercise 25.1. Exercise 1: Technology Trend Analysis

Analyze an emerging ML technology and assess its potential impact on your organization.

Deliverables:

1. Technology Overview:

- Select one technology: Edge ML, Federated Learning, LLMOps, or Continuous Learning
- Describe the technology and its current state
- Identify key vendors, open-source projects, and research

2. Technical Assessment:

- What problems does it solve?
- What are the technical requirements (infrastructure, expertise)?
- What are the limitations and challenges?
- How mature is the technology? (PoC, Production-ready, Mainstream)

3. Business Case:

- Potential use cases in your domain
- Expected benefits (cost savings, new capabilities, improved performance)
- Implementation costs and timeline
- ROI analysis

4. Adoption Roadmap:

- Phase 1: Exploration (PoC)
- Phase 2: Pilot (limited production)
- Phase 3: Scale (organization-wide adoption)
- Success metrics for each phase
- Risk mitigation strategies

5. Skills Gap Analysis:

- Required skills vs. current team capabilities
- Training needs
- Hiring requirements
- Partner/vendor support

Presentation Format: 10-15 slide deck with executive summary

Exercise 25.2. Exercise 2: Sustainable ML System Design

Design an ML system with sustainability as a primary objective.

Scenario: You're building a real-time recommendation system for a video streaming platform with 50M users. The system currently uses 100 GPUs for model serving, generating significant carbon emissions and costs.

Deliverables:**1. Baseline Carbon Footprint:**

- Calculate current training carbon footprint
- Calculate current inference carbon footprint (annual)
- Identify primary sources of emissions
- Benchmark against industry standards

2. Optimization Strategy:

- Model efficiency improvements (quantization, pruning, distillation)
- Infrastructure optimizations (right-sizing, auto-scaling)
- Regional optimization (low-carbon data centers)
- Temporal optimization (carbon-aware scheduling)
- Caching and request reduction strategies

3. Implementation Plan:

- Prioritized list of optimizations
- Expected carbon reduction for each
- Implementation complexity and timeline
- Trade-offs (accuracy, latency, cost)

4. Measurement and Monitoring:

- Carbon tracking system design
- Dashboard for carbon metrics
- Alerts for carbon budget violations
- Regular reporting cadence

5. Results Projection:

- Target carbon reduction (kg CO₂)
- Cost implications
- Performance impact
- Timeline to achieve targets

Constraints:

- Maintain >95% of current model accuracy

- Keep p99 latency <200ms
- Implementation budget: \$100K
- Timeline: 6 months

Exercise 25.3. Exercise 3: Future Skills Development Plan

Create a personalized 2-year skill development plan for advancing your ML engineering career.

Deliverables:**1. Current State Assessment:**

- Self-assessment of current skills (rate 1-5 for each category in Table 25.5)
- Identify strengths and gaps
- Career goals (specific role, company type, specialization)

2. Target Skills Profile:

- Skills required for target role
- Gap analysis (current vs. target)
- Prioritization based on industry trends and personal goals

3. Learning Plan (24 months):

- Quarter-by-quarter learning objectives
- Specific courses, books, projects for each skill
- Time commitment (hours per week)
- Milestones and checkpoints

4. Hands-On Projects:

- 4-6 portfolio projects demonstrating new skills
- Project descriptions, technologies used, and timelines
- How each project closes a skill gap
- Plan for showcasing work (GitHub, blog, talks)

5. Community Engagement:

- Conferences to attend
- Communities to join (online and local)
- Contribution plans (open source, blog posts, talks)
- Networking goals

6. Evaluation Framework:

- Quarterly self-assessment process
- Success metrics (skills acquired, projects completed, certifications)
- Adjustment triggers (when to pivot the plan)
- Career milestones (promotions, role changes)

Format: Document with Gantt chart showing 24-month timeline

Setup Guides and Installation

.1 Introduction

This appendix provides comprehensive installation and setup guides for all tools and frameworks mentioned throughout this book. Follow these guides to set up a complete ML engineering environment from scratch.

Prerequisites:

- Basic command-line proficiency
- Administrator/root access to your system
- Internet connection for downloads
- Minimum 16GB RAM, 50GB free disk space

.2 Python Environment Setup

.2.1 Installing Python 3.9+

Python is the foundation of modern ML engineering. We recommend Python 3.9 or later.

Windows:

```
1 # Method 1: Official installer
2 # Download from python.org and run installer
3 # Make sure to check "Add Python to PATH"
4
5 # Method 2: Using winget (Windows Package Manager)
6 winget install Python.Python.3.11
7
8 # Method 3: Using Chocolatey
9 choco install python --version=3.11.0
10
11 # Verify installation
12 python --version
13 pip --version
```

macOS:

```
1 # Method 1: Using Homebrew (recommended)
2 brew install python@3.11
3
4 # Method 2: Using pyenv for version management
5 brew install pyenv
6 pyenv install 3.11.0
```

```

7 pyenv global 3.11.0
8
9 # Add to ~/.zshrc or ~/.bash_profile
10 echo 'export PATH="$HOME/.pyenv/shims:$PATH"' >> ~/.zshrc
11
12 # Verify installation
13 python3 --version
14 pip3 --version

```

Linux (Ubuntu/Debian):

```

1 # Update package lists
2 sudo apt update
3
4 # Install Python 3.11
5 sudo apt install software-properties-common
6 sudo add-apt-repository ppa:deadsnakes/ppa
7 sudo apt update
8 sudo apt install python3.11 python3.11-venv python3.11-dev python3-pip
9
10 # Set as default (optional)
11 sudo update-alternatives --install /usr/bin/python3 python3 /usr/bin/python3.11 1
12
13 # Verify installation
14 python3 --version
15 pip3 --version

```

Linux (CentOS/RHEL 8+):

```

1 # Enable EPEL repository
2 sudo dnf install epel-release
3
4 # Install Python 3.11
5 sudo dnf install python3.11 python3.11-devel
6
7 # Verify installation
8 python3.11 --version
9 pip3.11 --version

```

.2.2 Virtual Environment Setup

Virtual environments isolate project dependencies.

```

1 # Create virtual environment
2 python -m venv myproject_env
3
4 # Activate virtual environment
5 # Windows
6 myproject_env\Scripts\activate
7
8 # macOS/Linux
9 source myproject_env/bin/activate
10
11 # Install packages in virtual environment
12 pip install --upgrade pip
13 pip install wheel setuptools
14
15 # Deactivate when done

```



```
16 deactivate
```

.2.3 Conda Environment Setup

Conda is an alternative package manager with better environment management.

```
1 # Install Miniconda
2 # Download from https://docs.conda.io/en/latest/miniconda.html
3
4 # Create conda environment
5 conda create -n ml_env python=3.11
6
7 # Activate environment
8 conda activate ml_env
9
10 # Install packages
11 conda install numpy pandas scikit-learn matplotlib
12
13 # List environments
14 conda env list
15
16 # Export environment
17 conda env export > environment.yml
18
19 # Create environment from file
20 conda env create -f environment.yml
21
22 # Deactivate
23 conda deactivate
```

.3 Docker Installation and Configuration

Docker containers provide consistent environments across different systems.

.3.1 Docker Desktop (Windows/macOS)

Installation:

1. Download Docker Desktop from <https://www.docker.com/products/docker-desktop>
2. Run the installer
3. Start Docker Desktop from Applications
4. Configure resources: Settings → Resources → Advanced
 - CPUs: 4+
 - Memory: 8GB+
 - Disk: 100GB+
5. Enable Kubernetes (optional): Settings → Kubernetes → Enable

Verification:

```

1 # Check Docker version
2 docker --version
3 docker-compose --version
4
5 # Test Docker installation
6 docker run hello-world
7
8 # Check running containers
9 docker ps
10
11 # Check Docker system info
12 docker system info

```

.3.2 Docker Engine (Linux)**Ubuntu/Debian:**

```

1 # Remove old versions
2 sudo apt remove docker docker-engine docker.io containerd runc
3
4 # Install prerequisites
5 sudo apt update
6 sudo apt install ca-certificates curl gnupg lsb-release
7
8 # Add Docker's official GPG key
9 sudo mkdir -p /etc/apt/keyrings
10 curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
11     sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
12
13 # Set up repository
14 echo \
15     "deb [arch=$(dpkg --print-architecture) \
16         signed-by=/etc/apt/keyrings/docker.gpg] \
17         https://download.docker.com/linux/ubuntu \
18         $(lsb_release -cs) stable" | \
19     sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
20
21 # Install Docker Engine
22 sudo apt update
23 sudo apt install docker-ce docker-ce-cli containerd.io \
24     docker-buildx-plugin docker-compose-plugin
25
26 # Add user to docker group (avoid sudo)
27 sudo usermod -aG docker $USER
28 newgrp docker
29
30 # Enable Docker service
31 sudo systemctl enable docker
32 sudo systemctl start docker
33
34 # Verify installation
35 docker run hello-world

```

CentOS/RHEL:

```

1 # Remove old versions

```

```
2 sudo yum remove docker docker-client docker-client-latest \  
3     docker-common docker-latest docker-latest-logrotate \  
4     docker-logrotate docker-engine  
5  
6 # Install yum-utils  
7 sudo yum install -y yum-utils  
8  
9 # Add Docker repository  
10 sudo yum-config-manager --add-repo \  
11     https://download.docker.com/linux/centos/docker-ce.repo  
12  
13 # Install Docker Engine  
14 sudo yum install docker-ce docker-ce-cli containerd.io \  
15     docker-buildx-plugin docker-compose-plugin  
16  
17 # Start Docker  
18 sudo systemctl start docker  
19 sudo systemctl enable docker  
20  
21 # Add user to docker group  
22 sudo usermod -aG docker $USER  
23  
24 # Verify  
25 docker run hello-world
```

.3.3 Docker Configuration for ML

Create a `daemon.json` file for optimized Docker settings:

```
1 # Linux: /etc/docker/daemon.json  
2 # macOS/Windows: Docker Desktop Settings > Docker Engine  
3  
4 {  
5     "default-runtime": "nvidia",  
6     "runtimes": {  
7         "nvidia": {  
8             "path": "nvidia-container-runtime",  
9             "runtimeArgs": []  
10        }  
11    },  
12    "storage-driver": "overlay2",  
13    "log-driver": "json-file",  
14    "log-opts": {  
15        "max-size": "10m",  
16        "max-file": "3"  
17    },  
18    "default-ulimits": {  
19        "nofile": {  
20            "Name": "nofile",  
21            "Hard": 64000,  
22            "Soft": 64000  
23        }  
24    }  
25 }
```

.4 Kubernetes Setup

.4.1 Minikube for Local Development

Minikube runs a single-node Kubernetes cluster locally.

Installation:

```
1 # macOS
2 brew install minikube
3
4 # Linux
5 curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
6 sudo install minikube-linux-amd64 /usr/local/bin/minikube
7
8 # Windows (using Chocolatey)
9 choco install minikube
10
11 # Install kubectl
12 # macOS
13 brew install kubectl
14
15 # Linux
16 curl -LO "https://dl.k8s.io/release/$(curl -L -s \
17     https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
18 sudo install kubectl /usr/local/bin/
19
20 # Windows
21 choco install kubernetes-cli
22
23 # Verify installation
24 minikube version
25 kubectl version --client
```

Starting Minikube:

```
1 # Start with custom resources
2 minikube start --cpus 4 --memory 8192 --disk-size 50g
3
4 # Enable addons
5 minikube addons enable metrics-server
6 minikube addons enable dashboard
7 minikube addons enable ingress
8
9 # Check status
10 minikube status
11
12 # Access dashboard
13 minikube dashboard
14
15 # Stop minikube
16 minikube stop
17
18 # Delete cluster
19 minikube delete
```

.4.2 Kind (Kubernetes in Docker)

Kind is another option for local Kubernetes development.

```
1 # Install kind
2 # macOS/Linux
3 curl -Lo ./kind https://kind.sigs.k8s.io/dl/v0.20.0/kind-linux-amd64
4 chmod +x ./kind
5 sudo mv ./kind /usr/local/bin/kind
6
7 # Create cluster
8 kind create cluster --name ml-cluster
9
10 # Create multi-node cluster
11 cat <<EOF | kind create cluster --config=-
12 kind: Cluster
13 apiVersion: kind.x-k8s.io/v1alpha4
14 nodes:
15 - role: control-plane
16 - role: worker
17 - role: worker
18 EOF
19
20 # List clusters
21 kind get clusters
22
23 # Delete cluster
24 kind delete cluster --name ml-cluster
```

.5 Cloud Provider Setup

.5.1 AWS CLI and SDK

```
1 # Install AWS CLI v2
2 # Linux
3 curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
4 unzip awscliv2.zip
5 sudo ./aws/install
6
7 # macOS
8 brew install awscli
9
10 # Windows
11 msixexec.exe /i https://awscli.amazonaws.com/AWSCLIV2.msi
12
13 # Configure AWS CLI
14 aws configure
15 # AWS Access Key ID: YOUR_ACCESS_KEY
16 # AWS Secret Access Key: YOUR_SECRET_KEY
17 # Default region name: us-west-2
18 # Default output format: json
19
20 # Verify configuration
21 aws sts get-caller-identity
22
23 # Install boto3 (AWS SDK for Python)
```

```
24 pip install boto3
25
26 # Test boto3
27 python -c "import boto3; print(boto3.__version__)"
```

Configure AWS Credentials:

Create ~/.aws/credentials:

```
1 [default]
2 aws_access_key_id = YOUR_ACCESS_KEY
3 aws_secret_access_key = YOUR_SECRET_KEY
4
5 [ml-project]
6 aws_access_key_id = PROJECT_ACCESS_KEY
7 aws_secret_access_key = PROJECT_SECRET_KEY
```

Create ~/.aws/config:

```
1 [default]
2 region = us-west-2
3 output = json
4
5 [profile ml-project]
6 region = us-east-1
7 output = json
```

.5.2 Google Cloud SDK

```
1 # Install gcloud CLI
2 # Linux
3 curl https://sdk.cloud.google.com | bash
4 exec -l $SHELL
5 gcloud init
6
7 # macOS
8 brew install --cask google-cloud-sdk
9
10 # Windows
11 # Download installer from cloud.google.com/sdk/docs/install
12
13 # Initialize gcloud
14 gcloud init
15
16 # Login
17 gcloud auth login
18
19 # Set project
20 gcloud config set project YOUR_PROJECT_ID
21
22 # List configurations
23 gcloud config list
24
25 # Install additional components
26 gcloud components install kubectl
27 gcloud components install beta
28
29 # Enable APIs
```

```
30 gcloud services enable compute.googleapis.com
31 gcloud services enable aiplatform.googleapis.com
32 gcloud services enable storage.googleapis.com
33
34 # Install Python client libraries
35 pip install google-cloud-storage google-cloud-aiplatform
```

.5.3 Azure CLI

```
1 # Install Azure CLI
2 # Linux (one-line install)
3 curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash
4
5 # macOS
6 brew install azure-cli
7
8 # Windows
9 # Download MSI from https://aka.ms/installazurecliwindows
10
11 # Login
12 az login
13
14 # Set subscription
15 az account set --subscription "YOUR_SUBSCRIPTION_ID"
16
17 # List subscriptions
18 az account list --output table
19
20 # Install Azure ML extension
21 az extension add -n ml
22
23 # Install Python SDK
24 pip install azure-ai-ml azure-identity
```

.6 ML Framework Installation

.6.1 PyTorch

```
1 # CPU-only version
2 pip install torch torchvision torchaudio
3
4 # CUDA 11.8 (NVIDIA GPU)
5 pip install torch torchvision torchaudio --index-url \
6     https://download.pytorch.org/whl/cu118
7
8 # CUDA 12.1
9 pip install torch torchvision torchaudio --index-url \
10    https://download.pytorch.org/whl/cu121
11
12 # Verify installation
13 python -c "import torch; print(torch.__version__); \
14     print(torch.cuda.is_available())"
15
```

```
16 # Check CUDA version
17 python -c "import torch; print(torch.version.cuda)"
```

.6.2 TensorFlow

```
1 # CPU version
2 pip install tensorflow
3
4 # GPU version (CUDA 11.8+, cuDNN 8.6+)
5 pip install tensorflow[and-cuda]
6
7 # Specific version
8 pip install tensorflow==2.15.0
9
10 # Verify installation
11 python -c "import tensorflow as tf; print(tf.__version__); \
12           print(tf.config.list_physical_devices('GPU'))"
13
14 # Enable mixed precision training
15 python -c "from tensorflow.keras import mixed_precision; \
16           mixed_precision.set_global_policy('mixed_float16')"
```

.6.3 JAX

```
1 # CPU version
2 pip install jax jaxlib
3
4 # CUDA 11 version
5 pip install jax[cuda11_pip] -f \
6     https://storage.googleapis.com/jax-releases/jax_cuda_releases.html
7
8 # CUDA 12 version
9 pip install jax[cuda12_pip] -f \
10    https://storage.googleapis.com/jax-releases/jax_cuda_releases.html
11
12 # Verify
13 python -c "import jax; print(jax.__version__); \
14           print(jax.devices())"
```

.6.4 Data Science Stack

```
1 # Core data science libraries
2 pip install numpy pandas scikit-learn scipy
3
4 # Visualization libraries
5 pip install matplotlib seaborn plotly
6
7 # Jupyter
8 pip install jupyter jupyterlab ipywidgets
9
10 # Additional ML libraries
11 pip install xgboost lightgbm catboost
```



```
12
13 # Start Jupyter Lab
14 jupyter lab --port=8888 --no-browser
```

.7 MLOps Tools Installation

.7.1 MLflow

```
1 # Install MLflow
2 pip install mlflow
3
4 # Install with extras
5 pip install mlflow[extras] # Includes scikit-learn, etc.
6
7 # Start MLflow tracking server
8 mlflow server \
9     --backend-store-uri sqlite:///mlflow.db \
10    --default-artifact-root ./mlruns \
11    --host 0.0.0.0 \
12    --port 5000
13
14 # Start with remote storage (S3)
15 mlflow server \
16    --backend-store-uri postgresql://user:pass@host:5432/mlflow \
17    --default-artifact-root s3://my-bucket/mlflow \
18    --host 0.0.0.0
19
20 # Set tracking URI
21 export MLFLOW_TRACKING_URI=http://localhost:5000
```

.7.2 DVC (Data Version Control)

```
1 # Install DVC
2 pip install dvc
3
4 # Install with remote storage support
5 pip install dvc[s3] # AWS S3
6 pip install dvc[gs] # Google Cloud Storage
7 pip install dvc[azure] # Azure Blob Storage
8 pip install dvc[all] # All remote storage options
9
10 # Initialize DVC in project
11 cd your_project
12 dvc init
13
14 # Configure remote storage
15 # S3
16 dvc remote add -d myremote s3://mybucket/dvcstore
17 dvc remote modify myremote region us-west-2
18
19 # GCS
20 dvc remote add -d myremote gs://mybucket/dvcstore
21
```

```
22 # Track data file
23 dvc add data/large_dataset.csv
24 git add data/large_dataset.csv.dvc .gitignore
25
26 # Push data to remote
27 dvc push
28
29 # Pull data from remote
30 dvc pull
31
32 # Create pipeline
33 dvc run -n prepare \
34     -d data/raw/data.csv \
35     -o data/prepared/data.csv \
36     python src/prepare.py
37
38 # Reproduce pipeline
39 dvc repro
```

.7.3 Apache Airflow

```
1 # Install Airflow
2 AIRFLOW_VERSION=2.7.3
3 PYTHON_VERSION="$(python --version | cut -d " " -f 2 | cut -d "." -f 1-2)"
4 CONSTRAINT_URL="https://raw.githubusercontent.com/apache/airflow/\
5 constraints-${AIRFLOW_VERSION}/constraints-${PYTHON_VERSION}.txt"
6
7 pip install "apache-airflow==${AIRFLOW_VERSION}" \
8     --constraint "${CONSTRAINT_URL}"
9
10 # Install with providers
11 pip install "apache-airflow[aws,google,postgres,redis]"
12
13 # Set Airflow home
14 export AIRFLOW_HOME=~/.airflow
15
16 # Initialize database
17 airflow db init
18
19 # Create admin user
20 airflow users create \
21     --username admin \
22     --firstname Admin \
23     --lastname User \
24     --role Admin \
25     --email admin@example.com \
26     --password admin
27
28 # Start web server
29 airflow webserver --port 8080
30
31 # Start scheduler (in separate terminal)
32 airflow scheduler
33
34 # Test DAG
35 airflow dags test my_dag 2024-01-01
```

.7.4 Kubeflow

```

1 # Install Kubeflow Pipelines standalone
2 export PIPELINE_VERSION=2.0.0
3 kubectl apply -k "github.com/kubeflow/pipelines/manifests/\
4 kustomize/cluster-scoped-resources?ref=$PIPELINE_VERSION"
5 kubectl wait --for condition=established --timeout=60s \
6     crd/applications.app.k8s.io
7 kubectl apply -k "github.com/kubeflow/pipelines/manifests/\
8 kustomize/env/platform-agnostic?ref=$PIPELINE_VERSION"
9
10 # Access Kubeflow Pipelines UI
11 kubectl port-forward -n kubeflow svc/ml-pipeline-ui 8080:80
12
13 # Install Kubeflow Pipelines SDK
14 pip install kfp
15
16 # Verify installation
17 python -c "import kfp; print(kfp.__version__)"

```

.7.5 Weights & Biases

```

1 # Install wandb
2 pip install wandb
3
4 # Login (requires account)
5 wandb login
6
7 # Or set API key
8 export WANDB_API_KEY=your_api_key
9
10 # Initialize in project
11 wandb init
12
13 # Test installation
14 python -c "import wandb; wandb.init(project='test'); wandb.finish()"

```

.8 GPU Drivers and CUDA

.8.1 NVIDIA Drivers

Ubuntu/Debian:

```

1 # Check GPU
2 lspci | grep -i nvidia
3
4 # Add NVIDIA repository
5 sudo add-apt-repository ppa:graphics-drivers/ppa
6 sudo apt update
7
8 # Install recommended driver
9 ubuntu-drivers devices
10 sudo ubuntu-drivers autoinstall
11

```

```
12 # Or install specific version
13 sudo apt install nvidia-driver-535
14
15 # Reboot
16 sudo reboot
17
18 # Verify installation
19 nvidia-smi
```

.8.2 CUDA Toolkit

```
1 # CUDA 11.8 (Ubuntu 22.04)
2 wget https://developer.download.nvidia.com/compute/cuda/repos/\
3 ubuntu2204/x86_64/cuda-keyring_1.0-1_all.deb
4 sudo dpkg -i cuda-keyring_1.0-1_all.deb
5 sudo apt update
6 sudo apt install cuda-11-8
7
8 # CUDA 12.1
9 sudo apt install cuda-12-1
10
11 # Add to PATH
12 echo 'export PATH=/usr/local/cuda/bin:$PATH' >> ~/.bashrc
13 echo 'export LD_LIBRARY_PATH=/usr/local/cuda/lib64:$LD_LIBRARY_PATH' >> ~/.bashrc
14 source ~/.bashrc
15
16 # Verify CUDA
17 nvcc --version
```

.8.3 cuDNN

```
1 # Download cuDNN from NVIDIA (requires login)
2 # https://developer.nvidia.com/cudnn
3
4 # Install cuDNN (example for Ubuntu)
5 sudo apt install ./cudnn-local-repo-ubuntu2204-8.9.5.29_1.0-1_amd64.deb
6 sudo cp /var/cudnn-local-repo-ubuntu2204-8.9.5.29/cudnn-*-keyring.gpg \
7 /usr/share/keyrings/
8 sudo apt update
9 sudo apt install libcudnn8 libcudnn8-dev
10
11 # Verify
12 cat /usr/local/cuda/include/cudnn_version.h | grep CUDNN_MAJOR -A 2
```

.8.4 NVIDIA Container Toolkit

For running GPU workloads in Docker:

```
1 # Add repository
2 distribution=$(. /etc/os-release;echo $ID$VERSION_ID)
3 curl -s -L https://nvidia.github.io/nvidia-docker/gpgkey | \
4     sudo apt-key add -
5 curl -s -L https://nvidia.github.io/nvidia-docker/$distribution/\
```

```

6 nvidia-docker.list | sudo tee /etc/apt/sources.list.d/nvidia-docker.list
7
8 # Install
9 sudo apt update
10 sudo apt install -y nvidia-container-toolkit
11
12 # Restart Docker
13 sudo systemctl restart docker
14
15 # Test GPU in Docker
16 docker run --rm --gpus all nvidia/cuda:11.8.0-base-ubuntu22.04 nvidia-smi

```

.9 Development Tools

.9.1 Git Configuration

```

1 # Configure Git
2 git config --global user.name "Your Name"
3 git config --global user.email "your.email@example.com"
4
5 # Set default editor
6 git config --global core.editor "vim"
7
8 # Set default branch name
9 git config --global init.defaultBranch main
10
11 # Configure line endings
12 git config --global core.autocrlf input # Linux/macOS
13 git config --global core.autocrlf true # Windows
14
15 # Configure aliases
16 git config --global alias.st status
17 git config --global alias.co checkout
18 git config --global alias.br branch
19 git config --global alias.ci commit
20 git config --global alias.unstage 'reset HEAD --'
21
22 # View configuration
23 git config --list

```

.9.2 VS Code Setup for ML

Install VS Code:

```

1 # macOS
2 brew install --cask visual-studio-code
3
4 # Ubuntu/Debian
5 wget -qO- https://packages.microsoft.com/keys/microsoft.asc | \
6     gpg --dearmor > packages.microsoft.gpg
7 sudo install -o root -g root -m 644 packages.microsoft.gpg \
8     /etc/apt/trusted.gpg.d/
9 sudo sh -c 'echo "deb [arch=amd64] \
10     https://packages.microsoft.com/repos/vscode stable main" > \

```

```

11 /etc/apt/sources.list.d/vscode.list'
12 sudo apt update
13 sudo apt install code

```

Essential Extensions:

```

1 # Python
2 code --install-extension ms-python.python
3 code --install-extension ms-python.vscode-pylance
4
5 # Jupyter
6 code --install-extension ms-toolsai.jupyter
7
8 # Docker and Kubernetes
9 code --install-extension ms-azuretools.vscode-docker
10 code --install-extension ms-kubernetes-tools.vscode-kubernetes-tools
11
12 # YAML
13 code --install-extension redhat.vscode-yaml
14
15 # Git
16 code --install-extension eamodio.gitlens
17
18 # Remote Development
19 code --install-extension ms-vscode-remote.remote-ssh
20 code --install-extension ms-vscode-remote.remote-containers
21
22 # AI assistance
23 code --install-extension GitHub.copilot

```

.10 Complete Environment Setup Script

This script sets up a complete ML engineering environment:

```

1 #!/bin/bash
2 set -e
3
4 echo "Setting up ML Engineering Environment..."
5
6 # Variables
7 PROJECT_NAME=${1:-ml_project}
8 PYTHON_VERSION=${2:-3.11}
9
10 echo "Project name: $PROJECT_NAME"
11 echo "Python version: $PYTHON_VERSION"
12
13 # Create project directory
14 mkdir -p $PROJECT_NAME
15 cd $PROJECT_NAME
16
17 echo "Creating virtual environment..."
18 python${PYTHON_VERSION} -m venv venv
19 source venv/bin/activate
20
21 # Upgrade pip
22 echo "Upgrading pip..."
23 pip install --upgrade pip setuptools wheel

```

```
24
25 # Install ML frameworks
26 echo "Installing ML frameworks..."
27 pip install torch torchvision torchaudio
28 pip install tensorflow
29 pip install scikit-learn xgboost lightgbm
30
31 # Install data science stack
32 echo "Installing data science libraries..."
33 pip install numpy pandas scipy matplotlib seaborn plotly
34
35 # Install Jupyter
36 echo "Installing Jupyter..."
37 pip install jupyter jupyterlab ipywidgets
38
39 # Install MLOps tools
40 echo "Installing MLOps tools..."
41 pip install mlflow dvc wandb
42
43 # Install serving frameworks
44 echo "Installing serving frameworks..."
45 pip install fastapi uvicorn pydantic
46
47 # Install testing tools
48 echo "Installing testing tools..."
49 pip install pytest pytest-cov pytest-mock
50
51 # Install code quality tools
52 echo "Installing code quality tools..."
53 pip install black flake8 mypy pylint isort
54
55 # Install additional tools
56 echo "Installing additional tools..."
57 pip install python-dotenv pyyaml requests boto3
58
59 # Initialize git
60 echo "Initializing git repository..."
61 git init
62 curl -sL https://raw.githubusercontent.com/github/gitignore/main/Python.gitignore \
63     -o .gitignore
64
65 # Initialize DVC
66 echo "Initializing DVC..."
67 dvc init
68
69 # Create project structure
70 echo "Creating project structure..."
71 mkdir -p data/{raw,processed,features,external}
72 mkdir -p models/{trained,registry}
73 mkdir -p notebooks/{exploratory,reporting}
74 mkdir -p src/{data,features,models,utils,api}
75 mkdir -p tests/{unit,integration}
76 mkdir -p config
77 mkdir -p logs
78 mkdir -p docs
79 mkdir -p scripts
80
81 # Create __init__.py files
```

```

82 touch src/__init__.py
83 touch src/data/__init__.py
84 touch src/features/__init__.py
85 touch src/models/__init__.py
86 touch src/utils/__init__.py
87 touch src/api/__init__.py
88 touch tests/__init__.py
89 touch tests/unit/__init__.py
90 touch tests/integration/__init__.py
91
92 # Create requirements.txt
93 echo "Creating requirements.txt..."
94 pip freeze > requirements.txt
95
96 # Create .env template
97 echo "Creating .env template..."
98 cat > .env.template << 'EOF'
99 # Environment variables template
100 MLFLOW_TRACKING_URI=http://localhost:5000
101 AWS_PROFILE=default
102 WANDB_API_KEY=your_key_here
103 PYTHONPATH=./src
104 EOF
105
106 # Create README
107 echo "Creating README.md..."
108 cat > README.md << EOF
109 # $PROJECT_NAME
110
111 ML Engineering Project
112
113 ## Setup
114
115 \'\'\bash
116 # Create and activate virtual environment
117 python -m venv venv
118 source venv/bin/activate # or venv\Scripts\activate on Windows
119
120 # Install dependencies
121 pip install -r requirements.txt
122
123 # Copy environment variables
124 cp .env.template .env
125 # Edit .env with your values
126 \'\'\
127
128 ## Project Structure
129
130 \'\'\
131 $PROJECT_NAME/
132 |--- data/ # Data files
133 | |--- raw/ # Original data
134 | |--- processed/ # Cleaned data
135 | \-- features/ # Feature sets
136 |--- models/ # Trained models
137 |--- notebooks/ # Jupyter notebooks
138 |--- src/ # Source code
139 | |--- data/ # Data processing

```



```

140 | |--- features/      # Feature engineering
141 | |--- models/       # Model training
142 | |-- api/           # API code
143 | |--- tests/        # Tests
144 | |-- config/        # Configuration files
145 | \'\''
146
147 ## Usage
148
149 TODO: Add usage instructions
150 EOF
151
152 # Create pyproject.toml
153 echo "Creating pyproject.toml..."
154 cat > pyproject.toml << 'EOF'
155 [tool.black]
156 line-length = 88
157 target-version = ['py311']
158
159 [tool.isort]
160 profile = "black"
161 line_length = 88
162
163 [tool.mypy]
164 python_version = "3.11"
165 warn_return_any = true
166 warn_unused_configs = true
167 disallow_untyped_defs = true
168 EOF
169
170 # Create pytest.ini
171 echo "Creating pytest.ini..."
172 cat > pytest.ini << 'EOF'
173 [pytest]
174 testpaths = tests
175 python_files = test_*.py
176 python_classes = Test*
177 python_functions = test_*
178 addopts = -v --cov=src --cov-report=html --cov-report=term
179 EOF
180
181 echo ""
182 echo "Setup complete!"
183 echo ""
184 echo "To activate the environment:"
185 echo "  source venv/bin/activate"
186 echo ""
187 echo "To start Jupyter Lab:"
188 echo "  jupyter lab"
189 echo ""
190 echo "To start MLflow UI:"
191 echo "  mlflow ui"

```

Listing 6: setup_ml_environment.sh

Usage:

```

1 # Make script executable
2 chmod +x setup_ml_environment.sh

```

```
3
4 # Run with default settings
5 ./setup_ml_environment.sh
6
7 # Run with custom project name and Python version
8 ./setup_ml_environment.sh my_ml_project 3.11
```

.11 Troubleshooting Common Issues

.11.1 Python Installation Issues

Issue: pip not found after Python installation

```
1 # Solution: Reinstall pip
2 python -m ensurepip --upgrade
3
4 # Or download get-pip.py
5 curl https://bootstrap.pypa.io/get-pip.py -o get-pip.py
6 python get-pip.py
```

Issue: Permission denied when installing packages

```
1 # Solution: Use --user flag
2 pip install --user package_name
3
4 # Or use virtual environment (recommended)
5 python -m venv venv
6 source venv/bin/activate
7 pip install package_name
```

.11.2 Docker Issues

Issue: Cannot connect to Docker daemon

```
1 # Solution 1: Start Docker service
2 sudo systemctl start docker
3
4 # Solution 2: Add user to docker group
5 sudo usermod -aG docker $USER
6 newgrp docker
7
8 # Solution 3: Check Docker socket permissions
9 sudo chmod 666 /var/run/docker.sock
```

.11.3 CUDA Issues

Issue: CUDA out of memory

```
1 # Solution 1: Clear cache
2 import torch
3 torch.cuda.empty_cache()
4
5 # Solution 2: Use smaller batch size
6 batch_size = 16 # Reduce from 32 or 64
```

```

7
8 # Solution 3: Use gradient accumulation
9 # See Chapter 9 for details

```

Issue: CUDA version mismatch

```

1 # Check CUDA version
2 nvcc --version
3 nvidia-smi
4
5 # Install matching PyTorch version
6 # Visit: pytorch.org/get-started/locally/

```

.12 Verification Checklist

After completing the setup, verify your environment:

1. **Python:** `python -version` shows Python 3.9+
2. **pip:** `pip -version` works
3. **Virtual environment:** Can create and activate `venv`
4. **Docker:** `docker run hello-world` succeeds
5. **PyTorch:** Can import and detect GPU if available
6. **TensorFlow:** Can import and list devices
7. **MLflow:** `mlflow -version` works
8. **DVC:** `dvc version` works
9. **Jupyter:** Can start and access Jupyter Lab
10. **Git:** `git -version` works

Run this verification script:

```

1 #!/usr/bin/env python
2 """Verify ML environment setup"""
3
4 import sys
5
6 def check_import(module_name, package_name=None):
7     """Check if module can be imported"""
8     try:
9         __import__(module_name)
10        print(f"✓ {package_name or module_name}")
11        return True
12    except ImportError:
13        print(f"x {package_name or module_name} - Not installed")
14        return False
15
16 def check_gpu():
17     """Check GPU availability"""

```

```
18     try:
19         import torch
20         if torch.cuda.is_available():
21             print(f"✓ GPU available: {torch.cuda.get_device_name(0)}")
22             return True
23         else:
24             print("x GPU not available (CPU only)")
25             return False
26     except:
27         return False
28
29 print("Python Environment Verification")
30 print("=" * 50)
31 print(f"Python version: {sys.version}")
32 print()
33
34 modules = [
35     ("torch", "PyTorch"),
36     ("tensorflow", "TensorFlow"),
37     ("sklearn", "scikit-learn"),
38     ("numpy", "NumPy"),
39     ("pandas", "pandas"),
40     ("matplotlib", "Matplotlib"),
41     ("mlflow", "MLflow"),
42     ("fastapi", "FastAPI"),
43     ("jupyter", "Jupyter"),
44 ]
45
46 print("Checking installed packages:")
47 all_ok = all(check_import(mod, name) for mod, name in modules)
48
49 print("\nChecking GPU:")
50 check_gpu()
51
52 print("\n" + "=" * 50)
53 if all_ok:
54     print("✓ All required packages installed!")
55 else:
56     print("x Some packages missing. Run: pip install -r requirements.txt")
```

Listing 7: verify_setup.py

Run with: `python verify_setup.py`

Code Examples and Templates

.13 Complete Training Pipeline Example

```
1 import mlflow
2 import pandas as pd
3 from sklearn.model_selection import train_test_split
4 from sklearn.ensemble import RandomForestClassifier
5 from sklearn.metrics import accuracy_score, classification_report
6 import joblib
7
8 def load_data(path):
9     """Load and prepare data"""
10    df = pd.read_csv(path)
11    X = df.drop('target', axis=1)
12    y = df['target']
13    return train_test_split(X, y, test_size=0.2, random_state=42)
14
15 def train_model(X_train, y_train, params):
16     """Train RandomForest model"""
17    model = RandomForestClassifier(**params)
18    model.fit(X_train, y_train)
19    return model
20
21 def evaluate_model(model, X_test, y_test):
22     """Evaluate model performance"""
23    y_pred = model.predict(X_test)
24    accuracy = accuracy_score(y_test, y_pred)
25    report = classification_report(y_test, y_pred)
26    return accuracy, report
27
28 def main():
29     # MLflow experiment tracking
30    mlflow.set_experiment("model_training")
31
32    with mlflow.start_run():
33        # Load data
34        X_train, X_test, y_train, y_test = load_data('data/processed/data.csv')
35
36        # Model parameters
37        params = {
38            'n_estimators': 100,
39            'max_depth': 10,
40            'random_state': 42
41        }
42        mlflow.log_params(params)
43
44        # Train model
```

```

45     model = train_model(X_train, y_train, params)
46
47     # Evaluate model
48     accuracy, report = evaluate_model(model, X_test, y_test)
49     mlflow.log_metric("accuracy", accuracy)
50
51     # Save model
52     mlflow.sklearn.log_model(model, "model")
53     joblib.dump(model, 'models/model.pkl')
54
55     print(f"Accuracy: {accuracy:.4f}")
56     print(report)
57
58 if __name__ == "__main__":
59     main()

```

Listing 8: train_pipeline.py

.14 REST API for Model Serving

```

1 from fastapi import FastAPI, HTTPException
2 from pydantic import BaseModel
3 import joblib
4 import numpy as np
5 import logging
6
7 # Setup logging
8 logging.basicConfig(level=logging.INFO)
9 logger = logging.getLogger(__name__)
10
11 # Load model
12 model = joblib.load('models/model.pkl')
13
14 # Create FastAPI app
15 app = FastAPI(title="ML Model API", version="1.0")
16
17 class PredictionRequest(BaseModel):
18     features: list[float]
19
20 class PredictionResponse(BaseModel):
21     prediction: int
22     probability: float
23
24 @app.get("/health")
25 async def health_check():
26     """Health check endpoint"""
27     return {"status": "healthy"}
28
29 @app.post("/predict", response_model=PredictionResponse)
30 async def predict(request: PredictionRequest):
31     """Make prediction"""
32     try:
33         # Prepare input
34         X = np.array(request.features).reshape(1, -1)
35
36         # Make prediction

```

```

37     prediction = model.predict(X)[0]
38     probability = model.predict_proba(X)[0].max()
39
40     logger.info(f"Prediction: {prediction}, Probability: {probability:.4f}")
41
42     return PredictionResponse(
43         prediction=int(prediction),
44         probability=float(probability)
45     )
46 except Exception as e:
47     logger.error(f"Prediction error: {str(e)}")
48     raise HTTPException(status_code=500, detail=str(e))
49
50 if __name__ == "__main__":
51     import uvicorn
52     uvicorn.run(app, host="0.0.0.0", port=8000)

```

Listing 9: serve.py

.15 Data Validation with Great Expectations

```

1 import great_expectations as ge
2 import pandas as pd
3
4 def create_expectation_suite(df):
5     """Create data validation suite"""
6     # Convert to GE DataFrame
7     ge_df = ge.from_pandas(df)
8
9     # Define expectations
10    ge_df.expect_table_row_count_to_be_between(min_value=100, max_value=1000000)
11    ge_df.expect_column_values_to_not_be_null('user_id')
12    ge_df.expect_column_values_to_be_unique('user_id')
13    ge_df.expect_column_values_to_be_between('age', min_value=0, max_value=120)
14    ge_df.expect_column_values_to_be_in_set('country', ['US', 'UK', 'CA', 'AU'])
15    ge_df.expect_column_mean_to_be_between('revenue', min_value=0, max_value=10000)
16
17    # Save expectation suite
18    ge_df.save_expectation_suite('data_expectations.json')
19
20    return ge_df
21
22 def validate_data(df, expectation_suite_path):
23     """Validate data against expectations"""
24     ge_df = ge.from_pandas(df)
25
26     # Load expectations
27     ge_df.read_json(expectation_suite_path)
28
29     # Validate
30     results = ge_df.validate()
31
32     return results
33
34 if __name__ == "__main__":
35     # Load data

```

```

36 df = pd.read_csv('data/raw/data.csv')
37
38 # Create and validate
39 ge_df = create_expectation_suite(df)
40 results = validate_data(df, 'data_expectations.json')
41
42 print(f"Validation success: {results.success}")
43 print(f"Number of expectations: {len(results.results)}")

```

Listing 10: validate_data.py

.16 Airflow DAG Template

```

1 from airflow import DAG
2 from airflow.operators.python import PythonOperator
3 from airflow.operators.bash import BashOperator
4 from datetime import datetime, timedelta
5 import sys
6
7 sys.path.insert(0, '/opt/airflow/dags/src')
8
9 from data_processing import extract_data, transform_data
10 from model_training import train_model
11 from model_evaluation import evaluate_model
12
13 default_args = {
14     'owner': 'ml_team',
15     'depends_on_past': False,
16     'start_date': datetime(2024, 1, 1),
17     'email': ['ml-team@example.com'],
18     'email_on_failure': True,
19     'email_on_retry': False,
20     'retries': 3,
21     'retry_delay': timedelta(minutes=5),
22 }
23
24 with DAG(
25     'ml_training_pipeline',
26     default_args=default_args,
27     description='Complete ML training pipeline',
28     schedule_interval='@daily',
29     catchup=False,
30     tags=['ml', 'training'],
31 ) as dag:
32
33     extract_task = PythonOperator(
34         task_id='extract_data',
35         python_callable=extract_data,
36     )
37
38     transform_task = PythonOperator(
39         task_id='transform_data',
40         python_callable=transform_data,
41     )
42
43     validate_task = BashOperator(

```



```

44     task_id='validate_data',
45     bash_command='python /opt/airflow/dags/src/validate_data.py',
46 )
47
48 train_task = PythonOperator(
49     task_id='train_model',
50     python_callable=train_model,
51 )
52
53 evaluate_task = PythonOperator(
54     task_id='evaluate_model',
55     python_callable=evaluate_model,
56 )
57
58 deploy_task = BashOperator(
59     task_id='deploy_model',
60     bash_command='python /opt/airflow/dags/src/deploy_model.py',
61 )
62
63 extract_task >> transform_task >> validate_task >> train_task >> evaluate_task >>
    deploy_task

```

Listing 11: ml_dag.py

.17 Model Monitoring Script

```

1  import pandas as pd
2  import numpy as np
3  from scipy import stats
4  from evidently.report import Report
5  from evidently.metric_preset import DataDriftPreset, DataQualityPreset
6
7  def detect_data_drift(reference_data, current_data, threshold=0.05):
8      """Detect data drift using KS test"""
9      drift_detected = {}
10
11     for column in reference_data.columns:
12         if pd.api.types.is_numeric_dtype(reference_data[column]):
13             statistic, pvalue = stats.ks_2samp(
14                 reference_data[column].dropna(),
15                 current_data[column].dropna()
16             )
17             drift_detected[column] = {
18                 'drift': pvalue < threshold,
19                 'pvalue': pvalue,
20                 'statistic': statistic
21             }
22
23     return drift_detected
24
25  def generate_monitoring_report(reference_data, current_data):
26      """Generate comprehensive monitoring report"""
27      report = Report(metrics=[
28          DataDriftPreset(),
29          DataQualityPreset(),
30      ])

```

```
31     report.run(reference_data=reference_data, current_data=current_data)
32     report.save_html('monitoring_report.html')
33
34
35     return report
36
37 if __name__ == "__main__":
38     # Load reference and current data
39     reference = pd.read_csv('data/reference/data.csv')
40     current = pd.read_csv('data/current/data.csv')
41
42     # Detect drift
43     drift_results = detect_data_drift(reference, current)
44
45     # Print results
46     for column, result in drift_results.items():
47         if result['drift']:
48             print(f"WARNING: Drift detected in {column}: p-value={result['pvalue']:.4f}
49             ")
50
51     # Generate detailed report
52     report = generate_monitoring_report(reference, current)
```

Listing 12: monitor.py

Tools and Frameworks Comparison

.18 Experiment Tracking Tools

Table 8: Comparison of Experiment Tracking Tools

Tool	Pros	Cons	Best For
MLflow	Open-source, flexible	Limited UI	General ML projects
Weights & Biases	Great UI, collaboration	Requires account	Team projects
Neptune.ai	Feature-rich	Paid service	Enterprise
TensorBoard	PyTorch/TF integration	Limited features	Deep learning

.19 Model Serving Frameworks

Table 9: Model Serving Framework Comparison

Framework	Language	Performance	Ease of Use
TensorFlow Serving	C++/gRPC	High	Medium
TorchServe	Java	High	Medium
FastAPI + Uvicorn	Python	Medium	High
BentoML	Python	Medium	High
Seldon Core	Multi	High	Medium
KServe	Multi	High	Complex

Table 10: Data Orchestration Platform Comparison

Tool	Learning Curve	Scalability	Community
Apache Airflow	Medium	High	Large
Prefect	Low	High	Growing
Dagster	Medium	High	Growing
Kubeflow Pipelines	High	Very High	Large
Argo Workflows	Medium	Very High	Large

Table 11: Feature Store Comparison

Solution	Type	Backend	Complexity
Feast	Open-source	Multi	Medium
Tecton	Managed	Cloud	High
Hopsworks	Open-source	Multi	High
AWS Feature Store	Cloud	AWS	Medium
Vertex AI Feature Store	Cloud	GCP	Medium

.20 Data Orchestration Tools

.21 Feature Store Solutions

.22 ML Monitoring Tools

.23 Cloud ML Platforms

.24 Container Orchestration

.25 CI/CD for ML

.26 Data Versioning

.27 Model Compression Tools

.28 Recommended Tool Stacks

.28.1 Startup Stack (Cost-Conscious)

- **Experiment Tracking:** MLflow
- **Orchestration:** Prefect
- **Serving:** FastAPI + Docker
- **Monitoring:** Evidently + Prometheus

Table 12: ML Monitoring Tool Comparison

Tool	Type	Features	Cost
Evidently	Open-source	Drift detection	Free
WhyLabs	SaaS	Comprehensive	Paid
Arize AI	SaaS	Advanced analytics	Paid
Fiddler	SaaS	Explainability	Paid
Prometheus + Grafana	Open-source	General monitoring	Free

Table 13: Cloud ML Platform Features

Platform	Training	Serving	MLOps	Cost
AWS SageMaker	Excellent	Excellent	Good	\$\$\$
Google Vertex AI	Excellent	Excellent	Excellent	\$\$\$
Azure ML	Good	Good	Good	\$\$
Databricks	Excellent	Good	Good	\$\$\$

- **Infrastructure:** Docker Compose / Single server

.28.2 Mid-Size Company Stack

- **Experiment Tracking:** MLflow or Weights & Biases
- **Orchestration:** Apache Airflow
- **Feature Store:** Feast
- **Serving:** Kubernetes + KServe
- **Monitoring:** Custom (Prometheus + Grafana) or Arize
- **Cloud:** AWS/GCP managed services

.28.3 Enterprise Stack

- **Platform:** AWS SageMaker or Google Vertex AI
- **Orchestration:** Kubeflow Pipelines or Airflow
- **Feature Store:** Tecton or Cloud provider
- **Serving:** Kubernetes + KServe or Cloud provider
- **Monitoring:** Enterprise monitoring solution (WhyLabs, Arize)
- **Governance:** Custom or vendor solution

Table 14: Container Orchestration Comparison

Platform	Complexity	Scalability	ML Support
Kubernetes	High	Excellent	Good (with KServe)
Docker Swarm	Low	Good	Limited
Amazon ECS	Medium	Good	Good (with SageMaker)
Google Cloud Run	Low	Good	Good

Table 15: CI/CD Tool Comparison

Tool	ML Features	Integration	Learning Curve
GitHub Actions	Basic	GitHub	Low
GitLab CI/CD	Basic	GitLab	Low
Jenkins	Plugins available	Universal	Medium
CircleCI	Basic	Multi	Low
Azure DevOps	Good	Azure	Medium

Table 16: Data Versioning Tools

Tool	Backend	Performance	Ease of Use
DVC	Multi (S3, GCS, etc.)	Good	Medium
LakeFS	S3-compatible	Excellent	Medium
Delta Lake	Spark	Excellent	High
Git LFS	Git	Limited	Low

Table 17: Model Optimization Frameworks

Framework	Techniques	Frameworks	Performance
ONNX Runtime	Multi	Multi-framework	Excellent
TensorRT	Quantization, pruning	TensorFlow, PyTorch	Excellent
PyTorch Mobile	Quantization	PyTorch	Good
TFLite	Quantization	TensorFlow	Good
OpenVINO	Multi	Multi-framework	Excellent

Mathematical Foundations

.29 Linear Algebra Fundamentals

.29.1 Matrix Operations

Matrix Multiplication:

$$\mathbf{C} = \mathbf{AB}, \quad C_{ij} = \sum_{k=1}^n A_{ik}B_{kj} \quad (1)$$

Matrix Transpose:

$$(\mathbf{A}^T)_{ij} = A_{ji} \quad (2)$$

Matrix Inverse:

$$\mathbf{AA}^{-1} = \mathbf{I} \quad (3)$$

.29.2 Eigenvalues and Eigenvectors

For a square matrix $\mathbf{A}$, an eigenvector $\mathbf{v}$ and eigenvalue λ satisfy:

$$\mathbf{Av} = \lambda\mathbf{v} \quad (4)$$

.30 Probability and Statistics

.30.1 Probability Distributions

Normal Distribution:

$$f(x \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right) \quad (5)$$

Bernoulli Distribution:

$$P(X = x) = p^x(1 - p)^{1-x}, \quad x \in \{0, 1\} \quad (6)$$

.30.2 Expected Value and Variance

Expected Value:

$$\mathbb{E}[X] = \sum_i x_i P(X = x_i) \quad \text{or} \quad \mathbb{E}[X] = \int x f(x) dx \quad (7)$$

Variance:

$$\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2 \quad (8)$$

.30.3 Covariance and Correlation

Covariance:

$$\text{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])] \quad (9)$$

Correlation:

$$\rho_{X,Y} = \frac{\text{Cov}(X, Y)}{\sqrt{\text{Var}(X)\text{Var}(Y)}} \quad (10)$$

.31 Optimization

.31.1 Gradient Descent

Update Rule:

$$\theta_{t+1} = \theta_t - \alpha \nabla_{\theta} L(\theta_t) \quad (11)$$

where α is the learning rate and $L(\theta)$ is the loss function.

.31.2 Stochastic Gradient Descent

Update with Mini-batch:

$$\theta_{t+1} = \theta_t - \alpha \frac{1}{m} \sum_{i=1}^m \nabla_{\theta} L(\theta_t; x^{(i)}, y^{(i)}) \quad (12)$$

.31.3 Adam Optimizer

First and Second Moment Estimates:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (13)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (14)$$

Bias-Corrected Estimates:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad (15)$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (16)$$

Parameter Update:

$$\theta_{t+1} = \theta_t - \frac{\alpha \hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \quad (17)$$

.32 Loss Functions

.32.1 Mean Squared Error

$$L_{\text{MSE}} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (18)$$

.32.2 Cross-Entropy Loss

Binary Cross-Entropy:

$$L_{\text{BCE}} = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (19)$$

Categorical Cross-Entropy:

$$L_{\text{CCE}} = -\frac{1}{n} \sum_{i=1}^n \sum_{c=1}^C y_{ic} \log(\hat{y}_{ic}) \quad (20)$$

.33 Regularization

.33.1 L1 Regularization (Lasso)

$$L = L_0 + \lambda \sum_{j=1}^p |\theta_j| \quad (21)$$

.33.2 L2 Regularization (Ridge)

$$L = L_0 + \lambda \sum_{j=1}^p \theta_j^2 \quad (22)$$

.33.3 Elastic Net

$$L = L_0 + \lambda_1 \sum_{j=1}^p |\theta_j| + \lambda_2 \sum_{j=1}^p \theta_j^2 \quad (23)$$

.34 Information Theory

.34.1 Entropy

$$H(X) = - \sum_{x \in \mathcal{X}} P(x) \log P(x) \quad (24)$$

.34.2 KL Divergence

$$D_{KL}(P \parallel Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)} \quad (25)$$

.34.3 Mutual Information

$$I(X; Y) = \sum_{x,y} P(x, y) \log \frac{P(x, y)}{P(x)P(y)} \quad (26)$$

.35 Evaluation Metrics

.35.1 Classification Metrics

Accuracy:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (27)$$

Precision:

$$\text{Precision} = \frac{TP}{TP + FP} \quad (28)$$

Recall:

$$\text{Recall} = \frac{TP}{TP + FN} \quad (29)$$

F1 Score:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (30)$$

.35.2 Regression Metrics

Mean Absolute Error (MAE):

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (31)$$

Root Mean Squared Error (RMSE):

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (32)$$

R-Squared:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (33)$$

Glossary

A

A/B Testing: A method of comparing two versions of a model or system to determine which performs better.

Adam: Adaptive Moment Estimation, a popular optimization algorithm for training neural networks.

Adversarial Attack: An attempt to fool an ML model by providing deceptive input.

API (Application Programming Interface): A set of protocols for building and integrating application software.

AutoML: Automated Machine Learning, tools and techniques for automating the ML workflow.

B

Backpropagation: Algorithm for training neural networks by computing gradients through the chain rule.

Batch Inference: Making predictions on large batches of data offline.

Batch Normalization: A technique to normalize layer inputs to stabilize neural network training.

Bias (Statistical): The difference between the expected value of an estimator and the true value.

Blue-Green Deployment: A deployment strategy that reduces downtime by running two identical environments.

C

Canary Deployment: Gradual rollout of changes to a small subset of users before full deployment.

CI/CD: Continuous Integration and Continuous Deployment, practices for automating software delivery.

Concept Drift: Changes in the underlying data distribution that affect model performance.

Container: A lightweight, standalone package containing code and all its dependencies.

Cross-Validation: A technique for assessing model generalization by partitioning data into training and validation sets.

D

Data Drift: Changes in the input data distribution over time.

Data Pipeline: A series of data processing steps that transform raw data into a usable format.

Data Versioning: Tracking changes to datasets over time.

Distributed Training: Training models across multiple machines or GPUs.

Docker: A platform for developing, shipping, and running applications in containers.

DVC (Data Version Control): A version control system for ML projects, tracking data and models.

E

Embedding: A learned representation mapping discrete variables to continuous vectors.

Ensemble Learning: Combining multiple models to improve performance.

ETL (Extract, Transform, Load): The process of extracting data from sources, transforming it, and loading it into a destination.

Experiment Tracking: Recording parameters, metrics, and artifacts from ML experiments.

F

Feature Engineering: Creating new features from raw data to improve model performance.

Feature Store: A centralized repository for storing and serving features for ML models.

Fine-Tuning: Adapting a pre-trained model to a specific task.

G

GPU (Graphics Processing Unit): Hardware accelerator commonly used for training neural networks.

Gradient: The vector of partial derivatives indicating the direction of steepest ascent.

gRPC: A high-performance RPC framework for communication between services.

H

Hyperparameter: A parameter set before training that controls the learning process.

Hyperparameter Optimization: The process of finding optimal hyperparameter values.

I

IaC (Infrastructure as Code): Managing infrastructure through code rather than manual processes.

Inference: Making predictions with a trained model.

K

Kubernetes: An open-source container orchestration platform.

KServe: A Kubernetes-based platform for serving ML models.

Kubeflow: A machine learning toolkit for Kubernetes.

L

Latency: The time taken to process a single inference request.

Learning Rate: A hyperparameter controlling the step size in gradient descent.

Loss Function: A function measuring the difference between predicted and actual values.

M

MLflow: An open-source platform for the ML lifecycle.

MLOps: Machine Learning Operations, practices for deploying and maintaining ML systems.

Model Registry: A centralized repository for managing model versions.

Model Serving: Deploying models to make predictions in production.

N

Neural Architecture Search (NAS): Automated search for optimal neural network architectures.

O

ONNX: Open Neural Network Exchange, a format for representing ML models.

Overfitting: When a model learns training data too well, including noise, hurting generalization.

P

Pipeline: A sequence of data processing or model training steps.

Precision: The fraction of positive predictions that are correct.

Pruning: Removing unnecessary weights or neurons from a neural network.

Q

Quantization: Reducing model size by using lower-precision numbers.

R

Recall: The fraction of actual positives correctly identified.

Regularization: Techniques to prevent overfitting by constraining model complexity.

REST API: Representational State Transfer, an architectural style for web services.

S

Scaling: Adjusting feature values to a common scale.

Shadow Deployment: Running a new model in parallel with the existing one without affecting users.

Sharding: Distributing data across multiple machines.

T

Tensor: A multi-dimensional array, the fundamental data structure in deep learning.

TensorFlow Serving: A system for serving TensorFlow models in production.

Throughput: The number of inference requests processed per unit time.

TorchServe: A tool for serving PyTorch models.

Training-Serving Skew: Discrepancies between training and serving environments.

Transfer Learning: Applying knowledge from one task to another related task.

U

Underfitting: When a model is too simple to capture underlying patterns.

V

Validation Set: Data used to evaluate model performance during training.

Variance: The amount by which model predictions vary for different training sets.

W

Weight Decay: A regularization technique equivalent to L2 regularization.

Acronyms

- **AI:** Artificial Intelligence
- **API:** Application Programming Interface
- **AUC:** Area Under the Curve
- **AWS:** Amazon Web Services
- **BERT:** Bidirectional Encoder Representations from Transformers

- **CI/CD:** Continuous Integration/Continuous Deployment
- **CNN:** Convolutional Neural Network
- **CPU:** Central Processing Unit
- **CV:** Cross-Validation
- **DL:** Deep Learning
- **DVC:** Data Version Control
- **EDA:** Exploratory Data Analysis
- **ETL:** Extract, Transform, Load
- **GAN:** Generative Adversarial Network
- **GCP:** Google Cloud Platform
- **GPU:** Graphics Processing Unit
- **gRPC:** gRPC Remote Procedure Call
- **IaC:** Infrastructure as Code
- **IoU:** Intersection over Union
- **KNN:** K-Nearest Neighbors
- **LSTM:** Long Short-Term Memory
- **MAE:** Mean Absolute Error
- **MAP:** Mean Average Precision
- **ML:** Machine Learning
- **MLP:** Multi-Layer Perceptron
- **MLOps:** Machine Learning Operations
- **MSE:** Mean Squared Error
- **NAS:** Neural Architecture Search
- **NLP:** Natural Language Processing
- **ONNX:** Open Neural Network Exchange
- **REST:** Representational State Transfer
- **RMSE:** Root Mean Squared Error
- **RNN:** Recurrent Neural Network
- **ROC:** Receiver Operating Characteristic

- **SaaS:** Software as a Service
- **SGD:** Stochastic Gradient Descent
- **SLA:** Service Level Agreement
- **SVM:** Support Vector Machine
- **TPU:** Tensor Processing Unit
- **URL:** Uniform Resource Locator

Code Style Guide

Code Style Guide

This appendix defines the coding standards and conventions used throughout the book. All code examples follow these guidelines to ensure consistency and readability.

.36 Python Style Standards

.36.1 General Principles

All Python code in this book adheres to:

- **PEP 8** style guide for Python code
- **PEP 257** for docstring conventions
- **Type hints** (PEP 484) for function signatures
- **Black** code formatter (line length: 88 characters)
- **isort** for import sorting

.36.2 Naming Conventions

```
1  # Classes: PascalCase
2  class FeatureStore:
3      pass
4
5  class ModelTrainer:
6      pass
7
8  # Functions and variables: snake_case
9  def train_model(data: pd.DataFrame) -> Model:
10     model_name = "fraud_detector"
11     training_data = prepare_data(data)
12     return model
13
14  # Constants: UPPER_SNAKE_CASE
15  MAX_RETRIES = 3
16  DEFAULT_TIMEOUT = 30
17  API_BASE_URL = "https://api.example.com"
18
19  # Private methods/attributes: leading underscore
20  class DataProcessor:
21     def __init__(self):
22         self._cache = {}
23
```

```

24     def _validate_data(self, data):
25         pass
26
27     # Type aliases: PascalCase
28     FeatureDict = Dict[str, Union[int, float, str]]
29     ModelConfig = Dict[str, Any]

```

Listing 13: Naming Convention Examples

.36.3 Import Organization

Imports are organized in three groups, separated by blank lines:

```

1  # Standard library imports
2  import os
3  import sys
4  from datetime import datetime
5  from pathlib import Path
6  from typing import Dict, List, Optional, Any, Tuple
7
8  # Third-party imports
9  import numpy as np
10 import pandas as pd
11 from sklearn.ensemble import RandomForestClassifier
12 import mlflow
13 import mlflow.sklearn
14
15 # Local application imports
16 from src.data.loaders import DataLoader
17 from src.features.transformers import FeatureTransformer
18 from src.models.trainer import ModelTrainer
19 from src.utils.logger import setup_logger

```

Listing 14: Import Organization

.36.4 Type Hints

All function signatures include type hints:

```

1  from typing import Dict, List, Optional, Union, Tuple, Any
2  import pandas as pd
3  import numpy as np
4
5  # Simple types
6  def calculate_score(value: float) -> float:
7      return value * 2.0
8
9  # Complex types
10 def process_features(
11     data: pd.DataFrame,
12     feature_names: List[str],
13     config: Dict[str, Any]
14 ) -> Tuple[np.ndarray, List[str]]:
15     """Process features with given configuration."""
16     processed_data = data[feature_names].values
17     return processed_data, feature_names
18

```

```

19 # Optional types
20 def load_model(path: str, version: Optional[str] = None) -> Model:
21     """Load model from path, optionally specific version."""
22     if version is None:
23         version = "latest"
24     return Model.load(path, version)
25
26 # Union types
27 def parse_value(value: Union[str, int, float]) -> float:
28     """Parse value to float."""
29     return float(value)
30
31 # Generic types
32 from typing import TypeVar, Generic
33
34 T = TypeVar('T')
35
36 class Cache(Generic[T]):
37     def __init__(self) -> None:
38         self._data: Dict[str, T] = {}
39
40     def get(self, key: str) -> Optional[T]:
41         return self._data.get(key)
42
43     def set(self, key: str, value: T) -> None:
44         self._data[key] = value

```

Listing 15: Type Hint Examples

.36.5 Docstrings

Use Google-style docstrings for all public functions and classes:

```

1 def train_model(
2     X_train: np.ndarray,
3     y_train: np.ndarray,
4     model_config: Dict[str, Any],
5     validation_split: float = 0.2
6 ) -> Tuple[Model, Dict[str, float]]:
7     """Train machine learning model with cross-validation.
8
9     This function trains a model using the provided training data
10    and configuration. It performs internal validation and returns
11    both the trained model and validation metrics.
12
13    Args:
14        X_train: Training features array of shape (n_samples, n_features)
15        y_train: Training labels array of shape (n_samples,)
16        model_config: Configuration dictionary containing model parameters
17        validation_split: Fraction of data to use for validation (default: 0.2)
18
19    Returns:
20        Tuple containing:
21        - Trained model instance
22        - Dictionary of validation metrics (accuracy, precision, recall, f1)
23
24    Raises:

```

```

25         ValueError: If validation_split is not in range [0, 1]
26         RuntimeError: If model training fails
27
28     Example:
29     >>> model_config = {"n_estimators": 100, "max_depth": 10}
30     >>> model, metrics = train_model(X_train, y_train, model_config)
31     >>> print(f"Accuracy: {metrics['accuracy']:.3f}")
32     Accuracy: 0.923
33
34     Note:
35     The model is automatically logged to MLflow if tracking is enabled.
36     See Chapter 8 for details on experiment tracking.
37
38     References:
39     See Section 7.3 for model architecture details.
40     """
41     if not (0 <= validation_split <= 1):
42         raise ValueError("validation_split must be in range [0, 1]")
43
44     # Implementation...
45     pass
46
47 class FeatureStore:
48     """Centralized feature storage and retrieval system.
49
50     The FeatureStore provides a unified interface for storing and
51     retrieving features for machine learning models. It supports
52     both batch and real-time feature access.
53
54     Attributes:
55         connection: Database connection instance
56         cache: In-memory feature cache
57         feature_specs: Dictionary of feature specifications
58
59     Example:
60     >>> store = FeatureStore(connection_string="postgresql://...")
61     >>> features = store.get_features(user_id="123", feature_names=["age", "income"])
62
63     >>> print(features)
64     {"age": 35, "income": 75000}
65
66     See Also:
67     Chapter 6: Feature Engineering and Feature Stores
68     """
69     def __init__(self, connection_string: str) -> None:
70         """Initialize feature store connection.
71
72         Args:
73             connection_string: Database connection string
74         """
75         self.connection = self._create_connection(connection_string)
76         self.cache = {}
77         self.feature_specs = {}

```

Listing 16: Docstring Examples

36.6 Error Handling

Explicit error handling with informative messages:

```

1 import logging
2 from typing import Optional
3
4 logger = logging.getLogger(__name__)
5
6 # Specific exceptions
7 class ModelNotFoundError(Exception):
8     """Raised when model cannot be found in registry."""
9     pass
10
11 class DataValidationError(Exception):
12     """Raised when data validation fails."""
13     pass
14
15 # Error handling with logging
16 def load_model(model_id: str) -> Model:
17     """Load model with proper error handling."""
18     try:
19         logger.info(f"Loading model: {model_id}")
20         model = Model.load(model_id)
21         logger.info(f"Successfully loaded model: {model_id}")
22         return model
23
24     except FileNotFoundError as e:
25         logger.error(f"Model file not found: {model_id}")
26         raise ModelNotFoundError(
27             f"Model {model_id} not found in registry"
28         ) from e
29
30     except Exception as e:
31         logger.exception(f"Unexpected error loading model: {model_id}")
32         raise
33
34 # Validation with early returns
35 def validate_config(config: Dict[str, Any]) -> None:
36     """Validate configuration dictionary."""
37     if not config:
38         raise ValueError("Configuration cannot be empty")
39
40     if "model_type" not in config:
41         raise ValueError("Configuration must specify 'model_type'")
42
43     if config["model_type"] not in ["classifier", "regressor"]:
44         raise ValueError(
45             f"Invalid model_type: {config['model_type']}. "
46             f"Must be 'classifier' or 'regressor'"
47         )
48
49     logger.info("Configuration validated successfully")

```

Listing 17: Error Handling Patterns

.37 Configuration Management

.37.1 YAML Configuration Files

All configuration uses YAML format:

```

1  # config/training_config.yaml
2  model:
3      name: "fraud_detector"
4      type: "xgboost"
5      hyperparameters:
6          n_estimators: 100
7          max_depth: 10
8          learning_rate: 0.1
9          subsample: 0.8
10
11  data:
12      training_path: "s3://bucket/data/train.parquet"
13      validation_path: "s3://bucket/data/val.parquet"
14      test_path: "s3://bucket/data/test.parquet"
15      batch_size: 1000
16
17  training:
18      num_epochs: 10
19      early_stopping_patience: 3
20      validation_split: 0.2
21      random_seed: 42
22
23  logging:
24      level: "INFO"
25      log_dir: "logs/"
26      mlflow_tracking_uri: "http://mlflow:5000"
27
28  deployment:
29      serving_endpoint: "http://api:8000/predict"
30      max_batch_size: 100
31      timeout_ms: 100

```

Listing 18: Configuration File Structure

.37.2 Configuration Loading

```

1  from pathlib import Path
2  from typing import Dict, Any
3  import yaml
4  from dataclasses import dataclass
5
6  @dataclass
7  class ModelConfig:
8      """Model configuration data class."""
9      name: str
10     type: str
11     hyperparameters: Dict[str, Any]
12
13     @classmethod
14     def from_dict(cls, config_dict: Dict[str, Any]) -> 'ModelConfig':
15         """Create ModelConfig from dictionary."""

```



```

16         return cls(**config_dict)
17
18 def load_config(config_path: str) -> Dict[str, Any]:
19     """Load configuration from YAML file.
20
21     Args:
22         config_path: Path to YAML configuration file
23
24     Returns:
25         Dictionary containing configuration
26
27     Raises:
28         FileNotFoundError: If config file doesn't exist
29         ValueError: If YAML is invalid
30     """
31     path = Path(config_path)
32
33     if not path.exists():
34         raise FileNotFoundError(f"Config file not found: {config_path}")
35
36     try:
37         with open(path, 'r') as f:
38             config = yaml.safe_load(f)
39         return config
40     except yaml.YAMLError as e:
41         raise ValueError(f"Invalid YAML in config file: {e}")
42
43 # Usage
44 config = load_config("config/training_config.yaml")
45 model_config = ModelConfig.from_dict(config['model'])

```

Listing 19: Configuration Loading Pattern

.38 Logging Standards

.38.1 Logger Setup

```

1 import logging
2 import sys
3 from pathlib import Path
4
5 def setup_logger(
6     name: str,
7     level: str = "INFO",
8     log_file: Optional[str] = None
9 ) -> logging.Logger:
10     """Configure logger with consistent format.
11
12     Args:
13         name: Logger name (usually __name__)
14         level: Logging level (DEBUG, INFO, WARNING, ERROR, CRITICAL)
15         log_file: Optional file path for logging
16
17     Returns:
18         Configured logger instance
19     """

```

```

20 logger = logging.getLogger(name)
21 logger.setLevel(getattr(logging, level.upper()))
22
23 # Console handler
24 console_handler = logging.StreamHandler(sys.stdout)
25 console_handler.setLevel(logging.INFO)
26
27 # Formatter
28 formatter = logging.Formatter(
29     fmt='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
30     datefmt='%Y-%m-%d %H:%M:%S'
31 )
32 console_handler.setFormatter(formatter)
33 logger.addHandler(console_handler)
34
35 # File handler (optional)
36 if log_file:
37     file_handler = logging.FileHandler(log_file)
38     file_handler.setLevel(logging.DEBUG)
39     file_handler.setFormatter(formatter)
40     logger.addHandler(file_handler)
41
42 return logger
43
44 # Usage in modules
45 logger = setup_logger(__name__)
46
47 logger.debug("Detailed debugging information")
48 logger.info("General information messages")
49 logger.warning("Warning messages")
50 logger.error("Error messages")
51 logger.critical("Critical error messages")

```

Listing 20: Logging Configuration

.39 Testing Patterns

.39.1 Unit Test Structure

```

1 import pytest
2 import numpy as np
3 import pandas as pd
4 from src.models.trainer import ModelTrainer
5 from src.data.loaders import DataLoader
6
7 class TestModelTrainer:
8     """Test suite for ModelTrainer class."""
9
10    @pytest.fixture
11    def sample_data(self):
12        """Create sample training data."""
13        np.random.seed(42)
14        X = np.random.randn(100, 10)
15        y = np.random.randint(0, 2, 100)
16        return X, y
17

```

```

18 @pytest.fixture
19 def trainer(self):
20     """Create ModelTrainer instance."""
21     config = {
22         "model": {
23             "type": "random_forest",
24             "hyperparameters": {
25                 "n_estimators": 10,
26                 "max_depth": 5
27             }
28         }
29     }
30     return ModelTrainer(config)
31
32 def test_train_model_shape(self, trainer, sample_data):
33     """Test that trained model produces correct output shape."""
34     X, y = sample_data
35     model = trainer.train(X, y)
36
37     predictions = model.predict(X)
38     assert predictions.shape == y.shape
39
40 def test_train_model_accuracy(self, trainer, sample_data):
41     """Test that model achieves minimum accuracy."""
42     X, y = sample_data
43     model = trainer.train(X, y)
44
45     accuracy = model.score(X, y)
46     assert accuracy > 0.5 # Better than random
47
48 def test_invalid_config_raises_error(self):
49     """Test that invalid configuration raises ValueError."""
50     invalid_config = {"model": {"type": "invalid_model"}}
51
52     with pytest.raises(ValueError):
53         trainer = ModelTrainer(invalid_config)
54
55 @pytest.mark.parametrize("n_estimators,expected_min_accuracy", [
56     (10, 0.6),
57     (50, 0.7),
58     (100, 0.75),
59 ])
60 def test_accuracy_improves_with_trees(
61     self,
62     sample_data,
63     n_estimators,
64     expected_min_accuracy
65 ):
66     """Test that accuracy improves with more trees."""
67     X, y = sample_data
68     config = {
69         "model": {
70             "type": "random_forest",
71             "hyperparameters": {"n_estimators": n_estimators}
72         }
73     }
74     trainer = ModelTrainer(config)
75     model = trainer.train(X, y)

```

```

76 accuracy = model.score(X, y)
77 assert accuracy >= expected_min_accuracy
78

```

Listing 21: Unit Test Example

.40 Code Comments

.40.1 When to Comment

1. **Complex algorithms:** Explain non-obvious logic
2. **Business rules:** Document domain-specific requirements
3. **Workarounds:** Explain why hacky code is necessary
4. **TODOs:** Mark incomplete implementations
5. **Performance considerations:** Explain optimization choices

.40.2 Comment Examples

```

1  # Good: Explains WHY, not WHAT
2  def calculate_fraud_score(transaction: Dict) -> float:
3      """Calculate fraud probability score."""
4      # Use log transformation to handle skewed amount distribution
5      # Reference: Section 5.2.3 for feature engineering rationale
6      log_amount = np.log1p(transaction['amount'])
7
8      # Apply higher weight to international transactions
9      # based on historical fraud patterns (85% of fraud is international)
10     int_multiplier = 2.5 if transaction['is_international'] else 1.0
11
12     return log_amount * int_multiplier
13
14 # Bad: States the obvious
15 x = x + 1 # Increment x by 1
16
17 # Good: Documents business rule
18 MAX_TRANSACTION_AMOUNT = 10000 # Regulatory limit from PCI-DSS v3.2.1
19
20 # Good: Explains workaround
21 def process_batch(data):
22     # HACK: pandas groupby has memory leak in version 1.2.3
23     # Explicitly delete intermediate dataframes to avoid OOM
24     # TODO: Remove after upgrading to pandas 1.3.0
25     grouped = data.groupby('user_id')
26     result = grouped.apply(aggregate_features)
27     del grouped
28     return result

```

Listing 22: Good Comment Examples

.41 Code Organization Patterns

.41.1 Module Structure

Each module follows this structure:

```
1  """Module docstring describing purpose.
2
3  This module provides functionality for...
4
5  Example:
6      >>> from module import ClassName
7      >>> instance = ClassName()
8      >>> result = instance.method()
9  """
10
11  # Imports (standard, third-party, local)
12  import os
13  from typing import Dict, List
14
15  import numpy as np
16
17  from .utils import helper_function
18
19  # Module-level constants
20  DEFAULT_CONFIG = {"param": "value"}
21
22  # Module-level variables (if necessary)
23  _cache: Dict[str, any] = {}
24
25  # Helper functions (private)
26  def _internal_helper(data: np.ndarray) -> np.ndarray:
27      """Internal helper function."""
28      pass
29
30  # Public functions
31  def public_function(param: str) -> str:
32      """Public API function."""
33      pass
34
35  # Classes
36  class PublicClass:
37      """Main class implementation."""
38      pass
39
40  # Main execution block
41  if __name__ == "__main__":
42      # CLI or testing code
43      pass
```

Listing 23: Standard Module Structure

.42 Performance Patterns

.42.1 Vectorization

Prefer vectorized operations over loops:

```

1 import numpy as np
2 import pandas as pd
3
4 # Bad: Slow loop
5 result = []
6 for x in data:
7     result.append(x ** 2 + 2 * x + 1)
8
9 # Good: Vectorized
10 result = data ** 2 + 2 * data + 1
11
12 # Bad: DataFrame iteration
13 for idx, row in df.iterrows():
14     df.at[idx, 'result'] = row['a'] + row['b']
15
16 # Good: Vectorized DataFrame operations
17 df['result'] = df['a'] + df['b']
18
19 # Good: Using .apply() when vectorization isn't possible
20 df['result'] = df.apply(lambda row: complex_function(row), axis=1)

```

Listing 24: Vectorization Examples

.43 Cross-Reference Style

Use consistent cross-references throughout the book:

```

1 % Chapter references
2 See Chapter~\ref{ch:data_pipelines} for details on data processing.
3
4 % Section references
5 As discussed in Section~\ref{sec:feature_engineering}, features must be...
6
7 % Figure references
8 Figure~\ref{fig:architecture} shows the system architecture.
9
10 % Table references
11 Table~\ref{tab:metrics} compares model performance.
12
13 % Equation references
14 Using Equation~\eqref{eq:precision}, we calculate...
15
16 % Code listing references
17 Listing~\ref{lst:training} demonstrates the training loop.
18
19 % Multiple references
20 Chapters~\ref{ch:serving}--\ref{ch:monitoring} cover production operations.

```

Listing 25: Cross-Reference Examples

.44 Summary

These coding standards ensure consistency and quality across all book examples:

- **Follow PEP 8** for all Python code
- **Use type hints** for function signatures
- **Write comprehensive docstrings** with examples
- **Handle errors explicitly** with informative messages
- **Use YAML for configuration** with validation
- **Set up proper logging** in all modules
- **Write tests** for critical functionality
- **Comment why, not what** for complex logic
- **Prefer vectorization** over loops
- **Use consistent cross-references** to other chapters

All code examples in this book are production-ready and can be adapted for real-world ML systems.

Bibliography

Cognilytica. Data engineering, preparation, and labeling for ai 2020. <https://www.cognilytica.com/>, 2020.

Gartner. Gartner survey reveals leading organizations expect to double the number of ai projects in place in 2020. <https://www.gartner.com/>, 2020.

David Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo, and Dan Dennison. Hidden technical debt in machine learning systems. *Advances in neural information processing systems*, 28, 2015.

VentureBeat. Why do 87% of data science projects never make it into production? <https://venturebeat.com/>, 2019.